

Dominium Conversation Human-Readable Reports Anthology

Mobile Dark Reading Edition

Status: DERIVED

Promotion Status: not_promoted

2026-05-30

Contents

1	Build And Future Proofing	2
2	Chronology Celestial Systems	65
3	Dominium Architecture I	75
4	Dominium Architecture Ii	81
5	Dominium Architecture Iii	91
6	Dominium Architecture Iv	98
7	Dominium Complete Conversation	103
8	Domino Dominium Workbench	508
9	Foundation Workbench Codex	580
10	Framework Open Source Provider	642
11	Language Platform Architecture	695
12	Launcher Setup Architecture	739
13	Modularity Aide Refactorability	746

14 Os Interface Repo Architecture	794
15 Portability Assurance Future Proof	849
16 Refactor Architecture	927
17 Refactor Control Plane	934
18 Release Identity And Versioning	971
19 Timekeeping And 2038 Resilience	1015
20 Ue6 Domino Deterministic Universe	1067
21 Universe Explorer Planning	1145
22 Workbench Aide Product Spine	1211
23 World Architecture	1335
24 Advanced Simulation Infrastructure	1342
25 App Runtime Platform Renderers	1348
26 App Testx Codehygiene	1356
27 Architecture Codex Prompts	1362
28 Architecture Ui Providers	1368

29 Canonical Structure And Framework	1422
30 Development Routes	1632
31 Documentation Standards Readme	1638
32 Dominion Domino Codex Planning	1645
33 Dominion Full Conversation	1657
34 Dominion Setup	1868
35 Domino Engine Refactor Prompts	1875
36 Engine Baseline Architecture	1882
37 Gui Binary Content	1956
38 Launcher App Layer	1964
39 Omega Xi Pi Architecture Future	1971
40 Platform Renderer Api Plan	2030
41 Platform Support	2036
42 Readme Ports Determinism	2047
43 Testx Repox Governance	2054

44	Ui Editor Tool Editor Planning	2060
45	Xstack Lab Galaxy	2067

Raw human-readable reports from the archived conversation corpus

Status: DERIVED. **Authority:** advisory source collection. **Promotion:** none.

This anthology preserves the raw human-readable report files from `docs/archive/conversations/` in one organized reading volume. It does not synthesize, promote, or reinterpret the reports.

0.1 How To Read This Anthology

Reports are grouped by their top-level conversation folder and then sorted by path. Each report is introduced by a compact provenance line, followed by the original human-readable text with heading levels shifted so the anthology remains navigable.

0.2 Source Coverage

- Conversation groups represented: 45
- Human-readable report files included: 131
- Machine manifests, ZIP files, raw packets, generated corpus outputs, and validation logs are excluded from the main anthology.

Chapter 1

Build And Future Proofing

1.1 Dominion Build And Future Proofing Architecture Human Readable Report

Source: docs/archive/conversations/Build_and_Future_Proofing/Dominium_Build_and_Future_Proofing_Architecture__01_human_readable_report.md

Report type: human_readable_report

Size: 32232 bytes

1.1.1 COMPLETE CHAT PRESERVATION REPORT - Dominion Build and Future-Proofing Architecture

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion Build and Future-Proofing Architecture

Row 2 Field: Date anchor; Assessment: 2026-05-27
Australia/Melbourne

Row 3 Field: Source scope; Assessment: This visible chat only unless labelled PROJECT-CONTEXT or REPO-CONTEXT

Row 4 Field: Apparent access; Assessment: Partial: visible current conversation, the uploaded prompt, and repo/tool results surfaced during this conversation were accessible; hidden or earlier full transcripts were not directly available.

Row 5 Field: Previously generated files available?; Assessment: No earlier downloadable files from this chat were available before this preservation task. New files were created by this task.

Row 6 Field: Uploaded files or artifacts present?; Assessment: Yes: Pasted `text.txt`, containing the maximum-fidelity preservation prompt.

Row 7 Field: Contains future plans?; Assessment: Yes. It contains build-system, CMake preset, CI, AIDE/XStack, modularity, public-surface, replacement-protocol, schema, and structure plans.

Row 8 Field: Contains decisions?; Assessment: Yes, with caveats. The chat contains firm user constraints and assistant recommendations; not every assistant recommendation was explicitly accepted by the user.

- Row 9** Field: Contains pending tasks?; Assessment: Yes. Several proposed task sequences remain pending.
- Row 10** Field: Contains unresolved questions?; Assessment: Yes. Most concern acceptance, sequencing, exact repo state, and which recommendations should become canon.
- Row 11** Field: Staleness risk; Assessment: Medium to High. Repo state, CMake/VS/toolchain facts, and CI state are time-sensitive.
- Row 12** Field: Extraction confidence; Assessment: 4/5 for the visible chat; 2-3/5 for any implied older-chat context.
- Row 13** Field: Safe for later aggregation?; Assessment: Yes, with caveats: preserve labels, do not merge assistant suggestions as decisions, and verify current repo/toolchain facts first.
- Row 14** Field: Main limitations; Assessment: The full old chat transcript was not available beyond visible context; repo facts were sampled via tool calls during the chat; some recommendations were not formally accepted by the user.

Plain-language limitation: this package is reliable as a preservation of the current visible chat. It should not be

treated as a complete archive of every Dominionium discussion in other chats. Where the report uses project or repository context, it is labelled as such. The most important caveat is that many items are recommendations, not final user decisions.

1. One-Page Orientation

This chat centered on how to make Dominionium durable as a serious long-term software project rather than a one-off game prototype. The user first provided a prior technical assessment about Dominionium's Windows build floors, Visual Studio toolchains, XP/Win7/Win10 support, CMake presets, static CRT policy, and runtime testing requirements. That pasted material treated Dominionium's canon as strict: engine in C89, game layer in C++98, deterministic fixed-point behavior, no hidden capability creep, separate artifacts per OS floor, no CRT mixing, and proof through RepoX/TestX-style governance.

The user's first active question then reframed the problem around build-system complexity. They were concerned that different machines would have different IDEs and compilers: one Windows 10 laptop might use VS2017, a desktop might carry VS2022/VS2026 plus XP toolchains, while old VMs or old hardware might host VS2010, VS2005, VC6, VC1.5, Xcode 9, CodeWarrior Pro 9, and other historical toolchains. The user asked for the best way to design CMake, CI, presets, distribution,

sync, and AIDE/XStack support so Dominion could manage those varied machines and outputs without becoming chaotic.

The response treated that as a build orchestration problem, not merely a preset list. The key recommendation was that CMake should remain the canonical build executor, but `CMakePresets.json` should stop acting as a single giant catch-all for host machines, IDE names, toolchains, OS floors, renderers, distribution variants, and release status. The proposed approach was tuple-driven: declare build tuples in contracts, probe the local machine, generate local user presets, execute CMake/CTest, collect TestX/RepoX evidence, then generate distribution/package manifests. AIDE would not replace CMake; it would detect local capabilities, generate local presets, run tuple verification, and preserve evidence. XStack would remain an orchestration/governance layer, while RepoX/TestX would supply repository-policy and deterministic-runtime proof.

The second major user question widened the scope. The user said it was very important that the code be portable, modular, extensible, and reusable: not only for other games on the Domino engine, but potentially for completely different engine or game projects. They wanted the codebase to support complete replacement or rewrite of code, data, files, and directories during future

refactors, and to be developed like a proper game engine or OS rather than a one-off indie repo.

The response reframed Dominion as a portable deterministic engine platform with one game mounted on top. It argued that the correct order is stable contracts first, replaceable implementations second, products third, and presentation last. It judged the current top-level structure as close to correct, especially after the repo's canonical spine and runtime naming cleanups, but said the missing layer was a mechanically enforced stability taxonomy. The proposed additions were a public surface registry, a replacement protocol, stronger private/internal boundary enforcement, dependency-edge contracts, ABI header conformance tests, schema compatibility harnesses, content-pack authority cleanup, and deeper game-rule/domain cleanup.

The preservation prompt uploaded at the end turned the whole visible chat into a documentation task. It requested a human-readable report, structured registers, a context transfer packet, spec sheet, aggregator packet, self-audit, and downloadable file package. This preservation report therefore captures the conversation's actual purpose: Dominion needs governance structures that let it scale across toolchains, platforms, games, engines, refactors, and future spec-book aggregation without turning recommendations into accidental canon.

2. The Story of the Conversation

2.1 The starting point: toolchains and

compatibility floors The chat began with the user pasting a prior technical assessment. That assessment evaluated a recommendation about Visual Studio 2022, older MSVC toolsets, XP support, SDK pinning, `_WIN32_WINNT`, static CRT linking, and per-floor build artifacts. The user context made clear that Dominium has strict existing canon: C89 engine, C++98 game layer, deterministic execution, fixed-point assumptions, RepoX/TestX governance, separate multi-floor artifacts, no CRT mixing, and no silent API creep.

The pasted material did not merely say "install VS2022." It warned that `v141_xp` is acceptable for MVP but not necessarily archival-perfect; that SDK pinning is mandatory; that XP support requires runtime testing, not just compilation; and that legacy toolchains such as VC7.1 or VC6 may later be needed for true archival support. It also included a repo-state assessment that the Dominium repo had improved structurally but was still not build/product/projection proven at that time.

2.2 The user's build-system problem The user then stated the practical issue: Dominium would be worked on from multiple machines with different IDEs and toolchains. The user planned to replace VS2015

with VS2017 on one laptop, keep VS2022/VS2026 or XP toolchains on a desktop, and use VMs or older machines for VS2010, VS2005, VS6, VS1.5, Xcode 9, CodeWarrior Pro 9, and similar toolchains. The question was how to design the build system, CI, presets, sync, distribution, and automation so that all of this remained understandable and mechanically governed.

2.3 The build architecture answer The assistant inspected the Dominion repo and docs through GitHub connector calls during the conversation. It found that the existing `CMakePresets.json` had the core smell the user was worried about: host/editor names, actual toolsets, target OS floors, renderer flags, release lanes, and local machine assumptions were mixed into one preset namespace. It also found existing docs for toolchain matrix, build matrix, verification paths, dist layout, component matrix, and CI.

The answer was that Dominion needs a build orchestration layer above CMake, not more manually curated presets. The proposed flow was: build contracts define canonical tuples; machine probes detect local capabilities; generated `CMakeUserPresets.json` exposes only what the local machine can actually build; CMake remains the build graph authority; CTest/TestX/RepoX validate; dist/package manifests preserve proof. AIDE should assist by probing, explaining, generating local

presets, verifying tuples, and emitting evidence. XStack should orchestrate and enforce, not become a second build system.

2.4 The broader modularity question The user then pushed beyond build mechanics. They asked what else they were missing about portability, modularity, extensibility, replacement, rewriteability, and reuse across future games and engines. They explicitly wanted Dominion to be developed like a proper OS or game-engine platform, not a one-off indie project.

The assistant checked newer repo state and current structural docs. It found that the repo had moved beyond the earlier POST-CONVERGE-09 state, including a canonical source spine collapse, pack payload moves into `content/packs`, and runtime second-level naming normalization. It cited current docs describing the final repository structure, component matrix, language strategy, code/data boundary, schema stability, and boundary enforcement.

The answer was that Dominion should be treated as a portable deterministic engine platform. The top-level structure was judged mostly right, but deeper work remained. The most important missing pieces were a public surface registry, a formal replacement protocol, stricter dependency-edge enforcement, ABI/public header conformance, schema compatibility tests, and a

clearer distinction between frozen ABI, stable API, stable data contracts, and internal mutable implementation.

2.5 The preservation task The final uploaded prompt requested a complete preservation package for this chat: a human-readable explanation, structured registers, context transfer packet, spec sheet, aggregator packet, self-audit, and files/ZIP package. This response completes that export task and creates downloadable files for later reading and aggregation.

3. Main Topics Discussed

Topic 1 - Multi-floor build and toolchain strategy

This topic came up because the user needed Dominion to build across many Windows floors and historical toolchains without accidental capability drift. The chat discussed VS2022 as a host for older toolsets, v141/v141_xp, Windows SDK pinning, XP and Win7 runtime proof, and later archival toolchains such as VC7.1 or VC6. The visible conclusion was that per-floor binaries are mandatory and that support cannot be claimed from compilation alone. Runtime testing on real or VM floors remains required.

Topic 2 - CMake presets versus build contracts

The major build-system conclusion was that

`CMakePresets.json` should not carry every human, host, IDE, renderer, OS floor, and release concern directly.

The proposed design is contract-driven: define build tuples, detect machine capabilities, generate local user presets, then run CMake. This makes presets projections of contract truth rather than the authority itself.

Topic 3 - AIDE, XStack, RepoX, and TestX roles

AIDE was positioned as a build intelligence and evidence layer. It should probe the machine, explain which tuples are available or blocked, generate local presets, run tuple verification, and record evidence. XStack should remain umbrella orchestration. RepoX should enforce repo/layout/policy truth. TestX should verify runtime/determinism/build proof. None of these should replace CMake as the native build graph authority.

Topic 4 - Repository structure and source

ownership The chat examined whether Dominium's directory structure is good enough. The answer was that the top-level structure is now close to correct: `apps`, `engine`, `game`, `runtime`, `contracts`, `content`, `docs`, `tests`, `tools`, `scripts`, `cmake`, `external`, `release`, and `archive`. The remaining issue is not top-level layout but second/third-level ownership clarity, public/private boundaries, and lifecycle/status words embedded in active source paths.

Topic 5 - Portability, modularity, and reuse The user wanted reusable code for different games on the Domino engine and possibly unrelated future engine/game projects. The answer was to freeze contracts rather than implementations. Stable ABI/API/data/protocol surfaces should be explicitly versioned and tested; implementations should be replaceable behind black-box conformance tests. This is the basis for future rewrites without breaking compatibility.

Topic 6 - Naming policy The chat emphasized that directory and function names should express stable ownership and responsibility, not temporary status. Names like `runtime/render/software` and `runtime/render/null` are preferable to `soft` or `stub`. Generic names like `helpers`, `utils`, `common`, `misc`, `old`, `new`, and `legacy` should be avoided in active source unless explicitly quarantined or versioned.

Topic 7 - Schemas, protocols, and compatibility The chat discussed schema stability, unknown-field preservation, frozen/evolving schemas, compatibility migrations, and the need to apply wire-format discipline to Dominion's saves, replays, package manifests, protocols, command IDs, registry IDs, and capability IDs. The recommendation was to add compatibility fixtures and reserved-ID checks, not rely only on

documentation.

Topic 8 - The preservation package itself The uploaded prompt requested a complete chat preservation package with human report, registers, handoff packet, spec sheet, aggregator packet, audit, and downloadable files. This output creates that package and labels its limitations.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted to design a robust Dominion build system that could handle many machines, IDEs, compilers, OS floors, CMake presets, CI lanes, distribution outputs, and historical toolchains. The user also explicitly wanted Dominion's codebase to be portable, modular, extensible, reusable across future games and engines, and refactorable at the level of whole files and directories.

The final explicit goal was preservation: create a maximum-fidelity package for this chat so the user can understand it later, ask questions in this chat, merge it with other old-chat reports, and eventually build a master project spec book.

4.2 Inferred Goals INFERENCE: The user is trying to prevent Dominion from accumulating accidental

architecture. They want mechanical governance rather than relying on memory or assistant recommendations. They also want future assistants to stop repeating conceptual advice and instead preserve actionable structure, constraints, risks, and next tasks.

4.3 Goals That Changed Over Time The conversation moved from a specific compatibility/toolchain question into a broader engineering governance question. It began with how to handle VS2017/VS2022/VS2026/legacy VMs and grew into how to structure the entire repo, public API, schema system, rewrite protocol, and project portability doctrine.

4.4 Goals Still Unresolved The unresolved goals are implementation goals: deciding which recommendations become canon, adding public-surface/dependency/build contracts, validating the current live repo state, and applying the proposed cleanup tasks. The preservation package is complete, but the engineering work it describes remains pending.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Treat C89 engine, C++98 game layer, deterministic execution, per-floor artifacts, no CRT mixing, and no silent API creep as locked canon for this discussion.; Status:

Established as user-provided constraint; Why it mattered: It bounds all build and architecture recommendations.; Confidence: High; Label: FACT

Row 2 ID: DECISION-02; Decision: Frame the build problem as orchestration/contracts, not just preset cleanup.; Status: Assistant recommendation; not explicitly accepted; Why it mattered: It prevents `CMakePresets.json` from becoming a machine-specific dumping ground.; Confidence: High; Label: INFERENCE/RECOMMENDATION

Row 3 ID: DECISION-03; Decision: Keep CMake as native build authority; use AIDE/XStack around it, not instead of it.; Status: Assistant recommendation; not explicitly accepted; Why it mattered: It preserves industry-standard build semantics while adding governance.; Confidence: High; Label: RECOMMENDATION

Row 4 ID: DECISION-04; Decision: Use tuple-driven build definitions: product + OS floor + arch + toolchain + SDK + CRT + config + renderer + package lane.; Status: Assistant recommendation; not explicitly accepted; Why it mattered: It makes artifacts and CI lanes explicit and auditable.; Confidence: High; Label: RECOMMENDATION

Row 5 ID: DECISION-05; Decision: Treat Dominium as a portable deterministic engine platform with

one game mounted on top.; Status: Assistant recommendation; not explicitly accepted; Why it mattered: It supports reuse across games/engines and prevents one-off project drift.; Confidence: Medium-High; Label: RECOMMENDATION

Row 6 ID: DECISION-06; Decision: Keep current top-level structure broadly, but continue second/third-level cleanup and boundary enforcement.; Status: Assistant conclusion; not explicitly accepted; Why it mattered: It avoids disruptive top-level churn while addressing real modularity debt.; Confidence: Medium; Label: RECOMMENDATION

Row 7 ID: DECISION-07; Decision: Add public-surface registry and replacement protocol.; Status: Assistant recommendation; not explicitly accepted; Why it mattered: It makes rewrites safe and mechanically checkable.; Confidence: Medium-High; Label: RECOMMENDATION

The most important caution is that only DECISION-01 is clearly a user-stated constraint in this visible chat. The other items are strong recommendations produced in response to the user's requests for best practice; they should not be merged into canon unless the user later accepts them.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

- More hand-written CMake presets were deprioritised. The better path is generated local presets from declared build contracts.
- One universal binary or one universal preset was rejected because it violates per-floor artifacts, CRT isolation, SDK pinning, and explicit floor proof.
- AIDE replacing CMake was rejected. AIDE should orchestrate, probe, and preserve evidence; CMake remains build authority.
- Manual drag-and-drop repo moves were rejected in earlier context and preserved here as a general risk; moves must go through migration maps, validators, shims, and proof.
- Freezing implementation files/directories was rejected in favor of freezing contracts, public surfaces, protocols, schemas, IDs, and compatibility semantics.
- Generic active-source names such as `common`, `misc`, `helpers`, `old`, `new`, `legacy`, `stub`, and status-coded paths were deprioritised for canonical code.

7. Important Reasoning, Rationale, and Tradeoffs

The visible rationale was governed by one central tradeoff: Dominion needs long-term portability and compatibility, but the project is already complex enough that uncontrolled abstraction would make it worse. The answer was not to introduce a universal mega-framework. The answer was to separate authorities: CMake builds, contracts define support truth, machine probes describe local capability, AIDE helps with orchestration/evidence, XStack coordinates governance, RepoX enforces repository policy, TestX proves behavior, and distribution manifests preserve artifact identity.

The second major tradeoff was between freezing too much and freezing too little. Freezing implementations would block refactors; freezing nothing would destroy compatibility. The recommended compromise was to freeze public contracts and conformance tests while allowing implementations to be replaced behind them.

The third tradeoff concerned structure. The repo's top-level layout is now relatively coherent, so another large top-level reshuffle would create churn. The deeper issue is ownership and dependency clarity below that level. Therefore the recommendation was to keep the top-level spine and invest in public surface registries, dependency contracts, schema compatibility, and name cleanup.

8. Plans, Future Work, and Next Steps

Recommended next-action sequence:

1. **Confirm acceptance boundary.** Decide which recommendations in this chat should become Dominion canon and which remain advisory.
2. **Verify current repo state.** Confirm the actual current HEAD, build proof, smoke tests, and outstanding warnings before acting.
3. **Add a public surface registry.** Define which headers, commands, schemas, protocols, and manifests are frozen/stable/internal.
4. **Add dependency-edge enforcement.** Convert intended module direction into a machine-readable contract.
5. **Add tuple build contracts and generated local presets.** Move build truth out of ad hoc preset names.
6. **Add ABI/header conformance tests.** Public engine/runtime headers should compile under declared floors.
7. **Add schema/protocol compatibility fixtures.** Old fixtures should load; unknown fields should round-trip; reserved IDs should be checked.

8. **Continue game/content cleanup.** Resolve `game/rule` versus `game/rules`, content pack taxonomy, and stale pack authority references.
9. **Document the replacement protocol.** Make whole-module rewrites safe through black-box conformance and replay/hash comparison.
10. **Feed this report into the master spec book aggregator.** Preserve uncertainty labels and avoid merging suggestions as decisions.

The strongest blocker is acceptance and verification: the user needs to decide what becomes binding, and the live repo state should be checked again before implementation.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences

- Engine is C89; game is C++98.
- Determinism, fixed-point behavior, and no hidden behavior are central constraints.
- Multi-floor builds must be separate artifacts.
- No CRT mixing.
- No silent API creep.

- RepoX/TestX governance matters.
- Human-readable explanations are preferred over machine-only handoffs.
- The preservation output must distinguish FACT, INFERENCE, UNCERTAIN/UNVERIFIED, and PROJECT-CONTEXT.

9.2 Inferred Constraints and Preferences

- The user values auditability over speed.
- The user dislikes vague architecture advice without next actions.
- The user wants future assistants to preserve tentative status and not over-canonize recommendations.
- The user wants project structure that can survive major rewrites.

9.3 Uncertain or Unestablished Preferences

- Whether all recommended structural tasks should be implemented now is not established.
- Whether the proposed build tuple naming scheme should become official is not yet established.

- Whether the current top-level structure is final-final or merely best current target remains subject to user decision and repo evidence.

10. Files, Artifacts, Outputs, and Prompts

ID	Artifact	Type	Purpose	Preserve?
ARTIFACT-01	Initial pasted GPT-5.2/ GPT-5.5 build/ toolchain and repo- state analysis	In-chat pasted context	Set the build/ governance problem frame	Yes
ARTIFACT-02	Assistant build- system answer	In-chat report	Proposed tuple- driven build architecture	Yes
ARTIFACT-03	Assistant modularity/ future- proofing answer	In-chat report	Proposed public- surface/ replacement/ structure doctrine	Yes
ARTIFACT-04	Pasted text.txt	Uploaded prompt	Requested this preservation package	Yes
ARTIFACT-05	GitHub connector snapshots of Dominium docs	Repo context surfaced in chat	Supported claims about structure, matrix, language, schema, and boundary docs	Yes, but verify freshness

No pre-existing downloadable files from this chat were available before this task. The downloadable files listed in the final section were created by the preservation task.

11. Open Questions and Unresolved Issues

1. Which assistant recommendations from this chat should become binding Dominion canon?
2. What is the exact current live repo HEAD and validation state after the latest commits?
3. Should the public surface registry be implemented before or after build tuple contracts?
4. What exact support floors are intended for MVP versus archival/research lanes?
5. Which schemas/protocols are truly frozen, and which are still evolving?
6. Which code surfaces must be reusable for non-Dominion future engine projects?
7. How should future master spec aggregation reconcile overlap with earlier Dominion chats?
8. What should be the immediate next implementation prompt after this preservation task?

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

- A future assistant may treat recommendations as final decisions. Mitigation: preserve labels and require explicit user acceptance.
- A future assistant may rely on stale repo state. Mitigation: verify live HEAD, docs, and CI before implementation.
- A future assistant may over-focus on folder names and miss contract/interface stability. Mitigation: start with public surface registry and dependency contracts.
- A future assistant may generate more abstract doctrine instead of actionable tasks. Mitigation: use the task register.
- A future assistant may merge this with other chats without preserving conflicts. Mitigation: use the aggregator packet and conflict warnings.
- A future assistant may confuse CMake/AIDE/XStack roles. Mitigation: CMake builds; AIDE orchestrates/probes; XStack coordinates; RepoX/TestX enforce/prove.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat contributes two major chapters to a future Dominionium project spec book:

1. **Build, Toolchain, CI, and Distribution Governance.** It defines the need for tuple-driven builds, machine probes, generated local presets, toolchain/floor evidence, and per-artifact manifests.
2. **Portability, Modularity, and Replacement Architecture.** It defines the need for public-surface registries, stable ABI/API/schema/protocol contracts, no reachable internals, dependency-edge enforcement, and rewrite-by-conformance.

It should not be merged as a list of final decisions. Much of it is recommendation-level material. The spec-book aggregator should convert user-stated constraints into requirements, assistant recommendations into candidate requirements, and repo facts into verified references only after current repo validation.

14. What I Should Remember

- The chat's central thesis is that Dominionium needs contract-governed portability, not more ad hoc presets or folders.

- The build-system answer was tuple-driven: contracts define what can be built; machines are probed; local presets are generated; CMake builds; TestX/RepoX prove; dist manifests preserve evidence.
- The modularity answer was contract-driven: freeze ABI/API/schema/protocol/public surfaces, not implementation files.
- The current top-level structure appears close to correct, but second/third-level names and boundaries still need cleanup.
- A public surface registry and replacement protocol are the most important missing architecture mechanisms.
- The user constraints around C89/C++98, determinism, no CRT mixing, per-floor artifacts, and no silent API creep must not be diluted.
- Do not merge assistant recommendations into canon without user acceptance.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- What are the three most important architecture ideas from this chat?

- Which parts of this chat are FACT versus assistant recommendation?

15.2 Decisions

- Which recommendations should I explicitly accept as Dominion canon?
- Which decisions are still only tentative?

15.3 Tasks and Next Actions

- Write the next implementation prompt for
STRUCTURE-01: Public Surface Registry.
- Write the next implementation prompt for tuple-driven build contracts and generated local presets.

15.4 Artifacts and Files

- Explain the generated ZIP package and which file I should use for aggregation.
- Turn the context transfer packet into a shorter bootstrap prompt.

15.5 Risks and Verification

- What live repo facts should I verify before implementing these tasks?

- What is the risk of doing build tuple work before public surface registry work?

15.6 Future Spec Book / Aggregation

- Convert this chat into spec-book chapter headings.
- Extract only formal requirement candidates from this chat.

15.7 Deep-Dive Questions Specific to This Chat

- Design the `contracts/public_surface` schema.
- Design the `contracts/build` tuple schema.
- Design a replacement protocol for rewriting a runtime backend without breaking compatibility.

16. Compact Human Summary

This chat was about making Dominium structurally durable. It began with a pasted analysis of Windows toolchains and compatibility floors, including VS2022 as a possible host for older MSVC toolsets, v141/v141_xp for practical XP-era builds, stricter historical toolchains for archival lanes, SDK pinning, static CRT requirements, and runtime testing across OS floors. That material framed Dominium as a project with strict canon: C89 engine, C++98 game, deterministic fixed-

point execution, per-floor artifacts, no CRT mixing, no hidden behavior, and RepoX/TestX governance.

The user's first active question asked how to handle many different development machines and toolchains without making CMake presets and CI unmanageable. The answer was that Dominion needs build orchestration above CMake, not a larger hand-maintained preset file. CMake should remain the build executor, but build truth should live in declared tuples: product, OS floor, architecture, toolchain, SDK, CRT, build config, renderer, and package lane. A machine probe should detect what a given laptop, desktop, VM, or CI runner can actually build, then generate local `CMakeUserPresets.json` entries. AIDE should help probe, explain, generate, verify, and record evidence. XStack should orchestrate. RepoX and TestX should enforce and prove. Distribution outputs should carry manifests that identify exactly what was built and under what contract.

The user's second question broadened the concern from build mechanics to the whole codebase. They wanted code that could be reused for other games on the Domino engine, or even different engine/game projects, and they wanted complete future rewrites of files/directories to be possible without destroying compatibility. The answer was that Dominion should be treated as a portable deterministic engine platform with

one game mounted on it. The correct order is stable contracts first, replaceable implementations second, products third, presentation last.

The repo's top-level structure was judged close to correct: `apps`, `engine`, `game`, `runtime`, `contracts`, `content`, `docs`, `tests`, `tools`, `scripts`, `cmake`, `external`, `release`, and `archive`. The remaining problem is deeper: public/private boundaries, dependency direction, schema compatibility, naming, and replacement protocols. The most important proposed additions were a public surface registry, a dependency-edge contract, ABI header conformance tests, schema compatibility fixtures, a replacement protocol, and further game/content cleanup.

The final uploaded prompt asked for this preservation package. The key thing to preserve is that most architecture proposals are recommendations, not accepted user decisions yet. The next best action is to decide which recommendations become canon, then implement either **STRUCTURE-01: Public Surface Registry** or the build tuple contract work, after verifying the current live repo state.

1.2 Dominium Build And Future Proofing Architecture Reader Brief

Source: docs/archive/conversations/Build_and_Future_Proofing/Dominium_Build_and_Future_Proofing_Architecture__06_reader_brief.md

Report type: reader_brief

Size: 1774 bytes

1.2.1 Reader Brief - Dominium Build and Future-Proofing Architecture

What this chat was about

This chat asked how Dominium should be engineered so it remains portable, modular, extensible, reusable, compatible with old and new toolchains, and safe to refactor over time. It focused first on build/toolchain governance and then on broader source-structure/API/schema/rewrite discipline.

Top 20 things to know

1. User's canon: C89 engine, C++98 game, determinism, per-floor artifacts, no CRT mixing, no silent API creep.

2. Build complexity comes from many machines and historical toolchains.
3. More hand-written presets are not the optimal solution.
4. Build truth should live in tuple contracts.
5. Local machine probes should generate local `CMakeUserPresets.json`.
6. CMake remains build authority.
7. AIDE should probe/explain/generate/verify/record evidence.
8. XStack orchestrates; RepoX/TestX enforce/prove.
9. Dist/package manifests should preserve artifact identity.
10. Dominion should be treated as an engine platform, not a one-off game.
11. Freeze contracts, not implementations.
12. The current top-level repo spine is close to right.
13. Deeper boundaries and naming still need cleanup.
14. Public surface registry is a key missing mechanism.
15. Replacement protocol is a key missing mechanism.
16. Dependency-edge enforcement is needed.

17. ABI/header conformance tests are needed.
18. Schema/protocol compatibility fixtures are needed.
19. Many recommendations are not yet accepted decisions.
20. Verify live repo state before implementation.

Best next step

Ask: "Which recommendation should become canon first: public surface registry or tuple-driven build contracts?" Then verify current repo state and write a narrow implementation prompt.

1.3 Dominium Build And Future Proofing Architecture In Chat Reader

Source: docs/archive/conversations/Build_and_Future_Proofing/Dominium_Build_and_Future_Proofing_Architecture__09_in_chat_reader.md

Report type: in_chat_reader

Size: 1602 bytes

1.3.1 In-Chat Reader - Dominion Build and Future-Proofing Architecture

Package overview

This package preserves the current visible chat about Dominion build/toolchain governance and modular future-proofing. It includes a human-readable report, structured registers, context transfer packet, YAML spec sheet, aggregator packet, audit, and future-chat prompt.

File index

- `00_manifest.md`: file inventory and caveats.
- `01_human_readable_report.md`: sections 0-16, best for reading first.
- `02_context_transfer_packet.md`: future-chat handoff.
- `03_spec_sheet.yaml`: machine-assisted aggregation format.
- `04_registers.md`: structured IDs and tables.
- `05_aggregator_packet.md`: compact merge packet.
- `06_reader_brief.md`: short summary.
- `07_verification_and_audit.md`: self-audit and verification queue.

- 08_future_chat_bootstrap_prompt.md: prompt for continuing elsewhere.
- 09_in_chat_reader.md: this guide.

Question menu

Ask next:

- Which recommendations are decisions versus advice?
- Write the implementation prompt for public surface registry.
- Write the implementation prompt for tuple-driven build contracts.
- Verify the current repo state and compare against this report.
- Convert this chat into spec-book requirements only.
- Build an aggregator packet from multiple chat reports.

Top things to preserve

C89/C++98 canon, deterministic/no hidden behavior, per-floor artifacts, no CRT mixing, tuple-driven builds, public surface registry, replacement protocol, schema

compatibility harness, and the warning that recommendations are not accepted canon yet.

1.4 Dominion Build And Future Proofing Architecture Accompanying Human Readable Detailed Summary Report

Source: docs/archive/conversations/Build_and_Future_Proofing/Dominium_Build_and_Future_Proofing_Architecture__10_accompanying_human_readable_detailed_summary_report.md

Report type: human_readable_detailed_summary

Size: 23777 bytes

GPT-5.5 Pro - 2026-05-27 21:03:00 AEST

1.4.1 Accompanying Human-Readable Detailed Conversation Report

Chat label: Dominion Build and Future-Proofing Architecture

Date anchor: 2026-05-27 Australia/Melbourne

Scope: This report covers the visible conversation and the preservation task that produced the accompanying handoff files. It does **not** claim complete access to any earlier hidden or retired chats.

Epistemic note: Items marked **FACT** were explicitly present in this conversation or in the provided/pasted material. Items marked **INFERENCE** are reasonable conclusions from the discussion. Items marked **RECOMMENDATION** are assistant proposals that should not be treated as accepted Dominion canon unless the user later confirms them.

1. Executive Summary

This conversation focused on how to make Dominion a durable, portable, modular, extensible, long-lived engine/game platform rather than a one-off indie-style project. The discussion moved through three major layers.

First, the user provided prior context about Dominion’s build and compatibility constraints. That context treated the engine as C89, the game layer as C++98, the simulation as deterministic/fixed-point, and the release process as governed by RepoX/TestX-style proof. It emphasized per-floor artifacts, no CRT mixing, no hidden capability creep, SDK pinning, and runtime proof on actual OS floors. The pasted material also summarized a recent repository state where layout governance had improved, but build/product/portable projection proof remained partial or blocked.

Second, the user asked how to handle many different build machines and toolchains. The examples included a

Windows 10 laptop moving from VS2015 to VS2017, a Windows 10 desktop with VS2022/VS2026 and XP toolchains, and older VMs or hardware with VS2010, VS2005, VC6, VC1.5, Xcode 9, CodeWarrior Pro 9, and other historical compilers. The answer was that Dominium should not solve this with a huge manually maintained `CMakePresets.json`. Instead, it should use a build orchestration layer above CMake: build tuples defined by contracts, local machine probing, generated local presets, CMake/CTest execution, TestX/RepoX proof, and distribution/package manifests.

Third, the user widened the scope from build mechanics to software architecture. They wanted all code to be portable, modular, extensible, reusable for other games on the Domino engine, and potentially reusable for other engine/game projects. They also wanted the ability to replace not just code/data but whole files and directories in future rewrites. The answer reframed Dominium as a portable deterministic engine platform with one game mounted on top. The central doctrine was: freeze public contracts and compatibility behavior, not implementation files. That means public surface registries, dependency-edge contracts, conformance tests, ABI/header checks, schema compatibility harnesses, and a formal replacement protocol.

Finally, the user uploaded a preservation prompt requesting a complete chat handoff package. That

package was created first, containing a human-readable report, context transfer packet, spec sheet, registers, aggregator packet, reader brief, verification/audit file, future-chat bootstrap prompt, in-chat reader, and ZIP. The current task then requested an accompanying detailed human-readable summary and a fresh single ZIP bundling all those files plus this report and an integrity check.

The most important thing future readers must not lose is the distinction between **constraints**, **recommendations**, and **accepted canon**. The visible chat clearly establishes the user's desired direction and hard constraints, but many specific implementation mechanisms remain assistant recommendations until accepted.

2. What the Conversation Was Really About

At the surface level, this was about build presets, CMake, old Visual Studio versions, and repository structure. At the deeper level, it was about governance.

The user is trying to prevent Dominion from becoming accidental software: a pile of source files, scripts, old folders, half-working build lanes, and documentation that cannot prove what the code actually supports. The user wants Dominion to be built more like a serious engine or operating environment: stable public

contracts, replaceable internals, strict boundaries, deterministic proof, portable build outputs, and long-term compatibility.

The conversation therefore kept returning to the same pattern:

```
contract first
implementation second
proof third
support claim last
```

This applied to toolchains, platforms, directory structure, public APIs, schemas, protocols, content packs, releases, and future refactors.

3. Build and Toolchain Discussion

3.1 Starting context **FACT:** The pasted context said VS2022 can host older toolsets such as v141/v141_xp, Windows 7.1A SDK, and Windows 8.1 SDK. It also emphasized that VS2022 is a host/projection and that the toolset defines the real compiler behavior.

FACT: The pasted context treated per-floor binaries as mandatory. A single Windows binary for all OS floors was rejected because different OS floors, CRT choices, SDK floors, and runtime compatibility surfaces must be separately proven.

FACT: `_WIN32_WINNT` was treated as floor-specific and something that should be set per CMake target/preset,

not manually scattered through source.

FACT: v141_xp was described as acceptable for MVP practicality but not equivalent to historically pure XP-era toolchains. VC7.1 was described as more mechanically honest for archival XP/Vista-style proof.

FACT: SDK locking was treated as mandatory. The warning was that allowing a newer Windows SDK to leak into old-floor builds can create symbol/runtime-loader failures and hidden dependency creep.

FACT: Static linking with /MT was treated as a compatibility requirement for XP-style builds, not a convenience.

FACT: Runtime testing on actual OS floors was required before support claims. Compile success alone was not enough.

3.2 User's practical build concern The user then explained that different development machines would not have the same IDEs/toolchains. They expected some machines to run VS Code/Codex while others would have specific Visual Studio or legacy tools installed. They were planning to uninstall VS2015 and replace it with VS2017 on one laptop, keep VS2022/VS2026 and possibly VS2017/XP toolchains on a desktop, and maintain older toolchains in virtual machines or old hardware.

The question was not "which single preset should I use?" It was "how can Dominion represent many possible host/toolchain combinations without losing determinism or governance?"

3.3 Recommendation: tuple-driven build system

RECOMMENDATION: Keep CMake as the canonical build executor, but do not make CMakePresets.json carry every host machine, IDE projection, OS floor, renderer, package lane, and development preference.

RECOMMENDATION: Add a build orchestration layer:

```
contracts/build/*.toml
?
local machine probe
?
generated CMakeUserPresets.json
?
cmake configure/build/test
?
CTest/TestX proof
?
RepoX/release policy proof
?
dist/package manifests
```

The recommended unit is a **build tuple**, not an IDE name:

product + OS floor + architecture + toolchain + SDK + CRT +
config + platform/backend + package lane

Example tuple names proposed:

verify.win10.x64.msvc143.mt.debug

release.xp.x86.msvc141_xp.mt.release

research.nt4.x86.vc6.mt.release

verify.linux.x64.gcc.glibc217.debug

3.4 What was put off Implementation was put off. No actual repo changes were made in this chat. The next implementation task was suggested as either a build contract/machine probe refactor or, after the later architecture discussion, a public surface registry. Both remain pending.

4. Repo State and Governance Context

The pasted material included a prior repo-state summary. It said the repo had moved from uncontrolled root chaos toward a governed state: strict layout validators passed, root allowlist passed, distribution validator passed, component matrix validator passed, and active exceptions were explicit. It also said the repo still lacked full build/product/portable projection proof.

Important state from the pasted material:

- Layout authority: mostly fixed.

- Exception count: reduced but not gone.
- Build proof: still blocked in that prior state.
- Product boot proof: partial.
- Portable projection proof: partial.
- Runtime/playtest proof: blocked.
- Feature expansion readiness: not yet.

FACT FROM PASTED CONTEXT: The prior material said POST-CONVERGE-09 recorded portable projection/package smoke proof as partial, not proven. It documented blockers such as missing Visual Studio 17 2022 local lane, no native product binaries, product boot proof limited to script/wrapper help, packaging assembly requiring build outputs, no real portable projection root, and missing portable manifests.

CAVEAT: Later visible repo/tool outputs in this chat showed more recent commits and improved runtime/source-spine cleanup. Therefore, repo-state details from the pasted material may already be stale and should be verified before action.

5. AIDE, XStack, RepoX, and TestX Roles

The conversation separated the roles of major project automation systems:

CMake = native build graph and compiler execution
CTest = native test execution layer
AIDE = agent/work-unit orchestration, probing, explanation, evidence
XStack = umbrella CI/control framework
RepoX = repository/layout/policy validation
TestX = runtime/test/determinism proof
Dist/package tools = generated artifact/projection proof

RECOMMENDATION: AIDE should not replace CMake. It should probe installed tools, explain available build tuples, generate local presets, run tuple verification, and write build evidence.

Suggested command shape:

```
python .aide/scripts/aide_lite.py build probe
python .aide/scripts/aide_lite.py build list
python .aide/scripts/aide_lite.py build gen-presets
python .aide/scripts/aide_lite.py build verify verify.win10.x64.msvc143.mt.debug
python .aide/scripts/aide_lite.py build package release.win10.x64.msvc143.mt.release
```

RECOMMENDATION: XStack should call the build contract and CMake; it should not invent a separate build matrix.

6. Code Portability, Modularity, and Reuse

The user's second major question was broader and more strategic. They said it was very important that Dominion code be portable, modular, and extensible so

it could be reused for:

- another game on the same Domino engine;
- other engine projects;
- other game projects;
- future complete rewrites/refactors where entire files and directories are replaced.

The answer reframed the project:

Dominium should not be "a game repo with reusable parts." It should be a portable deterministic engine platform with one game mounted on top.

The proposed design law was:

```
stable contracts first
replaceable implementations second
products third
presentation last
```

This means the engine should not expose unstable internal implementation details; the game should not depend on platform/rendering/runtime internals; apps should remain thin entrypoints; contracts should carry machine-readable law; content should be schema-validated authored data; and tools should validate,

migrate, codegen, and release rather than becoming runtime dependencies.

7. Directory Structure Discussion

The chat treated the current top-level repo structure as broadly correct:

```
apps/  
engine/  
game/  
runtime/  
contracts/  
content/  
docs/  
tests/  
tools/  
scripts/  
cmake/  
external/  
release/  
archive/
```

The top-level split was considered useful because it expresses ownership planes:

- `engine/`: deterministic C89 substrate.
- `game/`: simulation meaning, law, rules, domains, worlds, scenarios.
- `runtime/`: host services, shell, platform, render/

input/audio/network/storage/package/profile/
update services.

- `apps/`: thin products and workbench modules.
- `contracts/`: machine-readable law.
- `content/`: authored data/assets/packs/profiles/
templates/locales.
- `tools/`: validators, migration, codegen, audit,
release tooling.
- `scripts/`: thin human/CI wrappers.
- `docs/`: explanation and audits, not primary
machine law.

RECOMMENDATION: Keep the top level, refine
the second/third levels.

Specific naming guidance:

Good active-source names express ownership:

```
runtime/render/software/  
runtime/render/null/  
runtime/platform/win32/  
runtime/ui/service/  
runtime/shell/command/  
runtime/capability/core/
```

Bad active-source names encode status or vagueness:

```
old/
```


new/
legacy/
modern/
experimental/
temp/
misc/
common/
shared/
helpers/
stub/
soft/

CAVEAT: legacy, research, and similar terms may still appear in archive, compatibility matrices, or explicit support-tier contexts. The objection is to using them as active implementation ownership names.

8. Public Surface Registry and Replacement Protocol

The most important missing mechanism identified was a **public surface registry**.

RECOMMENDATION: Add machine-readable files such as:

```
contracts/public_surface/public_surface.contract.toml
contracts/public_surface/abi_surface.contract.toml
contracts/public_surface/schema_surface.contract.toml
contracts/public_surface/protocol_surface.contract.toml
contracts/public_surface/command_surface.contract.toml
```

Each public surface should specify:

- path;
- kind;
- owner;
- stability class;
- version;
- allowed dependencies;
- forbidden dependencies;
- compatibility/conformance tests.

The second missing mechanism was a **replacement protocol**:

1. Identify public contract.
2. Write black-box conformance tests.
3. Build old and new implementations behind the same API.
4. Run deterministic replay/save/protocol compatibility.
5. Compare output hashes.
6. Ship new implementation behind same public surface.
7. Retire old implementation only after compatibility proof.

This is what makes whole-file or whole-directory rewrites safe.

9. API, ABI, Schema, and Protocol Practices

The chat recommended that stable public C APIs use explicit ABI-safe patterns:

- opaque handles;
- public structs with `struct_size` and `api_version`;
- explicit result codes;
- no STL/exceptions/RTTI/templates/C++ name-mangled ABI at stable boundaries;
- explicit allocation ownership;
- explicit refusal/diagnostic semantics.

Schema/protocol rules recommended:

- never reuse numeric IDs;
- never reuse deleted field IDs;
- reserve deleted names/fields;
- add fields rather than renumbering;
- unknown fields round-trip unchanged;
- old saves load or explicitly refuse with reason;
- new artifacts are not silently accepted by old binaries unless compatible.

These were framed as candidate requirements for saves, replays, commands, registries, capabilities, packages, network protocols, TLV tags, and binary chunk IDs.

10. Testing and Compatibility Practices

The chat recommended clearer test categories:

- **Small:** deterministic logic only; no FS/network/wall-clock/thread/sleep.
- **Medium:** local filesystem, local IPC, package assembly, local cache.
- **Large:** VM floors, network, full install, launcher/client/server, packaging.
- **Golden:** exact replay/save/package hash expectations.
- **Compatibility:** old artifact read by new code; new artifact refused by old code when necessary.
- **ABI:** public headers compile under declared toolchains.
- **Contract:** schema/protocol/registry compatibility tests.

This aligns with the larger theme that support claims must be proven mechanically.

11. Decisions, Recommendations, and Non-Decisions

11.1 Clear user constraints **FACT:** The user provided/affirmed strong constraints through the pasted context and follow-up framing:

- Engine should be C89.
- Game layer should be C++98.
- Determinism matters.
- Fixed-point/no hidden behavior matters.
- RepoX/TestX-style governance matters.
- Multi-floor builds are separate artifacts.
- CRT mixing is forbidden.
- Silent API creep is forbidden.
- Portability, modularity, extensibility, and reuse are important.
- The project should be built like a serious engine/OS-like project, not a one-off indie project.

11.2 Assistant recommendations not yet accepted as canon These should be treated as strong proposals:

- tuple-driven build contracts;

- generated `CMakeUserPresets.json` from machine probe;
- public surface registry;
- dependency-edge contract;
- replacement protocol;
- specific `STRUCTURE-01 / BUILD-CONTRACT-01` task sequence;
- exact directory/naming rules;
- test classification scheme;
- formal schema/protocol compatibility harness.

11.3 Non-decisions The chat did not decide:

- exact contents of `contracts/build/*.toml`;
- exact public surface registry schema;
- which repo task should be run next;
- whether to prioritize public-surface registry or build tuple contracts;
- which OS/toolchain floors are MVP, T1, archival, or abandoned;
- which recommendations are now binding canon.

12. What Was Put Off for Later

The following were explicitly or implicitly deferred:

1. Implementing tuple-driven build contracts.
2. Adding a local machine toolchain probe.
3. Generating `CMakeUserPresets.json`.
4. Repairing/normalizing all naming debt under `game/content/tools/docs`.
5. Creating the public surface registry.
6. Adding dependency-edge enforcement.
7. Adding ABI/header compatibility tests for declared floors.
8. Adding schema/protocol compatibility fixtures.
9. Defining the replacement protocol as a formal contract.
10. Verifying current live repository state after newer commits.
11. Deciding which recommendations become canon.
12. Merging this chat into a future master project spec book.

13. Risks and Failure Modes

Risk 1 - Treating recommendations as accepted decisions Many details were proposed by the assistant. Future assistants must not convert them into canon without user confirmation.

Risk 2 - Relying on stale repo state The conversation contains pasted repo state, current visible repo/tool context, and previous assistant references. The live repository may have moved. Verify before implementation.

Risk 3 - Solving names but not boundaries Renaming directories is not enough. The real goal is stable ownership, public/private boundaries, dependency rules, ABI compatibility, and conformance proof.

Risk 4 - Expanding features before proof The earlier repo-state context warned against renderer/worldgen/fabrication/civilization work until build/product/projection proof is solid.

Risk 5 - Making AIDE too powerful AIDE should orchestrate and record evidence. It should not become an ungoverned source of build truth.

Risk 6 - Cross-chat aggregation drift When this report is merged with other reports, overlapping topics may conflict. The aggregator must preserve date anchors and source scopes.

14. Best Next Actions

Recommended next sequence:

1. **User confirmation:** identify which recommendations in this chat should become binding Dominion canon.
2. **Repo verification:** check current HEAD, CI status, build presets, CMake configure/build/CTest, layout validators, component matrix validators, and product/projection proof status.
3. **Choose first structural task:** either:
 - STRUCTURE-01: Public Surface Registry, or
 - BUILD-CONTRACT-01: Tuple Build Contract and Machine Probe.
1. **Create a narrow implementation prompt:** explicitly forbid feature work and scope creep.
2. **Run validators and record evidence:** preserve results in AIDE/RepoX/TestX-compatible form.

3. Feed this handoff into the master spec-book aggregator.

The best immediate question for the user to answer is:

Which recommendations in this report should become Dominion canon, and which should remain advisory?

15. What This Conversation Contributes to the Future Spec Book

This chat should feed these master spec-book chapters:

- Build, Toolchain, and CI Governance.
- Platform Floor and Artifact Policy.
- Repository Structure and Ownership Planes.
- Public Surface and ABI Policy.
- Schema/Protocol Compatibility Policy.
- Replacement and Refactor Protocol.
- AIDE/XStack/RepoX/TestX Governance Roles.
- Modularity, Portability, and Reuse Doctrine.
- Release, Dist, and Package Proof Requirements.

Candidate formal requirements:

- Build support claims require configure/build/test/package/runtime proof.
- Public surfaces must be registered with owner, version, stability, dependencies, and tests.
- Implementations can be replaced only behind conformance tests.
- CMake is build executor; orchestration layers must not replace build truth.
- Machine-specific presets should be generated/local, not hand-maintained as canon.
- Contracts govern; docs explain.

Background context that should not become requirement without review:

- Exact proposed file paths for future contracts.
- Exact task names.
- Exact tuple naming syntax.
- Specific toolchain floor recommendations if current support status changes.

16. What Was Created in the Preservation Work

Before this companion report, the previous preservation task created:

`Dominium_Build_and_Future_Proofing_Architecture__00_manifest.md`

`Dominium_Build_and_Future_Proofing_Architecture__01_human_readable_report.md`

`Dominium_Build_and_Future_Proofing_Architecture__02_context_transfer_packet.md`

`Dominium_Build_and_Future_Proofing_Architecture__03_spec_sheet.yaml`

`Dominium_Build_and_Future_Proofing_Architecture__04_registers.md`

`Dominium_Build_and_Future_Proofing_Architecture__05_aggregator_packet.md`

`Dominium_Build_and_Future_Proofing_Architecture__06_reader_brief.md`

`Dominium_Build_and_Future_Proofing_Architecture__07_verification_and_audit.md`

`Dominium_Build_and_Future_Proofing_Architecture__08_future_chat_bootstrap_prompt.md`

`Dominium_Build_and_Future_Proofing_Architecture__09_in_chat_reader.md`

`Dominium_Build_and_Future_Proofing_Architecture__handoff_package.zip`

This current task adds:

`Dominium_Build_and_Future_Proofing_Architecture__10_accompanying_human_readable_detailed_summary_report.md`

`Dominium_Build_and_Future_Proofing_Architecture__11_bundle_integrity_check.md`

`Dominium_Build_and_Future_Proofing_Architecture__complete_preservation_plus_companion_report.zip`

17. Final Check

I checked the existing preservation directory before creating this companion report. The original ten files were present, and the earlier ZIP contained those ten files. This new package bundles those ten files plus this companion report and an integrity check file. The new package is intended to be the one to keep if the user wants a single complete ZIP from this chat.

18. Suggested Follow-Up Questions

1. Which recommendations here should become binding canon?
2. Which should remain advisory until more evidence exists?
3. Write the implementation prompt for `STRUCTURE-01: Public Surface Registry`.
4. Write the implementation prompt for `BUILD-CONTRACT-01: Tuple Build Contracts and Machine Probe`.
5. Verify the current repo state and compare it to the older `POST-CONVERGE-09` state.
6. Turn the candidate requirements into a formal spec-book chapter.

7. Merge this package with another old-chat preservation package.
8. Extract only the build/toolchain requirements from this report.
9. Extract only the modularity/replacement/public surface requirements from this report.
10. Identify contradictions between this chat and later Dominion decisions.

Chapter 2

Chronology Celestial Systems

2.1 Dominion Chronology Celestial Systems Reader Brief

Source: docs/archive/conversations/Chronology_Celestial_Systems/Dominium_Chronology_Celestial_Systems__05_reader_brief.md

Report type: reader_brief

Size: 7486 bytes

2.1.1 Reader Brief - Dominion Chronology & Celestial Systems

What This Chat Was About

This chat defined a major Dominion Game subsystem covering celestial scope, time, calendars, chronology, diegetic time knowledge, and governance standards. It began with the question of what celestial objects and systems should be included in a logical universe simulation without redundant gameplay. The resulting content doctrine is scale-aware: Sol system gets maximum explicit detail because most playtime will

occur there; the Milky Way uses a named real-system backbone plus procedural/statistical expansion; the universe uses parametric cosmological structures.

The chat then developed a time architecture. The core rule is that time is continuous and authoritative, while calendars are interfaces. Atomic Continuous Time (ACT) is the simulation truth. Barycentric Sol Time, Galactic Coordinate Time, and Cosmological Proper Time support larger scales. Calendars, civil clocks, and naming systems are stateless renderers. Leap seconds are display-only. Proper time is derived for clocks/biology but never authoritative.

The most important specific calendar is the user-defined Perfect Earth Calendar (HPC-E): 13 months of 28 days, weekdays Monday-Sunday, year begins in March, month order March through February with Undecember as the 11th month, and all intercalary days appended after February. Year Day and Leap Day are canonically outside weeks/months, with optional compatibility projection to February 29/30.

The user also specified world start and UI rules: default date anchor is Gregorian January 1, 2000 AD, but the player spawns at local sunrise. At game start the player has no clock, calendar, date, or time HUD. Time/date knowledge is diegetic and must come from devices, calendars, organizations, jurisdictions, networks, or custom player-defined systems.

Most Important Things to Know

- FACT: ACT is authoritative; calendars are renderers only.
- FACT: Leap seconds are not in core.
- FACT: Proper time is derived, not authoritative.
- FACT: Sol system is the highest-detail play area.
- FACT: HPC-E month order and weekday names are user-defined and must be preserved.
- FACT: No January 0; intercalary days go after February.
- FACT: Default date anchor is Gregorian Jan 1, 2000 AD.
- FACT: Spawn time is local sunrise where possible.
- FACT: Player starts with no time/date HUD.
- FACT: Players can define calendars/clocks.
- FACT: Orgs, regions, jurisdictions, and individuals can designate standards.
- UNCERTAIN: HPC-E leap rule is not finalized.
- UNCERTAIN / UNVERIFIED: Assistant-generated astronomy data requires verification.

- UNCERTAIN / UNVERIFIED: Actual Plan G/
The Game implementation details were not accessible.

Active Plans or Workstreams

- WORKSTREAM-01: Multi-scale celestial content architecture (high)
- WORKSTREAM-02: Authoritative time core and coordinate frames (high)
- WORKSTREAM-03: Earth calendars and Perfect Earth Calendar (HPC-E) (high)
- WORKSTREAM-04: Planetary, moon, Pluto, and TNO calendars (medium-high)
- WORKSTREAM-05: Sol-wide, galactic, and universal epoch systems (medium-high)
- WORKSTREAM-06: World instantiation and local sunrise start (high)
- WORKSTREAM-07: Diegetic HUD, time knowledge, and fog-of-war (high)
- WORKSTREAM-08: Governance, jurisdictions, organizations, and time standards (high)
- WORKSTREAM-09: Modularity, extensibility, reliability, and integration architecture (high)

- WORKSTREAM-10: Prompt handoff and report packaging (high)

Decisions Already Made

- DECISION-01: Game operates across Earth, Sol, Milky Way, and universe scales. [FACT]
- DECISION-02: Sol system receives maximum explicit detail. [FACT]
- DECISION-03: Celestial content uses explicit/procedural/parametric classification. [INFERENCE]
- DECISION-04: Atomic Continuous Time (ACT) is authoritative simulation time. [FACT]
- DECISION-05: Use coordinate frames BST, GCT, and CPT above ACT. [FACT]
- DECISION-06: Calendars are pure renderers and never affect physics. [FACT]
- DECISION-07: Leap seconds are display-only and absent from core time. [FACT]
- DECISION-08: Proper time is derived only; calendars render coordinate time. [FACT]
- DECISION-09: Earth supports Gregorian, Julian, Islamic, Hebrew, Chinese, Persian, Holocene, Metric/French Revolutionary calendars. [FACT]

- DECISION-10: Perfect Earth Calendar has 13 months ? 28 days and Monday-Sunday weeks. [FACT]
- DECISION-11: Perfect Earth Calendar month order is March through February with Undecember as 11th month. [FACT]
- DECISION-12: HPC-E Year Day and Leap Day are appended after February; canonical mode is undated and compatibility mode may use Feb 29/30. [FACT / INFERENCE]
- DECISION-13: Default world date anchor is Gregorian January 1, 2000 AD. [FACT]
- DECISION-14: Player spawn time is local sunrise at spawn location. [FACT]
- DECISION-15: Player starts with no clock/calendar/date HUD. [FACT]

Pending Tasks

- TASK-01: Inspect actual Plan G/The Game architecture before implementation planning. (high)
- TASK-02: Verify celestial body/system lists against authoritative astronomy sources. (high)
- TASK-03: Finalize HPC-E leap rule. (high)

- TASK-04: Resolve acronym collisions and internal calendar IDs. (medium)
- TASK-05: Normalize week vs work-cycle terminology. (medium)
- TASK-06: Define TimeCore schema and API. (high)
- TASK-07: Define calendar renderer/plugin interface. (high)
- TASK-08: Define TimeStandard schema. (high)
- TASK-09: Define KnowledgeCapability schema for time/date access. (high)
- TASK-10: Define deterministic sunrise/fallback algorithm. (high)
- TASK-11: Define serialization/save policy. (high)
- TASK-12: Build recurrence/scheduling model. (medium)

Open Questions

- QUESTION-01: What is the actual Plan G/The Game architecture?
- QUESTION-02: Are Domino C89 / Dominion C++98 constraints real?

- QUESTION-03: What exact time scale maps Jan 1 2000 to ACT?
- QUESTION-04: What exact definition of sunrise is used?
- QUESTION-05: What is the exact HPC-E leap rule?
- QUESTION-06: Should compatibility projection Feb 29/Feb 30 be default?
- QUESTION-07: How many real Milky Way systems should be explicit?
- QUESTION-08: Which assistant-listed celestial systems are scientifically correct?
- QUESTION-09: Which exoplanets/sites per Milky Way system are explicit?
- QUESTION-10: Which planetary/moon calendar names are final?
- QUESTION-11: Should non-7-day cycles be weeks or work cycles?
- QUESTION-12: Do engineered mythological names remain despite no-mythology rule?

Files / Artifacts / Prompts to Preserve

- ARTIFACT-08: Final HPC-E definition.

- ARTIFACT-12: Leap seconds/relativity doctrine.
- ARTIFACT-13: World initialization/time discovery addendum.
- ARTIFACT-15: The Game handoff prompt.
- ARTIFACT-16: Celestial content inventory.
- ARTIFACT-17: Maximum-fidelity context transfer packet.
- ARTIFACT-18: This final report package.

What to Verify Before Acting

- Actual Plan G/The Game architecture.
- Codebase language/module assumptions.
- Celestial system list.
- Orbital/rotation constants.
- HPC-E leap rule.
- Non-Earth calendar names and cycle terminology.
- Sunrise algorithm and epoch mapping.

Best Next Step

Use this package in the new The Game/Plan G chat. First reconcile it with the actual project architecture.

Then generate a phased Codex implementation/
documentation/testing prompt, preserving uncertainty
labels and not assuming codebase details from this chat.

Chapter 3

Dominium Architecture I

3.1 Dominium Architecture I Reader Brief

Source: docs/archive/conversations/Dominium_Architecture_I/Dominium_Architecture_I__05_reader_brief.md

Report type: reader_brief

Size: 3901 bytes

3.1.1 Reader Brief - Dominium Architecture I

What This Chat Was About

This chat was a long architecture/specification-generation session for the Dominium Game project. It focused on turning a broad v3 game architecture into concrete, file-by-file `.txt` implementation specs for Codex automation. The user decided that Codex cannot read the entire full conversation at once, so the workflow changed from a single complete spec book to modular prompt files: one `.txt` per repo file.

The visible chat generated many implementation-spec prompts across engine, platform, render, audio, systems, and the start of game. It also preserved important decisions: UTF-8 internal text, retro fallbacks, semantic versioning, build metadata policy, deterministic simulation, strict layering, fixed-pool/no tick allocation, and lockstep/save/replay safety. The visible generation reached `game/g_world.h.txt`; the next file is `game/g_world.cpp.txt` if continuing in strict order.

The chat is partial: many earlier messages are skipped. Top-level docs, legal docs, context MDs, cross-dependencies, volume specs, and possibly early engine files are referenced but not fully visible.

Most Important Things to Know

- The chat label is **Dominium Architecture I**.
- Scope is this chat only, not the whole Project.
- The current package date anchor is 2026-05-27 Australia/Melbourne.
- Codex per-file devspecs are the active implementation-prep strategy.
- Top-level `.txt` files were skipped because `/docs/..` originals exist.
- If continuing, resume at `game/g_world.cpp.txt`.

- Determinism is mandatory.
- UTF-8 is canonical internally.
- Retro fallbacks are required where feasible.
- Semantic versioning is major.minor.patch.
- Build numbers/dates do not belong in filenames.
- Build metadata should include build numbers/dates.
- Many generated specs are drafts and need audit.
- Duplicate dweather/dhydro/dai_core specs are unresolved.
- dmem and dserialize APIs are inconsistent across drafts.
- C89/C++98 compliance is intended but violated by some generated text.
- External platform/tool/legal claims must be verified.

Active Plans or Workstreams

- Finish final handoff/report package.
- Continue devspec generation from `game/g_world.cpp.txt`.

- Canonicalise conflicting APIs before coding.
- Finish game/UI/launcher/mods/tools/data/tests/scripts specs.
- Convert specs into actual files.
- Use Codex to implement after canonicalisation.

Decisions Already Made

- Per-file `.txt` specs for Codex.
- Skip top-level `.txt` specs.
- Strict file order.
- UTF-8 canonical text.
- Semantic versioning and metadata build policy.
- Deterministic core systems.
- Strict layer separation.
- No dynamic allocation in deterministic ticks.

Pending Tasks

- Download/store this package.
- Continue at `game/g_world.cpp.txt` if requested.
- Resolve dweather/dhydro/dai_core conflicts.

- Normalise dmem/dserialize APIs.
- Audit C89/C++98 compliance.
- Verify platform/library/tool/legal claims.
- Inspect `/docs/...`

Open Questions

- Which duplicated system specs are canonical?
- Were missing early/platform specs generated in skipped context?
- Are `/docs/...` docs complete?
- What is the actual retro support tier?
- Is Codex 5.1 Max current and available?

Files / Artifacts / Prompts to Preserve

- The full generated devspec list.
- All visible generated specs through `g_world.h.txt`.
- Top-level docs references.
- Context MD references.
- Legal/policy doc references.

- Cross-dependencies doc reference.
- This report package.

What to Verify Before Acting

- /docs/... content.
- External API/tool/platform claims.
- Legal docs.
- C89/C++98 compatibility.
- dmem/dserialize canonical APIs.
- Duplicate specs.

Best Next Step

If continuing documentation: generate @@V10@@ next. If preparing implementation: run an **API canonicalisation** pass first.

Chapter 4

Dominium Architecture Ii

4.1 Dominium Architecture Ii Reader Brief

Source: docs/archive/conversations/Dominium_Architecture_II/Dominium_Architecture_II__05_reader_brief.md

Report type: reader_brief

Size: 6672 bytes

4.1.1 Reader Brief - Dominium Architecture II

What This Chat Was About

This chat developed the Dominium project from broad systems brainstorming into a detailed deterministic engine architecture and Codex-ready implementation plan. It covered the world model, building model, cut/fill and terrain layers, roads/rails/tunnels/bridges as vector-authored but grid-baked corridors, surface versus space domains, orbital construction/logistics, and the repository/documentation structure needed to guide

Codex 5.1 Max. The chat ended with the user preparing to retire the conversation and asking for a maximum-fidelity package for future continuation.

The most immediate practical output is a set of V4 spec replacements and Codex prompts. The important current target is a playable MVP: one full-size Earth surface; all systems minimally interactable; Lua for all data; Windows/DX9/Win32/SDL2 path; 2D vector top-down and 3D vector first-person. Multiplayer, Sol/space, all other renderers/platforms, textures, sounds and music are later steps.

Most Important Things to Know

- Dominion core must be deterministic, integer/fixed-point, C89-compatible.
- No floats in authoritative simulation.
- Renderer and platform are strictly separate from sim.
- MVP is vector-only and DX9/Windows-first.
- The world uses meters plus Q16.16 fixed substeps.
- Spatial ladder is subnano -> surface, with sparse microgrids/chunks.
- Buildings use blocks/cells/faces/edges as sim truth.

- Doors/devices are explicit fixtures, never automatic tunnel features.
- Terrain uses immutable base terrain plus earthworks deltas/cavities.
- Vector splines bake into chainpoints/linear microgrids.
- Space uses on-rails logic and local bubbles, not full n-body.
- Lua data lives through `/engine/script` with Lua 5.4.4 and sandboxed APIs.
- State hash uses FNV-1a 64-bit over canonical authoritative serialization.
- Codex should use only the listed files and current docs/dev files.

Active Plans or Workstreams

- WORKSTREAM-01: Unified documentation/source-of-truth
- WORKSTREAM-02: Deterministic core/kernel
- WORKSTREAM-03: ECS architecture
- WORKSTREAM-04: Messaging, events, jobs

- WORKSTREAM-05: Spatial hierarchy and cosmic identity
- WORKSTREAM-06: Terrain, cut/fill, and microgrids
- WORKSTREAM-07: Building system: cells, faces, edges, fixtures
- WORKSTREAM-08: Vector corridor/spline infrastructure
- WORKSTREAM-09: Utility networks: power, data, fluids, thermal
- WORKSTREAM-10: Transport systems
- WORKSTREAM-11: Pathfinding and deterministic movement
- WORKSTREAM-12: Surface vs space and orbital systems

Decisions Already Made

- DECISION-01: Use meter-based world coordinates with fixed substeps/Q16.16.
- DECISION-02: Use strict sim/render separation.
- DECISION-03: Adopt base-16 spatial ladder from subnano to surface.

- DECISION-04: Store surfaces as sparse microgrids/chunks, not continuous dense arrays.
- DECISION-05: Default local world is Earth/Sol/Milky Way.
- DECISION-06: Support canonical surface plus multiple surface instances for shards/mirrors.
- DECISION-07: Cross-surface interactions are asynchronous external events only.
- DECISION-08: Use on-rails orbital mechanics, not full n-body.
- DECISION-09: Docking, landing and takeoff are state-machine/domain transitions.
- DECISION-10: Use blocks/modules/cells/faces/edges as building simulation truth.
- DECISION-11: Use vector shapes/splines as authoring, generation and visual layers only.
- DECISION-12: Place walls/floors/roofs on grid faces/edges including diagonals.
- DECISION-13: Buildings rigid for now but schema supports destruction/collapse later.
- DECISION-14: Doors/devices are explicit fixtures placed manually or by explicit blueprint/pattern.

- DECISION-15: Immutable base terrain plus earthworks deltas and cavities.
- DECISION-16: Vector alignments bake into deterministic ChainPoints and linear microgrids.

Pending Tasks

- TASK-01: Apply/verify BUILDING.md V4
- TASK-02: Apply/verify SPEC_CORE.md V4
- TASK-03: Apply/verify DATA_FORMATS.md V4
- TASK-04: Apply/verify DIRECTORY_CONTEXT.md V4
- TASK-05: Verify dominium_new_addendum.txt contains renderer/platform addendum
- TASK-06: Generate CMake/build skeleton
- TASK-07: Implement core/sim/ECS shell
- TASK-08: Implement Win32 + DX9 vector shell
- TASK-09: Wire minimal client loop
- TASK-10: Implement 2D/3D vector camera/input controls
- TASK-11: Add /engine/script to DIRECTORY_CONTEXT/build

- TASK-12: Vendor Lua 5.4.4
- TASK-13: Implement script VM and sandbox
- TASK-14: Implement Lua bindings
- TASK-15: Create minimal Lua data prototypes

Open Questions

- QUESTION-01: Have V4 docs been applied to the actual repository?
- QUESTION-02: Does the actual repo contain /engine, /game, /data etc. yet?
- QUESTION-03: Should docs root remain flat or use /docs/spec?
- QUESTION-04: How to reconcile Lua-for-all-data with JSON-oriented DATA_FORMATS sections?
- QUESTION-05: What is the final DomDrawCmd/camera API?
- QUESTION-06: Native Win32 or SDL2 first?
- QUESTION-07: Can DX9.0c reliably target Windows 2000 SP4 through Windows 11?
- QUESTION-08: Can SDL2 support Windows 2000 target?

- QUESTION-09: Can Lua 5.4.4 build with old/retro compilers?
- QUESTION-10: What exact ECS component set is MVP?

Files / Artifacts / Prompts to Preserve

- ARTIFACT-19: User-provided current directory tree
- ARTIFACT-20: BUILDING.md V4
- ARTIFACT-21: SPEC_CORE.md V4
- ARTIFACT-22: DATA_FORMATS.md V4
- ARTIFACT-23: DIRECTORY_CONTEXT.md V4
- ARTIFACT-24: Codex consistency pass prompt
- ARTIFACT-25: Codex CMake/build skeleton prompt
- ARTIFACT-26: Codex core API skeleton prompt
- ARTIFACT-27: Codex sim/ECS shell prompt
- ARTIFACT-28: Codex platform/render shell prompt
- ARTIFACT-29: Codex minimal client loop prompt
- ARTIFACT-30: Lua clarification answer

What to Verify Before Acting

- VERIFY-01: Actual repo file contents and whether V4 docs applied.
- VERIFY-02: Actual directory existence under /engine, /game, /data, etc.
- VERIFY-03: DX9.0c support on Windows 2000 SP4 through Windows 11+.
- VERIFY-04: SDL2 support for Windows 2000 target.
- VERIFY-05: Lua 5.4.4 portability to required compilers.
- VERIFY-06: Restrictive LICENCE.md legal enforceability.
- VERIFY-07: CMake minimum/toolchains and flags.
- VERIFY-08: JSON-vs-Lua data policy.
- VERIFY-09: Final DomDrawCmd/camera API.
- VERIFY-10: Current docs use LICENCE.md and SPEC_CORE.md names consistently.

Best Next Step

Verify that BUILDING.md, SPEC_CORE.md, DATA_FORMATS.md, DIRECTORY_CONTEXT.md, and

docs/dev/dominium_new_addendum.txt on disk match the generated/current intent, then run the Codex consistency pass before any implementation.

Chapter 5

Dominium Architecture Iii

5.1 Dominium Architecture Iii Reader Brief

Source: docs/archive/conversations/Dominium_Architecture_III/Dominium_Architecture_III__05_reader_brief.md

Report type: reader_brief

Size: 4628 bytes

5.1.1 Reader Brief - Dominium Architecture III

What This Chat Was About

This chat developed Dominium's launcher, platform/render backend, software renderer, script, camera/view, keybinding, and support-tier plans. The launcher became a process-spawning system: it collects settings and executes separate client/server binaries rather than switching modes in-process. The backend/core launcher logic should be C89, while per-binary frontends may use C++98. Client and server binaries may be single-system

builds selected by the launcher and scripts.

The chat also replaced the earlier `vector_soft` idea with a single universal `software` renderer that works on every platform and supports both vector primitives and textured graphics. Every renderer should support vector-only and full graphics modes so the user can toggle CAD-style views, full graphics, missing-asset fallback, and performance modes. The latest renderer list is DX9.0c, DX11, GL1.1, GL2, VK1, and software. DX12 remains unresolved because it appeared earlier but was omitted from the latest user list.

The view system requirements expanded to include instant seamless zoom, instant switching between top-down 2D and first-person 3D, vector/graphics modes per view, and arbitrary cameras for free cam, map views, HUD cameras, CCTV, overlays, windows, and content creation. These must remain client/render-side and must not affect deterministic sim state.

The user finalized platform support tiers: Tier 1 is Windows NT 2000 SP4-latest, Mac OS X 10.6-10.14, Linux 3.2-current; Tier 2 extends to Windows 98 SE-Me, Mac OS 9-9.2, Mac OS X 10.6-15.0, Linux 2.6.18-current; Tier 3 covers MS-DOS 3.3-6.2, Windows 3, Windows 95-Me, NT 2000 SP4-latest, Mac OS 8.5-9.2, Mac OS X 10.0-latest, Linux 2.4-current. These are user decisions, but external feasibility must be verified.

Most Important Things to Know

- Launcher executes separate client/server binaries.
- Launcher backend/core is C89.
- Frontends may use C++98.
- Primary scripts: `dominium`, `setup`, `dom-client`, `dom-server`.
- `setup` supports install/repair/uninstall.
- `vector_soft` is superseded.
- Universal `software` renderer is required.
- Every renderer supports vector and graphics modes.
- Latest renderers: DX9.0c, DX11, GL1.1, GL2, VK1, software.
- DX12 is unresolved/deferred.
- Latest platform categories: POSIX, SDL1, SDL2, Native.
- X11/Wayland placement is unresolved.
- Instant zoom/2D/3D/vector/graphics switching is required.
- Arbitrary cameras are required.

- Final support tiers are user-defined and must be preserved.
- Uploaded launcher files must be inspected before code changes.

Active Plans or Workstreams

- Launcher C89 core and C++98 frontends.
- Launch plan and platform spawn API.
- Unified scripts and setup wrappers.
- Universal software renderer.
- Render/view/camera architecture.
- Input/keybinding spec.
- Platform support tiers and capability matrix.
- Uploaded launcher code refactor.

Decisions Already Made

- Use separate client/server binaries.
- Use C89 launcher backend/core.
- Allow C++98 frontends.
- Use `dominium` and `setup` as primary commands.

- Use universal **software** renderer.
- Every renderer must support vector/full modes.
- Preserve final Tier 1/2/3 OS support ranges.

Pending Tasks

- Inspect files.
- Resolve platform taxonomy.
- Resolve DX12 status.
- Update docs.
- Implement launcher core/model/viewmodel.
- Implement spawn API.
- Add scripts/CMake rules.
- Implement software renderer and render modes.
- Verify external compatibility claims.

Open Questions

- Is DX12 in scope?
- Are X11/Wayland explicit platform values or Native subtypes?
- Is POSIX headless-only?

- What launcher config format should be used?
- Are old script names aliases?
- What setup technology will be used?

Files / Artifacts / Prompts to Preserve

- `/mnt/data/dominium.7z`
- `/mnt/data/dom_launcher_view.c`
- `/mnt/data/dom_launcher_view.h`
- `/mnt/data/CMakeLists.txt`
- `/mnt/data/dom_launcher_main.c`
- Codex prompts for docs/platform/render/scripts, launcher build/test, input/keybindings, uploaded launcher refactor.
- F1-F12 map.
- Support tier table.
- Capability matrix draft.

What to Verify Before Acting

- Uploaded files and repo state.
- DX12 status.

- X11/Wayland/POSIX/Native taxonomy.
- Linux kernel/libc/toolchain claims.
- SDL/DirectX/OpenGL/Vulkan/Mac OS feasibility.
- Actual CMake target names and existing input/render APIs.

Best Next Step

Inspect the uploaded launcher files and repo state, then ask/decide the two blocking taxonomy questions: DX12 status and X11/Wayland/POSIX/Native mapping.

Chapter 6

Dominium Architecture Iv

6.1 Dominium Architecture Iv Reader Brief

Source: docs/archive/conversations/Dominium_Architecture_IV/Dominium_Architecture_IV__05_reader_brief.md

Report type: reader_brief

Size: 3586 bytes

6.1.1 Reader Brief - Dominium Architecture IV

What This Chat Was About

This chat defined and refined the architecture and implementation roadmap for the Dominium game and the reusable Domino engine. It moved from earlier platform/render/launcher/setup/game planning into a more rigorous ABIs-first roadmap. The current plan is to implement all ABIs/APIs first, then all platform backends, all renderer backends, then setup, launcher, tools/SDKs/editors, and finally the game.

The chat also established core architecture: Domino is the reusable engine layer; Dominion is the game/product layer. Products should be described once and presented through arbitrary UI/backend combinations using product contracts, capability descriptors, command/query APIs, data models, views, and canvases. The launcher/setup/tools can run headless/CLI/TUI/native GUI without a renderer; the game requires gfx. Major game-system design also emerged: deterministic fixed-point numeric policy, integer item storage, world origin at center/sea level, sparse 3D surface grid, Keplerian on-rails space, seamless surface/space transitions via reference frames and sim tiers, and arbitrary constructions as graphs of parts/materials for vehicles/buildings/machines.

Most Important Things to Know

- Current roadmap: ABIs/APIs -> platforms -> renderers -> setup -> launcher -> tools -> game.
- Actual repository state is unverified.
- Domino and Dominion must stay separate.
- Platform is mandatory; renderer optional for non-game products.
- UI mode and UI backend are separate.

- Product contracts control fallback/failure.
- Renderers consume command-buffer IR.
- Softref renderer is planned as canonical CPU reference.
- Numeric policy: Q4.12, Q16.16, Q48.16 plus integers.
- Inventory/material counts are whole integers.
- 0,0,0 is world center and sea level.
- Space uses Cartesian positions plus Keplerian on-rails orbits.
- Construction is unified across vehicles/buildings/machines.
- Blueprints are operations/plans, not world copies.
- Manual and queued work both flow through jobs.
- Setup should use native OS installer style where possible.
- Tools/SDKs/editors are first-class products.

Active Plans or Workstreams

Domino engine, Dominion product/game layer,
Repository structure, ABI/API spine, Platform API and

backends, Renderer API and backends, Audio API and backends, Unified UI system, Product contracts and runtime selection, Packages, instances, mods, packs, Setup product, Launcher product, Tools and SDKs, Game product

Decisions Already Made

See Decision Register in full report. Highest priority: DECISION-03, DECISION-05, DECISION-07, DECISION-08, DECISION-12, DECISION-19, DECISION-28.

Pending Tasks

First: verify repo state. Then apply Phase 1 prompts in order.

Open Questions

Key unresolved issues: schema format, scripting VM, default UPS, actual repo state, Carbon OS APIs, retro backend acceptance criteria, save/replay format, construction graph storage.

Files / Artifacts / Prompts to Preserve

Phase 1 prompts, platform prompt template, renderer prompt template/vtable/softref, setup prompts, launcher

prompts, tools prompts, Phase 7+ roadmap, numeric/construction/seamless transition specs.

What to Verify Before Acting

Repo tree/build state, README scale, schema format, external packaging/render/platform tools, dependency licenses.

Best Next Step

Verify the actual repository state, then begin Phase 1 with the Domino ABI/API prompt or ask for detailed Phase 7 prompts only if intentionally skipping implementation order.

Chapter 7

Dominium Complete Conversation

7.1 Dominium Canon Portability Audit Human Readable Report

Source: docs/archive/conversations/Dominium_Complete_Conversation/Accompanying_Report/Dominium_Canon_Portability_Audit__01_human_readable_report.md

Report type: human_readable_report

Size: 35882 bytes

7.1.1 COMPLETE CHAT PRESERVATION REPORT - Dominium Canon, Repository Alignment, and Portability Doctrine

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominium Canon, Repository Alignment, and Portability Doctrine

- Row 2** Field: Date anchor; Assessment: 2026-05-27
Australia/Melbourne
- Row 3** Field: Source scope; Assessment: This chat only
unless labelled PROJECT-CONTEXT
- Row 4** Field: Apparent access; Assessment: Partial:
current visible conversation plus uploaded
preservation prompt; not guaranteed full hidden
historical transcript
- Row 5** Field: Previously generated files available?;
Assessment: No earlier generated downloads in this
chat; this response creates files
- Row 6** Field: Uploaded files or artifacts present?;
Assessment: Yes: `Pasted text.txt` preservation
prompt
- Row 7** Field: Contains future plans?; Assessment: Yes
- Row 8** Field: Contains decisions?; Assessment: Yes, but
some are assistant recommendations and must
remain labelled
- Row 9** Field: Contains pending tasks?; Assessment: Yes
- Row 10** Field: Contains unresolved questions?;
Assessment: Yes
- Row 11** Field: Staleness risk; Assessment: Medium:
live repo state and external examples can change

Row 12 Field: Extraction confidence; Assessment: 4/5 for visible chat; 2/5 for any unavailable prior context

Row 13 Field: Safe for later aggregation?; Assessment: With caveats

Row 14 Field: Main limitations; Assessment: No full local checkout/test run; current repo inspection was partial; some project context comes from system memory and must be labelled PROJECT-CONTEXT

Plain-language limitation summary: I can preserve the visible current conversation, the uploaded `Pasted text.txt` prompt, and the repository evidence already inspected through the GitHub connector in this session. I cannot guarantee access to every older hidden transcript, expired upload, or external file from other chats. Where this report uses project-level memory about Dominion’s broader architecture, it is labelled PROJECT-CONTEXT or treated as background rather than direct proof.

1. One-Page Orientation

This chat was about preserving and re-grounding Dominion’s architecture around a very old constitutional contract, then checking whether the

current GitHub repository still follows that contract, and finally broadening the question into a platform-engineering doctrine for portability, modularity, extensibility, reuse, future-proofing, and long-term compatibility.

The chat began with the user pasting the old "DOMINIUM CONSTITUTIONAL ARCHITECTURE & EXECUTION CONTRACT v1" and the old "CANONICAL GLOSSARY v1". Those documents defined Dominion as a deterministic universe simulation platform built around Domino as a C89 deterministic engine, Dominion as a C++98 game layer, product applications such as Client/Server/Setup/Launcher, and XStack as a governance layer. The v1 ontology reduced all systems to Assemblies, Fields, Processes, Agents, and Law. It also insisted on fixed-point authoritative math, named RNG streams, thread-count invariance, replay hash equivalence, explicit AuthorityContext, LawProfile-driven behavior rather than runtime mode flags, and TruthModel -> PerceivedModel -> RenderModel separation. The glossary locked the vocabulary around terms such as Authority, LawProfile, Lens, Process, SessionSpec, UniverseIdentity, UniverseState, Macro Capsule, SRZ, XStack, RepoX, TestX, AuditX, CompatX, SecureX, and related concepts.

The user then asked how the current GitHub repository `julesc013/dominium` aligned with that old v1 contract

and glossary, asking specifically: what do the docs say, and what does the code do? The assistant inspected the live repository through the GitHub connector. The main finding was that the old v1 contract and glossary had not been discarded; they had been materialized into repo canon under `docs/canon/constitution_v1.md` and `docs/canon/glossary_v1.md`. The repo also had a newer canon-index regime saying prompts themselves are not authoritative unless materialized as canonical documents. This mattered because it changed the authority model: the old pasted prompt is historically important, but the repo version of the contract is the operational source of truth. The audit found strong alignment in documents, schemas, registries, governance, deterministic build constraints, and client session-pipeline code, but only partial implementation of the full runtime gameplay model. Survival fields and processes exist as registries; full agent gameplay loops, MMO/SRZ distributed runtime, embodiment, and other advanced systems were still described as deferred or future work. The user then reframed the issue around engineering quality: they wanted the code to be portable, modular, extensible, reusable for other games on the same Domino engine, reusable for different engine or game projects, and replaceable down to files and directories if a rewrite/refactor is needed. They explicitly said the project should be developed like a proper game or OS, not a

one-off indie project. The assistant's answer converted that into a doctrine: stable contracts, replaceable implementations, deterministic behavior, portable projections, and no accidental authority from paths, tools, UIs, or prompts. The answer emphasized that the goal is not to freeze every file, but to make every boundary explicit, versioned, testable, and replaceable. It identified the ownership-based source layout as broadly right: `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `docs/`, `tests/`, `tools/`, `scripts/`, `cmake/`, `external/`, `release/`, and `archive/`. It also identified remaining cleanup debt and recommended finishing ownership-contract convergence rather than starting a grand rewrite.

The unique value of this chat is that it bridges three layers: old constitutional intent, current repository reality, and future engineering doctrine. It is not merely a summary of Dominium's game design. It is a portability and governance checkpoint: what is canon, what is implemented, what remains declarative, what structure should remain, what structure still needs cleanup, and what practices should govern future coding.

2. The Story of the Conversation

The conversation opened with a deliberate reconstitution of canon. The user supplied an old self-contained constitutional architecture prompt for Dominium and

Domino. The assistant acknowledged it as authoritative within the conversation. The user then supplied a canonical glossary v1.0.0, and the assistant acknowledged that the glossary constrained future terms.

The next substantive turn asked for an alignment audit against the live GitHub repository. This changed the task from "accept the old constitution" to "measure the current repo against it." The assistant used the GitHub connector to inspect Julesc013/dominium, including README.md, docs/STATUS_NOW.md, docs/architecture/ARCHO_CONSTITUTION.md, docs/architecture/CANON_INDEX.md, docs/canon/constitution_v1.md, docs/canon/glossary_v1.md, docs/ARCHITECTURE.md, docs/XSTACK.md, CMakeLists.txt, client session-pipeline code, RNG code, world/sim code, survival registries, experience profiles, law profiles, and schemas for SessionSpec and AuthorityContext. The resulting assessment was nuanced: the repo strongly preserved the v1 constitution and glossary at the documentation and contract layer, but the implementation was not yet a full realization of the old vision. The repo had deterministic scaffolding, schema/registry discipline, C90/C++98 build constraints, XStack-style governance, and a real deterministic session pipeline. However, advanced gameplay and distributed-scale claims were mostly still future-facing.

The final major turn shifted from "does it align?" to

"what practices should we follow to make it robust for decades?" The user asked for broad and deep guidance on portability, modularity, extensibility, structure, file names, function names, schemas, protocols, APIs, big-company practices, future-proofing, backward compatibility, and prior related discussions. The assistant inspected newer repo layout/naming documents including ARCH_REPO_LAYOUT.md, REPO_LAYOUT_TARGET.md, contracts/repo/layout.contract.toml, contracts/repo/naming.contract.toml, OWNERSHIP_RULES.md, no_src_source_policy.md, file_naming.md, module_layout.md, language_ownership.md, and POST_CONVERGE_NEXT_STEPS.md. The answer concluded that the current ownership-root model is directionally right, while cleanup remains unfinished. It recommended stabilizing contracts rather than implementations, using explicit module descriptors, stable C89 ABI boundaries, versioned schemas and protocols, command/refusal spines, generated-artifact reproducibility, a second "minimal game on Domino" reuse proof, and strict separation of engine/game/runtime/apps/contracts/content/tools.

This preservation task was then uploaded as Pasted text.txt. It requested a maximum-fidelity report package for the current chat, with human-readable explanation first, structured registers, context-transfer packet, spec sheet, aggregator packet, self-audit, and

downloadable files.

3. Main Topics Discussed

Topic 1 - Old v1 Constitutional Architecture The old v1 contract established the original architecture language: Domino as deterministic C89 engine, Dominium as C++98 game layer, applications as product surfaces, and XStack as governance. It came up because the user needed old custom instructions restored or preserved. The most important conclusion was not merely "this old prompt exists," but that its concepts remain central: determinism, Process-only mutation, LawProfile and AuthorityContext, no omniscience, no mode flags, data-defined domains, and explicit governance.

Uncertainty: the raw pasted prompt is not the current repo authority unless materialized in repo canon. The later audit found that it had in fact been materialized under `docs/canon/constitution_v1.md`.

Topic 2 - Canonical Glossary and Terminology Control The glossary bound terms such as Authority, Law, Process, Lens, SessionSpec, UniverseIdentity, Macro Capsule, SRZ, RepoX, TestX, AuditX, CompatX, SecureX, and related terms. It matters because future assistants must not use sloppy synonyms like "mode"

where the canon requires ExperienceProfile or LawProfile. This topic directly supports modularity because stable vocabulary is part of stable architecture.

Topic 3 - Current Repository Alignment The user asked what the current GitHub repo says and what the code does. The audit found strong documentation/contract alignment, meaningful deterministic code scaffolding, and partial runtime implementation. Key repo docs said v1 canon and glossary are current canonical docs; the README and status pages are derived/provisional; and the repo contains a large canon index. Code inspection found C90/C++98 build settings, deterministic RNG stream support, tile-world save/load/checksum code, a deterministic client session pipeline, and registry-defined survival slice data.

The critical conclusion was: **the repo has strong architecture scaffolding, but the full game/platform vision is not yet implemented.**

Topic 4 - Portability, Modularity, and Extensibility Doctrine The user then asked what practices would make the code reusable for other games and even other engine projects. The assistant answered that the correct goal is not to make all files permanent, but to make boundaries explicit and stable. The doctrine became: stable contracts, replaceable implementations,

deterministic behavior, portable projections, and no accidental authority from paths/tools/UIs/prompts.

Topic 5 - Repository Layout and Naming The chat inspected repo layout/naming documents and concluded that the active ownership-root structure is broadly right: `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `docs/`, `tests/`, `tools/`, `scripts/`, `cmake/`, `external/`, `release/`, and `archive/`. Generic roots like `src/`, `source/`, `common/`, `shared`, and `misc` were rejected because they hide ownership.

Uncertainty: during this preservation pass, a potential inconsistency appeared: some docs claim product roots moved under `apps/`, while an earlier inspected code path under root `client/` was available and `apps/client` was not fetched successfully. This must be verified against the current physical tree.

Topic 6 - API, ABI, Schema, and Protocol

Practices The assistant recommended stable public C-compatible APIs for Domino, opaque handles, explicit struct sizes, no platform types in engine headers, explicit allocator/lifetime ownership, deterministic error/refusal codes, and no C++ ABI exposure across long-term stable boundaries. It also recommended versioned schemas, registries, manifests, save/replay formats, compatibility matrices, and migration/refusal rules.

Topic 7 - Preservation and Aggregation The uploaded prompt requested this full preservation package. It requires a human-readable report, registers, context transfer packet, spec sheet, aggregator packet, self-audit, exported files, and an in-chat reader.

4. What We Were Trying to Achieve

4.1 Explicit Goals The explicit goals were to restore/preserve old Dominion canon, assess current repository alignment, understand what the docs and code actually say/do, and identify practices to make the code portable, modular, extensible, replaceable, future-proof, and backward compatible.

4.2 Inferred Goals The inferred goal is to prevent Dominion from becoming a one-off fragile indie codebase. The user wants a durable platform architecture closer to a game engine/product family or OS-style codebase, where pieces can be replaced without breaking identity, compatibility, or determinism.

4.3 Goals That Changed Over Time The chat moved from accepting old canon, to auditing live repo reality, to producing broad engineering doctrine, to preserving the whole exchange for later aggregation.

4.4 Goals Still Unresolved Unresolved goals include verifying the latest physical repo tree, proving runtime implementation maturity, formalizing public ABI/API boundaries, completing layout/naming cleanup, and building a second Domino-based product to prove reuse.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Treat old v1 Constitution and Glossary as the comparison baseline for this chat.; Status: Accepted in-chat; repo authority later refined.; Why it mattered: Provides baseline for audit but must not override repo-materialized canon in live repo.; Confidence: 4; Label: FACT

Row 2 ID: DECISION-02; Decision: Prefer repo-materialized canonical documents over raw prompts for current repository authority.; Status: Established by audit; not directly stated by user, but supported by repo docs.; Why it mattered: Future work should cite `docs/canon/*` and canon index, not raw chat memory alone.; Confidence: 4; Label: FACT/INFERENCE

Row 3 ID: DECISION-03; Decision: Use ownership-based source roots instead of generic `src//source/` layout.; Status: Recommended and supported by current repo naming/layout docs.; Why it

mattered: Preserve apps/ engine/ game/
runtime/ contracts/ content/ ... model.;
Confidence: 4; Label: FACT/INFERENCE

Row 4 ID: DECISION-04; Decision: Do not freeze all
code; freeze contracts and make implementations
replaceable.; Status: Assistant recommendation;
user intent strongly supports it.; Why it mattered:
Directs future architecture toward ABI/schema/
protocol boundaries.; Confidence: 4; Label:
INFERENCE

Row 5 ID: DECISION-05; Decision: Treat survival/
runtime gameplay as not fully implemented yet
despite strong declarative scaffold.; Status: Audit
conclusion, not a user decision.; Why it mattered:
Avoid overstating implementation maturity.;
Confidence: 4; Label: INFERENCE

Row 6 ID: DECISION-06; Decision: Create this
preservation package with human-readable report
plus structured exports.; Status: Triggered by
uploaded preservation prompt.; Why it mattered:
This answer creates the artifact set.; Confidence: 5;
Label: FACT

Decision explanations

- **DECISION-01:** The old v1 contract/glossary
were accepted as the baseline for this chat because

the user supplied them as the old ground truth and asked later questions against them.

- **DECISION-02:** The live repo audit refined authority: the repo-materialized canon files and canon index are stronger for current repository work than raw prompts.
- **DECISION-03:** Ownership-root layout was preferred because it encodes who owns a file or subsystem, while generic `src/source` hides responsibility.
- **DECISION-04:** Stable contracts plus replaceable implementations best satisfy the user's desire for rewrites/refactors without losing compatibility.
- **DECISION-05:** The repo should not be described as fully implementing the old v1 vision yet; many systems are still declarative, scaffolded, or deferred.
- **DECISION-06:** The uploaded preservation prompt directly requested a package, so this response creates one.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

Row 1 ID: REJECTED-01; Option: Treat raw chat prompts as stronger than repo-materialized canon.; Status: Superseded; Reason: Repo canon index

says prompts are execution artifacts unless materialized.; Final or tentative?: Final for current repo authority unless canon changes.; Reconsider conditions: Only reconsider if repo authority order changes.; Related workstream: WORKSTREAM-01; Label: FACT

Row 2 ID: REJECTED-02; Option: Use generic `src/` or `source/` root as the active layout.; Status: Rejected/superseded; Reason: Current naming/no-src policy says ownership roots are active model.; Final or tentative?: Final for new active source work.; Reconsider conditions: Only reconsider by explicit repo contract amendment.; Related workstream: WORKSTREAM-04; Label: FACT

Row 3 ID: REJECTED-03; Option: Treat docs and registries as proof that full gameplay is implemented.; Status: Rejected as unsafe; Reason: Audit separated docs/schema from code behavior.; Final or tentative?: Final as epistemic discipline.; Reconsider conditions: Reconsider after runtime tests prove behavior.; Related workstream: WORKSTREAM-02; Label: INFERENCE

Row 4 ID: REJECTED-04; Option: Perform a grand rewrite/refactor without gated ownership migration.; Status: Deprioritised; Reason: Would risk breaking identity, imports, ABI, and

compatibility.; Final or tentative?: Tentative;
controlled rewrites remain allowed.; Reconsider
conditions: Only after contracts/tests/migration
plan exist.; Related workstream:
WORKSTREAM-04; Label: INFERENCE

7. Important Reasoning, Rationale, and Tradeoffs

The main tradeoff is between stability and replaceability. The user wants code and directories to be fully replaceable, but also wants long-term compatibility. The resolution is to stabilize contracts, identifiers, schemas, save/replay formats, command/refusal protocols, and tests while allowing implementations and storage paths to move behind those boundaries.

Another tradeoff is between a clean theoretical structure and the current repo's transitional debt. The current ownership-root target is good, but the repo still has cleanup work. The safe path is not broad movement by directory name; it is gated move/refactor batches with identity preservation, shims where needed, strict validators, and proof outputs.

A third tradeoff is between docs ambition and runtime proof. The docs are far ahead, which is acceptable for planning, but dangerous if treated as implementation fact. Future work must always separate "declared contract," "schema/registry exists," "code exists," "tests

pass," and "product behavior proven."

8. Plans, Future Work, and Next Steps

Recommended next-action sequence:

1. Verify the current physical repo tree against layout docs and `contracts/repo/layout.contract.toml`.
2. Run or inspect strict layout/root validators and active exception ledger.
3. Build a public API/ABI inventory for Domino, Dominium, runtime services, tools, and data contracts.
4. Audit survival/runtime Process execution to separate registry declarations from implemented behavior.
5. Create a compatibility policy map for schemas, saves, replays, packs, command packets, and ABI.
6. Add a second minimal Domino product/game as a reuse proof.
7. Continue move/refactor cleanup in small gated batches only.

8. Merge this chat into the master Project Spec Book using the aggregator packet, not by copying every assistant recommendation as a requirement.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences

Row 1 ID: CONSTRAINT-01; Constraint:

Determinism remains primary: fixed-point authoritative math, named RNG, replay/hash equivalence, thread-count invariance.; Type: Technical/canon; Hard/soft: Hard; Source / basis: Old v1 contract and live repo constitution; Practical implication: Do not introduce nondeterministic authoritative behavior.; Violation risk: High; Confidence: 5; Label: FACT

Row 2 ID: CONSTRAINT-02; Constraint: Process-

only mutation and law-gated authority.; Type: Technical/canon; Hard/soft: Hard; Source / basis: v1 and live constitution; Practical implication: All authoritative state changes must route through processes and AuthorityContext/LawProfile.; Violation risk: High; Confidence: 5; Label: FACT

Row 3 ID: CONSTRAINT-03; Constraint: No runtime mode flags; behavior derives from

ExperienceProfile/LawProfile/ParameterBundle.;

Type: Technical/canon; Hard/soft: Hard; Source / basis: v1 and live constitution; Practical implication: Avoid *_mode gameplay branches and mode-specific logic forks.; Violation risk: Medium; Confidence: 5; Label: FACT

Row 4 ID: CONSTRAINT-04; Constraint: TruthModel -> PerceivedModel -> RenderModel separation.; Type: Architecture; Hard/soft: Hard; Source / basis: v1 and repo docs; Practical implication: Renderer/runtime must not access or mutate truth directly.; Violation risk: High; Confidence: 5; Label: FACT

Row 5 ID: CONSTRAINT-05; Constraint: Ownership-root layout; no new generic `src/source/common/shared/misc` buckets.; Type: Repository structure; Hard/soft: Hard for new work; Source / basis: Naming/layout contracts; Practical implication: New files must be placed by ownership and role.; Violation risk: Medium; Confidence: 4; Label: FACT

Row 6 ID: CONSTRAINT-06; Constraint: Source repo layout is not install/package/runtime projection layout.; Type: Repository/distribution; Hard/soft: Hard; Source / basis: Layout docs and no-src policy; Practical implication: Do not infer runtime store/save/export roots from source roots.;

Violation risk: Medium; Confidence: 4; Label: FACT

Row 7 ID: CONSTRAINT-07; Constraint: Content/packs must not contain untrusted executable authority.; Type: Security/architecture; Hard/soft: Hard; Source / basis: Pack-driven integration doctrine; Practical implication: Future extension code needs separate trust/capability contract.; Violation risk: High; Confidence: 4; Label: INFERENCE

Row 8 ID: CONSTRAINT-08; Constraint: Human-readable explanation must come before machine-readable handoff in preservation tasks.; Type: Communication; Hard/soft: Hard for this task; Source / basis: Uploaded preservation prompt; Practical implication: This output must be understandable without downloading files.; Violation risk: Medium; Confidence: 5; Label: FACT

9.2 Inferred Constraints and Preferences The user likely wants future assistants to challenge weak framing, preserve uncertainty, avoid shallow "best practice" lists, and focus on decisions that survive future rewrites.

9.3 Uncertain or Unestablished Preferences It is not established whether the user wants to prioritize repo cleanup, API formalization, or second-product reuse proof first. That should be resolved by a future task selection.

10. Files, Artifacts, Outputs, and Prompts

Row 1 ID: ARTIFACT-01; Artifact / file / prompt / output: DOMINIUM CONSTITUTIONAL ARCHITECTURE & EXECUTION CONTRACT v1; Type: Pasted prompt/doc; Purpose: Baseline architecture contract for chat comparison.; Status: Used in chat; repo has canonical materialized version.; Origin: User pasted in current chat; Carry forward?: Yes; Notes: Treat raw prompt as baseline but repo canon as current authority.; Label: FACT

Row 2 ID: ARTIFACT-02; Artifact / file / prompt / output: DOMINIUM CANONICAL GLOSSARY v1.0.0; Type: Pasted prompt/doc; Purpose: Vocabulary authority for current chat.; Status: Used in chat; repo has canonical materialized version.; Origin: User pasted in current chat; Carry forward?: Yes; Notes: Preserve definitions and deprecated/reserved words.; Label: FACT

Row 3 ID: ARTIFACT-03; Artifact / file / prompt / output: Repository alignment audit answer; Type:

Chat output; Purpose: Compared live repo docs/
code to old v1.; Status: Produced in chat, not a file
until this package.; Origin: Assistant; Carry
forward?: Yes; Notes: Key finding: strong docs/
schema alignment, partial implementation.; Label:
FACT

Row 4 ID: ARTIFACT-04; Artifact / file / prompt /
output: Portability/modularity/future-proofing
doctrine answer; Type: Chat output; Purpose:
Broad guidance on coding practices and repo
structure.; Status: Produced in chat, not a file
until this package.; Origin: Assistant; Carry
forward?: Yes; Notes: Contains recommendations;
not all are user decisions.; Label: FACT

Row 5 ID: ARTIFACT-05; Artifact / file / prompt /
output: Uploaded Pasted text.txt; Type:
Uploaded text file; Purpose: Mega prompt
instructing this preservation/export task.; Status:
Current attachment read successfully.; Origin: User
upload; Carry forward?: Yes; Notes: This task's
direct instruction source.; Label: FACT

Row 6 ID: ARTIFACT-06; Artifact / file / prompt /
output: GitHub connector source observations;
Type: Connector output; Purpose: Evidence from
repo files used for audit.; Status: Available in
current chat through citations; not a local repo

checkout.; Origin: Assistant/tool; Carry forward?: Yes; Notes: Partial because not full tree/local test run.; Label: FACT

11. Open Questions and Unresolved Issues

Row 1 ID: QUESTION-01; Question / issue: Which repo layout state is current where docs claim product roots moved but earlier fetched code still appeared under root `client/`?; Why it matters: This affects source-layout truth and cleanup planning.; Known information: Docs say `apps/` is canonical; earlier code fetch succeeded under `client/`; `apps/client` fetch failed in this session.; Unknown information: Whether the connector saw stale docs, old paths, branch drift, or unresolved repo inconsistency.; Resolution path: Run full tree listing/local checkout validation.; Priority: P0; Related workstream: WORKSTREAM-04; Label: UNCERTAIN

Row 2 ID: QUESTION-02; Question / issue: How much of the survival vertical slice is implemented as runtime Process execution rather than registries?; Why it matters: Determines implementation maturity.; Known information: Fields/process IDs/law profiles exist in registries.; Unknown information: Concrete process executor/handlers/tests status.; Resolution path: Inspect

game/runtime process code and tests.; Priority: P1; Related workstream: WORKSTREAM-06; Label: UNCERTAIN

Row 3 ID: QUESTION-03; Question / issue: What are the exact public API/ABI boundaries for Domino vs Dominion vs runtime?; Why it matters: Needed for reuse and backward compatibility.; Known information: General doctrine and some headers exist.; Unknown information: Formal ABI stability map and public/private header inventory.; Resolution path: Create/API audit and contract map.; Priority: P1; Related workstream: WORKSTREAM-05; Label: INFERENCE

Row 4 ID: QUESTION-04; Question / issue: What remaining bad roots/exceptions exist today?; Why it matters: Necessary before feature expansion/refactors.; Known information: Docs mention exceptions and cleanup waves.; Unknown information: Current exact count/status at HEAD. ; Resolution path: Run strict layout validators and read exception ledger.; Priority: P1; Related workstream: WORKSTREAM-04; Label: UNCERTAIN

Row 5 ID: QUESTION-05; Question / issue: What should become formal requirements in the master Project Spec Book vs background context?; Why it

matters: Prevents over-formalizing suggestions.;
Known information: This chat contains both
accepted canon and assistant recommendations.;
Unknown information: User acceptance of specific
practices beyond high-level preference.; Resolution
path: Aggregator review with labels preserved.;
Priority: P1; Related workstream:
WORKSTREAM-08; Label: INFERENCE

Row 6 ID: QUESTION-06; Question / issue: Should a
second minimal Domino product be built now or
after layout cleanup?; Why it matters: Determines
next practical proof.; Known information:
Assistant recommended it as reuse proof.;
Unknown information: User/project priority and
current build readiness.; Resolution path: Decision
by project owner after current blockers reviewed.;
Priority: P2; Related workstream:
WORKSTREAM-07; Label: UNCERTAIN

**12. Risks, Failure Modes, and What Future Chats
Might Get Wrong**

Row 1 ID: RISK-01; Risk / failure mode: Treating repo
docs as fully implemented behavior.; Consequence:
Overstates maturity and causes bad planning.;
Likelihood: Medium; Severity: High; Mitigation:
Separate docs/schema/code/test status.; Related
workstream: WORKSTREAM-02; Label: FACT/

INFERENCE

Row 2 ID: RISK-02; Risk / failure mode: Treating assistant recommendations as accepted decisions.; Consequence: Spec book may formalize unapproved practices.; Likelihood: Medium; Severity: High; Mitigation: Use decision status and labels.; Related workstream: WORKSTREAM-08; Label: FACT

Row 3 ID: RISK-03; Risk / failure mode: Breaking semantic IDs during path cleanup.; Consequence: Saves/packs/contracts/replays may become incompatible.; Likelihood: Medium; Severity: High; Mitigation: Path moves must preserve IDs unless explicit migration.; Related workstream: WORKSTREAM-04; Label: INFERENCE

Row 4 ID: RISK-04; Risk / failure mode: Creating new generic buckets during refactor.; Consequence: Ownership ambiguity returns.; Likelihood: Medium; Severity: Medium; Mitigation: Use layout/naming contracts and validators.; Related workstream: WORKSTREAM-04; Label: FACT/INFERENCE

Row 5 ID: RISK-05; Risk / failure mode: Hidden coupling prevents Domino reuse by other games.; Consequence: Engine becomes Dominion-specific.; Likelihood: Medium; Severity: High; Mitigation: Add second minimal product reuse proof.; Related

workstream: WORKSTREAM-07; Label:
INFERENCE

Row 6 ID: RISK-06; Risk / failure mode: Runtime/
platform/UI layers gain truth authority.;
Consequence: Violates canon and harms
multiplayer fairness/determinism.; Likelihood:
Medium; Severity: High; Mitigation: Enforce
Truth/Perceived/Render and command/refusal
spine.; Related workstream: WORKSTREAM-03;
Label: FACT/INFERENCE

Row 7 ID: RISK-07; Risk / failure mode: Current
preservation is incomplete because full historical
transcript is unavailable.; Consequence: Future
aggregation may miss context outside visible chat.;
Likelihood: Medium; Severity: Medium;
Mitigation: State coverage limitation and use
PROJECT-CONTEXT labels.; Related
workstream: WORKSTREAM-08; Label: FACT

Main prevention rule: do not compress this chat into
"repo mostly aligns with canon." The more precise
takeaway is: **canon/docs/schemas/gates are strong,
runtime implementation is partial, layout
direction is good, cleanup remains, and
future-proofing requires stable contracts plus
replaceable implementations.**

13. What This Chat Contributes to the Larger Project or Future Spec Book

Row 1 Spec-book area: Architecture Constitution;
Contribution from this chat: Use v1 Constitution/
Glossary and live repo canon hierarchy as
foundational source hierarchy.; Source IDs:
DECISION-01, DECISION-02; Should become
requirement/context/open issue?: Requirement/
context; Confidence: 4; Notes: Preserve repo-
materialized canon as stronger than raw prompts.

Row 2 Spec-book area: Repository Layout and
Ownership; Contribution from this chat:
Ownership-root layout and no-src/source policy.;
Source IDs: DECISION-03, CONSTRAINT-05;
Should become requirement/context/open issue?:
Requirement; Confidence: 4; Notes: Verify current
tree/exception status before finalizing.

Row 3 Spec-book area: Portability and API Doctrine;
Contribution from this chat: Stable contracts,
replaceable implementations, C ABI, schema/
protocol versioning.; Source IDs: DECISION-04,
TASK-02, TASK-03; Should become requirement/
context/open issue?: Requirement candidate;
Confidence: 3; Notes: Some parts are assistant
recommendations, not explicit user decisions.

Row 4 Spec-book area: Implementation Status;

Contribution from this chat: Docs/schema strong; runtime gameplay partial; advanced systems deferred.; Source IDs: DECISION-05, QUESTION-02; Should become requirement/context/open issue?: Open issue/context; Confidence: 4; Notes: Avoid overstating current implementation.

Row 5 Spec-book area: Governance and Verification; Contribution from this chat: Use XStack/RepoX/TestX/AuditX and strict validators as gates.; Source IDs: CONSTRAINTS, TASK-06; Should become requirement/context/open issue?: Requirement/context; Confidence: 4; Notes: Latest status must be verified.

Row 6 Spec-book area: Reuse Proof; Contribution from this chat: Add second minimal game/product on Domino.; Source IDs: TASK-04; Should become requirement/context/open issue?: Open issue/requirement candidate; Confidence: 3; Notes: Needs user/project prioritization.

14. What I Should Remember

- The old v1 contract and glossary remain central, but current repo authority is materialized canon, not raw prompts.
- The current repo has strong deterministic/governance/schema scaffolding.

- The current repo should not be overstated as a complete runtime implementation of all v1 goals.
- The ownership-root layout is broadly right and better than a generic `src/` tree.
- The project should stabilize contracts and IDs, not file paths or implementation details.
- Portability depends on C-compatible public boundaries, no platform leakage into engine truth, data-driven domains, versioned schemas, and stable save/replay formats.
- The safest next technical move is a current repo layout/API/implementation audit, not a broad rewrite.
- This chat should feed the master spec book's architecture, repository layout, API doctrine, governance, and implementation-status sections.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- "Give me the shortest accurate version of the repo alignment audit."
- "What did this chat prove, and what did it only infer?"

15.2 Decisions

- "Which recommendations are actual decisions and which are only assistant suggestions?"
- "Which decisions should become formal requirements in the master spec book?"

15.3 Tasks and Next Actions

- "Turn the next-action sequence into a Codex task prompt."
- "Create a gated plan for verifying current repo layout and exceptions."

15.4 Artifacts and Files

- "Explain each file in the preservation package."
- "Extract only the aggregator packet from this package."

15.5 Risks and Verification

- "What are the biggest ways future assistants could overstate implementation maturity?"
- "What must be verified before claiming the repo has completed layout convergence?"

15.6 Future Spec Book / Aggregation

- "Which sections of a Dominion Project Spec Book should this chat feed?"
- "Convert the decisions and constraints into formal spec requirements."

15.7 Deep-Dive Questions Specific to This Chat

- "Design the second minimal Domino product reuse proof."
- "Draft a public C89 Domino ABI contract."
- "Audit the survival vertical slice from registry to runtime execution."

16. Compact Human Summary

This chat re-grounded Dominion around its old v1 constitution and glossary, then checked the current GitHub repository against that old canon, and finally broadened into a doctrine for building Dominion as a reusable platform rather than a one-off game. The old v1 contract defined Dominion as a deterministic universe simulation built on the Domino C89 engine, with a C++98 game layer, product applications, and XStack governance. It required deterministic math, named RNG streams, thread-count invariance, replay equivalence,

LawProfile-driven behavior, AuthorityContext gating, Process-only mutation, no omniscience, and TruthModel -> PerceivedModel -> RenderModel separation. The glossary supplied canonical vocabulary and deprecated sloppy mode-flag terminology.

The live repository audit found that these old ideas have been materialized in repo canon, especially `docs/canon/constitution_v1.md` and `docs/canon/glossary_v1.md`. It also found a newer rule: raw prompts are not authoritative unless materialized as canonical docs. The repo has strong documentation, schema, registry, build, and governance alignment, including C90/C++98 build discipline, XStack/RepoX/TestX/AuditX scaffolding, deterministic RNG code, world save/load/checksum scaffolding, SessionSpec and AuthorityContext schemas, and a deterministic client session pipeline. But the implementation is not yet the full old v1 vision. Survival exists strongly as registries and LawProfiles; full agent gameplay loops, embodiment, MMO-scale distributed authority runtime, and other advanced systems remain incomplete, deferred, or future-facing.

The later portability discussion established the most important engineering doctrine: the project should not try to freeze all code and directories. It should freeze contracts, semantic IDs, public APIs, save/replay formats, schemas, command/refusal protocols, and verification gates, while making implementations

replaceable. The current ownership-root layout is broadly the right direction: `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `docs/`, `tests/`, `tools/`, `scripts/`, `cmake/`, `external/`, `release/`, and `archive/`. Generic roots such as `src`, `source`, `common`, `shared`, and `misc` should not be used for active code because they hide ownership.

The most important unresolved issue is to verify the current physical repo tree and active exceptions. The docs say convergence has moved product roots under `apps/`, but current inspection in this session produced a possible inconsistency around old `client/` paths. That should be checked with a full tree listing or local checkout. Other priorities are to formalize stable API/ABI boundaries, audit which survival/gameplay processes are actually implemented, define data compatibility/migration rules, and eventually add a second minimal Domino-based product as a reuse proof.

The best next action is a current-state structural audit: tree layout, exception ledger, public API inventory, schema/version inventory, and runtime implementation proof. Only after that should broad cleanup or feature expansion proceed.

7.2 Dominium Canon Portability Audit Reader Brief

Source: docs/archive/conversations/Dominium_Complete_Conversation/Accompanying_Report/Dominium_Canon_Portability_Audit__06_reader_brief.md

Report type: reader_brief

Size: 2216 bytes

7.2.1 Reader Brief - Dominium Canon, Repository Alignment, and Portability Doctrine

What this chat was about

This chat restored old Dominium v1 canon, audited current GitHub repository alignment, and developed a future-proofing doctrine for building Dominium/Domino as a reusable platform rather than a one-off game.

Top 20 things to know

1. The old v1 Constitution and Glossary were pasted and used as baseline.
2. The live repo has materialized versions under docs/canon/constitution_v1.md and docs/canon/glossary_v1.md.

3. Repo canon index says raw prompts are not authoritative unless materialized.
4. The repo has strong docs/schema/registry/governance alignment.
5. The repo has real deterministic code scaffolding, including RNG/world/session pipeline pieces.
6. Full gameplay/runtime implementation is not yet proven.
7. Survival exists strongly in registries, but concrete runtime Process implementation needs audit.
8. Advanced systems such as MMO distributed authority are not proven complete.
9. The user wants portability, modularity, extensibility, reuse, and replaceability.
10. The correct doctrine is stable contracts and replaceable implementations.
11. Ownership-root layout is preferred over generic `src/source`.
12. Current target roots include `apps`, `engine`, `game`, `runtime`, `contracts`, `content`, `docs`, `tests`, `tools`, `scripts`, `cmake`, `external`, `release`, `archive`.

13. Paths are storage; IDs/contracts define semantic identity.
14. Public Domino boundaries should be C-compatible and stable.
15. Schemas/protocols/saves/replays/packs need versioning and migration/refusal rules.
16. Apps should stay thin.
17. Runtime must not own simulation truth.
18. Tools must not become runtime dependencies.
19. Future cleanup must be gated and preserve IDs.
20. This package should feed the master spec book with labels preserved.

Best next step

Verify current repo physical layout, exception ledger, public API inventory, and runtime implementation status before broad refactors or new feature work.

Key files in this package

Read file 01 for the human explanation, file 04 for registers, file 05 for aggregator merge, and file 08 for future-chat bootstrap.

7.3 Dominium Canon Portability Audit In Chat Reader

Source: docs/archive/conversations/Dominium_Complete_Conversation/Accompanying_Report/Dominium_Canon_Portability_Audit__09_in_chat_reader.md

Report type: in_chat_reader

Size: 1760 bytes

7.3.1 In-Chat Reader - Dominium Canon, Repository Alignment, and Portability Doctrine

Package overview

This package preserves a chat about Dominium's old v1 canon, current repository alignment, and future-proofing practices. It includes a human-readable report, registers, context packet, spec sheet, aggregator packet, audit, bootstrap prompt, and reader guide.

File index

- 00_manifest: package index and caveats.
- 01_human_readable_report: main explanation.
- 02_context_transfer_packet: handoff for future

chat.

- 03_spec_sheet: YAML-style structured summary.
- 04_registers: workstreams, decisions, tasks, risks, verification queue.
- 05_aggregator_packet: compact merge packet.
- 06_reader_brief: shorter human brief.
- 07_verification_and_audit: self-audit and verification queue.
- 08_future_chat_bootstrap_prompt: prompt for new chat.
- 09_in_chat_reader: this guide.

Question menu

- What did the repo audit actually prove?
- What is only an inference?
- Which recommendations should become formal requirements?
- What should be verified before cleanup?
- How do we design the second Domino reuse proof?
- How should public API/ABI boundaries be written?

- What belongs in engine vs game vs runtime vs apps?
- How do we avoid overclaiming implementation maturity?

Top things to preserve

- Old v1 canon and glossary.
- Repo-materialized canon authority.
- Ownership-root layout.
- Stable contracts/replaceable implementations doctrine.
- Docs/schema/code/test separation.
- Verification queue.

Safest next actions

1. Verify current physical repo tree.
2. Run/check strict layout validators.
3. Inventory public APIs and schemas.
4. Audit survival process runtime.
5. Only then plan refactor/cleanup waves.

7.4 Dominium Entire Conversation Accompanying Report Accompanying Detailed Summary Report

Source: docs/archive/conversations/Dominium_Complete_Conversation/Accompanying_Report/Dominium_Entire_Conversation_Accompanying_Report__10_accompanying_detailed_summary_report.md

Report type: detailed_summary_report

Size: 32068 bytes

7.4.1 Accompanying Human-Readable Detailed Summary and Report - Dominium Canon, Repository Alignment, and Portability Doctrine

Date anchor: 2026-05-27 Australia/Melbourne

Generated: 2026-05-27 21:01 AEST

Scope: This conversation, plus the currently available preservation files in /mnt/data and the uploaded preservation prompt.

Epistemic status: mixed. Items marked FACT were visible in this conversation or present in the generated files. Items marked INFERENCE are reasoned summaries. Items marked UNCERTAIN /

UNVERIFIED require a fresh repo checkout, tool run, or user confirmation.

1. Executive summary

This conversation centered on Dominion/Domino as a long-lived, deterministic, reusable simulation and game-platform project. It began with the user re-establishing an old **Dominium Constitutional Architecture & Execution Contract v1** and an old **Canonical Glossary v1** as the baseline for discussion. Those documents defined the project as a deterministic universe simulation stack: Domino as the C89 engine, Dominion as the C++98 game layer, Client/Server/Setup/Launcher as products, and XStack as the governance and verification layer.

The user then asked how the current GitHub repository `julesc013/dominium` aligned with those old documents. A repository inspection was performed through the GitHub connector. The result was not that the repository fully implements the old vision, but that it has strongly **materialized** much of that vision into current canonical documents, schemas, registries, governance tools, CMake policy, deterministic world scaffolding, session pipeline code, and layout/naming contracts. The key distinction established in the conversation was: **docs and registries show**

architectural intent; runtime code and tests prove implementation maturity. The repository appeared far more mature as a constitutional/governance scaffold than as a complete game runtime.

The user then broadened the question. They emphasized that the code must be portable, modular, extensible, reusable for different games on the same Domino engine, and reusable in completely different engine or game projects. They wanted a proper game/platform/OS-like development model rather than a one-off indie-project codebase. The answer developed a doctrine around **stable contracts and replaceable implementations.** The central recommendation was not to freeze every directory or every file, but to stabilize public boundaries: ABIs, schemas, semantic IDs, save/replay formats, command/refusal protocols, pack contracts, build/release manifests, and compatibility rules. Implementations behind those boundaries should remain replaceable.

The discussion also examined repository layout. The active layout doctrine discovered in the repo favors ownership-based roots: `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `docs/`, `tests/`, `tools/`, `scripts/`, `cmake/`, `external/`, `release/`, and `archive/`. This was judged directionally better than a generic `src/` or `source/` structure because it encodes ownership directly. The live repo naming policy explicitly rejects new `src/`, `source`, `common`, `shared`,

`misc`, `impl`, `old`, `new`, `legacy`, and similar generic/status terms in active paths.

A preservation package was then generated. This current task adds an accompanying detailed report and bundles it with the earlier preservation files, the uploaded preservation prompt, a new manifest, and a ZIP package. Some previously uploaded files were reported expired by the file-search system. Those expired files cannot be included unless re-uploaded. The package therefore includes only files currently available in `/mnt/data` plus the newly generated files.

The highest-priority practical next step remains: **run a fresh repository audit from an actual current checkout** to verify physical layout, API boundaries, runtime implementation maturity, public contract surfaces, and test/build status.

2. Conversation chronology

2.1 Re-establishing the old constitutional baseline

FACT: The user first pasted a self-contained **Dominium Constitutional Architecture & Execution Contract v1**. It defined the system identity, ontology, structural layers, survival vertical slice, domain/solver architecture, XStack governance, performance doctrine, agentic development doctrine, UX philosophy, packaging/distribution model, long-term

scalability model, and forbidden behaviors.

Important old-v1 principles included:

- Domino is the deterministic universe engine in C89.
- Dominion is the game layer in C++98.
- Applications include Client, Server, Setup, and Launcher.
- XStack governs development, audit, verification, compatibility, performance, security, and remediation.
- All simulation systems reduce to Assemblies, Fields, Processes, Agents, and Law.
- Process-only mutation is mandatory.
- AuthorityContext is required before intent execution.
- LawProfile, ExperienceProfile, ParameterBundle, ScenarioSpec, MissionSpec, and BundleProfile define behavior/data surfaces.
- Determinism requires fixed-point authoritative math, named RNG streams, thread-count invariance, and replay-hash equivalence.
- Epistemics require TruthModel -> PerceivedModel -> RenderModel separation and no omniscient truth leakage.

- No runtime mode flags are allowed; behavior differences derive from LawProfile and data.
- Survival defaults are diegetic-first and minimal.
- MMO/galaxy-scale/SRZ-0 scaling is a long-term architectural direction, not a baseline proof.

The assistant acknowledged this as ground truth for later work in the chat.

2.2 Binding the old glossary FACT: The user then pasted **Dominium - Canonical Glossary v1.0.0**. It defined canonical terms such as Account, Agent, AuthorityContext, AuditX, Canon, Capability, Client, Collapse, CompatX, Determinism, Domain, Engine, ExperienceProfile, Field, Game, Hardcore, LawProfile, Lens, Macro Capsule, MissionSpec, Observation Kernel, Pack, ParameterBundle, Process, Refusal Contract, RepoX, ScenarioSpec, SecureX, SessionSpec, TestX, TruthModel, UniverseIdentity, UniverseState, and many others.

The glossary also deprecated legacy mode-flag terms such as `survival_mode`, `creative_mode`, `hardcore_mode`, `spectator_mode`, `debug_mode`, `godmode`, and `sandbox`.

The assistant acknowledged the glossary as a terminology constraint layer.

2.3 Auditing the current GitHub repository against old v1 FACT: The user asked how the current GitHub repository `julesc013/dominium` aligned with the old contract and glossary. They asked specifically: "What do the docs say? What does the code do?"

The repository was inspected through the GitHub connector. Key files inspected or referenced included:

- `README.md`
- `docs/STATUS_NOW.md`
- `docs/architecture/ARCHO_CONSTITUTION.md`
- `docs/architecture/CANON_INDEX.md`
- `docs/canon/constitution_v1.md`
- `docs/canon/glossary_v1.md`
- `docs/ARCHITECTURE.md`
- `docs/XSTACK.md`
- `CMakeLists.txt`
- `client/core/session_pipeline.c`
- `client/core/session_stage_registry.c`
- `client/core/session_artifacts.h`
- `engine/include/domino/domain.h`

- engine/include/domino/core/rng_streams.h
- engine/modules/core/rng_streams.c
- engine/modules/sim/sim.c
- engine/modules/world/d_world.c
- data/registries/survival_vertical_slice.json
- data/registries/process_requirements.json
- data/registries/experience_profiles.json
- data/registries/law_profiles.json
- schema/session/session_spec.schema
- schema/authority/authority_context.schema
- scripts/dev/gate.py

The audit conclusion was:

- The repository strongly aligns with old v1 at the documentation/schema/registry/governance level.
- It partially aligns at implemented runtime-code level.
- The old v1 contract and glossary exist as current canonical repo files: docs/canon/constitution_v1.md and docs/canon/glossary_v1.md.

- The repo has a newer canon-index rule: prompts are not authoritative unless materialized as exactly one canonical document.
- The survival vertical slice exists structurally in data registries, but full runtime gameplay implementation was not proven.
- The session pipeline has real deterministic code and refusal handling.
- The engine has deterministic RNG and world/tile save-load/checksum scaffolding.
- Full agent gameplay loops, MMO distributed authority runtime, embodiment, and narrative/campaign systems were described as deferred in repo status docs.

2.4 Portability, modularity, extensibility, and future-proofing doctrine FACT: The user then stated that code portability, modularity, extensibility, and reuse are very important. They wanted the code reusable for different games on Domino and for unrelated engine/game projects. They also wanted everything connected so code/data/files/directories can be replaced or rewritten in a future refactor. The user explicitly said the project should be developed like a proper game or OS, not a one-off indie project.

The answer developed a general doctrine:

Stable contracts.

Replaceable implementations.

Deterministic behavior.

Portable projections.

No accidental authority from paths, tools, UIs, or prompts.

Important ideas discussed:

- Do not freeze every file. Stabilize contracts and semantic boundaries.
- Treat paths as storage, not identity.
- Introduce stability classes: frozen ABI, stable API, stable data contract, stable wire/replay contract, internal replaceable code, derived generated artifacts, and run-meta artifacts.
- Require replacement seams: every module should have a contract, version, owner, inputs, outputs, failure/refusal behavior, migration rule, tests, and dependency direction.
- Use negative architecture rules to prevent drift.
- Prove reuse with a second minimal Domino-based product.
- Keep public Domino APIs C-compatible and C89/C++98 parseable.

- Avoid exposing C++ classes across stable binary boundaries.
- Use versioned schemas, migration contracts, round-trip tests, and compatibility matrices.
- Keep runtime services from owning truth or Law.
- Keep apps/ thin.
- Keep packs data-only unless a separate trust/capability contract exists.
- Use ownership-based directory roots rather than generic source buckets.

2.5 Preservation package creation FACT: The user uploaded a preservation prompt in `Pasted_text.txt` requesting a maximum-fidelity chat preservation package. The assistant generated a preservation report and files named with the prefix `Dominium_Canon_Portability_Audit__...`

The generated files currently available in `/mnt/data` are:

- `Dominium_Canon_Portability_Audit__00_manifest.md`
- `Dominium_Canon_Portability_Audit__01_human_readable_report.md`

- `Dominium_Canon_Portability_Audit__02_context_transfer_packet.md`
- `Dominium_Canon_Portability_Audit__03_spec_sheet.yaml`
- `Dominium_Canon_Portability_Audit__04_registers.md`
- `Dominium_Canon_Portability_Audit__05_aggregator_packet.md`
- `Dominium_Canon_Portability_Audit__06_reader_brief.md`
- `Dominium_Canon_Portability_Audit__07_verification_and_audit.md`
- `Dominium_Canon_Portability_Audit__08_future_chat_bootstrap_prompt.md`
- `Dominium_Canon_Portability_Audit__09_in_chat_reader.md`
- `Dominium_Canon_Portability_Audit__handoff_package.zip`
- `Pasted_text.txt`

FACT: The user then asked for an accompanying human-readable detailed summary/report of the entire conversation and to bundle everything into a single ZIP.

This file is the accompanying report generated in response.

3. What was decided, what was only suggested, and what remains tentative

3.1 Decisions clearly established in the conversation

ID	Decision / established direction	Status	Label
DECISION-A01	Old v1 Constitution and Glossary are the comparison baseline for this chat.	Accepted for this chat	FACT
DECISION-A02	Current repo authority is repo-materialized canon, not raw prompts.	Established by repo docs inspected	FACT
DECISION-A03	Treat docs/registries as evidence of architectural intent, not runtime proof.	Adopted analysis principle	INFERENCE
DECISION-A04	Stable contracts matter more than stable file paths.	Recommended and aligned with user goals	INFERENCE
DECISION-A05	Ownership-based roots are preferable to generic <code>src//source</code> roots for this project.	Recommended; supported by repo docs	INFERENCE / FACT for repo policy
DECISION-A06	Future work should preserve	Recommended as strong doctrine	INFERENCE

3.2 Suggestions not yet confirmed as formal project law

The following were assistant recommendations. They should not be treated as binding project decisions unless the user or repo canon later adopts them:

- Add an explicit public API/ABI inventory.
- Create a second minimal Domino-based product example.
- Define module descriptors for statically and dynamically linked modules.
- Add more formal versioning per surface: engine ABI, runtime service API, schema, save format, replay format, pack format, wire protocol, command contract, refusal contract, component matrix, release manifest.
- Add an API stability report tool.
- Convert all transitional debt into owner/path/semantic-ID/risk/verification queues.
- Use the proposed function-prefix discipline (domino_*, d_*, dominium_*, runtime prefixes, etc.).

3.3 Things explicitly deferred or not proven

- Full runtime survival gameplay loop proof.
 - Full local build/test run of the repository.
 - Current physical repo layout verification from a fresh checkout.
 - Full API/ABI surface audit.
 - Public release/package proof.
 - Real second-product reuse proof.
 - Full TestX/XStack run.
 - Migration of remaining bad-root/transitional debt.
 - Confirmation of whether the inspected old paths such as `client/core/session_pipeline.c` still physically exist or were aliases/historical references relative to the later `apps/` convergence docs.
-

4. What the repository audit found

4.1 Documentation and canon **FACT:** The repository has canonical documents corresponding to the old v1 Constitution and Glossary. The repo's canon-index document says that if a document is not listed as canonical, it is not binding, and that prompts themselves are execution artifacts rather than authority.

INFERENCE: This matters because it prevents future assistants from treating pasted chat prompts as stronger than the current repo files. For future work, old chat text should be used as context unless it has been materialized into canonical repo files.

4.2 Architecture docs FACT: Current architecture docs describe Engine/Game/Client/Server/XStack boundaries, SessionSpec, AuthorityContext, UniverseIdentity/UniverseState separation, ExperienceProfile/LawProfile/ParameterBundle registries, session pipeline, Macro Capsule, Domain/Solver architecture, Truth/Perceived/Render separation, SRZ-0, determinism/replay, world representation, and worldgen.

INFERENCE: The architecture is highly documented and contract-heavy. The main risk is mistaking architectural density for implementation completeness.

4.3 Code implementation FACT: The inspected code includes a real deterministic session pipeline and session-stage registry. It supports explicit stage transitions, refusal codes, artifact readiness checks, and stage logs.

FACT: The engine contains deterministic RNG stream support and a deterministic world/tile substrate with save/load/checksum behavior.

FACT: CMake enforces C90/C++98 standards, reads version files, performs boundary checks, disables build-time downloads by default, and exposes many quality/governance targets.

UNCERTAIN / UNVERIFIED: The full runtime Process/Law/Authority execution path was not exhaustively traced. Survival process IDs exist in registries, but concrete C/C++ handlers for every survival process were not proven in the inspected surface.

4.4 Survival vertical slice **FACT:** The survival vertical slice registry contains the old v1 baseline fields and processes, including energy, hydration, exposure, health, alive status, environment temperature, daylight, need tick, exposure tick, consume item, gather resource, craft basic tool, build shelter, and death process.

INFERENCE: This shows strong structural alignment with old v1, but it remains largely declarative unless linked to runtime process execution and tests.

4.5 Current repo layout doctrine **FACT:** Later repo docs and contracts define an ownership-root layout model and explicitly say old `src/source` proposals are historical or superseded. The active roots include `apps`, `engine`, `game`, `runtime`, `contracts`, `content`, `docs`, `tests`, `tools`, `scripts`, `cmake`, `external`, `release`, and `archive`.

FACT: The naming contract forbids generic source buckets such as `src`, `source`, `code`, `impl`, `common`, `shared`, and `misc` for new active paths.

INFERENCE: This is a good long-term structure for modularity because it assigns ownership rather than merely grouping implementation files.

5. Portability and modularity doctrine distilled

The conversation produced a durable design doctrine. The core principle is:

A path answers who owns this. A file name answers what role this artifact plays. An ID answers what stable semantic object this is. A schema answers what shape and compatibility rules apply. A contract answers what behavior is allowed. A test answers how we know it still holds.

This doctrine has several practical consequences.

5.1 Stable contracts, replaceable implementations

Dominium should not aim to make every implementation permanent. It should make public contracts stable and explicit. Implementations should be rewritable behind those contracts.

Stable surfaces should include:

- public C ABI headers,
- schema IDs and schema versions,
- semantic IDs such as `process.*`, `law.*`, `lens.*`, and `pack.*`,
- save/replay formats,
- command/result/refusal protocols,
- wire protocols,
- package manifests,
- build/release manifests,
- migration rules,
- compatibility matrices.

Replaceable surfaces should include:

- internal engine algorithms,
- solver implementations,
- renderer backends,
- runtime adapters,
- product shells,

- UI implementations,
- tooling internals,
- generated-code internals.

5.2 Replacement seams A component is replaceable only when it has:

- a clear owner,
- an explicit contract,
- a version,
- declared dependencies,
- declared inputs/outputs,
- deterministic failure/refusal behavior,
- tests/golden fixtures,
- migration rules,
- diagnostics and proof artifacts.

A directory alone is not a module. A module is an enforceable boundary.

5.3 Reuse proof The strongest proposed test of reusability is a second minimal game/product on Domino. If a minimal non-Dominium product cannot be built without dragging in Dominium-specific LawProfiles, content, UI, or product assumptions, the engine is not cleanly reusable yet.

This remains a recommended future task, not a completed one.

5.4 Public API practice The recommendation was to keep public Domino APIs C-compatible and C89/C++98 parseable:

- opaque handles,
- `struct_size` fields,
- explicit allocator ownership,
- no platform types in public engine headers,
- no global mutable state,
- deterministic error codes,
- fixed-width integer types,
- append-only extension where possible,
- no exposed C++ ABI across stable plugin/module boundaries.

5.5 Data and schema practice Every schema/registry/manifest/save/replay/lockfile should declare:

- `schema_id`,
- `schema_version`,
- `stability`,
- `canonical encoding`,
- `unknown-field policy`,
- `migration/refusal policy`,
- `golden fixtures`,
- `round-trip tests`,
- `compatibility matrix`.

5.6 Runtime and apps practice The recommended ownership rule is:

- `engine/` owns deterministic truth substrate.
- `game/` owns Law, process semantics, and domain implementation.
- `runtime/` owns platform/render/input/audio/network/storage/shell/diagnostics adapters.
- `apps/` owns thin product entrypoints and product composition only.

- **contracts/** owns machine-readable law.
- **content/** owns authored data/packs/profiles/assets.
- **tools/** owns developer and governance tooling.

Runtime may host, adapt, present, connect, persist, and diagnose. It must not own simulation truth or Law.

6. What was put off for later

The conversation intentionally did not attempt to do all work immediately. Deferred items include:

1. **Fresh repo checkout audit** - Needed to resolve current physical layout, active code paths, and potentially stale references.
2. **Full build/test verification** - Needed to prove CMake, CTest, XStack, RepoX/TestX/AuditX, and product binaries in the current environment.
3. **Runtime Process execution audit** - Needed to distinguish survival registry declarations from implemented processes.
4. **Public API/ABI inventory** - Needed before stable versioning promises can be made.

5. **Schema/save/replay compatibility audit** - Needed for backward compatibility claims.
 6. **Second product reuse proof** - Needed to prove Domino can support a non-Dominium game cleanly.
 7. **Remaining layout-debt migration** - Needed to retire bad roots and transitional exceptions safely.
 8. **Function/file/schema naming audit** - Needed to determine whether current names are optimal or just transitional.
 9. **Formal adoption of recommendations as repo law** - Needed before assistant recommendations become binding project rules.
 10. **Master spec book aggregation** - This report is a source packet, not the final master spec book.
-

7. Open questions and verification queue

Open questions

1. **What is the exact current physical repo layout at HEAD?**

Reason: earlier inspection fetched old paths while later docs say product roots moved under `apps/`.

1. Which old v1 rules are fully implemented, partially implemented, only documented, or deferred?

Reason: architecture and runtime maturity must not be conflated.

1. Where is the complete LawProfile/AuthorityContext-gated Process execution path?

Reason: Process-only mutation and Law-gated authority are central to the constitution.

1. Are survival processes implemented as real runtime Process handlers?

Reason: the survival vertical slice exists structurally, but runtime proof is needed.

1. What are the stable public APIs/ABIs today?

Reason: long-term portability requires a known public surface.

1. Can a minimal second game use Domino without Dominium coupling?

Reason: that is the clearest test of engine reuse.

1. Which naming/layout recommendations should become formal repo law?

Reason: assistant recommendations are not binding until adopted.

1. Which generated artifacts are canonical, derived, or run-meta?

Reason: reproducibility and governance depend on artifact classification.

ID	Item	Why it needs verification	Priority
VERIFY-A01	Current physical repo tree	Resolve layout/path uncertainty	P0
VERIFY-A02	CMake configure/build status	Prove current build health	P0
VERIFY-A03	CTest/XStack/RepoX/TestX/AuditX status	Prove governance health	P0
VERIFY-A04	Runtime survival process implementation	Separate declaration from implementation	P1
VERIFY-A05	Public API/ABI inventory	Needed for stability/versioning plan	P1
VERIFY-A06	Save/replay compatibility surfaces	Needed for long-term backward compatibility	P1
VERIFY-A07	Pack/content trust enforcement	Needed for extensibility/security	P1
VERIFY-A08	Second Domino product proof	Needed for actual reuse proof	P2
VERIFY-A09	Naming and function-prefix audit	Needed for consistency and extensibility	P2
VERIFY-A10	Artifact-class	Needed for	P2

8. Artifacts included or referenced

8.1 Prior preservation package The earlier package contains the first structured preservation pass. It includes a manifest, human-readable report, context transfer packet, YAML spec sheet, registers, aggregator packet, reader brief, verification/audit report, future-chat bootstrap prompt, and in-chat reader.

This new bundle includes those files as source material.

8.2 Uploaded preservation prompt Pasted `text.txt` contains the user's detailed instructions for maximum-fidelity chat preservation. It is included so the future reader can understand why the preservation package has its structure.

8.3 This accompanying report This file summarizes the entire conversation in a more direct, narrative style and explains what was discussed, decided, suggested, deferred, and left open.

8.4 New bundle manifest The new manifest records what files were included in the final ZIP, their purpose, size, and SHA-256 hash.

8.5 Final ZIP package The final ZIP bundles the prior files, the uploaded prompt, this accompanying report, and the new manifest/checksum metadata.

9. Risks and future-assistant failure modes

RISK-A01 - Treating architectural docs as runtime proof A future assistant might say "Dominator implements X" when the repo only documents or declares X. Avoid this by classifying evidence as docs, schema, registry, code, tests, build, or runtime behavior.

RISK-A02 - Treating assistant suggestions as user decisions Several portability recommendations were assistant proposals. They should become project law only if the user or repo canon adopts them.

RISK-A03 - Losing uncertainty about repo state The repository can change. The prior GitHub inspection is time-bound and not equivalent to a fresh local checkout. Re-verify before modifying code.

RISK-A04 - Recreating forbidden generic roots Avoid `src/`, `source/`, `common/`, `shared/`, `misc`, `impl`, and `status/era` buckets unless a current contract explicitly allows them.

RISK-A05 - Breaking semantic identity during file moves Do not change process IDs, pack IDs, schema IDs, ABI IDs, product IDs, save IDs, or compatibility versions merely because storage paths change.

RISK-A06 - Over-cleanup before proof Mass moves or broad rewrites can break build, tests, imports, packaging, and compatibility. Use small gated migrations.

RISK-A07 - Conflating Domino and Dominion Domino should remain a reusable deterministic engine substrate. Dominion is one game/product layer on top. Keep that distinction strong.

RISK-A08 - Accidental runtime dependency on tools Tools may inspect and generate; runtime/product code must not depend on developer-only tooling.

RISK-A09 - UI/runtime owning truth Runtime and UI may project, present, and submit commands; they must not own simulation truth or decide Law.

RISK-A10 - Expired uploads and missing files Some previous uploads were reported expired. Future work should ask the user to re-upload any missing files needed for deeper analysis.

10. Recommended next-action sequence

1. Open a fresh current repo audit task.

Objective: verify physical tree, root exceptions, current active paths, and whether old `client/` paths remain.

1. Run or inspect strict layout validators.

Objective: compare actual tree against `contracts/repo/layout.contract.toml`, root allowlist, and layout exceptions.

1. Create an implementation maturity matrix.

Columns: docs, schema, registry, code, tests, build proof, runtime proof. Rows: determinism, authority, LawProfile, Process, survival, session, replay, packs, SRZ, XStack, packaging.

1. Create public API/ABI inventory.

Objective: identify what surfaces are stable, private, generated, transitional, or deprecated.

1. Trace survival process execution.

Objective: determine whether `process.need_tick`, `process.exposure_tick`, etc. are runtime-executable Processes or only registry entries.

1. Design the second Domino product proof.

Objective: prove engine reuse without Dominium coupling.

1. Convert recommendations to formal task prompts.

Objective: avoid ambiguous future work and preserve governance requirements.

1. Only then start large refactors or feature expansion.

Objective: preserve build/test/compatibility while improving structure.

11. What to preserve for the master spec book

This conversation should feed into these future spec-book chapters:

- Project identity and constitutional baseline.
- Glossary and terminology law.

- Evidence model: docs vs code vs tests vs runtime proof.
- Repository ownership layout.
- Naming and path identity doctrine.
- Stable contracts vs replaceable implementations.
- Public API/ABI policy.
- Schema, registry, save, replay, and pack compatibility.
- Runtime/app/game/engine boundary model.
- XStack governance and verification doctrine.
- Portability and second-product reuse proof.
- Migration/refactor rules.
- Risk and verification model.

Items that should become formal requirements if not already present:

- Path is storage, not identity.
- All stable surfaces require explicit versioning and compatibility rules.
- Every module needs a replacement seam.

- A second-product proof should be required before claiming engine reuse.
- No broad refactor without build/test/proof gates.
- Assistant suggestions cannot become canon without repo/user adoption.

Items that should remain background context unless adopted:

- Specific proposed function prefixes.
- Exact module descriptor sketch.
- Exact public C API sketch.
- Specific example-game ideas.
- External analogies to Linux, SQLite, NASA cFS, or SemVer.

12. Best questions to ask next

Understanding

1. What is the difference between the old v1 Constitution and the current repo canon?
2. Which parts of the repo audit were proven by code, and which were only docs/registries?

3. What does "path is storage, not identity" mean for Dominium?
4. Why is an ownership-root layout better than a `src/` layout here?

Decisions

1. Which portability recommendations should we promote into formal repo law?
2. Should we require a second-product proof before calling Domino reusable?
3. Should public Domino APIs be frozen as C ABI surfaces now or later?

Tasks

1. Turn the next-action sequence into a Codex-ready task prompt.
2. Draft an implementation maturity matrix for current Dominium.
3. Create a repo audit checklist for layout, naming, API, schemas, and runtime proof.
4. Design the minimal second Domino-based game/product proof.

Artifacts

1. Which file in the preservation package should be read first?
2. Which parts of this package should feed the master spec book?
3. Compare the prior human-readable report to this accompanying report and identify gaps.

Verification

1. What current repo facts must be re-verified before coding?
2. How should we verify survival Process implementation?
3. How should we audit public API/ABI stability?

Risk

1. What are the biggest ways a future assistant might misunderstand this conversation?
2. Which refactors are dangerous before build/test proof?
3. Which items are still assistant suggestions rather than user decisions?

13. Final package note

This accompanying report is meant to be read alongside the earlier preservation package. It does not replace the earlier structured registers and spec sheet; it provides a clearer narrative and a sharper explanation of what the conversation actually accomplished.

The final bundle includes both the earlier package files and this new report. Some older uploaded files are unavailable because they were reported expired. If those are needed, they must be uploaded again.

7.5 Dominium Deterministic Solar System Architecture Human Readable Report

Source: docs/archive/conversations/Dominium_Complete_Conversation/deterministic_solar_system_handoff_file/Dominium_Deterministic_Solar_System_Architecture__01_human_readable_report.md

Report type: human_readable_report

Size: 57540 bytes

**7.5.1 COMPLETE CHAT PRESERVATION
REPORT - Dominion Deterministic
Solar-System Architecture**

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion
Deterministic Solar-System Architecture

Row 2 Field: Date anchor; Assessment: 2026-05-27
Australia/Melbourne

Row 3 Field: Source scope; Assessment: This chat only
unless labelled PROJECT-CONTEXT

Row 4 Field: Apparent access; Assessment: Partial:
visible transcript and uploaded preservation
prompt are accessible; hidden/older unavailable
context is not.

Row 5 Field: Previously generated files available?;
Assessment: No prior generated files observed
before this response; this response creates files.

Row 6 Field: Uploaded files or artifacts present?;
Assessment: Yes: Pasted text.txt preservation
prompt.

Row 7 Field: Contains future plans?; Assessment: Yes

Row 8 Field: Contains decisions?; Assessment: Yes, but
many are design directions or assistant

recommendations rather than confirmed final decisions.

Row 9 Field: Contains pending tasks?; Assessment: Yes

Row 10 Field: Contains unresolved questions?;
Assessment: Yes

Row 11 Field: Staleness risk; Assessment: Medium:
engine/API/external-source claims and current
Unreal/NASA/JPL references require verification
before formal spec use.

Row 12 Field: Extraction confidence; Assessment: 4

Row 13 Field: Safe for later aggregation?; Assessment:
With caveats

Row 14 Field: Main limitations; Assessment: Only the
visible conversation and uploaded prompt were
used; no claim is made that this captures
unavailable prior project history.

This preservation is based on the visible chat plus the uploaded preservation prompt. The prompt explicitly requested a human-readable report first, then structured registers, a spec sheet, an aggregator packet, a self-audit, downloadable files, and a ZIP package. It also required uncertainty labels and warned not to treat brainstorm or assistant suggestions as final user decisions. The

uploaded prompt is therefore treated as the controlling artifact for this preservation task.

Plain-language limitations: I can see the visible transcript of this chat and the uploaded **Pasted text.txt** preservation instruction. I cannot verify whether there were older hidden turns, prior files, or separate project documents outside this chat unless they are present in the visible transcript or uploaded file. I also cannot treat the previous assistant's technical claims about Unreal, NASA/JPL data, MMO scale, or engine capabilities as permanently current; those belong in the verification queue.

1. One-Page Orientation

This chat was mainly about turning **Dominium**, built on or around the **Domino engine**, into a deterministic, procedural, large-scale civilisation-building game that can handle planetary and possibly solar-system scale without collapsing under simulation cost. The opening question asked whether it is possible to build a deterministic game with a full-scale real solar system, fully recreated planets, custom systems, procedural data-driven terrain, player terraforming, cut/fill construction, megaprojects, player-designed machines, factories, fog of war, sparse simulation, client-shared compute, single-player multiverse play, and MMO-style persistent universe play. The user also asked whether Unreal

Engine could do this with low latency, high FPS, and millions of players across shards or private servers.

The first assistant response framed the problem as an architecture problem rather than merely an engine-choice problem. The central recommendation was that **Domino should be the deterministic simulation and database layer**, while Unreal, Raylib, or another renderer should be treated as a client. It argued that the world should be represented as seeds, physical data, sparse deltas, event logs, field layers, and simulation LODs-not as a giant dense scene graph. It also warned that millions of players can share a logical universe if distributed across regions/shards, but not all occupy one fully simulated real-time bubble at high fidelity.

The user then refined the project substantially. The conversation moved from general engine feasibility to a concrete game premise: a robotic seed-civilisation game. The user described science-bounded procedural star systems and planets, where designers set physically meaningful parameters and can nudge or override results without manually painting every detail. The same underlying data/delta technology should support world generation, save files, planet packs, painted fields, and in-game terraforming. The lore premise is that humans launched a robotic mothership to a new solar system; the mothership lands on the most habitable planet and manufactures robot bodies intended to restart

civilisation, possibly before later human biological payloads arrive.

Gameplay, as discussed, centres on players spawning as robots from physical fabrication machines. They can choose body shapes such as biped, quadruped, spider, or tank-like forms. New spawn points are created by building remote laboratories/fabricators that can manufacture robot bodies from local materials. The mothership explains why players have a HUD, recipes, blueprints, quest-like guidance, and advanced technical knowledge from the start, while its finite resources, low throughput, heat, wear, and scale limits explain why players still need to build mines, refineries, factories, cities, and supply chains.

A major design shift was away from simulating human-like worker NPCs. The user explicitly identified labour/pathfinding as expensive and unnecessary if the civilisation is robotic. The preferred direction is automation from the beginning: nanobots, machine controllers, AI agents, process diagrams, and factories that can be represented as input/output graphs rather than thousands of pathfinding workers and moving parts. This became one of the most important chat contributions: **the game should simulate materials, energy, machines, logistics, sensors, and permissions-not pretend that invisible humans are doing the work.**

The future relevance of this chat is high. It contains the conceptual seed for Dominion's engine architecture, world generation philosophy, lore premise, spawn economy, construction interface, anti-NPC automation stance, machine abstraction model, and first vertical slice. A future reader should understand that not everything here is a final decision. Many items are user brainstorms or assistant recommendations. However, the strongest carry-forward ideas are: deterministic sparse simulation, procedural science-bounded worlds, robot avatars, mothership-constrained progression, nanobot blueprint construction, no worker NPC labour, process-graph factories, sensed-only fog-of-war simulation, and a first playable slice proving the mothership-to-remote-spawn-lab loop.

2. The Story of the Conversation

2.1 Opening: a maximum-scale technical

feasibility question The chat began with a broad but coherent question: how can Domino engine and Dominion support a deterministic universe-scale construction/civilisation game? The user wanted a real or custom solar system, procedurally recreated planets, terrain editing, terraforming, megaprojects, machine and factory design, fog of war, sparse simulation, MMO persistence, client-shared compute, and both single-player and multiplayer operation. The user also asked

whether Unreal could handle this without lag at high FPS and low latency.

This established the key tension: the desired game is extremely ambitious, but the user already understood some necessary constraints—fog of war, sparse simulation, collapsed unseen state, and distributed compute.

2.2 First response: architecture before engine

The first assistant answer reframed the problem around deterministic simulation. It recommended that the game be built as a deterministic simulation/database first, with renderers as clients. It proposed fixed ticks, deterministic commands, event logs, snapshots, state hashes, hierarchical coordinates, sparse deltas, procedural base terrain, simulation LOD, fog-of-war filtering, and server authority. It argued that Unreal can help with visualisation and local tooling but should not be expected to be the authoritative simulator for planets, factories, and millions of players.

The answer also drew a feasibility boundary: millions of accounts or players distributed across a universe may be possible with sharding and region authority, but millions in one fully simulated, low-latency visible place is not a practical design target.

2.3 User refinement: from engine question to

game identity The user then provided a long design

monologue that changed the centre of gravity. The game became not only a technical engine challenge but a specific fiction and gameplay loop. The user described a procedurally generated star system or "galaxy" with as many planets as desired; physically meaningful parameters; a worldgen editor where sliders are bounded by realism; and perhaps reverse generation, where the designer specifies the kind of planet they want and the system solves plausible orbital/planetary parameters.

The user wanted procedural fields that can be overlaid with painted or custom fields, allowing planet packs, designer edits, and gameplay terraforming to share the same underlying technology. This is important because it suggests one unified world-state model rather than separate systems for saves, mods, and worldgen.

The user then introduced the robotic mothership premise. Humans in the past sent a robotic mothership to another solar system to land on a habitable planet, manufacture robots using advanced technology, and restart civilisation. Players spawn as robots from physical machines. They can build new spawn labs on the frontier if they can gather resources, power them, and connect them to the mothership or local industry.

2.4 Gameplay simplification: automation replaces labour The user identified labour NPCs as a likely compute trap. Instead of workers with pathfinding,

collision, schedules, and animation, the game should lean into the fact that the civilisation is robotic. Nanobots, ship AI, automation, blueprints, and process diagrams provide a diegetic reason for construction and manufacturing to be abstracted. The user wanted cut/fill, blueprint stages, non-destructive planning, and deterministic execution. Machines and vehicles should be constructible, but once assembled, their functions should be summarised as process equations where possible.

This was a major narrowing of the simulation model. The game can still be deep, but it does not need to simulate every person, machine part, or worker step.

2.5 Second response: synthesis into a robotic seed-civilisation game The second assistant response named the direction: Dominion should be a robotic seed-civilisation game. It organised the user's ideas into systems: science-bounded planet generation, forward/reverse generation, field layer stack, mothership resource and recipe logic, physical robot body spawning, nanobot construction as an actuator rather than magic, no worker NPCs, machines compiled into graphs, easy-to-start cities, AI planning as quest markers, fog of war as robot sensor knowledge, separate single-player/private/official MMO modes, and a first vertical slice.

This response generated many useful framework elements, but it remains mostly assistant synthesis. The

user had not yet accepted every element before uploading the preservation prompt.

2.6 Preservation request: export this chat for future use

The final user action was uploading a detailed preservation prompt. It required this response to produce a maximum-fidelity human-readable report, structured registers, spec sheet, aggregator packet, audit, downloadable files, and final in-chat reader. That prompt also required uncertainty labels and warned against treating brainstorming as final decisions.

The final state of the chat is therefore a design handoff. The substantive design work is not complete, but this chat now has a coherent conceptual foundation ready to be turned into a formal project spec.

3. Main Topics Discussed

Topic 1 - Deterministic Domino engine

architecture The topic was how to make a massive procedural universe deterministic and persistent. It came up because the user explicitly asked for deterministic gameplay across solar-system scale, sparse construction/destruction, multiplayer, and client-shared compute. The discussion concluded that the authoritative game should be represented as deterministic data and commands: seeds, rules versions, event logs, state hashes, sparse

deltas, and snapshots. Rendering should not own truth. This remains a design recommendation rather than a confirmed implementation decision, but it is one of the strongest technical conclusions.

Uncertain points include exact language/runtime, math model, tick rates, coordinate scheme, physics boundaries, storage format, and how much determinism is required across platforms. Future work should define the Domino simulation core spec before building high-level content.

Topic 2 - Science-bounded procedural star

systems and planets The user wanted procedural systems and planets generated from physically meaningful parameters. Sliders should not be arbitrary; they should be bounded to plausible values. The user also raised reverse generation: specifying a desired planet and letting the system find plausible orbital/planetary parameters. This connects directly to the official server world, private universes, planet packs, and future editor tools.

Unresolved points include how realistic the model should be, what counts as an allowable sci-fi override, what external datasets should be used, how licensing works, and whether the initial scope is one star system or many.

Topic 3 - Field layers, painted overrides, saves, packs, and terraforming

The user wanted procedural

fields overlaid with painted fields, and wanted the same technology to support save files and packs. The assistant formalised this into a field-layer stack: base procedural generation plus real data, designer overrides, mods/packs, terraforming layers, construction layers, damage layers, and temporary preview layers.

The key conclusion is that worldgen, saves, packs, and terraforming should probably share a unified sparse field/delta model. The caveat is that editor/worldgen layers may freely create matter, while gameplay terraforming/construction layers must obey conservation rules.

Topic 4 - Robotic mothership lore and spawn economy The user described a robotic mothership launched by humans to restart civilisation. It lands, manufactures robot bodies, and possibly prepares the world for later human biological payloads. This premise explains respawn, HUDs, recipes, quest markers, and advanced knowledge.

The mothership should be powerful but limited. It can fabricate early bodies and precision items, but finite resources, low throughput, heat, wear, and mission reserves force players to mine, refine, fabricate, and build industry. The exact limits remain unresolved and are a top design task.

Topic 5 - Robot player bodies and physical

respawn The user proposed players spawning into robot forms such as biped, quadruped, spider, or tank-like bodies. New players enter through physical spawn machines. Frontier expansion depends on building remote labs that can make robot bodies from local materials.

This is not just lore. It creates a physical economy around respawn, exploration, risk, remote bases, communications, and industrial capacity. Open questions include body templates, balance, backup/mind-state rules, death penalties, and single-player versus MMO differences.

Topic 6 - Nanobot construction, blueprinting,

cut/fill, and terraforming The user proposed nanobots or an invisible force as a computationally cheap and diegetic construction mechanism. Robots can project HUD blueprints, preview non-destructively, then commit construction when resources and energy are available. This supports cut/fill, mining, foundations, buildings, and megaprojects.

The important caveat is that nanobots should be actuators, not magic. They should consume materials, energy, time, and heat budget, and should require capability tiers. Future work must specify exact operations and data formats.

Topic 7 - Automation instead of worker NPCs

This was one of the clearest user preferences. The user considered worker NPC labour but rejected it as expensive to compute. Since the society is robotic, there is no need to simulate people doing work. Instead, machines, nanobots, AI agents, and factory controllers can do it directly.

This affects the entire design. City-building becomes infrastructure-building, not labour management. The main risk is that cities may feel empty, so future design must create visible machine life, drones, sensor networks, traffic abstractions, and city operating systems.

Topic 8 - Machines, devices, factories, and process graphs

The user wanted players to design machines, devices, factories, vehicles, and rockets. However, the user also wanted the entire function of a machine assembly to be summarised into an equation or process step when possible. The assistant suggested a machine graph compiler.

The key challenge is expressiveness versus safety. Players need creative freedom, but the engine must prevent exploit machines, lag machines, impossible devices, and economy bypasses. This is one of the highest-priority technical design tasks.

Topic 9 - Fog of war and sensed-only simulation

The user required fog of war and only loading/simulating what players can sense. The assistant framed fog of war as a data-security and simulation model, not merely a visual overlay. A player should have observed, remembered, inferred, unknown, and secret states.

This connects directly to MMO anti-cheat and client-shared compute. If a client receives hidden information, players will extract it. Future design must define sensor rules, map sharing, hidden resources, and server-side filtering.

Topic 10 - MMO, single-player, private universes, and client-shared compute

The user wanted the same general game to support single-player multi-universe play, official MMO persistent universe play, and possibly user servers. The assistant recommended a model where single-player and private servers can relax trust assumptions, while official MMO mode uses server authority and strict fog-of-war filtering.

Unresolved issues include whether the official server is one designed star system, how many players can occupy a local area, what clients are allowed to compute, how region ownership works, and how to handle failure/recovery.

Topic 11 - Unreal's role The user asked whether Unreal can do the whole thing at high FPS and low latency. The assistant answered that Unreal can help with rendering, local tools, streaming, and visuals, but should not be assumed to provide deterministic planetary MMO simulation by itself.

This is a recommendation, not a user-confirmed decision. It requires current verification and prototype tests before being formalised.

Topic 12 - First vertical slice The assistant proposed a first playable slice: one planet region, one mothership, robot chassis, resource deposit, power, cut/fill, blueprint tool, nanobot construction, mine, refinery, fabricator, remote spawn lab, and fog-of-war sensor system. The gameplay loop is spawn, survey, find ore, blueprint mine, flatten site, build power, refine, fabricate, transport, build remote spawn lab, and spawn/swap there.

This is the best immediate next action, but it remains a recommendation until the user approves or modifies it.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted a deterministic Dominion/Domino game architecture capable of handling procedurally generated solar systems, planets, construction, terraforming, factories,

fog of war, sparse simulation, and MMO/single-player modes. The user also explicitly wanted science-bounded planet generation, procedural fields with painted overrides, robotic mothership lore, robot spawning, nanobot-like construction, automation over worker NPCs, and machine functions that can collapse into process diagrams.

These goals matter because they define the game's identity: a massive but compute-aware robotic civilisation simulator rather than a conventional survival/crafting game or ordinary Unreal open world.

4.2 Inferred Goals It is reasonable to infer that the user wants a future master spec book, wants old chats preserved for aggregation, wants technically honest feasibility boundaries, and wants the design to remain ambitious without becoming impossible. It is also inferred that the user values diegetic explanations for gameplay interfaces, because the user repeatedly tied HUDs, recipes, quest markers, and construction tools to robot/mothership fiction.

4.3 Goals That Changed Over Time The conversation began as a broad technical feasibility question. It changed into a richer game-design discussion once the user introduced the robotic seed-ship premise. The focus shifted from "can Unreal do a deterministic

solar-system MMO?" to "how should Dominion be designed as a robotic seed-civilisation game that happens to require a deterministic sparse engine?"

Another shift was away from worker NPCs. The user initially mentioned possible NPC workers/labour, then argued against simulating labour because it is expensive and redundant in a robot civilisation. This created a clearer design direction: automation from the jump.

4.4 Goals Still Unresolved The core goals remain unresolved at implementation level. The chat did not decide exact engine technology, exact data formats, exact planet generator equations, exact robot body rules, exact mothership resource numbers, exact MMO architecture, exact first milestone schedule, or whether the user accepts all assistant recommendations. The best next step is to convert this conceptual foundation into a formal requirements/spec document and then a vertical slice plan.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Treat Domino as the authoritative deterministic simulation/data core and rendering as a client layer.; Status: Recommended / not explicitly user-accepted; Why it mattered: Keeps deterministic state separate from frame-rate, rendering, physics effects, and

engine limitations.; Confidence: 4; Label:
INFERENCE

Row 2 ID: DECISION-02; Decision: Use science-bounded procedural generation rather than purely arbitrary sliders or manual painting.; Status: User-stated design direction; Why it mattered: Supports surprising worlds while preserving plausibility and design control.; Confidence: 5; Label: FACT

Row 3 ID: DECISION-03; Decision: Support procedural fields overlaid with painted/designer/player layers.; Status: User-stated direction, technical details open; Why it mattered: Unifies worldgen, packs, saves, and terraforming under one model.; Confidence: 4; Label: FACT +
INFERENCE

Row 4 ID: DECISION-04; Decision: Use a robotic seed-ship/mothership premise to explain spawning, recipes, HUD, and automation.; Status: User-stated design premise; Why it mattered: Provides diegetic explanation for robot avatars, HUD, knowledge database, spawn machines, and fabrication.; Confidence: 5; Label: FACT

Row 5 ID: DECISION-05; Decision: Make player characters robots with selectable manufactured chassis rather than ordinary humans.; Status: User-stated design direction; Why it mattered:

Makes HUD, strength, inventory, respawn, hostile environments, and automation coherent.;

Confidence: 5; Label: FACT

Row 6 ID: DECISION-06; Decision: Use nanobot/automation construction as the main interaction layer, bounded by material, energy, time, and capability.; Status: User-stated direction with assistant constraints; Why it mattered: Avoids expensive labour simulation and makes construction convenient while preserving economy.; Confidence: 4; Label: FACT + INFERENCE

Row 7 ID: DECISION-07; Decision: Deprioritise human-like worker NPC labour and use automation/process graphs instead.; Status: Strong user-stated preference / tentative design direction; Why it mattered: Avoids pathfinding/AI costs and aligns with robot civilisation premise.; Confidence: 5; Label: FACT

Row 8 ID: DECISION-08; Decision: Use the mothership as the recipe book and early precision fabricator, but constrain it with finite resources and throughput.; Status: User-stated direction; Why it mattered: Preserves progression while explaining why players know advanced designs.; Confidence: 5; Label: FACT

Row 9 ID: DECISION-09; Decision: Represent distant

machines/factories as processes/equations rather than simulating all physical parts.; Status: User-stated direction with assistant formalisation; Why it mattered: Enables scale and preserves performance.; Confidence: 5; Label: FACT

Row 10 ID: DECISION-10; Decision: Use sparse construction/destruction and collapsed simulation for unseen/unneeded areas.; Status: Core user requirement and assistant recommendation; Why it mattered: Makes planet-scale and MMO-scale persistence possible.; Confidence: 5; Label: FACT

Row 11 ID: DECISION-11; Decision: Do not trust clients with hidden information or authoritative MMO results.; Status: Assistant recommendation / not explicitly user-accepted; Why it mattered: Protects fog-of-war secrets and anti-cheat.; Confidence: 3; Label: INFERENCE

Row 12 ID: DECISION-12; Decision: Treat Unreal as useful for presentation/tooling but not as the whole authoritative deterministic MMO engine.; Status: Assistant recommendation / not explicitly user-accepted; Why it mattered: Prevents architecture from depending on ordinary engine scene graphs for planetary/MMO authority.; Confidence: 3; Label: INFERENCE

Row 13 ID: DECISION-13; Decision: Make the first

serious milestone a small vertical slice, not a full solar-system MMO.; Status: Assistant recommendation / not explicitly user-accepted; Why it mattered: Tests whether the core loop is fun and technically viable.; Confidence: 4; Label: INFERENCE

DECISION-01 - Domino as deterministic simulation core This was recommended by the assistant, not explicitly accepted by the user. It follows from the user's deterministic, persistent, sparse, MMO-scale requirements. The alternative would be to make Unreal or a normal scene graph the source of truth, but that conflicts with replayability, sparse planetary scale, and authority boundaries. This decision affects almost everything: save files, networking, terrain, factory simulation, and debugging. It should be revisited only if a much smaller game scope is chosen.

DECISION-02 - Science-bounded procedural generation This is a clear user-stated direction. The user does not want planets manually painted from scratch or shaped by arbitrary sliders. The alternative, fully arbitrary world creation, was deprioritised because it would undermine the "real science" premise. This affects the planet editor and official server world design.

DECISION-03 - Procedural fields plus overlays

The user explicitly wanted procedural fields overlaid with painted fields and wanted save-file technology to support packs. The decision is conceptually clear, but the technical implementation remains open. The key assumption is that the same sparse field system can serve worldgen, saves, mods, and gameplay deltas if conservation rules are separated by layer type.

DECISION-04 - Robotic mothership premise

The user made this a central lore/gameplay idea. It explains robot players, HUDs, recipes, spawning, and early constraints. It also creates faction and mystery possibilities. This decision could be revisited if the game changes away from robot civilisation, but in the current chat it is the strongest creative anchor.

DECISION-05 - Robot player chassis

The user explicitly described players choosing robot forms. This affects movement, inventory, environment access, progression, respawn, and player roles. Exact chassis are not final.

DECISION-06 - Nanobot construction bounded by costs

The user proposed nanobots/invisible force because it is computationally cheap and diegetic. The assistant added the constraint that nanobots should

consume materials, energy, time, and capability. This matters because otherwise construction becomes magic and invalidates factories.

DECISION-07 - Avoid worker NPC labour The user strongly favoured skipping expensive NPC labour and using automation. The main alternative was simulating worker-like agents, but that was deprioritised because of pathfinding and compute costs. This should be preserved carefully: future assistants should not reintroduce labour management as a default.

DECISION-08 - Mothership as recipe book but not infinite factory The user explicitly wanted the mothership to contain knowledge and some fabrication capacity, but with finite resources and low throughput so players still need to build industry. This affects early-game pacing and prevents a contradiction: if the ship can make everything, why play?

DECISION-09 - Machines/factories compile to equations The user explicitly wanted machine functions summarised into simple process steps where possible. This is a major performance strategy. The unresolved risk is how to keep player-designed machines expressive without creating exploits.

DECISION-10 - Sparse construction/destruction and collapsed simulation This was both user-stated and assistant-supported. The world must store player changes sparsely and simulate unobserved systems at lower fidelity. This is foundational and should become a formal requirement.

DECISION-11 - Client compute must be limited in public MMO mode This is an assistant recommendation. The user wanted clients to share compute; the assistant separated safe client compute from unsafe hidden/authoritative compute. It should be treated as a security recommendation pending final MMO design.

DECISION-12 - Unreal as client/tooling, not authoritative universe This is an assistant recommendation, not a user decision. It needs verification against current Unreal capabilities and the final scope. It matters because betting on Unreal as the whole simulation engine could lead to architectural failure.

DECISION-13 - First vertical slice before full MMO This is an assistant recommendation. It matters because the game is too large to build top-down. The proposed slice tests the unique loop: spawn from

mothership, survey, mine, build power/refinery/
fabricator, and create a remote spawn lab.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

Row 1 ID: REJECTED-01; Option: Manual painting as the main planet creation method; Status: Deprioritised; Reason: User wants procedural generation from plausible science, with painting as override rather than primary method.; Final or tentative?: Tentative but strong; Reconsider conditions: Use painting for designer overrides, mods, and exact authored locations.; Related workstream: WORKSTREAM-02, WORKSTREAM-03; Label: FACT

Row 2 ID: REJECTED-02; Option: Completely arbitrary sliders detached from physical plausibility; Status: Deprioritised; Reason: User said sliders should be bounded by what is realistic.; Final or tentative?: Tentative but strong; Reconsider conditions: Allow fantasy/sci-fi override modes if explicitly labelled.; Related workstream: WORKSTREAM-02; Label: FACT

Row 3 ID: REJECTED-03; Option: Human-like worker NPC labour as the main construction/economy mechanism; Status: Deprioritised; Reason: User

said labour is expensive to compute and suggested jumping straight to automation.; Final or tentative?: Tentative but strong; Reconsider conditions: Could return as local cosmetic drones/robots or abstracted AI agents, not core labour sim.; Related workstream: WORKSTREAM-07, WORKSTREAM-08; Label: FACT

Row 4 ID: REJECTED-04; Option: Simulating every machine part, belt item, worker, and device at full fidelity everywhere; Status: Rejected as infeasible by design rationale; Reason: User asked to collapse things; assistant recommended aggregate equations.; Final or tentative?: Tentative but fundamental; Reconsider conditions: Use local detailed visualization only when sensed/interactive. ; Related workstream: WORKSTREAM-07, WORKSTREAM-10; Label: FACT + INFERENCE

Row 5 ID: REJECTED-05; Option: Unreal Actors as the authoritative representation for all terrain, machines, cities, and planet state; Status: Rejected by assistant recommendation; Reason: Assistant argued Unreal should not own the authoritative universe.; Final or tentative?: Pending user acceptance; Reconsider conditions: Reconsider only for small prototypes or purely visual local scenes.; Related workstream: WORKSTREAM-11; Label:

INFERENCE

Row 6 ID: REJECTED-06; Option: Trusting public MMO clients with secret fog-of-war data or authority-critical results; Status: Rejected by assistant recommendation; Reason: Would leak secrets and enable cheating.; Final or tentative?: Strong recommendation; Reconsider conditions: Private servers/single-player can relax this.; Related workstream: WORKSTREAM-09, WORKSTREAM-10; Label: INFERENCE

Row 7 ID: REJECTED-07; Option: World reset as the normal permanent official-server failure response; Status: Deprioritised by assistant recommendation; Reason: User brainstormed restarting if first players fail; assistant suggested emergency recovery instead.; Final or tentative?: Uncertain; Reconsider conditions: Could be used for alpha seasons, roguelike servers, or explicit challenge modes.; Related workstream: WORKSTREAM-04, WORKSTREAM-10; Label: INFERENCE

The rejected/deprioritised items matter because they protect the design from sliding back into conventional assumptions. The game should not become a manual planet-painting tool, a worker-management city builder, an Unreal Actor planet, or a fully simulated machine-parts sandbox at all scales. Some options may return in

limited forms: painted terrain can exist as an override; NPC-like behaviour can exist as local drones or wildlife; Unreal Actors can represent local visuals; and world resets could be used in alpha seasons. But they should not be treated as the main architecture.

7. Important Reasoning, Rationale, and Tradeoffs

The visible reasoning in the chat revolved around scale versus fidelity. The user wanted extreme scale but already recognised that only sensed or relevant areas can be loaded and simulated. The assistant's rationale was that every system needs a compact representation when unobserved and a high-fidelity representation only when a player can interact with it.

A second tradeoff was realism versus design control. The user wants planets generated from real science, but also wants designers to nudge outcomes and possibly reverse-solve for a desired planet. The proposed compromise is plausibility-bounded generation plus labelled overrides.

A third tradeoff was convenience versus progression. Nanobots and mothership fabrication make the game smoother, but they can destroy the industrial challenge if unconstrained. The visible rationale was to let these systems provide precision, planning, and early capability while requiring materials, energy, throughput, heat management, and factories for scale.

A fourth tradeoff was creativity versus server safety. Player-designed machines are important, but arbitrary simulations can become lag machines or exploits. The proposed solution is machine graph compilation: visually rich designs are reduced to validated inputs, outputs, capabilities, costs, and failure modes.

A fifth tradeoff was MMO scale versus secrecy. Client-shared compute could help performance, but if clients receive hidden data, fog of war fails. The recommendation was to allow clients to compute visible/verifiable work but keep hidden state and authority on trusted servers.

8. Plans, Future Work, and Next Steps

The future work naturally begins with specification, not full implementation. The highest-priority tasks are to define the deterministic core, planet generator schema, field/delta format, mothership economy, nanobot construction pipeline, machine graph compiler, fog-of-war model, and first vertical slice.

Recommended sequence:

1. **Write the design pillars:** robotic seed civilisation; deterministic sparse world; science-bounded planets; automation-first cities; physical spawn economy.

2. **Define the first vertical slice:** one planet region, one mothership, one ore deposit, one power source, one cut/fill tool, one mine, one refinery, one fabricator, one remote spawn lab, one fog-of-war system.
3. **Specify the data core:** event logs, snapshots, state hashes, sparse deltas, field layers, command validation.
4. **Specify gameplay systems:** mothership constraints, robot chassis, nanobot construction, machine graphs, factory abstraction, spawn labs.
5. **Prototype determinism and terrain edits:** prove that cut/fill, save/load, and replay produce the same world state.
6. **Prototype machine graphs:** prove that a player-built machine can compile into an aggregate process safely.
7. **Verify engine choices:** test Unreal/Raylib/custom renderer assumptions only after the core data model is clear.

Main blockers are scope, unverified engine assumptions, the need for a safe machine compiler, and the need to prevent nanobots/mothership fabrication from trivialising the game.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences The user explicitly wants determinism, procedural data-driven solar-system/planet generation, realistic bounded sliders, field overlays and painted overrides, single-player/private/MMO possibilities, fog of war, sparse construction/destruction, collapsed unseen simulation, robot player avatars, physical spawn machines, nanobot-style construction, finite mothership resources, and automation rather than worker NPCs.

The uploaded preservation prompt explicitly requires human-readable explanation first, uncertainty labels, no invented facts, no silent inference, no treating brainstorming as decisions, structured registers, spec sheet, aggregator packet, self-audit, and files/ZIP when tools are available.

9.2 Inferred Constraints and Preferences The user appears to prefer diegetic explanations, practical feasibility boundaries, and systems that are both ambitious and computationally honest. The user also appears to prefer future-proof preservation so old chats can become a master Project Spec Book.

9.3 Uncertain or Unestablished Preferences It is not established whether the user prefers Unreal, Raylib,

or a fully custom renderer long-term. It is not established how realistic the science model should be, how punishing respawn should be, how many robot bodies should exist, whether official servers should ever reset, or how much PvP/conflict the MMO should support.

10. Files, Artifacts, Outputs, and Prompts

Row 1 ID: ARTIFACT-01; Artifact / file / prompt / output: Visible chat transcript; Type: Conversation context; Purpose: Primary source for preservation.; Status: Available in current chat; not separately exported before this response.; Origin: This chat; Carry forward?: Yes; Notes: Contains the user’s MMO/engine question, assistant architecture responses, and user’s detailed seed-civilisation concept.; Label: FACT

Row 2 ID: ARTIFACT-02; Artifact / file / prompt / output: Uploaded file: Pasted text.txt; Type: Prompt/instruction file; Purpose: Defines the preservation/export requirements for this response. ; Status: Uploaded in this turn.; Origin: User upload; Carry forward?: Yes; Notes: Requires human-readable report, registers, spec sheet, aggregator packet, audit, files, and ZIP.; Label: FACT

Row 3 ID: ARTIFACT-03; Artifact / file / prompt / output: Assistant architecture response 1; Type: In-chat output; Purpose: First structured answer about deterministic simulation, sparse deltas, fog of war, client compute, MMO scale, and Unreal limits.; Status: Existing in visible chat.; Origin: Assistant; Carry forward?: Yes with uncertainty labels; Notes: Treat as recommendations unless user later accepts.; Label: INFERENCE

Row 4 ID: ARTIFACT-04; Artifact / file / prompt / output: Assistant design response 2; Type: In-chat output; Purpose: Expanded design around robot seed civilisation, mothership, nanobots, automation, planet generation, and first vertical slice.; Status: Existing in visible chat.; Origin: Assistant; Carry forward?: Yes with uncertainty labels; Notes: Treat as recommendations and synthesis; user had not yet replied before preservation request.; Label: INFERENCE

Row 5 ID: ARTIFACT-05; Artifact / file / prompt / output: External references cited in assistant responses; Type: Links/citations; Purpose: Prior assistant cited Unreal, NASA/JPL/SPICE, Guinness, EVE, etc.; Status: Present as citations in visible chat, not reverified for this report.; Origin: Assistant/web citations in prior turns; Carry forward?: Carry forward to verification

queue; Notes: Useful for grounding engine/scale claims but may be stale.; Label: UNCERTAIN / UNVERIFIED

Row 6 ID: ARTIFACT-06; Artifact / file / prompt / output: This generated preservation package; Type: Markdown/YAML/ZIP files; Purpose: Exports the chat for future reading and aggregation.; Status: Created by this response.; Origin: Assistant; Carry forward?: Yes; Notes: Includes manifest, report, context transfer packet, spec sheet, registers, aggregator packet, reader brief, audit, bootstrap prompt, in-chat reader, and ZIP.; Label: FACT

Before this preservation task, no downloadable files were observed as having been created in the visible chat. The uploaded `Pasted text.txt` is the controlling prompt for the preservation package. This response creates the requested downloadable files and ZIP.

11. Open Questions and Unresolved Issues

Row 1 ID: QUESTION-01; Question / issue: Is the official scale one designed star system, many generated systems, or a true galaxy-level structure?; Why it matters: Determines generator scope, persistence model, and travel design.; Known information: User discussed a "Galaxy" but

clarified as essentially a solar system with configurable planets.; Unknown information: Final product scale and naming.; Resolution path: Choose initial official-server scope and name the editor accordingly.; Priority: P0; Related workstream: WORKSTREAM-02, WORKSTREAM-10; Label: FACT + UNCERTAIN / UNVERIFIED

Row 2 ID: QUESTION-02; Question / issue: How realistic should the planet generator be, and where should sci-fi overrides be allowed?; Why it matters: Controls content freedom and simulation complexity.; Known information: User wants sliders bounded by realism but also wants designer nudging and overrides.; Unknown information: Hard vs soft physics boundaries.; Resolution path: Define plausibility scoring and override categories.; Priority: P0; Related workstream: WORKSTREAM-02; Label: FACT

Row 3 ID: QUESTION-03; Question / issue: What is the exact field-layer format for terrain/material/climate/ownership/visibility data?; Why it matters: It is foundational for saves, mods, terraforming, and construction.; Known information: User wants procedural fields overlaid with painted fields and reused save technology.; Unknown information: Data schema, operations,

compression, conservation rules.; Resolution path: Draft field/delta spec.; Priority: P0; Related workstream: WORKSTREAM-03; Label: FACT + INFERENCE

Row 4 ID: QUESTION-04; Question / issue: What are the mothership's initial resources, fabrication rate, and failure/recovery rules?; Why it matters: This defines early progression and MMO sustainability.; Known information: User proposed finite resources, low throughput, and possible world reset/recovery if first players fail.; Unknown information: Exact numbers and whether official server can reset.; Resolution path: Design mothership economy and failure modes.; Priority: P0; Related workstream: WORKSTREAM-04; Label: FACT

Row 5 ID: QUESTION-05; Question / issue: How many robot chassis exist at launch and what tradeoffs do they have?; Why it matters: Chassis define player role expression and balance.; Known information: User listed biped/quadruped/spider/tank-like options.; Unknown information: Launch set, capabilities, costs, progression.; Resolution path: Draft first chassis table.; Priority: P1; Related workstream: WORKSTREAM-05; Label: FACT

Row 6 ID: QUESTION-06; Question / issue: What can

nanobots do at each tech tier?; Why it matters: Nanobots could break progression if too strong.; Known information: User proposed nanobots as easy compute interface; assistant recommended material/energy/time limits.; Unknown information: Capability ladder and limits.; Resolution path: Define construction tool tiers.; Priority: P0; Related workstream: WORKSTREAM-06; Label: FACT + INFERENCE

Row 7 ID: QUESTION-07; Question / issue: How should player-created machines compile into simple equations without killing creativity?; Why it matters: This is central to performance and user-generated engineering.; Known information: User wants machine assemblies summarised as unit steps/process diagrams.; Unknown information: Compiler limits, modules, validation, exploit prevention.; Resolution path: Draft machine graph compiler rules.; Priority: P0; Related workstream: WORKSTREAM-07; Label: FACT

Row 8 ID: QUESTION-08; Question / issue: How can cities feel alive without worker NPCs?; Why it matters: No labour NPCs removes common visual/social signals.; Known information: User prefers no intelligent worker NPCs and automation instead.; Unknown information: Ambient machines, drones,

UI, population abstraction, city OS.; Resolution path: Design city feedback loops and visual proxies.; Priority: P1; Related workstream: WORKSTREAM-08; Label: FACT + INFERENCE

Row 9 ID: QUESTION-09; Question / issue: What exact information does fog of war hide, remember, infer, or reveal?; Why it matters: Affects gameplay, security, and simulation.; Known information: User required fog of war and sensed-only loading.; Unknown information: Sensor types, map sharing, hidden resources, enemy bases, caves.; Resolution path: Draft sensor/perception rules.; Priority: P0; Related workstream: WORKSTREAM-09; Label: FACT

Row 10 ID: QUESTION-10; Question / issue: What client compute is allowed in public MMO mode?; Why it matters: Client-shared compute is a core scaling idea but dangerous.; Known information: User asked about clients sharing compute; assistant recommended only visible/verifiable jobs.; Unknown information: Precise trust boundaries and validation mechanisms.; Resolution path: Create client compute policy.; Priority: P0; Related workstream: WORKSTREAM-10; Label: FACT + INFERENCE

Row 11 ID: QUESTION-11; Question / issue: What should Unreal actually do in the production stack?; Why it matters: Engine commitment affects architecture and team skills.; Known information: Assistant recommended Unreal as renderer/client, not source of truth.; Unknown information: Whether user accepts this and whether current UE features suffice.; Resolution path: Verify and decide renderer strategy.; Priority: P1; Related workstream: WORKSTREAM-11; Label: UNCERTAIN / UNVERIFIED

Row 12 ID: QUESTION-12; Question / issue: What is the first vertical slice?; Why it matters: A huge design needs a concrete implementation target.; Known information: Assistant proposed mothership-to-remote-spawn-lab loop.; Unknown information: User acceptance and exact milestone scope.; Resolution path: Approve or revise vertical slice.; Priority: P0; Related workstream: WORKSTREAM-12; Label: INFERENCE

The most blocking unresolved issues are the exact first vertical slice, field/delta data format, mothership constraints, machine graph compiler, fog-of-war sensor rules, and whether Unreal is merely a renderer/client or a larger part of production.

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Row 1 ID: RISK-01; Risk / failure mode: Scope explosion from full solar system, MMO, terraforming, machines, cities, and megaprojects at once.; Consequence: Project never reaches a playable core.; Likelihood: High; Severity: High; Mitigation: Define and build the vertical slice first.; Related workstream: WORKSTREAM-12; Label: INFERENCE

Row 2 ID: RISK-02; Risk / failure mode: Future assistant treats brainstormed ideas as final decisions.; Consequence: Spec becomes too rigid or misrepresents user intent.; Likelihood: Medium; Severity: High; Mitigation: Use FACT/INFERENCE/UNCERTAIN labels and decision statuses.; Related workstream: WORKSTREAM-13; Label: FACT

Row 3 ID: RISK-03; Risk / failure mode: Unreal capability assumptions become stale or overoptimistic.; Consequence: Wrong engine architecture or performance promises.; Likelihood: Medium; Severity: High; Mitigation: Verify current Unreal docs and prototype critical path.; Related workstream: WORKSTREAM-11; Label: UNCERTAIN / UNVERIFIED

Row 4 ID: RISK-04; Risk / failure mode: Nanobots become magic and remove the need for industry/logistics.; Consequence: Gameplay loses progression and constraints.; Likelihood: Medium; Severity: High; Mitigation: Bound construction by energy, materials, heat, time, and capability.; Related workstream: WORKSTREAM-06; Label: INFERENCE

Row 5 ID: RISK-05; Risk / failure mode: Client-shared compute leaks hidden information or enables cheating.; Consequence: MMO security failure.; Likelihood: Medium; Severity: High; Mitigation: Do not send hidden state to clients; use server authority and validation.; Related workstream: WORKSTREAM-09, WORKSTREAM-10; Label: INFERENCE

Row 6 ID: RISK-06; Risk / failure mode: Machine graph compiler allows exploit/lag machines.; Consequence: Players can crash servers or bypass economy.; Likelihood: High; Severity: High; Mitigation: Use complexity budgets, validation, and collapsed representations.; Related workstream: WORKSTREAM-07; Label: INFERENCE

Row 7 ID: RISK-07; Risk / failure mode: Cities feel empty without NPCs.; Consequence: Player-built

civilisation lacks emotional/visual feedback.;
Likelihood: Medium; Severity: Medium;
Mitigation: Use drones, traffic abstractions, city
OS UI, visible infrastructure, sensors, and ambient
machine activity.; Related workstream:
WORKSTREAM-08; Label: INFERENCE

Row 8 ID: RISK-08; Risk / failure mode: Planet
generation realism conflicts with fun layout.;
Consequence: Official world is plausible but not
playable, or fun but implausible.; Likelihood:
Medium; Severity: Medium; Mitigation: Use
plausibility score plus gameplay score and allow
labelled overrides.; Related workstream:
WORKSTREAM-02; Label: INFERENCE

Row 9 ID: RISK-09; Risk / failure mode: Saves/mod
packs/terraforming share tech but require different
conservation rules.; Consequence: Worldgen and
gameplay data corrupt each other conceptually.;
Likelihood: Medium; Severity: Medium;
Mitigation: Separate editor layers from gameplay-
constrained layers.; Related workstream:
WORKSTREAM-03; Label: INFERENCE

Row 10 ID: RISK-10; Risk / failure mode: Mothership
fabricator trivialises the tech tree.; Consequence:
Players skip factories and cities.; Likelihood:
Medium; Severity: High; Mitigation: Limit

feedstock, throughput, heat, scale, wear, and mission reserves.; Related workstream: WORKSTREAM-04; Label: FACT + INFERENCE

Row 11 ID: RISK-11; Risk / failure mode: Future aggregation merges this chat with other chats without preserving caveats.; Consequence: Master spec contains contradictions or unsupported requirements.; Likelihood: Medium; Severity: High; Mitigation: Use artifact ledger, verification queue, and manual review before finalising spec.; Related workstream: WORKSTREAM-13; Label: FACT

The main future-chat risk is over-certainty. A future assistant might treat the assistant's architecture recommendations as user decisions, or treat the user's brainstorming as final design. The safest approach is to preserve labels: user-stated goals are FACT; assistant synthesis is INFERENCE unless accepted; engine and external-source claims are UNCERTAIN / UNVERIFIED until checked.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat contributes the conceptual foundation for Dominion as a robotic seed-civilisation game. It should inform the master spec book chapters on engine

architecture, procedural star-system generation, planetary field layers, lore premise, player bodies and spawning, construction/terraforming, automation/factories, fog of war, MMO architecture, and first vertical slice.

Some items should become formal requirements: deterministic core, sparse deltas, science-bounded generation, robot avatars, mothership-constrained progression, automation-first labour model, machine graph abstraction, fog-of-war secrecy, and vertical-slice-first planning. Other items should remain context or open issues: human eggs/next ship lore mystery, exact robot chassis, world reset rules, exact Unreal role, and exact MMO scale claims.

Likely overlaps with other chats include Domino engine architecture, deterministic simulation, procedural planets, resource economy, construction systems, multiplayer/server architecture, and Unreal feasibility. The aggregator should compare those reports before finalising requirements.

14. What I Should Remember

- The strongest design identity is **robotic seed civilisation**: players are robots created by a mothership to rebuild industrial capacity on a scientifically generated planet.

- The strongest technical identity is **deterministic sparse simulation**: the universe is seeds, fields, deltas, commands, graphs, ledgers, and snapshots, rendered locally when sensed.
- The strongest gameplay loop is **spawn, survey, mine, build power, refine, fabricate, expand spawn infrastructure, and scale toward cities/megaprojects**.
- The strongest user preference is **automation over worker NPCs**. Do not default back to human labour simulation.
- The mothership is both powerful and constrained: it explains knowledge and spawning but must not replace industry.
- Nanobots are an interface and actuator, not free matter or free energy.
- Factories and machines should compile into process graphs when possible.
- Fog of war is a simulation/security boundary, not just a shader.
- Unreal may be useful, but the authoritative universe should not be assumed to live inside Unreal without verification.

- The next best action is to formalise a vertical slice and data model.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- What are the five most important Dominion design pillars from this chat?
- Which ideas came directly from me and which were assistant recommendations?
- What changed when the design shifted from human labour to robotic automation?

15.2 Decisions

- Which decisions are safe to promote to formal requirements now?
- Which assistant recommendations still need explicit user approval?
- Should "Unreal as renderer/client only" become a project rule or stay a hypothesis?

15.3 Tasks and Next Actions

- Turn the proposed vertical slice into a milestone plan.

- Draft the deterministic core data model for fields, deltas, events, and state hashes.
- Define the mothership economy and starting resource constraints.

15.4 Artifacts and Files

- Show me the artifact ledger and explain what should feed into a master Project Spec Book.
- Convert the generated spec sheet into a formal design document outline.

15.5 Risks and Verification

- What are the biggest feasibility risks in the Dominion design?
- What current Unreal claims need verification before engine planning?
- What could make nanobot construction break the economy?

15.6 Future Spec Book / Aggregation

- Which chapters of the master Project Spec Book should this chat feed into?

- What cross-chat conflicts should an aggregator look for?

15.7 Deep-Dive Questions Specific to This Chat

- Design the first three robot chassis and their manufacturing costs.
- Design the mothership's finite fabrication system.
- Design the first machine graph compiler format.
- Design the field-layer stack for procedural terrain, painted overrides, and gameplay deltas.
- Design the remote spawn lab milestone loop.

16. Compact Human Summary

This chat captured the core identity and technical direction for *Dominium* as a deterministic, large-scale, procedural robotic civilisation game. It began with a very broad technical question: how can Domino engine and *Dominium* support full-scale solar systems, procedural planets, terraforming, cut/fill construction, megaprojects, player-designed machines and factories, fog of war, sparse simulation, client-shared compute, single-player universes, and MMO persistent universes? The user also asked whether Unreal could do this at low latency and high FPS with millions of players.

The first major conclusion was architectural: the world should not be built as a giant live Unreal scene. Instead, Domino should be treated as the authoritative deterministic simulation and database layer. The universe should be stored as seeds, rules, fields, sparse deltas, event logs, snapshots, state hashes, machine graphs, resource ledgers, and visibility records. Renderers such as Unreal can visualise the currently sensed local state, but rendering should not own world truth. This is an assistant recommendation, not an explicitly accepted user decision, but it follows strongly from the user's scale and determinism requirements.

The user then refined the game into a much more specific premise. The game should have procedurally generated star systems and planets, with parameters bounded by plausible science rather than arbitrary sliders. Designers should be able to guide results, perhaps even by reverse generation: specify the kind of planet desired, then have the system find plausible physical parameters. The same underlying field/delta technology should support procedural generation, painted overrides, planet packs, save files, terraforming, and construction.

The strongest creative idea was the robotic mothership. Humans launched a robotic seed ship to another solar system. The ship lands on a suitable planet and manufactures robot bodies to restart civilisation. Players spawn as robots from physical fabrication machines.

They may choose different chassis such as biped, quadruped, spider, or tank-like bodies. New frontier spawn points are not menu abstractions: players must build labs or fabricators that have power, materials, and manufacturing capability.

This premise solves many design problems. The HUD is diegetic because players are robots. Quest markers are diegetic because the ship AI can generate plans. Recipes exist because the mothership brought human knowledge. Players do not need to invent CPUs, railways, or calendars from scratch. However, the mothership cannot simply build everything forever. The user specifically wanted finite resources, limited throughput, and a need for players to build mines, refineries, factories, and cities. The mothership is excellent at knowledge and precision, but the planet is required for scale.

Another crucial design shift was rejecting worker NPC labour as a core simulation. The user initially considered NPC workers but then pointed out that labour, pathfinding, and person-like simulation are expensive and unnecessary if the civilisation is robotic. The game should start with automation. Nanobots, machine controllers, AI agents, construction swarms, and process diagrams can do the work. This means cities do not need to simulate thousands of workers. They need power grids, logistics, fabricators, roads, rails, sensors, spawn labs, permissions, and industrial processes.

Construction should be built around blueprinting and nanobots. Players can preview a design, stage it, reserve resources, and commit it. Cut/fill terrain editing, mining, foundations, buildings, and megaprojects can use the same deterministic sparse operation model. But nanobots must not be magic. Every action should require materials, energy, time, capability, and usually produce heat or waste.

Factories and machines should be visually rich when nearby, but computationally simple when distant. The user wanted machines and vehicles to be summarised as equations or process steps. The future design likely needs a machine graph compiler: players assemble components, and Domino compiles the result into capabilities, input/output rates, power draw, heat, reliability, and constraints. This lets the game support creative engineering without simulating every bolt, belt, worker, and moving part everywhere.

The main unresolved work is converting this concept into a formal spec and vertical slice. The best proposed first slice is: one planet region, one mothership, one robot body, one ore deposit, one power source, one cut/fill tool, one blueprint system, one mine, one refinery, one fabricator, one remote spawn lab, and one fog-of-war sensor model. If that loop is fun and deterministic, the project can scale outward. If it is not, adding solar systems and MMO scale will not fix it.

7.6 Dominium Deterministic Solar System Architecture Reader Brief

Source: docs/archive/conversations/Dominium_Complete_Conversation/deterministic_solar_system_handoff_file/Dominium_Deterministic_Solar_System_Architecture__06_reader_brief.md

Report type: reader_brief

Size: 3264 bytes

7.6.1 Reader Brief - Dominium Deterministic Solar-System Architecture

This chat defined a major direction for Dominium: a deterministic robotic seed-civilisation game where players are manufactured robot bodies spawned by a mothership, then expand across scientifically generated planets by surveying, mining, building power, refining materials, fabricating machines, constructing spawn labs, automating cities, terraforming, and eventually building megaprojects.

Top 20 things to know

1. Domino should probably be the deterministic simulation/data core.
2. Unreal should not be assumed to own the

authoritative universe.

3. Planets should be science-bounded procedural outputs, not arbitrary slider toys.
4. Designer painting should be an override layer, not the main workflow.
5. Save files, packs, painted fields, terraforming, and construction should share field/delta technology.
6. The main lore premise is a robotic mothership sent by humans.
7. Players spawn as robot bodies from physical machines.
8. Remote spawn labs are a key expansion milestone.
9. The mothership provides recipes/knowledge but has finite resources and low throughput.
10. Robot bodies explain the HUD and strong inventory/carry capacity.
11. Nanobots are the construction interface, but must consume materials, energy, time, and heat budget.
12. Worker NPC labour was strongly deprioritised.
13. Automation and process graphs are central.
14. Machines/factories should compile into aggregate equations when distant.

15. Cities should be easy to start but advanced industry should require large supply chains.
16. Fog of war is a data/security model, not merely visual shading.
17. Client-shared compute must not expose hidden MMO truth.
18. Millions across shards/universe is different from millions in one real-time area.
19. The proposed first vertical slice is mothership-to-remote-spawn-lab.
20. Many assistant suggestions are not yet accepted decisions.

Decisions

See DECISION-01 through DECISION-13. Strongest user-stated decisions: science-bounded procedural planets, robotic mothership premise, robot bodies, nanobot construction, automation over worker NPCs, mothership-limited fabrication, machine process abstraction, sparse/collapsed simulation.

Pending tasks

Highest priority: define design pillars, deterministic core, planet generator schema, field/delta format, mothership

economy, robot chassis, nanobot construction pipeline, machine graph compiler, fog-of-war model, MMO trust boundaries, and first vertical slice.

Open questions

Most important: official scope; realism bounds; field format; mothership limits; robot chassis; nanobot limits; machine compiler; city liveliness without NPCs; fog-of-war rules; client compute; Unreal role; first vertical slice.

Artifacts

Main artifacts are the visible chat, uploaded preservation prompt, prior assistant responses, external references cited earlier, and this generated package.

Verification items

Verify Unreal capabilities, NASA/JPL/SPICE/IAU data/licensing, deterministic coordinate/math approach, network scale assumptions, sparse terrain performance, and machine graph compiler feasibility.

Best next step

Turn the proposed vertical slice into a formal milestone/spec document.

7.7 Dominion Deterministic Solar System Architecture In Chat Reader

Source: docs/archive/conversations/Dominium_Complete_Conversation/deterministic_solar_system_handoff_file/Dominium_Deterministic_Solar_System_Architecture__09_in_chat_reader.md

Report type: in_chat_reader

Size: 2199 bytes

7.7.1 In-Chat Reader - Dominion Deterministic Solar-System Architecture

Package overview

This preservation package converts the visible Dominion/Domino chat into a human-readable report, structured registers, context transfer packet, YAML-style spec sheet, aggregator packet, audit, reader brief, and future-chat bootstrap prompt.

File index

- 00_manifest.md: package index and caveats.
- 01_human_readable_report.md: sections 0-16, the main report.

- `02_context_transfer_packet.md`: section 29 handoff.
- `03_spec_sheet.yaml`: section 30 YAML-style spec sheet.
- `04_registers.md`: sections 17-28.
- `05_aggregator_packet.md`: section 31.
- `06_reader_brief.md`: short reader-oriented brief.
- `07_verification_and_audit.md`: sections 32-34 and verification queue.
- `08_future_chat_bootstrap_prompt.md`: reusable prompt for a new chat.
- `09_in_chat_reader.md`: this guide.
- `handoff_package.zip`: all files bundled.

Plain-English explanation

The chat's substance is that *Dominium* should be shaped as a deterministic robotic seed-civilisation game. The user wants science-bounded procedural planets and solar systems, sparse editable terrain, physical robot spawning, nanobot construction, machine/factory abstraction, fog of war, and possible MMO/private/single-player modes. The strongest design move was replacing worker NPC labour with automation because the civilisation is robotic.

Question menu

Ask next:

- Which decisions are confirmed versus recommendations?
- Turn the vertical slice into a milestone plan.
- Define the field/delta data format.
- Define the mothership economy.
- Define first robot chassis.
- Define nanobot construction tiers.
- Define machine graph compiler.
- Define fog-of-war/sensor rules.
- Verify Unreal's role.
- Prepare this chat for master spec merge.

Top things to preserve

Robot mothership premise, deterministic sparse Domino core, science-bounded planets, automation over worker NPCs, nanobot blueprint construction, finite mothership fabrication, machine process graphs, fog-of-war secrecy, first vertical slice.

Safest next actions

Review decisions with uncertainty labels, then formalise the vertical slice.

7.8 Dominion Deterministic Solar System Architecture Accompanying Detailed Summary And Report

Source: docs/archive/conversations/Dominium_Complete_Conversation/deterministic_solar_system_handoff_file/Dominium_Deterministic_Solar_System_Architecture__10_accompanying_detailed_summary_and_report.md

Report type: accompanying_detailed_summary

Size: 27925 bytes

7.8.1 Accompanying Detailed Human-Readable Summary and Report - Dominion Deterministic Solar-System Architecture

0. Purpose of this accompanying report

This file accompanies the larger preservation package for the Dominion / Domino engine conversation. It is designed to be the human-readable bridge between the

raw chat, the previous preservation files, and the final single ZIP bundle.

It summarises what the conversation covered, what was decided or merely recommended, what was deferred, what artifacts were created, what was checked in this packaging pass, and what should happen next. It uses these epistemic labels where needed:

- **FACT** - explicitly stated or clearly established in this conversation.
- **INFERENCE** - reasonably inferred from the conversation, but not directly confirmed as a final decision.
- **UNCERTAIN / UNVERIFIED** - incomplete, stale, not independently checked here, or dependent on future technical validation.
- **PROJECT-CONTEXT** - coming from project-level context, files, or generated artifacts rather than only the visible discussion.

This report is not a replacement for the full preservation report. It is the readable companion and audit pass requested after the first package was generated.

1. High-level summary

This conversation defined the core direction for **Dominium**, a game built around or with the **Domino engine**. The project began as an extremely ambitious technical question: how could a deterministic game support a full-scale real or custom solar system, fully procedural planets, player terraforming, cut/fill terrain editing, megaprojects, custom machines, factories, fog of war, sparse construction and destruction, sensed-only simulation, MMO-scale persistence, single-player multi-universe play, and possible shared compute from player clients?

The first important architectural answer was that **Dominium** should not be treated as a normal open-world map inside Unreal. The authoritative world should be a deterministic simulation and data system. Unreal, Raylib, or another renderer can display the local world, but the source of truth should be Domino: fixed-tick rules, deterministic commands, procedural generation, sparse deltas, event logs, snapshots, state hashes, fog-of-war filtering, and multi-resolution simulation.

The user then shifted the discussion from pure feasibility to the actual game identity. The game became a **robotic seed-civilisation game**. In the fiction, humans launched a robotic mothership to another solar system. The mothership lands on a suitable planet and

manufactures robot bodies to restart civilisation. Players are not humans with backpacks; they are robot agents embodied in physical chassis. They can spawn from the mothership, later build remote spawn labs, and choose bodies such as bipeds, quadrupeds, spider-like robots, tank-like frames, drones, industrial frames, or other robot forms.

This premise solved many design problems at once. The HUD is diegetic because the player is a robot. Blueprints are diegetic because the player can project construction overlays through sensors. Quest markers are diegetic because the mothership or local AI planner can convert player goals into material, power, survey, and construction tasks. Respawn is diegetic because new bodies are physically fabricated. Missing human NPC labour is not a flaw because the civilisation is explicitly robotic and automated.

A major user-stated direction was to avoid using worker NPCs as the core labour system. The user called out the cost of pathfinding, scheduling, physics, and labour simulation. Instead, the game should lean into automation from the start: nanobots, construction swarms, AI controllers, drones, process graphs, machine diagrams, logistics networks, and factory equations. A city should not need 100,000 simulated workers to function. It needs power, material flow, fabrication capacity, spawn infrastructure, sensors, permissions,

logistics, and machine processes.

The most important technical design pattern is this: **store the world as fields, graphs, ledgers, sparse deltas, and event logs; render it as terrain, cities, machines, robots, and megaprojects only when players can sense it.** This pattern applies to planets, terrain, factories, machines, civilisations, fog of war, and multiplayer.

2. Conversation chronology

2.1 Initial large-scale architecture question The user first asked how to build a deterministic Dominion game with:

- full-scale real solar-system support;
- fully recreated or custom planets;
- procedural universes and planets generated from data;
- player-built civilisations;
- terraforming;
- cut/fill terrain editing;
- megaprojects;
- player-designed machines, devices, and factories;

- fog of war;
- sensed-only simulation;
- collapsed state for unobserved areas;
- sparse construction and destruction;
- single-player multi-universe support;
- MMO-style single persistent universe support;
- possible client-shared compute;
- low latency and high FPS;
- potential Unreal integration.

The assistant response framed this as a distributed deterministic simulation problem, not a conventional game-level streaming problem. It recommended hierarchical coordinates, sparse terrain deltas, simulation LOD, event sourcing, fog-of-war filtering, and server authority.

2.2 More detailed engine feasibility answer A later assistant response refined the architecture around three roles: Domino as deterministic simulation, Unreal as renderer/client/tooling, and Dominion as game rules. It also drew a feasibility boundary: millions of users can exist across a logical persistent universe, but millions cannot occupy one fully interactive real-time physics

bubble at high FPS and low latency. This was an assistant recommendation / assessment, not a user-confirmed final architectural decision.

2.3 User's game-identity monologue The user then described how planets and systems should be generated. The user wanted procedural generation with scientific bounds rather than arbitrary sliders. Designers should set physical parameters and get plausible planets, then nudge or override details. The user also raised a reverse-generation idea: design the desired planet outcome, then let a solver find plausible orbital/planetary parameters that make it work.

The user wanted procedural fields to be overlayable with painted fields and other authored data. The same underlying technology should support world generation, planet packs, save files, terrain edits, and terraforming.

The user then described the lore: a robotic mothership sent by humans finds another solar system, lands on the most habitable planet, and manufactures robots using advanced technology, potentially nanotechnology. There could be mystery around later ships carrying human eggs or biological payloads, but the main gameplay premise is robotic rebuilding of civilisation.

2.4 Spawning and expansion The user proposed that every player joins through a physical spawn point: a

machine that manufactures robot bodies. Initially, this is the mothership. Later, players can build remote labs or spawn fabricators to expand the frontier. This turns multiplayer spawning into a material, energy, logistics, and territory system rather than a menu abstraction.

The mothership has recipes and knowledge, so players do not need to invent CPUs, rail gauges, calendars, or machine tools from first principles. The progression challenge is not knowledge discovery; it is building the industrial capacity to manufacture advanced components locally.

2.5 Nanobot construction and blueprints The user suggested nanobots or an invisible construction force because it is computationally cheaper than simulating labourers. This allows cut/fill, blueprint previews, and deterministic committed construction. The assistant later formalised this as a staged pipeline: preview, plan, reserve, stage, execute, commit, and operate.

The key caveat is that nanobots must not create free matter or free work. They should require materials, power, time, heat handling, nearby capability, permissions, and sensor data.

2.6 Automation over worker NPCs This was one of the strongest design pivots. The user initially considered NPC workers because labour can create reasons for

automation, but then decided that worker labour is expensive to compute and unnecessary for a robotic civilisation game. The user said that if the computer is doing the work, the game should say the computer is doing the work rather than pretending there are person-like workers.

This means future design should avoid defaulting to human settlement simulation. Cities should be automated infrastructure systems, not primarily NPC population simulations.

2.7 Machine and factory abstraction The user wanted machines, vehicles, rockets, devices, and factories, but did not want every part simulated forever. A machine assembly should be reducible to one functional object or process equation when appropriate. The assistant proposed a machine graph compiler: nearby machines can be visual and detailed, while distant or stable machines collapse into input/output rates, power draw, heat, buffers, reliability, and maintenance debt.

2.8 Preservation package The user uploaded a detailed preservation prompt requiring a maximum-fidelity report, structured registers, spec sheet, aggregator packet, self-audit, export files, and ZIP package. A first preservation package was generated,

including a human-readable report, context transfer packet, spec sheet, registers, aggregator packet, reader brief, verification/audit file, future-chat bootstrap prompt, in-chat reader, and handoff ZIP.

The current request then asked for an accompanying human-readable detailed summary/report and a single ZIP with all files. This pass creates that companion report and a final complete bundle.

3. Main design outcomes

3.1 Dominium is a robotic seed-civilisation game

FACT: The user described a robotic mothership that manufactures robot bodies to restart civilisation.

INFERENCE: This is the cleanest game identity around which the technical systems should be organised.

This identity explains otherwise arbitrary game mechanics: HUD, blueprints, respawn, recipes, nanobot construction, automation, quest markers, body swapping, and frontier spawn points. It also creates natural long-term goals: restore human civilisation, build robot civilisation, terraform the planet, prepare for incoming humans, or diverge from the mothership's original mission.

3.2 Domino should be the deterministic

authoritative core **INFERENCE:** The assistant

repeatedly recommended that Domino own authoritative state.

UNCERTAIN: This was not explicitly accepted as a final engineering decision by the user, though it fits the user's stated requirements.

The reason is technical: full-scale planets, sparse edits, persistent construction, fog of war, factories, and MMO play require a deterministic data model. A renderer should not be the source of truth for every voxel, road, machine, resource, and entity.

3.3 Unreal should be treated as renderer/tooling unless proven otherwise INFERENCE /

UNVERIFIED: The assistant recommended Unreal as client, renderer, editor, and local interaction layer, not as the authoritative universe simulator.

This should remain in the verification queue. Current Unreal features and limits should be checked before becoming a binding project decision. The key architectural concern remains valid: even if Unreal is used, Domino must preserve deterministic truth, sparse persistence, fog-of-war filtering, and server authority.

3.4 Science-bounded procedural generation is central FACT:

The user wants realistic-bounded procedural generation, not arbitrary sliders and not

hand-painted worlds as the default.

The design should support forward generation from physical parameters and possible reverse generation from desired outcomes. It should allow designer nudges and overrides while preserving plausibility scoring.

3.5 Field layers unify worldgen, saves, packs, and terraforming **FACT + INFERENCE:** The user wanted procedural fields overlaid with painted fields and wanted the same technology to support saves and planet packs. The assistant formalised this as a field-layer/delta stack.

The distinction that must be preserved is that editor/worldgen layers can create authored data freely, while gameplay terraforming/construction layers must obey material, energy, time, and permission constraints.

3.6 Mothership knowledge does not equal infinite fabrication **FACT:** The user wanted the mothership to know how to make things but have limited resources, throughput, and scale.

This avoids asking players to reinvent basic technology while still requiring them to build infrastructure. The mothership provides recipes, guidance, starter fabrication, and precision; the planet provides mass, power, scale, and long-term industrial capacity.

3.7 Worker NPC labour is deprioritised FACT:

The user explicitly moved away from worker NPC labour because it creates expensive simulation and pathfinding problems.

This is a major design constraint. Future chats should not casually reintroduce a human-like worker economy unless it is explicitly reconsidered. If visible activity is needed, use drones, construction swarms, traffic flows, animated machine systems, or local non-authoritative visuals.

3.8 Machine and factory systems should compile into process graphs FACT:

The user wanted complex machines to be reducible to process equations when possible.

This is essential for scale. A factory can be visually represented when nearby, but collapsed into input/output rates, power draw, heat output, reliability, buffers, and maintenance debt when far away.

3.9 Fog of war is both gameplay and information security FACT:

The user required fog of war and sensed-only simulation.

INFERENCE: For MMO mode, this requires server-side filtering and cannot trust clients with hidden truth.

A robot civilisation makes this natural: sensors,

satellites, relays, surveys, map sharing, radar, seismic scans, and communication networks define what a player or faction knows.

4. What was decided, recommended, and left open

4.1 Strong user-stated directions These are the safest items to preserve as user-intended design direction:

1. Dominion should use procedural generation for systems and planets.
2. Planet generation should be bounded by realistic science rather than arbitrary sliders.
3. Designers should be able to nudge and override generated planets.
4. Procedural fields and painted/override fields should coexist.
5. Save-file and pack technology should likely share the same field/delta foundation as terraforming.
6. The lore involves a robotic mothership sent by humans.
7. Players spawn as robots from physical machines.
8. Remote spawn labs are a meaningful frontier expansion milestone.

9. The mothership provides knowledge/recipes but has limited resources and throughput.
10. Automation should replace worker NPC labour as the primary construction/economy model.
11. Machines and factories should be abstractable into process equations.
12. Cities should be easy to start but advanced industry should require real infrastructure.

4.2 Assistant recommendations not yet final

These should not be treated as final project decisions unless the user accepts them:

1. Domino should own deterministic authoritative simulation.
2. Unreal should be only renderer/client/tooling.
3. Public MMO clients should never receive hidden truth.
4. The first vertical slice should be mothership-to-remote-spawn-lab.
5. Heat and maintenance debt should be major bottlenecks.
6. Machine designs should go through a compiler/validator.

7. Planet generation should expose plausibility scores.
8. Official persistent servers should avoid hard resets except for alpha/seasons/disaster recovery.

4.3 Deferred design work The conversation put off many details for later. The most important deferred work is:

- exact launch scope: one designed star system, many systems, or galaxy-scale framework;
- deterministic math, tick rates, and serialization rules;
- field-layer and sparse-delta data schema;
- planet generator parameters and plausibility model;
- reverse-generation solver design;
- mothership resource budgets, reserves, throughput, fabrication wear, and safety protocols;
- exact robot chassis and body-swapping rules;
- death, backup, respawn, and communication-range rules;
- nanobot construction tiers and limits;
- blueprint validation and permissions;

- machine graph compiler rules;
- factory aggregation rules;
- city progression and how cities feel alive without NPCs;
- fog-of-war sensor model;
- MMO authority, sharding, server handoff, and client compute boundaries;
- anti-cheat, griefing, claims, war rules, and moderation;
- Unreal integration verification;
- legal/licensing review for real Solar System datasets;
- first playable vertical slice specification.

5. Things the conversation did

This conversation did four kinds of work.

First, it defined a high-level technical architecture: deterministic sparse world simulation, not a giant live scene. The architecture separated world truth from visual representation.

Second, it established a strong creative identity: robotic seed civilisation, physical robot spawning, mothership

knowledge, limited fabrication, nanobot construction, automation, machine graphs, and city-scale industrial progression.

Third, it created a preservation package. The earlier package included a full human-readable report, context packet, YAML spec sheet, structured registers, aggregator packet, reader brief, verification/audit file, future-chat bootstrap prompt, in-chat reader, and a handoff ZIP.

Fourth, this current pass checked the workspace, confirmed the presence of the exported files, created this accompanying detailed report, generated a complete bundle manifest and checksum file, and built a final single ZIP bundle.

6. What was checked in this pass

The packaging pass found the following relevant artifacts in the workspace:

Row 1 File: Dominion_Deterministic_Solar_System_Architecture__00_manifest.md; Group: current deterministic package; Size: 2389 bytes; SHA-256 prefix: d8e9ad908bbef8e5...

Row 2 File: Dominion_Deterministic_Solar_System_Architecture__01_human_readable_report.md; Group: current deterministic package; Size: 57540

bytes; SHA-256 prefix: 320d5e21af78e139...

Row 3 File: Dominion_Deterministic_Solar_System_Architecture__02_context_transfer_packet.md; Group: current deterministic package; Size: 4303 bytes; SHA-256 prefix: a07ff9d193d55bc2...

Row 4 File: Dominion_Deterministic_Solar_System_Architecture__04_registers.md; Group: current deterministic package; Size: 40599 bytes; SHA-256 prefix: 3c7af3db6949e29e...

Row 5 File: Dominion_Deterministic_Solar_System_Architecture__05_aggregator_packet.md; Group: current deterministic package; Size: 27431 bytes; SHA-256 prefix: 9674464e0379b64f...

Row 6 File: Dominion_Deterministic_Solar_System_Architecture__06_reader_brief.md; Group: current deterministic package; Size: 3264 bytes; SHA-256 prefix: 56d50d85c678d79c...

Row 7 File: Dominion_Deterministic_Solar_System_Architecture__07_verification_and_audit.md; Group: current deterministic package; Size: 4723 bytes; SHA-256 prefix: 6a7e79ed0adad485...

Row 8 File: Dominion_Deterministic_Solar_System_Architecture__08_future_chat_bootstrap_prompt.md; Group: current deterministic package;

Size: 978 bytes; SHA-256 prefix:
984d223840f548c1...

Row 9 File: Dominion_Deterministic_Solar_System_Architecture__09_in_chat_reader.md; Group: current deterministic package; Size: 2199 bytes; SHA-256 prefix: 6cd8b657a8b1b9cf...

Row 10 File: Dominion_Deterministic_Solar_System_Architecture__10_accompanying_full_conversation_report.md; Group: current deterministic package; Size: 32823 bytes; SHA-256 prefix: 9aba7fe219d17212...

Row 11 File: Dominion_Deterministic_Solar_System_Architecture__10_accompanying_human_readable_conversation_report.md; Group: current deterministic package; Size: 20191 bytes; SHA-256 prefix: 9c3b31be9005922e...

Row 12 File: Dominion_Deterministic_Solar_System_Architecture__10_accompanying_human_readable_detailed_summary_report.md; Group: current deterministic package; Size: 29184 bytes; SHA-256 prefix: 9e10ec545a94c73a...

Row 13 File: Dominion_Deterministic_Solar_System_Architecture__12_bundle_verification.md; Group: current deterministic package; Size: 4094 bytes; SHA-256 prefix: 30d4738ef8e979bf...

- Row 14** File: `Dominium_Deterministic_Solar_System_Architecture__03_spec_sheet.yaml`;
Group: current deterministic package; Size: 58575 bytes; SHA-256 prefix: `0dcbbc5509d71342...`
- Row 15** File: `Dominium_Deterministic_Solar_System_Architecture__handoff_package.zip`;
Group: current deterministic package zip; Size: 63147 bytes; SHA-256 prefix: `469bfb68b19198d9...`
- Row 16** File: `Dominium_Robotic_Seed_Civilisation_Architecture__00_manifest.md`; Group: older related workspace package; Size: 2627 bytes; SHA-256 prefix: `320c1fb070acf1ed...`
- Row 17** File: `Dominium_Robotic_Seed_Civilisation_Architecture__01_human_readable_report.md`;
Group: older related workspace package; Size: 33167 bytes; SHA-256 prefix: `0b1e56f0d8e57246...`
- Row 18** File: `Dominium_Robotic_Seed_Civilisation_Architecture__02_context_transfer_packet.md`;
Group: older related workspace package; Size: 5362 bytes; SHA-256 prefix: `2212c0a5be7f1ecf...`
- Row 19** File: `Dominium_Robotic_Seed_Civilisation_Architecture__04_registers.md`; Group: older related workspace package; Size: 37669 bytes; SHA-256 prefix: `d02a062de85e840f...`

Row 20 File: `Dominium_Robotic_Seed_Civilisation_Architecture__05_aggregator_packet.md`; Group: older related workspace package; Size: 25376 bytes; SHA-256 prefix: `056eb47ecc86358d...`

Row 21 File: `Dominium_Robotic_Seed_Civilisation_Architecture__06_reader_brief.md`; Group: older related workspace package; Size: 3058 bytes; SHA-256 prefix: `95b5d819810c1aaa...`

Row 22 File: `Dominium_Robotic_Seed_Civilisation_Architecture__07_verification_and_audit.md`; Group: older related workspace package; Size: 5158 bytes; SHA-256 prefix: `046540e67e94aea1...`

Row 23 File: `Dominium_Robotic_Seed_Civilisation_Architecture__08_future_chat_bootstrap_prompt.md`; Group: older related workspace package; Size: 980 bytes; SHA-256 prefix: `4a95d64912ca6a74...`

Row 24 File: `Dominium_Robotic_Seed_Civilisation_Architecture__09_in_chat_reader.md`; Group: older related workspace package; Size: 2406 bytes; SHA-256 prefix: `f52c2ca140b41e32...`

Row 25 File: `Dominium_Robotic_Seed_Civilisation_Architecture__03_spec_sheet.yaml`; Group: older related workspace package; Size: 53240 bytes; SHA-256 prefix: `3b0368853afb7ef7...`

Row 26 File: `Dominium_Robotic_Seed_Civilisation_Architecture__handoff_package.zip`; Group: older related workspace package zip; Size: 54889 bytes; SHA-256 prefix: `972af06794dd4713...`

Row 27 File: `Pasted text.txt`; Group: source prompt; Size: 31225 bytes; SHA-256 prefix: `6686114753accfa0...`

Important notes:

- The current deterministic package files are present as individual files and as a handoff ZIP.
- An older related package labelled `Dominium_Robotic_Seed_Civilisation_Architecture` is present in the workspace. It is included in the final bundle as an older related workspace artifact, not as the primary source of truth.
- The uploaded `Pasted text.txt` preservation prompt is included as the source prompt that defined the preservation/export requirements.
- This accompanying report and the complete bundle manifest/checksum file are new outputs from this pass.

7. Risks and what future chats might get wrong

Future chats could easily mishandle this material. The main risks are:

1. **Treating assistant recommendations as user decisions.** The user clearly described many design directions, but some architecture details are assistant recommendations.
2. **Reintroducing worker NPCs by default.** The user specifically moved away from that because it is computationally expensive and unnecessary for a robotic civilisation.
3. **Making nanobots magic.** Nanobots must require energy, material, time, heat handling, and capability limits.
4. **Assuming the mothership solves all progression.** It has knowledge and precision, not infinite mass production.
5. **Treating Unreal as already chosen or sufficient.** Unreal may be useful, but its role remains to be verified.
6. **Simulating too much.** The core pattern is collapsed simulation, sparse deltas, and sensed-only expansion.

7. **Leaking fog-of-war data to clients.** In MMO mode, clients cannot be trusted with hidden state.
8. **Confusing star system and galaxy scope.** The user used broad language, but the concrete design focuses on star systems and planets.
9. **Overlooking operations tooling.** A persistent MMO also needs backup, audit, moderation, rollback, migration, load testing, and admin tools.
10. **Ignoring versioning.** Procedural worlds, saves, mods, and server rules need generator/rules/schema versioning.

8. Missing or not-yet-specified areas worth adding later

The conversation already covered a lot, but future specs should also explicitly address:

- project naming boundaries: Dominion vs Domino engine vs official server vs private universes;
- exact data ownership model for saves, mods, and official servers;
- deterministic test harness and replay debugger;
- schema migration and old-universe compatibility;

- accessibility and onboarding for complex construction systems;
- tutorialising the mothership, blueprint, and spawn-lab loop;
- abuse prevention for offensive constructions and lag machines;
- public/private sharing of planet packs;
- UI for scientific sliders and plausibility warnings;
- player governance and territorial claims;
- market/economy controls if MMO trade exists;
- environmental ethics if terraforming native worlds is central;
- rules for native life, wildlife, hazards, mobs, or ecology;
- disaster simulation and recovery;
- cross-server universe consistency;
- mod trust and script sandboxing;
- data licensing for real astronomical or planetary data.

9. Best next action

The best next action is to turn the proposed first vertical slice into a formal milestone spec. The milestone should cover:

1. one generated planet region;
2. one mothership landing/spawn zone;
3. one or two robot chassis;
4. one survey mechanic;
5. one ore/material deposit;
6. one power source;
7. one cut/fill terrain tool;
8. one blueprint preview/commit workflow;
9. one nanobot construction actuator;
10. one mine;
11. one refinery;
12. one fabricator;
13. one remote spawn lab;
14. one fog-of-war/sensor model;
15. deterministic replay and save/load checks.

The success criterion should be simple: a player can spawn, survey, mine, build power, refine material, fabricate components, construct a remote spawn lab, and spawn or swap into a new body there, with deterministic replay and sparse saved state.

10. Final carry-forward summary

Dominium should be preserved as a deterministic robotic seed-civilisation project. Its core promise is not merely "large procedural planets," but a coherent system where players are robot agents using a mothership's knowledge and limited fabrication capacity to bootstrap civilisation on scientifically generated worlds.

The strongest design identity is automation-first. Players should not manage armies of worker NPCs. They should design, command, and expand robotic infrastructure: sensors, nanobots, machine graphs, factories, logistics networks, spawn labs, cities, terraforming systems, and megaprojects.

The strongest technical identity is sparse deterministic simulation. The world should be generated from seeds and data, modified through sparse deltas and event logs, and expanded into visual detail only when sensed.

Distant machines become process equations. Unknown terrain remains hidden. Unseen regions remain collapsed. Clients can assist with safe work, but hidden

truth and authority-critical state remain server-controlled in MMO mode.

The strongest unresolved issue is implementation focus. The project should not jump directly to galaxy/MMO scale. It should prove the mothership-to-remote-spawn-lab loop first. That loop tests the key systems: procedural terrain, robot spawning, fog of war, cut/fill, nanobot construction, resource accounting, fabrication, and sparse deterministic persistence.

This accompanying report, the prior preservation files, the source prompt, and the final ZIP bundle should be preserved as the current handoff package for future Dominion specification work.

7.9 Dominion Deterministic Solar System Architecture Accompanying Human Readable Conversation Report

Source: docs/archive/conversations/Dominium_Complete_Conversation/deterministic_solar_system_handoff_file/Dominium_Deterministic_Solar_System_Architecture__10_accompanying_human_readable_conversation_report.md

Report type: accompanying_human_readable

Size: 20191 bytes

7.9.1 Accompanying Human-Readable Detailed Summary and Report - Entire Conversation

Chat label: Dominion Deterministic Solar-System Architecture

Date anchor: 2026-05-27 Australia/Melbourne

Source scope: This conversation only, plus files visible in `/mnt/data` that were generated or uploaded during the preservation workflow.

Purpose: This is an additional plain-English companion report that sits beside the structured preservation package. It explains the whole conversation in a more continuous way and records what was discussed, what was decided, what was merely recommended, what was created, what was deferred, and what must be checked later.

1. What this conversation was about

This conversation was about shaping **Dominium** and the **Domino engine** into a deterministic, procedural, solar-system-scale robotic civilisation game. The user began with a broad technical question: how could a game support a full-scale real or custom solar system, fully recreated procedural planets, player-built civilisations, terraforming, cut-and-fill terrain editing,

megaprojects, machine/device/factory design, fog of war, sparse construction/destruction, and sensed-only simulation while also supporting single-player universes, private servers, and possibly a massive persistent MMO?

The central architectural answer developed in the chat was that *Dominium* should not be built as one enormous live scene graph. The world should instead be represented as **deterministic data**: seeds, physical parameters, procedural fields, event logs, sparse terrain and construction deltas, snapshots, state hashes, and machine/civilisation graphs. A rendering engine such as Unreal can display and interact with the local visible region, but the authoritative universe should be owned by Domino's deterministic simulation layer.

The design then evolved from abstract technical feasibility into a clearer game identity: **a robotic seed-civilisation game**. Humans in the backstory launched a robotic mothership to another star system. The mothership lands on a habitable or semi-habitable planet and manufactures robot bodies to restart civilisation. Players are not ordinary humans; they are robot agents instantiated into bodies fabricated by real machines. This premise explains respawning, the HUD, blueprints, recipe knowledge, construction assistance, and the lack of human worker NPCs.

2. Main creative direction that emerged

The strongest creative direction is that *Dominium* should be an **automation-first civilisation builder**, not a worker-labour simulation. The user explicitly reasoned that simulating worker NPCs would require expensive pathfinding, scheduling, collision, task planning, and animation. Since the inhabitants are robots, the game does not need to pretend that little people are doing the work. It can instead simulate machines, AI agents, nanobot swarms, process graphs, power grids, logistics networks, construction queues, sensor networks, and material flows.

This led to a clean premise:

- The mothership brings advanced knowledge and limited starter resources.
- Players spawn as manufactured robot bodies.
- The ship can fabricate some useful parts, but only with limited materials, throughput, heat budget, and wear tolerance.
- Players must eventually mine, refine, manufacture, build infrastructure, and create new spawn facilities.
- Cities should be relatively easy to start, but advanced industry should require serious supply

chains and infrastructure.

- Machines and factories can look detailed when nearby, but collapse into aggregate process equations when far away.
- Fog of war is not only visual; it defines what the player, faction, or client is actually allowed to know.

The result is a game where the central fantasy is not personally crafting every object by hand. The fantasy is using a seed ship's knowledge and robotic construction systems to grow an industrial civilisation from a landing site into remote settlements, cities, terraforming systems, orbital infrastructure, and megaprojects.

3. World generation and planets

A major topic was how to generate planets and star systems. The user wanted science-bounded procedural generation, not arbitrary toy sliders. Designers should set or solve for physically meaningful parameters such as star type, orbit, mass, radius, axial tilt, atmosphere, water inventory, moons, climate, geology, resource distribution, and habitability. The system should then generate plausible planets from those inputs.

The user also described a possible reverse-generation workflow. Instead of only saying "generate a planet from

these orbital parameters," a designer could say "I want a planet that looks and plays like this," and the system would search for plausible parameters that produce that outcome. This preserves creativity while keeping the output grounded in physical constraints.

Another key idea was the **field layer stack**. Procedural terrain should be overlayable with real data, designer-painted fields, mod/planet-pack layers, terraforming layers, construction layers, damage layers, and temporary blueprint previews. This matters because the same underlying data technology can support world generation, user-designed planets, saved games, in-game terraforming, and sparse construction/destruction.

A critical distinction was preserved: editor/worldgen layers can be more arbitrary, but gameplay layers should obey conservation rules. If a player cuts a tunnel, material should be removed, stored, wasted, transported, or accounted for. If they fill terrain, material and energy must come from somewhere.

4. Construction, nanobots, cut/fill, and blueprints

The user proposed using nanobots or a similar diegetic construction force. This was considered a good fit because it avoids having to simulate thousands of individual worker agents while still giving players a powerful construction interface. Since players are robots,

a blueprint HUD and nanobot construction system make sense within the fiction.

The design direction became:

1. The player previews a blueprint or terrain edit holographically.
2. The game validates feasibility, permissions, collision, and required resources.
3. Materials and energy are reserved or delivered.
4. Nanobots or construction machines execute the plan.
5. The committed result becomes deterministic sparse world state.

The important caveat is that nanobots must not become magic. They need limits: materials, power, time, heat dissipation, tool capability, local sensor knowledge, permissions, and waste handling. This is the difference between a useful construction interface and an economy-breaking cheat device.

Cut/fill terrain editing was treated as foundational. Terrain operations should create sparse deltas rather than rewriting entire planets. A cut operation might remove basalt volume, create rubble, consume energy, produce heat/dust, update structural support, and

update the local mesh. A fill operation might consume regolith, add compacted volume, update drainage, and affect future terrain stability.

5. Mothership, spawning, and progression

The mothership became the main diegetic anchor for progression. It explains why the player has recipe knowledge and advanced plans from the beginning. Players do not need to reinvent CPU design, rail gauges, calendars, machine tools, solar panels, or atmospheric chemistry; the ship already contains human knowledge.

Progression is therefore not about discovering that technology exists. Progression is about building the industrial capacity to produce it locally.

The mothership should be powerful but constrained. It can fabricate starter bodies, tools, and precision components, but it should have finite feedstock, low throughput, heat limits, power limits, fabricator wear, rare-material limits, mission reserves, and safety protocols. The proposed summary rule was: **the mothership is excellent at precision; the planet is required for scale.**

Player spawning should be physical. New players or respawns are manufactured by real machines. At first, the mothership is the spawn source. Later, players can build remote spawn labs that require materials, power,

fabricator cores, data links, local permissions, and maybe sensor coverage. This makes frontier expansion meaningful and gives multiplayer groups a concrete milestone: build the infrastructure that allows more players to appear at the edge of exploration.

Robot bodies were discussed as chassis rather than conventional RPG classes. Possible forms included biped, quadruped, spider, tracked/tank, drone, heavy industrial, aquatic, or aerial frames. This remains open design work.

6. Machines, factories, and cities

The user wanted players to design machines, devices, factories, vehicles, rockets, and other systems, but without simulating every moving part forever. The direction became a **machine graph compiler** model.

Players can build or assemble visible machines from modules such as motors, pumps, reactors, frames, wheels, tracks, arms, sensors, storage, processors, fabricators, and connectors. Domino then compiles the assembly into a functional representation: mass, power draw, heat output, throughput, input/output ports, capabilities, reliability, maintenance debt, and constraints.

This lets the game preserve creative engineering without turning every factory into an impossible physics simulation. Nearby, Unreal or another renderer can show

belts, items, sparks, arms, screens, and moving parts. Far away, the factory collapses into rates and buffers: input resources, output resources, energy use, heat, failure chance, and maintenance.

Cities should likewise be built from infrastructure rather than worker populations. A city in Dominion needs power, storage, logistics, fabrication, sensor coverage, roads/rails/pipes, spawn labs, claims/permissions, defenses, and construction queues. It does not need thousands of individual intelligent NPCs walking around. The open design problem is how to make those cities feel alive without worker NPCs. Candidate answers include drones, machine traffic, public infrastructure feedback, visible resource flows, active construction swarms, signage, lights, rail systems, and faction activity.

7. Fog of war, simulation collapse, and multiplayer

The user required fog of war and only loading/simulating what players can sense. The chat treated that as both a performance mechanism and an information-security model.

The core idea is that the game has truth, but each player or faction only receives observed or remembered state. Unknown or secret data should not be sent to clients. This matters especially if clients are allowed to help with compute. A public MMO client cannot be trusted with

hidden enemy bases, secret resource locations, or undiscovered underground features. Clients can help with visible mesh generation, local prediction, UI planning, non-secret procedural generation, or verifiable deterministic jobs, but the server must own authoritative hidden state.

Simulation collapse was also central. Local areas can run high-fidelity simulation. Distant factories, cities, wildlife, terraforming systems, and machines should collapse into aggregate deterministic state. When a player returns, the game can fast-forward or rehydrate the area into visible detail. This is how the project can support large planets and persistent worlds without simulating every rock, belt, machine, citizen, or drone at full fidelity everywhere.

On MMO scale, the important boundary was preserved: millions of players might share a logical universe if spread across regions/shards and interest-managed properly, but millions cannot all occupy one fully interactive real-time area at low latency and high FPS. This remains an architecture and verification topic, not a solved promise.

8. Unreal and Domino

The user asked whether Unreal can do this. The answer developed in the conversation was cautious. Unreal can be valuable for rendering, local world streaming, visual

tools, UI, animation, effects, and local interaction. It may help with procedural content tools and large-world rendering. But Unreal should not be assumed to be the authoritative deterministic universe engine.

The recommended architecture was:

- **Domino** owns deterministic simulation, sparse data, event logs, world generation, fog of war, construction validation, resource ledgers, machine graphs, and server authority.
- **Unreal** renders local state, handles presentation, local controls, previews, animation, audio, UI, effects, and editor tooling.
- **Dominium** defines the gameplay rules and player experience on top of those systems.

This is still an assistant recommendation rather than a user-ratified final decision. It should be verified and revisited as implementation constraints become clearer.

9. What was actually created in this conversation

A preservation prompt was uploaded as `Pasted text.txt`. That prompt requested a maximum-fidelity preservation package with a human-readable report, structured registers, a context transfer packet, a YAML-

style spec sheet, an aggregator packet, self-audit, and downloadable files.

A previous response created a full preservation package under the title **Dominium Deterministic Solar-System Architecture**. It included:

- a manifest,
- a main human-readable report,
- a context transfer packet,
- a YAML spec sheet,
- structured registers,
- an aggregator packet,
- a reader brief,
- a verification/audit file,
- a future chat bootstrap prompt,
- an in-chat reader guide,
- and a handoff ZIP.

The filesystem also contains an older or alternate preservation set named **Dominium Robotic Seed Civilisation Architecture**. Because the current user asked to bundle everything, the consolidated package includes both preservation sets, the uploaded source

prompt, this new companion report, and a complete bundle manifest.

10. What was decided, versus what remains tentative

Strong user-stated directions

- Dominion should use science-bounded procedural planets and systems.
- Designers should be able to nudge or override procedural outputs.
- Procedural fields and painted fields should be composable.
- Save-file/delta technology should also support packs, terraforming, construction, and destruction.
- The lore premise involves a robotic mothership sent by humans.
- Players spawn as robot bodies from physical machines.
- Remote spawn labs should be expansion milestones.
- Mothership fabrication should be useful but limited.
- Construction can be explained through robot HUDs, blueprints, nanobots, and AI planning.

- Worker NPC labour should be deprioritised because it is expensive and unnecessary for robotic civilisation.
- Machines/factories should be reducible to input/output process abstractions when not locally simulated.
- Fog of war and sensed-only simulation are fundamental.

Assistant recommendations needing user confirmation

- Domino should be the authoritative deterministic backend and Unreal only a renderer/client/tooling layer.
- Client compute in public MMO mode should be limited to visible/verifiable work.
- The first vertical slice should be a mothership-to-remote-spawn-lab loop.
- Machine design should use a formal compiler that produces functional process graphs.
- The project should use strict state hashes, event logs, rules versions, and replay tools.
- Official server world resets should be avoided outside alpha/seasonal testing.

Deferred decisions

- Final engine/runtime split.
 - Exact scope: one official star system, multiple systems, or larger galaxy-like structure.
 - Exact planet generation realism model.
 - Exact field/delta schema.
 - Exact mothership resource and fabrication limits.
 - Exact robot chassis set.
 - Exact nanobot capability tiers.
 - Exact machine graph compiler design.
 - Exact city liveliness model without worker NPCs.
 - Exact fog-of-war/sensor/security model.
 - Exact MMO authority and sharding model.
 - Exact first playable milestone.
-

11. What was put off for later

Several major items were deliberately left as future work because the conversation was defining the high-level architecture and game identity rather than implementing systems.

The most important deferred work is the **vertical slice specification**. The proposed slice is: one planet region, one mothership, one robot body, one ore deposit, one power source, cut/fill tools, blueprinting, nanobot construction, a mine, a refinery, a fabricator, a remote spawn lab, and fog-of-war sensors. This should become the next formal milestone document.

Other deferred work includes:

- formalising deterministic core data structures;
- specifying the planet generator parameter schema;
- designing the field-layer and sparse delta system;
- defining how saves, packs, terraforming, and construction share data technology;
- balancing mothership reserves and throughput;
- defining robot bodies and death/backup rules;
- designing nanobot construction limits;
- designing machine graph compilation and validation;
- making cities feel alive without NPC labour;
- defining resource, heat, maintenance, and logistics systems;
- verifying Unreal's current technical role;

- verifying external Solar System data sources and licensing;
 - designing MMO region authority and client compute trust boundaries.
-

12. Checks performed for this package

For this consolidated package, the filesystem was checked for previously generated files in `/mnt/data`. The current package found:

- the newer **Dominium Deterministic Solar-System Architecture** preservation set;
- the older/alternate **Dominium Robotic Seed Civilisation Architecture** preservation set;
- the uploaded source prompt `Pasted text.txt`;
- the previously generated deterministic handoff ZIP.

This companion report was then created, along with a complete bundle manifest. A new ZIP package was built containing the existing preservation files, the source prompt, this companion report, and the new manifest.

The package does not claim to contain hidden prior chats, private chain-of-thought, or project history that is not visible or present in files. It should still be manually

reviewed before being merged into a master Project Spec Book because some items are recommendations, not final user decisions.

13. Best next action

The best next action is to turn the proposed first playable slice into a formal milestone/spec document.

Recommended title:

Dominium Vertical Slice 01 - Mothership to Remote Spawn Lab

It should define:

- gameplay loop;
- player body/chassis;
- mothership spawn and fabrication rules;
- terrain field/delta requirements;
- cut/fill operations;
- nanobot construction pipeline;
- first ore/resource chain;
- power and heat constraints;
- refinery/fabricator process graph;

- remote spawn lab requirements;
- fog-of-war sensor rules;
- save/replay/state-hash tests;
- success criteria;
- non-goals;
- risks;
- and what must be prototyped before adding MMO scale.

If that loop is fun and technically stable, the project can expand outward into cities, vehicles, factories, terraforming, star-system travel, megaprojects, private universes, and MMO infrastructure.

14. The short version to remember

Dominium is becoming a deterministic robotic seed-civilisation game. The world should be stored as procedural fields, sparse deltas, ledgers, graphs, event logs, and state hashes. Players are robot bodies manufactured by a mothership and later by remote spawn labs. The mothership gives knowledge but not unlimited industrial scale. Construction is handled through blueprints, nanobots, energy, materials, heat, and time. Cities and factories should be automation-

first, not worker-NPC simulations. Machines should compile into process graphs. Fog of war controls what players and clients know. Unreal may be useful as a renderer/client, but Domino should own deterministic truth unless a later verified architecture says otherwise. The next step is the mothership-to-remote-spawn-lab vertical slice.

7.10 **Dominium Deterministic Solar System Architecture**

Accompanying Human Readable Detailed Summary Report

Source: docs/archive/conversations/Dominium_Complete_Conversation/deterministic_solar_system_handoff_file/Dominium_Deterministic_Solar_System_Architecture__10_accompanying_human_readable_detailed_summary_report.md

Report type: human_readable_detailed_summary

Size: 29184 bytes

7.10.1 Accompanying Human-Readable Detailed Summary and Report

Chat label: Dominion Deterministic Solar-System Architecture

Date anchor: 2026-05-27 Australia/Melbourne

Source scope: This conversation only, plus the generated preservation files and the uploaded preservation prompt present in this chat.

Epistemic policy: User-stated content is treated as **FACT** where clearly stated. Assistant architecture/design recommendations are treated as **INFERENCE** unless the user explicitly accepted them. External engine/data claims remain **UNCERTAIN** / **UNVERIFIED** until checked against current sources.

1. Purpose of This Companion Report

This companion report is meant to sit beside the larger preservation package already created in this chat. The earlier package contains the formal preservation report, context transfer packet, spec sheet, registers, aggregator packet, reader brief, verification/audit file, bootstrap prompt, and in-chat reader. This new file adds a more directly readable narrative of the whole conversation, including the design discussion, what was decided or deferred, what artifacts were created, and what a future assistant or project aggregator should not lose.

The practical purpose is simple: a future reader should be able to open this one file and quickly understand the substance of the conversation without needing to reread the whole transcript. It is not a replacement for the structured package; it is the human companion to it.

2. Coverage and Reliability Check

Access and scope The conversation available for this report includes:

1. The user's initial architecture question about a deterministic solar-system-scale Dominion/Domino game.
2. Assistant responses proposing a deterministic sparse simulation architecture and warning about Unreal/MMO limits.
3. The user's longer design monologue defining the robotic mothership, robot players, science-bounded planet generation, procedural field overlays, nanobot construction, automation-first design, and process-graph machines.
4. The assistant synthesis that framed the project as a robotic seed-civilisation game.
5. The uploaded preservation prompt that required a maximum-fidelity human-readable report, structured registers, spec sheet, aggregator packet, self-audit, downloadable files, and ZIP package.
6. The prior generated preservation package files and ZIP.

7. The current request to create this accompanying detailed report and bundle everything into one complete ZIP.

Reliability limits This report is reliable for the visible conversation and generated files. It should not be treated as a full history of the wider **Dominium** project. Any information not visible in this chat, not present in the generated files, or not clearly marked as project-level context should be treated as outside scope.

The strongest uncertainty is that many assistant architecture proposals were not explicitly accepted by the user as final decisions. They are useful recommendations and should be preserved, but they should not be promoted automatically into binding requirements.

Engine-specific claims about Unreal, MMO scale, NASA/JPL data, SPICE, and current software capabilities were part of earlier discussion but remain verification items. This companion report does not newly verify external facts.

3. One-Page Summary of the Whole Conversation

The conversation began with a very large technical/game-design question: how can **Domino engine** and **Dominium game** support a deterministic, procedural,

full-scale solar-system or universe-scale game where players can build civilisations, terraform planets, cut and fill terrain, build megaprojects, design machines and factories, and play either single-player or in an MMO-style persistent universe? The user also wanted fog of war, sparse simulation, sensed-only loading, and possibly client-shared compute, while asking whether Unreal could do it at high FPS, low latency, and very large player counts.

The first major conclusion was architectural. The assistant recommended that Dominium should not be built as one giant live Unreal scene. Instead, **Domino should be a deterministic simulation/data engine**, and Unreal or another graphics engine should act as a renderer/client. The universe should be stored as seeds, parameters, event logs, field layers, sparse deltas, state hashes, and snapshots. Only what players can sense should be expanded into high-fidelity simulation or graphics. Everything else should collapse into aggregate state, delayed events, or deterministic fast-forward. This was an assistant recommendation, not a user-confirmed final decision, but it fits the user's stated requirements.

The user then refined the project into a more concrete game. Dominium should generate star systems and planets procedurally, with realistic scientific bounds rather than arbitrary sliders. Designers should set meaningful parameters such as orbits, planetary

properties, atmosphere, climate, resources, and geography, then nudge or override results. The user also suggested a reverse-generation workflow: define the kind of planet desired, then let the system solve plausible parameters to make it fit.

The world data model became a central theme. The user wanted procedural fields that can be overlaid with painted fields, designer overrides, save-file changes, planet packs, terraforming changes, and player construction. The assistant formalised this as a field-layer and sparse-delta system: base procedural planet plus real data overlays, designer layers, mod layers, construction layers, damage layers, and gameplay terraforming layers. A key distinction is that editor/designer layers can be unconstrained, while gameplay terraforming and construction should conserve material, energy, time, and capability.

The strongest creative premise was the **robotic seed-civilisation** idea. Humans in the past built a robotic mothership, sent it to another solar system, and tasked it with landing on a habitable planet and manufacturing robots to restart civilisation. Players are not normal human colonists. They are robot bodies fabricated by physical machines. Their software identity can be instantiated into different chassis, such as bipeds, quadrupeds, spiders, tracked/tank forms, drones, or industrial frames. New players or respawns must come

from physical spawn machines, and frontier expansion requires building remote labs/fabricators that can manufacture bodies using local resources and power.

The mothership solves several design problems at once. It explains why players have a HUD, blueprints, recipes, construction guidance, and advanced technical knowledge. Players do not need to reinvent CPUs, railway gauges, smelting, computing, or calendars because the mothership already contains human knowledge. However, the mothership cannot simply build everything forever. It should have finite feedstock, limited throughput, heat constraints, fabricator wear, mission reserves, and scale limits. The core progression is not "discover how technology works"; it is "build the industrial base capable of manufacturing and maintaining that technology locally."

A major design pivot was away from worker NPC labour. The user explicitly recognised that simulating workers requires pathfinding, scheduling, collision, animation, and a large amount of compute. Since the civilisation is robotic, work can instead be done by nanobots, AI controllers, drones, machine graphs, logistics systems, and abstract process diagrams. This is one of the most important conclusions of the conversation: **Dominium should not default to worker-NPC city simulation.** It should simulate materials, energy, machines, logistics, sensors, construction jobs,

permissions, infrastructure, and automated systems. Construction should be blueprint-based and diegetic. Robots can project plans into their HUD, issue construction jobs, reserve resources, and command nanobot or machine swarms to execute cut/fill/build operations. Nanobots are useful because they provide a convenient invisible construction interface, but they must not become magic. Every committed operation should require material, energy, time, capability, heat handling, sensor coverage, and permission.

Factories and machines should also be multi-representation systems. Nearby, they can look detailed and physical. Far away, they should collapse into process graphs and equations: inputs, outputs, power draw, heat, throughput, buffers, maintenance debt, and failure probability. This supports the user's goal of allowing players to design machines, vehicles, rockets, factories, and devices without requiring the engine to simulate every bolt, belt item, gear tooth, or worker everywhere.

The conversation ended by generating a full preservation package. The uploaded prompt required a human-readable report first, then structured registers, a context transfer packet, spec sheet, aggregator packet, self-audit, downloadable files, and ZIP. A set of preservation files was created. This current companion report and complete bundle extend that work by adding a more direct narrative summary and bundling the package into

one new ZIP.

4. What Was Discussed in Detail

4.1 Deterministic simulation and sparse world representation The user wanted determinism at a very large scale. The assistant recommended fixed simulation ticks, deterministic commands, event sourcing, snapshots, hashes, stable procedural seeds, sparse terrain deltas, and simulation LOD. This matters because the game cannot store or simulate whole planets at high fidelity. The world must be mostly generated from reproducible functions, with only player changes and important dynamic state persisted.

The key architecture phrase is:

Store worlds as fields, graphs, ledgers, and sparse deltas; render them as terrain, cities, robots, machines, and megaprojects only when players can sense them.

4.2 Full-scale systems, planets, and procedural science The user wanted planets generated from realistic parameters rather than arbitrary artistic sliders. The assistant proposed forward generation and reverse generation. Forward generation starts with star/planet/orbit/atmosphere/resource parameters and generates

terrain, climate, oceans, caves, ores, and habitability. Reverse generation starts with desired outcomes and solves plausible parameters.

This is still unresolved at implementation level. The project needs a planet/system schema, parameter bounds, plausibility scoring, and a decision about how hard-science versus sci-fi the generator should be.

4.3 Procedural fields, painted overrides, packs, saves, and terraforming The user wanted the same technology to support worldgen, save files, user packs, designer overrides, and in-game terraforming. This is a strong direction because it avoids building separate incompatible systems.

The proposed model is:

- base procedural field;
- real-data layer if applicable;
- designer override layer;
- mod/planet-pack layer;
- construction layer;
- terraforming layer;
- damage/destruction layer;
- temporary blueprint preview layer.

The unresolved detail is the exact representation: voxel chunks, signed-distance functions, heightfields, material fields, climate grids, graph overlays, or hybrid systems.

4.4 Robotic mothership and seed-civilisation

premise The user defined the fiction: a human-built robotic mothership travels to another solar system, lands on a habitable planet, and manufactures robots to restart civilisation. Later human eggs or biological payloads may arrive, but that remains a lore possibility rather than a final decision.

The premise is unusually useful because it explains many UI and gameplay abstractions:

- HUDs exist because players are robots.
- Blueprints exist because the mothership has design databases.
- Quest markers exist because the ship AI can plan tasks.
- Respawn exists because robot bodies are manufactured.
- Progression exists because knowledge is available but manufacturing capacity is not.
- Automation exists because the civilisation is robotic.

4.5 Robot bodies and physical spawn economy

Players should spawn from real machines, not abstract menu points. The starting mothership can fabricate early robot bodies. Later, players build remote spawn labs that require power, materials, fabricator components, communications, coolant, and permissions.

This creates a good exploration loop: expansion to new regions is not just walking there; it is building the infrastructure that allows new bodies to be made there.

Open questions remain around body templates, death/backup rules, communication range, respawn cost, body upgrading, and whether player identity is software, hardware, or both.

4.6 Nanobots, blueprints, cut/fill, and

construction The user liked nanobots or an invisible construction force because it is computationally simpler than simulating labourers. The assistant recommended treating nanobots as construction actuators rather than magic. They can cut, fill, weld, print, assemble, smooth, reinforce, repair, and survey, but committed operations should consume resources and update sparse deltas.

A likely construction pipeline is:

1. Preview as a hologram.
2. Convert preview to a formal plan.

3. Validate terrain, permissions, collisions, and requirements.
4. Reserve resources.
5. Stage materials.
6. Execute with nanobots/machines.
7. Commit deterministic deltas.
8. Operate the built object as part of the simulation.

4.7 Automation instead of worker NPCs This is one of the clearest user-stated directions. The user considered NPC workers but rejected them as computationally expensive and conceptually unnecessary. Future assistants should not keep proposing worker labour as the main solution unless the user explicitly reopens it.

Instead, intelligent activity should appear as ship AI, construction swarms, machine controllers, city operating systems, logistics schedulers, drone hives, and process graphs.

This does create a risk: cities could feel empty. The solution is not worker NPCs by default; it is visible drones, traffic, signals, industrial movement, lights, alarms, logistics flows, public infrastructure, faction interfaces, and active machine systems.

4.8 Machines, vehicles, factories, and process graph compilation The user wants players to create machines and vehicles, but also wants the engine to summarise a whole assembly as one functional object or process equation where possible. The assistant proposed a machine compiler: player-built assemblies are validated and reduced to capabilities, input/output ports, throughput, energy use, heat, mass, reliability, constraints, and maintenance debt.

This is essential for scale. If every remote factory must simulate every belt item and moving arm, the game cannot grow. If each remote factory can collapse into a process graph, large cities and planets become plausible.

4.9 Fog of war and sensed-only simulation The user wanted fog of war and only simulating/loading what players can sense. The assistant emphasised that fog of war is not just visual; it is an information security model. Hidden enemy bases, undiscovered resources, caves, and secret construction should not be sent to clients in MMO mode.

This directly affects client-shared compute. Clients may generate visible meshes, predict local actions, and help with non-secret verifiable jobs, but they should not compute hidden authoritative truth in public MMO mode.

4.10 Unreal, Raylib, and engine roles The assistant recommended separating Domino simulation from rendering. Unreal can be useful for high-end visuals, client interaction, UI, animation, tooling, and local streaming. Raylib or a custom renderer may be useful for early deterministic prototypes. However, Unreal should not be assumed to solve deterministic planetary MMO simulation by itself.

This is not yet a user-confirmed final architecture. It remains a recommendation and belongs in the verification/prototype queue.

4.11 Single-player, private servers, and official MMO The user wants multiple modes: single-player instantiated universes, private servers, and possibly an official canonical MMO server. The assistant proposed using the same core rules but different trust and scaling assumptions.

Single-player can allow pausing, local fast-forward, relaxed trust, and custom universes. Private servers can allow mods and custom systems. Official MMO mode needs server authority, anti-cheat, fog-of-war filtering, persistent event logs, and strict resource accounting.

5. Decisions, Design Directions, and Their Status

Strong user-stated design directions These should be treated as high-confidence carry-forward items:

- **Dominium should support deterministic, procedural, sparse large-scale worlds.**
- **Planets should be generated from realistic/science-bounded parameters, not arbitrary sliders alone.**
- **Procedural fields should be overlayable with painted/designer/player layers.**
- **The premise is a robotic mothership that manufactures robots to rebuild civilisation.**
- **Players are robot bodies/chassis manufactured by physical spawn machines.**
- **Frontier expansion should involve building new spawn labs/fabricators.**
- **The mothership gives recipes and knowledge but has finite resources and limited throughput.**
- **Nanobot/automation construction is preferred over labour simulation.**

- Worker NPC labour should be deprioritised because pathfinding and labour simulation are expensive.
- Machines/factories should be collapsible into input/output/process equations.
- Fog of war and sensed-only simulation are core requirements.

Assistant recommendations not automatically final These are useful but should not be treated as confirmed user decisions unless accepted later:

- Domino should be the authoritative deterministic simulation/data core.
- Unreal should be a renderer/client/tooling layer rather than the whole authoritative engine.
- Clients in MMO mode should not be trusted with hidden state.
- The first vertical slice should be mothership-to-remote-spawn-lab.
- Machine assemblies should be handled by a formal compiler.
- External datasets such as JPL/SPICE/IAU-style data should be considered for real solar-system accuracy.

Decisions deferred The chat did not finalise:

- exact engine stack;
- programming language;
- deterministic math model;
- exact terrain representation;
- planet generator equations;
- field-layer storage format;
- exact robot chassis list;
- death/backup/respawn rules;
- mothership resource quantities;
- nanobot tier limits;
- factory/machine compiler format;
- official server architecture;
- exact MMO population and shard model;
- PvP, governance, claims, and permissions;
- how alive cities should look without worker NPCs;
- modding/package format;
- external data licensing.

6. What Was Done in This Chat

The conversation produced both design substance and artifacts.

Design work completed

- Defined the project as a deterministic sparse simulation problem.
- Established the robotic seed-civilisation premise.
- Identified the mothership as lore, recipe source, spawn source, and early bottleneck.
- Clarified that automation should replace worker NPC labour.
- Proposed field-layer and sparse-delta representation.
- Proposed nanobot blueprint construction.
- Proposed process-graph factories and machine compilation.
- Identified fog-of-war as both gameplay and data-security architecture.
- Identified a likely first vertical slice.

Preservation work completed A full preservation package was generated before this companion report. It contained:

- manifest;
- human-readable report;
- context transfer packet;
- YAML spec sheet;
- structured registers;
- aggregator packet;
- reader brief;
- verification and audit file;
- future-chat bootstrap prompt;
- in-chat reader;
- handoff ZIP.

This current task completed This current task adds:

- a new accompanying detailed human-readable summary/report;
- a complete bundle manifest;
- a file inventory with hashes;

- a new ZIP containing the existing preservation files, the new companion report, the new manifest, the inventory, the original uploaded prompt for provenance, and the earlier handoff ZIP.
-

7. What Was Put Off for Later

The most important deferred work is implementation specification. The chat established the concept and architecture direction, but not the formal engine spec.

Highest-priority deferred work

1. **Formal first vertical slice:** one planet region, mothership, one or more robot bodies, survey, ore, power, cut/fill, blueprinting, nanobot construction, mine, refinery, fabricator, remote spawn lab, and fog-of-war sensors.
2. **Deterministic core spec:** ticks, commands, event logs, snapshots, state hashes, reproducibility rules, math model, save format, and replay tooling.
3. **Planet generator schema:** physical parameters, plausibility scoring, reverse generation, designer overrides, resource maps, climate, terrain, and data layers.

4. **Field/delta model:** how procedural terrain, painted overlays, gameplay edits, construction, destruction, terraforming, and saves share one representation.
5. **Mothership economy:** reserves, throughput, heat, wear, mission reserve, spawn costs, starter body costs, fabrication limits, and recovery/failure modes.
6. **Robot chassis system:** body categories, tradeoffs, carrying capacity, locomotion, sensors, manipulation, power, and upgrade paths.
7. **Nanobot construction rules:** tiers, energy, mass, heat, speed, permissions, validation, and commit mechanics.
8. **Machine graph compiler:** how player-made assemblies become functional processes and how impossible/laggy designs are rejected.
9. **Fog-of-war model:** what is known, remembered, inferred, hidden, sensed, shared, scanned, or unknown.
10. **MMO/private/single-player authority rules:** trust boundaries, client compute, region servers, official server rules, and shard/handoff model.

Lower-priority but still important deferred work

- Faction/governance systems.
 - PvP and griefing protections.
 - City visual liveliness without worker NPCs.
 - Wildlife and non-intelligent mobs.
 - Terraforming ecology/climate depth.
 - Megaproject structural simulation.
 - UI/UX for blueprints and AI planning.
 - Mod/planet-pack packaging.
 - External dataset licensing and update rules.
-

8. Risks and What Future Work Could Get Wrong

Risk 1 - Treating brainstorming as final decisions

The user brainstormed many ideas quickly. Some are strong directions, but not every detail is final. For example, later human eggs, official server resets, and exact robot forms remain open. Future assistants should preserve status labels.

Risk 2 - Reintroducing worker NPC labour as the default This would contradict one of the clearest pivots in the chat. The user explicitly preferred automation because labour/pathfinding is expensive. Worker-like agents might exist locally or cosmetically later, but not as the core construction/economy model unless reopened.

Risk 3 - Letting nanobots become magic Nanobots are the construction interface, not free matter. They must require energy, material, time, heat handling, tool capability, and permissions.

Risk 4 - Letting the mothership trivialise progression The mothership provides knowledge and precision, but the planet provides scale. If the mothership can fabricate everything cheaply and instantly, mines, factories, cities, and logistics stop mattering.

Risk 5 - Trusting clients with hidden MMO truth Client-shared compute is tempting, but hidden data cannot be sent to clients in a public MMO. Fog-of-war security should be treated as an architecture constraint.

Risk 6 - Building too much before the core loop is proven The project should not start with the whole

solar system, MMO, terraforming, and megaprojects. The immediate proof should be a small deterministic loop: spawn, survey, build, mine, refine, fabricate, and create a new spawn point.

Risk 7 - Using Unreal as if it solves the backend
Unreal may be useful, but the core deterministic universe needs its own data/simulation architecture. Current engine claims should be verified, not assumed.

9. File and Artifact Inventory Summary

The prior package files were checked on disk before creating this companion bundle. The key generated files are:

- `00_manifest.md` - inventory and caveats for the prior package.
- `01_human_readable_report.md` - the main complete preservation report.
- `02_context_transfer_packet.md` - future-chat handoff.
- `03_spec_sheet.yaml` - machine-readable spec-prep sheet.
- `04_registers.md` - workstreams, decisions, tasks, constraints, risks, questions, artifacts.

- `05_aggregator_packet.md` - compact merge packet for a master aggregator.
- `06_reader_brief.md` - shorter human brief.
- `07_verification_and_audit.md` - self-audit and verification queue.
- `08_future_chat_bootstrap_prompt.md` - prompt for continuing in a new chat.
- `09_in_chat_reader.md` - guide for inspecting the package.
- `handoff_package.zip` - previous ZIP of the above files.
- `10_accompanying_human_readable_detailed_summary_report.md` - this new companion report.
- `11_complete_bundle_manifest.md` - manifest for the new complete bundle.
- `12_file_inventory.json` - file sizes and SHA-256 hashes for verification.
- `complete_bundle_with_accompanying_report.zip` - the new single ZIP bundle.

The uploaded `Pasted text.txt` prompt is also preserved in the new complete ZIP as provenance for the preservation requirements.

10. Best Next Action

The best next action is to turn the proposed first vertical slice into a formal milestone specification.

A useful title would be:

Vertical Slice 01 - Mothership to Remote Spawn Lab

It should define:

- player starting state;
- starting mothership capabilities and limits;
- one robot chassis;
- one terrain region;
- one ore/resource type;
- one power system;
- one cut/fill operation;
- one blueprint construction flow;
- one nanobot construction job;
- one mine/refinery/fabricator chain;
- one remote spawn lab;

- one fog-of-war sensor model;
- deterministic save/load/replay tests;
- success criteria.

This would convert the conceptual design into an implementable milestone without prematurely committing to the full solar-system MMO.

11. Human Memory Section

The most important things to remember are:

- **Dominium is now best understood as a robotic seed-civilisation game.** The player fantasy is not just survival or factory building; it is rebuilding civilisation as machines after a human-launched seed ship lands on a new world.
- **Domino should probably be a deterministic sparse simulation/data engine.** Rendering and high-fidelity local visuals should be secondary to authoritative world truth.
- **The world should be generated and stored as fields, graphs, ledgers, sparse deltas, and event logs.** Full planets should not be stored as dense mutable maps.

- **The mothership explains knowledge but not scale.** It knows how to make things and can fabricate precise starter components, but players must build industry for mass production.
 - **Automation replaces worker NPC labour.** This is a key design pivot and should not be lost.
 - **Nanobots are construction actuators, not magic.** They need materials, energy, time, heat budgets, permissions, and capability tiers.
 - **Machines and factories should collapse into process graphs.** Local visuals can be detailed, but distant simulation should be aggregate.
 - **Fog of war is an information security model.** The client should only know what the player can sense, especially in MMO mode.
 - **The first real milestone should be small.** Prove spawn, survey, build, mine, refine, fabricate, and create a new spawn point before scaling to planets, cities, and MMO systems.
-

12. Recommended Follow-Up Questions

- Turn the mothership-to-remote-spawn-lab loop into a formal vertical slice spec.

- Draft the deterministic Domino core data model.
 - Define the field-layer and sparse-delta representation.
 - Design the first three robot chassis.
 - Design the mothership reserve and fabrication economy.
 - Define nanobot construction tiers and limitations.
 - Draft the machine graph compiler.
 - Define fog-of-war knowledge states and sensor rules.
 - Specify what Unreal should and should not own.
 - Identify which assistant recommendations should become formal requirements.
-

13. Final Companion Report Status

- Companion report created: yes.
- Existing preservation package checked: yes.
- Complete bundle manifest created: yes.
- File inventory with hashes created: yes.
- Complete ZIP created: yes.

- Safe for aggregation: with caveats.
- Extraction confidence: 4/5.
- Main caveats: hidden prior project history is not included; assistant recommendations are not automatically final decisions; external engine/data claims require verification.

7.11 Dominion Conversation Complete Companion Report

Source: docs/archive/conversations/Dominium_Complete_Conversation/dominium_related_file/Dominium_Conversation_Complete_Companion_Report.md

Report type: companion_report

Size: 22109 bytes

7.11.1 Accompanying Human-Readable Detailed Summary and Report - Dominion Conversation Complete Preservation Bundle

0. Purpose of this companion report

This companion report sits beside the larger preservation package already created for the Dominion/Domino conversation. Its purpose is to give a clear human-

readable explanation of the entire conversation, what was discussed, what was decided or recommended, what was deferred, what artifacts were produced, and how the bundled files should be used later.

This report is based on the visible conversation, the uploaded preservation prompt, and the generated files present in the sandbox at packaging time. It does **not** claim access to hidden prior chats or external project records. Where something came from the user’s own statements in this chat, it is treated as stronger than assistant recommendations. Where something was suggested by the assistant and not explicitly accepted by the user, it remains a recommendation or inference, not a final product decision.

Date anchor: 2026-05-27 Australia/Melbourne

Package creation timestamp: 2026-05-27T11:04:03

Source scope: this chat and files generated from this chat.

1. One-paragraph orientation

This conversation developed the Dominion/Domino project into a deterministic, sparse, procedural, robotic seed-civilisation game architecture. The user began with a maximum-scale technical ambition: full-scale procedural solar systems and planets, realistic planet generation, terraforming, cut/fill terrain, megaprojects,

player-built machines and factories, fog of war, sensed-only simulation, sparse construction/destruction, MMO-style persistence, single-player multi-universe play, private/custom servers, and possible Unreal integration. The conversation then evolved into a much more specific game premise: humanity sends a robotic mothership to another solar system; it manufactures robot bodies to restart civilisation; players spawn physically from fabricators; expansion requires building remote spawn labs; automation replaces worker NPC labour; nanobots and blueprints make construction convenient; machines and factories compile into process graphs; and the authoritative universe is stored as deterministic data, fields, graphs, ledgers, event logs, and sparse deltas rather than as a giant always-live scene.

2. What the conversation was mainly about

The central problem was how to make an enormous simulation feel real without actually simulating everything everywhere at maximum fidelity. The user wanted full planets, player-made civilisations, construction, terraforming, machines, factories, fog of war, and possibly millions of players across a persistent universe. The core technical conclusion was that this cannot be treated as a normal open-world level full of live actors. Instead, *Dominium* needs a deterministic simulation core.

The most important architectural idea was:

Store the universe as deterministic data.

Render only the local sensed world.

Expand detail only when players can perceive or affect it.

Collapse everything else into sparse deltas, aggregate graphs, ledgers, and scheduled simulation.

This means Domino, the engine/simulation layer, should own world truth. Unreal, Raylib, or another runtime can be used as renderer, client, visualisation layer, or tooling layer, but should not be the source of authoritative planetary/MMO truth. That was an assistant recommendation, not a confirmed final user decision, but it follows from the user's stated goals around determinism, fog of war, sparse simulation, and MMO feasibility.

3. Main creative/gameplay premise

The user's most concrete design contribution was the robotic mothership premise. In the lore, humans sent a robotic seed ship to another solar system. It finds or lands on a suitable planet and begins manufacturing robot bodies that can rebuild civilisation. The user also floated mystery/lore possibilities around later human payloads, embryos, or future ships, but those remain optional and unresolved.

This premise solves multiple design problems at once:

- Players can respawn because robot bodies are physically manufactured.
- New players can join through real spawn machines rather than abstract menus.
- The HUD is diegetic because players are robots.
- Blueprint overlays are diegetic because robot vision can project construction plans.
- Quest markers can be explained as mothership AI planning or construction guidance.
- Advanced recipes are known because the mothership brought human technical knowledge.
- The game does not need simulated human workers, because the civilisation is robotic.

The user described robot bodies such as bipeds, quadrupeds, spider-like robots, tank-like bodies, and other chassis. The specific chassis list is not final. The important principle is that player identity can be separated from physical body hardware. A player may be software/mind-state/agent identity inhabiting a manufactured robot body.

4. World generation and planet design

The user wanted procedural star systems and planets that are not arbitrary slider toys. Parameters should be

bounded by plausible science: orbits, star properties, planetary mass/radius, atmosphere, climate, resources, terrain, and related physical constraints. The designer should be able to set high-level physical parameters and let the system generate believable planets.

The user also wanted a reverse or guided workflow: instead of only entering orbital/physical inputs and accepting whatever comes out, a designer could specify a desired planet outcome and ask the generator to solve for plausible parameters that produce it. This is not yet a final implementation plan, but it is an important design idea.

A major data-model idea was that planets should be built from fields and layers rather than handcrafted maps. Procedural fields can be overlaid with real data, designer overrides, painted fields, gameplay terraforming, construction deltas, and damage. This same technology can support:

- procedural generation;
- save files;
- player terraforming;
- cut/fill terrain modification;
- planet packs;
- designer-authored worlds;

- modded overlays;
- sparse persistence.

The key caveat is that not all layers follow the same rules. Editor/worldgen layers can create authored terrain freely. Gameplay terraforming layers should conserve mass, material, energy, time, and heat so construction and destruction remain meaningful.

5. Mothership economy and progression

The mothership should not trivialise the game. The user explicitly discussed the problem: if the mothership has all human knowledge and advanced nanotechnology, why does it not simply synthesize CPUs, machines, infrastructure, and cities for the players?

The emerging answer was that the mothership has knowledge and precision, but not unlimited scale. Its fabrication should be limited by:

- finite feedstock;
- finite stored materials;
- low throughput;
- energy requirements;
- heat limits;

- wear and maintenance;
- reserve stockpiles;
- rare material shortages;
- scale limits;
- mission or safety protocols.

So the mothership can start the game, spawn early robots, fabricate starter tools, and provide recipes, but players must build local industry to scale. This creates the core progression loop: survey, mine, refine, power, fabricate, build, automate, expand, and eventually support remote spawn labs, cities, advanced industry, terraforming, and megaprojects.

A useful preserved design principle is:

The mothership is excellent at knowledge and precision.

The planet is required for scale.

6. Construction, nanobots, and cut/fill

The user suggested using nanobots or an invisible construction force because it is computationally efficient and fits robot lore. Instead of simulating thousands of workers physically digging and building, the player can use a HUD/blueprint interface and command nanobots or automated systems to alter terrain and construct structures.

The assistant formalised this into a staged construction model:

1. preview a hologram/blueprint;
2. validate the plan;
3. reserve resources;
4. stage materials;
5. execute cut/fill/build operations;
6. commit deterministic sparse deltas;
7. operate the resulting structure or machine.

The key constraint is that nanobots must not become free magic. They should require material, energy, time, heat handling, tooling capability, permissions, sensor coverage, and possibly maintenance. This preserves challenge and gives engineering meaning to the construction game.

Cut/fill terrain is central. Terrain should not be stored as one gigantic dense mesh or heightmap. It should be a base procedural field plus sparse operation deltas: subtract volume, add fill, reinforce material, smooth, excavate, compact, or construct. This enables terraforming and digging while keeping storage and simulation manageable.

7. Automation instead of worker NPCs

This was one of the strongest design shifts in the conversation. The user considered worker NPCs but rejected/deprioritised them because labour simulation implies pathfinding, scheduling, collision, animation, traffic, housing, and many other expensive systems. The user's view was that if the computer is doing the work, the game should not pretend it is a society of people doing the work.

The preserved direction is that *Dominium* should be an automation-first civilisation game. Cities are built by robotic systems, nanobots, AI controllers, drone hives, process graphs, and factories. Intelligent agents can exist as software or city-control systems rather than human-like NPC workers.

This does not mean cities must feel dead. The design challenge is to make them feel alive through:

- visible machine activity;
- drones and construction swarms;
- power grids;
- logistics networks;
- sensor towers;
- factories;

- traffic abstractions;
- UI overlays;
- city operating systems;
- player/faction activity;
- environmental changes.

The important thing to avoid in future chats is accidentally reintroducing worker NPC labour as the default city simulation model.

8. Machines, factories, and process graphs

The user wants players to build machines, vehicles, rockets, factories, and devices, but not to force the engine to simulate every bolt, belt, gear, and item everywhere. The proposed solution is to let machines have visual/interactive detail locally, then compile their actual function into a graph or equation when distant.

For example, a remote factory should be stored as something like:

Consumes: iron ore, carbon, electricity

Produces: steel, slag, waste heat

State: efficiency, buffers, maintenance debt, failure risk

A nearby factory can render belts, robots, machines, lights, smoke, items, arms, and interfaces. A distant

factory should collapse into input/output rates, storage buffers, energy demand, heat output, and reliability.

For player-designed machines, the project likely needs a machine graph compiler. Players assemble components, but Domino validates and reduces the assembly into capabilities and constraints. This protects the server from lag machines, exploits, and impossible designs while preserving creativity.

This system is important enough to become a major future workstream.

9. Fog of war and sensed-only simulation

The user required fog of war and sensed-only loading/simulation. The assistant expanded this into both a simulation rule and a security rule.

The game should distinguish:

- what the player currently senses;
- what the player remembers from past exploration;
- what the player can infer;
- what remains unknown;
- what is secret and must not be sent to the client.

This is crucial for MMO mode. If a client receives hidden enemy bases, resource locations, undiscovered

caves, or secret construction data, players can extract it. Public MMO clients should only receive state their body, faction, sensors, satellites, or map-sharing network can legitimately know.

This also connects to compute scaling. Clients can help with rendering, local prediction, visible terrain generation, previews, and other non-secret jobs. They should not be trusted with authoritative hidden truth, combat-critical state, resource maps, economy settlement, or anti-cheat-critical simulation.

10. MMO, single-player, private servers, and scale

The user wants the design to support multiple play modes:

- single-player across instantiated universes;
- private/custom servers;
- possibly one official canonical MMO server;
- perhaps many shards or region authorities;
- client-shared compute where possible.

The main feasibility conclusion was that millions of players can share a logical universe if spread across regions, shards, planets, servers, and interest-

management zones. Millions cannot all be in one low-latency, fully simulated interaction bubble. That claim was an assistant feasibility boundary and should be verified against current technology before formal use, but it is a practical design constraint.

The assistant recommended distributed authority: universe coordinator, region servers, cell servers, persistence/event logs, interest gateways, validation systems, and client prediction. This remains architectural recommendation, not a fully accepted final spec.

11. Unreal's role

The user specifically asked whether Unreal can do this. The assistant's answer was that Unreal can help with rendering, local world streaming, tooling, animation, UI, visual effects, and maybe local gameplay, but it should not own the authoritative deterministic universe. That is because the project needs deterministic data, sparse deltas, planet-scale coordinates, fog-of-war truth filtering, MMO authority, replay, and collapsed simulation.

This is a recommendation rather than a confirmed decision. It also depends on current Unreal capabilities, which were flagged for verification. Future work should not rely on stale claims about Unreal, networking, Large

World Coordinates, World Partition, MassEntity, Iris, Nanite, or runtime geometry without checking current documentation and prototyping.

12. What was actually done in this chat

The conversation produced several things:

1. A high-level architecture argument for deterministic sparse simulation.
2. A game-identity synthesis around robotic seed civilisation.
3. A detailed preservation package with reports, registers, spec sheet, context transfer packet, aggregator packet, self-audit, and reader guide.
4. A set of downloadable files under the `Dominium_Deterministic_Solar_System_Architecture_..` naming scheme.
5. An earlier related set of files under the `Dominium_Robotic_Seed_Civilisation_Architecture_..` naming scheme, also present in the sandbox and included here for preservation.
6. This companion report and consolidated archive.

The uploaded `Pasted text.txt` file contained the detailed preservation prompt. It required a human-

readable report first, structured registers, a spec sheet, aggregator packet, self-audit, downloadable files, ZIP package, and final in-chat reader. It also warned not to invent facts, not to turn brainstorming into decisions, and not to claim files were created unless they actually were.

13. Decisions and directions to preserve

The strongest user-stated directions are:

- Dominion should use procedural star-system and planet generation bounded by realistic science.
- Planets should be adjustable through nudges/overrides, not primarily manual painting.
- Procedural fields and painted fields should be overlayable.
- The same data/delta technology should support saves, packs, terraforming, and construction.
- The core lore is a robotic mothership manufacturing robot bodies to restart civilisation.
- Players spawn physically from fabrication machines.
- Remote spawn labs should be buildable expansion milestones.

- The mothership provides knowledge/recipes but should be constrained by resources and throughput.
- Nanobot/blueprint construction is desirable because it is diegetic and computationally efficient.
- Worker NPC labour should be avoided or deprioritised because it is expensive to compute.
- Machines/factories should be collapsible into process equations or graphs.

The strongest assistant recommendations are:

- Domino should be the deterministic authoritative simulation/data core.
- Unreal should be renderer/client/tooling rather than authoritative universe truth.
- Public MMO clients should not compute hidden/secret state.
- The first milestone should be a small vertical slice around the mothership-to-remote-spawn-lab loop.

Those assistant recommendations should not be treated as final decisions unless the user later confirms them.

14. What was put off for later

The conversation intentionally left many items unresolved. The most important deferred work is:

- exact launch scope: one star system, many systems, or galaxy-like universe;
- exact planet-generation schema;
- exact realism/plausibility scoring rules;
- field-layer/delta data format;
- save-file format;
- terrain chunking and sparse edit representation;
- mothership resource amounts, throughput, heat, reserves, and failure/recovery rules;
- robot chassis list and stats;
- death/backup/respawn rules;
- nanobot capabilities by tech tier;
- machine graph compiler design;
- factory aggregation rules;
- city progression and how cities feel alive without worker NPCs;
- fog-of-war and sensor model;

- permissions, claims, griefing protection, and governance;
- single-player versus MMO balance differences;
- client-shared compute trust boundaries;
- Unreal/Raylib/custom engine final split;
- verification of current engine and data-source assumptions;
- the first formal vertical-slice milestone document.

The most important next deliverable remains the vertical slice spec: one planet region, mothership, robot body, survey tool, ore deposit, power source, cut/fill tool, blueprint system, nanobot construction, mine, refinery, fabricator, remote spawn lab, and fog-of-war sensor logic.

15. Key risks and failure modes

The biggest risk is scope explosion. The project idea contains planets, terraforming, MMO scale, procedural generation, cities, machines, factories, fog of war, automation, and megaprojects. Without a vertical slice, it could become too broad to implement.

The second risk is accidentally treating brainstorming as final decisions. Several details were floated but not decided, including exact lore around human eggs, official

server resets, robot types, exact engine choices, and MMO scale.

The third risk is letting nanobots become magic. If they ignore material, energy, heat, time, and logistics, the game's industrial progression collapses.

The fourth risk is reintroducing human-like worker NPCs by default. That would contradict a major user preference and create compute problems.

The fifth risk is relying on clients for hidden MMO truth. That would undermine fog of war and anti-cheat.

The sixth risk is stale technical assumptions about Unreal, networking, planetary datasets, or current engine features. Those need verification before becoming formal spec claims.

16. Recommended next action sequence

1. Convert the mothership-to-remote-spawn-lab loop into a formal vertical slice spec.
2. Define the deterministic data model: ticks, commands, state hashes, event logs, snapshots, sparse deltas.
3. Define the planet field-layer model: procedural base, override layers, gameplay deltas, conservation rules.

4. Define mothership economy: reserves, throughput, power, heat, fabricator wear, spawn costs.
 5. Define the first robot chassis and their capabilities.
 6. Define nanobot construction tiers and blueprint commit rules.
 7. Define the first machine graph format and factory aggregation model.
 8. Define fog-of-war knowledge states and sensor rules.
 9. Decide how Unreal, Raylib, or custom rendering fit into the prototype.
 10. Verify current external assumptions before turning them into formal requirements.
-

17. File package verification summary

This companion package includes the existing generated preservation files, the uploaded preservation prompt, the earlier related Dominium package files that were present in `/mnt/data`, and new files created for this request. The package was checked by confirming that the archive could be opened and that each file could be read by Python's zipfile integrity test.

The final consolidated archive should be treated as the current best downloadable record of this conversation,

with caveats. It is safe for later aggregation, but manual review is still recommended before turning recommendations into formal requirements.

18. Best follow-up questions

- Turn the mothership-to-remote-spawn-lab loop into a formal vertical slice spec.
- Which assistant recommendations should become official Dominion requirements?
- Draft the deterministic Domino core data model.
- Draft the planet field-layer and sparse delta schema.
- Design the first five robot chassis.
- Design mothership reserves, fabrication throughput, heat, and spawn costs.
- Design nanobot construction tiers.
- Design the machine graph compiler.
- Design the fog-of-war sensor and map-sharing model.
- Decide whether Unreal is renderer-only, prototype client, or deeper engine dependency.

- Create the first master Project Spec Book chapter outline from these files.
-

19. Final status

Files created now:

- `Dominium_Conversation_Complete_Companion_Report.md`
- `Dominium_Conversation_Consolidated_File_Manifest.md`
- `Dominium_Conversation_Package_Verification_Report.md`
- `Dominium_Conversation_Complete_Preservation_Bundle.zip`

Main value of this companion report: it explains the whole conversation in plain language and ties together the existing preservation package files.

Main value of the ZIP: it preserves the prior generated package files plus the new companion report and manifest in one download.

7.12 Dominion Complete Conversation Companion Report Accompanying Human Readable Detailed Report

Source: docs/archive/conversations/Dominium_
Complete_Conversation/existing_companion_
report_package/Dominium_Complete_Conversation_
Companion_Report__01_accompanying_human_
readable_detailed_report.md

Report type: accompanying_human_readable

Size: 30030 bytes

7.12.1 Accompanying Human-Readable Detailed Report - Dominion / Domino Conversation Preservation

0. What this companion report is

This file is an accompanying human-readable report for the Dominion/Domino conversation package. It is intended to sit beside the earlier preservation files, not replace them. The earlier handoff package already contains a formal preservation report, context packet, registers, spec sheet, aggregator packet, reader brief, audit, and future-chat prompt. This companion report focuses on giving a coherent, readable account of the entire visible conversation, including what was discussed,

what was decided or recommended, what was deliberately postponed, what artifacts were created, and what should happen next.

Source scope: this report uses the visible conversation plus the uploaded preservation prompt and the files already present in `/mnt/data`. It does not claim access to hidden older chats, unprovided project materials, or private reasoning. Anything that came from the assistant rather than the user is treated as a recommendation or inference unless the user clearly accepted it.

Date anchor: 2026-05-27, Australia/Melbourne.

Reliability summary: high for the visible conversation; medium for external technology claims; with caveats for project-wide decisions because other **Dominium** chats may contain additional or conflicting context.

1. Plain-English summary

This conversation developed the foundations for **Dominium**, a very large-scale deterministic construction, terraforming, civilisation, and automation game built with or around the **Domino engine**. The initial question was about feasibility: how could the project support a procedurally generated full-scale solar system, fully recreated planets, custom planets, terraforming, cut/fill terrain editing, megaprojects,

player-designed machines, factories, fog of war, sparse simulation, single-player universes, persistent MMO universes, and potentially client-shared compute?

The first major conclusion was architectural: the game should not be built as a normal giant Unreal level full of actors. The world should be a deterministic data model first. Domino should own world truth through fixed ticks, event logs, sparse terrain/construction deltas, simulation LOD, state hashes, and replayable commands. Unreal, Raylib, or another renderer should be treated as a client layer that visualises and interacts with only the local, sensed portion of the universe. That recommendation remains an assistant recommendation, but it is strongly aligned with the user's requirement for determinism, scale, fog of war, and sparse simulation.

The second major shift came when the user described the game identity in much more detail. *Dominium* became a **robotic seed-civilisation game**. The lore/gameplay premise is that humans sent a robotic mothership to another solar system. The mothership lands on the most habitable planet, manufactures robot bodies, and begins rebuilding civilisation. Players are not humans chopping trees by hand. They are robot agents instantiated into physical bodies fabricated by machines. They may choose body forms such as bipeds, quadrupeds, spider-like bodies, tank-like forms, drones, or industrial chassis. A spawn point is not just a menu

location; it is a physical fabrication machine with power, materials, recipes, permissions, and network connection.

The mothership solves many design problems at once. It explains the HUD because players are robots. It explains blueprinting because robots can project plans into their sensor overlay. It explains recipes because the ship carries human knowledge. It explains quest markers because an onboard AI can translate goals into material and construction requirements. It explains respawning because bodies can be manufactured. It explains why players do not need to reinvent CPUs, rail gauges, machine tools, calendars, or basic industrial recipes. However, the mothership must be constrained so it does not trivialise the game. It can have finite resources, limited fabrication throughput, heat constraints, wear, mission reserves, rare feedstock constraints, and scale limits. The ship is excellent at knowledge and precision; the planet is required for scale.

A critical design direction was moving away from worker NPC labour. The user explicitly identified worker simulation as expensive because of pathfinding, scheduling, collision, animation, stuck states, and replication. Since the society is robotic, the game should not pretend that little human-like workers are doing the work. Instead, it should use automation from the start: nanobots, machine controllers, drones, AI agents, fabrication plans, process diagrams, and infrastructure

graphs. This is one of the most important points to preserve. Cities in *Dominium* should not be defined by thousands of simulated people walking around. They should be defined by power, logistics, materials, spawn capacity, sensors, permissions, factories, roads, rails, pipes, storage, fabrication, and machine networks.

Planet generation was also refined. The user wants science-bounded procedural star systems or solar systems. Designers should set real physical parameters such as orbital elements, star type, planet mass, radius, atmosphere, water inventory, rotation, climate, tectonics, and geology. The generated planet should come from plausible science rather than arbitrary sliders. At the same time, designers need artistic and gameplay control. The conversation introduced two complementary generation modes: forward generation, where parameters produce planets, and reverse generation, where the designer specifies a target planet feel and a solver searches for plausible parameters that could create it. The user also wanted procedural fields to be overlayable with painted/designer fields, and wanted the same underlying technology to support save files, planet packs, worldgen overrides, terraforming, construction, destruction, and terrain edits.

Construction and terrain editing were framed around sparse deterministic deltas. Players can preview blueprints freely, but committed changes should be

recorded as validated operations: cut, fill, reinforce, place, weld, disassemble, repair, excavate, and terraform. Nanobots or robotic construction swarms are the diegetic interface, but they must still consume material, energy, time, and heat budget. A tunnel is not just a mesh hole; it produces material, uses power, creates dust/heat, affects structure, and updates the persistent terrain delta store.

Machines and factories should be visually rich when observed but abstract when distant. The user wants players to create machines, vehicles, rockets, factories, and devices, but does not want every motor, belt item, gear tooth, pipe molecule, or joint simulated everywhere forever. The proposed solution is a **machine graph compiler**. Player-made assemblies compile into capabilities, input/output ports, throughput, power draw, heat output, mass, reliability, maintenance debt, and allowed actions. Nearby machines can be expanded visually. Far-away machines collapse into process equations such as input rates, output rates, buffers, energy consumption, and failure probabilities.

Fog of war is both a gameplay and technical requirement. The game should only load, simulate, and reveal what players can sense. In an MMO, hidden enemy bases, hidden ore fields, secret tunnels, undiscovered caves, and unknown entities must not be sent to clients. This limits client-shared compute.

Clients can help with rendering, local prediction, visible terrain generation, mesh generation, and non-secret verifiable work, but public MMO clients should not be trusted with hidden truth or economy/combat authority.

The conversation also addressed Unreal. The answer was not that Unreal is useless; rather, Unreal should be used carefully. It can be useful for rendering, UI, local terrain chunks, blueprint previews, animation, audio, effects, editor tooling, world partitioning, procedural content tools, and local client presentation. It should not be treated as the authoritative simulation database for a full deterministic solar-system MMO. External claims about Unreal capabilities, network scale, and engine features should be verified before becoming binding technical requirements.

Finally, the chat moved into preservation. The user uploaded a preservation/export prompt requiring a human-readable report first, plus structured registers, a spec sheet, an aggregator packet, self-audit, downloadable files, ZIP package, and future-chat handoff. The assistant generated a preservation package. In this follow-up task, the existing package files were checked, a zero-byte extracted spec sheet was noticed on disk, the correct nonzero spec sheet was recovered from the prior ZIP, and this consolidated companion package was created.

2. Chronological reconstruction

2.1 The conversation began with maximum-scale feasibility The user's first major question described an extremely broad technical target: a deterministic game with a full-scale real solar system, fully recreated planets, custom planets, procedural data-driven generation, player-built civilisations, terraforming, cut/fill terrain, megaprojects, machines, devices, factories, fog of war, sensed-only simulation, client-shared compute, single-player multi-universe play, and MMO-style persistent universes.

The important thing is that the user already understood some of the hard constraints. The question did not just ask for "a huge open world." It explicitly mentioned sparse construction/destruction, only loading what players can sense, collapsing everything else, and sharing compute where possible. Those details show that scale and fidelity were the real design challenge.

2.2 The first response separated simulation from rendering The assistant responded by saying the project should be a deterministic simulation/database first. That answer proposed that Domino own world truth through fixed ticks, stable command ordering, event logs, snapshots, sparse deltas, hashes, and deterministic procedural generation. Renderers such as

Raylib or Unreal would visualise state, but not be the source of truth.

It also introduced several technical pillars: hierarchical coordinate frames, procedural planets as base seeds plus sparse deltas, sparse voxel/SDF chunks for edited terrain, simulation LOD, server-side fog of war, careful client compute boundaries, and a distributed MMO architecture using region authorities rather than a single monolithic server.

The response drew an important feasibility boundary: millions of players can share a logical universe if spread across regions, shards, or universes, but millions cannot occupy one fully simulated low-latency physics bubble at high FPS. This remains a critical anti-scope-creep point.

2.3 The user refined the creative and systemic premise The next major user message gave the game a clearer identity. The user described a science-bounded star-system generator where designers set parameters and get plausible planets, rather than painting everything manually. The user also wanted the ability to nudge results, override fields, and perhaps reverse-generate worlds from desired outcomes.

Then the user introduced the lore and gameplay premise: humans in the past launched a robotic mothership to another solar system. The ship lands on the most habitable planet and manufactures robots to restart

civilisation. There may be mystery lore involving later ships containing human eggs or a human payload, and the robots may be preparing the world for humans, but the user clearly treated this as flexible lore rather than locked final story.

The user then described player spawning. Every new player enters through a physical machine that makes robot bodies. Players choose a body form. The starting mothership contains materials and fabrication capabilities, but players who want to expand across the world must physically travel, extract resources, and build new spawn labs. A remote spawn lab becomes an exploration and expansion milestone.

The user also explained the construction interface. Nanobots or some diegetic invisible force are attractive because they are computationally easy. Players can use a HUD to blueprint cut/fill and construction operations, preview them non-destructively, then commit them. This supports a construction fantasy without expensive worker simulation.

The user then made the key automation pivot. Initially, there might have been NPC workers, but the user explicitly rejected that as too expensive. If computers and robots are doing the work, the game should not pretend human-like labourers are doing it. The game should jump straight to automation. Work becomes energy, materials, machine capability, and process

scheduling rather than pathfinding workers.

The user also clarified the mothership's role. The ship carries recipes, designs, and knowledge so players do not have to reinvent CPUs, rail gauges, calendars, or industrial standards. But it must not be able to produce unlimited advanced goods forever. Its reserves are finite, its throughput is limited, and eventually players must build local industry to replenish or surpass it.

2.4 The second response synthesised the game as robotic seed civilisation

The assistant then summarised the design direction as a robotic seed-civilisation game. It organised the ideas into a more formal framework: science-bounded planet generation, reverse generation, field layer stacks, mothership constraints, robot chassis, physical inventory, nanobot construction, cut/fill terrain, no worker NPCs, machine graphs, city progression, AI-planner quest markers, fog-of-war sensors, single-player/private/MMO modes, Unreal's role, and a first vertical slice.

That response is useful as a design framework, but it should not all be treated as final. The user clearly supplied the core premise and many constraints, while the assistant provided architecture and structuring recommendations.

2.5 The preservation/export phase began The user uploaded a preservation prompt asking for maximum-fidelity chat preservation. The prompt required a human-readable report first, then registers, spec sheet, aggregator packet, self-audit, downloadable files, ZIP package, and final in-chat reader. It also required uncertainty labels and warned against treating brainstorming as final decisions.

The assistant generated a full preservation package. That package included a human-readable report, context packet, spec sheet, registers, aggregator packet, reader brief, verification/audit, future-chat bootstrap prompt, in-chat reader, and ZIP archive.

2.6 This companion package was requested and created The user then asked for an accompanying human-readable detailed summary/report of the entire conversation, including what was discussed, decided, done, put off, and anything else missed, bundled with all files into a single ZIP.

A file audit found two existing preservation package families in /mnt/data: one labelled `Dominium_Deterministic_Solar_System_Architecture` and one labelled `Dominium_Robotic_Seed_Civilisation_Architecture`. It also found the uploaded prompt `Pasted text.txt`. One extracted deterministic spec sheet on disk was zero bytes, but the deterministic ZIP

contained a correct nonzero spec sheet. This consolidated package therefore includes the correct file recovered from the ZIP rather than silently bundling the broken zero-byte extracted copy.

3. What was decided, what was recommended, and what remains tentative

3.1 User-stated design directions The following are the strongest user-stated directions from the visible conversation:

- 1. Dominion should be deterministic and procedural.** The game should generate solar systems/planets from data and rules, not hand-authored static maps.
- 2. Planet generation should be science-bounded.** Sliders and parameters should be constrained by plausibility rather than arbitrary fantasy controls, while still allowing designer nudges and overrides.
- 3. Procedural fields must support overlays.** Painted/designer overrides, save files, world packs, terraforming, construction, and destruction should share compatible field/delta technology.
- 4. The lore premise is a robotic mothership.** Humans launched a ship that manufactures robots

to restart civilisation on another planet.

5. **Players spawn as robots from physical machines.** Spawn points are fabricated infrastructure, not abstract menus.
6. **Expansion requires new spawn labs.** Players can push the frontier by building remote fabrication/spawn facilities.
7. **The HUD and blueprints are diegetic.** Robots can have sensor overlays and receive construction programs from the ship or AI systems.
8. **Nanobots or equivalent construction actuators are desirable.** They allow powerful cut/fill/build interactions without simulating worker bodies.
9. **Worker NPC labour is deprioritised.** The user explicitly called out labour/pathfinding as expensive and moved toward automation from the start.
10. **The mothership is a recipe book and starter fabricator, but limited.** It contains knowledge and some resources, but players must build local industry for scale.
11. **Cities should be relatively easy to build at the basic level.** Advanced technology should be

hard because it requires infrastructure, not because players must micromanage labourers.

12. **Machines and factories should collapse into process equations when possible.** The user wants input/tick/output abstractions rather than full simulation of every part everywhere.

3.2 Assistant recommendations that should not be treated as final decisions without confirmation

The assistant recommended several architecture choices. They are useful, but they should be treated as recommendations unless confirmed later:

1. Domino should be the authoritative deterministic simulation/database core.
2. Unreal should be a renderer/client/tooling layer, not the authoritative world model.
3. The world should use fixed ticks, event logs, snapshots, state hashes, and deterministic replays.
4. Planets should use hierarchical coordinate frames, base procedural generation, and sparse deltas.
5. Terrain should use sparse voxel/SDF or equivalent chunk systems for cut/fill.
6. Fog of war should be enforced server-side, with clients receiving only observed/remembered state.

7. Public MMO clients should not be trusted with hidden truth or authority-critical computation.
8. Distant machines/factories/cities should use simulation LOD and aggregate process graphs.
9. The first vertical slice should be the mothership-to-remote-spawn-lab loop.

These recommendations are compatible with the user's goals, but a future spec should mark them as accepted only if the user explicitly approves them.

3.3 Decisions and questions still unresolved Many implementation details were deliberately left open:

- exact engine stack;
- exact programming language and deterministic math model;
- final planet generator schema;
- reverse-generation solver design;
- field-layer operation format;
- terrain delta storage;
- robot chassis list and balance;
- mothership reserves and fabrication limits;

- nanobot capabilities by tier;
- material/energy/heat model;
- machine graph compiler rules;
- city progression and liveliness without NPC workers;
- fog-of-war sensor model;
- MMO authority/shard design;
- client compute trust model;
- official server reset/recovery policy;
- first vertical slice specification.

4. What was put off for later

The conversation intentionally stayed at architecture/design level and put off detailed specs. The most important postponed work is listed below.

4.1 Planet and star-system generation spec The conversation established the principle of science-bounded generation, but did not define the complete schema. Future work needs to define parameters such as star mass/luminosity/age, orbit, eccentricity, axial tilt, rotation, moons, atmosphere, water, tectonics, erosion,

cratering, resources, magnetosphere, radiation, climate, and gameplay scoring.

It also needs to define how reverse generation works: what the designer can target, how the solver searches, how plausibility is scored, and how unrealistic requests are handled.

4.2 Field/delta architecture The user wanted procedural and painted fields to overlay, and the assistant suggested a stack such as base procedural field plus real data plus designer overrides plus mod layers plus terraforming plus construction plus temporary previews. But the exact data representation remains open.

Future work needs to define field types, resolution, chunking, blending operations, conservation rules, versioning, save formats, delta compression, conflict resolution, and how gameplay edits differ from editor overrides.

4.3 Mothership economy The mothership is central, but its exact constraints are not defined. Future work needs to choose its initial reserves, power limits, heat limits, fabrication throughput, wear, emergency modes, recipe access, body fabrication cost, mission reserve rules, and how players replenish it.

This is critical because if the mothership is too powerful it removes the need for industry; if it is too weak it becomes frustrating.

4.4 Robot body and respawn system The user mentioned possible body forms but did not settle the body list, costs, abilities, or progression. Future work should define initial chassis, chassis upgrades, cargo limits, terrain mobility, sensor packages, tool hardpoints, death/backup rules, and spawn lab requirements.

4.5 Construction and nanobot tiers Nanobot construction is powerful, but its limits remain open. Future work should define which operations are allowed early, which require special tools, how material is staged, how blueprints are validated, how energy/time/heat are calculated, and how construction can be grief-proofed in multiplayer.

4.6 Machine graph compiler The user wants machines and vehicles to become summary equations when possible, but the compiler does not yet exist. Future work must define allowed modules, graph topology, ports, capabilities, constraints, performance budgets, validation, exploit resistance, and how the visual machine maps to the abstract process.

4.7 City design without worker NPCs The decision to avoid worker NPCs solves compute problems but creates a design problem: cities must still feel alive. Future work should design visible machine life: drones, moving cargo, lights, process displays, traffic abstractions, industrial sounds, sensor networks, construction swarms, and city operating systems.

4.8 Fog of war and sensors The chat established that fog of war matters, but not the full model. Future work should define observed, remembered, inferred, unknown, and secret states; sensor types; map sharing; scan resolution; hidden underground data; stealth; communication relays; satellites; and client data filtering.

4.9 MMO/distributed architecture The conversation sketched one logical universe with many region authorities, but the exact design is open. Future work must define region handoff, authority boundaries, persistence, tick rates, load balancing, anti-cheat, client prediction, and how single-player/private servers differ from official servers.

4.10 Verification of external engine/data facts Claims about Unreal features, NASA/JPL/SPICE, networking scale, and MMO feasibility should be verified before becoming formal technical requirements. The

conversation used these as supporting context, not as final implementation proof.

5. Artifact and file history

Several artifacts now exist. The earlier generated package family was `Dominium_Deterministic_Solar_System_Architecture`. It contains a manifest, human-readable report, context transfer packet, spec sheet, registers, aggregator packet, reader brief, verification/audit, future-chat bootstrap prompt, in-chat reader, and ZIP archive.

A second existing package family, `Dominium_Robotic_Seed_Civilisation_Architecture`, was also present in `/mnt/data`. It appears to be an earlier or parallel preservation package for the same overall subject, with the same file categories.

The uploaded `Pasted text.txt` is the preservation prompt. It is important because it specified the output requirements: human-readable report first, structured registers, spec sheet, aggregator packet, self-audit, file export, and final reader. It also required caveats, uncertainty labels, preservation of tentative status, and no false claims about file creation.

This new consolidated package includes:

- a new manifest;

- this companion report;
- a file/artifact audit;
- a decisions/tasks/open-questions digest;
- a next-step vertical-slice brief;
- a future-chat prompt;
- the uploaded preservation prompt;
- both prior preservation package families, extracted from their ZIP files;
- both prior ZIP archives;
- a final inventory with file sizes and hashes.

Important audit note: the extracted disk copy of `Dominium_Deterministic_Solar_System_Architecture__03_spec_sheet.yaml` was zero bytes, but the prior deterministic ZIP contained a valid nonzero spec sheet. The consolidated package uses the ZIP-recovered version, so the bundled copy is not zero bytes.

6. Main risks and future-assistant failure modes

The biggest project risk is scope explosion. The vision includes planet generation, terrain editing, automation, factories, cities, fog of war, MMO scale, client compute, and Unreal integration. A future assistant should not

treat the whole vision as a single initial implementation target. The correct next step is a small vertical slice.

The biggest preservation risk is turning brainstorming into decisions. The user's robot/mothership/automation directions are strong, but many assistant architecture recommendations remain recommendations unless explicitly accepted.

The biggest design risk is nanobots becoming magic. If nanobots can make anything instantly without material, energy, heat, time, or capability constraints, the industrial game collapses.

The biggest technical risk is representing too much as live simulation. Every system needs a collapsed form: terrain deltas, process graphs, aggregate cities, sensor fields, and local expansion only when observed.

The biggest MMO risk is trusting clients with hidden information or authority. Client-shared compute is useful only for safe, visible, verifiable tasks in public MMO mode.

The biggest experiential risk is empty cities. Removing worker NPCs is computationally sensible, but the game still needs machine life and visible industrial feedback.

The biggest aggregation risk is future reports conflicting. Other chats may have different assumptions about NPCs, engine choice, world scale, or progression. This package should be merged with labels and caveats intact.

7. Recommended next-action sequence

The safest next action is to turn the proposed first playable loop into a formal milestone plan. That loop should include:

1. one planet region;
2. one mothership;
3. one robot body;
4. one simple sensor/fog-of-war model;
5. one ore deposit;
6. one power source;
7. one cut/fill tool;
8. one blueprint tool;
9. one nanobot construction swarm;
10. one mine;
11. one refinery;
12. one fabricator;
13. one remote spawn lab;
14. one respawn/body-switch event from that lab.

The success condition is not visual grandeur. The success condition is proving the core loop: a robot leaves the mothership, surveys terrain, finds material, prepares a site, builds infrastructure, fabricates parts, constructs a new spawn lab, and uses it. If that loop is fun and deterministic, the larger game has a foundation. If it is not fun, adding planets, cities, megaprojects, and MMO scale will not fix it.

After that, the next work should be:

1. define design pillars;
2. specify deterministic core invariants;
3. specify field/delta data model;
4. define mothership economy;
5. define robot chassis;
6. define nanobot construction tiers;
7. define machine graph compiler;
8. define fog-of-war sensors;
9. verify Unreal/current engine assumptions;
10. merge this report with other old-chat reports only after preserving caveats.

8. Best questions to ask next

- Turn the mothership-to-remote-spawn-lab loop into a formal vertical-slice spec.
- Define the field-layer and sparse delta format.
- Design the mothership resource economy.
- Define the first five robot chassis and their tradeoffs.
- Design nanobot construction tiers and limits.
- Design the machine graph compiler.
- Design the fog-of-war sensor model.
- Design cities that feel alive without worker NPCs.
- Decide whether Unreal is renderer/client only or also part of the simulation stack.
- Convert this package into a master Project Spec Book chapter outline.

9. Final memory capsule

Dominium should be remembered as a deterministic robotic seed-civilisation game. Domino should probably be the sparse deterministic simulation core. Worlds should be generated from science-bounded procedural data plus overlays and sparse deltas. Players are robot

bodies fabricated by physical machines. The mothership provides knowledge, recipes, starter resources, and limited precision fabrication, but players need local industry for scale. Construction should use diegetic blueprints and nanobots, but must obey material, energy, time, heat, and capability constraints. Cities should be infrastructure and automation systems, not worker-NPC simulations. Machines should compile into process graphs so they can collapse when distant. Fog of war must be real, not just visual. MMO clients can help only with safe, visible, verifiable compute. The immediate next step is the first vertical slice: go from mothership spawn to remote spawn lab through surveying, mining, refining, fabrication, and construction.

7.13 **Dominium Complete Conversation Companion Report Bundle Manifest**

Source: docs/archive/conversations/Dominium_
Complete_Conversation/existing_companion_
report_package/Dominium_Complete_Conversation_
Companion_Report__bundle_manifest.md

Report type: companion_report

Size: 5882 bytes

7.13.1 Complete Bundle Manifest -
Dominium / Domino Conversation
Preservation

Created: 2026-05-27 Australia/Melbourne

Purpose: Single ZIP bundle containing the existing Dominium preservation files, prior handoff packages, the uploaded preservation prompt, and the new human-readable companion report.

Source scope: This conversation and files present in /mnt/data at bundle creation time.

Bundle Contents

Row 1 File: Dominium_Complete_Conversation_Companion_Report__human_readable_detailed_summary.md; Size bytes: 29791; SHA-256: 012efa1d441d5ba37c66bf1e31b34b479127f7a1b48ba

Row 2 File: Dominium_Conversation_Complete_Companion_Report.md; Size bytes: 22109; SHA-256: 3a8213b11ae92c281798850bc8c8b30a43be17b6476a3

Row 3 File: Dominium_Conversation_Complete_Preservation_Bundle.zip; Size bytes: 516816; SHA-256: 4971b3775d7f7c7033851eb644d2c136401f9d1cba8af

Row 4 File: `Dominium_Conversation_Consolidated_File_Manifest.md`; Size bytes: 6167; SHA-256: `b14978389453751440dc63a48176945b77ee6013020076`

Row 5 File: `Dominium_Conversation_Package_Verification_Report.md`; Size bytes: 4006; SHA-256: `557106f08d8112a21f3908169c835ce545fe4348c5a75a`

Row 6 File: `Dominium_Deterministic_Solar_System_Architecture__00_manifest.md`; Size bytes: 2389; SHA-256: `d8e9ad908bbef8e55e6346bde1dcda1edba52db7671e76`

Row 7 File: `Dominium_Deterministic_Solar_System_Architecture__01_human_readable_report.md`; Size bytes: 57540; SHA-256: `320d5e21af78e1398f3784a6e1ecc612c047bfb15e6f2c`

Row 8 File: `Dominium_Deterministic_Solar_System_Architecture__02_context_transfer_packet.md`; Size bytes: 4303; SHA-256: `a07ff9d193d55bc216fa97770b6512922bc0def272e9d7`

Row 9 File: `Dominium_Deterministic_Solar_System_Architecture__03_spec_sheet.yaml`; Size bytes: 58575; SHA-256: `0dcbbc5509d713428dbc0ddd09fe79eedca35e9887b87c`

Row 10 File: `Dominium_Deterministic_Solar_System_Architecture__04_registers.md`; Size

bytes: 40599; SHA-256:

3c7af3db6949e29ef45f402f8a85b9d6cbbefccac4fd6

Row 11 File: Dominion_Deterministic_Solar_System_Architecture__05_aggregator_packet.md; Size bytes: 27431; SHA-256:

9674464e0379b64f13eb3d9d404b56a58311661dda0cd9

Row 12 File: Dominion_Deterministic_Solar_System_Architecture__06_reader_brief.md; Size bytes: 3264; SHA-256:

56d50d85c678d79cae032dda8ad31f794d2aaaa4f96719

Row 13 File: Dominion_Deterministic_Solar_System_Architecture__07_verification_and_audit.md; Size bytes: 4723; SHA-256:

6a7e79ed0adad4851c6cf8d7b9909c1bc6416482aeace5

Row 14 File: Dominion_Deterministic_Solar_System_Architecture__08_future_chat_bootstrap_prompt.md; Size bytes: 978; SHA-256:

984d223840f548c1ed1bb07fe6f59c23ecc570db35009

Row 15 File: Dominion_Deterministic_Solar_System_Architecture__09_in_chat_reader.md; Size bytes: 2199; SHA-256:

6cd8b657a8b1b9cf9140517e61c2d772b64d59cfe2457c

Row 16 File: Dominion_Deterministic_Solar_System_Architecture__10_accompanying_human_readable_conversation_report.md; Size

bytes: 20191; SHA-256:

9c3b31be9005922ef28971375c7ba7c62ce8fc3291d22a

Row 17 File: Dominion_Deterministic_Solar_System_Architecture__11_complete_bundle_manifest.md; Size bytes: 4303; SHA-256:

e7bfaad1453d4bda7befe09eddebc997272d92935a8770

Row 18 File: Dominion_Deterministic_Solar_System_Architecture__complete_conversation_package.zip; Size bytes: 253679; SHA-256:

c59fabaf6bb524f133336519a5b9edb2a6ad2481a989e1

Row 19 File: Dominion_Deterministic_Solar_System_Architecture__handoff_package.zip; Size bytes: 63147; SHA-256:

469bfb68b19198d9a1eb3e2e37b985516079cb4b82f960

Row 20 File: Dominion_Robotic_Seed_Civilisation_Architecture__00_manifest.md; Size bytes: 2627; SHA-256:

320c1fb070acf1ed313d93c571712ba1d2e973e93f2820

Row 21 File: Dominion_Robotic_Seed_Civilisation_Architecture__01_human_readable_report.md; Size bytes: 33167; SHA-256:

0b1e56f0d8e572465e285b65cdf0e8a40bea76ffec70a0

Row 22 File: Dominion_Robotic_Seed_Civilisation_Architecture__02_context_

transfer_packet.md; Size bytes: 5362; SHA-256:
2212c0a5be7f1ecf672e4cab1a8cd72465379ca1683506

Row 23 File: Dominion_Robotic_Seed_
Civilisation_Architecture__03_spec_sheet.
yaml; Size bytes: 53240; SHA-256:
3b0368853afb7ef7af9bcba5c763ee3ca7c99333d1e74d

Row 24 File: Dominion_Robotic_Seed_
Civilisation_Architecture__04_registers.md;
Size bytes: 37669; SHA-256:
d02a062de85e840f600fae79af2ad3cb99f5c62ddcfb86

Row 25 File: Dominion_Robotic_Seed_
Civilisation_Architecture__05_aggregator_
packet.md; Size bytes: 25376; SHA-256:
056eb47ecc86358dae13b0a1581f167c4defa281071cb8

Row 26 File: Dominion_Robotic_Seed_
Civilisation_Architecture__06_reader_
brief.md; Size bytes: 3058; SHA-256:
95b5d819810c1aaac6171653dff39856d0730566b4515f

Row 27 File: Dominion_Robotic_Seed_
Civilisation_Architecture__07_
verification_and_audit.md; Size bytes: 5158;
SHA-256:
046540e67e94aea1436177d481117b7ea2df5b967cca3f

Row 28 File: Dominion_Robotic_Seed_
Civilisation_Architecture__08_future_chat_

bootstrap_prompt.md; Size bytes: 980; SHA-256:
4a95d64912ca6a74b63f66059945e50db09ae3c733d423

Row 29 File: Dominion_Robotic_Seed_
Civilisation_Architecture__09_in_chat_
reader.md; Size bytes: 2406; SHA-256:
f52c2ca140b41e32d8c1497117ad4078ea18ec1a756fd1

Row 30 File: Dominion_Robotic_Seed_
Civilisation_Architecture__handoff_
package.zip; Size bytes: 54889; SHA-256:
972af06794dd47134da18a0c99f8235ff3e8df6e333d67

Row 31 File: Pasted_text.txt; Size bytes: 31225;
SHA-256:
6686114753accfa0ddd2aa1b3ef159bc685c53892dc58a

Notes

- The bundle includes both the earlier Dominion_Robotic_Seed_Civilisation_Architecture preservation set and the later Dominion_Deterministic_Solar_System_Architecture preservation set.
- The bundle includes Pasted_text.txt, the uploaded preservation/export prompt.
- The bundle includes Dominion_Complete_Conversation_Companion_Report__human_

readable_detailed_summary.md, the new accompanying narrative report.

- Prior ZIP handoff packages are preserved inside this complete ZIP as artifacts. This creates some duplication by design.
- External engine/data claims in earlier reports remain verification items and should not be treated as freshly verified.

7.14 **Dominium Complete Conversation Companion Report Human Readable Detailed Summary**

Source: docs/archive/conversations/Dominium_Complete_Conversation/existing_companion_report_package/Dominium_Complete_Conversation_Companion_Report__human_readable_detailed_summary.md

Report type: human_readable_detailed_summary

Size: 29791 bytes

7.14.1 **Dominium / Domino - Complete Conversation Companion Report**

Date anchor: 2026-05-27 Australia/Melbourne

Source scope: This conversation and the files created during this conversation.

Purpose: This report is a human-readable companion to the structured preservation package. It explains, in ordinary language, what happened in the conversation, what was discussed, what was decided or only proposed, what was deferred, what artifacts were created, and what should be preserved for future **Dominium / Domino** work.

1. Executive Orientation

This conversation developed the foundation for **Dominium**, a deterministic, procedural, solar-system-scale robotic civilisation game built with or around a custom **Domino engine**. The central challenge was how to make a game that can contain full-scale solar systems, procedurally recreated or custom planets, terraforming, cut/fill construction, machines, devices, factories, megaprojects, fog of war, sparse destruction/construction, and both single-player/private-universe play and possible MMO-style persistent-universe play.

The key technical conclusion was that the world cannot be treated as a normal game-engine level full of live actors. The world must be a deterministic simulation and data system first. **Domino** should store the true universe as seeds, rules, fields, sparse deltas, event logs,

snapshots, hashes, graphs, and ledgers. Rendering engines such as Unreal can be used to display and interact with the local area, but they should not be the sole source of authoritative world truth.

The key creative conclusion was that *Dominium* should be a **robotic seed-civilisation game**. Humans in the past launched a robotic mothership to another star system. The ship lands on a suitable planet and manufactures robot bodies to restart civilisation. Players are not ordinary human settlers. They are robot minds instantiated into physical chassis fabricated by machines. This premise explains the HUD, respawning, blueprint projection, recipe knowledge, construction automation, remote spawn labs, and the absence of worker NPC labour.

The strongest gameplay direction was to avoid simulating human-like workers as the foundation of cities and construction. The user explicitly identified labour, worker pathfinding, scheduling, and human-like simulation as expensive and unnecessary. Since the civilisation is robotic, the game should use nanobots, drones, machine controllers, AI planners, construction swarms, process graphs, and infrastructure systems instead. Cities should be made from power, logistics, fabrication, sensors, permissions, spawn labs, roads, rails, pipes, machines, and production networks rather than thousands of simulated people walking around.

The strongest technical design pattern was **high-fidelity when sensed, collapsed when not sensed**. A factory near the player can be visually rich. It can show belts, arms, lights, sparks, vehicles, modules, robots, and moving parts. Far away, the same factory should collapse into a deterministic process equation: inputs, outputs, power draw, heat, throughput, maintenance debt, buffers, and failure chances. A planet nobody has modified should exist mostly as seed plus generator version. A mined or terraformed area should exist as sparse deltas over the procedural base.

The conversation also produced a preservation/export package: a manifest, human-readable report, context transfer packet, spec sheet, structured registers, aggregator packet, reader brief, verification/audit file, future-chat bootstrap prompt, in-chat reader, and ZIP handoff package. This companion report adds a more narrative explanation and bundles all existing files into a single checked ZIP.

2. Chronological Story of the Conversation

2.1 The opening feasibility question The user began with a broad, ambitious feasibility question: could Domino and Dominium support a deterministic game with a full-scale real solar system, recreated planets, custom solar systems and planets, procedural universe

generation, player-built civilisations, terraforming, cut/fill terrain, megaprojects, machine and factory design, fog of war, sensed-only loading/simulation, sparse construction/destruction, MMO persistence, single-player multiverse play, and client-shared compute? The user also asked whether Unreal could do this without lag, with low latency and high FPS, and potentially millions of players spread across shards or private servers.

This set the technical frame: the problem was not simply graphical scale. It was about determinism, simulation ownership, sparse persistence, fog of war, distributed compute, and scalable interaction.

2.2 The first architecture response The assistant responded by reframing *Dominium* as a deterministic simulation/database problem. The recommendation was that Domino should be the authoritative world engine, and rendering engines should be clients. The world should be represented through deterministic rules, event logs, sparse deltas, state hashes, generator versions, fixed ticks, hierarchical coordinates, and simulation levels of detail.

The answer drew an important feasibility boundary: millions of players can share a logical universe if distributed across many cells, regions, shards, or private servers, but millions cannot all be in the same fully simulated low-latency local interaction bubble at once.

This is not a stylistic point; it is a network, CPU, bandwidth, and simulation-limit point.

2.3 The user refined the project into a game identity The user then gave the most important creative design monologue of the conversation. The project shifted from a generic scale question into a specific premise and loop.

The user described a procedurally generated solar system or configurable system, with as many planets as desired and realistic parameter bounds. Planet generation should use real science rather than arbitrary sliders. Designers should not manually paint everything by default; instead, they should set high-level physical parameters, get a procedural planet, and then nudge or override details where needed.

The user also proposed a reverse-generation idea. Instead of only choosing orbital and planetary parameters and seeing what planet results, a designer might specify the desired planet outcome and let the system search for plausible parameters that produce it. This would let designers make playable worlds without breaking the scientific tone.

The user then described field overlays. Procedural fields should be layered with painted fields and designer overrides. The same technology that powers save files should also support packs, planet design, terraforming,

cut/fill, and in-game edits.

The lore premise then emerged: humans in the past launched a robotic mothership to another solar system. It lands on the most habitable planet and manufactures robots using advanced technology, possibly nanotechnology, in order to restart civilisation. There could be mystery lore involving later ships containing human eggs or payloads, but the main gameplay premise is the robotic restart of civilisation.

Players spawn from physical machines. A new player chooses a robot body type, such as a biped, quadruped, spider, tank-like body, or other chassis. The original mothership can fabricate early bodies from limited reserves. To expand across the planet, players must physically travel and build new labs or spawn points that can fabricate bodies from local materials and power.

Construction should be explained diegetically through robotic HUDs, blueprints, nanobots, and electromagnetic communication with construction swarms. The player can preview a build nondestructively as a holographic or HUD blueprint, then commit the build once resources, permissions, and construction capacity are available.

The user considered worker NPCs but moved away from them. Worker NPC labour would require pathfinding, scheduling, collision, and simulation. Since the civilisation is robotic, the game should not make the

computer pretend to be people doing labour. It should let the computer be the worker: automation, nanobots, AI agents, machines, process diagrams, and construction systems.

The mothership should also explain why players have recipes and do not need to invent CPUs, rail gauges, calendars, or basic industrial theory from scratch. The ship already carries human knowledge. The progression challenge is not discovering the idea of a CPU; it is building the industrial chain capable of fabricating CPUs locally.

The user also identified the need to limit the mothership. If the mothership can fabricate anything perfectly at any scale, it invalidates the game. Therefore it should have finite material reserves, low throughput, heat limits, size limits, mission reserves, wear, and other constraints. It can help early, but players eventually need mines, refineries, factories, logistics, power grids, cities, and industrial capacity.

2.4 The second synthesis response The assistant then synthesised the design into the phrase: **robotic seed-civilisation game**. It organised the ideas into systems: science-bounded procedural worldgen, reverse planet generation, field-layer stacks, mothership constraints, robot chassis, physical inventory, nanobot construction, cut/fill terrain, no worker NPCs, machine

graph factories, city progression, AI-planner quest markers, fog-of-war sensors, single-player/private/MMO modes, Unreal as client/rendering/tooling, and the first vertical slice.

A key recommendation was that nanobots should be construction actuators, not magic. They can cut, fill, weld, assemble, disassemble, reinforce, repair, and survey, but each committed operation must require material, energy, time, heat handling, capability, permission, and deterministic validation.

Another key recommendation was that machines and factories should compile into process graphs. The player may build visually complex machines, but the simulation should reduce them to validated capabilities, inputs, outputs, throughput, power draw, heat, mass, maintenance, and reliability. This allows creative engineering while avoiding the need to simulate every part everywhere forever.

The assistant also proposed a first vertical slice: one planet region, one mothership, two robot chassis, one resource deposit, one power source, cut/fill, blueprinting, a nanobot construction swarm, a mine, a refinery, a fabricator, a remote spawn lab, and fog-of-war sensors.

2.5 The preservation/export package The user uploaded a detailed preservation prompt requiring a maximum-fidelity human-readable report, structured

registers, context transfer packet, spec sheet, aggregator packet, self-audit, files, and ZIP export. The assistant produced a complete package with sections 0-39, including the human-readable report, registers, spec sheet, future-chat packet, aggregator packet, audit, reader guide, and downloadable files.

2.6 This companion report and complete bundle

The user then asked for an accompanying human-readable detailed summary/report of the entire conversation and requested that everything be checked and bundled into a single ZIP. This file is that companion report. It sits alongside the earlier structured preservation files and is bundled with them in the new complete ZIP.

3. Major Design Outcomes

3.1 Domino should be treated as the authoritative deterministic core **Status:** Assistant

recommendation, strongly aligned with the user’s requirements.

Label: INFERENCE unless explicitly ratified later.

The conversation repeatedly pointed toward a custom deterministic engine core. Domino should own true world state, deterministic simulation, field layers, sparse construction/destruction, event logs, state hashes, fog-of-

war truth, material ledgers, machine graphs, and persistence. A renderer such as Unreal should display only local, sensed, permitted state.

This matters because a full-scale editable solar-system MMO cannot be built as one normal live scene.

Determinism, replay, sparse persistence, and secret-state filtering require a simulation/data model deeper than a renderer.

3.2 Procedural planets should be science-bounded but designer-steerable **Status:** User-stated design direction.

Label: FACT.

Planet and system generation should be controlled by plausible physical parameters: orbital configuration, star type, planet mass/radius/density, atmosphere, water, climate, geology, resources, and related fields. Sliders should be bounded to realistic ranges where possible.

But the designer still needs control. The system should allow nudging, overrides, painted layers, and possibly reverse-solving from target planet qualities to plausible physical parameters.

3.3 The field-layer/delta model is central **Status:** User-stated direction plus assistant formalisation.

Label: FACT + INFERENCE.

The same data model should support world generation, designer overrides, planet packs, save files, terraforming, terrain edits, construction, damage, and destruction. The proposed structure is a stack of fields and sparse deltas over a procedural base.

The most important distinction is that editor/worldgen layers may create content without conservation requirements, while gameplay layers should generally conserve mass, material, energy, or equivalent game resources.

3.4 The mothership premise solves many design problems **Status:** User-stated premise.

Label: FACT.

The mothership explains why the player has a HUD, why respawn is physical, why recipes are known, why blueprints exist, why construction is automated, why the early game has a central hub, and why new spawn points are meaningful frontier infrastructure.

The mothership also creates progression tension. It has knowledge and precision fabrication, but limited material reserves, low throughput, heat limits, wear, and mission constraints. This forces players to build local industry.

3.5 Players are robot bodies/chassis, not ordinary human settlers **Status:** User-stated direction.

Label: FACT.

Players spawn into manufactured robot bodies. Possible forms include bipeds, quadrupeds, spiders, tank-like frames, drones, heavy industrial bodies, aquatic bodies, and aerial bodies. The exact launch list is deferred.

This supports diegetic inventory, construction, HUD, respawn, harsh environments, exploration, and body switching.

3.6 Automation replaces worker NPC labour

Status: Strong user-stated preference.

Label: FACT.

This is one of the most important outcomes. The user explicitly moved away from intelligent worker NPCs because labour and pathfinding are expensive. The world should not need to simulate a society of human-like workers to explain construction or production.

Instead, the civilisation is machine-first: nanobots, drones, AI agents, process graphs, logistics systems, controllers, and construction swarms. Wildlife or mobs may exist, but cities do not rely on worker NPCs as the central productive system.

3.7 Machines and factories should compile into process graphs

Status: User-stated direction.

Label: FACT.

The user wants players to build machines, vehicles, factories, rockets, and devices, but does not want every component simulated forever. The answer is a machine compiler: visible assemblies become validated functional graphs with inputs, outputs, energy, heat, throughput, reliability, storage, and constraints.

This supports player creativity without allowing unlimited performance costs.

3.8 Fog of war is both gameplay and data security

Status: User required fog of war; assistant formalised security implications.

Label: FACT + INFERENCE.

Fog of war is not just a visual overlay. It controls what exists in a client's knowledge, what the server sends, what can be simulated by the player's machine, and what remains hidden. A client cannot help compute hidden enemy bases or secret resource locations in a public MMO without compromising the game.

3.9 Unreal should probably be a client/tooling layer, not the universe authority **Status:** Assistant recommendation, not user-final.

Label: INFERENCE / UNVERIFIED.

The conversation repeatedly suggested that Unreal can help with rendering, local interaction, UI, visual

streaming, animation, PCG tooling, and high-fidelity presentation. It should not be the sole authoritative state engine for a full deterministic editable universe. This needs verification against current engine capabilities and project needs before becoming a binding architecture decision.

3.10 The first vertical slice should prove the core loop **Status:** Assistant recommendation, not yet user-approved.

Label: INFERENCE.

The proposed first slice is not a full MMO and not a full solar system. It is one playable loop:

1. Spawn from the mothership.
2. Survey terrain.
3. Find ore.
4. Use cut/fill and blueprints.
5. Build power.
6. Build mine/refinery/fabricator.
7. Transport or reserve materials.
8. Construct a remote spawn lab.
9. Spawn or switch body from that lab.

10. Preserve fog-of-war and deterministic replay.

This loop tests almost every core system at small scale.

4. What Was Decided, What Was Only Proposed, and What Was Deferred

4.1 Strong user-stated decisions or directions

These are safest to carry forward as user-originated design direction:

- Dominion should involve procedural solar systems and planets.
- World generation should be science-bounded rather than arbitrary slider art.
- Designers should be able to nudge or override generated planets.
- Field overlays should support procedural generation, painted data, saves, planet packs, and terraforming.
- The lore centres on a robotic mothership that manufactures robots to restart civilisation.
- Players spawn as robot bodies from physical machines.

- Remote spawn points should be built physically by players as frontier infrastructure.
- The mothership contains recipes/knowledge so players do not need to reinvent industrial basics.
- The mothership must have finite capacity, limited resources, or other bottlenecks so local industry matters.
- Nanobot/automated construction is preferred over person-like manual labour simulation.
- Worker NPC labour is deprioritised because pathfinding and labour simulation are expensive.
- Machines/factories should be reducible to process diagrams/equations when possible.
- The game should support single-player/private universes and possibly official MMO-style persistent play.

4.2 Assistant recommendations not yet final

These are useful, but should not be treated as user-final decisions unless later accepted:

- Domino as a custom deterministic simulation core separate from Unreal.
- Unreal as a renderer/client/tooling layer rather than authoritative engine.

- Fixed-tick event-sourced architecture with snapshots and hashes.
- Hierarchical coordinate frames for solar-system scale.
- Machine graph compiler as the formal implementation model.
- Server authority over public MMO truth.
- Strict hidden-state filtering for fog of war.
- The specific first vertical slice composition.
- Exact planet generator scoring categories.
- Exact chassis categories and city tier table.

4.3 Things explicitly or implicitly put off for later

The conversation deliberately left many items unresolved:

- The exact launch scope: one designed star system, many systems, or true galaxy-scale play.
- The exact deterministic math model.
- The field-layer and sparse-delta file format.
- The exact planet generation parameter schema.
- The reverse-generation solver design.

- The mothership's reserves, throughput, heat limits, and fabrication rules.
- The robot chassis list and body progression.
- Death, backup, respawn, and memory-state rules.
- Nanobot construction tiers and limitations.
- The machine graph compiler and validation rules.
- Vehicle, rocket, and megaproject abstraction.
- City liveliness without worker NPCs.
- Economy, permissions, factions, governance, griefing, war, and claims.
- Fog-of-war sensor mechanics and map sharing.
- MMO architecture, shard/cell boundaries, authority, and client compute policy.
- Unreal integration details.
- External data and licensing verification.
- The final vertical slice milestone spec.

5. Artifacts Created or Present

This conversation now contains two preservation/export sets plus this companion report.

5.1 Earlier robotic seed-civilisation package Files named `Dominium_Robotic_Seed_Civilisation_Architecture__...` appear to be an earlier preservation package. They include a manifest, report, context packet, spec sheet, registers, aggregator packet, reader brief, verification/audit file, bootstrap prompt, in-chat reader, and handoff ZIP.

5.2 Later deterministic solar-system package Files named `Dominium_Deterministic_Solar_System_Architecture__...` are the later and more complete preservation package generated after the uploaded preservation prompt. These files cover the human-readable report, context transfer, structured registers, spec sheet, aggregator packet, audit, future-chat bootstrap, and package ZIP.

5.3 Uploaded preservation prompt The file `Pasted text.txt` contains the user's maximum-fidelity preservation/export instructions. It is an important artifact because it defines the required shape of the preservation package.

5.4 This companion report and complete bundle This report adds a narrative companion summary that focuses on human understanding rather than only structured handoff. It is included in the new complete

ZIP together with the existing files.

6. Key Risks and Failure Modes

Risk 1 - Treating brainstorming as final decisions

Much of the conversation contains ideas and recommendations. Future work must not treat every assistant-supplied detail as accepted canon. The preservation package uses labels such as FACT, INFERENCE, and UNVERIFIED to avoid this.

Risk 2 - Reintroducing worker NPC labour by default The user clearly moved away from worker NPCs because they are computationally expensive and unnecessary for a robotic civilisation. Future assistants should not casually propose a colony-sim labour model unless explicitly exploring an alternative.

Risk 3 - Making nanobots into magic Nanobots are useful as a construction interface, but they must be bounded by energy, material, time, heat, capability, storage, permissions, and deterministic validation. Otherwise they trivialise mining, construction, logistics, and industry.

Risk 4 - Letting the mothership trivialise progression The mothership can know recipes and

perform precision fabrication. It should not be an infinite free factory. Its limits are what force players to build civilisation.

Risk 5 - Confusing rendering with simulation

truth Unreal or any renderer can display local state, but the project's scale requires a deeper authoritative data model. If future design puts all truth into live engine actors, the architecture will likely collapse under scale.

Risk 6 - Hidden-state leaks through client

compute Client-shared compute sounds attractive, but a public MMO cannot give clients hidden fog-of-war state. Clients can help with rendering, visible generation, prediction, or verifiable jobs, not secret enemy or resource simulation.

Risk 7 - Over-scoping before proving the core loop

The project includes planets, civilisations, terraforming, factories, megaprojects, and MMO persistence. The immediate risk is trying to build all of it before proving the smallest fun loop.

7. Recommended Next Work

The best next step is to turn the proposed first vertical slice into a formal milestone document. That document should include goals, non-goals, required systems, data structures, success criteria, tests, risks, and scope limits.

Recommended order:

1. Define the core design pillars.
2. Specify the first vertical slice.
3. Define deterministic simulation invariants.
4. Draft the field-layer/sparse-delta model.
5. Define the mothership economy.
6. Define the initial robot chassis.
7. Define nanobot construction operations.
8. Define the first machine graph/process model.
9. Define fog-of-war sensor states.
10. Decide Unreal's role after current verification.
11. Create a master spec book chapter outline.
12. Compare this package with other old-chat reports and merge carefully.

The first vertical slice should be deliberately small. It should prove that the player can spawn as a robot, survey terrain, find resources, use blueprint/cut-fill construction, build power, mine/refine materials, fabricate parts, create a remote spawn lab, and persist/replay the result deterministically.

8. What Future Assistants Must Preserve

Future assistants should preserve the following without forcing the user to re-explain it:

- Dominion is being shaped as a robotic seed-civilisation game.
- Domino should probably be a deterministic sparse simulation core.
- Planets are generated from science-bounded parameters and editable field overlays.
- The mothership explains recipes, HUD, spawning, blueprints, and early fabrication.
- The mothership must be limited so local industry matters.
- Players are robot bodies/chassis, not normal humans.

- Remote spawn labs are physical expansion milestones.
 - Worker NPC labour is not the default city/construction model.
 - Automation, nanobots, AI agents, machine controllers, and process graphs are central.
 - Machines/factories should collapse into equations when not locally observed.
 - Fog of war is a data/security model, not only a visual layer.
 - Public MMO client compute must not reveal hidden state.
 - Unreal may be useful as client/tooling, but its role is not final.
 - The next practical task is a first vertical-slice spec.
-

9. Open Questions to Carry Forward

The following unresolved issues should remain visible:

1. Is the official scope one star system, multiple star systems, or a true galaxy-scale universe?

2. What exact scientific constraints define realistic planet generation?
3. How does reverse planet generation work?
4. What is the field-layer and sparse-delta schema?
5. How are designer/editor layers separated from gameplay-conserved layers?
6. What exact resources does the mothership start with?
7. How fast can the mothership fabricate, and what limits it?
8. What robot bodies exist at launch?
9. How does player identity, death, backup, and respawn work?
10. What can nanobots do at each technology tier?
11. How does the machine graph compiler validate player-created devices?
12. How do cities feel alive without worker NPCs?
13. What are the ownership, claims, permissions, and anti-griefing rules?
14. What exactly does fog of war hide, remember, reveal, or infer?

15. What client compute is safe in public MMO mode?
 16. What should Unreal do, and what should remain in Domino?
 17. What external data sources and licenses are needed for real solar-system recreation?
 18. What is the exact first vertical slice?
-

10. Short Human Summary

This conversation defined the foundations of Dominium as a deterministic robotic civilisation game. The first question asked how to support a vast procedural game with full-scale solar systems, planets, terraforming, cut/fill terrain editing, megaprojects, factories, fog of war, sparse simulation, single-player universes, and MMO persistence. The key answer was that this cannot be built as a normal giant Unreal map. It needs a deterministic data/simulation core. Domino should own true world state; a renderer should display only what the player can sense.

The user then gave the project its strongest identity. Humans sent a robotic mothership to another solar system. It lands, manufactures robot bodies, and begins rebuilding civilisation. Players spawn as robots from physical machines. They can choose different chassis and

later build remote spawn labs on the frontier. The mothership carries knowledge and recipes, so players do not reinvent basic technology, but it has limited materials and throughput, so players must build mines, refineries, factories, power systems, logistics, cities, and eventually megaprojects.

The user also clarified the world-generation philosophy. Star systems and planets should be procedurally generated from realistic scientific parameters, not arbitrary sliders. Designers should be able to nudge or override outcomes, and the same field/delta technology should support planet packs, save files, painted overrides, terraforming, cut/fill construction, and destruction.

A major design pivot was rejecting worker NPC labour as the main construction/city model. Since the game is about robots, the project does not need to simulate human workers doing jobs. It should use nanobots, drones, AI controllers, machine graphs, and process diagrams. Machines and factories can be visually rich when the player is nearby, but far away they should collapse into deterministic equations for input, output, energy, heat, throughput, buffers, reliability, and maintenance.

The main work still to do is specification. The project needs a first vertical slice, deterministic data model, field-layer schema, mothership economy, chassis list, nanobot construction rules, machine graph compiler, fog-

of-war system, multiplayer authority model, and Unreal integration decision. The best next action is to formalise the first playable loop: spawn from the mothership, survey terrain, find ore, use blueprints and cut/fill, build power, build a mine/refinery/fabricator, construct a remote spawn lab, and prove that the result is deterministic, persistent, and fun.

7.15 Dominion Complete Conversation Preservation Accompanying Human Readable Detailed Summary Report

Source: docs/archive/conversations/Dominium_Complete_Conversation/existing_complete_preservation_package/Dominium_Complete_Conversation_Preservation__10_accompanying_human_readable_detailed_summary_report.md

Report type: human_readable_detailed_summary

Size: 42353 bytes

7.15.1 Accompanying Human-Readable Detailed Summary and Report

Conversation label: Dominion Complete
Conversation Preservation

Primary design label: Dominion Deterministic Solar-
403

System / Robotic Seed-Civilisation Architecture

Date anchor: 2026-05-27 Australia/Melbourne

Source scope: This chat only, plus explicitly generated/exported files and the uploaded preservation prompt in this conversation.

Epistemic labels used: FACT, INFERENCE, UNCERTAIN / UNVERIFIED, PROJECT-CONTEXT.

0. What this companion report is

This is a standalone human-readable companion report for the current Dominion conversation. It is designed to sit beside the previously generated structured preservation package. The earlier package already contains a large formal report, context transfer packet, spec sheet, registers, aggregator packet, reader brief, verification/audit file, future-chat bootstrap prompt, in-chat reader, and ZIP archive. This companion report adds a narrative explanation of what happened in the conversation and checks that the exported files and artifacts have been preserved together in one bundle.

This report should be read as a summary of the whole visible conversation, not as a final design specification. Some items were clearly stated by the user and are marked as FACT. Some were assistant-proposed architecture or design synthesis and are marked as INFERENCE unless the user clearly accepted them.

Current-world claims about Unreal Engine, NASA/JPL data, network scale, or software capabilities remain **UNCERTAIN / UNVERIFIED** unless they are checked again with current sources before becoming formal project requirements.

1. Coverage and reliability

Row 1 Field: Apparent chat access; Assessment:
Partial but useful: visible conversation context and generated artifacts are available; hidden older project history is not assumed.

Row 2 Field: Uploaded source prompt; Assessment:
Present: **Pasted text.txt**, the maximum-fidelity preservation instruction.

Row 3 Field: Previously generated files; Assessment:
Present: two generated file sets were found in the workspace: a **Dominium_Deterministic_Solar_System_Architecture** set and a **Dominium_Robotic_Seed_Civilisation_Architecture** set.

Row 4 Field: Main design content covered; Assessment:
Yes: deterministic simulation, procedural planets, robotic mothership, robot players, spawn labs, nanobot construction, machine/factory abstraction, fog of war, MMO/private/single-player modes, Unreal role, vertical slice.

- Row 5** Field: Decisions present; Assessment: Yes, but many are design directions or assistant recommendations rather than final project decisions.
- Row 6** Field: Future work present; Assessment: Yes: vertical slice, data schemas, field layers, mothership economy, machine graph compiler, sensor/fog-of-war model, engine verification.
- Row 7** Field: Open questions present; Assessment: Yes: first vertical slice scope, realism bounds, official server scope, data models, robot chassis, nanobot tiers, machine compiler, Unreal role.
- Row 8** Field: Staleness risk; Assessment: Medium: engine/API/software/network claims require verification before formal use.
- Row 9** Field: Extraction confidence; Assessment: 4/5 for the visible conversation and files; lower for any unavailable prior project history.
- Row 10** Field: Safe for later aggregation; Assessment: Yes, with caveats and labels preserved.

Plain-language limitation: this package does not prove that every Dominion idea ever discussed elsewhere has been captured. It captures what is visible and what was generated in this chat. The files should be merged into a

future master spec only after cross-checking other reports and preserving uncertainty labels.

2. One-page orientation

This conversation was about turning the Dominion/Domino project into a coherent technical and creative architecture for an extremely ambitious deterministic game: a procedurally generated, scientifically bounded, solar-system-scale civilisation/construction simulation with editable planets, terraforming, machine building, factories, megaprojects, fog of war, sparse simulation, single-player universes, private servers, and possible MMO operation.

The original technical problem was scale. The user asked how a game could contain a full-scale real solar system or custom solar systems, fully recreated planets, procedural planets and universes, civilisations, terraforming, cut/fill, megaprojects, devices, machines, factories, and fog of war, while only loading and simulating what players can sense. The user also asked whether clients could share compute and whether Unreal could run this without lag, at low latency and high FPS, across many shards or private universes.

The first main architecture conclusion was that Dominion should not be built as a normal giant Unreal level. Instead, Domino should be treated as a

deterministic simulation and data engine. The universe should be stored as seeds, source data, procedural fields, sparse deltas, event logs, state hashes, snapshots, graphs, and ledgers. Rendering should be separate. Unreal, Raylib, or another engine can present local visible state, but the authoritative truth of the universe should not be ordinary engine actors or frame-rate-driven gameplay logic.

The conversation then shifted from pure engine feasibility to game identity. The user described a procedural star-system generator where planets are created from scientifically plausible parameters rather than arbitrary sliders. The designer should be able to nudge or override the result, and maybe even use reverse generation: design the desired planet, then let the system solve plausible orbital and planetary parameters that could produce it. The user also wanted procedural planet fields to be overlayable with painted/designer fields and to reuse the same underlying technology for saves, planet packs, terraforming, construction, and destruction.

The strongest creative identity that emerged was the robotic seed-civilisation premise. Humanity launched a robotic mothership to another solar system. The ship lands on the most habitable planet and manufactures robot bodies intended to restart civilisation. Players spawn as robot bodies from physical fabrication machines. They can later build remote spawn labs or

fabrication facilities that allow new players or respawns to appear at frontier locations. This ties respawn, progression, exploration, construction, and settlement expansion into one physical in-world economy.

A major design pivot was rejecting worker NPC labour as the core of city and construction simulation. The user identified labour/pathfinding as computationally expensive and unnecessary if the civilisation is explicitly robotic. Instead, construction and production should be handled by automation: nanobots, AI controllers, machine graphs, process diagrams, drones, logistics systems, and computer-managed factories. This is one of the most important ideas to preserve. *Dominium* should not become a human-worker city sim by default; it should be a robotic infrastructure and automation civilisation game.

The mothership explains why players know advanced recipes. The ship already contains human knowledge: CPUs, rail gauges, calendars, construction methods, circuits, refineries, and industrial design. The progression problem is not "invent a CPU from scratch." It is "build the industrial chain capable of locally making CPUs at scale." The mothership can fabricate some things, but its resources, energy, heat rejection, throughput, wear, and mission reserves must be limited so it cannot solve the whole game alone.

The best next step is not "build the full MMO." It is to

formalise and prototype the first vertical slice: one planet region, one mothership, one robot body, one ore deposit, one power source, cut/fill, blueprinting, nanobot construction, mine, refinery, fabricator, remote spawn lab, and fog-of-war sensor model. If that loop is fun and deterministic, the project can scale outward.

3. Chronological story of the conversation

3.1 The initial maximum-scope question The user began by asking whether Domino engine and Dominion could support a deterministic game with a full-scale real solar system, recreated planets, custom systems, procedural planets, player-built civilisations, terraforming, cut/fill, megaprojects, machines, devices, factories, fog of war, sparse simulation, client-shared compute, MMO persistence, private servers, and Unreal support. The user already framed the problem in terms of simulation load: only load and simulate what players can sense, collapse everything else, and make construction/destruction sparse.

3.2 First architecture response The assistant responded with an architecture-first answer. The main point was that the game should be built as a deterministic simulation/database first, with rendering as a client. It recommended fixed ticks, deterministic commands, seeded procedural generation, hierarchical

coordinates, sparse terrain deltas, field layers, simulation LOD, event logs, snapshots, state hashes, server authority, and fog-of-war filtering. The assistant also drew a scale boundary: millions of accounts or players spread across shards/regions is possible in principle, but millions in one fully simulated real-time bubble is not a practical target.

3.3 User refinement into a robotic civilisation

game The user then gave a long design monologue. The star-system generator should produce planets using real science. The designer should not manually paint everything and should not use arbitrary sliders. Sliders should be bounded to plausible physical ranges. There should also be a way to nudge results or possibly use reverse generation: specify the desired outcome and let the system infer plausible physical parameters.

The user also introduced the main lore and gameplay premise. Humans in the past created a robotic mothership that travels to another solar system, lands on a suitable planet, manufactures robots with advanced technology, and tries to restart civilisation. A future ship might bring human eggs or another payload, but the main gameplay does not depend on that being finalised.

The player is a robot. The player spawns from a physical machine. They may choose a chassis, such as biped, quadruped, spider, tank-like body, or other robot forms.

Early spawns come from the mothership. Later, players expand by building remote labs/fabricators that can manufacture bodies from local materials. This makes frontier spawn points an earned industrial milestone.

The user then moved into construction. Robots can justify HUDs, blueprints, task markers, recipe knowledge, and remote AI guidance. Nanobots or an invisible construction force can be used because they are easy to compute and can be explained diegetically. A player can blueprint a plan in the world, inspect it, and then commit it. Cut/fill and construction should be deterministic and non-destructive in the planning phase.

The user considered NPC workers but deprioritised them. Worker NPCs imply pathfinding, labour scheduling, collisions, and simulation complexity. The user's conclusion was that if computers and robots are doing the work, the game should admit that directly. The player should have automation from the beginning, even if advanced machines still require progression.

The user also explained why the mothership does not trivialise everything. It contains recipes and can synthesize some objects, but it has finite resources and limited throughput. It can help early players but eventually must be replenished with planet-derived materials and supported by local factories. The ship is a knowledge and precision bottleneck, not an infinite city printer.

3.4 Second synthesis response The assistant then synthesised the game as a robotic seed-civilisation design. It framed the project as: players are robot avatars; the mothership contains human knowledge, starter resources, spawn fabrication, and AI planning; players explore, survey, mine, refine, build cities, create spawn labs, automate industry, and eventually terraform or build megaprojects.

The assistant proposed several major systems: science-bounded forward/reverse planet generation, field layer stacks, mothership constraints, robot chassis, physical inventory, nanobot construction pipelines, no worker NPCs, machine graph compilers, easy-to-start/hard-to-advance cities, AI-planner quest markers, fog-of-war sensors, single-player/private/MMO modes, Unreal as a visual client, and a first vertical slice.

3.5 Preservation/export phase The user then uploaded a maximum-fidelity preservation prompt. That prompt requested a human-readable report first, then structured registers, a context transfer packet, a YAML-style spec sheet, an aggregator packet, a self-audit, downloadable files, a ZIP package, and a final in-chat reader. A preservation package was generated with 10 files plus a ZIP. In the workspace, two related file sets exist: one labelled `Dominium_Deterministic_Solar_System_Architecture` and one labelled `Dominium_`

Robotic_Seed_Civilisation_Architecture. This companion task now bundles those existing files, the uploaded prompt, this new report, and an updated package index into a single consolidated ZIP.

4. Main design topics preserved

4.1 Deterministic Domino simulation core FACT/

INFERENCE: The user wants determinism, sparse simulation, fog of war, and large-scale persistence. The assistant recommended that Domino be the deterministic authoritative simulation core, with rendering separated. This should be treated as a strong architecture recommendation, not necessarily a final engine decision until explicitly accepted and prototyped.

The core idea is that the universe is not a giant live map. It is a deterministic state transition system. Commands, seeds, source data, field layers, sparse deltas, graphs, ledgers, snapshots, and hashes reconstruct the world. Rendering converts currently sensed state into visible terrain, machines, robots, effects, and UI.

4.2 Full-scale solar systems and procedural planets

FACT: The user wants star systems/solar systems and planets generated from realistic parameters. The wording briefly used "galaxy," but the described scope was closer to custom solar systems with many planets.

This remains an open scope issue: whether the project should call the launch scope a star system, solar system, galaxy, universe, or multiverse.

The design should support physical parameters such as orbit, mass, radius, atmosphere, water, rotation, moons, climate, geology, and resource distribution. Sliders should be bounded by plausible science. The user wants designer control, but not arbitrary manual painting as the primary workflow.

4.3 Reverse generation and designer nudging

FACT: The user suggested a reverse process where a designer creates or describes the desired planet and the generator infers plausible parameters that fit it. This is an important tool idea. It lets designers say "I want a fun planet with these features" while still preserving scientific plausibility.

UNCERTAIN: The actual algorithm remains undecided. It could be constraint solving, search over parameters, evolutionary generation, simulated annealing, or manual tuning with plausibility feedback.

4.4 Field layers, saves, packs, terraforming, construction, and destruction

FACT: The user wants procedural fields overlaid with painted/designed data and wants the same technology to support save files, planet packs, and in-game terraforming. The

assistant formalised this as a field layer stack.

The key preservation point is that worldgen layers and gameplay layers should not be treated the same. A designer/editor layer can create a mountain or ocean as an authored override. A gameplay terraforming layer should conserve mass, energy, heat, time, permissions, and tool capability. This distinction prevents the save/packs system from turning into free in-game creation.

4.5 Robotic mothership lore and game loop

FACT: The user described a robotic mothership sent by humans to restart civilisation. The mothership lands on a habitable planet, manufactures robots, and contains knowledge and starter resources. Possible mystery lore around future humans/human eggs was mentioned but not finalised.

This premise is extremely useful because it explains many game systems at once: robot bodies, HUD, blueprints, recipe book, AI guidance, spawn fabrication, early tools, finite starting resources, and the need to build local industry.

4.6 Robot players and physical spawn economy

FACT: Players are robots, not humans. New players or respawns are manufactured by physical machines. Early bodies are fabricated at the mothership. Later bodies can be created at remote spawn labs built by players.

This changes the normal survival-game loop. Expansion is not only geographic; it is reproductive/infrastructural. Building a remote spawn lab means extending civilisation's ability to create new agents at the frontier.

4.7 Nanobot construction and blueprinting

FACT: The user proposed nanobots or an invisible force to do construction because it is computationally easy and fits robots. The assistant recommended bounding this by materials, energy, time, heat, sensor coverage, permissions, and tool capability.

The key implementation idea is a staged pipeline: preview, plan, reserve, stage, execute, commit, operate. The preview can be free and client-side; the committed build must be validated and deterministic.

4.8 Automation over worker NPC labour

FACT: The user explicitly moved away from worker NPC labour because it is expensive to compute and because a robotic civilisation does not need simulated human workers. This is one of the strongest design decisions from the conversation.

The preserved rule should be: do not make worker NPCs the default mechanism for construction, production, or city simulation. Use automation, AI agents, machine controllers, drones, construction swarms, logistics graphs, and process abstractions. Wildlife/mobs may

exist locally, but intelligent labour should be abstract unless deliberately added for a narrow reason.

4.9 Machines, factories, and process graphs

FACT: The user wants players to make machines, vehicles, rockets, devices, and factories, but wants the engine to summarise machines as process equations or functional objects wherever possible. The assistant called this a machine graph compiler.

This is central to performance and gameplay. A factory can be visually expanded when a player is nearby, but far away it should become a graph with inputs, outputs, throughput, energy use, heat, reliability, maintenance debt, and buffers. The same principle applies to player-created machines: the shape and parts create capabilities, but the server should compile the result into bounded functions.

4.10 Cities and progression **FACT/INFERENCE:**

The user wants cities to be easy to build but advanced industry to require city-scale infrastructure. The assistant proposed a tiered progression: mothership camp, outpost, settlement, industrial city, planetary infrastructure, terraforming civilisation, megaproject civilisation.

The important design point is that the basics should be accessible quickly. Advanced technology should require

materials, power, precision, logistics, fabrication chains, clean environments, cooling, and coordination.

4.11 Fog of war and sensed-only simulation

FACT: The original user question required fog of war and simulating/loading only what players can sense. The assistant formalised this as an information model: current observation, remembered state, inferred state, unknown state, and server-only hidden truth.

For MMO play, this is not just a rendering feature. A client must not receive hidden enemy bases, rare resources, secret terrain edits, or other unavailable truth. This directly limits client-shared compute.

4.12 Single-player, private universes, and MMO

FACT: The user wants the game to support single-player multi-universe play and possibly MMO/private server play. The assistant recommended using the same core rules across modes, with trust and authority varying by mode.

Single-player can be local and more relaxed. Private servers can allow custom systems and mods. Official MMO mode should be strict: server authority, filtered fog-of-war data, auditability, anti-cheat, and careful limits on client compute.

4.13 Unreal's role INFERENCE/UNCERTAIN: The assistant recommended Unreal as a renderer/client/tooling layer rather than the authoritative universe engine. This is a strong architecture recommendation, but it remains unverified as a project decision. Future work should verify current Unreal features and decide how much of the client/runtime should be Unreal versus Domino custom code.

4.14 First vertical slice INFERENCE: The assistant recommended a first vertical slice: one planet region, one mothership, one robot chassis, one resource deposit, one power source, cut/fill, blueprinting, nanobot construction, mine, refinery, fabricator, remote spawn lab, and fog-of-war sensor system. This has not been explicitly approved as final, but it is the best candidate next milestone because it tests the central gameplay loop without requiring full planet/MMO scale.

5. Decisions and status

ID	Decision or design direction	Status	Label	Confidence
DECISION-01	Dominium should use deterministic final sparse simulation rather than full dense world simulation.	Strong direction; implementation details open	FACT + INFERENCE	4/5
DECISION-02	Planet generation should be scientifically bounded, not arbitrary.	User-stated direction	FACT	5/5
DECISION-03	Designer override/painted fields should layer over procedural generation.	User-stated direction; format undecided	FACT	4/5
DECISION-04	Save/pack/terraforming/construction systems should share	User-stated direction plus assistant formalisation	FACT + INFERENCE	4/5

The main caveat is that DECISION-13 through DECISION-15 are not direct user decisions. They are strong recommendations that follow from the conversation, but they should be confirmed before becoming binding.

6. What was deferred or put off for later

Several important subjects were not fully specified and should remain open:

1. **Exact launch scope:** The user used broad language around solar systems, galaxies, universes, and multiverses. The first official scope still needs to be chosen.
2. **Planet generator schema:** The exact physical parameters, realism scoring, and solver mechanics remain undefined.
3. **Field/delta data format:** The project needs concrete schemas for procedural fields, painted overlays, saved deltas, terraforming, construction, destruction, and mod packs.
4. **Mothership economy:** Starting reserves, fabricator throughput, spawn costs, mission reserves, heat limits, replenishment rules, and failure/recovery modes need design.

5. **Robot chassis:** Candidate bodies were discussed, but launch chassis, stats, hardpoints, locomotion, inventory, and progression remain open.
6. **Nanobot tiers:** The ability set, range, throughput, energy cost, and construction rules remain open.
7. **Machine graph compiler:** The system that turns player-built machines into validated capabilities is a major unresolved technical design.
8. **City liveliness without workers:** The project needs ways to make automated cities feel alive through drones, logistics, lights, traffic, machine sound, sensors, screens, and infrastructure feedback.
9. **Fog-of-war sensor model:** Current observation, remembered maps, inferred maps, map sharing, survey tools, satellites, and hidden state rules need definition.
10. **Client-shared compute:** Public MMO, private server, and single-player trust boundaries need separate rules.
11. **Unreal integration:** Current Unreal capabilities and limits need verification before committing to an Unreal-heavy pipeline.

12. **Vertical slice:** The proposed first playable loop needs a formal milestone document.
13. **Governance, permissions, and griefing:** Claims, ownership, terraforming rights, destruction permissions, salvage rules, decay, and anti-lag budgets were identified but not designed.
14. **Versioning and migration:** Ruleset versions, generator versions, schema versions, mod versions, and old-save compatibility need design.
15. **Verification:** Current technical claims require fresh checking before inclusion in a formal spec.

7. Rejected, superseded, or deprioritised ideas

ID	Idea	Status	Why preserved
REJECTED-01	Manual painting as the primary planet creation method	Deprioritised	The user wants procedural science-bounded generation first, with painting as override.
REJECTED-02	Arbitrary unrealistic planet sliders	Deprioritised	The user wants realistic bounds and plausibility.
REJECTED-03	Worker NPC labour as the construction/city core	Strongly deprioritised	It creates pathfinding/scheduling cost and conflicts with robotic automation premise.
REJECTED-04	Simulating every machine part everywhere	Rejected as infeasible	Machine functions should collapse into process graphs/equations.
REJECTED-05	Treating Unreal Actors as every piece of authoritative world truth	Assistant-rejected	Too heavy and not appropriate for deterministic games

These are important because future chats may accidentally reintroduce them as defaults. The most important rejected/default trap is worker NPC labour.

8. Key tradeoffs

8.1 Scale versus fidelity The project wants full planets, cities, machines, factories, and possibly MMO scale. That cannot be done with dense active simulation everywhere. The proposed solution is multi-resolution simulation: active local fidelity near players, aggregate process graphs for distant systems, sparse deltas for terrain/building edits, and lazy fast-forward for unobserved systems.

8.2 Realism versus designer control The user wants realistic generation, but also wants fun and designer control. The implied solution is bounded sliders, plausibility scores, and explicit override layers.

8.3 Convenience versus progression Nanobots, blueprints, and mothership recipes make construction convenient. Finite materials, power, heat, throughput, logistics, and machine capability preserve progression.

8.4 Creativity versus exploit prevention Players should create machines and megaprojects. But arbitrary

designs can become server-load exploits or economy exploits. A machine compiler and construction complexity budgets are needed.

8.5 Client compute versus hidden information

Clients can help with rendering, mesh generation, local prediction, and visible/verifiable simulation. They cannot be trusted with hidden MMO truth, secret resources, enemy bases, or economy-critical state.

9. Artifact and file inventory explanation

Two generated file sets are present:

1. `Dominium_Deterministic_Solar_System_Architecture__...`

This is the latest full preservation package created after the uploaded prompt. It contains a manifest, human-readable report, context packet, spec sheet, registers, aggregator packet, reader brief, verification/audit file, future-chat bootstrap prompt, in-chat reader, and ZIP archive.

1. `Dominium_Robotic_Seed_Civilisation_Architecture__...`

This appears to be an earlier or alternate generated preservation file set focused on the robotic seed-

civilisation framing. Because it exists in the workspace and is relevant, it is preserved in the consolidated ZIP rather than discarded.

1. Pasted `text.txt`

This is the uploaded maximum-fidelity preservation prompt. It is preserved as source context because it defines the required output structure and preservation standards.

1. This new companion report

This file adds a readable narrative explanation and checks that the current package preserves the conversation and generated artifacts.

1. Updated consolidated package index

A new index file describes what is in the consolidated ZIP and how to use it.

1. New consolidated ZIP

The final ZIP bundles the previous generated files, prior package variant, source prompt, this companion report, and the updated index into one archive.

10. Open questions and unresolved issues

The highest-priority open questions are:

1. What is the first formal vertical slice?

Recommended answer: mothership to remote spawn lab, but the user should confirm.

1. What is the exact data model for fields and sparse deltas?

This affects planets, saves, packs, terraforming, cut/fill, construction, and destruction.

1. What exactly are the mothership's limits?

These limits are the foundation of progression.

1. What are the initial robot chassis?

This affects locomotion, inventory, construction capability, exploration, and identity.

1. How do machines compile into simulation graphs?

This is central to performance and player creativity.

1. How does fog of war work mechanically and securely?

It affects sensors, map data, client replication, hidden enemies/resources, and MMO trust.

1. What can clients compute?

Single-player, private servers, and official MMO need different trust models.

1. What should Unreal be used for?

The project needs a verified decision on Unreal as renderer/tooling/client versus deeper runtime integration.

1. How do cities feel alive without worker NPCs?

The automation-first model needs strong visual and systemic feedback.

1. What governance and anti-griefing systems exist?

Terraforming, construction, destruction, spawn labs, and megaprojects require claims, permissions, decay, and moderation rules.

11. Risks and failure modes

Row 1 Risk: Scope explosion; Why it matters: Full solar systems, MMOs, machines, cities, and terraforming could overwhelm development.; Mitigation: Build the vertical slice first.

Row 2 Risk: Treating brainstorming as final; Why it matters: User's exploratory ideas could be over-promoted into hard requirements.; Mitigation: Preserve labels and confirmation status.

Row 3 Risk: Losing the automation pivot; Why it matters: Future designs might reintroduce worker NPC labour.; Mitigation: Mark no-worker-NPC labour as a core design direction unless explicitly revisited.

Row 4 Risk: Nanobots becoming magic; Why it matters: Free construction would destroy progression and economy.; Mitigation: Require material, energy, time, heat, capability, permissions, and logistics.

Row 5 Risk: Mothership trivialising industry; Why it matters: Infinite high-end fabrication erases the game loop.; Mitigation: Limit reserves, throughput, heat, wear, scale, and mission protocols.

Row 6 Risk: Machine designs becoming lag machines; Why it matters: Player creativity can weaponise

simulation cost.; Mitigation: Use compiler validation and complexity budgets.

Row 7 Risk: Client compute leaking secrets; Why it matters: MMO fog-of-war can fail if hidden data is sent to clients.; Mitigation: Server-authoritative truth and filtered observation packets.

Row 8 Risk: Unreal overreach; Why it matters: Treating Unreal as the whole universe engine may create deterministic and scale problems.; Mitigation: Keep Domino authoritative unless proven otherwise.

Row 9 Risk: Empty automated cities; Why it matters: No worker NPCs may make cities feel lifeless.; Mitigation: Use drones, moving logistics, sound, lights, machine activity, sensor overlays, traffic abstractions.

Row 10 Risk: Stale technical claims; Why it matters: Engine/network/API data may change.; Mitigation: Put external claims in verification queue.

Row 11 Risk: Aggregation errors; Why it matters: Later master spec may merge conflicting reports incorrectly.; Mitigation: Use artifact IDs, labels, and manual review.

12. Verification queue

Before formalising the technical spec, verify:

- Current Unreal Engine large-world, World Partition, PCG, MassEntity, Iris/Replication, runtime mesh, and deterministic limitations.
- Current licensing and usage conditions for NASA/JPL/Horizons/SPICE/IAU or any real solar-system datasets.
- Deterministic math strategy across platforms.
- Viability of sparse terrain deltas with planetary coordinate systems.
- Network/server authority strategy for fog-of-war and persistent MMO regions.
- Feasibility of a machine graph compiler and performance budgets.
- Storage and replay strategy for large persistent universes.
- Mod/planet-pack security and versioning.

13. Recommended next-action sequence

1. Confirm the design pillars.

Draft 5-8 pillars and mark which are hard requirements versus guiding principles.

1. **Define the first vertical slice.**

Write a milestone for the mothership-to-remote-spawn-lab loop.

1. **Specify the deterministic core invariants.**

Define tick model, command log, world hash, save/load, replay, and deterministic RNG.

1. **Specify field/delta data structures.**

Define terrain/material/climate/resource fields, overlays, edit logs, and conservation rules.

1. **Design the mothership economy.**

Define starting materials, spawn costs, fabrication throughput, energy, heat, wear, and replenishment.

1. **Design the first robot chassis.**

Pick two or three bodies that test locomotion, construction, cargo, and exploration.

1. **Design nanobot construction tiers.**

Define what early nanobots can cut, fill, assemble, repair, and survey.

1. Draft the machine graph compiler.

Define machine modules, ports, inputs, outputs, throughput, heat, reliability, and validation.

1. Draft the fog-of-war model.

Define sensors, current/remembered/inferred/unknown state, and hidden server truth.

1. Verify Unreal integration assumptions.

Decide what Unreal does, what Domino does, and where the boundary is.

14. What to preserve above all

- Dominium's identity as a **robotic seed-civilisation game**.
- Domino's likely role as a **deterministic sparse simulation/data engine**.
- The rule that **worlds are fields, graphs, ledgers, sparse deltas, event logs, and snapshots**, not just giant maps.

- The user's strong preference to **avoid worker NPC labour** and use automation instead.
- The mothership as **knowledge, spawn source, finite precision fabricator, and early bottleneck**.
- The physical spawn economy: **remote spawn labs are built infrastructure**.
- The construction model: **blueprint, reserve, stage, execute, commit**.
- The machine model: **visual locally, aggregate/process-graph remotely**.
- The fog-of-war rule: **do not send hidden truth to clients in public MMO mode**.
- The immediate practical milestone: **prove the mothership-to-remote-spawn-lab loop**.

15. Best questions to ask next

Understanding

1. What is the shortest accurate description of Dominion after this conversation?
2. Which parts are user-stated FACT and which are assistant INFERENCE?

3. What changed when the user moved away from worker NPC labour?

Decisions

1. Should "automation over worker NPC labour" become a hard project pillar?
2. Should "Domino owns truth; Unreal renders local state" become a formal architecture rule?
3. Should the official launch scope be one designed star system rather than galaxy-scale?

Tasks

1. Turn the first vertical slice into a formal milestone plan.
2. Draft the field/delta data model.
3. Draft the mothership economy.
4. Draft the machine graph compiler.

Artifacts

1. Which of the two preservation file sets should be treated as canonical?
2. Merge the two file sets into one master report without duplication.

Risks and verification

1. What current Unreal claims need verification before we commit?
2. How do we stop nanobot construction from breaking progression?
3. How do we make automated cities feel alive without worker NPCs?

Spec-book integration

1. Which future master spec chapters should this chat feed into?
2. Which recommendations need user confirmation before becoming requirements?
3. What cross-chat conflicts should the aggregator look for?

16. Compact few-minute summary

This conversation established a major design direction for Dominium. The project should not be thought of as a normal open-world map with a huge number of objects. It should be a deterministic, sparse, data-driven universe simulation. The world is reconstructed from seeds, procedural fields, overlays, sparse deltas,

command logs, graphs, ledgers, snapshots, and hashes. Rendering engines such as Unreal can show the local visible area, but they should not be assumed to own the authoritative world truth.

The user wants procedurally generated solar systems and planets that are scientifically bounded. Designers should set realistic physical parameters and nudge results rather than manually painting entire planets or using arbitrary fantasy sliders. The user also wants a possible reverse-generation workflow: specify the planet desired, then let the generator search for plausible orbital/geophysical parameters. Procedural fields should be layerable with painted fields, and the same field/delta technology should support saves, planet packs, terraforming, construction, and destruction.

The game premise became much clearer. Humans launched a robotic mothership to another solar system. The mothership lands on a habitable planet and manufactures robot bodies to restart civilisation. Players are robots. They spawn from physical fabrication machines, choose different robot chassis, and later build remote spawn labs so civilisation can expand across the planet. This premise explains the HUD, recipe knowledge, blueprints, AI guidance, respawn, construction, and why the player starts with advanced knowledge but not unlimited industrial capacity.

The mothership is not an infinite factory. It has finite

reserves, limited throughput, energy limits, heat limits, wear, scale limits, and mission protocols. It can provide recipes and precision fabrication, but players must eventually mine, refine, build power, create factories, construct cities, and replenish the mothership. The ship is excellent at knowledge and precision; the planet is required for scale.

The strongest design pivot was away from worker NPC labour. The user considered workers, but rejected or deprioritised them because they require pathfinding, scheduling, physics, collision, and expensive simulation. Since the civilisation is robotic, work should be done directly by automation: nanobots, AI controllers, drones, machine graphs, logistics systems, and process diagrams. Cities should be easy to start and can be automated from the beginning, while advanced industry still requires complex infrastructure.

Construction should use blueprints and nanobots. Players can preview a plan for free, then commit it when materials, energy, permissions, tool capability, time, heat, and logistics are available. Cut/fill, terraforming, and construction should be deterministic and saved as sparse deltas. Nanobots provide the convenient interaction fantasy, but they must not become magic.

Factories and machines should be visually rich near the player but collapse into process graphs when distant. A player-built machine should compile into a functional

object with inputs, outputs, power draw, heat output, throughput, buffers, reliability, and maintenance debt. This allows player creativity without simulating every gear, belt, item, or robot everywhere in the universe.

Fog of war matters both for gameplay and multiplayer security. The game should only load and simulate what players can sense at high fidelity. In public MMO mode, clients should not receive hidden truth. They may help with rendering, local prediction, visible mesh generation, or verifiable non-secret jobs, but they should not compute hidden enemy bases, secret resources, or authority-critical economy/combat outcomes.

The main unfinished work is formal specification. The best next action is to define the first vertical slice: one planet region, one mothership, one robot body, one resource deposit, one power source, cut/fill, blueprinting, nanobot construction, a mine, refinery, fabricator, remote spawn lab, and fog-of-war sensors. That loop proves the core identity of the game before attempting full planets, megaprojects, or MMO scale.

17. Final check performed

I checked the workspace for generated files and found two relevant preservation sets plus the uploaded source prompt. I created this new companion report, an updated consolidated package index, and a single ZIP

containing the existing files plus the new materials. The package is intended to be readable by humans and useful for later aggregation.

7.16 Dominion Conversation Complete Accompanying Report Accompanying Human Readable Summary And Report

Source: docs/archive/conversations/Dominium_Complete_Conversation/new_companion_report_and_bundle/Dominium_Conversation_Complete_Accompanying_Report__10_accompanying_human_readable_summary_and_report.md

Report type: accompanying_human_readable

Size: 30887 bytes

7.16.1 Accompanying Human-Readable Summary and Report - Dominion Conversation Complete Bundle

Chat label: Dominion Conversation Complete
Accompanying Report

Date anchor: 2026-05-27 Australia/Melbourne

Source scope: This conversation only, plus the files
already generated inside this conversation and the

uploaded preservation prompt.

Purpose: This report accompanies the earlier handoff package. It explains the whole conversation in plain language, including what was discussed, what was decided or only recommended, what was deferred, what files were produced, and how the final bundle should be used.

0. Coverage and reliability

This companion report covers the visible conversation about **Dominium** and the **Domino engine**, including the earlier architectural answers, the user's long design monologue about the robotic mothership and automation-first civilisation, the uploaded preservation prompt, the previously generated preservation package, and the current request to create an additional human-readable report and bundle everything into one ZIP.

Access is **partial but sufficient for this task**. I can use the visible conversation and the files present in the workspace. I cannot claim access to hidden project history, older chats not present here, or private external documents. When this report says **FACT**, it means the idea was directly stated or clearly established in this visible conversation. When it says **INFERENCE**, it means the idea is a reasonable interpretation or assistant synthesis, but not necessarily a final user-approved

decision. When it says **UNCERTAIN** / **UNVERIFIED**, it means the item needs future checking before it becomes a specification or implementation claim.

The main limitation is that several earlier assistant responses included technical claims about Unreal, NASA/JPL data sources, MMO scale, and distributed simulation. Those claims were useful for framing, but they should be verified against current documentation before becoming binding technical requirements. The conversation itself also contains many brainstorms. These should not be upgraded into final product decisions without explicit user confirmation.

1. Plain-language overview

This conversation was about defining the foundations of **Dominium**, a very large-scale procedural civilisation and construction game that would run on or alongside a deterministic engine called **Domino**. The initial user question asked how to build a deterministic game with a full-scale real solar system, procedurally generated planets, custom solar systems, terraforming, terrain cutting and filling, player-built civilisations, megaprojects, machines, devices, factories, fog of war, sparse simulation, single-player universes, private servers, and an MMO-like official persistent universe. The user also asked whether Unreal Engine could make

that possible without lag, at low latency and high frame rate, even with very large numbers of players spread across shards or private universes.

The early architectural conclusion was that *Dominium* cannot be treated as a normal "giant Unreal map." The world must be a **deterministic data system first**. The important state should live in Domino as seeds, parameters, fields, sparse deltas, event logs, snapshots, hashes, resource ledgers, machine graphs, and visibility records. Unreal, Raylib, or any other visual engine should be treated as a renderer/client/tooling layer that asks Domino what the player can currently see, sense, edit, or command. This does not mean Unreal is useless. It means Unreal should not be the source of authoritative planetary truth.

The user then refined the project from a pure technical feasibility question into a much clearer game identity. *Dominium* became a **robotic seed-civilisation game**. The user described a procedural star system or "galaxy" generator where planets are created from plausible scientific parameters rather than arbitrary art sliders. Designers should be able to choose large physical parameters, let the system produce a planet, then nudge or override results. There might also be a reverse-generation workflow: the designer specifies the desired world, such as gravity, continents, atmosphere, oceans, moons, and resource distribution, and the generator

searches for plausible orbital and planetary parameters that could produce it.

A major user-stated idea was that procedural generation, designer overrides, save files, world packs, terraforming, cut/fill construction, and destruction should all use a related field/delta technology. The planet should have a procedural base, then layers on top: real data if used, designer edits, mod or pack data, player terraforming, construction, damage, and temporary blueprint previews. This matters because it avoids building separate incompatible systems for worldgen, saves, mods, and gameplay editing.

The strongest lore/gameplay premise was the robotic mothership. In the backstory, humans sent a robotic seed ship to another solar system. The mothership lands on the most habitable planet and manufactures robot bodies using advanced technology, possibly to restart civilisation and prepare the planet for later human biological payloads. Players are not ordinary humans. They spawn as robots from physical fabrication machines. This explains the HUD, blueprint overlays, recipe knowledge, AI guidance, respawn, machine bodies, and high-strength manipulation.

Spawn points are not just menu abstractions. They are physical machines that need materials, power, feedstock, and permissions. Early players spawn from the mothership. Later players can spawn at frontier labs

that existing players build. This turns expansion into a concrete gameplay milestone: to move civilisation outward, players must survey terrain, find resources, mine, refine, power infrastructure, fabricate parts, transport them, and construct new spawn labs.

The conversation also established a major anti-pattern: **do not build the game around worker NPC labour**. The user explicitly called out labour NPCs as expensive because they require pathfinding, scheduling, collision, animation, and constant simulation. Since the civilisation is robotic, the better solution is to use automation from the start. Construction and production can be done by nanobots, AI agents, drones, machine controllers, process diagrams, and factory graphs. Cities do not need thousands of simulated workers to be real; they need power, logistics, sensors, fabricators, spawn labs, resource flows, ownership, permissions, and infrastructure.

Nanobots and blueprinting became the proposed interaction model for construction. Players can preview plans diegetically in their robotic HUD, reserve or gather the required materials, then commit deterministic cut/fill/build operations. Nanobots are useful because they avoid the need to physically simulate every worker and tool movement. However, they must not be free magic. They should require material, energy, time, heat dissipation, tool capability, sensor coverage, permissions,

and possibly swarm capacity or maintenance.

Factories and machines were treated similarly. Players should be able to design machines, vehicles, rockets, and devices, but the engine should not simulate every internal part everywhere forever. Instead, a machine or factory should be represented locally in detail when the player can see or interact with it, and collapsed into an input/output process graph when distant. A factory far away might be simulated as iron ore plus carbon plus electricity producing steel, slag, heat, wear, and maintenance debt. When a player returns, Unreal can render belts, arms, items, and sparks consistent with that aggregate state.

The final preservation work then turned this conversation into a structured handoff package. The previously generated package included a main human-readable report, context transfer packet, spec sheet, registers, aggregator packet, reader brief, audit, bootstrap prompt, in-chat reader, and ZIP. This new companion report adds a plain-language overview of the whole conversation and bundles the existing files with this new report and a combined manifest.

2. What was actually discussed

2.1 Deterministic simulation and sparse world

state **FACT:** The user wanted determinism,

procedural worlds, sparse construction/destruction, fog of war, and sensed-only simulation.

INFERENCE: The correct architecture is a deterministic Domino simulation core with renderer clients.

Reason: A planet-scale or solar-system-scale game cannot store and simulate everything at full fidelity. The world must be represented as procedural base data plus sparse changes. A full mountain that no one has modified should be regenerated from seed and planet parameters. A mined tunnel, building foundation, or megaproject should be stored as deltas, ledgers, and graphs.

The important technical direction is: **store what matters, regenerate what can be regenerated, and only expand detail where causality or player perception requires it.** This applies to terrain, factories, machines, cities, wildlife, weather, and civilisation processes.

2.2 Full-scale solar systems and planet generation

FACT: The user wanted procedural star systems/planets bounded by realistic science, not arbitrary sliders.

FACT: The user wanted designers to be able to tweak, nudge, override, and perhaps reverse-generate planets.

UNCERTAIN: Whether launch scope is one official star system, many systems, or a true galaxy remains unresolved.

The discussed generator should expose physical parameters such as star type, orbit, eccentricity, axial tilt, mass, radius, gravity, atmosphere, water inventory, moons, climate, resources, and geology. The designer should not have to manually paint every mountain, biome, and ocean, but should be able to guide the generated result.

The reverse-generation idea is important. Instead of only saying "given this orbit, produce a planet," designers could say "I want this kind of planet," and the generator tries to find plausible parameters that produce it. The result should probably include plausibility scoring, warnings, and sci-fi override labels.

2.3 Field layers, saves, packs, and terraforming

FACT: The user wanted procedural fields overlaid with painted/designer data, and wanted the same technology to support saves and planet packs.

INFERENCE: The best model is a layer stack of procedural fields, override fields, mod fields, terraforming fields, construction fields, and damage fields.

This is one of the most reusable ideas in the conversation. The same basic system can represent

worldgen, manual design, player edits, save files, terraforming, and destruction. The key caveat is that not all layers obey the same rules. A designer painting a mountain for a custom planet pack does not need to conserve mass. A player raising terrain during gameplay should consume regolith or fill material, use energy, produce heat, and alter drainage or structural conditions.

2.4 Robotic mothership and player spawning

FACT: The user proposed a robotic mothership sent by humans to restart civilisation.

FACT: Players spawn as robots from physical fabrication machines.

FACT: Remote spawn labs become expansion milestones.

UNCERTAIN: The exact lore around human eggs, later ships, or robot autonomy remains open.

The mothership solves many design problems elegantly. It explains why players know recipes, why they have a HUD, why they can access blueprints, why respawn exists, and why players can begin with advanced knowledge without personally inventing CPUs, rail gauges, or calendars. It also gives the game a central progression constraint: the mothership can make some things, but not infinite infrastructure.

The best summary from the conversation is: **the**

mothership is excellent at knowledge and precision, but the planet is required for scale. The ship can start civilisation but cannot replace civilisation.

2.5 Robot chassis and inventory **FACT:** The user mentioned possible robot forms including quadruped, biped, spider, and tank-shaped robots.

INFERENCE: Body types should be chassis with different physical capabilities rather than fantasy classes.

Robotic bodies allow more generous inventory than human characters, but inventory should remain physical enough to preserve logistics. A robot can carry tools, modules, and some cargo, but tonnes of ore, steel beams, or reactor assemblies should require vehicles, containers, pallets, conveyors, pipes, drones, rails, or other logistics infrastructure.

2.6 Nanobots, blueprinting, cut/fill, and construction **FACT:** The user proposed nanobots or an invisible force as a computationally cheap diegetic construction system.

FACT: The user wanted blueprint stages and non-destructive planning before committing work.

INFERENCE: The right pipeline is preview -> plan -> reserve -> stage -> execute -> commit -> operate.

This makes construction pleasant while preserving

deterministic world state. Previewing a blueprint can be client-side and free. Committing it should produce authoritative deltas and resource ledger changes. Cut/fill should remove or add material, consume energy, generate heat/waste/dust, and update terrain and structural support.

2.7 Automation instead of worker NPCs FACT:

The user explicitly moved away from worker NPC labour because labour is expensive to compute.

FACT: The user preferred automation from the beginning.

Importance: This is one of the clearest user-driven design pivots in the conversation.

Future assistants should not keep proposing human-like worker economies as the default. Dominion's civilisation should be robotic and infrastructural. The living feeling of a city should come from machines, drones, logistics, construction queues, sensors, power grids, factories, communication relays, weather effects, player activity, and visible resource flows-not from simulating thousands of worker NPCs.

2.8 Machines and factory graphs FACT: The user wanted machine assemblies and vehicles to be summarised as functional equations where possible.

INFERENCE: Domino should contain a machine graph compiler that validates player designs and converts them into capabilities, rates, costs, heat, reliability, and failure modes.

This is essential for both performance and creativity. If every player-designed machine remains a pile of individually simulated moving parts forever, the game will collapse. If machines are too abstract, players lose engineering creativity. The middle path is to let players assemble meaningful modules, then compile the result into a deterministic process graph.

2.9 Cities and progression **FACT:** The user wanted cities to be easy to build at the basic level but hard to make advanced.

INFERENCE: City progression should be about industrial capability rather than worker population.

A basic city might need power, storage, roads, material handling, simple fabrication, and a spawn lab. An advanced city would need precision manufacturing, clean rooms, heat management, rare materials, high-grade power, high-throughput logistics, sensors, governance/permissions, and maintenance.

2.10 Fog of war and sensed-only simulation

FACT: Fog of war and loading/simulating only what

players can sense were part of the original user requirement.

INFERENCE: Fog of war must be an information-security model, not merely a screen effect.

The client should not receive hidden enemy bases, undiscovered resources, secret tunnels, or unseen construction. This is especially important if clients help compute anything. Clients can safely generate visible meshes or predict local motion, but they cannot be trusted with hidden authoritative truth in a public MMO.

2.11 MMO, private servers, and Unreal FACT:

The user wanted single-player, multiple instantiated universes, private servers, and official MMO possibilities.

INFERENCE: Public MMO mode requires server authority, interest management, region/cell ownership, and strict fog-of-war filtering.

UNCERTAIN / UNVERIFIED: Current Unreal capabilities and network limits must be checked before finalising the implementation plan.

The conversation's practical boundary is that millions of players can share a logical universe across regions/shards, but not all be in one high-fidelity, low-latency simulation bubble. Unreal can help with visuals, local world streaming, editor tooling, and client-side

presentation. It should not be assumed to solve deterministic planet-scale simulation, MMO server meshing, or hidden-state security by itself.

3. What was decided, recommended, or left tentative

The conversation produced a mixture of user-stated decisions, assistant recommendations, and open design questions. The following distinction matters.

3.1 Strong user-stated directions

- **Dominium should use procedural star/planet generation bounded by realism.** This came directly from the user.
- **Planets should support procedural fields plus overrides.** The user wanted painted fields, planet packs, saves, and terraforming to relate.
- **The game premise should involve a robotic mothership and manufactured robot players.** The user explicitly described this lore and spawn model.
- **Players should be robot bodies, not ordinary humans.** The user listed chassis examples.
- **The mothership should provide knowledge and recipes but have finite materials and**

throughput. The user explicitly worried about the ship making everything and proposed limits.

- **The game should lean toward automation instead of worker NPCs.** This was a clear design pivot.
- **Factories/machines should be able to collapse into input/output process diagrams.** The user explicitly described this.

3.2 Strong assistant recommendations not automatically final

- **Domino should be the deterministic authoritative core.** This follows from the user's goals but should still be formally confirmed.
- **Unreal should be a renderer/client/tooling layer, not the authoritative universe engine.** This is a technical recommendation needing future verification and user acceptance.
- **Use a field layer stack and sparse delta/event-log model.** This formalises the user's idea but still needs actual schema design.
- **Use a vertical slice focused on mothership**
-> ore -> power -> mine -> refinery ->
fabricator -> remote spawn lab. This is the

best next step recommended by the assistant, not yet a user-approved milestone.

- **Use server authority and restrict client compute in public MMO mode.** Strong security recommendation, still to be specified.

3.3 Main unresolved items

- Exact launch scope: one star system, many systems, or galaxy-scale.
- Exact realism level and sci-fi override policy.
- Exact field/delta data model.
- Exact mothership reserves, throughput, heat, wear, and mission-reserve rules.
- First robot chassis and their gameplay roles.
- Exact nanobot capabilities and limitations.
- Machine graph compiler design.
- How cities feel alive without worker NPCs.
- Fog-of-war sensor states and map sharing rules.
- MMO authority, shard/region/cell boundaries, and client compute policy.
- Final role of Unreal or any renderer.
- First vertical slice milestone definition.

4. What we did in this conversation

The conversation produced several kinds of output.

First, it produced a **technical architecture framing**: deterministic Domino simulation core, renderer/client separation, sparse worlds, hierarchical coordinates, simulation LOD, fog-of-war secrecy, server authority, and caution around Unreal as a full solution.

Second, it produced a **creative/gameplay premise**: robotic mothership seed-civilisation, manufactured robot bodies, physical spawn points, frontier spawn labs, finite mothership fabrication, advanced knowledge without free industrial scale, and automation-first construction.

Third, it produced a **systems design frame**: science-bounded planet generation, reverse-generation possibility, field layers, cut/fill, nanobot blueprint construction, machine process graphs, factory aggregation, city progression, and sensor-based fog of war.

Fourth, it produced a **preservation package** from the uploaded prompt. The existing package includes a main human-readable report, context transfer packet, YAML-style spec sheet, structured registers, aggregator packet, reader brief, verification/audit, bootstrap prompt, in-chat reader, and ZIP.

Fifth, this current step produced an **accompanying**

report and a **combined bundle** that packages the existing generated files, the original uploaded preservation prompt, and this new companion summary into one ZIP.

5. What we postponed for later

The conversation deliberately did not solve implementation details. The following were deferred:

1. **Formal deterministic core spec.** The chat established the need, but not the final tick model, math rules, command schema, state hashes, or replay format.
2. **Planet generator schema.** The chat described science-bounded generation but did not define the parameter list, constraints, plausibility model, or reverse solver.
3. **Field/delta format.** The chat proposed the concept but did not define storage layouts, operations, conservation rules, chunking, or conflict resolution.
4. **Mothership economy.** The chat established finite resources and throughput limits, but did not decide the actual resource budgets or fabrication rules.

5. **Robot chassis design.** The chat listed examples but did not specify stats, tradeoffs, manufacturing costs, or upgrade paths.
6. **Nanobot construction rules.** The chat described blueprinting and cost constraints but not exact equations or tech tiers.
7. **Machine graph compiler.** The chat identified the need but did not define module libraries, validation rules, or exploit controls.
8. **City progression.** The chat suggested infrastructure-first cities but did not define city systems in detail.
9. **Fog-of-war sensor model.** The chat established sensed-only simulation but did not specify sensor types, visibility states, map sharing, or deception.
10. **MMO architecture.** The chat discussed server authority, shards, and client compute constraints, but not final network architecture.
11. **Unreal integration.** The chat recommended Unreal as client/renderer, but current documentation and prototypes need verification.
12. **First vertical slice.** The chat recommended one, but it still needs a formal milestone document.

6. What future assistants must not get wrong

A future assistant should not turn this conversation into a generic survival-crafting summary. The game direction is more specific: robotic seed civilisation, procedural scientific planets, deterministic sparse simulation, physical spawn fabrication, nanobot construction, automation-first infrastructure, and machine graph factories.

A future assistant should also not treat every assistant recommendation as a final user decision. The user clearly stated many design preferences, but several architecture choices were assistant synthesis. They are strong candidates, not automatically final requirements.

A future assistant should not reintroduce worker NPC labour as the default solution. The user explicitly moved away from that because of pathfinding and compute cost. If worker-like entities are ever used, they should probably be drones, local cosmetics, or collapsed aggregate systems rather than a full labour economy.

A future assistant should not assume nanobots mean free building. Nanobots are a construction interface. They must still obey material, energy, time, heat, capability, permission, and logistics constraints.

A future assistant should not assume a single Unreal level can contain the whole game. The conversation's architecture was about separating deterministic truth

from local rendering.

7. Most important preservation points

The most important things to preserve are:

1. **Game identity:** Dominion is being shaped as a robotic seed-civilisation game.
2. **Engine identity:** Domino should likely be a deterministic sparse simulation core.
3. **World identity:** planets and systems are science-bounded procedural data, not hand-painted maps by default.
4. **Data identity:** procedural fields, designer overlays, saves, packs, terraforming, and construction should share field/delta technology.
5. **Spawn identity:** players are robot bodies manufactured at physical spawn machines.
6. **Progression identity:** the mothership provides knowledge and precision, but finite throughput/resources force local industry.
7. **Construction identity:** nanobots and blueprints are diegetic, deterministic, and resource-bound.

8. **City identity:** cities are automation/infrastructure systems, not worker-NPC simulations.
9. **Factory identity:** machines compile into process graphs when not locally observed.
10. **Multiplayer identity:** fog of war is an information-security model, not a screen shader.
11. **Roadmap identity:** prove the remote spawn lab loop before attempting solar-system/MMO scale.
12. **Spec process identity:** keep FACT, INFERENCE, and UNCERTAIN labels to avoid corrupting future aggregation.

8. Recommended next action after this bundle

The best next action is to turn the proposed first vertical slice into a formal milestone:

A player spawns from the mothership as a robot, surveys terrain, finds ore, uses cut/fill to prepare a site, builds power, builds a mine, refines material, fabricates parts, transports them, constructs a remote spawn lab, and successfully spawns or swaps into a robot body from that lab.

That milestone should produce:

- a design document;
- a system list;
- data schemas for terrain deltas and construction jobs;
- a mothership resource budget;
- one or two robot chassis;
- one ore and one power system;
- one machine graph example;
- one fog-of-war/sensor rule set;
- deterministic replay/snapshot/hash success tests;
- clear non-goals so the prototype does not become the whole game at once.

9. File package explanation

This companion bundle includes the prior preservation files and this new report. The previously generated files are still useful because they contain the full formal structure: sections 0-39, registers, spec sheet, aggregator packet, and audit. This companion report is meant to be more direct and readable, explaining the whole conversation in one place and clarifying what the package contains.

The bundle also includes the uploaded `Pasted text.txt` preservation prompt, because that prompt defined the required format for the preservation package and is useful context for why the prior files look the way they do.

10. Bundle verification

The new ZIP bundle was created after scanning the available files in `/mnt/data`. It includes:

- the current deterministic solar-system architecture handoff files;
- the earlier robotic seed-civilisation architecture handoff files found in the workspace;
- the uploaded preservation prompt;
- this new companion report;
- a combined bundle manifest with file hashes and sizes.

The ZIP was opened and tested after creation. No archive corruption was detected by the ZIP test.

11. Quick reference: what was done versus deferred

Done in this conversation

- Established the overall technical feasibility framing.
- Identified deterministic sparse simulation as the core approach.
- Separated Domino's simulation role from Unreal/client rendering role.
- Developed the robotic mothership premise.
- Established robot players and physical spawn machines.
- Established automation over worker NPC labour as a major direction.
- Established nanobot blueprint construction as the construction interface.
- Established machine/factory process graphs as a scaling tool.
- Created a preservation package with reports, registers, spec sheet, aggregator packet, and ZIP.
- Created this companion report and combined bundle.

Deferred for later

- Formal core engine spec.
- First vertical slice milestone document.

- Planet generator parameter schema.
- Field/delta data schema.
- Mothership economy numbers.
- Robot chassis stats.
- Nanobot construction rules.
- Machine compiler rules.
- City liveliness design without NPC workers.
- Sensor/fog-of-war model.
- MMO authority model.
- Unreal integration validation.
- Master Project Spec Book merge.

12. Final human summary

This conversation matters because it turned a very broad "can we build a huge deterministic procedural solar-system MMO?" question into a more concrete and distinctive game direction. Dominion is not just a space game, survival game, factory game, city builder, or MMO. The emerging concept is a deterministic robotic seed-civilisation simulator where players are machine bodies produced by a finite mothership, then expand

across a procedurally generated scientific planet by building automated infrastructure.

The technical foundation is sparse deterministic state. The world should not be stored as one dense map or simulated everywhere at full fidelity. It should be generated from seeds and physical data, then modified through sparse deltas, construction jobs, resource ledgers, machine graphs, and event logs. Only areas that players can sense or affect need to be expanded into full local simulation. Everything else should collapse into aggregate forms that preserve causality and consistency without wasting compute.

The creative foundation is the mothership. It gives players knowledge, recipes, HUD overlays, blueprints, respawn, and a reason to be robots. It also creates the central progression constraint: the ship can begin civilisation but cannot scale civilisation alone. Players must build the industrial base needed to replenish, expand, and eventually surpass the mothership.

The gameplay foundation is automation. The user specifically rejected worker NPCs as the default because they would require expensive pathfinding and simulation. Dominium should instead make automation diegetic: nanobots, AI controllers, drones, fabricators, process graphs, power grids, logistics systems, and machine networks. Cities become living infrastructure rather than crowds of simulated workers.

The next step should not be a full solar system or MMO. The next step should be a small but complete vertical slice: mothership spawn, robot body, survey, ore, power, cut/fill, blueprint construction, mine, refinery, fabricator, remote spawn lab, and deterministic replay. That loop tests the core promise of the game. If it works, larger planets, cities, factories, terraforming, multiplayer, and megaprojects can be layered on top.

7.17 Dominion Robotic Seed Civilisation Architecture Human Readable Report

Source: docs/archive/conversations/Dominium_Complete_Conversation/robotic_seed_civilisation_handoff_file/Dominium_Robotic_Seed_Civilisation_Architecture__01_human_readable_report.md

Report type: human_readable_report

Size: 33167 bytes

**7.17.1 COMPLETE CHAT PRESERVATION
REPORT - Dominion Robotic
Seed-Civilisation Architecture**

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion
Robotic Seed-Civilisation Architecture

Row 2 Field: Date anchor; Assessment: 2026-05-27
Australia/Melbourne

Row 3 Field: Source scope; Assessment: This chat only
unless labelled PROJECT-CONTEXT

Row 4 Field: Apparent access; Assessment: Partial /
unclear

Row 5 Field: Previously generated files available?;
Assessment: No previously generated downloadable
files visible before this preservation task

Row 6 Field: Uploaded files or artifacts present?;
Assessment: Yes: Pasted text.txt preservation
prompt; visible chat messages are also source
artifacts

Row 7 Field: Contains future plans?; Assessment: Yes

Row 8 Field: Contains decisions?; Assessment: Yes, but
many are tentative design directions rather than
formally ratified decisions

Row 9 Field: Contains pending tasks?; Assessment:
Yes, mostly assistant-derived spec/prototype tasks

Row 10 Field: Contains unresolved questions?;
Assessment: Yes

Row 11 Field: Staleness risk; Assessment: Medium

Row 12 Field: Extraction confidence; Assessment: 4/5
for visible chat; lower for any unavailable prior/
invisible context

Row 13 Field: Safe for later aggregation?; Assessment:
With caveats

Row 14 Field: Main limitations; Assessment: Only the
visible transcript and uploaded preservation
prompt were available; no hidden older project files
or previous generated packages were available for
inspection.

The extraction is based on the visible conversation and the uploaded preservation prompt. I can see the substantive Domino/Dominium design exchange in this chat: the original large-scale deterministic solar-system/MMO feasibility question, the assistant architecture responses, the user’s spoken brainstorm transcript about procedural planets and robotic seed civilisation, and the assistant’s design synthesis. I cannot verify whether there were earlier turns outside the visible context, and I

cannot inspect any files that were not uploaded or visible here. Therefore the package is reliable for this visible chat, but it should be merged into a larger project only with caveats.

The uploaded file is an instruction artifact, not a gameplay artifact. It asks for a human-readable report first, then structured registers, a context transfer packet, a spec sheet, an aggregator packet, self-audit, and generated files. That file is preserved as ARTIFACT-06.

1. One-Page Orientation

This chat was about the high-level architecture and game design of **Dominium**, a proposed deterministic, procedural, full-scale planetary/star-system civilisation game built on or alongside a custom **Domino engine**. The user's initial question was very broad: how to make a deterministic game with a full-scale real solar system, fully recreated planets, customizable solar systems and planets, procedural generation from data, player construction/destruction, terraforming, megaprojects, machines, factories, fog of war, sparse loading/simulation, MMORPG persistence, single-player multi-universe play, and client-shared compute. The user also asked whether Unreal could do this without lag at high FPS and low latency for millions of players across shards or player-hosted universes.

The assistant's first major answer framed the core technical principle: the project should be a deterministic simulation and database first, with rendering as a downstream client concern. In that framing, Domino should own fixed-tick deterministic simulation, sparse state, procedural generation, fog-of-war truth, event logs, snapshots, authoritative servers, and simulation LOD. Unreal could help with visual presentation, tools, local rendering, and streaming, but should not be treated as the sole authoritative simulation engine for the whole universe. The answer also argued that millions of players can plausibly share one persistent universe across many regions/shards, but not all participate in one fully simulated, low-latency, high-FPS interaction bubble.

The user then gave a long spoken brainstorm that shifted the discussion from pure architecture into a more concrete game premise. The game became a **robotic seed-civilisation** game: humanity sent a robotic mothership to another solar system; the ship lands on the most habitable planet; it fabricates robot bodies using nanotechnology; players spawn as robots from physical machines; they explore, mine, build factories, create new spawn labs, and eventually build cities and terraform. The mothership carries human knowledge, which explains why players do not need to reinvent CPUs, railway gauges, calendars, or advanced recipes. However, the mothership is limited by resources,

throughput, scale, and replenishment needs, so players still need to build industrial chains.

A major design transition occurred in the user's brainstorm: the user moved away from human-like worker NPC labour. The user reasoned that workers would require pathfinding, scheduling, physics, and costly simulation, and suggested going straight to automation. Work should be performed by computers, nanobots, machines, and abstract AI agents rather than simulated people. Likewise, machines and factories should not be simulated as every moving part everywhere. They should collapse into process diagrams: inputs, ticks, outputs, throughput, energy costs, heat, and maintenance. This is central to the project because it preserves the fantasy of civilisation-building while keeping computation bounded.

The assistant's final design synthesis in the visible chat organized this into a coherent direction: science-bounded procedural star-system and planet generation; layered fields for procedural base data, painted overrides, saves, mod packs, terraforming, and construction; physical robot spawn points; robot chassis instead of classes; nanobot-assisted blueprint construction; cut/fill terrain deltas; no worker NPC economy; machine graphs/compilers; AI-planner quest markers; fog-of-war via sensors; and a practical first vertical slice. That vertical slice would prove the loop: spawn from the mothership,

survey terrain, find ore, blueprint a mine, cut/fill a site, build power, refine materials, fabricate parts, construct a remote spawn lab, and spawn or swap into a new robot body there.

The future relevance of this chat is high. It contains the transition from "can this impossible-scale engine exist?" to "what is the actual game?" The most important thing a future reader must understand is that many items are still brainstormed or assistant recommendations. The strongest user-stated directions are: science-bounded procedural planets; overlayable field data; robot bodies and physical spawn fabrication; mothership as knowledge source but not infinite factory; nanobot/blueprint construction; avoiding worker NPC labour; and machine/factory abstraction as process graphs. The main architectural recommendation-Domino as deterministic backend and Unreal as renderer/client-is useful but should still be verified and ratified before becoming a final requirement.

2. The Story of the Conversation

2.1 From impossible-scale feasibility to architecture The chat began with an ambitious feasibility question. The user wanted to know how Domino/Dominium could support deterministic gameplay across a full-scale solar system with procedurally recreated planets, sparse construction/

destruction, terraforming, fog of war, machines, factories, megaprojects, MMO persistence, client-shared compute, and single-player multi-universe gameplay. The assistant answered by reducing the problem to architecture: make Domino a deterministic simulation/database first and use Unreal or another engine as a renderer/client. This created the first major structural split between **truth simulation** and **visual presentation**.

2.2 From architecture to game premise The user then provided a spoken design brainstorm. This introduced the seedship premise: humans in the past sent a robotic mothership to find another star system, land on a habitable planet, and fabricate robot bodies to restart civilisation. This transformed the project from a generic planetary MMO simulation into a game with a coherent player fantasy: players are robots building a new civilisation from a seedship.

2.3 From procedural planets to field overlays The user described a system where designers generate planets by setting science-bounded parameters rather than manually painting worlds. The user wanted forward generation from orbital/planetary parameters and also a reverse process where a desired planet outcome can suggest plausible physical parameters. The user also wanted procedural fields to be overlaid with painted

fields, specific data, save-file deltas, worldgen edits, and terraforming changes. The assistant later formalized this as a field-layer stack.

2.4 From labour simulation to automation The user considered NPC workers but then deprioritised them because labour implies pathfinding, physics, scheduling, and many failure cases. The user preferred to make automation available from the start and to justify it diegetically: computers, robot bodies, nanobots, and AI agents do the work. This became one of the strongest design directions in the chat.

2.5 From machine parts to process graphs The user wanted machines, vehicles, rockets, factories, and devices to be creatable by players, but not continuously simulated as every internal part. Instead, a machine or assembly should compile into a functional object or equation: inputs, tick, outputs, energy, and constraints. The assistant framed this as a machine compiler and active/collapsed simulation model.

2.6 Final state of the chat By the end of the design discussion, the chat had produced a coherent candidate direction: *Dominium* as a deterministic robotic seed-civilisation game, with *Domino* as a sparse deterministic simulation engine, *Unreal* as a possible renderer/client,

and a first playable slice around building a remote spawn lab from mothership beginnings. The preservation request then asked for this chat to be turned into a high-fidelity handoff and spec-prep package.

3. Main Topics Discussed

Topic 1 - Deterministic universe architecture The initial topic was whether a full-scale deterministic solar-system game could be technically structured. The assistant proposed fixed-tick simulation, deterministic command logs, sparse deltas, snapshots, hierarchical coordinates, fog-of-war filtering, and simulation LOD. The conclusion was not a user-ratified decision but a strong recommendation: Domino should be the authoritative deterministic core and renderer engines should not own truth state.

What remains uncertain is the exact implementation language, math model, networking model, physics compromises, and determinism guarantees. This connects directly to future engine architecture, replay debugging, anti-cheat, server authority, and save compatibility.

Topic 2 - Science-bounded procedural star-system and planet generation The user wanted procedural planets and solar systems generated from realistic

bounds, not arbitrary sliders or manual painting. The generator should allow forward generation from parameters such as orbit and planetary properties, and possibly reverse generation from desired surface/world outcomes. The user also wanted designers to tweak results without losing procedural foundation.

The key conclusion is that the generator/editor should distinguish hard validity, soft plausibility, and gameplay/science-fiction overrides. This needs future verification against planetary science and gameplay needs.

Topic 3 - Layered fields for planet data, packs, saves, and terraforming The user wanted the same underlying technology to support worldgen, planet packs, save files, painted overrides, and in-game terraforming. The assistant formalized this as a field-layer stack. This is important because it unifies procedural data, authored changes, player edits, and persistence.

The unresolved issue is the exact schema: which fields exist, how they are chunked, how operations are versioned, which layers conserve matter/energy, and how conflicts are resolved.

Topic 4 - Robotic seedship lore and player identity The user introduced the mothership: a human-created robotic ship that lands on a suitable planet and manufactures robots to restart human civilisation.

Players spawn as robot bodies from physical machines. They may choose body forms such as biped, quadruped, spider, or tank-like chassis. The mothership carries knowledge and early resources.

This topic matters because it ties together respawn, HUD, construction, automation, progression, and lore. It remains unresolved whether the long-term story is human restoration, robot autonomy, human embryo arrival, mystery lore, or player faction divergence.

Topic 5 - Mothership as recipe book but not infinite factory The user used the mothership to solve the "why do players know advanced recipes?" problem. The ship contains designs for CPUs, railways, calendars, manufacturing processes, and other human knowledge. The user also identified the risk: if the mothership can synthesize anything, progression collapses. The proposed answer is finite reserves, low throughput, scale limits, and eventual need for player-supplied resources.

Future work must turn this into hard mechanics: starter reserves, fabrication rates, heat limits, safety protocols, mission reserves, and replenishment recipes.

Topic 6 - Nanobot construction, HUDs, and blueprinting The user proposed nanobots as a diegetic construction interface and compute-saving abstraction. Robots have HUDs; they can preview

nondestructive blueprints, commit builds, cut and fill terrain, and use nanobot swarms to manipulate matter. The assistant recommended a staged workflow: preview, plan, reserve, stage, execute, commit, operate.

The unresolved risk is that nanobots become magic unless bounded by material, energy, time, range, heat, permissions, and tool capability.

Topic 7 - No worker NPC economy; automation from the start The user considered labour NPCs but rejected or deprioritised them because they require expensive pathfinding and simulation. Instead, work should be represented as computers, AI agents, machines, nanobots, and process graphs. Wildlife and mobs may exist, but intelligent worker societies are not core.

This is one of the clearest user design preferences. Future design must still make cities feel alive without worker NPCs.

Topic 8 - Machine/factory graphs and compiled devices The user wanted machines and vehicles to be player-creatable but computationally collapsible. A machine assembly should become an object with a functional summary: inputs, tick rate, outputs, power, throughput, and constraints. The assistant called this a machine compiler.

This is central to scalability. Future work must define component catalogs, graph rules, allowed functions, budgets, exploit prevention, visual expansion near players, and collapsed aggregate simulation far away.

Topic 9 - MMO, single-player, private universes, and scale limits The user wanted single-player multi-universe play and possibly an official MMO canonical server. The assistant distinguished logical universe persistence from one giant real-time bubble. It recommended region authority, interest management, fog-of-war filtering, and sparse simulation.

The unresolved issue is how far MMO ambition should go, how client-shared compute can be used safely, and how modes should differ in progression and persistence.

Topic 10 - Unreal's role The user asked whether Unreal can do this without lag and with low latency/high FPS for millions of players. The assistant's answer was that Unreal can help with rendering, local streaming, tooling, UI, animation, and maybe client/network pieces, but should not own the deterministic authoritative universe.

This is a recommendation requiring verification. Current Unreal features and limits should be checked before becoming formal specification.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted to understand how to make *Dominium* deterministic, full-scale, procedural, sparse, fog-of-war-aware, multiplayer-capable, and playable in both MMO and single-player modes. The user also explicitly wanted realistic procedural solar-system/planet generation; player terraforming; cut/fill construction; megaprojects; machine/device/factory design; robot spawning; physical expansion across a planet; and automation without expensive worker NPCs.

These goals were addressed conceptually, not implemented. The chat produced architecture principles and a design direction, but not code, data schemas, or prototypes.

4.2 Inferred Goals The user appears to want a game where large-scale civilisation building feels physically grounded but is computationally viable. The user also appears to prefer diegetic explanations for mechanics: HUDs because players are robots, quest markers because an AI planner knows the player's goals, spawn points because robot bodies are fabricated, and automation because the colony is robotic.

4.3 Goals That Changed Over Time The conversation shifted from generic full-scale universe/MMO architecture toward a more specific robotic seed-civilisation premise. It also shifted from potential NPC labour to direct automation and process graphs. The world scope clarified from "galaxy/universe" language to an initial star/solar system with customizable planets and possible larger universe framing later.

4.4 Goals Still Unresolved The main unresolved goals are implementation-specific: generator science model, field layer schema, mothership economy, chassis stats, nanobot limits, machine compiler rules, fog-of-war model, server architecture, Unreal integration, and the first vertical slice.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: The main playable setting should start as a procedurally generated star/solar system, not a fully simulated galaxy.; Status: Tentative user design direction; Why it mattered: Narrows initial scope while preserving future universe/multiverse framing.; Confidence: 4/5; Label: FACT

Row 2 ID: DECISION-02; Decision: Planet/system generation should be science-bounded, using

plausible parameters rather than arbitrary sliders.; Status: User-stated design direction; Why it mattered: Provides design freedom without random fantasy settings unless marked as overrides.; Confidence: 5/5; Label: FACT

Row 3 ID: DECISION-03; Decision: The editor should allow both forward generation from orbital/planet parameters and reverse fitting from desired planet outcomes.; Status: Tentative design direction; Why it mattered: Gives designers control over results without manually painting everything.; Confidence: 4/5; Label: FACT

Row 4 ID: DECISION-04; Decision: Procedural fields, painted overrides, save files, mod packs, and terraforming should share a common layered data approach.; Status: Tentative design direction; Why it mattered: Unifies persistence, modding, editing, and gameplay terrain changes.; Confidence: 4/5; Label: FACT

Row 5 ID: DECISION-05; Decision: Players are robots fabricated by a mothership or remote spawn labs.; Status: Strong user-stated direction; Why it mattered: Makes respawn, HUD, carrying strength, AI assistance, and automation diegetic.; Confidence: 5/5; Label: FACT

Row 6 ID: DECISION-06; Decision: The mothership

contains knowledge/recipes but is constrained by finite resources, throughput, scale, heat, and replenishment needs.; Status: Tentative user direction, assistant refined; Why it mattered: Preserves progression despite advanced seedship technology.; Confidence: 5/5; Label: FACT + INFERENCE

Row 7 ID: DECISION-07; Decision: Use nanobots/robot HUD/blueprints as the player-facing construction interface.; Status: Tentative user design direction; Why it mattered: Makes construction powerful and cheap to represent while still requiring rules.; Confidence: 4/5; Label: FACT

Row 8 ID: DECISION-08; Decision: Avoid worker NPC labour as a core construction/economy system; use automation and AI/process abstraction instead.; Status: Strong user-stated direction; Why it mattered: Reduces compute cost and aligns with robot-civilization premise.; Confidence: 5/5; Label: FACT

Row 9 ID: DECISION-09; Decision: Machines/factories should compile into process graphs/equations rather than simulate every physical part continuously.; Status: Strong user-stated direction; Why it mattered: Enables player creativity without unbounded simulation cost.; Confidence:

5/5; Label: FACT

Row 10 ID: DECISION-10; Decision: Cities should be easy to start, but advanced technology should require city-scale industry.; Status: Tentative user direction; Why it mattered: Creates progression while supporting the fantasy of large-scale settlement building.; Confidence: 4/5; Label: FACT

Row 11 ID: DECISION-11; Decision: Domino should own deterministic simulation truth; Unreal, if used, should be a client/renderer/tool layer.; Status: Assistant recommendation, not formally user-ratified; Why it mattered: Avoids relying on Unreal actors/physics for a massive deterministic MMO universe.; Confidence: 3/5; Label: INFERENCE

Row 12 ID: DECISION-12; Decision: A first playable slice should prove one region loop before solar-system/MMO scale.; Status: Assistant recommendation, not formally user-ratified; Why it mattered: Provides a testable path from concept to implementation.; Confidence: 3/5; Label: INFERENCE

The most important distinction is that user-stated design directions are stronger than assistant recommendations. The user clearly advanced the robot

seedship premise, science-bounded procedural planets, mothership as knowledge source, limited mothership fabrication, physical robot spawn points, nanobot construction, no worker NPC labour, and machine process abstraction. The assistant's Domino/Unreal architectural split and vertical-slice plan are useful recommendations, but they should be treated as not yet formally accepted unless the user later ratifies them.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

The chat rejected or deprioritised several tempting but costly paths. Manual painting should not be the primary planet creation method; it should exist as override data. Arbitrary planet sliders should be constrained or clearly marked as non-plausible. Worker NPC labour was strongly deprioritised because it adds pathfinding, scheduling, simulation, and replication cost. Simulating every machine part everywhere was rejected in favour of process graphs. An unlimited mothership fabricator was rejected because it would erase the need for player industry. Unreal as the complete authoritative solution was rejected by the assistant, but this remains a recommendation pending verification and user ratification.

7. Important Reasoning, Rationale, and Tradeoffs

The visible reasoning is compute-first and diegetic. The user repeatedly looked for ways to get the fantasy of a large civilisation game without paying the full computational cost. Robots solve HUD, inventory, respawn, strength, and automation. Nanobots solve manipulation and construction interface, but only if bounded. Process graphs solve factories and machines, but only if the compiler prevents exploit designs. The mothership solves knowledge progression, but only if limited by resources and throughput. Field layers solve the connection between procedural generation, authored overrides, saves, packs, and terraforming, but only if gameplay layers obey conservation.

The central tradeoff is fidelity versus simulation cost. The answer throughout the chat was to keep high fidelity where players can sense and interact, and collapse everything else into deterministic sparse fields, graphs, ledgers, and aggregates.

8. Plans, Future Work, and Next Steps

The recommended next work is to convert the design into a prototype/spec sequence:

1. Formalize creative pillars: robotic seedship, science-bounded procedural worlds, automation-

first civilisation, sparse deterministic simulation.

2. Define the generator/editor parameter model and plausibility scoring.
3. Define the field-layer schema for terrain/resources/climate/terraforming/construction/saves.
4. Design mothership reserves, spawn economy, body chassis, and remote spawn labs.
5. Design nanobot construction and cut/fill commit rules.
6. Design the machine compiler/process graph model.
7. Define the first playable vertical slice: spawn, survey, mine, cut/fill, power, refine, fabricate, build remote spawn lab.
8. Verify Unreal/current engine assumptions before locking renderer/client decisions.
9. Define MMO/single-player/private-server differences.

The best immediate next action is to produce a **Dominium First Playable Slice Spec** based on this chat.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences The user explicitly wants science-bounded procedural generation, not arbitrary sliders. The user explicitly wants overlayable data, including painted fields, saves, packs, and terraforming. The user explicitly prefers automation over worker NPC labour. The user explicitly wants robots, physical spawn machines, mothership knowledge, limited mothership resources, blueprint construction, and process-equation machines.

9.2 Inferred Constraints and Preferences The user appears to prefer diegetic mechanics, compute-aware systems, large-scale ambition with sparse simulation, and systems that can later become formal spec chapters.

9.3 Uncertain or Unestablished Preferences It is not yet established whether the user wants Unreal as the primary renderer, whether the official MMO should be permanent or seasonal, how hard-science the setting should be, how much mystery lore matters, whether humans eventually arrive, or how strict anti-griefing rules should be.

10. Files, Artifacts, Outputs, and Prompts

Before this preservation response, no generated downloadable files were visible. The uploaded file `Pasted text.txt` is the preservation-task prompt. The visible chat itself contains the core artifacts: the initial feasibility question, assistant architecture answers, user brainstorm transcript, and assistant design synthesis. This response creates the preservation files and ZIP listed in Section 35.

11. Open Questions and Unresolved Issues

The largest unresolved issues are: exact science bounds for generation; reverse-generation method; field-layer schema; mothership resource/throughput balance; chassis list and spawn costs; nanobot limits; machine compiler rules; city liveliness without NPCs; official MMO architecture; single-player/private-server differences; fog-of-war security; Unreal integration; and first vertical slice acceptance criteria.

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Future chats might incorrectly treat every brainstorm item as a final decision, especially assistant recommendations. They might over-focus on Unreal and ignore Domino's need for deterministic data/simulation.

They might reintroduce worker NPC labour despite the user's compute objection. They might let nanobots or the mothership become unlimited magic. They might ignore fog-of-war secrecy while discussing client-shared compute. They might also merge stale external claims about Unreal or NASA data without verification.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat contributes the likely core identity of Dominion: a deterministic robotic seed-civilisation game built around science-bounded procedural planets, a resource-limited mothership, physical robot spawn fabrication, nanobot construction, automation-first cities, and machine process graphs. It should feed into spec-book chapters on creative pillars, world generation, engine architecture, terrain/fields, player identity, construction, factories, progression, fog of war, multiplayer modes, and vertical slice planning.

14. What I Should Remember

- The project should not be reduced to "space MMO."
The distinctive idea is robotic seed-civilisation on scientifically generated worlds.
- The strongest user-stated decisions are robots, mothership, physical spawn points, finite ship

resources, automation, nanobot blueprinting, field overlays, and machine process graphs.

- The most important assistant recommendation is Domino as deterministic truth and Unreal as renderer/client, but this needs verification and user ratification.
- The first playable slice should prove the loop from mothership spawn to remote spawn lab.
- The largest risks are over-scope, treating brainstorm as final, unlimited nanobots/mothership fabrication, player-created lag machines, fog-of-war leaks, and stale engine assumptions.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- "Summarize the robotic seed-civilisation premise in five design pillars."
- "Which parts of this chat are user-stated facts versus assistant recommendations?"

15.2 Decisions

- "Which tentative decisions should I ratify, revise, or reject before creating the spec book?"

- "Is Domino-as-deterministic-core and Unreal-as-renderer now an accepted architecture, or still just a recommendation?"

15.3 Tasks and Next Actions

- "Turn the first playable slice into a detailed milestone plan."
- "Write the field-layer schema for procedural planets, saves, packs, and terraforming."

15.4 Artifacts and Files

- "Extract the generated package into a concise project-spec outline."
- "Compare the human report and registers for contradictions."

15.5 Risks and Verification

- "Make a verification plan for Unreal, SPICE/Horizons, determinism, and sparse terrain."
- "List the top ten ways this design could collapse under compute cost."

15.6 Future Spec Book / Aggregation

- "Convert this chat into spec-book chapters with requirement IDs."
- "Identify what should not be merged into the master spec until confirmed."

15.7 Deep-Dive Questions Specific to This Chat

- "Design the mothership resource economy so it explains recipes but does not bypass industry."
- "Design the machine compiler that turns player-built devices into process graphs."
- "Design robot chassis and remote spawn labs for the first playable slice."

16. Compact Human Summary

This chat captured a major design transition for Dominion. It began as a technical feasibility question about a deterministic full-scale solar-system game with procedural planets, terraforming, machines, megaprojects, fog of war, MMO persistence, client-shared compute, and Unreal support. The assistant's initial answer reduced the problem to a practical architecture: make Domino a deterministic simulation/database first, use sparse procedural state and deltas,

apply simulation LOD, enforce fog-of-war at the data/network level, and treat Unreal as a possible renderer/client rather than the authoritative universe.

The user then supplied the most important creative material: *Dominium* should be a robotic seed-civilisation game. Humans launched a robotic mothership into another star system. It lands on the most habitable planet and fabricates robot bodies to restart civilisation. Players spawn as robots from physical fabrication machines, choose chassis such as biped, quadruped, spider, or tank-like forms, and expand across the planet by building remote spawn labs. The mothership carries the knowledge of human civilisation, explaining why players know recipes for CPUs, rail systems, calendars, machines, and advanced construction. However, the mothership cannot simply build everything forever: its materials, throughput, heat handling, scale, and mission reserves must be limited, forcing players to mine, refine, manufacture, and build cities.

The user also clarified how planet generation should work. The game should procedurally generate a star/solar system and planets using realistic science-bounded parameters, not arbitrary sliders. Designers should be able to tweak and override results without manually painting everything. The user also proposed reverse generation: choose the kind of planet you want, then let the generator search for plausible orbital and planetary

parameters that fit. The same underlying technology should support procedural fields, painted fields, planet packs, save files, and gameplay terraforming.

A second major shift was the rejection or strong deprioritisation of worker NPC labour. The user reasoned that simulated workers would require pathfinding, physics, scheduling, and high compute cost. Instead, the civilisation should be robotic and automated from the beginning. Construction can be done through nanobots, HUD blueprints, AI planners, and machines. Factories and machines should not simulate every part everywhere. They should compile into process graphs or equations: inputs, ticks, outputs, power, throughput, heat, failure, and maintenance. High-detail visuals can appear near players, but the authoritative simulation should remain collapsed and deterministic where possible.

The chat did not produce final implementation details or code. It produced a coherent design direction and a set of future workstreams. The most important next action is to turn the first playable loop into a formal spec: spawn from mothership, survey terrain, find ore, blueprint a mine, cut/fill terrain, build power, refine materials, fabricate parts, construct a remote spawn lab, and spawn/swap into a new body. If that loop is fun and technically coherent, it can become the foundation for larger planet, city, and MMO systems.

7.18 Dominion Robotic Seed Civilisation Architecture Reader Brief

Source: docs/archive/conversations/Dominium_Complete_Conversation/robotic_seed_civilisation_handoff_file/Dominium_Robotic_Seed_Civilisation_Architecture__06_reader_brief.md

Report type: reader_brief

Size: 3058 bytes

7.18.1 Reader Brief - Dominion Robotic Seed-Civilisation Architecture

What this chat was about

This chat developed Dominion from an enormous deterministic full-scale solar-system/MMO feasibility question into a candidate game identity: a robotic seed-civilisation game. Players are robot bodies fabricated by a mothership that landed on a procedurally generated, science-bounded planet. They use ship knowledge, limited materials, nanobot construction, automation, and machine process graphs to build industry, remote spawn labs, cities, and eventually terraforming/megaproject systems.

Top 20 things to know

1. The user wants science-bounded procedural solar-system/planet generation.
2. The initial scope is better understood as a star/solar system, not a full galaxy.
3. Planet generation should not rely primarily on manual painting.
4. Painted/data overrides are still needed.
5. A shared field-layer model should support generation, saves, packs, and terraforming.
6. Humans sent a robotic mothership to restart civilisation.
7. Players spawn as robot bodies fabricated by physical machines.
8. Remote spawn labs are a major exploration/expansion milestone.
9. The mothership contains recipes/knowledge so players do not reinvent all technology.
10. The mothership must be resource/throughput/heat/scale limited.
11. Robots provide diegetic HUDs, strength, bodies, tools, and respawn logic.

12. Nanobots provide diegetic blueprint construction and cut/fill manipulation.
13. Nanobots must not be free magic.
14. Worker NPC labour was strongly deprioritised.
15. Automation should exist from the start.
16. Machines/factories should compile into process graphs/equations.
17. Cities should be easy to start but advanced tech should require city-scale industry.
18. Fog of war and sensing limits matter for both gameplay and networking.
19. Unreal is recommended as renderer/client/tooling, not authoritative universe core, pending verification.
20. The best next step is a first playable slice spec from mothership spawn to remote spawn lab.

Decisions

See Decision Register. The strongest user-stated decisions are DECISION-02, DECISION-04, DECISION-05, DECISION-06, DECISION-08, and DECISION-09. DECISION-11 and DECISION-12 are assistant recommendations pending user ratification.

Pending tasks

The highest-priority tasks are the first playable slice spec, generator/editor model, field-layer schema, mothership economy, chassis/spawn rules, nanobot construction workflow, and machine compiler.

Open questions

The key open questions concern physical realism bounds, reverse generation, field schema, mothership quantities, robot chassis, nanobot limits, machine graph rules, city liveliness without NPCs, MMO architecture, and Unreal integration.

Verification items

Verify Unreal capabilities, real Solar System data sources/licensing, planetary science models, determinism risks, sparse terrain tech, MMO scalability, and client-shared compute security.

Best next step

Write the **Dominium First Playable Slice Spec**.

7.19 Dominion Robotic Seed Civilisation Architecture In Chat Reader

Source: docs/archive/conversations/Dominium_Complete_Conversation/robotic_seed_civilisation_handoff_file/Dominium_Robotic_Seed_Civilisation_Architecture__09_in_chat_reader.md

Report type: in_chat_reader

Size: 2406 bytes

7.19.1 In-Chat Reader - Dominion Robotic Seed-Civilisation Architecture

Package overview

This package preserves the visible Dominion/Domino chat about deterministic full-scale procedural world simulation, robotic seed-civilisation gameplay, nanobot construction, automation-first factories, machine process graphs, and Unreal/Domino architecture.

File index

- 00 Manifest: file list, purpose, caveats, status.
- 01 Human-readable report: Sections 0-16.

- 02 Context transfer packet: Section 29.
- 03 Spec sheet: Section 30 in YAML.
- 04 Registers: Sections 17-28.
- 05 Aggregator packet: Section 31.
- 06 Reader brief: short brief and top 20 things.
- 07 Verification and audit: Sections 32-34 and verification queue.
- 08 Future chat bootstrap prompt: prompt for continuing in a new chat.
- 09 In-chat reader: this guide.
- ZIP: all files bundled.

Plain-English explanation

The chat's main value is that it connects extreme-scale simulation architecture to an actual game premise. The project is not just a technical "solar-system MMO"; it is a robotic seed-civilisation game where the user wants scientifically bounded procedural planets, robot bodies fabricated from physical spawn machines, a limited knowledge-carrying mothership, nanobot blueprint construction, automation instead of worker NPCs, and factories/machines represented as process graphs.

Question menu

- Which items are definitely user-stated facts?
- Which assistant recommendations still need ratification?
- What should the first playable slice include?
- How should the field-layer system work?
- How do nanobots remain useful without becoming magic?
- How does the mothership provide recipes without erasing progression?
- How should machine graphs compile player-made devices?
- What must be verified before choosing Unreal boundaries?

Top things to preserve

Preserve robotic seed-civilisation, science-bounded procedural generation, field layers, limited mothership, robot spawn labs, nanobot blueprinting, no worker NPC labour, machine process graphs, sparse deterministic simulation, and the first-slice plan.

Safest next actions

1. Confirm which tentative decisions become formal requirements.
2. Write the first playable slice spec.
3. Define the field-layer schema.
4. Define the mothership/spawn economy.
5. Define the machine compiler.
6. Verify Unreal and external data assumptions.

Chapter 8

Domino Dominion Workbench

8.1 Dominion Workbench Full Conversation Companion Human Readable Detailed Summary Report

Source: docs/archive/conversations/Domino_Dominium_Workbench/dominium_workbench_full_conversation_companion__human_readable_detailed_summary_report.md

Report type: human_readable_detailed_summary

Size: 28340 bytes

8.1.1 Dominion Workbench Full Conversation Companion Report

Date anchor: 2026-05-27 Australia/Melbourne

Scope: This report summarizes the visible conversation and accompanies the existing preservation package.

Reliability note: This is a human-readable consolidation, not a live repository audit. Repo-current

statements that came from pasted material or prior-chat excerpts remain **UNVERIFIED** until checked against the live `julesc013/dominium` repo and current AIDE/validator outputs.

1. Executive Summary

This conversation started as a practical plan to build a Windows-first internal UI Editor / Tool Editor to fix the Dominium launcher UI and visually author setup, launcher, game, and tool interfaces. It then evolved into a much larger and more durable architecture:

Dominium Workbench Platform, a cross-platform rendered, TUI-capable, CLI/headless-compatible, modular production environment for building the entire Domino/Dominium ecosystem.

The original UI Editor / Tool Editor plan is now **superseded as a final product**. It is not lost: its useful parts become Workbench capabilities, especially Interface Studio, UI/HUD Sandbox, Theme Laboratory, TUI Studio, Rendered GUI Studio, Preview Matrix, validation panels, import/export tools, and document-patch workflows.

The central decision is that Workbench must not become semantic authority. Workbench is a surface over contracts, commands, services, documents, patches, providers, packs, modules, apps, artifacts, diagnostics,

evidence, tests, and replay/proof. It is the richest human/agent interface over the system, but it must not bypass command, capability, refusal, validation, document, patch, or proof law.

The second central decision is that CLI, TUI, rendered GUI, OS-native GUI, and headless automation must be **projections of the same command/result/refusal/document/view spine**. A GUI button, a TUI hotkey, a CLI command, a headless JSON job, and a future OS-native widget action should route through the same command/service path and produce the same typed result, diagnostics, refusals, logs, and evidence.

The conversation also expanded the target from "build a tool" to "build the production environment for a deterministic simulation operating environment." Domino is the reusable substrate. Dominion is one game/product family. Workbench is the production, validation, editing, inspection, packaging, and evidence environment. AIDE governs work. Codex executes bounded patches. Contracts are law. Tests, replay, and evidence are proof.

The practical next objective, subject to live repo verification, is

COMMAND-RESULT-VIEW-SLICE-01: prove one real command through a typed result, view contract, projection descriptor, CLI output, TUI/text output,

rendered-placeholder output, diagnostics, refusals, and evidence. The Validation Dashboard remains the best first Workbench module because it is immediately useful and proves command/result/diagnostic/evidence parity without requiring full gameplay.

2. How the Conversation Developed

2.1 The initial UI Editor / Tool Editor phase The first phase focused on a Windows launcher UI that looked mangled and flickered. The proposed solution was an internal graphical editor capable of designing pixel-perfect UIs and broad-strokes functionality for setup, launcher, game, and other tools. The early plan assumed Windows first, later Windows/macOS/Linux support, true OS controls where possible, CMake integration, stable file formats, generated stubs, and a visual authoring environment.

The user provided concrete details about the existing DUI system: the launcher used a TLV schema/state model, with Win32, DGMX, and null backends. The likely flicker sources were Win32 layout on WM_SIZE, RedrawWindow, list resets/invalidates, and visibility toggles mapping to ShowWindow on child controls. This grounded the plan in actual existing systems rather than abstract UI-tool speculation.

The resulting old plan included canonical UI IR,

deterministic TLV output, JSON mirrors, safe atomic saves, stable IDs, capability validation, layout engines, splitter/tabs/scroll widgets, action stubs, event dispatch, UI Editor GUI, Tool Editor bootstrap, import/export, CLI scripting, `ops.json`, and tests. Many Codex prompts were generated around those ideas.

2.2 The import/CLI/ops and launcher/setup capability-test phase The plan was then extended so the UI Editor could import existing tool UIs, provide a CLI suitable for Codex automation, and use scripted `ops.json` changes to create logical "Minecraft-style" launcher and setup layouts. The user clarified that "Minecraft-style" referred to layout and flow, not visual skin. All controls were to remain native Win32 controls in that earlier phase.

That phase generated more Codex prompts: UI discovery/import/export, CLI validate/format/codegen, `ops.json` editing, launcher UI generation, setup UI generation, and hardening/CI. These are now **historical/superseded** but still useful as implementation material for future document/patch/CLI authoring workflows.

2.3 The pivot to Workbench The user then rejected the old UI Editor / Tool Editor as final products. The user wanted cross-platform tools that use the same CLI,

TUI, and rendered GUI systems as the client, not tools dependent on OS-native widgets. OS-native tools like Visual Studio and Xcode remain useful, but they are not the core presentation platform. This changed the plan from a UI editor to a **Workbench Platform**.

The new goal became a cross-platform rendered Workbench app: one app, many modules, many workspaces, many themes, many renderer backends, and one shared command/document/patch/evidence spine. This Workbench would eventually be used to build Domino, Dominion, packs, modules, UI, themes, tests, releases, and AIDE/Codex work units.

2.4 The unified presentation spine The conversation then focused on how all apps could share CLI, TUI, rendered GUI, OS-native GUI, and headless behavior. The conclusion was that each product should declare which presentation modes it supports, but all products should share the same semantic command/result/refusal/document/view model.

This avoids four independent implementations for the same behavior. A validation command should produce one result schema and then be projected as CLI table/JSON, TUI panel, rendered dashboard, native list/detail, and headless report.

2.5 Workbench as production environment The user emphasized that Workbench is not only for inspection. It should be the integrated environment for building the entire project: code, data, contracts, packs, modules, UIs, themes, renderer tests, sounds, release artifacts, and agentic workflows. The Workbench should support human development, agent work, CLI/TUI/rendered workflows, and evidence/proof.

This led to the idea of Workbench as:

- Command OS.
- Creation Studio.
- Evidence Notebook.
- System Graph.
- Preview/Sandbox.
- Agent Control Board.
- Release Forge.

The user-facing large tools became **workspaces** rather than base modules: Project Graph Explorer, Interface Studio, Module/Pack Foundry, App Composer, Release Forge, Universe Explorer, and so on. These are built from smaller reusable modules, views, documents, commands, services, validators, inspectors, graph views, preview canvases, evidence panels, and patch/diff viewers.

2.6 Modular UI, themes, TUI, and renderer The conversation then described how layout, controls, themes, styles, views, and projections should be modular. The answer was a layered UI architecture: view model, view document, style/theme tokens, runtime controls/widgets/layouts, and renderer backend. Themes are data/token/control-skin profiles, not renderer backends or product logic. Renderers consume draw lists and do not know about products, commands, or OS theme identities.

The user explored OEM+ theme profiles inspired by Windows, Linux desktops, and macOS eras. The answer was that code-only primitive rendering can support useful themes such as barebones, instrument, kernel, paper, terminal, blueprint, phosphor, gridline, tactical, ledger, cartographic, and high contrast. Richer icons, textures, fonts, sounds, and photoreal elements can become optional packs.

The TUI was also treated as a first-class projection. It should not be a second-class ASCII clone; it should be the deterministic, keyboard-first, low-dependency projection of the same operating environment.

2.7 Engineering law and repo future-proofing The user then asked about portability, modularity, extensibility, naming, schemas, protocols, ABI, APIs, and directory structure. The discussion established that

the real problem is not just folders; it is engineering law. Public surfaces, ABI rules, schemas, protocols, commands, diagnostics, artifacts, capabilities, providers, modules, packs, replacements, versions, deprecations, tests, and evidence must all be contract-governed.

A precise vocabulary was adopted:

- Component = source/build ownership unit.
- Service = callable runtime capability.
- Provider = replaceable implementation.
- Pack = distributable authored payload.
- Module = declared functional extension unit.
- Workspace = large user-facing Workbench composition.
- App = shipped product composition.
- Artifact = versioned persisted output/input.

The key warning is that paths are not identity. Stable identity lives in contracts, registries, manifests, IDs, schemas, and artifacts. Implementations can be replaced.

2.8 Self-hosting ladder The user asked how to build Workbench and then use it to build itself and everything else. The answer was **progressive self-hosting**:

1. Hand-code seed substrate.
2. Define commands/results/refusals/diagnostics/documents/views/patches.
3. Prove CLI/TUI/rendered/headless projection for one command.
4. Build minimal Workbench shell.
5. Workbench edits safe non-critical artifacts.
6. Workbench edits its own layout/theme/module documents.
7. Workbench builds product UIs, modules, packs, app compositions, releases, and agent work units.

This avoids the circular trap of expecting Workbench to build itself before the substrate exists.

2.9 Universe Explorer and the robot

seed-civilization game The final major expansion connected Workbench to the actual game direction. Universe Explorer was framed as a lawful inspection/materialization/reference-frame proof surface, not a renderer/free-camera demo. It should prove scale continuity, no modal loading, reference frames, materialization, representation ladders, interest sets, fidelity, provenance, renderer purity, and evidence.

The user then introduced a robot seed-civilization game concept: robotic player avatars, mothership knowledge and constrained fabrication, physical spawn labs, nanobot construction, terrain cut/fill, field layers, machine graphs, factories, fog-of-war through sensors, logistics, cities, claims, governance, and civilization. The answer was that Workbench modules and workspaces can build this if they author real runtime artifacts.

Proposed Workbench spaces include Universe Explorer, Star System/Planet Generator Lab, Field Layer Stack Editor, Cut/Fill Terrain Delta Lab, Construction Blueprint Studio, Machine Graph Lab, Factory/Process Graph Editor, Robot Body/Spawn Fabricator Lab, Mothership Planner, Sensor/Fog-of-War Lab, Logistics/Infrastructure Planner, City/Civilization Planner, Governance/Claims/Permissions Lab, Performance/Complexity Budget Lab, and Renderer/External Bridge Lab.

3. What Was Decided

3.1 Final product direction **Decision:** Old UI Editor and Tool Editor are superseded as final products.

Replacement: Dominion Workbench Platform.

Reason: The old plan risked becoming a separate one-off tool architecture. Workbench can instead prove and reuse the same systems as the client, launcher, setup,

server, and future tools.

3.2 Workbench authority boundary Decision:

Workbench is not authority.

Reason: If Workbench mutates files, truth, or product behavior directly, it will drift from CLI/headless/TUI/runtime semantics. Workbench must issue commands, produce patches, preview, validate, commit transactions, and record evidence.

3.3 Unified presentation spine Decision:

CLI, TUI, rendered GUI, OS-native GUI, and headless are projections of the same command/result/refusal/document/view model.

Reason: This lets one command and one result schema drive every interface. It prevents duplicated logic and lets Workbench modules be reused by client, launcher, setup, server, and tools.

3.4 Native GUI role Decision:

OS-native GUI is optional projection, not the core architecture.

Reason: Visual Studio, Xcode, Win32, AppKit, GTK, etc. remain useful for OS SDK native shells, but the cross-platform rendered Workbench/client path should not depend on them.

3.5 TUI role **Decision:** TUI is first-class.

Reason: It supports servers, SSH, recovery, setup fallback, CI, low-end systems, accessibility, and agent supervision. It must project the same commands and results as GUI/CLI.

3.6 Renderer and theme model **Decision:**

Software renderer first; hardware renderers later. Themes are token/control-skin/layout-metric profiles. Renderer consumes draw lists only.

Reason: This maximizes portability, testability, and reuse.

3.7 Vocabulary and modularity **Decision:** Use the component/service/provider/pack/module/workspace/app/artifact vocabulary.

Reason: It prevents `module`, `tool`, and `component` from becoming vague junk drawers.

3.8 Progressive self-hosting **Decision:** Workbench should become self-hosting gradually.

Reason: It avoids circular bootstrapping and prevents unsafe self-modification.

3.9 Universe Explorer role

Decision/recommendation: Universe Explorer should

be a lawful inspection/materialization proof, not a free-camera renderer demo.

Reason: It should prove Dominion's reality doctrine and multiscale representation system.

4. What Was Superseded or Put Off for Later Superseded

- Windows-first UI Editor as final product.
- Native-widget Tool Editor as final product.
- GUI-only authoring workflows.
- Tooling that does not use the runtime/client presentation systems.
- Rendering-focused Universe Explorer without command/materialization/refinement law.

Recycled

- Visual layout authoring.
- Import/export.
- UI documents.
- CLI automation.
- `ops.json`-style command scripts.

- Codegen/stubs.
- Validation panels.
- Property inspectors.
- Hierarchy trees.
- Launcher/setup logical layout tests.

These now belong in Workbench modules and services.

Deferred

- Arbitrary third-party native plugins.
- Full OS-native GUI authoring as core Workbench behavior.
- Full Interface Studio before command/result/view/projection proof.
- Full Universe Explorer before package/replay/client/workbench foundations.
- Full robot civilization simulation before field layers, terrain deltas, machine graphs, and proof systems exist.
- Exact final physical directory structure while repo layout remains under review.

5. The Architecture We Converged On

The final conceptual architecture is:

Contracts define law.

Services execute behavior.

Commands expose behavior.

Documents hold editable state.

Patches mutate state lawfully.

Providers replace implementations.

Packs distribute authored payloads.

Modules declare capabilities.

Workspaces present workflows.

Apps compose products.

Artifacts preserve evidence and state.

Tests prove invariants.

AIDE governs repo work.

Codex executes bounded patches.

Workbench operates the system.

The key integration chain is:

User input / script / agent

-> command

-> capability/refusal law

-> service

-> document patch or typed result

-> diagnostics/evidence

-> view model

-> projection:

CLI

TUI

rendered GUI

OS-native GUI

-> same artifacts, same logs, same proof

Workbench fits here as the rich human/agent projection and authoring environment.

6. How CLI, TUI, Rendered GUI, Native GUI, and Headless Integrate

The shared truth is not a button or widget. The shared truth is a command/result/document/view contract.

Example:

```
command:  tools.validation.run
result:   validation_run_result.v1
view:    view.validation.summary
```

This can project as:

- CLI table or JSON.
- TUI panel.
- Rendered dashboard.
- Native list/detail view.
- Headless report/evidence packet.

Each projection has different affordances but the same semantics. This is how Workbench modules, server tools, setup, launcher, and client debug surfaces can reuse the same systems.

7. How the Rendered GUI Is Built

The rendered GUI pipeline is:

```
command/result/document state
-> view model
-> UI document / workspace layout
-> layout engine
-> style/theme resolver
-> control tree
-> draw command list
-> renderer backend
```

The renderer only knows primitives:

- rectangles,
- lines,
- text,
- images/bitmaps eventually,
- clipping,
- alpha,
- patterns,
- gradients,
- 9-slice-like panels,
- screenshot/capture.

The renderer does not know what a Validation Dashboard, Pack Browser, Windows XP theme, or Workbench module is. This keeps the renderer reusable by client, setup, launcher, server admin surfaces, Workbench, and future games.

8. How the TUI Is Built

TUI is a projection over the same views and commands. It uses:

- panels,
- split panes,
- tabs,
- tables,
- trees,
- forms,
- log views,
- command line,
- status bars,
- keyboard navigation,
- Unicode box drawing with ASCII fallback,

- limited color.

The TUI is not a toy fallback. It is the deterministic, keyboard-first, low-dependency surface for servers, recovery, SSH, CI, accessibility, AIDE/Codex supervision, setup fallback, launcher fallback, and client strategic/debug fallback.

9. How Workbench Modules and Workspaces Work

A module contributes:

- commands,
- documents,
- views,
- editors,
- inspectors,
- preview panes,
- validators,
- import/export definitions,
- patch generators,
- test fixtures,

- agent task templates,
- UI panels.

A workspace is a large user-facing composition made from modules and services.

Examples:

- Project Graph Explorer.
- Interface Studio.
- Module / Pack Foundry.
- App Composer.
- Release Forge.
- Agent Control Workspace.
- Renderer / Theme Laboratory.
- Replay / Trace Workspace.
- Universe Explorer.

Smaller reusable modules include:

- Command Browser.
- Document Inspector.
- Patch/Diff Viewer.

- Validation Runner.
 - Diagnostics Viewer.
 - Evidence Viewer.
 - Pack Browser.
 - Provider Browser.
 - Capability Matrix Viewer.
 - Theme Previewer.
 - Renderer Test Panel.
 - Asset Browser.
 - Project Graph View.
 - Build/Job Console.
 - Agent Task Queue.
-

10. How Workbench Builds Itself and Everything Else

The self-hosting staircase is:

1. Build `command/result/refusal` spine by hand.
2. Build `document/patch/transaction` spine by hand.
3. Build CLI projection by hand.
4. Build TUI projection enough for validation dashboard.
5. Build software-rendered draw-list and primitive controls.

6. Build rendered validation dashboard.
7. Use validation dashboard to monitor future work.
8. Build project graph viewer.
9. Use project graph viewer to understand/repair architecture.
10. Build theme lab + renderer sandbox.
11. Use them to harden rendered UI and themes.
12. Build Interface Studio.
13. Use Interface Studio to build Workbench panels.
14. Use Interface Studio to build launcher/setup/client/server UI docs.
15. Build Module/Pack Foundry.
16. Use it to create modules/packs/themes/templates.
17. Build App Composer.
18. Use it to assemble client/launcher/setup/server/workbench variants.
19. Build Agent Work Board.
20. Use it to create/review AIDE/Codex work units.
21. Build Release Forge.
22. Use it to package and prove releases.

This is progressive self-hosting. Workbench does not build itself from nothing. It gradually gains authority over safer artifact classes.

11. Workbench Modules for the Robot Seed-Civilization Game

The robot seed-civilization game direction is compatible with Workbench if game content is represented as real runtime artifacts.

Key Workbench spaces:

- Universe Explorer.
- Star System / Planet Generator Lab.
- Field Layer Stack Editor.
- Cut / Fill / Terrain Delta Lab.
- Construction Blueprint Studio.
- Machine Graph Compiler / Lab.
- Factory / Process Graph Editor.
- Robot Body / Spawn Fabricator Lab.
- Mothership Planner.
- Sensor / Fog-of-War Lab.
- Logistics / Infrastructure Planner.
- City / Civilization Planner.
- Governance / Claims / Permissions Lab.
- Performance / Complexity Budget Lab.
- Renderer / Unreal or external bridge lab.

These should produce artifacts that the runtime actually consumes:

- planet specs,

- field layer stacks,
 - terrain delta operations,
 - construction plans,
 - machine graphs,
 - factory graphs,
 - spawn lab descriptors,
 - mothership policies,
 - sensor/fog policies,
 - logistics graphs,
 - claim policies,
 - complexity budgets,
 - render snapshots,
 - evidence packets.
-

12. What Was Done in the Conversation

The conversation produced:

1. Multiple old UI Editor Codex prompts and prompt plans.

2. UU1-UU6 prompt plans for UI import/CLI/ops/launcher/setup tests.
3. A VS/Xcode/Linux IDE live-editing prompt.
4. A major architecture pivot to Workbench.
5. A full Workbench/product/presentation doctrine.
6. A TUI doctrine.
7. Theme and renderer architecture.
8. A component/service/provider/pack/module/workspace/app vocabulary.
9. Progressive self-hosting doctrine.
10. Universe Explorer doctrine.
11. Robot seed-civilization Workbench module roadmap.
12. A previous detailed preservation package with reports, registers, spec sheet, aggregator packet, and ZIP.
13. This companion summary package and ZIP.

13. What Was Put Off for Later

Items explicitly or implicitly delayed:

- Full Workbench GUI implementation.
- Interface Studio.
- Theme Laboratory.
- Renderer Sandbox.
- Universe Explorer 0.
- Third-party module/plugin runtime.
- Arbitrary native plugin support.
- Full OS-native GUI projection system.
- Full visual UI/HUD editing.
- Full robot civilization simulation.
- Unreal/external renderer bridge.
- Full package/mod trust runtime.
- Exact final folder layout.
- Exact final Workbench app classification.
- Current live repo gate verification.

14. What Is Missing or Needs More Thought

The plan is strong, but several important systems need formal specs:

1. Presentation contract

The minimal schema linking commands/results/documents to CLI/TUI/rendered/native/headless projections.

1. View/action binding model

How actions are exposed, validated, and mapped across modes.

1. Document/patch/transaction runtime

The safe mutation model for Workbench, CLI, TUI, agents, and migrations.

1. Projection conformance tests

Golden CLI output, TUI cell buffers, rendered layout trees/screenshots, and native view descriptors.

1. Module conformance kits

Modules need tests beyond descriptors.

1. **Provider conformance kits**

Renderers, storage, package loaders, input systems, etc. need standard conformance tests.

1. **Project graph service**

Workbench needs a system graph connecting files, contracts, commands, services, modules, packs, apps, tests, artifacts, and evidence.

1. **Localization/accessibility contracts**

String IDs, keyboard focus, screen reader labels, contrast, reduced motion, large text, and TUI fallback should not be bolted on late.

1. **Asset/license/provenance model**

Theme packs, sound packs, icon packs, and mod packs need origin/license/hash/provenance metadata.

1. **Security/trust model**

Data-only packs first, built-in compiled modules second, external adapters later, sandboxed plugins later, arbitrary native plugins last if ever.

1. **Performance budgets**

Every renderer/UI/module/game artifact should eventually expose sim/render/network/storage/pathfinding/expansion budgets.

1. Rollback/recovery model

For Workbench edits, package changes, releases, save migrations, and agent patches.

15. Key Risks

15.1 Workbench becomes authority If Workbench directly mutates source, runtime truth, or product behavior, it breaks the whole doctrine. It must issue commands and patches and rely on services, validation, and evidence.

15.2 UI modes drift If CLI/TUI/rendered/native/headless paths are built independently, products will diverge. Projection contracts and parity tests prevent this.

15.3 Modules become a junk drawer Without strict vocabulary, modules will absorb services, providers, apps, packs, and workspaces. The component/service/provider/pack/module/workspace/app/artifact split prevents this.

15.4 Repo-current facts go stale Several queue/foundation/current-state claims came from pasted context from other chats. They must be verified before implementation.

15.5 Universe Explorer becomes a renderer demo

If built as a free camera first, it may violate materialization, reference frame, interest, and truth/render separation. It must be a lawful inspection/projection surface.

15.6 Theme mimicry crosses legal lines OEM+mimic themes should use generic tokens and assets, not copied proprietary OS fonts/icons/textures/wallpapers.

16. Best Current Next Action

The safest immediate next action is:

Verify current live repo status and task gate state.

If gates allow, proceed with:

COMMAND-RESULT-VIEW-SLICE-01

Minimum proof:

registered command

-> typed result schema

-> view contract

-> projection descriptor

-> CLI output

- > TUI/text output
- > rendered-placeholder output
- > diagnostics/refusals/evidence

Recommended first command:

```
dominium.validation.run
```

Recommended first Workbench module:

Validation Dashboard

This is the smallest useful bridge from governance and command infrastructure to Workbench, TUI, rendered GUI, client shell, setup, launcher, and server admin surfaces.

17. Human-Readable Decision Table

Decision	Status	Why it matters
Old UI Editor/ Tool Editor no longer final products	Accepted	Avoids one-off native editor architecture
Workbench is the new production environment	Accepted	Unifies human, agent, CLI, TUI, GUI, modules, packs, releases
Workbench is not authority	Accepted	Preserves determinism, reuse, validation, evidence
CLI/TUI/ rendered/native/ headless share one spine	Accepted	Prevents duplicated product logic
TUI is first-class	Accepted	Supports servers, recovery, CI, SSH, agents
Rendered GUI uses software-first draw- list renderer	Accepted direction	Portable/testable baseline before GPU
Native GUI is optional projection	Accepted	Keeps OS SDK tools useful but non-authoritative
Themes are data/ token/control-skin profiles	Accepted direction	Moddable and renderer-neutral
Modules/packs/ providers/apps/ workspaces are distinct	Accepted direction	Prevents taxonomy drift
Workbench self- hosts progressively	Accepted	Avoids circular bootstrap
Universe Explorer	Recommended	Prevents renderer /

18. Reader Guidance

If you want to continue this work, do not restart with UI Editor. Start from Workbench and the presentation spine.

If you need a short formulation:

Dominium Workbench is the visual, TUI, CLI, and agentic production environment for building Domino/Dominium artifacts. It edits documents through patches, validates through contracts, previews through projections, packages through packs, and proves work through evidence. It is not authority; contracts and runtime services are authority.

If you need the immediate task:

Verify repo state, then generate `COMMAND-RESULT-VIEW-SLICE-01`.

If you need the long-term target:

Build Workbench modules and workspaces that can author every real runtime artifact

used by client, server, launcher, setup, Workbench, packs, mods, themes, Universe Explorer, and robot seed-civilization gameplay systems.

**8.2 Dominion Workbench
Presentation Spine Universe
Explorer Planning Human
Readable Report**

Source: docs/archive/conversations/Domino_Dominium_Workbench/prior_preservation_files/dominium_workbench_presentation_spine_universe_explorer_planning__01_human_readable_report.md
Report type: human_readable_report

Size: 34528 bytes

**8.2.1 COMPLETE CHAT PRESERVATION
REPORT - Dominion Workbench,
Presentation Spine, and Universe
Explorer Planning**

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion Workbench, Presentation Spine, and Universe Explorer Planning

- Row 2** Field: Date anchor; Assessment: 2026-05-27
Australia/Melbourne
- Row 3** Field: Source scope; Assessment: This chat only
unless labelled PROJECT-CONTEXT
- Row 4** Field: Apparent access; Assessment: Partial-to-
broad visible transcript context; not guaranteed
raw/full transcript
- Row 5** Field: Previously generated files available?;
Assessment: No confirmed previously generated
downloadable files before this package
- Row 6** Field: Uploaded files or artifacts present?;
Assessment: Yes: source.zip, screenshot bundles,
multiple image mockups, pasted preservation
request, pasted Universe Explorer note
- Row 7** Field: Contains future plans?; Assessment: Yes
- Row 8** Field: Contains decisions?; Assessment: Yes
- Row 9** Field: Contains pending tasks?; Assessment: Yes
- Row 10** Field: Contains unresolved questions?;
Assessment: Yes
- Row 11** Field: Staleness risk; Assessment: Medium-to-
high for live repo facts and external platform facts;
low for in-chat decisions
- Row 12** Field: Extraction confidence; Assessment: 4/5

Row 13 Field: Safe for later aggregation?; Assessment: Yes, with caveats

Row 14 Field: Main limitations; Assessment: The full raw transcript may not be accessible; some repo-state claims were pasted from other chats and were not independently verified in this preservation pass; screenshots were used as reference but not deeply re-processed here.

Plain-language limitation: this package reconstructs the conversation from the visible chat context and uploaded/pasted supporting files. It should be treated as a high-fidelity handoff for this chat, but not as an independently verified snapshot of the live GitHub repo. Any statement about the repo's current status, latest queue item, or latest gate should be verified before becoming formal project law.

1. One-Page Orientation

This chat began as a focused planning conversation about a Windows launcher UI problem and a proposed internal "Tool Editor" / "UI Editor." The early objective was concrete: build a graphical editor that could author pixel-perfect UI definitions, generate TLV or code, use true native widgets where necessary, and fix the mangled/flickering launcher. Over the course of the chat, the plan grew and then corrected itself several times.

The user clarified that Windows 10 was the immediate platform for the initial UI Editor, that the eventual tool should support Windows, macOS, and Linux, and that the UI system had to support both native-looking controls and custom rendered widgets. Early assistant output produced detailed Codex prompt plans for a Windows-only UI Editor, a future Tool Editor, TLV schema handling, codegen, layout, capability validation, and launcher/setup redesign tests.

The major turn in the conversation was the user's rejection of the old UI Editor / Tool Editor as final products. The user concluded that those were useful general ideas but poor final architecture. The conversation then reframed the direction around a cross-platform rendered Workbench: one modular, rendered, CLI/TUI/GUI/headless-capable tools environment using the same runtime, command, renderer, UI, pack, diagnostics, and evidence systems as the client. Workbench became a production environment for building Domino and Dominion, not just an editor. It would host modules such as Validation Dashboard, Project Graph Explorer, Agent Work Board, Pack Browser, Interface Studio, Renderer Sandbox, Theme Laboratory, Release Forge, Replay/Trace Viewer, and later domain-specific labs for planet generation, terrain, machines, factories, robot bodies, spawn fabricators, fog of war, logistics, and civilization systems.

A second major theme was unification of presentation surfaces. The user wanted CLI, TUI, rendered GUI, and OS-native GUI to share code and semantics. The decision emerged that these should not be four separate systems. Instead, all products should expose the same command/result/refusal/document/view spine, while CLI, TUI, rendered GUI, native GUI, and headless modes are projections over that spine. This unifies the client, server, setup, launcher, Workbench, validators, AIDE/Codex tasks, and future admin tools.

A third major theme was portability and future-proofing. The user repeatedly emphasized that Domino should be reusable for different games, not just Dominion. The architecture therefore separated terms: components, services, providers, packs, modules, workspaces, apps, artifacts, commands, documents, patches, and evidence. The guiding idea became that path is not identity, implementation is not contract, UI is not authority, and generated output is not source truth. Reuse should be achieved through contracts, registries, commands, capabilities, modules, packs, providers, and conformance tests rather than through fragile folder references.

A fourth theme was the robot seed-civilization / universe-scale game vision. The user introduced a large pasted design about Domino as deterministic authority and Unreal or other renderers as optional high-fidelity clients, plus a robot colonization game built around

motherships, nanobot construction, physical spawning, fog of war, terrain cut/fill, machine graphs, factory graphs, and civilization/infrastructure progression. The chat concluded that Workbench can and should eventually contain modules to author and test these artifacts: planet specs, field layers, terrain deltas, construction blueprints, machine graphs, factory process graphs, spawn systems, sensor/fog models, logistics/infrastructure, governance/claims, performance budgets, and universe exploration.

The most important final state is that Workbench should be built by progressive self-hosting. It does not build everything from nothing. First come the foundation gates and command/result/view/projection slice. Then package mounting, replay proof, barebones client shell, and minimal Workbench shell. Then Workbench modules begin editing safe artifacts. Later Workbench edits its own layouts, themes, modules, app compositions, packs, and eventually creates AIDE/Codex work units for code changes. The chat's lasting contribution is a unified architecture for building the project through the same systems that the project itself will expose to players, modders, developers, servers, tools, agents, and future games.

2. The Story of the Conversation

2.1 Original UI Editor / Tool Editor phase The chat opened with the user asking for help designing an internal "Tool Editor" that could visually design pixel-perfect UIs and broad-strokes functionality for setup, launcher, game, and other tools. The first questions focused on platforms, languages, native widgets, editor UX, output files, codegen, and dependency constraints. The user supplied details about the existing Dominium UI/DUI TLV system, Win32/DGFX/null backends, likely flicker paths, and current launcher files.

A Windows-first plan emerged: Phase A would be a minimal Windows UI Editor to generate canonical TLV; Phase B would be a fuller Tool Editor that edits DUI schemas/states directly. Detailed Codex prompts were generated for scaffolding, IR, TLV, capability validation, layout, widgets, action codegen, UI Editor, Tool Editor bootstrap, Win32 batching/flicker fixes, and hardening. Later the user expanded the UI Editor requirements to include import/export of existing tools, a headless CLI/ops.json interface for Codex, and "Minecraft-style" logical launcher/setup layouts using native controls only.

2.2 Pivot away from native-widget editor products

The user then determined that the old UI Editor and Tool Editor were good general ideas but bad final

products. This pivot was central. The plan moved away from OS-native widget authoring as the core direction. The user wanted cross-platform tools using the same CLI/TUI/rendered GUI path as the client, rendered by software first and hardware later, not reliant on a specific OS SDK or native widgets.

The discussion reframed the product as Dominion Workbench: one rendered, modular, command-driven development environment for building Domino, Dominion, packs, modules, UI, tests, themes, assets, releases, and agent workflows. The Workbench would be an integrated shell, not a monolithic editor.

2.3 Unified product surfaces and presentation

spine The conversation then developed a general rule: all apps should share the same command/result/refusal/document/view spine. CLI, TUI, rendered GUI, headless mode, and optional OS-native GUI should be projections of that spine, not independent implementations. This addressed client, server, launcher, setup, Workbench, validators, future tools, and native SDK GUI bridges.

OS-native GUI authoring with Visual Studio/Xcode remained valid, but native GUI was demoted to an optional projection. Rendered GUI became the core shared route for client and Workbench. TUI was treated as a first-class low-dependency projection, not a toy fallback. CLI/headless remained mandatory for

automation, tests, AIDE, and Codex.

2.4 Workbench modules and workspaces The user proposed early Workbench modules: Project Editor, UI Editor, Module Editor, and App Editor. The conversation refined these into larger user-facing workspaces built from smaller reusable modules and services. Names evolved into Project Graph Explorer, Interface Studio, Module/Pack Foundry, App Composer, Validation Dashboard, Agent Work Board, Pack Browser, Renderer/Theme Laboratory, Build/Test Console, Release Forge, Replay/Trace Viewer, and later domain-specific labs.

A vocabulary split emerged: component, service, provider, pack, module, workspace, app, artifact. This prevents "module" from becoming a junk drawer. Workbench modules are one consumer of the general module system, not the general module system itself.

2.5 UI, theme, renderer, and TUI architecture

The conversation specified a modular UI stack: view model, view document, layout, control registry, widget registry, style tokens, theme profiles, projection adapters, and renderer draw lists. The renderer should consume primitives like rects, text, lines, images, clips, alpha, patterns, and 9-slice commands; it should not know about modules or product semantics.

The user wanted OEM+ theme profiles that mimic OS-native styles for Windows, Linux, and macOS eras. The plan accepted this as a strong system test: theme profiles are data/control-skin/layout metric packs, not renderer backends and not copied proprietary assets. Built-in primitive-only themes such as barebones, instrument, paper, terminal, blueprint, high contrast, tactical, phosphor, ledger, and gridline can work without external assets.

TUI received a detailed doctrine: it should be a terminal-native projection of the same operating environment, with panels, tables, split panes, command input, diagnostics, logs, server/admin views, setup recovery, launcher TUI, client strategic fallback, Agent Work Board, and Replay Viewer. It should support Unicode and ASCII fallback and always preserve the command/result/refusal spine.

2.6 Engineering law and foundation gates The user then pushed for portability, modularity, extensibility, and replaceability like a proper game/OS-scale project rather than a one-off indie tool. The conversation developed engineering principles: public surfaces, API/ABI canon, dependency direction, command surfaces, diagnostics, artifact identity, schema/protocol evolution, capability/refusal law, provider model, document/patch/transaction law, module

composition law, replacement protocol, version/deprecation law, mod/pack trust model, and portability matrix.

Several queue plans were discussed. Later pasted context stated that foundation scaffolding had happened and the active blocker or next task had changed over time. The final preserved task sequence settled around the need to proceed with COMMAND-RESULT-VIEW-SLICE-01, subject to whatever current gate status applies, because that slice is the bridge between command services and projection/UI/Workbench authoring.

2.7 Universe Explorer and game systems The final phase introduced a Universe Explorer and then a robot seed-civilization game vision. Universe Explorer was framed as the first lawful window into the world, not the world itself. It should prove reference frames, materialization, inspection, no modal loading, fidelity degradation, representation continuity, interest sets, and renderer purity.

The robot seed-civilization direction then introduced player robots, mothership knowledge and constraints, nanobot construction actuators, physical spawn fabricators, terrain cut/fill, field layers, machine graph compilers, factory/process graphs, fog-of-war/sensor models, logistics, infrastructure, governance/claims, and planet generation labs. The answer concluded that

Workbench modules and workspaces can be created to build all of this, but they should produce artifacts the actual runtime consumes.

3. Main Topics Discussed

Topic 1 - Original Windows UI Editor and Tool Editor The chat began with a practical UI tool design. The user wanted a Windows launcher UI fixed and an internal editor capable of producing pixel-perfect, DPI-aware, extensible UI definitions. We explored native Win32 widgets, DUI TLV, codegen, layout engines, stable IDs, canonical TLV, JSON mirrors, action stubs, and imports. This was later superseded as a final product direction but remains relevant as background for Interface Studio and presentation artifact authoring.

Topic 2 - Dominion Workbench Platform Workbench became the replacement for UI Editor/Tool Editor. It is a cross-platform, rendered, modular, command-driven production environment. It is not a monolithic IDE or GUI-only editor. It hosts modules and workspaces over shared services, commands, documents, patches, providers, packs, diagnostics, and evidence. It should eventually let a sole developer and agents build code, data, packs, UI, themes, tests, release artifacts, and future games.

Topic 3 - Unified CLI/TUI/Rendered/Native

Presentation Spine A major conclusion was that all apps should expose one command/result/refusal/document/view spine. CLI, TUI, rendered GUI, OS-native GUI, and headless operation are projections. This preserves semantic parity and reuse across client, server, launcher, setup, Workbench, validators, and future tools. Native GUI remains optional; rendered GUI and TUI are shared runtime surfaces.

Topic 4 - Modular

UI/Layout/Control/Theme/Renderer System

The chat specified a data-driven UI architecture: layouts as data, controls as registered capabilities, themes as token overlays, widgets as compositions, views as typed result/document projections, and renderers as primitive draw-list consumers. OEM+ mimic themes for Windows, macOS, and Linux were identified as an ambitious but valuable test of renderer/UI extensibility.

Topic 5 - TUI as First-Class Projection

TUI should be a deterministic, keyboard-first, low-dependency projection of the operating environment. It supports the same modules and commands as GUI where practical. It is especially useful for servers, SSH, recovery, setup, CI, agent workflows, and low-end systems. It should not be treated as a second-class

ASCII clone.

Topic 6 - Engineering Constitution / Foundation

Queue The user wanted portability and replaceability like a proper OS or large game engine. The discussion produced a foundation law queue: public surfaces, ABI/API, dependency direction, commands, diagnostics, artifact identity, schemas/protocols, document/patch/transactions, capabilities/refusals, providers, modules, replacement protocols, version/deprecation, mod/pack trust, and portability. This is intended to keep Domino reusable beyond Dominium.

Topic 7 - Modules, Packs, Providers, Apps,

Workspaces The chat clarified vocabulary.

Components are source/build ownership units. Services are callable runtime capabilities. Providers are replaceable implementations. Packs are distributable authored payloads. Modules are declared functional extension units. Workspaces are large Workbench compositions. Apps are shipped product compositions. Artifacts are versioned persisted things. This vocabulary is essential for avoiding future architecture drift.

Topic 8 - Progressive Self-Hosting Workbench should not build itself from nothing. It should start from a hand-coded seed substrate, then prove commands/

results/views/projections, then minimal Workbench shell, then safe artifact editing, then editing its own layouts/themes/modules, then product UIs, packs, modules, releases, and agent WorkUnits. This avoids circular bootstrapping.

Topic 9 - Universe Explorer Universe Explorer was framed as a lawful inspection/materialization/reference-frame proof, not a free-camera renderer demo. It should prove scale continuity, reference-frame correctness, no modal loading, explicit materialization, renderer purity, representation auditability, and evidence export. It should initially be a Workbench module/workspace and later support client/worldgen/server/admin/replay tools.

Topic 10 - Robot Seed-Civilization Game Systems The game direction evolved toward robot avatars seeded by a mothership, physical spawn fabrication, nanobot construction as lawful actuator, field-layer planets, sparse cut/fill terrain, machine graphs, factories as process graphs, sensor/fog truth models, infrastructure/logistics, and civilization building without simulating human labor NPCs first. Workbench modules can author/test these artifacts.

4. What We Were Trying to Achieve

4.1 Explicit Goals

- Build or plan tools to create and improve Dominion UIs.
- Replace defective launcher UI/flicker with a better architecture.
- Build a cross-platform tools environment using the same CLI/TUI/rendered GUI stack as the client.
- Make the system modular, portable, extensible, reusable for other Domino-based games and even other engine projects.
- Support graphical and agentic development through Workbench, AIDE, and Codex.
- Unify code/data/modules/packs/apps/themes across client, server, setup, launcher, and tools.
- Build Workbench modules that can author the actual game systems and runtime artifacts.

4.2 Inferred Goals

- Avoid another directory/refactor disaster by establishing governance first.
- Make Workbench a test harness for the client architecture.
- Make modder/player expectations first-class: themes, packs, modules, UI/HUD authoring, game system extension.

- Avoid vendor/editor lock-in while still using Visual Studio/Xcode for native GUI work where useful.

4.3 Goals That Changed Over Time The goal changed from a Windows-first TLV UI Editor to a cross-platform rendered Workbench. Native-widget UI authoring was deprioritized as the final product direction, while rendered GUI, TUI, command surfaces, modules, packs, documents, patches, evidence, and progressive self-hosting became the core.

4.4 Goals Still Unresolved

- Exact current repo gate status must be verified before implementation.
- Exact final directory layout remains under review.
- Presentation contract details are not fully specified.
- Module/pack/provider runtime loading trust model remains staged.
- Workbench implementation order after COMMAND-RESULT-VIEW-SLICE-01 remains dependent on review.

5. Decisions Made and Why

- Row 1** ID: DECISION-01; Decision: Abandon old UI Editor/Tool Editor as final products; recycle concepts into Workbench modules; Status: Accepted; Why it mattered: Prevents Windows/native-widget one-off tool drift; Confidence: High; Label: FACT
- Row 2** ID: DECISION-02; Decision: Build Dominion Workbench Platform as modular rendered production environment; Status: Accepted conceptually; Why it mattered: Gives one reusable production surface for project, tools, modules, packs, agent work; Confidence: High; Label: FACT/INFERENCE
- Row 3** ID: DECISION-03; Decision: Center architecture on contracts/commands/services/documents/patches/providers/packs/modules/artifacts/proof, not Workbench; Status: Accepted; Why it mattered: Keeps Workbench as surface, not authority; Confidence: High; Label: FACT
- Row 4** ID: DECISION-04; Decision: CLI, TUI, rendered GUI, OS-native GUI, and headless are projections of one spine; Status: Accepted; Why it mattered: Enables reuse and prevents GUI-only semantics; Confidence: High; Label: FACT

Row 5 ID: DECISION-05; Decision: OS-native GUI remains optional projection; rendered GUI is core shared path; Status: Accepted; Why it mattered: Lets Visual Studio/Xcode remain useful without defining the core architecture; Confidence: High; Label: FACT

Row 6 ID: DECISION-06; Decision: TUI is first-class, not fallback-only; Status: Accepted; Why it mattered: Supports server/admin/CI/recovery/SSH/low-end/agent workflows; Confidence: High; Label: FACT

Row 7 ID: DECISION-07; Decision: Use progressive self-hosting; Status: Accepted; Why it mattered: Avoids circular "Workbench builds itself from nothing"; Confidence: High; Label: FACT

Row 8 ID: DECISION-08; Decision: Use module/pack/provider/app/workspace vocabulary; Status: Accepted; Why it mattered: Prevents module/tool/pack junk-drawer confusion; Confidence: High; Label: FACT

Row 9 ID: DECISION-09; Decision: Build code-only primitive themes first; richer packs/assets optional; Status: Accepted conceptually; Why it mattered: Ensures portable baseline and modder extensibility; Confidence: Medium-high; Label: FACT/INFERENCE

Row 10 ID: DECISION-10; Decision: Universe Explorer should be lawful inspection/materialization proof, not renderer demo; Status: Accepted in assistant recommendation; user asked follow-up; Why it mattered: Important for scale and world truth doctrine; Confidence: Medium; Label: INFERENCE

6. Ideas Considered but Rejected, Superseded, or Deprioritised

- Windows-only native UI Editor as final product: superseded. It remains useful as historical bootstrap thinking but not final direction.
- Tool Editor as separate all-in-one editor: superseded by Workbench platform and modules.
- OS SDK native widgets as core cross-platform tool strategy: deprioritized. Native GUI remains optional projection.
- Renderer/free-camera universe as first world implementation: rejected in favor of lawful Universe Explorer.
- Workbench as architecture center: rejected. Contracts/commands/services/proof are the center.
- Arbitrary native plugins early: deferred until trust/sandbox/capability law matures.

- Pixel-perfect proprietary OS theme reproduction: rejected as unsafe/unnecessary; OEM+ mimic through generic assets/tokens is acceptable.

7. Important Reasoning, Rationale, and Tradeoffs

The visible rationale centered on avoiding architectural drift. A tool that directly edits private state or owns semantics would become another isolated subsystem. A command/result/refusal spine lets CLI, TUI, GUI, headless, tests, and agents share behavior. A document/patch/transaction model lets visual edits and agent patches remain auditable. A provider/capability/refusal model keeps platforms, renderers, storage, input, UI, and modules replaceable. A pack/module/app descriptor system lets future games and mods reuse Domino without hardcoding Dominion-specific assumptions.

The major tradeoff is upfront governance versus implementation speed. The chat repeatedly balanced not over-governing the project against preventing another refactor disaster. The chosen strategy is foundation first, then narrow product slices. Workbench comes after command/result/view/projection proof because otherwise it would build UI over unstable semantics.

8. Plans, Future Work, and Next Steps

Current best near-term sequence, subject to repo verification:

1. `COMMAND-RESULT-VIEW-SLICE-01` - prove one real command, typed result, view contract, projection descriptors, CLI/text/rendered-placeholder parity, diagnostics/evidence.
2. `PHASE-REVIEW-02` - assess whether presentation/projection law is adequate.
3. `PACKAGE-MOUNT-SLICE-01` or `PRESENTATION-CONTRACT-01` depending on the review.
4. `REPLAY-PROOF-SLICE-01`.
5. `BAREBONES-CLIENT-SHELL-01`.
6. `WORKBENCH-SHELL-01`.
7. First Workbench modules: Validation Dashboard, Project Graph Explorer, Agent Work Board, Pack Browser.
8. Later authoring modules: Interface Studio, Theme Laboratory, Renderer Sandbox, Module/Pack Foundry, App Composer, Release Forge.
9. Later world/game modules: Universe Explorer, Field Layer Stack Editor, Planet Generator Lab,

Terrain Delta Lab, Construction Blueprint Studio,
Machine Graph Lab.

If the live repo still has dependency-direction blockers,
repair those before product slices.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences

- User wants rigorous, source-grounded, direct, audit-ready answers.
- User wants modularity, portability, extensibility, replaceability, backwards compatibility, and reuse across other Domino games/projects.
- User does not want one-off indie project practices.
- User values CLI/TUI/rendered/native/headless parity.
- User wants Workbench to build real artifacts consumed by runtime, not pretty mockups.
- User wants AIDE/Codex work to be visible, structured, governed, and evidence-producing.

9.2 Inferred Constraints and Preferences

- Prefer contracts and validators over prose-only docs.

- Prefer data-driven UI/theme/module/packs where safe.
- Prefer code-only baseline with optional packs for richer assets.
- Prefer foundation gates before broad Workbench implementation.

9.3 Uncertain or Unestablished Preferences

- Exact naming of final Workbench product.
- Exact final paths while repo layout remains under review.
- Exact degree of native GUI support per app.
- Exact renderer backend priority beyond software-first.

10. Files, Artifacts, Outputs, and Prompts

Important artifacts from the chat include:

- **source.zip**: uploaded repo/source snapshot; used early for UI/DUI context but not deeply preserved here.
- **SetupC.zip** and **LauncherC.zip**: uploaded screenshot bundles for setup/launcher logical layout references.

- Multiple image mockups: visual references for Workbench modules, themes, TUI, UI/UX directions.
- `Pasted markdown.md`: the user's detailed preservation-package instructions for this response.
- `Pasted text.txt`: a pasted Universe Explorer / reference-frame / materialization note used in the later discussion.
- Generated in-chat Codex prompts: Prompt 1-11 for old UI Editor plan, UU1-UU6 for import/CLI/ops/launcher/setup/hardening, IDE live-editing prompt, and many planning prompt sequences.
- This preservation package: new downloadable Markdown/YAML/ZIP files created by this task.

11. Open Questions and Unresolved Issues

- What is the verified current repo gate status?
- Has `COMMAND-RESULT-VIEW-SLICE-01` already started, or is it next?
- Does dependency-direction strict still block Foundation Closeout?
- What exact physical directories will be used once repo layout review stabilizes?

- What minimal projection descriptor schema should be adopted?
- What is the earliest practical Workbench rendered shell milestone?
- How should Workbench module descriptors interact with general module descriptors?
- What sandbox/trust model will eventually allow third-party modules?
- How soon should Universe Explorer become a Workbench module?

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Future assistants might incorrectly restart the old UI Editor/Tool Editor plan, treat Workbench as authority, treat GUI as the primary semantics, ignore TUI/CLI/headless parity, overclaim current repo status, lock directory paths prematurely, conflate modules with packs/providers/services, or build Universe Explorer as a renderer demo. Avoid this by preserving the central spine: contracts -> commands -> services -> documents/patches -> providers -> packs/modules/apps -> artifacts/evidence -> tests/gates.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat contributes the Workbench doctrine, presentation spine, UI/theme/rendering architecture, module/pack/app/provider terminology, progressive self-hosting plan, TUI doctrine, OEM+ theme strategy, engineering-law framing, and Workbench modules for robot seed-civilization systems. It should feed spec-book chapters on Workbench Platform, Presentation Architecture, Module/Packs/Providers, Engineering Governance, UI/Theme/Renderer, TUI, AIDE/Codex Workflow, Universe Explorer, and Robot Seed-Civilization Design.

14. What I Should Remember

- The old UI Editor / Tool Editor was superseded as a final product.
- Workbench is the new production environment, but it is not authority.
- CLI, TUI, rendered GUI, native GUI, and headless must be projections of the same command/result/refusal/document/view system.
- TUI is first-class.
- Rendered GUI uses software renderer first and hardware later.

- Themes are data/token/control-skin profiles; OEM+ mimic is allowed without copied proprietary assets.
- Everything edited should become a document patch validated and committed with evidence.
- Modules, packs, providers, apps, workspaces, artifacts, services, and components are distinct.
- Progressive self-hosting is the only safe bootstrapping path.
- Universe Explorer should prove lawful inspection/materialization, not just free camera rendering.
- Workbench can author game systems if those systems are real runtime artifacts.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- "Explain the difference between module, pack, provider, service, component, workspace, app, and artifact again with examples."
- "What parts of the old UI Editor plan should be recycled into Interface Studio?"

15.2 Decisions

- "Which decisions are final versus tentative?"
- "What would cause us to revisit the Workbench-first direction?"

15.3 Tasks and Next Actions

- "Write the Codex/AIDE prompt for COMMAND-RESULT-VIEW-SLICE-01."
- "Write the no-apply plan for PRESENTATION-CONTRACT-01."

15.4 Artifacts and Files

- "List every prompt generated in this chat and what it was for."
- "Create a compact artifact map from this preservation package."

15.5 Risks and Verification

- "What repo-current facts in this chat must be verified before implementation?"
- "What are the top 10 ways Workbench could accidentally become architectural authority?"

15.6 Future Spec Book / Aggregation

- "Which sections of a master Project Spec Book should this chat feed?"
- "What overlaps should be reconciled with other chats?"

15.7 Deep-Dive Questions Specific to This Chat

- "Design the minimal presentation descriptor schema."
- "Design the Validation Dashboard vertical slice."
- "Design the Field Layer Stack Editor artifact model."

16. Compact Human Summary

This chat began with a concrete UI tooling problem: the Dominion launcher UI looked mangled and flickered, and the user wanted a Windows-first UI Editor / Tool Editor to author pixel-perfect UIs and generate DUI/TLV code. We explored that plan in detail: platform choices, native widgets, canonical TLV, JSON mirrors, layouts, stable IDs, action codegen, import/export, CLI ops scripts, launcher/setup redesign tests, and Codex prompt plans.

The plan then changed substantially. The user concluded that the old UI Editor and Tool Editor were useful ideas but not good final products. The conversation moved toward a cross-platform rendered Workbench: one modular production environment using the same CLI, TUI, rendered GUI, command, pack, diagnostics, renderer, and evidence systems as the client. Workbench should not be a Visual Studio clone, not a native-widget-first editor, and not a one-off GUI app. It should be a modular shell over shared services and commands.

The central architectural conclusion was that CLI, TUI, rendered GUI, OS-native GUI, and headless execution should all be projections of the same command/result/refusal/document/view system. This applies to client, server, launcher, setup, Workbench, validators, AIDE/Codex workflows, and future admin/modding tools. Workbench becomes the richest projection and authoring environment, but it is not semantic authority.

The chat also established a vocabulary and modularity model. Components are source/build ownership units. Services are callable runtime capabilities. Providers are replaceable implementations. Packs are authored distributable payloads. Modules are functional extension units. Workspaces are user-facing Workbench compositions. Apps are product compositions. Artifacts are persisted versioned things. This vocabulary prevents "module" from becoming a junk drawer.

The UI architecture became data-driven and renderer-neutral: view models, view documents, layout engines, controls, widgets, style tokens, themes, and projection descriptors are separate. The rendered GUI should use a software renderer baseline and later hardware backends. TUI is a first-class projection. OS-native GUI is optional and should still call the command/view spine. Themes can mimic OEM+ Windows/macOS/Linux styles through generic tokens, metrics, and skins without copying proprietary assets.

The user also emphasized portability and future-proofing. The project should be built like a proper OS/game engine, not a one-off indie project. This led to engineering-law queues around public surface registry, API/ABI canon, dependency direction, commands, diagnostics, artifact identity, schemas/protocols, document/patch/transaction law, capabilities/refusals, providers, module composition, replacement, versioning, mod/pack trust, and portability.

Later, the discussion connected Workbench to game systems. The user introduced a universe-scale/robot seed-civilization concept: players are robots seeded by a mothership, using nanobots, fabricators, physical spawn labs, field-layer planets, terrain cut/fill, machine graphs, factories, fog-of-war sensors, logistics, infrastructure, governance, and civilization-scale development. The conclusion was that Workbench modules can build all of

this if they author runtime-consumed artifacts rather than mockups. Important future modules include Universe Explorer, Planet Generator Lab, Field Layer Stack Editor, Terrain Delta Lab, Construction Blueprint Studio, Machine Graph Lab, Factory Graph Editor, Spawn Fabricator Lab, Fog-of-War Lab, Logistics Planner, City/Civilization Planner, and Governance/Claims Lab.

The best next step is not a big Workbench GUI. It is the smallest projection proof: COMMAND-RESULT-VIEW-SLICE-01, unless the live repo gate still requires dependency-direction repair first. That slice should prove one real command, typed result, view contract, projection descriptors, CLI output, TUI/text output, rendered-placeholder output, diagnostics, and evidence. This is the bridge from foundation law to future Workbench, client, launcher, setup, server, and game UI.

8.3 **Dominium Workbench Presentation Spine Universe Explorer Planning Reader Brief**

Source: docs/archive/conversations/Domino_Dominium_Workbench/prior_preservation_files/dominium_workbench_presentation_spine_universe_explorer_planning__06_reader_brief.md

Report type: reader_brief

Size: 1996 bytes

8.3.1 Reader Brief - Dominion Workbench, Presentation Spine, and Universe Explorer Planning

This chat is a major planning pivot. It began with a Windows UI Editor/Tool Editor design and ended with a broader Dominion Workbench Platform architecture. The old editor plan is superseded as a final product. Its useful pieces should be recycled into Workbench modules.

Top things to know:

1. Workbench is not authority; it is a projection over contracts/services/commands/documents/evidence.
2. CLI, TUI, rendered GUI, native GUI, and headless should share one semantic spine.
3. TUI is first-class.
4. Rendered GUI is built on software renderer first; hardware later.
5. Native SDK GUIs remain optional projections.
6. UI is data-described through views, layouts, controls, widgets, themes, style tokens, and projections.

7. Visual edits should produce document patches and evidence.
8. Components, services, providers, packs, modules, workspaces, apps, and artifacts are distinct.
9. Workbench modules are built from smaller reusable services/views/documents.
10. Progressive self-hosting is required.
11. The immediate likely slice is COMMAND-RESULT-VIEW-SLICE-01, subject to repo verification.
12. Validation Dashboard is the best first Workbench module.
13. Project Graph Explorer and Agent Work Board are early high-value modules.
14. Interface Studio and Theme Lab come after projection/runtime proof.
15. Universe Explorer should be lawful inspection/materialization, not renderer free camera.
16. Robot seed-civilization systems can be built as runtime artifacts and Workbench labs.
17. Code-only primitive themes can support useful UI; richer packs/assets are optional.

18. OEM+ themes are mimic profiles, not copied OS assets.
19. Current repo facts in pasted contexts need verification.
20. The future spec book should treat this chat as a core Workbench/presentation architecture source.

Best next step: verify current repo state, then draft
COMMAND-RESULT-VIEW-SLICE-01 if gates allow.

8.4 Dominium Workbench Presentation Spine Universe Explorer Planning In Chat Reader

Source: docs/archive/conversations/Domino_
Dominium_Workbench/prior_preservation_files/
dominium_workbench_presentation_spine_universe_
explorer_planning__09_in_chat_reader.md

Report type: in_chat_reader

Size: 1484 bytes

8.4.1 In-Chat Reader - Dominion Workbench, Presentation Spine, and Universe Explorer Planning

Package overview

This package preserves a chat that pivoted from a Windows UI Editor plan to a modular Dominion Workbench Platform architecture. It captures decisions, tasks, risks, artifacts, open questions, and spec-book integration guidance.

File index

- 00 Manifest: package inventory.
- 01 Human-readable report: full narrative and sections 0-16.
- 02 Context transfer packet: future-chat handoff.
- 03 Spec sheet: YAML-style structured data.
- 04 Registers: workstreams, decisions, tasks, risks, artifacts.
- 05 Aggregator packet: compact merge packet.
- 06 Reader brief: quick overview.
- 07 Verification and audit: self-audit and verification items.

- 08 Future chat bootstrap prompt: prompt for next chat.
- 09 In-chat reader: this guide.

Question menu

- What was the biggest decision in this chat?
- What superseded the old UI Editor plan?
- What is COMMAND-RESULT-VIEW-SLICE-01 supposed to prove?
- What are the first Workbench modules?
- How does Universe Explorer fit?
- What needs verification before implementation?

Top things to preserve

Workbench as projection, unified presentation spine, progressive self-hosting, module/pack/provider/app vocabulary, document/patch/evidence model, and the old UI Editor supersession.

Safest next actions

Verify live repo state. Then draft a bounded AIDE/Codex work packet for COMMAND-RESULT-VIEW-SLICE-01 if allowed.

Chapter 9

Foundation Workbench Codex

9.1 Dominion Foundation
 Workbench Parallel Codex
 Preservation Human Readable
 Report

Source: docs/archive/conversations/Foundation_Workbench_Codex/Dominium_Foundation_Workbench_Parallel_Codex_Preservation__01_human_readable_report.md

Report type: human_readable_report

Size: 34540 bytes

9.1.1 COMPLETE CHAT PRESERVATION
 REPORT - Dominion Foundation
 Lock, Workbench Spine, and Parallel
 Codex Handoff

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion Foundation Lock, Workbench Spine, and Parallel Codex Handoff

- Row 2** Field: Date anchor; Assessment: 2026-05-27
Australia/Melbourne
- Row 3** Field: Source scope; Assessment: This chat only
unless labelled PROJECT-CONTEXT
- Row 4** Field: Apparent access; Assessment: Partial /
high-context
- Row 5** Field: Previously generated files available?;
Assessment: No prior downloadable package was
available to this run; prior in-chat handoff text was
visible
- Row 6** Field: Uploaded files or artifacts present?;
Assessment: Yes - current uploaded preservation
prompt is available; some older uploads may have
expired
- Row 7** Field: Contains future plans?; Assessment: Yes
- Row 8** Field: Contains decisions?; Assessment: Yes
- Row 9** Field: Contains pending tasks?; Assessment: Yes
- Row 10** Field: Contains unresolved questions?;
Assessment: Yes
- Row 11** Field: Staleness risk; Assessment: Medium
- Row 12** Field: Extraction confidence; Assessment: 4 / 5
- Row 13** Field: Safe for later aggregation?; Assessment:
Yes, with caveats

Row 14 Field: Main limitations; Assessment: The visible transcript contains extensive excerpts and generated prompts, but some earlier turns were elided as "Skipped"; live repo state after the latest pasted transcript was not independently rechecked in this preservation step; some older uploaded files have expired.

Plain-language limitation statement:

This report reconstructs the visible substance of this chat and the user-uploaded preservation instructions. It does not claim access to every hidden or skipped turn. It also does not claim to have rerun live GitHub checks at the end of the chat. When repo state is described, the report distinguishes between state that was explicitly checked earlier in the conversation and state that was later pasted by the user. Some previous uploaded files are no longer available; if the user wants those exact files incorporated later, they should be re-uploaded.

The current uploaded preservation instruction file is part of the source context for this response.

?filecite?turn85file0?

1. One-Page Orientation

This chat was a high-intensity strategic, architectural, and operational continuation of the Dominion project: a large simulation/game/engine ecosystem in the

Julesc013/dominium repository. The user's central concern was that the project had spent a very long time in refactor and restructuring work, and that without a clear, enforceable architecture and faster execution model, the project would either drift back into chaos or stall forever before reaching real Workbench and gameplay/product work.

The conversation began from the aftermath of extensive repo cleanup, AIDE installation, build proof, internal pilot proof, and directory/root restructuring work. Earlier in the project, Dominium had a large number of root-level folders such as `core/`, `control/`, `data/`, `packs/`, `profiles/`, `bundles/`, `lib/`, `libs/`, `validation/`, `meta/`, `governance/`, `net/`, and others. The user was extremely frustrated by how long the cleanup took and repeatedly threatened to manually move folders if the process did not converge. This created the need for a more aggressive but still governed model: deterministic routing of bad-root files, quarantine for unknowns, canonical roots, and later a move away from physical folder cleanup into contract/governance hardening.

The conversation ultimately converged on a stable architectural model: **Domino** is the reusable deterministic/runtime substrate; **Dominium** is the game/product family built on Domino; **Workbench** is the production, validation, editing, inspection,

packaging, and evidence environment; **AIDE** is the repo/control-plane harness; **Codex** is a bounded patch executor; **Contracts are law**; and **tests, replay, diagnostics, and evidence are proof**. The most important principles became: path is not identity; implementation is not contract; UI is not authority; generated output is not source truth.

The project's root skeleton and governance spine were treated as mostly established by the end of the chat. Foundation Lock was originally blocked by dependency-direction strict validation, with 358 violations and 38 warnings. Later the user and assistant verified/preserved a state where **FOUNDATION-CLOSEOUT-02** closed Foundation Lock with warnings: dependency-direction strict had 0 violations and 68 warnings, fast strict passed, CMake configure/build and smoke CTest passed through fast strict, and full CTest remained T4/full-gate debt. Narrow product slices were authorized, but broad feature work remained blocked.

A major operational shift occurred near the end of the chat: instead of queueing all prompts sequentially, the user decided to run many tasks in parallel through multiple Codex chats and git worktrees. The assistant defined strict parallel-worker rules to prevent conflicts: no direct pushes to **main**, no edits to global AIDE latest-status files, task-local evidence only, explicit allowed paths, targeted validators instead of full test suites, and

coordinator-controlled serial merges.

Several Wave 1 parallel prompts were generated: SERVICE-CONFORMANCE-LAW-01, DOCUMENT-PATCH-TRANSACTION-RUNTIME-01, PROJECT-GRAPH-SERVICE-01, COMPOSITION-RESOLVER-LAW-01, and DOMINIUM-DOCTRINE-RECOVERY-MATRIX-01. A later pasted transcript stated that Wave 1 and WORKBENCH-VALIDATION-SLICE-01 were effectively complete, and that the next task should be COMMAND-RESULT-VIEW-SLICE-01. That task is now the most important next action unless live repo verification contradicts it.

The future relevance of this chat is high. It contains the transition from repo cleanup to governed product development; the final architectural vocabulary; the current parallel Codex workflow; the task ordering for Workbench/product spine; the rejected paths; the user's frustration and speed requirements; and a doctrine recovery thread that preserves old Dominion design ambitions such as truth totality, sparse materialization, Truth/Perceived/Render separation, representation ladders, semantic ascent/descent, formalization chains, failure ontology, domain constitutions, ordinary life, and civilization-scale simulation.

2. The Story of the Conversation

2.1 From root cleanup frustration to controlled refactor The early visible part of the conversation was dominated by the user's frustration with the Dominion repository's directory structure. The user wanted the ugly old root folders gone and did not want to keep fixing builds only to move directories again. The assistant initially argued for cautious, AIDE-controlled moves: inventory, salvage maps, move plans, gates, apply waves, and proof after every move. Over time, as Codex reports came back, the process generated small moves, root inventories, and proof reports.

The user became increasingly impatient because the visible root mess persisted after many tasks. The assistant eventually shifted from tiny low-risk moves to a deterministic bad-root router model: known files should move to their canonical homes, and unknown files should be routed to `archive/quarantine/<root>/`, rather than blocking cleanup. This was a major change in tempo. It preserved the principle of not guessing unknown semantics while allowing the directory structure to become clean.

2.2 From folders to semantic authority After the root skeleton improved, the assistant and user recognized that the deeper problem was no longer top-level

directories. The problem became semantic duplication and governance: what is public, what is private, what is stable, what is provisional, what is generated, what is a fixture, what must stay compatible, what can be replaced, and what proof is required.

This led to the Foundation Lock queue: public surface registry, API/ABI canon, dependency direction, command surface, diagnostics/evidence, artifact identity, schema/protocol evolution, capability/refusal, provider model, module/workbench/app composition, replacement protocol, version/deprecation, mod/pack trust, portability matrix, and fast strict test tiers.

2.3 Language and architecture pivot Earlier project doctrine emphasized C89/C++98 to preserve broad historical portability. The conversation later revised this. Once the effective platform floor was treated as Windows 7 SP1, macOS 10.9.5, and Linux, the user and assistant concluded that C17 + C++17 was the better mainline baseline. Public ABI must remain C-compatible, but internal C++17 can be used for resource ownership, providers, runtime systems, apps, and Workbench. Later, the project also adopted a 64-bit source-native doctrine for full native products: x86_64 and arm64, little-endian, fixed-width persisted formats, and no pointer-sized truth.

2.4 Foundation Lock and dependency direction

The live repo check showed Foundation Lock initially blocked by dependency-direction strict validation. This was the first serious foundation validator still red: 358 violations and 38 warnings. The next prompt was generated to repair dependency direction. A later check showed Foundation Lock closed with warnings after dependency repair: 0 violations and 68 warnings. This unlocked narrow product slices such as `WORKBENCH-VALIDATION-SLICE-01`, while broad feature work remained blocked.

2.5 Workbench as product surface, not authority

The user and assistant explored Workbench deeply. The user proposed large modules such as Project Editor, UI Editor, Module Editor, and App Editor. The assistant refined these into workspaces: Project Graph Explorer, Interface Studio, Module/Pack Foundry, App Composer, Release Forge, Agent Control Workspace, Renderer/Theme Laboratory, and Replay/Trace Workspace. Internally, these should be built from smaller reusable modules: validation runner, command browser, document inspector, patch/diff viewer, diagnostics viewer, evidence viewer, pack browser, provider browser, capability matrix viewer, theme previewer, renderer test panel, asset browser, graph view, build/job console, and agent task queue.

The key conclusion was that Workbench is not the general module system; it is one consumer of the module/command/service/provider/pack/artifact system. Workbench must not call private tools directly. It must route through registered commands and typed results, diagnostics, refusals, views, and evidence.

2.6 Parallel Codex mode Once Foundation Lock closed with warnings and Workbench validation was authorized, the user wanted to generate multiple prompts in parallel rather than continue sequentially. The assistant designed a parallel worktree system. The first wave included service conformance law, document/patch/transaction law, project graph law, composition resolver law, and doctrine recovery matrix. Each prompt included branch/worktree rules, path allowlists, non-goals, deliverables, targeted validators, evidence, and commit instructions. A later pasted transcript said Wave 1 was effectively complete and the next task should be `COMMAND-RESULT-VIEW-SLICE-01`.

2.7 Final state before preservation By the end, the conversation had produced a complete continuity handoff and then this more detailed preservation request. The current working interpretation is that the next substantive Codex prompt should be `COMMAND-RESULT-VIEW-SLICE-01`. This task should prove that one

command result can become a semantic view with multiple projections, using `dominium.validation.run` if possible, without building a full Workbench shell, rendered GUI, native GUI, module loader, package runtime, provider runtime, gameplay, renderer implementation, or release publication.

3. Main Topics Discussed

Topic 1 - Dominium's final architecture

Dominium's final architecture was repeatedly refined. The strongest synthesis is that Dominium is not merely a game repo or a literal OS, but a deterministic, contract-governed simulation platform. Domino is the reusable substrate. Dominium is the game/product family. Workbench is the production and evidence environment. AIDE is the repo-control harness. Contracts are law. Tests, replay, diagnostics, and evidence are proof.

The topic came up because the project needed a model that could support world simulation, modding, tooling, Workbench, release, portability, and future games without repeating endless refactors. The conclusion was that stable contracts and semantic IDs must be separated from replaceable private implementation.

What remains uncertain is how quickly this architecture will move from contracts and fixtures into runtime implementations and product slices. The next chat

should not assume runtime provider resolvers, package runtime, or Workbench shell exist.

Topic 2 - Repository structure and root cleanup

The repo root cleanup dominated much of the conversation. The old root structure was visibly unacceptable and triggered intense user frustration. The conversation explored cautious moves, gates, root inventories, salvage maps, bulk routing, and deterministic quarantine. The final stance was that the root skeleton is now mostly settled and should not be re-litigated unless validators show a live violation.

The key outcome is the closed root set: `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `docs/`, `tests/`, `tools/`, `scripts/`, `cmake/`, `external/`, `release/`, and `archive/`, plus control roots like `.aide/` and `.github/`. `Generated/` local roots stay local. Old junk roots should not return.

Topic 3 - Foundation Lock

Foundation Lock was the governance gate that determined whether narrow product work could begin. It initially blocked on dependency-direction strict validation. After repair, it reportedly closed with warnings. This topic matters because it marks the transition from infrastructure cleanup to narrow product slices.

It remains important not to misread Foundation Lock.

It does not mean full CTest is green, broad Workbench exists, providers are implemented, or gameplay can start. It means the governance spine exists and narrow slices may proceed under its laws.

Topic 4 - C17/C++17 and 64-bit mainline The language policy shifted from historical C89/C++98 to C17/C++17. The mainline native architecture policy shifted toward 64-bit source-native: x86_64 and arm64. Public ABI remains C-compatible. Persisted formats must be fixed-width, explicit little-endian, and pointer-width independent.

This matters because it enables modern implementation patterns while preserving stable ABI and artifact compatibility. The risk is overusing C++17 library features that break macOS 10.9.5 or allowing `size_t`, `long`, raw pointers, or native layout to leak into save/replay/network/package formats.

Topic 5 - Workbench Workbench is a future product surface, not the center of authority. The user wants to get to Workbench and code soon, but Workbench must route through commands/services/views/evidence rather than private direct mutation. The first Workbench slice was narrow validation. The next needed bridge is command-result-view projection.

Workbench's final shape is a shell hosting modules and

workspaces. It eventually includes Project Graph Explorer, Interface Studio, Module/Pack Foundry, App Composer, Release Forge, Agent Work Board, Pack Browser, Renderer/Theme Laboratory, Replay/Trace Viewer, and related tools.

Topic 6 - Commands, results, views, projections

This became the immediate next frontier. The system must avoid separate CLI, TUI, Workbench, rendered, native, and headless implementations of the same behavior. The next slice should prove a command result can become a semantic view projected across multiple surfaces, preserving the same result schema, refusal codes, diagnostics, and evidence.

Topic 7 - Parallel Codex work The user wanted to run many autonomous tasks concurrently. The assistant designed branch/worktree isolation and strict parallel-worker rules. Workers do not push to main, do not update global latest AIDE files, and produce task-local evidence. Coordinator merges serially and runs fast strict.

This topic matters operationally because the user wants speed. The new chat should be ready to generate more parallel prompts or a coordinator review prompt depending on current repo state.

Topic 8 - Doctrine preservation An uploaded transcript/audit said much of the old Dominionum doctrine was preserved in repo canon, but some important areas remain partial: deep primitives, failure ontology, player-facing formalization workflows, ordinary life grounding, domain constitutions, playable-baseline hardening, and domain-by-domain verification. The doctrine recovery matrix was generated to prevent old requirements from being forgotten.

This matters for the eventual master Project Spec Book and future domain feature work.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted:

- to stop endless refactor churn,
- to make the repository tidy and future-proof,
- to preserve all old requirements and design decisions,
- to front-load autonomous Codex work in parallel,
- to get to Workbench and code as soon as possible,
- to avoid repeating explanations in new chats,
- to produce a comprehensive preservation package for this chat.

4.2 Inferred Goals

Reasonably inferred goals:

- minimize future architectural reversals;
- ensure assistants do not lose constraints;
- make Codex prompts operational rather than theoretical;
- preserve emotional and motivational context so future chats do not repeat slow, cautious patterns;
- build toward a master Project Spec Book.

4.3 Goals That Changed Over Time

The goal changed from:

`clean up the directory structure`

to:

`close Foundation Lock and build governed product slices`

to:

`parallelize autonomous contract/product-spine tasks`

to:

`preserve this chat for future aggregation.`

4.4 Goals Still Unresolved

Still unresolved:

- verify latest live repo state after user-pasted Wave 1 completion;

- generate/run `COMMAND-RESULT-VIEW-SLICE-01`;
- prove package mount, replay proof, barebones client shell;
- instantiate domain constitutions;
- stabilize playable baseline;
- build Workbench shell and tools;
- classify/fix full CTest/T4 debt;
- merge this chat with other old-chat reports into a master spec book.

5. Decisions Made and Why

Decision overview

Row 1 ID: DECISION-01; Decision: Dominion should be a deterministic contract-governed simulation platform, not merely a game repo.; Status: final; Why it mattered: Enables modularity, replacement, proof-driven development.; Confidence: High; Label: FACT

Row 2 ID: DECISION-02; Decision: Domino is the reusable substrate; Dominion is the product family; Workbench is production environment; AIDE is control-plane harness.; Status: final; Why it mattered: Prevents Workbench/Dominion from

becoming vague everything-platform.; Confidence: High; Label: FACT

Row 3 ID: DECISION-03; Decision: Contracts are law; tests/replay/evidence are proof.; Status: final; Why it mattered: Docs explain, generated output is evidence unless promoted.; Confidence: High; Label: FACT

Row 4 ID: DECISION-04; Decision: Path is not identity; implementation is not contract; UI is not authority; generated output is not source truth.; Status: final; Why it mattered: Stable IDs/contracts/registries/manifests own identity; paths are replaceable.; Confidence: High; Label: FACT

Row 5 ID: DECISION-05; Decision: Keep the closed canonical source-root set.; Status: final-with-caveats; Why it mattered: No new top-level modules/plugins/services/sdk roots unless future contract introduces them.; Confidence: High; Label: FACT

Row 6 ID: DECISION-06; Decision: Unknown files should not remain in bad roots; route to archive/quarantine rather than guess active ownership.; Status: final for cleanup phase; Why it mattered: Prevents unknown files blocking cleanup or being wrongly active.; Confidence: Medium; Label: FACT

Row 7 ID: DECISION-07; Decision: Mainline language baseline is C17 + C++17, not C89/C++98.; Status: final/live per checks; Why it mattered: C-compatible public ABI remains; C89/C++98 becomes research/historical.; Confidence: Medium; Label: FACT/VERIFY

Row 8 ID: DECISION-08; Decision: Full native products should be 64-bit source-native: x86_64 + arm64; 32-bit is constrained/projection/archive-runner.; Status: decided, policy encoding pending/in-flight; Why it mattered: Avoid old-toolchain/memory/matrix drag; keep pointer-width out of persisted law.; Confidence: Medium; Label: FACT/UNCERTAIN

Row 9 ID: DECISION-09; Decision: Little-endian mainline with explicit little-endian persisted/protocol encodings.; Status: decided; Why it mattered: Avoid expensive big-endian support; formats are explicit.; Confidence: Medium; Label: FACT

Row 10 ID: DECISION-10; Decision: Fast strict is normal dev gate; full CTest is T4/full-gate debt.; Status: final; Why it mattered: Prompts use targeted validators; full CTest only release/trust or specific need.; Confidence: High; Label: FACT

Row 11 ID: DECISION-11; Decision: Foundation Lock

gates narrow product work; broad feature work remains blocked.; Status: final for current phase; Why it mattered: Narrow slices allowed; not gameplay/renderer/native GUI/release.; Confidence: High; Label: FACT

Row 12 ID: DECISION-12; Decision: Workbench must use registered commands/services/evidence, not private validators/tools.; Status: final; Why it mattered: Prevents Workbench from becoming authority or bypass.; Confidence: High; Label: FACT

Row 13 ID: DECISION-13; Decision: Use parallel Codex worktrees with strict worker rules.; Status: active decision; Why it mattered: Speed up setup while limiting merge conflicts.; Confidence: High; Label: FACT

Row 14 ID: DECISION-14; Decision: Wave 1 prompt set: service conformance, document/patch/transaction, project graph, composition resolver, doctrine recovery.; Status: executed in chat; completion user-reported; Why it mattered: Foundation hardening runs in parallel.; Confidence: High; Label: FACT

Row 15 ID: DECISION-15; Decision: Next task after Wave 1 is COMMAND-RESULT-VIEW-SLICE-01.; Status: current plan; Why it mattered: Bridge

command result to semantic view/projection before package mount.; Confidence: Medium; Label: FACT/UNCERTAIN

Row 16 ID: DECISION-16; Decision: Do not start broad Workbench UI before command/result/view/projection law.; Status: final for sequencing; Why it mattered: Presentation spine first.; Confidence: High; Label: FACT

Row 17 ID: DECISION-17; Decision: Composition is the product.; Status: architectural decision; Why it mattered: Apps compose profiles/packs/modules/providers/capabilities into lockfiles/evidence.; Confidence: High; Label: FACT

Row 18 ID: DECISION-18; Decision: Doctrine recovery should map old doctrine to current canon rather than create a competing master doctrine.; Status: final for doctrine phase; Why it mattered: Preserve old requirements without overriding contracts.; Confidence: High; Label: FACT

Row 19 ID: DECISION-19; Decision: Major game features must enter as domain constitutions/bridges/capabilities/representation/failure/proof, not ad hoc features.; Status: future design law; Why it mattered: Prevents shallow feature scripts and semantic drift.; Confidence: Medium; Label: FACT/INFERENCE

Row 20 ID: DECISION-20; Decision: Do not put domino/dominium in every path; use project prefixes for public C symbols and fully qualified stable IDs.; Status: final naming law; Why it mattered: Short readable paths; collision-proof public symbols/IDs.; Confidence: High; Label: FACT

Decision explanation The decisions above should be treated as the working canon of this chat. The strongest decisions are the architectural identity split, the contract/proof law, the closed root model, the C17/C++17 + C-compatible ABI baseline, fast strict as the normal gate, and the sequencing rule that Workbench/product work must be narrow and command/service/evidence-backed.

The most tentative decisions are those depending on current live repo state after the latest pasted transcript: Wave 1 completion, current queue state, and exact readiness for **COMMAND-RESULT-VIEW-SLICE-01**. These should be verified before action.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

- Manual drag/drop directory cleanup was rejected as dangerous, even though the user threatened it out of frustration.

- Tiny sequential micro-move waves were superseded by deterministic routing and later by governed parallel task work.
- C89/C++98 as mainline was superseded by C17/C++17.
- 32-bit full-native mainline was rejected/deprioritized in favor of constrained/projection/archive lanes.
- Literal OS framing was softened into deterministic simulation operating environment / platform.
- Workbench as authority was rejected.
- Top-level `modules/`, `plugins/`, `services/`, or `sdk/` roots were rejected for now.
- Full Workbench UI before `command/view/` projection spine was rejected.
- Full CTest as routine gate was rejected.
- Broad feature work before `product/runtime` spine was rejected.

7. Important Reasoning, Rationale, and Tradeoffs

The main tradeoff throughout was speed versus safety. The user wanted speed because the project had spent months in refactor; the assistant tried to preserve

correctness by moving from cautious sequential gates to parallel but isolated worktrees.

Another tradeoff was governance versus product progress. Too little governance risks repeating the refactor disaster. Too much governance before product work risks stalling again. The compromise was Foundation Lock plus narrow product slices.

The C17/C++17 choice balanced modern implementation power against portability. The compromise was C++17 internally but C-compatible ABI and fixed-width persisted formats externally.

The 64-bit choice balanced modern platform practicality against legacy reach. The compromise was 64-bit full-native products while retaining 32-bit/vintage through constrained-native, contract-projection, or archive-runner lanes.

The Workbench plan balanced ambition against authority separation. Workbench should become powerful, but only as a command/evidence surface over contracts and services.

8. Plans, Future Work, and Next Steps

Immediate:

1. Verify live repo state.
2. If accurate, generate `COMMAND-RESULT-VIEW-`

SLICE-01.

3. After completion, run PHASE-REVIEW-02.

Then:

1. PACKAGE-MOUNT-SLICE-01
2. REPLAY-PROOF-SLICE-01
3. BAREBONES-CLIENT-SHELL-01
4. RUNTIME-SPINE-REVIEW-01

Then:

1. PRESENTATION-CONTRACT-01
2. PROJECTION-CONFORMANCE-01
3. ACCESSIBILITY-CONTRACT-01
4. TEXT-LOCALIZATION-CONTRACT-01
5. PRESENTATION-SPINE-REVIEW-01

Then:

1. WORKBENCH-SHELL-01
2. PROJECT-GRAPH-EXPLORER-01
3. PACK-BROWSER-01

4. AGENT-WORK-BOARD-01

Hardening:

1. COMPATIBILITY-CORPUS-01

2. PERFORMANCE-BUDGETS-01

3. ASSURANCE-PROFILE-00

4. RELEASE-PROMOTION-GATE-01

5. FULL-GATE-DEBT-01

6. POINTER-WIDTH-SERIALIZATION-AUDIT-01

7. GENERATED-SOURCE-LAW-01

Doctrine/domain:

1. SPEC-DEEP-PRIMITIVES-01

2. SPEC-FAILURE-ONTOLOGY-01

3. SPEC-PLAYER-FORMALIZATION-WORKFLOWS-01

4. SPEC-REPRESENTATION-PROOF-01

5. DOMAIN-CONSTITUTION-WAVE-01

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences The user wants direct, evidence-grounded, audit-ready answers; timestamps/model labels; explicit uncertainty; no repeated clarification unless necessary; large continuous Codex prompt blocks when generating prompts; targeted tests instead of full suites; and rapid progress toward Workbench/code.

9.2 Inferred Constraints and Preferences The user is likely to reject slow, report-only, overcautious process. Future assistants should produce executable next prompts quickly after verifying state.

9.3 Uncertain or Unestablished Preferences It is uncertain how many concurrent Codex workers the user will actually run at once and whether the user wants coordinator prompts or worker prompts next.

10. Files, Artifacts, Outputs, and Prompts

Important artifacts include:

- current uploaded preservation prompt file;
- prior handoff text;
- Foundation docs/audits;

- language baseline docs;
- portability docs;
- Wave 1 generated prompts;
- proposed `COMMAND-RESULT-VIEW-SLICE-01` task;
- doctrine recovery matrix prompt;
- root cleanup/router prompts.

See Artifact Ledger for details.

11. Open Questions and Unresolved Issues

Key unresolved issues:

- latest live repo state;
- whether Wave 1 is merged;
- whether `PORTABILITY-ARCH-POLICY-02` completed;
- exact implementation state of Workbench validation slice;
- exact command/result schema for `dominium.validation.run`;
- current warning disposition;
- full CTest/T4 debt;
- doctrine gaps such as deep primitives and failure ontology.

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

The highest-risk misunderstanding is to restart root cleanup or broad planning instead of moving to `COMMAND-RESULT-VIEW-SLICE-01`. Another risk is assuming Workbench may call private validators directly. Another risk is treating user-pasted state as live without verification. A future assistant may also accidentally run full CTest or edit global AIDE latest files in a parallel worker.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat contributes:

- final architecture split;
- root cleanup to foundation transition;
- language/architecture baseline;
- Foundation Lock/governed product-slice model;
- parallel Codex workflow;
- Wave 1 task prompts;
- next product-spine plan;
- doctrine recovery strategy;

- user preferences and frustration context.

It should feed spec-book chapters on:

- Project Architecture,
- Repo Governance,
- Build/Language/Portability,
- Workbench,
- Command/View/Diagnostics,
- Composition/Providers/Packs,
- Testing/Proof,
- Doctrine and Domain Constitutions,
- Development Workflow.

14. What I Should Remember

- The next task is `COMMAND-RESULT-VIEW-SLICE-01` unless live repo says otherwise.
- Workbench is not authority.
- Commands/services/results/views/diagnostics/evidence are the bridge from governance to product.

- Foundation Lock is closed with warnings, not "everything complete."
- Full CTest remains T4 debt.
- Broad feature work remains blocked.
- Parallel Codex workers must not edit global AIDE latest files.
- Old doctrine is mostly preserved but needs domain instantiation.
- The user wants speed, concrete prompts, targeted tests, and no repeated slow refactor cycles.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- "Explain why `COMMAND-RESULT-VIEW-SLICE-01` comes before `PACKAGE-MOUNT-SLICE-01`."
- "What is the difference between Workbench module, workspace, service, provider, pack, and app?"

15.2 Decisions

- "Which decisions in this chat are final versus tentative?"

- "What would make us revisit the C17/C++17 or 64-bit decisions?"

15.3 Tasks and Next Actions

- "Generate the `COMMAND-RESULT-VIEW-SLICE-01` prompt."
- "Generate `PHASE-REVIEW-02`."
- "What should the coordinator do after merging Wave 1?"

15.4 Artifacts and Files

- "List all generated Codex prompts from this chat."
- "Which files should a future assistant inspect first?"

15.5 Risks and Verification

- "What live repo facts must be verified before the next prompt?"
- "What could go wrong if Workbench bypasses registered commands?"

15.6 Future Spec Book / Aggregation

- "Convert this chat into spec-book chapter candidates."

- "Merge this preservation package with another old-chat report."

15.7 Deep-Dive Questions Specific to This Chat

- "What is the doctrine recovery matrix meant to preserve?"
- "How should deep primitives and failure ontology be turned into future specs?"
- "How do composition resolver, project graph, service conformance, and document/patch/transaction law fit together?"

16. Compact Human Summary

This chat was about the Dominion project's transition from painful repo restructuring into a coherent, governed product-building phase. The project had spent a long time cleaning up a messy repo full of bad roots and partial systems. The user was frustrated by how slowly the cleanup moved and wanted to stop endlessly planning so Workbench and real code could begin. The assistant and user gradually converged on a model where directory structure matters, but stable contracts, IDs, commands, services, providers, modules, packs, artifacts, and proof matter more.

The final architecture is that Domino is the reusable deterministic/runtime substrate; Dominion is the game/product family; Workbench is the production and validation environment; AIDE is the repo/control harness; and Codex is a bounded patch executor. The most important rules are: path is not identity, implementation is not contract, UI is not authority, and generated output is not source truth. This protects the project from future folder-name mistakes and allows implementation replacement behind stable surfaces.

The repo was treated as having moved beyond broad root cleanup. Foundation Lock became the main governance gate. It was first blocked by dependency-direction strict validation, then reportedly closed with warnings after repair. Fast strict passed, CMake configure/build and smoke CTest passed through fast strict, and full CTest remained T4/full-gate debt. Narrow product slices became allowed, but broad Workbench UI, gameplay, renderer expansion, native GUI, runtime module/package/provider systems, and release publication remained blocked.

The conversation then shifted to a parallel Codex workflow. The user wanted multiple concurrent Codex chats working on isolated worktrees. The assistant created strict worker rules: no direct pushes to main, no editing global AIDE latest files, task-local evidence only, allowed path lists, targeted validators only, and local

commits on task branches. Five Wave 1 prompts were generated: service conformance law, document/patch/transaction law, project graph law, composition resolver law, and doctrine recovery matrix. A later transcript said these were effectively complete, along with a narrow Workbench validation slice.

The next correct task is **COMMAND-RESULT-VIEW-SLICE-01**. It should prove that one registered command result can become a semantic view and be projected consistently across CLI/headless/text/TUI/Workbench-style surfaces. It should use `dominium.validation.run` if possible. It must preserve the same command, result schema, refusal codes, diagnostics, and evidence packet. It must not implement a full Workbench shell, rendered GUI, native GUI, runtime view engine, module loader, package runtime, provider runtime, gameplay, renderer implementation, or release.

The chat also preserved long-running Dominium doctrine: deterministic law, Truth/Perceived/Render separation, sparse materialization, representation ladders, semantic ascent/descent, formalization chains, cross-domain bridges, process-only mutation, pack-driven integration, explicit refusal/degradation, ordinary life, institutions, and civilization-scale simulation. The remaining doctrine gaps are deep primitives, failure ontology, player-facing formalization workflows, domain constitutions, and playable-baseline hardening.

The most important continuation is: verify current origin/main, confirm Wave 1 and Workbench validation state, then generate or run COMMAND-RESULT-VIEW-SLICE-01.

9.2 Dominium Foundation Workbench Parallel Codex Preservation Reader Brief

Source: docs/archive/conversations/Foundation_Workbench_Codex/Dominium_Foundation_Workbench_Parallel_Codex_Preservation__06_reader_brief.md

Report type: reader_brief

Size: 1489 bytes

9.2.1 Reader Brief - Dominium Foundation Lock, Workbench Spine, and Parallel Codex Handoff

What this chat was about

This chat was about getting Dominium out of endless repo refactor and into governed product work. It preserved the architecture, task queue, constraints, and next steps.

Top 20 things to know

1. Dominium is now framed as a deterministic contract-governed simulation platform.
2. Domino is reusable substrate.
3. Workbench is production/evidence environment, not authority.
4. AIDE is repo/control-plane harness.
5. Contracts are law; tests/replay/evidence are proof.
6. Path is not identity.
7. Implementation is not contract.
8. UI is not authority.
9. Generated output is not source truth.
10. Canonical roots are settled; do not restart root cleanup.
11. Mainline is C17 + C++17.
12. Full native products are 64-bit x86_64/arm64.
13. Public ABI stays C-compatible.
14. Persisted data is fixed-width/little-endian/pointer-width independent.
15. Fast strict is normal gate.

16. Full CTest is T4/full-gate debt.
17. Foundation Lock reportedly closed with warnings.
18. Broad feature work remains blocked.
19. Parallel Codex worktrees are allowed with strict rules.
20. Next task is likely COMMAND-RESULT-VIEW-SLICE-01.

Best next step

Verify live repo, then generate COMMAND-RESULT-VIEW-SLICE-01.

Important warnings

Do not start broad Workbench UI, renderer, native GUI, package runtime, provider runtime, module loader, gameplay, or public release yet.

9.3 Dominion Foundation Workbench Parallel Codex Preservation In Chat Reader

Source: docs/archive/conversations/Foundation_
Workbench_Codex/Dominium_Foundation_Workbench_

Parallel_Codex_Preservation__09_in_chat_reader.
md

Report type: in_chat_reader

Size: 997 bytes

9.3.1 In-Chat Reader - Dominion Foundation Lock, Workbench Spine, and Parallel Codex Handoff

Package overview

This preservation package contains a human-readable report, structured registers, a context transfer packet, a YAML-style spec sheet, an aggregator packet, a reader brief, audit notes, and a future chat bootstrap prompt.

File index

Read:

1. Human-readable report for full understanding.
2. Reader brief for short review.
3. Context transfer packet for new chat.
4. Registers for tasks/decisions/artifacts.
5. Spec sheet for aggregation.
6. Audit file for uncertainty and verification queue.

Question menu

Ask:

- "Generate COMMAND-RESULT-VIEW-SLICE-01."
- "Verify live origin/main and compare with this handoff."
- "List all Wave 1 tasks and their dependencies."
- "Explain why Workbench is not authority."
- "Turn this into a Master Project Spec Book chapter outline."
- "Create a merge plan for parallel branches."
- "Generate PHASE-REVIEW-02."

9.4 Dominium Foundation Workbench Parallel Codex Preservation Accompanying Detailed Summary Report

Source: docs/archive/conversations/Foundation_
Workbench_Codex/Dominium_Foundation_Workbench_
Parallel_Codex_Preservation__10_accompanying_
detailed_summary_report.md

Report type: detailed_summary_report

Size: 15237 bytes

9.4.1 Accompanying Human-Readable Detailed Summary and Continuity Report

Chat label: Dominion Foundation Lock, Workbench Spine, and Parallel Codex Handoff

Date anchor: 2026-05-27 Australia/Melbourne

Source scope: This chat only, except where marked as project context or live-repo status previously checked during the conversation.

Reliability note: This report is a human-readable companion to the preservation package. It summarizes the visible conversation and the previously generated package. Some earlier uploads or skipped transcript portions were unavailable at generation time, so live repo details should be rechecked before acting.

1. What this conversation was really about

This conversation captured a major transition in the Dominion project. Earlier project work had been dominated by repository cleanup, root-folder consolidation, AIDE installation, Codex task generation, build proof, internal pilot packaging, and repeated

attempts to make the repository structurally safe. The user was frustrated because this phase had taken months and still seemed to block work on the actual game, Workbench, and product code.

The conversation gradually shifted away from "how do we move every bad folder?" toward a more durable model: **make the project contract-governed so folders and implementations can be replaced without breaking identity, saved artifacts, packs, modules, Workbench surfaces, or downstream games.**

The final architecture that emerged was:

Domino = reusable deterministic/runtime substrate

Dominium = game/product family built on Domino

Workbench = production, validation, editing, inspection, packaging, evidence, and agent-control environment

AIDE = repo/control-plane harness

Codex = bounded patch executor

Contracts = law

Tests, replay, diagnostics, and evidence = proof

The most important rules were:

Path is not identity.

Implementation is not contract.

UI is not authority.

Generated output is not source truth.

This matters because Dominium's future depends on stable IDs, contracts, schemas, commands, capabilities, providers, modules, packs, artifacts, diagnostics, and

proof. The private implementation can move or be replaced, but public identity and compatibility cannot drift silently.

2. The emotional and practical context

The user was not calmly asking for abstract architecture. The user was trying to stop a long-running refactor from consuming the project indefinitely.

A recurring theme was urgency. The user repeatedly expressed that the directory and root-structure cleanup had taken too long, and at points threatened to manually drag/drop folders if the assistant and Codex process failed to produce real progress. That context matters for future assistants: the user does not want more gentle open-ended planning when the project already has a decided path.

The correct future posture is:

- verify live repo state;
- identify the current next task;
- generate concrete Codex prompts with strict scope;
- keep prompts implementation/documentation/test focused;
- avoid full CTest unless necessary;

- avoid restarting solved debates;
 - avoid asking for clarification unless genuinely blocking.
-

3. The repository cleanup arc

The early and middle parts of this chat dealt with the repo's ugly historical roots. The project had accumulated top-level roots such as:

```
core/  
control/  
data/  
packs/  
profiles/  
bundles/  
compat/  
lib/  
libs/  
locks/  
repo/  
safety/  
security/  
specs/  
updates/  
meta/  
governance/  
ide/  
performance/  
validation/
```

```
modding/  
models/  
templates/  
net/
```

The initial cleanup model was very cautious:

```
inventory  
-> classify  
-> salvage map  
-> move map  
-> gate  
-> apply  
-> proof
```

This proved safe but slow. The user became impatient because the visible folder mess remained. The process then evolved into deterministic routing:

- semantically known files move to canonical owners;
- unknown or ambiguous files go to `archive/quarantine/<root>/`;
- bad roots should not remain as active source;
- active exceptions require owners and retirement plans.

Later live-state checks suggested that the visible root problem was largely resolved and that the project had moved to the next problem: **semantic governance inside the approved roots.**

The canonical source-root model became:

```
apps/  
engine/  
game/  
runtime/  
contracts/  
content/  
docs/  
tests/  
tools/  
scripts/  
cmake/  
external/  
release/  
archive/
```

Generated/local only:

```
.aide.local/  
.dominium.local/  
build/  
out/  
dist/  
tmp/  
__pycache__/
```

Future assistants should not restart root cleanup unless a live validator shows an active old-root violation.

4. The governance phase and Foundation Lock

Once the folder skeleton became acceptable, the focus shifted to the governance platform. The user and

assistant converged on the need for a **Foundation Lock**: a proof gate showing that core repo law exists and validates before narrow product work resumes.

The Foundation layers included:

1. fast strict test tier;
2. public surface registry;
3. API/ABI canon;
4. dependency direction law;
5. command/refusal/result surface law;
6. diagnostic and evidence registry;
7. artifact identity law;
8. schema/protocol evolution law;
9. capability/refusal law;
10. provider model;
11. module/workbench/app composition law;
12. replacement protocol;
13. version/deprecation law;
14. mod/pack trust model;
15. portability matrix.

A live repo check during the conversation found Foundation Lock initially blocked by dependency-direction strict validation:

```
358 violations
```

```
38 warnings
```

A later live check found that FOUNDATION-CLOSEOUT-02 had closed Foundation Lock with warnings:

```
dependency-direction strict: PASS, 0 violations, 68 warnings
```

```
fast strict: PASS
```

```
RepoX STRICT: PASS with stale AuditX warning
```

```
CMake configure/build: PASS through fast strict
```

```
smoke CTest: PASS through fast strict
```

```
full CTest: T4/full-gate debt
```

This was a major threshold. It meant narrow product slices could begin, but broad feature work remained blocked.

5. The technical baseline

The project moved away from C89/C++98 as the mainline rule.

The final baseline discussed was:

```
Language: C17 + C++17
```

```
Architecture: 64-bit source-native: x86_64 + arm64
```

```
Endian: little-endian mainline only
```

```
ABI: C-compatible, POD-only, versioned, no C++ ABI leakage
```

```
Data formats: fixed-width, explicit little-endian, no padding,  
no pointer-sized fields
```

Simulation: deterministic, fixed-point where authoritative, stable IDs, canonical ordering

Runtime: provider/capability/refusal model

Renderer: null + software baseline; GPU backends are replaceable providers

UI: CLI/headless always; TUI/rendered/native shells are capability layers

Legacy targets: constrained_native or contract_projection, not mainline constraints

C17 owns the stable deterministic and ABI-adjacent surfaces:

- fixed-width math and IDs;
- save/replay/wire records;
- renderer command IR;
- provider facades;
- canonical hashing.

C++17 owns higher-level implementation:

- game orchestration;
- domain systems;
- runtime services;
- providers;
- apps;

- Workbench;
- resource ownership;
- job systems;
- compiled tooling where useful.

The public ABI remains C-compatible. Stable formats must not use `size_t`, `long`, pointer values, native padding, object layout, address ordering, or pointer-derived identity.

6. Workbench's role

Workbench became a major focus, but the critical decision was that **Workbench is not authority**.

Workbench is the richest human/agent interface over the system, but it must use the same command/result/refusal/diagnostic/evidence spine as CLI, TUI, headless, server/admin, tests, and future UI projections.

The conversation distinguished:

```

component = source/build ownership unit
service = callable runtime capability
provider = replaceable implementation
pack = distributable authored payload
module = declared functional extension unit
workspace = large Workbench composition
app = shipped product composition

```

```
artifact = persisted versioned result
```

Workbench has:

```
apps/workbench/shell/  
apps/workbench/module/  
apps/workbench/workspace/
```

Workbench modules include:

```
validation/  
pack_browser/  
agent_board/  
replay_trace/  
renderer_lab/  
package_planner/  
evidence_viewer/
```

Workbench workspaces include:

```
project_graph/  
interface_studio/  
module_foundry/  
app_composer/  
release_forge/
```

The first Workbench slice was WORKBENCH-VALIDATION-SLICE-01, which was reported as complete but narrow. It did not implement a broad Workbench shell, rendered GUI, native UI, runtime module loader, provider runtime, package runtime, gameplay, or release publication.

7. The transition to parallel Codex work

After Foundation Lock closed with warnings, the user wanted to stop queueing prompts one at a time. The assistant proposed a parallel worktree system.

Parallel workers must:

- use dedicated branches/worktrees;
- not push to main;
- not edit global AIDE latest files;
- write task-local evidence only;
- stay within allowed path lists;
- run targeted validators;
- avoid full CTest;
- commit locally;
- leave final merge to a coordinator.

Files parallel workers must not edit:

```
.aide/context/latest-task-packet.md
.aide/context/latest-review-packet.md
.aide/reports/latest-dominium-status.md
.aide/reports/latest-warning-disposition.md
.aide/queue/current.toml
.aide/ledgers/migration_ledger.jsonl
docs/repo/POST_CONVERGE_NEXT_STEPS.md
```

This enabled Wave 1 prompts.

8. Wave 1 parallel prompts

The assistant generated five major parallel Codex prompts:

1. SERVICE-CONFORMANCE-LAW-01
2. DOCUMENT-PATCH-TRANSACTION-RUNTIME-01
3. PROJECT-GRAPH-SERVICE-01
4. COMPOSITION-RESOLVER-LAW-01
5. DOMINIUM-DOCTRINE-RECOVERY-MATRIX-01

A later pasted transcript reported these as complete, with PASS or PASS_WITH_WARNINGS.

8.1 Service conformance Purpose: a provider/service/module is not available merely because it compiles or has a descriptor. It must pass a conformance kit.

8.2 Document/patch/transaction Purpose: Workbench, AIDE, CLI, and tools must mutate structured state through:

document

-> patch

-> dry run

-> validation

-> transaction apply

-> evidence packet

-> rollback handle

not ad hoc edits.

8.3 Project graph Purpose: typed relationship graph across files, components, public surfaces, contracts, commands, services, providers, modules, packs, apps, tests, artifacts, evidence, and releases.

Project Graph is derived evidence, not authority.

8.4 Composition resolver Purpose: composition is the product.

The chain:

app descriptor

+ profile

+ packs

+ modules

+ providers

+ platform

+ renderer

+ trust policy

+ capabilities

-> composition resolver

- > selected providers
- > mounted packs
- > enabled modules
- > capability set
- > refusals/degradations
- > lockfile
- > evidence packet

8.5 Doctrine recovery matrix Purpose: preserve long-running Dominion doctrine by mapping old ideas to current canon, without creating a competing master doctrine.

9. Doctrine preserved and still missing

The uploaded transcript/audit said most major old Dominion ideas were preserved in the repo:

- truth totality;
- sparse materialization;
- Truth / Perceived / Render separation;
- representation ladders;
- semantic ascent/descent;
- formalization chains;
- process-only mutation;

- cross-domain bridge law;
- ordinary life;
- institutions and civilization;
- pack-driven integration;
- explicit refusal/degradation.

Partial or missing areas include:

- deep primitives master doctrine;
- failure ontology master taxonomy;
- player-facing formalization workflows;
- ordinary-life grounding;
- domain constitutions;
- playable-baseline hardening;
- domain-by-domain verification.

The doctrine recovery matrix was generated to preserve those threads for later spec work.

10. Current next task

The latest pasted planning state said the next correct task is:

COMMAND-RESULT-VIEW-SLICE-01

Why this comes next:

WORKBENCH-VALIDATION-SLICE-01 proved a narrow command/result/diagnostic/evidence path, but the next missing bridge is:

```
command result
-> semantic view
-> projection
-> CLI / headless / Workbench
```

This task should use `dominium.validation.run` if possible and prove that one command result can produce a semantic view projected consistently across multiple surfaces.

It must not implement:

```
full Workbench shell
rendered GUI
native GUI
runtime view engine
package runtime
provider runtime
module loader
gameplay
renderer implementation
release publication
```

After it, the next checkpoint should be:

PHASE-REVIEW-02

Then the runtime/product spine:

PACKAGE-MOUNT-SLICE-01

REPLAY-PROOF-SLICE-01

BAREBONES-CLIENT-SHELL-01

RUNTIME-SPINE-REVIEW-01

11. What remains to do

Immediate

- Verify current origin/main.
- Confirm whether COMMAND-RESULT-VIEW-SLICE-01 has already run.
- Generate or execute COMMAND-RESULT-VIEW-SLICE-01.

Next checkpoint

- Run PHASE-REVIEW-02 after command-result-view.

Product/runtime spine

- PACKAGE-MOUNT-SLICE-01
- REPLAY-PROOF-SLICE-01

- BAREBONES-CLIENT-SHELL-01
- RUNTIME-SPINE-REVIEW-01

Presentation and UI law

- PRESENTATION-CONTRACT-01
- PROJECTION-CONFORMANCE-01
- ACCESSIBILITY-CONTRACT-01
- TEXT-LOCALIZATION-CONTRACT-01

Workbench expansion

- WORKBENCH-SHELL-01
- PROJECT-GRAPH-EXPLORER-01
- PACK-BROWSER-01
- AGENT-WORK-BOARD-01

Hardening

- COMPATIBILITY-CORPUS-01
- PERFORMANCE-BUDGETS-01
- ASSURANCE-PROFILE-00
- RELEASE-PROMOTION-GATE-01

- FULL-GATE-DEBT-01
- POINTER-WIDTH-SERIALIZATION-AUDIT-01
- GENERATED-SOURCE-LAW-01

Doctrine/domain

- SPEC-DEEP-PRIMITIVES-01
 - SPEC-FAILURE-ONTOLOGY-01
 - SPEC-PLAYER-FORMALIZATION-WORKFLOWS-01
 - SPEC-REPRESENTATION-PROOF-01
 - DOMAIN-CONSTITUTION-WAVE-01
-

12. What a future assistant should do first

A future assistant should:

1. Verify live repo state.
2. Inspect the current AIDE queue/status/audit docs.
3. Confirm whether COMMAND-RESULT-VIEW-SLICE-01 is still the next task.
4. If yes, generate the Codex prompt for that task as one large continuous text block.

5. Keep it narrow and targeted.
 6. Do not launch broad Workbench UI or gameplay.
 7. Avoid full CTest unless required.
 8. Preserve the parallel-worker rules if using concurrent Codex chats.
-

13. Final package check summary

This companion report should be used alongside the original preservation package files. The original package already contains:

- manifest;
- full human-readable preservation report;
- context transfer packet;
- YAML spec sheet;
- registers;
- aggregator packet;
- reader brief;
- verification/audit;
- future chat bootstrap prompt;

- in-chat reader guide.

This new report gives a more narrative explanation of the entire conversation and is intended for human reading before diving into the registers.

Chapter 10

Framework Open Source
Provider

10.1 Domino Framework Open
Source Provider Architecture
Human Readable Report

Source: docs/archive/conversations/Framework_
Open_Source_Provider/Domino_Framework_Open_
Source_Provider_Architecture__01_human_
readable_report.md

Report type: human_readable_report

Size: 48188 bytes

10.1.1 COMPLETE CHAT PRESERVATION
REPORT - Domino Framework and
Open-Source Provider Architecture

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Domino
Framework and Open-Source Provider Architecture

Row 2 Field: Date anchor; Assessment: 2026-05-27

Australia/Melbourne

- Row 3** Field: Source scope; Assessment: This chat only unless labelled PROJECT-CONTEXT
- Row 4** Field: Apparent access; Assessment: Partial but substantial: visible chat context plus uploaded preservation prompt; not guaranteed full historical transcript or inaccessible past-chat links
- Row 5** Field: Previously generated files available?; Assessment: No previously generated downloadable package was visible before this task
- Row 6** Field: Uploaded files or artifacts present?; Assessment: Yes: Pasted `text.txt`, the preservation/export instruction prompt `?filecite?turn29file0?`
- Row 7** Field: Contains future plans?; Assessment: Yes
- Row 8** Field: Contains decisions?; Assessment: Yes, mostly architecture decisions and accepted directions
- Row 9** Field: Contains pending tasks?; Assessment: Yes
- Row 10** Field: Contains unresolved questions?; Assessment: Yes
- Row 11** Field: Staleness risk; Assessment: Medium to High for external project facts, licenses, repo state, library versions, and platform support

Row 12 Field: Extraction confidence; Assessment: 4 / 5 for visible-chat substance; 3 / 5 for full-history completeness

Row 13 Field: Safe for later aggregation?; Assessment: With caveats

Row 14 Field: Main limitations; Assessment: I can preserve the visible conversation and uploaded prompt. I cannot guarantee access to every old chat behind the pasted "Past chat" links, hidden transcript outside current context, or future/current repo state.

Plain-language limitation: this package is a high-fidelity preservation of the conversation that is visible in this chat context. It should not be treated as a complete extraction of every related past chat. The user pasted references to other ChatGPT conversations, but those links were not browsed or available as full transcripts here. All facts about external libraries, licenses, project status, and the `julesc013/dominium` repo should be verified before implementation.

1. One-Page Orientation

This chat was about how to accelerate the Dominium/Domino game-engine project by using existing open-source code without letting outside engines or libraries

define the architecture. The user's recurring concern was speed: instead of starting from nothing, could Dominium bootstrap a working engine, game client, Workbench, scripting layer, and provider ecosystem using open-source systems such as raylib, SDL2, Lua, Celestia, SpaceEngine-like references, voxel/RTS/factory games, and other modular libraries? The conversation gradually converged on a clear doctrine: use outside code aggressively, but only as replaceable providers behind first-party Domino/Dominium contracts.

The first major theme was the difference between forking a full engine and assembling a framework from modular components. The chat rejected the idea that Dominium should become a modified copy of a large engine.

Instead, it preferred a framework approach: Domino should define stable service contracts, provider ABIs, capability/refusal law, profiles, tests, and deterministic simulation interfaces; a Domino engine implementation should satisfy those contracts; the Dominium game implementation should consume those contracts. This means the game should not directly include `raylib.h`, `SDL.h`, `lua.h`, or any other third-party API in its canonical layers.

The second major theme was the raylib ecosystem. The user liked raylib and asked whether its subprojects could be used: raylib itself, `rlgl`, `rlsw`, `raymath`, `raygui`, `raudio`, texture/model/font systems, and examples. The

conclusion was yes, but with clear classification. `raylib` is a broad high-level provider suite. `rlgl` is a raylib-family OpenGL abstraction provider, not the final Dominium OpenGL 3.3 backend. `rlsw` is a raylib software-render provider, not the canonical Dominium reference software renderer. `raygui` is an early Workbench/debug UI provider, not UI law. `raudio` is a first audio provider. `raymath` is safe for presentation/editor math, not deterministic simulation law. Asset-loading helpers are import/preview helpers, not Dominium asset identity.

The third major theme was service-first architecture. Earlier suggestions used paths like `runtime/render/raylib`, but the conversation refined that to `runtime/<service>/providers/<provider>`. The intended services include platform, input, render, draw, audio, asset, script, UI, world, simulation, diagnostics, storage, authority, and work delegation. Provider profiles, stored in `release/profiles/` or `content/profiles/`, should select combinations such as `client.raylib`, `client.sd12_raylib`, `workbench.raylib`, and `server.null`. Apps remain generic: `apps/client`, `apps/workbench`, `apps/server`.

The fourth major theme was reference projects. The user asked about SpaceEngine, Celestia, PCGUniverse2, Valian/pgg, and other engines/games. The answer separated "use as dependency" from "study as reference."

SpaceEngine is a design reference only. Celestia is useful but GPL-constrained. PCGUniverse2 and pgg were useful procedural-generation references but license-unclear. Larger projects such as Luant/Minetest, OpenRA, Spring RTS, Mindustry, Freeciv, Endless Sky, Pioneer, Godot, O3DE, Torque3D, and others were classified by subsystem relevance, mostly as research material rather than bases to fork.

The fifth major theme was the intended game architecture: deterministic, sparse, distributed, and able to support arbitrary CAD-like building. The conversation developed the phrase "sparse deterministic delegated simulation." The world should be infinite by addressability, not by simulating everything. Active cells are finite and scheduled; cold regions are seeds plus deltas. Clients may contribute processing power through work leases, but the host/server verifies results before committing state. Player/agent/NPC creations should be arbitrary at design time but compiled into bounded runtime representations: construct graphs, machine graphs, physics proxies, render proxies, and event-sourced state.

The future relevance of this chat is high. It should feed the master Project Spec Book sections on Domino Framework architecture, provider system, first-wave dependencies, Workbench strategy, scripting/modding, deterministic simulation, sparse-world architecture,

client-contributed compute, CAD/machine systems, and license/provenance policy. The most important thing a future assistant must understand is that *Dominium* is not meant to become a raylib game, a Godot fork, or a Celestia derivative. It is meant to become a first-party framework/game system that can use those tools as replaceable providers or references.

2. The Story of the Conversation

2.1 From "Can we use open source?" to a provider

doctrine The chat began with the user asking whether, because the project is portable, modular, and extensible, it could use existing open-source systems instead of starting from nothing. The first answer accepted that direction but drew a boundary: third-party code could accelerate execution, but it must not define *Dominium* law. That produced the early doctrine: fork for speed, wrap for control, test for proof, replace by boundary.

The user then stated a platform and language target: Windows 7 SP1+, macOS 10.9.5+, Linux, C17/C++17, Win32, Cocoa, SDL2, and OpenGL 3.3. The user also strongly endorsed Lua, SDL, and raylib. That shifted the conversation from abstract open-source reuse to specific first-wave providers.

2.2 The raylib phase The chat spent significant time unpacking raylib. Initial suggestions put raylib under directories like `runtime/render/raylib` and `apps/client/rendered/raylib`. The user then pasted and compared refinements that pushed the architecture toward a cleaner service/provider model. The key correction was that source ownership should be service-first, not vendor-first. A renderer should not become "raylib-shaped"; instead, raylib should implement the render service as one provider.

The discussion then moved into raylib subcomponents. The user asked specifically whether `rlsw` could be used for software rendering and `rlgl` for OpenGL. The conclusion was nuanced. Yes, `rlsw` can be a raylib software-render provider, especially for headless screenshot evidence and CI, but Dominium should still own a canonical software reference renderer. Yes, `rlgl` can be an OpenGL-family provider, but a future direct `opengl33` provider should remain separate. `raymath`, `raygui`, `raudio`, `rtextures`, and `rmodels` are usable, but only in presentation/tool/import/provider layers.

2.3 Repo comparison phase The user asked how raylib compared to current implementations at `julesc013/dominium`. The assistant inspected repository files through GitHub connector results. The chat found, at that time, that Dominium already had

system and render abstractions such as `dsys` and `d_gfx`, plus stubs and software-backed paths. This supported the claim that the repo already had seams where raylib could fit as a concrete backend/provider. However, one important contradiction appeared later: a pasted transcript claimed the repo had already moved to C17/C++17, while a later assistant check said the visible `main` CMake still appeared to set C90/C++98. This must be verified before implementation.

2.4 Reference projects and open-source ecosystem phase

The user then asked about SpaceEngine, Celestia, PCGUniverse2, and Valian/pgg. The chat classified them mostly as reference projects rather than code dependencies. SpaceEngine was treated as proprietary/commercial design inspiration. Celestia was useful but GPL-constrained. PCGUniverse2 and pgg were procedural-generation references with unclear license status. Later, when the user asked about other open-source engines/games, the assistant listed projects useful by subsystem: Lianti/Minetest for sparse voxel/mod ecosystems, OpenRA/Spring/0 A.D./Freeciv/Widelands for command/ruleset/RTS/logistics patterns, Mindustry for factory graphs, Endless Sky/Pioneer/Celestia for space models, Godot/O3DE/Torque3D for editor/engine references, and raylib/SDL/Lua/SQLite/ImGui/Nuklear and similar libraries as better first-wave

dependencies.

2.5 Sparse deterministic delegated simulation

phase The user then asked how to make the envisioned game deterministic and sparse, with clients dynamically taking load from the host and contributing processing power, while supporting arbitrary infinite building, CAD creation, machines, inventions, agents, NPCs, and other features. This produced a central conceptual architecture: infinite address space, finite active set, sparse materialization, deterministic local simulation islands, event-sourced state, client-contributed work, and host-verified authority.

The visible rationale was that clients can compute but cannot be blindly trusted. Work should be delegated through leases: a host assigns a deterministic packet, a client returns results with hashes/traces, and the host verifies by replay, quorum, spot-checking, or invariants. Arbitrary CAD designs should be free at design time but compiled to bounded runtime graphs and proxies. Players, agents, and NPCs should all mutate the world through the same command/event/build transaction system.

2.6 Domino Framework phase The conversation culminated when the user said they liked the framework approach and asked whether raylib and other sources/

projects could make a Domino framework that can be provided by any Domino engine implementation and any Dominion game implementation. The final architectural synthesis was: Domino Framework defines contracts and provider ABI; a Domino engine implementation provides those services; Dominion is a game implementation consuming the framework; raylib, SDL2, Lua, ImGui, raygui, raudio, rlgl, and rlsw are providers, not law. This is the main conceptual deliverable of the chat.

2.7 Preservation phase Finally, the user uploaded a detailed preservation prompt and requested a maximum-fidelity report, registers, spec sheet, aggregator packet, audit, and downloadable files. This package is the response to that request.

3. Main Topics Discussed

Topic 1 - Open-source bootstrap strategy The first topic was whether Dominion should use existing open-source systems to accelerate development. The chat concluded that this is desirable, but only if third-party code is treated as replaceable scaffolding or providers. The preferred approach is not to fork a full engine and gradually mutate it, because that risks inheriting the engine's object model, editor workflow, build system, content format, and architectural assumptions. The framework approach gives more

control and better alignment with Dominion's portability/modularity/extensibility goals.

Future work should convert this into a dependency policy: use permissive/weak-copyleft libraries as providers where appropriate; use GPL/proprietary/unclear projects as research references unless explicitly quarantined; require provider manifests, license manifests, and conformance tests.

Topic 2 - raylib ecosystem as first visible provider suite

raylib came up because the user liked it and because it provides a fast way to get windows, input, rendering, audio, assets, and tooling visible. The conversation concluded that raylib should be used heavily, but only under `runtime/<service>/providers/<provider>` and proof/experiment areas. `raylib.h` and related headers should be forbidden from framework public headers, contracts, game law, content, saves, replays, and public SDK.

`rlsw` can help with headless and CPU rendering, but not as canonical deterministic reference renderer. `rlgl` can help as OpenGL-family provider, but not as final direct `opengl33`. `raygui` can bootstrap Workbench/debug panels, but Workbench documents remain first-party. `raudio` can be audio provider. `raymath` is safe for render/editor math only. Asset import helpers can load/preview images/models/fonts/audio, but Dominion

asset identity must remain first-party.

Topic 3 - SDL2 and Lua SDL2 was treated as complementary to raylib, not a replacement or afterthought. SDL2 should be a platform/input/audio provider, and profiles can combine SDL2 platform/input with raylib/rlgl rendering. Lua was accepted as a good scripting provider for packs, mods, CLI, Workbench automation, scenario hooks, and debug tooling. The important caveat is that Lua must be pinned and hidden behind `dominium.script.v1` or equivalent. Mods should target Dominium's script API, not the raw Lua ABI.

Topic 4 - Service-first/provider-first architecture

This became the central architectural theme. The conversation refined earlier vendor-shaped paths into service-first paths. The stable pattern is:

`runtime/<service>/providers/<provider>`

`contracts/provider`

`contracts/capability`

`contracts/schema/runtime/<service>`

`release/profiles/*.toml`

`apps/client`, `apps/workbench`, `apps/server` remain generic

The reason is that service identity is first-party and provider implementation is replaceable. This allows multiple engine implementations and multiple provider profiles to run the same game implementation.

Topic 5 - Domino Framework / Domino Engine / Dominium Game split The final synthesis separated framework, engine, and game. Domino Framework is the stable API/contract/provider system. A Domino engine implementation satisfies it. Dominium is a game implementation using it. This is likely the most important conceptual result of the chat. It means the project can produce one reference engine and later allow alternate engines to run the same game or allow other games to run on the same framework.

Topic 6 - Current Dominium repository fit The chat inspected `julesc013/dominium` and found evidence of existing abstractions such as system and graphics layers, stubs, and soft-backed backends. The finding was that raylib could fill concrete provider gaps without replacing the architecture. However, current repo facts are stale/uncertain and must be verified, especially the C17/C++17 vs C90/C++98 baseline contradiction.

Topic 7 - Reference projects The user asked about SpaceEngine, Celestia, PCGUniverse2, pgg, and later other open-source engines/games. The conclusion was that these are valuable mostly as research/reference material. They can inform worldgen, catalogs, addons, space navigation, terrain LOD, voxel worlds, RTS command systems, factory graphs, mod ecosystems, and

editor designs. They should not be blindly vendored or forked because of license, stack, or architecture mismatch.

Topic 8 - Sparse deterministic delegated

simulation The user's large game vision requires an architecture where the world is infinite by addressability but finite by active simulation. The chat proposed activity states such as cold, warm, scheduled, active, and hot. It also proposed cells/islands/constructs as units of scheduling and authority. Clients can contribute compute, but state changes must be host-verified. This topic should become a formal spec chapter.

Topic 9 - CAD, machines, arbitrary inventions

The chat addressed the user's desire for arbitrary creation systems, machines, and inventions. The core conclusion was that raw arbitrary CAD geometry cannot be simulated forever everywhere. Designs must compile into bounded runtime forms: construct graphs, machine graphs, port graphs, physics proxies, damage graphs, and render proxies. Players, agents, and NPCs should all submit build intents through the same transaction system.

Topic 10 - Preservation and aggregation The final topic is this package. The user requested a preservation

report that is human-readable first, with registers, spec sheet, aggregator packet, self-audit, and downloadable files. This report preserves the visible chat, not any inaccessible past-chat transcripts.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted to speed up Dominion development by using existing open-source systems. The conversation addressed this by comparing framework vs fork approaches and converging on modular providers.

The user explicitly wanted Lua support for modules/packs/mods/CLI and maybe all scriptable layers. The chat addressed this by recommending a pinned Lua provider behind a Dominion script API.

The user explicitly wanted SDL2 and raylib support. The chat addressed this by assigning SDL2 to platform/input/audio and raylib to visible rendering/tooling/provider suite.

The user explicitly wanted to know whether raylib subprojects like `rlsw`, `rlgl`, `raymath`, `raygui`, and `raudio` could be used. The chat answered yes with boundaries.

The user explicitly wanted a deterministic sparse game architecture where clients contribute compute and players/agents/NPCs can build arbitrary/infinite things.

The chat addressed this with sparse deterministic delegated simulation, work leases, host verification, and CAD-to-runtime graph compilation.

The user explicitly wanted a preservation/export package for this chat. This output addresses that.

4.2 Inferred Goals The user appears to want a long-lived, auditable project spec that can survive chat transitions and aggregation. The preservation prompt confirms this.

The user appears to want architecture that remains portable across older desktop systems and future render/platform providers. This was inferred from the platform floor and repeated concern with portability/modularity/extensibility.

The user appears to want a clean distinction between reusable framework, engine implementation, and game implementation. This became explicit later but was also implied by the framework approach.

4.3 Goals That Changed Over Time The conversation began as "Can we use open source to avoid starting from nothing?" It evolved into a much more precise doctrine: "Build a Domino Framework with service contracts and providers, and use raylib/SDL2/Lua as first providers."

Early directory ideas were more vendor-shaped. Later they became service-first. Early raylib use was described as `runtime/render/raylib`; later the refined pattern became `runtime/render/providers/raylib`.

The idea of using open-source games shifted from potential code reuse to reference/research material due to license and architecture risks.

4.4 Goals Still Unresolved The exact first implementation branch, versions, and dependency pins are unresolved. The exact Domino Framework ABI is unresolved. The exact Lua version is unresolved. The current repository baseline must be verified. The formal policy for GPL/LGPL/unclear license material is unresolved. The sparse simulation and CAD systems need formal specs and prototypes.

5. Decisions Made and Why

Decision overview

Row 1 ID: DECISION-01; Decision: Use open-source code primarily through a framework/provider approach, not by mutating one large fork.; Status: Accepted direction; Why it mattered: Preserves Dominion law and allows replaceability.; Confidence: 4; Label: FACT/INFERENCE

Row 2 ID: DECISION-02; Decision: raylib should be used heavily as the first visible provider suite.; Status: Accepted direction; Why it mattered: Fast client, Workbench, rendering, audio, import preview.; Confidence: 5; Label: FACT

Row 3 ID: DECISION-03; Decision: Dominion/Domino contracts, simulation, replay, saves, packs, commands, UI documents, provider law remain first-party.; Status: Accepted doctrine; Why it mattered: Avoids lock-in and supports deterministic/replayable game law.; Confidence: 5; Label: FACT/INFERENCE

Row 4 ID: DECISION-04; Decision: Use service-first runtime layout: runtime/<service>/providers/<provider>; Status: Accepted direction; Why it mattered: Avoids vendor-shaped architecture.; Confidence: 5; Label: FACT/INFERENCE

Row 5 ID: DECISION-05; Decision: Profiles select provider combinations; apps should remain generic.; Status: Accepted direction; Why it mattered: Avoids product forks per backend.; Confidence: 4; Label: FACT/INFERENCE

Row 6 ID: DECISION-06; Decision: Keep SDL2 as a first-wave provider, not a fallback afterthought.; Status: Accepted direction; Why it mattered:

Gives stable platform/input/audio substrate.;
Confidence: 4; Label: FACT/INFERENCE

Row 7 ID: DECISION-07; Decision: Use Lua, but pin the Lua provider and expose Dominium script API separately.; Status: Accepted direction; Why it mattered: Prevents raw Lua ABI from becoming mod law.; Confidence: 4; Label: FACT/INFERENCE

Row 8 ID: DECISION-08; Decision: rlsw can be used as a raylib software-render provider, but not as Dominium canonical reference renderer.; Status: Recommended; likely accepted direction; Why it mattered: Maintains first-party deterministic evidence path.; Confidence: 4; Label: FACT/INFERENCE

Row 9 ID: DECISION-09; Decision: rlgl can be used as OpenGL-family provider, but should not be named as final opengl33 law.; Status: Recommended; likely accepted direction; Why it mattered: Preserves future direct OpenGL provider.; Confidence: 4; Label: FACT/INFERENCE

Row 10 ID: DECISION-10; Decision: SpaceEngine, Celestia, PCGUniverse2, and pgg should mainly be reference material, not direct code dependencies.; Status: Recommended; Why it mattered: Avoids

legal and architectural contamination.; Confidence: 4; Label: FACT/INFERENCE

Row 11 ID: DECISION-11; Decision: For sparse/infinite game design, use infinite address space with finite active set and deterministic cells.; Status: Recommended design doctrine; Why it mattered: Makes infinite/arbitrary world computationally feasible.; Confidence: 4; Label: INFERENCE

Row 12 ID: DECISION-12; Decision: Clients may contribute compute but should not be blindly trusted as authoritative.; Status: Recommended doctrine; Why it mattered: Prevents cheating/desync in distributed compute.; Confidence: 4; Label: INFERENCE

Row 13 ID: DECISION-13; Decision: Arbitrary CAD creations should compile into bounded runtime representations.; Status: Recommended doctrine; Why it mattered: Separates design-time freedom from runtime tractability.; Confidence: 4; Label: INFERENCE

Row 14 ID: DECISION-14; Decision: Create a full preservation package for this chat.; Status: Explicit user instruction; Why it mattered: Allows aggregation and future continuation.; Confidence: 5; Label: FACT

Decision explanations The main accepted direction was the framework/provider approach. The user explicitly said they liked the framework approach and asked whether it could be used to make a Domino framework provided by any engine implementation and consumed by any Dominion game implementation. This makes sense because it lets the project use raylib and other libraries quickly without baking them into game law.

The raylib decision was also directionally accepted. The user repeatedly expressed enthusiasm for raylib and asked about using as much of raylib infrastructure as possible. The caveat is that raylib is a provider suite, not architecture. This affects rendering, audio, input, asset preview, and Workbench bootstrap.

The service-first layout decision matters because it prevents vendor-shaped directories and product variants. A profile can choose raylib or SDL2 providers without making a separate client architecture.

The SDL2 and Lua decisions matter because they balance speed with portability and extensibility. SDL2 gives a stable platform/input/audio substrate. Lua gives scripting, but the chat decided raw Lua version should not define mod law.

The sparse deterministic delegated simulation decisions are design doctrines, not implemented decisions. They

are accepted as the recommended direction but still need formal proof and prototypes.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

Row 1 ID: REJECTED-01; Option: Forking a full engine as the main architecture.; Status: Deprioritised; Reason: Would overwrite Dominion law and create lock-in.; Final or tentative?: Tentative but strong; Reconsider conditions: Reconsider only for isolated prototype or tool, not canonical core.; Related workstream: WORKSTREAM-01; Label: FACT/INFERENCE

Row 2 ID: REJECTED-02; Option: Letting raylib define game architecture.; Status: Rejected; Reason: raylib should be provider, not law.; Final or tentative?: Strong; Reconsider conditions: Never reconsider unless project goal changes.; Related workstream: WORKSTREAM-04; Label: FACT

Row 3 ID: REJECTED-03; Option: Placing provider identity into app folder names such as apps/client/rendered/raylib.; Status: Superseded; Reason: Profiles should select providers; apps remain generic.; Final or tentative?: Strong; Reconsider conditions: Temporary proof apps may exist under proof/experiments.; Related workstream:

WORKSTREAM-03; Label: FACT/INFERENCE

Row 4 ID: REJECTED-04; Option: Top-level labs/ or profiles/ roots as main architecture.; Status: Deprioritised; Reason: Could reopen top-level-root sprawl; prefer content/profiles or release/profiles and tools/experiments.; Final or tentative?: Tentative; Reconsider conditions: Reconsider if repo governance adopts those roots deliberately.; Related workstream: WORKSTREAM-03; Label: INFERENCE

Row 5 ID: REJECTED-05; Option: Using rlsw as canonical Dominion software renderer.; Status: Rejected; Reason: rlsw is raylib ecosystem provider; Dominion reference should be first-party.; Final or tentative?: Strong; Reconsider conditions: Could be temporary fallback but not proof law.; Related workstream: WORKSTREAM-04; Label: FACT/INFERENCE

Row 6 ID: REJECTED-06; Option: Using rlgl as the final direct opengl33 provider.; Status: Rejected/ superseded; Reason: rlgl is raylib abstraction; direct opengl33 should be separate.; Final or tentative?: Strong; Reconsider conditions: rlgl can implement first OpenGL-family provider.; Related workstream: WORKSTREAM-04; Label: FACT/ INFERENCE

Row 7 ID: REJECTED-07; Option: Trusting client-contributed simulation results without verification.; Status: Rejected; Reason: Cheating/desync risk.; Final or tentative?: Strong; Reconsider conditions: Only in local trusted sandbox/co-op if explicitly chosen.; Related workstream: WORKSTREAM-09; Label: INFERENCE

Row 8 ID: REJECTED-08; Option: Simulating every arbitrary object everywhere continuously.; Status: Rejected; Reason: Impossible/unbounded; sparse activation required.; Final or tentative?: Strong; Reconsider conditions: Local dense simulation can exist in bounded active cells.; Related workstream: WORKSTREAM-09; Label: INFERENCE

Row 9 ID: REJECTED-09; Option: Copying code from proprietary/GPL/unclear-license projects by default.; Status: Rejected; Reason: License and architecture risk.; Final or tentative?: Strong; Reconsider conditions: Reconsider only with explicit license strategy/quarantine.; Related workstream: WORKSTREAM-08; Label: FACT/INFERENCE

The most important rejected idea is using a full engine fork as the project’s main foundation. It was rejected because it would likely make Dominion inherit the full engine’s assumptions. The second important rejected

idea is allowing raylib to become the engine architecture. The chat repeatedly preserved the boundary: use raylib heavily, but only behind providers.

A more subtle superseded idea was app-variant architecture. Early structures such as `apps/client/rendered/raylib` were useful for proof bootstraps, but the stronger model is generic apps plus provider profiles.

Another rejected idea was using external reference projects directly without license review. This matters because later assistants might see Celestia, PCGUniverse2, pgg, or SpaceEngine and assume code can be copied. The chat did not authorize that.

7. Important Reasoning, Rationale, and Tradeoffs

The central tradeoff was speed versus architectural control. The user wanted fast progress. The answer was to use existing libraries, but only behind first-party service contracts. This keeps the project from getting stuck in a third-party object model or rendering/input/save system.

Another tradeoff was convenience versus determinism. raylib, SDL2, Lua, and physics/rendering helpers are convenient, but deterministic simulation cannot depend on wall-clock time, raw floating-point platform differences, uncontrolled Lua execution, GPU physics, thread races, or arbitrary library internals. Therefore

rendering/audio/UI are non-authoritative providers, while simulation commands, state hashes, replay, and save law remain first-party.

A third tradeoff was arbitrary creativity versus finite runtime cost. The user wants arbitrary CAD-style creations and machines. The answer was not to restrict design-time creativity, but to compile designs into bounded runtime structures.

A fourth tradeoff was client-contributed compute versus trust. Clients can help with meshing, planning, simulation proposals, and non-authoritative work, but valuable state must be verified by the host/server.

A fifth tradeoff was open-source reuse versus licensing risk. Permissive libraries can be providers. GPL/proprietary/unclear projects should be research references unless the project deliberately adopts a compatible license/quarantine strategy.

8. Plans, Future Work, and Next Steps

The main implementation path is:

1. Verify repo baseline and dependency versions.
2. Create Domino Framework/provider ABI skeleton.
3. Add null providers and validators.
4. Add raylib/rlgl/rlsw/raygui/raudio providers.

5. Add SDL2 platform/input/audio provider.
6. Add Lua script provider.
7. Add profiles: client.raylib, client.sdl2_raylib, workbench.raylib, server.null.
8. Add conformance tests and deterministic replay/state hash tests.
9. Develop worldgen/sparse simulation/CAD schemas and prototypes.

The first concrete wedge should be **DOMINO-FRAMEWORK-WEDGE-01**: framework headers, provider registry, profile loader, null providers, raylib providers, and validation tools. The expected output is a server-null profile that runs deterministic ticks and a client-raylib profile that opens a window and renders non-authoritative presentation.

Risks include stale repo assumptions, platform floor incompatibility, license mistakes, third-party type leakage, and overbuilding sparse/CAD systems before a minimal deterministic cell simulator exists.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences The user requires source-grounded, audit-ready, uncertainty-labelled responses. The preservation prompt explicitly

requires FACT/INFERENCE/UNCERTAIN/
PROJECT-CONTEXT labels, no invented facts, no
treating brainstorm as decisions, and no over-
compression. The prompt also requires a human-
readable report first and downloadable files if possible
?filecite?turn29file0?.

Technical constraints explicitly mentioned include
Windows 7 SP1+, macOS 10.9.5+, Linux, C17/C++17,
Win32, Cocoa, SDL2, and OpenGL 3.3.

9.2 Inferred Constraints and Preferences The user
prefers modular architecture over monolithic engine
forks. The user prefers long-term portability,
extensibility, and replaceability. The user cares about
preserving reasoning and rejected options for future
aggregation.

9.3 Uncertain or Unestablished Preferences The
exact project license is not established here. The exact
dependency versions are not established. The exact
tolerance for GPL/LGPL code is not established. The
exact first implementation branch is not established.

10. Files, Artifacts, Outputs, and Prompts

The main uploaded artifact is `Pasted text.txt`,
containing the preservation task prompt and output

requirements ?filecite?turn29file0?. Before this task, no generated report/ZIP package was visible in this chat. This response creates the requested package.

Important in-chat artifacts include the raylib deep-dive outputs, directory structure proposals, provider manifest examples, the julesc013/dominium repo comparison findings, open-source project classifications, sparse deterministic delegated simulation architecture, and Domino Framework wedge proposal.

Row 1 ID: ARTIFACT-01; Artifact / file / prompt / output: Pasted text.txt preservation prompt; Type: Uploaded file / prompt; Purpose: Defines required preservation package, sections, files, and labels.; Status: Available in this chat; Origin: User upload; Carry forward?: Yes; Notes: Primary instruction artifact for this task.

?filecite?turn29file0?; Label: FACT

Row 2 ID: ARTIFACT-02; Artifact / file / prompt / output: Prior assistant raylib/provider architecture messages; Type: In-chat outputs; Purpose: Developed service-first provider doctrine, raylib/SDL/Lua role, directory structures.; Status: Visible in chat context; Origin: This chat; Carry forward?: Yes; Notes: Substantive source for report.; Label: FACT

Row 3 ID: ARTIFACT-03; Artifact / file / prompt /

output: GitHub inspection snippets for julesc013/
dominium; Type: Tool results in chat; Purpose:
Compared current repo dsys/dgfx stubs and
possible CMake baseline.; Status: Visible in chat
context; freshness uncertain; Origin: GitHub
connector during chat; Carry forward?: Yes, with
verification; Notes: Use as historical inspection, not
current truth.; Label: UNCERTAIN/
UNVERIFIED

Row 4 ID: ARTIFACT-04; Artifact / file / prompt /
output: Lists of external projects and reference
links; Type: In-chat lists; Purpose: SpaceEngine,
Celestia, PCGUniverse2, pgg, open-source engines/
games.; Status: Visible in chat; Origin: User and
assistant messages; Carry forward?: Yes; Notes:
Feed research/reference lane.; Label: FACT

Row 5 ID: ARTIFACT-05; Artifact / file / prompt /
output: DOMINO-FRAMEWORK-WEDGE-01
proposal; Type: Plan/output; Purpose: First
implementation wedge for framework/provider
architecture.; Status: Proposed, not implemented;
Origin: Assistant suggestion accepted directionally;
Carry forward?: Yes; Notes: Should become task/
epic.; Label: INFERENCE

Row 6 ID: ARTIFACT-06; Artifact / file / prompt /
output: PROVIDER-WEDGE / RENDER-

WEDGE / SCRIPT-WEDGE naming; Type: Plan/output; Purpose: Staged implementation plan.; Status: Proposed, not implemented; Origin: Assistant outputs; Carry forward?: Yes; Notes: Useful for backlog.; Label: INFERENCE

Row 7 ID: ARTIFACT-07; Artifact / file / prompt / output: This preservation package files; Type: Generated files; Purpose: Markdown/YAML/ZIP handoff package.; Status: Created by current response; Origin: Assistant file export; Carry forward?: Yes; Notes: Use for aggregation and future continuation.; Label: FACT

11. Open Questions and Unresolved Issues

Row 1 ID: QUESTION-01; Question / issue: Which exact versions of raylib, SDL2, Lua, raygui, ImGui, and related libraries should be pinned?; Why it matters: Platform floors and API stability depend on versions.; Known information: The conversation recommended using them.; Unknown information: Exact versions and compatibility with Win7/macOS 10.9.5.; Resolution path: Verify upstream release notes and build tests.; Priority: P0; Related workstream: WORKSTREAM-04; Label: UNCERTAIN/UNVERIFIED

Row 2 ID: QUESTION-02; Question / issue: Is

Dominium repo main branch actually C17/C++17 or still C90/C++98?; Why it matters: Build baseline affects raylib and framework.; Known information: Earlier assistant inspection found possible C90/C++98 contradiction; later transcript claimed C17/C++17.; Unknown information: Current branch truth.; Resolution path: Inspect current repo branch before implementation.; Priority: P0; Related workstream: WORKSTREAM-07; Label: UNCERTAIN/UNVERIFIED

Row 3 ID: QUESTION-03; Question / issue: Should the canonical Lua provider be lua54 or lua55?; Why it matters: Script ABI and mod compatibility.; Known information: Lua should be pinned and hidden behind dominium.script.v1.; Unknown information: Specific series choice.; Resolution path: Compare maturity, compatibility, platform support.; Priority: P1; Related workstream: WORKSTREAM-06; Label: UNCERTAIN

Row 4 ID: QUESTION-04; Question / issue: What is the first minimal Domino Framework ABI?; Why it matters: Blocks implementation.; Known information: Concept defined but not specified fully.; Unknown information: Exact structs/functions/services.; Resolution path: Draft DOMINO-FRAMEWORK-WEDGE-01 header.;

Priority: P0; Related workstream:

WORKSTREAM-02; Label: INFERENCE

Row 5 ID: QUESTION-05; Question / issue: What first provider profile should be implemented: client.raylib, client.sdl2_raylib, or server.null?; Why it matters: Implementation order.; Known information: All were proposed.; Unknown information: Priority/order and acceptance tests.; Resolution path: Choose wedge scope.; Priority: P0; Related workstream: WORKSTREAM-02; Label: INFERENCE

Row 6 ID: QUESTION-06; Question / issue: How strict must deterministic simulation be across platforms?; Why it matters: Affects math, threads, providers.; Known information: Determinism required at sim/command layer, not render/audio/UI.; Unknown information: Exact determinism class and tolerated divergence.; Resolution path: Define determinism spec.; Priority: P0; Related workstream: WORKSTREAM-09; Label: INFERENCE

Row 7 ID: QUESTION-07; Question / issue: What license policy will Dominium adopt for GPL/LGPL/unclear-license references?; Why it matters: Code reuse boundaries.; Known information: Reference-only default recommended.; Unknown information: Formal project legal policy.;

Resolution path: Write license/provenance policy.;
Priority: P1; Related workstream:
WORKSTREAM-08; Label: UNCERTAIN

Row 8 ID: QUESTION-08; Question / issue: What is the minimal CAD construct graph for first implementation?; Why it matters: Arbitrary invention system is broad.; Known information: Graph/ports/machine model proposed.; Unknown information: First component set and compile pipeline.; Resolution path: Define CAD-WEDGE-01.; Priority: P1; Related workstream: WORKSTREAM-10; Label: INFERENCE

Row 9 ID: QUESTION-09; Question / issue: How will client-contributed work be verified and what trust tiers exist?; Why it matters: Distributed compute feasibility and security.; Known information: Work lease/quorum/spot-check model proposed.; Unknown information: Exact verification policy.; Resolution path: Prototype host/client lease smoke.; Priority: P0; Related workstream: WORKSTREAM-09; Label: INFERENCE

The most blocking unresolved issues are repo baseline verification, exact dependency pins, minimal framework ABI, provider profile order, deterministic simulation spec, and license policy.

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Row 1 ID: RISK-01; Risk / failure mode: Future assistant treats recommendations as final decisions.; Consequence: Spec may overcommit.; Likelihood: Medium; Severity: High; Mitigation: Use status labels: accepted direction vs proposed vs unresolved.; Related workstream: All; Label: FACT/INFERENCE

Row 2 ID: RISK-02; Risk / failure mode: Third-party libraries leak into public contracts.; Consequence: Long-term lock-in, save/replay breakage.; Likelihood: Medium; Severity: High; Mitigation: Forbidden include validator and provider manifests.; Related workstream: WORKSTREAM-03; Label: FACT/INFERENCE

Row 3 ID: RISK-03; Risk / failure mode: C17/C++17 baseline assumed without repo verification.; Consequence: Build failures or wrong plan.; Likelihood: Medium; Severity: High; Mitigation: Verify current branch before implementation.; Related workstream: WORKSTREAM-07; Label: UNCERTAIN/UNVERIFIED

Row 4 ID: RISK-04; Risk / failure mode: raylib/SDL/ Lua current versions may not support stated old OS floors.; Consequence: Platform target failure.;

Likelihood: Medium; Severity: High; Mitigation: Pin/test exact versions on Win7/macOS 10.9.5/Linux.; Related workstream: WORKSTREAM-04; Label: UNCERTAIN/UNVERIFIED

Row 5 ID: RISK-05; Risk / failure mode: GPL/unclear-license project code copied accidentally.; Consequence: Legal contamination.; Likelihood: Low-medium; Severity: High; Mitigation: Research-only lane, license validator, provenance policy.; Related workstream: WORKSTREAM-08; Label: FACT/INFERENCE

Row 6 ID: RISK-06; Risk / failure mode: Distributed client compute becomes cheat vector.; Consequence: Invalid world state/economy/combat.; Likelihood: High if unmitigated; Severity: High; Mitigation: Host verification, work leases, state hashes, trust tiers.; Related workstream: WORKSTREAM-09; Label: INFERENCE

Row 7 ID: RISK-07; Risk / failure mode: Arbitrary CAD runtime becomes computationally unbounded.; Consequence: Performance collapse/desync.; Likelihood: High if raw CAD simulated; Severity: High; Mitigation: Compile to bounded graphs/proxies and enforce budgets.; Related workstream: WORKSTREAM-10; Label: INFERENCE

- Row 8** ID: RISK-08; Risk / failure mode: Report overstates access to full chat or past-chat links.; Consequence: Bad aggregation.; Likelihood: Medium; Severity: Medium; Mitigation: State source limitation; use this chat only.; Related workstream: WORKSTREAM-11; Label: FACT
- Row 9** ID: RISK-09; Risk / failure mode: External facts go stale.; Consequence: Wrong dependency/legal choices.; Likelihood: High over time; Severity: Medium-high; Mitigation: Verification queue before implementation.; Related workstream: All; Label: FACT

A future assistant might incorrectly say "we decided to use raylib as the engine." The correct statement is that raylib is a first visible provider suite. Another common mistake would be to treat every assistant proposal as a user decision. Several items are recommended directions, not implemented decisions.

A future assistant might also rely on stale external facts or repository inspections. Dependency versions, OS floors, repo CMake state, and licenses must be verified.

13. What This Chat Contributes to the Larger Project or Future Spec Book

- Row 1** Spec-book area: Framework architecture; Contribution from this chat: Domino Framework

API, engine implementation, game implementation split.; Source IDs: DECISION-01, DECISION-14, WORKSTREAM-02; Should become requirement/context/open issue?: Requirement; Confidence: 5; Notes: Central spec chapter.

Row 2 Spec-book area: Provider system; Contribution from this chat: Service-first runtime providers, manifests, profiles, conformance tests.; Source IDs: DECISION-03..05, WORKSTREAM-03; Should become requirement/context/open issue?: Requirement; Confidence: 5; Notes: Core architecture.

Row 3 Spec-book area: First-wave dependencies; Contribution from this chat: raylib, rlgl, rlsw, raygui, raudio, SDL2, Lua.; Source IDs: DECISION-02, DECISION-06..09; Should become requirement/context/open issue?: Requirement/context with verification; Confidence: 4; Notes: Needs version/platform checks.

Row 4 Spec-book area: License/provenance; Contribution from this chat: Reference-only lane for GPL/proprietary/unclear projects.; Source IDs: DECISION-10, CONSTRAINT-07; Should become requirement/context/open issue?: Requirement; Confidence: 4; Notes: Should become legal/provenance policy.

- Row 5** Spec-book area: Sparse deterministic simulation; Contribution from this chat: Infinite address space, finite active set, work leases, host verification.; Source IDs: DECISION-11..13, WORKSTREAM-09; Should become requirement/context/open issue?: Requirement/open issue; Confidence: 4; Notes: Needs formal protocols.
- Row 6** Spec-book area: CAD/invention system; Contribution from this chat: CAD documents compile into bounded construct/machine graphs.; Source IDs: DECISION-13, WORKSTREAM-10; Should become requirement/context/open issue?: Requirement/open issue; Confidence: 4; Notes: Needs first component model.
- Row 7** Spec-book area: Workbench direction; Contribution from this chat: Workbench edits Dominion documents, not raygui objects; uses providers.; Source IDs: WORKSTREAM-04, WORKSTREAM-10; Should become requirement/context/open issue?: Requirement/context; Confidence: 4; Notes: Ties UI documents to tooling.
- Row 8** Spec-book area: Preservation/aggregation; Contribution from this chat: This chat is converted into report, registers, spec sheet, ZIP.; Source IDs: WORKSTREAM-11; Should become requirement/

context/open issue?: Context; Confidence: 5;
Notes: Feeds master spec book.

This chat contributes the high-level architecture for the Domino Framework and provider system. It also contributes first-wave dependency strategy, research/reference project classification, sparse deterministic delegated simulation, and CAD/machine system framing. These should become formal chapters in a master Project Spec Book.

14. What I Should Remember

- Dominion should not become a raylib game or a fork of another engine. It should become a contract-governed framework/game system.
- The framework/provider approach is the central conclusion.
- raylib is useful now, but only as providers: raylib, rlg, rls, raygui, raudio, asset import.
- SDL2 and Lua remain first-wave providers.
- Third-party types must not enter contracts, game law, content, saves, replays, or public SDK.
- Profiles select providers; apps stay generic.
- External games/engines are mostly reference material, not code bases to copy.

- The world architecture should be sparse, deterministic, delegated, and host-verified.
- Arbitrary CAD creations should compile into bounded runtime graphs.
- The next practical step is a minimal Domino Framework/provider wedge with null and raylib providers.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- Explain the difference between Domino Framework, Domino engine implementation, and Dominion game implementation.
- Explain why raylib is a provider suite rather than the engine architecture.
- Summarize the service-first/provider-first directory model.

15.2 Decisions

- Which decisions in this chat are explicit user-accepted decisions versus assistant recommendations?

- Which decisions should become formal requirements in the spec book?
- Which decisions need user confirmation before implementation?

15.3 Tasks and Next Actions

- Draft the exact `DOMINO-FRAMEWORK-WEDGE-01` file tree and PR checklist.
- Draft the first provider ABI header in C.
- Draft the forbidden-include validator.

15.4 Artifacts and Files

- Show me the artifact ledger and explain which artifacts should feed the master spec book.
- Generate the provider manifest templates for raylib, SDL2, Lua, and null providers.

15.5 Risks and Verification

- Verify the current `julesc013/dominium` CMake language baseline.
- Build a verification checklist for raylib/SDL2/Lua platform support.

- Create a license/provenance policy for reference projects.

15.6 Future Spec Book / Aggregation

- Convert this package into a spec-book chapter outline.
- Identify likely duplicates/conflicts with other old chats.

15.7 Deep-Dive Questions Specific to This Chat

- Design the sparse deterministic work-lease protocol.
- Design the first CAD construct graph schema.
- Design the `client.raylib` and `server.null` profiles.

16. Compact Human Summary

This chat established how Dominium should accelerate development without losing its own architecture. The user wanted to know whether existing open-source systems could be used instead of starting from nothing. The answer evolved into a clear framework/provider doctrine: use open-source libraries and projects heavily, but only through first-party Domino contracts and replaceable providers.

The central architectural conclusion is that Domino should be a framework: it defines stable contracts, service APIs, provider manifests, profiles, capability/refusal law, conformance tests, and deterministic simulation boundaries. A Domino engine implementation provides those services. Dominium is one game implementation that consumes them. Providers such as raylib, SDL2, Lua, raygui, raudio, rlgl, rlsw, ImGui, and others implement specific services, but they do not define the game's law.

raylib was the most deeply discussed dependency. The chat concluded that raylib should be used aggressively because it can quickly provide a visible client, Workbench panels, drawing, audio, asset preview, and screenshots. But raylib must be fenced. `rlsw` can be a raylib software-render provider but not the canonical Dominium reference renderer. `rlgl` can be an OpenGL-family provider but not the final direct `opengl33` provider. `raygui` can bootstrap tools but not define UI documents. `raudio` can provide early audio. `raymath` is only for editor/render math, not deterministic simulation. Asset loaders are import/preview helpers, not Dominium asset law.

SDL2 remains important as a first-wave platform/input/audio provider. Lua is a good scripting provider for packs, mods, CLI, Workbench automation, and scenario hooks, but it must be pinned and hidden behind a

Dominium script API such as `dominium.script.v1`.
The raw Lua ABI should not become mod law.

The conversation also evaluated external projects. SpaceEngine, Celestia, PCGUniverse2, and Valian/pgg can help as references, but not as direct dependencies by default. Celestia is GPL-constrained. SpaceEngine is proprietary. PCGUniverse2 and pgg had unclear license status in the chat. Other open-source games and engines such as Lianti, OpenRA, Spring, Mindustry, Freeciv, Endless Sky, Pioneer, Godot, O3DE, and Torque3D are valuable mostly as research references for subsystems: voxel worlds, command simulation, factories, rulesets, space content, editor architecture, and plugin ecosystems.

The user's broader game vision led to a second major architecture: sparse deterministic delegated simulation. The game should be infinite by addressability, not by simulating everything everywhere. The active world should be partitioned into cells/islands/constructs. Cold regions are procedural seeds plus saved deltas. Active cells tick deterministically. Clients may contribute compute, but only through work leases whose outputs are verified by the host/server before becoming authoritative. Player/agent/NPC creations can be arbitrary at design time, but must compile into bounded runtime structures: construct graphs, machine graphs, physics proxies, render proxies, and event-sourced state.

The most important next action is to implement DOMINO-FRAMEWORK-WEDGE-01: framework headers, provider ABI, null providers, raylib providers, provider manifests, profiles, and validators. The immediate blocker is verification: confirm the actual repo language baseline, dependency versions, platform floors, and license constraints before implementing.

10.2 Domino Framework Open Source Provider Architecture Reader Brief

Source: docs/archive/conversations/Framework_Open_Source_Provider/Domino_Framework_Open_Source_Provider_Architecture__06_reader_brief.md

Report type: reader_brief

Size: 2726 bytes

10.2.1 Reader Brief - Domino Framework and Open-Source Provider Architecture

What this chat was about

This chat developed a framework/provider strategy for Dominion. The user wanted to accelerate development with open-source code while preserving portability,

modularity, extensibility, determinism, and future replaceability. The final architecture is Domino Framework + Domino engine implementation + Dominium game implementation. Third-party libraries become providers.

Top 20 things to know

1. Use the framework approach, not a monolithic fork.
2. Domino Framework defines contracts and provider ABI.
3. A Domino engine implementation provides services.
4. Dominium is one game implementation using those services.
5. raylib is a first visible provider suite, not the engine.
6. r1sw is a raylib software provider, not Dominium reference renderer.
7. r1gl is an OpenGL-family provider, not final opengl33.
8. raygui is Workbench/debug UI provider, not UI document law.
9. raudio is audio provider.
10. raymath is render/editor math only.

11. SDL2 remains platform/input/audio provider.
12. Lua should be pinned behind `dominium.script.v1`.
13. Use `runtime/<service>/providers/<provider>`.
14. Apps remain generic; profiles select providers.
15. Third-party types are forbidden in `contracts/game/content/saves/replays`.
16. External games/engines are mostly references.
17. Sparse deterministic delegated simulation is core future architecture.
18. Clients can compute but host verifies results.
19. CAD creations compile into bounded runtime graphs.
20. Next step is `DOMINO-FRAMEWORK-WEDGE-01`.

Decisions

See Decision Register. Most important: framework/provider architecture, raylib as first provider suite, service-first layout, profiles for provider selection, Lua/SDL2 as first-wave providers, and sparse deterministic delegated simulation as design doctrine.

Pending tasks

Top tasks: verify repo baseline, draft framework ABI, add provider manifests, add forbidden include validator, implement null providers, implement raylib providers, create profiles, and define work-lease/CAD schemas.

Open questions

Main open questions: dependency versions, C17/C++17 repo state, Lua version, framework ABI, profile order, deterministic class, license policy, CAD graph scope, and distributed compute verification.

Artifacts

Primary artifact: uploaded preservation prompt. This package creates the full handoff files and ZIP.

Verification items

Verify repository state, raylib/SDL2/Lua versions and platform support, licenses for reference projects, and any second-wave dependency licenses.

Best next step

Draft DOMINO-FRAMEWORK-WEDGE-01 into an implementation-ready PR plan.

10.3 Domino Framework Open Source Provider Architecture In Chat Reader

Source: docs/archive/conversations/Framework_Open_Source_Provider/Domino_Framework_Open_Source_Provider_Architecture__09_in_chat_reader.md

Report type: in_chat_reader

Size: 2072 bytes

10.3.1 In-Chat Reader - Domino Framework and Open-Source Provider Architecture

Package overview

This package preserves a chat about building Dominion/Domino through a modular open-source framework/provider strategy. It includes a human-readable report, structured registers, context transfer packet, YAML spec sheet, aggregator packet, audit, reader brief, future-chat prompt, and ZIP package.

File index

- 00_manifest.md: what was created and why.
- 01_human_readable_report.md: sections 0-16,

best for understanding the chat.

- `02_context_transfer_packet.md`: paste into a future chat to continue.
- `03_spec_sheet.yaml`: machine-friendly aggregation spec.
- `04_registers.md`: decisions, tasks, risks, questions, artifacts, verification items.
- `05_aggregator_packet.md`: compact merge packet for a master spec aggregator.
- `06_reader_brief.md`: short human summary.
- `07_verification_and_audit.md`: audit and verification queue.
- `08_future_chat_bootstrap_prompt.md`: standalone prompt for continuation.
- `09_in_chat_reader.md`: this guide.

Plain-English explanation

The chat concluded that Dominium should use open-source code through providers, especially raylib, SDL2, Lua, raygui, raudio, rlgl, and rlsw, while preserving first-party simulation and contracts. It also defined a sparse deterministic delegated simulation model and CAD-to-bounded-graphs model for the user's long-term game vision.

Question menu

Ask next: "Draft DOMINO-FRAMEWORK-WEDGE-01," "Define the provider ABI header," "Verify repo C17/C++17 state," "Design the work-lease protocol," "Create raylib/SDL/Lua provider manifests," or "Turn this into a spec book chapter."

Top things to preserve

- service-first/provider-backed architecture
- third-party type fencing
- raylib as provider suite, not engine law
- profiles select providers
- sparse deterministic delegated simulation
- CAD designs compile into bounded runtime graphs

Safest next actions

Verify current repo/build facts, then draft the first framework/provider wedge.

Chapter 11

Language Platform Architecture

11.1 Dominion Language Platform Architecture Baseline Human Readable Report

Source: docs/archive/conversations/Language_
Platform_Architecture/Dominium_Language_
Platform_Architecture_Baseline__01_human_
readable_report.md

Report type: human_readable_report

Size: 38542 bytes

11.1.1 COMPLETE CHAT PRESERVATION REPORT - Dominion Language, Platform, and Architecture Baseline

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion
 Language, Platform, and Architecture Baseline

Row 2 Field: Date anchor; Assessment: 2026-05-27
 Australia/Melbourne

- Row 3** Field: Source scope; Assessment: This chat only, with PROJECT-CONTEXT/repo citations where used
- Row 4** Field: Apparent access; Assessment: Partial-to-high for visible/current chat; not full independent access to all other old chats
- Row 5** Field: Previously generated files available?; Assessment: No prior generated downloadable files visible in this chat; this response creates files
- Row 6** Field: Uploaded files or artifacts present?; Assessment: Yes: Pasted text.txt preservation prompt; tool also noted some earlier uploaded files may have expired
- Row 7** Field: Contains future plans?; Assessment: Yes
- Row 8** Field: Contains decisions?; Assessment: Yes, with some recommendations still not implemented
- Row 9** Field: Contains pending tasks?; Assessment: Yes
- Row 10** Field: Contains unresolved questions?; Assessment: Yes
- Row 11** Field: Staleness risk; Assessment: Medium: toolchain/platform/library facts are time-sensitive
- Row 12** Field: Extraction confidence; Assessment: 4/5 for this visible chat; 3/5 for claims relying on external repo/toolchain state

Row 13 Field: Safe for later aggregation?; Assessment: Yes, with caveats and labels

Row 14 Field: Main limitations; Assessment: Cannot guarantee full contents of "other chats"; repo may change; external platform claims require verification; some uploaded files outside current prompt may have expired

This report covers the visible/current chat and the attached preservation prompt. The prompt itself requests a maximum-fidelity human-readable and structured export package for this chat. The file is available as `Pasted text.txt` and is treated as an artifact to preserve. Some repo/source facts were already fetched in this chat and are carried forward with citations where available. The main caveat is that the first user message asked to scan "other chats in this repo," but this preservation report cannot independently reconstruct all other chats beyond project context and the visible transcript. External toolchain/platform claims are treated as verification items, not permanently settled facts.

1. One-Page Orientation

This chat was a design-decision and architecture-baseline discussion for Dominion, focused on language standards, platform floors, binary architecture, portability strategy,

and the project's long-term modular structure. The user began by asking whether Dominium would gain performance or features by moving beyond the earlier C89 engine and C++98 game plan. Over the conversation, the topic expanded into a full re-evaluation of what should be written in C versus C++, what standards should define the mainline, which operating systems should still count as full native targets, and how old or exotic systems should be supported without distorting the modern architecture.

The most important evolution was away from the crude split "engine = C, game = C++." The better rule that emerged was semantic: C should own stable law, ABI, fixed-width data, serialization, replay, packets, renderer command IR, and deterministic primitives; C++ should own resource-heavy machinery, runtime services, providers, apps, game orchestration, tools, Workbench, and composition. This is the central idea to preserve. The project does not need pure C, pure C++, or a language ideology. It needs stable contracts at the boundaries and modern implementation power internally.

The user initially considered staged old-to-new plans: versions 0.x/1.x using C89/C++98 and later versions moving to C11/C++17. That plan was superseded as the user accepted a more decisive baseline: C17 and C++17 across the mainline, with 64-bit source-native builds and little-endian assumptions. The old C89/

C++98 goal moved from active mainline law into retro/projection/archive thinking. The current repo state discussed in the chat supports this: root and engine CMake were observed as already requiring C17/C++17, while the repository still contains documentation and comments that should be updated to match the new doctrine.

The chat also clarified portability. The user wanted future support for Windows XP, Windows 98, Mac OS X 10.6, Mac OS 9.2, Android, iOS, PS5, Xbox, and Switch. The answer was not to pretend one primitive binary can execute everywhere. The correct model is one canonical project with contracts, content, tests, deterministic law, C-compatible ABI, and renderer/platform/provider interfaces, then multiple products or projections: full 64-bit source-native builds for modern supported systems, constrained native builds for some older environments, contract projections for extreme legacy systems, and archive/emulator lanes for museum targets.

Another major thread was modularity. The user wanted code, data, modules, packs, tools, apps, and Workbench to integrate cleanly and be reusable. The answer became a contract-centered composition model: apps compose modules; modules require services; services are implemented by providers; providers declare capabilities; packs provide data/modules/content; profiles select packs/settings; contracts define compatibility; lockfiles

make composition reproducible; commands invoke behavior; artifacts record results; tests prove it; Workbench operates it; AIDE governs it. This is the most future-proof part of the design, but it also revealed missing pieces: a composition resolver, lockfiles, compatibility corpus, trust/permission model, virtual filesystem/root model, performance budgets, and a public-surface promotion protocol.

The final state of the chat was not "done." It reached a coherent architecture direction, but the latest project state said Foundation Lock was blocked by dependency-direction strict validation failures: 358 violations and 38 warnings. Therefore, the immediate next step is not more Workbench UI, renderer work, or gameplay expansion. It is FOUNDATION-REPAIR-DEPENDENCY-DIRECTION-01: classify and repair those dependency-direction violations, then rerun Foundation Closeout. Only after that should WORKBENCH-VALIDATION-SLICE-01 begin.

2. The Story of the Conversation

The conversation began with a practical question: whether Dominium would gain runtime performance or features by abandoning the older C89/C++98 split. Early answers distinguished compiler improvements from language-standard effects. A newer compiler can optimize old C and C++ code, but newer standards

allow the programmer to express ownership, atomics, alignment, move semantics, copy elision, and compile-time policies in ways that reduce implementation risk and can improve runtime behavior in targeted places.

The user then asked for separate C and C++ standard comparisons. That framed C99/C11/C17/C23 and C++98/C++11/C++17/C++20+ as capability choices, not just syntax choices. C99 was identified as a meaningful step up from C89 for C ergonomics and some optimization opportunities, but not sufficient for the overall Dominion architecture. C11/C17 provide useful modern C features but cannot be trusted as the cross-platform threading/atomic layer under MSVC. C++11 provides a major leap over C++98, but C++17 is the better current target if the project is already using modern toolchains and carefully restricting standard-library deployment assumptions.

The user proposed an early staged plan: v0/v1 in C89/C++98 and v2+ in C11/C++17. That was coherent for maximum retro reach, but later became obsolete when the project floor moved toward Windows 7 SP1, macOS 10.9.5, and Linux. The conversation then explored whether the engine and game could mix standards, such as a C11 engine with a C++98 game. The answer was yes in principle, as long as the ABI boundary was C-compatible and semantic assumptions did not leak across the boundary.

The discussion then corrected the conceptual split. The user asked what actually needs C++, whether everything should be converted to C89, whether everything should be C++98, and whether there is any reason to keep regular C. The answer that matters is that folders should not determine language.

Deterministic law, ABI, packets, saves, replays, stable IDs, renderer command IR, and low-level facades should be C-compatible. Resource ownership, high-level orchestration, providers, apps, Workbench, and tools should use C++ where it helps.

The user then asked for repo-specific advice based on `julesc013/dominium`. The assistant inspected repo files across the conversation. Earlier, the repo still carried C89/C++98 assumptions. Later, the live CMake state showed root and engine target settings had moved to C17/C++17. The repo also showed a broad `domino__engine` target that aggregates engine, game, runtime, renderer, platform, UI, network, packaging, ECS, scheduler, SIMD/GPU, and legacy C sources. That is useful as a convergence artifact but not the final modular target graph.

The user then shifted decisively: abandon the old plan and move to C17/C++17. That became the mainline recommendation. The plan was then expanded to support old and future platforms. The answer was that Windows 98, Mac OS 9.2, and similar targets should not

govern mainline code. They can consume projections, replay streams, snapshot clients, or generated subsets. Full source-native builds should be modern and eventually 64-bit.

The 32-bit question followed. The answer was to make full native products 64-bit and keep 32-bit as constrained or projection support only. The key caveat was not to make native pointer size part of game law. Save, replay, network, IDs, and renderer IR must use fixed-width types and never serialize `size_t`, `long`, `uintptr_t`, or raw pointers.

The user then consolidated a broad future plan around Domino, Dominion, Workbench, AIDE, contracts, tests, replay, and evidence. The answer agreed that the plan was strong, but identified missing central pieces: composition resolver, lockfiles, compatibility corpus, trust/permissions, virtual filesystem roots, performance budgets, and stable public-surface promotion rules.

Finally, the user pasted advice favoring C99 or C++11 due to raylib/SDL and legacy targets. The answer rejected a pivot. Raylib and SDL2 being C APIs only means provider boundaries should be C-callable; it does not force the whole engine or game to C99. The final recommendation remained C17 + C++17, with raylib/SDL2 treated as providers and with external deployment claims placed into the verification queue.

3. Main Topics Discussed

Topic 1 - Language baseline The chat repeatedly compared C89, C99, C11/C17, C23, C++98, C++11, C++17, and newer C++ standards. The final working direction is C17 + C++17 for the mainline. C99 and C++11 were considered but not adopted as the project-wide baseline. Newer C23/C++20/C++23/C++26 were treated as future provider/tool lanes, not current mainline law.

Topic 2 - C versus C++ responsibility split The major conclusion was not "engine C, game C++," but "stable law C-compatible, machinery C++17." C17 should own deterministic primitives, ABI, fixed-point math, save/replay/wire formats, renderer command IR, stable IDs, and low-level facades. C++17 should own game orchestration, runtime services, providers, apps, Workbench, resource ownership, and tools.

Topic 3 - ABI and data contracts The project must retain a C-compatible ABI. The public boundary should remain POD-only, versioned, return-code/refusal based, and free of exceptions, STL containers, classes, templates, allocator ownership, and C++ ABI assumptions. This allows providers, plugins, tools, projections, and future bindings to survive implementation changes.

Topic 4 - Determinism and parallelism The deterministic simulation law remains more important than the language standard. Authoritative simulation must use stable IDs, canonical ordering, fixed-width values, fixed-point math, explicit little-endian encoding, and deterministic scheduler phases. Threads may accelerate derived work, but final authoritative commit must be canonical and not depend on OS timing or thread completion.

Topic 5 - Platform and legacy support The mainline should be 64-bit source-native for x86_64 and arm64. Windows 7 SP1, macOS 10.9.5, and Linux were discussed as the practical active floor, subject to verification. Windows XP, Windows 98, Mac OS X 10.6, Mac OS 9.2, and similar systems should be constrained native, projection, or archive lanes rather than mainline constraints.

Topic 6 - Modularity, providers, packs, and modules The design moved toward contracts and composition. Apps compose modules; modules require services; services are implemented by providers; packs provide data/modules/content; profiles select packs; lockfiles make composition reproducible; Workbench operates through commands; AIDE governs and validates.

Topic 7 - Repo state and Foundation blocker The live repo was discussed as already having C17/C++17 CMake settings, but also retaining a broad domino__ engine target and old wording in docs/headers. The immediate blocker is Foundation Lock: dependency-direction strict validation reports 358 violations and 38 warnings, so Workbench/product work is not authorized until repair.

4. What We Were Trying to Achieve

4.1 Explicit Goals

- Determine whether Dominium should move beyond C89/C++98.
- Decide whether to use C, C++, or both, and which language versions.
- Support Windows 7, macOS Mavericks, Linux, and possibly older/future systems.
- Preserve modularity, performance, extensibility, distributed simulation, and long-term portability.
- Understand how code, data, modules, packs, providers, apps, Workbench, and AIDE integrate.
- Preserve this chat in a human-readable and aggregator-ready form.

4.2 Inferred Goals

- Avoid another cleanup cycle by locking hard-to-change decisions early.
- Prevent old retro ambitions from governing the modern mainline.
- Keep future assistants from repeating rejected advice such as pure C99 or pure C++11.
- Prepare material for a future Project Spec Book.

4.3 Goals That Changed Over Time The goal changed from "should old C/C++ standards be upgraded?" to "what is the full future architecture baseline?" The user initially entertained C89/C++98 staging, then accepted C17/C++17 as mainline. The user also moved from "maybe support old systems directly" to "support old systems through constrained/projection/archive lanes."

4.4 Goals Still Unresolved The exact platform floors, C++17 subset policy, composition resolver schema, lockfile taxonomy, trust model, performance budgets, and dependency-direction repair remain open.

5. Decisions Made and Why

- Row 1** ID: DECISION-01; Decision: Mainline should use C17 and C++17 rather than C89/C++98, pure C99, or pure C++11.; Status: accepted direction; Why it mattered: Keeps modern implementation leverage while staying conservative enough for Windows 7/Mavericks-era planning.; Confidence: 4; Label: FACT/INFERENCE
- Row 2** ID: DECISION-02; Decision: Full source-native products should be 64-bit-first/64-bit-only: x86_64 and arm64.; Status: accepted direction; Why it mattered: Reduces matrix complexity and matches modern desktop/mobile/console direction.; Confidence: 4; Label: FACT/INFERENCE
- Row 3** ID: DECISION-03; Decision: Keep little-endian mainline and explicit little-endian data formats.; Status: accepted direction; Why it mattered: Avoids large portability burden while retaining cross-platform determinism across chosen targets.; Confidence: 5; Label: FACT
- Row 4** ID: DECISION-04; Decision: Public boundary remains C-compatible, POD-only, versioned, and no C++ ABI leakage.; Status: accepted direction; Why it mattered: Enables C/C++/tools/projections/plugins without binding to compiler-specific C++ ABI.; Confidence: 5; Label: FACT

Row 5 ID: DECISION-05; Decision: C17 owns deterministic law and stable formats; C++17 owns machinery and orchestration.; Status: accepted direction; Why it mattered: Gives determinism and stable contracts while using C++ where it provides real value.; Confidence: 4; Label: FACT/INFERENCE

Row 6 ID: DECISION-06; Decision: Raylib and SDL2 should be providers/backends, not architectural identity.; Status: accepted direction; Why it mattered: Allows direct C API integration without forcing the whole engine/game to C99.; Confidence: 4; Label: INFERENCE

Row 7 ID: DECISION-07; Decision: Do not pursue a universal primitive binary for all systems.; Status: accepted direction; Why it mattered: Prevents false portability assumptions for Windows 98/Mac OS 9/consoles.; Confidence: 4; Label: FACT/INFERENCE

Row 8 ID: DECISION-08; Decision: Foundation Lock must not close while dependency-direction strict validation is red.; Status: accepted/current blocker; Why it mattered: Prevents product/UI work before dependency law is enforceable.; Confidence: 5; Label: FACT

Row 9 ID: DECISION-09; Decision: Workbench is not

authority; it is a command/evidence surface over contracts/services/providers.; Status: accepted direction; Why it mattered: Prevents Workbench from bypassing CLI/tests/AIDE and fragmenting behavior.; Confidence: 4; Label: FACT/
INFERENCE

Row 10 ID: DECISION-10; Decision: Composition resolver and lockfiles are missing central pieces.; Status: recommended, not yet implemented; Why it mattered: Needed to make modules/packs/providers/apps reproducible and extensible.; Confidence: 4; Label: INFERENCE

Decision explanations

- **DECISION-01:** The accepted direction is C17 + C++17 mainline. This superseded the old C89/C++98 plan and the later temptations to pivot to pure C99 or pure C++11. It depends on restricted deployment/library policy for legacy floors.
- **DECISION-02:** Full source-native builds should be 64-bit. This simplifies the release matrix and avoids old memory constraints, while fixed-width data keeps saves/replays portable.
- **DECISION-03:** Little-endian is accepted as a mainline invariant. Explicit little-endian

serialization remains necessary so that files are stable and auditable.

- **DECISION-04:** The public ABI remains C-compatible. This is the key decision that lets modern C++ implementation coexist with tools, providers, projections, and future bindings.
- **DECISION-05:** C and C++ are separated by semantic role, not by folder name.
- **DECISION-06:** Raylib and SDL2 can be useful providers but should not determine the whole architecture.
- **DECISION-07:** Universal primitive binaries were rejected; projections or generated subsets are the correct extreme-legacy strategy.
- **DECISION-08:** Foundation Lock remains blocked while dependency-direction strict validation is red.
- **DECISION-09:** Workbench must operate through the same command/evidence system as CLI/tests.
- **DECISION-10:** Composition resolver and lockfiles are recommended as missing pieces; they are not yet implemented.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

Row 1 ID: REJECTED-01; Option: Stay permanently on C89/C++98 as mainline.; Status: superseded; Reason: Good for old compiler reach, but too constraining once platform floor moved to Windows 7/Mavericks/Linux and repo already moved to C17/C++17.; Final or tentative?: mostly final for mainline; Reconsider conditions: May remain relevant for retro projection/archive lanes.; Related workstream: WORKSTREAM-01; Label: FACT/INFERENCE

Row 2 ID: REJECTED-02; Option: Pure C99 for entire engine and game.; Status: rejected; Reason: Suitable for small raylib/SDL games but poor fit for Dominion-scale providers, Workbench, resource ownership, composition, and modularity.; Final or tentative?: tentative but strong; Reconsider conditions: Could be reconsidered for tiny projection clients or isolated C providers.; Related workstream: WORKSTREAM-02; Label: INFERENCE

Row 3 ID: REJECTED-03; Option: Pure C++11 for entire engine and game.; Status: rejected; Reason: Viable but unnecessarily conservative; loses useful C++17 features and does not solve ABI/data-

contract issues.; Final or tentative?: tentative but strong; Reconsider conditions: Could be used for constrained native targets if toolchains require it.; Related workstream: WORKSTREAM-01; Label: INFERENCE

Row 4 ID: REJECTED-04; Option: Expose C++17 ABI/classes/STL across module boundaries.; Status: rejected; Reason: Compiler/runtime ABI instability and ownership/lifetime hazards.; Final or tentative?: final; Reconsider conditions: Never for stable public ABI; private in-process internals allowed.; Related workstream: WORKSTREAM-03; Label: FACT

Row 5 ID: REJECTED-05; Option: Universal primitive binary that runs on any OS/era.; Status: rejected; Reason: Native binaries depend on ISA, binary format, loader, imports, runtime, and OS APIs.; Final or tentative?: final; Reconsider conditions: Bytecode or generated projections could serve a different purpose.; Related workstream: WORKSTREAM-10; Label: FACT/INFERENCE

Row 6 ID: REJECTED-06; Option: Top-level platform folders such as windows/, mac/, android/.; Status: rejected; Reason: Violates ownership-based canonical root model and creates platform-owned source identity.; Final or tentative?: final unless

repo contract changes; Reconsider conditions: Platform profiles/adapters belong under runtime/contracts/cmake/release as appropriate.; Related workstream: WORKSTREAM-05; Label: FACT/INFERENCE

Row 7 ID: REJECTED-07; Option: Let Workbench call private tools directly.; Status: rejected; Reason: Would bypass command/evidence spine and create inconsistent behavior.; Final or tentative?: final; Reconsider conditions: Debug-only internal panels may need explicit experimental status.; Related workstream: WORKSTREAM-09; Label: INFERENCE

The most important rejected idea is pure C99. It is attractive for small raylib/SDL projects, but Dominion's scope includes deterministic simulation, distributed zones, provider replacement, Workbench, packs/modules, AIDE governance, and long-lived artifacts. A hand-written C object system would not actually be simpler at this scale. Pure C++11 was also rejected as unnecessarily conservative. Unrestricted C++17 across ABI/data boundaries was rejected even though C++17 remains the internal implementation standard.

7. Important Reasoning, Rationale, and Tradeoffs

The visible rationale repeatedly balanced three forces: modern implementation leverage, long-term compatibility, and deterministic auditability. C17/C++17 gives enough modern power without depending on the latest unstable library/runtime ecosystems. A C-compatible ABI prevents compiler-specific C++ details from becoming public law. Fixed-width, little-endian formats prevent native architecture details from corrupting saves, replays, packets, or renderer commands. Provider/capability contracts prevent a backend such as SDL2, raylib, D3D11, or Metal from becoming the identity of the product. Support tiers prevent old systems from distorting mainline architecture while still leaving room for projections or archive builds. The largest tradeoff is that this architecture needs more governance and proof than a simple game. The mitigation is to require vertical slices: one command, one provider, one pack/profile artifact, one diagnostic/refusal path, and one evidence packet before broad Workbench or plugin work.

8. Plans, Future Work, and Next Steps

The immediate plan is not more doctrine. The next active task is

FOUNDATION-REPAIR-DEPENDENCY-DIRECT

It should classify the 358 dependency-direction violations and 38 warnings, repair real boundary leaks, add only narrow justified exceptions, and rerun Foundation Closeout. After that, the next authorized product slice is **WORKBENCH-VALIDATION-SLICE-01**:

validate one pack/profile/content artifact through a registered command, show the same result in CLI and Workbench, and emit diagnostics/evidence.

Recommended sequence:

1. Repair dependency-direction blocker.
2. Close Foundation Lock.
3. Update stale C89/C++98 wording in ABI and determinism specs.
4. Add architecture/source-native profiles for 64-bit/little-endian policy.
5. Design the smallest composition resolver and lockfile slice.
6. Run the first Workbench validation slice.
7. Build compatibility corpus and performance budgets.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences

- Row 1** ID: PREF-01; Preference: User wants direct, audit-ready, source-grounded answers.; Area: communication; Explicit/inferred/uncertain: explicit; Strength: high; Practical implication: Use labels, citations, uncertainty, and concrete next actions.; Risk if wrong: User may distrust vague or overconfident answers.; Label: FACT
- Row 2** ID: PREF-02; Preference: User prefers human-readable narrative before machine-readable registers.; Area: handoff/reporting; Explicit/inferred/uncertain: explicit; Strength: high; Practical implication: Start preservation reports with prose explanation, then registers/spec sheets.; Risk if wrong: Machine-only output misses the stated goal.; Label: FACT
- Row 3** ID: PREF-03; Preference: User values preserving rejected/superseded options and contradictions.; Area: knowledge preservation; Explicit/inferred/uncertain: explicit; Strength: high; Practical implication: Do not flatten the history into only final decisions.; Risk if wrong: Future assistants may repeat rejected paths.; Label: FACT
- Row 4** ID: PREF-04; Preference: User wants not to re-explain everything when moving chats.; Area: workflow; Explicit/inferred/uncertain: explicit; Strength: high; Practical implication: Create

context transfer packets, aggregators, and bootstrap prompts.; Risk if wrong: Loss of continuity and repeated prompting.; Label: FACT

Row 5 ID: PREF-05; Preference: User wants strong modularity but not arbitrary folder proliferation.; Area: architecture; Explicit/inferred/uncertain: explicit/inferred; Strength: high; Practical implication: Use canonical roots and ownership/category names, not platform-specific top-level roots.; Risk if wrong: Repo rot or bad abstractions.; Label: FACT/INFERENCE

Row 6 ID: PREF-06; Preference: User is sensitive to assistant suggestions being mistaken for accepted decisions.; Area: epistemic; Explicit/inferred/uncertain: explicit; Strength: high; Practical implication: Label decisions vs recommendations vs inferences.; Risk if wrong: Incorrect spec book aggregation.; Label: FACT

9.2 Inferred Constraints and Preferences The user wants architectural clarity more than fashionable language choices. The user also wants future assistants to preserve tradeoffs, not collapse everything into an oversimplified final answer.

9.3 Uncertain or Unestablished Preferences Exact tolerance for exceptions/RTTI inside private C++17 code remains unsettled. Exact platform floors and raylib/SDL2 priority remain unresolved.

10. Files, Artifacts, Outputs, and Prompts

Row 1 ID: ARTIFACT-01; Artifact / file / prompt / output: Pasted text.txt preservation prompt; Type: uploaded prompt; Purpose: Defines this preservation task and required output package.; Status: available; Origin: User upload in current turn; Carry forward?: yes; Notes: This file is the direct task source for the preservation package.; Label: FACT

Row 2 ID: ARTIFACT-02; Artifact / file / prompt / output: CMakeLists.txt root C17/C++17 settings; Type: repo file; Purpose: Evidence that repo mainline currently sets C17 and C++17.; Status: referenced; Origin: GitHub repo; Carry forward?: yes; Notes: Cited in report; also indicates language migration already occurred.; Label: FACT

Row 3 ID: ARTIFACT-03; Artifact / file / prompt / output: engine/CMakeLists.txt broad domino__engine target; Type: repo file; Purpose: Evidence of current aggregation of engine/game/runtime/render/platform/UI/network sources.; Status:

referenced; Origin: GitHub repo; Carry forward?: yes; Notes: Important for target split and dependency repair.; Label: FACT

Row 4 ID: ARTIFACT-04; Artifact / file / prompt / output: contracts/repo/layout.contract.toml; Type: repo contract; Purpose: Defines canonical root model and strict unknown-root policy.; Status: referenced; Origin: GitHub repo; Carry forward?: yes; Notes: Central to avoiding bad folder structures.; Label: FACT

Row 5 ID: ARTIFACT-05; Artifact / file / prompt / output: engine/include/domino/abi.h; Type: repo header; Purpose: Current ABI helper design and stale C89/C++98 wording.; Status: referenced; Origin: GitHub repo; Carry forward?: yes; Notes: Needs terminology update but design should be preserved.; Label: FACT

Row 6 ID: ARTIFACT-06; Artifact / file / prompt / output: SPEC_DETERMINISM.md; Type: repo spec; Purpose: Defines deterministic law: stable IDs, fixed-point, no pointers/floats, little-endian.; Status: referenced; Origin: GitHub repo; Carry forward?: yes; Notes: Needs language wording update but remains canonical law.; Label: FACT

Row 7 ID: ARTIFACT-07; Artifact / file / prompt / output: SPEC_SIM_SCHEDULER.md; Type:

repo spec; Purpose: Defines tick/phase/budget/
ordering law and forbids OS time/thread timing as
truth.; Status: referenced; Origin: GitHub repo;
Carry forward?: yes; Notes: Core to distributed/
parallel simulation.; Label: FACT

Row 8 ID: ARTIFACT-08; Artifact / file / prompt /
output: component/support/platform matrices;
Type: repo docs/contracts; Purpose: Define
backend status vocabulary, support tiers, and
evidence requirements.; Status: referenced; Origin:
GitHub repo; Carry forward?: yes; Notes: Need
expansion for architecture/source-native tiers.;
Label: FACT

Row 9 ID: ARTIFACT-09; Artifact / file / prompt /
output: latest task/review packets; Type: AIDE
context files; Purpose: Record Foundation Lock
blocked by dependency-direction strict failures.;
Status: referenced; Origin: GitHub repo; Carry
forward?: yes; Notes: Current immediate blocker
and next action.; Label: FACT

Row 10 ID: ARTIFACT-10; Artifact / file / prompt /
output: Current preservation package files; Type:
generated files; Purpose: Human-readable report,
registers, context packet, spec sheet, aggregator
packet, reader brief, audit, bootstrap prompt, in-
chat reader, ZIP.; Status: created now; Origin:

This response; Carry forward?: yes; Notes: Use these to continue or aggregate this chat.; Label: FACT

11. Open Questions and Unresolved Issues

Row 1 ID: QUESTION-01; Question / issue: What exact platform floors should be formalized: Windows 7 SP1, macOS 10.9.5, which Linux baseline, Android/iOS min versions?; Why it matters: Support floors determine toolchain, standard library, SDK, packaging, and testing.; Known information: Windows 7/Mavericks/Linux were discussed; mobile/consoles future.; Unknown information: Exact versions and support tiers remain to be locked and verified.; Resolution path: Create platform profile contracts after verifying official/toolchain facts.; Priority: P1; Related workstream: WORKSTREAM-10; Label: UNCERTAIN

Row 2 ID: QUESTION-02; Question / issue: How should the 358 dependency-direction violations be classified?; Why it matters: Foundation Lock is blocked until repaired.; Known information: Counts and blocker status known.; Unknown information: Specific files/causes not in this preservation prompt.; Resolution path: Run validator, classify real boundary leaks vs justified

exceptions.; Priority: P0; Related workstream:
WORKSTREAM-05; Label: FACT/UNCERTAIN

Row 3 ID: QUESTION-03; Question / issue: What is the first minimal composition resolver slice?; Why it matters: Resolver is central but can be overbuilt. ; Known information: Need resolver for packs/modules/providers/apps.; Unknown information: Smallest useful schema and runtime path not yet specified.; Resolution path: Start with one pack/profile validation command and one provider.; Priority: P1; Related workstream: WORKSTREAM-07; Label: INFERENCE

Row 4 ID: QUESTION-04; Question / issue: How permissive should C++17 be internally?; Why it matters: Too strict wastes C++; too loose risks ABI/determinism leaks.; Known information: Broad policy exists; restricted subset discussed.; Unknown information: Exact exceptions/RTTI/stdlib policy per target not finalized.; Resolution path: Draft C++17 subset policy by layer.; Priority: P1; Related workstream: WORKSTREAM-01; Label: UNCERTAIN

Row 5 ID: QUESTION-05; Question / issue: Should raylib be a serious provider, a prototyping provider, or excluded in favor of SDL2/native backends?; Why it matters: It affects render/

platform/audio/input provider shape.; Known information: Raylib/SDL2 treated as providers, not identity.; Unknown information: Priority and long-term role of raylib are unsettled.; Resolution path: Research support matrix and decide provider tier.; Priority: P2; Related workstream: WORKSTREAM-06; Label: UNCERTAIN

Row 6 ID: QUESTION-06; Question / issue: What exact trust/permission model should native modules, scripts, packs, and Workbench tools use?; Why it matters: Unsafe extensibility can compromise users and saves.; Known information: Need trust levels and permissions.; Unknown information: Concrete policy/schema absent.; Resolution path: Draft trust contract before native plugin implementation.; Priority: P1; Related workstream: WORKSTREAM-08; Label: INFERENCE

Row 7 ID: QUESTION-07; Question / issue: What are the first performance budgets?; Why it matters: Optimization needs measurable gates.; Known information: Tick/replay/render/load/allocation budgets proposed.; Unknown information: Specific thresholds absent.; Resolution path: Add baseline benchmarks, then set budget thresholds from measurements.; Priority: P1; Related workstream: WORKSTREAM-11; Label: INFERENCE

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Row 1 ID: RISK-01; Risk / failure mode: Dependency-direction blocker is ignored while product work continues.; Consequence: Foundation rot and false green status.; Likelihood: medium; Severity: high; Mitigation: Repair validator failures before Workbench/product slices.; Related workstream: WORKSTREAM-05; Label: FACT

Row 2 ID: RISK-02; Risk / failure mode: Assistant/future spec treats tentative recommendations as accepted user decisions.; Consequence: Incorrect master spec and repeated bad advice.; Likelihood: medium; Severity: high; Mitigation: Use decision status labels and cite direct user acceptance.; Related workstream: all; Label: FACT

Row 3 ID: RISK-03; Risk / failure mode: C++17 leaks into ABI/data formats.; Consequence: Compiler lock-in, plugin breakage, save/replay incompatibility.; Likelihood: medium; Severity: high; Mitigation: C-compatible ABI validators and public header checks.; Related workstream: WORKSTREAM-03; Label: FACT

Row 4 ID: RISK-04; Risk / failure mode: Determinism breaks through thread timing, unordered iteration, pointer identity, or floats.; Consequence: Replay/

lockstep failures.; Likelihood: medium; Severity: critical; Mitigation: Keep scheduler/determinism validators and golden replay hashes.; Related workstream: WORKSTREAM-04; Label: FACT

Row 5 ID: RISK-05; Risk / failure mode: False support claims for Windows 7/Mavericks/Linux/legacy/mobile/consoles.; Consequence: Users/builds fail; credibility loss.; Likelihood: medium; Severity: high; Mitigation: Support claims require toolchain/preset/smoke/package/release evidence.; Related workstream: WORKSTREAM-10; Label: FACT

Row 6 ID: RISK-06; Risk / failure mode: Over-architecture delays useful vertical slices.; Consequence: Lots of contracts but no working product proof.; Likelihood: medium; Severity: medium; Mitigation: Use one pack/profile validation slice after Foundation green.; Related workstream: WORKSTREAM-09; Label: INFERENCE

Row 7 ID: RISK-07; Risk / failure mode: Raylib/SDL provider choice becomes product identity.; Consequence: Backend lock-in and architecture distortion.; Likelihood: low; Severity: medium; Mitigation: Keep backends behind provider facades.; Related workstream: WORKSTREAM-06; Label: INFERENCE

Row 8 ID: RISK-08; Risk / failure mode: Generated files or stale context are mistaken for canonical source.; Consequence: Aggregation and repo decisions use wrong authority.; Likelihood: medium; Severity: medium; Mitigation: Respect source hierarchy and label PROJECT-CONTEXT/UNCERTAIN.; Related workstream: all; Label: FACT

The biggest future-chat failure would be to answer the language question as if this were a small raylib project. It is not. Another failure would be to treat the final architecture as permission to build every layer immediately. The current red gate is still dependency direction.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat should feed the Project Spec Book chapters on language baseline, ABI/data contracts, deterministic simulation, platform support tiers, provider/capability architecture, modular composition, Workbench/AIDE operating model, and release/support evidence. It should not be merged as a final implementation spec until the dependency-direction repair and platform/toolchain verification are done.

14. What I Should Remember

- The mainline is C17 + C++17, but stable boundaries remain C-compatible.
- 64-bit source-native and little-endian mainline are the intended architecture assumptions.
- C is for law; C++ is for machinery.
- Raylib/SDL2 are providers, not the product identity.
- Workbench is not authority; contracts/commands/evidence are authority.
- Extreme legacy support uses constrained builds or projections, not one universal binary.
- The immediate blocker is dependency-direction strict validation.
- The missing architectural center is composition resolver + lockfiles.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- Explain the difference between "C for law" and "C++ for machinery" using Dominion examples.

- Which parts of the old C89/C++98 plan are fully superseded and which survive as projection/archive ideas?

15.2 Decisions

- Which decisions in this chat are user-accepted versus assistant recommendations?
- What would cause us to revisit the C17/C++17 decision?

15.3 Tasks and Next Actions

- Draft the task packet for FOUNDATION-REPAIR-DEPENDENCY-DIRECTION-01.
- Draft the acceptance criteria for WORKBENCH-VALIDATION-SLICE-01.

15.4 Artifacts and Files

- Which repo docs need wording updates after the C17/C++17 decision?
- What should go into the architecture profile contract?

15.5 Risks and Verification

- Verify the Windows 7 and macOS 10.9 toolchain/library assumptions using official sources.
- List ways C++17 could accidentally break deterministic replay.

15.6 Future Spec Book / Aggregation

- Convert this chat's decisions into formal spec-book requirements.
- Merge this chat with other Dominium platform/renderer chats and identify conflicts.

15.7 Deep-Dive Questions Specific to This Chat

- Design the composition resolver's first minimal schema.
- Define trust levels for packs, modules, scripts, and native providers.
- Propose a CMake target split from the current broad domino_engine target.

16. Compact Human Summary

This chat is the point where Dominium's mainline architecture moved from the old C89 engine + C++98

game idea into a modern but still conservative C17 + C++17 baseline. The user repeatedly tested alternative language strategies: old C/C++, pure C99, pure C11/C17, pure C++11, pure C++17, and newer C++ standards. The final position is not a language ideology. It is a layered architecture: C-compatible stable law at the boundaries and C++17 implementation machinery inside.

The key rule is that C should own deterministic law and stable formats: ABI structs, fixed-width types, stable IDs, packets, saves, replays, fixed-point math, renderer command IR, capability/refusal structs, and provider facades. C++17 should own game orchestration, domain systems, runtime services, providers, renderer/platform backends, apps, tools, Workbench, jobs, resource ownership, and composition. This lets Dominion benefit from RAI, move semantics, variants/optionals internally, templates, constexpr, and safer orchestration, without exporting C++ ABI or native object layouts into long-term contracts.

The chat also established a 64-bit source-native direction. Full native products should target x86_64 and arm64. 32-bit is not deleted from imagination, but it is downgraded to constrained-native or projection support. Old systems such as Windows 98 and Mac OS 9 should consume replay viewers, snapshots, thin clients, or generated subsets rather than forcing the mainline to

remain old-standard or 32-bit. The project should also remain little-endian mainline and use explicit little-endian serialization for stable data.

The repo context matters. The current CMake state discussed in this chat already reflects C17 and C++17, but the repo still has stale C89/C++98 wording in some docs/headers and a broad `domino_engine` target that aggregates many ownership domains. That target should eventually be split into layered targets once dependency-direction repairs are complete. The layout contract is already directionally right: keep canonical roots and avoid top-level platform folders or arbitrary module roots.

The user's broader future plan around Domino, Dominion, Workbench, AIDE, contracts, tests, replay, and evidence was mostly accepted. The missing pieces identified were composition resolver, lockfiles, compatibility corpus, trust/permission model, virtual roots, performance budgets, and public-surface promotion protocol. These are necessary to make code, data, modules, packs, providers, apps, Workbench, and AIDE actually compose rather than becoming manually wired subsystems.

The immediate next action is not more architecture brainstorming. The latest project task/review packets say Foundation Lock is blocked because dependency-direction strict validation reports 358 violations and 38

warnings. The next task is FOUNDATION-REPAIR-DEPENDENCY-DIRECTION-01. Only after that should Workbench validation or product-facing module work proceed.

11.2 Dominium Language Platform Architecture Baseline Reader Brief

Source: docs/archive/conversations/Language_Platform_Architecture/Dominium_Language_Platform_Architecture_Baseline__06_reader_brief.md

Report type: reader_brief

Size: 1758 bytes

11.2.1 Reader Brief - Dominium Language, Platform, and Architecture Baseline

What this chat was about

This chat decided Dominium's modern language/platform architecture. The key outcome is C17 + C++17, 64-bit, little-endian, C-compatible ABI, deterministic fixed-width data, and provider/capability modularity.

Top 20 things to know

1. C17 + C++17 is the mainline direction.
2. C89/C++98 is superseded for mainline, preserved only for retro/projection/archive thinking.
3. Pure C99 was rejected as too weak for Dominium-scale modularity.
4. Pure C++11 was rejected as unnecessarily conservative.
5. C17 owns stable law and ABI-adjacent substrate.
6. C++17 owns machinery, providers, apps, Workbench, and tools.
7. Public boundaries remain C-compatible, POD-only, versioned, and no-exception.
8. No STL/classes/templates/RTTI/exceptions cross stable ABI.
9. Full source-native builds should be 64-bit: x86_64/arm64.
10. 32-bit is constrained/projection/archive only.
11. Little-endian mainline is accepted.
12. Save/replay/network/renderer IR cannot use pointer-sized/native-layout fields.

13. Raylib and SDL2 are providers, not identity.
14. Workbench is not authority; it must use commands/services/results/refusals/evidence.
15. Apps compose modules; modules require services; services use providers.
16. Packs provide data/modules/content.
17. Lockfiles and a composition resolver are missing central pieces.
18. Compatibility corpus and trust/permission model are missing.
19. Foundation Lock is currently blocked by dependency-direction strict failures.
20. The next action is dependency-direction repair, not product feature work.

Best next step

Run and repair FOUNDATION-REPAIR-DEPENDENCY-DIRECTION-01, then rerun Foundation Closeout.

11.3 Dominion Language Platform Architecture Baseline In Chat Reader

Source: docs/archive/conversations/Language_Platform_Architecture/Dominium_Language_Platform_Architecture_Baseline__09_in_chat_reader.md

Report type: in_chat_reader

Size: 1673 bytes

11.3.1 In-Chat Reader - Dominion Language, Platform, and Architecture Baseline

Package overview

This package preserves the chat where Dominion's mainline architecture settled around C17 + C++17, 64-bit, little-endian, C-compatible ABI, deterministic law, provider/capability modularity, Workbench/AIDE governance, and support-tier/projection strategy.

File index

- 00_manifest.md: package inventory.
- 01_human_readable_report.md: full narrative report sections 0-16.

- `02_context_transfer_packet.md`: paste into a future chat.
- `03_spec_sheet.yaml`: machine-friendly aggregation sheet.
- `04_registers.md`: all structured registers.
- `05_aggregator_packet.md`: central aggregator packet.
- `06_reader_brief.md`: shorter human brief.
- `07_verification_and_audit.md`: self-audit and verification queue.
- `08_future_chat_bootstrap_prompt.md`: future chat starter prompt.
- `09_in_chat_reader.md`: this guide.

Question menu

Ask next:

- What exactly does C17 own versus C++17 in Dominium?
- Draft FOUNDATION-REPAIR-DEPENDENCY-DIRECTION-01.
- Draft the architecture profile contract for 64-bit/little-endian/source-native tiers.

- Design the first composition resolver schema.
- Define the trust model for packs/modules/native providers.
- Verify Windows 7 and macOS 10.9 support assumptions.
- Propose a CMake target split for `domino__engine`.

Top things to preserve

C17 + C++17 mainline; C-compatible ABI;
deterministic fixed-width law; 64-bit source-native; little-
endian; providers not identity; Workbench not authority;
Foundation blocker first.

Safest next action

Run dependency-direction repair before product or
Workbench feature work.

Chapter 12

Launcher Setup Architecture

12.1 Dominium Launcher Setup Architecture Reader Brief

Source: docs/archive/conversations/Launcher_Setup_Architecture/Dominium_Launcher_Setup_Architecture__05_reader_brief.md

Report type: reader_brief

Size: 4384 bytes

12.1.1 Reader Brief - Dominium Launcher Setup Architecture

What This Chat Was About

This chat designed the Dominium launcher, setup system, runtime-launcher contract, and related modular architecture. It began from the user's broad Dominium philosophy: deterministic C89 engine core, stable APIs, versioned/TLV file formats, data-first content/mods, optional plugins, and migration rather than silent breakage. The conversation then narrowed into how the launcher should behave for users: install the game, run

the launcher, create profiles, choose versions/mods/settings, and launch one or more client/server/tool instances.

The launcher became an optional all-in-one Dominionium control center, not a hard dependency. The game must still run directly without launcher involvement. The setup system became a separate canonical install/repair/uninstall authority. The user then fixed the built-in launcher tabs as News, Changes, Mods, Instances, and Settings, and required these tabs to be fully interactive.

The final major direction change was the user's requirement that the launcher be designed and implemented using the Domino platform/rendering layers: **dsys** for platform services and **dgfx** for rendering, with strict C89 APIs and retro/modern support. This latest dsys/dgfx direction supersedes earlier C++98/ncurses/SDL/ImGui launcher frontend prompts.

Most Important Things to Know

- Latest active launcher implementation direction: C89 + dsys + dgfx.
- Game/runtime must run standalone with no launcher.
- Launcher supervision is optional and disableable.
- Instance display modes are NONE, CLI, TUI, GUI.

- Setup, launcher, and game should remain separate binaries unless user revises this.
- Setup must support portable, per-user, and system-wide installs.
- Setup must support defaults and full customization.
- Launcher tabs are News, Changes, Mods, Instances, Settings.
- Tabs must be fully interactive.
- Mods tab is a full mod manager.
- Instances tab manages instances and installations.
- Settings tab must be intuitive and powerful.
- Launcher/setup/instances should be modular and extensible.
- Plugins should be ABI-versioned.
- Earlier JSON schemas conflict with later TLV/dmeta direction.
- Actual repo, build system, dsys/dgfx APIs were not inspected.

Active Plans or Workstreams

- WORKSTREAM-01: Global Dominium architecture and repo strategy.
- WORKSTREAM-03: Runtime/game executable integration.
- WORKSTREAM-04: Setup system.
- WORKSTREAM-05: Launcher all-in-one hub.
- WORKSTREAM-06: Launcher tabs and interaction model.
- WORKSTREAM-07: Plugin/extensibility.
- WORKSTREAM-08: dsys/dgfx C89 launcher implementation.

Decisions Already Made

- Use one codebase / monorepo.
- Prefer separate setup, launcher, and game binaries.
- Engine remains deterministic C89.
- Game runs without launcher.
- Display modes: NONE, CLI, TUI, GUI.
- Setup is canonical install/repair/uninstall authority.

- Tabs are News, Changes, Mods, Instances, Settings.
- Latest launcher uses dsys and dgfx.

Pending Tasks

- Inspect repo.
- Resolve JSON vs TLV/dmeta.
- Verify dsys/dgfx API names.
- Generate dsys/dgfx Work-order 1.
- Implement launcher data layer.
- Implement UI widgets/layout/events.
- Implement dgfx renderer.
- Implement process supervision.
- Implement tabs.
- Implement harness.
- Verify runtime capabilities.

Open Questions

- JSON or TLV/dmeta?
- Is setup also C89/dsys?
- What are real dsys/dgfx API names?

- What is actual runtime binary structure?
- How much retro support is first-class now?
- Does dsys provide dynamic plugin loading?
- How to handle News/Changes open/browse on retro platforms?

Files / Artifacts / Prompts to Preserve

- Global architecture philosophy.
- Runtime CLI contract.
- Setup system prompt/spec.
- Launcher tabs spec.
- dsys/dgfx launcher architecture.
- C89 header sketches.
- Codex work-order sequence.
- Context Transfer Packet.
- This report package.

What to Verify Before Acting

- Repo layout/build system.
- dsys/dgfx APIs.

- TLV/metadata implementation.
- Runtime capabilities.
- Setup language/system-layer requirements.
- Plugin loading APIs.

Best Next Step

Generate dsys/dgfx Work-order 1: core launcher config/install/profile/mods data layer in C89 using dsys filesystem and versioned metadata, after resolving or explicitly flagging the JSON-vs-TLV/dmeta format question.

Chapter 13

Modularity Aide Refactorability

13.1 Dominion Modularity Aide
 Refactorability Architecture
 Human Readable Report

Source: docs/archive/conversations/Modularity_ AIDE_ Refactorability/ Dominion_ Modularity_ AIDE_ Refactorability_ Architecture_ _01_ human_ readable_ report. md

Report type: human_readable_report

Size: 45544 bytes

13.1.1 COMPLETE CHAT PRESERVATION
 REPORT - Dominion Modularity,
 AIDE Refactorability, and
 Future-Proof Architecture

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion
Modularity, AIDE Refactorability, and Future-
Proof Architecture

- Row 2** Field: Date anchor; Assessment: 2026-05-27
Australia/Melbourne
- Row 3** Field: Source scope; Assessment: This chat only,
except explicitly labelled PROJECT-CONTEXT
- Row 4** Field: Apparent access; Assessment: Partial-to-
good: visible current chat plus uploaded
preservation prompt; not the full live repo or all
prior generated files
- Row 5** Field: Previously generated files available?;
Assessment: Unclear / mostly no: prior files are
referenced, but not available as downloadable
artifacts in this chat
- Row 6** Field: Uploaded files or artifacts present?;
Assessment: Yes: `Pasted text.txt`, containing
the preservation-package output contract
- Row 7** Field: Contains future plans?; Assessment: Yes
- Row 8** Field: Contains decisions?; Assessment: Yes, but
many are assistant recommendations unless clearly
adopted by user language
- Row 9** Field: Contains pending tasks?; Assessment: Yes
- Row 10** Field: Contains unresolved questions?;
Assessment: Yes
- Row 11** Field: Staleness risk; Assessment: Medium:
repo status, commit IDs, live tooling status, and

external practice references need verification before implementation

Row 12 Field: Extraction confidence; Assessment: 4/5 for visible-chat substance; 2/5 for live repo facts and older unavailable artifacts

Row 13 Field: Safe for later aggregation?; Assessment: With caveats

Row 14 Field: Main limitations; Assessment: No live repo inspection in this turn; no full old-chat transcript beyond visible current conversation; prior generated files and older uploads may be unavailable/expired; several repo-status claims were pasted from earlier assistant outputs and are not independently verified here.

This package is based on the visible current chat and the uploaded `Pasted text.txt` prompt. It does not include live repository inspection, execution of validators, or complete access to older files generated in other chats. Some older uploaded files appear unavailable or expired in the broader chat environment. Where this report mentions repo state, CTest failures, specific commit IDs, or current file paths from pasted prior assistant outputs, those are treated as **UNCERTAIN / UNVERIFIED** unless independently verified later. The package is safe for aggregation if future readers preserve these labels

and do not treat every assistant recommendation as an adopted user decision.

1. One-Page Orientation

This chat was about making Dominion structurally durable: portable, modular, extensible, reusable across future games, and refactorable without the user having to repeat a painful top-level cleanup. The discussion began from an already-developed architectural thread about Dominion's source layout, distribution layout, package layout, install roots, runtime roots, bundle formats, and component matrices. The user brought in several previous assistant outputs that had converged on a broad direction: Dominion should not be treated like a one-off indie project, but like a long-lived platform with a stable architecture, stable contracts, deterministic build/release/install semantics, and a repo structure that can survive multiple products, renderers, platforms, shells, and content systems.

The key early idea was that Dominion needs more than a source repo directory layout. It needs a unified logical-root model that can be projected into CI output, `.dompkg` packages, portable installs, installed desktop/server layouts, read-only media, caches, staging roots, rollback state, symbols, provenance, save bundles, instance bundles, replay bundles, and diagnostics bundles. This led to the principle that `.zip` archives,

installers, portable folders, package files, media images, save bundles, and runtime installs should not invent separate semantics. They should all bind to the same logical roots, with physical paths depending on mode and mutability.

The next topic was the source repo itself. The user and assistant treated the existing root clutter as a serious architectural problem, not cosmetic untidiness. The target root set was repeatedly narrowed toward stable ownership categories: `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `docs/`, `tests/`, `tools/`, `scripts/`, `cmake/`, `external/`, `release/`, `archive/`, and generated `dist/` under strict policy. The important conclusion was that future top-level roots should be refused by default. New work should fit into existing ownership surfaces unless it truly has a separate lifecycle and long-term purpose.

A major correction happened when the user rejected the old XStack/AuditX/RepoX/TestX framing. The user wanted to transition to AIDE as soon as possible and recycle existing code/docs/tests rather than ignore them. The answer shifted from "make old checks green first, then clean" to "install AIDE as the restructuring control plane now, wrap and salvage old tools, and then drive cleanup through AIDE work units." This became one of the central outcomes of the chat: AIDE should own inventory, classification, move maps, salvage maps,

reference rewriting, validators, evidence ledgers, path aliases, shims, and refactor history.

The user then broadened the goal. The concern was not only Dominion's current folder tree, but whether the codebase could become a proper reusable game/engine platform. The final answer introduced a product-line architecture view. Code should be split by reuse level: reusable across Dominion products, reusable across future games on the same domino/Dominion engine, and reusable across unrelated engine or game projects. That means contracts, runtime infrastructure, tooling, deterministic base engine pieces, content-addressed storage, diagnostics, packaging, schemas, tests, and AIDE machinery should be reusable beyond Dominion where possible, while `game/` should hold Dominion-specific truth.

The most important stable principles were: paths are not identity; contracts and manifests define identity; apps are thin; runtime owns host adaptation; engine owns deterministic substrate; game owns Dominion-specific rules; durable APIs need stability levels; C89-compatible ABI seams should be used for stable boundaries without forcing every internal implementation to be C89; schemas and protocols must be versioned; capability negotiation should replace platform-era assumptions; generated outputs must be quarantined; and every future refactor should be

mechanical, evidence-backed, and reversible.

This chat contributes a high-level architecture constitution and refactorability doctrine for Dominion. It should feed directly into a future Project Spec Book section on repository architecture, reusable engine architecture, runtime/platform modularity, packaging/install semantics, AIDE governance, API/ABI stability, schemas/protocols, deterministic data, testing, and compatibility policy. Its main limitation is that several concrete repo-status claims came from pasted earlier outputs and were not verified against the live repository in this turn.

2. The Story of the Conversation

The conversation opened with the user pasting a prior analysis about Dominion's distribution and install layouts. That analysis argued that the source repository is only one layer. Dominion also needs canonical layouts for release build output, compressed packages, portable installs, installed layouts, instances and saves, bundles, diagnostics, caches, staging, rollback, symbols, provenance, and media. The core idea was that all these physical layouts should be projections of one logical virtual-root contract. The user then pasted another prior analysis about root convergence. That analysis treated the live repository as cluttered, with many roots mixing ownership levels, and proposed a staged convergence

program around a stable source layout, distribution layout, runtime/install layout, and mechanical enforcement.

The next pasted section focused on component naming. It argued that Dominion should support many platforms, renderers, shells, package types, and presentation qualities, but should avoid names like `compat`, `universal`, `legacy`, `modern`, `gl2`, `vk1`, `dx11`, `winui3`, and `portable_zip` as primary component IDs. The better pattern was to name the component by what it is, then put version, support status, host age, package format, and constraints into matrix fields. This led to a clean modular architecture model: fixed contracts, runtime components, product entryptoints, and domain/content components.

The user asked whether this was the best possible direction. The response accepted the broad direction but strengthened it: the goal should be a contract-driven projection system, not just a directory plan. The answer added machine-readable contracts, a projection model, split host/platform axes, evidence levels in matrix rows, refusal codes, generated artifact policy, thin-app enforcement, a truth pipeline, capability negotiation, a minimum viable proof path, and migration safety rules.

The user then pasted another prior analysis about remaining root clutter and CTest/AuditX/RepoX blockers. That analysis recommended not doing visual

folder cleanup and instead stabilizing failing checks before root exception cleanup. The user objected to the XStack/AuditX/RepoX/TestX framing and said they wanted to transition to AIDE as soon as possible while recycling existing code and docs. This changed the direction of the plan. The response agreed that the old names were temporary scaffolding and should not become durable architecture. It proposed AIDE as the control plane, with old tools inventoried, wrapped, adapted, renamed, archived, or retired through evidence-backed work units.

The user then asked whether this was truly the best plan and emphasized not wanting to go through the restructuring headache again. The answer refined the objective: the aim is not a perfect structure that never changes, but a stable top-level constitution plus a refactor operating system. Future internal movement should be cheap because it is scripted, validated, reversible, and driven by AIDE move maps and salvage maps. This produced the idea of **AIDE-STRUCTURE-00**, a repo constitution and refactorability framework.

Finally, the user made the broadest request: Dominion code must be portable, modular, extensible, reusable for another game on the same engine, and potentially reusable for different engine or game projects. The user wanted to know what was missing, what practices apply generally and specifically, whether the directory

structure and names were the best they could be, what major engineering organizations do, and how to maximize future-proofing and backwards compatibility. The response reframed Dominion as a product-line architecture rather than a single game. It proposed explicit reuse levels, stricter internal structure for `contracts/`, `engine/`, `game/`, `runtime/`, `apps/`, `content/`, and `release/`, and detailed practices for ABI design, API stability, schemas, protocols, naming, target-based builds, dependency visibility, component manifests, data design, tests, compatibility, AIDE-driven refactors, portability threats, determinism, memory, errors, logging, concurrency, serialization, and host abstraction.

The final user action was uploading a preservation-package prompt. That prompt requested a maximum-fidelity preservation report, structured registers, context transfer packet, spec sheet, aggregator packet, self-audit, downloadable files, and a final in-chat reader. This package is the result.

3. Main Topics Discussed

Topic 1 - Unified layout projection system The chat repeatedly distinguished source layout from distribution/build output, packages, portable installs, installed layouts, media, caches, rollback roots, symbols, bundles, saves, and diagnostics. The conclusion was that

these should not be separate semantic inventions. They should be physical projections of a shared virtual-root contract. This matters because `.zip`, `.dompkg`, `installer`, `media image`, and `runtime folder` semantics can otherwise drift apart. The uncertainty is that the exact contract file names and root names still need implementation against the live repo.

Topic 2 - Stable source repo constitution The target top-level layout stabilized around `.aide/`, `.github/`, `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `docs/`, `tests/`, `tools/`, `scripts/`, `cmake/`, `external/`, `release/`, `archive/`, and generated `dist/`. The purpose is to make root ownership long-lived and prevent future root sprawl. The most important rule is that new top-level roots are refused by default unless they pass a strict lifecycle and ownership test.

Topic 3 - AIDE as restructuring control plane The user objected to the `XStack/AuditX/RepoX/TestX` framing and wanted AIDE adopted quickly. The resulting plan made AIDE the repo-native control plane for inventory, task queues, policies, move maps, salvage maps, evidence ledgers, validation, and refactor history. Existing tools should be recycled, not discarded. This is central to future refactorability.

Topic 4 - Product-line architecture and reuse levels The final discussion broadened Dominion from a single game to a reusable platform. Code should be separated by whether it is reusable across Dominion products, reusable across games on the same domino/ Dominion engine, or reusable across unrelated engine/ game projects. This changes where code belongs: reusable infrastructure should live in `contracts/`, `engine/base/`, `runtime/`, or `tools/`, while Dominion-specific rules belong in `game/`.

Topic 5 - Contracts, API, ABI, schemas, and protocols The chat emphasized that durable identity and compatibility should live in contracts and manifests, not paths. Public seams need stability levels. C89-compatible ABI headers are valuable for stable boundaries, using opaque handles, explicit allocators, versioned structs, and stable error codes. Schemas and protocols need versioning, negotiation, compatibility ranges, refusal codes, and migration policy.

Topic 6 - Naming and component matrices The chat rejected names based on era, temporary project status, support status, or implementation-version encoding. Components should have boring literal names such as `opengl`, `direct3d`, `vulkan`, `software`, `win32`, `posix`, `cocoa`, `portable`, `installed`, with version/

status/format fields carried separately. This reduces long-term naming debt.

Topic 7 - Build, dependency, and visibility discipline The discussion proposed target-based CMake and module boundaries rather than path mythology. Public headers, private headers, allowed dependencies, and forbidden dependencies should be explicit. Apps must remain thin. Engine must not depend on game/apps/runtime UI. Runtime adapts host/platform/rendering without owning simulation truth.

Topic 8 - Determinism, data, compatibility, and testing The chat stressed deterministic serialization, named RNGs, no wall-clock dependency in simulation, no filesystem ordering dependence, schema round-trip tests, replay tests, package verification tests, migration tests, renderer parity tests, and hermetic tests. Compatibility should be promised only for selected public seams, not every internal implementation.

Topic 9 - Preservation and aggregation The uploaded prompt converted the conversation into a preservation task. It requires a human-readable report first, then registers, spec sheet, aggregator packet, self-audit, and downloadable files. This final package is

intended to be merged later with old-chat reports into a master Project Spec Book.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted Dominion's code and architecture to be portable, modular, extensible, reusable for other games on the same engine, and reusable where possible for different engine or game projects. The user also explicitly wanted to avoid repeating a painful restructuring process and wanted future full refactors to be as quick and easy as possible. The user wanted AIDE adopted quickly as the restructuring control plane and wanted existing code/docs/tests recycled rather than ignored.

4.2 Inferred Goals The conversation implies a goal of turning Dominion into a long-lived product-line platform rather than a single-game repository. It also implies a desire for auditability: decisions should be grounded, visible, mechanically enforceable, and later aggregatable into a master spec. The user appears to value stable boundaries, explicit compatibility policy, and practical Codex/AIDE tasks.

4.3 Goals That Changed Over Time The initial focus was directory and distribution structure. It then widened to component naming, repository convergence,

AIDE governance, refactorability, and finally broad software engineering doctrine for reusable engine/platform development. The most important change was the rejection of the XStack-style framing and the shift toward AIDE as the near-term control plane.

4.4 Goals Still Unresolved The live repo still needs verification. The exact AIDE files, contracts, validators, and migration tasks must be implemented. It remains unresolved which existing roots and tools map to which final homes, which APIs become stable, which code is reusable across products/games/projects, and which prior assistant recommendations should become formal requirements after user review.

5. Decisions Made and Why

- Row 1** ID: DECISION-01; Decision: Use a stable top-level root constitution rather than ad hoc root growth.; Status: accepted direction; Why it mattered / basis: Repeated user concern about avoiding future restructure; assistant proposals align.; Confidence: WORKSTREAM-01; Label: high
- Row 2** ID: DECISION-02; Decision: Treat AIDE as the restructuring control plane.; Status: accepted by user framing; Why it mattered / basis: User

explicitly said to transition to AIDE ASAP and recycle existing material.; Confidence: WORKSTREAM-03; Label: high

Row 3 ID: DECISION-03; Decision: Recycle old tools/docs/tests rather than ignore or discard them.; Status: accepted direction; Why it mattered / basis: User explicitly said there is code and docs to recycle.; Confidence: WORKSTREAM-04; Label: high

Row 4 ID: DECISION-04; Decision: Paths are not identity; contracts and manifests define identity.; Status: strong recommendation; Why it mattered / basis: Repeated in final architecture answer.; Confidence: WORKSTREAM-05; Label: high

Row 5 ID: DECISION-05; Decision: Design Dominion as a product-line architecture, not a one-off game.; Status: accepted objective; Why it mattered / basis: User explicitly requested proper game/OS-like architecture and reusable code.; Confidence: WORKSTREAM-05; Label: high

Row 6 ID: DECISION-06; Decision: Use C89-compatible stable ABI seams, without forcing all internals to be C89.; Status: recommendation; Why it mattered / basis: Assistant proposed this refinement.; Confidence: WORKSTREAM-06; Label: medium

Row 7 ID: DECISION-07; Decision: Avoid durable architecture names based on temporary tooling or status labels.; Status: accepted direction; Why it mattered / basis: User disliked XStack framing; prior component naming analysis rejected legacy/modern/compat labels.; Confidence: WORKSTREAM-07; Label: high

Row 8 ID: DECISION-08; Decision: Future refactors should be AIDE work units with move maps and salvage maps.; Status: recommendation aligned with user goal; Why it mattered / basis: User wanted future refactors easy and quick.; Confidence: WORKSTREAM-03; Label: high

Row 9 ID: DECISION-09; Decision: Apps should be thin; runtime adapts host; engine/game own truth.; Status: strong recommendation; Why it mattered / basis: Repeated in architecture doctrine.; Confidence: WORKSTREAM-05; Label: medium-high

Row 10 ID: DECISION-10; Decision: Do not promise compatibility for every internal interface.; Status: recommendation; Why it mattered / basis: Final response emphasized stabilizing correct public seams only.; Confidence: WORKSTREAM-06; Label: medium

DECISION-01 - Use a stable top-level root constitution rather than ad hoc root growth.

Status: accepted direction. Basis: Repeated user concern about avoiding future restructure; assistant proposals align. Rationale: Prevents root sprawl and preserves ownership clarity. Implications: Future roots require admission tests and validators. Related workstream: WORKSTREAM-01. Confidence: high. Label: FACT/INFERENCE. This should be revisited if live repo evidence contradicts the assumed structure, or if the user later chooses a different product-line strategy.

DECISION-02 - Treat AIDE as the restructuring control plane.

Status: accepted by user framing. Basis: User explicitly said to transition to AIDE ASAP and recycle existing material. Rationale: AIDE provides queue, evidence, move-map, and salvage discipline. Implications: Old XStack-style tools become transitional assets. Related workstream: WORKSTREAM-03. Confidence: high. Label: FACT. This should be revisited if live repo evidence contradicts the assumed structure, or if the user later chooses a different product-line strategy.

DECISION-03 - Recycle old tools/docs/tests rather than ignore or discard them.

Status: accepted direction. Basis: User explicitly said there is

code and docs to recycle. Rationale: Prevents loss of useful validators and policies. Implications: Requires classification: keep/adapt/extract/convert/archive/drop. Related workstream: WORKSTREAM-04. Confidence: high. Label: FACT. This should be revisited if live repo evidence contradicts the assumed structure, or if the user later chooses a different product-line strategy.

DECISION-04 - Paths are not identity; contracts and manifests define identity. Status: strong recommendation. Basis: Repeated in final architecture answer. Rationale: Makes directories replaceable and data portable. Implications: Packs, saves, schemas, modules, and packages need IDs, versions, hashes, capabilities. Related workstream: WORKSTREAM-05. Confidence: high. Label: INFERENCE. This should be revisited if live repo evidence contradicts the assumed structure, or if the user later chooses a different product-line strategy.

DECISION-05 - Design Dominium as a product-line architecture, not a one-off game. Status: accepted objective. Basis: User explicitly requested proper game/OS-like architecture and reusable code. Rationale: Supports reuse across products, games, and projects. Implications: Requires stricter separation of engine/game/runtime/contracts/tools/content.

Related workstream: WORKSTREAM-05. Confidence: high. Label: FACT. This should be revisited if live repo evidence contradicts the assumed structure, or if the user later chooses a different product-line strategy.

DECISION-06 - Use C89-compatible stable ABI seams, without forcing all internals to be C89.

Status: recommendation. Basis: Assistant proposed this refinement. Rationale: Stable external seams with flexible implementation. Implications: Public headers need opaque handles, allocators, versioned structs, error codes. Related workstream: WORKSTREAM-06. Confidence: medium. Label: INFERENCE. This should be revisited if live repo evidence contradicts the assumed structure, or if the user later chooses a different product-line strategy.

DECISION-07 - Avoid durable architecture names based on temporary tooling or status labels.

Status: accepted direction. Basis: User disliked XStack framing; prior component naming analysis rejected legacy/modern/compat labels. Rationale: Prevents temporary migration scaffolding becoming permanent ontology. Implications: Use boring literal names and matrix fields. Related workstream: WORKSTREAM-07. Confidence: high. Label: FACT/INFERENCE. This should be revisited if live repo evidence contradicts the

assumed structure, or if the user later chooses a different product-line strategy.

DECISION-08 - Future refactors should be AIDE work units with move maps and salvage maps.

Status: recommendation aligned with user goal. Basis:

User wanted future refactors easy and quick. Rationale:

Turns restructuring into a mechanical process.

Implications: Requires AIDE refactor schemas, ledgers, validators, and temporary aliases. Related workstream:

WORKSTREAM-03. Confidence: high. Label:

INFERENCE. This should be revisited if live repo evidence contradicts the assumed structure, or if the user later chooses a different product-line strategy.

DECISION-09 - Apps should be thin; runtime adapts host; engine/game own truth. Status:

strong recommendation. Basis: Repeated in architecture

doctrine. Rationale: Preserves portability and prevents

UI/rendering/platform code owning simulation truth.

Implications: Boundary validators needed. Related

workstream: WORKSTREAM-05. Confidence: medium-

high. Label: INFERENCE. This should be revisited if

live repo evidence contradicts the assumed structure, or

if the user later chooses a different product-line strategy.

DECISION-10 - Do not promise compatibility for every internal interface. Status: recommendation.

Basis: Final response emphasized stabilizing correct public seams only. Rationale: Avoids maintenance paralysis. Implications: Need stability levels and deprecation policy. Related workstream: WORKSTREAM-06. Confidence: medium. Label: INFERENCE. This should be revisited if live repo evidence contradicts the assumed structure, or if the user later chooses a different product-line strategy.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

REJECTED-01 - Visual drag-and-drop cleanup of root folders Status: rejected. It was rejected/superseded because Would break paths, build, tests, package identity, and authority semantics. Finality: final unless repo becomes trivial. Reconsider conditions: Only reconsider for tiny isolated folders with move map and proof.. Related workstream: WORKSTREAM-01. Label: FACT/INFERENCE.

REJECTED-02 - XStack/AuditX/RepoX/TestX as durable architecture ontology Status: rejected/superseded. It was rejected/superseded because User disliked it; names are temporary scaffolding. Finality: mostly final. Reconsider conditions: May remain in

history/ledgers during transition.. Related workstream: WORKSTREAM-03. Label: FACT.

REJECTED-03 - Discarding old tools/docs/tests during AIDE transition Status: rejected. It was rejected/superseded because User explicitly wants recycling. Finality: final. Reconsider conditions: Only drop after classification and evidence.. Related workstream: WORKSTREAM-04. Label: FACT.

REJECTED-04 - Perfect structure that never changes Status: superseded. It was rejected/superseded because Unrealistic; better target is stable top-level roots plus cheap internal refactors. Finality: final as framing. Reconsider conditions: N/A.. Related workstream: WORKSTREAM-01. Label: INFERENCE.

REJECTED-05 - Using paths as identity Status: rejected. It was rejected/superseded because Makes moves/refactors break artifacts. Finality: final as doctrine. Reconsider conditions: N/A.. Related workstream: WORKSTREAM-05. Label: INFERENCE.

REJECTED-06 - Stabilizing every internal interface forever Status: deprioritized. It was rejected/superseded because Too costly; stabilize public seams and let internals evolve. Finality: tentative but

strong. Reconsider conditions: Revisit per API if external dependents emerge.. Related workstream: WORKSTREAM-06. Label: INFERENCE.

REJECTED-07 - Component IDs based on legacy/modern/compat/universal or version-coded names Status: rejected direction. It was rejected/superseded because Creates naming debt and mixes axes. Finality: strong. Reconsider conditions: Use only as status/version fields where needed..Related workstream: WORKSTREAM-07. Label: FACT/INFERENCE.

7. Important Reasoning, Rationale, and Tradeoffs

The visible rationale centered on avoiding architectural lock-in and path fragility. A neat folder tree alone does not make code reusable or portable. The durable solution requires contracts, manifests, APIs, build targets, validators, and AIDE-managed refactor machinery. The main tradeoff is between rigidity and flexibility: if every internal interface is frozen, the project becomes hard to evolve; if no seam is stable, future games, tools, saves, packs, and integrations cannot rely on anything. The proposed answer is to stabilize public seams and durable data formats, while allowing internals to change behind tests, migrations, and compatibility policies.

A second tradeoff is between immediate cleanup and preserving working checks. The user rejected throwing away old tools. The better path is wrap-before-rename: inventory existing tools, call them through AIDE, classify them, then migrate names and paths after their value is preserved. Another tradeoff is naming clarity versus historical continuity. Names like XStack or AuditX may be useful as migration history but should not become long-term product architecture.

The project's portability goal creates specific constraints: deterministic engine code must avoid hidden OS calls, unnamed randomness, wall-clock dependence, filesystem-order dependence, unversioned binary formats, and public implementation structs. Runtime should adapt the host; engine/game truth should remain host-independent. The user's deeper concern is not just this refactor, but preventing future architectural debt from accumulating unseen.

8. Plans, Future Work, and Next Steps

The recommended path is to implement AIDE-backed architecture contracts before moving large amounts of code. The highest-value next tasks are: verify live repo state; create the repo constitution/refactorability framework; create the modularity and reuse constitution; inventory and wrap old XStack-style tooling; define logical-root projection contracts; add non-blocking

boundary validators; and only then perform ownership extraction waves.

Recommended next-action sequence:

1. **TASK-09** - Verify live repo baseline: current branch, root inventory, validators, CTest status, generated artifacts.
2. **TASK-01** - Implement AIDE-STRUCTURE-00: root constitution, ownership slots, AIDE refactor framework, move/salvage map schemas, inventory tooling.
3. **TASK-03** - Inventory old XStack/AuditX/RepoX/TestX-style tools and classify them.
4. **TASK-04** - Wrap old checks behind AIDE without changing behavior.
5. **TASK-02** - Implement AIDE-ARCH-00: modularity/reuse doctrine, dependency layers, C89 ABI rules, API stability levels, boundary checker.
6. **TASK-05** - Define distribution/install/media/bundle projection contracts.
7. **TASK-06/TASK-07/TASK-08** - Add boundary validators, component manifests, and portability/determinism proof checks.

8. Use AIDE work units for actual cleanup, with move maps, salvage maps, validators, evidence, and shim retirement.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences The user wants human-readable explanation first, not only machine-readable artifacts. The user wants broad/deep reasoning and does not want to repeat restructuring pain. The user explicitly wants portable, modular, extensible, reusable code, and wants AIDE adoption plus recycling of existing code/docs/tests.

9.2 Inferred Constraints and Preferences The user prefers source-grounded, audit-ready decisions; bounded uncertainty; explicit task prompts; and reusable engineering doctrine. Future assistants should avoid shallow affirmation and should distinguish accepted user decisions from assistant recommendations.

9.3 Uncertain or Unestablished Preferences It is not yet established which specific compatibility promises the user wants to make for each API, save format, protocol, or package format. It is also not established how much generated output should be tracked in the repo. These need future decisions after repo verification.

10. Files, Artifacts, Outputs, and Prompts

Row 1 ID: ARTIFACT-01; Artifact / file / prompt / output: Pasted text.txt; Type: uploaded prompt; Purpose: Defines required preservation package structure and rules.; Status: available in this chat; Origin: user upload; Carry forward?: yes; Notes: Must be treated as active instruction for this turn.; Label: FACT

Row 2 ID: ARTIFACT-02; Artifact / file / prompt / output: Prior distribution/install layout analysis pasted by user; Type: pasted assistant output; Purpose: Provided logical-root projection doctrine and layout examples.; Status: visible in chat, not independently verified; Origin: user-pasted text; Carry forward?: yes; Notes: Use as chat content; repo/doc claims need verification.; Label: FACT/UNCERTAIN

Row 3 ID: ARTIFACT-03; Artifact / file / prompt / output: Prior convergence/root cleanup analysis pasted by user; Type: pasted assistant output; Purpose: Provided staged repo convergence and root allowlist plan.; Status: visible in chat, not independently verified; Origin: user-pasted text; Carry forward?: yes; Notes: Some specific live repo claims are unverified.; Label: FACT/UNCERTAIN

Row 4 ID: ARTIFACT-04; Artifact / file / prompt /

output: Prior component naming/matrix analysis
pasted by user; Type: pasted assistant output;
Purpose: Defined clean component naming
doctrine.; Status: visible in chat; Origin: user-
pasted text; Carry forward?: yes; Notes: Should
inform spec-book naming/matrix chapter.; Label:
FACT

Row 5 ID: ARTIFACT-05; Artifact / file / prompt /
output: AIDE transition prompt/task proposal;
Type: assistant-generated prompt; Purpose:
Suggested AIDE-00 bootstrap and recycling plan.;
Status: visible in chat; Origin: assistant response;
Carry forward?: yes, with user review; Notes:
Prompt is recommendation, not executed.; Label:
INFERENCE

Row 6 ID: ARTIFACT-06; Artifact / file / prompt /
output: AIDE-STRUCTURE-00 prompt; Type:
assistant-generated prompt; Purpose: Defines repo
constitution/refactorability framework task.;
Status: visible in chat; Origin: assistant response;
Carry forward?: yes; Notes: Candidate next
Codex/AIDE task.; Label: INFERENCE

Row 7 ID: ARTIFACT-07; Artifact / file / prompt /
output: AIDE-ARCH-00 prompt; Type: assistant-
generated prompt; Purpose: Defines modularity
and reuse constitution task.; Status: visible in

chat; Origin: assistant response; Carry forward?: yes; Notes: Candidate next Codex/AIDE task.; Label: INFERENCE

Row 8 ID: ARTIFACT-08; Artifact / file / prompt / output: This preservation package; Type: generated files + ZIP; Purpose: Preserves this chat for reading, handoff, aggregation, and future spec-book work.; Status: created in this turn; Origin: assistant generated; Carry forward?: yes; Notes: Contains report, registers, context packet, spec sheet, aggregator packet, audit, reader brief.; Label: FACT

Important caveat: the only current uploaded file available for this task is `Pasted text.txt`, which contains the preservation-package prompt. Earlier generated handoff files, ZIP packages, or uploaded docs referenced in prior conversation are not available in this chat unless the user re-uploads them.

11. Open Questions and Unresolved Issues

Row 1 ID: QUESTION-01; Question / issue: What is the current live repo state and which of the pasted commit/status claims remain true?; Why it matters: Implementation depends on baseline accuracy.; Known information: Pasted text asserted specific repo status and failures.;

Unknown information: Live branch, CTest, validators, current root inventory.; Resolution path: Inspect repo with GitHub/Codex and produce baseline report.; Priority: P0; Related workstream: WORKSTREAM-01; Label: UNCERTAIN

Row 2 ID: QUESTION-02; Question / issue: What exact parts of old XStack/AuditX/RepoX/TestX tooling are useful?; Why it matters: Recycling requires classification.; Known information: User wants recycling, not ignoring.; Unknown information: Actual file contents and usefulness.; Resolution path: AIDE inventory with keep/adapt/extract/convert/archive/drop.; Priority: P0; Related workstream: WORKSTREAM-04; Label: FACT/UNCERTAIN

Row 3 ID: QUESTION-03; Question / issue: Which APIs deserve stable/frozen compatibility promises?; Why it matters: Over-stabilizing internals is costly; under-stabilizing public seams breaks reuse.; Known information: Candidate seams include C ABI, schemas, commands, saves, packages, protocols.; Unknown information: Actual implementation maturity and external use cases.; Resolution path: Define stability levels and public API declarations.; Priority: P1; Related workstream: WORKSTREAM-06; Label: INFERENCE

Row 4 ID: QUESTION-04; Question / issue: Where should domino-engine reusable code stop and Dominion-specific game code begin?; Why it matters: Determines reuse for future games.; Known information: Final answer proposed reuse levels.; Unknown information: Actual current code ownership.; Resolution path: Inventory modules and classify reuse level.; Priority: P0; Related workstream: WORKSTREAM-05; Label: INFERENCE

Row 5 ID: QUESTION-05; Question / issue: What generated artifacts are intentionally tracked versus accidental?; Why it matters: Generated-output policy affects cleanup.; Known information: Pasted text references dist/artifacts and generated outputs.; Unknown information: Actual repository policy and files.; Resolution path: Generated artifact audit.; Priority: P1; Related workstream: WORKSTREAM-01; Label: UNCERTAIN

Row 6 ID: QUESTION-06; Question / issue: What should become formal Project Spec Book requirements versus background context?; Why it matters: Avoid prematurely hardening brainstorm.; Known information: This chat proposes many doctrines.; Unknown information: User confirmation and cross-chat consistency.; Resolution path: Aggregator review.; Priority: P1;

Related workstream: WORKSTREAM-09; Label: INFERENCE

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Row 1 ID: RISK-01; Risk / failure mode: Future assistant treats assistant recommendations as user decisions.; Consequence: Spec book hardens tentative ideas incorrectly.; Likelihood: medium; Severity: high; Mitigation: Keep labels and require user confirmation for major implementation commitments.; Related workstream: WORKSTREAM-09; Label: FACT

Row 2 ID: RISK-02; Risk / failure mode: Live repo claims from pasted text are stale or wrong.; Consequence: Codex tasks may target wrong blockers or paths.; Likelihood: medium; Severity: high; Mitigation: Verify repo state before implementation.; Related workstream: WORKSTREAM-01; Label: UNCERTAIN

Row 3 ID: RISK-03; Risk / failure mode: AIDE becomes another mythology instead of a control plane.; Consequence: Product architecture polluted by workflow names.; Likelihood: medium; Severity: medium-high; Mitigation: Keep AIDE under .aide/, tools/aide/, docs/aide; do not name

runtime/game concepts after it.; Related
workstream: WORKSTREAM-03; Label:
INFERENCE

Row 4 ID: RISK-04; Risk / failure mode: Old
validators are renamed before being wrapped.;
Consequence: Regression detection lost.;
Likelihood: medium; Severity: high; Mitigation:
Wrap before rename; preserve behavior first.;
Related workstream: WORKSTREAM-04; Label:
INFERENCE

Row 5 ID: RISK-05; Risk / failure mode: Root
constitution becomes too rigid.; Consequence:
Useful evolution blocked.; Likelihood: low-medium;
Severity: medium; Mitigation: Allow root
admission test with high bar and validator update.;
Related workstream: WORKSTREAM-01; Label:
INFERENCE

Row 6 ID: RISK-06; Risk / failure mode:
Compatibility promises are too broad.;
Consequence: Maintenance burden and slow
evolution.; Likelihood: medium; Severity: medium-
high; Mitigation: Define stability levels; stabilize
only public seams.; Related workstream:
WORKSTREAM-06; Label: INFERENCE

Row 7 ID: RISK-07; Risk / failure mode: Reusable
code remains tangled with Dominion-specific game

rules.; Consequence: Future games/projects cannot reuse the engine cleanly.; Likelihood: medium-high; Severity: high; Mitigation: Classify modules by reuse level and enforce dependencies.; Related workstream: WORKSTREAM-05; Label: FACT/INFERENCE

Row 8 ID: RISK-08; Risk / failure mode: Generated artifacts leak into source roots.; Consequence: Repo becomes hard to reason about and refactor.; Likelihood: medium; Severity: medium; Mitigation: Generated-output policy and validator. ; Related workstream: WORKSTREAM-01; Label: INFERENCE

The most serious risk is that a future assistant treats this chat as if it verified the live repo. It did not. The second serious risk is that it hardens every assistant suggestion into a requirement. Several items are strong recommendations aligned with user goals, but still require implementation review. Future chats must preserve labels and verify stale facts.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat contributes the architecture constitution layer: stable source layout, logical-root projection, AIDE governance, product-line reuse, modular engine/

runtime/game separation, API/ABI/schema/protocol discipline, naming doctrine, generated-output policy, and refactorability workflow. It should feed future chapters on repo architecture, AIDE operations, engine modularity, distribution/install layout, API stability, portability, determinism, testing, and compatibility. It overlaps with other Dominion architecture chats and may conflict if earlier chats used different root names, older XStack terminology, or more specific platform/render priorities.

14. What I Should Remember

- The goal is not a perfect layout that never changes. The goal is stable top-level ownership plus mechanical, reversible internal refactors.
- AIDE should become the refactor/control plane now, not after all old XStack-style tooling is magically cleaned up.
- Existing tools/docs/tests should be inventoried and recycled, not discarded.
- Paths are not identity. Contracts, manifests, IDs, versions, hashes, and capabilities define identity.
- Dominion should be designed as a product-line architecture: reusable across products, future

games on the same engine, and some unrelated engine projects where possible.

- `game/` should contain Dominion-specific truth. Reusable engine/runtime/tooling/contracts should not be trapped there.
- Stable C-style ABI seams are useful for durability, but internal implementation should remain flexible.
- Future assistants must verify live repo state before implementing cleanup.
- The best next action is AIDE-STRUCTURE-00 plus live repo baseline verification.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- Explain the difference between source layout, install layout, package layout, and AIDE refactor layout.
- Summarize the single most important architecture doctrine from this chat.

15.2 Decisions

- Which decisions in this chat were clearly accepted by me, and which are still assistant recommendations?

- Which compatibility promises should be made stable first?

15.3 Tasks and Next Actions

- Turn AIDE-STRUCTURE-00 into a Codex-ready implementation prompt.
- Turn AIDE-ARCH-00 into a shorter first-pass task that cannot break the build.

15.4 Artifacts and Files

- Extract the generated prompts from this report into a standalone implementation queue.
- Create a shorter checklist from the artifact ledger.

15.5 Risks and Verification

- What live repo facts must be verified before running Codex?
- Which pasted repo-status claims are most likely stale?

15.6 Future Spec Book / Aggregation

- Convert this chat into formal requirements for the master Project Spec Book.

- Compare this chat against another Dominium architecture report and identify conflicts.

15.7 Deep-Dive Questions Specific to This Chat

- Define a reusable engine/module boundary model for Dominium.
- Design the exact `contracts/repo/root_constitution.toml` schema.
- Design the exact AIDE move-map and salvage-map schema.

16. Compact Human Summary

This chat was about preventing Dominium from becoming a one-off, fragile game repository. The user wanted a structure and engineering doctrine strong enough for a proper long-lived game or OS-like project: portable, modular, extensible, reusable, backwards-compatible where appropriate, and easy to refactor in the future. The conversation began from earlier architecture outputs about repository layout, distribution/install layouts, package formats, runtime roots, component matrices, and root cleanup. The initial answer was that Dominium needs more than one folder tree: it needs a logical-root projection system. Source repo layout, build output, packages, portable installs,

installed layouts, media, saves, instances, bundles, diagnostics, cache, staging, rollback, symbols, and provenance should all be governed by shared contracts rather than invented separately.

The source repo target stabilized around a small constitutional root set: `.aide/`, `.github/`, `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `docs/`, `tests/`, `tools/`, `scripts/`, `cmake/`, `external/`, `release/`, `archive/`, and generated `dist/` only under strict policy. The user's concern was not simply aesthetic clutter; it was future portability and refactorability. The answer was that root sprawl should be refused by default and new top-level roots should pass a strict admission test.

A major correction came when the user objected to XStack/AuditX/RepoX/TestX-style framing. The user wanted AIDE adopted quickly and existing code/docs/tests recycled rather than ignored. This shifted the plan: AIDE should become the restructuring control plane now. Existing old tools should be inventoried, wrapped, classified, adapted, renamed, or archived through AIDE. They should not be deleted blindly, and their temporary names should not leak into durable Dominion architecture.

The deepest part of the chat reframed Dominion as product-line architecture. Code should be separated by reuse level: reusable across Dominion products, reusable

across future games on the same domino/Dominium engine, and reusable across unrelated projects. This implies a strict split: **contracts/** for stable seams and identity; **engine/** for reusable deterministic substrate and domino engine; **game/** for Dominion-specific rules; **runtime/** for host/platform/render/audio/input/network/storage/UI adaptation; **apps/** for thin entrypoints; **content/** for authored data; **release/** for recipes; **tools/** and **.aide/** for validation and refactor machinery.

The most important doctrine is that paths are not identity. Durable things need IDs, versions, schemas, manifests, hashes, capabilities, compatibility ranges, and migration/refusal policy. Public APIs need stability levels. Stable C89-compatible ABI seams are useful, using opaque handles, explicit allocators, versioned structs, and stable error codes, but internals should remain flexible. Schemas and protocols need version negotiation and capability negotiation. Tests should be deterministic and hermetic. Generated outputs should be quarantined. Future refactors should run through AIDE inventory, move maps, salvage maps, reference rewriting, validators, build/test proof, evidence ledgers, and shim retirement.

The best next action is to verify live repo state, then implement AIDE-STRUCTURE-00 and AIDE-ARCH-00 as non-invasive control-plane and architecture-

constitution tasks. Do not start broad moves until contracts and inventories exist. Do not trust pasted repo-status claims without verification. Do not treat every assistant recommendation here as an accepted decision. Preserve the uncertainty labels when merging this chat into the future Project Spec Book.

13.2 **Dominium Modularity Aide Refactorability Architecture Reader Brief**

Source: docs/archive/conversations/Modularity_AIDE_Refactorability/Dominium_Modularity_AIDE_Refactorability_Architecture__06_reader_brief.md

Report type: reader_brief

Size: 2403 bytes

13.2.1 **Reader Brief - Dominium Modularity, AIDE Refactorability, and Future-Proof Architecture**

What this chat was about

This chat established a future-proofing doctrine for Dominium: stable repository ownership, AIDE-driven refactorability, portable modular code, reusable engine/

runtime/contracts, and mechanically enforced compatibility boundaries.

Top 20 things to know

1. The user wants Dominion built like a serious long-lived game/engine platform.
2. The goal is not just cleaner folders; it is long-term portability and refactorability.
3. A stable root constitution is needed.
4. New top-level roots should be refused by default.
5. AIDE should become the refactor control plane.
6. Old XStack/AuditX/RepoX/TestX-style tools should be recycled, not deleted.
7. Temporary workflow names should not become product architecture.
8. Paths are not identity.
9. Contracts and manifests define identity.
10. Apps should be thin.
11. Runtime owns host adaptation.
12. Engine owns deterministic reusable substrate.
13. Game owns Dominion-specific rules.

14. Content is authored data, not runtime cache.
15. Public APIs need stability levels.
16. Stable C ABI seams should use opaque handles, allocators, versioned structs, error codes.
17. Schemas/protocols need versioning and negotiation.
18. Generated artifacts need quarantine.
19. Future refactors should use AIDE move maps and salvage maps.
20. Live repo status must be verified before implementation.

Decisions

See DECISION-01 through DECISION-10 in the registers. The clearest user-accepted decisions are AIDE adoption and recycling existing material.

Pending tasks

Highest priority: verify repo state; implement AIDE-STRUCTURE-00; inventory old tooling; implement AIDE-ARCH-00.

Open questions

Main questions: current live repo state; useful old tooling; stable API boundaries; reuse-level classification; generated artifact policy; which recommendations become formal requirements.

Artifacts

The uploaded `Pasted text.txt` prompt requested this preservation package. This package generated Markdown, YAML, and ZIP outputs.

Verification items

Verify live repo head, CTest/validator status, referenced docs, old tool paths, component matrix names, and prior generated file availability.

Best next step

Run live repo baseline verification, then implement AIDE-STRUCTURE-00 as a non-invasive control-plane task.

13.3 Dominion Modularity Aide Refactorability Architecture In Chat Reader

Source: docs/archive/conversations/Modularity_AIDE_Refactorability/Dominium_Modularity_AIDE_Refactorability_Architecture__09_in_chat_reader.md

Report type: in_chat_reader

Size: 2145 bytes

13.3.1 In-Chat Reader - Dominion Modularity, AIDE Refactorability, and Future-Proof Architecture

Package overview

This package preserves the current chat about Dominion modularity, future-proof structure, AIDE refactorability, and reusable engine architecture. It contains a human-readable report, structured registers, future-chat handoff, YAML spec sheet, aggregator packet, audit, reader brief, and bootstrap prompt.

File index

1. 00_manifest.md - file list and package status.
2. 01_human_readable_report.md - sections 0-16.

3. `02_context_transfer_packet.md` - future-chat handoff.
4. `03_spec_sheet.yaml` - machine-readable aggregation sheet.
5. `04_registers.md` - workstreams, decisions, tasks, constraints, preferences, risks, verification.
6. `05_aggregator_packet.md` - compact merge packet for master spec aggregation.
7. `06_reader_brief.md` - short human reader guide.
8. `07_verification_and_audit.md` - self-audit and verification queue.
9. `08_future_chat_bootstrap_prompt.md` - prompt for a new chat.
10. `09_in_chat_reader.md` - this guide.

Plain-English explanation

The chat's core contribution is that Dominium needs stable ownership roots, AIDE-driven refactor mechanics, and strict separation of reusable engine/runtime/contracts from Dominium-specific game rules. The main next action is to verify live repo state and implement non-invasive AIDE structure and architecture contracts.

Question menu

Ask: "Which decisions are accepted versus recommendations?", "Draft the AIDE-STRUCTURE-00 Codex prompt", "Design root__constitution.toml", "What repo facts must be verified?", "How should old XStack-style tools be recycled?", or "Turn this into Project Spec Book requirements."

Top things to preserve

AIDE-first control plane; recycle existing tools/docs/tests; paths are not identity; stable root constitution; product-line architecture; public API stability levels; deterministic proof matrix.

Safest next actions

Verify repo baseline, add AIDE scaffolding, inventory old tooling, then add modularity/reuse contracts. Avoid broad moves first.

Chapter 14

Os Interface Repo Architecture

14.1 Dominion Os Interface Repo
 Architecture Human Readable
 Report

Source: docs/archive/conversations/OS_Interface_
Repo_Architecture/Dominium_OS_Interface_Repo_
Architecture__01_human_readable_report.md

Report type: human_readable_report

Size: 40311 bytes

14.1.1 COMPLETE CHAT PRESERVATION
 REPORT - Dominion OS-like
 Architecture, Repository Convergence,
 and Interface Operating Layer

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion OS-
like Architecture, Repository Convergence, and
Interface Operating Layer

Row 2 Field: Date anchor; Assessment: 2026-05-27

Australia/Melbourne

Row 3 Field: Source scope; Assessment: This chat only unless labelled PROJECT-CONTEXT

Row 4 Field: Apparent access; Assessment: Partial

Row 5 Field: Previously generated files available?; Assessment: No previously generated preservation files visible in this chat before this task

Row 6 Field: Uploaded files or artifacts present?; Assessment: Yes - **Pasted text.txt** contains the preservation/export prompt for this task

Row 7 Field: Contains future plans?; Assessment: Yes

Row 8 Field: Contains decisions?; Assessment: Yes, mostly tentative architecture decisions and accepted directions

Row 9 Field: Contains pending tasks?; Assessment: Yes

Row 10 Field: Contains unresolved questions?; Assessment: Yes

Row 11 Field: Staleness risk; Assessment: Medium

Row 12 Field: Extraction confidence; Assessment: 4/5

Row 13 Field: Safe for later aggregation?; Assessment: Yes, with caveats

Row 14 Field: Main limitations; Assessment: I can see the visible transcript and the loaded preservation prompt. I cannot guarantee access to every historical turn if the interface omitted earlier messages. I also cannot see a `docs.zip` attachment in the currently loaded file set. Repository facts were taken from prior tool-accessed live GitHub files during this chat, but those should be treated as observations made during the chat, not necessarily current as of all future dates.

Plain-language limitation note: this report preserves the visible conversation and the supporting repository/file evidence surfaced during the chat. It should not be treated as a complete audit of every file in `/docs` oldest-to-newest; the connector did not provide a chronological full-docs walk. It is nevertheless a high-fidelity preservation of the architectural discussion that actually occurred here.

1. One-Page Orientation

This chat was about rethinking Dominion from a conventional game project into a much broader, more modular, deterministic, extensible software environment. The user repeatedly asked whether the current plan was the best possible version, and the discussion evolved from GUI/binary planning, to repository restructure, to

release/package strategy, to code-vs-data analysis, to an "OS-like" deterministic simulation operating environment, and finally to a unified Interface Operating Layer and Workbench platform.

The user's underlying objective was not simply to clean folders or choose GUI frameworks. The larger goal was to preserve the long-term ambition of Dominium: a deterministic, moddable, inspectable, replayable universe platform where client, server, launcher, setup, tools, Workbench, packs, validation, release, and agent workflows all share common contracts. The user was concerned that the existing code and docs were extensive but not yet aligned with the vision, and that early wrong decisions could create brittle product architectures, GUI drift, duplicated command systems, or a repo that looked cleaner while remaining conceptually confused.

The conversation began from an earlier baseline: CLI should always work, TUI should be expected, and multiple native or rendered GUI shells should be thin, replaceable frontends over a stable backend/UI contract. Windows, macOS, Linux, and Android lanes were discussed, including legacy/modern toolchains and compatibility floors. The chat then moved into repository convergence: the user wanted to know whether a generic `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `tools/` style structure was enough. The answer became more nuanced: the

repository should be organized by architectural ownership and contract boundaries, not by topic folders or temporary product categories. The key split became contracts, engine/kernel, game/domains, runtime/services/drivers, apps/userland products, content/packs, tools/dev tooling, and release/distribution.

A major shift happened when the user asked whether to recycle the repo into something "closer to an OS than to a game." That became the strongest conceptual reframing. The proposed target was not a literal hardware operating system, but a deterministic simulation operating environment: a kernel-like substrate, contracts as law, services, drivers, domain modules, apps as userland shells, packages/content as mounted data, and proof/replay/release around everything. This reframing made the client no longer "the game" but one shell over the environment; the server another authoritative shell; launcher/setup/tools additional userland apps; and Workbench a major self-hosting proof surface.

The later part of the chat narrowed into UI/UX. The user presented a detailed plan for abandoning the old UI Editor / Tool Editor product and instead creating a rendered cross-platform Dominion Workbench / Tools Host. The answer refined this into a broader Interface Operating Layer: one command/result/refusal/document/event spine with multiple projections: CLI,

TUI, rendered GUI, headless report mode, and optional native wrappers later. This would support launcher, setup, client HUDs, server admin, Workbench modules, validation dashboards, pack tools, modding tools, release tooling, renderer sandboxes, replay viewers, and agent work boards. The key product principle is that UI projections do not own semantics; every action goes through commands, capabilities, services, typed results, diagnostics, and evidence.

The future relevance of this chat is high. It contributes the conceptual bridge between repository convergence and product architecture. It also clarifies that the desired MVP should not be a shallow gameplay demo, but a proof spine: boot product binaries, unify command dispatch, mount/validate packages, create/load a universe, render a deterministic no-assets view, accept one lawful intent, commit state, emit event/replay/proof artifacts, and replay to the same hash. Future assistants must understand that many suggestions were architectural directions rather than fully accepted final decisions; they should preserve uncertainty, avoid overclaiming implementation status, and keep the current repo's post-CONVERGE authority stack in mind.

2. The Story of the Conversation

2.1 Starting point: GUI and binary lanes The chat began with a transfer-style discussion of GUI and binary rebuild strategy. The starting architecture was that every product should always support CLI, should be expected to support TUI, and may have modular GUI shells. Native GUI families would vary by host and era: Win32/WinForms/WinUI on Windows, AppKit/SwiftUI on macOS, fewer GUI families on Linux, and Android treated as a distinct mobile host family. The important conclusion was that GUI shells must remain thin adapters over the same product backend and command/UI contract used by CLI and TUI.

2.2 Repository restructure and convergence The user then asked how to actually do the work, given the rough repo state. The assistant inspected live repository documentation and code through the GitHub connector. The existing docs already contained many convergence ideas: AppShell, product boundaries, entrypoint maps, future layout proposals, distribution and versioning models, and later post-CONVERGE repository layout docs. The conversation concluded that the repo was not simply missing ideas; it had too many partially overlapping architecture documents and needed convergence around a single ownership model.

The current source-root authority after the repo's CONVERGE series was identified as `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `docs/`, `tests/`, `tools/`, `scripts/`, `cmake/`, `external/`, `release/`, and `archive/`, with `sdk/` and `examples/` as optional future roots. The chat stressed that older stale layout docs should not override the machine-readable layout contract and post-CONVERGE docs.

2.3 Shipping, versions, packages, installers The discussion moved to what binaries, executables, packages, and setups should ship. The strongest model was layered packaging: canonical `.dcompkg` component packages, install profiles, assembled distribution trees, thin OS-native installer wrappers, and release manifests/index/archive records. The answer recommended public deliverables such as full bundles, server bundles, tools bundles, symbols/debug artifacts, release metadata, and component feeds. It warned not to make raw binaries the primary release unit and not to make installer wrappers own install logic.

2.4 Old/new platform compatibility and binary matrix The user asked whether to ship old and modern binaries per OS, and proposed very broad support from Win95/NT4, .NET 2.0, Mac OS 8.6 Carbon, Mac OS X 10.6, Linux 3.x/GTK, and modern WinUI/AppKit lanes.

The assistant accepted the compatibility-lane idea but rejected the universal all-products/all-era matrix as too broad. The better model was legacy/compat/modern lanes with explicit support tiers. In particular, .NET 2.0 was discouraged as an old Windows default; Win32 C/C++ was preferred for old setup/server surfaces.

2.5 What current code really does The user asked how the current repo code actually works: code vs data, hardcoded vs data-driven, modularity, extensibility, modding, and alignment with archived documentation. The assistant inspected key repo files: client CMake, client main, client shell, command registries, session pipeline/stage registry, runtime app, engine/game CMake, product registry, code/data report, modularity proof, audio and render backends. The conclusion was blunt: current code is modular by build/source architecture and partly data-driven through registries/contracts/packs, but not yet deeply moddable or runtime-extensible from boot to playable world. The client has many hard-coded transitional slices: CLI help, command descriptors, session stages, interaction objects, UI overlay, software glyphs, and null audio. The vision is present in docs and scaffolding, but runtime proof remains partial.

2.6 OS-like reframing The user then proposed recycling everything into a system closer to an OS than a game. This was treated as the strongest reframing: Dominion should be a deterministic simulation operating environment, not literally an OS and not merely a game engine. The analogy mapped OS concepts to Dominion concepts: kernel, syscalls, processes, drivers, filesystem, packages, users/permissions, shell, services, logs. The proposed architecture became contracts + engine/kernel + services + drivers + domains + apps + packs + proof/release.

2.7 Workbench and interface platform The user then introduced a new specific direction: abandon the old UI Editor / Tool Editor and build a cross-platform rendered Domino/Dominium tools environment using the same runtime, renderer, UI, command, pack, and data systems as the client. The assistant accepted this but tightened it: the Workbench should be a modular userland tools host, not a monolithic editor, and shipped tool modules should not live under repo-only `tools/`. A critical correction was that AppShell currently says rendered mode is client-only, so rendered mode must become product-declared by capability and contract.

The final refinement was even broader: the Workbench should not be the platform; it should be the largest proof surface for a reusable Interface Operating Layer. That

layer provides one command/result/refusal/document/event spine and projects it into CLI, TUI, rendered GUI, headless JSON/report mode, and optional native wrappers. The guaranteed UI floor should work without assets, GPU, icons, fonts, sounds, themes, or optional modules. Richer assets can later be validated, packable, and replaceable.

3. Main Topics Discussed

Topic 1 - GUI and frontend architecture The chat discussed how Dominion should handle GUI across products and platforms. The conclusion was that CLI is mandatory, TUI is expected, and GUI shells are modular adapters. This avoids forcing one universal GUI framework across all host eras. Windows can have Win32/WinForms/WinUI lanes; macOS can have AppKit/SwiftUI lanes; Linux should have fewer GUI families; Android is separate. The important memory is that GUI technology is not product architecture. Product behavior belongs behind shared commands/contracts.

Topic 2 - Repository convergence and ownership

The repo discussion established that physical layout should reflect architectural ownership, not simulation topics or product history. The current target roots are apps, engine, game, runtime, contracts, content, docs,

tests, tools, scripts, cmake, external, release, and archive. Domain roots should split across contracts/game/content/docs/tests. The current post-CONVERGE docs are the layout authority; older docs may be historical or stale. This topic matters because platform/render/native GUI expansion should not proceed over root-folder chaos.

Topic 3 - Shipping, versioning, packaging, and distribution The chat discussed shipping bundles and packages. The best model was deterministic component packages, install profiles, assembled bundles, thin setup wrappers, manifests, release indexes, archive history, and offline verification. Versioning should stay layered: suite version, product version, protocol/schema/format versions, semantic contract hashes, build IDs, artifact hashes, release-control-plane IDs, target families, and exact target descriptors. The conversation rejected treating filenames or one SemVer string as truth.

Topic 4 - Platform and binary compatibility lanes The user explored very old and modern targets. The answer kept the compatibility-lane strategy but warned against making every product ship every lane. The future should distinguish official, compatibility, experimental, and preservation/retro targets. Old Windows should use native Win32 where possible rather

than .NET 2.0. macOS old-host support implies frozen toolchains. Linux target descriptors must include more than kernel version.

Topic 5 - Current code/data reality A central part of the chat examined how the current code actually behaves. The repo has native product targets and libraries, but product boot proof remains partial. Engine/game/runtime/client boundaries exist at build level. Data registries exist. However, runtime command dispatch is not yet fully unified, UI is minimal, audio is null, and many client surfaces are hard-coded. Current code is scaffold-heavy and doctrine-rich, but not yet a fully data-driven MVP.

Topic 6 - OS-like deterministic simulation environment The biggest conceptual change was viewing Dominion as a deterministic simulation operating environment. Engine becomes kernel-like substrate; contracts become system law; runtime services become OS services; drivers are platform/render/audio/input/network/storage adapters; game domains are installed domain modules; apps are userland shells; packs are mounted data; replay/proof/release wrap the whole system. This framing integrates the ambitions better than "game plus engine."

Topic 7 - Workbench / Studio / Tools Host The old UI Editor/Tool Editor idea was superseded by a modular Dominion Workbench. The Workbench should be rendered, cross-platform, and built on the same runtime as the client. It should have a module registry, command palette, diagnostics/log panels, validation dashboard, pack browser, UI/HUD sandbox, renderer sandbox, replay/trace viewer, release forge, agent work board, and later material/process editors. It should not be a Visual Studio replacement or one giant editor.

Topic 8 - Interface Operating Layer The final topic turned the Workbench idea into a general Interface Operating Layer. The core is one command/result/refusal/document/event spine with projections into CLI, TUI, rendered GUI, headless report mode, and native wrappers later. Everything user-visible should pass through command/capability/service/result/diagnostic/evidence paths. The no-assets GUI becomes the guaranteed floor, not a weak fallback. The Workbench becomes the largest dogfooding surface for this layer.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted to determine whether the current architecture was the best possible version; how to refactor the repo; what binaries, setups, packages, and distributions to ship; how

modular/data-driven the current code is; whether the code aligns with the project vision; whether Dominium should be more OS-like than game-like; and whether the Workbench/UI direction could produce a complete cross-product interface platform.

4.2 Inferred Goals INFERENCE: The user wanted long-term architectural leverage rather than short-term MVP shortcuts. The recurring concerns were modularity, extensibility, compatibility, modding, future-proofing, robustness, auditability, data-driven behavior, and avoiding duplicated GUI/product logic. The user also appears to want a foundation that can become self-hosting in spirit: a Workbench that uses Dominium systems to create, validate, package, and prove Dominium artifacts.

4.3 Goals That Changed Over Time The goal changed from "which GUI/binary lanes should we use?" to "how should the repo converge?" to "how does code actually work?" to "should Dominium be an OS-like deterministic operating environment?" to "can we build a unified Interface Operating Layer?" The trajectory moved upward in abstraction while becoming more concrete about proof requirements.

4.4 Goals Still Unresolved The unresolved goals are implementation-level. The repo still needs command dispatch unification, product boot proof, portable projection proof, AppShell rendered-mode law update, Workbench module contracts, document/patch/result/refusal schemas, and a boot-to-replay MVP. It also needs continued exception retirement, CTest/RepoX remediation, and verification of current build status.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: CLI mandatory, TUI expected, GUI modular; Status: Accepted direction; Why it mattered: Prevents product behavior from being owned by GUI frameworks; Confidence: High; Label: FACT

Row 2 ID: DECISION-02; Decision: GUI shells must be thin over shared contracts; Status: Accepted direction; Why it mattered: Prevents feature drift and duplicate product architectures; Confidence: High; Label: FACT/INFERENCE

Row 3 ID: DECISION-03; Decision: Repo structure should follow ownership boundaries; Status: Accepted direction; Why it mattered: Avoids domain/root chaos and supports future validation; Confidence: High; Label: FACT

Row 4 ID: DECISION-04; Decision: Use post-

CONVERGE roots as current layout authority;
Status: Accepted as repo fact; Why it mattered:
Prevents stale docs from overriding current
contract; Confidence: High; Label: FACT

Row 5 ID: DECISION-05; Decision: Treat Dominion
as deterministic simulation operating environment;
Status: Strong accepted direction; Why it
mattered: Better fits kernel/contracts/services/
drivers/domains/apps/packs model; Confidence:
Medium-high; Label: INFERENCE

Row 6 ID: DECISION-06; Decision: Workbench should
be modular Tools Host, not monolithic editor;
Status: Accepted direction; Why it mattered:
Avoids old UI Editor mistake; Confidence: High;
Label: FACT/INFERENCE

Row 7 ID: DECISION-07; Decision: Interface
Operating Layer should underlie all products;
Status: Strong direction; Why it mattered: Makes
CLI/TUI/rendered/headless projections share
semantics; Confidence: Medium-high; Label:
INFERENCE

Row 8 ID: DECISION-08; Decision: Rendered mode
must become product-declared, not client-only;
Status: Proposed requirement; Why it mattered:
Required for Workbench rendered GUI to be
lawful; Confidence: Medium; Label: INFERENCE

- Row 9** ID: DECISION-09; Decision: Shipped tool modules should not live under repo-only `tools/`; Status: Proposed requirement aligned with repo docs; Why it mattered: Prevents runtime dependency on developer tooling; Confidence: High; Label: FACT/INFERENCE
- Row 10** ID: DECISION-10; Decision: No-assets GUI is guaranteed floor; Status: Accepted direction; Why it mattered: Ensures recovery/product usability without optional assets; Confidence: High; Label: FACT/INFERENCE
- Row 11** ID: DECISION-11; Decision: Package/distribution should be layered and deterministic; Status: Accepted direction; Why it mattered: Supports offline verification, rollback, reproducibility; Confidence: High; Label: FACT
- Row 12** ID: DECISION-12; Decision: Avoid arbitrary native plugins early; Status: Proposed direction; Why it mattered: Reduces trust/safety/replay risks; Confidence: Medium; Label: INFERENCE

Decision explanations **DECISION-01 - CLI mandatory, TUI expected, GUI modular.** This was part of the initial architecture baseline and was repeatedly reaffirmed. It matters because every product must remain operable in recovery/headless/automation

contexts. GUI is allowed but not authoritative.

DECISION-02 - Thin GUI shells over shared contracts. The chat consistently rejected GUI families becoming separate product architectures. This affects client, launcher, setup, server admin, tools, and Workbench modules.

DECISION-03 - Repo ownership layout. The user pushed for repository convergence; the discussion established that folders should map to ownership and contract boundaries. This decision is final enough to guide work, but details remain subject to machine-readable layout contracts.

DECISION-04 - Current post-CONVERGE authority. The final audit and layout target docs define current roots and authority stack. Older docs remain context but not current physical-layout authority.

DECISION-05 - OS-like deterministic operating environment. The user proposed this framing and the assistant endorsed it. It is not a literal OS decision; it is a conceptual architecture decision. It should be formalized before implementation.

DECISION-06 - Workbench modular host. The user explicitly accepted the idea of one integrated environment with many focused modules. This supersedes the old UI Editor / Tool Editor final-product plan.

DECISION-07 - Interface Operating Layer. The final UI discussion generalized Workbench into a cross-product interface platform. This is a strong direction, but it should be formalized as doctrine and contract work before being treated as implemented.

DECISION-08 - Rendered mode product-declared. This was an assistant correction based on AppShell docs. It is necessary because current AppShell docs say rendered mode is client-only. User has not yet separately confirmed the doctrine change, so treat it as a required proposed decision.

DECISION-09 - Keep shipped modules out of repo-only @@V10@@. This follows repo ownership rules. The Workbench's shipped modules should live under `apps/tools/modules/` or equivalent, while developer validators/codegen/audit tooling remains under `tools/`.

DECISION-10 - No-assets GUI floor. The UI baseline doc already requires a zero-asset GUI floor. The chat expanded this into a product-grade primitive UI system.

DECISION-11 - Deterministic layered packaging. The chat recommended component packages, install profiles, distribution trees, thin installer wrappers, release manifests and indexes. This aligns with release/distribution docs.

DECISION-12 - Avoid arbitrary native plugins early. This is a risk-management choice. Early modules should be built-in or data-described; later IPC/WASM/script/native extensions require trust and sandbox policy.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

1. **One universal GUI framework for all hosts.**
Rejected because long-lived Dominion needs native/near-native families and compatibility lanes.
2. **One GUI per OS version.** Rejected as matrix explosion. The better model is compatibility lanes and visual profiles.
3. **Generic game repo layout without Dominion-specific ownership rules.**
Superseded by ownership-root and domain-split doctrine.
4. **Old UI Editor -> Tool Editor as final product.** Superseded by modular Dominion Workbench and later Interface Operating Layer.
5. **One monolithic "everything editor."** Rejected. The Workbench should be a shell plus modules.
6. **OS-native widgets as the core tools strategy.** Deprioritized. Rendered UI using Dominion

runtime should be first; native wrappers later.

7. **Visual Studio/Xcode replacement as immediate goal.** Rejected. Workbench is self-hosting in spirit, not a full IDE replacement.
8. **.NET 2.0 as old Windows GUI default.** Rejected/deprioritized in favor of native Win32 for old setup/server surfaces.
9. **Raw binaries as primary release unit.** Rejected in favor of deterministic package/profile/bundle/release-manifest model.
10. **Arbitrary native plugin marketplace early.** Deprioritized until trust/capability/sandbox law exists.
11. **Treating the client as the game.** Superseded by client as userland shell over deterministic operating environment.
12. **Treating Workbench as the whole platform.** Superseded by Interface Operating Layer; Workbench is a host/proof surface.

7. Important Reasoning, Rationale, and Tradeoffs

The central tradeoff was between short-term visible functionality and long-term architectural leverage. The user repeatedly asked whether the plan could be more

modular, future-proof, optimized, robust, reliable, useful, and powerful. The answer consistently favored building shared contracts and proof paths before expanding product surfaces.

The repo's current state shaped the reasoning. It already has rich documentation, validators, contracts, registries, and build targets, but current runtime code remains partially hard-coded. Therefore, the best strategy is not greenfield invention nor continuing the old product/UI plan; it is recycling existing code through stronger contracts, services, generated descriptors, and proof gates.

Another tradeoff was between data-driven runtime flexibility and deterministic portability. The conversation did not recommend arbitrary scripting everywhere. It recommended descriptor-driven systems, validation, deterministic code generation, static/built-in modules first, and richer plugin forms only after trust/capability policy matures.

For UI, the tradeoff was between polish through assets and guaranteed recoverability. The chat chose a strong no-assets floor: built-in primitives, text, layout, and software rendering should produce a complete product-grade experience even when packs/themes/fonts/GPU/sound are missing. Richer assets are optional validated packs later.

For repository architecture, the tradeoff was between intuitive names and actual ownership. The chat chose ownership and contract boundaries over topical roots. That is less immediately pretty, but it prevents architecture drift and makes validators possible.

8. Plans, Future Work, and Next Steps

8.1 Immediate proof-spine work

1. Remediate or classify remaining RepoX/CTest drift.
2. Repair canonical verify test discovery.
3. Ensure native product binaries boot.
4. Fix or classify setup Python bridge compatibility, missing `dist/bin/dom`, and server CLI arg forwarding.
5. Prove portable projection assembly with required manifests.
6. Rerun product boot and portable projection proof.

These are blocking because current docs still classify product boot and portable projection proof as partial or blocked.

8.2 Interface-law work

1. Write INTERFACE-LAW-00.
2. Update AppShell rendered-mode law so rendered mode is product-declared.
3. Define command/result/refusal/document/patch/module/theme/diagnostic/evidence schemas.
4. Ensure UI projections do not own product semantics.

8.3 Command-service unification Create one dispatch path used by CLI, TUI, rendered GUI, and headless report mode. Acceptance test: same command, same result, same refusal, same diagnostics, different presentation.

8.4 Workbench shell Build a minimal rendered Workbench shell with software renderer, module registry, command palette, console overlay, log/diagnostics panel, and first module.

8.5 First modules Recommended order: Validation Dashboard, Pack Browser, Renderer Sandbox, UI/HUD Sandbox, Replay/Trace Viewer, Release Forge, Agent Work Board.

8.6 Boot-to-replay MVP The true MVP should boot, mount a base package, load/create a world, render a deterministic view, accept one intent, commit a lawful transition, emit event/replay/proof, and replay to the same hash.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences

- The user wants direct, source-grounded, audit-ready answers.
- The user wants uncertainty labels and correction of incorrect framing.
- The user wants human-readable explanations, not only machine-readable handoffs.
- This preservation task specifically required FACT / INFERENCE / UNCERTAIN / PROJECT-CONTEXT labels, stable IDs, registers, spec sheets, an aggregator packet, self-audit, and downloadable files if tools are available.
- The source scope is this chat only unless external project context is labelled.
- Future facts involving platforms, software support, toolchains, APIs, laws, schedules, prices, current roles, etc. require verification.

9.2 Inferred Constraints and Preferences

- **INFERENCE:** The user prefers architectural correctness over quick demos.
- **INFERENCE:** The user wants future assistants to preserve reasoning and not collapse tentative ideas into final decisions.
- **INFERENCE:** The user values modularity, extensibility, determinism, compatibility, and long-term maintainability.
- **INFERENCE:** The user wants usable product concepts, not just abstract architecture.

9.3 Uncertain or Unestablished Preferences

- **UNCERTAIN:** Whether the user wants to rename `engine` conceptually to `kernel` in code, or only use `kernel` as a conceptual layer.
- **UNCERTAIN:** Whether the final public product name should be Dominium Workbench, Dominium Studio, Domino Workbench, or something else.
- **UNCERTAIN:** How aggressive the next actual code refactor should be versus staged adapters and codegen.

10. Files, Artifacts, Outputs, and Prompts

Important artifacts in or referenced by this chat:

1. **Uploaded @@V10@@.** Contains the preservation/export prompt that generated this package.
2. **Repo docs inspected during chat.** Key examples: AppShell constitution, GUI baseline, repo layout target, ownership rules, domain split rules, final convergence audit, post-CONVERGE next steps, versioning/distribution docs, code/data report, modularity proof, product boot proof.
3. **GitHub repository references.** The repo Julesc013/dominium was inspected via GitHub connector.
4. **Conceptual artifacts produced in chat.** These include proposed architectures, matrices, lane plans, Workbench modules, Interface Operating Layer, boot-to-replay MVP, and structured roadmaps.
5. **This preservation package.** Newly generated files and ZIP at the end of this task.

No prior downloadable handoff ZIP from this chat was visible before this task. The earlier user mentioned a

`docs.zip`, but no such file was available in the currently loaded file set.

11. Open Questions and Unresolved Issues

1. Should `engine` remain the physical name while being conceptually treated as kernel, or should a future rename be considered?
2. What exact schema should define `module_descriptor_v1`?
3. How should rendered-mode capability law be updated without breaking existing AppShell doctrine?
4. Which command registry becomes the single runtime authority?
5. How should existing hard-coded client commands, session stages, and interaction definitions be externalized?
6. What is the first real Workbench module: Validation Dashboard or Pack Browser?
7. Which product boot blockers are still current after the latest repo commits?
8. How much of the current game/domain code should be salvaged into domain interpreters versus rewritten?

9. What plugin tiers are acceptable after built-in/data-described modules?
10. What is the exact boot-to-replay MVP artifact set?
11. How should future reports from other old chats be reconciled against this one?

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Future assistants might overstate implementation status. Much of this chat defines direction, not completed code. Product boot and portable projection proof were partial/blocked in inspected docs.

A future assistant might treat Workbench as the platform. The correct model is Interface Operating Layer as platform; Workbench is a major host/proof surface.

A future assistant might recreate the old UI Editor mistake by building a monolithic GUI-only tool. The correct model is command/document/result-driven modules with CLI/TUI/rendered/headless parity.

A future assistant might put shipped runtime modules under repo-only `tools/`. The ownership rules prohibit runtime/product code depending on developer tooling.

A future assistant might ignore the rendered-mode law conflict. Current AppShell docs say rendered mode is client-only; Workbench needs a deliberate law update.

A future assistant might treat all old platform compatibility targets as near-term release targets. The chat distinguished official, compatibility, experimental, and preservation lanes.

A future assistant might forget the no-assets floor. The UI must work without optional assets, fonts, GPU, sound, themes, modules, or packs.

A future assistant might merge tentative assistant proposals as user decisions. Preserve status labels.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat should feed several chapters of a future Project Spec Book:

- Repository and ownership architecture.
- Deterministic simulation operating environment doctrine.
- AppShell and product shell model.
- Interface Operating Layer and UI/UX platform doctrine.
- Workbench/productivity environment design.
- Packaging/versioning/distribution model.
- Platform/binary compatibility lanes.

- Code-vs-data and modding/extensibility strategy.
- MVP/proof spine and boot-to-replay definition.
- Risk management and rejected options.

The unique contribution is the synthesis: Dominionium should not be framed only as a game or editor suite; it should be a deterministic operating environment with a reusable interface layer. This likely overlaps with other chats on worldgen, domain realism, UI tools, release packaging, and repo convergence. Conflicts to watch: product names, physical directory names, toolchain/platform targets, and how aggressive the OS-like refactor should be.

14. What I Should Remember

- Dominionium's strongest architecture is a deterministic simulation operating environment, not just a game.
- The current source-root authority is post-CONVERGE: apps, engine, game, runtime, contracts, content, docs, tests, tools, scripts, cmake, external, release, archive.
- GUI shells must not own product semantics.
- CLI, TUI, rendered GUI, and headless modes should project the same command/result/refusal/

document spine.

- Workbench should be a modular userland app over the Interface Operating Layer, not a monolithic editor.
- No-assets rendered UI is the guaranteed floor, not a weak fallback.
- Current code is scaffold-rich but not yet fully data-driven or deeply moddable at runtime.
- Product boot proof, portable projection proof, command unification, and AppShell law updates remain critical.
- Treat many items as direction unless the user explicitly accepted them as final.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- "Explain the difference between Dominion as a game, engine, OS-like environment, and Interface Operating Layer."
- "What were the most important shifts in direction across this chat?"

15.2 Decisions

- "Which decisions are final enough to implement, and which need confirmation?"
- "What does rendered-mode product-declared law need to say?"

15.3 Tasks and Next Actions

- "Turn the next-action sequence into concrete PR-sized tasks."
- "Draft INTERFACE-LAW-00."

15.4 Artifacts and Files

- "Which repo docs should be promoted into the master spec book?"
- "Which files from this preservation package should I combine with other chat reports?"

15.5 Risks and Verification

- "Audit the risk that Workbench becomes another monolithic editor."
- "What repository facts need re-verification before coding?"

15.6 Future Spec Book / Aggregation

- "Convert this report into a master-spec chapter outline."
- "Compare this chat's decisions against another old-chat report."

15.7 Deep-Dive Questions Specific to This Chat

- "Design `module_descriptor_v1`."
- "Design the unified command/result/refusal model."
"
- "Design the no-assets UI control set."
- "Define the boot-to-replay MVP acceptance tests."

16. Compact Human Summary

This chat was about turning Dominium from a sprawling game/engine/tooling project into a coherent deterministic platform. The user repeatedly asked whether the plan was the best possible version and pushed for more modularity, extensibility, robustness, future-proofing, compatibility, data-driven behavior, and utility. The discussion began with GUI and binary strategy, moved through repo convergence and distribution packaging, audited current code vs data

reality, reframed the project as something closer to an operating environment than a game, then focused on a unified Workbench and finally a full Interface Operating Layer.

The earliest baseline was that all products must have CLI, should have TUI, and may have modular GUIs. GUI shells should be thin adapters over the same backend/UI contract. That led into platform lanes: Windows legacy/compat/modern, macOS AppKit/SwiftUI/legacy toolchain, Linux reduced GUI family count, and Android as a distinct host family. The chat rejected one universal GUI framework and one GUI per OS version.

The repository discussion established that Dominium should be organized by ownership and contracts, not by topic folders. The post-CONVERGE layout authority uses roots such as `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `tools/`, `release/`, and `archive/`. Domain work must split into contracts, implementation, content, docs, and tests. Older layout docs may be historical but should not override machine-readable contracts.

The shipping discussion recommended deterministic layers: `.dompkg` packages, install profiles, assembled bundles, thin installer wrappers, release manifests, release indexes, archive history, and offline verification. Versioning must remain layered: `suite/product/`

protocol/schema/format/semantic-contract/build/artifact/release/target descriptors. Filenames are projections, not truth.

The code audit concluded that the current repo is modular at source/build level and partly data-driven through registries and validators, but not yet a fully data-driven moddable runtime. The client still has hard-coded help, command descriptors, session stages, UI overlay, interaction definitions, and null audio. Product boot and portable projection proof were partial or blocked in inspected docs.

The biggest conceptual shift was accepting the user's "closer to an OS than a game" idea, with precision: Dominium should be a deterministic simulation operating environment, not a literal OS. Engine becomes kernel-like substrate; contracts become law; runtime services provide commands/packages/world/replay/diagnostics; drivers handle platform/render/audio/input/network/storage; domains are installed interpreters; apps are userland shells; packs are mounted data; proof/replay/release wrap everything.

The Workbench discussion superseded the old UI Editor / Tool Editor plan. The best product is a modular Dominium Workbench: one rendered tools host, many modules, same runtime as client, same command/result/refusal model as CLI/TUI/headless. However, the final refinement was broader: the Workbench is not the

platform; the Interface Operating Layer is. The layer provides one command/result/refusal/document/event spine and projections into CLI, TUI, rendered GUI, headless reports, and native wrappers later.

The most important next step is not a pretty GUI. It is law and proof: update AppShell rendered-mode capability law, unify command dispatch, define typed result/refusal/document/patch/module schemas, prove product boot, then build a minimal Workbench shell with Validation Dashboard and Pack Browser. The best MVP is a boot-to-replay proof: mount package, create/load world, render deterministic view, submit one intent, commit state, emit event/replay/proof, and replay to same hash.

14.2 Dominium Os Interface Repo Architecture Reader Brief

Source: docs/archive/conversations/OS_Interface_Repo_Architecture/Dominium_OS_Interface_Repo_Architecture__06_reader_brief.md

Report type: reader_brief

Size: 1697 bytes

14.2.1 Reader Brief - Dominion OS-like Architecture, Repository Convergence, and Interface Operating Layer

This chat preserved a major architectural synthesis for Dominion. The most important idea is that Dominion should be treated as a deterministic simulation operating environment with a reusable Interface Operating Layer.

Top 20 Things to Know

1. CLI is mandatory, TUI expected, GUI modular.
2. GUI shells are thin projections over command/service contracts.
3. Current repo authority is post-CONVERGE roots.
4. Domain work splits across contracts/game/content/docs/tests.
5. Current code is partly modular but not fully data-driven.
6. Product boot proof was partial/blocked in inspected docs.
7. The OS-like reframing is the strongest conceptual direction.
8. Engine is conceptually kernel-like, but physical rename is unresolved.

9. Workbench replaces old UI Editor / Tool Editor final product.
10. Workbench is modular shell plus modules, not monolithic editor.
11. Interface Operating Layer is broader than Workbench.
12. No-assets GUI is guaranteed floor.
13. Rendered mode currently conflicts with AppShell client-only rule.
14. Rendered mode should become product-declared by capability.
15. Shipped modules should not depend on repo-only `tools/`.
16. Command dispatch must be unified.
17. First modules should be Validation Dashboard and Pack Browser.
18. Boot-to-replay is the best MVP definition.
19. Packaging should be deterministic and manifest-driven.
20. Future assistants must not overstate implementation status.

Best Next Step

Draft INTERFACE-LAW-00 and DOMINIUM_OPERATING_ENVIRONMENT.md, then turn them into PR-sized tasks after verifying current repo state.

14.3 Dominium Os Interface Repo Architecture In Chat Reader

Source: docs/archive/conversations/OS_Interface_Repo_Architecture/Dominium_OS_Interface_Repo_Architecture__09_in_chat_reader.md

Report type: in_chat_reader

Size: 1032 bytes

14.3.1 In-Chat Reader - Dominium OS-like Architecture, Repository Convergence, and Interface Operating Layer

Package Overview

This preservation package explains the chat, records decisions, tasks, artifacts, risks, and open questions, and exports a structured spec/aggregation packet for future merging.

File Index

- File 00 manifest: package overview.
- File 01 report: sections 0-16.
- File 02 transfer packet: future-chat handoff.
- File 03 spec sheet: YAML aggregation data.
- File 04 registers: structured tables.
- File 05 aggregator packet: compact merge-ready summary.
- File 06 reader brief: short practical guide.
- File 07 verification/audit: self-audit and verification queue.
- File 08 bootstrap prompt: prompt for new chat.
- File 09 in-chat reader: this guide.

Question Menu

Ask:

- What is the Interface Operating Layer?
- Which decisions are final?
- What should the next PR be?

- What does `module_descriptor_v1` need?
- How do we avoid another monolithic editor?
- How do we define boot-to-replay MVP?

14.4 Dominion Os Interface Repo Architecture Accompanying Detailed Summary Report

Source: docs/archive/conversations/OS_Interface_Repo_Architecture/Dominium_OS_Interface_Repo_Architecture__10_accompanying_detailed_summary_report.md

Report type: detailed_summary_report

Size: 12846 bytes

14.4.1 Accompanying Human-Readable Detailed Summary and Report

Chat label: Dominion OS-like Architecture, Repository Convergence, and Interface Operating Layer

Generated: 2026-05-27 20:36:00 AEST

Purpose: Companion narrative and bundle index for the preservation package already generated in this chat.

Scope: This chat only, with repository facts limited to files and evidence surfaced during the conversation.

1. Why this companion report exists

The earlier preservation package already contains a full human-readable report, context transfer packet, structured registers, spec sheet, aggregator packet, reader brief, verification/audit note, and future-chat bootstrap prompt. This companion report adds a more direct narrative recap of the entire discussion, explains how the pieces fit together, records what was actually produced, and bundles every visible package file into one fresh ZIP.

The user asked for a single downloadable bundle containing all the files plus an accompanying human-readable detailed report of the conversation: what was discussed, what was decided, what was deferred, what was produced, and what still needs follow-up. This report serves that role. It is intentionally redundant with the longer report in `01_human_readable_report.md`, because preservation packages should remain understandable even if one file is read in isolation.

2. Core story of the conversation

This conversation started from a broad architectural concern: Dominion had many ambitions, many docs, many possible GUI/toolchain/platform lanes, and no working MVP yet. The user repeatedly asked whether the proposed plan was "the best we can do." The answer

evolved over the course of the chat.

The first level of the discussion was about products and user interfaces. Dominion should not rely on a single GUI framework, and it should not let GUI frameworks become product architecture. The stable rule became: every product must have CLI, TUI should be expected, and GUI should be modular. GUI/TUI/CLI/headless modes must be projections over the same command/service/backend contract. This is the point that later drove the Interface Operating Layer.

The second level was repository structure. The user was already refactoring the repo and asked how the structure should actually work. The discussion rejected a merely generic `apps/engine/game/runtime/contracts/content/tools` layout unless each folder was tied to Dominion's own constitutional boundaries. The important repo doctrine became ownership-based structure: `apps/` for thin product entrypoints, `engine/` for deterministic substrate, `game/` for domain/rule logic, `runtime/` for AppShell/platform/render/audio/input/network/storage/diagnostics/UI, `contracts/` for machine-readable law, `content/` for authored packs/data/assets, `tools/` for repo/dev/validation tooling, `release/` for release definitions, and `archive/` for historical material. Domain work should split across contracts, implementation, content, docs, and tests, rather than creating new root folders.

The third level was shipping. The chat discussed binaries, installers, versioning, packaging, release indexes, portable bundles, server bundles, tools bundles, symbols, and thin installer wrappers. The best model was not "ship raw binaries and call it done." The preferred model was layered: deterministic component packages, install profiles, assembled distribution trees, thin OS-native wrappers that call setup, release manifests, release indexes, retained history, and offline-verifiable archives. Versioning must remain layered: suite version, product versions, protocol/schema/format versions, semantic contract hashes, build IDs, artifact hashes, release/channel IDs, target families, and exact target descriptors.

The fourth level was reality checking. The user asked how the current repo code actually works from process start to client UI/world interaction. The audit found that the repo has real modular scaffolding: native product targets, an engine library, a game library, runtime app helpers, product registries, command registries, null/software renderers, and extensive docs. But it is not yet a fully data-driven, moddable, runtime-extensible game/platform. Current client code still contains hard-coded CLI help, command descriptors, session stages, interaction IDs, UI overlay strings, renderer glyphs, and null/no-op audio. Data-driven capability extension exists at the manifest-validation

level, but the command graph is not yet fully the runtime source of truth. Product boot and portable projection proofs were partial or blocked in inspected docs.

The fifth and most important level was conceptual reframing. The user proposed recycling the project into something closer to an OS than a game. The refined answer was: not a literal hardware operating system, but a deterministic simulation operating environment. In that model, **engine/** is the kernel-like deterministic substrate; **contracts/** are system law; **runtime/** contains services and drivers; **game/domains/** contains lawful domain interpreters; **apps/** are userland shells; **content/packs** are mounted data; and **replay/proof/** release infrastructure surrounds every state transition. This reframing made the client one shell over the environment, not "the game" itself.

The sixth level was tools and UI/UX. The old UI Editor / Tool Editor plan was superseded. The better direction became a rendered, cross-platform Dominion Workbench / Tools Host, but even that was later generalized. The Workbench is not the platform; the reusable platform is the Interface Operating Layer. That layer gives every product and tool one command/result/refusal/document/event spine, projected into CLI, TUI, rendered GUI, headless JSON/report mode, and optional native wrappers later. The Workbench becomes the largest proof surface for that layer and can host

modules such as Validation Dashboard, Pack Browser, UI/HUD Sandbox, Renderer Sandbox, Replay/Trace Viewer, Release Forge, Agent Work Board, Project Graph Explorer, and Theme Laboratory.

The final result of the conversation is an architectural doctrine candidate: Dominion should become a deterministic world operating environment with a complete, recoverable, extensible interface and production environment. Everything essential should work from built-in no-assets primitives; richer assets, themes, modules, sounds, fonts, icon packs, and platform-native polish should be optional, validated, packable, and replaceable.

3. Most important conclusions

1. **Dominium should be treated as a deterministic simulation operating environment, not merely a game.** The game client is one shell over the environment, not the whole product.
1. **The Interface Operating Layer is broader than the Workbench.** Workbench is the largest host/proof surface, but launcher, setup, client, server admin, tools, mod workflows, pack workflows, release workflows, and agent workflows should all use the same interface law.

1. **All user-visible behavior should route through command/result/refusal/document/event semantics.** Button clicks, TUI actions, CLI commands, and headless scripts should not fork behavior.
1. **Repo structure should preserve post-CONVERGE ownership roots.** Do not start a new root-level revolution unless a contract explicitly changes it.
1. **Domain work must use the five-way split:** contracts, game implementation, content/data, docs, and tests.
1. **The no-assets GUI is a guaranteed product floor, not a weak fallback.** Dominion should be usable with no textures, icon packs, fonts, sounds, GPU, theme packs, or optional modules.
1. **The current code is promising but not yet the full vision.** It is modular at build/source level and partly data-driven, but many runtime pathways are still hard-coded or only partially wired.
1. **Rendered Workbench requires AppShell law changes.** The inspected AppShell doctrine treated

rendered mode as client-only, so rendered mode must become product-declared by capability and rendered-shell contract.

1. **Shipped Workbench modules should not live under repo-only @@V10@@.** Runtime or shipped code should use `apps/tools/`, `runtime/`, and `contracts/`, while repo-only validators/dev tooling stay under `tools/`.
1. **The best MVP is a boot-to-replay proof spine.** Boot, mount/validate packages, load/create a world, render a deterministic view, submit one lawful intent, commit state, emit event/replay/proof artifacts, and replay to the same hash.

4. Decisions and their status

The conversation made several strong decisions or near-decisions, but not all should be treated as final implementation law. The strongest accepted directions were:

- CLI mandatory, TUI expected, GUI modular.
- GUI/TUI/CLI/headless should share semantics.
- Repo layout should be ownership-based.
- Domain roots should split across `contracts/``game/``content/``docs/``tests/`.

- Raw binaries and filenames are not release truth.
- Workbench should replace the old UI Editor / Tool Editor as the major tools direction.
- The Interface Operating Layer is the correct generalized framing.

The most important proposed-but-still-needing-formalization decisions are:

- Rendered mode should be product-declared, not client-only.
- INTERFACE-LAW-00 should define command/result/refusal/document/projection semantics.
- A module descriptor contract should define how Workbench modules register commands, panels, documents, capabilities, diagnostics, and result schemas.
- The boot-to-replay MVP should become the real MVP gate.

5. What was put off for later

The conversation explicitly or implicitly deferred many things:

- serious native GUI expansion;

- WinForms/WinUI/AppKit/GTK/SwiftUI details beyond lanes;
- retro platform binaries;
- arbitrary plugin systems;
- full module marketplace;
- rich asset/theme/icon/font/sound packs;
- photoreal previews;
- hardware renderer expansion beyond proof stages;
- deep domain realism work;
- full Workbench module suite;
- public release packaging;
- full code recycling/rewrite;
- final SDK design;
- all current repo blocker remediation;
- formal Spec Book integration with other chats.

These were deferred because the proof spine and command/interface law need to be stable first.

6. What we actually produced in this chat

The prior preservation task produced a set of downloadable handoff files, including a manifest, human-readable report, context transfer packet, spec sheet, registers, aggregator packet, reader brief, verification/audit note, future-chat bootstrap prompt, in-chat reader, and a ZIP archive. This companion task adds this report plus a fresh complete ZIP bundle that contains all visible preservation files, the original uploaded preservation prompt, and verification metadata.

7. Current blockers and verification needs

Before coding from this architecture, the project should re-verify current repo state. The important verification items are:

- build/test status;
- product boot proof;
- portable projection proof;
- current AppShell rendered-mode law;
- current product registry capabilities;
- command registry and dispatch ownership;
- current status of RepoX/CTest blockers;

- whether any missing `docs.zip` or other file context exists elsewhere.

8. Recommended next action sequence

1. Re-verify build/product/projection status in the current repo.
2. Draft `DOMINIUM_OPERATING_ENVIRONMENT.md` to formalize the OS-like framing without changing roots.
3. Draft `INTERFACE-LAW-00` to define CLI/TUI/rendered/headless projections over one command/document/result spine.
4. Update AppShell/product capability law so rendered mode can be declared by non-client products.
5. Define `command_result_v1`, `document_patch_v1`, `module_descriptor_v1`, `theme_descriptor_v1`, and `diagnostic_result_v1`.
6. Unify runtime command dispatch so CLI, TUI, rendered, and headless paths call the same command service.
7. Build a minimal Workbench shell with command palette, console overlay, module registry, log panel, and Validation Dashboard.

8. Prove the boot-to-replay MVP.

9. How to use this package

Read the companion report first if you want the narrative. Read the original full human-readable report for maximum preservation detail. Read the registers to turn the discussion into tasks. Read the aggregator packet when merging this chat with other old-chat reports. Read the verification/audit file before implementation work.

10. Final warning

This chat contains strong architecture direction, not a completed implementation. Future assistants should not treat every assistant suggestion as a user-approved requirement. The safest path is to formalize the direction into doctrine/spec documents, verify the current repo state, and then implement in small proof-backed increments.

Chapter 15

Portability Assurance Future Proof

15.1 Domino Dominium Portability Assurance Future Proof Architecture Human Readable Report

Source: docs/archive/conversations/Portability_Assurance_Future_Proof/Domino_Dominium_Portability_Assurance_Future_Proof_Architecture__01_human_readable_report.md

Report type: human_readable_report

Size: 33664 bytes

15.1.1 COMPLETE CHAT PRESERVATION
REPORT - Domino/Dominium
Portability, Assurance, and
Future-Proof Architecture

0. Coverage and Reliability Assessment

Row 1	Field: Chat label; Assessment: Domino/ Dominium Portability, Assurance, and Future- Proof Architecture
Row 2	Field: Date anchor; Assessment: 2026-05-27 Australia/Melbourne
Row 3	Field: Source scope; Assessment: This chat only unless explicitly labelled PROJECT-CONTEXT.
Row 4	Field: Apparent access; Assessment: Partial / visible context. I can use the visible transcript and uploaded prompt, but cannot guarantee hidden or earlier transcript access.
Row 5	Field: Previously generated files available?; Assessment: No prior downloadable files visible before this response.
Row 6	Field: Uploaded files or artifacts present?; Assessment: Yes. Pasted text.txt was uploaded and used.
Row 7	Field: Contains future plans?; Assessment: Yes. DDAP, DIL levels, contracts, conformance tests,

registries, validators, repo-structure work.

Row 8 Field: Contains decisions?; Assessment: Yes, but mostly assistant recommendations plus user-stated priorities.

Row 9 Field: Contains pending tasks?; Assessment: Yes. Architecture/validation tasks proposed; preservation package completed by this response.

Row 10 Field: Contains unresolved questions?; Assessment: Yes. Repo state, accepted architecture, first pilot module, compatibility promises.

Row 11 Field: Staleness risk; Assessment: Medium.

Row 12 Field: Extraction confidence; Assessment: 4/5 for visible chat; 2/5 for unseen full-history completeness.

Row 13 Field: Safe for later aggregation?; Assessment: With caveats.

Row 14 Field: Main limitations; Assessment: No repo inspection; no guaranteed hidden transcript access; external standards need verification.

Limitations: this package preserves visible current-chat context and the uploaded preservation prompt. It does not prove that hidden transcript segments, other

conversations, repository files, or older generated artifacts were accessible. Broader project context is not treated as current-chat fact unless labelled.

1. One-Page Orientation

This chat was mainly about turning Domino/Dominium from a one-off game project into a durable, reusable engine/product platform. It began with a question about whether standards from high-assurance domains-especially DO-178B/C and adjacent secure-development, supply-chain, and metadata standards-could be used for Domino and Dominium. The proposed answer was that the project should use standards as design inputs, not as compliance targets. The useful parts are traceability, requirements-based tests, tool-impact classification, independent review, secure development, release provenance, SBOM/license metadata, and risk-based assurance. The wrong move would be claiming or pursuing avionics-style certification for a game/engine project.

The user then widened the topic. They explicitly said that portability, modularity, extensibility, reuse, refactorability, and future-proofing are very important. They want code reusable for another game on Domino, and possibly for different engine or game projects. They also want files and directories replaceable during rewrites or major refactors. The desired quality bar is closer to a

proper game platform or OS-style system than a throwaway indie project.

The central proposed answer was that Domino should be a contract-governed engine platform, while Dominion should be one product built on it. The assistant's core doctrine was "stable outside, replaceable inside." Public contracts-headers, save formats, pack formats, replay formats, schemas, protocols, command/result APIs, capability IDs, and migration rules-should be explicit, versioned, tested, and hard to break casually. Private internals, algorithms, backend modules, and directory-internal helper files should remain replaceable.

A recurring point was that modularity is not created by folder names alone. A real module needs an owner, purpose, public contract, private implementation, declared dependencies, forbidden dependencies, fixtures, conformance tests, versioning policy, diagnostics/error policy, and replacement criteria. The replacement test is: can this directory be deleted and reimplemented against only public contracts, while passing the same conformance tests and loading the same data? If yes, the boundary is meaningful. If no, the project is probably coupled through hidden assumptions.

The chat also proposed concrete practices: public C-style API discipline; opaque handles; size/version/reserved fields in public structs; no C++ ABI across plugin boundaries; no platform handles in core APIs; stable

registries for tags, opcodes, error codes, capability IDs, and format versions; deterministic simulation isolated from OS/UI/renderers/wall-clock time; conformance, migration, fuzz, replay, and layer tests; setup as a transactional system; launcher as a control plane; and tools/UI as public-contract consumers.

The final user action uploaded a preservation-package prompt. This response turns the visible chat into a human-readable report, registers, context packet, spec sheet, aggregator packet, audit, and downloadable ZIP. The main caveat is that many technical details are assistant recommendations, not explicit user-ratified decisions, and the actual repository was not inspected. A future assistant should preserve that distinction.

2. The Story of the Conversation

Opening: standards as inspiration, not compliance

The chat began with the user pasting a standards-focused answer originally framed around Eureka and asking whether similar ideas could be used for Domino and Dominium. The answer developed into a recommendation: borrow assurance patterns, but do not claim or pursue DO-178C or similar compliance.

Expansion: from assurance to platform

architecture The user then clarified that the real

concern was broader: all code should be portable, modular, extensible, reusable, replaceable, and future-proof. This shifted the conversation from standards to the whole structure of the engine/product ecosystem.

Concrete architecture The assistant proposed treating Domino as a reusable engine/runtime/toolchain and Dominion as one product built on it. It proposed stable public contracts, replaceable internals, conformance tests, versioned data formats, deterministic simulation boundaries, explicit naming rules, and a target ownership-based repository layout.

Preservation checkpoint The user then uploaded a maximum-fidelity preservation prompt. The task became preserving this chat into a package suitable for human reading and future aggregation. This response is that package.

3. Main Topics Discussed

Topic 1 - Standards-informed assurance The chat proposed using standards such as DO-178C, NIST SSDF, OWASP ASVS, SLSA, and SPDX as design inputs only. The proposed internal version is DDAP v0 with DIL levels. Uncertainty: user has not explicitly ratified DDAP.

Topic 2 - Domino as reusable engine and

Dominium as product The user's explicit reuse requirement led to the proposed split: Domino is engine/runtime/toolchain; Dominium is one product using Domino. Domino should not depend on Dominium.

Topic 3 - Stable contracts and replaceable

internals The main doctrine was stable outside, replaceable inside. Public contracts and persistent formats are stable; internals remain free to change.

Topic 4 - Directory structure and module

ownership A target tree was proposed: include/, contracts/, source/domino/, source/dominium/, tests/conformance/, tests/migration/, tests/fuzz/, tools/validators/, docs/architecture/, docs/assurance/. This is a candidate, not a repo-verified plan.

Topic 5 - Public API/ABI discipline

The proposed rules include opaque handles, size/version/reserved fields, explicit allocator ownership, project-owned primitive types, no C++ ABI at plugin boundaries, no platform types in core APIs, and lifecycle labels.

Topic 6 - Data compatibility Saves, packs, replays, protocols, and schemas need versioning, registries,

migration tests, hashes/signatures where appropriate, and no reuse of released IDs.

Topic 7 - Determinism and replay Simulation should be isolated from wall-clock time, OS APIs, UI, renderer backends, and unordered platform-specific behavior. Replay equality should be testable.

Topic 8 - Testing and validators Conformance tests prove replaceability. High-trust paths need valid, invalid, malformed, old-version, future-version, migration, roundtrip, determinism, and negative-permission tests.

Topic 9 - Setup, launcher, tools, and UI boundaries Setup should be transactional; launcher should orchestrate; tools should use public contracts; UI should use command/result/view-model contracts.

Topic 10 - Preservation and aggregation The uploaded prompt required this preservation report, registers, spec sheet, aggregator packet, audit, files, and ZIP.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user wanted to know whether standards can be used wisely for Domino/Dominium.

The user also explicitly wanted portability, modularity, extensibility, reuse, replaceability, proper-game/OS-grade structure, better directories, better names, better APIs, better schemas, and future-proof/backward-compatible design. The uploaded prompt explicitly requested a maximum-fidelity preservation package.

4.2 Inferred Goals The user likely wants these outputs to feed a future master Project Spec Book and prevent long-chat context loss.

4.3 Goals That Changed Over Time The conversation widened from standards to full engine architecture, then shifted to preservation/export.

4.4 Goals Still Unresolved The actual repo was not inspected. The exact accepted directory structure, API policy, DDP profile, compatibility promises, and first pilot module remain unresolved.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Use standards as design inputs, not literal compliance targets.; Status: Assistant recommendation; not explicitly user-ratified.; Why it mattered: Useful assurance patterns should be borrowed; external certification

wrappers are misaligned.; Confidence: 4; Label: INFERENCE / assistant recommendation

Row 2 ID: DECISION-02; Decision: Separate Domino as reusable engine/runtime/toolchain from Dominion as one product built on it.; Status: Assistant recommendation aligned with explicit user goal.; Why it mattered: Engine/product separation prevents product assumptions contaminating reusable code.; Confidence: 5; Label: FACT goal + INFERENCE architecture

Row 3 ID: DECISION-03; Decision: Stable public contracts, replaceable internals.; Status: Assistant recommendation; not separately accepted after answer.; Why it mattered: Preserves compatibility while allowing deep refactors.; Confidence: 4; Label: assistant recommendation

Row 4 ID: DECISION-04; Decision: Do not make every internal function/header stable.; Status: Assistant recommendation.; Why it mattered: Freezing internals creates accidental compatibility burdens.; Confidence: 4; Label: assistant recommendation

Row 5 ID: DECISION-05; Decision: Use internal DIL-0 through DIL-5 levels for assurance gates.; Status: Assistant recommendation; not user-confirmed.; Why it mattered: Applies rigor only where trust/

persistence/authority/external impact exists.;
Confidence: 4; Label: assistant recommendation

Row 6 ID: DECISION-06; Decision: Prioritise assurance for saves, packs, replay, authority, updater/installer, multiplayer, and mod trust.; Status: Assistant recommendation.; Why it mattered: Failures here can corrupt data, break compatibility, or undermine trust/security.; Confidence: 4; Label: assistant recommendation

Row 7 ID: DECISION-07; Decision: Use conformance tests as proof of replaceability.; Status: Assistant recommendation.; Why it mattered: Replacement must be objectively testable.; Confidence: 4; Label: assistant recommendation

Row 8 ID: DECISION-08; Decision: Treat data evolution as seriously as code evolution.; Status: Assistant recommendation.; Why it mattered: Long-lived projects often fail through broken data compatibility.; Confidence: 4; Label: assistant recommendation

Row 9 ID: DECISION-09; Decision: Do not present this package as guaranteed full transcript extraction.; Status: Decision in this package.; Why it mattered: Preservation must be honest about source scope.; Confidence: 5; Label: FACT / UNCERTAIN scope

DECISION-01 - Use standards as design inputs, not literal compliance targets. Status: Assistant recommendation; not explicitly user-ratified.

Basis: User asked whether standards could be used for Domino/Dominium; assistant answered yes, but not as certification/compliance.

Rationale: Useful assurance patterns should be borrowed; external certification wrappers are misaligned.

Implications: Creates DDP-style internal profile rather than compliance claim.

Caveat: carry forward as INFERENCE / assistant recommendation.

DECISION-02 - Separate Domino as reusable engine/runtime/toolchain from Dominium as one product built on it. Status: Assistant recommendation aligned with explicit user goal.

Basis: User explicitly wanted reuse for another game and other engine/game projects.

Rationale: Engine/product separation prevents product assumptions contaminating reusable code.

Implications: Domino should not depend on Dominium; Dominium consumes public contracts.

Caveat: carry forward as FACT goal + INFERENCE architecture.

DECISION-03 - Stable public contracts, replaceable internals. Status: Assistant recommendation; not separately accepted after answer.

Basis: Assistant framed this as the core doctrine.

Rationale: Preserves compatibility while allowing deep refactors.

Implications: Public headers/formats/protocols/tests stable; algorithms/private modules replaceable.

Caveat: carry forward as `assistant recommendation`.

DECISION-04 - Do not make every internal function/header stable. Status: Assistant recommendation.

Basis: Assistant warned against making all internal headers public or stable.

Rationale: Freezing internals creates accidental compatibility burdens.

Implications: Need explicit public/private split and lifecycle labels.

Caveat: carry forward as `assistant recommendation`.

DECISION-05 - Use internal DIL-0 through DIL-5 levels for assurance gates. Status: Assistant recommendation; not user-confirmed.

Basis: Assistant proposed levels for experiments through

external-impacting systems.

Rationale: Applies rigor only where trust/persistence/authority/external impact exists.

Implications: DIL-3+ receives stronger evidence/tests/review; UI/content stays lighter.

Caveat: carry forward as **assistant recommendation**.

DECISION-06 - Prioritise assurance for saves, packs, replay, authority, updater/installer, multiplayer, and mod trust. Status: Assistant recommendation.

Basis: Assistant identified highest-value targets.

Rationale: Failures here can corrupt data, break compatibility, or undermine trust/security.

Implications: These paths need versioning, validation, negative tests, review gates, audit evidence.

Caveat: carry forward as **assistant recommendation**.

DECISION-07 - Use conformance tests as proof of replaceability. Status: Assistant recommendation.

Basis: Assistant stated stable interface without conformance suite is only a promise.

Rationale: Replacement must be objectively testable.

Implications: tests/conformance becomes first-class repo

area.

Caveat: carry forward as **assistant recommendation**.

DECISION-08 - Treat data evolution as seriously as code evolution. Status: Assistant recommendation.

Basis: Assistant emphasized saves/packs/schemas/protocols and no tag reuse.

Rationale: Long-lived projects often fail through broken data compatibility.

Implications: Need registries, fixtures, migration tests, compatibility policy.

Caveat: carry forward as **assistant recommendation**.

DECISION-09 - Do not present this package as guaranteed full transcript extraction. Status: Decision in this package.

Basis: Visible context and uploaded prompt are available; hidden transcript/repo are not guaranteed.

Rationale: Preservation must be honest about source scope.

Implications: Safe for aggregation only with caveats.

Caveat: carry forward as **FACT / UNCERTAIN** scope.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

REJECTED-01 - Claim or pursue DO-178C compliance for Domino/Dominium. Status: Rejected by assistant recommendation.

Reason: Misaligned with game/engine project and implies certification obligations.

Reconsideration: Only reconsider for regulated safety-critical external use.

REJECTED-02 - Apply heavy assurance to every line of code and all gameplay/content/UI. Status: Rejected/deprioritised by assistant recommendation.

Reason: Would slow creative iteration and overburden low-risk work.

Reconsideration: Reconsider only if subsystem becomes trust-bearing.

REJECTED-03 - Freeze all internal APIs/functions for compatibility. Status: Rejected by assistant recommendation.

Reason: Internal stability preserves bad designs and blocks refactoring.

Reconsideration: Only for intentionally public/plugin APIs.

REJECTED-04 - Use directory structure alone as proof of modularity. Status: Rejected implicitly.

Reason: Folders need contracts, dependencies, tests, replacement criteria.

Reconsideration: None; structure remains necessary but insufficient.

REJECTED-05 - Persist data by dumping raw C structs. Status: Rejected by assistant recommendation.

Reason: Not portable across compilers/endian/alignment/schema evolution.

Reconsideration: Only for temporary debug snapshots marked nonportable.

REJECTED-06 - Let setup/launcher/gameplay/tooling freely share privileged internals. Status: Rejected by assistant recommendation.

Reason: Collapses boundaries and makes refactors/security/audits harder.

Reconsideration: Internal diagnostic privileged mode only if explicit.

7. Important Reasoning, Rationale, and Tradeoffs

The main tradeoff was between long-term rigor and practical development. The user wants platform/OS-grade discipline, but Domino/Dominium is not an avionics certification program. The assistant therefore proposed borrowing assurance practices only where they manage real project risk: saves, packs, replay, release integrity, authority boundaries, installers, mod trust, and multiplayer.

Another tradeoff was between stability and refactorability. Stability is necessary for reuse and backward compatibility, but too much stability too early freezes bad design. The answer therefore separated stable public contracts from replaceable private internals.

A third tradeoff was between folder organization and real modularity. The assistant argued that folder names are secondary to contracts, dependency rules, tests, fixtures, and replacement criteria.

8. Plans, Future Work, and Next Steps

Recommended next-action sequence:

1. Ratify or revise the doctrine: standards-informed not compliance-claiming; Domino separate from Dominium; stable outside/replaceable inside; data compatibility as first-class.

2. Inspect the actual repo tree, headers, build files, schemas, formats, tests, and dependency leaks.
3. Define public/private/experimental/stable/deprecated boundaries.
4. Draft public-api-policy.md, data-evolution-policy.md, layering.md, and ddap.md.
5. Add registries for stable IDs and formats.
6. Choose one conformance-test pilot, preferably pack validator or save reader.
7. Add layer checker and schema validators.
8. Build migration and malformed-input fixtures.
9. Use DIL gates only for trust-bearing paths.
10. Feed this package into the master Project Spec Book with caveats.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences The user explicitly required human-readable preservation, uncertainty labels, no invented facts, no silent inference, and no treating assistant recommendations as user decisions. The user explicitly values portability, modularity, extensibility, reuse, replaceability, future-proofing, and proper long-lived engineering.

9.2 Inferred Constraints and Preferences The user likely wants spec-book-ready artifacts and dislikes compressed summaries that lose rationale.

9.3 Uncertain or Unestablished Preferences It is not yet established whether the user accepts the exact DDAP/DIL structure, exact directory tree, exact naming prefixes, or exact C89/C++98 enforcement scope.

10. Files, Artifacts, Outputs, and Prompts

Row 1 ID: ARTIFACT-01; Artifact / file / prompt / output: User-pasted standards/DO-178B/C/Eureka analysis; Type: prompt content; Purpose: Frame whether standards-inspired assurance could apply to Domino/Dominium.; Status: Referenced; not independently verified here.; Origin: User message; Carry forward?: Yes, as background and verification input.; Notes: Contains external standards claims/links needing verification.; Label: FACT / VERIFY

Row 2 ID: ARTIFACT-02; Artifact / file / prompt / output: Assistant response: DDAP/DIL standards-informed assurance answer; Type: generated chat output; Purpose: Proposed project-native assurance profile and high-trust paths.; Status: Produced; recommendations not explicitly ratified.;

Origin: Assistant; Carry forward?: Yes, with tentative label.; Notes: Includes DIL levels, repo impact, AIDE enforcement, boundary conditions.; Label: assistant recommendation

Row 3 ID: ARTIFACT-03; Artifact / file / prompt / output: User question on portability, modularity, extensibility, replaceability, future-proofing; Type: user instruction / design goal; Purpose: Defines main architectural concern of the chat.; Status: Explicit user requirement/preference.; Origin: User message; Carry forward?: Yes, high priority.; Notes: Core source for platform doctrine.; Label: FACT

Row 4 ID: ARTIFACT-04; Artifact / file / prompt / output: Assistant response: contract-governed engine platform answer; Type: generated chat output; Purpose: Proposed stable contracts, replaceable internals, repo tree, naming, schema, determinism, testing.; Status: Produced; recommendations not explicitly ratified.; Origin: Assistant; Carry forward?: Yes, as architecture draft.; Notes: Contains many candidate requirements/tasks.; Label: assistant recommendation

Row 5 ID: ARTIFACT-05; Artifact / file / prompt / output: Pasted text.txt preservation mega-prompt; Type: uploaded file; Purpose: Instructs assistant to

870

create complete preservation package and ZIP.;
Status: Loaded and used.; Origin: User upload;
Carry forward?: Yes, as method/template.; Notes:
Source for current preservation task.; Label: FACT

Row 6 ID: ARTIFACT-06; Artifact / file / prompt /
output: Generated preservation package files;
Type: generated files; Purpose: Preserve this chat
in human-readable and machine-aggregatable form.
; Status: Created by this response.; Origin:
Assistant file export; Carry forward?: Yes.; Notes:
Includes manifest, report, context packet, spec
sheet, registers, aggregator packet, reader brief,
audit, future-chat prompt, in-chat reader, ZIP.;
Label: FACT

11. Open Questions and Unresolved Issues

QUESTION-01 - Is DDAP v0 accepted as project direction or only a recommendation pending user review? Why it matters: Determines whether DIL levels become canonical governance.

Known: Assistant recommended DDAP; user has not explicitly accepted it.
Unknown: User’s final acceptance and desired strictness.
Resolution path: Ask user to approve, revise, or reject DDAP.

QUESTION-02 - What is the current actual repo structure? Why it matters: Directory

recommendations were not based on repo inspection.

Known: Only proposed structures and PROJECT-CONTEXT memories are available.

Unknown: Actual files, build scripts, and current coupling.

Resolution path: Inspect repo or have user provide tree.

QUESTION-03 - Which public boundaries must be stable first? Why it matters: Stabilizing too much too early freezes bad design; too little undermines portability.

Known: Assistant recommended public headers, formats, protocols, command/result APIs.

Unknown: Which systems are mature enough to freeze.

Resolution path: Define experimental/stable/deprecated lifecycle policy and boundary list.

QUESTION-04 - How strict should C89/C++98 portability be across all code? Why it matters:

Portability rules depend on toolchains/platforms.

Known: PROJECT-CONTEXT says C89 engine and C++98 shells; current user asked portability but did not restate versions.

Unknown: Whether all current work enforces this now.

Resolution path: User confirms target platform/
toolchain matrix.

QUESTION-05 - What should be the first pilot module for conformance/replacement testing?

Why it matters: A concrete pilot prevents abstract architecture only.

Known: Assistant suggested pack validator, save reader, or render.soft.

Unknown: Which module exists and is easiest to isolate.

Resolution path: Choose one module and write module.
toml + fixtures + tests.

QUESTION-06 - What exact compatibility promises should be made for saves, packs, mods, and protocols? Why it matters: Compatibility guarantees define future burden and user trust.

Known: Assistant recommended versioning and no ID reuse.

Unknown: Support window, migration guarantees, breaking-change process.

Resolution path: Write compatibility policy with tiers.

QUESTION-07 - Which external standards claims need verification before formal spec use? Why it matters: Standards change and some references are high-level/paywalled.

Known: Prior assistant cited several standards and practices.

Unknown: Current exact wording/applicability.

Resolution path: Verify against official/current sources before normative text.

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

RISK-01 - Future assistant treats recommendations as user-approved decisions.

Consequence: Spec book may encode unratified advice as canon.

Likelihood/severity: Medium / High

Mitigation: Keep status labels; ask user to approve/reject DDAP and target structure.

RISK-02 - Architecture becomes certification cosplay. Consequence: Paperwork without real assurance; slowed development.

Likelihood/severity: Medium / Medium

Mitigation: Use project-native DIL levels; restrict heavy

gates to trust paths.

RISK-03 - Portability remains aspirational without conformance tests. Consequence: Modules cannot actually be replaced; hidden coupling accumulates.

Likelihood/severity: High / High

Mitigation: Create tests/conformance and replacement criteria.

RISK-04 - Data formats evolve without registries and migration tests. Consequence: Old saves/mods/packs/replays break or corrupt silently.

Likelihood/severity: Medium / High

Mitigation: No-ID-reuse, registries, golden fixtures, migration tests.

RISK-05 - Dominium-specific assumptions leak into Domino. Consequence: Domino becomes un reusable.

Likelihood/severity: Medium / High

Mitigation: Layer checker; no Domino->Dominium dependencies.

RISK-06 - Public API is frozen too early.

Consequence: Bad early designs become compatibility burdens.

Likelihood/severity: Medium / Medium

Mitigation: Use experimental/stable/deprecated lifecycle labels and promotion reviews.

RISK-07 - External standards claims become stale/inaccurate. Consequence: Spec book cites incorrect/outdated standards framing.

Likelihood/severity: Medium / Medium

Mitigation: Verification queue before normative incorporation.

RISK-08 - Preservation package overstates access.

Consequence: Aggregator assumes complete extraction when only visible context was available.

Likelihood/severity: Medium / Medium

Mitigation: State coverage limitations clearly.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat contributes architecture doctrine, assurance framing, API/ABI policy candidates, data-compatibility doctrine, conformance testing requirements,

determinism/replay principles, and setup/launcher/tool/UI boundary principles. It should feed future spec-book chapters but should not be merged as final canon without checking recommendation status and actual repo state.

14. What I Should Remember

- The explicit user requirement is portable, modular, extensible, reusable, replaceable, future-proof code.
- The strongest proposed doctrine is stable public contracts with replaceable internals.
- Domino should be reusable engine/runtime/toolchain; Dominium should be one product using it.
- Modularity must be proven by conformance tests, not just folders.
- Data compatibility is as important as code compatibility.
- Saves, packs, replays, schemas, protocols, IDs, and migrations need policy from the beginning.
- Standards should inform the project but not become compliance claims.
- DDAP/DIL is proposed, not yet user-ratified.

- The actual repo was not inspected.
- The best next action is repo boundary audit plus first policy/registry/conformance pilot.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

1. Explain stable-outside/replaceable-inside in simpler terms.
2. Which recommendations are strongest and which are speculative?

15.2 Decisions

1. Which recommendations should become project canon?
2. Should DDAP/DIL be accepted, revised, or rejected?

15.3 Tasks and Next Actions

1. Convert the task register into a first-month implementation plan.
2. Pick the best first conformance-test pilot.

15.4 Artifacts and Files

1. Which generated file should I use for a future chat bootstrap?
2. Extract only formal requirement candidates.

15.5 Risks and Verification

1. What could go wrong if we freeze public APIs too early?
2. Which standards claims need verification?

15.6 Future Spec Book / Aggregation

1. Which spec-book chapters should this feed into?
2. What overlaps with other Dominion chats?

15.7 Deep-Dive Questions Specific to This Chat

1. Draft public-api-policy.md.
2. Draft data-evolution-policy.md.
3. Draft ASSURANCE_PROFILE.md.
4. Turn the "20 laws" into CI checks.

16. Compact Human Summary

This chat is a major architecture checkpoint for Domino/Dominium. It first asked whether standards such as DO-178B/C and related security/supply-chain frameworks should influence the project. The proposed answer was yes, but only as design input. Domino/Dominium should not claim aviation, automotive, medical, or industrial compliance. It should borrow useful patterns: traceability, requirements-based tests, tool-impact classification, review independence, secure-development practices, release provenance, SBOM/license metadata, and risk-based assurance.

The internal version proposed was DDAP v0, using DIL levels from low-risk experiments to external-impacting systems. Most ordinary game/content/UI work should stay light. Stronger controls should apply to trust-bearing paths such as saves, packs, mod loading, replay, authority/capability logic, installers/updaters, signed releases, multiplayer authority, native/offline clients, and automated execution. This is a recommendation, not yet a user-ratified decision.

The user then clarified the deeper issue: all code should be portable, modular, extensible, reusable, replaceable, and future-proof. The user wants Domino reused for other games and possibly other engine/game projects. They want files and directories replaceable during major

rewrites. The desired standard is a proper long-lived platform, closer to OS-grade discipline than a one-off indie project.

The assistant's strongest doctrine was stable outside, replaceable inside. Public contracts and persistent formats should be stable, versioned, documented, and tested. Internal implementation code should remain free to change. Public contracts include headers, save formats, pack formats, replay formats, protocols, schemas, command/result APIs, capability IDs, error codes, and migration rules. Private internals include helper files, algorithms, backend details, and non-public structs.

The conversation emphasized that directory structure is not enough. A real module needs an owner, purpose, public contract, private implementation, declared dependencies, forbidden dependencies, fixtures, conformance tests, versioning policy, diagnostics/error policy, and replacement criteria. The replacement test is whether a directory can be deleted and reimplemented against only public contracts while still passing the same tests and loading the same data.

Several concrete repo ideas were proposed: include/domino, include/dominium, contracts, source/domino, source/dominium, tests/conformance, tests/migration, tests/fuzz, tools/validators, docs/architecture, and docs/assurance. These are proposals; the actual repo was not

inspected.

API/ABI discipline was a major theme: opaque handles, desc structs with size/version/reserved fields, explicit allocator ownership, project-owned integer types, no platform handles in core APIs, no exposed private structs, no public bitfields, no C++ ABI across plugin boundaries, no global singleton state, and lifecycle labels such as experimental, stable, deprecated, internal, and removed.

Persistent data compatibility was treated as central. The assistant rejected raw struct dumps for long-lived saves. Instead it recommended versioned, endian-explicit, chunked/TLV-based formats with magic values, format IDs, writer versions, minimum reader versions, schema IDs, chunk tables, hashes, optional signatures, migration paths, unknown-field policies, and reserved ID registries. Released IDs should never be reused.

Testing was framed as the proof of modularity. Unit tests are insufficient. The project needs contract tests, conformance tests, golden tests, replay tests, migration tests, fuzz tests, fault-injection tests, platform tests, layer tests, build-matrix tests, performance budgets, and security tests. High-trust paths require valid, invalid, old-version, future-version, malformed, roundtrip, migration, determinism, and negative-permission fixtures.

The uploaded prompt requested this preservation package. The main caveat is that the package preserves visible chat context and the uploaded prompt, not a guaranteed hidden full transcript or inspected repository. The best next action is to inspect the actual repo and turn the strongest recommendations into a small set of enforceable policies and one conformance-test pilot.

15.2 Domino Dominion Portability Assurance Future Proof Architecture Reader Brief

Source: docs/archive/conversations/Portability_Assurance_Future_Proof/Domino_Dominium_Portability_Assurance_Future_Proof_Architecture__06_reader_brief.md

Report type: reader_brief

Size: 2099 bytes

15.2.1 Reader Brief - Domino/Dominium Portability, Assurance, and Future-Proof Architecture

What this chat was about

Domino/Dominium future-proof architecture: standards-informed assurance, portability, modularity, extensibility,

883

replaceability, public contracts, data compatibility, deterministic replay, testing, and preservation.

Top 20 things to know

- The explicit user requirement is portable, modular, extensible, reusable, replaceable, future-proof code.
- The strongest proposed doctrine is stable public contracts with replaceable internals.
- Domino should be reusable engine/runtime/toolchain; Dominium should be one product using it.
- Modularity must be proven by conformance tests, not just folders.
- Data compatibility is as important as code compatibility.
- Saves, packs, replays, schemas, protocols, IDs, and migrations need policy from the beginning.
- Standards should inform the project but not become compliance claims.
- DDAP/DIL is proposed, not yet user-ratified.
- The actual repo was not inspected.
- The best next action is repo boundary audit plus first policy/registry/conformance pilot.

Additional points: DDAP/DIL is proposed; repo not inspected; conformance tests prove modularity; data compatibility must be formalized; generated output needs validation; future aggregation must preserve recommendation-vs-decision labels.

Decisions

See `04_registers.md`, especially DECISION-01 through DECISION-09.

Pending tasks

Highest priority: TASK-02, TASK-03, TASK-04, TASK-05, TASK-06, TASK-10.

Open questions

Main unresolved issues: DDAP acceptance, actual repo structure, stable boundary list, toolchain strictness, first conformance pilot, compatibility guarantees.

Artifacts

Primary artifacts: user standards paste, assistant DDAP answer, user portability question, assistant architecture answer, uploaded preservation prompt, generated files.

Verification items

Verify standards, repo structure, toolchain targets, persistent formats, and cross-chat conflicts.

Best next step

Inspect the actual repo and create a boundary audit.

15.3 Domino Dominion Portability Assurance Future Proof Architecture In Chat Reader

Source: docs/archive/conversations/Portability_Assurance_Future_Proof/Domino_Dominium_Portability_Assurance_Future_Proof_Architecture__09_in_chat_reader.md

Report type: in_chat_reader

Size: 2501 bytes

15.3.1 In-Chat Reader - Domino/Dominium Portability, Assurance, and Future-Proof Architecture

Package overview

This package preserves the visible current chat about standards-informed assurance, portability, modularity,

extensibility, replaceability, directory/API/schema/protocol design, and future-proof Domino/Dominium architecture.

File index

- `00_manifest.md` - package map and caveats.
- `01_human_readable_report.md` - sections 0-16.
- `02_context_transfer_packet.md` - future-chat bootstrap context.
- `03_spec_sheet.yaml` - structured aggregation spec.
- `04_registers.md` - workstreams, decisions, tasks, constraints, preferences, questions, artifacts, rejected options, risks, verification queue, timeline, spec contributions.
- `05_aggregator_packet.md` - merge packet.
- `06_reader_brief.md` - short human reference.
- `07_verification_and_audit.md` - self-audit and verification.
- `08_future_chat_bootstrap_prompt.md` - standalone future-chat prompt.
- `09_in_chat_reader.md` - this guide.

Plain-English explanation

The chat says Domino/Dominium should be treated as a reusable platform: stable public contracts, replaceable internals, standards-informed assurance, formal data compatibility, and conformance tests.

Question menu

Ask about DDAP, DIL levels, API policy, data-evolution policy, repo structure, conformance tests, deterministic replay, setup/launcher boundaries, or spec-book integration.

Top things to preserve

- The explicit user requirement is portable, modular, extensible, reusable, replaceable, future-proof code.
- The strongest proposed doctrine is stable public contracts with replaceable internals.
- Domino should be reusable engine/runtime/toolchain; Dominium should be one product using it.
- Modularity must be proven by conformance tests, not just folders.
- Data compatibility is as important as code compatibility.

- Saves, packs, replays, schemas, protocols, IDs, and migrations need policy from the beginning.
- Standards should inform the project but not become compliance claims.
- DDAP/DIL is proposed, not yet user-ratified.
- The actual repo was not inspected.
- The best next action is repo boundary audit plus first policy/registry/conformance pilot.

Safest next actions

1. Read the human-readable report.
2. Review the decision register and mark accepted/rejected/needs revision.
3. Inspect the actual repo.
4. Draft public API and data evolution policies.
5. Choose one conformance-test pilot.

15.4 Domino Dominion Portability Assurance Future Proof Architecture Accompanying Detailed Summary Report

Source: docs/archive/conversations/Portability_Assurance_Future_Proof/Domino_Dominium_Portability_Assurance_Future_Proof_Architecture__10_accompanying_detailed_summary_report.md

Report type: detailed_summary_report

Size: 33375 bytes

15.4.1 Accompanying Human-Readable Detailed Summary and Report

Chat label: Domino/Dominium Portability, Assurance, and Future-Proof Architecture

Date anchor: 2026-05-27 Australia/Melbourne

Generated: 2026-05-27 21:02:29 AEST

Source scope: This report covers the visible current chat plus the generated preservation files currently available in /mnt/data. It does not claim access to hidden model reasoning, a full external repository, or other old chats unless explicitly labelled as PROJECT-CONTEXT.

1. Executive summary

This conversation was about turning Domino and Dominionium from a one-off game project into a long-lived, reusable, portable, modular engine-and-product ecosystem. The user's core requirement was not merely "clean code." The user wanted code and data that can survive major rewrites, directory replacement, engine reuse, new games, new platforms, and future refactors without collapsing into project-specific coupling.

The conversation first addressed whether high-assurance standards such as DO-178B/C and adjacent security/supply-chain standards could inform Domino and Dominionium. The answer was that such standards are useful as design inputs, but not as literal compliance targets. The proposed approach was a project-native assurance profile: standards-informed, but not certification cosplay. This became the proposed **Domino/Dominium Assurance Profile** concept, with internal integrity levels rather than aviation labels. The discussion then moved into a broader architecture doctrine. The central idea was:

**Stable public contracts; replaceable
internals.**

This means Domino should expose durable, versioned, tested contracts at the boundaries that matter: public C

APIs, save formats, pack formats, replay formats, schemas, protocol IDs, command/result APIs, error codes, capability IDs, and migration rules. It does **not** mean every internal function, file, or helper should be frozen. A future-proof system must be strict where external compatibility matters and flexible where internal redesign is valuable.

The conversation also identified the need for conformance tests, schema evolution rules, deterministic simulation boundaries, platform abstraction, render abstraction, transactional setup/update behavior, launcher separation, toolchain discipline, and a hierarchy between Domino as engine/platform and Dominion as one product built on that engine.

A later uploaded preservation prompt requested a maximum-fidelity preservation package for the chat. A full package was generated with ten files: manifest, human-readable report, context transfer packet, YAML spec sheet, registers, aggregator packet, reader brief, verification/audit file, future-chat bootstrap prompt, and in-chat reader. This accompanying report is now being added to that set and bundled into a consolidated ZIP.

The most important unresolved point is that the proposed architecture is still a **candidate doctrine**, not a completed repo migration. The actual repository was not inspected in this chat, and many assistant outputs are recommendations rather than user-ratified decisions.

The next proper step is to inspect the repo, identify the real coupling points, decide which recommendations become canonical, and draft enforceable policies for public APIs, data evolution, module boundaries, and conformance tests.

2. Coverage and reliability check

- Row 1** Field: Apparent chat access; Assessment: Partial-to-visible context only. The report covers what is visible in this chat and the generated files available in `/mnt/data`.
- Row 2** Field: Repository access; Assessment: No actual Domino/Dominium repository was inspected.
- Row 3** Field: Uploaded files; Assessment: Yes. The user uploaded a preservation/export prompt as `Pasted text.txt`.
- Row 4** Field: Previously generated files; Assessment: Yes. Ten preservation package files and one prior handoff ZIP existed before this companion report.
- Row 5** Field: Current new files; Assessment: This companion report, a consolidated bundle manifest, checksums, and a new complete ZIP.
- Row 6** Field: Contains explicit user goals; Assessment: Yes. Portability, modularity, extensibility,

replaceability, future-proofing, OS-grade/game-engine-grade development, and preservation/export.

Row 7 Field: Contains assistant recommendations; Assessment: Yes. DDAP/DIL, target directory tree, API rules, schema policy, tests, module contracts, and next actions.

Row 8 Field: Contains final decisions; Assessment: Some explicit user priorities are clear; most technical architecture items remain recommendations until accepted by the user.

Row 9 Field: Staleness risk; Assessment: Medium. Standards references and external claims should be verified before being placed in formal specs.

Row 10 Field: Extraction confidence; Assessment: 4/5 for visible conversation and generated files; lower for anything outside visible context.

Row 11 Field: Safe for aggregation; Assessment: Yes, with caveats. Keep FACT / INFERENCE / UNCERTAIN status and do not convert suggestions into ratified decisions.

Plain-language limitation statement This report is reliable for explaining what was discussed in the visible chat. It is not proof that the real repository has the

recommended structure, not proof that the user formally accepted every recommendation, and not a substitute for verifying the current official standards or inspecting actual code. Future aggregation should treat this as a design-intent and architecture-doctrine package, not as a completed implementation record.

3. Chronological story of the conversation

3.1 Standards and assurance question The conversation began with the user presenting a long assessment of whether Eureka should borrow from DO-178B/C and related standards. The pasted assessment argued that the right approach is not full DO-178C compliance, but a DO-178C-inspired assurance layer for high-trust paths. It also corrected several overstatements about DO-178C, such as treating it as simply "stricter everywhere" or as a proof of correctness.

The user then asked whether something similar should be used for Domino and Dominium, and whether it was wise.

The assistant's answer was that it is wise only if used as **standards-informed design discipline**, not as a literal certification regime. The recommended doctrine was that Domino/Dominium should borrow from DO-178C-style traceability and verification, NIST SSDF secure-development practices, OWASP ASVS web/API

security verification, SLSA build provenance, SPDX/SBOM metadata, and functional-safety thinking where the project creates trust, authority, persistence, execution, or external impact.

This led to the proposed **DDAP v0 - Domino/Dominium Assurance Profile**. It defined internal levels such as DIL-0 exploratory through DIL-5 external-impacting. It also separated Domino as an engine/toolchain substrate from Dominium as the product/game/content ecosystem using that substrate.

3.2 User's deeper portability/modularity requirement The user then clarified that the core concern was broader than assurance standards. They wanted all code to be portable, modular, extensible, reusable, replaceable, and future-proof. They wanted the codebase to support different games on the same Domino engine and also reuse modules in entirely different engine or game projects.

The user also emphasized that code, data, files, and directories should be replaceable or rewriteable during future refactors. The project should be developed like a proper game or OS, not like a one-off indie project.

The assistant responded with the main architectural doctrine:

Stable outside, replaceable inside.

This was framed as the central rule for the project. Public contracts, save/pack/replay formats, command protocols, schemas, and tool outputs should be stable and versioned. Private implementation details should remain free to change.

3.3 Architecture, naming, directory, API, and data-evolution recommendations The assistant then proposed a much broader set of practices.

The most important architectural recommendation was to structure the repo around ownership and replacement boundaries. Domino should remain reusable and product-independent. Dominium should use Domino through public contracts and should not become entangled with Domino internals. The proposed tree included `include/`, `contracts/`, `source/domino/`, `source/dominium/`, `data/`, `state/`, `tests/`, `tools/`, `scripts/`, `docs/`, `examples/`, `external/`, `cmake/`, `release/`, and `dist/`.

The answer also recommended clear layering: contracts first, public headers next, Domino core and subsystem implementations below that, then Dominium product code. It emphasized dependency direction and suggested layer-checking scripts to prevent accidental reverse dependencies.

The API guidance focused on C-compatible public boundaries: opaque handles, explicit `size` and `version`

fields in public structs, project-owned integer types, explicit allocator ownership, no platform types in core APIs, no exposed private structs, no bitfields in public ABI, no C++ ABI across plugin boundaries, and no hidden global singleton state.

Naming guidance recommended prefixes such as `dom_`, `dcore_`, `dsys_`, `dgfx_`, `dui_`, `dsim_`, `dpack_`, `dnet_`, `dreplay_`, `dser_`, and `dtlv_`. It warned against names such as `utils.c`, `helpers.c`, `misc.c`, `manager.c`, `new_render.c`, and `legacy_render.c`.

Data-format guidance emphasized that persistent formats must include magic values, format IDs, versions, writer version, minimum reader version, endianness, schema/profile IDs, chunk tables or TLV registries, checksums, optional signatures, migrations, unknown-field policies, and reserved tag registries. The key rule was that no tag, enum value, error code, protocol opcode, save chunk ID, or capability ID should ever be reused after release.

3.4 Testing, determinism, tooling, and OS-grade discipline The conversation then moved into testing and maintainability. The assistant argued that every stable boundary needs conformance tests. A boundary is not really replaceable unless another implementation can satisfy the same public API, fixtures, golden outputs, invalid/malformed cases, and migration tests.

The answer also emphasized deterministic simulation: fixed tick rates, named RNG streams, no wall-clock calls in simulation, deterministic serialization order, deterministic replay logs, and clear separation between truth state, perceived state, view model, render commands, and backend presentation.

Setup, launcher, tools, and UI were separated conceptually:

- **Setup** should be transactional and auditable: plan, stage, verify, commit, receipt, rollback.
- **Launcher** should orchestrate products, instances, profiles, packs, diagnostics, and launching, without containing gameplay logic.
- **Tools** should consume the same public contracts as runtime code.
- **UI** should submit commands and receive results rather than directly mutating authoritative game state.

3.5 Preservation package generation The user then uploaded a preservation/export prompt requesting a maximum-fidelity preservation package for this chat. The prompt required a human-readable report, structured registers, spec sheet, aggregator packet, self-audit, file exports, and a ZIP.

The assistant created the package with the inferred label:

**Domino/Dominium Portability,
Assurance, and Future-Proof
Architecture**

The generated files were:

1. 00_manifest.md
2. 01_human_readable_report.md
3. 02_context_transfer_packet.md
4. 03_spec_sheet.yaml
5. 04_registers.md
6. 05_aggregator_packet.md
7. 06_reader_brief.md
8. 07_verification_and_audit.md
9. 08_future_chat_bootstrap_prompt.md
10. 09_in_chat_reader.md
11. handoff_package.zip

The user then asked for an accompanying human-readable detailed summary/report of the entire conversation, including what was discussed, decided,

deferred, and any missing considerations, and asked for everything to be bundled into one ZIP. This current report and bundle fulfill that request.

4. What was actually decided versus only proposed

A key preservation issue is separating user decisions from assistant recommendations.

4.1 Clear user-stated requirements The user explicitly cared about:

- code portability;
- modularity;
- extensibility;
- reuse across multiple Domino-engine games;
- reuse across completely different engine or game projects;
- ability to replace or rewrite code, data, files, and directories during major refactors;
- developing the project like a proper game/platform/OS-style system rather than a one-off indie project;

- future-proofing;
- backward compatibility;
- directory structure, file names, function names, schemas, protocols, and APIs;
- broad/deep reasoning about what was missing;
- preserving the chat and generated outputs for future aggregation and a master Project Spec Book.

These are FACTS from the visible user messages.

4.2 Strong assistant recommendations not yet fully ratified The following were assistant recommendations. They should not be treated as final canon unless the user later accepts them:

- Create **DDAP v0** as a Domino/Dominium Assurance Profile.
- Use internal **DIL-0 through DIL-5** integrity levels.
- Adopt the doctrine **stable public contracts, replaceable internals**.
- Use **source/domino/** for reusable engine/platform code and **source/dominium/** for product/game code.

- Add **contracts**/, conformance tests, migration tests, fuzz tests, and validators.
- Use C89-style public ABI boundaries where appropriate.
- Use opaque handles and **size/version** public structs.
- Define registries for tags, IDs, error codes, capabilities, opcodes, and subsystem IDs.
- Make **save/pack/replay/protocol** formats explicitly versioned and migration-tested.
- Build setup as transactional and launcher as orchestration only.
- Enforce architecture using layer-check scripts, public-header checks, schema-evolution checks, and traceability checks.

These recommendations are highly aligned with the user's stated goals, but they remain provisional until accepted and adapted to the actual repository.

4.3 Items explicitly deferred or not completed

The following were put off for later or remain unresolved:

- Inspecting the actual Domino/Dominium repository.

- Verifying the proposed directory tree against real files.
- Deciding which proposed architecture policies become project canon.
- Verifying current official details of external standards before including them in formal docs.
- Choosing final naming conventions.
- Creating actual `docs/architecture/*.md` policy files.
- Creating actual `contracts/` schemas and registries.
- Implementing layer-check, schema-evolution, conformance, fuzz, and replay test scripts.
- Deciding where C89, C++98, or other language constraints apply exactly.
- Defining the real platform matrix.
- Defining the first conformance-test pilot.
- Determining how much backward compatibility to promise at each stage of project maturity.
- Aggregating this chat with other old chats into a master Project Spec Book.

5. Main architecture doctrine preserved from the conversation

5.1 Stable public contracts; replaceable internals

This is the most important architecture doctrine. It means the project should deliberately distinguish between public contracts and private implementation. Public contracts should be rare, well-designed, versioned, documented, tested, and hard to break. Internals should be easy to delete and rewrite.

A practical replacement test was proposed:

Can we delete this directory and reimplement it against only public contracts, then pass the same conformance tests and load the same data?

If yes, the system is modular. If no, coupling has leaked through private assumptions.

5.2 Domino as reusable platform, Dominium as product Domino should own reusable engine/platform concepts: deterministic simulation, public ABI boundaries, platform abstraction, render command abstraction, data/pack/replay systems, mod/package validation, and conformance targets.

Dominium should own product-specific game rules, content, UI composition, launcher policy, setup/install

plans, server/client/headless modes, and game-specific tooling.

The boundary matters because a reusable engine dies if its flagship game infects its internals with product-specific assumptions.

5.3 Contracts before implementation The conversation emphasized that real modularity is created by contracts, not folders. A folder is not a module unless it has a purpose, owner, public API, private implementation, dependency declaration, conformance tests, fixtures, replacement criteria, versioning policy, diagnostics/error policy, and migration policy if relevant.

5.4 Data compatibility is equal to code compatibility The chat made clear that saves, packs, replays, protocols, and content schemas are long-term project assets. Bad data evolution can destroy portability even if the code is clean.

The preserved rule is:

No released ID is reused. Removed IDs are reserved forever. Old data gets migration tests.

5.5 Testing is the enforcement mechanism The conversation treated tests not merely as bug-finding tools

but as architectural enforcement. The recommended test classes included unit tests, contract tests, conformance tests, golden tests, replay tests, migration tests, fuzz tests, fault injection, platform tests, layer tests, build-matrix tests, performance budgets, and security tests.

5.6 Assurance should scale by risk The standards discussion produced a risk-scaled approach. Ordinary UI, lore, concept art, and experiments should not carry heavy assurance burdens. Saves, packs, replay, deterministic state, installers, updaters, signed releases, mod loading, multiplayer authority, and external-impacting features should receive stronger controls.

6. What "big systems" contributed to the reasoning

The conversation used several large-system lessons as analogies:

- **Linux-like lesson:** stable external interfaces, flexible internals.
- **SQLite-like lesson:** trust comes from aggressive testing of observable behavior, malformed inputs, failure modes, and old data.
- **Protocol Buffers / FlatBuffers-like lesson:** schema evolution rules must be established early

and IDs must not be reused.

- **Google-like lesson:** small, reviewable changes are better than huge uncontrolled refactors.
- **LLVM-like lesson:** large codebases need local style discipline and avoid unrelated mass churn.
- **DO-178C-like lesson:** traceability, verification evidence, and independent review are valuable for high-trust paths, but aviation certification itself is not the goal.
- **NIST SSDF / OWASP ASVS / SLSA / SPDX-like lesson:** security, web/API verification, build provenance, and SBOM/component metadata are useful where relevant.

These were not presented as external compliance obligations. They were used as design inspiration.

7. Directory structure and naming conclusions

The assistant proposed a target repo structure with major roots such as:

```
include/  
contracts/  
source/domino/  
source/dominium/  
data/
```

```
state/  
tests/  
tools/  
scripts/  
docs/  
examples/  
external/  
cmake/  
release/  
dist/
```

The most important structural point was not the exact names. It was the separation of stable contracts, reusable engine code, product-specific game code, data, tools, tests, and documentation.

The naming recommendations emphasized ownership and role. Names like `utils.c`, `helpers.c`, `misc.c`, and `manager.c` were discouraged because they hide responsibility and usually become dumping grounds.

More explicit names like `d_tlv_reader.c`, `d_pack_validate.c`, `d_replay_verify.c`, `d_gfx_command_buffer.c`, and `d_sys_win32_window.c` were proposed.

Function naming should be consistent and subsystem-oriented, such as:

```
dpack_manifest_parse  
dpack_manifest_validate  
dpack_manifest_write  
dreplay_log_open  
dreplay_log_append
```

```
dgfx_command_buffer_begin
dgfx_command_buffer_push_rect
```

The open issue is whether these exact prefixes and names should become canon. That requires repo inspection and user acceptance.

8. API and ABI guidance preserved

The conversation strongly emphasized public API discipline:

- use opaque handles;
- use explicit result codes;
- include **size**, **version**, and reserved fields in public structs;
- define project-owned primitive types;
- avoid exposing platform headers in core public APIs;
- avoid public bitfields;
- avoid compiler-dependent struct layout;
- avoid STL/C++ ABI across plugin/runtime boundaries;
- avoid exceptions/RTTI crossing engine boundaries;

- clarify allocator ownership;
- avoid global singleton state;
- label APIs as experimental, stable, deprecated, reserved, internal, or removed.

This matters because portability across compilers, OSes, language bindings, plugin boundaries, and future rewrites requires stricter discipline than ordinary application code.

9. Data, protocol, save, pack, and replay rules preserved

The conversation treated persistent formats as first-class public contracts. Every persistent format should include:

- magic identifier;
- format ID;
- major/minor version;
- writer version;
- minimum reader version;
- endianness;
- schema/profile ID;

- chunk table or TLV registry;
- checksum or hash;
- optional signature;
- migration path;
- unknown-field policy;
- reserved tag registry.

It explicitly warned against raw `fwrite/fread` of in-memory structs as a save format because that bakes in padding, endian, alignment, compiler layout, and evolution assumptions.

The proposed long-term policy is that released IDs are never reused and old data gets fixtures and migration tests.

10. Testing, validation, and conformance preserved

The conversation said that every stable boundary should have:

- public spec;
- valid fixtures;
- invalid fixtures;

- malformed fixtures;
- golden outputs;
- migration cases;
- fuzz cases;
- roundtrip tests;
- deterministic replay tests where relevant;
- conformance tests that a replacement implementation must pass.

High-trust paths such as save/load, pack/mod loading, replay, network protocol, installer/updater, authority/capability systems, and content promotion were identified as needing stronger validation.

This is one of the biggest future-proofing points: without conformance tests, "modular" APIs are only promises.

11. Setup, launcher, tools, UI, and command boundaries preserved

The conversation separated operational roles:

- **Setup** modifies installed system state and should be transactional: plan, stage, verify, commit, receipt, rollback.

- **Launcher** is orchestration: discover products, select profiles, validate installs, resolve packs/mods, start the game, export diagnostics.
- **Tools** should use the same public contracts as runtime code.
- **UI** should not mutate authoritative state directly; it should send commands and receive results, refusals, diagnostics, progress, and view models.

This supports multiple frontends: CLI, TUI, rendered GUI, native wrapper, headless JSON, server admin, launcher, setup.

12. What was put off for later

The conversation deferred implementation. The major deferred work includes:

1. **Repo boundary audit** - inspect actual directories, headers, dependencies, naming, and data formats.
2. **Canonical architecture decision** - decide what parts of the proposed doctrine become binding.
3. **Public API policy** - draft and adopt rules for public headers, ABI compatibility, ownership, versioning, and lifecycle labels.

4. **Data evolution policy** - draft and adopt rules for saves, packs, replays, protocols, schemas, registries, and migrations.
 5. **DDAP/DIL acceptance** - decide whether the assurance profile and integrity levels become project canon.
 6. **Conformance test pilot** - choose one subsystem and prove the replacement-contract model.
 7. **Layer checker** - implement dependency direction enforcement.
 8. **Schema/ID registry checker** - enforce non-reuse and reservation of released IDs.
 9. **Standards verification** - verify official sources before including external standards in the formal spec book.
 10. **Cross-chat aggregation** - combine this report with other old-chat reports into a master Project Spec Book.
-

13. What the generated preservation package already contains

Before this companion report, the package already contained:

Row 1 File: 00_manifest.md; Role: Package inventory and caveats.

Row 2 File: 01_human_readable_report.md; Role: Main preservation report for the chat.

Row 3 File: 02_context_transfer_packet.md; Role: Handoff packet for a future chat.

Row 4 File: 03_spec_sheet.yaml; Role: YAML-style aggregation/spec sheet.

Row 5 File: 04_registers.md; Role: Workstream, decision, task, constraint, preference, artifact, risk, verification, timeline, and spec contribution registers.

Row 6 File: 05_aggregator_packet.md; Role: Compact cross-chat aggregation packet.

Row 7 File: 06_reader_brief.md; Role: Short human reference.

Row 8 File: 07_verification_and_audit.md; Role: Self-audit and verification queue.

Row 9 File: 08_future_chat_bootstrap_prompt.md; Role: Prompt for continuing in a new chat.

Row 10 File: 09_in_chat_reader.md; Role: Navigation guide and question menu.

Row 11 File: `handoff_package.zip`; Role: The prior ZIP bundle containing files 00-09.

This new report adds a more direct narrative accompaniment that explicitly answers: what did we talk about, what did we decide, what did we do, what did we put off, and what should not be lost.

14. Most important carry-forward items

14.1 Most important idea The project should be treated as a contract-governed engine platform, not just a collection of game files.

14.2 Most important architecture law Domino must not depend on Dominion. Dominion should consume Domino through stable public contracts.

14.3 Most important compatibility law Released IDs, tags, opcodes, capability IDs, save chunks, and error codes must not be reused.

14.4 Most important testing law A boundary is not stable until it has conformance tests and fixtures.

14.5 Most important process law Heavy assurance should apply only where risk justifies it. Do not burden

ordinary creative iteration with certification-style process.

14.6 Most important unresolved issue The actual repository must be inspected before declaring the proposed structure "best" or canonical.

15. Recommended next action sequence

- 1. Read the main human-readable report and this companion report.**

Purpose: ensure the doctrine is understood before formalizing anything.

- 1. Inspect the actual repo.**

Purpose: identify where existing structure, naming, headers, data formats, tools, and tests match or contradict the proposed architecture.

- 1. Produce a boundary audit.**

Output: list of current public contracts, accidental public contracts, private internals, coupling violations, data formats, and migration risks.

- 1. Accept/revise/reject DDAP and DIL.**

Output: clear decision on assurance profile and integrity levels.

1. **Draft @@V10@@.**

Output: rules for public headers, ABI, lifecycle labels, ownership, allocator rules, and versioning.

1. **Draft @@V10@@.**

Output: rules for save/pack/replay/protocol/schema IDs, migrations, unknown fields, and reserved values.

1. **Pick one conformance-test pilot.**

Recommended candidates: pack validator, save reader, replay verifier, or render command buffer.

1. **Create enforcement scripts.**

Output: layer checker, public header checker, schema evolution checker, reserved ID checker, and traceability checker.

1. **Aggregate with other old-chat reports.**

Output: master Project Spec Book draft with conflicts and duplicates identified.

16. Risks and failure modes

Risk: treating recommendations as ratified

decisions Many architecture items are strong recommendations but not yet explicit user decisions. A future assistant should not say "the project decided X" unless the user ratifies it.

Risk: freezing internals too early The project needs stable public contracts, not frozen implementation details. Over-freezing private APIs would make the system harder to improve.

Risk: delaying data compatibility too long Save, pack, replay, protocol, and schema formats become hard to fix after release. A compatibility policy should be drafted early.

Risk: over-applying assurance DDAP/DIL should not turn creative iteration into bureaucracy. Heavy controls belong on high-trust paths.

Risk: under-testing contracts Without conformance fixtures and golden/malformed/migration tests, the claim of replaceability is weak.

Risk: stale standards claims External standards and official references should be verified before being merged into a formal spec book.

Risk: repository mismatch The proposed tree may not match the current repo. It should be used as a target model, not as proof of current state.

17. Decisions and statuses

Item	Status	Notes
User wants portability/modularity/extensibility/reuse	FACT / user-stated	Strongly established.
User wants replaceability of files/directories/code/data	FACT / user-stated	Strongly established.
User wants OS-grade/proper-game-grade discipline	FACT / user-stated	Strongly established.
Use standards as design inputs, not compliance targets	Assistant recommendation; likely aligned	Needs user ratification for canon.
Create DDAP/DIL	Assistant recommendation	Open decision.
Stable public contracts, replaceable internals	Assistant recommendation; central doctrine	Should be confirmed by user.
Domino reusable, Dominium product-specific	Assistant recommendation; highly aligned	Should be confirmed by user.
Proposed directory structure	Candidate design	Needs repo inspection.
API/ABI rules	Candidate policy	Needs policy draft and acceptance.
Data-evolution rules	Candidate policy	High priority to formalize.
Preservation package generation	Completed	Files and ZIP generated.
Companion report and consolidated	Completed by this task	See new ZIP and manifest.

18. Open questions

1. Which assistant recommendations should become formal Domino/Dominium canon?
2. Should DDAP and DIL levels be adopted, renamed, simplified, or rejected?
3. What is the actual current repo structure?
4. Which APIs already exist and which are accidental/public/private?
5. Which data formats already exist and need migration policy?
6. What platforms are genuinely target platforms versus aspirational future targets?
7. Which language constraints are mandatory in each layer?
8. What is the first subsystem to receive conformance tests?
9. What is the minimum backward-compatibility promise before first public release?
10. How should this chat's package merge with other project preservation packages?

19. Best follow-up questions

The best next questions to ask in this chat are:

1. Which recommendations from this package should become formal Domino/Dominium canon?
2. Can you turn the stable-contracts doctrine into a concise architecture constitution?
3. Can you draft `docs/architecture/public-api-policy.md`?
4. Can you draft `docs/architecture/data-evolution-policy.md`?
5. Can you make a repo audit checklist for checking whether our current structure violates these rules?
6. Can you convert the 20 architecture laws into CI-enforceable checks?
7. Can you design the first conformance-test pilot for the pack validator?
8. Can you design the first conformance-test pilot for the save format?
9. Can you make a decision matrix for DDAP/DIL adoption?
10. Can you prepare this package for merger into a master Project Spec Book?

20. Final summary

This conversation established the high-level architecture doctrine for Domino/Dominium as a serious, reusable, long-lived engine/game ecosystem. It started with a question about whether high-assurance standards should influence the project and expanded into a much broader set of engineering practices for portability, modularity, extensibility, replaceability, future-proofing, and compatibility.

The most important concept is stable public contracts with replaceable internals. The project should make deliberate promises at durable boundaries-public APIs, schemas, data formats, protocols, IDs, packs, saves, replays, and tool outputs-while keeping internal code and directory implementation free to evolve.

The conversation produced a candidate architecture with Domino as reusable engine/platform and Dominium as one product built on top. It proposed internal assurance levels, strict public API rules, data-evolution policies, conformance tests, deterministic simulation boundaries, transactional setup, launcher separation, toolchain discipline, and structured preservation artifacts.

What has been completed is a preservation and handoff package. What remains is formal adoption, repo inspection, policy drafting, test implementation, standards verification, and cross-chat aggregation.

The safest immediate next action is to inspect the real repository and produce a boundary audit. After that, the user should decide which recommendations become canon and begin drafting the public API and data evolution policies.

Chapter 16

Refactor Architecture

16.1 Dominium Domino Refactor Architecture Reader Brief

Source: docs/archive/conversations/Refactor_Architecture/Dominium_Domino_Refactor_Architecture__05_reader_brief.md

Report type: reader_brief

Size: 5050 bytes

16.1.1 Reader Brief - Dominium + Domino Refactor Architecture

What This Chat Was About

This chat was a detailed architecture and refactor planning session for the Dominium + Domino project. The central result is a stable separation between **Domino**, the deterministic engine under `source/domino`, and **Dominium**, the product suite under `source/dominium`. The user explicitly chose to flatten the Dominium product source layout by removing `source/dominium/products` and placing common, game,

`launcher`, `setup`, and `tools` directly under `source/dominium`. The chat also established that Domino should not be installed as a separate end-user runtime; it should be linked/bundled into Dominium products, with a separate developer SDK only as a future possibility.

The product model is four products: Game, Launcher, Setup, and Tools. Game is a single product with startup modes for client, listen server, dedicated server, and demo/full. Launcher is the main access point to the suite. Setup handles portable/per-user/system-wide install, import, repair, uninstall, and GC. Tools is an aggregated product beginning with existing `dominium_modcheck`.

The chat also defined the `DOMINIUM_HOME` model for multi-version installs. Product builds, mods, and packs live in a shared versioned repo; instances reference versions instead of copying whole suites. Versioning is split across core, products, formats, protocols, mods, and packs, with Suite version equal to Game version. The base official modpack should match Game version by release convention, but engine validity should use compatibility ranges.

Several Codex prompts were generated, including a master refactor prompt and a post-refactor consistency prompt. It is not verified that any prompt was applied.

Most Important Things to Know

- Domino stays under `source/domino`; do not rename to `source/engine`.
- Dominion source is flattened under `source/dominium/common, game, launcher, setup, tools`.
- Do not keep `source/dominium/products` in the final layout.
- Domino is not a separate end-user runtime product.
- Dominion has four products: Game, Launcher, Setup, Tools.
- Game is one binary/product with modes, not separate client/server/demo products.
- Demo is content/instance policy, not a separate product.
- Launcher is the main suite hub.
- Launcher should use actions, not hardcoded executable paths.
- Setup supports portable, per-user, and system-wide install modes.
- `DOMINIUM_HOME` stores `repo/products`, `repo/mods`, `repo/packs`, and instances.

- Instances reference versions; they do not copy full suites.
- Suite version equals Game version.
- Core/product/protocol/format versions are separate.
- Base mod version matching Game version is a convention, not engine invariant.
- DLC/user mods/packs use independent versions and compatibility ranges.
- Use explicit architecture tags such as x86-64, not x64.
- All product UI/rendering must go through Domino dsys/dgfx/canvas.
- Codex prompts were generated; application status is unknown.

Active Plans or Workstreams

- Apply source layout refactor.
- Add compat/platform headers and product introspection.
- Unify Game client/server/demo modes.

- Implement DOMINIUM_HOME repo/instance APIs.
- Implement actions loader and Launcher integration.
- Implement Setup import/install/GC.
- Update content manifests and packaging scripts.
- Run post-refactor consistency sweep.

Decisions Already Made

See DECISION-01 through DECISION-22 in the registers. Highest impact: DECISION-01, DECISION-02, DECISION-03, DECISION-06, DECISION-08, DECISION-11, DECISION-13, DECISION-14, DECISION-18, DECISION-19.

Pending Tasks

Highest priority tasks are TASK-24, TASK-01, TASK-02, TASK-03, TASK-04, TASK-05, TASK-06, TASK-07, TASK-08, TASK-09, TASK-10, TASK-11, TASK-12, and TASK-13.

Open Questions

- Has Codex applied the generated prompts?

- What are the current actual version numbers?
- Which parser should repo/instance/actions manifests use?
- What exact DOMINIUM_HOME defaults should each OS use?
- Which root-level tools should become DominiumTools?
- How much UI/packs work should be included in the next implementation phase?

Files / Artifacts / Prompts to Preserve

- User-provided existing repo directory tree.
- Master Refactor Prompt.
- Post-Refactor Consistency Pass Prompt.
- Starter Prompt Generator Prompt.
- Extended Master Starter Prompt Generator Prompt.
- Initial Context Transfer Packet.
- This report package.

What to Verify Before Acting

- Repo state after any Codex run.
- Current version macros and data versions.
- Whether new headers/targets exist.
- Whether `source/dominium/products` remains.
- Whether product introspection works.
- Whether CMake/tests build.
- Whether docs/scripts still contain stale paths or x64.

Best Next Step

Ask the user whether Codex has already applied the master refactor prompt. If not, run or refine the master refactor prompt. If yes, run the post-refactor consistency prompt and then verify the build/test results.

Chapter 17

Refactor Control Plane

17.1 Aide Xstack Dominium Refactor Control Plane Human Readable Report

Source: docs/archive/conversations/Refactor_Control_Plane/AIDE_XStack_Dominium_Refactor_Control_Plane__01_human_readable_report.md

Report type: human_readable_report

Size: 33174 bytes

17.1.1 COMPLETE CHAT PRESERVATION REPORT - AIDE, XStack, and Dominium Refactor Control Plane

0. Coverage and Reliability Assessment

Row 1	Field: Chat label; Assessment: AIDE, XStack, and Dominium Refactor Control Plane
Row 2	Field: Date anchor; Assessment: 2026-05-27 Australia/Melbourne
Row 3	Field: Source scope; Assessment: This chat only,

except live GitHub checks explicitly labelled as current repo verification

Row 4 Field: Apparent access; Assessment: Partial-to-broad visible chat context; not guaranteed full hidden transcript

Row 5 Field: Previously generated files available?; Assessment: Canvas documents exist in this conversation but not as direct downloadable files until this package

Row 6 Field: Uploaded files or artifacts present?; Assessment: Yes: current Pasted text.txt; earlier uploads expired

Row 7 Field: Contains future plans?; Assessment: Yes

Row 8 Field: Contains decisions?; Assessment: Yes

Row 9 Field: Contains pending tasks?; Assessment: Yes

Row 10 Field: Contains unresolved questions?; Assessment: Yes

Row 11 Field: Staleness risk; Assessment: Medium-High for external tools/repo state; mitigated by current GitHub checks

Row 12 Field: Extraction confidence; Assessment: 4/5 for visible chat substance; lower for expired uploads

Row 13 Field: Safe for later aggregation?; Assessment: Yes, with caveats

Row 14 Field: Main limitations; Assessment: Some earlier uploaded files expired; exact raw re-analysis not available; live repos may advance after package creation

Limitations in plain language FACT: This package is based on the visible chat context, the user-provided preservation prompt in `Pasted text.txt`, canvas summaries created in this chat, and live GitHub checks performed during this chat. FACT: several older uploaded files expired, including earlier PDFs and source ZIPs. Their contents are represented here only from the visible discussion and earlier extracted summaries, not from fresh reinspection. UNCERTAIN: the live AIDE and Dominion repositories may have advanced after the latest checks used in this package. For current implementation work, the first action should be to verify repository HEAD and relevant files again.

1. One-Page Orientation

This chat was a long-running architecture, workflow, and project-control discussion around how to use AI coding tools more effectively, how to evolve Dominion's XStack governance system, and how to build AIDE as a portable

repo-native operating layer. It began with practical questions about using Codex Pro, the Codex VS Code extension, ChatGPT project spaces, the ChatGPT/Codex apps, Visual Studio, and the API. The user wanted a better way to develop projects, perhaps concurrently and across tools, without manually handing prompts from ChatGPT to Codex.

The conversation quickly shifted from simple prompt engineering toward harness engineering. The central lesson became that high-quality agentic development depends less on one perfect prompt and more on the surrounding system: repo instructions, task contracts, memory, tool permissions, worktree isolation, evidence packets, validation, review gates, branch discipline, and deterministic run paths. This framing then became the lens for evaluating Codex, Claude Code, OpenHands, Continue, oh-my-codex, claw-code, claude-mem, Graphify, and other systems.

A major middle phase focused on XStack. XStack began as a Dominion-specific governance stack grown out of TestX, then expanded into RepoX, AuditX, and other concepts. We initially explored making XStack a general portable metaharness, but that proved too large and too Dominion-shaped. The accepted resolution was that XStack should remain Dominion's strict local profile and proving ground, while AIDE becomes the new portable public layer.

AIDE itself also changed direction. It was first imagined broadly as an Automated Integrated Development Environment: perhaps a desktop app, extension family, multi-agent runtime, daemon, service, or control plane. Over time, the plan narrowed into something more practical: AIDE should first be Profiles + Harness + Compatibility + Dominion Bridge. That means AIDE defines repo contracts in `.aide/`, compiles those contracts into current tool-facing artifacts, validates and migrates them, produces compact task and evidence packets, and later may grow into Runtime, Gateway, Service, Commander, and IDE hosts only after its smaller layer proves useful.

The latest Dominion-specific thread reframed AIDE as the restructuring and execution control plane for the messy Dominion repo. Dominion had completed CONVERGE and POST-CONVERGE work that improved layout authority and build proof, but it remained physically messy, with root exceptions and CTest/RepoX blockers. The conclusion was that AIDE should now control refactors through root inventories, per-file classification, salvage maps, move maps, path aliases, migration ledgers, and validation gates. XStack/AuditX/RepoX/TestX should not be deleted; their useful checks should be wrapped, classified, migrated, and retired behind AIDE adapters.

The future relevance of this chat is high. It defines the

operating doctrine for AIDE, the relationship between AIDE and XStack, the immediate Dominion cleanup sequence, the longer AIDE Q35-Q57 roadmap, the branch/commit/workunit discipline, the versioning/capability model, and the anti-patterns to avoid. A future reader must understand one key idea: AIDE is not being built as another coding agent first. It is being built as the repo-native operating layer that lets humans, Codex, Claude Code, local tools, and future agents work through stable contracts, evidence, and validation.

2. The Story of the Conversation

2.1 From tool workflow to harness doctrine The chat opened with the user describing a workflow: paying for Codex Pro, using the Codex VS Code extension on one project at a time, and using ChatGPT project spaces to generate prompts that were manually fed into Codex. The user had also installed the ChatGPT and Codex apps and Visual Studio Community. The initial question was whether there was a better, more professional, more integrated way to develop projects.

The first conclusion was that the user's workflow worked but was not optimal. ChatGPT should be used for architecture, research, task design, documentation, review, and high-level reasoning. Codex should handle bounded repo-local implementation, testing, review, and worktree tasks. API/Responses/Agents-style systems

could later support durable automation. This was the first move away from manual prompt relay and toward structured work.

The user then asked about optimizing prompts for GPT-5.3/GPT-5.4 and later supplied advice about harness engineering. That advice argued that teams succeed by engineering the environment around agents: tools, docs, linters, feedback loops, memory, and observability. The conversation accepted the core point: the model is not the whole system; the harness is the multiplier.

2.2 From harness ideas to XStack metaharness

The user proposed a rich stack: repo tree as topology, AGENTS.md as law, skills as procedures, MCP as integration bus, Responses as orchestration substrate, hosted shell as execution, evals as truth filter, CI/merge queue as integration governor, and recursive subagents working across branches and machines. The response refined this into bounded delegation: task graphs, evidence packets, branch leases, role separation, traces, eval layers, and privilege separation. The important correction was that unbounded recursive agents are not the goal; bounded tasks, contracts, evidence, and integration governance are.

The discussion then mapped these ideas onto Dominion's XStack. We considered adding HarnessX, XPlan, XExec, MergeX, ContextX, SkillX, TaskX,

BridgeX, SessionX, PermitX, DoctorX, TraceX, and more. Over time this became too many public concepts. We simplified toward higher-level planes and then eventually concluded that XStack should not be the general product at all.

2.3 External research inputs The user supplied or referenced many external projects and reports: Claude Code leak mirrors, claw-code, oh-my-codex, claude-code-from-source, claude-mem, OpenHands, Continue, Graphify, prompt archives, and leak/resource guides. The repeated pattern was: use them for pattern mining, not as foundations. oh-my-codex looked like a useful Codex-facing workflow/runtime reference. claw-code looked useful as research/parity scaffolding, not a base. Claude Code leak analysis confirmed harness patterns: coordinator-worker, tool partitioning, memory, hooks, remote sessions, and multi-agent orchestration. Graphify looked useful as an optional graph-analysis backend, not a canonical memory layer. claude-mem contributed ideas for observer-driven typed memory and progressive disclosure.

The Grug Brained Developer became a design filter: avoid complexity, avoid premature abstraction, do small safe refactors, prefer integration tests, invest in logging, and respect existing structures before tearing them down. This influenced the decision to narrow AIDE and

not overbuild runtime/hosts too early.

2.4 XStack becomes Dominium profile; AIDE becomes public layer The user eventually named AIDE: Automated Integrated Development Environment. The accepted split became: XStack remains Dominium's strict local governance/proof profile; AIDE becomes the portable public layer. AIDE would live in its own repo, be usable as a standalone repo pack, later as CLI/app/extensions, and eventually perhaps a runtime/service/host system.

But AIDE was narrowed again: AIDE v0 should not be a full coding agent. It should be a portable repo harness/profile standard plus compiler/adapters, validator, bakeoff kit, and Dominium bridge. Later, after more AIDE development, the plan matured into AIDE Core / Hosts / Bridges with internal Core bands: Contract, Harness, Runtime, Compatibility, Control, SDK. The first shipped stack remained Contract/Profile + Harness + Compatibility + Dominium Bridge.

2.5 Dominium repo convergence and root recycling The user then integrated live Dominium repo state. CONVERGE had improved layout authority but not finished physical cleanup. Later POST-CONVERGE tasks improved native build proof: VS2022/v143 configure and build pass, native binaries

are produced, but full CTest remains blocked by `invariant_units_present`, `inv_repo_x_rules`, and wall-time. The repo is governed and build-configurable but still messy and not feature-expansion-ready.

The plan changed from "clean folders" to "AIDE-controlled root recycling." The remaining bad top-level roots are mixed strata, not junk. Every file must be inventoried and assigned a fate: Keep, Adapt, Extract, Convert, Archive, or Drop. Moves require salvage maps, move maps, path aliases, migration ledgers, validators, build/test proof, and rollback notes. XStack/AuditX/RepoX/TestX should be wrapped and recycled behind AIDE before renaming or deletion.

2.6 Current final state By the end of the chat, AIDE had become the staged, compatibility-preserving repo operating system. For AIDE itself, the authoritative development path runs from Q35 through Q57: GitHub/CI advisory, governance audit, intent compiler, repo intelligence index, file quality ledger, refactor control, root recycling, tool absorption, install/repair/upgrade/rollback, stable release bundle, Dominion/Eureka installs, and only later runtime/gateway/hosts. For Dominion, the next sequence is CTest/RepoX blocker remediation/classification, then AIDE-STRUCTURE-00, tool recycling, root recycling framework, inventory waves, move waves, product boot proof, portable

projection proof, and only then feature work.

3. Main Topics Discussed

Topic 1 - Personal AI development workflow This topic began the chat. The user wanted to know whether their Codex Pro, ChatGPT, VS Code, and app workflow could be improved. The conclusion was to stop using ChatGPT mainly as a prompt generator for Codex and instead use each tool for its proper layer. ChatGPT should be the architecture/spec/review/documentation environment. Codex should do bounded repo implementation and tests. AIDE later becomes the portable harness layer that reduces prompt relay entirely.

Topic 2 - Harness engineering Harness engineering became the major doctrine. The visible rationale was that agents perform better when the repo provides stable context, reliable commands, test recipes, permission models, worktree isolation, memory, and evidence. This moved the discussion away from "best prompts" and toward "best environment." The future implication is that AIDE should create the repo-native structure that existing agent tools consume.

Topic 3 - XStack XStack was first considered as the central metaharness, but that became too broad. The final decision is that XStack remains Dominion-specific

and strict. It should be narrowed, modularized, and eventually wrapped behind AIDE. It should not be deleted immediately because its checks and evidence are valuable. It should not remain public mythology forever because the names and structure are too Dominion-era and too complex.

Topic 4 - AIDE AIDE was introduced as the new general system. It evolved from a broad Automated Integrated Development Environment into a more staged repo-native operating layer. The first real product is Profiles + Harness + Compatibility + Dominion Bridge. Later AIDE may include Runtime, Gateway, Service, Commander, mobile, and IDE hosts, but those are deferred until the stable control plane proves itself.

Topic 5 - Existing tool ecosystem The chat studied or referenced Codex, Claude Code, OpenHands, Continue, oh-my-codex, claw-code, claude-code-from-source, claude-mem, Graphify, and other projects. The conclusion was that tools already have agents, subagents, skills, hooks, project files, and config surfaces. The gap is a portable repo-owned harness/profile standard that can compile into those dialects. AIDE should fill that gap.

Topic 6 - Dominion repo convergence and root recycling The current Dominion repo has improved

governance and build proof but still contains many messy roots. The final plan is AIDE-controlled root recycling: inventory, classify, salvage, move, rewrite references, validate, shim, and retire exceptions. No broad feature work or root moving should occur before CTest/RepoX blockers are classified and AIDE refactor machinery exists.

Topic 7 - Versioning and compatibility The chat developed a layered identity model: versions identify releases, capability contracts determine compatibility, and GBN/BII/hash identify exact build provenance. This matters to Dominion and AIDE because they handle products, suites, components, packs, schemas, protocols, plugins, and runtime targets. Capabilities such as `save.schema@5` or `net.protocol.session@3` are better compatibility truth than release version numbers alone.

Topic 8 - Git workflow and WorkUnit recovery The user supplied AIDE advice around dev branches, structured commits, and WorkUnit recovery. The accepted model is `main` as canonical truth, `dev` as governed integration, `task/*` as bounded work, and `release/hotfix` branches as needed. AIDE should enforce commit trailers, branch roles, `safe sync/land/promote/prune`, and `repo-state-first` recovery when prompts are

repeated or out of order.

Topic 9 - Knowledge preservation The user asked to preserve this chat as a static knowledge base. This package is the result. Earlier canvas documents captured intermediate summaries. This report supersedes them as the most complete current package but should not be treated as perfect raw transcript extraction, especially because earlier uploads expired.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted to improve AI-assisted development, design a better harness, decide XStack's role, create AIDE, align Dominion cleanup with AIDE, use AIDE in Dominion safely, and generate a complete preservation package. These goals mattered because the user is managing long-running complex projects and wants future chats, Codex sessions, and repos to stop losing context or repeating mistakes.

4.2 Inferred Goals INFERENCE: The user also wants a durable personal/team development operating system that can survive tool churn, support local and remote agents, avoid vendor lock-in, and make repo cleanup mechanical. This was inferred from repeated interest in Codex, Claude Code, open-source harnesses,

local models, multi-repo AIDE adoption, and
Dominium/Eureka integration.

4.3 Goals That Changed Over Time AIDE changed most. It began as a possible full development environment and became a portable repo operating layer. XStack changed from a candidate universal system to a Dominium strict profile. Repo cleanup changed from moving folders to root recycling under AIDE. AI workflow changed from prompt optimization to harness engineering.

4.4 Goals Still Unresolved Unresolved goals include finishing AIDE Q35-Q57, stabilizing Dominium CTest/RepoX blockers, defining final AIDE install/upgrade bundles, implementing root recycling machinery, deciding when product boot proof can proceed, and later deciding whether AIDE Runtime/Gateway/Hosts are truly needed.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Use harness engineering as central frame; Status: Accepted; Why it mattered: Models/prompts alone do not solve reliability; environment/contracts/evidence do; Confidence: 5; Label: FACT

- Row 2** ID: DECISION-02; Decision: Keep ChatGPT/Codex roles distinct; Status: Accepted; Why it mattered: ChatGPT for architecture/spec/review; Codex for repo-local implementation; Confidence: 4; Label: INFERENCE
- Row 3** ID: DECISION-03; Decision: XStack remains Dominion-specific strict profile; Status: Accepted; Why it mattered: XStack grew from Dominion and carries local law; universalizing it causes bloat; Confidence: 5; Label: FACT
- Row 4** ID: DECISION-04; Decision: AIDE becomes new portable repo-native operating layer; Status: Accepted; Why it mattered: New clean identity avoids XStack baggage; Confidence: 5; Label: FACT
- Row 5** ID: DECISION-05; Decision: AIDE v0 is Profiles + Harness + Compatibility + Dominion Bridge, not full runtime; Status: Accepted; Why it mattered: Existing tools already provide runtimes; missing layer is portable repo contract; Confidence: 5; Label: FACT
- Row 6** ID: DECISION-06; Decision: Profile and Harness are distinct; Status: Accepted; Why it mattered: Declarative truth and executable machinery must not blur; Confidence: 5; Label: FACT

Row 7 ID: DECISION-07; Decision: AIDE must self-host from a small seed, not zero magic; Status: Accepted; Why it mattered: Honest bootstrap requires repo kind/commands/policy seed; Confidence: 4; Label: FACT

Row 8 ID: DECISION-08; Decision: Use AIDE in Dominion as subordinate pack/control layer first; Status: Accepted; Why it mattered: Dominion AGENTS/XStack authority must not be overwritten; Confidence: 5; Label: FACT

Row 9 ID: DECISION-09; Decision: Do not delete XStack checks; wrap/classify/migrate/retire; Status: Accepted; Why it mattered: Old names are bad, checks are valuable; Confidence: 5; Label: FACT

Row 10 ID: DECISION-10; Decision: Dominion cleanup must be file-by-file root recycling, not wholesale moves; Status: Accepted; Why it mattered: Mixed roots contain source/docs/policy/tests/generated/history; Confidence: 5; Label: FACT

Row 11 ID: DECISION-11; Decision: Do not start broad feature work until build/CTest/product/projection proof is acceptable; Status: Accepted; Why it mattered: Feature expansion amplifies

disorder while proof gaps remain; Confidence: 5;
Label: FACT

Row 12 ID: DECISION-12; Decision: Use versions for release identity, capabilities/contracts for compatibility, GBN/BII for provenance; Status: Accepted as strong refinement; Why it mattered: Multi-dimensional compatibility cannot be encoded in one version string; Confidence: 4; Label: FACT/INFERENCE

Row 13 ID: DECISION-13; Decision: Use dev as integration branch, not source of truth; Status: Accepted from AIDE Git plan; Why it mattered: main remains accepted canonical truth; Confidence: 4; Label: FACT/INFERENCE

Row 14 ID: DECISION-14; Decision: Structured commits and WorkUnit recovery should become AIDE policy; Status: Accepted; Why it mattered: Future automation needs task IDs, branch roles, evidence, trailers; Confidence: 4; Label: FACT/INFERENCE

Row 15 ID: DECISION-15; Decision: External leak-derived projects are research inputs, not foundations; Status: Accepted; Why it mattered: Legal/supply-chain risk and vendor-specific semantics; Confidence: 5; Label: FACT/INFERENCE

- Row 16** ID: DECISION-16; Decision: Graphify-like tools are optional analysis backends, not canon; Status: Accepted; Why it mattered: Graph outputs are derived and may be noisy/stale; Confidence: 4; Label: FACT/INFERENCE
- Row 17** ID: DECISION-17; Decision: AIDE must support template, managed product, and release bundle adoption; Status: Accepted; Why it mattered: Different repos need different adoption modes; Confidence: 4; Label: FACT
- Row 18** ID: DECISION-18; Decision: Generated outputs are evidence unless promoted; Status: Accepted; Why it mattered: Prevents generated files from competing with canon; Confidence: 5; Label: FACT
- Row 19** ID: DECISION-19; Decision: Use documentation/contract first before runtime/app/extension expansion; Status: Accepted; Why it mattered: Avoids overbuilding and premature abstraction; Confidence: 5; Label: FACT
- Row 20** ID: DECISION-20; Decision: This chat should be preserved as a high-fidelity handoff package; Status: Accepted by current user request; Why it mattered: Creates persistent artifact for future aggregation; Confidence: 5; Label: FACT

Decision explanations The most important decisions were DECISION-03 through DECISION-10. The user accepted that XStack should remain Dominion's strict profile, not the public general system. AIDE should become the portable repo-native operating layer. AIDE's first product should be Profiles + Harness + Compatibility + Dominion Bridge. XStack checks should be wrapped and migrated, not deleted. Dominion root cleanup must be file-by-file root recycling, not wholesale moves. These decisions all manage the same risk: losing hard-won validation and project knowledge while trying to clean up complexity. Several decisions are strategic rather than fully implemented. The version/capability model, Git workflow model, install/repair/upgrade model, and tool absorption model are accepted directions but require schemas and code. Future assistants must not mistake them for already completed implementations.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

The biggest rejected idea was building AIDE as a full coding-agent competitor first. That was superseded by the smaller Profiles + Harness + Compatibility + Bridge wedge. Another rejected idea was keeping XStack forever as public mythology; it should be wrapped and retired. Conversely, deleting XStack outright was also

rejected because its checks matter. Wholesale folder moves were rejected because roots are mixed strata. Feature work in Dominion was deferred until proof improves. Leak-derived code and graph tools were limited to pattern research or optional backends.

7. Important Reasoning, Rationale, and Tradeoffs

The visible rationale was consistent: preserve value while reducing complexity. The chat repeatedly balanced ambition against proof. AIDE should be powerful but start small. XStack should be replaced as public ontology but not discarded functionally. Dominion should become cleaner but not by moving files blindly. External projects should inform AIDE but not become dependencies. Versions should remain useful but not be abused as compatibility truth. Dev branches should help multi-agent work but not become uncontrolled truth.

The main managed risks were drift, over-abstraction, lost validation, hidden path assumptions, bad agent memory, and premature feature work. The dominant tradeoff was speed versus safety: moving folders or deleting old tools would look faster but would likely break build/test/proof paths and lose semantics. Root recycling is slower but safer.

8. Plans, Future Work, and Next Steps

The current plan splits into AIDE work and Dominion work.

For AIDE, finish Q35, run QCHECK-03, then implement Q36-Q42 for intent/repo intelligence/refactor/tool absorption, then Q43-Q48 for install/repair/upgrade/rollback/release bundle, then Q49-Q57 for Dominion/Eureka stable install and absorption. Runtime/Gateway/Hosts wait until QCHECK-04.

For Dominion, handle the current CTest/RepoX blockers, then establish AIDE-STRUCTURE-00, inventory old tooling, wrap tools through AIDE, build root recycling framework, inventory roots, and only later perform move waves. Product boot proof, portable projection proof, and first test release lane follow. Feature systems wait.

Recommended immediate sequence:

1. Verify current Dominion HEAD and POST-CONVERGE status.
2. Remediate/classify `invariant_units_present`.
3. Remediate/classify `inv_repox_rules`.
4. Decide if POST-CONVERGE-11 can proceed with CTest-warning status.

5. Finish AIDE Q35 and QCHECK-03 if not already done.
6. Start AIDE Q36-Q42 in AIDE repo.
7. In Dominion, start AIDE-STRUCTURE-00 only after current blocker status is understood.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences The user values direct, source-grounded, audit-ready, detailed answers. The user asked for preservation packages that are human-readable and machine-aggregatable. The user repeatedly prefers not to lose nuance, rejected options, uncertainties, or artifacts. The user wants current facts verified when possible and wants future assistants to avoid re-asking answered questions.

9.2 Inferred Constraints and Preferences

INFERENCE: The user strongly prefers systems that make future agent work less dependent on long chats. The user also prefers evidence-backed engineering over vibes, and wants repo state to dominate memory. The user appears willing to invest in infrastructure if it reduces future chaos.

9.3 Uncertain or Unestablished Preferences

UNCERTAIN: Exact public-facing naming for AIDE

Core/Kernel remains partly open. Exact threshold for accepting CTest-warning status remains a user/repo governance decision. Exact final AIDE installation packaging details remain pending.

10. Files, Artifacts, Outputs, and Prompts

Important artifacts include the canvas documents created earlier, this generated handoff package, live repo files fetched from GitHub, and many external references discussed. The earlier uploaded PDFs/ZIPs have expired, so they cannot be freshly audited. The newly uploaded `Pasted text.txt` provided the preservation-task prompt and drove the package structure.

The most important live repo artifacts are AIDE README/profile/Harness/AIDE Lite files and Dominion AGENTS/POST_CONVERGE_NEXT_STEPS. The most important generated artifacts are this report package, the context transfer packet, the spec sheet, the registers, the aggregator packet, and the future-chat bootstrap prompt.

11. Open Questions and Unresolved Issues

The main unresolved issues are whether Dominion's current CTest blockers will be fixed or explicitly accepted; what exact AIDE schema and stable release bundle will be used for target repos; how strict AIDE

validation will be; when AIDE Runtime becomes necessary; and how each remaining Dominion root family should be classified. These are not philosophical issues anymore. They are implementation and verification tasks.

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Future chats may incorrectly assume AIDE is already a full runtime, delete old XStack tools prematurely, move roots wholesale, start features before proof, treat generated files as canon, or forget that expired uploads limit raw-source certainty. The mitigation is to use the task register and verification queue, check live repo state first, preserve authority hierarchy, and require evidence for every cleanup/migration.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat contributes the architectural doctrine for AIDE, the migration strategy from XStack to AIDE, the Dominion root-recycling plan, the AIDE Q35-Q57 roadmap, the version/capability compatibility model, the Git/WorkUnit recovery model, and the anti-patterns to avoid. It should feed future spec-book chapters on AIDE doctrine, repo harness architecture, Dominion

cleanup, migration/compatibility, agent workflow, and evidence governance.

It should not be merged prematurely as final implementation law where items remain planning-level. In particular, Q35-Q57, root recycling, install/upgrade, and capability contracts require current verification and implementation.

14. What I Should Remember

- AIDE is the portable repo operating layer; XStack is Dominion's strict local profile.
- AIDE first ships Profiles + Harness + Compatibility + Dominion Bridge.
- AIDE Runtime/Gateway/Hosts are later, not now.
- Dominion is build-producing but not CTest-clean or feature-ready.
- Current Dominion cleanup must be AIDE-controlled root recycling, not folder dragging.
- XStack/AuditX/RepoX/TestX checks are valuable; names are not.
- Every mixed-root file gets one fate: Keep, Adapt, Extract, Convert, Archive, or Drop.
- Versions identify releases; capabilities/contracts decide compatibility; build IDs prove provenance.

- AIDE should support install/repair/upgrade/rollback before wide target-repo adoption.
- Future assistants must verify live repo state before implementing.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- "Give me the shortest possible explanation of why AIDE should not replace XStack by deletion."
- "Explain the difference between AIDE Profile, Harness, Compatibility, and Runtime."

15.2 Decisions

- "Which decisions here are final, and which are still tentative?"
- "Which decisions should become formal AIDE requirements?"

15.3 Tasks and Next Actions

- "Generate the next Codex prompt for POST-CONVERGE-10F."
- "Generate AIDE-STRUCTURE-00 as an implementation prompt."

- "Generate Q36 Intent Compiler v0 as a precise AIDE WorkUnit."

15.4 Artifacts and Files

- "Which files from this package should I paste into a future chat?"
- "Which live repo files should be checked before continuing Dominion work?"

15.5 Risks and Verification

- "What are the top risks if we import AIDE into Dominion too early?"
- "What must be verified before root moves begin?"

15.6 Future Spec Book / Aggregation

- "Convert this package into a chapter outline for the master spec book."
- "Merge this with another chat's preservation package without losing uncertainty labels."

15.7 Deep-Dive Questions Specific to This Chat

- "Design the root recycling schema in detail."

- "Design the AIDE install/upgrade/rollback model."
- "Design the capability contract registry for Dominion packs/plugins/protocols."

16. Compact Human Summary

This chat produced the current strategic architecture for AIDE, XStack, and Dominion cleanup. It began with a practical question about getting more out of Codex Pro and ChatGPT. The answer evolved into a much deeper conclusion: the core leverage is not better prompts but better harnesses. Agents need repo contracts, stable context, reliable commands, validation, permissions, evidence, and structured tasks. This led to the idea that AIDE should be a repo-native operating layer rather than another coding agent.

XStack was originally considered as the broader metaharness, but that was rejected. XStack grew from Dominion-specific needs and carries too much local vocabulary. It should remain Dominion's strict local governance/proof profile, but it should not become the public universal system. AIDE should become the portable public layer.

AIDE's first real product is not Runtime, Gateway, Service, Commander, desktop app, or IDE extension. It is Profiles + Harness + Compatibility + Dominion Bridge. AIDE Profiles declare repo truth. AIDE Harness

compiles, validates, doctors, migrates, and eventually installs/repairs/upgrades. Compatibility preserves schema, migration, replay, shim, and upgrade rules. Dominion Bridge maps Dominion's XStack strictness into AIDE without overwriting Dominion authority.

The current AIDE development plan runs through Q35-Q57. Q35/QCHECK-03 finish governance/Git/release readiness. Q36-Q38 build intent compilation, repo intelligence, and file quality. Q39-Q42 build refactor/root recycling/tool absorption/move-map mechanics. Q43-Q48 build install/repair/upgrade/rollback/release bundle support. Q49-Q53 prove Dominion stable install and absorption. Q54-Q57 do the same for Eureka and a product vertical slice. Only after QCHECK-04 should AIDE return to Gateway/provider/Service/Commander/IDE host work.

Dominium is not clean yet. The repo has improved: POST-CONVERGE work made layout authority machine-visible, strict validators pass, VS2022/v143 configure/build pass, and native binaries are produced. But full CTest is still blocked by unit/RepoX issues and wall-time, product boot proof is partial, portable projection proof is partial, and many root exceptions remain. So Dominion should not start major features yet.

The next Dominion strategy is root recycling. Remaining bad roots are not simply junk. They contain

mixed useful source, docs, policy, tests, generated evidence, obsolete scaffolding, and stale agent-era machinery. Every file must be classified as Keep, Adapt, Extract, Convert, Archive, or Drop. AIDE should create inventories, salvage maps, move maps, path aliases, migration ledgers, reference rewrites, validators, shims, and exception retirement. XStack/AuditX/RepoX/TestX tools should first be wrapped behind AIDE, then inventoried, adapted, migrated, and retired from public ontology. Do not delete them first.

The final operating rule is simple: AIDE should make Dominion refactors mechanical, validated, reversible, and evidence-backed before feature work resumes.

17.2 Aide Xstack Dominion Refactor Control Plane Reader Brief

Source: docs/archive/conversations/Refactor_Control_Plane/AIDE_XStack_Dominium_Refactor_Control_Plane__06_reader_brief.md

Report type: reader_brief

Size: 1976 bytes

17.2.1 Reader Brief -

AIDEXStackDominiumRefactorControl__

What this chat was about

This chat developed the architecture and migration strategy for AIDE, XStack, and Dominium. It started from improving AI coding workflows and ended with a concrete plan: AIDE becomes the repo-native operating/refactor control plane; XStack becomes Dominium's strict local profile and legacy validation layer; Dominium cleanup proceeds through AIDE-controlled root recycling.

Top 20 things to know

1. AIDE is not a full coding agent first.
2. AIDE first ships Profiles + Harness + Compatibility + Dominium Bridge.
3. XStack remains Dominium strict profile, not public general product.
4. Old XStack/AuditX/RepoX/TestX checks are valuable.
5. Old X names should eventually retire behind AIDE adapters.
6. Dominium currently builds but is not CTest-clean.

7. POST-CONVERGE-10E improved AuditX blockers but left unit/RepoX blockers.
8. Feature work should wait.
9. Root cleanup must be file-by-file root recycling.
10. Every file gets one fate: Keep, Adapt, Extract, Convert, Archive, Drop.
11. AIDE should own install/repair/upgrade/rollback plans before wide use.
12. AIDE should support dev/main/task branch discipline.
13. Generated outputs are not canon unless promoted.
14. Target repo truth wins over AIDE pack defaults.
15. Versions identify releases; capabilities decide compatibility.
16. External projects are pattern sources, not foundations.
17. Graph tools may help but are derived artifacts.
18. AIDE Runtime/Gateway/Hosts are deferred until QCHECK-04.
19. Live repo state must be rechecked before implementation.

20. This package is the current human and aggregator handoff.

Best next step

Verify live AIDE and Dominion heads. If Dominion remains at/near POST-CONVERGE-10E, generate a bounded WorkUnit for `invariant_units_present` and `inv_repox_rules` remediation/classification. In parallel, ensure AIDE Q35 and QCHECK-03 are complete before Q36-Q42 work.

17.3 Aide Xstack Dominion Refactor Control Plane In Chat Reader

Source: docs/archive/conversations/Refactor_Control_Plane/AIDE_XStack_Dominium_Refactor_Control_Plane__09_in_chat_reader.md

Report type: in_chat_reader

Size: 2207 bytes

17.3.1 In-Chat Reader - AIDEXStackDominiumRefactorControl__

Package overview

This package preserves the AIDE/XStack/Dominium refactor-control conversation. It contains a long human-

readable report, context transfer packet, YAML spec sheet, registers, aggregator packet, reader brief, verification audit, and future chat bootstrap prompt.

File index

- `00_manifest.md`: package index and caveats.
- `01_human_readable_report.md`: full prose report.
- `02_context_transfer_packet.md`: future-chat handoff.
- `03_spec_sheet.yaml`: machine-readable structured sheet.
- `04_registers.md`: workstreams, decisions, tasks, constraints, artifacts, risks, verification.
- `05_aggregator_packet.md`: compact cross-chat merge packet.
- `06_reader_brief.md`: short reader version.
- `07_verification_and_audit.md`: self-audit and verification queue.
- `08_future_chat_bootstrap_prompt.md`: prompt for a new chat.
- `09_in_chat_reader.md`: this guide.

Plain-English explanation

The chat decided that AIDE should become the portable repo operating layer and Dominion refactor control plane. XStack should become a strict local profile and migration layer, not be deleted or kept as public mythology. Dominion cleanup should be AIDE-controlled root recycling: inventory, classify, salvage, move, validate, and retire exceptions. AIDE itself should first build stable Profiles/Harness/Compatibility/Bridge functions before Runtime/Gateway/Hosts.

Question menu

- What should AIDE do next?
- What should Dominion do next?
- Which XStack pieces should be wrapped first?
- How should root recycling classify files?
- What are the biggest risks?
- Which repo files should be verified live?
- How do I use this package in a future chat?

Top things to preserve

- XStack remains strict Dominion profile.

- AIDE becomes repo-native operating layer.
- No wholesale root moves.
- No deleting old checks before wrapping.
- No feature work before proof status is acceptable.

Safest next actions

1. Verify live repo state.
2. Finish/classify Dominium CTest/RepoX blockers.
3. Finish AIDE Q35/QCHECK-03.
4. Build AIDE Q36-Q42.
5. Start Dominium AIDE-STRUCTURE-00 only after blocker status is understood.

Chapter 18

Release Identity And Versioning

18.1 Dominion Xstack Release Identity And Versioning Human Readable Report

Source: docs/archive/conversations/Release_Identity_and_Versioning/Dominium_XStack_Release_Identity_and_Versioning__01_human_readable_report.md

Report type: human_readable_report

Size: 37953 bytes

18.1.1 COMPLETE CHAT PRESERVATION REPORT - Dominion XStack Release Identity and Versioning

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion
 XStack Release Identity and Versioning

Row 2 Field: Date anchor; Assessment: 2026-05-27
 Australia/Melbourne

- Row 3** Field: Source scope; Assessment: This chat only unless explicitly labelled PROJECT-CONTEXT.
- Row 4** Field: Apparent access; Assessment: Partial: current visible transcript/context and uploaded prompt are available; raw hidden/full transcript outside context is not independently accessible.
- Row 5** Field: Previously generated files available?; Assessment: No prior downloadable files visible in this task; this response creates new files.
- Row 6** Field: Uploaded files or artifacts present?; Assessment: Yes: Pasted text.txt, the preservation mega-prompt.
- Row 7** Field: Contains future plans?; Assessment: Yes.
- Row 8** Field: Contains decisions?; Assessment: Yes, with some tentative directions that need confirmation.
- Row 9** Field: Contains pending tasks?; Assessment: Yes.
- Row 10** Field: Contains unresolved questions?; Assessment: Yes.
- Row 11** Field: Staleness risk; Assessment: Low for design decisions; medium for SemVer primary-source wording and OS/toolchain feasibility.

- Row 12** Field: Extraction confidence; Assessment: 4/5 for visible chat substance; lower for actual repo/build-system state.
- Row 13** Field: Safe for later aggregation?; Assessment: With caveats.
- Row 14** Field: Main limitations; Assessment: No direct repo inspection in this task; no full raw transcript beyond the conversation context; some earlier assistant claims about existing XStack files remain unverified.

The uploaded instruction file defines this as a maximum-fidelity preservation, explanation, handoff, audit, spec-prep, and export task for the current chat. The extraction below is grounded in the visible conversation context and the uploaded prompt. The report is intentionally conservative: assistant proposals are not treated as final user decisions unless the user clearly accepted, echoed, or built on them. Any claim about actual repository files, build scripts, toolchains, or OS support is marked as uncertain where the chat discussed it but this task did not independently verify it.

1. One-Page Orientation

This chat was about designing a durable release identity, versioning, compatibility, build, and packaging system

973

for Dominion/XStack. The user began from dissatisfaction with products that change versioning policies midstream, citing examples such as Windows NT, Minecraft, macOS, .NET, Linux, and TeX. The underlying concern was not aesthetic only. The user wanted a versioning system that could survive long-term without arbitrary "vibe" bumps, without getting trapped in permanent 1.x, and without overloading one number with too many incompatible meanings.

The discussion first compared Semantic Versioning with other versioning strategies, then moved into a project-specific model. Early suggestions included alternatives such as generation/epoch/feature/patch, but the conversation gradually converged on a stronger principle: one version number should not carry all truth. A large platform needs separate identities for public release naming, component compatibility, suite composition, build provenance, lifecycle/support channel, target platform, package type, and internal compatibility. This became the main conceptual outcome of the chat.

The user then clarified that Dominion/XStack has standalone products such as Stack, Tools, SDK, Engine, Game, Client, Server, Launcher, and Setup, plus suites or distributions such as All, Full, Lite, and Net. This led to a product-vs-suite distinction: products/components can have their own versions, while suites are curated bundles with human-facing release identities. The chat

also established that setup should remain a standalone product capable of running portably and then managing installation, repair, uninstall, rollback, upgrade, and migration using bundled or network sources.

A central decision was to use strict SemVer 2.0.0 only where it genuinely applies: components with declared public APIs or compatibility contracts. Examples include SDKs, engine libraries, protocol/schema/plugin surfaces, reusable runtime libraries, and stable CLI/API tools. For user-facing products and suites, the user liked the shape of SemVer - `1.2.3`, `1.2.3-beta.1`, `1.2.3+gbn.7137` - but agreed that these should be explicitly labelled as release identifiers unless they truly satisfy strict SemVer semantics.

The chat also integrated XStack's build concepts. Global Build Number (GBN) should be used for exact CI artifact identity and provenance, normally in build metadata such as `+gbn.7137`, but not as SemVer precedence. Build Identification / BII should primarily live in structured manifests, with optional compact projection into filenames or metadata. The conversation also split channel semantics: `dev`, `alpha`, `beta`, and `rc` can be prerelease ordering labels; `stable`, `lts`, `nightly`, `internal`, and `hotfix` are better treated as lifecycle or release-class metadata.

The final important shift was the user's proposal that internal compatibility should use capabilities instead of

relying only on versions. This was accepted as a strong tentative direction: versions identify releases, while capabilities/contracts decide interoperability. Examples include `save.schema@5`, `plugin.host@2`, `net.protocol@3`, and `renderer/install/pack` capabilities. This refines the earlier "compatibility axes" idea into an operational model.

The chat's future value is high because it can become the basis for a Release Constitution, SemVer Component Inventory, Suite Release Policy, Build Identity Spec, Artifact Naming Spec, Target Taxonomy, Release Manifest Schema, and Capability Compatibility Spec. The main unresolved work is to formalize these into repo-ready documents and verify them against actual XStack/RepoX build files and real platform/toolchain constraints.

2. The Story of the Conversation

The conversation began with a broad comparison of Semantic Versioning against other versioning methods. The initial answer positioned SemVer as compatibility-focused, while CalVer, serial build numbers, marketing versions, build numbers, git hashes, protocol/schema versions, and release channels each solve different problems. This established the idea that version numbers are information encodings, not merely labels.

The user then objected to versioning policy drift in real products and expressed concern about SemVer's "1.x forever" failure mode. The first proposed answer was a four-part public version scheme, `GEN.EPOCH.FEATURE.PATCH`, meant to avoid fake major bumps. That idea was useful as an intermediate step but later became less central because XStack already has GBN for dense build history and separate build identity.

The conversation then moved into how this might fit XStack. The model shifted from "better public version number" to "layered identity model." The assistant recommended preserving per-product versions, a global build number, build IDs, compatibility versions, and suite versions as separate layers. This became the foundation for later decisions.

The user suggested that suites might use a separate consumer-facing or marketing version, while each component could use stricter SemVer. The chat accepted this as a mature pattern: suites are curated bundles, while components may have technical versions. The model was refined to avoid synchronized fake versioning and to avoid treating a suite major version as universal breakage.

The user then asked if the suite version should still encode enough meaning to infer internal and external compatibility. The conversation clarified that the suite version can communicate a compatibility envelope or

release family, but exact compatibility must live in explicit metadata. This led to the idea of compatibility profiles and later capabilities.

A broader packaging discussion followed. The user listed standalone products and suites and proposed filename patterns including version, channel, build, platform, and architecture. The chat improved this by separating scope (product or suite), id, version, target, arch, and pkg kind. It also clarified that platform family and binary baseline are not the same thing: WinNT5, Linux5, or MacOSX4 may be useful support-family labels, but exact compatibility requires a tested baseline/runtime/toolchain profile.

The user then reset the discussion from scratch around SemVer 2.0.0. The chat reconstructed SemVer's core rules: public API requirement, major/minor/patch meanings, prerelease labels, build metadata, precedence, and common mistakes. The key consequences for XStack were that `stable` should not be encoded as `-stable`, `hotfix` should not normally be encoded as `-hotfix`, and GBN inside `+...` cannot determine SemVer precedence.

The user liked this distinction: strict SemVer for public APIs, explicitly non-SemVer but SemVer-shaped identifiers for end-user facing things. The chat then classified true SemVer components, product release identifiers, suite identifiers, GBN, BII, channels, and filenames. It concluded that file names are projections of

metadata, while manifests are canonical.

The chat was then summarized into a knowledge base. After that, the user proposed an even deeper internal rule: use capabilities instead of versions for compatibility. The response accepted this as a stronger model: versions identify releases; capabilities decide interoperability; GBN/BII/hash identify exact artifacts. That is the current final design direction.

3. Main Topics Discussed

Topic 1 - SemVer versus other versioning strategies The chat compared SemVer, CalVer, serial numbering, marketing versions, build numbers, git hashes, protocol/schema versions, and channels. The conclusion was that each scheme answers a different question. SemVer is strong for public API compatibility, but weak as a universal product identity. CalVer is good for recency and support windows, but weak for compatibility. Build IDs and hashes are precise provenance tools, but poor human release labels.

The user should remember that the chat did not reject SemVer. It limited SemVer to the domain where it is truthful.

Topic 2 - Versioning policy drift and "1.x hell"

The user's concern was that many projects change

versioning policy after the original scheme stops fitting. The chat identified the root cause as overloaded version strings. A scheme that tries to encode release age, compatibility, marketing generation, build identity, platform, and suite composition in one number will eventually break down.

The early four-part version proposal was superseded by a layered identity architecture.

Topic 3 - Layered release identity architecture

The main design conclusion was that release identity should be layered: product/suite version, component version, build identity, compatibility/capability contracts, lifecycle channel, target/platform, package kind, and manifest metadata should remain separate. This is the central principle to carry forward.

Topic 4 - Strict SemVer 2.0.0 for public APIs

The chat rebuilt SemVer from first principles and decided that strict SemVer should apply only to components with declared public contracts. Candidate true-SemVer entities include SDKs, engine libraries, tool APIs/CLIs, protocol libraries, plugin hosts, schema libraries, and reusable runtime libraries. It remains unresolved which exact repo modules qualify.

Topic 5 - SemVer-shaped release IDs for non-SemVer products and suites The user liked the visual shape of SemVer even for consumer-facing items. The chat accepted `X.Y.Z[-pre][+build]` as a useful shape, but required that non-SemVer products/suites be documented as release identifiers rather than API-compatibility promises.

Topic 6 - Products versus suites The chat separated standalone products from suites/distributions. Products include Stack, Tools, SDK, Engine, Game, Client, Server, Launcher, and Setup. Suites include All, Full, Lite, Net, User, Dev, and similar distributions. Metadata should include `scope=product|suite` to avoid ambiguity.

Topic 7 - GBN and BII GBN should identify CI/public/internal builds and appear as provenance, e.g. `+gbn.7137`, but it must not be used as SemVer precedence. BII should primarily be structured manifest metadata, because it may encode lane, target, kind, configuration, and other build identity fields. The exact BII schema remains unresolved.

Topic 8 - Channels, lifecycle, and release class The chat separated prerelease ordering labels from lifecycle/support metadata. `dev`, `alpha`, `beta`, and `rc`

may belong in the version string. `stable`, `lts`, `nightly`, `internal`, `archival`, `hotfix`, `security`, and `rollback` should usually be lifecycle or release-class metadata.

Topic 9 - Artifact naming and manifests The proposed filename grammar became `Dominium-<scope>-<id>-<version>-<target>-<arch>-<pkg>.<ext>`. Examples included product SDKs, setup installers, and suite bundles. The chat stressed that filenames are not canonical truth; manifests are.

Topic 10 - Platform and architecture taxonomy The user proposed platform buckets for DOS, Windows, classic Mac, macOS, and Linux. The chat concluded that coarse platform labels are useful, but exact compatibility requires target baselines and runtime/toolchain profiles. Linux kernel major alone is not a reliable binary target. Windows 9x/NT families often need separate targets rather than one generic Windows binary.

Topic 11 - Capabilities for internal compatibility The final conceptual refinement was that internal compatibility should use capabilities/contracts rather than version ranges alone. A version says what release something is; a capability says what it can do. This is especially useful for saves, packs, plugins, network handshakes, installers, renderers, and platform/runtime

constraints.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted a versioning/release identity policy that would not need to change halfway through. They wanted to avoid SemVer stagnation, arbitrary major bumps, and product-history confusion. Later, the user wanted the conversation reconstructed into a knowledge base and preservation package for future aggregation.

4.2 Inferred Goals The user likely wanted a repo-ready architecture for XStack/Dominium releases that can support many products, suites, channels, build identities, package types, targets, and internal compatibility checks. This is inferred from the repeated movement from theory into concrete product/suite/build/platform examples.

4.3 Goals That Changed Over Time The conversation began as a comparison of versioning schemes. It evolved into a release identity architecture. It then became a SemVer adaptation exercise. Finally, it became a capability-based compatibility architecture layered under versioning.

4.4 Goals Still Unresolved The main unresolved goals are to produce formal specs: Release Constitution, SemVer Component Inventory, Suite Release Policy, Build Identity Spec, Channel/Lifecycle Spec, Artifact Naming Spec, Manifest Schema, Target Taxonomy, and Capability Registry.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Use a layered release identity architecture rather than one universal version string.; Status: Accepted direction; Why it mattered: A single string cannot safely encode release identity, compatibility, provenance, support class, platform, package type, and suite composition.; Confidence: 4; Label: FACT/INFERENCE

Row 2 ID: DECISION-02; Decision: Strict SemVer 2.0.0 applies only where there is a declared public API/contract.; Status: Accepted direction; Why it mattered: SemVer is compatibility-centric and requires a public API; applying it everywhere would create fake semantics.; Confidence: 5; Label: FACT

Row 3 ID: DECISION-03; Decision: End-user products and suites may use SemVer-shaped identifiers without claiming strict SemVer semantics.; Status:

Accepted direction; Why it mattered: Preserves readable familiar shape while avoiding false API compatibility claims.; Confidence: 5; Label: FACT

Row 4 ID: DECISION-04; Decision: Suites/distributions are separate from products/components.; Status: Accepted direction; Why it mattered: Full/Lite/Net/All are curated distributions, not equivalent to Engine/SDK/Client.; Confidence: 4; Label: FACT

Row 5 ID: DECISION-05; Decision: Suite versions should be human-curated and not strict SemVer.; Status: Accepted direction; Why it mattered: A suite is a tested bundle/release family, not one API.; Confidence: 4; Label: FACT/INFERENCE

Row 6 ID: DECISION-06; Decision: GBN belongs in build metadata/manifests as provenance, not SemVer precedence.; Status: Accepted direction; Why it mattered: SemVer build metadata after + does not affect precedence; GBN is exact build identity/order outside SemVer ordering.; Confidence: 5; Label: FACT

Row 7 ID: DECISION-07; Decision: BII should primarily be structured manifest metadata, with optional compact projection.; Status: Tentative accepted direction; Why it mattered: Full BII may

become too structured/long for version strings.;
Confidence: 4; Label: INFERENCE

Row 8 ID: DECISION-08; Decision: Do not encode stable as -stable and do not encode hotfix as a prerelease suffix.; Status: Accepted direction; Why it mattered: Under SemVer, -stable and -hotfix are prerelease identifiers below the plain release; hotfix should normally be a patch release plus release-class metadata.; Confidence: 5; Label: FACT

Row 9 ID: DECISION-09; Decision: Split prerelease ordering labels from lifecycle/support metadata.; Status: Accepted direction; Why it mattered: dev/alpha/beta/rc affect ordering; stable/lts/nightly/internal/hotfix describe policy/lifecycle.; Confidence: 5; Label: FACT

Row 10 ID: DECISION-10; Decision: Filenames are readable projections; manifests are canonical truth. ; Status: Accepted direction; Why it mattered: Filenames cannot safely carry all metadata or evolve as the only authority.; Confidence: 4; Label: FACT/INFERENCE

Row 11 ID: DECISION-11; Decision: Separate support family from exact binary target baseline.; Status: Accepted direction; Why it mattered: A broad family like Linux5 or WinNT5 is not always an

exact compatibility contract.; Confidence: 4; Label: FACT/INFERENCE

Row 12 ID: DECISION-12; Decision: Use capabilities/ contracts internally for compatibility rather than relying on versions alone.; Status: Tentative direction; Why it mattered: Capabilities directly express what can interoperate, especially with backports, partial implementations, optional features, and multidimensional compatibility.; Confidence: 3; Label: INFERENCE

The decisions above are not all equally final. The strongest decisions are the separation of strict SemVer from SemVer-shaped release identifiers, the separation of product/suite/build/compatibility identities, and the rule that GBN/BII are provenance rather than SemVer precedence. The most tentative direction is the internal capability model, because the user proposed it and the assistant endorsed it, but the conversation has not yet produced a formal capability schema or user-ratified registry.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

Row 1 ID: REJECTED-01; Option: Pure SemVer for everything.; Status: Rejected/deprioritised; Reason: Does not fit suites, apps, packs, installers,
987

and multidimensional compatibility.; Final or tentative?: Tentative but strong; Reconsider conditions: Only reconsider for entities with declared public APIs.; Related workstream: WORKSTREAM-01; Label: FACT

Row 2 ID: REJECTED-02; Option: One universal version number for suite, products, builds, compatibility, and packages.; Status: Rejected; Reason: Overloads one field and causes future policy drift.; Final or tentative?: Strong; Reconsider conditions: Only tiny projects could accept this.; Related workstream: WORKSTREAM-01; Label: FACT/INFERENCE

Row 3 ID: REJECTED-03; Option: Four-part public version as the main answer.; Status: Superseded; Reason: GBN already provides dense chronological/build identity, so public release need not add fourth field for density.; Final or tentative?: Tentative; Reconsider conditions: May reappear for specific products if required by platform conventions.; Related workstream: WORKSTREAM-01; Label: FACT/INFERENCE

Row 4 ID: REJECTED-04; Option: Encoding stable as -stable.; Status: Rejected; Reason: Under SemVer, any hyphen suffix is prerelease and sorts below stable.; Final or tentative?: Strong; Reconsider

conditions: Do not reconsider unless abandoning SemVer ordering semantics entirely.; Related workstream: WORKSTREAM-05; Label: FACT

Row 5 ID: REJECTED-05; Option: Encoding hotfix as -hotfix.; Status: Rejected; Reason: Hotfix is usually a patch release plus release_class metadata, not a prerelease of the old version.; Final or tentative?: Strong; Reconsider conditions: Only use for unreleased hotfix candidates if explicitly defined.; Related workstream: WORKSTREAM-05; Label: FACT

Row 6 ID: REJECTED-06; Option: Using Linux kernel major alone as exact binary target.; Status: Rejected as exact target; Reason: Linux compatibility depends heavily on libc/runtime/toolchain/userland.; Final or tentative?: Strong; Reconsider conditions: Can remain a coarse support family label.; Related workstream: WORKSTREAM-07; Label: FACT/INFERENCE

Row 7 ID: REJECTED-07; Option: Generic one-size Windows binary target.; Status: Rejected as default; Reason: Windows 9x/NT families and old baselines differ materially.; Final or tentative?: Tentative; Reconsider conditions: Could exist for very constrained CLI/tools after testing.; Related workstream: WORKSTREAM-07; Label: FACT/

INFERENCE

Row 8 ID: REJECTED-08; Option: Filenames as canonical release truth.; Status: Rejected; Reason: Filenames are too lossy and mutable.; Final or tentative?: Strong; Reconsider conditions: Filename parsing may be used as convenience only. ; Related workstream: WORKSTREAM-06; Label: FACT/INFERENCE

These rejected or superseded options matter because future assistants may otherwise reintroduce them. The biggest trap is suggesting "just use SemVer everywhere." That was explicitly narrowed. Another trap is treating **stable** or **hotfix** as prerelease suffixes even after SemVer ordering was explained.

7. Important Reasoning, Rationale, and Tradeoffs

The visible rationale was that version numbers are compact signals, but compactness becomes dangerous when one field carries too many meanings. SemVer is valuable because it ties version changes to public API compatibility, but that value disappears when applied to things without clear public APIs. Suite versions are valuable because humans need a simple release label, but they should not be mistaken for exact internal compatibility truth.

The main tradeoff was between readability and correctness. The user liked SemVer's familiar shape, and the design preserves that shape. The correction is semantic: only true API/contract-bearing components claim strict SemVer. Products and suites may use SemVer-shaped identifiers but are explicitly not pure SemVer if their compatibility promises are not SemVer public-API promises.

GBN created another tradeoff. It gives dense build identity and repo-wide chronology, but SemVer build metadata does not affect precedence. The chosen model keeps GBN as provenance while requiring version/pre-release fields to handle release ordering.

Capabilities were introduced to reduce overreliance on versions for compatibility. This solves backports, partial implementations, optional features, multi-surface systems, and cross-product interoperability better than version ranges alone. The risk is capability chaos unless the registry is governed.

8. Plans, Future Work, and Next Steps

The immediate future work is to convert this design into formal repo/spec artifacts. Recommended order:

1. **Release Constitution:** define permanent field meanings and the rule that field meanings cannot be reinterpreted.

2. **Entity Classification / SemVer Component Inventory:** classify every product, suite, API, schema, protocol, plugin surface, and build field.
3. **Suite Release Policy:** define exactly what suite X.Y.Z means.
4. **Build Identity Spec:** define GBN, BII, git SHA, local build, CI build, and manifest usage.
5. **Channel/Lifecycle Spec:** define prerelease labels, lifecycle/support lanes, and release classes.
6. **Artifact Naming Spec:** freeze the filename grammar and package-kind vocabulary.
7. **Release Manifest Schema:** make manifests canonical.
8. **Target Taxonomy Spec:** define support families, exact baselines, architectures, runtime/toolchain profiles.
9. **Capability Compatibility Spec:** define capability namespaces, provided/required capabilities, versioned contracts, negotiation, migration, and validation.

The major blockers are repo verification and formal user decisions on suite semantics, BII schema, target taxonomy, and capability registry design.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences The user prefers direct, audit-ready, source-grounded, high-fidelity reporting. They dislike overcompressed handoffs and machine-readable-only outputs. They want versioning policy stability and dislike arbitrary version policy shifts. They explicitly like strict SemVer for public APIs and SemVer-shaped identifiers for other release IDs.

9.2 Inferred Constraints and Preferences The user values long-term maintainability over short-term prettiness. They prefer explicit metadata over hidden inference. They want future assistants to preserve tentative status and not silently turn brainstorm into decisions.

9.3 Uncertain or Unestablished Preferences It is still uncertain how strict the user wants suite X.Y.Z field meanings to be, whether capabilities should fully replace compatibility profiles or coexist with them, and how much old-platform support should be real versus aspirational.

10. Files, Artifacts, Outputs, and Prompts

Row 1 ID: ARTIFACT-01; Artifact / file / prompt /
output: Uploaded preservation mega-prompt:

Pasted text.txt; Type: Prompt/instruction file; Purpose: Defines this preservation/export task.; Status: Provided in current turn.; Origin: User upload; Carry forward?: Yes; Notes: 1109-line prompt requiring human-readable report, registers, spec sheet, aggregator packet, self-audit, and file exports.; Label: FACT

Row 2 ID: ARTIFACT-02; Artifact / file / prompt / output: In-chat versioning summary produced before upload; Type: Chat output; Purpose: Summarized decisions and unresolved issues up to that point.; Status: Visible transcript; Origin: Assistant output; Carry forward?: Yes; Notes: Served as prior knowledge-base baseline, but must not be treated as user-final decisions where not accepted.; Label: FACT

Row 3 ID: ARTIFACT-03; Artifact / file / prompt / output: Canonical version examples; Type: Examples/spec fragments; Purpose: Illustrate strict SemVer and SemVer-shaped identifiers.; Status: Visible discussion; Origin: Chat content; Carry forward?: Yes; Notes: Examples include 1.2.3, 1.2.3-beta.1, +gbn.7137, +sha.deadbeef.; Label: FACT

Row 4 ID: ARTIFACT-04; Artifact / file / prompt / output: Proposed artifact filename grammar; Type:

Spec fragment; Purpose: Readable artifact naming model.; Status: Visible discussion; Origin: Chat content; Carry forward?: Yes; Notes: Pattern: `Dominium-<scope>-<id>-<version>-<target>-<arch>-<pkg>.<ext>`; not final.; Label: FACT/ INFERENCE

Row 5 ID: ARTIFACT-05; Artifact / file / prompt / output: Proposed capability model examples; Type: Spec fragment; Purpose: Internal compatibility strategy.; Status: Visible discussion; Origin: Chat content; Carry forward?: Yes; Notes: Examples: `save.schema@5`, `plugin.host@2`, `net.protocol@3`. Tentative.; Label: INFERENCE

This preservation task creates the first actual downloadable package in this chat. Prior artifacts were primarily in-chat examples, summaries, and proposed spec fragments.

11. Open Questions and Unresolved Issues

Row 1 ID: QUESTION-01; Question / issue: Which exact products/components are strict SemVer components?; Why it matters: Needed before Release Constitution can be enforced.; Known information: Candidates identified.; Unknown information: Final repo/component inventory and public API boundaries.; Resolution path: Audit

repo and create SemVer Component Inventory.;
Priority: P0; Related workstream:
WORKSTREAM-02; Label: FACT

Row 2 ID: QUESTION-02; Question / issue: What exactly does suite version X.Y.Z mean?; Why it matters: Prevents future drift and marketing reinterpretation.; Known information: It is human-curated and non-SemVer.; Unknown information: Field semantics still not final.; Resolution path: Choose and document suite-major/train/update or equivalent.; Priority: P0; Related workstream: WORKSTREAM-03; Label: FACT

Row 3 ID: QUESTION-03; Question / issue: What is the exact BII schema?; Why it matters: BII must integrate with manifests, filenames, and support/debugging.; Known information: It should be structured metadata first.; Unknown information: Fields, tokenization, allowed values.; Resolution path: Inspect existing XStack/RepoX build identity and formalize.; Priority: P1; Related workstream: WORKSTREAM-04; Label: FACT/INFERENCE

Row 4 ID: QUESTION-04; Question / issue: What is the canonical artifact filename grammar?; Why it matters: Needed for releases/downloads.; Known information: Strong pattern proposed.; Unknown

information: Final delimiters/case/fields/package kinds.; Resolution path: Draft naming spec and test examples.; Priority: P1; Related workstream: WORKSTREAM-06; Label: FACT

Row 5 ID: QUESTION-05; Question / issue: What should the manifest schema contain and validate?; Why it matters: Manifest is canonical truth.; Known information: Candidate fields listed.; Unknown information: Exact schema, required/optional fields, versioning.; Resolution path: Create YAML/JSON schema draft.; Priority: P1; Related workstream: WORKSTREAM-06; Label: FACT

Row 6 ID: QUESTION-06; Question / issue: What target family/baseline taxonomy should be used for DOS/Windows/Mac/Linux?; Why it matters: Avoids overclaiming binary compatibility.; Known information: Need to separate support family and exact baseline.; Unknown information: Exact labels and feasibility.; Resolution path: Research/test compiler/runtime support.; Priority: P1; Related workstream: WORKSTREAM-07; Label: FACT/UNVERIFIED

Row 7 ID: QUESTION-07; Question / issue: Should compatibility profiles remain distinct from capability contracts, or are capabilities the profile mechanism?; Why it matters: Affects internal

resolver design.; Known information: Both ideas discussed.; Unknown information: Final relationship.; Resolution path: Design capability registry and map to suite manifests.; Priority: P1; Related workstream: WORKSTREAM-08; Label: INFERENCE

The most important unresolved issue is formal classification. Without knowing which entities are strict SemVer components, suite versions, product release IDs, build fields, or capabilities, the rest of the policy cannot be enforced safely.

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Row 1 ID: RISK-01; Risk / failure mode: Future assistant treats SemVer-shaped product/suite IDs as strict SemVer.; Consequence: False compatibility claims and wrong bump advice.; Likelihood: Medium; Severity: High; Mitigation: Always classify entity type first.; Related workstream: WORKSTREAM-01; Label: FACT/INFERENCE

Row 2 ID: RISK-02; Risk / failure mode: GBN used as SemVer ordering despite being in +build metadata. ; Consequence: Incorrect update ordering/resolver behavior.; Likelihood: Medium; Severity: High; Mitigation: Keep separate ordering fields and tests.

; Related workstream: WORKSTREAM-04; Label: FACT

Row 3 ID: RISK-03; Risk / failure mode: BII becomes an overloaded version-string dump.; Consequence: Fragile filenames and unreadable versions.; Likelihood: Medium; Severity: Medium; Mitigation: Keep full BII in manifest; expose compact token only.; Related workstream: WORKSTREAM-04; Label: INFERENCE

Row 4 ID: RISK-04; Risk / failure mode: OS/platform labels overclaim compatibility.; Consequence: Artifacts fail on older claimed systems.; Likelihood: High; Severity: High; Mitigation: Separate support_family and target_baseline; test baselines.; Related workstream: WORKSTREAM-07; Label: FACT/UNVERIFIED

Row 5 ID: RISK-05; Risk / failure mode: Capabilities become undocumented string soup.; Consequence: Compatibility resolver becomes arbitrary and hard to audit.; Likelihood: Medium; Severity: High; Mitigation: Create namespace, ownership, lifecycle, and validation rules.; Related workstream: WORKSTREAM-08; Label: INFERENCE

Row 6 ID: RISK-06; Risk / failure mode: Tentative ideas are merged as final requirements.; Consequence: Spec book contains unstable or

wrong commitments.; Likelihood: Medium;
Severity: High; Mitigation: Preserve status labels
and require user confirmation.; Related
workstream: ALL; Label: FACT/INFERENCE

Row 7 ID: RISK-07; Risk / failure mode: Human
narrative is replaced by registers only.;
Consequence: User cannot understand old chat
without rereading transcript.; Likelihood: Medium;
Severity: Medium; Mitigation: Keep prose report
as primary artifact.; Related workstream:
Documentation; Label: FACT

The highest-risk misunderstanding is collapsing the
layered model back into one universal version scheme.
Future assistants should classify the entity first, then
apply the correct policy.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat contributes the release identity doctrine for
Dominium/XStack. It should feed into chapters on
release engineering, versioning, build identity, packaging,
platform targeting, compatibility, capability negotiation,
installer/setup workflow, and artifact manifests. The
chat's strongest formal-requirement candidates are:
strict SemVer only for declared public APIs; SemVer-
shaped identifiers for products/suites as separate release

IDs; GBN/BII as provenance; manifests as canonical truth; channels split from lifecycle metadata; and capabilities/contracts as internal compatibility truth.

Items that should remain background until confirmed include exact suite field semantics, platform target families, and capability registry details.

14. What I Should Remember

- The core conclusion is layered identity: versions, suite labels, GBN, BII, capabilities, channels, targets, and package kinds each have separate jobs.
- Strict SemVer is for declared public APIs/contracts, not everything.
- User-facing products and suites may use SemVer-shaped IDs without claiming SemVer compatibility semantics.
- GBN identifies builds; it does not create SemVer precedence when placed after `+`.
- BII belongs primarily in structured metadata/manifests.
- `stable` and `hotfix` should not be encoded as prerelease suffixes.
- Filenames are projections. Manifests are truth.

- Platform support families are not exact binary baselines.
- Internal compatibility should likely be capability/contract-based.
- The next real work is formal spec writing and repo verification.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- Explain the difference between strict SemVer and SemVer-shaped release identifiers again using Dominion examples.
- What is the central design principle of this chat in one page?

15.2 Decisions

- Which decisions in this chat are firm, and which are only tentative?
- Which assistant suggestions have not yet been accepted as final user decisions?

15.3 Tasks and Next Actions

- Draft the Release Constitution from this chat.

- Create the SemVer Component Inventory table for Stack, Tools, SDK, Engine, Game, Client, Server, Launcher, and Setup.

15.4 Artifacts and Files

- Turn the artifact filename grammar into a normative spec.
- Convert the manifest field list into a JSON/YAML schema.

15.5 Risks and Verification

- What external facts need verification before this becomes a repo standard?
- Which OS/platform target assumptions are most likely to be wrong?

15.6 Future Spec Book / Aggregation

- Which sections of a master Dominion Project Spec Book should this chat feed into?
- What should an aggregator preserve versus leave tentative?

15.7 Deep-Dive Questions Specific to This Chat

- Design the internal capability registry for saves, packs, plugins, networking, renderers, installers, and runtimes.
- Decide whether compatibility profiles should be derived from capability sets or exist as separate named profiles.

16. Compact Human Summary

This chat developed a durable versioning and release identity strategy for Dominion/XStack. The user began with frustration about products that change versioning schemes over time and with SemVer's tendency to create either permanent 1.x stagnation or arbitrary major bumps. The conversation first compared versioning methods, then moved into a broader architecture: one version number cannot safely encode release identity, compatibility, build provenance, suite composition, support status, platform target, package type, and internal capability.

The strongest conclusion was to use a layered release identity system. Strict SemVer 2.0.0 should apply only to components with declared public APIs or compatibility contracts, such as SDKs, engine libraries, protocol/schema/plugin surfaces, reusable runtime libraries, and stable CLI/API tools. User-facing

products and suites can still use a familiar SemVer-shaped syntax such as `1.2.3`, `1.2.3-beta.1`, and `1.2.3+gbn.7137`, but those identifiers must be documented as product or suite release IDs, not strict API SemVer unless a real public contract exists.

The chat separated products from suites. Products include Stack, Tools, SDK, Engine, Game, Client, Server, Launcher, and Setup. Suites/distributions include All, Full, Lite, Net, User, Dev, and similar bundles. The recommended metadata should include `scope=product|suite` and an `id`, so that `Engine` and `Full` are not flattened into the same namespace.

The chat integrated existing XStack ideas. GBN should identify exact CI/public/internal builds and may appear as build metadata, e.g. `+gbn.7137`, but it must not be used as SemVer precedence. BII should primarily be structured manifest metadata, with optional compact projection in filenames. Git SHA can identify local builds, e.g. `+sha.deadbeef`. The build/release system should distinguish exact provenance from release ordering.

Channels were split into two meanings. `dev`, `alpha`, `beta`, and `rc` can be prerelease ordering labels inside the version. `stable`, `lts`, `nightly`, `internal`, `archival`, and `hotfix` should generally be lifecycle or release-class metadata outside the version. A stable release is simply `1.2.3`, not `1.2.3-stable`. A hotfix should normally be

a new patch version plus metadata, not `1.2.3-hotfix`.

Artifact names should be deterministic but not authoritative. The preferred filename pattern is `Dominium-<scope>-<id>-<version>-<target>-<arch>-<pkg>.<ext>`, but the manifest is the canonical truth. The manifest should contain product/suite identity, version, lifecycle, release class, target family, target baseline, architecture, package kind, GBN, BII, git hash, compatibility/capability information, and suite membership.

Platform targeting remains unresolved. The chat concluded that coarse families like WinNT5, WinNT10, MacOSX4, Linux5, or DOS5 are useful support labels, but exact binary compatibility needs target baselines and runtime/toolchain profiles. Linux kernel major alone is not enough. Windows 9x/NT/NT5/NT6/NT10 likely need separate build lanes or at least explicit tested baselines.

The final conceptual refinement was internal capability-based compatibility. Versions should identify releases; capabilities should decide whether things interoperate. Examples include `save.schema@5`, `plugin.host@2`, `net.protocol@3`, renderer capabilities, installer capabilities, pack schema capabilities, and runtime/platform capabilities. This is a tentative but strong direction and should become a formal capability registry if adopted.

The best next action is to draft a Release Constitution and SemVer Component Inventory, then formalize GBN/BII, channel/lifecycle, artifact naming, manifest schema, target taxonomy, and capability registry.

18.2 **Dominium Xstack Release Identity And Versioning Reader Brief**

Source: docs/archive/conversations/Release_Identity_and_Versioning/Dominium_XStack_Release_Identity_and_Versioning__06_reader_brief.md

Report type: reader_brief

Size: 2328 bytes

18.2.1 **Reader Brief - Dominium XStack Release Identity and Versioning**

What this chat was about

This chat designed a release identity/versioning architecture for Dominium/XStack. It moved from general versioning theory to a concrete layered model that separates strict SemVer components, SemVer-shaped product/suite release IDs, GBN/BII build provenance, channels/lifecycle, artifact names, platform

targets, manifests, and capability-based compatibility.

Top 20 things to know

1. Do not solve release identity with one number.
2. Strict SemVer is only for declared public APIs/contracts.
3. Products and suites may use SemVer-shaped IDs without strict SemVer semantics.
4. Products and suites are separate entity classes.
5. GBN is build provenance, not SemVer precedence.
6. BII should be structured manifest metadata.
7. Git SHA can identify local builds.
8. `stable` should not be encoded as `-stable`.
9. Hotfixes should normally be patch releases plus metadata.
10. `dev/alpha/beta/rc` are prerelease labels.
11. `stable/lts/nightly/internal/hotfix` are lifecycle/release-class metadata.
12. Filenames are projections, not canonical truth.
13. Manifests are canonical.

14. Package kind should be explicit.
15. Support family and binary target baseline are separate.
16. Linux kernel major alone is not an exact target.
17. Windows legacy families likely need separate baselines.
18. Setup is a standalone product with install/repair/rollback/migration duties.
19. Internal compatibility should likely use capabilities/contracts.
20. The next action is Release Constitution + SemVer Component Inventory.

Decisions

See Decision Register in file 04.

Pending tasks

See Task Register in file 04. Highest priority: Release Constitution, SemVer Component Inventory, suite version policy, GBN/BII spec, capability registry.

Open questions

Exact suite semantics, exact SemVer component list, BII schema, filename grammar, manifest schema, target taxonomy, and capability registry.

Artifacts

The uploaded prompt and this generated package are the main artifacts.

Verification items

Verify SemVer primary spec, repo/build files, OS/toolchain feasibility, final product/suite inventory, and capability-profile relationship.

Best next step

Draft the Release Constitution and SemVer Component Inventory.

18.3 Dominion Xstack Release Identity And Versioning In Chat Reader

Source: docs/archive/conversations/Release_Identity_and_Versioning/Dominium_XStack_

Release_Identity_and_Versioning__09_in_chat_reader.md

Report type: in_chat_reader

Size: 2329 bytes

18.3.1 In-Chat Reader - Dominion XStack Release Identity and Versioning

Package overview

This package preserves the chat's versioning/release-identity design work for later human reading and aggregation. It includes a full report, context transfer packet, YAML-style spec sheet, structured registers, aggregator packet, reader brief, verification/audit file, bootstrap prompt, and this in-chat reader.

File index

File	Purpose
00_manifest	Package inventory and caveats.
01_human_readable_report	Main narrative report, sections 0-16.
02_context_transfer_packet	Future-chat handoff.
03_spec_sheet	YAML-style aggregation/spec data.
04_registers	Workstream/decision/task/constraint/open-question/artifact/risk registers.
05_aggregator_packet	Compact merge packet for a master aggregator chat.
06_reader_brief	Short human overview.
07_verification_and_audit	Self-audit and verification queue.
08_future_chat_bootstrap_prompt	Prompt for continuing in a new chat.
09_in_chat_reader	Guide to this package.

Plain-English explanation

This chat decided that Dominionum/XStack should use a layered identity architecture: strict SemVer for public API components, SemVer-shaped release IDs for products/suites, GBN/BII/hash for exact provenance, manifests for canonical truth, and capabilities/contracts for internal compatibility. The exact suite semantics, BII schema, target taxonomy, manifest schema, and capability registry still need formal work.

Question menu

- What is the Release Constitution implied by this chat?
- Which entities should be strict SemVer components?
- What should suite X.Y.Z mean?
- How should GBN and BII appear in metadata and filenames?
- What should the manifest schema be?
- What target families and baselines should exist?
- How should capabilities/contracts be named and negotiated?

Top things to preserve

Preserve layered identity, strict SemVer scoping, product/suite separation, GBN/BII provenance, lifecycle/prerelease split, manifest authority, and capability-based internal compatibility.

Safest next actions

Start with Release Constitution and SemVer Component Inventory. Then define suite policy, build identity,

channel/lifecycle, artifact names, manifest schema, target taxonomy, and capabilities.

Chapter 19

Timekeeping And 2038
Resilience

19.1 Dominion Timekeeping And
 2038 Resilience Human
 Readable Report

Source: docs/archive/conversations/Timekeeping_
and_2038_Resilience/Dominium_Timekeeping_and_
2038_Resilience__01_human_readable_report.md

Report type: human_readable_report

Size: 24741 bytes

19.1.1 COMPLETE CHAT PRESERVATION
 REPORT - Dominion Timekeeping
 and 2038 Resilience

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion
Timekeeping and 2038 Resilience

Row 2 Field: Date anchor; Assessment: 2026-05-27
Australia/Melbourne

Row 3 Field: Source scope; Assessment: This chat only unless labelled PROJECT-CONTEXT

Row 4 Field: Apparent access; Assessment: Partial-to-substantial: visible current transcript and tool outputs are available; hidden/deleted transcript is not guaranteed

Row 5 Field: Previously generated files available?; Assessment: No, before this preservation task; files are created by this task

Row 6 Field: Uploaded files or artifacts present?; Assessment: Yes: Pasted `text.txt`, containing the preservation/export prompt
`?filecite?turn30file0?`

Row 7 Field: Contains future plans?; Assessment: Yes

Row 8 Field: Contains decisions?; Assessment: Yes, mostly assistant recommendations and repo-supported conclusions; few explicit user acceptances

Row 9 Field: Contains pending tasks?; Assessment: Yes

Row 10 Field: Contains unresolved questions?; Assessment: Yes

Row 11 Field: Staleness risk; Assessment: Medium: platform/library facts and repository state can change

Row 12 Field: Extraction confidence; Assessment: 4/5

Row 13 Field: Safe for later aggregation?; Assessment:
With caveats

Row 14 Field: Main limitations; Assessment: The full GitHub repository was not exhaustively audited; earlier external facts were not reverified in this preservation turn; some conclusions are assistant recommendations rather than explicit user decisions.

The visible chat is enough to reconstruct the main technical arc: Y2038 risk, durable timestamp representation, cross-platform time architecture, and application to Dominion. The main caveat is that the Dominion audit was selective: the assistant fetched and read several key docs/source files, but not every backend, serializer, save format, or replay path. The preservation prompt itself was uploaded as `Pasted text.txt` and is treated as a user-provided instruction artifact. [?filecite?turn30file0?](#)

1. One-Page Orientation

This chat was mainly about making timekeeping code survive old platforms, future dates, deterministic simulation requirements, and project-scale architecture constraints. It began with a practical question: what

happens to 32-bit operating systems and 32-bit applications after 2038? The user wanted to know whether they would stop working completely, whether critical functions would fail, whether rolling back the clock would work, and whether unsigned 32-bit or signed 64-bit integers could be used in 16-bit and 32-bit code to avoid the problem.

The conversation then moved from the general 2038 problem into representation design. The user clarified that they did not need millions of years of date support, only roughly 10,000 years into the future and past. This turned the problem into a range/precision tradeoff: with one signed 64-bit scalar, the chosen tick quantum determines the span. The assistant recommended either a single signed 64-bit scalar using 100 ns or microsecond ticks, or, preferably where possible, a split representation using `int64 seconds + int32 nanoseconds`.

The user then asked for the most compatible framework for cross-platform software intended to compile and run across past, present, and future target machines and operating systems. The answer shifted from arithmetic to architecture: durable code should not use one time type for everything. It should separate absolute instants, durations, monotonic elapsed-time measurement, and civil/local calendar time. This separation is the core future-proofing idea from the chat. It avoids not just Y2038, but also Y2K-style date-format bugs, wall-clock

jumps, timezone rule changes, and replay nondeterminism.

The final technical stage applied this framework to Dominium. The user asked the assistant to read the attached docs/code or the GitHub repository `julesc013/dominium`. The assistant used the GitHub connector to inspect selected Dominium documents and source files. The key finding was that Dominium already largely follows the correct model. Its authoritative simulation time is logical ACT, not wall-clock. Its core ACT type is signed 64-bit. Its platform runtime DSYS exposes monotonic microsecond timing for runtime/app pacing. Distributed ordering is based on logical ticks and deterministic keys, not host time. Observer/replay clocks are documented as derived and non-authoritative. Civil/astronomical conversions exist only as placeholder projections.

The main outcome is not that Dominium needs a conceptual rewrite. The outcome is that Dominium's existing model is directionally correct, but needs boundary hardening: audit all DSYS backend timers, freeze ACT units and serialization semantics, prevent platform event timestamps from becoming authoritative, keep build/provenance timestamps out of deterministic state, and design civil/astronomical time as a projection layer.

The preservation task at the end changes the purpose of

the chat from discussion to archive. The user uploaded a detailed preservation prompt requiring a human-readable report, structured registers, a context-transfer packet, a YAML-style spec sheet, an aggregator packet, audit sections, and downloadable files. This report is therefore both a reader-friendly reconstruction and a future-spec handoff.

A future assistant should understand that this chat contributes a time architecture doctrine for Dominion: **ACT is authority; DSYS time is runtime-only; observer clocks are derived; civil/astronomical time is projection-only; wall-clock time must never drive authoritative ordering.**

2. The Story of the Conversation

2.1 From 2038 anxiety to exact failure model The chat began with a broad concern about what would happen after 2038 to 32-bit operating systems and applications. The user's framing included several possible outcomes: complete failure, partial critical failure, clock rollback as workaround, or no meaningful breakage.

The assistant narrowed the issue: the dangerous case is not "32-bit" by itself, but signed 32-bit Unix-style time in APIs, ABIs, file formats, and persistent data. This distinction mattered because it prevented an overbroad

conclusion. Some 32-bit systems and applications may keep working if they use safe time representations; others may fail in time-related functions.

2.2 From "avoid 2038" to "choose a durable representation" The user then clarified a design target: roughly 10,000 years forward and backward was enough. The assistant explained the range/detail tradeoff for one signed 64-bit scalar. Nanoseconds are too fine for that range if stored as one signed 64-bit count; 100 ns ticks or microseconds are practical. The assistant also distinguished scalar storage from split storage, recommending seconds plus nanoseconds when the design is not forced into one integer.

2.3 From representation to architecture The user then asked for the "most compatible" combination of timekeeping for cross-platform software architecture, including past and future target machines. The assistant answered with a layered model: absolute instant, duration, monotonic elapsed time, and civil/local time. This was the main conceptual upgrade of the chat. It broadened the issue from integer width to semantic separation.

2.4 Dominion-specific application The user then asked how the timekeeping framework applies to

Dominium and asked for docs/code to be read. The assistant searched and fetched Dominium repository files, including timing docs, runtime loop docs, distributed time model, deterministic ordering policy, macro time model, observer clocks, app runtime code, DSYS system code, ACT core code, cross-shard schema/log docs, and related tests.

The assistant concluded that Dominium already largely follows the right strategy. The authoritative engine clock is logical and signed 64-bit. DSYS/platform time is runtime-only. Cross-shard ordering uses logical ticks and deterministic keys. Observer clocks are derived. The remaining work is edge hardening.

2.5 Preservation task The user then uploaded a detailed prompt requiring this preservation package. The task became a maximum-fidelity archive and handoff for the current chat.

3. Main Topics Discussed

Topic 1 - The 2038 problem The first topic was the post-2038 behavior of 32-bit operating systems and applications. The chat established that 2038 is a representation problem: signed 32-bit seconds since 1970 overflow in January 2038. The important conclusion was that not all 32-bit systems stop working, but vulnerable

time-related paths can fail. Rolling back the clock may keep some isolated local systems running but is not a valid general solution for networked, security-sensitive, or correctly timestamped systems.

What remains uncertain is the current exact status of specific libc/OS/toolchain combinations, because those facts can change and were not reverified during this preservation turn.

Topic 2 - Integer representation of long-range time

The second topic was how to represent about 10,000 years with good detail. The chat discussed signed 64-bit scalar time values and the precision/range tradeoff. The key recommendation was 100 ns ticks or microsecond ticks if one scalar is required, and `int64 seconds + int32 nanoseconds` if a split type is acceptable.

The user should remember that unsigned 32-bit is not a future-proof absolute timestamp format. It only delays the overflow.

Topic 3 - Cross-platform durable time architecture

The third topic was the architecture needed for software intended to survive platform changes. The main conclusion was to separate absolute instants, durations, monotonic elapsed-time counters, and civil/local calendar time. This is the core "immortal code" framework from the chat.

What remains uncertain is the precise target list of legacy machines/toolchains and how far compatibility must go, especially for 16-bit compilers with weak or absent 64-bit arithmetic.

Topic 4 - Dominion's time model The fourth topic was how the framework maps onto Dominion. The repo-grounded conclusion was that Dominion already uses logical ACT for authoritative simulation, DSYS monotonic microseconds for platform/runtime pacing, deterministic ordering keys for cross-shard messages, and derived observer clocks for perception/replay.

The major future work is not to invent the model from scratch, but to harden boundaries and serialization.

Topic 5 - Preservation and aggregation The final topic is this export task. The user requested a human-readable report first, followed by registers, spec prep, context-transfer packet, aggregator packet, self-audit, and files. This package is designed to let the chat be read later without reopening the full transcript and to support merger into a larger Dominion spec book.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted to understand Y2038 consequences; choose safe timekeeping

representations for 16-bit and 32-bit code; find the most compatible timekeeping framework for cross-platform software; apply that framework to Dominium; and preserve this chat in a complete human-readable and aggregation-ready package.

4.2 Inferred Goals The user appears to be trying to prevent long-term architecture mistakes in Dominium and related C/C++ software. The user values deterministic simulation, legacy compatibility, future-proof serialization, and avoiding subtle date/time bugs that would require painful migrations later.

4.3 Goals That Changed Over Time The goal evolved from "will things break after 2038?" to "how should I design time for projects that should last?" and then to "does Dominium already match that doctrine?" The final upload changed the goal again into preservation and handoff.

4.4 Goals Still Unresolved The main unresolved goals are backend audit, ACT serialization audit, civil/astronomical projection design, and verification of external platform facts.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: 2038 is a time representation problem, not simply a 32-bit hardware problem.; Status: Assistant conclusion; Why it mattered: Prevents wrong audit target; Confidence: 4; Label: INFERENCE

Row 2 ID: DECISION-02; Decision: Prefer signed 64-bit time to unsigned 32-bit for absolute timestamps.; Status: Assistant recommendation; Why it mattered: Avoids 2106 cliff and signedness issues; Confidence: 4; Label: INFERENCE

Row 3 ID: DECISION-03; Decision: For ~10,000-year scalar time, use 100 ns or 1 ~s ticks; otherwise seconds+nanos.; Status: Assistant recommendation; Why it mattered: Balances range and precision; Confidence: 4; Label: INFERENCE

Row 4 ID: DECISION-04; Decision: Use separate Instant, Duration, Monotonic, and Civil time domains.; Status: Assistant recommendation; Why it mattered: Prevents semantic leakage; Confidence: 4; Label: INFERENCE

Row 5 ID: DECISION-05; Decision: Dominium ACT remains logical authoritative time.; Status: Repo-supported conclusion; Why it mattered: Preserves

determinism/replay; Confidence: 5; Label: FACT/
INFERENCE

Row 6 ID: DECISION-06; Decision: DSYS monotonic
time remains runtime-only, non-authoritative.;
Status: Repo-supported conclusion; Why it
mattered: Keeps OS quirks out of simulation
authority; Confidence: 4; Label: FACT/
INFERENCE

Row 7 ID: DECISION-07; Decision: Observer clocks
remain derived from ACT.; Status: Repo-
supported conclusion; Why it mattered: Supports
replay/perception without changing outcomes;
Confidence: 4; Label: FACT/INFERENCE

Row 8 ID: DECISION-08; Decision: Civil/astronomical
time belongs in projection layer.; Status: Assistant
recommendation; Why it mattered: Avoids
calendar/lore leakage into authority; Confidence:
4; Label: INFERENCE

The most important distinction is decision status. Many
items are strong assistant conclusions or
recommendations, not explicit user ratifications. The
Dominium conclusions are stronger because they are
grounded in fetched repo files: `DISTRIBUTED_TIME_`
`MODEL` says simulation time is logical and not wall-clock,
`dom_time_core.h` defines signed 64-bit ACT seconds,

and deterministic ordering/cross-shard docs forbid wall-clock ordering. [?filecite?turn9file0?](#) [?filecite?turn16file0?](#) [?filecite?turn10file0?](#) [?filecite?turn29file0?](#)

6. Ideas Considered but Rejected, Superseded, or Deprioritised

The chat rejected the idea that all 32-bit systems necessarily stop working after 2038. It rejected clock rollback as a general fix. It rejected unsigned 32-bit absolute time as a future-proof solution. It rejected one universal time type for all purposes. In the Dominionum context, it rejected any design where wall-clock/platform timestamps drive authoritative ordering.

These rejections matter because future chats may otherwise repeat simplistic suggestions: "just use unsigned," "just set the clock back," or "just use one timestamp everywhere." Those ideas are inadequate for Dominionum's deterministic simulation architecture.

7. Important Reasoning, Rationale, and Tradeoffs

The visible rationale centered on semantic separation. Integer width is necessary but not sufficient. A signed 64-bit value avoids 2038 only if it is used consistently in storage, protocols, and APIs. It does not by itself solve wall-clock jumps, timezone changes, deterministic replay, or distributed ordering.

For Dominion, the highest-value tradeoff is preserving deterministic authority over convenience. Platform clocks are convenient for frame pacing and telemetry, but they are unsafe as authoritative time because they can vary by OS, machine, scheduling, clock adjustments, and backend implementation. Logical ACT, deterministic event ordering, and replay logs are harder to design but are necessary for reproducible simulation. The main uncertainty is completeness of repo inspection. The assistant read key files, but not all serialization paths and not all platform backends.

8. Plans, Future Work, and Next Steps

The recommended next-action sequence is:

1. Audit all DSYS backend timer implementations.
2. Search all save/replay/network schema paths for ACT serialization and host time usage.
3. Formalize a canonical Dominion time architecture document.
4. Add CI/analyzer checks forbidding wall-clock time in authority paths.
5. Define civil/astronomical/lore time as a projection layer.

6. Verify current platform/library facts before using them in formal public documentation.
7. Feed this preservation package into the later master Dominion Project Spec Book.

The highest priority is backend/serialization audit, because that is where long-lived time bugs are most likely to enter despite the core model being sound.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences The preservation prompt explicitly requires a human-readable report first, not only a machine-readable handoff. It requires uncertainty labels, stable registers, preservation of artifacts, no invented facts, and downloadable files if tools are available. `?filecite?turn30file0?`

9.2 Inferred Constraints and Preferences The user prefers source-grounded, audit-ready technical reasoning, direct structure, and careful distinction between facts, recommendations, and unresolved issues. PROJECT-CONTEXT indicates Dominion values C89/C++98 portability, deterministic architecture, CLI/TUI support, and clean platform/runtime separation.

9.3 Uncertain or Unestablished Preferences The exact acceptable precision for Dominion ACT is not established. The exact legacy OS/toolchain floor is not fully established inside this chat. The user has not explicitly accepted every assistant recommendation as final doctrine.

10. Files, Artifacts, Outputs, and Prompts

The main uploaded artifact is `Pasted text.txt`, the preservation prompt used to generate this package.
?filecite?turn30file0?

The main in-chat artifacts are the earlier assistant answers about 2038, int64 range/precision, cross-platform time architecture, and Dominion application. The main external/repo artifacts are the fetched Dominion files, including timing docs, ACT core code, DSYS runtime code, deterministic ordering policy, distributed time model, macro time model, observer clock spec, cross-shard schema/log docs, and runtime loop docs.
?filecite?turn6file0? ?filecite?turn16file0?
?filecite?turn22file0? ?filecite?turn9file0?
?filecite?turn10file0? ?filecite?turn25file0?

This preservation task creates Markdown/YAML files and a ZIP package listed in section 35.

11. Open Questions and Unresolved Issues

The biggest unresolved issue is whether all actual Dominionium platform backends implement safe monotonic 64-bit time on their target operating systems. Another unresolved issue is whether ACT seconds are permanently sufficient or whether future simulation needs subsecond/subtick authority. The civil/ astronomical/lore time model remains unspecified. The current state of external platform facts also needs verification before being treated as formal spec material.

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Future chats might overstate this chat by treating assistant recommendations as user decisions. They might confuse DSYS runtime time with ACT authority. They might assume the repo audit was exhaustive. They might merge external platform claims without re-verification. They might repeat rejected fixes like unsigned 32-bit timestamps or clock rollback. They might also lose the distinction between civil time and simulation time.

Avoid these failures by preserving the source hierarchy, labels, and verification queue.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat contributes a clear time architecture doctrine for Dominion. It should feed into chapters on deterministic simulation, distributed time, platform runtime, serialization/replay compatibility, legacy compatibility, and future civil/astronomical projection systems.

Likely formal requirements include: authoritative logic must not depend on wall-clock time; ACT must remain versioned/fixed-width/logical; cross-shard ordering must use deterministic keys; platform time must remain DSYS-owned and non-authoritative; civil time must be projection-only.

14. What I Should Remember

- The 2038 issue is not "32-bit machines die"; it is signed 32-bit time leaking into APIs/data.
- Unsigned 32-bit is not a durable absolute-time fix.
- For broad range and precision, split **seconds** + **nanos** is cleaner than forcing everything into one scalar.
- Dominion already has the right conceptual split: ACT authority, DSYS runtime, observer projections, and deterministic cross-shard ordering.

- The main work is hardening boundaries and serialization, not rewriting the time model.
- The strongest unresolved technical task is auditing actual backend timers and persistence formats.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- Explain the difference between ACT, DSYS time, observer clocks, and civil time in Dominium.
- Reconstruct the full Y2038 reasoning in one technical note.

15.2 Decisions

- Which recommendations in this chat should become formal Dominium requirements?
- Which decisions were repo-grounded versus merely assistant suggestions?

15.3 Tasks and Next Actions

- Create a concrete DSYS backend time-audit checklist.
- Draft the canonical Dominium time architecture document.

15.4 Artifacts and Files

- List every Dominion file inspected and why it mattered.
- Turn this package into a shorter aggregator-ready capsule.

15.5 Risks and Verification

- What external platform facts must be verified before formalizing the spec?
- Where could wall-clock time still leak into authority logic?

15.6 Future Spec Book / Aggregation

- Which parts of this chat should become requirements in the master spec book?
- What could conflict with other Dominion architecture chats?

15.7 Deep-Dive Questions Specific to This Chat

- Should Dominion ACT remain seconds or be migrated to ticks/subticks?
- How should Dominion represent planetary/civil/lore calendars without compromising replay?

16. Compact Human Summary

This chat started as a question about what happens after 2038 to 32-bit operating systems and applications. The central correction was that 2038 is not really about all 32-bit machines failing. It is about signed 32-bit Unix-style time values overflowing. A vulnerable system can keep running in some ways while time-dependent paths fail, misorder events, reject future dates, break expiry logic, or corrupt scheduling. Rolling the clock back is not a proper general solution because it damages correctness for networked, security, logging, and synchronization contexts.

The conversation then turned to representation design. The user clarified that they only needed about 10,000 years into the future and past. That made the problem a range-versus-precision calculation. A single signed 64-bit nanosecond counter is too narrow for this target, but 100 ns ticks or microsecond ticks are practical. If not forced into one scalar, the cleaner representation is a split timestamp: signed 64-bit seconds plus a bounded nanosecond field.

Next, the user asked for a framework that could make cross-platform software more durable across old and future machines. The answer was to stop thinking in terms of one universal timestamp and instead separate time by semantics. Absolute instants, durations,

monotonic elapsed-time counters, and civil/local calendar time should be different types. This helps avoid Y2038, Y2K-style formatting bugs, clock jumps, timezone rule changes, and simulation nondeterminism.

The Dominion-specific part of the chat was the most important. The user asked how this applies to Dominion and requested repo/docs/code inspection. The assistant inspected selected files from Julesc013/dominium, including timing docs, runtime-loop docs, platform runtime docs, ACT core code, deterministic ordering policy, distributed time model, cross-shard message schema/log docs, observer clock specs, and runtime code. The conclusion was that Dominion already has the right conceptual architecture: authoritative simulation time is logical ACT, not wall-clock; ACT is signed 64-bit seconds; DSYS owns platform monotonic microseconds for runtime pacing; cross-shard messages use logical ticks and deterministic ordering keys; observer clocks are derived and non-authoritative.

The main conclusion is that Dominion does not need a conceptual rewrite for time. It needs boundary hardening. Future work should audit all DSYS backends, freeze ACT units and serialization, ban wall-clock time from authority paths, make civil/astronomical time a derived projection layer, and verify any external platform facts before making them formal documentation.

The final user upload requested this preservation

package. The purpose of this report is to preserve the chat for later reading and aggregation into a larger Dominion spec book.

19.2 Dominion Timekeeping And 2038 Resilience Reader Brief

Source: docs/archive/conversations/Timekeeping_and_2038_Resilience/Dominium_Timekeeping_and_2038_Resilience__06_reader_brief.md

Report type: reader_brief

Size: 2914 bytes

19.2.1 Reader Brief - Dominion Timekeeping and 2038 Resilience

What this chat was about

This chat developed a durable timekeeping framework, starting from Y2038 and ending with Dominion-specific architecture guidance. The key result is that Dominion should preserve a strict split: ACT for authoritative logical simulation, DSYS monotonic time for runtime/pacing, observer clocks for derived perception/replay, and civil/astronomical time as projection only.

Top 20 things to know

1. Y2038 is about signed 32-bit time representations, not all 32-bit machines.
2. Some 32-bit apps will continue; vulnerable time functions may fail.
3. Clock rollback is not a general solution.
4. Unsigned 32-bit is not a future-proof absolute timestamp fix.
5. Signed 64-bit is the right baseline for long-lived project time.
6. One signed 64-bit nanosecond scalar is too narrow for $\sim 10,000$ years.
7. 100 ns and 1 μ s scalar ticks are practical for that range.
8. Split `int64 seconds + int32 nanos` is cleaner when possible.
9. Durable software needs separate Instant, Duration, Monotonic, and Civil types.
10. Monotonic time should be used for intervals and pacing, not authority.
11. Civil/local time must not be treated as a raw offset only.

12. Dominion already documents simulation time as logical, not wall-clock.
13. `dom_act_time_t` is signed 64-bit ACT seconds.
14. DSYS owns platform monotonic microsecond timing.
15. Cross-shard ordering uses logical ticks and deterministic keys.
16. Observer clocks are derived and non-authoritative.
17. ACT->BST/GCT/CPT conversions are currently placeholders/projections.
18. Main work is hardening boundaries, not rewriting the model.
19. External platform facts must be verified before formal spec use.
20. This package should feed the future Dominion master spec book.

Decisions

The most important repo-supported decision is that Dominion authority must remain logical ACT, not wall-clock. The strongest recommendations are to freeze ACT units/serialization, audit DSYS timers, and keep civil time as projection-only.

Pending tasks

Audit DSYS timer backends, audit persistence/protocols for host time, write a canonical Dominion time architecture doc, add tests/analyzers, design civil/astronomical projection.

Open questions

Should ACT remain seconds forever? Are all DSYS backends safe on target legacy OSes? What human/lore/calendar display system does Dominion need?

Artifacts

Key artifacts include the uploaded preservation prompt, the earlier in-chat technical answers, selected inspected GitHub repo files, and the files/ZIP generated by this task.

Verification items

Verify glibc/musl/MSVC/Android timing facts, current repo state, DSYS backend implementations, and ACT serialization paths.

Best next step

Run a repo-wide time audit, starting with DSYS backend `time_now_us` implementations and any serialized ACT/timestamp fields.

19.3 Dominium Timekeeping And 2038 Resilience In Chat Reader

Source: docs/archive/conversations/Timekeeping_and_2038_Resilience/Dominium_Timekeeping_and_2038_Resilience__09_in_chat_reader.md

Report type: in_chat_reader

Size: 2113 bytes

19.3.1 In-Chat Reader - Dominium Timekeeping and 2038 Resilience

Package overview

This package preserves a chat about Y2038-safe timekeeping, durable timestamp representations, cross-platform temporal architecture, and Dominium's ACT/DSYS/observer time model.

File index

File	Purpose	Use
00_manifest	Package map	Start here if downloading files
01_human_readable_report	Main narrative	Best full read
02_context_transfer_packet	Future chat handoff	Paste into a new chat
03_spec_sheet	YAML-style aggregation data	Use for spec book merge
04_registers	Structured IDs	Track decisions/tasks/risks
05_aggregator_packet	Merge packet	Use in central aggregator chat
06_reader_brief	Short brief	Read if time-limited
07_verification_and_audit	Caveats and checks	Use before formalizing
08_future_chat_bootstrap_prompt	New-chat prompt	Starts continuation chat
09_in_chat_reader	This guide	Inspect package quickly

Plain-English explanation

The chat found that Dominion’s core time model is already mostly correct. The project should keep ACT as logical authority, DSYS time as runtime/pacing only, observer clocks as derived perception/replay, and civil/astronomical time as a projection layer. The key future work is auditing edges: backend timers, serialization,

wall-clock leakage, and current external platform facts.

Question menu

- What exactly did we decide about ACT?
- Which repo files supported the Dominium conclusion?
- What should be audited first?
- Which facts are stale or unverified?
- How should this feed into the master spec book?

Top things to preserve

- ACT is authority; wall-clock is not.
- DSYS time is non-authoritative runtime timing.
- Observer clocks are derived.
- Civil time is projection-only.
- Backend and serialization audits are still required.

Safest next actions

1. Save the ZIP.
2. Confirm which recommendations are accepted as requirements.

3. Audit DSYS timer backends.
4. Audit ACT serialization.
5. Draft canonical Dominion time architecture documentation.

19.4 Dominion Timekeeping And 2038 Resilience Accompanying Detailed Summary Report

Source: docs/archive/conversations/Timekeeping_and_2038_Resilience/Dominium_Timekeeping_and_2038_Resilience__10_accompanying_detailed_summary_report.md

Report type: detailed_summary_report

Size: 21997 bytes

19.4.1 Accompanying Human-Readable Detailed Summary and Report - Dominion Timekeeping and 2038 Resilience

Date anchor: 2026-05-27 Australia/Melbourne

Source scope: this chat only, except where explicitly marked as repo/tool context.

Purpose: give a readable, consolidated explanation of the entire conversation and attach it to the existing

preservation package.

0. Reliability and Scope Note

FACT: This report is based on the visible conversation, the generated preservation files now present in `/mnt/data`, the uploaded preservation prompt (`Pasted text.txt`), and the selected GitHub repository files inspected during the chat. It does not claim access to hidden chain-of-thought or to any transcript content that is not visible in the current conversation context.

FACT: A prior preservation package was already generated in this chat. This new report is an accompanying narrative layer intended to sit beside that package, not replace it.

INFERENCE: The main value of this extra report is that it reads less like a structured export and more like a coherent explanation of what happened, what mattered, what was decided, and what remains for later.

UNCERTAIN / UNVERIFIED: The earlier platform-specific claims about glibc, musl, Windows/MSVC, Android, POSIX, and other external systems were not freshly reverified while creating this companion bundle. Treat those as technical background requiring refresh before formal publication.

1. Executive Overview

This conversation was about timekeeping durability. It began with a general systems question: what happens to 32-bit operating systems and applications after the 2038 signed-time overflow boundary? The key correction was that the dangerous failure mode is not "all 32-bit software stops working." The actual risk is software, file formats, protocols, APIs, or ABIs that represent real time as signed 32-bit seconds since a Unix-like epoch. Such software may fail when converting, storing, comparing, scheduling, expiring, or serializing times outside that range. Non-time-related code can keep working, and 32-bit hardware is not automatically doomed.

The user then narrowed the design target. They did not need millions of years of support; they wanted roughly 10,000 years forward and backward. That changed the topic from a yes/no 2038 answer into a representation-design problem: how much range and precision can a signed 64-bit integer provide? The answer was that a single signed 64-bit nanosecond counter is too narrow for 10,000 years, while 100 ns ticks or microsecond ticks cover the target. The stronger architectural recommendation was to avoid forcing all semantics into one scalar when possible. A split representation such as `int64 seconds + int32 nanoseconds` is more durable for absolute timestamps and large durations.

The user then asked for the most compatible cross-platform architecture: the best combination of timekeeping choices for software intended to compile and run across many machines and operating systems, past and present. The answer moved beyond integer size. The durable framework is to keep separate time domains: absolute instants, durations, monotonic elapsed time, and civil/local calendar time. Those domains must not be collapsed into one type because they answer different questions. Wall-clock time is not safe for measuring elapsed time; local time is not safe as an authoritative machine timestamp; monotonic time is not a human calendar; simulation time is not operating-system time.

The conversation then became Dominion-specific. The user asked how this all applies to Dominion and requested inspection of the `Julesc013/dominium` repository. The assistant inspected selected docs and source files. The main finding was that Dominion already has the right conceptual separation: authoritative simulation time is logical ACT, not wall-clock; ACT is represented as signed 64-bit seconds; DSYS owns platform-facing monotonic microsecond time for runtime pacing and UI loops; cross-shard messages are ordered by logical ticks and deterministic keys; observer clocks are derived, non-authoritative mappings over ACT; and civil/astronomical/lore time should remain a projection layer rather than becoming

simulation authority.

The final stage was preservation. The user uploaded a detailed preservation/export prompt and asked for a maximum-fidelity package. A full package was generated with a human-readable report, context-transfer packet, YAML-style spec sheet, structured registers, aggregator packet, reader brief, verification/audit file, future-chat bootstrap prompt, in-chat reader, and ZIP archive. This current file adds another companion summary and bundles everything into one complete report ZIP.

2. Chronological Reconstruction

2.1 Initial 2038 question The user asked what would happen to 32-bit operating systems and applications after 2038. The question included multiple possible outcomes: total failure, critical-function failure, clock rollback as a workaround, or no breakage.

The answer distinguished system width from time representation. The important conclusion was that 2038 is caused by signed 32-bit time counters reaching their maximum representable value. The risk appears where signed 32-bit Unix-style timestamps are used directly or indirectly. Applications that use safer time representations, libraries, or operating-system APIs may continue normally. Applications that persist or exchange 32-bit time may fail before or at the boundary when

asked to handle future dates.

2.2 Question about using unsigned 32-bit or signed 64-bit in 16-bit and 32-bit code The user asked whether unsigned 32-bit or signed 64-bit integers could be used in 16-bit and 32-bit code to avoid the issue. The answer was that signed 64-bit is the safer long-term choice for real timestamps. Unsigned 32-bit only moves the overflow boundary to roughly 2106 and loses easy representation of pre-epoch dates. The CPU or operating-system bitness does not by itself prevent 64-bit integer arithmetic; many 16-bit and 32-bit environments can emulate or compile 64-bit integer operations, though very old toolchains may need software helpers or multiword arithmetic.

2.3 Range/precision design for ~10,000 years The user clarified that the project did not need millions of years of support, only about 10,000 years in the future and past. The answer explained that tick size determines the range of a signed 64-bit scalar. A signed 64-bit nanosecond scalar only covers a few centuries. A 100 ns scalar covers tens of thousands of years. A microsecond scalar provides even more range. If the representation need not be a single scalar, `int64 seconds + int32 nanoseconds` is cleaner and avoids the range/precision tradeoff of one packed integer.

The important design takeaway was: use 100 ns ticks if one scalar is mandatory and sub-microsecond precision is required; use microseconds if simpler arithmetic and broad headroom matter more; use seconds+nanos if a two-field type is acceptable.

2.4 Cross-platform "immortal" time architecture

The user asked for the most compatible combination of timekeeping for cross-platform software architecture, including old and future targets, and asked how to avoid Y2K, Y2038, and similar future problems.

The answer established a four-domain model:

1. **Instant** - an absolute timestamp, preferably independent of host ABI types.
2. **Duration** - a span between instants or logical times.
3. **Monotonic elapsed time** - local measurement for timeouts, pacing, profiling, and scheduling deltas.
4. **Civil/local time** - human-facing date, time zone, calendar, and scheduling-intent representation.

The central conclusion was that "immortal" code is not achieved by a single magic integer. It is achieved by isolating unstable layers: OS clocks, time-zone databases, human calendars, file/protocol encodings, and replay/simulation semantics.

2.5 Dominion-specific repository inspection The user then asked how all this applies to Dominion and requested that the assistant read attached docs/code or julesc013/dominium. The assistant used the GitHub connector and inspected selected repository files. Important inspected areas included:

- docs/app/TIMING_AND_CLOCKS.md
- docs/app/RUNTIME_LOOP.md
- docs/platform/PLATFORM_RUNTIME.md
- engine/include/domino/core/dom_time_core.h
- engine/modules/core/dom_time_frames.c
- engine/modules/system/sys.c
- app/include/dominium/app/app_runtime.h
- app/app_runtime.c
- docs/architecture/DISTRIBUTED_TIME_MODEL.md
- docs/architecture/DETERMINISTIC_ORDERING_POLICY.md
- docs/architecture/MACRO_TIME_MODEL.md
- docs/architecture/CROSS_SHARD_LOG.md
- schema/cross_shard_message.schema

- `schema/time/SPEC_OBSERVER_CLOCKS.md`
- `docs/architecture/REPLAY_AND_TIME_`
`ASYMMETRY.md`
- `engine/modules/system/dsys_perf.c`
- `tests/app/mmo0_distributed_contract_tests.`
`py`

The conclusion was that Dominium already separates authoritative logical simulation time from platform runtime time. `dom_act_time_t` is a signed 64-bit ACT seconds type. The docs state that simulation time is logical, not wall-clock. Cross-shard ordering is deterministic and logical-tick-based. Observer clocks are derived and non-authoritative. DSYS owns OS-facing monotonic time and sleep. This is the correct architectural direction for Y2038 resilience and deterministic distributed simulation.

2.6 Preservation package generation The user uploaded a very detailed preservation prompt asking for a complete chat preservation report, structured registers, spec sheet, aggregator packet, audit, and downloadable package. The assistant generated a full preservation package and ZIP archive. The previous package is now included inside this new complete bundle.

2.7 Current companion report and complete

bundle The user then asked for an accompanying human-readable detailed summary/report of the entire conversation and a single ZIP containing all files. This document is that companion report. It adds a more direct narrative overview, an explicit list of what was done and deferred, and a file-integrity inventory.

3. What Was Discussed and Why It Mattered

3.1 Y2038 and failure modes The conversation clarified that Y2038 is not a universal apocalypse for 32-bit systems. It is a boundary condition for signed 32-bit time counters. This matters because the correct remediation is an audit of time representation and external interfaces, not a blanket abandonment of 32-bit code or platforms.

The user should remember that a project can be partly safe and partly unsafe. Core code might use 64-bit time while a save file, network protocol, database column, log parser, or imported library still uses 32-bit time.

3.2 Clock rollback Clock rollback was considered as a possible workaround. The answer rejected it as a general fix. Rolling back the wall clock can keep some isolated legacy software inside its representable date range, but it breaks real-world ordering, certificate validity, network

protocols, logs, file timestamps, synchronization, and user trust.

For Dominion, this reinforces a broader rule: wall-clock time should not become authoritative simulation time.

3.3 Unsigned 32-bit versus signed 64-bit Unsigned 32-bit time was considered and deprioritised. It extends the Unix-seconds boundary from 2038 to 2106 but does not solve the long-term problem. Signed 64-bit is a much stronger baseline. For absolute timestamps, signedness is helpful because it preserves dates before the chosen epoch and avoids interoperability surprises.

3.4 Single scalar versus split representation A single scalar is attractive for storage, sorting, and ABI simplicity. The tradeoff is that range and precision are locked together. A split representation is a little larger but much more expressive. The best general answer was `int64 seconds + int32 nanoseconds`, with normalization rules.

For Dominion specifically, the existing ACT type is simpler: signed 64-bit seconds. That is currently adequate for broad simulation time unless future requirements demand subsecond authoritative precision.

3.5 Deterministic simulation time The most important Dominion-specific point is that simulation

time is not real time. ACT should mean "authoritative logical time in the simulation," not "current time on the operating system clock." This protects replay, distributed determinism, and cross-platform consistency.

3.6 Runtime pacing time DSYS monotonic time is still necessary, but it serves different purposes: frame pacing, UI loops, sleep caps, profiling, and non-authoritative telemetry. It should be replaceable, mockable, and local to platform/runtime layers.

3.7 Observer and replay time Dominion has observer clocks and replay/asymmetric-time concepts. Those should remain derived from ACT. The viewer can perceive time differently, buffer events, or replay past state, but those perception clocks must not modify the authoritative timeline.

3.8 Civil, astronomical, and lore time Dominion may eventually need human-readable calendars, planetary cycles, galactic/cosmological frames, or lore chronology. The inspected `dom_time_frames.c` file showed placeholder conversions for ACT to derived frames. The correct future design is projection-only: civil/astronomical time should be derived from ACT and scenario metadata, not treated as the core authority.

4. What Was Decided, Recommended, or Left Tentative

4.1 Strong conclusions from the chat

- 2038 is a representation/interface problem, not a pure 32-bit hardware problem.
- Signed 64-bit is the safer baseline for absolute timestamps.
- Unsigned 32-bit absolute time is not a durable fix.
- A split seconds+nanos representation is the strongest general-purpose timestamp model.
- Cross-platform durable software should separate instant, duration, monotonic, and civil time.
- Dominium's authoritative time should remain logical ACT, not wall-clock.
- Dominium's DSYS time should remain runtime-only and non-authoritative.
- Dominium observer/replay clocks should remain derived and non-authoritative.
- Dominium civil/astronomical/lore time should be projection-only.

4.2 Recommendations not yet explicitly accepted as formal project doctrine

The assistant recommended several things that should not be treated as final user-approved requirements without review:

- formalizing `int64 seconds + int32 nanoseconds` for general persisted real-world timestamps;
- using 100 ns or microsecond ticks where a single scalar is required;
- adding explicit wall-clock leak analyzers/tests;
- writing a `Dominium TIME_ARCHITECTURE.md` or equivalent canonical spec;
- auditing all DSYS backends and serialization paths.

These recommendations are technically motivated, but they should be reviewed before becoming binding project doctrine.

4.3 Tentative or unresolved decisions

- Whether ACT seconds are sufficient forever, or whether future authoritative subticks are needed.
- What exact format should be used for non-authoritative real-world timestamps in logs/build metadata/import-export.

- How civil, planetary, galactic, or cosmological time should be represented for Dominion gameplay and tooling.
- Which old OS/toolchain targets Dominion actually intends to support.

5. What We Did

The conversation produced both technical analysis and package artifacts.

5.1 Technical work performed

- Explained Y2038 failure modes.
- Compared signed/unsigned 32-bit and signed 64-bit timekeeping choices.
- Calculated broad range/precision tradeoffs for signed 64-bit scalar time.
- Proposed cross-platform time architecture using separated temporal domains.
- Inspected selected Dominion repository docs and source files.
- Mapped the generic timekeeping framework onto Dominion.

- Identified current Dominion strengths and boundary risks.
- Created a maximum-fidelity preservation package.
- Created this accompanying detailed human-readable report.
- Created an integrity inventory and complete ZIP bundle.

5.2 Files generated before this companion report

The earlier preservation task generated:

- 00_manifest.md
- 01_human_readable_report.md
- 02_context_transfer_packet.md
- 03_spec_sheet.yaml
- 04_registers.md
- 05_aggregator_packet.md
- 06_reader_brief.md
- 07_verification_and_audit.md
- 08_future_chat_bootstrap_prompt.md
- 09_in_chat_reader.md
- handoff_package.zip

5.3 Files generated now This task adds:

- 10_accompanying_detailed_summary_report.md
- 11_file_inventory_and_integrity_check.md
- README_START_HERE.md
- complete_report_bundle.zip

6. What Was Put Off for Later

6.1 Repo-wide backend audit The assistant inspected selected files but did not audit every backend implementation. This remains the highest-priority technical task. The platform abstraction is only as safe as its actual backend implementations.

6.2 Serialization and persistence audit It remains necessary to inspect save files, replay logs, schema files, network protocols, tool outputs, telemetry, and import/export formats to ensure time values are fixed-width, versioned, and not host-ABI-dependent.

6.3 ACT precision decision Dominionium currently defines ACT as signed 64-bit seconds. If gameplay, physics, or scheduling later need subsecond authority, that should be added through a new explicit type or companion field, not by silently changing the meaning of existing ACT values.

6.4 Civil/astronomical/lore calendar design The chat identified the correct layer but did not design the actual calendar or astronomical projection model. That should become its own specification effort.

6.5 External platform verification Earlier responses relied on platform and library facts. Before they become formal project documentation, they should be verified against current official sources.

6.6 Cross-chat aggregation This chat's report should later be merged with other old-chat reports into a master Dominion Project Spec Book. That merge has not been done here.

7. Main Risks and Failure Modes

7.1 Mistaking recommendations for accepted decisions Some conclusions are assistant recommendations rather than explicit user-approved decisions. Future assistants must preserve that distinction.

7.2 Assuming the Dominion audit was exhaustive The repo inspection was selective. The package should not be read as proof that all Dominion time handling is safe.

7.3 Wall-clock leakage into authority Platform event timestamps, build timestamps, log timestamps, OS time APIs, or profiler clocks could accidentally enter deterministic simulation logic. That would undermine replay and cross-platform determinism.

7.4 Silent ACT unit migration If ACT seconds are later reinterpreted as milliseconds, ticks, or nanoseconds without versioning, old saves/replays/logs would become ambiguous or corrupted.

7.5 Stale external facts Platform behavior, standard-library capabilities, and operating-system APIs can change. These facts require verification before formal publication.

7.6 Losing package context The generated files are useful only if their roles are remembered. The manifest, reader brief, and this companion report are intended to prevent that.

8. Best Next Actions

Recommended order:

1. **Read this companion report** for the full plain-English story.

2. **Read @@V10@@** for the formal preservation report.
3. **Read @@V10@@** for stable IDs, tasks, risks, and verification items.
4. **Run a Dominium time audit** focused on DSYS backend timers, ACT serialization, wall-clock leakage, and event timestamp usage.
5. **Draft a canonical Dominium time architecture document** that formalizes ACT/DSYS/observer/civil separation.
6. **Verify external platform facts** before using earlier platform claims in formal docs.
7. **Feed the aggregator packet into the future master Project Spec Book process.**

9. What a Future Assistant Must Not Forget

- Do not treat 2038 as a generic 32-bit hardware death date.
- Do not recommend unsigned 32-bit time as a durable solution.
- Do not collapse wall-clock time, simulation time, and monotonic time into one concept.

- Do not use platform event timestamps for authoritative ordering.
- Do not silently change ACT units.
- Do not treat derived observer clocks as authority.
- Do not merge unverified platform facts into canonical docs.
- Do not claim a complete repo audit was performed.
- Do not lose the distinction between user decisions and assistant recommendations.

10. File Package Guide

Use the files as follows:

- **Start with @@V10@@** if opening the package cold.
- **Use this file** for the detailed story of the whole conversation.
- **Use @@V10@@** for the formal preservation report.
- **Use @@V10@@** for planning and task execution.
- **Use @@V10@@** when starting a new chat.

- Use @@V10@@ when merging with other old-chat reports.
- Use @@V10@@ before turning conclusions into formal requirements.
- Use @@V10@@ for machine-assisted aggregation.
- **Keep the original uploaded prompt** as evidence of the preservation requirements.

11. Final Companion Report Status

FACT: This report was created as an additional human-readable explanation and bundled with the prior preservation package files.

INFERENCE: It should make the package easier to use because it explains the conversation as a coherent narrative rather than only as registers and handoff structures.

UNCERTAIN / UNVERIFIED: It does not replace a repo-wide audit, a platform-facts refresh, or user confirmation of which recommendations should become formal Dominion requirements.

Chapter 20

Ue6 Domino Deterministic Universe

20.1 Dominion Ue6 Domino
 Deterministic Universe Human
 Readable Report

Source: docs/archive/conversations/UE6_Domino_Deterministic_Universe/Dominium_UE6_Domino_Deterministic_Universe__01_human_readable_report.md

Report type: human_readable_report

Size: 36315 bytes

20.1.1 COMPLETE CHAT PRESERVATION
 REPORT - Dominion UE6, Domino,
 and Deterministic Universe Feasibility

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion UE6,
 Domino, and Deterministic Universe Feasibility

Row 2 Field: Date anchor; Assessment: 2026-05-27

Australia/Melbourne

Row 3 Field: Source scope; Assessment: THIS CHAT ONLY, plus clearly labelled PROJECT-CONTEXT where the current project summary is used

Row 4 Field: Apparent access; Assessment: Partial

Row 5 Field: Previously generated files available?; Assessment: No prior generated files from this chat were available before this task; this package creates new files

Row 6 Field: Uploaded files or artifacts present?; Assessment: Yes: Pasted `text.txt`, containing the preservation-package prompt applied here

Row 7 Field: Contains future plans?; Assessment: Yes

Row 8 Field: Contains decisions?; Assessment: Yes, but some are assistant recommendations rather than user-confirmed final decisions

Row 9 Field: Contains pending tasks?; Assessment: Yes

Row 10 Field: Contains unresolved questions?; Assessment: Yes

Row 11 Field: Staleness risk; Assessment: Medium to high for UE6/UE5 requirements, engine roadmaps, and external-source claims

Row 12 Field: Extraction confidence; Assessment: 3/5

Row 13 Field: Safe for later aggregation?; Assessment:
With caveats

Row 14 Field: Main limitations; Assessment: The full long-running Dominion project transcript is not visible. Only the current visible turns, the uploaded prompt, and project-context summaries are available. Earlier uploaded files in the broader environment may have expired. External facts cited in the earlier answers were not freshly reverified during this packaging pass.

Plain-language limitations FACT: This package preserves the currently visible chat about whether Dominion should use UE6, UE5, Domino, Unreal, or a custom engine layer for a highly ambitious deterministic solar-system-scale game. It also preserves the current uploaded prompt as an artifact because it defines this preservation task.

UNCERTAIN / UNVERIFIED: This is not a complete reconstruction of every Dominion conversation ever held. The project context shows that many other chats exist about platforms, renderers, setup, game architecture, ecology, ECS, launcher design, and development planning, but their full transcripts are not visible here. Those items are labelled PROJECT-

CONTEXT when referenced.

PROJECT-CONTEXT: The broader Dominion doctrine includes a deterministic C89 engine, C++98 product shells, platform/renderer separation, CLI/TUI/native shells, distribution contracts, and cross-renderer architecture. That context strongly informs how the UE/Unreal/Domino discussion should be interpreted, but this report does not claim those details were all explicitly discussed in the visible turns of this chat.

1. One-Page Orientation

This chat was mainly about whether Dominion could or should be built on Unreal Engine 6, Unreal Engine 5, Domino, or some custom architecture that uses a commercial engine only as a frontend. The user began by pasting a confident external-style answer claiming that UE6 had been unveiled, that UE6 would raise baseline OS requirements, and that high-performance UE6 games would probably require modern Windows, macOS, and Linux targets. The user then asked the key strategic question: could Dominion be made on UE6 instead of Domino, and could it be made portable to Domino later?

The first answer established the central architectural position of this chat: Dominion should not wait for UE6 or treat UE6 as the whole game platform. The recommended path was to build using current Unreal

technology where useful, structure the project for a future UE6 transition, and preserve portability to Domino by keeping the durable game logic outside Unreal-specific systems. The important distinction was between the game itself and the engine used to render or host it. The answer framed DominionCore as the permanent game, Unreal or UE6 as one possible renderer/runtime, and Domino as a later renderer/runtime. That distinction matters because Dominion's broader design is not a conventional Unreal game. It is intended to be a long-lived, portable, deterministic, multi-platform, simulation-heavy project.

The user then escalated the technical scope substantially. They asked whether UE5, UE6, or any other engine could handle a deterministic full-scale real solar system with fully recreated procedural planets, player-built civilizations, terraforming, cut-and-fill terrain, megaprojects, machine/device/factory design, fog of war, sparse simulation, collapsed unobserved regions, single-player multi-universe gameplay, an MMORPG single universe, and client-shared compute where each client simulates its own area or assists other areas when servers cannot keep up. The user also asked whether Unreal could do this without lag, with low latency, at high FPS, and with millions of players in the same universe.

The answer was deliberately blunt: no commercial engine, including UE5 or publicly known UE6, solves

this combination out of the box. The decisive conclusion was not that Unreal is useless, but that Unreal should not be the authoritative simulation engine. Unreal can be valuable as the visual client, tooling environment, editor, animation/rendering stack, construction preview layer, and possibly debug frontend. The durable game must be a custom deterministic simulation core, backed by a sparse world database and an authority/server layer. The proposed permanent architecture became `DominiumSim + DominiumWorldDB + DominiumServer`, with `DominiumClient_UE` and a future `DominiumClient_Domino` as adapters.

The conversation mattered because it clarified the difference between a game engine decision and a world-simulation architecture decision. If `Dominium` is treated as "an Unreal game," it risks becoming locked into non-deterministic actor ticking, Unreal physics, Unreal replication, `.uasset` content, and engine-specific assumptions. If `Dominium` is treated as a portable deterministic simulation with Unreal as one client, it remains theoretically movable to `Domino` or another runtime later. The result is more difficult up front, but it preserves the design ambition.

The main outcome was a practical doctrine: use commercial engines for what they are good at, but do not delegate the canonical universe state to them. Determinism, resource accounting, fog-of-war authority,

sparse planet edits, region handoff, client/server trust boundaries, simulation LOD, and persistence must be designed as first-class Dominion systems. The chat also produced a high-level phased roadmap: first build a deterministic headless simulation, then sparse planet chunks, then collapse/refine mechanics, then server authority and fog of war, then region authority, then MMO-scale load testing.

For a future assistant or human reader, the key thing to understand is that the chat did not reject Unreal. It rejected the idea that Unreal or UE6 could be the complete solution. The direction is hybrid: custom simulation and backend as the real engine, Unreal as the first high-quality client, UE6 as a possible future upgrade, and Domino as a possible later frontend/runtime if the core remains portable.

2. The Story of the Conversation

The visible conversation began after the user had obtained or drafted a confident answer about UE6 platform requirements. That pasted answer treated UE6 as a coming bleeding-edge technology with raised OS and hardware expectations and implied that modern Windows 10/11, macOS Sequoia or newer, and modern Linux distributions would be the practical targets. The user's real question was not merely about OS compatibility. It was about whether the Dominion

project could pivot from Domino to UE6, and whether that would still leave a path back to Domino later.

The first response corrected the framing. It said that public UE6 information is still too thin to treat UE6 as an available production base. It recommended a UE5-first path: build with a stable UE5 release now, structure the project so that a later UE6 migration is plausible, and maintain Domino portability by isolating engine-independent game systems. The important early architecture was a separation between **Core**, **UnrealAdapter**, **DominoAdapter**, source assets, Unreal-specific assets, and tools.

That response also introduced a rule that shaped the rest of the chat: Unreal should render and host **Dominium**, but Unreal should not define **Dominium**. That was the first major conceptual decision point. It reframed **Dominium** not as an engine project in the narrow sense, but as a portable game/simulation system with replaceable frontends.

The user then asked a far more ambitious question. They described a game with a deterministic real-scale solar system, data-driven procedural planets, player-built civilizations, terraforming, cut/fill terrain, megaprojects, player-designed machines and factories, fog of war, sparse simulation, collapsed unobserved state, MMORPG single-universe scale, single-player multi-universe options, and client-shared computation. They

also asked if Unreal could do all of this without lag, with low latency and high FPS, for millions of players in the same universe.

The second response answered that no existing commercial engine can solve that full combination out of the box. It separated solvable requirements from impossible or tradeoff-bound requirements. Solar-system scale is possible with hierarchical coordinate systems. Procedural planets are possible with seeds, real data, and sparse deltas. Deterministic rules are possible with a fixed-step custom simulation. Terraforming is possible with sparse voxel, signed-distance-field, or hybrid terrain storage. Player-built factories are possible if treated as graph simulations rather than huge piles of individual physics actors. Fog of war is possible if the server never sends hidden state. A single shared universe is possible if it means one persistent world split into regions and authority domains. Millions of players in the same local mutually interacting space at low latency is not realistically possible.

The answer then proposed the architecture that became the main deliverable of the chat: `DominiumSim`, `DominiumWorldDB`, `DominiumServer`, `DominiumClient_UE`, and `DominiumClient_Domino`. This architecture makes the core simulation, world storage, and server authority independent from the renderer. Unreal becomes an adapter, not the source of truth. Domino

becomes a future adapter if the project requires it.

The discussion then walked through major hard problems: determinism, coordinate scale, procedural planets, terraforming, factories, collapsed simulation, fog of war, client-shared compute, and MMO authority. It introduced the idea that the project must distinguish between what exists, what is simulated, what is rendered, what is knowable, and what is authoritative. That distinction is one of the most important concepts in the chat because it prevents the project from trying to run a full-fidelity universe everywhere all at once.

Finally, the user uploaded a preservation-package prompt asking for this chat to be turned into a human-readable report, registers, spec-prep packet, aggregator packet, audit, and downloadable files. This output is the resulting preservation package. The final state of the chat is therefore not a final engine selection, but a clearly stated architecture strategy: do not rely on UE6 as the game; build a portable deterministic core, use Unreal where it is useful, and preserve the option to port to Domino later.

3. Main Topics Discussed

Topic 1 - UE6, UE5, and Domino as strategic options The first topic was whether Dominium should be made on UE6 instead of Domino. The answer

separated availability from suitability. UE6 was treated as not yet available enough for production planning. UE5 was treated as a practical near-term renderer/tooling base. Domino was treated as a possible future runtime or custom engine path, not something that can be reached by automatically exporting an Unreal project.

Conclusion: use Unreal/UE5 as a practical client layer if useful, do not wait for UE6, and do not make Unreal-specific systems the canonical game.

Uncertainty: public UE6 technical capabilities and requirements remain time-sensitive and need verification before any future hard commitment.

Topic 2 - Engine-independent Dominium core

The chat repeatedly returned to the need for an engine-independent core. This means rules, simulation, save/load, economy, fog of war, machine graphs, resource accounting, and deterministic replay should live outside Unreal `UObject`, `AActor`, Blueprint, Chaos physics, or Unreal replication assumptions.

Conclusion: Dominium's durable product should be its simulation and world model. Unreal and Domino should be adapters.

Topic 3 - Deterministic full-scale simulation

The user's vision requires deterministic behavior across

machines, platforms, and possibly long-running worlds. The answer explained that Unreal's ordinary gameplay stack is not appropriate as a canonical deterministic simulation because of floating-point behavior, engine ticking, physics, and network replication. A custom fixed-step simulation with fixed-point or integer math, deterministic RNG, stable ordering, replay logs, and state hashes is needed.

Conclusion: strict determinism is possible only if treated as a core architecture requirement from the beginning.

Topic 4 - Solar-system and planet scale The user wanted real solar-system scale and fully recreated planets. The answer distinguished rendering scale from simulation scale. Unreal Large World Coordinates may help render large environments, but a real solar system should not be treated as a single flat Unreal coordinate space. The correct model is hierarchical: solar-system coordinates, orbit coordinates, planet surface coordinates, region coordinates, chunk coordinates, and local render coordinates.

Conclusion: solar-system scale is possible, but only through coordinate hierarchy and local render bubbles.

Topic 5 - Procedural planets, terrain edits, and megaprojects The user described planets generated procedurally from data, with player terraforming, cut/

fill, construction, and megaprojects. The answer argued that base terrain should be procedural and immutable by default, while player changes are sparse deltas or operation logs. Terrain should likely use sparse voxels, SDF chunks, heightfield-plus-edit layers, or a hybrid model.

Conclusion: arbitrary terrain modification is feasible only if sparse, chunked, versioned, and separate from the renderer's terrain representation.

Topic 6 - Factories, machines, and designed

devices The game vision includes factories and player-designed devices. The response warned that simulating every physical object everywhere is infeasible. Factories should be graphs: machines as nodes, belts/pipes/cables/routes as edges, and deterministic transfer rules. Distant or unseen factories should collapse into macrostate rates and inventories while preserving accounting.

Conclusion: machine gameplay must be designed as a scalable graph simulation, not as unrestricted physics.

Topic 7 - Fog of war, secrets, and knowability

Fog of war means clients must not receive hidden state. This conflicts with naive client-shared compute. The answer distinguished safe client work from unsafe authority.

Clients can generate meshes, predict local motion,

preview builds, suggest paths, and simulate non-authoritative UI states. They should not authoritatively decide hidden resources, enemy bases, persistent production, combat results, or market-relevant outcomes.

Conclusion: server-side sensory authority is mandatory for an MMO with fog of war.

Topic 8 - Single-universe MMO and millions of players The user asked about millions of players in the same universe. The answer distinguished a single persistent universe from a single local mutually interacting space. A single universe can exist through region authority, sharded or meshed simulation, shared databases, global economy/social systems, and interest management. Millions of players in one local space at low latency and high FPS is not a realistic target.

Conclusion: design for one universe identity, not one physics room containing everyone.

Topic 9 - Client-shared compute The user asked whether every client can share compute load. The answer said this is only safe in limited forms. The core pattern must be client proposes, server verifies, server commits. Full client authority would break the economy, fog of war, and anti-cheat model.

Conclusion: client compute can assist rendering,

prediction, meshing, path proposals, and single-player/private worlds, but not persistent MMO authority.

Topic 10 - Engines and systems to study The response named several references: Unreal for rendering/tooling, UNIGINE for geospatial precision and planet-scale simulation, Factorio for deterministic factory simulation and desync discipline, EVE Online for single-shard/social scale and time dilation, Unity DOTS/Bevy for data-oriented ECS design, Cesium/3D Tiles for geospatial streaming, and SpaceEngine/Outerra-like systems for astronomical/procedural planet-scale ideas. Conclusion: copy architecture ideas, not entire engines.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted to know whether Dominium could be built on UE6 instead of Domino and whether it could later be ported to Domino. Later, the user explicitly wanted to know whether any engine could handle the full deterministic solar-system/MMO/civilization-building vision with high performance and low latency.

These goals mattered because they affect the deepest architecture of Dominium. If the project starts in an engine-specific way, portability and determinism may become impossible later.

4.2 Inferred Goals INFERENCE: The user is trying to avoid choosing a technology stack that prematurely blocks the full Dominion vision. The user also seems to want an engine strategy that supports both near-term development velocity and long-term architectural control.

INFERENCE: The user wants a high-level but technically honest feasibility assessment, not encouragement that ignores determinism, networking, scale, or persistence constraints.

4.3 Goals That Changed Over Time The conversation moved from a narrower platform question, "Can Dominion use UE6 instead of Domino?", to a deeper systems question: "What architecture could ever support this game at all?" The engine choice became secondary to the simulation/backend architecture.

4.4 Goals Still Unresolved The chat did not settle whether Unreal should definitely be used as the first client. It recommended Unreal as a plausible first high-quality frontend, but the final choice still depends on prototype results, licensing, team capacity, platform targets, and how much custom simulation work is required.

The chat also did not resolve Domino's exact role because Domino's current technical state and intended capabilities were not fully specified in the visible

transcript.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Do not treat UE6 as the current production base; Status: Assistant recommendation; not explicitly user-final; Why it mattered: UE6 public details are insufficient for direct production planning; Confidence: 4/5; Label: INFERENCE / UNVERIFIED external facts

Row 2 ID: DECISION-02; Decision: Use UE5/Unreal only as a client/tooling layer if used; Status: Assistant recommendation; Why it mattered: Prevents Unreal from defining the canonical universe; Confidence: 4/5; Label: INFERENCE

Row 3 ID: DECISION-03; Decision: Keep DominionCore engine-independent; Status: Assistant recommendation strongly supported by user goals; Why it mattered: Preserves UE6/Domino portability; Confidence: 5/5; Label: INFERENCE

Row 4 ID: DECISION-04; Decision: Build a custom deterministic simulation core; Status: Assistant recommendation; Why it mattered: Required for deterministic factories, world state, and replay; Confidence: 5/5; Label: INFERENCE

Row 5 ID: DECISION-05; Decision: Do not trust clients with MMO authority; Status: Assistant recommendation; Why it mattered: Prevents cheating, fog-of-war leaks, and economy corruption; Confidence: 5/5; Label: INFERENCE

Row 6 ID: DECISION-06; Decision: Treat "single universe" as shared persistence plus partitioned authority; Status: Assistant recommendation; Why it mattered: Makes MMO scale plausible while rejecting impossible local concurrency; Confidence: 5/5; Label: INFERENCE

Row 7 ID: DECISION-07; Decision: Preserve Domino as a future adapter, not an automatic Unreal export target; Status: Assistant recommendation; Why it mattered: Avoids false portability assumptions; Confidence: 4/5; Label: INFERENCE

DECISION-01 - Do not treat UE6 as the current production base The assistant recommended against starting directly on UE6 because public technical access and production documentation were not treated as available in the visible chat. This was not framed as a permanent rejection of UE6. It was a timing decision: do not wait for a future engine to solve current architecture problems.

DECISION-02 - Use UE5/Unreal only as a client/tooling layer if used The chat treated Unreal as valuable for rendering, animation, editor workflows, UI, construction preview, and visual streaming. It should not own the canonical simulation. The alternative was an "Unreal is the whole game" approach, which was rejected because it would entangle gameplay with engine-specific non-deterministic systems.

DECISION-03 - Keep DominiumCore engine-independent This is the highest-value decision candidate. Core rules, simulation, save format, economy, factory logic, fog-of-war state, and replay/hash systems should be portable. This matters because Domino portability is only plausible if the core does not depend on Unreal object lifecycles and assets.

DECISION-04 - Build a custom deterministic simulation core The user's requirements imply determinism. The assistant recommended a fixed-step deterministic core with fixed-point/integer math, stable iteration order, deterministic random numbers, command logs, and state hashes. This is not optional if deterministic multiplayer, replay, collapsed regions, and auditability are serious requirements.

DECISION-05 - Do not trust clients with MMO authority The chat rejected the idea that clients can authoritatively simulate resource production, hidden state, enemy bases, combat, or persistent terrain changes in an MMO. Clients can assist, but the server must verify and commit.

DECISION-06 - Define single-universe MMO carefully The chat accepted a practical definition of a single universe: one persistent identity and shared world, partitioned across regions, servers, databases, and interest sets. It rejected the literal idea that millions of players can all share one local mutually interacting space at low latency and high FPS.

DECISION-07 - Preserve Domino portability through adapters The chat recommended making Domino a future adapter target. Portability comes from architecture, not from a future conversion button.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

- **REJECTED-01:** Treating Unreal or UE6 as the whole game engine. Rejected because determinism, persistence, server authority, and portability need custom control.

- REJECTED-02: Waiting for UE6 before beginning serious work. Deprioritised because UE6 availability and capabilities remain uncertain.
- REJECTED-03: Trusting clients with persistent MMO simulation. Rejected for anti-cheat, fog-of-war, and economy-integrity reasons.
- REJECTED-04: Supporting millions of players in one local fully interactive space at low latency. Rejected as a networking and computation impossibility under normal internet/consumer-client assumptions.
- REJECTED-05: Simulating full fidelity everywhere at all times. Rejected because it directly contradicts finite compute and storage.
- REJECTED-06: Making `.uasset` or Unreal Blueprints the source of truth for portable game rules. Rejected because they are not engine-portable.
- REJECTED-07: Simulating every machine/device/factory as full physics everywhere. Rejected because player-made factories require graph and LOD simulation.

7. Important Reasoning, Rationale, and Tradeoffs

The central tradeoff is between engine convenience and architectural control. Unreal gives rendering, tooling, animation, UI, asset workflows, and a mature editor. But the more Dominium relies on Unreal for gameplay truth, the harder determinism, portability, and Domino migration become.

A second tradeoff is between scale and fidelity. The user wants full-scale planets and civilizations, but a full-fidelity simulation everywhere would be computationally impossible. The chat's solution is simulation LOD: active areas are detailed, distant areas are summarized, and unobserved details are generated or refined deterministically as needed.

A third tradeoff is between client-shared compute and authority. Allowing clients to compute more can reduce server load, but allowing them to decide persistent outcomes creates cheating and secrecy problems. The safe model is proposal, verification, and commit.

A fourth tradeoff is between single-universe identity and local concurrency. A single persistent universe is plausible. A single local space with millions of fully relevant players is not. Dominium must use spatial partitioning, interest management, time dilation or degradation modes, region caps, and possibly soft instancing for local overload.

8. Plans, Future Work, and Next Steps

The recommended roadmap is staged:

1. Build a deterministic headless core with fixed ticks, deterministic RNG, replay logs, and state hashes.
2. Build a sparse planet-region prototype with procedural terrain, chunked edits, material inventory, and save/load.
3. Implement collapse/refine simulation for factories and regions, proving that active and macro simulation preserve resource accounting.
4. Add server authority and fog-of-war filtering so clients receive only knowable state.
5. Implement region authority, handoff, persistence, and load balancing.
6. Run MMO-scale testing with bots before claiming large player counts.
7. Only after these prototypes should the project decide how much should live in Unreal, Domino, or custom runtime layers.

The best immediate next action is to define the first deterministic simulation proof: a headless prototype with a small factory graph, terrain chunk edits, player command logs, and cross-platform hash checks.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences The user wants direct, source-grounded, audit-ready answers; explicit uncertainty; bounded estimates; and correction when framing is wrong. The preservation prompt explicitly requires human-readable explanation first, structured registers second, uncertainty labels, no invented facts, no hidden chain-of-thought, and downloadable files if tools are available.

9.2 Inferred Constraints and Preferences

INFERENCE: The user values long-term architectural optionality more than short-term engine convenience. They also prefer explicit handling of failure modes, stale facts, and future aggregation.

9.3 Uncertain or Unestablished Preferences

UNCERTAIN: The final intended role of Domino remains unclear in the visible transcript. It may be a custom engine, target runtime, renderer, or broader platform plan, but the current chat does not fully define it.

10. Files, Artifacts, Outputs, and Prompts

- ARTIFACT-01: Pasted UE6 OS/requirements answer from the user. Purpose: triggered the UE6

vs Domino question. Status: preserved as context; external claims require verification.

- ARTIFACT-02: Assistant response recommending UE5-now/UE6-later/Domino-portable architecture. Purpose: first strategic answer.
- ARTIFACT-03: User's expanded deterministic solar-system/MMO feasibility question. Purpose: defined the true scale of the technical challenge.
- ARTIFACT-04: Assistant response proposing `DominiumSim + DominiumWorldDB + DominiumServer`. Purpose: core architecture answer.
- ARTIFACT-05: Uploaded `Pasted text.txt`. Purpose: maximum-fidelity preservation prompt.
- ARTIFACT-06 to ARTIFACT-15: The files created by this preservation task, listed in the file index.

11. Open Questions and Unresolved Issues

- QUESTION-01: Should Unreal definitely be used as the first high-fidelity client, or should an in-house/experimental renderer come first?
- QUESTION-02: What exactly is Domino's current technical scope and target role?

- QUESTION-03: Which language should the deterministic core use: C, C++, Rust, or a split architecture?
- QUESTION-04: What determinism level is required across platforms: strict lockstep, server-only deterministic, replay deterministic, or audit deterministic?
- QUESTION-05: How much terrain modification is required: heightfield-only, voxel, SDF, or hybrid?
- QUESTION-06: What is the minimum viable prototype scope for factories, machines, and devices?
- QUESTION-07: How will collapsed simulation reconcile with exact resource accounting?
- QUESTION-08: What player count target is meaningful for phase-one MMO testing?
- QUESTION-09: What external engine claims from UE5/UE6 need re-verification before planning?
- QUESTION-10: How will the project prevent player-designed machines from becoming lag weapons?

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Future chats could incorrectly treat this as a decision to use Unreal for the whole game. They could also incorrectly treat UE6 as currently available and documented enough for production. Another risk is treating assistant recommendations as user-final decisions. The safe interpretation is that this chat produced a recommended architecture, not a signed-off implementation plan.

Future assistants might also overstate portability. A UE project cannot simply be exported to Domino. Portability exists only if the simulation core, data schemas, protocol, and source assets remain engine-independent.

A major technical risk is underestimating the backend. For this game, persistence, region authority, event logs, chunk deltas, anti-cheat, fog-of-war filtering, and simulation LOD are not secondary services. They are the game.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat should feed the following future spec-book areas:

- Engine and runtime strategy
- Renderer/frontend separation
- Deterministic simulation requirements
- Planetary coordinate and terrain architecture
- Sparse world persistence
- MMO authority and trust model
- Simulation LOD / collapse-refine design
- Fog-of-war and secrecy model
- Portability to Domino or future clients
- Prototype roadmap and validation gates

It should not be merged as a final commitment to Unreal. It should be merged as a strong argument for separating the permanent Dominion core from any commercial rendering frontend.

14. What I Should Remember

- Dominion should not be architected as "an Unreal game" if determinism and Domino portability matter.
- UE5 or future UE6 can be useful for rendering, tooling, and iteration.

- The durable architecture is `DominiumSim + DominiumWorldDB + DominiumServer`.
- Single universe is plausible as shared persistence plus partitioned authority, not millions of players in one local fully interactive space.
- Client-shared compute is useful only when the server validates or when the work has no persistent competitive value.
- Fog of war means hidden state must never be sent to unauthorized clients.
- Procedural planets should be seeds plus sparse deltas, not fully stored static worlds.
- Factories should be graph simulations with LOD, not physics actors everywhere.
- The next serious task is a deterministic headless proof, not an engine beauty prototype.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- "Explain again why Unreal should be a client rather than the canonical Dominium engine."
- "What is the smallest architecture that preserves UE, UE6, and Domino options?"

15.2 Decisions

- "Which recommendations in this chat should become actual project decisions?"
- "Which decisions still require my explicit confirmation?"

15.3 Tasks and Next Actions

- "Define the first deterministic DominionSim prototype in detail."
- "Write a milestone plan for proving sparse terrain edits and collapsed factories."

15.4 Artifacts and Files

- "Turn the architecture answer into a formal engine strategy document."
- "Extract a clean requirements list from the MMO feasibility answer."

15.5 Risks and Verification

- "List all claims about UE5/UE6 that need current-source verification."
- "What are the top ten ways this architecture could fail?"

15.6 Future Spec Book / Aggregation

- "Convert this chat into a spec-book chapter on engine/runtime strategy."
- "Merge this with the Dominionium platform/renderer doctrine from other chats."

15.7 Deep-Dive Questions Specific to This Chat

- "Design the collapse/refine algorithm for factories."
- "Design the fog-of-war authority model for a single-universe MMO."
- "Define the coordinate hierarchy for a real-scale solar system."

16. Compact Human Summary

This chat clarified that Dominionium's engine choice cannot be answered by simply choosing UE6, UE5, Domino, Unity, or any other commercial engine. The user's requirements are far beyond a normal engine decision: deterministic simulation, full-scale solar-system structure, procedural/data-driven planets, player-built civilizations, terraforming, cut/fill terrain, megaprojects, machine and factory design, fog of war, collapsed unobserved regions, single-player multi-universe possibilities, and MMO-scale single-universe ambitions.

The first question asked whether *Dominium* could be made on UE6 instead of *Domino* and later ported to *Domino*. The answer was that UE6 should not be treated as a current production base because public information and tooling are too uncertain. UE5 or Unreal may be useful now, but only if *Dominium*'s core remains independent. The recommended model was *DominiumCore* as the permanent game, with Unreal/UE6 and *Domino* as adapters or frontends.

The second question pushed much deeper: can Unreal, UE6, or any other engine actually support the dream game? The answer was no, not out of the box. No commercial engine gives strict determinism, massive sparse world persistence, real-scale planetary systems, arbitrary construction/destruction, fog-of-war secrecy, distributed compute, and millions of players in one universe at high FPS and low latency. Some requirements are solvable, but only with a custom architecture and hard tradeoffs.

The most important architectural output was the proposed split: *DominiumSim* for deterministic rules and simulation, *DominiumWorldDB* for sparse planets, terrain edits, events, and persistence, *DominiumServer* for authority, interest management, fog of war, and region handoff, and then separate clients such as *DominiumClient_UE* and *DominiumClient_Domino*. This makes Unreal a renderer/editor/client rather than the

source of truth.

The chat also established key constraints. Clients cannot be trusted with persistent MMO authority because they can cheat, leak hidden state, or corrupt the economy.

Fog of war means the server must not send hidden state to the client. Millions of players can exist in one persistent universe only if the universe is partitioned into regions and interest sets; millions cannot all be in one fully interactive local space with low latency. Factories and machines must be graph simulations with LOD, not physics actors everywhere. Terrain must be sparse and chunked, using procedural base state plus deltas and operation logs. Unobserved areas must collapse into macrostate, exact fast-forward, or deterministic late-bound detail depending on gameplay need.

The practical next step is not choosing UE6. It is building a small deterministic headless prototype. That prototype should prove fixed ticks, deterministic commands, cross-platform hashes, basic factory graphs, terrain chunk edits, and replay. Only after that should Dominion invest heavily in a renderer. Unreal remains useful, but the real Dominion engine is the custom simulation, world database, and authority layer.

20.2 Dominium Ue6 Domino Deterministic Universe Reader Brief

Source: docs/archive/conversations/UE6_Domino_
Deterministic_Universe/Dominium_UE6_Domino_
Deterministic_Universe__06_reader_brief.md

Report type: reader_brief

Size: 2667 bytes

20.2.1 READER BRIEF - Dominium UE6, Domino, and Deterministic Universe Feasibility

What this chat was about

This chat asked whether Dominium should use UE6, UE5/Unreal, Domino, or a custom architecture for a deterministic, planet-scale, sparse, MMO-capable civilization/factory/terraforming game. The answer was that no commercial engine solves the full problem. Unreal can be useful as a frontend; the canonical game must be a custom deterministic simulation/world/server system.

Top 20 things to know

1. Do not treat UE6 as the current production foundation without verification.
2. UE5/Unreal can be useful for rendering and tooling.
3. Unreal should not own canonical game state.
4. Domino portability requires engine-independent rules and data.
5. Dominium's real engine is `DominiumSim` + `DominiumWorldDB` + `DominiumServer`.
6. Determinism requires fixed-step simulation and controlled math/order.
7. Full-scale solar systems require coordinate hierarchy, not one flat scene.
8. Procedural planets should use seeds plus sparse deltas.
9. Terraforming needs chunked/sparse terrain representation.
10. Factories should be graph simulations.
11. Unseen areas need collapse/refine simulation.
12. Fog of war requires server-side sensory authority.

13. Hidden state must not be sent to clients.
14. Clients can propose work but not commit persistent MMO outcomes.
15. One shared universe is plausible with partitioned authority.
16. Millions of players in one local fully interactive area is not plausible.
17. Player-made machines need operation/gas limits.
18. External UE facts need verification.
19. Domino's exact role is unresolved.
20. The next task is a deterministic headless prototype spec.

Decisions

Main decision candidates: custom deterministic core; Unreal as frontend only; server-authoritative MMO state; sparse procedural planets; single universe via partitioning; Domino as future adapter.

Pending tasks

Define the simulation MVP, prototype deterministic tick/replay/hash, design coordinate hierarchy, prototype

terrain chunks, design factory graphs, define collapse/
refine invariants, clarify Domino, verify UE facts.

Open questions

What is Domino exactly? Should Unreal be first
frontend? Which language/math stack should
DominiumSim use? What terrain model is best? How
exact must collapsed simulation be? What first MMO
scale target is realistic?

Artifacts

The package includes a human report, context transfer
packet, spec sheet, registers, aggregator packet, reader
brief, audit file, future chat bootstrap prompt, in-chat
reader, and ZIP.

Best next step

Write a formal DominiumSim deterministic MVP spec.

20.3 Dominium Ue6 Domino Deterministic Universe In Chat Reader

Source: docs/archive/conversations/UE6_Domino_
Deterministic_Universe/Dominium_UE6_Domino_

Deterministic_Universe__09_in_chat_reader.md

Report type: in_chat_reader

Size: 4057 bytes

20.3.1 IN-CHAT READER - Dominion UE6, Domino, and Deterministic Universe Feasibility

Package overview

This package preserves the visible chat about Dominion's engine/runtime strategy. It explains why Unreal/UE6 should not be treated as the whole game, why a custom deterministic core is required, how Domino portability can be preserved, and what work should happen next.

File index

File	Purpose	What it contains	When to use it	Importance
00_manifest.md	Package overview	File list, purposes, caveats, status	First file to inspect	High
01_human_readable_report.md	Main explanation	Sections 0-16	To understand the chat	Highest
02_context_transfer_packet.md	Future chat handoff	Section 29	To continue in a new chat	High
03_spec_sheet.yaml	Structured spec	Section 30	For machine-assisted aggregation	High
04_registers.md	Structured records	Sections 17-28	For decisions/tasks/risks	High
05_aggregator_packet.md	Merge packet	Section 31	For central spec book	High
06_reader_brief.md	Short guide	Top things to know	Fast review	Medium-high
07_verification_and_audit.md	Audit	Sections 32-34 and verification	Before relying on facts	High
08_future_chat_bootstrap_	Prompt	Standalone continuation prompt	To start a new chat	Medium-high

Plain-English explanation

The chat concluded that Dominium's requirements exceed what UE5, UE6, Domino, or any single commercial engine can provide out of the box. The game should be architected as a custom deterministic simulation and persistent world backend, with Unreal as a possible frontend and Domino as a possible later adapter. The most important immediate task is a deterministic headless prototype, not a final renderer choice.

Question menu

- Explain the hybrid architecture again in simpler terms.
- Which items should become formal project requirements?
- What does the first deterministic prototype need to include?
- How should collapse/refine simulation work?
- How should server authority and fog of war interact?
- What does Domino need to support for a later port?
- What UE5/UE6 facts need verification?

Top things to preserve

- `DominiumSim` + `DominiumWorldDB` + `DominiumServer` is the main architecture.
- Unreal is frontend/tooling, not canonical truth.
- Domino portability requires core/adapter separation.
- Client compute must not commit persistent MMO outcomes.
- Single universe means shared persistence plus partitioned authority.
- External engine facts require current verification.

Safest next actions

1. Confirm whether the architecture recommendations should become project decisions.
2. Define Domino's role.
3. Verify UE5/UE6 facts.
4. Specify the deterministic MVP.
5. Prototype fixed-tick replay and state hashes.

20.3.2 Final Package Status

- Chat label: Dominium UE6, Domino, and Deterministic Universe Feasibility
- Report type: full human-readable + structured handoff + spec-prep package
- Files created: yes
- ZIP created: yes
- Safe for later aggregation: with caveats
- Extraction confidence: 3/5
- Main value of this chat: It establishes the Dominium engine/runtime strategy: custom deterministic simulation and backend first, Unreal/UE as frontend, Domino as possible adapter.
- Most important decision: Keep the canonical game state outside Unreal and commercial engine systems.
- Most important unresolved issue: Domino's exact role and capabilities are not defined in the visible chat.
- Most important next action: Define the first deterministic DominiumSim MVP.

- Main caveats: Partial transcript access; UE5/UE6 facts need verification; assistant recommendations need user confirmation before becoming final project decisions.
- Best thing for me to read first: 01_human_readable_report.md
- Best question for me to ask next: "Define the first deterministic DominionSim prototype in detail."

20.4 Dominion Ue6 Domino Deterministic Universe Accompanying Human Readable Detailed Summary And Report

Source: docs/archive/conversations/UE6_Domino_Deterministic_Universe/Dominium_UE6_Domino_Deterministic_Universe__10_accompanying_human_readable_detailed_summary_and_report.md

Report type: human_readable_detailed_summary

Size: 35405 bytes

20.4.1 ACCOMPANYING HUMAN-READABLE DETAILED SUMMARY AND REPORT - Dominium UE6, Domino, and Deterministic Universe Feasibility

Date anchor: 2026-05-27 Australia/Melbourne

Generated: 2026-05-27 21:03 AEST

Source scope: This visible chat, the uploaded preservation prompt `Pasted text.txt`, and the previously generated package files available in `/mnt/data`.

Reliability note: This is an accompanying narrative report. It does not replace the formal preservation package; it explains it in a more direct, human-readable way and adds the current packaging step.

1. Executive Summary

This conversation clarified a core technical direction for **Dominium**: the project should not be treated as a normal Unreal, UE6, Domino, Unity, or single-engine game. The user asked whether Dominium could be made on **Unreal Engine 6** instead of **Domino**, whether it could remain portable to Domino later, and whether any engine could support a much larger vision: a deterministic full-scale real solar system with procedurally recreated planets, terraforming, cut-and-fill

terrain, player-built civilizations, megaprojects, designed machines/devices/factories, fog of war, sparse construction/destruction, collapsed simulation for unseen areas, client-assisted compute, single-player multi-universe gameplay, and an MMO-scale single persistent universe.

The central conclusion was that no commercial engine can provide this whole stack out of the box. Unreal Engine can be useful, especially as a renderer, editor, visual client, construction-preview tool, and asset pipeline. UE6 may eventually be useful if and when its public tooling becomes available. Domino may also become useful as a future frontend or runtime target. But the canonical game cannot live inside Unreal, UE6, Domino, or any other general-purpose commercial engine if the requirements remain deterministic, portable, persistent, sparse, fog-of-war-secure, and MMO-scale.

The durable architecture that emerged was:

```
DominiumSim
+ DominionWorldDB
+ DominionServer
+ Unreal / UE6 / Domino / other engines as replaceable clients or
adapters
```

The important shift was from asking "**Which engine should Dominion use?**" to asking "**What must Dominion itself be, independent of any renderer?**" The answer is that Dominion needs its own

deterministic simulation core, sparse persistent world database, region-authoritative server layer, trust model, and simulation-level-of-detail system. Engines can render or host slices of that state, but should not define the authoritative universe.

The conversation also produced a practical near-term direction: do not wait for UE6. Build a deterministic headless prototype first. Keep gameplay rules, world state, resource accounting, factory logic, terrain edits, fog-of-war authority, and persistence outside Unreal-specific systems. Use Unreal only if it accelerates visualization and tooling. Keep a Domino adapter possible by making all engine integrations adapters around a portable core.

This package now contains the original preservation outputs plus this companion report and a verification/index file. The result is intended to let the user read, preserve, aggregate, and continue the conversation later without needing to reconstruct it from memory.

2. Coverage and Reliability

Row 1 Field: Chat label; Assessment: Dominion UE6, Domino, and Deterministic Universe Feasibility

Row 2 Field: Date anchor; Assessment: 2026-05-27
Australia/Melbourne

Row 3 Field: Source scope; Assessment: Visible chat plus uploaded preservation prompt and generated files present in `/mnt/data`

Row 4 Field: Apparent access; Assessment: Partial, but enough to preserve this visible discussion

Row 5 Field: Previously generated files available?; Assessment: Yes; manifest, human report, registers, spec sheet, aggregator packet, audit, reader brief, bootstrap prompt, full combined report, and prior ZIP were present

Row 6 Field: Uploaded files available?; Assessment: Yes; Pasted `text.txt` was present and contains the preservation-package instruction prompt

Row 7 Field: Contains future plans?; Assessment: Yes

Row 8 Field: Contains decisions?; Assessment: Yes, though several are assistant recommendations needing user confirmation before becoming formal project decisions

Row 9 Field: Contains unresolved questions?; Assessment: Yes

Row 10 Field: Staleness risk; Assessment: Medium to high for UE5/UE6 facts, engine roadmaps, Domino capabilities, and external-source claims

Row 11 Field: Extraction confidence; Assessment: 3/5 for full project history; 4/5 for the visible conversation and generated artifacts

Row 12 Field: Safe for later aggregation?; Assessment: Yes, with caveats

Limitations This report does **not** claim to capture every older Dominion conversation. The accessible material shows that this is part of a broader Dominion project, but the complete historical project transcript is not visible here. Some previously uploaded files in the broader environment may have expired. Therefore, this package should be treated as a preservation package for **this chat's visible**

UE6/Domino/deterministic-universe discussion, not as the master Dominion specification.

The package also preserves assistant recommendations, but it does not treat every recommendation as a final user decision. The user clearly asked strategic questions and requested preservation. They did not explicitly ratify every architecture detail as a locked requirement. Future chats should preserve this distinction.

External factual claims about UE5, UE6, macOS/Windows/Linux support, Epic roadmaps, Unreal features, and third-party engines should be verified before being used as final project basis. The earlier

answers used web citations, but this packaging step is primarily a preservation task, not a fresh technical verification pass.

3. Chronological Story of the Conversation

3.1 Initial UE6 and Domino question The visible technical discussion began with a pasted answer about Unreal Engine 6. That pasted answer claimed that UE6 had been officially unveiled, that it would be a bleeding-edge architecture, that modern OS baselines would be necessary, and that UE6 public preview or production timelines would be years out. The user’s underlying question was not simply about operating systems. The real question was whether Dominion could be made on UE6 instead of Domino, and whether it could remain portable to Domino later.

The answer challenged the framing. It said that UE6 should not be treated as a present production target because public UE6 tooling and detailed requirements were not established in the visible sources. It recommended building in current Unreal technology where useful, likely UE5, while keeping the core independent enough to migrate to UE6 later if UE6 becomes useful. The answer also emphasized that portability to Domino would not happen automatically. A UE project cannot simply be exported into Domino.

Portability would require a clean separation between Dominion's permanent game logic and engine-specific rendering/runtime layers.

The first major architectural idea therefore became:

```
DominiumCore = permanent game logic and state
Unreal / UE6 = possible first renderer/runtime
Domino = possible future renderer/runtime
```

This mattered because it rejected both extremes: do not wait for UE6 as a savior, but also do not discard Unreal if it can help as a visual and tooling layer.

3.2 Expansion into the full Dominion vision The user then asked a much larger feasibility question. They described a deterministic game with a full-scale real solar system, fully recreated procedural planets, player-built civilizations, terraforming, cut-and-fill construction, megaprojects, designed machines/devices/factories, fog of war, sparse loading and simulation, collapsed unobserved areas, single-player multi-universe gameplay, an MMO single universe, client-shared compute, and low-latency high-FPS gameplay with millions of players.

The answer framed this as far beyond what UE5, UE6, Domino, or any ordinary game engine can do out of the box. It clarified that the problem is not merely rendering scale. The hard problems include deterministic simulation, server authority, interest management, persistence, database write throughput, world streaming,

terrain edits, trust boundaries, fog-of-war secrecy, client cheating, factory graph scaling, state collapse/refinement, and network fanout.

The answer's central position was:

Unreal should not be the simulation engine.

Unreal can be the visual client and tooling layer.

The real Dominium engine is DominiumSim + DominiumWorldDB + DominiumServer.

3.3 Clarifying what is possible and impossible

The conversation separated the user's large vision into solvable, partially solvable, and impossible-in-literal-form components.

A single persistent universe is possible if it means one shared identity, persistence layer, economy, map, and social/civilizational reality spread across many authoritative regions. Millions of players can exist in the same universe in this sense.

Millions of players in the same local area, mutually interacting at low latency and high FPS, is not realistic. The bottleneck is not only graphics; it is interest management, bandwidth, latency, simulation load, server authority, and database writes. The answer emphasized that the design must avoid making all players mutually relevant at once.

The conversation also clarified that full fidelity everywhere is not possible under compute constraints.

The system must distinguish between:

- what exists
- what is simulated
- what is rendered
- what is knowable
- what is authoritative

This distinction became one of the most important conceptual tools from the chat. A distant planet can exist as procedural data and sparse edits without being actively simulated at per-object fidelity. A factory can exist and produce resources as a graph/rate model without every belt item being simulated. A hidden base can be authoritative on the server but unknown to the client. A region can be collapsed into macrostate and later refined when observed.

3.4 The proposed hybrid architecture The answer proposed a layered architecture:

- DominiumSim
 - deterministic fixed-step simulation
 - fixed-point/integer math
 - deterministic RNG
- factory graph simulation
- economy/civilization state
- terrain/material state model
- fog-of-war and sensing rules
- replay/hash/desync testing
- DominiumWorldDB
 - coordinate hierarchy

planet/tile/chunk storage
sparse voxel/SDF/terrain deltas
construction operation logs
event sourcing
snapshots/checkpoints
rollback/version migration
DominiumServer
region authority
interest management
authority handoff
anti-cheat validation
simulation LOD scheduling
global persistence/economy/social systems
DominiumClient_UE
rendering
animation/audio/UI
construction preview
terrain visualization
debugging/editor tools
DominiumClient_Domino
later adapter if Domino becomes an appropriate target

This architecture is the main technical carry-forward from the chat.

3.5 Preservation task and generated package The user uploaded a long preservation prompt in Pasted text.txt. That prompt instructed the assistant to produce a maximum-fidelity chat preservation package, including a human-readable report, structured registers, context transfer packet, spec sheet, aggregator packet,

self-audit, file export package, and final in-chat reader.

A set of files was then created in `/mnt/data`, including a manifest, human-readable report, context transfer packet, spec sheet, registers, aggregator packet, reader brief, verification/audit file, future chat bootstrap prompt, in-chat reader, full combined report, and a ZIP package.

The current user request asked for an accompanying human-readable detailed summary/report of the entire conversation and a new single ZIP containing all files. This companion report is the answer to that request, and the package was rebuilt with this file included.

4. Main Technical Conclusions

4.1 Unreal Engine is useful but should not own the game The conversation did not reject Unreal. It rejected treating Unreal as the authoritative world engine for Dominium. Unreal can provide high-quality rendering, animation, editor workflows, UI, asset import, debugging views, and construction previews. It may be a practical first client for a complex simulation game because it gives a mature visual stack.

However, Unreal’s actor model, physics, replication, world partition, and asset system should not define canonical game truth. Dominium’s deterministic simulation, resource accounting, terrain edit state,

machine logic, fog-of-war visibility, persistent economy, and server authority need to live outside Unreal-specific systems.

4.2 UE6 should not be waited on as a solution

The earlier pasted answer discussed UE6 as if it were a near-term design target. The assistant's answer treated UE6 facts as uncertain and warned that public UE6 details were insufficient for production planning. The recommendation was not to stop work until UE6 arrives. Instead, build the portable core now and make future UE6 support an adapter/runtime question.

4.3 Domino portability requires architecture, not hope

Portability to Domino is possible only if the game is designed with engine separation from the start. If Dominium logic is built into Unreal Blueprints, Actors, Components, UMG, Materials, and Unreal replication, it will not be portable in a meaningful way. If the gameplay exists in a deterministic core with clear data schemas and adapter APIs, Domino can later become another frontend/runtime.

The key rule is:

Unreal should render and host Dominium.

Unreal should not define Dominium.

4.4 Determinism requires a custom simulation

layer Strict determinism is incompatible with naive use of engine physics, variable tick ordering, nondeterministic floating-point behavior, and platform-dependent computation. Dominion needs fixed tick simulation, deterministic RNG, stable iteration order, fixed-point/integer math where needed, replay logs, and state hashes.

This is especially important because the user wants factories, machines, resource accounting, sparse world edits, and possibly multiplayer. If the same input log does not produce the same state hash, the system cannot support reliable replays, desync detection, deterministic collapse/refinement, or authoritative verification.

4.5 The solar system should be hierarchical, not one giant world coordinate space

Even if Unreal's Large World Coordinates help with large rendered worlds, a full real-scale solar system should not be represented as one continuous Unreal physics space. Dominion should use a coordinate hierarchy:

Solar system

Planet/orbit

Planet surface

Region

Chunk

Local render bubble

The server/database can store precise absolute coordinates using deterministic integer/fixed-point representations. The client only renders a local bubble around the player or camera.

4.6 Terrain and construction must be sparse Fully storing every planet at maximum resolution is infeasible. The correct model is procedural base data plus sparse player deltas. Unchanged areas come from seeds, real-world datasets where appropriate, and procedural generation. Edited areas store compressed operations, material deltas, ownership/permission metadata, and version history.

Terrain should not rely solely on Unreal Landscape if arbitrary cut/fill, tunnels, caves, overhangs, and megaproject excavation are central mechanics. A hybrid heightfield/SDF/sparse-voxel approach is more plausible.

4.7 Factories and machines should be graph simulations Factories should not be simulated as arbitrary physics actors everywhere. They should be deterministic graphs of machines, storage, pipes, belts, routes, power, heat, and signals. Nearby systems can be animated at high fidelity; distant systems should collapse into rate/inventory/accounting models.

The resource accounting must remain exact or auditable, while the visual microstate can be reconstructed or

approximated when observed.

4.8 Fog of war limits client-shared compute The user wanted clients to share simulation load. The answer distinguished safe client assistance from unsafe authority. Clients can safely help with rendering, mesh generation, local prediction, build previews, pathfinding proposals, and non-authoritative local views. Clients should not authoritatively simulate hidden enemies, rare resources, production outputs, combat, market-affecting state, or secret world data.

The safe rule is:

```
client proposes
server verifies
server commits
```

4.9 Single-universe MMO means shared persistence, not one local room The chat concluded that a single persistent universe is possible in principle with partitioned regional authority, interest management, global persistence, and simulation LOD. But millions of players in one mutually interacting local space at high FPS and low latency is not a credible literal target. The design must use region boundaries, caps, time dilation, instancing or soft instancing, abstraction, and relevance filtering.

4.10 Simulation collapse/refinement must be a game rule The idea of "collapsing" everything players cannot sense is viable only if it is designed into the rules from the beginning. A collapsed factory must still preserve resource totals. A collapsed civilization must still have population/economy/security/climate states. When observed, it must refine into a visible state that is consistent with the macrostate, even if the exact per-object microstate was not simulated the whole time. This is a core mechanic, not an optimization that can be bolted on later.

5. Decisions, Recommendations, and Their Status

Row 1 ID: D-01; Decision / Recommendation: Do not wait for UE6 as the starting production engine; Status: Assistant recommendation; not separately user-ratified; Why it matters: Avoids delaying work for unavailable/uncertain tooling; Confidence: Medium-high; Label: INFERENCE / UNVERIFIED external facts

Row 2 ID: D-02; Decision / Recommendation: Use Unreal, if used, as renderer/client/tooling rather than canonical simulation; Status: Strong recommendation; Why it matters: Prevents engine lock-in and nondeterministic world truth; Confidence: High; Label: INFERENCE

Row 3 ID: D-03; Decision / Recommendation: Keep Dominium's core simulation engine-independent; Status: Strong recommendation aligned with user's portability goal; Why it matters: Enables UE6/Domino/other adapters later; Confidence: High; Label: INFERENCE

Row 4 ID: D-04; Decision / Recommendation: Build custom DominiumSim + DominiumWorldDB + DominiumServer; Status: Strong recommendation; Why it matters: Required for determinism, persistence, fog of war, sparse world state, and MMO authority; Confidence: High; Label: INFERENCE

Row 5 ID: D-05; Decision / Recommendation: Do not trust clients with persistent MMO authority; Status: Strong recommendation; Why it matters: Protects anti-cheat, economy, secrets, and fog of war; Confidence: High; Label: INFERENCE

Row 6 ID: D-06; Decision / Recommendation: Treat millions in one local space as infeasible; design one persistent universe instead; Status: Strong recommendation; Why it matters: Prevents impossible networking/performance target; Confidence: High; Label: INFERENCE

Row 7 ID: D-07; Decision / Recommendation: Start with a deterministic headless MVP; Status: Strong

recommendation; Why it matters: Proves the hardest invariant before renderer work dominates; Confidence: High; Label: INFERENCE

Row 8 ID: D-08; Decision / Recommendation: Preserve all generated reports and package files for future aggregation; Status: User requested and assistant executed; Why it matters: Maintains continuity across chats; Confidence: High; Label: FACT

Important caveat: these are mostly preserved as **recommendations and working conclusions**, not as final locked product requirements. A future planning pass should explicitly ask which items become formal Dominion requirements.

6. What Was Put Off for Later

The conversation deferred several necessary decisions and implementation details:

1. **Domino's exact role.** The chat did not define what Domino can currently do, what platforms it targets, whether it supports necessary rendering or scripting features, or whether it is meant to be a primary runtime or later adapter.
1. **Final engine/client choice.** Unreal was recommended as a plausible first visual client, but

the project has not formally committed to Unreal as the first frontend.

1. **UE6 verification.** Public UE6 details, release timing, feature set, and system requirements need current verification before they influence planning.
1. **Deterministic math design.** The chat recommended fixed-point/integer deterministic math but did not specify bit widths, units, overflow rules, coordinate encoding, or numerical libraries.
1. **Planet data model.** The chat proposed procedural base data plus sparse edits but did not decide the exact terrain representation: heightfield, voxel, SDF, hybrid, or another system.
1. **Collapse/refine rules.** The concept was discussed, but exact macrostate variables, refinement constraints, and fairness rules remain open.
1. **Networking architecture.** The chat described server authority, region partitioning, and interest management, but did not specify protocols, tick rates, authority handoff mechanics, database layout, or failure recovery.

1. **MMO scale target.** The chat separated "single universe" from "millions in one local space," but actual concurrency targets and performance budgets remain undefined.
 1. **Database/persistence system.** Event sourcing, snapshots, sparse chunk diffs, rollback, and audit trails were proposed, but no database technology or schema was chosen.
 1. **Security and abuse controls.** Lag machines, player-made computation, economy exploits, griefing, and client tampering were identified, but no concrete rule limits were designed.
 1. **Prototype scope.** The recommended next action is a deterministic `DominiumSim` MVP, but its exact acceptance tests still need to be defined.
-

7. Engines, Systems, and Ideas to Copy or Study

The conversation suggested copying ideas from multiple systems rather than trying to combine full engines at runtime.

Row 1 Reference: Unreal Engine 5 / future UE6; What to study or copy: Rendering, editor, animation,

asset tools, client frontend, construction
visualization; What not to assume: Do not assume
it can be the deterministic MMO universe engine

Row 2 Reference: Domino; What to study or copy:
Possible future frontend/runtime adapter; What
not to assume: Do not assume portability unless
core/adapter separation is built now

Row 3 Reference: Factorio; What to study or copy:
Deterministic factory simulation, replay discipline,
desync detection, graph-like production systems;
What not to assume: Its lockstep model does not
directly solve MMO authority at planetary scale

Row 4 Reference: EVE Online; What to study or copy:
Single-shard social/economic world, regional load
handling, time dilation philosophy; What not to
assume: Does not solve fully destructible planets or
deterministic factory terraforming

Row 5 Reference: Cesium / 3D Tiles; What to study or
copy: Massive geospatial streaming and
hierarchical tiles; What not to assume: Not a game
simulation solution by itself

Row 6 Reference: UNIGINE / Outerra-like systems;
What to study or copy: Planet-scale precision and
world rendering ideas; What not to assume: Does
not solve the entire MMO simulation and economy
stack

Row 7 Reference: Unity DOTS / Bevy / ECS systems; What to study or copy: Data-oriented design and parallel simulation ideas; What not to assume: Not enough by themselves for deterministic persistent planetary MMO

Row 8 Reference: SpaceEngine-like procedural universe systems; What to study or copy: Astronomical scale, procedural generation, LOD concepts; What not to assume: Rendering/exploration scale is not the same as editable persistent multiplayer simulation

The strategic recommendation was to copy architectural ideas and algorithms, not to bolt multiple full engines together.

8. Near-Term Recommended Roadmap

Phase 1 - Deterministic headless core Build a small command-driven simulation with fixed ticks, deterministic RNG, stable iteration order, state hashing, replay logs, and identical results across supported platforms.

Suggested MVP target from the earlier report:

- 10,000 machines
 - 1,000 terrain chunks
 - 100 players as command streams
- 1131

fixed tick
state hash every tick
replay from command log
same result on Windows/Linux/macOS

The exact numbers are candidates, not final requirements.

Phase 2 - Sparse planet region Create one procedural planet region with chunked terrain, editable material state, construction placement, save/load, and sparse deltas. Render it in a simple viewer or Unreal client.

Phase 3 - Collapse/refine simulation Implement active vs collapsed factories and settlements. Test whether resource accounting remains consistent when regions sleep, fast-forward, and wake.

Phase 4 - Server authority and fog of war Add server-owned hidden state, client visibility filtering, sensor rules, interest management, and anti-cheat validation.

Phase 5 - Region authority and persistence Implement multiple authoritative regions, handoff, persistent chunk database, event logs, snapshots, and graceful overload behavior.

Phase 6 - MMO-scale testing Scale gradually: local deterministic tests, then simulated clients, then multiple regions, then stress tests. Do not design around "millions in one room." Design around load-bounded regions in a single persistent universe.

9. Artifact Ledger and File Package

The package contains the previously created preservation files plus this companion report and verification index.

Artifact	Purpose	Notes
00_manifest.md	Overview of files and package status	Updated to include the companion report and verification file
01_human_readable_report.md	Main formal preservation report	Sections 0-16 from the original preservation task
02_context_transfer_packet.md	Handoff for a future chat	Use when continuing work elsewhere
03_spec_sheet.yaml	Structured YAML-style spec sheet	For later aggregation into a master project spec book
04_registers.md	Workstreams, decisions, tasks, constraints, risks, verification queue	Use for project tracking and cross-chat merging
05_aggregator_packet.md	Compact merge packet	For a central aggregation chat
06_reader_brief.md	Short human-readable brief	Use for fast reacquaintance
07_verification_and_audit.md	Audit and verification needs	Use before turning recommendations into formal requirements
08_future_chat_bootstrap_prompt.md	Prompt for starting a future chat	Paste with the context transfer packet
09_in_chat_reader.md	In-chat navigation guide	Use for asking follow-up questions inside this chat
10_accompanying_human_readable_detailed_summary_and_report.md	This file	Companion summary/report requested in the latest user turn

10. Main Open Questions

1. **What exactly is Domino?** The chat assumes Domino may be a future runtime or adapter, but its actual capabilities and constraints need definition.
1. **Should Unreal be the first visual client?** It is recommended as plausible, not formally chosen.
1. **What is the exact deterministic MVP?** The project needs a concrete first prototype specification.
1. **What numerical model should Dominium use?** Fixed-point/integer math is recommended, but the coordinate and units system remains open.
1. **How should collapsed simulation preserve fairness and accounting?** Exact rules need design.
1. **What is the terrain representation?** Sparse voxels, SDFs, heightfields, or hybrid systems remain to be tested.

1. **What MMO scale is actually targeted?**
"Millions in universe" needs bounded definitions:
concurrent, daily active, region-local, visible,
simulated, and interactable.

1. **What can clients safely compute?** A trust
matrix should be formalized.

1. **How should the database handle persistent
edits?** Event logs, snapshots, diffs, compression,
and rollback need design.

1. **Which earlier assistant statements require
re-verification?** UE6, UE5, platform
requirements, and engine feature claims should be
checked before formal adoption.

11. Risks and Failure Modes

Risk	Consequence	Mitigation
Treating Unreal as the whole game	Engine lock-in, nondeterminism, hard Domino migration	Keep canonical state in DominiumSim/WorldDB/Server
Waiting for UE6	Delayed progress based on uncertain tooling	Prototype deterministic core now
Trusting clients with MMO authority	Cheating, economy corruption, fog-of-war leaks	Server verifies and commits authoritative state
Trying to simulate full fidelity everywhere	Impossible compute and bandwidth costs	Use simulation LOD and collapse/refine rules
Designing player machines without limits	Lag weapons and unbounded computation	Add operation, graph, power, bandwidth, and tick-budget limits
Treating recommendations as final decisions	Future confusion and bad aggregation	Preserve status labels and require user confirmation
Forgetting artifact provenance	Misuse of stale or partial reports	Preserve manifest, verification, and source scope labels
Relying on stale UE6 claims	Wrong platform and roadmap assumptions	Verify before final planning
Confusing single universe with single local space	Impossible networking/performance target	Define universe, region, visibility, and interaction scopes separately

12. What a Future Assistant Must Not Get Wrong

A future assistant should not reduce this chat to "use Unreal" or "do not use Unreal." The actual conclusion is more specific: use Unreal only if it helps as a replaceable client/tooling layer, while keeping the authoritative deterministic game outside it.

A future assistant should also not claim that UE6 solves the project. The chat treated UE6 as uncertain and not currently sufficient as a foundation for planning.

A future assistant should not assume Domino portability happens automatically. It only happens if Dominium is architected around a portable core and engine adapters.

A future assistant should preserve the distinction between a single persistent universe and a single local mutually interacting space. The first is plausible with partitioning. The second is not plausible at the scale of millions with low latency and high FPS.

A future assistant should not treat every assistant suggestion as a user-approved final decision. Many items are strong recommendations that need user confirmation before entering the formal spec.

13. Best Next Action

The best next action is:

Define the first deterministic `DominiumSim` prototype in detail.

That should produce a small, testable specification covering:

- simulation tick rate,
- command input format,
- deterministic RNG,
- fixed-point/integer math rules,
- state hashing,
- replay logs,
- machine/factory graph model,
- terrain chunk edit model,
- save/load format,
- cross-platform determinism tests,
- pass/fail acceptance criteria.

This should happen before deep Unreal/UE6/Domino integration work, because it validates the hardest architectural invariant: whether `Dominium` can have a portable deterministic canonical simulation at all.

14. Compact Carry-Forward Summary

Dominium should be treated as a custom deterministic simulation and persistent-world system with replaceable visual clients. Unreal/UE5/UE6 can help render and edit the world, but should not own canonical game state. Domino may be a later frontend or runtime, but portability requires deliberate core/adaptor separation from the beginning. No commercial engine currently solves the full target: deterministic real-scale solar system, sparse procedural planets, terrain editing, machines/factories, fog of war, client compute, and MMO-scale single-universe persistence.

The permanent architecture is:

`DominiumSim + DominiumWorldDB + DominiumServer`

The renderer/client layer can be:

`Unreal now, UE6 later if useful, Domino later if useful, or other
adapters`

The most important rule is that the game must separate existence, simulation, rendering, knowability, and authority. Distant or unseen areas can exist in the database without being rendered or fully simulated. Factories can collapse into exact accounting graphs or rate models. Hidden enemy state can remain authoritative on the server without being sent to clients. Players can help compute visuals or proposals, but clients must not authoritatively decide MMO-persistent

outcomes.

The next project step is to define and build a deterministic headless `DominiumSim` MVP with replay, state hashes, fixed ticks, deterministic math, and a small factory/terrain model. Only after that should renderer choice dominate the work.

15. Specific Follow-Up Questions to Ask Next

Architecture

1. Define the first deterministic `DominiumSim` prototype in detail.
2. What belongs in `DominiumSim`, `DominiumWorldDB`, `DominiumServer`, and `DominiumClient`?
3. What is the exact API boundary between the simulation core and Unreal/Domino adapters?

Determinism

1. What fixed-point or integer coordinate system should `Dominium` use?
2. How should state hashing and replay logs work?
3. What operations must be forbidden or wrapped to preserve determinism?

Planet and terrain

1. Should Dominion use sparse voxels, SDF chunks, heightfields, or a hybrid for planets?
2. How should cut/fill operations store material conservation and ownership?
3. How should a real-scale solar system coordinate hierarchy be encoded?

Factories and machines

1. How should player-designed machines be represented as deterministic graphs?
2. What limits prevent player-made lag machines or general-computation exploits?
3. How should active and collapsed factories reconcile state?

Networking and MMO scale

1. What is a realistic first multiplayer target for Dominion?
2. How should fog of war and client interest filtering be designed?
3. What computations can clients safely assist with?

4. How should server region authority and handoff work?

Project preservation and spec book

1. Which recommendations from this chat should become formal Dominion requirements?
 2. Which parts should remain background context rather than hard requirements?
 3. How should this package be merged with other Dominion preservation reports?
 4. What external facts should be verified before final architecture planning?
-

16. Final Status of This Companion Report

This companion report was created to make the package easier to read without opening the full formal preservation report first. It preserves the entire visible conversation at a high but human-readable level, adds the latest packaging request, and explains how to use the generated files.

The report is safe to carry forward into future Dominion planning with these caveats:

- It is not a full master project spec.

- It does not include every older Dominium chat.
- It relies on the visible conversation and existing package files.
- It distinguishes assistant recommendations from user-final decisions.
- It marks UE6/engine facts as needing verification before formal use.

Chapter 21

Universe Explorer Planning

21.1 Dominion Universe Explorer
 Planning Handoff Human
 Readable Report

Source: docs/archive/conversations/Universe_Explorer_Planning/Dominium_Universe_Explorer_Planning_Handoff__01_human_readable_report.md

Report type: human_readable_report

Size: 33832 bytes

21.1.1 COMPLETE CHAT PRESERVATION
 REPORT - Dominion Universe
 Explorer Planning and Repo Handoff

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion
 Universe Explorer Planning and Repo Handoff

Row 2 Field: Date anchor; Assessment: 2026-05-27
 Australia/Melbourne

Row 3 Field: Source scope; Assessment: This chat only,
 1145

except clearly labelled PROJECT-CONTEXT from live repo docs and Project-level context referenced inside this chat

Row 4 Field: Apparent access; Assessment: Partial-to-current-visible transcript

Row 5 Field: Previously generated files available?; Assessment: No previously generated downloadable files in this chat before this package; repo files were read through GitHub connector

Row 6 Field: Uploaded files or artifacts present?; Assessment: Yes: Pasted `text.txt`, the preservation-task instruction file uploaded at the end of the chat

Row 7 Field: Contains future plans?; Assessment: Yes

Row 8 Field: Contains decisions?; Assessment: Yes, with caveats: some items are user-stated preferences or assistant recommendations not yet explicitly accepted

Row 9 Field: Contains pending tasks?; Assessment: Yes

Row 10 Field: Contains unresolved questions?; Assessment: Yes

Row 11 Field: Staleness risk; Assessment: Medium to High for live repo status and GitHub-derived facts; low for visible chat content

- Row 12** Field: Extraction confidence; Assessment: 4/5 for visible chat; 2/5 for external Dominion One/Two chats not visible here
- Row 13** Field: Safe for later aggregation?; Assessment: Yes, with caveats
- Row 14** Field: Main limitations; Assessment: I can preserve the visible chat and the repo facts accessed inside it, but I cannot recover full Dominion One/Two transcripts or unshown Project history. Repo status may have changed after the checked files.

Plain-language limitation: this preservation package is strong for this chat’s visible content. It is not a complete archive of all Dominion planning across all chats. Several early materials were pasted into this chat as excerpts, so they are preserved as content of this chat, but they are not independently verified against their original chat transcripts.

1. One-Page Orientation

This chat was a high-level Dominion planning, recovery, repo-audit, and forward-strategy session. The user began by pasting substantial prior discussion from earlier Dominion chats. Those excerpts covered the project’s deep simulation architecture: total truth with sparse materialization, assemblies/fields/processes/constraints,

epistemic and diegetic refinement, classifier-based emergence, macro-to-micro scaling, representation ladders, semantic ascent/descent, formalization chains, capability surfaces, institutional/cultural substrate, knowledge propagation, failure ontology, and deep primitives for matter, geometry, constraints, processes, affordances, and design knowledge.

The user then asked whether the assistant could recover the thread of thought from the *Dominium Two* chat. The assistant answered that it could not recover a hidden transcript, but could reconstruct the direction from Project and repo context. That reconstruction emphasized the transition from abstract architecture into repo-grounded semantic constitutionalization: Lambda core, semantic ownership review, Lambda bridge closure, Sigma, Phi, Upsilon, and later Zeta. The user then corrected the frame: this chat was not for implementation but for planning, exploration, and discussion. That correction mattered because it kept the session focused on conceptual synthesis and future planning rather than coding.

Next, the user requested a major recap and asked the assistant to inspect the live `julesc013/dominium` repo, especially long-standing specification documents. The assistant read many repo docs through the GitHub connector. These included the README, AGENTS, constitution, architecture summary, Universal Reality

Framework, Domain Contract Template, Capability Surfaces, Representation Ladders, Semantic Ascent/Descent, Formalization Chain, Cross-Domain Bridges, Player Desire Acceptance Map, Post-Pi execution plan, semantic ownership review, repo layout target, runtime kernel/component/service docs, interaction model, and Real-World Affordance Matrix. The assistant concluded that most of the important ideas from prior discussions were preserved in the repo, but some areas still needed consolidation: deep primitives, a master failure ontology, instantiated domain constitutions, player-facing formalization workflows, and a stable playable baseline.

The user then proposed a first major objective: build a seamless 1:1 scale universe explorer with continuous 6-axis movement, travel, inspection, reference-frame switching, streaming materialization, and no visible loading screens before embodiment and deep simulation systems. The assistant checked additional docs on no-modal-loading, streaming budgets, fidelity degradation, reference frames, orbits/timewarp, interest sets, universe model, spacetime, logical travel, reality/existence, visitability/refinement, renderer responsibility, SRZ, macro capsules, refinement contracts/lattices, universe-scale strategy, and Workbench docs. The assistant agreed with the goal but refined it: the first objective should be a governed Universe Explorer / Inspection Workbench, not a free-camera renderer. It should prove

lawful inspection, representation, reference frames, materialization, streaming, and continuity before full embodiment or gameplay.

Finally, the user pasted a newer repo-status summary saying the structure was now clean enough to stop broad structure refactors: canonical structure passed with warnings, fast strict and smoke proof passed, full CTest remained blocked by stale full-gate tests, broad feature work remained blocked, and the next lane should be presentation/projection contracts. The assistant verified current repo docs including `CANON_STRUCTURE_ACTUAL_FINAL_CLEANUP_01.md`, `FOUNDATION_LOCK.md`, `.aide/queue/current.toml`, and the doctrine recovery matrix. The final strategy was: stop broad structure cleanup, run `PRESENTATION-CONTRACT-01`, `PROJECTION-CONFORMANCE-01`, `WORKBENCH-SHELL-READONLY-01`, then create a Universe Explorer contract, headless proof, no-modal-loading proof, reference-frame proof, materialization proof, visual proof, interactive explorer, and only then embodiment/local-domain entry. This chat therefore contributes a focused bridge between the project's broad doctrine and the first product-shaped proof slice.

2. The Story of the Conversation

Opening: recovering the old direction The chat began with the user pasting long prior *Dominium*

planning text. That text already contained a mature architecture: not just a simulation design, but a meta-architecture for growth, collapse, substitution, verification, modding, alternate universes, and civilization-scale expansion. The initial theme was that *Dominium* needed a lawful framework that could absorb arbitrary future domains without special-case creep.

The pasted material proposed seven universal layers: ontology, materialization, recognition, encapsulation, substitution, verification, and governance. It argued for representation ladders rather than graphical LOD, explicit semantic ascent/descent, multiple truth-state classes, player intuition as a design constraint, institutional/cultural substrate, knowledge propagation, failure ontology, unknown-unknown support, and master doctrine documents.

Learning from other games and deep primitives

The user also pasted a later discussion about games like *Valheim*, *Stormworks*, *KSP*, *Eco*, *EVE*, *Foxhole*, *Vintage Story*, *Boundless*, etc. The point was that these games feel deep by choosing tractable substrates: voxels, blocks, parts, recipes, graphs, markets, law systems, or macro abstractions. They do not simulate arbitrary reality from first principles. The lesson for *Dominium* was to avoid both shallow prefabs and impossible atom-by-atom simulation. The best route was: define deep

primitives, allow stable behavior to emerge, recognize it, formalize it, and scale it.

The next pasted section went deeper into how players should manipulate matter, shape, constraints, tools, stations, processes, and affordances rather than item classes. Examples included cutting a tree, making a screw, throwing pottery, using a pottery wheel, building an axe, making a table, recognizing a chair affordance, and industrializing a handmade switch. This section established the "deep primitives plus design knowledge substrate" theme.

Recovering Dominium Two and correcting the frame The user then asked whether the assistant could recover the discussion from another project chat. The assistant responded with a reconstruction: the later thread had probably moved from abstract architecture into repo-grounded execution doctrine. It identified a sequence of Lambda, Sigma, Phi, Upsilon, and Zeta planning artifacts and suggested the recovered continuation was "repo-grounded semantic constitutionalization." The user clarified that this chat's job was not implementation, but planning, exploration, and discussion. The assistant adjusted: the right level was conceptual constitution, semantic stability, civilizational substrate, player-reality continuity, and boundaries.

Live repo audit and recap The user then asked for a major recap and a live repo review. The assistant read many live repository docs and produced a large synthesis. It concluded that the repo already contained most of the earlier high-level concepts as stable doctrine: Universal Reality Framework, Domain Contract Template, Capability Surfaces, Representation Ladders, Semantic Ascent/Descent, Formalization Chain, Player Desire Acceptance Map, Cross-Domain Bridges, and runtime constitutional docs. It also found structural governance and authority ordering in AGENTS, constitution, and planning docs.

The assistant identified what remained weak: domain constitutions, deep primitives consolidation, failure ontology, five truth-state taxonomy, player-facing formalization workflows, ordinary-life grounding, biology/ecology/agriculture/economy/cities/conflict/offworld domains, and playable-baseline fragility.

The Universe Explorer pivot The user then proposed a first major development objective: a seamless 1:1 universe explorer with continuous 6-axis movement, travel, inspection, reference-frame switching, streaming materialization, and no loading screens. The assistant checked further docs and agreed, but reframed it: this should be a governed Universe Explorer / Inspection Workbench, not a renderer/free-camera feature. The

explorer should prove no-modal-loading, fidelity degradation, frame switching, universe graph traversal, spacetime discipline, materialization requests, interest/fidelity control, macro capsules, SRZ compatibility, Workbench projection discipline, renderer purity, and provenance.

This changed the future plan from "eventually implement everything" to "first prove seamless lawful inspection/materialization at universe scale."

Final planning synthesis The user then supplied a newer structure-status summary: active old-path debt was gone, canonical structure passed with warnings, full CTest remained blocked, broad work was still blocked, and targeted cleanup plus presentation/projection contracts should be next. The assistant verified current repo files and gave the best plan forward: stop broad structure cleanup, follow the queue, build presentation/projection contracts, read-only Workbench shell, Universe Explorer contract, headless explorer proof, no-modal-loading proof, reference-frame proof, materialization proof, visual explorer, interactive explorer, then embodiment and local-domain entry. Parallel cleanup should be targeted, not broad.

The current preservation task then asked for this full chat package.

3. Main Topics Discussed

Topic 1 - Meta-architecture for Dominion The chat repeatedly treated Dominion as more than a game engine. The user's pasted content argued that the project needed governance of complexity: every future subsystem must prove it belongs, competing models must coexist, abstraction changes must preserve semantics, and future contributors must not smuggle chaos into the architecture. The conclusion was that Dominion needs a constitutional, self-policing architecture, not just a collection of simulation features. Uncertain: whether all seven universal layers are now formally captured under exactly that naming. The repo has equivalent or related doctrine, but not every early phrase appears as a single artifact.

Topic 2 - Representation ladders and semantic movement The user's earlier text argued that "LOD" was too weak and should be replaced with representation ladders. The assistant later verified that the repo contains a representation ladder spec. The important conclusion is that Dominion should move between lawful representations of the same truth, not merely reduce detail. Semantic ascent and descent are central: raw substrate can become recognized meaning; high-level summaries can descend into detail only if invariants and

lineage are preserved.

Topic 3 - Deep primitives and player manipulation

The pasted discussion on trees, screws, pottery, axes, chairs, tables, and machines established that the player should manipulate material, geometry, constraints, processes, tools, stations, and affordances. The conclusion is that "item classes" must not be the truth substrate. Recipes and blueprints can exist, but as higher-order formalizations.

Uncertain: the repo contains pieces of this through affordance/capability/domain docs, but the assistant did not find one single master Deep Primitives spec in the docs it read.

Topic 4 - Formalization chain and civilization

scaling A major theme was that useful local inventions must become portable, standardized, industrialized, and institutionally adopted. The repo now has a Formalization Chain spec and Player Desire Acceptance Map that strongly preserve this. The future work is making this playable: drafting, measuring, testing, blueprinting, certifying, teaching, manufacturing, maintaining, and revising designs.

Topic 5 - Live repo authority and current state

The assistant read the repo and found strong authority

ordering: canon/glossary/AGENTS outrank lower docs, generated evidence, and chat memory. The repo had completed major semantic doctrine and significant structure convergence. Later repo state indicates canonical structure passes with warnings, but feature readiness is limited and full CTest remains blocked.

Topic 6 - Universe Explorer first objective The user proposed the first major objective: seamless 1:1 universe explorer. The assistant refined it into a governed inspection/materialization proof. This became the most important forward-looking outcome of the chat. The explorer should prove reference frames, no-modal-loading, streaming, fidelity degradation, universe graph inspection, materialization requests, Workbench projection, and renderer purity before embodiment.

Topic 7 - Workbench and presentation/projection Workbench was treated as the likely first surface for the Universe Explorer, but only if it remains projection, not authority. The repo queue says PRESENTATION-CONTRACT-01 is next, with PROJECTION-CONFORMANCE-01 as alternate. The conclusion was that presentation/projection contracts must come before building UI or renderer features.

Topic 8 - Stop broad structure cleanup The final plan accepted the newer repo status: structure is clean enough to stop giant structure prompts. Remaining cleanup should be targeted: stale full-gate tests, pack layout canon, residual taxonomy, AIDE state classification, public header ABI, provider split. This is a planning recommendation, not yet an explicitly accepted user decision.

4. What We Were Trying to Achieve

4.1 Explicit Goals

1. Recover the lost train of thought from Dominionum One/Two.
2. Keep this chat at the planning/exploration level, not implementation.
3. Recap everything discussed and inspect live repo docs to avoid forgetting older specs.
4. Determine what the first major game/engine objective should be.
5. Integrate latest repo-structure status into a best plan forward.
6. Preserve this chat in a maximal-fidelity package for future aggregation.

4.2 Inferred Goals

1. Avoid losing important architecture from many months of chats.
2. Avoid starting implementation before the repo and doctrine are ready.
3. Ensure future assistants do not confuse repo truth with chat memory.
4. Convert a huge architecture vision into a realistic near-term product path.
5. Prepare material for a later master Project Spec Book.

4.3 Goals That Changed Over Time The early goal was broad recovery of lost architecture. It became repo-audited synthesis. Then it shifted toward a concrete first milestone: Universe Explorer. Finally it shifted toward packaging the chat for preservation.

4.4 Goals Still Unresolved

1. Whether PRESENTATION-CONTRACT-01 has been completed after the latest queue state.
2. Whether full CTest has been repaired.

3. Whether the Universe Explorer has a formal repo task yet.
4. Whether deep primitives and failure ontology should be authored as new specs.
5. How to merge this chat with other old-chat reports without conflicts.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Treat this chat as planning/exploration, not implementation; Status: Accepted by user; Why it mattered: Prevented implementation drift; Confidence: High; Label: FACT

Row 2 ID: DECISION-02; Decision: Repo truth outranks chat memory; Status: Established by repo doctrine and assistant framing; Why it mattered: Prevents stale reconstruction overriding canon; Confidence: High; Label: FACT/
PROJECT-CONTEXT

Row 3 ID: DECISION-03; Decision: Most old doctrine is preserved in the repo; Status: Assistant conclusion based on doc audit; Why it mattered: Avoids rewriting master doctrine from scratch; Confidence: Medium-high; Label: INFERENCE

Row 4 ID: DECISION-04; Decision: First major product objective should be a Universe Explorer / inspection proof; Status: User-proposed, assistant refined; Why it mattered: Gives product direction; Confidence: Medium-high; Label: FACT + INFERENCE

Row 5 ID: DECISION-05; Decision: Universe Explorer must be governed/projection-first, not free-camera renderer-first; Status: Assistant recommendation from docs; Why it mattered: Prevents renderer/camera becoming truth authority; Confidence: Medium; Label: INFERENCE

Row 6 ID: DECISION-06; Decision: Stop broad structure cleanup; use targeted cleanup only; Status: Assistant recommendation after repo status; Why it mattered: Avoids churn now structure is clean enough; Confidence: Medium; Label: INFERENCE

Row 7 ID: DECISION-07; Decision: Presentation/projection contracts should precede Workbench UI/explorer; Status: Strongly supported by queue and Foundation Lock; Why it mattered: Aligns product work with gates; Confidence: High; Label: FACT/PROJECT-CONTEXT

Row 8 ID: DECISION-08; Decision: Headless explorer proof should precede visual explorer; Status:

Assistant recommendation; Why it mattered:
Keeps first slice within narrow product-spine
constraints; Confidence: Medium; Label:
INFERENCE

Row 9 ID: DECISION-09; Decision: Embodiment
should come after explorer/materialization proof;
Status: Assistant recommendation consistent with
user proposal; Why it mattered: Avoids conflating
observer travel with entity travel; Confidence:
Medium-high; Label: INFERENCE

Row 10 ID: DECISION-10; Decision: Create
preservation package for this chat; Status: User
explicitly requested; Why it mattered: Enables
future aggregation and continuation; Confidence:
High; Label: FACT

Decision explanations:

- DECISION-01 was made by the user when they corrected the assistant: the chat's job is not to implement anything, but to plan, explore, and discuss. This governs the rest of the chat.
- DECISION-04 is the user's strongest product-direction statement. The user said the first major objective should be a seamless 1:1 scale universe explorer before embodiment and deeper simulation

systems. The assistant refined it but did not reject it.

- DECISION-06 is not fully user-accepted yet. It is a best-plan recommendation based on the pasted repo-status and verified docs.
- DECISION-08 is also a recommendation, not a user-confirmed final plan.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

1. Full arbitrary atom-by-atom simulation: rejected as impractical for game-scale systems; deep primitives are preferred.
2. Hardcoded object taxonomies: deprioritised; recognizers/formalization over substrate are preferred.
3. Graphics-only LOD: superseded by representation ladders.
4. Pure recipe crafting: rejected as too shallow; recipes may exist only as one expression of formalization.
5. Clean-room master framework replacement: rejected/deprioritised because the repo already has a Lambda doctrine spine.

6. Broad structure refactor prompts: now deprioritised because current structure is clean enough with warnings.
7. Renderer-first Universe Explorer: rejected as unsafe; projection/contract/headless proof should come first.
8. Embodiment-first gameplay: deprioritised until universe inspection/materialization is proven.
9. Treating Workbench as authority: rejected; Workbench is projection/orchestration surface.
10. Treating assistant reconstruction as transcript recovery: rejected; the assistant cannot recover hidden old transcripts verbatim.

7. Important Reasoning, Rationale, and Tradeoffs

The main tradeoff was between ambition and discipline. Dominion's vision is huge: universe-scale exploration, arbitrary invention, civilization simulation, institutions, knowledge propagation, and moddable alternate realities. But the repo and doctrine enforce a strict route: no silent semantic drift, no renderer authority, no mode flags, process-only mutation, pack-driven integration, explicit refusal, and repo truth over chat memory.

The Universe Explorer plan balances product motivation and architecture law. Starting with an explorer gives a

tangible first goal and tests the most foundational hard problem: seamless movement across scale. But if done as a normal free-camera renderer, it would violate multiple doctrines. Therefore the explorer must start as an inspection/materialization/projection proof.

The repo-state tradeoff is also important. Structure is no longer the main crisis, but full proof is not complete.

The correct response is not more massive cleanup and not broad feature implementation. It is targeted cleanup plus narrow product-spine work.

8. Plans, Future Work, and Next Steps

Recommended sequence:

1. PRESENTATION-CONTRACT-01
2. PROJECTION-CONFORMANCE-01
3. WORKBENCH-SHELL-READONLY-01
4. UNIVERSE-EXPLORER-CONTRACT-01
5. UNIVERSE-EXPLORER-HEADLESS-01
6. NO-MODAL-LOADING-PROOF-01
7. REFERENCE-FRAME-EXPLORER-PROOF-01
8. MATERIALIZATION-REQUEST-PROOF-01
9. UNIVERSE-EXPLORER-VISUAL-01

10. UNIVERSE-EXPLORER-INTERACTIVE-01

11. EMBODIMENT-ANCHOR-01

12. LOCAL-DOMAIN-ENTRY-01

Parallel maintenance lane:

1. FULL-GATE-LEGACY-TEST-ROUTE-01

2. PACK-INTERNAL-LAYOUT-CANON-01

3. RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01

4. PUBLIC-HEADER-ABI-PROMOTION-01

5. STORAGE-PACKAGE-PROVIDER-SPLIT-01

Potential planning-spec lane:

1. Deep Primitives reconciliation.

2. Master Failure Ontology.

3. Domain Constitution priority wave.

4. Player-facing Formalization UX model.

5. Representation proof doctrine.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences

- Planning/exploration only for this chat.
- Human-readable report first for preservation.
- Do not invent facts.
- Do not silently infer.
- Preserve uncertainty.
- Do not treat assistant suggestions as user decisions.
- Create downloadable files if tools are available.
- Preserve all important artifacts, files, prompts, risks, open questions, and tasks.

9.2 Inferred Constraints and Preferences

- User values direct, source-grounded, audit-ready answers.
- User prefers long-term correctness over speed.
- User wants repo docs and old planning preserved together.
- User dislikes losing train of thought across chats.
- User wants future assistants to avoid repeating already-corrected frames.

9.3 Uncertain or Unestablished Preferences

- Whether the user wants UNIVERSE-EXPLORER-CONTRACT-01 authored immediately after this preservation task.
- Whether the user wants full-gate cleanup before any explorer planning.
- Whether the user wants a formal Project Spec Book chapter next or more repo-task planning.

10. Files, Artifacts, Outputs, and Prompts

Artifacts in or produced for this chat:

1. `Pasted text.txt`: uploaded preservation task prompt. It defines the required output structure and must be preserved.
2. GitHub repo docs read during the chat: README, AGENTS, constitution, Universal Reality Framework, Domain Contract Template, Capability Surfaces, Representation Ladders, Semantic Ascent/Descent, Formalization Chain, Player Desire Acceptance Map, Cross-Domain Bridges, repo structure audit, Foundation Lock, queue, Workbench docs, no-modal-loading, reference frames, interest/fidelity, etc.

3. Assistant-produced narrative reports in chat: recovery, recap, Universe Explorer framing, best plan forward.
4. This export package: generated files and ZIP, created by this task.

No prior downloadable file package was created in this chat before the current preservation task.

11. Open Questions and Unresolved Issues

1. Has PRESENTATION-CONTRACT-01 been completed after the latest queue state?
2. Should the next chat generate UNIVERSE-EXPLORER-CONTRACT-01?
3. Should full CTest cleanup be prioritized before explorer planning?
4. Is the latest repo state still the same as 2026-05-23 audit files?
5. Does the repo already have a newer Universe Explorer or presentation contract task after the checked queue?
6. Should Deep Primitives and Failure Ontology become separate canonical specs?

7. How should this report merge with Dominion One/Two preservation reports?
8. Which assistant recommendations in this chat should be elevated to explicit user decisions?

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

- Treating assistant recommendations as user decisions.
- Forgetting the user's correction that this chat is planning-only.
- Treating the Universe Explorer as a broad renderer/gameplay task rather than a governed proof slice.
- Ignoring Foundation Lock and queue constraints.
- Restarting broad structure cleanup despite current clean-enough status.
- Assuming full CTest is green.
- Treating repo docs accessed earlier as permanently current.
- Confusing visible pasted excerpts with full original Dominion One/Two transcripts.
- Forgetting Workbench is projection, not authority.

- Letting camera/render distance drive materialization or truth.
- Over-compressing doctrine recovery and losing deep primitives/failure ontology gaps.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat contributes the bridge from abstract Dominion doctrine to first product-spine strategy. It should feed chapters on:

- Project continuity and doctrine preservation.
- Universal reality and representation law.
- Deep primitives and player fabrication.
- Formalization and civilization scaling.
- Workbench/product-spine development.
- Universe Explorer first milestone.
- No-modal-loading and streaming materialization.
- Reference frames and 1:1 scale navigation.
- Repo governance and Foundation Lock.
- Future task sequence.

Formal requirement candidates:

- First major product milestone should prove lawful universe-scale inspection/materialization.
- Workbench and renderer must not be truth authorities.
- Presentation/projection contracts must precede UI/explorer.
- Headless/null proof should precede visual explorer.
- Embodiment must follow visitability/refinement proof.

Background context candidates:

- Comparison to other games.
- Deep primitives examples: tree, screw, pottery, axe, chair, table.
- Prior Dominion One/Two recovery narrative.

Open issues for spec book:

- Deep Primitives master doctrine.
- Failure Ontology.
- Domain constitution order.
- Exact Universe Explorer task contract.

14. What I Should Remember

- This chat established a major pivot: the best first product objective is a seamless 1:1 universe explorer, but only as a governed inspection/materialization proof.
- The repo already preserves most old architecture doctrine; do not rewrite it from scratch.
- The next safe repo lane is presentation/projection/read-only Workbench, not broad features.
- Structure cleanup is no longer the main blocker; full CTest and targeted warning buckets remain.
- Workbench, renderer, and camera must not own truth.
- Universe Explorer should start headless/read-only before visual/interactive.
- Embodiment comes after reference-frame/materialization/visitability proof.
- Deep Primitives and Failure Ontology remain important consolidation gaps.
- This preservation package is for this chat only and should be merged carefully with other chat reports.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- "Explain again why Universe Explorer should be the first milestone but not renderer-first."
- "What parts of the old Dominion architecture were preserved in the repo?"

15.2 Decisions

- "Which recommendations in this chat should I explicitly accept as decisions?"
- "What decisions are still only assistant suggestions?"

15.3 Tasks and Next Actions

- "Draft UNIVERSE-EXPLORER-CONTRACT-01 as a planning spec."
- "Draft the prompt for PRESENTATION-CONTRACT-01."

15.4 Artifacts and Files

- "List all repo docs this chat relied on and group them by authority."
- "Turn this handoff package into a master index."

15.5 Risks and Verification

- "What repo facts in this chat should be re-verified before acting?"
- "What could go wrong if we implement the explorer too early?"

15.6 Future Spec Book / Aggregation

- "Convert this chat into chapters for a Project Spec Book."
- "Compare this report with a Dominion One report and find conflicts."

15.7 Deep-Dive Questions Specific to This Chat

- "Design the headless Universe Explorer proof."
- "Design the no-modal-loading proof harness."
- "Define Deep Primitives as a future spec."
- "Define Failure Ontology as a future spec."

16. Compact Human Summary

This chat was about recovering, checking, and then refocusing the Dominion project plan. The user began by pasting long earlier Dominion planning material.

That material described an ambitious simulation/game architecture based on truth totality, sparse materialization, representation ladders, semantic ascent/descent, formalization, substitution, verification, governance, deep primitives, affordances, and civilization-scale knowledge/institution systems. It also compared other sandbox and simulation games and concluded that Dominion should not hardcode every object or simulate every atom; it should define deep lawful primitives and let stable behavior emerge, be recognized, formalized, standardized, propagated, and scaled.

The user asked whether the assistant could recover the lost discussion from another project chat. The assistant reconstructed a likely continuation from Project/repo context: the project had moved toward repo-grounded semantic constitutionalization through Lambda, Sigma, Phi, Upsilon, and Zeta planning. The user corrected the frame: this chat was for planning and exploration, not implementation. That correction shaped everything afterward.

The user then asked for a complete recap and a live repo review. The assistant read many `julesc013/dominium` docs. The result was that most of the old conceptual doctrine was already preserved in the repo: Universal Reality Framework, Domain Contract Template, Capability Surfaces, Representation Ladders, Semantic

Ascent/Descent, Formalization Chain, Player Desire Acceptance Map, Cross-Domain Bridges, and runtime kernel/component/service doctrine. The repo also had strong authority ordering and structure governance. Remaining gaps included deep primitives consolidation, failure ontology, domain constitutions, player-facing formalization workflows, and a stable playable baseline.

The user proposed the first major development objective: a seamless 1:1 universe explorer with continuous movement, reference-frame switching, inspection, streaming materialization, and no visible loading screens. The assistant checked docs on no-modal-loading, streaming, fidelity, reference frames, orbits, interest sets, universe model, spacetime, logical travel, reality, visitability, renderer responsibility, SRZ, macro capsules, refinement, universe-scale strategy, and Workbench. The conclusion was that the Universe Explorer is a good first objective if framed correctly: it must be a governed inspection/materialization proof, not a renderer-first free-camera feature. It must prove reference frames, no-modal-loading, streaming, materialization, projection, and continuity before embodiment.

The user then supplied a newer status summary: repo structure is now clean enough to stop broad structure refactors, but not perfect; full CTest still has stale full-gate debt; broad feature work is blocked; presentation/projection contracts are next. The assistant verified repo

files and gave a final plan: stop broad structure cleanup, run PRESENTATION-CONTRACT-01, PROJECTION-CONFORMANCE-01, WORKBENCH-SHELL-READONLY-01, then create a Universe Explorer contract, headless proof, no-modal-loading proof, reference-frame proof, materialization proof, visual proof, interactive explorer, then embodiment/local-domain entry. Targeted cleanup should continue in parallel for full-gate legacy tests, pack layout, residual taxonomy, ABI/public header warnings, and provider split.

The most important unresolved issue is that the Universe Explorer plan is still a proposed/recommended next strategy, not yet a completed repo task. The next best action is either to draft PRESENTATION-CONTRACT-01 / UNIVERSE-EXPLORER-CONTRACT-01, or to preserve this chat and merge it with other old-chat reports into a master Project Spec Book.

21.2 **Dominium Universe Explorer Planning Handoff Reader Brief**

Source: docs/archive/conversations/Universe_Explorer_Planning/Dominium_Universe_Explorer_Planning_Handoff__06_reader_brief.md

Report type: reader_brief

Size: 1819 bytes

21.2.1 Reader Brief - Dominion Universe Explorer Planning and Repo Handoff

What this chat was about

This chat recovered old Dominion architecture discussion, checked the live repo, identified what had been preserved, reframed the first major product objective as a governed seamless Universe Explorer, and produced a plan to move from structure cleanup into presentation/projection and read-only explorer proof.

Top 20 things to know

1. The user wants planning/exploration here, not implementation.
2. The visible chat includes pasted earlier Dominion architecture content.
3. The assistant cannot recover hidden Dominion One/Two transcripts.
4. Repo truth must outrank chat memory.
5. Most old doctrine appears preserved in repo specs.
6. Deep primitives still need consolidation.
7. Failure ontology still needs a master taxonomy.
8. Domain constitutions remain major future work.

9. The user proposed a seamless 1:1 Universe Explorer as first major objective.
10. The assistant refined it as a governed inspection/materialization proof.
11. Workbench must be projection, not authority.
12. Renderer must not mutate truth.
13. No-modal-loading and streaming/fidelity degradation are core requirements.
14. Reference frames and fixed-point/segmented spacetime matter.
15. Camera movement must not create truth or activate simulation by itself.
16. Structure is now clean enough to stop broad structure prompts.
17. Foundation Lock allows narrow product-spine slices only.
18. Current queue says Presentation Contract is next.
19. Headless explorer proof should precede visual explorer.
20. Embodiment should come after materialization/reference-frame/visitability proof.

Best next step

Verify latest queue. If unchanged, proceed with PRESENTATION-CONTRACT-01; then draft UNIVERSE-EXPLORER-CONTRACT-01.

21.3 Dominium Universe Explorer Planning Handoff In Chat Reader

Source: docs/archive/conversations/Universe_Explorer_Planning/Dominium_Universe_Explorer_Planning_Handoff__09_in_chat_reader.md

Report type: in_chat_reader

Size: 1108 bytes

21.3.1 In-Chat Reader - Dominium Universe Explorer Planning and Repo Handoff

Package overview

This package preserves the current chat as a human-readable report, structured registers, a context-transfer packet, a YAML-style spec sheet, an aggregator packet, audit notes, a reader brief, and a future-chat bootstrap prompt.

File index

Read first: 01_human_readable_report.md.

For continuation: 02_context_transfer_packet.md.

For aggregation: 03_spec_sheet.yaml and 05_aggregator_packet.md.

For task planning: 04_registers.md.

For quick review: 06_reader_brief.md.

For caution: 07_verification_and_audit.md.

For new chat: 08_future_chat_bootstrap_prompt.md.

Top things to preserve

- Planning-only scope.
- Universe Explorer as first major objective, but projection/headless-first.
- Presentation/projection contracts before UI.
- Workbench and renderer are non-authoritative.
- Stop broad structure cleanup.
- Targeted cleanup only.
- Deep primitives and failure ontology gaps.

Safest next action

Verify current repo queue and then draft the next authorized planning artifact.

21.4 Dominion Universe Explorer Planning Handoff Accompanying Human Readable Detailed Summary

Source: docs/archive/conversations/Universe_Explorer_Planning/Dominium_Universe_Explorer_Planning_Handoff__10_accompanying_human_readable_detailed_summary.md

Report type: human_readable_detailed_summary

Size: 22428 bytes

21.4.1 Accompanying Human-Readable Detailed Summary and Report

Dominium Universe Explorer Planning and Repo Handoff

Date anchor: 2026-05-27 Australia/Melbourne

Scope: This conversation only, with clearly noted repo/project context where used

Purpose: A human-readable companion to the

structured preservation package already created in this chat

1. What this conversation was fundamentally about

This conversation was about turning a very large amount of Dominionium planning into a usable forward direction.

It began from old, high-density Dominionium design material. That older material described a project far larger than a conventional game: a deterministic simulation/game architecture that could eventually support invention, manufacturing, industry, logistics, economics, institutions, law, knowledge propagation, civilizations, alternate universes, modding, large-scale compression and expansion, and unknown player-created systems.

The key issue was not simply "what features should Dominionium have?" It was:

How can Dominionium grow for years or decades without collapsing into special cases, silent semantic drift, contradictory abstractions, or unmaintainable simulation layers?

The user wanted to recover the lost train of thought from prior chats, especially the transition between the earlier conceptual "Dominium One" discussion and the

later "Dominium Two" project/repo-grounded continuation. The assistant reconstructed the likely direction, then corrected course when the user clarified that this chat should remain a planning, exploration, and discussion chat rather than an implementation chat.

After that, the conversation became a repo-grounded planning audit. The assistant inspected the live `julesc013/dominium` repository docs and compared the repo's current doctrine with the older planning ideas pasted into the chat. The key conclusion was that most of the old major architecture doctrine was not lost. It had been captured in current repo artifacts.

The final stage of the conversation focused on the best plan forward. The user proposed that the first major objective for the engine/game should be a seamless 1:1-scale universe explorer with continuous 6-axis movement, inspection, reference-frame switching, streaming materialization, and no visible loading screens. The assistant agreed with the direction but reframed it as a governed inspection/materialization proof rather than a renderer-first free-camera feature.

The final strategic plan became:

Stop broad repo structure cleanup. Follow the narrow product-spine queue. Build presentation/projection contracts first. Then build a read-only Workbench shell. Then

build a headless Universe Explorer proof.
Then prove no-modal-loading, reference
frames, materialization, and only later visual/
interactive explorer and embodiment.

2. What the older **Dominium** material contributed

The opening material preserved several major
architecture ideas.

2.1 The seven-layer meta-architecture The old
pasted material proposed that every future domain in
Dominium should pass through a larger constitutional
structure:

1. **Ontology layer** - assemblies, fields, processes,
constraints, law, time, truth/perceived/render
separation.
2. **Materialization layer** - what is expanded,
summarized, deferred, observable, or causally
required.
3. **Recognition layer** - how stable patterns become
usable structures without hardcoding every object
taxonomy.

4. **Encapsulation layer** - how repeated stable things become blueprints, archetypes, capsules, and reusable patterns.
5. **Substitution layer** - when a summary, proxy, capsule, or equivalent representation can safely stand in for another.
6. **Verification layer** - replay, equivalence bounds, invariants, classifier stability, macro/micro tests, border reconciliation.
7. **Governance layer** - how new domains prove they belong, declare contracts, and avoid future chaos.

The important point was that Dominion needs not only simulation architecture, but a meta-architecture that governs how the architecture grows.

2.2 Representation ladders instead of LOD A central correction was replacing "LOD" with **representation ladders**. LOD is too graphical and too weak. Dominion needs lawful alternate representations of the same underlying reality.

Examples discussed:

- transport can move from raw terrain traces to paths, roads, route networks, district mobility summaries, and civilization-scale logistics flows;

- biology can move from chemistry and biomass fields to populations, species distributions, ecosystems, regional attractors, and biosphere balance;
- machines can move from raw materials and constraints to recognized machines, blueprints, factory-line archetypes, and industrial production networks.

The conclusion was that scale is not solved by drawing less detail. It is solved by lawful representation transitions that preserve identity, causality, provenance, invariants, and player-observable continuity.

2.3 Semantic ascent and descent Another key idea was **semantic ascent** and **semantic descent**.

Semantic ascent is how raw or lower-level substrate becomes higher-order meaning:

- repeated ruts become a path;
- a path becomes a road;
- a road becomes a logistics corridor;
- a settlement cluster becomes a village, town, or city;
- conductive assemblies become a circuit;
- circuit logic becomes a controller.

Semantic descent is the reverse direction: expanding high-level commitments back into lower-level details without inventing arbitrary facts.

Examples:

- an industrial town expands into factories, warehouses, streets, sidings, workers, smoke, lighting, and inventories;
- a rail corridor expands into rails, sleepers, switches, depots, maintenance crews, and signaling;
- a forest biome expands into trees, understory, fungi, streams, and animal populations.

The crucial conclusion was that ascent and descent must be explicit architecture, not accidental content generation.

2.4 Deep primitives The old material strongly argued that players should not primarily manipulate named object classes. They should manipulate deep primitives:

- **materials** with physical, chemical, electrical, thermal, and workability properties;
- **geometry** such as terrain volumes, rods, beams, sheets, fibers, channels, surfaces, and fluids;

- **constraints** such as hinges, sliders, joins, axles, gears, ropes, bearings, seals, conductive links, and fluid continuity;
- **processes** such as cutting, carving, forging, casting, firing, weaving, grinding, machining, repairing, routing fluids, or switching signals;
- **affordances** such as can be held, sat on, mounted, cut, controlled, traversed, read, instrumented, filled, connected, or slept on;
- **design knowledge** such as measurements, tolerances, standards, blueprints, testing procedures, maintenance procedures, process routes, and institutional know-how.

This is how players can make things the designers did not explicitly enumerate. Not by simulating every atom, and not by hardcoding every object, but by composing lawful primitives and allowing recognition/formalization systems to identify stable useful behavior.

2.5 Formalization chain The conversation preserved a major idea: one-off inventions must be able to become reusable civilization-scale knowledge.

The chain discussed was:

discovery -> recognition -> explicit capture -
 > specification/blueprinting -> validation/

testing -> standardization -> institutional
adoption -> production/deployment/diffusion
-> maintenance/revision -> deprecation/
supersession/retirement

This chain explains how a handmade switch, wheel, kiln, machine, route, or procedure becomes a repeated industrial or institutional standard.

It also prevents the project from collapsing into recipe-only crafting. A blueprint is not just a recipe. It is a lineage-bearing, testable, revisable, standardizable artifact.

2.6 Institutional and cultural substrate The earlier material emphasized that civilization is not just agents plus buildings. It requires institutions, law, administration, logistics doctrine, education, language, standards, engineering tolerances, manufacturing conventions, trade customs, scientific traditions, social classes, military organization, and maintenance traditions.

This was important because Dominium's long-term dream includes far-civilization scale. Without institutional/cultural substrate, distant societies would feel like decorative settlements rather than real civilizations.

2.7 Failure ontology The older planning also stressed that systems must fail plausibly. Every domain should declare failure modes:

- overload, fatigue, fracture, buckling;
- corrosion, clogging, leakage, shorting;
- erosion breach, habitat collapse, logistics starvation;
- institutional paralysis, administrative corruption;
- protocol incompatibility, cultural fragmentation.

The repo has domain contract requirements for failure semantics, but a master failure ontology remains a likely future planning task.

3. What the repo audit showed

The assistant inspected many current repo docs. The main result was that the project has moved from abstract architecture into formal repo doctrine.

3.1 What is already captured The live repo contains a strong semantic doctrine stack. The discussion identified these as the most important preserved artifacts:

- Universal Reality Framework;
- Domain Contract Template;
- Capability Surfaces;
- Representation Ladders;
- Semantic Ascent and Descent;
- Formalization Chain;
- Player Desire Acceptance Map;
- Cross-Domain Bridges;
- Semantic Ownership Review;
- Runtime Kernel Model;
- Component Model;
- Runtime Services;
- Foundation Lock;
- Canon structure cleanup audit;
- Doctrine Recovery Matrix.

The user's old ideas are largely present in these artifacts, though sometimes under different names or split across several docs.

3.2 What remains incomplete The repo is not done.
The main gaps identified were:

1. **Domain constitutions** need to be instantiated.
The template exists, but every serious domain needs its own contract.
2. **Deep primitives** exist indirectly, but likely need a master reconciliation spec.
3. **Failure ontology** is required per domain, but a cross-domain master taxonomy is still needed.
4. **Five truth-state classes** from the old discussion are conceptually near repo doctrine, but not clearly preserved as one named taxonomy.
5. **Player-facing formalization workflows** need to become concrete.
6. **Ordinary life** is accepted as first-class but not yet fully grounded in domain systems.
7. **Biology, ecology, agriculture, economy, urban systems, conflict, and offworld systems** remain less grounded than governance, authority, formalization, and engineering doctrine.
8. **Playable/product baseline** remains the practical near-term challenge.

3.3 Repo structure status Later in the conversation, the user pasted and the assistant verified a newer repo status:

- active tracked source structure now passes the canonical structure hard gate;
- no suspicious active paths are reported;
- fast strict, CMake verify/build, smoke CTest, and targeted capability tests pass;
- full CTest remains blocked by unrelated legacy/full-gate tests expecting old roots;
- feature readiness remains limited;
- broad feature work remains blocked.

This changed the plan. The project should stop broad structure cleanup and switch to narrow product-spine work.

4. The Universe Explorer objective

The user proposed the first major engine/game objective:

A seamless 1:1-scale universe explorer with continuous 6-axis movement, travel, inspection, reference-frame switching,

streaming materialization, and zero visible loading screens.

The assistant agreed with the direction but refined it.

The correct first objective is not just a renderer. It is:

a governed Universe Explorer / Inspection Workbench proof that validates space, time, reference frames, materialization, streaming, projection, and continuity.

4.1 Why this is the right first objective The Universe Explorer is a good first objective because it proves the foundation required by all later systems:

- enormous scale;
- no visible loading screens;
- continuous navigation;
- reference-frame switching;
- inspection;
- sparse materialization;
- macro/micro representation transitions;
- no renderer truth mutation;
- no fake pop-in;

- explicit refusal and degradation.

It lets the project prove that the world can be traversed and inspected before adding embodiment, survival, materials, NPCs, simulation responsibility zones, or civilization-scale systems.

4.2 Why it must be governed and projection-first

The danger is that a normal free-camera universe renderer could violate Dominion doctrine.

Bad version:

Camera goes near a thing, so the thing exists and micro-simulation activates.

Good version:

Observer/camera may request interest or inspection; refinement/materialization occurs only through explicit contracts, budgets, authority, provenance, and refusal/degradation paths.

The explorer must not be the world. It must be a lawful window into the world.

4.3 What the explorer needs to prove The assistant identified several proof areas:

- no-modal-loading and streaming budgets;
- fidelity degradation instead of stalls;
- deterministic reference-frame registry and switching;
- universe graph identity;
- spacetime and coordinate correctness;
- logical travel versus physical travel versus observer inspection;
- explicit existence states;
- visitability/refinement rules;
- representation ladders;
- semantic ascent/descent;
- macro capsules;
- simulation responsibility zone compatibility;
- state externalization;
- lifecycle visibility;
- domain-service bindings;
- capability surfaces;
- Workbench projection discipline;
- renderer purity.

5. Best plan forward

The final plan was:

5.1 Stop broad structure cleanup The repo is now structurally credible enough. More giant move passes would likely create churn. Remaining cleanup should be targeted.

Targeted cleanup lane:

- FULL-GATE-LEGACY-TEST-ROUTE-01;
- PACK-INTERNAL-LAYOUT-CANON-01;
- RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01;
- AIDE-STATE-CLASSIFICATION-01;
- PUBLIC-HEADER-ABI-PROMOTION-01;
- STORAGE-PACKAGE-PROVIDER-SPLIT-01.

5.2 Follow current product-spine queue The verified current queue showed:

- next task: PRESENTATION-CONTRACT-01;
- alternate: PROJECTION-CONFORMANCE-01;
- secondary follow-up: POINTER-WIDTH-SERIALIZATION-AUDIT-01;

- tertiary follow-up: WORKBENCH-SHELL-READONLY-01;
- broad feature work not authorized.

Therefore, the first practical path is not to start rendering. It is to define presentation/projection contracts.

5.3 Build the explorer in staged proofs

Recommended sequence:

1. PRESENTATION-CONTRACT-01
 2. PROJECTION-CONFORMANCE-01
 3. WORKBENCH-SHELL-READONLY-01
 4. UNIVERSE-EXPLORER-CONTRACT-01
 5. UNIVERSE-EXPLORER-HEADLESS-01
 6. NO-MODAL-LOADING-PROOF-01
 7. REFERENCE-FRAME-EXPLORER-PROOF-01
 8. MATERIALIZATION-REQUEST-PROOF-01
 9. UNIVERSE-EXPLORER-VISUAL-01
 10. UNIVERSE-EXPLORER-INTERACTIVE-01
 11. EMBODIMENT-ANCHOR-01
- 1200

12. LOCAL-DOMAIN-ENTRY-01

The key principle is:

Headless/projection proof before visual proof.

Visual proof before interactive explorer.

Explorer proof before embodiment.

Embodiment before full gameplay simulation.

6. What was decided versus recommended

This matters because the user asked for preservation and future aggregation, and the chat includes both firm decisions and assistant recommendations.

User-stated or clearly accepted

- This chat is for planning, exploration, and discussion, not implementation.
- The first major objective should be a seamless 1:1-scale universe explorer.
- The current project direction should integrate the latest repo state and not ignore Foundation Lock / structure warnings.
- A full preservation package should be generated.

Assistant recommendations

- Treat the Universe Explorer as a governed inspection/materialization proof, not a renderer-first free camera.
- Stop broad structure cleanup.
- Run presentation/projection contracts before Workbench UI or Explorer implementation.
- Build a headless explorer proof before a visual explorer.
- Put embodiment after explorer/materialization/frame proof.
- Draft Deep Primitives and Failure Ontology as future specs.

These recommendations are strong, but they should not be recorded as final user decisions unless the user later explicitly accepts them.

7. What was put off for later

Several things were intentionally deferred.

7.1 Broad feature work Deferred because Foundation Lock and queue say broad feature work is blocked:

- broad Workbench UI;
- runtime module loading;
- provider runtime;
- package runtime;
- gameplay;
- renderer implementation;
- native GUI;
- release publication;
- broad rewrites.

7.2 Embodiment and gameplay Deferred until after the Universe Explorer proves:

- reference frames;
- no-modal-loading;
- materialization;
- inspection;
- projection;
- renderer purity.

7.3 Simulation domains Deferred until after the first explorer/product proof:

- materials;
- worldgen expansion;
- biology;
- ecology;
- economy;
- settlements;
- institutions;
- civilization;
- SRZ;
- capability law realization;
- full ordinary-life systems.

7.4 Conceptual specs Deferred but identified as important:

- Deep Primitives master spec;
- Failure Ontology master spec;
- player-facing formalization workflows;

- domain constitution wave;
- representation proof doctrine.

7.5 Full-gate cleanup Deferred to a parallel targeted cleanup lane:

- full CTest stale old-root expectations;
 - pack internal layout;
 - residual taxonomy;
 - ABI/public header warnings;
 - provider split warnings.
-

8. What could go wrong if future chats mishandle this

Future assistants could easily derail the project by:

1. treating the Universe Explorer as a normal renderer/free-camera task;
2. ignoring the current queue and trying to implement broad Workbench UI or gameplay;
3. restarting broad repo structure cleanup;

4. treating assistant recommendations as accepted user decisions;
5. assuming repo state has not changed;
6. forgetting that this chat is planning-only;
7. over-compressing old doctrine and losing the deep primitives/formalization/failure ideas;
8. confusing observer movement with entity travel;
9. letting camera distance drive materialization;
10. letting Workbench or renderer mutate truth;
11. using generated/status artifacts as canon;
12. failing to distinguish fact, inference, project context, and uncertainty.

The main mitigation is to keep the generated preservation package, re-verify the repo, and proceed through the staged product-spine sequence.

9. What this new companion report adds beyond the prior package

The earlier package already included a comprehensive preservation report, registers, spec sheet, context transfer packet, aggregator packet, and file set.

This new companion report adds:

- a single continuous human-readable narrative summary;
- a clearer explanation of the conversation's arc;
- a clearer separation between user decisions and assistant recommendations;
- a clearer summary of what was deferred;
- a plain-language explanation of the Universe Explorer strategy;
- a final bundle manifest with file sizes and checksums;
- a new complete ZIP containing the original package files, the uploaded prompt, and this companion report.

10. Recommended next use

If continuing the work in this chat or a new chat, the safest next prompt is:

Verify the latest repo queue and Foundation Lock, then draft PRESENTATION-CONTRACT-01 if it is not complete; if it is complete, draft UNIVERSE-EXPLORER-CONTRACT-01 as a planning artifact, preserving Workbench

projection discipline, no-modal-loading, reference-frame, materialization, and renderer-purity constraints.

If building a master project spec book later, this conversation should feed these chapters:

- Project continuity and doctrine recovery;
- Universal Reality and Representation Doctrine;
- Workbench and Projection Discipline;
- Universe Explorer first milestone;
- No-modal-loading and streaming;
- Reference frames and spacetime;
- Materialization and refinement;
- Deep primitives;
- Failure ontology;
- Repo governance and task sequencing.

11. Final compressed summary

This conversation preserved the transition from Dominion's older abstract simulation architecture into a

repo-grounded forward plan. The old material established a deep philosophy: Dominion should be based on lawful primitives, representation ladders, semantic ascent/descent, formalization chains, domain contracts, substitution, verification, governance, institutions, knowledge propagation, and failure ontology. The repo audit showed that most of this has already been captured in current docs and specs, though gaps remain around deep primitives, failure ontology, domain instantiation, and player-facing formalization.

The user then proposed the first major product objective: a seamless 1:1-scale universe explorer. The assistant agreed, but reframed it as a governed inspection/materialization proof rather than a renderer-first free camera. It should prove continuous navigation, no-modal-loading, streaming, reference frames, universe graph inspection, materialization/refinement, Workbench projection, renderer purity, provenance, refusals, and degradation before embodiment or gameplay.

The latest repo state says structure is clean enough to stop broad refactors, but full CTest and broad feature readiness are not done. Foundation Lock allows only narrow product-spine work. Therefore the best plan is to run presentation/projection contracts, then a read-only Workbench shell, then a Universe Explorer contract and headless proof, then no-modal-loading/reference-frame/materialization proofs, then minimal visual and

interactive explorer, and only after that embodiment/
local-domain entry.

The most important thing to remember is:

Dominium's first product proof should be a
lawful window into a huge universe, not a
graphics demo that accidentally becomes the
universe.

Chapter 22

Workbench Aide Product Spine

22.1 Dominion Architecture Workbench Aide Product Spine Planning Human Readable Report

Source: docs/archive/conversations/Workbench_AIDE_Product_Spine/Dominium_Architecture_Workbench_AIDE_Product_Spine_Planning__01_human_readable_report.md

Report type: human_readable_report

Size: 85664 bytes

22.1.1 COMPLETE CHAT PRESERVATION REPORT - Dominion Architecture, Workbench, AIDE, and Product-Spine Planning

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion Architecture, Workbench, AIDE, and Product-Spine Planning

- Row 2** Field: Date anchor; Assessment: 2026-05-27
Australia/Melbourne
- Row 3** Field: Source scope; Assessment: This chat only
unless labelled PROJECT-CONTEXT
- Row 4** Field: Apparent access; Assessment: Partial,
with high contextual coverage
- Row 5** Field: Previously generated files available?;
Assessment: No previously generated downloadable
handoff files were visible; this task creates new files.
- Row 6** Field: Uploaded files or artifacts present?;
Assessment: Yes. The uploaded `Pasted text.txt`
contains the preservation/export instructions for
this task.
- Row 7** Field: Contains future plans?; Assessment: Yes
- Row 8** Field: Contains decisions?; Assessment: Yes
- Row 9** Field: Contains pending tasks?; Assessment: Yes
- Row 10** Field: Contains unresolved questions?;
Assessment: Yes
- Row 11** Field: Staleness risk; Assessment: Medium
- Row 12** Field: Extraction confidence; Assessment: 4 / 5
- Row 13** Field: Safe for later aggregation?; Assessment:
Yes, with caveats

Row 14 Field: Main limitations; Assessment: The visible transcript contains skipped-message markers; not every earlier raw turn is available. Several repo states were reported by the user/Codex and/or queried during this chat, but the live repo can advance after this report. This preservation therefore records the conversation state and flags live repo verification as required.

Plain-language limitations:

This report is not a perfect raw transcript substitute. It reconstructs the working state from the visible conversation, user-pasted transcripts, assistant outputs, repo-state checks performed in this chat, and the final uploaded preservation instruction. Some prior messages were collapsed as "Skipped messages" in the visible conversation. Therefore, this report should be treated as a high-fidelity reconstruction, not a legal archive of every token. The next chat should always verify live repo state before acting on task order or claiming a prompt has been executed.

The largest uncertainty is not the conceptual architecture; that has converged. The largest uncertainty is live execution state: whether prompts generated near the end of the chat have already been run locally, committed, or pushed. The next chat should first inspect `.aide/queue/current.toml`, recent commits, and

relevant audit files.

1. One-Page Orientation

This chat was primarily about moving Dominium from a large, ambitious but risk-prone game/simulation project into a disciplined, contract-governed, deterministic simulation platform with a reusable engine substrate, a production Workbench, and an AIDE-driven development workflow. The user's underlying concern was that Dominium should not become a one-off indie project with ad hoc code, drifting folders, duplicated UI systems, unverified docs, and fragile feature work. The conversation repeatedly returned to the same long-term aim: make all code, data, systems, and development workflows portable, modular, extensible, deterministic where needed, replaceable, testable, and compatible with future refactors or even total rewrites.

The main architectural identity that emerged is:

`Domino = reusable deterministic substrate`

`Dominium = game/product family on Domino`

`Workbench = production, validation, editing, inspection, packaging, and evidence environment`

`AIDE = repo/control-plane harness`

`Codex = bounded patch executor`

`Contracts = law`

`Diagnostics / tests / replay / evidence = proof`

A major shift was that the discussion stopped treating

"the game," "the UI," "the Workbench," or "the folders" as the center. Instead, the center became contracts, commands, services, providers, packs, modules, artifacts, capabilities, diagnostics, tests, replay, and evidence. The system should be able to replace private implementations and even directories while stable semantic IDs, public contracts, conformance tests, and artifact identities remain intact.

The user also wanted to understand how to build extremely realistic worlds without brute-force simulation. Earlier in the conversation, the thread moved from "procedurally generate a world and simulate billions of years" to "truth is total, materialization is sparse, history can be lazily and deterministically evaluated, and observation/causality controls refinement. " That thinking later generalized into the whole project: sparse materialization, macro/micro collapse, epistemic/diegetic gating, process-only mutation, representation ladders, and formalization chains should apply across worldgen, fabrication, ecology, civilization, UI, tools, and agentic development.

A second major stream focused on player creation and deep primitives. Rather than hardcoding every object as a primitive, the conversation converged on materials, geometry, fields, constraints, processes, affordances, recognizers, formalizations, blueprints, standards, and production routes. This is how players might eventually

make things like tools, wheels, machines, vehicles, rails, computers, factories, infrastructure, and civilizations without the engine requiring every object to be hand-coded as a special case.

A third stream focused on Workbench. The old idea of a UI Editor / Tool Editor was superseded by the Dominion Workbench Platform: a modular, command-driven, visual and agentic production environment for building Dominion itself. Workbench should eventually help create, validate, preview, package, and prove code, data, contracts, packs, modules, UI/HUD documents, themes, renderer tests, release artifacts, and AIDE/Codex work units. But Workbench must not become a monolith or authority. It edits documents and dispatches registered commands; runtime executes services; contracts define legality; content packages authored payloads; tools validate.

A fourth stream focused on AIDE and development workflow. The final doctrine is: development should be non-blocking, but promotion should be evidence-blocked. The user wants to run many prompts in parallel across dev branches, machines, and Codex sessions. The agreed architecture is task branches/worktrees per prompt, AIDE-managed blocker classification and repair queues, `origin/dev` as an integration branch, checkpoint branches for proof, and `origin/main` as promoted truth. The new chat must not let agents directly mutate shared

dev without isolation.

At the end of the conversation, the immediate repo plan was narrow and sequenced. Foundation Lock is `PASS_WITH_WARNINGS`, broad feature work remains blocked, `PACKAGE-MOUNT-SLICE-01` is complete, and the next intended product-spine work is `REPLAY-PROOF-SLICE-01`, then `BAREBONES-CLIENT-SHELL-01`, then `PRODUCT-SPINE-REVIEW-01`. After that, the project can begin limited parallel development, especially AIDE workflow law and presentation contracts.

2. The Story of the Conversation

2.1 Starting point: preserving architecture while expanding worldgen The conversation began from a handoff prompt for continuing Dominion worldgen and life systems after a prior architecture freeze. The user established that the project already had a Constitutional Architecture with pinned semantic contracts, determinism discipline, stability tags, CAP-NEG, `PACK-COMPAT`, `LIB/APPSHELL/TIME-ANCHOR` concepts, and frozen `v0.0.0-mock` architecture. The new development focus was procedural worldgen and domain realism: erosion, hydrology, ecology, evolution, speciation, biomes, climate feedback loops, geology, resource formation, and long time-scale planetary transformations.

The user asked whether realistic worlds could be generated by simulating years to billions of years, and whether that conflicts with sparse, on-the-fly generation. The resulting decision was that brute-force global simulation is not required or desirable. Instead, *Dominium* should use deterministic lazy historical evaluation: total truth defined by seed/laws/processes, but materialized only where observation, interaction, or causality requires it.

2.2 Generalizing worldgen into a project-wide doctrine The user then asked whether epistemics and diegetics could further optimize what is simulated, loaded, or rendered. This expanded the idea beyond worldgen. The conversation converged on a general pattern: the simulation's authoritative truth can be total and deterministic, while representation, materialization, detail, and presentation can vary based on observation, capability, epistemic state, and causal need.

This logic was then applied to terrain modification, structures, intelligent life, machines, civilizations, and player-authored worlds. The assistant repeatedly framed *Dominium* as a multi-representation reality substrate rather than a feature pile.

2.3 Player construction, machines, and civilization realism A large portion of the chat explored how a

player might create things from first principles: screws, planks, pottery wheels, kilns, vehicles, switches, wires, rails, solenoids, CPUs, factories, cities, and more. The problem was how the game knows when arbitrary shapes become a vehicle, a pipe, a rail, a chair, a table, or a machine.

The solution that emerged was not to hardcode every object and not to rely purely on player declarations. Instead, use deep primitives: materials, geometry, constraints, fields, processes, affordances, recognizers, formalizations, and design knowledge. The system can detect stable useful patterns, classify affordances, and let players formalize them into blueprints, standards, production routes, and mass-manufacturable artifacts.

This same approach supports distant civilizations: macro truth for unvisited places, expansion into detail when observed, and collapse back to capsules while preserving observed facts and causal continuity.

2.4 Repo convergence and Foundation Lock A major later phase shifted from pure architecture into repo-specific planning. The repo had undergone extensive CONVERGE and POST-CONVERGE work: layout contracts, root allowlists, stale doc supersession, root recycling, AIDE control plane, build proof recovery, product boot proof, package projection, internal release baseline, move-family cleanup, Foundation scaffolding,

C17/C++17 language-policy direction, public surface registry, ABI policy, dependency direction law, command/diagnostic/artifact/provider/module/replacement/version/trust/portability scaffolds, and fast strict gates.

The live repo state changed repeatedly during the conversation. Earlier, Foundation Lock was blocked by dependency-direction violations. Later, Foundation Lock closed as `PASS_WITH_WARNINGS`. The user and assistant repeatedly emphasized that `PASS_WITH_WARNINGS` is not "clean/no warnings"; full CTest remains T4/full-gate debt, and broad feature work remains blocked.

2.5 Workbench and Interface Operating Layer

The old "UI Editor / Tool Editor" plan was superseded by the Dominion Workbench Platform. Workbench should become the visual and agentic production environment for building Dominion: code, data, contracts, packs, modules, UI/HUD layouts, renderer tests, sounds/assets, tests, releases, and AIDE work units.

The conversation refined this further: the real platform is not Workbench itself, but the reusable Interface Operating Layer underneath it. CLI, text/TUI, rendered GUI, native GUI, and headless modes should be projections over the same command/result/refusal/document/view/action spine. Workbench is the richest

consumer and authoring environment for that spine.

2.6 Prompt-generation and task sequencing In the later chat, the user requested a series of Codex/AIDE prompts. The assistant generated prompts for:

```
MATRIX-CLEANUP-00
WORKBENCH-VALIDATION-SLICE-01
SERVICE-CONFORMANCE-LAW-01
DOCUMENT-PATCH-TRANSACTION-RUNTIME-01
PROJECT-GRAPH-SERVICE-01
COMPOSITION-RESOLVER-LAW-01
DOMINIUM-DOCTRINE-RECOVERY-MATRIX-01
COMMAND-RESULT-VIEW-SLICE-01
PHASE-REVIEW-02
QUEUE-RECONCILE-01
REPLAY-PROOF-SLICE-01
BAREBONES-CLIENT-SHELL-01
```

The user noted that earlier prompts were hard to copy because they were not always contained in one code block, and that prompts should better handle dirty worktrees and concurrent tasks. From then on, generated prompts included detailed dirty worktree handling, allowed/forbidden paths, non-goals, validation, blocker classification, and commit/final-response formats.

2.7 Parallel development and AIDE OS Near the end, the user wanted to run prompts in parallel on a permanent `origin/dev` and `local/dev`, then

checkpoint/merge to **main**. The conversation converged on the AIDE OS model: agents should not mutate shared dev directly; each prompt should run in a task branch/worktree; AIDE integrates to dev; checkpoint branches prove waves; main receives only evidence-backed promotions.

The final recommended sequence was: finish replay proof and barebones client, run product spine review, then begin limited parallel dev. Larger parallel waves should wait until minimum AIDE workflow law exists.

2.8 Chat retirement / preservation The user then requested this full preservation package so a new chat could continue without re-explaining everything. This final handoff is itself part of the preservation output.

3. Main Topics Discussed

Topic 1 - Deterministic Sparse Worldgen and Aged Worlds

The user wanted worlds that feel geologically, biologically, historically, and civilizationally real. The first idea considered was generating a world and simulating it for years to billions of years. The refined conclusion was that naive full simulation is incompatible with scale and unnecessary. Instead, Dominion should define a total deterministic truth and materialize/evaluate detail lazily.

Key conclusions:

Truth is total.

Materialization is conditional.

History can be lazily evaluated.

Observation and causality drive refinement.

Global brute-force simulation is avoided.

Remaining uncertainty: exact implementation of erosion/ecology/evolution proxies remains future domain work.

Topic 2 - Epistemics and Diegetics as

Optimization Doctrine The conversation treated epistemics and diegetics as more than flavor. They determine what must be simulated, loaded, rendered, or refined. If a player has not observed something, it can remain summarized so long as future expansion is consistent with seed, law, and causal history.

This supports:

unknown/unobserved detail summarized

measured domains refined

instrumentation affects knowledge

observation anchors prevent continuity breaks

Topic 3 - Deep Primitives for Player Creation

The user asked how players could make arbitrary things: axes, tables, screws, kilns, pottery wheels, springs, machines, switches, wires, vehicles, rails, computers,

factories, and more.

The concluded primitive stack:

materials

geometry

fields

constraints

processes

affordances

recognizers

formalizations

measurements

tolerances

standards

blueprints

production procedures

institutions/knowledge

Rejected: hardcoding all objects as primary truth or requiring all player intent through non-diegetic declarations.

Topic 4 - Civilization Macro/Micro Expansion

Distant societies should not require continuous detailed simulation. They can exist as macro truth: population distributions, infrastructure, culture, industry, governance, institutions, logistics corridors, and knowledge propagation. When observed, they expand into buildings, agents, institutions, and local histories. They can collapse back while preserving observed facts.

Topic 5 - Repo Architecture and Foundation Lock

The chat discussed multiple repo states over time: CONVERGE, POST-CONVERGE, AIDE, root cleanup, Foundation Lock, public surface registry, ABI law, dependency-direction, language baseline, component matrix cleanup, package mount, and command-result-view.

Current expected state:

```
Foundation Lock = PASS_WITH_WARNINGS
fast strict = PASS
dependency direction = 0 violations, known warnings
full CTest = T4/full-gate debt
broad feature work = blocked
```

Topic 6 - Workbench Platform Workbench evolved from "UI editor/tool editor" into "visual and agentic production environment." It should eventually author views, layouts, themes, widgets, TUI panels, rendered screens, HUDs, setup flows, launcher pages, admin dashboards, packs, modules, release evidence, and AIDE/Codex work units.

Key correction: Workbench should host workflows but not own truth.

Topic 7 - Interface / Presentation Architecture

The UI architecture converged on:

```
command/result/refusal/document spine
1225
```

-> view model
-> projection adapter
-> CLI / text / rendered / native / headless

CLI, text/TUI, rendered GUI, native GUI, and headless are projections, not separate semantic systems.

Topic 8 - AIDE Development Operating System

AIDE should evolve from prompt generator into task operating system:

Scheduler

Ledger

Repair Engine

Checkpoint Engine

Promotion Engine

Core rule:

Development is non-blocking.

Promotion is evidence-blocked.

Topic 9 - Prompt Design The user needed copyable prompts in a single code block. Future prompts should include:

status

context

goal

allowed paths

forbidden paths

dirty worktree handling

task details

non-goals
validation
expected result
block conditions
commit format
final response format

Topic 10 - Parallel Dev and Branching Agents should not all mutate dev directly. Use:

task/<task-id>
repair/<task-id>
origin/dev
checkpoint/<wave-id>
origin/main

Topic 11 - Replay Proof and Barebones Client PACKAGE-MOUNT-SLICE-01 completed; next product-spine tasks are replay proof and barebones client shell.

Topic 12 - Future Domain Systems Worldgen, ecology, evolution, fabrication, economy, institutions, cities, conflict, and offworld systems remain future domain constitution work, not immediate implementation.

4. What We Were Trying to Achieve

4.1 Explicit Goals

1. Build realistic procedural worlds without brute-force global simulation.
2. Make Dominionium modular, extensible, deterministic, portable, and future-proof.
3. Avoid repeating refactor spirals.
4. Create a Workbench that helps build the project.
5. Use AIDE/Codex effectively for repo development.
6. Generate copyable prompts for each next task.
7. Transition to parallel dev safely.
8. Preserve this chat fully for a new chat.

4.2 Inferred Goals

1. Preserve project momentum without losing accumulated reasoning.
2. Reduce user's cognitive load by making AIDE manage blockers and tasks.
3. Ensure future assistants do not restart settled debates.
4. Turn a highly ambitious game idea into a disciplined platform.

4.3 Goals That Changed Over Time

- Worldgen realism expanded into full platform architecture.
- UI Editor/Tool Editor became Workbench Platform.
- Workbench-centered view was refined into contracts-centered architecture with Workbench as surface.
- Serial prompt execution evolved toward AIDE-managed parallel development.
- Old C89/C++98 portability discipline shifted toward C17/C++17 with C-compatible ABI.

4.4 Goals Still Unresolved

- Full CTest cleanup.
- Runtime package mount.
- Provider runtime.
- Runtime module loader.
- Workbench shell.
- Presentation contracts.
- Replay proof execution status.

- Barebones client execution status.
- AIDE dev/main workflow law.
- Actual gameplay/domain implementation.

5. Decisions Made and Why

Decision Overview Table

Row 1 ID: DECISION-01; Decision: Dominion is a deterministic simulation operating environment; Status: Accepted; Why it mattered: Avoid feature pile, enable platform longevity; Confidence: High; Label: FACT

Row 2 ID: DECISION-02; Decision: Domino is reusable substrate; Dominion is product family; Status: Accepted; Why it mattered: Enables reuse in other games/projects; Confidence: High; Label: FACT

Row 3 ID: DECISION-03; Decision: Workbench is production environment, not authority; Status: Accepted; Why it mattered: Prevents editor truth drift; Confidence: High; Label: FACT

Row 4 ID: DECISION-04; Decision: AIDE is repo/control-plane harness; Status: Accepted; Why it mattered: Needed for parallel, repairable development; Confidence: High; Label: FACT

- Row 5** ID: DECISION-05; Decision: Development non-blocking, promotion evidence-blocked; Status: Accepted; Why it mattered: Balances speed and correctness; Confidence: High; Label: FACT
- Row 6** ID: DECISION-06; Decision: Use task branches/worktrees, not direct shared-dev mutation; Status: Accepted; Why it mattered: Prevents concurrency chaos; Confidence: High; Label: FACT
- Row 7** ID: DECISION-07; Decision: Keep canonical source roots closed; Status: Accepted; Why it mattered: Prevents root sprawl; Confidence: High; Label: FACT
- Row 8** ID: DECISION-08; Decision: C17/C++17 target baseline with C-compatible ABI; Status: Accepted as doctrine; verify implementation; Why it mattered: Modernizes without ABI loss; Confidence: Medium-high; Label: FACT + VERIFY
- Row 9** ID: DECISION-09; Decision: CLI/text/rendered/native/headless are projections; Status: Accepted; Why it mattered: Avoids four UI systems; Confidence: High; Label: FACT
- Row 10** ID: DECISION-10; Decision: Full CTest remains T4 debt; Status: Accepted; Why it mattered: Keeps product-spine progress moving; Confidence: High; Label: FACT

Row 11 ID: DECISION-11; Decision: Renderer first wave: null, software, OpenGL 3.3, D3D11; Status: Accepted; Why it mattered: Practical platform floor; Confidence: High; Label: FACT

Row 12 ID: DECISION-12; Decision: `vector2d` is drawing/canvas, not renderer backend; Status: Accepted; Why it mattered: Avoids matrix confusion; Confidence: High; Label: FACT

Row 13 ID: DECISION-13; Decision: Broad feature work remains blocked; Status: Accepted; Why it mattered: Prevents premature implementation; Confidence: High; Label: FACT

Row 14 ID: DECISION-14; Decision: Package mount is fixture-level, not runtime; Status: Accepted / current; Why it mattered: Avoid false support claims; Confidence: High; Label: FACT

Prose Explanation of Key Decisions DECISION-01 matters because the user's ambitions include worldgen, player invention, civilizations, modding, Workbench, AIDE, and future interfaces. A normal game-feature model would collapse. A deterministic simulation operating environment lets each capability become a contract-backed subsystem.

DECISION-05 matters because the user wants many agents and machines to work in parallel without being

stopped by every blocker. But main cannot become a dumping ground. This is why dev is permissive and main is evidence-blocked.

DECISION-09 matters because separate CLI, TUI, GUI, and native systems would duplicate behavior and drift. The chosen architecture makes command/result/refusal/view/action the truth and projections the surface.

DECISION-14 matters because PACKAGE-MOUNT-SLICE-01 has only proven fixtures and reports. Treating it as package runtime would create false implementation claims.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

Row 1 ID: REJECTED-01; Option: Simulate whole worlds continuously for billions of years; Status: Rejected; Reason: Too expensive; lazy deterministic history preferred; Reconsider conditions: Local/refined time simulation remains possible; Label: FACT

Row 2 ID: REJECTED-02; Option: Hardcode every object/machine/vehicle; Status: Rejected; Reason: Does not scale to arbitrary creation; Reconsider conditions: Specific built-ins may exist as modules/templates; Label: FACT

Row 3 ID: REJECTED-03; Option: Require all player-

created things be manually declared; Status: Rejected as sole method; Reason: Too rigid/non-diegetic; Reconsider conditions: Optional formalization remains useful; Label: FACT

Row 4 ID: REJECTED-04; Option: Pure intent inference; Status: Rejected; Reason: Engine should detect patterns/affordances, not read minds; Reconsider conditions: Assistant/UX can infer suggestions; Label: FACT

Row 5 ID: REJECTED-05; Option: Workbench as monolithic everything editor; Status: Rejected; Reason: Would become a second platform/authority; Reconsider conditions: Integrated shell with modules is accepted; Label: FACT

Row 6 ID: REJECTED-06; Option: Native-widget-first UI editor; Status: Rejected; Reason: Does not reuse client/rendered UI path; Reconsider conditions: OS-native projections later; Label: FACT

Row 7 ID: REJECTED-07; Option: Separate CLI/TUI/rendered/native semantics; Status: Rejected; Reason: Causes drift; Reconsider conditions: Projection model accepted; Label: FACT

Row 8 ID: REJECTED-08; Option: All agents mutate dev directly; Status: Rejected; Reason: Conflict/

attribution risk; Reconsider conditions: Task branches/worktrees accepted; Label: FACT

Row 9 ID: REJECTED-09; Option: Build full AIDE OS before product-spine work; Status: Deprioritised; Reason: Would stall product progress; Reconsider conditions: Minimum AIDE law before large parallel wave; Label: FACT

Row 10 ID: REJECTED-10; Option: Treat full CTest as normal immediate blocker; Status: Deprioritised; Reason: Too heavy; T4/full-gate debt; Reconsider conditions: Release/trust phase; Label: FACT

Row 11 ID: REJECTED-11; Option: OpenGL 1.1/D3D9 first-wave renderer; Status: Rejected; Reason: Back-port/research only; Reconsider conditions: Future constrained lanes; Label: FACT

Row 12 ID: REJECTED-12; Option: C89/C++98 as active mainline constraint; Status: Superseded; Reason: C17/C++17 baseline accepted; Reconsider conditions: Retro/research lanes; Label: FACT + VERIFY

7. Important Reasoning, Rationale, and Tradeoffs

The conversation repeatedly balanced ambition against collapse. The user wants an ultimate simulation platform, but the chosen path is incremental proof slices.

The system must avoid both extremes: building only doctrine with no usable product, and rushing into broad feature implementation that breaks modularity.

The most important tradeoff is speed versus evidence. The user wants non-blocking development on `dev`, but main must remain evidence-backed. This led to the doctrine "development is non-blocking, promotion is evidence-blocked."

Another tradeoff is simulation realism versus compute cost. The chosen solution is deterministic sparse materialization rather than global brute-force simulation.

A third tradeoff is UI richness versus architectural reuse. Workbench should eventually be powerful, but it must use the same command/view/projection system as client, launcher, setup, server, CLI, and TUI.

A fourth tradeoff is portability versus modern code quality. The final doctrine accepts C17/C++17 for mainline while preserving C-compatible ABI and explicit serialization.

8. Plans, Future Work, and Next Steps

Recommended Immediate Sequence 1. Verify current repo state.

2. If queue stale after `PACKAGE-MOUNT-SLICE-01`, run `QUEUE-RECONCILE-01`.

3. Run `REPLAY-PROOF-SLICE-01` if missing.

4. Run `BAREBONES-CLIENT-SHELL-01` if missing.

- 5. Run PRODUCT-SPINE-REVIEW-01.
- 6. If review passes, begin limited parallel dev:
AIDE-WORKFLOW-LAW-01
PRESENTATION-CONTRACT-01
POINTER-WIDTH-SERIALIZATION-AUDIT-01

Why this order Replay proof turns deterministic claims into a proof fixture. Barebones client proves product survival floor. Product Spine Review determines whether limited parallel development is safe.

- Next Phase after Product Spine Review** AIDE-WORKFLOW-LAW-01
- AIDE-WORKUNIT-SCHEMA-01
 - AIDE-DEV-MAIN-POLICY-01
 - AIDE-CHECKPOINT-LOOP-01
 - AIDE-CAPABILITY-REALITY-LEDGER-01

- Presentation Phase** PRESENTATION-CONTRACT-01
- PROJECTION-CONFORMANCE-01
 - ACCESSIBILITY-CONTRACT-01
 - TEXT-LOCALIZATION-CONTRACT-01

- Workbench Expansion** WORKBENCH-SHELL-01
- PROJECT-GRAPH-EXPLORER-01
 - PACK-BROWSER-01
 - AGENT-WORK-BOARD-01
 - COMPONENT-MATRIX-VIEWER-01
 - RENDERER-SANDBOX-01
 - THEME-LAB-01

Domain Phase Later only:

SPEC-DEEP-PRIMITIVES-01

SPEC-FAILURE-ONTOLOGY-01

SPEC-PLAYER-FORMALIZATION-WORKFLOWS-01

DOMAIN-CONSTITUTION-WAVE-01

CROSS-DOMAIN-BRIDGE-INSTANTIATION-01

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences

- Answers should be direct, source-grounded, and audit-ready.
- Explicitly label uncertainty.
- Do not invent facts.
- Do not smooth over contradictions.
- Prompts must be copyable in one code block.
- Do not ask whether to proceed when user already asked to proceed.
- Use repo state as source of truth.
- Preserve warnings; do not claim warning-free if repo says `PASS_WITH_WARNINGS`.

- No broad feature work without gate authorization.
- No false implementation claims.
- Do not re-ask already answered questions.

9.2 Inferred Constraints and Preferences

- User prefers architecture before implementation.
- User wants strong challenge/correction.
- User prefers long, structured answers when preserving context.
- User wants future assistants to avoid restarting doctrine.

9.3 Uncertain or Unestablished Preferences

- Exact desired level of final file detail vs chat detail may vary; user requested both.
- Whether future prompts should always include optional fast strict depends on task scope.
- Whether `origin/dev` already exists needs verification.

10. Files, Artifacts, Outputs, and Prompts

Uploaded File

ID	Artifact	Type	Purpose	Status
ARTIFACT-01	/mnt/ data/ Pasted text.txt	Uploaded text file	Contains the preservation/ export task instruction	Loaded

Important repo files/audits referenced

.aide/queue/
current.toml

- .aide/context/latest-task-packet.md
- .aide/context/latest-review-packet.md
- .aide/reports/latest-dominium-status.md
- .aide/reports/latest-warning-disposition.md
- docs/repo/FOUNDATION_LOCK.md
- docs/repo/audits/PACKAGE_MOUNT_SLICE_01.md
- docs/repo/audits/REPLAY_PROOF_SLICE_01.md
- docs/repo/audits/BAREBONES_CLIENT_SHELL_01.md
- docs/repo/audits/QUEUE_RECONCILE_01.md
- docs/repo/audits/COMMAND_RESULT_VIEW_SLICE_01.md
- docs/repo/audits/WORKBENCH_VALIDATION_SLICE_01.md
- docs/repo/audits/PHASE_REVIEW_02.md

Prompts generated in this chat

MATRIX-CLEANUP-00

- WORKBENCH-VALIDATION-SLICE-01
- SERVICE-CONFORMANCE-LAW-01
- DOCUMENT-PATCH-TRANSACTION-RUNTIME-01
- PROJECT-GRAPH-SERVICE-01

COMPOSITION-RESOLVER-LAW-01
DOMINIUM-DOCTRINE-RECOVERY-MATRIX-01
COMMAND-RESULT-VIEW-SLICE-01
PHASE-REVIEW-02
QUEUE-RECONCILE-01
REPLAY-PROOF-SLICE-01
BAREBONES-CLIENT-SHELL-01

Files created by this preservation task See sections 35-39 and linked files in final chat response.

11. Open Questions and Unresolved Issues

Row 1 ID: QUESTION-01; Question / issue: Has QUEUE-RECONCILE-01 run?; Why it matters: Determines next task; Known: Prompt generated; Unknown: Execution status; Priority: P0

Row 2 ID: QUESTION-02; Question / issue: Has REPLAY-PROOF-SLICE-01 run?; Why it matters: Required before barebones client; Known: Prompt generated; Unknown: Execution status; Priority: P0

Row 3 ID: QUESTION-03; Question / issue: Has BAREBONES-CLIENT-SHELL-01 run?; Why it matters: Required before product spine review; Known: Prompt generated; Unknown: Execution status; Priority: P0

Row 4 ID: QUESTION-04; Question / issue: Does CMake fully reflect C17/C++17?; Why it matters: Avoid docs/build mismatch; Known: Doctrine says yes; Unknown: Live build files need check; Priority: P1

Row 5 ID: QUESTION-05; Question / issue: Does origin/dev exist?; Why it matters: Parallel dev planning; Known: Desired branch model; Unknown: Repo state; Priority: P1

Row 6 ID: QUESTION-06; Question / issue: What warnings remain after latest prompts?; Why it matters: Promotion policy; Known: Known warning categories; Unknown: Exact live count; Priority: P1

Row 7 ID: QUESTION-07; Question / issue: When to start large parallel dev?; Why it matters: Workflow scale; Known: After AIDE workflow layer; Unknown: Exact readiness; Priority: P1

Row 8 ID: QUESTION-08; Question / issue: What should be first Workbench runtime shell task?; Why it matters: Workbench roadmap; Known: Validation slice exists; Unknown: Shell readiness; Priority: P2

Row 9 ID: QUESTION-09; Question / issue: When to tackle full CTest?; Why it matters: Release trust;

Known: T4 debt; Unknown: Burn-down schedule;
Priority: P2

Row 10 ID: QUESTION-10; Question / issue: When to
begin domain/game work?; Why it matters: Main
ambition; Known: Later after product spine;
Unknown: Exact gate; Priority: P2

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Risk	Consequence	Mitigation
Treating PASS_WITH_WARNINGS as clean	False confidence	Always preserve warnings
Restarting doctrine	Wasted time, inconsistent plans	Use this handoff and live repo
Treating Workbench as authority	Architectural drift	Workbench must use commands/services
Starting broad feature work too soon	Refactor spiral	Respect blockers
Assuming docs are implementation	False support claims	Capability Reality Ledger
Agents mutating shared dev	Conflicts	Use task branches/worktrees
Overbuilding AIDE OS before product proof	Stalls progress	Minimum AIDE layer only
Ignoring full CTest debt	Release risk	Keep T4 debt visible
Creating new roots	Root sprawl	Closed root model
Making separate UI systems	Drift	Projection architecture
Running destructive git cleanup	Data loss	Use blocker-handling rules
Assuming latest prompt ran	Wrong task order	Verify audits and queue

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat should feed into future spec book chapters on:

Project identity and doctrine
Repo governance and root model
Foundation Lock and gates
AIDE task operating system
Dev/main/checkpoint workflow
Workbench production environment
Presentation/projection architecture
Package/pack/module/provider composition
Replay/proof evidence
Barebones product floors
Worldgen/domain doctrine
Deep primitives and player creation
Civilization macro/micro simulation
Renderer/platform baseline
Language/ABI policy
Testing and release gates

Formal requirement candidates:

Path is not identity.
Implementation is not contract.
UI is not authority.
Generated output is not source truth.
Development is non-blocking.
Promotion is evidence-blocked.
Every stable capability must have evidence.
Every prompt must preserve known warnings and broad blockers.

Context-only candidates:

nixie/VFD/industrial UI inspirations

closed-eye cognitive UI concept

OEM+ OS-style theme ideas

long-term BCI/multiverse vision

Needs verification before merging into master spec:

live status of latest prompts

C17/C++17 build implementation

AIDE branch policy files

current queue state

current warning counts

14. What I Should Remember

- The project identity has converged; do not restart it.
- The next chat should verify repo state first.
- Package mount is complete as fixture proof, not runtime.
- Replay proof and barebones client are the next product-spine tasks if not already done.
- AIDE should enable non-blocking dev but evidence-blocked promotion.
- Workbench is a production environment, not authority.
- UI modes are projections of shared semantics, not separate systems.

- Full CTest remains T4 debt.
- Broad feature work is blocked.
- Future parallel dev should use task branches/worktrees, not direct shared-dev mutation.
- Do not hide warnings.
- Do not claim support without evidence.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

1. "Explain the final Dominion / Domino / Workbench / AIDE architecture in simple terms."
2. "What are the three most important doctrines from this chat?"
3. "How did we get from worldgen to AIDE dev/main workflow?"

15.2 Decisions

1. "Which decisions are final, and which are still tentative?"
2. "Why did we reject Workbench as a monolithic editor?"

3. "Why is full CTest not blocking current product-spine work?"

15.3 Tasks and Next Actions

1. "Generate PRODUCT-SPINE-REVIEW-01 if replay and barebones are complete."
2. "Generate AIDE-WORKFLOW-LAW-01."
3. "Generate PRESENTATION-CONTRACT-01."

15.4 Artifacts and Files

1. "List all audits and queue files the next chat should inspect first."
2. "What should be in the Capability Reality Ledger?"
3. "What file should record dev/main promotion policy?"

15.5 Risks and Verification

1. "What could go wrong if we start parallel dev too early?"
2. "What repo facts must be verified before the next prompt?"
3. "How do we prevent docs from outrunning implementation?"

15.6 Future Spec Book / Aggregation

1. "Which parts of this chat should become formal spec requirements?"
2. "Which parts are background context only?"
3. "What should be merged with other old-chat reports?"

15.7 Deep-Dive Questions Specific to This Chat

1. "Explain the Workbench self-hosting ladder."
2. "Explain how player-created machines fit the deep primitive model."

16. Compact Human Summary

This chat was about turning Dominion into a durable, deterministic, modular simulation platform rather than a one-off game. It began around worldgen and realism, asking whether worlds should be procedurally generated and then simulated for years to billions of years. The answer became a larger doctrine: truth can be deterministic and total, while materialization can be lazy and sparse. Instead of simulating everything everywhere, the system can use seeds, laws, processes, checkpoints, capsules, and observation-driven refinement to create

aged worlds, histories, terrain, climates, biomes, species, and civilizations without brute-force global simulation.

That idea expanded to the whole project. The user wanted players to be able to make arbitrary things: tools, screws, pottery wheels, kilns, rails, vehicles, switches, computers, factories, infrastructure, and cities. The answer was not to hardcode every object or force players to declare everything. Instead, *Dominium* should use deep primitives: materials, geometry, fields, constraints, processes, affordances, recognizers, formalizations, blueprints, standards, and production routes. Useful structures can be recognized, formalized, optimized, and later mass-produced.

The project's architecture then became the main focus. *Dominium* should be a deterministic simulation operating environment. *Domino* is the reusable substrate; *Dominium* is the game/product family; *Workbench* is the production and validation environment; *AIDE* is the repo/control-plane harness; *Codex* is a bounded patch executor; contracts are law; diagnostics, replay, tests, and evidence are proof. The core laws are: path is not identity; implementation is not contract; UI is not authority; generated output is not source truth.

Repo planning converged on a stable root structure: apps, engine, game, runtime, contracts, content, docs, tests, tools, scripts, cmake, external,

release, and archive. The project moved through Foundation Lock, component matrix cleanup, Workbench validation, service conformance, document/patch/transaction law, project graph law, composition resolver law, doctrine recovery, command-result-view proof, and package mount proof. The current status is `PASS_WITH_WARNINGS`, not warning-free. Fast strict passes. Dependency-direction strict has zero violations but known warnings. Full CTest remains T4/full-gate debt. Broad feature work remains blocked.

Workbench was another major topic. The old idea of a UI Editor / Tool Editor was superseded. Workbench should eventually become the visual and agentic production environment for building Dominion itself: views, layouts, themes, widgets, workspaces, packs, modules, release evidence, and AIDE/Codex work units. But Workbench must not become a separate authority. It should edit documents, emit patches, run validation, preview outputs, and package artifacts. Runtime executes; contracts constrain; content supplies; apps consume; tests prove.

The presentation architecture also converged. CLI, text/TUI, rendered GUI, native GUI, and headless are not separate semantic systems. They are projections of the same command/result/refusal/document/view/action spine. This prevents drift and allows client, server, launcher, setup, Workbench, CI, and future tools to

reuse the same semantics.

At the end, the user wanted to run many prompts in parallel on `dev`. The answer was yes, but only after the next product-spine slices and a product-spine review. The doctrine is: development is non-blocking, promotion is evidence-blocked. Agents should not all mutate shared `dev`; each prompt should run on its own task branch/worktree. AIDE integrates into `dev`, checkpoint branches prove waves, and `main` receives only evidence-backed promotions.

The immediate next steps are to verify live repo state, reconcile the queue if stale, run `REPLAY-PROOF-SLICE-01` if missing, run `BAREBONES-CLIENT-SHELL-01` if missing, then run `PRODUCT-SPINE-REVIEW-01`. After that, limited parallel development can begin with `AIDE-WORKFLOW-LAW-01`, `PRESENTATION-CONTRACT-01`, and `POINTER-WIDTH-SERIALIZATION-AUDIT-01`. Larger parallel waves should wait until AIDE work-unit schema, dev/main policy, and checkpoint loop are defined.

17. Workstream Register

Row 1 ID: WORKSTREAM-01; Name: Domino/
Dominium Platform Architecture; Objective:
Establish deterministic, modular platform identity;
Current state: Converged doctrine; Desired end
state: Reusable substrate + game family +

evidence-backed runtime; Status: Active; Priority: P0; Confidence: High; Label: FACT

Row 2 ID: WORKSTREAM-02; Name: Repo Foundation and Governance; Objective: Keep repo governable and validated; Current state: Foundation PASS_WITH_WARNINGS; Desired end state: Stable foundation with classified warnings; Status: Active; Priority: P0; Confidence: High; Label: FACT

Row 3 ID: WORKSTREAM-03; Name: Product Spine; Objective: Build narrow usable command/package/replay/client spine; Current state: Package mount done; replay/barebones pending/uncertain; Desired end state: Package mount + replay proof + barebones client + review; Status: Active; Priority: P0; Confidence: High; Label: FACT

Row 4 ID: WORKSTREAM-04; Name: AIDE Task OS; Objective: Enable parallel non-blocking development; Current state: Conceptual, partial AIDE exists; Desired end state: Scheduler/ledger/repair/checkpoint/promotion system; Status: Planned; Priority: P1; Confidence: High; Label: FACT

Row 5 ID: WORKSTREAM-05; Name: Workbench; Objective: Production/validation/editing

environment; Current state: Validation slice only;
Desired end state: Self-hosting modular authoring
environment; Status: Planned/partial; Priority:
P1; Confidence: High; Label: FACT

Row 6 ID: WORKSTREAM-06; Name: Presentation
Architecture; Objective: Unify CLI/text/rendered/
native/headless; Current state: Command-result-
view slice exists; Desired end state: Projection
conformance and presentation contracts; Status:
Planned/partial; Priority: P1; Confidence: High;
Label: FACT

Row 7 ID: WORKSTREAM-07; Name: Package/
Composition; Objective: Compose apps from
packs/modules/providers; Current state:
Composition law + package mount fixture; Desired
end state: Runtime-safe package/profile/provider
composition; Status: Active/planned; Priority: P1;
Confidence: High; Label: FACT

Row 8 ID: WORKSTREAM-08; Name: Replay/Proof;
Objective: Prove deterministic command replay;
Current state: Prompt generated, execution
uncertain; Desired end state: Replay/proof slice
and later full replay systems; Status: Pending;
Priority: P0; Confidence: Medium; Label:
UNCERTAIN

Row 9 ID: WORKSTREAM-09; Name: Barebones

Product Floors; Objective: Products run without optional content; Current state: Client prompt generated, execution uncertain; Desired end state: No-content help/status/diag floor; Status: Pending; Priority: P0; Confidence: Medium; Label: UNCERTAIN

Row 10 ID: WORKSTREAM-10; Name: Worldgen/Simulation Domains; Objective: Realistic long-term simulation domains; Current state: Future doctrine only; Desired end state: Domain constitutions and slices; Status: Deferred; Priority: P2; Confidence: High; Label: FACT

Row 11 ID: WORKSTREAM-11; Name: Renderer/Platform; Objective: Define and later implement render/platform providers; Current state: Matrix cleaned; Desired end state: Software + OpenGL + D3D11 etc. providers; Status: Deferred implementation; Priority: P2; Confidence: High; Label: FACT

Row 12 ID: WORKSTREAM-12; Name: Spec Book Aggregation; Objective: Merge old-chat reports into master project spec; Current state: This report created; Desired end state: Master Project Spec Book; Status: Planned; Priority: P1; Confidence: High; Label: FACT

18. Decision Register

- Row 1** ID: DECISION-01; Decision: Dominion is a deterministic simulation platform; Status: Accepted; Evidence / basis: Repeated user acceptance; Rationale: Needed for full ambition; Implications: Contracts/evidence centered architecture; Related workstream: WORKSTREAM-01; Confidence: High; Label: FACT
- Row 2** ID: DECISION-02; Decision: Domino is reusable substrate; Status: Accepted; Evidence / basis: Chat doctrine; Rationale: Enables other games/projects; Implications: Split engine/runtime/contracts from game/content; Related workstream: WORKSTREAM-01; Confidence: High; Label: FACT
- Row 3** ID: DECISION-03; Decision: Workbench is not authority; Status: Accepted; Evidence / basis: Repeated corrections; Rationale: Prevents editor truth drift; Implications: Workbench must use commands/services; Related workstream: WORKSTREAM-05; Confidence: High; Label: FACT
- Row 4** ID: DECISION-04; Decision: AIDE should become repo task OS; Status: Accepted; Evidence / basis: User explicitly agreed; Rationale: Needed

for parallel agents; Implications: Workflow law and work-unit schema needed; Related workstream: WORKSTREAM-04; Confidence: High; Label: FACT

Row 5 ID: DECISION-05; Decision: Development non-blocking, promotion evidence-blocked; Status: Accepted; Evidence / basis: User + Codex agreement; Rationale: Allows dev progress but protects main; Implications: dev/task/checkpoint/main policy; Related workstream: WORKSTREAM-04; Confidence: High; Label: FACT

Row 6 ID: DECISION-06; Decision: Task branches/worktrees per prompt; Status: Accepted; Evidence / basis: Final correction; Rationale: Avoid shared dev chaos; Implications: Future parallel prompt model; Related workstream: WORKSTREAM-04; Confidence: High; Label: FACT

Row 7 ID: DECISION-07; Decision: Full CTest is T4 debt; Status: Accepted; Evidence / basis: Repo status; Rationale: Keeps current work unblocked; Implications: Must remain visible; Related workstream: WORKSTREAM-02; Confidence: High; Label: FACT

Row 8 ID: DECISION-08; Decision: Package mount slice is fixture-level only; Status: Accepted;

Evidence / basis: Package audit; Rationale: Avoid false runtime claims; Implications: Next replay proof; Related workstream: WORKSTREAM-07; Confidence: High; Label: FACT

Row 9 ID: DECISION-09; Decision: C17/C++17 target baseline; Status: Accepted as doctrine; Evidence / basis: User/Codex plan; Rationale: Modern code while keeping ABI; Implications: Verify live CMake; Related workstream: WORKSTREAM-01; Confidence: Medium; Label: FACT + VERIFY

Row 10 ID: DECISION-10; Decision: CLI/text/rendered/native/headless are projections; Status: Accepted; Evidence / basis: UI architecture discussion; Rationale: Avoid duplicate UI systems; Implications: Presentation contract needed; Related workstream: WORKSTREAM-06; Confidence: High; Label: FACT

Row 11 ID: DECISION-11; Decision: Use closed source-root model; Status: Accepted; Evidence / basis: Repo convergence; Rationale: Avoid root sprawl; Implications: No new top roots; Related workstream: WORKSTREAM-02; Confidence: High; Label: FACT

Row 12 ID: DECISION-12; Decision: Vector2D is canvas/drawing, not renderer; Status: Accepted;

Evidence / basis: Matrix cleanup; Rationale: Avoid backend confusion; Implications: Renderer matrix updated; Related workstream: WORKSTREAM-11; Confidence: High; Label: FACT

19. Task Register

Row 1 ID: TASK-01; Task: Verify current repo state; Priority: P0; Urgency: U0; Owner: Next chat; Dependencies: None; Inputs needed: Repo access; Expected output: Accurate next task; Next step: Inspect queue/audits/commits; Related workstream: All; Label: FACT

Row 2 ID: TASK-02; Task: QUEUE-RECONCILE-01; Priority: P0 if queue stale; Urgency: U0; Owner: Codex/AIDE; Dependencies: PACKAGE-MOUNT done; Inputs needed: Queue and package audit; Expected output: Queue points replay; Next step: Run/generate if stale; Related workstream: WORKSTREAM-03; Label: FACT

Row 3 ID: TASK-03; Task: REPLAY-PROOF-SLICE-01; Priority: P0; Urgency: U0; Owner: Codex/AIDE; Dependencies: Package mount; Inputs needed: Prompt generated; Expected output: Command replay/proof fixture; Next step: Run if missing; Related workstream: WORKSTREAM-08; Label: FACT

Row 4 ID: TASK-04; Task: BAREBONES-CLIENT-SHELL-01; Priority: P0; Urgency: U1; Owner: Codex/AIDE; Dependencies: Replay proof preferred; Inputs needed: Prompt generated; Expected output: Client no-content floor; Next step: Run if missing; Related workstream: WORKSTREAM-09; Label: FACT

Row 5 ID: TASK-05; Task: PRODUCT-SPINE-REVIEW-01; Priority: P0; Urgency: U1; Owner: Codex/AIDE; Dependencies: Replay + barebones; Inputs needed: New prompt needed; Expected output: Decide limited parallel readiness; Next step: Generate after prior tasks; Related workstream: WORKSTREAM-03; Label: FACT

Row 6 ID: TASK-06; Task: AIDE-WORKFLOW-LAW-01; Priority: P1; Urgency: U1; Owner: Codex/AIDE; Dependencies: Product spine review; Inputs needed: Plan; Expected output: Branch/lifecycle/blocker law; Next step: Generate after review; Related workstream: WORKSTREAM-04; Label: FACT

Row 7 ID: TASK-07; Task: AIDE-WORKUNIT-SCHEMA-01; Priority: P1; Urgency: U1; Owner: Codex/AIDE; Dependencies: AIDE workflow law; Inputs needed: Schemas; Expected output: WorkUnit/task attempt schemas; Next step: Later;

Related workstream: WORKSTREAM-04; Label: FACT

Row 8 ID: TASK-08; Task: AIDE-DEV-MAIN-POLICY-01; Priority: P1; Urgency: U1; Owner: Codex/AIDE; Dependencies: AIDE workflow law; Inputs needed: Branch doctrine; Expected output: dev/main/checkpoint policy; Next step: Later; Related workstream: WORKSTREAM-04; Label: FACT

Row 9 ID: TASK-09; Task: PRESENTATION-CONTRACT-01; Priority: P1; Urgency: U1; Owner: Codex/AIDE; Dependencies: Command/result/view proof; Inputs needed: Existing view/action docs; Expected output: Presentation/action contracts; Next step: Parallel candidate; Related workstream: WORKSTREAM-06; Label: FACT

Row 10 ID: TASK-10; Task: POINTER-WIDTH-SERIALIZATION-AUDIT-01; Priority: P1; Urgency: U2; Owner: Codex/AIDE; Dependencies: Arch policy; Inputs needed: Code/audit access; Expected output: Serialization audit; Next step: Parallel candidate; Related workstream: WORKSTREAM-01; Label: FACT

Row 11 ID: TASK-11; Task: PROJECTION-CONFORMANCE-01; Priority: P1; Urgency: U2; Owner: Codex/AIDE; Dependencies: Presentation

contract; Inputs needed: Fixtures; Expected output: Projection conformance tests; Next step: Later; Related workstream: WORKSTREAM-06; Label: FACT

Row 12 ID: TASK-12; Task: ACCESSIBILITY-CONTRACT-01; Priority: P2; Urgency: U2; Owner: Codex/AIDE; Dependencies: Presentation contract; Inputs needed: View/action model; Expected output: Accessibility law; Next step: Later; Related workstream: WORKSTREAM-06; Label: FACT

Row 13 ID: TASK-13; Task: TEXT-LOCALIZATION-CONTRACT-01; Priority: P2; Urgency: U2; Owner: Codex/AIDE; Dependencies: Presentation contract; Inputs needed: Text/UI docs; Expected output: Text catalog law; Next step: Later; Related workstream: WORKSTREAM-06; Label: FACT

Row 14 ID: TASK-14; Task: WORKBENCH-SHELL-01; Priority: P2; Urgency: U2; Owner: Codex/AIDE; Dependencies: Product/presentation spine; Inputs needed: Workbench descriptors; Expected output: Minimal shell; Next step: Later; Related workstream: WORKSTREAM-05; Label: FACT

Row 15 ID: TASK-15; Task: DOMAIN-CONSTITUTION-WAVE-01; Priority: P2;

Urgency: U3; Owner: Codex/AIDE; Dependencies: Product/AIDE/presentation layers; Inputs needed: Doctrine matrix; Expected output: Domain constitutions; Next step: Much later; Related workstream: WORKSTREAM-10; Label: FACT

20. Constraint Register

- Row 1** ID: CONSTRAINT-01; Constraint: Broad feature work blocked; Type: Technical/process; Hard/soft: Hard; Source / basis: Queue/foundation state; Practical implication: Only narrow slices; Violation risk: Refactor spiral; Confidence: High; Label: FACT
- Row 2** ID: CONSTRAINT-02; Constraint: Full CTest not green; Type: Testing; Hard/soft: Hard fact; Source / basis: Repo status; Practical implication: Do not claim release-ready; Violation risk: Release risk; Confidence: High; Label: FACT
- Row 3** ID: CONSTRAINT-03; Constraint: Runtime package/provider/module not implemented; Type: Technical; Hard/soft: Hard fact; Source / basis: Audits; Practical implication: Avoid runtime claims; Violation risk: False support; Confidence: High; Label: FACT
- Row 4** ID: CONSTRAINT-04; Constraint: Workbench shell not implemented; Type: Technical; Hard/soft:

Hard fact; Source / basis: Audits; Practical implication: Use projections/fixtures; Violation risk: False UI claims; Confidence: High; Label: FACT

Row 5 ID: CONSTRAINT-05; Constraint: Public ABI C-compatible; Type: Architecture; Hard/soft: Hard; Source / basis: Doctrine; Practical implication: No C++ ABI leakage; Violation risk: Portability break; Confidence: High; Label: FACT

Row 6 ID: CONSTRAINT-06; Constraint: Paths are not identity; Type: Architecture; Hard/soft: Hard; Source / basis: Doctrine; Practical implication: Use stable IDs; Violation risk: Incompatibility; Confidence: High; Label: FACT

Row 7 ID: CONSTRAINT-07; Constraint: Runtime must not depend on tools; Type: Dependency; Hard/soft: Hard; Source / basis: Ownership law; Practical implication: Tools repo-only; Violation risk: Product fragility; Confidence: High; Label: FACT

Row 8 ID: CONSTRAINT-08; Constraint: Prompts in one code block; Type: UX/process; Hard/soft: Hard preference; Source / basis: User explicit; Practical implication: Easy copy/paste; Violation risk: User friction; Confidence: High; Label: FACT

- Row 9** ID: CONSTRAINT-09; Constraint: No destructive git cleanup in prompts; Type: Safety; Hard/soft: Hard; Source / basis: Prompt design; Practical implication: Protect concurrent work; Violation risk: Data loss; Confidence: High; Label: FACT
- Row 10** ID: CONSTRAINT-10; Constraint: Evidence-backed promotion; Type: Process; Hard/soft: Hard; Source / basis: AIDE doctrine; Practical implication: main protected; Violation risk: False completion; Confidence: High; Label: FACT
- Row 11** ID: CONSTRAINT-11; Constraint: Dev non-blocking but structured; Type: Process; Hard/soft: Soft-to-hard future; Source / basis: User/Codex; Practical implication: Blockers become tasks; Violation risk: Chaotic dev; Confidence: High; Label: FACT
- Row 12** ID: CONSTRAINT-12; Constraint: Do not invent facts; Type: Epistemic; Hard/soft: Hard; Source / basis: User explicit; Practical implication: Label uncertainty; Violation risk: Trust loss; Confidence: High; Label: FACT

21. User Preference Register

- Row 1** ID: PREF-01; Preference: Direct, audit-ready answers; Area: Communication; Explicit/inferred/
- 1265

uncertain: Explicit; Strength: Strong; Practical implication: Use structured evidence; Risk if wrong: Frustration; Label: FACT

Row 2 ID: PREF-02; Preference: Distinguish facts/inferences/uncertainty; Area: Epistemic; Explicit/inferred/uncertain: Explicit; Strength: Strong; Practical implication: Label claims; Risk if wrong: Loss of trust; Label: FACT

Row 3 ID: PREF-03; Preference: Preserve detail, do not over-compress; Area: Output; Explicit/inferred/uncertain: Explicit; Strength: Strong; Practical implication: Long reports acceptable; Risk if wrong: Missing context; Label: FACT

Row 4 ID: PREF-04; Preference: Generate prompts in one code block; Area: Prompting; Explicit/inferred/uncertain: Explicit; Strength: Strong; Practical implication: Use single fenced block; Risk if wrong: Copy failure; Label: FACT

Row 5 ID: PREF-05; Preference: Challenge bad framing; Area: Reasoning; Explicit/inferred/uncertain: Explicit/inferred; Strength: Strong; Practical implication: Correct user when needed; Risk if wrong: Bad plans; Label: FACT

Row 6 ID: PREF-06; Preference: No re-explaining settled issues; Area: Continuity; Explicit/inferred/

uncertain: Explicit; Strength: Strong; Practical implication: Use handoff; Risk if wrong: Wasted time; Label: FACT

Row 7 ID: PREF-07; Preference: Source/repo-grounded planning; Area: Evidence; Explicit/inferred/uncertain: Explicit/inferred; Strength: Strong; Practical implication: Verify repo; Risk if wrong: Stale plan; Label: FACT

Row 8 ID: PREF-08; Preference: Proceed unless clarification materially needed; Area: Workflow; Explicit/inferred/uncertain: Explicit/inferred; Strength: Medium-high; Practical implication: Avoid blocking questions; Risk if wrong: Slowdown; Label: FACT

Row 9 ID: PREF-09; Preference: Preserve rejected options; Area: Continuity; Explicit/inferred/uncertain: Explicit; Strength: Strong; Practical implication: Include rejected register; Risk if wrong: Repeated work; Label: FACT

Row 10 ID: PREF-10; Preference: Prefer modular/reusable design; Area: Architecture; Explicit/inferred/uncertain: Explicit; Strength: Strong; Practical implication: Enforce contracts/providers; Risk if wrong: Fragility; Label: FACT

22. Open Questions Register

Row 1 ID: QUESTION-01; Question / issue: Has REPLAY-PROOF-SLICE-01 executed?; Why it matters: Determines next prompt; Known information: Prompt generated; Unknown information: Live status; Resolution path: Check commits/audit; Priority: P0; Related workstream: WORKSTREAM-08; Label: UNCERTAIN

Row 2 ID: QUESTION-02; Question / issue: Has BAREBONES-CLIENT-SHELL-01 executed?; Why it matters: Determines Product Spine Review; Known information: Prompt generated; Unknown information: Live status; Resolution path: Check commits/audit; Priority: P0; Related workstream: WORKSTREAM-09; Label: UNCERTAIN

Row 3 ID: QUESTION-03; Question / issue: Is queue reconciled after package mount?; Why it matters: Avoid stale prompts; Known information: Queue may be stale; Unknown information: Live status; Resolution path: Inspect .aide/queue/current.toml; Priority: P0; Related workstream: WORKSTREAM-03; Label: UNCERTAIN

Row 4 ID: QUESTION-04; Question / issue: Is C17/C++17 fully implemented in build?; Why it matters: Avoid docs/build mismatch; Known information: Doctrine accepted; Unknown

information: Live CMake status; Resolution path: Inspect build files; Priority: P1; Related workstream: WORKSTREAM-01; Label: VERIFY

Row 5 ID: QUESTION-05; Question / issue: Does origin/dev exist?; Why it matters: Parallel dev; Known information: Desired; Unknown information: Repo status; Resolution path: `git branch -r`; Priority: P1; Related workstream: WORKSTREAM-04; Label: UNCERTAIN

Row 6 ID: QUESTION-06; Question / issue: What are current warning counts?; Why it matters: Promotion gates; Known information: Known categories; Unknown information: Exact live counts; Resolution path: Run validators; Priority: P1; Related workstream: WORKSTREAM-02; Label: UNCERTAIN

Row 7 ID: QUESTION-07; Question / issue: When to burn down full CTest?; Why it matters: Release readiness; Known information: T4 debt; Unknown information: Schedule; Resolution path: Plan full-gate phase; Priority: P2; Related workstream: WORKSTREAM-02; Label: OPEN

Row 8 ID: QUESTION-08; Question / issue: Which product should follow client shell?; Why it matters: Roadmap; Known information: Launcher/setup/server later; Unknown information: Order;

Resolution path: Product Spine Review; Priority: P2; Related workstream: WORKSTREAM-03; Label: OPEN

23. Artifact Ledger

Row 1 ID: ARTIFACT-01; Artifact / file / prompt / output: Pasted text.txt; Type: Uploaded file; Purpose: Preservation task instruction; Status: Loaded; Origin: User upload; Carry forward?: Yes; Notes: Cite if referencing uploaded instruction; Label: FACT

Row 2 ID: ARTIFACT-02; Artifact / file / prompt / output: FOUNDATION_LOCK.md; Type: Repo doc; Purpose: Foundation status; Status: Existing; Origin: Repo; Carry forward?: Yes; Notes: Verify live; Label: FACT

Row 3 ID: ARTIFACT-03; Artifact / file / prompt / output: .aide/queue/current.toml; Type: Repo status; Purpose: Current queue; Status: Existing; Origin: Repo; Carry forward?: Yes; Notes: Must inspect first; Label: FACT

Row 4 ID: ARTIFACT-04; Artifact / file / prompt / output: PACKAGE_MOUNT_SLICE_01.md; Type: Audit; Purpose: Package mount slice result; Status: Existing; Origin: Repo; Carry forward?:

Yes; Notes: Said PASS_WITH_WARNINGS;
Label: FACT

Row 5 ID: ARTIFACT-05; Artifact / file / prompt /
output: QUEUE-RECONCILE-01 prompt; Type:
Prompt; Purpose: Update queue after package
mount; Status: Generated; Origin: This chat;
Carry forward?: Yes; Notes: May need to run;
Label: FACT

Row 6 ID: ARTIFACT-06; Artifact / file / prompt /
output: REPLAY-PROOF-SLICE-01 prompt; Type:
Prompt; Purpose: Command-level replay proof;
Status: Generated; Origin: This chat; Carry
forward?: Yes; Notes: Next if missing; Label:
FACT

Row 7 ID: ARTIFACT-07; Artifact / file / prompt /
output: BAREBONES-CLIENT-SHELL-01 prompt;
Type: Prompt; Purpose: Minimal client floor;
Status: Generated; Origin: This chat; Carry
forward?: Yes; Notes: After replay; Label: FACT

Row 8 ID: ARTIFACT-08; Artifact / file / prompt /
output: PRODUCT-SPINE-REVIEW-01; Type: Future
prompt; Purpose: Review product spine; Status:
Not yet generated; Origin: Planned; Carry
forward?: Yes; Notes: Generate after replay +
barebones; Label: FACT

Row 9 ID: ARTIFACT-09; Artifact / file / prompt / output: Workbench validation slice audit; Type: Audit; Purpose: First Workbench validation proof; Status: Existing; Origin: Repo; Carry forward?: Yes; Notes: No full shell; Label: FACT

Row 10 ID: ARTIFACT-10; Artifact / file / prompt / output: Command-result-view slice audit; Type: Audit; Purpose: View/projection proof; Status: Existing/expected; Origin: Repo; Carry forward?: Yes; Notes: Verify live; Label: FACT

Row 11 ID: ARTIFACT-11; Artifact / file / prompt / output: Handoff package files; Type: Generated package; Purpose: Preserve this chat; Status: Created by this response; Origin: This task; Carry forward?: Yes; Notes: See file links; Label: FACT

24. Rejected / Superseded Options Register

Row 1 ID: REJECTED-01; Option: Brute-force global simulation; Status: Rejected; Reason: Too costly; Final or tentative?: Final for global approach; Reconsider conditions: Local refinement remains; Related workstream: WORKSTREAM-10; Label: FACT

Row 2 ID: REJECTED-02; Option: Hardcode all objects/machines; Status: Rejected; Reason: Does not scale; Final or tentative?: Final as core model;

Reconsider conditions: Templates okay; Related workstream: WORKSTREAM-10; Label: FACT

Row 3 ID: REJECTED-03; Option: Workbench monolith; Status: Rejected; Reason: Becomes authority/blob; Final or tentative?: Final; Reconsider conditions: Shell+modules accepted; Related workstream: WORKSTREAM-05; Label: FACT

Row 4 ID: REJECTED-04; Option: Native-widget-first editor; Status: Rejected; Reason: Not reusable with client UI; Final or tentative?: Final for first wave; Reconsider conditions: Native projection later; Related workstream: WORKSTREAM-06; Label: FACT

Row 5 ID: REJECTED-05; Option: Separate UI systems; Status: Rejected; Reason: Causes drift; Final or tentative?: Final; Reconsider conditions: Projection model accepted; Related workstream: WORKSTREAM-06; Label: FACT

Row 6 ID: REJECTED-06; Option: Agents directly mutate shared dev; Status: Rejected; Reason: Concurrency chaos; Final or tentative?: Final; Reconsider conditions: Task branches accepted; Related workstream: WORKSTREAM-04; Label: FACT

Row 7 ID: REJECTED-07; Option: Build full AIDE OS before product spine; Status: Deprioritised; Reason: Would stall progress; Final or tentative?: Tentative; Reconsider conditions: Minimum AIDE layer soon; Related workstream: WORKSTREAM-04; Label: FACT

Row 8 ID: REJECTED-08; Option: Treat full CTest as current blocker; Status: Deprioritised; Reason: Too broad now; Final or tentative?: Tentative; Reconsider conditions: Release/full-gate; Related workstream: WORKSTREAM-02; Label: FACT

Row 9 ID: REJECTED-09; Option: OpenGL 1.1/D3D9 first wave; Status: Rejected; Reason: Backport only; Final or tentative?: Final first-wave; Reconsider conditions: Research lanes later; Related workstream: WORKSTREAM-11; Label: FACT

Row 10 ID: REJECTED-10; Option: C89/C++98 active mainline; Status: Superseded; Reason: Modern baseline accepted; Final or tentative?: Tentative until build verified; Reconsider conditions: Retro lanes; Related workstream: WORKSTREAM-01; Label: FACT+VERIFY

25. Risk Register

Row 1	ID: RISK-01; Risk / failure mode: Stale AIDE queue; Consequence: Wrong prompts run; Likelihood: Medium; Severity: High; Mitigation: Verify queue first; Related workstream: WORKSTREAM-04; Label: FACT
Row 2	ID: RISK-02; Risk / failure mode: Docs outrun code; Consequence: False support claims; Likelihood: High; Severity: High; Mitigation: Capability Reality Ledger; Related workstream: WORKSTREAM-04; Label: FACT
Row 3	ID: RISK-03; Risk / failure mode: Broad feature work starts early; Consequence: Refactor spiral; Likelihood: Medium; Severity: High; Mitigation: Preserve blockers; Related workstream: All; Label: FACT
Row 4	ID: RISK-04; Risk / failure mode: Agents mutate dev directly; Consequence: Merge chaos; Likelihood: Medium; Severity: High; Mitigation: Task branches/worktrees; Related workstream: WORKSTREAM-04; Label: FACT
Row 5	ID: RISK-05; Risk / failure mode: Full CTest ignored forever; Consequence: Release weakness; Likelihood: Medium; Severity: Medium-high;

Mitigation: FULL-GATE-DEBT-01; Related
workstream: WORKSTREAM-02; Label: FACT

Row 6 ID: RISK-06; Risk / failure mode: Workbench
becomes authority; Consequence: Architecture
drift; Likelihood: Medium; Severity: High;
Mitigation: Command/service boundaries; Related
workstream: WORKSTREAM-05; Label: FACT

Row 7 ID: RISK-07; Risk / failure mode: Runtime
depends on tools; Consequence: Product fragility;
Likelihood: Medium; Severity: High; Mitigation:
Dependency-direction law; Related workstream:
WORKSTREAM-02; Label: FACT

Row 8 ID: RISK-08; Risk / failure mode: UI
projections diverge; Consequence: Duplicated
behavior; Likelihood: Medium; Severity: High;
Mitigation: Presentation contract; Related
workstream: WORKSTREAM-06; Label: FACT

Row 9 ID: RISK-09; Risk / failure mode: C17/C++17
docs/build mismatch; Consequence: Build
confusion; Likelihood: Medium; Severity: Medium;
Mitigation: Verify CMake; Related workstream:
WORKSTREAM-01; Label: VERIFY

Row 10 ID: RISK-10; Risk / failure mode: New chat
restarts doctrine; Consequence: Wasted time;
Likelihood: Medium; Severity: Medium;

Mitigation: Use handoff; Related workstream: All;
Label: FACT

26. Verification Queue

- Row 1** ID: VERIFY-01; Item requiring verification:
Current `.aide/queue/current.toml`; Why
verification is needed: Determines next task;
Suggested source/type: Repo file; Priority: P0;
Related workstream: WORKSTREAM-03; Label:
VERIFY
- Row 2** ID: VERIFY-02; Item requiring verification:
Recent commits; Why verification is needed: Check
which prompts landed; Suggested source/type: `git
log`; Priority: P0; Related workstream: All; Label:
VERIFY
- Row 3** ID: VERIFY-03; Item requiring verification:
`REPLAY_PROOF_SLICE_01.md`; Why verification is
needed: Determine replay status; Suggested
source/type: Repo audit; Priority: P0; Related
workstream: WORKSTREAM-08; Label: VERIFY
- Row 4** ID: VERIFY-04; Item requiring verification:
`BAREBONES_CLIENT_SHELL_01.md`; Why verification
is needed: Determine barebones status; Suggested
source/type: Repo audit; Priority: P0; Related
workstream: WORKSTREAM-09; Label: VERIFY

Row 5 ID: VERIFY-05; Item requiring verification: CMake language standard; Why verification is needed: Confirm C17/C++17; Suggested source/type: CMake files; Priority: P1; Related workstream: WORKSTREAM-01; Label: VERIFY

Row 6 ID: VERIFY-06; Item requiring verification: origin/dev existence; Why verification is needed: Parallel plan; Suggested source/type: Git branches; Priority: P1; Related workstream: WORKSTREAM-04; Label: VERIFY

Row 7 ID: VERIFY-07; Item requiring verification: Warning counts; Why verification is needed: Avoid false clean claim; Suggested source/type: Validator output; Priority: P1; Related workstream: WORKSTREAM-02; Label: VERIFY

Row 8 ID: VERIFY-08; Item requiring verification: Full CTest status; Why verification is needed: Release readiness; Suggested source/type: CI/local test; Priority: P2; Related workstream: WORKSTREAM-02; Label: VERIFY

Row 9 ID: VERIFY-09; Item requiring verification: Product app descriptor state; Why verification is needed: Barebones client; Suggested source/type: contracts/apps; Priority: P1; Related workstream: WORKSTREAM-09; Label: VERIFY

Row 10 ID: VERIFY-10; Item requiring verification:
Package runtime absence; Why verification is
needed: Avoid false claims; Suggested source/type:
repo/audits/code; Priority: P1; Related
workstream: WORKSTREAM-07; Label: VERIFY

27. Chronological Timeline Register

Row 1 Sequence: 1; Event / topic: Worldgen handoff;
What changed or was decided: Continue after
architecture freeze; Why it mattered: Preserves
invariants; Current relevance: Background
doctrine; Confidence: High

Row 2 Sequence: 2; Event / topic: Aged worlds
question; What changed or was decided: Lazy
deterministic history preferred; Why it mattered:
Avoid brute force; Current relevance: Domain
future; Confidence: High

Row 3 Sequence: 3; Event / topic: Epistemics/
diegetics; What changed or was decided: Became
optimization doctrine; Why it mattered: Sparse
refinement; Current relevance: Platform doctrine;
Confidence: High

Row 4 Sequence: 4; Event / topic: Player construction;
What changed or was decided: Deep primitives
identified; Why it mattered: Arbitrary invention;

Current relevance: Future domain work;
Confidence: High

Row 5 Sequence: 5; Event / topic: Civilization macro/
micro; What changed or was decided: Collapse/
expand model; Why it mattered: Scales societies;
Current relevance: Future domain work;
Confidence: High

Row 6 Sequence: 6; Event / topic: Universal reality
framework; What changed or was decided: Domain
contracts proposed; Why it mattered: Avoid
feature creep; Current relevance: Superseded by
repo-grounded tasks; Confidence: Medium

Row 7 Sequence: 7; Event / topic: Repo CONVERGE;
What changed or was decided: Layout and
authority cleanup; Why it mattered: Governed
repo; Current relevance: Foundation context;
Confidence: High

Row 8 Sequence: 8; Event / topic: AIDE introduced;
What changed or was decided: Control plane
installed; Why it mattered: Refactor management;
Current relevance: Still active; Confidence: High

Row 9 Sequence: 9; Event / topic: Foundation Lock;
What changed or was decided: Scaffolds/gates
built; Why it mattered: Enables narrow slices;
Current relevance: Current base; Confidence: High

- Row 10** Sequence: 10; Event / topic: Language baseline; What changed or was decided: C17/C++17 doctrine; Why it mattered: Modernize; Current relevance: Verify build; Confidence: Medium
- Row 11** Sequence: 11; Event / topic: Workbench direction; What changed or was decided: Production environment; Why it mattered: Replaces UI editor; Current relevance: Active future; Confidence: High
- Row 12** Sequence: 12; Event / topic: Presentation model; What changed or was decided: Projection architecture; Why it mattered: Avoid UI drift; Current relevance: Future prompt; Confidence: High
- Row 13** Sequence: 13; Event / topic: Prompt phase; What changed or was decided: Generated seven prompts; Why it mattered: Built contract layers; Current relevance: Mostly complete; Confidence: High
- Row 14** Sequence: 14; Event / topic: Package mount; What changed or was decided: Fixture proof landed; Why it mattered: Product spine progress; Current relevance: Done; Confidence: High
- Row 15** Sequence: 15; Event / topic: AIDE dev/main; What changed or was decided: Non-blocking dev/

evidence main; Why it mattered: Parallel future;
Current relevance: Next phase; Confidence: High

Row 16 Sequence: 16; Event / topic: Preservation request; What changed or was decided: Full handoff required; Why it mattered: Retire chat; Current relevance: This document; Confidence: High

28. Spec Book Contribution Register

Row 1 Spec-book area: Project Identity; Contribution from this chat: Domino/Dominium/Workbench/AIDE roles; Source IDs: DECISION-01-05; Should become requirement/context/open issue?: Requirement; Confidence: High; Notes: Core chapter

Row 2 Spec-book area: Repo Architecture; Contribution from this chat: Closed root model; Source IDs: DECISION-07; Should become requirement/context/open issue?: Requirement; Confidence: High; Notes: Check live docs

Row 3 Spec-book area: ABI/Language; Contribution from this chat: C17/C++17 + C ABI; Source IDs: DECISION-08; Should become requirement/context/open issue?: Requirement with verification; Confidence: Medium; Notes: Verify CMake

- Row 4** Spec-book area: AIDE Workflow; Contribution from this chat: dev/main/task/checkpoint doctrine; Source IDs: DECISION-04-06; Should become requirement/context/open issue?: Requirement; Confidence: High; Notes: Needs files
- Row 5** Spec-book area: Workbench; Contribution from this chat: Production environment, not authority; Source IDs: DECISION-03; Should become requirement/context/open issue?: Requirement; Confidence: High; Notes: Future Workbench chapter
- Row 6** Spec-book area: Presentation; Contribution from this chat: Projection architecture; Source IDs: DECISION-09; Should become requirement/context/open issue?: Requirement; Confidence: High; Notes: UI chapter
- Row 7** Spec-book area: Package Composition; Contribution from this chat: Fixture mount, composition law; Source IDs: WORKSTREAM-07; Should become requirement/context/open issue?: Requirement/context; Confidence: High; Notes: Runtime later
- Row 8** Spec-book area: Replay Proof; Contribution from this chat: Next product-spine task; Source IDs: TASK-03; Should become requirement/

context/open issue?: Open issue; Confidence: Medium; Notes: Execution unknown

Row 9 Spec-book area: Worldgen Doctrine; Contribution from this chat: Sparse/lazy deterministic truth; Source IDs: WORKSTREAM-10; Should become requirement/context/open issue?: Requirement/context; Confidence: High; Notes: Domain chapters

Row 10 Spec-book area: Deep Primitives; Contribution from this chat: Creation model; Source IDs: WORKSTREAM-10; Should become requirement/context/open issue?: Requirement/context; Confidence: High; Notes: Domain chapters

Row 11 Spec-book area: Testing Gates; Contribution from this chat: Fast strict vs full gate; Source IDs: DECISION-10; Should become requirement/context/open issue?: Requirement; Confidence: High; Notes: Test strategy

Row 12 Spec-book area: Renderer Baseline; Contribution from this chat: Null/software/OpenGL/D3D11; Source IDs: DECISION-11; Should become requirement/context/open issue?: Requirement; Confidence: High; Notes: Renderer chapter

29. Context Transfer Packet for a Future Chat

29.1 Ultra-Condensed Bootstrap Brief This chat developed and preserved the current Dominion architecture and task roadmap. Dominion is no longer being treated as a normal game project. The converged identity is: Domino is the reusable deterministic substrate, Dominion is the game/product family built on it, Workbench is the production/validation/editing/evidence environment, AIDE is the repo/control-plane harness, Codex is a bounded patch executor, contracts are law, and diagnostics/tests/replay/evidence are proof.

The core doctrines are: path is not identity; implementation is not contract; UI is not authority; generated output is not source truth; development should be non-blocking; promotion should be evidence-blocked. The project should be modular, deterministic, portable, extensible, replaceable, and evidence-backed.

Current expected state from this chat: Foundation Lock is `PASS_WITH_WARNINGS`, fast strict passes, dependency-direction strict has 0 violations with known warnings, full CTest remains T4/full-gate debt, and broad feature work remains blocked. `PACKAGE-MOUNT-SLICE-01` landed as `PASS_WITH_WARNINGS` and proved only a fixture-level package/profile mount decision, not runtime package mounting. The next intended product-spine task is `REPLAY-PROOF-SLICE-01`, followed by `BAREBONES-`

CLIENT-SHELL-01, then PRODUCT-SPINE-REVIEW-01. Verify live repo state first, especially `.aide/queue/current.toml`, recent commits, and relevant audits.

Workbench should eventually be a visual and agentic production environment, but it is not authority. It edits documents and dispatches registered commands; runtime executes; contracts constrain; content packages; apps consume; tests prove. CLI, text/TUI, rendered GUI, native GUI, and headless must be projections of one command/result/refusal/document/view/action spine, not separate UI systems.

Future AIDE workflow: do not let agents all mutate shared `dev`. Use one task branch/worktree per prompt. AIDE integrates safe work into `origin/dev`; checkpoint branches prove integrated waves; `origin/main` receives only evidence-backed promotion. Limited parallel dev can start after replay proof, barebones client, and product-spine review. Large parallel dev should wait for AIDE-WORKFLOW-LAW-01, AIDE-WORKUNIT-SCHEMA-01, and AIDE-DEV-MAIN-POLICY-01.

29.2 Source Hierarchy

1. Direct user statements in this chat.
2. Explicit decisions accepted by the user.
3. Current task register.

4. Constraint register.
5. Artifact ledger.
6. Inferences.
7. Assistant suggestions not accepted by the user.
8. General model knowledge.

29.3 Operating Rules for Future Assistants

- Preserve FACT / INFERENCE / UNCERTAIN / PROJECT-CONTEXT labels.
- Verify live repo state before acting.
- Do not re-ask answered questions.
- Ask clarifying questions only when materially necessary.
- Do not invent missing details.
- Do not treat tentative items as final.
- Do not repeat rejected options.
- Preserve artifacts and warnings.
- Use structured outputs.
- Put future Codex prompts in one code block.
- Do not start broad feature work prematurely.

29.4 Active Workstreams Active: product spine, AIDE workflow, Workbench platform, presentation architecture, package/replay/barebones product proof, repo governance.

Deferred: gameplay, renderer implementation, native GUI, runtime package/provider/module loaders, domain constitutions.

29.5 Current Priorities

1. Verify current repo state.
2. Run missing queue/replay/barebones prompts.
3. Run Product Spine Review.
4. Add minimum AIDE workflow law.
5. Begin limited parallel task-branch development.
6. Add presentation contracts.
7. Continue Workbench/runtime/product slices.

29.6 Current Open Questions

- Has replay proof run?
- Has barebones client shell run?
- Is queue reconciled?

- Does live build use C17/C++17?
- Does origin/dev exist?
- What warnings remain?

29.7 Recommended First Action Inspect the live repo. Determine whether `REPLAY-PROOF-SLICE-01` and `BAREBONES-CLIENT-SHELL-01` have run. Generate the next missing prompt accordingly.

30. Spec Sheet

```
spec_sheet:

metadata:

chat_label: "Dominium Architecture, Workbench, AIDE, and
Product-Spine Planning"

date_anchor: "2026-05-27 Australia/Melbourne"

source_scope: "This chat only unless labelled PROJECT-CONTEXT"

apparent_coverage: "Partial transcript with high contextual
reconstruction"

confidence_1_to_5: 4

staleness_risk: "Medium"

safe_for_aggregation: "Yes, with caveats"

main_limitations:

- "Visible transcript includes skipped-message markers."

- "Live repo may advance after report creation."

- "Execution status of latest generated prompts must be verified.
"

summary:

one_sentence: "This chat converged Dominium into a
deterministic, contract-governed simulation platform with
Workbench as production environment and AIDE as
```

short_brief: "The chat established architecture, repo gates, Workbench direction, AIDE dev/main workflow, product-spine task sequencing, and future domain r

main_topics:

- "Deterministic sparse worldgen"
- "Deep primitives and player creation"
- "Repo Foundation Lock"
- "Workbench production environment"
- "AIDE task operating system"
- "Package/replay/barebones product spine"
- "Presentation/projection architecture"

main_outputs:

- "Prompt series for post-foundation slices"
- "AIDE dev/main workflow doctrine"
- "Product-spine roadmap"
- "This preservation report"

highest_priority_carry_forward:

- "Verify repo state."
- "Run missing REPLAY-PROOF-SLICE-01 and BAREBONES-CLIENT-SHELL-01."
- "Generate PRODUCT-SPINE-REVIEW-01."
- "Add AIDE-WORKFLOW-LAW-01 before large parallel dev."

source_rules:

labels_used:

- "FACT"
- "INFERENCE"
- "UNCERTAIN / UNVERIFIED"
- "PROJECT-CONTEXT"

conflict_rules:

- "Live repo evidence overrides this handoff."
- "User-accepted decisions override assistant brainstorming."
- "Prompt-generated plans are not implementation proof."

staleness_rules:

- "Verify current queue and commits before acting."
- "Treat repo statuses as potentially stale if not checked live."

user_preferences:

explicit:

- "Detailed, source-grounded, audit-ready answers."
- "Do not invent facts."
- "Prompts in one code block."
- "Preserve uncertainty and rejected options."
- "Do not over-compress."

inferred:

- "Prefers architecture and proof before broad implementation."
- "Values correction over agreement."
- "Wants future assistants to avoid repeated doctrine discussion."

uncertain_or_not_established:

- "Exact desired cadence once parallel dev begins."

workstreams:

- id: "WORKSTREAM-03"

name: "Product Spine"

label: "FACT"

objective: "Build package mount, replay proof, and barebones product floor."

current_state: "Package mount complete; replay and barebones execution uncertain."

desired_end_state: "Product spine review authorizes limited parallel development."

status: "Active"

priority: "P0"

background: "Current narrow product slices are permitted after Foundation Lock."

decisions_made:

- "Package mount is fixture-level only."

decisions_pending:

- "Whether product spine review passes."

tasks:

- "REPLAY-PROOF-SLICE-01"
- "BAREBONES-CLIENT-SHELL-01"
- "PRODUCT-SPINE-REVIEW-01"

constraints:

- "No broad package runtime."
- "No gameplay."

dependencies:

- "PACKAGE-MOUNT-SLICE-01"

timeline: "Immediate"

blockers:

- "Execution status unknown."

risks:

- "False runtime claims."

artifacts:

- "PACKAGE_MOUNT_SLICE_01.md"

success_criteria:

- "Replay proof and barebones client pass with warnings or better."

next_action: "Verify live repo and run missing prompt."

verification_needed:

- "Check REPLAY_PROOF_SLICE_01 audit."

confidence: "High"

decisions:

- id: "DECISION-05"

decision: "Development is non-blocking; promotion is evidence-blocked."

status: "accepted"

label: "FACT"

evidence_or_basis: "User and assistant convergence near end of chat."

rationale: "Allows parallel progress without corrupting main."

```
implications: "Use dev/task/checkpoint/main model."
related_workstreams:
- "WORKSTREAM-04"
uncertainty: "Exact automation not implemented yet."
tasks:
- id: "TASK-03"
task: "REPLAY-PROOF-SLICE-01"
priority: "P0"
urgency: "U0"
owner: "Codex/AIDE"
dependencies:
- "PACKAGE-MOUNT-SLICE-01"
inputs_needed:
- "Package mount command/result fixture"
expected_output: "Command-level replay/proof fixture and
validator."
next_step: "Run if audit missing."
related_workstreams:
- "WORKSTREAM-08"
label: "FACT"
confidence: "Medium"
constraints:
- id: "CONSTRAINT-01"
constraint: "Broad feature work remains blocked."
type: "process"
hard_or_soft: "hard"
source_or_basis: "Queue/Foundation status"
implication: "Only narrow slices."
violation_risk: "Refactor spiral"
label: "FACT"
confidence: "High"
open_questions:
```

```
- id: "QUESTION-01"
question: "Has REPLAY-PROOF-SLICE-01 executed?"
why_it_matters: "Determines next task."
known: "Prompt generated."
unknown: "Execution status."
resolution_path: "Check commits and audit file."
priority: "P0"
related_workstreams:
- "WORKSTREAM-08"
label: "UNCERTAIN"
rejected_or_superseded_options:
- id: "REJECTED-05"
option: "Workbench as monolithic editor."
status: "rejected"
reason: "Would become authority/blob."
final_or_tentative: "final for current architecture"
reconsider_conditions: "None for near term."
related_workstreams:
- "WORKSTREAM-05"
label: "FACT"
artifacts:
- id: "ARTIFACT-06"
name_or_description: "REPLAY-PROOF-SLICE-01 prompt"
type: "prompt"
purpose: "Generate narrow command-level replay proof."
status: "generated; execution uncertain"
origin: "This chat"
carry_forward: true
notes: "Run after queue reconciliation if missing."
label: "FACT"
risks:
- id: "RISK-02"
```

risk: "Docs outrun code."
consequence: "False support claims."
likelihood: "High"
severity: "High"
mitigation: "Capability Reality Ledger."
related_workstreams:
- "WORKSTREAM-04"
label: "FACT"
verification_queue:
- id: "VERIFY-01"
item: "Current AIDE queue"
why_verification_needed: "Determines next prompt."
suggested_source_type: ".aide/queue/current.toml"
priority: "P0"
related_workstreams:
- "WORKSTREAM-03"
label: "VERIFY"
spec_book_notes:
likely_sections:
- "Architecture doctrine"
- "Repo governance"
- "AIDE workflow"
- "Workbench"
- "Presentation system"
- "Domain primitives"
- "Testing gates"
unique_contributions:
- "Development non-blocking / promotion evidence-blocked doctrine."
- "Workbench progressive self-hosting."
- "Product-spine sequencing."
possible_duplicates_with_other_chats:

- "Worldgen architecture"
- "Constitutional Architecture"
- "Repo convergence"

conflicts_to_watch_for:

- "C89/C++98 vs C17/C++17 status."
- "Queue next-task state."

formal_requirements_candidates:

- "Path is not identity."
- "UI is not authority."
- "Task branches per prompt."

background_context_candidates:

- "Nixie/VFD/OEM+ UI inspirations."

needs_user_confirmation:

- "Whether origin/dev exists and should become official."

final_recommendations:

next_action_if_continuing_this_chat: "Verify queue and run the next missing product-spine prompt."

next_action_for_aggregator: "Merge this report under architecture/AIDE/Workbench/product-spine sections."

user_checks_required:

- "Check live repo state."
- "Confirm whether latest prompts have run."

31. Aggregator Packet

22.1.2 Aggregator Packet - Dominion Architecture, Workbench, AIDE, and Product-Spine Planning

Packet Metadata

- Chat label: Dominion Architecture, Workbench, AIDE, and Product-Spine Planning
- Date anchor: 2026-05-27 Australia/Melbourne
- Source scope: This chat only unless labelled PROJECT-CONTEXT
- Coverage: Partial raw transcript, high reconstruction from visible content
- Confidence: 4 / 5
- Staleness risk: Medium
- Merge priority: High
- Main limitations: latest repo prompt execution state must be verified

Ultra-Condensed Carry-Forward Capsule

This chat is a major architecture and planning checkpoint for Dominion. It converged the project identity around Domino as reusable deterministic

substrate, Dominion as the game/product family, Workbench as the production and validation environment, AIDE as repo/control-plane harness, and contracts/evidence as the core truth layer. The chat should feed into the master spec as a core source for repo governance, Workbench, AIDE, presentation architecture, package/replay/product spine, and future simulation-domain planning.

The strongest doctrines are: path is not identity; implementation is not contract; UI is not authority; generated output is not source truth; development is non-blocking; promotion is evidence-blocked. These should become formal requirements.

The repo state during this chat moved through Foundation Lock and multiple post-foundation slices. Foundation Lock is `PASS_WITH_WARNINGS`, not warning-free. Fast strict passes. Dependency-direction strict is green with known warnings. Full CTest remains T4/full-gate debt. Broad feature work remains blocked. `PACKAGE-MOUNT-SLICE-01` is complete as a fixture-level proof, not package runtime. The next tasks are queue reconciliation if stale, replay proof, barebones client shell, and product spine review.

Workbench should become the visual and agentic production environment, but not authority. It edits documents and dispatches commands. Runtime executes services; contracts constrain; content packages; apps

consume; tests prove. UI modes are projections: CLI, text/TUI, rendered GUI, native GUI, and headless share command/result/refusal/view/action semantics.

AIDE should evolve into a repo-native task operating system. The branch doctrine is task branches/worktrees per prompt, `origin/dev` for integration, checkpoint branches for proof, and `origin/main` for promoted truth. Ordinary blockers should become repair/prerequisite/resume tasks. Unsafe/destructive/authority blockers require human decision.

The chat also preserves future domain doctrine: deterministic sparse worldgen, lazy history, truth/perceived/render separation, epistemic/diegetic refinement, process-only mutation, deep primitives for player creation, macro/micro civilization capsules, and future domain constitutions. These are not immediate implementation tasks.

Top Carry-Forward Items

Row 1 Priority: P0; Item: Verify live repo state; Type: Task; Related ID: TASK-01; Why it matters: Prevent stale next-task errors; Label: FACT; Confidence: High

Row 2 Priority: P0; Item: Run missing replay proof; Type: Task; Related ID: TASK-03; Why it matters: Deterministic evidence spine; Label:

FACT; Confidence: Medium

Row 3 Priority: P0; Item: Run missing barebones client; Type: Task; Related ID: TASK-04; Why it matters: Product survival floor; Label: FACT; Confidence: Medium

Row 4 Priority: P0; Item: Product Spine Review; Type: Task; Related ID: TASK-05; Why it matters: Authorize limited parallel dev; Label: FACT; Confidence: High

Row 5 Priority: P1; Item: AIDE workflow law; Type: Task; Related ID: TASK-06; Why it matters: Enables safe parallel dev; Label: FACT; Confidence: High

Row 6 Priority: P1; Item: Presentation contract; Type: Task; Related ID: TASK-09; Why it matters: Prevents UI drift; Label: FACT; Confidence: High

Row 7 Priority: P1; Item: Capability Reality Ledger; Type: Task; Related ID: Future; Why it matters: Prevents docs/code drift; Label: FACT; Confidence: High

Compact Registers for Merge

See sections 17-28 above for full registers. Merge the following as high priority:

- DECISION-05: development non-blocking / promotion evidence-blocked.
- DECISION-09: UI modes are projections.
- TASK-03 to TASK-05: replay, barebones, product spine review.
- CONSTRAINT-01: broad feature work blocked.
- RISK-02: docs outrun code.
- VERIFY-01 to VERIFY-04: queue/replay/barebones/CMake verification.

Possible Cross-Chat Duplicates

- Constitutional Architecture
- Worldgen sparse materialization
- Repo convergence
- AIDE planning
- Workbench/UI editor discussions
- Domain constitution planning

Possible Cross-Chat Conflicts

- C89/C++98 vs C17/C++17.

- Earlier roadmap saying Workbench next vs later product-spine sequencing.
- Old root layout vs canonical root layout.
- Full CTest as blocker vs T4 debt.
- Workbench as product vs Workbench as authority.

Spec Book Integration Guidance

This chat should inform chapters on:

Project identity

Repo governance

AIDE workflow

Workbench

Presentation system

Package/composition/replay/product spine

Testing gates

Domain doctrine

Worldgen/deep primitives

Formalize:

Path is not identity.

Implementation is not contract.

UI is not authority.

Generated output is not source truth.

Development non-blocking.

Promotion evidence-blocked.

Do not merge as implemented features:

runtime package mount

runtime replay
provider runtime
module loader
Workbench shell
renderer
native GUI
gameplay
release publication

Aggregator Warnings

- Do not assume latest prompts executed.
- Do not flatten `PASS_WITH_WARNINGS` into `clean`.
- Do not treat doctrine recovery as canon supersession.
- Do not treat fixture proof as runtime implementation.
- Do not restart doctrine from scratch.

32. Adversarial Self-Audit

Issue	Severity	Correction needed	Correction applied?	Residual risk
Raw transcript incomplete due to skipped messages	High	Label access as partial	Yes	Some details may be missing
Live repo may advance	High	Add verification queue	Yes	New chat must verify
C17/C++17 implementation uncertain	Medium	Mark VERIFY	Yes	Still requires repo check
Latest prompts execution unknown	High	Mark replay/barebones uncertain	Yes	Must inspect audits
Domain/worldgen details compressed	Medium	Added doctrine summary	Partial	Full old brainstorm not exhaustive
Generated prompts not fully reproduced	Medium	Listed prompt names and roles	Partial	User can regenerate
Artifacts may be incomplete	Medium	Ledger includes known artifacts	Yes	Missing repo files possible
Assistant suggestions vs decisions	High	Decision status and confidence included	Yes	Some acceptance inferred from user continuation

33. Corrections Applied

After audit, the handoff was revised to:

- Mark transcript access as partial, not full.
- Add explicit verification items for live repo state.
- Add uncertainty around `REPLAY-PROOF-SLICE-01` and `BAREBONES-CLIENT-SHELL-01`.
- Clarify C17/C++17 as doctrine needing live build verification.
- Preserve `PASS_WITH_WARNINGS` warnings.
- Add branch model and AIDE workflow doctrine.
- Add rejected/superseded options register.
- Add product-spine next-task sequence.
- Include worldgen and deep primitive context.
- Add "what new chat should not assume."

34. Final Reliability Assessment

- Completeness rating: 4 / 5
- Reliability rating: 4 / 5
- Human-readability rating: 4 / 5

- Aggregation-readiness rating: 4 / 5
- Main remaining uncertainty sources:
- skipped transcript sections,
- live repo changes after this report,
- execution status of latest prompts,
- CMake language baseline,
- exact current warning counts.
- Manual review recommended before merging into a master spec: Yes.

35. File Export Package

Files were created for this preservation package. See final chat links.

36. File Index and Explanation

File	Purpose	What it contains	When to use it	Importance
Manifest	Package overview	File list, caveats, counts	First when downloading	High
Human-readable report	Main preservation report	Sections 0-16	Read for understanding	Highest
Context transfer packet	New-chat context	Section 29	Paste/brief future chat	Highest
Spec sheet	YAML-style structured data	Section 30	Aggregation/spec merge	High
Registers	Tables/IDs	Sections 17-28	Tracking decisions/tasks	High
Aggregator packet	Cross-chat merge	Section 31	Master spec aggregation	High
Reader brief	Shorter human brief	Top items and next steps	Quick review	Medium-high
Verification/audit	Self-audit and verification	Sections 32-34 + queue	Before relying on state	High
Bootstrap prompt	Future chat starter	Copy-paste prompt	Start new chat	Highest
In-chat reader	Guide to package	What to read, questions	Human navigation	Medium
ZIP	Bundle	All files	Archive/backup	Highest

37. Human Reader Guide

Read first:

1. Section A / Ultra-condensed brief.
2. Section 0 / Coverage and Reliability.
3. Section 1 / Orientation.
4. Section 16 / Compact summary.
5. Section 29 / Context Transfer Packet.
6. Task Register and Verification Queue if continuing work.

For aggregation:

- Use sections 17-31.
- Use Spec Sheet YAML.
- Use Aggregator Packet.

For verification:

- Use sections 11, 26, 32-34.
- Check live repo before acting.

For next actions:

- Verify queue/current.
- Determine whether replay and barebones tasks have run.
- Generate Product Spine Review when ready.

38. Best Follow-Up Questions

Understanding

1. "Explain the final project identity in 5 minutes."
2. "What does 'development non-blocking, promotion evidence-blocked' mean in practice?"

Decisions

1. "Which decisions are final and which need verification?"
2. "Why did we decide Workbench is not authority?"

Tasks

1. "What prompt should be generated next given current repo state?"
2. "Generate PRODUCT-SPINE-REVIEW-01."

Artifacts

1. "Which repo files should I inspect first?"
2. "What audits prove the product-spine state?"

Risks

1. "What are the top risks before starting parallel dev?"
2. "How do we prevent docs outrunning code?"

Verification

1. "Create a checklist for verifying live repo state."
2. "What does `PASS_WITH_WARNINGS` currently mean?"

Spec Book / Aggregation

1. "Which parts of this chat should become formal spec requirements?"
2. "Which parts are background only?"

Deep Dive

1. "Explain Workbench progressive self-hosting."

2. "Explain the deep primitives model for player-created machines."
3. "Explain sparse worldgen and lazy historical evaluation."

39. Final Package Status

- Chat label: Dominion Architecture, Workbench, AIDE, and Product-Spine Planning
- Report type: full human-readable + structured handoff + spec-prep package
- Files created: yes
- ZIP created: yes
- Safe for later aggregation: yes, with caveats
- Extraction confidence: 4 / 5
- Main value of this chat: converged project identity, AIDE workflow doctrine, Workbench architecture, and product-spine sequencing.
- Most important decision: development should be non-blocking, promotion evidence-blocked.
- Most important unresolved issue: verify whether replay proof and barebones client prompts have executed.

- Most important next action: verify live repo state and run/generate the next missing product-spine prompt.
- Main caveats: visible transcript was partial; live repo may have advanced; latest prompts execution status uncertain.
- Best thing for me to read first: Ultra-Condensed Brief and Context Transfer Packet.
- Best question for me to ask next: "Given the live repo state, what is the next prompt to generate?"

22.2 Dominium Architecture Workbench Aide Product Spine Planning Reader Brief

Source: docs/archive/conversations/Workbench_AIDE_Product_Spine/Dominium_Architecture_Workbench_AIDE_Product_Spine_Planning__06_reader_brief.md

Report type: reader_brief

Size: 1840 bytes

22.2.1 Reader Brief - Dominion Architecture, Workbench, AIDE, and Product-Spine Planning

What this chat was about

This chat preserved and refined the architecture, repo-control strategy, Workbench plan, AIDE development model, and immediate product-spine task sequence for Dominion. It established Dominion as a deterministic, contract-governed simulation platform, not merely a game. Domino is the reusable substrate; Dominion is the product/game family; Workbench is the production and validation environment; AIDE is the repo control plane; Codex is the bounded patch executor.

Top 20 things to know

1. Foundation Lock is PASS_WITH_WARNINGS, not clean.
2. Fast strict passes; full CTest remains T4 debt.
3. Dependency-direction has 0 violations but known warnings.
4. Broad feature work remains blocked.
5. Package mount slice is complete only as fixture-level proof.

6. Replay proof is the next product-spine task if not already run.
7. Barebones client follows replay proof.
8. Product Spine Review follows replay + barebones.
9. Workbench is not authority.
10. AIDE should manage task branches, blockers, repairs, and checkpoints.
11. Dev should be non-blocking; main should be evidence-blocked.
12. Agents should use task branches/worktrees, not shared dev directly.
13. Paths are not identity.
14. Implementation is not contract.
15. UI modes are projections, not separate systems.
16. Public ABI remains C-compatible.
17. C17/C++17 is intended target baseline; verify live build files.
18. Full gameplay/worldgen work is deferred.
19. Presentation contracts are a near-future task.
20. Verify live queue before generating any new prompt.

Best next step

Check live repo state and determine whether QUEUE-RECONCILE-01, REPLAY-PROOF-SLICE-01, and BAREBONES-CLIENT-SHELL-01 have run. Then generate the next missing prompt.

22.3 Dominium Architecture Workbench Aide Product Spine Planning In Chat Reader

Source: docs/archive/conversations/Workbench_AIDE_Product_Spine/Dominium_Architecture_Workbench_AIDE_Product_Spine_Planning__09_in_chat_reader.md

Report type: in_chat_reader

Size: 938 bytes

22.3.1 In-Chat Reader - Dominium Architecture, Workbench, AIDE, and Product-Spine Planning

Package overview

This preservation package explains the conversation, records the current task state, preserves decisions and rejected options, and prepares the chat for aggregation into a master Project Spec Book.

File index

Read:

1. Human-readable report for full understanding.
2. Context transfer packet for continuing in a new chat.
3. Spec sheet for aggregation.
4. Registers for task/decision tracking.
5. Verification/audit before relying on the state.

Question menu

Ask:

- What is the next prompt given live repo state?
- What is the product-spine sequence?
- What is the AIDE dev/main workflow?
- What is Workbench allowed to do next?
- What are the known warnings and blockers?
- Which parts should become spec requirements?

Safest next action

Verify live repo state and generate the next missing product-spine prompt.

22.4 Dominion Architecture Workbench Aide Product Spine Planning Accompanying Detailed Summary

Source: docs/archive/conversations/Workbench_AIDE_Product_Spine/Dominium_Architecture_Workbench_AIDE_Product_Spine_Planning__10_accompanying_detailed_summary.md

Report type: accompanying_detailed_summary

Size: 14735 bytes

22.4.1 Accompanying Human-Readable Detailed Summary and Report

**Dominium Architecture, Workbench, AIDE, and
Product-Spine Planning**

Date anchor: 2026-05-27 Australia/Melbourne

Source scope: This chat only unless explicitly labelled as PROJECT-CONTEXT.

Reliability note: This report reconstructs the visible

conversation, the user-pasted transcripts, the generated handoff files, and the current preservation task. Some earlier messages were collapsed as skipped/summary content in the visible transcript, so this is a high-fidelity reconstruction, not a verbatim transcript.

1. What this conversation was really about

FACT: This conversation was not a normal feature-planning chat. It began from a Dominion world-generation and simulation-realism handoff and expanded into a project-wide architecture, repo governance, Workbench, and AIDE development-control discussion.

The user’s underlying goal was to avoid building Dominion as a brittle one-off indie project. The user wants Dominion to become a durable, modular, deterministic, portable, extensible, testable, replaceable, long-lived simulation platform. The conversation repeatedly returned to the same concern: if the project is not built with stable contracts, evidence, versioning, and strict ownership boundaries from the beginning, future feature work will collapse into refactor debt.

The final project identity that emerged is:

Domino = reusable deterministic substrate

Dominium = game/product family on Domino

Workbench = production, validation, editing, inspection, packaging, and evidence environment

AIDE = repo/control-plane harness

Codex = bounded patch executor

Contracts = law

Diagnostics / tests / replay / evidence = proof

The most important conceptual shift was from "game systems" to "contract-governed operating environment." Dominion is intended to support a realistic universe simulation, but also the tools, packages, mods, UI systems, proof systems, and development workflow required to build that simulation without losing control. The conversation produced three major outputs:

1. A settled architectural doctrine for Domino / Dominion / Workbench / AIDE.
2. A near-term product-spine task sequence after Foundation Lock.
3. A preservation package for future chats and eventual spec-book aggregation.

2. The conversation arc

2.1 Worldgen and the realism problem FACT: The user first explored whether realistic procedural worlds could be made by generating a world and then simulating years, millennia, or billions of years so rivers, erosion, mountains, climates, biomes, forests, species, tribes, and materials emerge naturally.

The conclusion was not to globally simulate everything. Instead, the architecture should support deterministic lazy historical evaluation:

Truth is total.

Materialization is conditional.

History can be evaluated lazily.

Observation and causality determine refinement.

This means a world can have a lawful, deterministic past without requiring every point in space and time to have been fully simulated at all times.

INFERENCE: This idea became a root pattern for the whole project: do not eagerly expand expensive detail unless observation, interaction, or proof requires it.

2.2 Epistemics and diegetics as optimization

doctrine FACT: The user asked whether epistemics and diegetics could optimize what is simulated, loaded, and rendered. The answer became yes: epistemic state and diegetic access are not merely flavor. They help decide what detail must exist, what can remain summarized, and what must be preserved because it was observed.

This led to a broader doctrine:

Truth / Perceived / Render separation must hold.

No presentation state belongs in truth.

Observation anchors preserve continuity.

Unobserved domains can remain summarized.

Refinement is governed by causality and capabilities.

2.3 Player creation, machines, and arbitrary invention FACT: The user repeatedly asked how players can make things like planks, beams, screws, pottery wheels, kilns, rails, machines, vehicles, solenoids, computers, factories, and infrastructure without the game hardcoding everything.

The resulting answer was the deep-primitives model. Instead of making every object a special truth object, Dominium should model:

materials
geometry
fields
constraints
processes
affordances
recognizers/classifiers
formalizations
blueprints
measurements
tolerances
standards
procedures
institutions/knowledge

This enables a player to make something experimentally, have the system recognize useful affordances, then formalize it into a reusable design, blueprint, standard,

production procedure, module, or pack.

Rejected options included:

hardcode every possible object

force every player creation to be manually declared

try to infer player intent directly as primary truth

The preferred approach is recognition plus optional formalization.

2.4 Civilization, macro/micro simulation, and long history FACT: The user wanted far civilizations and distant planets to be believable without simulating every individual and institution in full detail. The conversation settled on macro truth that can expand and collapse.

A distant civilization can exist initially as:

population distributions

infrastructure density

industrial capacity

knowledge base

institutions

governance

legitimacy

logistics corridors

land use

culture and standards

When observed, it expands into more concrete settlements, buildings, agents, documents, laws, and history. When no longer needed, it can collapse into capsules while preserving observed facts and causal

constraints.

2.5 Repo convergence and Foundation Lock

FACT: A large part of the conversation became repo-grounded. The repo had already gone through many convergence and cleanup phases before or during the chat. Foundation Lock moved from blocked to `PASS_WITH_WARNINGS`.

The expected current state at the time of this preservation task is:

```
Foundation Lock = PASS_WITH_WARNINGS
```

```
fast strict = PASS
```

```
dependency-direction strict = PASS with 0 violations and known warnings
```

```
full CTest = NOT_RUN_T4_DEBT
```

```
broad feature work = blocked
```

FACT: `PACKAGE-MOUNT-SLICE-01` was reported as complete with `PASS_WITH_WARNINGS`, and the next intended product-spine task was `REPLAY-PROOF-SLICE-01`.

UNCERTAIN / UNVERIFIED: Whether `QUEUE-RECONCILE-01`, `REPLAY-PROOF-SLICE-01`, and `BAREBONES-CLIENT-SHELL-01` have actually been executed depends on live repo state after this report. The next chat must verify.

2.6 Workbench FACT: The old "UI Editor / Tool Editor" idea was superseded. The preferred model is Dominion Workbench as the production environment for building the project.

Workbench should eventually support:

- validation dashboards
- project graph explorer
- pack browser
- agent work board
- component matrix viewer
- theme laboratory
- renderer sandbox
- interface studio
- layout studio
- TUI studio
- rendered GUI studio
- release forge
- module / pack foundry
- app composer

But Workbench is not the center of truth. It presents and edits through contracts, commands, documents, patches, services, diagnostics, and evidence. It must not call private validators or mutate files directly as authority.

2.7 Presentation architecture FACT: The conversation settled on CLI, text/TUI, rendered GUI, native GUI, and headless as projections over one

semantic spine.

The spine is:

```
intent
-> command
-> capability/refusal check
-> service
-> result | document | snapshot
-> diagnostics/evidence
-> view
-> action model
-> projection
-> shell
```

This prevents four separate UI systems.

2.8 AIDE workflow and parallel development

FACT: Near the end of the chat, the user asked about running prompts in parallel on dev. The chosen doctrine became:

Development should be non-blocking.
Promotion should be evidence-blocked.

The corrected branch model is:

```
origin/main promoted truth
origin/dev shared integration branch
local/dev local tracking/integration branch
task/<task-id> one branch per prompt / Codex / human task
repair/<task-id> generated repair branch from blocker evidence
checkpoint/<id> merge/proof branch before main promotion
```

Agents should not directly mutate shared dev. AIDE

should integrate task branches, manage blockers, create repair/resume tasks, run checkpoints, and promote only evidence-backed states to main.

3. What was decided

3.1 Core identity decisions FACT: Dominion is no longer being treated as just a game. It is a deterministic simulation operating environment.

FACT: Domino is the reusable deterministic/runtime substrate. Dominion is the game/product family. Workbench is the production environment. AIDE is the repo/control-plane harness. Codex is a bounded patch executor.

FACT: The real center is not Workbench or folders. The center is:

- contracts
- commands
- services
- providers
- packs
- modules
- artifacts
- capabilities
- diagnostics
- tests
- replay
- evidence

3.2 Repo and implementation boundaries

FACT:

The canonical root set is:

apps/
engine/
game/
runtime/
contracts/
content/
docs/
tests/
tools/
scripts/
cmake/
external/
release/
archive/

Rejected as active top-level roots:

src/
source/
core/
control/
data/
packs/
profiles/
modules/
plugins/
services/
sdk/
workspaces/
repo/
validation/

meta/

FACT: Paths are not identity. Stable IDs, contracts, manifests, registries, and public surfaces are identity.

3.3 Product-spine sequencing FACT: The current path is narrow slices, not broad implementation.

The immediate path:

QUEUE-RECONCILE-01, if queue stale

REPLAY-PROOF-SLICE-01

BAREBONES-CLIENT-SHELL-01

PRODUCT-SPINE-REVIEW-01

Then limited parallel development can begin if review passes.

3.4 AIDE workflow FACT: The user accepted the doctrine that development is non-blocking and promotion is evidence-blocked.

FACT: AIDE should ultimately manage task lifecycle, blockers, repair queues, evidence, checkpoint branches, and promotion to main.

3.5 Workbench FACT: Workbench is not a monolithic editor. It is a modular production environment over the shared command/document/view/service/projection spine.

FACT: Workbench should become progressively self-

hosting only after the seed substrate and projection proof exist.

4. What was postponed or explicitly not done

The chat repeatedly deferred broad implementation.

Not yet authorized:

- broad Workbench UI
- runtime module loader
- provider runtime
- broad package runtime or mod loader
- gameplay/domain implementation
- renderer implementation
- native GUI
- release publication
- full game/world replay
- save/load runtime
- world runtime
- Interface Studio
- runtime projection engine
- full AIDE OS automation

Postponed but important:

- FULL-GATE-DEBT-01
- DEPENDENCY-DIRECTION-WARNING-BURNDOWN-01
- AIDE-REVIEW-REF-REPAIR-01
- SERVICE-CONFORMANCE-WARNING-DISPOSITION-01
- PRESENTATION-CONTRACT-01
- PROJECTION-CONFORMANCE-01
- ACCESSIBILITY-CONTRACT-01

TEXT-LOCALIZATION-CONTRACT-01
AIDE-WORKFLOW-LAW-01
AIDE-WORKUNIT-SCHEMA-01
AIDE-DEV-MAIN-POLICY-01
AIDE-CHECKPOINT-LOOP-01
AIDE-CAPABILITY-REALITY-LEDGER-01
SPEC-DEEP-PRIMITIVES-01
SPEC-FAILURE-ONTOLOGY-01
DOMAIN-CONSTITUTION-WAVE-01

5. What the next chat must verify first

The most important uncertainty is live repo state.

The next chat should verify:

```
git log / recent commits
.aide/queue/current.toml
.aide/context/latest-task-packet.md
docs/repo/FOUNDATION_LOCK.md
docs/repo/audits/PACKAGE_MOUNT_SLICE_01.md
docs/repo/audits/QUEUE_RECONCILE_01.md, if present
docs/repo/audits/REPLAY_PROOF_SLICE_01.md, if present
docs/repo/audits/BAREBONES_CLIENT_SHELL_01.md, if present
```

Decision logic:

If package mount is complete but queue still points to PACKAGE-MOUNT-SLICE-01:

```
run/generate QUEUE-RECONCILE-01.
```

If replay proof is missing:

```
run/generate REPLAY-PROOF-SLICE-01.
```

If replay proof exists and barebones client shell is missing:

```
run/generate BAREBONES-CLIENT-SHELL-01.
```


If package mount, replay proof, and barebones client are complete:

generate PRODUCT-SPINE-REVIEW-01.

6. What this package adds

This refreshed package contains the original preservation files plus this new accompanying detailed human-readable summary and updated manifest. It is intended to make the conversation understandable without rereading the transcript and also usable for later aggregation into a master spec book.

The package is useful for:

continuing in a new chat
checking task sequence
understanding decisions and rationale
preventing repeated debates
preparing a master spec book
feeding a future aggregator chat
auditing warnings and uncertainties

7. Final recommended next action

If continuing development from this chat:

1. Verify live repo state.
2. If stale, run QUEUE-RECONCILE-01.

3. If missing, run `REPLAY-PROOF-SLICE-01`.
4. If replay is complete, run `BAREBONES-CLIENT-SHELL-01`.
5. Then run `PRODUCT-SPINE-REVIEW-01`.
6. If review passes, start limited parallel development with:
 - `AIDE-WORKFLOW-LAW-01`
 - `PRESENTATION-CONTRACT-01`
 - `POINTER-WIDTH-SERIALIZATION-AUDIT-01`

Do not start broad Workbench, runtime package/provider/module, renderer, native GUI, gameplay, or release work until gates explicitly authorize it.

8. Summary of current package files

The updated ZIP contains:

```
00_manifest.md
01_human_readable_report.md
02_context_transfer_packet.md
03_spec_sheet.yaml
04_registers.md
05_aggregator_packet.md
06_reader_brief.md
07_verification_and_audit.md
```

08_future_chat_bootstrap_prompt.md
09_in_chat_reader.md
10_accompanying_detailed_summary.md

The most useful files for a human are:

10_accompanying_detailed_summary.md
01_human_readable_report.md
02_context_transfer_packet.md
06_reader_brief.md

The most useful files for aggregation are:

03_spec_sheet.yaml
04_registers.md
05_aggregator_packet.md
07_verification_and_audit.md

The most useful file for starting a new chat is:

08_future_chat_bootstrap_prompt.md

9. Final reliability note

This report is intentionally conservative. It does not claim that every prompt has run, every warning is gone, or every planned system is implemented. It preserves the distinction between:

contract-level law
fixture-level proof
runtime implementation
product support
release support

The next chat should keep that distinction. The main risk is false promotion: treating contracts, docs, or fixture proofs as full runtime implementation. That mistake must be avoided.

Chapter 23

World Architecture

23.1 Dominion World Architecture Reader Brief

Source: docs/archive/conversations/World_Architecture/Dominium_World_Architecture__05_reader_brief.md

Report type: reader_brief

Size: 4059 bytes

23.1.1 Reader Brief - Dominion World Architecture

What This Chat Was About

This chat designed a deterministic, fixed-point, modular world architecture for a Dominion-style game. It covered the core spatial hierarchy, coordinate system, save format, runtime memory model, continuous terrain, material/media fields, hydrology, weather, climate, gases, liquids, oil/gas reservoirs, flooding, ruptures, networks, modding, tooling, launcher architecture, and a Codex implementation prompt. The main design principle is

that chunks and segments are storage/cache boundaries only, while canonical simulation state lives in deterministic global/topological systems.

The final user-confirmed world model uses a toroidal horizontal SurfaceGrid of 2^{24} metres per axis, one vertical Page of 4096 metres, 16 metre cubic chunks, runtime Q16.16 coordinates with 8-bit Segment addressing, and Q4.12 chunk-local save coordinates. The core and saves must not use floats. The core target is C89; platform/renderer/runtime may use C++98 and floats only outside the deterministic core.

Most Important Things to Know

- SurfaceGrid is toroidal 2^{24} m in X/Y.
- Z is one Page: 4096m total, 2km down/up.
- Chunk = $16 \times 16 \times 16$ m.
- Runtime = Segment + Q16.16.
- Save = Q4.12 local-to-chunk.
- Correct signedness bug: local X/Y and chunk-local Q4.12 should be unsigned or otherwise redesigned.
- No floats in core or saves.
- Core C89; platform/renderer C++98 and floats allowed one-way.

- Cosmic IDs: 2^8 galaxies, 2^{24} systems, 2^{16} planets.
- One canonical surface saved per planet for now.
- Chunks are caches, not authorities.
- Terrain is continuous ?, not blocks.
- Ground and air are abstract materials/media, not blocks.
- Use FluidSpaces, Connectors, SpatialVolumes, and Networks for environmental systems.
- content.lock and deterministic registries are required.
- Previous Codex prompt is useful but defective and must be repaired.

Active Plans or Workstreams

- Fixed-point deterministic core.
- Sparse TLV save format.
- Chunk/segment runtime memory.
- Continuous terrain/material/media.
- Fluid/weather/hydrology systems.
- Oil/gas/flooding/rupture support.

- Network systems for pipes/electric/heat/logic.
- Mod/content registry and deterministic Lua.
- Repo-wide modular engine/runtime/launcher/tools architecture.
- Corrected Codex implementation prompt.
- Determinism and seam tests.

Decisions Already Made

- 2^{24} m toroidal horizontal surface.
- One Page $Z = 4096$ m.
- 16m chunks.
- Runtime Q16.16.
- Save Q4.12.
- 8-bit Segment addressing.
- No floats in core/saves.
- C89 core, C++98 platform/renderer.
- One canonical saved surface per planet.
- Directory names should encode high address bits.
- Chunks must not own canonical simulation state.

Pending Tasks

- Repair Codex prompt.
- Inspect actual repo.
- Define unsigned fixed-point types.
- Define C89 portability layer.
- Correct Region bit decomposition.
- Implement world address conversions.
- Implement TLV save/load.
- Implement minimal geom_sample and chunk cache.
- Add deterministic tests.
- Specify solvers later.

Open Questions

- How exactly to handle 64-bit integers under C89 constraints?
- Unsigned local coordinates vs centered signed representation?
- Exact endian/alignment/compression save policy?
- Exact terrain meshing algorithm and sample resolution?

- Exact weather/hydrology/hydraulic/thermal formulas?
- Exact content manifest and Lua sandbox?
- Actual repo/build system/toolchain?

Files / Artifacts / Prompts to Preserve

- Previous Codex 5.1 implementation prompt, but only after correction.
- Original Context Transfer Packet.
- This report package and ZIP.
- Registers and YAML spec sheet.

What to Verify Before Acting

- Fixed-point signedness corrections.
- Region bit widths.
- C89 compatibility.
- Saved rotations are 8-bit.
- Actual repo structure.
- Compiler and platform targets.

Best Next Step

Revise the prior Codex implementation prompt using this report, then inspect the actual git repo before generating or applying code changes.

Chapter 24

Advanced Simulation Infrastructure

24.1 Dominion Advanced Simulation Infrastructure Architecture Reader Brief

Source: docs/archive/conversations/advanced_simulation_infrastructure/dominium_advanced_simulation_infrastructure_architecture__05_reader_brief.md

Report type: reader_brief

Size: 4449 bytes

24.1.1 Reader Brief - Dominion Advanced Simulation and Infrastructure Architecture

What This Chat Was About

[FACT] This chat developed architecture for the Dominion/Domino deterministic engine, beginning with advanced simulation systems and expanding into

gameplay layering, infrastructure construction, arbitrary placement, corridors/splines, buildings/structures, signage, markings, decor, and implementation/refactor prompts. The central constraint is that Domino Engine must remain ISO C89, deterministic, fixed-point, portable, and free of hardcoded gameplay semantics, while Dominion UI/tools use a portable C++98 subset and emits deterministic commands.

[FACT] The largest outcome was a unified BUILD / TRANS / STRUCT / DECOR / SIM model. BUILD owns construction intent, phases, costs, and jobs. TRANS owns linear/networked topology such as corridors, junctions, slots, attachments, z-profiles, microsegments, and lane/track/utility graphs. STRUCT owns volumetric/load-bearing/enclosed structures, buildings, interiors, surface/anchor graphs, and generated carriers such as bridges/tunnels. DECOR owns visual signs, markings, decals, props, generated baselines, manual overrides, semantic keys, and tiles. SIM owns deterministic physics domains. RES/content owns semantics.

[FACT] The key late requirement is no grid lock: the engine should support arbitrary position, rotation, inclination, and full 3D curves. Grids are optional Dominion UI snapping aids only. Determinism should come from fixed-point math, quantized commands, stable IDs, ordered iteration, fixed iteration counts, and

parametric host-relative anchoring.

Most Important Things to Know

- Domino Engine = ISO C89; Dominion UI/tools = portable C++98.
- No OS APIs in engine.
- No floats where determinism matters; use fixed-point Q formats.
- No hardcoded gameplay semantics; content defines meaning.
- UI emits deterministic commands; it does not mutate engine state directly.
- Overlapping infrastructure should use corridor bundles with slots, not competing splines.
- Runtime performance uses compiled microsegments, chunk spatial indices, dirty flags, and hot/cold data split.
- Complex infrastructure should be built through corridors, junctions, layers, modules, and facilities.
- Signage/markings are DECOR: generated by rules and manually overridden with pin/suppress semantics.

- Host Surfaces + Anchor Frames generalize placement to buildings, interiors, terrain, props, and corridors.
- DECOR is visual-only unless promoted to a STRUCT/TRANS device.
- Engine is grid-agnostic; grids are UI snapping only.
- No repo files were inspected or modified in the original chat.
- Proposed paths and command names are conceptual until verified.

Active Plans or Workstreams

WORKSTREAM-01, WORKSTREAM-02,
WORKSTREAM-03, WORKSTREAM-04,
WORKSTREAM-05, WORKSTREAM-06,
WORKSTREAM-07, WORKSTREAM-08,
WORKSTREAM-09, WORKSTREAM-10,
WORKSTREAM-11, WORKSTREAM-12,
WORKSTREAM-13

Decisions Already Made

The hard user-stated decisions are DECISION-01 through DECISION-05. The strongest design commitments are DECISION-08, DECISION-10,

DECISION-13, DECISION-15, DECISION-16, and DECISION-17. See the registers file for exact statuses and labels.

Pending Tasks

The highest-priority tasks are TASK-01 through TASK-05: verify repo, compare existing refactor plan, formalize placement spec, choose deterministic frame math, and audit grid assumptions. Implementation should then follow TASK-06 through TASK-14.

Open Questions

Most important: actual repo structure, existing GPT-5.2 refactor plan contents, final Q formats, orientation representation, grid assumptions in current code, microsegment length, packing VM instruction set, bridge/tunnel ownership, and checksum rules.

Files / Artifacts / Prompts to Preserve

Preserve ARTIFACT-11 (nine Codex prompts), ARTIFACT-12 (existing-refactor merge prompt), ARTIFACT-13 (previous Context Transfer Packet), and this package's generated Markdown/YAML/ZIP files.

What to Verify Before Acting

Verify repo structure, existing refactor plan, current grid assumptions, command/replay/TLV systems, fixed-point math utilities, and whether any prompts have already been executed elsewhere.

Best Next Step

Download and save this package. If continuing implementation, inspect the repo and then convert TASK-03 into a repo-specific placement/quantization spec before running any Codex implementation prompts.

Chapter 25

App Runtime Platform Renderers

25.1 Dominium App0 Runtime Platform Renderers Reader Brief

Source: docs/archive/conversations/app_runtime_
platform_renderers/dominium_app0_runtime_
platform_renderers__05_reader_brief.md

Report type: reader_brief

Size: 5601 bytes

25.1.1 Reader Brief - Dominium APP0 Runtime, Platform, and Renderer Architecture

What This Chat Was About

This chat focused on Dominium / Domino APP0: the runtime/application/platform/renderer layer. The user provided an APP0 prompt defining client, server, launcher, setup/installer, tools, renderers, and platform

support while forbidding simulation-rule redesign, content-definition changes, life/civ/economy changes, or gameplay shortcuts. The central boundary is that engine + game remain authoritative, while applications are shells and orchestrators.

A first assistant response produced a large Codex-style implementation pack, but the user then explicitly asked to discuss before planning or generating prompts.

Discussion shifted to architectural correctness, missing enforcement mechanisms, and the eventual goal: a real executable capable of displaying an interactive, resizable window on supported platform/renderer combinations.

The chat then developed a proposed architecture: add or formalize a platform runtime layer for windows/events/surfaces/timing; keep renderers separate from window creation; bring up null and software rendering before GPU backends; treat GPU vendors as metadata rather than renderer families; use capability tiers and support classes for modern, legacy, vintage, and experimental targets; and consider runtime modules/plugins/manifests for plug-and-play distribution. The user granted write access to docs plus render, platform, application, client, and server, but not engine/game.

The most important unresolved item is repository verification. The user requested inspection of an old code snapshot in project attachments. A prior assistant claimed to inspect it, but that claim is unverified in this

package. The next assistant should inspect the actual files before producing repo-specific plans or prompts.

Most Important Things to Know

- APP0 scope is application/runtime/platform/render only.
- Engine + game remain authoritative.
- Applications are orchestrators, not decision-makers.
- Rendering never affects simulation.
- Client is non-authoritative.
- Server is authoritative and headless-capable.
- Launcher must not install content or alter simulation state.
- Setup owns install/version/integrity responsibilities.
- Tools must obey authority/law and be auditable.
- Current write permission covers docs plus render/platform/application/client/server only.
- Desired macro-plan end state includes a real interactive resizable window.
- Platform/windowing should be separate from rendering.

- Renderers should consume platform surfaces, not create windows.
- Null/software renderer path should come before GPU backends.
- Do not model renderers by GPU vendor.
- Module/plugin/framegraph/support-tier ideas are proposals, not accepted final decisions.
- Inspect old code snapshot before any repo-specific prompt or implementation plan.

Active Plans or Workstreams

- WORKSTREAM-01 APP0 application/runtime layer.
- WORKSTREAM-02 Client runtime and window bring-up.
- WORKSTREAM-03 Server runtime.
- WORKSTREAM-04 Renderer architecture and capability model.
- WORKSTREAM-05 Platform runtime.
- WORKSTREAM-06 Launcher orchestration.
- WORKSTREAM-07 Setup/installer/integrity.
- WORKSTREAM-08 Tools and audit/authority.

- WORKSTREAM-09 Distribution/modules/plugins.
- WORKSTREAM-10 Old repo snapshot verification.

Decisions Already Made

- APP0 is runtime/application layer only.
- Authoritative logic remains in engine + game.
- Client must not author authoritative state.
- Server must be authoritative and headless.
- Launcher must not install content or mutate sim state.
- Tools must obey same authority rules and be auditable.
- Do not generate more prompts until user asks after discussion/verification.
- Current permitted write areas exclude engine/game.

Pending Tasks

- Inspect old repository snapshot.
- Verify actual directory layout.

- Verify build system/languages.
- Verify engine/game public APIs.
- Map dependency boundaries.
- Define platform runtime contract.
- Define renderer capability schema.
- Decide support classes and module strategy.
- Draft repo-specific architecture plan.
- Generate Codex prompts only when requested.

Open Questions

- What is the actual repo tree?
- Is `app`, `apps`, or `application` canonical?
- Are engine/game APIs sufficient without modification?
- Do renderers already exist?
- Does platform runtime already exist?
- Static modules, dynamic plugins, or hybrid?
- Which platforms/renderers are canonical vs compatibility vs experimental?
- What is the setup trust model?

- What is the tools elevation/audit model?
- What are server sharding semantics?

Files / Artifacts / Prompts to Preserve

- User APP0 prompt.
- Assistant APP0 implementation pack, with caveats.
- APP0 critique.
- Platform runtime proposal.
- Renderer/platform taxonomy proposal.
- Advanced runtime/module/framegraph proposal, with caveats.
- Prior Context Transfer Packet.
- This final report package.
- Old repo snapshot/project attachments, not yet verified.

What to Verify Before Acting

- Actual old repository files.
- Build system and languages.

- Current client/server/render/platform dependencies.
- Engine/game public APIs.
- Whether renderers create windows.
- Whether server depends on graphics.
- Existing docs/tests.
- External graphics API/platform facts.

Best Next Step

Inspect the old code snapshot/project attachments and produce a verified repository inventory with FACT / INFERENCE / UNCERTAIN labels before generating implementation prompts or architecture changes.

Chapter 26

App Testx Codehygiene

26.1 Dominion App Testx Codehygiene Handoff Reader Brief

Source: docs/archive/conversations/app_testx_codehygiene/dominium_app_testx_codehygiene_handoff__05_reader_brief.md

Report type: reader_brief

Size: 4168 bytes

26.1.1 Reader Brief - Dominion Application/TestX/CodeHygiene Handoff

What This Chat Was About

This retired chat consolidated a large amount of Dominion / Domino planning into current carry-forward state for application-layer implementation, testing/version/changelog governance, code hygiene, and chat handoff packaging. The project itself is a deterministic universe engine/game, but this chat's latest active focus

is no longer core simulation design. The latest user-pasted PROJECT-CONTEXT says architecture is closed and canon locked; future work should implement, audit, optimise, and maintain within canon.

The most important current artifact is APP-UNIFIED-CANON, which unifies setup/launcher/application-layer plans. It requires a pure application contract layer (`libs/contracts`), a shared appcore (`libs/appcore`), a single canonical command graph, declarative UI IR, setup as install authority, launcher as reference app, thin client/server, read-only tools, RepoX/TestX integration, accessibility/localisation, and strict failure behavior.

The chat also produced TESTX, a replacement for TEST0, to create long-lived tests, assertions, comment-density checks, blocker crossrefs, version/build/changelog automation, and git discipline.

CODEHYGIENE-X addresses hardcoded taxonomies, magic numbers, CUSTOM/OTHER enums, admin/cheat flags, and data/code boundary enforcement.

Generated prompts are not proof of repository implementation. The next chat must inspect repo/source/docs before claiming files exist.

Most Important Things to Know

- Architecture is closed; do not redesign.
- Core ontology from latest bootstrap: Assemblies,

Fields, Processes, Agents, Law.

- Engine is C89; game is C++98.
- Apps may use newer toolchains but cannot impose them on engine/game.
- Applications are orchestration shells only.
- CLI is canonical.
- UI is data (UI IR), never business logic.
- Setup is the only install mutation authority.
- Launcher is the reference application.
- Tools are read-only by default.
- Apps must run with zero content packs.
- RepoX is source of truth for build/changelog/compatibility.
- No manual changelog editing.
- BUILD-ID-0 semantics supersede earlier build-number plans.
- TESTX supersedes TEST0.
- CODEHYGIENE-X is current hygiene plan.
- Verify actual repo state before implementation.

Active Plans or Workstreams

- Application layer implementation.
- Testing/version/changelog governance.
- Source-code hygiene/data-code boundary.
- Docs/canon/package handoff.
- Content/data later in separate chat.

Decisions Already Made

- APP-UNIFIED-CANON is latest application plan.
- CLI canonical; GUI/TUI are views.
- Setup handles install/repair/rollback; launcher does not duplicate setup logic.
- Tools default read-only.
- Changelogs generated automatically.
- Hardcoded open taxonomies should migrate to data/registries.

Pending Tasks

- Verify which prompts were actually executed.
- Inspect repository for `libs/contracts`, `libs/appcore`, `RepoX`, `TestX`, `VALIDATE-0`.

- Generate concrete application implementation prompt if continuing app work.
- Run or adapt TESTX.
- Run or adapt CODEHYGIENE-X.
- Store this package for aggregation.

Open Questions

- Which generated prompts ran?
- Does CANON_INDEX.md exist?
- Where are RepoX/TestX/VALIDATE-0 files?
- Are app contract/appcore directories present?
- How are DOMINIUM_RUN_ROOT and DOMINIUM_HOME finalized?

Files / Artifacts / Prompts to Preserve

- APP-UNIFIED-CANON
- TESTX
- CODEHYGIENE-X
- DOCS0
- CANON0 / REALITY0 / LIFE0+ / CIV0+ / FUTURE0

- Latest user-pasted global bootstrap
- Latest user-pasted application bootstrap
- This final report package

What to Verify Before Acting

- Actual repo state.
- Actual docs/canon files.
- Actual RepoX/TestX/VALIDATE-0 commands.
- Whether BUILD-ID-0 is implemented.
- Whether APP-CANON1/APP-AUTO-0/APP-UI-BIND-0 docs exist.

Best Next Step

Open a new chat with the bootstrap prompt and this package. If continuing application work, inspect repo or ask for source/docs, then generate a concrete Codex prompt to implement `libs/contracts` and `libs/appcore` according to APP-UNIFIED-CANON.

Chapter 27

Architecture Codex Prompts

27.1 Dominium Domino Architecture Codex Prompts Reader Brief

Source: docs/archive/conversations/architecture_codex_prompts/dominium_domino_architecture_codex_prompts__05_reader_brief.md

Report type: reader_brief

Size: 4349 bytes

27.1.1 Reader Brief - Dominium/Domino Architecture and Codex Prompts

What This Chat Was About

This chat developed the architecture and implementation roadmap for the Dominium/Domino project. The user provided a master starter prompt defining Domino as a deterministic, fixed-point, ISO C89 engine and Dominium as a portable C++98 product suite. The chat expanded from resources, items, entities, recipes, belts, buildings, vehicles, hydrology, atmosphere, blueprints, and saves into a full engine/product/content/

tooling architecture.

The main outcome is a layered, data-driven design: Domino provides generic deterministic systems; Dominium provides products, GUI, launcher/setup/tools; content packs/mods define all actual gameplay data through TLV schemas. The user explicitly corrected the direction to require GUI-first testing and to keep actual data definitions out of engine/core. A sequence of Codex prompts was generated from Prompt 1 through Prompt 18, covering core infrastructure, content, world subsystems, GUI, demo slice, determinism, advanced simulation, logistics, production, research/economy/policies, multiplayer, and tools/editors.

The latest strategic direction is to retire this chat and start a new conversation focused on Path D: advanced simulation, including heat, power grids, fluids, atmosphere, vehicle physics, structural loads/destruction, and mod-extensible deterministic physics.

Most Important Things to Know

- Domino Engine = ISO C89.
- Dominium products/tools/UI = portable C++98 subset.
- Simulation must be deterministic and fixed-point.
- Engine/core must not know actual gameplay data

definitions.

- All real materials/items/machines/recipes/resources live in packs/mods/TLV.
- GUI-first minimal testing is preferred; do not rely only on CLI.
- Rendering goes through dgfx/DVIEW/DUI; no OS-native drawing.
- Platform operations go through dsys.
- The main architecture pattern is Core + Model + Proto + Instance + Registry + TLV.
- Prompts 1-18 are artifacts, not proof of implementation.
- Existing code may already exist and be stale; inspect and reconcile before coding.
- Path D advanced simulation is the next-chat focus.

Active Plans or Workstreams

- WORKSTREAM-01: Domino engine core infrastructure.
- WORKSTREAM-04: Content/TLV/packs/mods/base pack.
- WORKSTREAM-15: GUI/render/view/UI.

- WORKSTREAM-17: Advanced simulation Path D.
- WORKSTREAM-18: Prompt roadmap and handoff/report packaging.

Decisions Already Made

- Use C89 for engine and C++98 for products.
- Use fixed-point deterministic simulation.
- Keep platform/rendering behind dsys/dgfx.
- Keep actual content out of engine/core.
- Use GUI-first smoke testing.
- Use base/base_demo packs for data-driven demos.
- Use spline/packet/container logistics.
- Compile vehicles/weapons from blueprints into runtime aggregate objects.
- Focus next chat on Path D.

Pending Tasks

- Save this report package.
- Inspect actual repository state.
- Start Path D new chat using the bootstrap prompt.

- Verify whether prompts have been implemented.
- Choose/verify GUI backend.
- Define Path D solver architecture and fixed-point ranges.
- Avoid duplicate systems if old code exists.

Open Questions

- Has any Codex prompt been applied to the repo?
- Which GUI backend is available?
- Should new Path D chat produce architecture or Codex prompts first?
- What exact fixed-point ranges should advanced solvers use?
- How strict should domain-neutrality be in engine comments/docs?
- What exact TLV schema tags are canonical?

Files / Artifacts / Prompts to Preserve

- ARTIFACT-01: Extended Master Starter Prompt.
- ARTIFACT-02: dres proposal.
- ARTIFACT-09 through ARTIFACT-26: Prompts 1-18.

- ARTIFACT-27: New Chat Advanced Simulation Starter Prompt.
- ARTIFACT-28: Prior Context Transfer Packet.
- ARTIFACT-29: This final report package.

What to Verify Before Acting

- Actual repo state.
- Build system.
- Existing stale components.
- GUI backend availability.
- Content/TLV schema code.
- Whether user wants architecture or implementation prompt next.
- Any external/current dependency facts.

Best Next Step

Start the new Path D chat with the bootstrap prompt from the full report or prior transfer packet, then produce the advanced simulation top-level architecture before requesting implementation prompts.

Chapter 28

Architecture Ui Providers

28.1 Dominion Architecture Ui Providers Robot Os Strategy Human Readable Report

Source: docs/archive/conversations/architecture_ui_providers/dominium_architecture_ui_providers_robot_os_strategy__01_human_readable_report.md

Report type: human readable report

Size: 49673 bytes

28.1.1 COMPLETE CHAT PRESERVATION REPORT - Dominium Architecture, UI, Providers, and Robot OS Strategy

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion
Architecture, UI, Providers, and Robot OS
Strategy

Row 2 Field: Date anchor: Assessment: 2026-05-27

Australia/Melbourne

Row 3 Field: Source scope; Assessment: This chat only unless labelled PROJECT-CONTEXT

Row 4 Field: Apparent access; Assessment: Partial with caveats

Row 5 Field: Previously generated files available?; Assessment: No prior downloadable files visible in this chat; new files generated by this task

Row 6 Field: Uploaded files or artifacts present?; Assessment: Yes: `Pasted text.txt`, containing the preservation/export prompt

Row 7 Field: Contains future plans?; Assessment: Yes

Row 8 Field: Contains decisions?; Assessment: Yes, with distinction between explicit user decisions and accepted synthesis

Row 9 Field: Contains pending tasks?; Assessment: Yes

Row 10 Field: Contains unresolved questions?; Assessment: Yes

Row 11 Field: Staleness risk; Assessment: Medium

Row 12 Field: Extraction confidence; Assessment: 4/5

Row 13 Field: Safe for later aggregation?; Assessment: With caveats

Row 14 Field: Main limitations; Assessment: The visible chat includes extensive pasted transcripts and assistant summaries, but not guaranteed raw full transcripts of every earlier referenced chat. Live repo state and external library/version claims require re-verification before implementation.

Plain-language limitation: this package reconstructs the conversation available in this chat, including user-pasted prior-chat transcripts and the assistant's responses here. It does not claim independent access to every old chat or every latest repo file beyond what appeared or was retrieved during the conversation. External facts about raylib, SDL, Lua, Windows, macOS, OpenGL, Direct3D, Unreal, and tooling are preserved as discussed but placed in the verification queue where they affect implementation.

1. One-Page Orientation

This chat was a strategic architecture and design convergence session for Dominion/Domino. It began from a transferred knowledge base about rebuilding Dominion's GUI and binary strategy, then evolved into a broad platform-specification discussion covering repository structure, product shells, renderer/platform priorities, native applications, language standards, provider architecture, Workbench, UI authoring, and

finally the game's core identity as a robotic seed-civilisation simulator with a diegetic robot operating-system interface.

The user was trying to prevent *Dominium* from becoming a brittle one-off project. The repeated concern was that the codebase, files, directories, APIs, schemas, renderers, GUIs, and data formats should remain portable, modular, extensible, replaceable, and reusable for other Domino-engine games or even unrelated engine/game projects. The conversation therefore treated the project less like a simple indie game and more like a long-lived platform or OS-like codebase. That framing shaped almost every technical recommendation: stable contracts, replaceable implementations, service boundaries, provider manifests, compatibility matrices, command/action/view/document systems, and validators.

The earliest phase dealt with GUI and binary architecture. The retained direction was that every product should have CLI, every product should have at least TUI or deterministic TUI stubs, and GUI support should be modular. Setup and launcher should use OS-native/OEM+ apps where appropriate; the client should use a rendered UI; server remains headless/CLI/TUI first; tools are case-by-case. The user then pushed on directory structure, wanting fewer top-level folders and a more future-proof organization. The resulting doctrine

was ownership-based roots: apps compose, runtime implements, contracts define, content supplies, tools validate, and archive preserves historical material.

The conversation then reworked the platform and rendering plan. A key correction was that `vector2d` should not be treated as a renderer backend; it is drawing/canvas functionality. A software renderer remains useful as a CPU framebuffer/reference/test/fallback path. After the user announced the project had moved to C17/C++17 with floors of Windows 7 SP1, Mac OS X 10.9.5, and Linux, older fixed-function targets such as OpenGL 1.1/2.1 and Direct3D 5/7/8/9 were deprioritised to research/back-port lanes. OpenGL 3.3 became the preferred first direct hardware target, followed by Direct3D 11, then Metal/Vulkan/Direct3D 12 later. The user also brought in raylib, SDL2, Lua, rlgl, rlsw, raygui, raudio, rtextures, and rmodels as potential accelerators. The best synthesis became: use them aggressively, but only as replaceable providers behind Dominion-owned service contracts.

A major part of the chat was UI/UX architecture. The user wanted CLI, TUI, rendered GUI, native GUI, and later VR/immersive UI to be projections of one shared presentation model rather than four independent UI systems. Workbench became the planned authoring environment for views, widgets, layouts, themes, HUDs, TUI panels, rendered screens, workspaces, and UI packs.

The final UX turn connected this to gameplay: because players are robot avatars whose consciousness boots into nodes and downloads into manufactured bodies, the UI can be diegetic. Menus, HUD, Workbench, TUI, CLI, and immersive shells can all feel like a robot OS.

The most important design shift came near the end. The user stated that *Dominium* should be a robotic seed-civilisation game, not a labour-management game.

Players are machine consciousnesses/robot avatars. The mothership contains human knowledge, starter resources, fabrication capacity, and AI planning. The planet is generated from science-bound parameters. Players explore, mine, refine, fabricate, build cities, spawn labs, industrial networks, and eventually terraform or build megaprojects. The sim focuses on energy, materials, machines, logistics, sensors, fields, permissions, heat, maintenance, construction, and machine graphs, not human workers.

Future relevance: this chat should feed several chapters of a master Project Spec Book, including architecture doctrine, repo layout, provider strategy, renderer plan, language/toolchain baseline, UI/Workbench platform, Robot OS UX, and game design pillars. A future assistant must not flatten everything into "use raylib" or "robot UI theme." The essence is: first-party contracts and simulation law; replaceable providers; profile-selected products; shared UI projections; robotic seed-

civilisation as the game identity.

2. The Story of the Conversation

2.1 GUI/binary transfer baseline The chat began with a transfer-ready knowledge base from a prior discussion about rebuilding Dominion's GUI and binary surface. That brief established the original non-negotiables: CLI mandatory, TUI expected, modular GUIs, stable backend contracts, native or near-native frontend families per platform, and avoidance of one universal GUI framework or one GUI per OS version. Products in scope were setup, launcher, client, server, and tools.

2.2 Live repo and directory convergence The user then said the latest docs and code were live at `julesc013/dominium`. The assistant treated the repo as an implementation baseline and observed that AppShell, UI mode resolution, distribution doctrine, and component matrices already existed. This changed the task from inventing a GUI strategy from scratch to reconciling the transfer brief with existing repository doctrine.

The user next emphasized that the directory structure was a "jumped mess" and needed to become more logical and future-proof. The discussion established that root

structure should be ownership-based, not topic-based. The resulting model used roots such as `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `docs/`, `tests/`, `tools/`, `release/`, and `archive/`. Later pasted reports claimed much of this convergence had happened, with residual issues in schema taxonomy, pack internal layout, runtime taxonomy, AIDE state classification, and report integrity.

2.3 Platform/render/native-app evolution The chat then moved into platform and renderer planning. Initially the discussion included older lanes such as OpenGL 1.x/2.x, Direct3D 9, DX12, Vulkan, Metal, Win32, Cocoa, X11/Wayland, SDL2, AppKit, WinForms, SwiftUI, WinUI, GTK, and others. Over time, the plan sharpened. The user challenged `vector2d`, leading to the conclusion that it should be a drawing/canvas layer, not a renderer backend. Software renderer was defined as a CPU framebuffer renderer for tests, fallback, and reference behavior, not necessarily a full 3D engine.

When the user announced the transition to C17/C++17 and minimum systems of Windows 7 SP1, Mac OS X 10.9.5, and Linux, many old lanes became irrelevant for first-wave work. OpenGL 1.1/2.1 and old DirectX were deferred. OpenGL 3.3 became the first direct hardware target, Direct3D 11 the Windows target, and Metal/

Vulkan/DX12 later advanced lanes.

2.4 Broad platform engineering doctrine The user asked what practices would make the code portable, modular, extensible, reusable, replaceable, and future-proof. The response expanded into general engineering doctrine: stable contracts, replaceable implementations, schema/version discipline, C-compatible ABI boundaries, command/view/result/refusal models, deterministic replay, provider manifests, forbidden include checks, conformance tests, semantic IDs separate from file paths, and validators. The large principle became: contracts are stable; implementations are replaceable.

2.5 Workbench and shared UI platform The user then focused on CLI/TUI/rendered/native GUI sharing and Workbench. The conversation converged on a command/action/result/document/view/patch/projection spine. Workbench should not be a one-off editor or raygui object editor. It should author Dominion UI artifacts: views, layouts, widgets, themes, TUI panels, HUDs, workspaces, and UI packs. Those artifacts are then projected into command, text, rendered, native, headless, and later immersive shells.

2.6 Raylib/SDL/Lua provider doctrine The user pasted a long discussion about using raylib, SDL2, Lua,

rlgl, rlsw, raygui, raudio, rtextures, and rmodels. The final synthesis was not "make Dominium a raylib game." It was: use raylib aggressively as a seed provider suite, SDL2 as a platform/input/audio provider, Lua as a pinned script provider, and Dominium's own contracts as the authority. Directory structure should remain service-first: `runtime/render/providers/raylib/`, not `runtime/raylib/render/`.

2.7 Game design shift: robotic seed civilisation

The user then supplied the strongest game-design direction. Dominium should become a robotic seed-civilisation game, not a labour-management game. Players are robot avatars/machine consciousnesses. The mothership provides knowledge, starter fabrication, spawn capacity, and AI planning but is limited by feedstock, throughput, heat, energy, safety protocols, reserves, rare materials, and scale. Gameplay becomes exploration, surveying, mining, refining, fabrication, construction, logistics, sensor networks, spawn labs, terraforming, and megaprojects. Human worker NPCs are deliberately absent from the core.

2.8 Robot OS UI/UX Finally, the user connected the game premise to UI/UX. The player boots into an OS, chooses or enters a universe, downloads consciousness into a robot body, disconnects into "ice"/

backup, and operates through a mesh of robot bodies and constructed nodes obeying speed-of-light/communication constraints. The UI should feel like a customizable robot operating system: CLI, TUI, GUI, VR, custom themes, era styles, and HUD all share the same underlying menu/HUD/Workbench code. The assistant synthesized this as Robot OS: boot shell, node shell, body link shell, embodied HUD, Workbench shell, and protected diegetic epistemics.

3. Main Topics Discussed

Topic 1 - GUI, binary, and product shell

architecture The chat preserved the earlier direction that every Dominionium product should always work through CLI, should have TUI or deterministic TUI stubs, and may have modular GUIs. Setup and launcher should be OS-native/OEM+ where useful; the client should be rendered-first; server is headless/CLI/TUI first; tools are case-by-case. The conclusion remains that GUIs are thin shells over shared contracts, not independent product architectures. Uncertainty remains around exact native-shell implementation order and how much native GUI is first-wave versus deferred.

Topic 2 - Repository and directory structure The discussion repeatedly emphasized ownership-based layout. Top-level folders should answer "what

architectural layer owns this?" rather than "what topic is this?" The target roots include apps, engine, game, runtime, contracts, content, docs, tests, tools, release, archive, scripts, cmake, external, and possibly sdk/examples. The user later pasted a report saying top-level structure is largely clean, but residual issues remain: schema taxonomy, pack internal layout, runtime/engine compatx, project graph, serialization/session placement, AIDE state classification, and report-bundle integrity. This should feed the repo-layout and governance chapters of a spec book.

Topic 3 - Renderer architecture and software renderer The chat clarified that renderer backends and drawing primitives are separate. **software** means a CPU framebuffer renderer useful for tests, fallback, deterministic screenshots, and a proof that UI/game code does not require GPU APIs. **vector2d** should not be a renderer; it is drawing/canvas functionality. After the C17/C++17 and system-floor change, OpenGL 3.3 became the preferred first direct hardware renderer, D3D11 second, Metal/Vulkan later, and old fixed-function/old DirectX deferred.

Topic 4 - Platform/provider architecture The strongest synthesis was service-first/provider-backed/profile-selected/contract-governed/third-party-fenced.

Services are Dominion-owned: platform, input, render, draw, audio, asset, script, UI, storage, diagnostics. Providers implement them: null, raylib, rlgl, rlsw, SDL2, raudio, Lua, later OpenGL33/D3D11/Metal/Vulkan/native host providers. Profiles select provider combinations. Third-party types must not cross into engine, game, contracts, content, saves, replays, packs, or public SDK.

Topic 5 - Language and compatibility baseline

The user stated that the project had transitioned to C17 and C++17 and raised minimum systems to Windows 7 SP1, Mac OS X 10.9.5, and Linux. The visible conclusion was to use C17 for ABI-facing law-like structures, packets, fixed-width types, deterministic math, save/replay records, and low-level facades; C++17 for runtime machinery, provider implementations, Workbench, apps, resource ownership, and orchestration. Public boundaries remain C-compatible and POD/versioned.

Topic 6 - Workbench as UI authoring environment

Workbench should be the tool that designs and edits rendered GUI, TUI, HUD, widget, theme, and workspace artifacts. It should edit Dominion documents and patches, not provider-specific widgets. It should support View Studio, Layout Studio, Control Catalog, Theme Lab, TUI Studio, Rendered GUI Studio, Preview

Matrix, and Validation/Evidence panels. The first proof should not be a full interface editor; it should be a practical dashboard such as validation summary or node/body status shown across projections.

Topic 7 - Shared UI/UX system CLI, TUI, rendered GUI, native GUI, headless, and immersive/VR should be projections of the same command/action/view/document system. The UI is not just buttons and panels; the reusable truth is commands, results, refusals, diagnostics, evidence, actions, views, documents, patches, and capabilities. This lets menus, setup, launcher, Workbench, server admin, client HUD, and robot OS shells share one architecture.

Topic 8 - Robotic seed-civilisation game design
The user explicitly defined the core game direction. Players are robots/machine consciousnesses seeded by a mothership. The game simulates energy, materials, machines, logistics, sensors, permissions, fields, construction, heat, maintenance, and machine graphs rather than human workers. The mothership gives knowledge and precision but not scale. Players build industrial chains, remote spawn labs, cities, and eventually terraforming/megaproject infrastructure. Nanotech is an actuator, not magic. Machine assemblies compile into functional process graphs.

Topic 9 - Planet generation, fields, and

terraforming The game should use star-system/planet generation, not initially a "galaxy generator." It should support forward generation from physical parameters and reverse generation from desired gameplay/planet characteristics. A common field-layer stack should support procedural generation, real data, overrides, mod packs, terraforming, construction, damage, and temporary previews. Gameplay layers must conserve matter/energy where appropriate; editor/worldgen layers need not.

Topic 10 - Robot OS, diegetics, and epistemics

The final UX direction is that the player boots into an OS, connects to nodes/universes, downloads consciousness into fabricated bodies, and operates through sensor/communication networks. The UI should reflect what the player's body, faction, node, or sensor network can know. Facts should have metadata such as source, confidence, observation tick, network path, latency, and permission. Fog-of-war becomes diegetic and anti-cheat relevant.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted a clean, future-proof architecture for Dominion's GUI, binaries, platform support, renderer support, native apps, and

directory structure. Later explicit goals included deep portability, modularity, extensibility, full replaceability of code/data/files/directories, reusable Domino engine code, a robot seed-civilisation game direction, raylib/SDL/Lua use as providers, and OS-like fully customizable UI/UX.

4.2 Inferred Goals INFERENCE: The user wants to turn Dominium into a platform-level project whose architecture can survive decades of refactors, ports, new games, new providers, and evolving standards. The user also wants future assistants to stop re-litigating already-settled principles and instead build upon the emerging doctrine.

4.3 Goals That Changed Over Time The GUI/binary goal started around native GUIs and compatibility lanes, then shifted to ownership-based repo convergence, then to renderer/provider matrices, then to Workbench/UI authoring, then to robot OS diegetics. The project language baseline changed from older C89/C++98 framing to C17/C++17. The game design changed from a broad MMO with invention/industry/logistics/trade to a more specific robotic seed-civilisation premise.

4.4 Goals Still Unresolved The exact first playable slice remains unresolved. The first provider wedge is clear, but exact sequence between client/workbench gameplay, Robot OS, and gameplay mechanics still needs a formal milestone. The exact Lua version, Linux baseline, current repo status, and external library/version support need verification. The exact degree of Unreal involvement remains unclear.

5. Decisions Made and Why

Decision overview table

Row 1	ID: DECISION-01; Decision: Architectural direction is service-first, provider-backed, profile-selected, contract-governed, third-party-fenced.; Status: accepted by user / current doctrine; Why it mattered: Allows raylib/SDL/Lua acceleration without architecture lock-in.; Confidence: 4; Label: FACT
Row 2	ID: DECISION-02; Decision: Use C17 + C++17 as the mainline language baseline, not C89/C++98, pure C99, or pure C++11.; Status: accepted by user; Why it mattered: Modernizes implementation while preserving C-compatible boundaries.; Confidence: 4; Label: FACT
Row 3	ID: DECISION-03; Decision: Minimum first-wave systems are Windows 7 SP1+, Mac OS X 10.

9.5+, and Linux, with 64-bit x86_64/ARM64 prioritized.; Status: accepted by user, external specifics unverified; Why it mattered: Defines what renderers, OS APIs, and toolchains matter first.; Confidence: 4; Label: FACT/VERIFY

Row 4 ID: DECISION-04; Decision: OpenGL 3.3 should be the first direct hardware-rendering target before Direct3D 11.; Status: accepted direction; Why it mattered: Keeps early rendering cross-platform and shader-first.; Confidence: 4; Label: FACT/INFERENCE

Row 5 ID: DECISION-05; Decision: OpenGL 1.1, OpenGL 2.1, and Direct3D 5/7/8/9 are deferred indefinitely/back-port research rather than first-wave targets.; Status: accepted by user; Why it mattered: Reduces architectural burden and lets project focus on shader graphics.; Confidence: 4; Label: FACT

Row 6 ID: DECISION-06; Decision: vector2d should not be a renderer backend; it should become a drawing/canvas feature layer.; Status: assistant recommendation apparently accepted by continued discussion; Why it mattered: Prevents backend taxonomy confusion.; Confidence: 3; Label: INFERENCE

Row 7 ID: DECISION-07; Decision: Use raylib

ecosystem heavily now as seed providers, not as architecture.; Status: accepted by user through subsequent synthesis; Why it mattered: Enables visible progress without letting third-party types become law.; Confidence: 4; Label: FACT/INFERENCE

Row 8 ID: DECISION-08; Decision: Keep Dominion apps generic and select provider combinations through profiles, not app-directory variants.; Status: accepted in refined synthesis; Why it mattered: Preserves modularity and replaceability.; Confidence: 4; Label: FACT/INFERENCE

Row 9 ID: DECISION-09; Decision: Workbench should author Dominion UI documents, not raygui/raylib objects directly.; Status: accepted by sustained discussion; Why it mattered: Keeps UI customization reusable across CLI/TUI/rendered/native/immersive shells.; Confidence: 4; Label: INFERENCE

Row 10 ID: DECISION-10; Decision: Dominion should become a robotic seed-civilisation game, not a labour-management game.; Status: explicit user statement; Why it mattered: Sets the gameplay, UI diegetics, fog-of-war, spawning, automation, and world simulation model.; Confidence: 5; Label: FACT

Row 11 ID: DECISION-11; Decision: The UI should feel like a customizable robot operating system, with boot/node/body/HUD/workbench shells sharing code and artifacts.; Status: explicit user statement + accepted synthesis; Why it mattered: Unifies diegetics, menu/HUD continuity, Workbench, TUI/CLI/rendered/VR shells.; Confidence: 5; Label: FACT/INFERENCE

Row 12 ID: DECISION-12; Decision: Setup/launcher should allow user choice only through verified compatible component profiles, not arbitrary unchecked binaries.; Status: accepted direction; Why it mattered: Supports customization without compatibility fraud or unsafe combinations.; Confidence: 3; Label: INFERENCE

Decision explanations DECISION-01 through DECISION-12 above are the decision set that should carry forward. The strongest accepted formulation is service-first, provider-backed, profile-selected, contract-governed, and third-party-fenced. This decision matters because it reconciles the user’s desire to use raylib/SDL/Lua now with the deeper requirement that Dominion remain portable and replaceable. It supersedes vendor-shaped architecture.

The language baseline decision matters because it changes what old renderers and toolchains matter. C17/1387

C++17 reduces the need to design around C89/C++98 constraints but does not remove the need for C-compatible ABI boundaries. The system floor decision similarly moves many older renderers and OS targets into research/back-port categories.

The Robot OS and robotic seed-civilisation decisions are the most important design decisions. They are explicit user statements, not merely assistant proposals. They should be formalized as product/game requirements. They affect UI, fog-of-war, spawning, respawn, automation, industry, tutorial/planner design, and MMO anti-cheat.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

Row 1 ID: REJECTED-01; Option: One universal GUI framework defines the product family.; Status: rejected/superseded; Reason: Risk of framework capture and poor old-host/native fit.; Final or tentative?: likely final; Reconsider conditions: Reconsider only if project narrows to one platform/toolkit.; Related workstream: WORKSTREAM-05; Label: FACT

Row 2 ID: REJECTED-02; Option: One GUI per OS version.; Status: rejected; Reason: Creates combinatorial explosion.; Final or tentative?: likely

final; Reconsider conditions: Only reconsider for small high-value host lanes.; Related workstream: WORKSTREAM-05; Label: FACT

Row 3 ID: REJECTED-03; Option: Content/domains as default destination for all domain material.; Status: rejected; Reason: Domain material includes code, schemas, registries, docs, packs, fixtures.; Final or tentative?: final; Reconsider conditions: Only use content for authored data/packs/assets.; Related workstream: WORKSTREAM-01; Label: FACT

Row 4 ID: REJECTED-04; Option: vector2d as a render backend.; Status: superseded; Reason: It is drawing/canvas functionality, not backend equivalent to OpenGL/D3D.; Final or tentative?: final for first-wave; Reconsider conditions: Could exist as feature name, not renderer backend.; Related workstream: WORKSTREAM-04; Label: INFERENCE

Row 5 ID: REJECTED-05; Option: OpenGL 1.1/2.1 and Direct3D 5/7/8/9 as first-wave renderers.; Status: deprioritized; Reason: New floors allow shader-first OpenGL 3.3 and D3D11.; Final or tentative?: temporary/back-port possible; Reconsider conditions: Reconsider for explicit old-host back-port.; Related workstream:

WORKSTREAM-04; Label: FACT

Row 6 ID: REJECTED-06; Option: raylib as the game architecture.; Status: rejected; Reason: Would contaminate core contracts/saves/replays/game law.; Final or tentative?: final; Reconsider conditions: raylib remains provider suite.; Related workstream: WORKSTREAM-02; Label: FACT/ INFERENCE

Row 7 ID: REJECTED-07; Option: Pure C99 or pure C++11 pivot.; Status: rejected; Reason: C17/ C++17 gives better machinery while preserving C-compatible ABI.; Final or tentative?: likely final; Reconsider conditions: Reconsider only if host/ toolchain constraints radically change.; Related workstream: WORKSTREAM-03; Label: FACT

Row 8 ID: REJECTED-08; Option: Human worker/ labour-management core game.; Status: rejected by user; Reason: User chose robotic seed civilisation; no human labour simulation.; Final or tentative?: final for current design; Reconsider conditions: Could appear only as later narrative/ecology layer, not core.; Related workstream: WORKSTREAM-07; Label: FACT

Row 9 ID: REJECTED-09; Option: Workbench edits provider-specific UI objects as source of truth.; Status: rejected; Reason: Would lock UI to raygui/

raylib/native toolkit.; Final or tentative?: final;
Reconsider conditions: Workbench edits Dominionum
documents and projections.; Related workstream:
WORKSTREAM-05; Label: INFERENCE

7. Important Reasoning, Rationale, and Tradeoffs

The central tradeoff was speed versus purity. Raylib, SDL2, Lua, raygui, rlgl, rlsw, and related libraries can accelerate visible progress, but if they leak into game law, UI documents, saves, replays, or public contracts, they become architectural capture. The chosen compromise is to use them as providers behind Dominionum-owned services and conformance tests.

A second tradeoff was breadth of compatibility versus first-wave progress. Earlier ambitions included very old Windows, old OpenGL, old DirectX, old macOS, 32-bit, and even exotic/retro systems. After the C17/C++17 and system-floor shift, these became back-port/research lanes. The project still preserves modularity for them, but it does not let them dictate the first renderer or UI architecture.

A third tradeoff was deep simulation versus playability. The seed-civilisation design can explode into full physics, ecology, climate, fluid, heat, economics, logistics, and AI. The visible rationale was to start with systems that produce gameplay leverage: field layers, cut/fill,

material/energy-constrained construction, machine graphs, logistics, heat, maintenance, permissions, and fog-of-war. Full groundwater, ecology, dust storms, and atmospheric chemistry can come later.

A fourth tradeoff was customization versus trust. The user wants fully customizable UI/UX like an OS. That is compatible with MMO integrity only if the client can style, arrange, and summarize known information but cannot reveal hidden data, bypass permissions, hide critical warnings, or automate unauthorized actions.

8. Plans, Future Work, and Next Steps

The immediate technical plan is PROVIDER-WEDGE-01: add provider manifest schema, third-party manifests, forbidden include validator, null providers, and raylib/rlgl/rlsw/raygui/raudio/asset providers. This produces a client window, Workbench viewport, simple drawing, audio smoke, asset preview, frame export, and ensures server/headless/replay still work without raylib.

The immediate design plan is ROBOT-OS-WEDGE-01: define command/action/view/document/patch/UI contracts and build one shared `view.node.status` projected as command output, text panel, rendered OS screen, and body HUD panel. This proves menu/HUD/Workbench continuity.

The immediate game-design plan is to formalize the

robotic seed-civilisation doctrine into a spec chapter and choose a first gameplay loop, likely survey -> mine -> refine -> fabricate -> build simple power/logistics/spawn infrastructure.

Recommended next-action sequence:

1. Verify and pin external library/toolchain facts.
2. Create/update provider manifest and forbidden include validators.
3. Implement null providers and raylib provider wedge.
4. Clean component/render/provider matrices.
5. Build `view.node.status` as Robot OS proof.
6. Formalize game design pillars into a spec.
7. Design UI customization/security constraints.
8. Define first playable vertical slice.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences

Row 1 ID: PREF-01; Preference: Prefers rigorous, source-grounded, audit-ready answers with explicit uncertainty.; Area: communication; Explicit/
1393

inferred/uncertain: explicit; Strength: high;
Practical implication: Use citations where possible;
mark FACT/INFERENCE/VERIFY.; Risk if
wrong: Loss of trust if assistant overstates.; Label:
FACT

Row 2 ID: PREF-02; Preference: Prefers intuitive,
literal, modular names; dislikes vague labels like
compat/universal/ugly compound strings.; Area:
naming; Explicit/inferred/uncertain: explicit;
Strength: high; Practical implication: Use
platform/input/render/provider/profile terms;
avoid status words as IDs.; Risk if wrong:
Architecture taxonomy becomes confusing.; Label:
FACT

Row 3 ID: PREF-03; Preference: Prefers long-term
portability, modularity, extensibility, and
replaceability over quick one-off indie design.; Area:
architecture; Explicit/inferred/uncertain: explicit;
Strength: high; Practical implication: Favor
contracts, provider boundaries, manifests, tests,
matrices.; Risk if wrong: Project becomes one-off
and hard to refactor.; Label: FACT

Row 4 ID: PREF-04; Preference: Prefers native/
OEM+ OS apps and CLI/TUI support across
products.; Area: UX/platform; Explicit/inferred/
uncertain: explicit; Strength: high; Practical

implication: Setup/launcher/server/tools should have command/text/native where appropriate.; Risk if wrong: OS integration feels inconsistent.; Label: FACT

Row 5 ID: PREF-05; Preference: Prefers broad, deep strategic synthesis over narrow answers.; Area: conversation style; Explicit/inferred/uncertain: explicit; Strength: high; Practical implication: Explain rationale, tradeoffs, risks, and alternatives. ; Risk if wrong: User may need to re-explain context.; Label: FACT

Row 6 ID: PREF-06; Preference: Prefers human-readable reports before machine-readable registers.; Area: preservation; Explicit/inferred/uncertain: explicit; Strength: high; Practical implication: Produce prose first, registers after.; Risk if wrong: Machine dump fails preservation goal.; Label: FACT

Row 7 ID: PREF-07; Preference: Wants downloadable files/ZIP when tools are available.; Area: artifact/export; Explicit/inferred/uncertain: explicit; Strength: high; Practical implication: Create markdown/yaml/zip package.; Risk if wrong: Need to manually copy long output.; Label: FACT

Row 8 ID: PREF-08; Preference: Wants UI/UX to feel like an OS, with deep customization and diegetic

robot epistemics.; Area: game UX; Explicit/
inferred/uncertain: explicit; Strength: high;
Practical implication: Design robot OS shells,
themes, HUDs, Workbench UI docs.; Risk if wrong:
Generic game UI breaks concept.; Label: FACT

9.2 Inferred Constraints and Preferences The user appears to prefer source layouts and naming that remain meaningful years later. They dislike vendor-shaped, framework-shaped, and status-word architecture. Future assistants should not propose "just use X framework" as the answer. They should instead map libraries to providers, profiles, and contracts.

9.3 Uncertain or Unestablished Preferences It remains uncertain how much the user wants Unreal in the near-term after raylib-first discussion. It also remains uncertain whether Lua 5.4 or 5.5 is preferred; the user appears to value pinning and stability more than "latest" for script ABI.

10. Files, Artifacts, Outputs, and Prompts

Row 1 ID: ARTIFACT-01; Artifact / file / prompt /
output: Transfer Knowledge Base - GUI and
Binary Rebuild; Type: pasted prior-chat brief;
Purpose: Provided initial context for GUI/binary/
product/platform doctrine.; Status: preserve;

Origin: user paste in chat; Carry forward?: yes;
Notes: Contains architecture/non-negotiables/open
questions; some later superseded.; Label: FACT

Row 2 ID: ARTIFACT-02; Artifact / file / prompt /
output: Live repo reference `julesc013/dominium`;
Type: repository reference; Purpose: Established
latest docs/code baseline for repo-aware discussion.
; Status: preserve; Origin: user statement +
assistant repo lookups; Carry forward?: yes; Notes:
Current repo status may have changed; verify
before implementing.; Label: FACT/VERIFY

Row 3 ID: ARTIFACT-03; Artifact / file / prompt /
output: Repo/layout and component matrix
excerpts; Type: assistant retrieved repo files in
chat; Purpose: Supported claims about AppShell,
component matrix, layout target, CMake C17/
C++17.; Status: preserve as context; Origin: api_
tool/GitHub connector in prior turns; Carry
forward?: yes; Notes: Repo-source evidence should
be refreshed for implementation.; Label:
PROJECT-CONTEXT/VERIFY

Row 4 ID: ARTIFACT-04; Artifact / file / prompt /
output: Pasted raylib/SDL/Lua provider synthesis
transcript; Type: user-pasted prior chat material;
Purpose: Captured service-first provider
architecture and third-party fencing.; Status:

preserve; Origin: user paste; Carry forward?: yes;
Notes: External version/support claims need
verification.; Label: FACT/VERIFY

Row 5 ID: ARTIFACT-05; Artifact / file / prompt /
output: Pasted game design direction document;
Type: user-pasted design doctrine; Purpose:
Defines robotic seed-civilisation premise, planet
generation, field layers, mothership, nanotech,
machine graphs, fog of war.; Status: preserve;
Origin: user paste; Carry forward?: yes; Notes:
Should be formalized into game-design spec.;
Label: FACT

Row 6 ID: ARTIFACT-06; Artifact / file / prompt /
output: Pasted preservation prompt file; Type:
uploaded file; Purpose: Defines this preservation
task and output package requirements.; Status:
preserve; Origin: uploaded Pasted text.txt; Carry
forward?: yes; Notes: Primary task spec for this
response.; Label: FACT

Row 7 ID: ARTIFACT-07; Artifact / file / prompt /
output: Generated preservation Markdown/
YAML/ZIP package; Type: generated files;
Purpose: This response's export package for later
reading and aggregation.; Status: new; Origin:
assistant-generated; Carry forward?: yes; Notes:
Files linked at end.; Label: FACT

Row 8 ID: ARTIFACT-08; Artifact / file / prompt / output: Renderer naming and vector2d discussion; Type: conversation output; Purpose: Clarified software renderer, drawing/canvas, null, OpenGL/D3D/Metal/Vulkan roles.; Status: preserve; Origin: this chat; Carry forward?: yes; Notes: Convert to renderer matrix cleanup task.; Label: INFERENCE

Row 9 ID: ARTIFACT-09; Artifact / file / prompt / output: Workbench/UI authoring architecture; Type: conversation output; Purpose: Explains how Workbench edits views/layouts/widgets/themes/documents and previews projections.; Status: preserve; Origin: this chat; Carry forward?: yes; Notes: Forms UI platform spec foundation.; Label: INFERENCE

Row 10 ID: ARTIFACT-10; Artifact / file / prompt / output: Robot OS UI/UX synthesis; Type: conversation output; Purpose: Defines boot/node/body/HUD/Workbench shells and diegetic epistemics.; Status: preserve; Origin: this chat; Carry forward?: yes; Notes: Forms UX spec foundation.; Label: FACT/INFERENCE

11. Open Questions and Unresolved Issues

- Row 1** ID: QUESTION-01; Question / issue: Which Lua provider version should be pinned first: lua54 or lua55?; Why it matters: Script ABI stability and support.; Known information: Lua should be provider behind dominium.script.v1; versions differ.; Unknown information: Which version best balances maturity and future features.; Resolution path: Verify current Lua status and choose explicitly.; Priority: P1; Related workstream: WORKSTREAM-02; Label: VERIFY
- Row 2** ID: QUESTION-02; Question / issue: Should raylib be vendored under external/upstream or pulled as submodule/package?; Why it matters: Reproducibility and patch control.; Known information: Preferred structure says external/upstream with licenses/manifests/patches.; Unknown information: Exact supply-chain method.; Resolution path: Define third-party dependency policy.; Priority: P1; Related workstream: WORKSTREAM-02; Label: UNCERTAIN
- Row 3** ID: QUESTION-03; Question / issue: How much of raygui should be used for Workbench before first-party UI controls mature?; Why it matters: Avoid raygui lock-in while moving fast.; Known information: Raygui is early provider, not

UI law.; Unknown information: When to replace or supplement it.; Resolution path: Build Workbench proof then assess limits.; Priority: P1; Related workstream: WORKSTREAM-05; Label: INFERENCE

Row 4 ID: QUESTION-04; Question / issue: What exact first-wave Linux baseline is targeted?; Why it matters: Toolchain/glibc/windowing compatibility. ; Known information: Linux is in floor; x11/wayland custom providers deferred.; Unknown information: Distro/glibc/minimum compiler assumptions.; Resolution path: Create platform matrix and verify toolchain support.; Priority: P0; Related workstream: WORKSTREAM-03; Label: UNCERTAIN

Row 5 ID: QUESTION-05; Question / issue: How should official MMO UI customization restrict scripts/macros?; Why it matters: Anti-cheat and fairness.; Known information: Customization allowed; hidden automation forbidden.; Unknown information: Specific server policy and sandbox model.; Resolution path: Threat model and UI security spec.; Priority: P0; Related workstream: WORKSTREAM-06; Label: INFERENCE

Row 6 ID: QUESTION-06; Question / issue: What is the first gameplay vertical slice for robotic seed-

1401

civilisation?; Why it matters: Implementation order.; Known information: User defined many mechanics.; Unknown information: Minimal playable loop still unchosen.; Resolution path: Choose first loop: survey -> mine -> refine -> fabricate -> build spawn node?; Priority: P0; Related workstream: WORKSTREAM-07; Label: UNCERTAIN

Row 7 ID: QUESTION-07; Question / issue: Should Unreal remain part of the visible-local-world plan or is raylib-first enough for near term?; Why it matters: Visual tech stack and scope.; Known information: User pasted Unreal role in game design transcript; later focus shifted to raylib providers.; Unknown information: Current commitment to Unreal is unclear.; Resolution path: Decide whether Unreal is future renderer/provider, external visual client, or deprioritized.; Priority: P1; Related workstream: WORKSTREAM-04; Label: UNCERTAIN

Row 8 ID: QUESTION-08; Question / issue: How should robot consciousness backup, disconnect, and body persistence work on official servers?; Why it matters: Core UX, fairness, and diegetic epistemics.; Known information: User proposed consciousness on ice and speed-of-light mesh nodes. ; Unknown information: Exact gameplay and anti-

abuse rules.; Resolution path: Design session/
backup/body-link spec.; Priority: P0; Related
workstream: WORKSTREAM-06; Label:
INFERENCE

Row 9 ID: QUESTION-09; Question / issue: Which
repo residual taxonomy items are actually still
present after latest repo changes?; Why it matters:
Avoid acting on stale tree reports.; Known
information: User pasted a report claiming
structure mostly clean with residual warnings.;
Unknown information: Current live repo state not
fully revalidated in this task.; Resolution path:
Run current tree/report integrity checks.; Priority:
P1; Related workstream: WORKSTREAM-01;
Label: VERIFY

**12. Risks, Failure Modes, and What Future Chats
Might Get Wrong**

Row 1 ID: RISK-01; Risk / failure mode: Future
assistant treats assistant suggestions as accepted
decisions.; Consequence: False spec requirements.;
Likelihood: medium; Severity: high; Mitigation:
Use labels and decision statuses; cite user
acceptance where clear.; Related workstream: all;
Label: FACT

Row 2 ID: RISK-02; Risk / failure mode: Third-party

provider leakage into engine/game/contracts/content.; Consequence: Vendor lock-in, broken saves/replays, hard rewrites.; Likelihood: medium; Severity: high; Mitigation: Forbidden include validator, provider manifests, type translation boundaries.; Related workstream: WORKSTREAM-02; Label: INFERENCE

Row 3 ID: RISK-03; Risk / failure mode: UI customization becomes cheating or hides critical warnings.; Consequence: MMO unfairness, griefing, bad UX safety.; Likelihood: medium; Severity: high; Mitigation: Protected alert classes, sandbox rules, server-side authority, UI security spec.; Related workstream: WORKSTREAM-06; Label: INFERENCE

Row 4 ID: RISK-04; Risk / failure mode: Raylib proof becomes permanent architecture accidentally.; Consequence: Provider lock-in.; Likelihood: medium; Severity: medium-high; Mitigation: Provider conformance tests and planned replacement path.; Related workstream: WORKSTREAM-02; Label: INFERENCE

Row 5 ID: RISK-05; Risk / failure mode: Robot OS diegetic UI overreaches before core gameplay exists.; Consequence: Scope creep.; Likelihood: medium; Severity: medium; Mitigation: Build

wedge with view.node.status and small modules first.; Related workstream: WORKSTREAM-06; Label: INFERENCE

Row 6 ID: RISK-06; Risk / failure mode: Old compatibility/back-port lanes distract from C17/C++17 first-wave.; Consequence: Delayed progress.; Likelihood: medium; Severity: medium; Mitigation: Matrix phases: first-wave vs research/back-port.; Related workstream: WORKSTREAM-03; Label: FACT/INFERENCE

Row 7 ID: RISK-07; Risk / failure mode: Game design becomes over-simulated too early.; Consequence: Performance/design paralysis.; Likelihood: medium; Severity: high; Mitigation: Start with simplified fields, cut/fill, machine graph compiler; add fluids/climate later.; Related workstream: WORKSTREAM-07; Label: INFERENCE

Row 8 ID: RISK-08; Risk / failure mode: Fog-of-war and epistemics are implemented only visually, not server-authoritatively.; Consequence: Cheating and lore breakage.; Likelihood: medium; Severity: high; Mitigation: Server must constrain known data; UI metadata tracks source/confidence/staleness.; Related workstream: WORKSTREAM-06; Label: INFERENCE

Row 9 ID: RISK-09; Risk / failure mode: Repo reports

are stale or inconsistent.; Consequence: Wrong cleanup actions.; Likelihood: medium; Severity: medium; Mitigation: Verify current repo tree and report-integrity pipeline.; Related workstream: WORKSTREAM-01; Label: VERIFY

Row 10 ID: RISK-10; Risk / failure mode: Preservation package over-compresses nuance.; Consequence: Loss of context for future spec book.; Likelihood: low-medium; Severity: medium; Mitigation: Human report + registers + spec sheet + aggregator packet created.; Related workstream: WORKSTREAM-10; Label: FACT

13. What This Chat Contributes to the Larger Project or Future Spec Book

Row 1 Spec-book area: Architecture doctrine; Contribution from this chat: Service-first/provider-backed/profile-selected/contract-governed/third-party-fenced runtime architecture.; Source IDs: DECISION-01, WORKSTREAM-02; Should become requirement/context/open issue?: requirement; Confidence: 4; Notes: Central project architecture.

Row 2 Spec-book area: Repository layout; Contribution from this chat: Ownership-based source tree and residual taxonomy cleanup.; Source

IDs: WORKSTREAM-01, TASK-13; Should become requirement/context/open issue?: requirement/open issue; Confidence: 3; Notes: Verify current repo before merging.

Row 3 Spec-book area: Renderer strategy; Contribution from this chat: null/software/raylib/rlsw/rlgl/opengl33/direct3d11/metal/vulkan staged renderer plan.; Source IDs: DECISION-04-07, WORKSTREAM-04; Should become requirement/context/open issue?: requirement/open issue; Confidence: 4; Notes: External support claims need verification.

Row 4 Spec-book area: Language/toolchain; Contribution from this chat: C17+C++17 mainline, C ABI boundaries, Windows 7 SP1/macOS 10.9.5/Linux floor.; Source IDs: DECISION-02-03; Should become requirement/context/open issue?: requirement with verification; Confidence: 4; Notes: Formal matrix update needed.

Row 5 Spec-book area: UI platform; Contribution from this chat: Command/action/view/document/projection spine across CLI/TUI/rendered/native/immersive.; Source IDs: DECISION-09, WORKSTREAM-05-06; Should become requirement/context/open issue?: requirement;

Confidence: 4; Notes: Core for Workbench and Robot OS.

Row 6 Spec-book area: Game design; Contribution from this chat: Robotic seed-civilisation, mothership, body spawning, nanotech as actuator, field layers, machine graphs.; Source IDs: DECISION-10, WORKSTREAM-07; Should become requirement/context/open issue?: requirement/context; Confidence: 5; Notes: Largest design contribution from chat.

Row 7 Spec-book area: Diegetic epistemics; Contribution from this chat: Sensor/source/confidence/staleness/latency/permitted data constrains UI.; Source IDs: WORKSTREAM-06, CONSTRAINT-07; Should become requirement/context/open issue?: requirement; Confidence: 4; Notes: Ties lore to anti-cheat.

Row 8 Spec-book area: Third-party dependencies; Contribution from this chat: raylib/SDL2/Lua provider manifest/fencing/conformance.; Source IDs: WORKSTREAM-02, TASK-01-04; Should become requirement/context/open issue?: requirement/open issue; Confidence: 4; Notes: Requires license verification.

Row 9 Spec-book area: Spec aggregation; Contribution from this chat: This chat preservation report and

registers.; Source IDs: WORKSTREAM-10; Should become requirement/context/open issue?: context; Confidence: 5; Notes: Can be merged with other chat reports.

14. What I Should Remember

- The strongest architecture phrase is: service-first, provider-backed, profile-selected, contract-governed, third-party-fenced, evidence-tested.
- Dominium should not become a raylib game; raylib is a seed provider suite.
- SDL2 remains a useful platform/input/audio provider.
- Lua should be pinned behind `dominium.script.v1`.
- C17/C++17 is the current mainline language baseline; C-compatible ABI boundaries still matter.
- OpenGL 3.3 first, D3D11 second, Metal/Vulkan later; old OpenGL/DirectX lanes are research/back-port.
- Workbench authors Dominium documents, views, widgets, layouts, themes, and workspaces, not provider-specific UI objects.

- The final game direction is robotic seed civilisation, not labour management.
- The UI should feel like a customizable robot OS whose HUD, menus, TUI, CLI, Workbench, and future VR shell are projections of the same system.
- The most important immediate next task is provider/Robot OS wedge planning with validators and manifests.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

1. "Explain the difference between Dominion services, providers, profiles, and apps."
2. "Summarize the final renderer plan after the C17/C++17 transition."
3. "Explain the Robot OS idea in terms of boot shell, node shell, body shell, and HUD."

15.2 Decisions

1. "Which decisions in this chat were explicit user decisions versus assistant synthesis?"
2. "Which renderer/platform decisions are final, and which are still provisional?"

3. "Which parts of the robotic seed-civilisation design are hard requirements?"

15.3 Tasks and Next Actions

1. "Turn PROVIDER-WEDGE-01 into a Codex-ready implementation task."
2. "Turn ROBOT-OS-WEDGE-01 into a file-by-file spec."
3. "Create the first gameplay vertical slice plan for survey -> mine -> fabricate -> build spawn lab."

15.4 Artifacts and Files

1. "List all files created by the preservation package and what I should do with them."
2. "Extract the game-design portion into a standalone spec chapter."
3. "Extract the provider architecture into a standalone technical design document."

15.5 Risks and Verification

1. "Make a verification checklist for raylib, SDL2, Lua, Windows 7 SP1, macOS 10.9.5, and OpenGL 3.3."

2. "Threat-model UI customization for official MMO servers."

15.6 Future Spec Book / Aggregation

1. "Map this chat's registers into Project Spec Book chapters."
2. "Identify likely conflicts with other chats' renderer/platform plans."

15.7 Deep-Dive Questions Specific to This Chat

1. "Design the UI document schema for a customizable robot OS HUD."
2. "Design the consciousness backup/disconnect/session model."
3. "Define protected alert classes that themes cannot hide."
4. "Design the machine graph compiler concept for player-built machines."

16. Compact Human Summary

This chat was one of the major convergence points for Dominion. It began from a transfer brief about GUI and binary strategy, but it grew into a broad architecture

and design foundation for the whole project. The user's recurring concern was that *Dominium* must not become a one-off indie codebase or a pile of platform-specific UI and renderer hacks. The project should be portable, modular, extensible, replaceable, and reusable: not just for *Dominium*, but for other games on the *Domino* engine and possibly other future engine/game projects.

The early discussion preserved the GUI doctrine: every product should have CLI, TUI should exist or be stubbed deterministically, GUIs are optional/modular, and the product logic must remain behind stable backend contracts. Setup and launcher can use OS-native/OEM+ GUI shells; client is rendered-first; server is headless/CLI/TUI first; tools are case-by-case. This was tied to the repo structure problem: the source tree should be organized by architectural ownership, not by topics or historical accidents. The desired top-level roots are apps, engine, game, runtime, contracts, content, docs, tests, tools, release, archive, scripts, cmake, external, and later sdk/examples if needed.

The renderer discussion evolved significantly. Initially there were old lanes like OpenGL 1.x/2.x and Direct3D 9, but after the user stated that the project had moved to C17/C++17 and now targets Windows 7 SP1, Mac OS X 10.9.5, and Linux, the first-wave plan changed. Old fixed-function renderers and old DirectX should be deferred to research/back-port lanes. OpenGL 3.3

should be first because it supports early portable shader work across Windows/Mac/Linux, then Direct3D 11 for Windows, then Metal, Vulkan, and Direct3D 12 later. A key correction was that `vector2d` should not be a renderer backend; it is drawing/canvas functionality. Software rendering remains useful as a CPU framebuffer/reference/test/fallback path.

The strongest architecture synthesis was the service/provider/profile model. Dominion owns services and contracts: platform, input, render, draw, audio, asset, script, UI, storage, diagnostics, commands, actions, views, documents, patches, saves, replays, packs, profiles, and provider law. Third-party libraries implement providers. Raylib, rlgl, rlsw, raygui, raudio, rtextures, rmodels, SDL2, and Lua can be used aggressively, but only as replaceable provider implementations. Their types must not leak into engine, game, contracts, content, saves, replays, packs, game law, or public SDK. Apps remain generic; profiles select provider combinations.

The Workbench and UI discussion established that CLI, TUI, rendered GUI, native GUI, headless, and future immersive/VR interfaces should not be separate UI systems. They should be projections of one command/action/view/document model. Workbench becomes the authoring environment for UI artifacts: widgets, layouts, themes, TUI panels, rendered screens, HUDs,

workspaces, preview fixtures, and UI packs. It edits Dominion documents and patches, then validates and previews them through providers.

The biggest design decision came near the end: Dominion should become a robotic seed-civilisation game, not a labour-management game. Players are robot avatars or machine consciousnesses. A mothership contains human knowledge, starter resources, spawn fabrication, and AI planning, but is limited by feedstock, energy, heat, rare materials, safety protocols, and throughput. Gameplay is exploration, surveying, mining, refining, fabrication, power, logistics, sensors, permissions, spawn labs, industrial cities, terraforming, and megaprojects. Nanotech is a construction actuator, not free matter or magic. Machines compile into process graphs. Fog-of-war is diegetic because robots only know what their sensors, satellites, relays, and faction networks know.

The final UI/UX synthesis connects directly to that premise. The interface should feel like a robot operating system. The player boots into an OS, connects to a universe, downloads consciousness into a fabricated robot body, and disconnects into backup/on-ice state. Menus, Workbench, HUD, CLI, TUI, GUI, and future VR shell are all projections of that OS. Users can customize themes, layouts, shells, HUDs, terminal styles, workspaces, and sensor overlays, but customization must

not reveal hidden data, hide critical warnings, bypass permissions, or automate illegal actions.

The best next action is to implement the first provider and Robot OS wedges: provider manifest schema, forbidden include validator, null providers, raylib/rlgl/rlsw/raygui/raudio providers, client/workbench/server profiles, and one shared `view.node.status` projected into command, text, rendered OS screen, and HUD.

28.2 **Dominium Architecture Ui Providers Robot Os Strategy Reader Brief**

Source: docs/archive/conversations/architecture_
ui_providers/dominium_architecture_ui_
providers_robot_os_strategy__06_reader_brief.md

Report type: reader_brief

Size: 1810 bytes

28.2.1 **Reader Brief - Dominium Architecture, UI, Providers, and Robot OS Strategy**

This chat is a major Dominium convergence chat. The top 20 things to know:

1. Dominium should be a long-lived platform, not a

one-off game.

2. The core architecture is service-first, provider-backed, profile-selected, contract-governed, third-party-fenced.
3. C17/C++17 is the current mainline language baseline.
4. Public/stable boundaries should remain C-compatible, POD/versioned, and explicit.
5. First-wave floors are Windows 7 SP1+, Mac OS X 10.9.5+, and Linux, subject to verification.
6. Old renderers such as OpenGL 1.1/2.1 and Direct3D 9 are back-port/research, not first wave.
7. OpenGL 3.3 is first direct hardware renderer; Direct3D 11 follows; Metal/Vulkan later.
8. Software renderer remains important for fallback, CI, reference, and evidence.
9. `vector2d` is not a renderer backend; drawing/canvas is a feature layer.
10. `raylib`, `rlgl`, `rlsw`, `raygui`, `raudio`, `SDL2`, and `Lua` are providers, not architecture.
11. Third-party types must not leak into engine/game/contracts/content/saves/replays/packs/public SDK.

12. Apps stay generic; provider choices are profiles.
13. Workbench authors Dominion UI documents, not raygui or raylib objects.
14. CLI/TUI/rendered/native/immersive are projections of one command/action/view/document system.
15. Dominion should be a robotic seed-civilisation game.
16. Players are robots or machine consciousnesses, not human labour managers.
17. The mothership provides knowledge and precision but not infinite scale.
18. Nanotech is a construction actuator, not magic.
19. UI should feel like a customizable robot operating system.
20. Fog-of-war and UI epistemics are sensor/network/permission constrained.

Best next step: implement PROVIDER-WEDGE-01 and ROBOT-OS-WEDGE-01 specs.

28.3 Dominion Architecture Ui Providers Robot Os Strategy In Chat Reader

Source: docs/archive/conversations/architecture_ui_providers/dominium_architecture_ui_providers_robot_os_strategy__09_in_chat_reader.md

Report type: in_chat_reader

Size: 1396 bytes

28.3.1 In-Chat Reader - Dominion Architecture, UI, Providers, and Robot OS Strategy

Package overview

This package preserves the chat's substance in human-readable and machine-mergeable forms. It covers architecture, repo layout, provider strategy, renderer plan, Workbench/UI, Robot OS UX, robotic seed-civilisation design, tasks, risks, verification items, and aggregation guidance.

File index

- 00 manifest: package map.
- 01 human-readable report: sections 0-16.

- 02 context transfer packet: future-chat handoff.
- 03 spec sheet: YAML-style aggregation data.
- 04 registers: workstreams, decisions, tasks, constraints, questions, artifacts, risks, verification, timeline, spec contributions.
- 05 aggregator packet: compact merge packet.
- 06 reader brief: shorter guide.
- 07 verification and audit: self-audit and verification queue.
- 08 future chat bootstrap prompt: prompt for new chat.
- 09 in-chat reader: this guide.

Plain-English explanation

Read file 01 first if you want to understand what happened. Read file 04 if you need tasks and decisions. Read file 05 if merging into a master Project Spec Book. Read file 07 before treating external support claims as implementation truth.

Question menu

Ask about provider wedge, robot OS wedge, renderer plan, Workbench UI authoring, robotic seed-civilisation

mechanics, UI customization security, or spec-book merge.

Chapter 29

Canonical Structure And Framework

29.1 Dominion Canonical Architecture Repo Foundation Human Readable Report

Source: docs/archive/conversations/canonical_structure_and_framework/dominium_canonical_architecture_repo_foundation__01_human_readable_report.md

Report type: human_readable_report

Size: 42232 bytes

29.1.1 COMPLETE CHAT PRESERVATION REPORT - Dominion Canonical Architecture, Repository Foundation, and Provider Model

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion Canonical Architecture, Repository Foundation,
1422

and Provider Model

Row 2 Field: Date anchor; Assessment: 2026-05-27
Australia/Melbourne

Row 3 Field: Source scope; Assessment: This chat only,
except where explicitly marked PROJECT-
CONTEXT

Row 4 Field: Apparent access; Assessment: Partial

Row 5 Field: Previously generated files available?;
Assessment: Yes: prior generated Markdown/plain-
text handoff files are present in /mnt/data; older
uploaded-file context has partly expired.

Row 6 Field: Uploaded files or artifacts present?;
Assessment: Yes: the preservation prompt,
multiple directory-tree/report bundles, and prior
handoff files are available in the sandbox. Some
older uploaded files are expired or unavailable
through file search.

Row 7 Field: Contains future plans?; Assessment: Yes

Row 8 Field: Contains decisions?; Assessment: Yes,
with caveats: some were explicit user acceptances;
many were assistant recommendations that the
user explored, refined, or sent to other chats.

Row 9 Field: Contains pending tasks?; Assessment: Yes

- Row 10** Field: Contains unresolved questions?; Assessment: Yes
- Row 11** Field: Staleness risk; Assessment: Medium to High for current repo state, library version facts, and live validation results; Low for conversation-internal doctrine and user preferences.
- Row 12** Field: Extraction confidence; Assessment: 4/5 for visible conversation substance; 3/5 for exact sequence because portions of the transcript are skipped/expired.
- Row 13** Field: Safe for later aggregation?; Assessment: Yes, with caveats
- Row 14** Field: Main limitations; Assessment: The visible conversation includes skipped message markers; some uploaded files expired; live repo state reports changed repeatedly; several "current" statuses came from user-provided summaries and should be verified before acting.

Plain-language limitation note: this report reconstructs this chat from the visible transcript, the user’s embedded status summaries, and available sandbox files. It cannot guarantee perfect coverage of turns hidden behind "Skipped messages" markers or expired uploaded files. Where the chat reported repo commits, tests, and live validation statuses, those are preserved as FACTs as

reported in the chat, not independently verified current repository truth unless explicitly noted.

1. One-Page Orientation

This chat was an extended architecture, repository-structure, and project-governance conversation for the Dominion game/project. The user's core concern was that months of planning and refactoring had stalled actual development and that the repository had repeatedly grown new root directories, duplicate taxonomies, generic `src//source/` wrappers, and unclear ownership boundaries. The user wanted to force the project toward a durable, future-proof structure before building Workbench, the client, the engine, the game systems, modules, packs, renderers, and provider integrations.

The conversation evolved from immediate directory cleanup into a much larger design model. The early problem was physical: too many roots, duplicate structures, vague names, product-specific code in reusable places, pack payloads mixed with package contracts, schemas organized around old roots, tests and docs mirroring obsolete paths, and tools acting like a second source tree. The user repeatedly challenged weak or over-broad directory proposals, especially ones that reintroduced `src/`, new top-level roots, long product names, status words like `legacy/modern`, and vendor-

shaped structure.

The durable architectural answer that emerged was that the top-level root model should remain small and closed: `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `docs/`, `tests/`, `tools/`, `scripts/`, `cmake/`, `external/`, `release/`, and `archive/`, plus project/tooling roots like `.aide/`, `.github/`, and `.vscode/`. Generated/local roots such as `.dominium.local/`, `build/`, `out/`, `dist/`, `artifacts/`, and `reports/` must not become active source authority. The most important naming and ownership rule became: no new top-level roots unless a root contract intentionally allows them; no first-party `src/` or `source/`; no generic junk drawers like `core`, `shared`, `common`, `misc`, `lib`, or `data` in active source.

As the chat deepened, the model became less about folders and more about stable public surfaces. The user wanted modularity strong enough that code, directories, renderers, providers, and even large implementations could be replaced later without breaking saves, packs, mods, replays, or downstream games. The key doctrine became: **stable semantic IDs, stable public contracts, replaceable private implementations, and proof before promotion**. The assistant and user converged on "Domino" as the reusable deterministic substrate/framework layer, "Dominator" as one game/product family built on Domino, "Workbench" as the

production/editing/validation/evidence environment, and "AIDE" as the repo/control-plane harness.

Contracts define law; runtime implements services/projections/providers; apps compose products; content supplies authored payloads; tools validate/generate/migrate/audit; tests/replay/evidence prove behavior; archive preserves history without active authority.

The conversation produced many Codex/AIDE mega-prompts for concrete cleanup: runtime names, game rules, engine include boundaries, pack authority, schema canonicalization, content pack layout, apps-thin enforcement, Workbench naming, AIDE scan boundaries, tools/docs/test taxonomy cleanup, RepoX/TestX proof repair, full remediation, dependency-direction repair, provider structure, framework boundary, and full-gate legacy path routing. Several user status reports said these tasks landed with commits and passing validations, though many still ended `PASS_WITH_WARNINGS` rather than full green.

The current end-state of the chat is not "everything is perfect." It is that the old root/directory crisis is largely resolved, while full release proof, full CTest, stale generated evidence, provider implementation, presentation/projection conformance, pack internal layout, and remaining warning buckets remain future work. The best next direction identified in the chat is not more giant structure thrashing, but targeted proof

hygiene and governed product-spine work: clean stale full-gate/generated evidence, continue presentation/projection contracts, build provider conformance, and use raylib/SDL2/Lua as fenced providers rather than architecture.

2. The Story of the Conversation

2.1 From directory frustration to canonical root

doctrine The user began from intense frustration: months of planning had not produced enough actual development, and the repository still looked structurally unstable. The recurring symptom was root sprawl: new top-level folders, repeated `src//source/` wrappers, redundant `appcore/appshell/shell` variants, product-specific roots, and duplicate plural/singular taxonomies. The user emphasized that the original warning sign was the assistant repeatedly adding new root directories and source folders despite the repo doctrine forbidding that pattern.

The first major conclusion was that the repo needed a closed set of ownership roots, not a new clever hierarchy. The top-level roots were settled as the standard canonical model: `apps`, `engine`, `game`, `runtime`, `contracts`, `content`, `docs`, `tests`, `tools`, `scripts`, `cmake`, `external`, `release`, and `archive`, with `AIDE/`/`GitHub/``editor` roots allowed. This did not mean every internal directory was perfect; it meant root churn

should stop.

2.2 From structure to enforcement The discussion then shifted from "what should the folders be?" to "how do we stop the repo from drifting again?" The user and assistant emphasized validators, allowlists, no-src/source rules, forbidden path terms, generated/local roots, bad-root absence, docs/tools/test taxonomy checks, dependency direction, and structure report integrity. The user asked repeatedly whether the repo was fixed after each status report, and the assistant kept distinguishing top-level success from internal residual debt.

This produced a pattern: generate a large Codex/AIDE task, the user ran it or reported a result, then the assistant evaluated what was truly fixed versus what was only validator/document churn. The user eventually demanded a one-shot "actual final cleanup" prompt because previous passes had sometimes added validators without moving directories. That prompt explicitly required real `git mv` routing, not just reports.

2.3 Converging on Domino, Dominium, Workbench, AIDE Over time, the project model became clearer. Domino was defined as the reusable deterministic/runtime substrate; Dominium as the specific game/product family; Workbench as a

production environment for editing, validation, inspection, packaging, UI authoring, evidence, and agent workflows; and AIDE as the repo/control-plane harness. This distinction mattered because the user wanted code and data reusable not only for Dominion but for other games or engine projects.

The assistant proposed that the framework should not be a new top-level `framework/` root. Instead, "Domino Framework" should be the public surface package formed by contracts, public headers, service/provider law, ABI law, conformance tests, and release profiles. The actual engine implementation remains in `engine/` and `runtime/`; the Dominion game implementation remains in `game/`, `content/`, and `apps/`.

2.4 From folders to public surfaces, APIs, ABIs, and compatibility A major part of the chat concerned how to make code portable, modular, extensible, and replaceable. The user wanted stable C APIs where possible but also needed modern implementation power. The plan evolved from older C89/C++98 ambitions to a C17/C++17 mainline, while keeping the public ABI C-compatible, POD-only, versioned, and free of C++ ABI leakage. The user also considered C99/pure C/C++11 alternatives, but the settled doctrine was: C17 for boring law, ABI-facing types, packets, saves/replays, deterministic math, and

low-level facades; C++17 for game orchestration, runtime services, providers, Workbench, tools, resource ownership, and large modular machinery.

The chat repeatedly emphasized that the stable boundary is not a folder; it is a registered public surface: C headers, ABI tables, schemas, protocols, command/view/event/refusal contracts, package/save/replay/profile formats, provider ABIs, capability IDs, diagnostic codes, and release artifacts. This motivated future tasks such as Public Surface Registry, API/ABI Canon, Dependency Direction, Compatibility Corpus, Replacement Protocol, Capability/Refusal Law, Provider Model, Presentation Contract, and Projection Conformance.

2.5 Workbench and the semantic spine The Workbench brainstorming originally proposed large user-facing workspaces such as Project Editor, Interface Studio, Module/Pack Foundry, and App Composer. The discussion refined this: those are workspaces, not primitive modules. The system should distinguish components, services, providers, packs, modules, workspaces, apps, and artifacts. A module is a declared functional extension unit, not just a folder. A Workbench module is one consumer of the general module system, not the module system itself.

The presentation architecture also matured. CLI, TUI/

text, rendered GUI, native GUI, and headless reports should not become separate systems. They should be projections over the same semantic spine: intent -> command -> capability/refusal check -> service -> result/document/snapshot -> diagnostics/evidence -> view -> action model -> projection -> shell. This led to the recommendation for PRESENTATION-CONTRACT-01 and PROJECTION-CONFORMANCE-01.

2.6 Provider model and third-party acceleration

Later the chat considered raylib, SDL2, Lua, raygui, rlgl, rlsw, raudio, and Dear ImGui. The user was dissatisfied with vendor-shaped structures, and the assistant refined the model: use raylib aggressively, but only as replaceable providers behind Dominium/Domino service contracts. SDL2 is a platform/input/audio provider family; Lua is a pinned script provider; raygui and ImGui are UI providers; rlgl and rlsw are raylib ecosystem render providers. Third-party types must not leak into engine, game, contracts, content, saves, replays, public SDK headers, or deterministic game law.

The final provider directory doctrine became service-first: runtime/<service>/providers/<provider>/, not runtime/<vendor>/<service>/. Provider choices belong in release/profiles/ or content/profiles/, not app path names. External code belongs under external/upstream/ or external/vendor/, but the

repo should choose one convention.

2.7 Current reported state By the end, user reports indicated that several actual cleanup commits landed. Important reported commits included provider-structure work, actual final structure cleanup, full-gate legacy path routing, and Domino framework boundary definition. The assistant judged the repo as structurally credible, with canonical structure and old active paths mostly clean, but not perfect. Full CTest/full-gate release proof remained ungreen, with stale AuditX/identity evidence, launcher marker debt, and other full-gate warnings still needing targeted maintenance. The final conclusion was that broad structure cleanup should largely stop; future work should be targeted proof hygiene and governed product-spine tasks.

3. Main Topics Discussed

Topic 1 - Canonical repository root structure The chat repeatedly returned to the question of which top-level roots should exist. The conclusion was that the root set should be closed and stable: `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `docs/`, `tests/`, `tools/`, `scripts/`, `cmake/`, `external/`, `release/`, and `archive/`, plus `.aide/`, `.aide.local.example/`, `.github/`, and `.vscode/`. The user strongly rejected proposals that reintroduced `src/`, `source/`, new

top-level modules/, plugins/, profiles/, labs/, framework/, or sdk/ roots. The goal was to keep roots as durable ownership planes, not topic buckets.

Topic 2 - Source layout versus

runtime/install/package layout A recurring distinction was that source repo layout is not runtime/install/media/package layout. store/, instances/, saves/, exports/, cache/, ops/, portable install roots, media layouts, and .dompkg package projections are runtime/distribution concepts, not source roots. The conversation repeatedly separated build output, portable installs, installed layout, package cache, save bundles, instance bundles, diagnostics bundles, and source repo structure.

Topic 3 - Apps, runtime, engine, game, contracts, content ownership The chat developed strong ownership rules. apps/ are thin product shells. engine/ is deterministic substrate. game/ is Dominion law, rule, world, and domain meaning. runtime/ adapts host/platform/render/input/audio/network/storage/package/UI/services and implements projections/providers. contracts/ is machine-readable law. content/ is authored payload. tools/ is repo-only machinery. archive/ is inactive history/evidence.

Topic 4 - Public surface, API, ABI, and language baseline The user wanted reusable code not only for Dominion but for other games and engine projects. The conclusion was that stable public surfaces matter more than frozen folders. The repo should use C17 + C++17 as the active mainline baseline, with a C-compatible public ABI: opaque handles, versioned structs, fixed-width types, explicit allocation/lifetime rules, no STL/C++ ABI across public boundaries, no exceptions crossing ABI, and no raw object-layout serialization.

Topic 5 - Dependency direction and replacement

A key design target was replaceability: whole implementations or directories should be rewritable behind stable contracts. Dependency direction became a crucial enforcement layer: apps may depend on runtime/game/contracts; game may depend on engine/contracts; runtime may depend on engine/contracts; engine must not depend on runtime/game/apps; contracts must not import implementation; runtime must not depend on tools; archive must not be active dependency. Replacement should be governed by replacement packets, conformance tests, compatibility corpora, and proof artifacts.

Topic 6 - Workbench, modules, workspaces, and composition The Workbench discussion distinguished

workspaces from modules. Workspaces are large human-facing compositions like Project Graph, Interface Studio, Module Foundry, App Composer, and Release Forge. Workbench modules are smaller UI/functional units like validation dashboard, pack browser, evidence viewer, replay trace, renderer lab, and agent board. The general module system belongs in `contracts/module/` and manifests; Workbench modules are only one consumer.

Topic 7 - Presentation and UI architecture The user and assistant converged on a shared presentation spine: CLI, text/TUI, rendered GUI, native GUI, and headless should project the same `command/result/refusal/document/view/action/evidence` contracts. Contracts define semantic surfaces; runtime implements projection machinery; apps compose products; Workbench hosts workflows. Presentation modes should not become separate semantic authorities.

Topic 8 - Packs, profiles, saves, replays, and artifacts Packs are distributable authored payloads. The preferred pack layout became category-based, such as `content/packs/<category>/<pack_id>/`. Pack IDs must not depend on paths. Saves and replays must use stable IDs, schema versions, canonical serialization, deterministic hashes, and explicit migration/refusal policies. Backward compatibility must be a

compatibility corpus plus tests, not intention.

Topic 9 - Provider model, raylib, SDL2, Lua, and third-party boundaries The later chat focused on raylib/SDL/Lua and other third-party accelerators. The doctrine became service-first and provider-backed: `runtime/<service>/providers/<provider>/`; external code under `external/upstream/` or `external/vendor/`; provider selection in profiles; third-party types fenced out of `engine/game/contracts/content/saves/replays`. Raylib is a seed provider suite, SDL2 a platform/input/audio provider, Lua a pinned script provider, raygui/ImGui UI providers, and rlgl/rlsw render providers.

Topic 10 - Validators, proof gates, and full-gate debt The chat produced a strong proof model. Fast strict is the normal development gate, while full CTest/full proof is T4/full-gate debt. Many validators were discussed: root allowlist, bad-root, no-src/source, forbidden names, canonical structure, dependency direction, public surface, provider structure, third-party include, report integrity, schema taxonomy, content layout, Workbench taxonomy, docs/tools/test taxonomy, and full-gate legacy paths. Full release readiness remains blocked until full-gate failures are resolved.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted to stop the repository from blocking development. They wanted the directory structure fixed, not endlessly discussed. They wanted no more root sprawl, no `src//source/` wrappers, no generic buckets, no long meaningless paths, and no vendor/product-shaped architecture. They wanted a repo structure suitable for a serious reusable game/engine platform rather than a one-off indie project. They also wanted future chats to have enough context to continue without re-explaining everything.

4.2 Inferred Goals INFERENCE: The user wanted a durable method for deciding future structure changes, not just a one-time layout. They also wanted the assistant to become more rigorous: distinguish decisions from brainstorming, stop over-planning without execution, and preserve uncertainty. The user also wanted a structure that supports other Domino-based games and possibly different engine implementations.

4.3 Goals That Changed Over Time The goal shifted from "fix folders" to "define stable public surfaces and make private implementations replaceable." It also shifted from old C89/C++98 portability ambition to C17/C++17 mainline with C-compatible public ABI. Workbench shifted from a monolithic editor idea to a

projection/command/workspace system over shared contracts. Raylib shifted from a possible app framework to a fenced provider suite.

4.4 Goals Still Unresolved Full release proof is not green. Some warning buckets remain: generated evidence, full CTest/T4 failures, provider runtime implementation, presentation/projection conformance, pack internal layout, public ABI promotion, and possibly AIDE state classification. The user still needs to decide when "good enough" for product work is reached versus continuing maintenance cleanup.

5. Decisions Made and Why

Decision overview

Row 1 ID: DECISION-01; Decision: Keep a closed top-level root model; Status: Accepted/current; Why it mattered: Prevents root sprawl and repeated refactor churn; Confidence: High; Label: FACT

Row 2 ID: DECISION-02; Decision: No first-party src//source/ wrappers; Status: Accepted/current; Why it mattered: Avoids meaningless nesting and ownership ambiguity; Confidence: High; Label: FACT

Row 3 ID: DECISION-03; Decision: Domino is reusable

substrate; Dominion is game/product family;
Status: Accepted/current; Why it mattered:
Enables reuse by other games/projects; Confidence:
High; Label: FACT/INFERENCE

Row 4 ID: DECISION-04; Decision: Workbench is not authority; contracts/commands/services are authority; Status: Accepted/current; Why it mattered: Prevents GUI-specific product logic; Confidence: High; Label: FACT

Row 5 ID: DECISION-05; Decision: C17 + C++17 mainline, C-compatible public ABI; Status: Accepted/current; Why it mattered: Balances implementation power with portability and stable ABI; Confidence: Medium-High; Label: FACT as discussed; external details require verification

Row 6 ID: DECISION-06; Decision: Service-first provider structure; Status: Accepted/current; Why it mattered: Keeps raylib/SDL/Lua replaceable; Confidence: High; Label: FACT

Row 7 ID: DECISION-07; Decision: Do not add top-level `framework/` root; Status: Accepted/current; Why it mattered: Avoids competing public API roots; Confidence: High; Label: FACT

Row 8 ID: DECISION-08; Decision: Profiles select providers; apps remain generic; Status: Accepted/

current; Why it mattered: Prevents product variants from becoming directory architecture; Confidence: High; Label: FACT

Row 9 ID: DECISION-09; Decision: Full CTest is full-gate debt, not normal daily gate; Status: Accepted/current; Why it mattered: Keeps development usable while preserving release proof; Confidence: Medium-High; Label: FACT from reports

Row 10 ID: DECISION-10; Decision: Continue with targeted tasks, not giant structure passes, after cleanup credibility; Status: Current recommendation; Why it mattered: Prevents churn after structure is credible; Confidence: Medium; Label: INFERENCE

Explanatory decision notes The closed root model was the most emotionally and technically important decision. The user repeatedly objected to root proliferation; the final root set became the anchor for all later advice. This decision affects every future proposal: if a concept can fit under an existing root, it should not become a new top-level root.

The no-**src** rule was also critical. The user considered repeated **src/** and **source/** folders as a central symptom of bad architecture. The final doctrine is that ownership

directories are already source roots; implementation should live under meaningful module/subsystem folders, not generic `src` wrappers.

The Domino/Dominium split emerged as the long-term reuse strategy. Domino is the framework/substrate identity; Dominium is one game/product family. This lets the current repo host Dominium while still exposing reusable runtime/engine/contract/provider surfaces.

The framework-boundary decision rejected a top-level `framework/` root. The accepted model is that Domino Framework exists as public surfaces, ABI headers, contracts, and provider/service law. This prevents root sprawl while still supporting SDK/export concepts later.

The provider model decision was a major response to raylib/SDL/Lua discussions. Third-party libraries should be providers behind first-party service contracts. They can accelerate visible progress but must not own saved data, game law, public ABI, or content schemas.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

Option	Status	Reason
Top-level <code>src/</code> or repeated module <code>src/</code> folders	Rejected	Violates explicit repo doctrine and user preference.
New top-level <code>framework/</code>	Rejected	Framework identity should be public surfaces/ contracts/headers, not another root.
Top-level <code>modules/</code> , <code>plugins/</code> , <code>services/</code> , <code>profiles/</code> , <code>labs/</code>	Rejected for now	Concepts fit under <code>contracts/runtime/content/release/apps/tools</code> .
Vendor-shaped structure such as <code>runtime/raylib/render</code>	Rejected	Repo should be service-first, provider-backed.
Product variants like <code>apps/client/rendered/raylib</code> as architecture	Rejected except temporary proof bootstraps	Apps should be generic; profiles select providers.
Pure C99 or pure C++11 baseline	Rejected	C17+C++17 better matches project scale and target floor.
C89/C++98 as active mainline	Superseded	Preserved as historical/research lane, not mainline.
Workbench as authority	Rejected	Workbench must consume <code>command/view/document/evidence</code> contracts.
<code>content/modules/</code> and <code>content/</code>	Rejected ¹⁴⁴³	Packs are distributable

7. Important Reasoning, Rationale, and Tradeoffs

The central tradeoff was between speed and long-term architectural integrity. The user wanted immediate progress after months of structure work, but also wanted to avoid repeating the same cleanup cycle. The resulting strategy became: enforce enough structure to prevent collapse, then proceed with narrow governed product slices instead of waiting for every possible governance layer to be perfect.

Another major tradeoff was between stable APIs and replaceable internals. The chat rejected the idea that every folder or private helper must be stable. Instead, stable identity lives in contracts, manifests, registries, public headers, artifact IDs, and compatibility tests. Private directories and implementations can move or be rewritten if public surfaces and compatibility proofs remain intact.

The raylib/SDL/Lua discussion raised a speed-versus-lock-in tradeoff. Raylib can accelerate rendering, audio, input, and Workbench prototypes. SDL2 can provide robust platform/input/audio provider roles. Lua can enable scripting and automation. But none should become architecture law. The solution was provider manifests, forbidden include validators, conformance tests, and profile-selected provider combinations.

The Workbench discussion involved a similar tradeoff.

The user wanted powerful authoring workspaces, but the architecture must not make Workbench a privileged bypass. Therefore, Workbench modules and workspaces should operate through the same commands, views, actions, documents, diagnostics, and evidence packets that CLI, CI, headless, server, and future apps use.

A persistent uncertainty is the exact current repo state. The user supplied many status reports and directory exports, but several exports were internally inconsistent or stale at various points. The chat therefore repeatedly emphasized structure report integrity and verification before trusting tree analyses.

8. Plans, Future Work, and Next Steps

The visible next work falls into three tracks.

8.1 Maintenance / proof hygiene The strongest immediate maintenance tasks are: refresh stale generated evidence, repair launcher pack-verification marker debt if still blocking fast strict, run or audit full CTest/T4 failures, and classify remaining full-gate failures by cause. The task names discussed include FULL-GATE-GENERATED-EVIDENCE-REFRESH-01, FAST-STRICT-EVIDENCE-MARKER-REPAIR-01, and FULL-CTEST-AUDIT-NONPATH-01.

8.2 Architecture/product-spine mainline Mainline tasks discussed include PRESENTATION-CONTRACT-01, PROJECTION-CONFORMANCE-01, COMMAND-RESULT-VIEW-SLICE-01, PACKAGE-MOUNT-SLICE-01, and WORKBENCH-VALIDATION-SLICE-01. The rule is to prove narrow slices before broad UI, gameplay, renderer, provider runtime, or package runtime expansion.

8.3 Provider/third-party implementation The provider path includes PROVIDER-WEDGE-01, RAYLIB-SEED-PROVIDER-01, SDL2-PROVIDER-01, LUA-PROVIDER-PIN-01, and follow-ups for render/draw/software/OpenGL/Direct3D/native platform providers. The structure rule is service-first: `runtime/<service>/providers/<provider>/`, with provider choices in profiles.

Recommended sequence as of the chat's end

1. Verify fast-strict/RepoX/structure report status from a clean current export.
2. If fast-strict is blocked, repair stale generated evidence/markers first.
3. If fast-strict is green, proceed to PROJECTION-CONFORMANCE-01 or PRESENTATION-CONTRACT-01 depending the live queue.

4. Do not run broad structure cleanup unless a validator reports hard active-path blockers.
5. Keep full CTest/T4 debt as release-gate work, not normal development gating.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences

- The user wants direct, audit-ready, source-grounded answers.
- The user dislikes vague reassurance and wants uncertainty labelled.
- The user wants no root sprawl, no arbitrary new top-level roots, and no repeated `src//source/` wrappers.
- The user wants structure to support modularity, portability, extensibility, reuse, modding, backwards compatibility, and future replacement.
- The user strongly prefers actual execution/move prompts over endless micro-planning when structure is blocking work.
- The user wants future assistants not to treat assistant brainstorms as accepted decisions unless accepted by the user.

9.2 Inferred Constraints and Preferences

- **INFERENCE:** The user prefers aggressive cleanup when structural debt is blocking product work, but increasingly accepts targeted tasks once structure is credible.
- **INFERENCE:** The user values consistent vocabulary: component, service, provider, pack, module, workspace, app, artifact.
- **INFERENCE:** The user wants the repo to behave more like a long-lived engine/platform or OS-grade project than a one-off game.

9.3 Uncertain or Unestablished Preferences

- **UNCERTAIN:** The final exact convention for `external/upstream` versus `external/vendor` should be checked against current repo policy.
- **UNCERTAIN:** Whether Lua should be pinned to 5.4 or 5.5 is not settled in this chat.
- **UNCERTAIN:** Whether pack-internal `content/` directories are legitimate pack law or legacy remains to be verified.

10. Files, Artifacts, Outputs, and Prompts

Important artifacts in or associated with this chat include:

- `dominium_canonical_handoff.md` and `dominium_canonical_handoff.txt`: earlier generated handoff files summarizing a major phase of the repository/architecture discussion. Preserve and compare with this preservation package.
- Multiple directory-tree/report bundles: `dir_tree.json`, `dir_tree.txt`, `dirfiles_manifest.json`, `dirfiles_run.log`, `dirfiles.zip`. These were used to assess repo structure repeatedly. Several bundles were stale or internally inconsistent, which became a major risk and motivated report-integrity validators.
- `Pasted text.txt`: the uploaded preservation-package instruction that triggered this report.
- Numerous Codex/AIDE mega-prompts generated in-chat, including but not limited to: `RUNTIME-NAME-01`, `PACK-AUTHORITY-01`, `SCHEMA-CANON-01`, `CONTENT-PACK-CANON-01`, `APPS-THIN-01`, `WORKBENCH-NAME-01`, `AIDE-SCAN-BOUNDARY-01`, `TOOLS-FOLD-01`, `DOCS-CANON-01`, `REPOX-TESTX-CANON-PATHS-01`, `CANON-REMEDIATION-FULL-PROOF-01`, `REPOX-STRICT-DEBT-01`, `CANON-TASK-`

STATUS-RECONCILE-01, API-ABI-CANON-01, PUBLIC-SURFACE-REGISTRY-01, PROVIDER-STRUCTURE-CANON-01, CANON-STRUCTURE-ACTUAL-FINAL-CLEANUP-01, and more.

- User-reported commits and status summaries: these include commits like 6e0dd93, 1406490, 3243fab, ce9ca, and others. Treat them as FACTs reported in the chat, but verify live repo state before acting.
- Generated reports/audits mentioned by the user: CANON_STRUCTURE_ACTUAL_FINAL_CLEANUP_01.md, FULL_GATE_LEGACY_TEST_ROUTE_01.md, PROVIDER_STRUCTURE_CANON_01.md, domino_framework_boundary.md, and many AIDE/RepoX/TestX reports.

11. Open Questions and Unresolved Issues

- Is full CTest currently green? The chat repeatedly says no or not run; verify before any release/trust claim.
- Are stale AuditX/identity evidence and launcher marker debt still blocking fast strict? User reports changed over time; verify live.
- Are provider split warnings for storage/package still present? If yes, finish STORAGE-PACKAGE-

PROVIDER-SPLIT-01.

- Are `pack-internal content/` directories canonical or legacy? This needs a pack-law decision.
- Are `.aide/cache`, `.aide/queue`, `.aide/reports`, and `.aide/ledgers` canonical control-plane state or mutable local state?
- Is the final live queue pointing to `PROJECTION-CONFORMANCE-01`, `PRESENTATION-CONTRACT-01`, or a maintenance task? Verify current `.aide/queue/current.toml`.
- Which Lua version should be pinned if/when Lua provider work begins?
- Should `external/upstream` or `external/vendor` be canonical? Verify current repo convention.

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Future assistants may think the directory structure is either completely broken or completely done. The correct state is more nuanced: broad structure appears credible, but release/full proof and warning debt remain.

A future assistant might also reintroduce top-level roots like `framework`, `profiles`, `labs`, `modules`, or `plugins`. This should be avoided unless the repo's root contract explicitly changes.

Another risk is treating raylib, SDL2, Lua, ImGui, raygui, rlgl, rlsw, or raudio as architecture instead of providers. These libraries should be fenced behind service contracts and provider manifests.

A future assistant might over-trust stale directory exports. Structure-report integrity was a recurring issue. Always check commit, branch, dirty state, source mode, and hashes before using uploaded structure reports as truth.

A future assistant might also confuse Workbench modules with the general module system. Workbench modules are product UI modules; general module law belongs in contracts and manifests.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat should feed major sections of a future Dominion/Domino Project Spec Book:

- Repository layout and ownership model.
- Domino Framework boundary and public surface registry.
- C17/C++17 language baseline and C-compatible ABI rules.
- Dependency-direction law.

- Service/provider/profile architecture.
- Workbench/module/workspace model.
- Presentation/projection semantic spine.
- Pack/content/profile/artifact identity law.
- Third-party provider fencing for raylib, SDL2, Lua, raygui, ImGui, rlgl, rlsw, and raudio.
- Proof gates, fast strict, full CTest/T4, and full-gate legacy routing.
- AIDE/Codex workflow and structure-report integrity.

Formal requirements candidates include: no top-level root sprawl; no first-party `src/source`; path is not identity; public surfaces must be registered; apps compose, runtime implements, contracts define; provider implementations must not leak third-party types across stable boundaries; full release proof requires full-gate green.

14. What I Should Remember

- The repo's top-level root model is settled; do not redesign it casually.

- The user was angry because repeated structure planning had delayed real work; future work should be execution-oriented and honest about results.
- The strongest doctrine is: stable semantic IDs and contracts, replaceable implementations.
- Domino is the reusable substrate/framework; Dominium is one game/product family.
- Workbench is a production surface, not the authority.
- Third-party libraries are providers, not architecture.
- C17+C++17 is the mainline baseline, with C-compatible public ABI.
- Feature readiness is limited until proof gates are clean.
- Full CTest/T4 is release/full-proof debt, not the normal fast development gate.
- Do not trust stale structure exports; validate report bundle integrity.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- "Explain the Domino/Dominium/Workbench/AIDE distinction again in one page."
- "What changed between the early folder-cleanup phase and the final provider-framework model?"

15.2 Decisions

- "Which decisions were explicitly accepted by me versus merely suggested by the assistant?"
- "Which root names and directory patterns are now forbidden?"

15.3 Tasks and Next Actions

- "Generate the next prompt for PROJECTION-CONFORMANCE-01."
- "Generate a maintenance prompt for stale generated evidence and fast-strict blockers."
- "Generate a targeted PACK-INTERNAL-LAYOUT-CANON-01 prompt."

15.4 Artifacts and Files

- "List all handoff/report files generated in this chat and what each should be used for."

- "Compare this preservation report with the earlier `dominium_canonical_handoff.md`."

15.5 Risks and Verification

- "What facts need live repo verification before I continue?"
- "What are the current risks if I start broad Workbench or renderer work now?"

15.6 Future Spec Book / Aggregation

- "Which parts of this chat should become formal spec-book requirements?"
- "What might conflict with other chats?"

15.7 Deep-Dive Questions Specific to This Chat

- "Should `external/upstream` or `external/vendor` be canonical?"
- "How should `raylib/SDL2/Lua` provider manifests be named?"
- "What exactly belongs in `apps/workbench/module/` versus `contracts/module/`?"

16. Compact Human Summary

This chat was about turning Dominionium from a structurally unstable game repository into a credible, reusable, contract-governed Domino/Dominium platform. The user's core frustration was that months of structure planning and refactoring had prevented actual development. The repo had repeatedly accumulated new roots, generic wrappers, duplicate directories, old path names, and unclear ownership. The user wanted the structure fixed decisively so Workbench, client, engine, game, provider, module, and pack work could begin without another refactor cycle.

The first major outcome was a closed top-level root model: `apps`, `engine`, `game`, `runtime`, `contracts`, `content`, `docs`, `tests`, `tools`, `scripts`, `cmake`, `external`, `release`, and `archive`, plus project roots like `.aide`, `.github`, and `.vscode`. The user strongly rejected top-level `src`, `source`, `modules`, `plugins`, `profiles`, `labs`, `framework`, and `sdk`. The conversation established that `apps` compose products; `runtime` implements reusable services, projections, providers, and host/platform integrations; `engine` owns deterministic substrate; `game` owns Dominionium law, rule, world, and domain meaning; `contracts` define law; `content` supplies authored payload; `tools` validate/generate/migrate/audit; `archive` preserves history and evidence.

The chat then moved beyond folders into public-surface governance. The user wanted code to be reusable by other games and engine projects. The answer became stable contracts and IDs, not stable private folders. Domino is the reusable deterministic substrate/framework; Dominion is one game/product family; Workbench is the production/editing/validation/evidence environment; AIDE is the repo/control-plane harness. The framework should not be a new **framework/** root. It should be the set of public surfaces, ABI headers, contracts, service/provider law, and conformance tests.

Language policy was also settled. The old C89/C++98 ambition was superseded by C17/C++17 as the mainline baseline, while preserving a C-compatible public ABI: opaque handles, versioned structs, fixed-width types, explicit ownership/lifetime, no STL/C++ ABI leakage, no exceptions crossing the boundary, and no raw object-layout serialization. C17 owns boring law, ABI-facing data, packets, saves, replays, deterministic math, and low-level facades. C++17 owns runtime services, game orchestration, providers, apps, Workbench, tools, and resource ownership.

The Workbench model was refined heavily. Workbench should not be the authority. CLI, text/TUI, rendered GUI, native GUI, headless, Workbench, CI, and AIDE should project the same command/result/refusal/

document/view/action/evidence contracts. A module is a declared functional extension unit, not just a folder. Workbench modules are only one consumer of general module law. Workspaces are larger user-facing compositions.

Third-party libraries were treated as providers. Raylib can be used aggressively, along with rlgl, rlsw, raygui, raudio, SDL2, Lua, and possibly ImGui, but they must be fenced. The final structure should be service-first: `runtime/<service>/providers/<provider>/`. Apps stay generic; profiles select providers; external third-party code lives under `external/upstream/` or `external/vendor/`; third-party types must not leak into engine, game, contracts, content, saves, replays, or public ABI.

Many concrete Codex/AIDE prompts were generated and many user status reports described commits landing. The repo reportedly reached a credible canonical state: old active paths routed, provider/profile structure added, full-gate legacy stale paths fixed, Domino framework boundary defined, fast strict and smoke tests passing in several reports. However, full CTest/T4 release proof remains ungreen or unverified, stale generated evidence and marker debt appear in some reports, and feature readiness remains limited. The best future work is targeted: repair proof hygiene, run/triage full CTest, continue projection conformance and presentation

contracts, and then proceed with provider and Workbench product-spine slices.

29.2 Dominium Canonical Architecture Repo Foundation Reader Brief

Source: docs/archive/conversations/canonical_
structure_and_framework/dominium_canonical_
architecture_repo_foundation__06_reader_brief.
md

Report type: reader_brief

Size: 1732 bytes

29.2.1 Reader Brief - Dominium Canonical Architecture, Repository Foundation, and Provider Model

This package preserves a long chat about fixing Dominium's repository structure and broader architecture. The top things to know are:

1. The user wanted to end months of structure churn and get back to building real systems.
2. The top-level root model is now settled and should not be casually changed.

3. No first-party `src/` or `source/` wrappers.
4. Domino is the reusable substrate/framework;
Dominium is one game/product family.
5. Workbench is a production environment, not the authority.
6. Contracts define law; runtime implements; apps compose; content supplies payload; tools validate; tests prove.
7. Stable IDs and public contracts matter more than stable private folders.
8. C17+C++17 is the mainline baseline, with C-compatible public ABI.
9. Raylib, SDL2, Lua, ImGui, raygui, rlgl, rlsw, and raudio are providers, not architecture.
10. Providers live under `runtime/<service>/providers/<provider>/`.
11. Apps should not encode provider choices in paths.
12. Profiles select provider combinations.
13. Module identity is not path; Workbench modules are one consumer of the module system.
14. CLI/TUI/rendered/native/headless are projections over shared semantic contracts.

15. Full CTest/T4 remains release/full-proof debt.
16. Fast strict is the normal development gate.
17. Stale structure reports were a recurring risk.
18. Do not treat assistant brainstorms as decisions unless the user accepted them.
19. Future work should be targeted, not another root-structure reset.
20. Best next action is to verify current live repo status and proceed with proof hygiene or projection conformance depending on gate status.

29.3 Dominium Canonical Architecture Repo Foundation In Chat Reader

Source: docs/archive/conversations/canonical_
structure_and_framework/dominium_canonical_
architecture_repo_foundation__09_in_chat_
reader.md

Report type: in_chat_reader

Size: 1360 bytes

29.3.1 In-Chat Reader

Package overview

This preservation package is a human-readable and machine-mergeable record of this chat about Dominion repository structure, Domino framework boundaries, provider architecture, Workbench, public surfaces, and proof gates.

File index

- `00_manifest.md`: file list and package metadata.
- `01_human_readable_report.md`: main explanatory report.
- `02_context_transfer_packet.md`: future-chat handoff.
- `03_spec_sheet.yaml`: structured spec data.
- `04_registers.md`: workstream/decision/task/risk registers.
- `05_aggregator_packet.md`: compact cross-chat merge packet.
- `06_reader_brief.md`: short human brief.
- `07_verification_and_audit.md`: self-audit and verification queue.

- `08_future_chat_bootstrap_prompt.md`: prompt for continuing in a new chat.
- `09_in_chat_reader.md`: this guide.

Safest next actions

1. Verify current live repo status from a fresh tracked export.
2. If fast strict/RepoX is blocked, repair proof hygiene.
3. If gates are clean, proceed with projection conformance or presentation contracts.
4. Do not restart root-structure debate unless a hard validator fails.

Question menu

Ask: "What decisions are final?", "What is still uncertain?", "What is the next prompt?", "Which rejected options should we avoid?", "What should go into the master spec book?"

29.4 Dominion Canonical Structure And Framework Human Readable Report

Source: docs/archive/conversations/canonical_
structure_and_framework/dominium_canonical_
structure_and_framework__01_human_readable_
report.md

Report type: human_readable_report

Size: 84019 bytes

29.4.1 COMPLETE CHAT PRESERVATION REPORT - Dominion Canonical Structure and Domino Framework Architecture

0. Coverage and Reliability Assessment

Row 1	Field: Chat label; Assessment: Dominion Canonical Structure and Domino Framework Architecture
Row 2	Field: Date anchor; Assessment: 2026-05-27 Australia/Melbourne
Row 3	Field: Source scope; Assessment: This chat only unless labelled PROJECT-CONTEXT
Row 4	Field: Apparent access; Assessment: Partial

- Row 5** Field: Previously generated files available?; Assessment: Yes: earlier Markdown/plain-text handoff files are mounted in `/mnt/data`; some previously uploaded files expired.
- Row 6** Field: Uploaded files or artifacts present?; Assessment: Yes: repeated directory-tree exports, dirfiles manifests, run logs, ZIP bundles, and the current preservation prompt text.
- Row 7** Field: Contains future plans?; Assessment: Yes
- Row 8** Field: Contains decisions?; Assessment: Yes, with some decisions tentative or dependent on repo proof status.
- Row 9** Field: Contains pending tasks?; Assessment: Yes
- Row 10** Field: Contains unresolved questions?; Assessment: Yes
- Row 11** Field: Staleness risk; Assessment: Medium to High for live repo state and third-party library/version facts; Low for architectural decisions explicitly discussed.
- Row 12** Field: Extraction confidence; Assessment: 4/5 for the visible chat and mounted summaries; 3/5 for exact historical file states because some uploads expired and several structure bundles were mixed/stale.

Row 13 Field: Safe for later aggregation?; Assessment: Yes, with caveats

Row 14 Field: Main limitations; Assessment: The full original transcript is not available as a separate complete export. This report reconstructs from the visible chat context, uploaded preservation prompt, earlier generated handoff files, and user-provided status reports. Some file uploads expired and live-repo facts may have changed after the last status report.

Plain-language limitation note: this report is a high-fidelity reconstruction of the accessible conversation, not a forensic transcript. It preserves the substance, decisions, prompts, tasks, tradeoffs, and current state as established in the chat. It does not claim to know every file in the user's local repository beyond the uploaded snapshots and user-reported task results.

1. One-Page Orientation

This chat was a long-running architecture and repository-convergence session for the Dominion project. The central problem was that Dominion's repository structure had become too sprawling, redundant, and unstable for serious development. The user was blocked from building real features because the repo kept accumulating new roots, duplicated concepts, `src/` and

source/ folders, old domain roots, tool roots, app roots, and generated-state confusion. The user repeatedly emphasized that the project needed to become future-proof, portable, modular, extensible, moddable, reusable, backward-compatible, and not another one-off indie project.

The conversation gradually moved from directory cleanup into a broader architectural doctrine. Early discussion focused on distinguishing source repo layout from distribution, installed, portable, package, save, media, cache, rollback, and diagnostics layouts. A major conclusion was that Dominium needed one logical virtual-root model projected into multiple physical layouts, not a different ad hoc tree for every artifact type. This then became a source-layout convergence effort: keep a small closed set of top-level roots-apps/, engine/, game/, runtime/, contracts/, content/, docs/, tests/, tools/, scripts/, cmake/, external/, release/, and archive/-and prevent new root sprawl.

As the work progressed, the user pushed back hard against structure proposals that reintroduced the very mistakes the refactor was meant to eliminate, especially root-level `src/`, nested `src/`, `source/`, vague `core`, `lib`, `common`, `shared`, top-level `modules`, `plugins`, `profiles`, `labs`, and product/vendor-shaped roots. This produced a stronger naming and ownership doctrine: paths encode ownership, public symbols encode namespace, stable IDs

encode semantic identity, and private names stay concise. The user and assistant converged on the rule that **path is not identity, implementation is not contract, UI is not authority, and generated output is not source truth.**

A major architectural shift was the emergence of the "Domino Framework / Dominion Game" distinction. Domino is the reusable deterministic substrate and service/provider framework; Dominion is one game/product family built on it; Workbench is the production, editing, validation, evidence, and inspection environment; AIDE is the repo/control-plane harness. The project should not create a top-level **framework/** root; instead, the framework exists through **contracts/**, public headers under **engine/include/domino** and **runtime/include/domino**, provider/service law, public-surface registries, conformance tests, and release/profile descriptors.

Another large theme was provider architecture. The chat settled on a service-first provider model: Dominion/DOMINO owns service contracts; third-party libraries like raylib, rlgl, rlsw, raygui, raudio, SDL2, Lua, and possibly Dear ImGui are replaceable providers. Provider implementations live under **runtime/<service>/providers/<provider>/**; third-party source lives under **external/upstream/** or equivalent; provider choices live in **release/profiles/** or **content/profiles/**, not app

directory names. The user asked whether names should be `sd12` or `sd1`, `raylib` or `rlgl`, `win32` or `windows`, `d3d11` or `directx`; the answer was to use generic service roots, exact implementation provider folder names, and fully qualified provider IDs.

The chat also defined the Workbench/presentation architecture. Workbench should not be the center of the system. It is one consumer of semantic contracts, commands, services, providers, packs, modules, and artifacts. The correct presentation spine is: intent -> command -> capability/refusal check -> service -> result/document/snapshot -> diagnostics/evidence -> view/action model -> projection -> shell. CLI, text/TUI, rendered GUI, native GUI, headless reports, Workbench panels, CI, and AIDE should be projections over the same command/view/action/evidence contracts, not separate UI systems.

Many large Codex/AIDE prompts were generated or outlined to force structure cleanup and proof hardening. The user reported several commits and validation states, including canonical structure cleanup, provider structure enforcement, framework boundary definition, and full-gate legacy path routing. The most recent established status is that the structure is credible and much improved, but not perfect: fast strict, canonical structure, CMake verify/build, smoke tests, and targeted capability tests pass or pass with warnings; full CTest/

T4 release proof remains debt; broad feature work is still blocked; narrow governed tasks such as projection conformance, presentation contracts, provider wedges, and full-gate evidence repair remain next.

A future assistant must understand that this chat was not merely about folders. It was about turning a messy game repository into a deterministic, contract-governed, provider-backed, pack-composed simulation platform. The most important carry-forward is the doctrine: freeze public surfaces, replace private implementations, preserve artifacts, verify behavior.

2. The Story of the Conversation

The chat began from the user's frustration that Dominion had been stuck in planning/refactoring for months. The user wanted an "ultimate" directory structure that would not require another large restructure. Early answers proposed broad final layouts and routing maps, but the user strongly rejected anything that reintroduced root sprawl, `src/source/`, long paths, proper-name-heavy directories, or vague wrapper folders. This produced one of the central constraints: **no root @@V12@@, no root @@V13@@, no nested module @@V14@@, no generic wrappers, no new top-level roots without contract approval.**

The first major phase was source-root convergence. The conversation established the canonical top-level root set and clarified that source layout must be separate from release output, install layout, portable layout, media layout, package layout, save/bundle layout, diagnostics bundles, and setup/cache/rollback state. The repo should not contain runtime install roots like `store/`, `instances/`, `saves/`, `cache/`, or `ops/` as source roots. Distribution and install layouts should be projections of virtual roots, not drivers of source-tree ownership.

The second phase was bad-root routing. The chat generated large Codex prompts for residual cleanup: runtime name cleanup, pack authority cleanup, schema taxonomy cleanup, content pack layout cleanup, app thinness, Workbench naming, AIDE scan boundary, tools fold, docs canon, RepoX/TestX canonical path repair, remediation proof, RepoX strict debt repair, and task-status reconciliation. The user repeatedly reported commits and validation results from Codex/AIDE runs. Over time, the repo moved from obvious root chaos toward a canonical root model with progressively fewer stale active paths.

The third phase was frustration-driven tightening. The user objected that earlier suggestions still permitted architectural sloppiness. The assistant then articulated stricter naming and structure laws: singular/plural distinctions, short local paths, project prefixes only at

public/global boundaries, stable IDs in contracts/manifests rather than paths, and no status words like **legacy**, **modern**, **experimental**, **compat**, **universal**, **old**, **new**, **future**, **v2**, **v3** in active source paths except where semantically justified.

The fourth phase shifted from folders to API/ABI and public surface governance. The user wanted code to be reusable not only for Dominion but for other games and even other engine projects. This led to the Domino/Dominium split: Domino is the reusable deterministic substrate; Dominion is the game/product family. The project needs stable C-compatible public APIs/ABIs, versioned structs, opaque handles, explicit ownership, allocator rules, error/refusal codes, provider descriptors, capability negotiation, compatibility corpora, replacement protocols, dependency-direction validation, and public-header consumer tests. Initially the doctrine referenced C89/C++98, but later the user introduced and accepted a C17/C++17 mainline baseline for Windows 7 SP1+, macOS 10.9.5+, and Linux, while preserving C-compatible ABI boundaries.

The fifth phase defined Workbench and modules. The user brought in brainstorming from another chat about Project Editor, Interface Studio, Module/Pack Foundry, App Composer, Release Forge, etc. The chat refined this into a vocabulary: component = source/build unit; service = callable runtime capability; provider =

replaceable implementation; pack = distributable authored payload; module = declared functional extension unit; workspace = large Workbench composition; app = product composition; artifact = persisted/versioned object. This prevented `module/` from becoming the next junk drawer. Reusable modules should not live under `apps/workbench/`; only Workbench UI modules belong there. Module law belongs under `contracts/module/`, module payloads can be delivered through packs, reusable behavior lives under `runtime/` or `game/`, and Workbench presents/edits modules through commands and views.

The sixth phase defined presentation architecture. The user asked about unifying CLI, TUI, rendered GUI, native GUI, Workbench, launcher, setup, server, and client. The chat concluded that all should project the same semantic truth: commands, results, refusals, documents, snapshots, diagnostics, evidence, views, and actions. It rejected separate contract worlds for `tui`, `rendered`, and `native`; those are projection modes under `runtime/projection/`, not semantic authorities under `contracts/`. This produced a refined runtime target: `runtime/command`, `runtime/action`, `runtime/document`, `runtime/patch`, `runtime/view`, `runtime/projection/{cli,text,rendered,native,headless}`, `runtime/ui`, `runtime/render`, `runtime/platform`, etc.

The seventh phase was raylib/SDL/Lua provider

architecture. The user pasted or summarized advice about using raylib heavily. The chat accepted the accelerator strategy but rejected vendor-shaped structure. Raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua should be providers behind first-party service contracts, not engine law. Provider implementations should live under `runtime/<service>/providers/<provider>/`; provider choices should be in profiles, not app paths; third-party types must not leak into `engine/`, `game/`, `contracts/`, `content/`, `saves`, `replays`, `packs`, or public SDK headers. Lua should be pinned as a script provider with Dominium script API versioning, not raw Lua ABI as mod law.

The eighth phase handled the framework question. The user considered adding a `framework/` root and splitting `engine/domino_reference` and `game/dominium`. The chat rejected a new top-level `framework/` root. Domino Framework is not a source root; it is the collection of public surfaces, ABI headers, contracts, `service/provider` law, and conformance tests. The reference implementation remains in `engine/` and `runtime/`; Dominium remains in `game/`, `content/`, and `apps/`. A later external SDK can be generated from registered public surfaces, but the repo should not add a new active `framework/` root.

The final visible phase was proof and maintenance status. The user reported commits such as

refactor(repo): finish canonical structure cleanup, tests: route full-gate legacy path expectations, and repo: define Domino framework boundary. The assistant evaluated each status. The conclusion: the directory structure is now credible and mostly fixed, with canonical structure gates passing, old active paths cleaned, provider/profile structure in place, and stale full-gate path expectations reduced. But the repo is not perfect and not full-release-ready. Remaining work includes stale AuditX/identity evidence, launcher pack marker debt, full CTest/T4 nonpath failures, provider split warnings, public header ABI promotion warnings, storage/package provider split, projection conformance, presentation contracts, and targeted residual taxonomy cleanups.

3. Main Topics Discussed

Topic 1 - Canonical source-root structure FACT: The user wanted to end repeated repo-structure churn and stop new roots from appearing. The conversation converged on a closed top-level root model: apps, engine, game, runtime, contracts, content, docs, tests, tools, scripts, cmake, external, release, archive, plus .aide, .aide.local.example, .github, and .vscode as project/tooling roots. Generated/local roots such as .aide.local, .dominium.local, build, out, dist, artifacts, reports, tmp, and __pycache__

must not become active tracked source.

This mattered because the repo had repeatedly accumulated roots that mixed product, runtime, domain, tooling, generated, archive, and documentation concerns. The canonical model was intended to make future growth predictable and enforceable.

Topic 2 - No `src/` / `source/` and no generic wrappers FACT: The user explicitly objected that earlier proposals violated the rule against adding `src/` and `source/` folders. The chat locked the doctrine that first-party source modules should not contain root or nested `src/`, `source`, `sources`, `code`, `impl`, `common`, `shared`, `misc`, or `lib` wrappers. Implementation files should live under ownership paths directly.

The exception is third-party/external source, archive/history, or fixtures preserving upstream/provenance layout.

Topic 3 - Source layout versus distribution/install layouts FACT: The conversation emphasized that source layout is not runtime/install layout. Distribution artifacts, `.dmpkg` packages, compressed archives, portable installs, installed roots, media layouts, caches, staging, rollback, save bundles, instance bundles, replay/diagnostic bundles, and symbols/provenance layouts are separate projection models. The source repo should not

contain active install roots like `store/`, `instances/`, `saves/`, `exports`, `cache`, or `ops`.

This matters because Dominium needs multiple physical layouts but one semantic virtual-root contract.

Topic 4 - Domino versus Dominium FACT: The chat established a strong distinction between Domino and Dominium. Domino is the reusable deterministic substrate/framework. Dominium is one game/product family built on Domino. Workbench is a production/editing/validation/evidence environment. AIDE is the repo/control-plane harness.

This distinction matters for reuse. Future games or engine projects should be able to reuse Domino surfaces, providers, tools, and contracts without inheriting Dominium-specific game law, content packs, app descriptors, or domains.

Topic 5 - Public surface, API, and ABI governance FACT: The chat concluded that directory cleanliness is not enough. Public/private/stable/internal surfaces need explicit registry and validation. Public/stable surfaces include C headers, ABI tables, schemas, registries, protocols, commands, views, events, refusals, package/save/replay formats, provider ABIs, capability IDs, diagnostic codes, and release artifacts.

The visible rationale was that stable boundaries must be frozen, versioned, owned, and tested, while private implementation remains replaceable. This is one of the main future-proofing mechanisms.

Topic 6 - C17/C++17 baseline with C-compatible ABI FACT: The chat moved from older C89/C++98 discipline to a C17/C++17 mainline baseline. The target floor discussed was Windows 7 SP1+, macOS 10.9.5+, and Linux, with 64-bit little-endian x86_64 and arm64 first. Public ABI should remain C-compatible, POD-only, versioned, explicit about ownership/lifetime, and free of C++ ABI leakage.

UNCERTAIN / UNVERIFIED: Exact compiler/toolchain support details and third-party library versions were discussed with references in prior turns; current real-world facts should be verified before release policy is finalized.

Topic 7 - Commands, views, actions, projections, and UI unification FACT: The chat established that CLI, text/TUI, rendered GUI, native GUI, headless reports, Workbench, server/admin surfaces, CI, and AIDE should be projections of the same command/result/refusal/document/view/action system. The system should not grow four separate UI systems.

The refined spine was: intent -> command ->

capability/refusal check -> service -> result/document/
snapshot -> diagnostics/evidence -> view/action model
-> projection -> shell.

Topic 8 - Workbench and module composition

FACT: The chat clarified that Workbench modules are not the general module system. Workbench modules are user-facing UI/presentation modules under `apps/workbench/module/`; large user-facing compositions are Workbench workspaces under `apps/workbench/workspace/`; module law belongs under `contracts/module/`; pack-delivered module payloads live under `content/packs/.../modules`; reusable behavior lives under `runtime/`, `game/`, or `engine` depending ownership.

This matters because a world creation workspace, pack foundry, app composer, or interface studio must be reusable through commands, services, providers, packs, and artifacts-not hardwired into Workbench folders.

Topic 9 - Service-first provider architecture and third-party libraries

FACT: The chat concluded that raylib, rlgl, rlsw, raygui, raudio, SDL2, Lua, and possibly ImGui should be providers, not architecture. Dominion owns service contracts; providers satisfy them. Provider implementations live under `runtime/<service>/providers/<provider>/`; provider choices live in profiles; third-party source lives under `external/`

`upstream/` or equivalent; third-party headers/types must be fenced.

The user specifically asked naming questions such as `SDL2` versus `SDL`, `raylib` versus `rlgl`, `win32` versus `windows`, and `d3d11` versus `directx`. The answer: use generic service roots, exact provider implementation folder names, fully qualified provider IDs, family metadata in profiles, and readable user labels.

Topic 10 - Domino Framework boundary **FACT:**

The user considered whether a framework approach required a `framework/` root. The chat decided no: Domino Framework is a public surface package made from contracts and public headers; the reference implementation remains in `engine/` and `runtime/`. `framework/` and `sdk/` top-level roots should be guarded against unless explicitly contracted later.

The user later reported a commit defining this boundary and adding validators.

Topic 11 - Proof gates and validation state **FACT:**

The chat tracked a shift from broad structure cleanup to proof hygiene. Fast strict, AIDE doctor/validate, RepoX strict, CMake verify/build, smoke CTest, canonical structure, provider structure, and targeted capability tests were reported as passing or passing with warnings at various stages. Full CTest/T4 release proof remained

debt.

This matters because the repo can support narrow governed development but is not broad-release ready.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted to stop months of refactor churn, make the directory structure future-proof, avoid new root directories, eliminate redundant `src//source` patterns, make the code portable and modular, allow reuse across other games and engine projects, support Workbench and product development, and preserve the chat for future aggregation.

These goals were addressed by defining a closed root model, ownership rules, service/provider architecture, public surface governance, ABI policy, command/view/projection spine, provider profiles, and maintenance prompts.

4.2 Inferred Goals INFERENCE: The user also wanted psychological closure: a clear point where structure work stops being the main activity. The assistant repeatedly distinguished "structure credible with warnings" from "perfect" and recommended targeted maintenance instead of more giant move passes once the major structure gates passed.

4.3 Goals That Changed Over Time The conversation began as directory cleanup. It became architecture governance. It then became proof, provider, Workbench, framework, and preservation planning. The language baseline shifted from C89/C++98 ideas to C17/C++17 with a C-compatible ABI. The provider approach shifted from "use raylib" to "raylib is a seed provider suite behind first-party service contracts." The framework idea shifted from adding `framework/` to defining framework boundaries through contracts and public headers.

4.4 Goals Still Unresolved Full CTest/T4 remains unresolved. Some warnings remain: stale AuditX/identity evidence, launcher pack-verification marker debt, provider split warnings, public header ABI promotion warnings, pack internal `content/` layout warnings, residual schema buckets, AIDE state-like roots, and full-gate nonpath failures. Broader Workbench UI, runtime module loading, provider runtime, package runtime, gameplay, renderer implementation, native GUI, and release publication remain blocked or limited until proof gates advance.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Keep a closed canonical top-level root set.; Status: Accepted; 1483

Why it mattered: Prevents root sprawl.;
Confidence: High; Label: FACT

Row 2 ID: DECISION-02; Decision: No first-party root/nested `src` or `source.`; Status: Accepted; Why it mattered: Prevents generic wrappers and repeated confusion.; Confidence: High; Label: FACT

Row 3 ID: DECISION-03; Decision: Treat Domino as reusable substrate and Dominion as game/product family.; Status: Accepted; Why it mattered: Enables reuse beyond one game.; Confidence: High; Label: FACT

Row 4 ID: DECISION-04; Decision: Define framework through contracts/public headers, not a top-level `framework/` root.; Status: Accepted and later reportedly committed.; Why it mattered: Avoids new root ambiguity.; Confidence: High; Label: FACT

Row 5 ID: DECISION-05; Decision: Use C17/C++17 mainline with C-compatible ABI boundaries.; Status: Accepted as doctrine in chat.; Why it mattered: Balances modern implementation with portable ABI.; Confidence: Medium-High; Label: FACT/UNVERIFIED for external toolchains

Row 6 ID: DECISION-06; Decision: Use service-first provider structure for raylib/SDL/Lua.; Status:

Accepted and reportedly committed.; Why it mattered: Enables third-party acceleration without lock-in.; Confidence: High; Label: FACT

Row 7 ID: DECISION-07; Decision: Workbench is a consumer/projection surface, not authority.; Status: Accepted; Why it mattered: Prevents product/UI bypassing contracts.; Confidence: High; Label: FACT

Row 8 ID: DECISION-08; Decision: Use module/pack/app/workspace terminology precisely.; Status: Accepted; Why it mattered: Prevents module/ becoming a junk drawer.; Confidence: High; Label: FACT

Row 9 ID: DECISION-09; Decision: Continue narrow governed slices; broad feature work remains blocked until gates improve.; Status: Accepted through reported queue/status.; Why it mattered: Prevents architecture regression.; Confidence: High; Label: FACT

Row 10 ID: DECISION-10; Decision: Stop giant structure refactors after canonical structure gates are clean; use targeted maintenance.; Status: Recommended and broadly accepted after cleanup passes.; Why it mattered: Avoids endless churn.; Confidence: Medium; Label: INFERENCE

Each decision depended on the assumption that the repository is being actively validated by AIDE, RepoX, structure validators, CMake, and smoke/full tests. Some exact statuses depend on user-reported task results and should be rechecked in the live repo before new work.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

`framework/` as a top-level root was rejected. It seemed attractive as a way to separate framework and implementation, but it competed with `contracts/`, `engine/include`, `runtime/include`, and `public_surface` law. It may be reconsidered only if a future root contract explicitly defines an external SDK/source package root.

Top-level `modules/`, `plugins/`, `services/`, `profiles/`, and `labs/` were rejected. These concepts belong under existing roots: `contracts/module`, `runtime/<service>`, `content/packs`, `release/profiles`, `apps/*/proof`, `tools/experiments` if allowed, or `archive/generated/experiments`.

`apps/client/rendered/raylib` and `apps/workbench/raylib` as product architecture were rejected. Apps should be generic product shells; provider choices belong in profiles.

Raylib-shaped architecture was rejected. Raylib can be

heavily used, but only behind provider/service contracts. Raylib types must not leak into engine/game/contracts/content/saves/replays/public ABI.

Pure C99 or pure C++11 were rejected as mainline baselines. C17/C++17 with C-compatible ABI was preferred.

C89/C++98 as mainline was superseded by C17/C++17, while the discipline of stable C-shaped ABI and deterministic serialization was retained.

Putting reusable modules under `apps/workbench/module/` was rejected. Workbench modules are UI/presentation modules only; the general module system lives in `contracts/manifests/packs/runtime/game` as appropriate.

Separate UI systems for CLI/TUI/rendered/native were rejected. Projection over shared semantic contracts was preferred.

7. Important Reasoning, Rationale, and Tradeoffs

The core tradeoff was between early speed and long-term replaceability. Raylib, SDL2, Lua, and raygui could accelerate visible progress, but if their types leaked into public contracts, saves, game law, or engine state, the project would become vendor-bound. The service-first provider model preserves speed while fencing contamination.

Another tradeoff was between structure purity and endless churn. The user wanted perfection, but the chat repeatedly distinguished between hard blockers, warning-level debt, and speculative cleanup. Once major old paths were removed and canonical validators passed, the correct move became targeted maintenance and proof repair rather than another broad restructure.

A third tradeoff was between generic framework ambitions and repo root simplicity. Adding `framework/`, `sdk/`, `modules/`, or `profiles/` would appear clean in isolation but would undermine the hard-won closed root model. The better solution is generated/exported packages under `release/`, public surface registries under `contracts/`, and implementation under existing roots.

The user cared most about avoiding another cleanup cycle, making `code/data/modules/packs` reusable, preserving determinism and backward compatibility, supporting `Workbench/client/server/apps`, and keeping the architecture audit-ready.

8. Plans, Future Work, and Next Steps

The most recently discussed next workstreams were:

1. Proof hygiene: fix stale AuditX/identity evidence and launcher marker debt so fast-strict/RepoX gates are clean.

2. Full-gate maintenance: run or shard full CTest and classify nonpath failures.
3. Projection conformance: prove CLI/text/rendered/native/headless projections over existing command/result/view/evidence contracts.
4. Presentation contract: formalize view/action/projection/document/snapshot contracts.
5. Provider implementation wedges: raylib/SDL2/Lua providers, null providers, conformance tests, forbidden include checks.
6. Pack internal layout cleanup: decide whether pack-internal `content/` is canonical or legacy.
7. Runtime/engine residual taxonomy: classify session/serialization/foundation/compatibility residuals.
8. AIDE state classification: ensure `.aide/cache`, `.aide/queue`, `.aide/reports`, `.aide/ledgers` are canonical control-plane/evidence or moved to local/generated roots.

Recommended immediate sequence based on the latest reported state:

1. FAST-STRICT-EVIDENCE-MARKER-REPAIR-01 or FULL-GATE-GENERATED-EVIDENCE-REFRESH-01
2. PROJECTION-CONFORMANCE-01

3. PRESENTATION-CONTRACT-01, if not already completed or if projection work exposes gaps
4. PROVIDER-WEDGE-01 / RAYLIB-SEED-PROVIDER-01, once provider law and gates are clean
5. FULL-CTEST-AUDIT-NONPATH-01

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences The user prefers direct, rigorous, source-grounded, audit-ready answers with explicit uncertainty. The user repeatedly asked for "best possible," "future-proof," "modular," and "no excuses" structure. The user dislikes vague or redundant directory names and strongly rejects root/nested `src` and `source` structures. The user wants no new top-level roots unless truly justified and contracted. The user wants downloadable handoff/report packages where possible.

9.2 Inferred Constraints and Preferences

INFERENCE: The user values high control over architectural naming and does not want assistant suggestions to silently become decisions. The user wants prompts that Codex/AIDE can execute without repeatedly asking for clarification. The user prefers explicit blockers and status classifications such as PASS, PASS_WITH_WARNINGS, BLOCKED, LIMITED, and NO.

9.3 Uncertain or Unestablished Preferences

UNCERTAIN: Exact accepted spelling of some roots (diagnostic versus diagnostics, external/upstream versus external/vendor, lua54 versus lua55) may depend on current repo canon after the last commit. Future assistants should check the latest contracts and validators before renaming.

10. Files, Artifacts, Outputs, and Prompts

Key artifacts include:

- Earlier generated handoff files: `dominium_canonical_handoff.md` and `.txt` in `/mnt/data`. These summarize the state as of 2026-05-20 and should be preserved as historical handoff context.
- Multiple uploaded `dir_tree.json`, `dir_tree.txt`, `dirfiles_manifest.json`, `dirfiles_run.log`, and `dirfiles.zip` bundles. Their usefulness varied because several were mixed/stale. They established the need for structure-report integrity validation.
- Large Codex/AIDE prompts generated in chat, including runtime naming, pack authority, schema canon, content pack canon, app thinness, Workbench naming, AIDE scan boundary, tools fold, docs canon, RepoX/TestX canonical paths, remediation/full proof, strict debt repair, structure

finalization, residual loops, provider structure, framework boundary, and full-gate legacy path routing.

- User-reported audits and commits: `CANON_STRUCTURE_FINALIZE_NOW_01`, `CANON_STRUCTURE_ACTUAL_FINAL_CLEANUP_01`, `FULL_GATE_LEGACY_TEST_ROUTE_01`, `PROVIDER_STRUCTURE_CANON_01`, and `domino_framework_boundary.md`.
- The current uploaded preservation prompt: `Pasted text.txt`, which requested this complete preservation package.

11. Open Questions and Unresolved Issues

1. Is full CTest/T4 now green? Known answer from chat: no, not yet. It needs auditing and repair.
2. Are stale AuditX/identity evidence and launcher marker debt fixed? Last visible status said no or still pending.
3. Are raylib/SDL2/Lua providers implemented or only planned? Last status said no third-party implementation was introduced; provider work remains separate.
4. Is `external/upstream` or `external/vendor` canonical in the latest repo? Needs live repo verification.

5. Is Lua provider `lua54` or `lua55` chosen? The chat discussed pinning one but did not establish a final user decision.
6. Is `contracts/diagnostic` or `contracts/diagnostics` canonical? The chat suggested consistency but did not confirm final repo state.
7. Are `pack-internal content/` folders law or legacy? Still an open cleanup item.
8. Are `.aide/cache` and `.aide/queue` canonical control-plane data or mutable state? Needs classification assurance.

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

A future assistant might restart broad structure refactoring despite the repo being past the structure crisis. Avoid this by checking the latest validators and focusing on targeted warnings.

A future assistant might add `framework/`, `modules/`, `profiles/`, or `labs/` roots because they seem conceptually clean. This would violate the hard-won closed-root doctrine unless explicitly approved by contracts.

A future assistant might treat raylib, SDL2, Lua, or ImGui as architecture rather than providers. Avoid by

enforcing provider manifests, forbidden include validators, and service-first structure.

A future assistant might assume Workbench modules are general modules. They are not. Workbench modules are UI/product modules; general module law lives under contracts/manifests/packs.

A future assistant might confuse generated reports with source truth. The chat repeatedly identified structure-report integrity as a problem.

A future assistant might treat PASS_WITH_WARNINGS as full green. It is not. Feature readiness remains limited until full-gate proof improves.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat should feed into spec-book chapters on repository structure, root policy, naming law, distribution/install layout, public surface registry, API/ABI policy, provider architecture, module/pack/app/workspace composition, Workbench architecture, presentation/projection design, testing/proof gates, AIDE workflow, and third-party provider policy.

Formal requirements candidates include: closed top-level root set; no `src/source`; service-first provider structure; provider profile model; third-party leakage fences; C17/C++17 with C-compatible ABI; public surface registry;

command/result/refusal/view/action projection spine; Workbench non-authority rule; and package/save/replay compatibility law.

Potential overlaps with other chats: worldgen, Workbench modules, raylib integration, provider wedges, language baseline, and AIDE workflow law. Potential conflicts: exact Lua version, exact external third-party path convention, exact contract singular/plural root names, and how soon to implement raylib providers versus projection contracts.

14. What I Should Remember

- The top-level structure is mostly settled; do not redesign it casually.
- Domino Framework is not a **framework**/ root; it is contracts, public surfaces, service/provider law, public headers, and conformance tests.
- Dominium is the game/product family; Domino is the reusable substrate.
- Raylib/SDL2/Lua are providers, not architecture.
- Apps should not hardwire providers in their paths; profiles select providers.
- Workbench is a projection/operator over commands, views, evidence, modules, packs, and artifacts; it is not authority.

- Module identity, pack identity, provider identity, app identity, and workspace identity are not paths.
- Fast strict and smoke can be green while full CTest remains red. Do not overclaim.
- Remaining work is proof hygiene and targeted hardening, not another broad root cleanup.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- "Summarize the Domino/Dominium split in one page."
- "Explain why we rejected a top-level `framework/` root."
- "Explain the difference between component, service, provider, pack, module, workspace, app, and artifact."

15.2 Decisions

- "Which decisions are definitely accepted versus only suggested?"
- "Which naming choices still need live repo verification?"

15.3 Tasks and Next Actions

- "Generate the next prompt for FAST-STRICT-EVIDENCE-MARKER-REPAIR-01."
- "Generate PROJECTION-CONFORMANCE-01."
- "Generate PROVIDER-WEDGE-01 for raylib/SDL2/Lua."

15.4 Artifacts and Files

- "List every generated handoff and audit artifact from this chat."
- "Which dirfiles exports were inconsistent and why?"

15.5 Risks and Verification

- "What are the highest-risk ways a future assistant could corrupt the architecture?"
- "What live repo facts should be verified before continuing?"

15.6 Future Spec Book / Aggregation

- "Turn this chat into a spec-book chapter outline."
- "Extract formal requirements from the chat."

15.7 Deep-Dive Questions Specific to This Chat

- "Should provider profiles live under `release/profiles` or `content/profiles`?"
- "How should `pack-internal content/` versus `data/` be resolved?"
- "What should the first raylib provider wedge implement?"

16. Compact Human Summary

This chat was a long, high-stakes architecture and repository convergence session for Dominium. The user wanted to end months of refactor churn and finally establish a structure capable of supporting real development. The conversation began with directory layout and grew into a broader architecture for a reusable Domino substrate, a Dominium game/product family, a Workbench production environment, and an AIDE control-plane harness.

The most important structural conclusion was to keep a small closed set of active roots: `apps`, `engine`, `game`, `runtime`, `contracts`, `content`, `docs`, `tests`, `tools`, `scripts`, `cmake`, `external`, `release`, and `archive`, plus a few project/tooling roots. The repo should not add `framework`, `modules`, `plugins`, `services`, `profiles`, `labs`, `src`, `source`, or vague junk roots. Paths encode

ownership, not identity. Public identity belongs in contracts, manifests, registries, stable IDs, public headers, and compatibility tests.

The chat then established the Domino versus Dominion distinction. Domino is the reusable deterministic/runtime framework. Dominion is one game/product family built on it. Workbench is a production and validation environment over the same contracts. AIDE is the control-plane harness. The "Domino Framework" should not be a new top-level framework/root; it is the collection of public surfaces, contracts, ABI law, service/provider law, public headers under `engine/include/domino` and `runtime/include/domino`, and conformance tests.

The main technical doctrine became: freeze contracts, replace implementations, preserve artifacts, verify behavior. C17/C++17 is the mainline language baseline, but public ABI remains C-compatible: POD structs, explicit versioning, opaque handles, explicit ownership, no C++ ABI leakage, and no third-party types in stable surfaces. Deterministic law remains first-party.

The Workbench/module discussion produced a precise vocabulary. A component is a source/build unit; a service is a callable runtime capability; a provider is a replaceable implementation; a pack is distributable authored payload; a module is a declared functional extension unit; a workspace is a large user-facing

Workbench composition; an app is a shipped product composition; an artifact is a persisted versioned object. Reusable modules should not live under `apps/workbench`; Workbench modules are UI modules only. General module law belongs under `contracts/module`, module payloads can be delivered through packs, and reusable behavior lives under `runtime`, `game`, or `engine` depending ownership.

The presentation/UI model was also unified. CLI, text/TUI, rendered GUI, native GUI, headless reports, Workbench panels, CI, and AIDE should all project the same semantic contracts. The spine is intent -> command -> capability/refusal check -> service -> result/document/snapshot -> diagnostics/evidence -> view/action model -> projection -> shell. This avoids building separate UI systems and prevents Workbench from becoming authority.

Third-party libraries were then placed into the provider model. Raylib, rlgl, rlsw, raygui, raudio, SDL2, Lua, and possibly ImGui are valuable accelerators, but only as providers. Runtime services own the stable identity; providers implement them. Correct paths look like `runtime/render/providers/raylib`, `runtime/platform/providers/sdl2`, `runtime/script/providers/lua54`, and provider choices live in profiles, not app folder names. Third-party types must not leak into `engine`, `game`, `contracts`, `content`, `saves`, `replays`,

packs, or public ABI.

Many executable Codex/AIDE prompts were generated or summarized, and the user reported commits that progressively cleaned the repo. The most recent state is credible but not perfect: canonical structure and old-path cleanup are mostly green; fast strict, CMake verify/build, smoke CTest, and targeted capability tests pass; full CTest/T4 remains debt; feature readiness is limited; broad feature work remains blocked. The next meaningful work is proof hygiene and targeted hardening, not another wide directory refactor.

**29.4.2 STRUCTURED REGISTERS -
 Dominium Canonical Structure and
 Domino Framework Architecture**

17. Workstream Register

Row 1 ID: WORKSTREAM-01; Name: Canonical
Repository Structure; Objective: End root/path
chaos and enforce ownership roots.; Current state:
Mostly clean; residual warnings remain.; Desired
end state: Closed root set with validators and no
stale active paths.; Status: PASS_WITH_
WARNINGS; Priority: P0; Confidence: High;
Label: FACT

Row 2 ID: WORKSTREAM-02; Name: Domino
Framework Boundary; Objective: Define

framework without new top-level root.; Current state: Reported committed and validated.; Desired end state: Framework identity lives in contracts/public headers/service law.; Status: PASS_WITH_WARNINGS; Priority: P0; Confidence: High; Label: FACT

Row 3 ID: WORKSTREAM-03; Name: Public Surface / ABI Governance; Objective: Register stable/public surfaces and enforce C-compatible ABI.; Current state: Scaffolds exist; warnings remain.; Desired end state: Every stable surface registered, versioned, tested.; Status: Active; Priority: P0; Confidence: Medium; Label: FACT

Row 4 ID: WORKSTREAM-04; Name: Service-first Provider Architecture; Objective: Make raylib/SDL/Lua providers, not architecture.; Current state: Provider structure partly committed; implementation still pending.; Desired end state: Runtime service providers with conformance tests and profiles.; Status: Active; Priority: P0; Confidence: High; Label: FACT

Row 5 ID: WORKSTREAM-05; Name: Presentation / Projection Spine; Objective: Unify CLI/TUI/rendered/native/headless over semantic views/actions.; Current state: Concept accepted; task remains active.; Desired end state: Projection

conformance over shared commands/views/
evidence.; Status: Active; Priority: P1; Confidence:
High; Label: FACT

Row 6 ID: WORKSTREAM-06; Name: Workbench
Product Spine; Objective: Build Workbench
modules/workspaces through contracts.; Current
state: Validation slice reported done; broad UI
blocked.; Desired end state: Workbench shell/
module/workspace over command/view/projection
system.; Status: Limited; Priority: P1; Confidence:
High; Label: FACT

Row 7 ID: WORKSTREAM-07; Name: Full-gate Proof
/ CTest; Objective: Remove stale tests and achieve
full release proof.; Current state: Legacy path
subset fixed; full CTest not green.; Desired end
state: Full CTest/T4 classified or green.; Status:
Active debt; Priority: P0; Confidence: High; Label:
FACT

Row 8 ID: WORKSTREAM-08; Name: AIDE Control
Plane; Objective: Govern AIDE queue/tasks/
evidence and avoid stale generated state.; Current
state: Some warnings remain.; Desired end state:
AIDE workflow law and clean evidence/queue
state.; Status: Active debt; Priority: P1;
Confidence: Medium; Label: FACT

Row 9 ID: WORKSTREAM-09; Name: Pack/Content

Authority; Objective: Keep authored packs under content and package law under contracts.; Current state: Mostly fixed; pack internal layout warnings remain.; Desired end state: Clear pack authority and consistent internal layout.; Status: Active debt; Priority: P1; Confidence: High; Label: FACT

18. Decision Register

Row 1 ID: DECISION-01; Decision: Keep closed canonical root set.; Status: Accepted; Evidence / basis: Repeated prompts/reports.; Rationale: Prevent root sprawl.; Implications: No casual new roots.; Related workstream: WORKSTREAM-01; Confidence: High; Label: FACT

Row 2 ID: DECISION-02; Decision: No first-party `src/source`.; Status: Accepted; Evidence / basis: User explicit objection.; Rationale: Avoid generic wrappers.; Implications: Validators should enforce.; Related workstream: WORKSTREAM-01; Confidence: High; Label: FACT

Row 3 ID: DECISION-03; Decision: Domino ? Dominium.; Status: Accepted; Evidence / basis: Multiple later summaries.; Rationale: Enables reuse.; Implications: Separate reusable substrate from game/product.; Related workstream: WORKSTREAM-02; Confidence: High; Label:

FACT

- Row 4** ID: DECISION-04; Decision: No top-level framework/.; Status: Accepted; Evidence / basis: User-reported commit repo: `define Domino framework boundary.`; Rationale: Avoid root ambiguity.; Implications: Framework is contracts/public headers.; Related workstream: WORKSTREAM-02; Confidence: High; Label: FACT
- Row 5** ID: DECISION-05; Decision: C17/C++17 mainline with C-compatible ABI.; Status: Accepted doctrine.; Evidence / basis: User pasted language-baseline synthesis.; Rationale: Modern implementation plus portable boundary.; Implications: ABI validators and public header rules.; Related workstream: WORKSTREAM-03; Confidence: Medium; Label: FACT/VERIFY
- Row 6** ID: DECISION-06; Decision: Third-party libraries are providers.; Status: Accepted; Evidence / basis: Raylib/SDL/Lua discussions.; Rationale: Avoid vendor lock-in.; Implications: Runtime provider paths and manifests.; Related workstream: WORKSTREAM-04; Confidence: High; Label: FACT
- Row 7** ID: DECISION-07; Decision: Workbench is not authority.; Status: Accepted; Evidence / basis:

Workbench/presentation discussions.; Rationale: Prevent UI bypass.; Implications: Workbench calls commands/services.; Related workstream: WORKSTREAM-06; Confidence: High; Label: FACT

Row 8 ID: DECISION-08; Decision: CLI/TUI/ rendered/native are projections.; Status: Accepted; Evidence / basis: Presentation spine discussions.; Rationale: Avoid four UI systems.; Implications: runtime/projection/*, semantic contracts.; Related workstream: WORKSTREAM-05; Confidence: High; Label: FACT

Row 9 ID: DECISION-09; Decision: Provider choices live in profiles, not app paths.; Status: Accepted; Evidence / basis: Provider structure reports.; Rationale: Avoid product variants.; Implications: Use release/profiles/content/profiles.; Related workstream: WORKSTREAM-04; Confidence: High; Label: FACT

19. Task Register

Row 1 ID: TASK-01; Task: Repair stale AuditX/ identity/launcher marker fast-strict debt.; Priority: P0; Urgency: U0; Owner: AIDE/Codex; Dependencies: Current repo state; Inputs needed: Latest fast-strict failure output; Expected output:

Clean normal gate; Next step: Generate/execute repair prompt; Related workstream: WORKSTREAM-07; Label: FACT

Row 2 ID: TASK-02; Task: Projection conformance.; Priority: P1; Urgency: U1; Owner: AIDE/Codex; Dependencies: Presentation contract basics; Inputs needed: Existing command/result/view artifacts; Expected output: Conformance tests for projection modes; Next step: Run PROJECTION-CONFORMANCE-01; Related workstream: WORKSTREAM-05; Label: FACT

Row 3 ID: TASK-03; Task: Presentation contract finalization.; Priority: P1; Urgency: U1; Owner: AIDE/Codex; Dependencies: Command/diagnostic contracts; Inputs needed: Current contracts tree; Expected output: View/action/projection schemas and validators; Next step: Run PRESENTATION-CONTRACT-01; Related workstream: WORKSTREAM-05; Label: FACT

Row 4 ID: TASK-04; Task: Raylib/SDL/Lua provider wedge.; Priority: P1; Urgency: U2; Owner: AIDE/Codex; Dependencies: Provider structure and third-party fences; Inputs needed: Provider manifests, vendored sources; Expected output: First provider implementation scaffold; Next step: Run PROVIDER-WEDGE-01; Related workstream:

WORKSTREAM-04; Label: INFERENCE

Row 5 ID: TASK-05; Task: Full CTest audit nonpath failures.; Priority: P0; Urgency: U1; Owner: AIDE/Codex; Dependencies: Legacy path route done; Inputs needed: Full CTest output; Expected output: Failure ledger; Next step: Run FULL-CTEST-AUDIT-NONPATH-01; Related workstream: WORKSTREAM-07; Label: FACT

Row 6 ID: TASK-06; Task: Pack internal layout canon.; Priority: P1; Urgency: U2; Owner: AIDE/Codex; Dependencies: Pack authority mostly fixed; Inputs needed: Current pack leaves; Expected output: Documented or normalized pack internals; Next step: Run pack layout task; Related workstream: WORKSTREAM-09; Label: FACT

Row 7 ID: TASK-07; Task: AIDE state classification.; Priority: P1; Urgency: U2; Owner: AIDE/Codex; Dependencies: AIDE control-plane model; Inputs needed: `.aide` tree; Expected output: Clear source/local/generated classification; Next step: Run AIDE state task; Related workstream: WORKSTREAM-08; Label: FACT

Row 8 ID: TASK-08; Task: Runtime/engine residual taxonomy.; Priority: P2; Urgency: U2; Owner: AIDE/Codex; Dependencies: Canonical structure cleanup; Inputs needed: Current roots; Expected

output: Classified session/serialization/foundation
etc.; Next step: Targeted cleanup; Related
workstream: WORKSTREAM-01; Label: FACT

20. Constraint Register

Row 1 ID: CONSTRAINT-01; Constraint: No new top-level roots without contract approval.; Type: Structure; Hard/soft: Hard; Source / basis: User and repeated doctrine.; Practical implication: Reject `framework/`, `modules/`, `profiles/`, `labs.`; Violation risk: Root sprawl.; Confidence: High; Label: FACT

Row 2 ID: CONSTRAINT-02; Constraint: No first-party `src/source` wrappers.; Type: Structure; Hard/soft: Hard; Source / basis: User explicit objection.; Practical implication: Implementation lives under ownership paths.; Violation risk: Refactor recurrence.; Confidence: High; Label: FACT

Row 3 ID: CONSTRAINT-03; Constraint: No third-party type leakage into stable law.; Type: Architecture; Hard/soft: Hard; Source / basis: Provider doctrine.; Practical implication: Raylib/SDL/Lua only in providers/proofs.; Violation risk: Vendor lock-in.; Confidence: High; Label: FACT

Row 4 ID: CONSTRAINT-04; Constraint: Public ABI

is C-compatible, not C++ ABI.; Type: ABI;
Hard/soft: Hard; Source / basis: Language
baseline doctrine.; Practical implication: POD/
opaque/versioned structs.; Violation risk: Binary
instability.; Confidence: Medium-High; Label:
FACT

Row 5 ID: CONSTRAINT-05; Constraint: Workbench
must use command/service/evidence spine.; Type:
Product; Hard/soft: Hard; Source / basis:
Workbench doctrine.; Practical implication: No
private validator/tool bypass.; Violation risk: UI
becomes authority.; Confidence: High; Label:
FACT

Row 6 ID: CONSTRAINT-06; Constraint: Full release
readiness requires full-gate proof.; Type: Testing;
Hard/soft: Hard for release; Source / basis:
Reported status.; Practical implication: Do not
claim release green.; Violation risk: False readiness.
; Confidence: High; Label: FACT

21. User Preference Register

Row 1 ID: PREF-01; Preference: Direct, rigorous,
audit-ready answers.; Area: Communication;
Explicit/inferred/uncertain: Explicit profile +
behavior; Strength: Strong; Practical implication:
Use clear status, caveats, evidence.; Risk if wrong:

User distrust.; Label: FACT

Row 2 ID: PREF-02; Preference: Avoid vague/soft language when structure is bad.; Area: Communication; Explicit/inferred/uncertain: Inferred; Strength: Strong; Practical implication: Say not done when not done.; Risk if wrong: Misleading reassurance.; Label: INFERENCE

Row 3 ID: PREF-03; Preference: Prefer executable prompts for Codex/AIDE.; Area: Workflow; Explicit/inferred/uncertain: Explicit through repeated requests; Strength: Strong; Practical implication: Generate concrete tasks with acceptance criteria.; Risk if wrong: Work stalls.; Label: FACT

Row 4 ID: PREF-04; Preference: Do not ask unnecessary clarifying questions.; Area: Workflow; Explicit/inferred/uncertain: Explicit system/user preference; Strength: Strong; Practical implication: Proceed with assumptions and labels.; Risk if wrong: User frustration.; Label: FACT

Row 5 ID: PREF-05; Preference: Keep architecture future-proof and reusable.; Area: Technical; Explicit/inferred/uncertain: Explicit; Strength: Strong; Practical implication: Favor contracts/providers/profiles over vendor/product paths.; Risk if wrong: Future refactor.; Label: FACT

22. Open Questions Register

Row 1 ID: QUESTION-01; Question / issue: Is full CTest green after latest fixes?; Why it matters: Determines release/full proof readiness.; Known information: Last visible status: not green/not fully run.; Unknown information: Current live status.; Resolution path: Run full CTest or audited shard.; Priority: P0; Related workstream: WORKSTREAM-07; Label: FACT

Row 2 ID: QUESTION-02; Question / issue: Lua 5.4 or 5.5?; Why it matters: Script ABI/provider pinning.; Known information: Chat says pin one.; Unknown information: Final choice.; Resolution path: Decide after compatibility/security review.; Priority: P1; Related workstream: WORKSTREAM-04; Label: UNCERTAIN

Row 3 ID: QUESTION-03; Question / issue: external/upstream or external/vendor?; Why it matters: Consistency and provenance.; Known information: Both discussed; final should choose one.; Unknown information: Latest repo canon.; Resolution path: Check contracts/docs.; Priority: P1; Related workstream: WORKSTREAM-04; Label: UNCERTAIN

Row 4 ID: QUESTION-04; Question / issue: Is pack internal content/ canonical?; Why it matters:

Pack layout and tooling.; Known information: Warning remains.; Unknown information: Pack law.; Resolution path: Run pack internal layout task.; Priority: P1; Related workstream: WORKSTREAM-09; Label: FACT

Row 5 ID: QUESTION-05; Question / issue: Are `.aide/cache` and `.aide/queue` source or local state?; Why it matters: Prevent generated/live state confusion.; Known information: Need classification.; Unknown information: Exact contents.; Resolution path: AIDE state classification task.; Priority: P1; Related workstream: WORKSTREAM-08; Label: FACT

23. Artifact Ledger

Row 1 ID: ARTIFACT-01; Artifact / file / prompt / output: `dominium_canonical_handoff.md/.txt`; Type: Handoff files; Purpose: Earlier self-contained summary.; Status: Created earlier; Origin: This chat; Carry forward?: Yes; Notes: Historical snapshot as of 2026-05-20.; Label: FACT

Row 2 ID: ARTIFACT-02; Artifact / file / prompt / output: `dir_tree.json, dirfiles_manifest.json`, ZIP exports; Type: Structure evidence; Purpose: Inspect repo tree.; Status: Mixed quality; Origin: User uploads; Carry forward?: Yes with

caveats; Notes: Many bundles were inconsistent/
stale.; Label: FACT

Row 3 ID: ARTIFACT-03; Artifact / file / prompt /
output: CANON_STRUCTURE_ACTUAL_FINAL_
CLEANUP_01.md; Type: Audit; Purpose: Actual
structure cleanup.; Status: User-reported
committed; Origin: Repo; Carry forward?: Yes;
Notes: Key proof of structure cleanup.; Label:
FACT

Row 4 ID: ARTIFACT-04; Artifact / file / prompt /
output: FULL_GATE_LEGACY_TEST_ROUTE_01.md;
Type: Audit; Purpose: Routed stale full-gate tests.
; Status: User-reported committed; Origin: Repo;
Carry forward?: Yes; Notes: Important proof-gate
improvement.; Label: FACT

Row 5 ID: ARTIFACT-05; Artifact / file / prompt /
output: domino_framework_boundary.md; Type:
Architecture doc; Purpose: Defines framework
without root.; Status: User-reported committed;
Origin: Repo; Carry forward?: Yes; Notes: Core
decision artifact.; Label: FACT

Row 6 ID: ARTIFACT-06; Artifact / file / prompt /
output: Large Codex/AIDE prompts; Type:
Prompts; Purpose: Execute cleanup and
governance tasks.; Status: Many generated in chat;

Origin: This chat; Carry forward?: Yes; Notes:
Useful prompt templates.; Label: FACT

Row 7 ID: ARTIFACT-07; Artifact / file / prompt /
output: Current preservation package; Type:
Report package; Purpose: Preserve this chat.;
Status: Created now; Origin: This response; Carry
forward?: Yes; Notes: See file links.; Label: FACT

24. Rejected / Superseded Options Register

Row 1 ID: REJECTED-01; Option: Top-level
framework/; Status: Rejected; Reason: Competes
with contracts/public headers.; Final or tentative?:
Mostly final; Reconsider conditions: External
SDK/root contract requires it.; Related
workstream: WORKSTREAM-02; Label: FACT

Row 2 ID: REJECTED-02; Option: Top-level
modules/; Status: Rejected; Reason: Would
become junk drawer.; Final or tentative?: Final
unless separate module repo.; Reconsider
conditions: True module repository.; Related
workstream: WORKSTREAM-06; Label: FACT

Row 3 ID: REJECTED-03; Option: Product paths like
apps/client/rendered/raylib; Status: Rejected;
Reason: Provider choice belongs in profiles.; Final
or tentative?: Final; Reconsider conditions:

Temporary proof boot only.; Related workstream: WORKSTREAM-04; Label: FACT

Row 4 ID: REJECTED-04; Option: Raylib as engine architecture; Status: Rejected; Reason: Vendor lock-in.; Final or tentative?: Final; Reconsider conditions: None; raylib remains provider.; Related workstream: WORKSTREAM-04; Label: FACT

Row 5 ID: REJECTED-05; Option: C89/C++98 as mainline; Status: Superseded; Reason: Too restrictive for current target.; Final or tentative?: Tentative on platform verification; Reconsider conditions: Retro/research lane.; Related workstream: WORKSTREAM-03; Label: FACT

Row 6 ID: REJECTED-06; Option: Separate UI systems for CLI/TUI/rendered/native; Status: Rejected; Reason: Duplicates behavior.; Final or tentative?: Final; Reconsider conditions: None; projections only.; Related workstream: WORKSTREAM-05; Label: FACT

25. Risk Register

Row 1 ID: RISK-01; Risk / failure mode: Future assistant restarts broad structure churn.; Consequence: Lost time/regression.; Likelihood: Medium; Severity: High; Mitigation: Use targeted

tasks only; check validators.; Related workstream:
WORKSTREAM-01; Label: INFERENCE

Row 2 ID: RISK-02; Risk / failure mode: PASS_
WITH_WARNINGS treated as full green.;
Consequence: Premature feature/release work.;
Likelihood: Medium; Severity: High; Mitigation:
Preserve readiness labels.; Related workstream:
WORKSTREAM-07; Label: FACT

Row 3 ID: RISK-03; Risk / failure mode: Third-party
types leak into contracts/game/saves.;
Consequence: Vendor lock-in.; Likelihood:
Medium; Severity: High; Mitigation: Forbidden
include/type validators.; Related workstream:
WORKSTREAM-04; Label: FACT

Row 4 ID: RISK-04; Risk / failure mode: Workbench
calls private tools directly.; Consequence: UI
becomes authority.; Likelihood: Medium; Severity:
High; Mitigation: Command/view/action
contracts.; Related workstream: WORKSTREAM-
06; Label: FACT

Row 5 ID: RISK-05; Risk / failure mode: Mixed
structure reports mislead agents.; Consequence:
Chasing stale paths.; Likelihood: High historically;
Severity: Medium; Mitigation: Structure report
integrity validator.; Related workstream:
WORKSTREAM-01; Label: FACT

Row 6 ID: RISK-06; Risk / failure mode: Full CTest remains unclassified.; Consequence: Hidden release blockers.; Likelihood: Medium; Severity: High; Mitigation: Full-gate audit ledger.; Related workstream: WORKSTREAM-07; Label: FACT

26. Verification Queue

Row 1 ID: VERIFY-01; Item requiring verification: Latest live repo status after last commit.; Why verification is needed: User reports may be stale.; Suggested source/type: Git status / latest audit.; Priority: P0; Related workstream: All; Label: FACT

Row 2 ID: VERIFY-02; Item requiring verification: Full CTest result.; Why verification is needed: Full release readiness.; Suggested source/type: CTest output ledger.; Priority: P0; Related workstream: WORKSTREAM-07; Label: FACT

Row 3 ID: VERIFY-03; Item requiring verification: Third-party versions/support floors.; Why verification is needed: External facts change.; Suggested source/type: Official raylib/SDL/Lua docs.; Priority: P1; Related workstream: WORKSTREAM-04; Label: VERIFY

Row 4 ID: VERIFY-04; Item requiring verification: Provider profile path convention.; Why verification

is needed: Avoid path churn.; Suggested source/
type: Current contracts/docs.; Priority: P1;
Related workstream: WORKSTREAM-04; Label:
UNCERTAIN

Row 5 ID: VERIFY-05; Item requiring verification:
AIDE state classification.; Why verification is
needed: Prevent local/generated contamination.;
Suggested source/type: Inspect `.aide` dirs.;
Priority: P1; Related workstream:
WORKSTREAM-08; Label: FACT

27. Chronological Timeline Register

Row 1 Sequence: 1; Event / topic: Initial directory-
layout debate; What changed or was decided: Need
multiple layouts but one virtual-root/projection
model.; Why it mattered: Prevent install/package/
source confusion.; Current relevance: Background
doctrine.; Confidence: High

Row 2 Sequence: 2; Event / topic: User backlash
against `src`/root sprawl; What changed or was
decided: No `src`/`source`, no new roots.; Why it
mattered: Core constraint.; Current relevance: Still
active.; Confidence: High

Row 3 Sequence: 3; Event / topic: Canonical root
model established; What changed or was decided:
Closed source root set.; Why it mattered: End root

chaos.; Current relevance: Current basis.;
Confidence: High

Row 4 Sequence: 4; Event / topic: Cleanup prompts
generated/executed; What changed or was decided:
Runtime/game/schema/content/tools/docs/tests
moved.; Why it mattered: Practical convergence.;
Current relevance: Historical/audit context.;
Confidence: High

Row 5 Sequence: 5; Event / topic: Public surface/API/
ABI phase; What changed or was decided: Need
stable contract governance.; Why it mattered:
Reuse/future-proofing.; Current relevance: Active.;
Confidence: High

Row 6 Sequence: 6; Event / topic: C17/C++17
baseline; What changed or was decided: Mainline
implementation modernized.; Why it mattered:
Language policy.; Current relevance: Active, verify
toolchains.; Confidence: Medium

Row 7 Sequence: 7; Event / topic: Module/workbench
vocabulary split; What changed or was decided:
Component/service/provider/pack/module/
workspace/app terms.; Why it mattered: Prevent
junk drawers.; Current relevance: Active.;
Confidence: High

Row 8 Sequence: 8; Event / topic: Presentation spine;
What changed or was decided: UI modes become

projections.; Why it mattered: Reusable
Workbench/client/UI.; Current relevance: Active.;
Confidence: High

Row 9 Sequence: 9; Event / topic: Provider/raylib/
SDL/Lua model; What changed or was decided:
Third-party as providers.; Why it mattered: Fast
progress + replaceability.; Current relevance:
Active.; Confidence: High

Row 10 Sequence: 10; Event / topic: Framework
boundary; What changed or was decided: No
framework/ root.; Why it mattered: Prevent root
sprawl.; Current relevance: Active.; Confidence:
High

Row 11 Sequence: 11; Event / topic: Full-gate legacy
routing; What changed or was decided: Retired
path tests updated.; Why it mattered: Cleaner
proof signal.; Current relevance: Recent status.;
Confidence: High

28. Spec Book Contribution Register

Row 1 Spec-book area: Repo structure; Contribution
from this chat: Closed root model, no src/source.;
Source IDs: DECISION-01, DECISION-02; Should
become requirement/context/open issue?:
Requirement; Confidence: High; Notes: Core
canon.

- Row 2** Spec-book area: Framework architecture;
Contribution from this chat: Domino framework
via contracts/public headers.; Source IDs:
DECISION-03, DECISION-04; Should become
requirement/context/open issue?: Requirement;
Confidence: High; Notes: Avoid top-level
framework.
- Row 3** Spec-book area: Provider architecture;
Contribution from this chat: Service-first providers
and profiles.; Source IDs: DECISION-06; Should
become requirement/context/open issue?:
Requirement; Confidence: High; Notes: Raylib/
SDL/Lua policy.
- Row 4** Spec-book area: Workbench architecture;
Contribution from this chat: Workbench as
projection/operator.; Source IDs: DECISION-07,
DECISION-08; Should become requirement/
context/open issue?: Requirement/context;
Confidence: High; Notes: Guides first UI work.
- Row 5** Spec-book area: ABI/language policy;
Contribution from this chat: C17/C++17 + C-
compatible ABI.; Source IDs: DECISION-05;
Should become requirement/context/open issue?:
Requirement after verification; Confidence:
Medium; Notes: External facts need verification.
- Row 6** Spec-book area: Testing/proof gates;

Contribution from this chat: Fast strict vs full gate.; Source IDs: TASK-01, TASK-05; Should become requirement/context/open issue?: Requirement/open issue; Confidence: High; Notes: Full CTest debt remains.

29.4.3 Context Transfer Packet for a Future Chat

Ultra-Condensed Bootstrap Brief

This old chat concerned Dominium's transition from a messy, root-sprawling game repository into a deterministic, contract-governed, provider-backed, pack-composed simulation platform built on a reusable Domino substrate. The user's core need was to stop months of structure churn and establish an architecture that supports real development of engine, game, Workbench, apps, modules, packs, providers, and future games without another large refactor.

The major accepted root model is: `apps/`, `engine/`, `game/`, `runtime/`, `contracts/`, `content/`, `docs/`, `tests/`, `tools/`, `scripts/`, `cmake/`, `external/`, `release/`, `archive/`, plus `.aide/`, `.aide.local.example/`, `.github/`, `.vscode/`. Do not add top-level `framework/`, `modules/`, `plugins/`, `services/`, `profiles/`, `labs/`, `sdk/`, `src/`, or `source/` unless explicitly contracted.

The key doctrine is: path is not identity; implementation

is not contract; UI is not authority; generated output is not source truth. Domino Framework is not a source root; it is the public-surface package made of contracts, ABI law, service/provider law, public headers under `engine/include/domino` and `runtime/include/domino`, and conformance tests. Dominion is the game/product family. Workbench is the production/editor/evidence environment. AIDE is the control-plane harness.

The system vocabulary is: component = source/build unit; service = callable runtime capability; provider = replaceable implementation; pack = authored distributable payload; module = declared functional extension unit; workspace = Workbench composition; app = product composition; artifact = persisted versioned object. Raylib, rlgl, rlsw, raygui, raudio, SDL2, Lua, and similar libraries are providers, not architecture. Provider implementations live under `runtime/<service>/providers/<provider>/`; provider selections live in `release/profiles/` or `content/profiles/`; apps stay generic.

The presentation model is: intent -> command -> capability/refusal check -> service -> result/document/snapshot -> diagnostics/evidence -> view/action model -> projection -> shell. CLI, text/TUI, rendered GUI, native GUI, headless, Workbench, CI, and AIDE should be projections of the same semantic surfaces.

Current repo status per user reports: broad structure chaos is mostly resolved; canonical structure and fast strict pass or pass with warnings; full CTest/T4 release proof remains debt; broad feature work remains blocked; narrow governed tasks continue. Recent reported work included final canonical structure cleanup, full-gate legacy path routing, provider-structure enforcement, and Domino framework boundary definition.

Recommended next action depends on latest repo status: if fast-strict/RepoX still fail on stale AuditX/identity or launcher marker debt, repair that first. Otherwise proceed with PROJECTION-CONFORMANCE-01 and/or PRESENTATION-CONTRACT-01, then provider wedge tasks for raylib/SDL/Lua.

Source Hierarchy

1. Direct user statements in this chat.
2. Explicit user-accepted decisions and user-reported commits/statuses.
3. Task and constraint registers in this preservation package.
4. Uploaded directory tree/status artifacts, with caveats for stale/mixed runs.
5. Earlier generated handoff files in /mnt/data.

6. Inferences clearly marked.
7. Assistant suggestions not accepted by the user.
8. General model knowledge.

Operating Rules for Future Assistants

Preserve FACT / INFERENCE / UNCERTAIN / PROJECT-CONTEXT labels. Do not assume access to the old chat. Do not re-ask questions already answered. Verify stale facts before relying on them. Do not treat tentative brainstorming as decisions. Do not repeat rejected options such as adding top-level **framework/**, **modules/**, or **profiles/**. Preserve artifacts and use structured outputs when continuing. Do not restart broad structure cleanup unless a validator finds a hard blocker.

Active Workstreams

- Canonical structure and root policy.
- Domino Framework boundary and public surface governance.
- C17/C++17 and C-compatible ABI policy.
- Service-first provider architecture and third-party fencing.

- Presentation/projection spine.
- Workbench module/workspace development.
- Full-gate proof repair.
- AIDE workflow/evidence classification.

Current Priorities

1. Repair stale proof/evidence/marker debt if fast-strict is blocked.
2. Proceed to PROJECTION-CONFORMANCE-01 / PRESENTATION-CONTRACT-01 when gates allow.
3. Implement provider wedge for raylib/SDL/Lua only through service/provider contracts.
4. Continue full CTest/T4 audit.

Current Open Questions

- Is full CTest green in the live repo?
- Which Lua version is pinned?
- Is external/upstream or external/vendor canonical?
- Are AIDE cache/queue/reports canonical control-plane data or local/generated state?
- Is pack-internal content/ canonical or legacy?

Recommended First Action

Check the latest live repo gate status. If fast-strict or RepoX still fails due stale evidence/marker debt, run a focused proof-hygiene repair. If normal gates are clean, run **PROJECTION-CONFORMANCE-01** next.

spec_sheet:

metadata:

chat_label: "Dominium Canonical Structure and Domino Framework Architecture"

date_anchor: "2026-05-27 Australia/Melbourne"

source_scope: "This chat only unless labelled PROJECT-CONTEXT"

apparent_coverage: "Partial but substantial"

confidence_1_to_5: 4

staleness_risk: "medium-high for live repo state; medium for external library facts; low for explicit architecture decisions"

safe_for_aggregation: "yes_with_caveats"

main_limitations:

- "Full transcript export unavailable."
- "Some uploaded files expired."
- "Several repo structure bundles in the chat were mixed/stale."

- "Latest live repo state should be verified before action."

summary:

one_sentence: "This chat converged Dominium from directory-structure chaos toward a Domino framework model based on contracts, public surfaces, services, providers, packs, modules, profiles, proofs, and narrow governed product slices."

short_brief: "The chat established a closed top-level root model, rejected src/source/root sprawl, distinguished Domino from Dominium, defined service-first provider architecture for raylib/SDL/Lua, unified Workbench and UI around command/view/action/projection contracts, and tracked multiple cleanup/proof commits while preserving full-gate debt as unresolved."

main_topics:

- "Canonical repository structure"
- "Domino/Dominium framework boundary"
- "Public surface and ABI governance"
- "Service-first providers"
- "Workbench modules/workspaces"
- "Presentation/projection architecture"

- "Full-gate proof debt"

main_outputs:

- "Earlier handoff files"
- "Large Codex/AIDE cleanup prompts"
- "Structured doctrine for roots, providers, modules, packs, apps, workspaces"

highest_priority_carry_forward:

- "Do not add top-level framework/modules/profiles/labs/sdk roots."
- "Keep third-party libraries behind runtime providers."
- "Repair proof-gate debt before broad feature work."

source_rules:

labels_used: ["FACT", "INFERENCE", "UNCERTAIN / UNVERIFIED", "PROJECT-CONTEXT"]

conflict_rules:

- "User-reported current repo state supersedes older uploaded directory trees if clearly later."
- "Fresh tracked-only exports supersede mixed or stale dirfiles bundles."

- "Explicit user acceptance supersedes assistant brainstorm."

staleness_rules:

- "External library versions and platform support require verification."
- "Live repo state requires fresh git/validator checks."

user_preferences:

explicit:

- "Direct, audit-ready, source-grounded answers."
- "No src/source folder sprawl."
- "No casual new top-level roots."
- "Generate executable prompts when requested."

inferred:

- "The user wants decisive status labels and no false reassurance."
- "The user values architecture that can survive future rewrites."

uncertain_or_not_established:

- "Exact singular/plural spelling for some contract roots."
- "Exact external/upstream versus external/vendor convention."

workstreams:

- id: WORKSTREAM-01

name: "Canonical Repository Structure"

label: FACT

objective: "Stop root/path chaos."

current__state: "Mostly clean with warnings."

desired__end__state: "Closed root set and no active stale paths."

status: "PASS_WITH_WARNINGS"

priority: P0

background: "Repo had many bad roots and redundant structures."

decisions__made: [DECISION-01, DECISION-02]

decisions__pending: []

tasks: [TASK-05, TASK-08]

constraints: [CONSTRAINT-01, CONSTRAINT-02]

dependencies: []

timeline: "Major cleanup occurred across many reported commits."

blockers: []

risks: [RISK-01, RISK-05]

artifacts: [ARTIFACT-02, ARTIFACT-03]

success_criteria: ["no unexpected top-level roots",
"canonical structure validator passes"]

next_action: "Targeted residuals only."

verification_needed: [VERIFY-01]

confidence: high

- id: WORKSTREAM-04

name: "Service-first Provider Architecture"

label: FACT

objective: "Use third-party libraries as replaceable providers."

current_state: "Provider structure reportedly committed; provider implementations pending."

desired_end_state: "Providers selected by profiles and tested by conformance suites."

status: "active"

priority: P0

background: "Raylib/SDL/Lua considered as

accelerators."

decisions__made: [DECISION-06]

decisions__pending: ["Lua version", "external source path convention"]

tasks: [TASK-04]

constraints: [CONSTRAINT-03]

dependencies: [WORKSTREAM-03]

timeline: "Provider model refined after rejecting raylib-shaped architecture."

blockers: []

risks: [RISK-03]

artifacts: []

success__criteria: ["forbidden include validator passes", "provider conformance tests exist"]

next__action: "Provider wedge after proof gates."

verification__needed: [VERIFY-03, VERIFY-04]

confidence: high

decisions:

- id: DECISION-04

decision: "Do not add top-level framework/."

status: "accepted"

label: FACT

evidence_or_basis: "User-reported Domino framework boundary commit."

rationale: "Framework should be contracts/public headers/service law, not a new source root."

implications: "Public APIs live under contracts and include roots."

related_workstreams: [WORKSTREAM-02]

uncertainty: "Need latest repo verification for exact files."

- id: DECISION-06

decision: "Third-party libraries are providers."

status: "accepted"

label: FACT

evidence_or_basis: "Multiple raylib/SDL/Lua turns and provider-structure commit report."

rationale: "Allows acceleration without vendor lock-in."

implications: "runtime/<service>/providers/
<provider>; apps select profiles."

related_workstreams: [WORKSTREAM-04]

uncertainty: "Provider implementations not yet complete."

tasks:

- id: TASK-01

task: "Repair fast-strict stale evidence/launcher marker debt."

priority: P0

urgency: U0

owner: "AIDE/Codex"

dependencies: []

inputs__needed: ["latest fast-strict/RepoX output"]

expected__output: "clean normal gate or exact failure ledger"

next__step: "generate focused repair prompt"

related__workstreams: [WORKSTREAM-07]

label: FACT

confidence: high

constraints:

- id: CONSTRAINT-01

constraint: "No new top-level roots without explicit contract approval."

type: "repository structure"

hard__or__soft: hard

source__or__basis: "repeated user requirement and cleanup doctrine"

implication: "Reject framework/modules/profiles/labs as

roots."

violation_risk: "root sprawl and renewed restructure cycle"

label: FACT

confidence: high

open_questions:

- id: QUESTION-02

question: "Which Lua version should be pinned?"

why_it_matters: "Lua ABI/version affects script provider and mod compatibility."

known: "Scripts should target dominium/domino script API, not raw Lua ABI."

unknown: "Final Lua 5.4 vs 5.5 choice."

resolution_path: "Verify platform/library constraints and choose one."

priority: P1

related_workstreams: [WORKSTREAM-04]

label: UNCERTAIN

rejected_or_superseded_options:

- id: REJECTED-01

option: "Top-level framework/ root"

status: "rejected"

reason: "Duplicates contracts/public headers and
reopens root sprawl."

final_or_tentative: "mostly final"

reconsider_conditions: "only if root contract is explicitly
changed for external SDK/source package"

related_workstreams: [WORKSTREAM-02]

label: FACT

artifacts:

- id: ARTIFACT-01

name_or_description: "dominium_canonical_handoff.
md and .txt"

type: "handoff"

purpose: "Earlier continuation summary"

status: "created earlier and mounted"

origin: "this chat"

carry_forward: true

notes: "Historical snapshot, not latest state"

label: FACT

risks:

- id: RISK-03

risk: "Third-party type leakage into stable surfaces"

consequence: "vendor lock-in and replay/save
incompatibility"

likelihood: medium

severity: high

mitigation: "forbidden include/type validators and
provider boundary docs"

related_workstreams: [WORKSTREAM-04]

label: FACT

verification_queue:

- id: VERIFY-01

item: "latest live repo state"

why_verification_needed: "user-reported commits may
not reflect current branch"

suggested_source_type: "git status, latest audits,
validator run"

priority: P0

related_workstreams: [WORKSTREAM-01,
WORKSTREAM-07]

label: FACT

spec_book_notes:

likely_sections:

- "Repository structure"
- "Domino Framework boundary"
- "Service/provider architecture"
- "Workbench/presentation architecture"
- "Testing and proof gates"

unique_contributions:

- "Detailed root-policy refinement"
- "Raylib/SDL/Lua provider doctrine"
- "No-framework-root decision"

possible_duplicates_with_other_chats:

- "Workbench brainstorming"
- "Language baseline discussions"
- "Raylib provider discussions"

conflicts_to_watch_for:

- "Exact external path convention"
- "Lua version"
- "Diagnostic root singular/plural"

formal_requirements_candidates:

- "No top-level framework/modules/profiles/labs roots"
- "runtime/<service>/providers/<provider> provider structure"
- "C-compatible public ABI"

background_context_candidates:

- "Historical frustration and repeated refactor churn"

needs_user_confirmation:

- "Final Lua version"
- "Provider implementation sequencing"

final_recommendations:

next_action_if_continuing_this_chat: "Verify latest repo gate status, then repair fast-strict/evidence debt or proceed to projection conformance."

next_action_for_aggregator: "Merge this as the canonical repository/framework/provider architecture thread."

user_checks_required:

- "Confirm latest commit and full CTest state."
- "Confirm provider convention names if repo changed after this chat."

29.4.4 Aggregator Packet - Dominion Canonical Structure and Domino Framework Architecture

Packet Metadata

- Chat label: Dominion Canonical Structure and Domino Framework Architecture
- Date anchor: 2026-05-27 Australia/Melbourne
- Source scope: This chat only unless labelled PROJECT-CONTEXT
- Coverage: Partial but substantial
- Confidence: 4/5 for accessible substance; lower for exact latest repo state
- Staleness risk: Medium to High for live repo status and third-party versions
- Merge priority: High
- Main limitations: Full transcript not separately exported; some uploads expired; several structure bundles were stale/mixed.

Ultra-Condensed Carry-Forward Capsule

This chat is the main architecture/convergence thread for Dominion's repository structure and Domino framework model. The user wanted to stop months of structure churn and finally make the repo suitable for real development. The conversation converged on a closed top-level root model: apps, engine, game, runtime, contracts, content, docs, tests, tools, scripts, cmake, external, release, archive, plus limited project/tooling roots. New top-level framework/modules/plugins/services/profiles/labs/sdk/src/source roots were rejected.

The central doctrine is: path is not identity; implementation is not contract; UI is not authority; generated output is not source truth. Domino is the reusable deterministic/runtime framework; Dominion is the game/product family; Workbench is the production/evidence/editor environment; AIDE is the control-plane harness. Domino Framework should not become a **framework/** root. It is represented by contracts, public surfaces, ABI law, service/provider law, public headers, and conformance tests.

The chat defined precise vocabulary: component = source/build unit; service = callable runtime capability; provider = replaceable implementation; pack = authored distributable payload; module = declared extension unit; workspace = Workbench composition; app = product

composition; artifact = persisted versioned object.
Workbench modules are not the general module system.
Presentation architecture became a major result. CLI, text/TUI, rendered GUI, native GUI, headless reports, Workbench panels, AIDE, and CI should all project the same semantic spine: intent -> command -> capability/refusal check -> service -> result/document/snapshot -> diagnostics/evidence -> view/action model -> projection -> shell.

The chat also set the raylib/SDL/Lua doctrine. Raylib, rlgl, rlsw, raygui, raudio, SDL2, Lua, and possibly ImGui are providers. Their code belongs under external and runtime provider paths. Third-party types must not leak into engine, game, contracts, content, saves, replays, packs, or public ABI. Provider choices live in profiles, not app path names.

Multiple cleanup tasks/prompts were generated and user-reported commits landed: actual canonical structure cleanup, full-gate legacy path routing, provider structure enforcement, and Domino framework boundary definition. The current state is credible but not full release green. Structure is mostly fixed; fast strict/smoke generally pass or pass with warnings; full CTest/T4 remains debt; broad feature work remains blocked; narrow governed work can continue after gate-specific blockers are repaired.

Top Carry-Forward Items

Row 1 Priority: P0; Item: Closed top-level root set;
Type: Requirement; Related ID: DECISION-01;
Why it matters: Prevents root sprawl; Label:
FACT; Confidence: High

Row 2 Priority: P0; Item: No top-level framework/;
Type: Requirement; Related ID: DECISION-04;
Why it matters: Preserves contract/public-header
model; Label: FACT; Confidence: High

Row 3 Priority: P0; Item: Service-first provider
structure; Type: Requirement; Related ID:
DECISION-06; Why it matters: Enables raylib/
SDL/Lua without lock-in; Label: FACT;
Confidence: High

Row 4 Priority: P0; Item: C-compatible ABI; Type:
Requirement; Related ID: DECISION-05; Why it
matters: Enables portability/replacement; Label:
FACT; Confidence: Medium-High

Row 5 Priority: P1; Item: Projection spine; Type:
Requirement; Related ID: DECISION-08; Why it
matters: Prevents four UI systems; Label: FACT;
Confidence: High

Row 6 Priority: P1; Item: Full CTest/T4 debt remains;
Type: Open issue; Related ID: QUESTION-01;

Why it matters: Blocks release/full proof; Label: FACT; Confidence: High

Workstream Summaries

- ID: WORKSTREAM-01
- Name: Canonical Repository Structure
- Objective: Keep source roots closed and active paths canonical.
- Current state: Mostly clean with warnings.
- Desired end state: No stale active paths; validators enforce structure.
- Priority: P0
- Decisions: DECISION-01, DECISION-02
- Tasks: targeted residuals only.
- Constraints: no new roots, no src/source.
- Artifacts: structure cleanup audits, dirfiles exports.
- Risks: restarting broad churn.
- Open questions: latest live status.
- Next action: verify latest tree if needed.

- ID: WORKSTREAM-04

- Name: Service-first Provider Architecture
- Objective: Use raylib/SDL/Lua as replaceable providers.
- Current state: Provider law/structure reported in place; implementation pending.
- Desired end state: runtime/<service>/providers/<provider> with profiles and conformance.
- Priority: P0
- Decisions: DECISION-06
- Tasks: Provider wedge after proof hygiene.
- Constraints: no third-party leakage.
- Risks: vendor-shaped architecture.
- Open questions: Lua version, external path convention.

Compact Registers for Merge

Use sections 17-28 of the full register file as authoritative compact registers.

Possible Cross-Chat Duplicates

Workbench brainstorming, language baseline, raylib/SDL/Lua provider discussions, AIDE workflow law, and

worldgen/module composition may overlap with other old chats.

Possible Cross-Chat Conflicts

Exact naming of `external/upstream` versus `external/vendor`, Lua version, diagnostic/diagnostics root, and the timing of provider wedge versus presentation contract may conflict with later chats.

Spec Book Integration Guidance

This chat should feed into chapters on repo structure, framework boundary, provider architecture, public surfaces, ABI, Workbench/presentation, modules/packs/apps, testing/proof gates, and AIDE governance. Treat most architecture doctrine as candidate requirements; treat exact current repo status as verification-dependent.

Aggregator Warnings

Do not merge assistant brainstorming as final user decisions unless user accepted it. Do not overclaim full green. Do not add new roots. Verify live repo state before executing tasks.

29.4.5 Verification and Audit - Dominion Canonical Structure and Domino Framework Architecture

Adversarial Self-Audit

Issue	Severity	Correction needed	Correction applied?	Residual risk
Full transcript not accessible as separate export.	Medium	Mark coverage partial.	Yes	Some turn-level detail may be omitted.
Some uploaded files expired.	Medium	Note file-access limitation.	Yes	Missing historical artifacts.
User-reported repo commits may be stale.	High	Add verification queue.	Yes	Must check live repo before action.
Assistant suggestions may be mistaken for decisions.	High	Label accepted vs suggested.	Yes	Some implicit acceptance still inferred.
External library facts may be stale.	Medium	Mark for verification.	Yes	Need web/current docs before release policy.

Corrections Applied

- Coverage was labelled partial rather than full.
- Tentative decisions were separated from accepted decisions where possible.
- External facts and live repo state were placed in the verification queue.
- Rejected options were preserved.
- The report warns against treating PASS_WITH_WARNINGS as full green.

Final Reliability Assessment

- Completeness rating: 4/5 for accessible substance.
- Reliability rating: 4/5 for architectural decisions; 3/5 for exact latest repo state.
- Human-readability rating: 4/5.
- Aggregation-readiness rating: 4/5 with caveats.
- Main remaining uncertainty sources: live repo state, full CTest status, exact provider version choices, expired uploads, and whether later chats superseded some details.
- Manual review before merge: recommended.

Verification Queue

See register section 26.

29.4.6 In-Chat Reader - Dominion Canonical Structure and Domino Framework Architecture

Package overview

This preservation package captures the substance of the Dominion canonical repository structure and Domino framework architecture chat. It is intended for human reading, future chat continuation, and later aggregation into a project spec book.

File index

Read first: 01_human_readable_report.md.

Use for continuing in another chat: 02_context_transfer_packet.md.

Use for aggregation: 03_spec_sheet.yaml and 05_aggregator_packet.md.

Use for precise tables: 04_registers.md.

Use for caveats: 07_verification_and_audit.md.

Top things to preserve

- Closed root model.
- No `framework/` root.
- Service-first provider model.
- Workbench is not authority.
- Third-party libraries are providers.
- C17/C++17 with C-compatible ABI.
- Full CTest remains debt.

Safest next actions

1. Verify latest live repo status.
2. Repair fast-strict evidence/marker debt if present.
3. Run projection conformance and presentation contract work.
4. Do provider wedge work only behind contracts and validators.

29.5 Dominium Canonical Structure And Framework Reader Brief

Source: docs/archive/conversations/canonical_
structure_and_framework/dominium_canonical_

structure_and_framework__06_reader_brief.md

Report type: reader_brief

Size: 2184 bytes

29.5.1 Reader Brief - Dominion Canonical Structure and Domino Framework Architecture

This chat was about turning Dominion from a messy, repeatedly refactored game repository into a stable, reusable Domino/Dominion architecture. The central result is that the repo should have a closed root model, and the architecture should be contract-centered rather than folder-, UI-, app-, or vendor-centered.

Top 20 things to know

1. Domino is the reusable deterministic/runtime substrate.
2. Dominion is the game/product family built on Domino.
3. Workbench is a production/evidence/editor surface, not authority.
4. AIDE is the repo/control-plane harness.
5. The canonical root model is mostly settled.

6. Do not add top-level `framework/`, `modules/`, `profiles/`, `labs`, or `src`.
7. Domino Framework lives in `contracts`, `public` surfaces, `ABI` law, `service/provider` law, and `public` headers.
8. Public headers live in `engine/include/domino`, `runtime/include/domino`, and `game/include/dominium` where needed.
9. Public ABI is C-compatible even though implementation is C17/C++17.
10. Raylib/SDL/Lua are providers, not architecture.
11. Provider implementations belong under `runtime/<service>/providers/<provider>`.
12. Provider choices belong in `profiles`, not app paths.
13. Third-party types must not leak into stable `contracts/saves/replays/game` law.
14. Workbench modules are UI modules only; general module law is under `contracts/manifests/packs`.
15. CLI/TUI/rendered/native/headless should be projections of the same semantic spine.
16. The spine is `command -> result/refusal -> document/snapshot/patch -> diagnostics/evidence -> view/action -> projection`.

17. Multiple structure cleanup passes were reported and many active old paths were removed.
18. Structure is credible but still PASS_WITH_WARNINGS, not perfect.
19. Full CTest/T4 remains debt.
20. Next work should be proof hygiene, projection conformance, and provider wedges-not broad root cleanup.

Best next step

Verify latest live repo gate state. If fast-strict/RepoX still fails on stale evidence or launcher marker debt, fix that first. If normal gates are clean, continue with PROJECTION-CONFORMANCE-01.

29.6 Dominion Canonical Structure And Framework In Chat Reader

Source: docs/archive/conversations/canonical_
structure_and_framework/dominium_canonical_
structure_and_framework__09_in_chat_reader.md

Report type: in_chat_reader

Size: 1120 bytes

29.6.1 In-Chat Reader - Dominion Canonical Structure and Domino Framework Architecture

Package overview

This preservation package captures the substance of the Dominion canonical repository structure and Domino framework architecture chat. It is intended for human reading, future chat continuation, and later aggregation into a project spec book.

File index

Read first: `01_human_readable_report.md`.

Use for continuing in another chat: `02_context_transfer_packet.md`.

Use for aggregation: `03_spec_sheet.yaml` and `05_aggregator_packet.md`.

Use for precise tables: `04_registers.md`.

Use for caveats: `07_verification_and_audit.md`.

Top things to preserve

- Closed root model.
- No `framework/` root.
- Service-first provider model.

- Workbench is not authority.
- Third-party libraries are providers.
- C17/C++17 with C-compatible ABI.
- Full CTest remains debt.

Safest next actions

1. Verify latest live repo status.
2. Repair fast-strict evidence/marker debt if present.
3. Run projection conformance and presentation contract work.
4. Do provider wedge work only behind contracts and validators.

29.7 Dominium Canonical Structure And Framework Accompanying Detailed Summary

Source: docs/archive/conversations/canonical_
structure_and_framework/dominium_canonical_
structure_and_framework__10_accompanying_
detailed_summary.md

Report type: accompanying_detailed_summary

Size: 19554 bytes

29.7.1 Dominion / Domino Canonical Structure - Accompanying Detailed Human Summary

Generated: 2026-05-27 21:03:35 AEST

Scope: This conversation's accessible content, the prior generated handoff package, visible user status reports, and uploaded/mounted summaries.

Purpose: A human-readable companion report that explains what happened in the conversation, what was decided, what was done, what was deferred, and what the next continuation chat should preserve.

1. Executive Status

The conversation started as a directory-structure emergency and ended as a broader architecture-and-proof program for Dominion and the reusable Domino substrate.

The main outcome is:

Domino Framework = contracts + public headers + service/provider law + conformance tests.

Dominion = the game/product family built on Domino.

Workbench = production/editing/validation/evidence environment.

AIDE = repo/control-plane harness.

The structure is no longer primarily a root-chaos problem. The canonical top-level root model has been established and repeatedly defended:

apps/
engine/
game/
runtime/
contracts/
content/
docs/
tests/
tools/
scripts/
cmake/
external/
release/
archive/

Allowed project/tooling roots:

.aide/
.aide.local.example/
.github/
.vscode/

Generated/local roots must not become active tracked source:

.aide.local/
.dominium.local/
build/
out/
dist/
artifacts/
reports/
tmp/
__pycache__/

The most important current status from the conversation:

Canonical structure: `credible, PASS_WITH_WARNINGS.`

Major old active path debt: `mostly cleared.`

Fast strict / CMake / smoke: `reported passing in latest maintenance lanes.`

Full CTest / T4 release gate: `still not green.`

Broad feature work: `still blocked.`

Narrow governed work: `allowed only under the proof/queue model.`

The single most important rule to preserve:

`Path is not identity.`

`Implementation is not contract.`

`UI is not authority.`

`Generated output is not source truth.`

2. Conversation Narrative

2.1 Initial problem: repo structure blocked real work The user began from the position that months of planning and refactoring had not produced a clean enough repo to continue development. The user was especially frustrated that prior attempts kept adding new root directories and reintroducing `src/` and `source/` folders, which had already been identified as a core failure mode.

The early goal was to determine the best future-proof directory layout for Dominion. The first answers proposed broad structures, routing maps, and

distribution/install/media layouts. The user rejected proposals that reintroduced root sprawl, long ambiguous paths, source wrappers, vague folders, or status words in active paths.

That rejection hardened the core root doctrine:

No root src/.

No root source/.

No nested first-party src/.

No generic common/shared/misc/lib/core/data junk drawers.

No new top-level roots without contract approval.

2.2 Source layout versus distribution/install

layout The conversation clarified that source repo layout is separate from:

release dist output

.dcompkg files

compressed archives

portable installs

installed desktop/server layouts

read-only media layouts

cache/staging/rollback layouts

save/instance/replay/diagnostic bundles

symbols/provenance layouts

The source repo should not contain install/runtime roots such as:

store/

instances/

saves/

exports/

cache/
ops/
media/

Those are projections of virtual roots, not source ownership roots.

2.3 Canonical root model The conversation converged on the durable root model:

apps thin product shells
engine deterministic substrate
game Dominion game meaning/law/domain
runtime services, providers, projections, platform/render/
storage/input/audio/package infrastructure
contracts machine-readable law
content authored payloads: packs, profiles, domains, themes,
assets, templates
docs explanation, not law
tests proof and fixtures
tools validators, codegen, migration, release, audit
scripts thin automation wrappers
cmake build-system support
external third-party source/provenance
release release definitions, profiles, packaging, signing,
updates
archive retained historical/generated/superseded material

This root model became the basis for all later architecture.

2.4 Massive cleanup prompts and status reports

Many Codex/AIDE prompts were generated to fix actual

directories, not only document the ideal state. The prompts included cleanup of runtime names, game rules, engine include boundaries, schema taxonomy, content packs, Workbench naming, AIDE scan boundaries, tools taxonomy, docs taxonomy, RepoX/TestX stale paths, full remediation, and final canonical structure passes.

The user reported many task results and commits. Some important reported commits/statuses include:

```
6e0dd93f263815667135bbf94b445c44cff6f733
refactor(repo):  finalize canonical structure
52ac5707e
repo:  enforce service-first provider structure
1406490bba4a8db617911f54cc85a8e939d29baa
refactor(repo):  finish canonical structure cleanup
3243fab7d7e4b9c32dc296a2583d4c5fa5ad8301
tests:  route full-gate legacy path expectations
ce9ca005f5922fd562f7f4c1aeb3240edd2d4823
repo:  define Domino framework boundary
```

The reported outcomes moved the repo from obvious active root chaos to `PASS_WITH_WARNINGS` structure credibility.

2.5 Shift from folders to public-surface governance

The user asked how to make code portable, modular, extensible, and reusable across not just Dominion but other games and engine projects. The conversation moved from folders to public surface governance:

```
public surface registry
```

C-compatible ABI rules
dependency-direction validation
compatibility corpus
replacement protocol
capability/refusal law
provider model
schema/protocol evolution law
assurance profiles
fast strict and full release gates

The key doctrine became:

Freeze public contracts.
Replace private implementations.
Preserve artifacts.
Verify behavior.

2.6 Domino Framework versus Dominion Game

The conversation then clarified that Domino is the reusable substrate/framework and Dominion is one game/product family. The user considered a new framework/ root, but the final decision rejected it.

Accepted doctrine:

Domino Framework is not a top-level framework/ source root.
Domino Framework = contracts + public headers + service/provider law + conformance tests.

The public API homes are:

engine/include/domino/ deterministic substrate public headers
runtime/include/domino/ runtime/service/provider public headers
game/include/dominium/ game/product-specific public headers where needed

A future SDK can be generated from registered public surfaces, likely under `release/sdk/` or equivalent, but not as a new active root unless explicitly contracted.

2.7 Language baseline The conversation moved from old C89/C++98 ideas to:

Mainline language baseline: C17 + C++17

Public ABI boundary: C-compatible, POD-only, versioned, no C++ ABI leakage

Architecture: 64-bit first, little-endian first, x86_64 + arm64 first

C17 should own boring stable law:

ABI-facing structs

packets

save/replay records

fixed-point math

stable IDs/handles

renderer command IR

low-level facades

C++17 should own machinery:

game orchestration

runtime services

providers

renderer/platform backends

apps

Workbench

tools

resource ownership

composition/profile resolution

The important carry-forward is not to expose C++ classes, STL containers, exceptions, templates, or compiler object layout across stable ABI/data boundaries.

2.8 Workbench, modules, packs, and composition

The user brought in Workbench brainstorming around Project Editor, Interface Studio, Module/Pack Foundry, App Composer, Release Forge, and related modules. The conversation clarified the vocabulary:

```
component = source/build ownership unit
service = callable runtime capability
provider = replaceable implementation
pack = distributable authored payload
module = declared functional extension unit
workspace = large Workbench composition
app = shipped product composition
artifact = persisted/versioned object
```

The key rule:

```
A folder can contain a component implementation.
A manifest defines a module.
A pack can provide modules.
An app composes modules, providers, services, and packs.
A workspace presents workflows over modules and services.
```

Reusable modules do not live under `apps/workbench`. Workbench modules are Workbench UI/presentation modules only. General module law lives under `contracts/module/`, and module payloads may be

delivered through `content/packs/.../modules/`.

2.9 Presentation and projection model The chat refined the UI architecture so CLI, text/TUI, rendered GUI, native GUI, headless reports, Workbench, CI, server/admin, and AIDE project the same semantic truth.

Final spine:

```
intent
-> command
-> capability/refusal check
-> service
-> result | document | snapshot
-> diagnostics/evidence
-> view
-> action model
-> projection
-> shell
```

Projection modes belong under runtime implementation, not semantic contract roots:

```
runtime/projection/cli/
runtime/projection/text/
runtime/projection/rendered/
runtime/projection/native/
runtime/projection/headless/
```

Contracts should define semantic surfaces:

```
contracts/command/
contracts/action/
```

contracts/result/
contracts/refusal/
contracts/diagnostic/
contracts/document/
contracts/patch/
contracts/view/
contracts/presentation/
contracts/projection/

Do not create separate contract worlds for `contracts/tui`, `contracts/rendered`, or `contracts/native`.

2.10 Raylib, SDL2, Lua, ImGui and provider architecture The user asked how raylib, SDL2, Lua, and other libraries should fit. The final doctrine:

Dominium owns service contracts.
Providers satisfy service contracts.
Profiles select providers.
Third-party libraries are fenced.
Apps stay generic.

Raylib, rlgl, rlsw, raygui, raudio, SDL2, Lua, and ImGui are providers, not architecture. Correct structure:

external/upstream/<third_party>/
runtime/<service>/providers/<provider>/
contracts/provider/
contracts/capability/
contracts/schema/runtime/<service>/
release/profiles/<profile>.toml
content/profiles/<runtime_profile>.toml
apps/<product>/

Use names like:

sdl2 now, sdl as family

raylib for broad provider

rlgl for low-level raylib GL abstraction

rlsw for raylib software renderer provider

win32 for Windows API provider, windows as OS family

direct3d11 for renderer provider, direct3d as family

opengl33 for direct OpenGL 3.3 provider, opengl as family

lua54 or lua55 for concrete script provider, lua as family

Third-party headers and types are forbidden in:

engine/

game/

contracts/

content/

saves/replays/packs/public ABI/stable data law

2.11 Framework boundary The user then considered whether the framework approach implied `framework/`, `engine/domino_reference/`, and `game/dominium/` roots. The final answer was no.

Corrected model:

Domino Framework = public-surface package generated from contracts and public headers.

Domino Reference Implementation = current engine/ + runtime/.

Dominium Game = current game/ + content/ + apps/.

Providers = runtime/<service>/providers/<provider>.

Profiles = release/content profile descriptors.

The user later reported a commit defining the Domino framework boundary and adding a validator that guards

against top-level `framework/` and `sdk/`.

2.12 Current proof posture The latest visible statuses indicate:

Canonical structure: `PASS_WITH_WARNINGS`
Provider structure: `PASS_WITH_WARNINGS`
Domino framework boundary: `PASS`
Full-gate legacy path subset: `PASS`
Fast strict / AIDE / CMake / smoke: mostly `PASS` by report
RepoX/fast-strict stale evidence/launcher marker debt: still blocking in a later status
Full CTest/T4: still not green
Feature readiness: `LIMITED`
Broad feature/release readiness: `NO`

A future assistant must verify latest live repo state before acting.

3. What Was Done

The conversation produced both reasoning and executable task prompts. The user reported actual maintenance runs and commits. The main completed categories were:

1. **Canonical structure cleanup** - major old active paths routed away; top-level roots normalized.
2. **Runtime/game/engine include cleanup** - old runtime names, `game/rules`, and engine include leakage were reportedly removed in active paths.

3. **Schema residual routing** - 130+ residual schema paths reportedly moved into canonical categories in one major cleanup pass.
4. **Test taxonomy and full-gate retired-root routing** - stale tests expecting retired roots were updated, and a `check_full_gate_legacy_paths.py` validator was added.
5. **Provider structure** - service-first provider structure, release provider profiles, provider policy/docs, and validators were added.
6. **Domino framework boundary** - framework defined as `contracts/public headers/provider-service law`, not a root; guards added against `framework/` and `sdk/`.
7. **Structure report integrity** - validators and reports were discussed/added to prevent mixed stale tree exports.
8. **Handoff package** - prior preservation files were generated, and this companion report now extends them.

4. What Was Put Off for Later

The following were not finished or intentionally deferred:

Full CTest/T4 release proof.

Stale AuditX/identity evidence refresh.
Launcher pack-verification marker debt.
Storage/package provider split.
Actual raylib/SDL2/Lua provider implementations.
Provider conformance suites beyond skeleton/planned warnings.
Projection conformance.
Presentation contract completion if not already done.
Public header ABI promotion warnings.
Pack internal layout content/ vs data/ final law.
AIDE state-like directory classification.
Runtime/engine residual session/serialization/foundation taxonomy.
Broad Workbench UI.
Runtime module loader.
Provider runtime implementation beyond planned structure.
Package runtime.
Gameplay expansion.
Renderer implementation beyond provider planning.
Native GUI.
Release publication.

The deliberate policy was to stop broad structure churn once the active tree became credible and then move to targeted maintenance and narrow governed product slices.

5. Final Canonical Root Doctrine

Active roots:

.aide/
.aide.local.example/

.github/
.vscode/
apps/
engine/
game/
runtime/
contracts/
content/
docs/
tests/
tools/
scripts/
cmake/
external/
release/
archive/

Forbidden active roots:

src/
source/
sources/
framework/
sdk/
modules/
plugins/
services/
profiles/
labs/
core/
control/
data/
lib/

libs/
packs/
profiles/
bundles/
meta/
validation/
governance/
performance/
models/
templates/
net/
ide/

Allowed exceptions:

archive/legacy/
archive/generated/
archive/historical/
archive/quarantine/
docs/compatibility/
contracts/compatibility/
external/upstream or external/vendor third-party source
preserving upstream layout
tests/fixtures intentionally modeling old layouts
finite documented compatibility shims

6. Final Architectural Laws

Path is not identity.

Implementation is not contract.

UI is not authority.

Generated output is not source truth.

Module identity is not path.

Pack identity is not path.
Provider identity is not implementation directory.
App identity is not executable filename.
Workspace identity is not layout file.
Commands are behavioral boundaries.
Documents/snapshots/results/patches are data boundaries.
Diagnostics/evidence are audit boundaries.
Contracts and tests decide stability.

7. Final Provider Directory Doctrine

Best pattern:

```
external/upstream/<third_party>/  
runtime/<service>/providers/<provider>/  
contracts/provider/  
contracts/capability/  
contracts/schema/runtime/<service>/  
release/profiles/<profile>.toml  
content/profiles/<runtime_profile>.toml  
apps/<product>/
```

Example runtime layout:

```
runtime/platform/providers/sdl2/  
runtime/platform/providers/raylib/  
runtime/input/providers/sdl2/  
runtime/input/providers/raylib/  
runtime/render/providers/null/  
runtime/render/providers/software/  
runtime/render/providers/rlsw/  
runtime/render/providers/raylib/  
runtime/render/providers/rlgl/
```

```
runtime/render/providers/opengl33/  
runtime/render/providers/direct3d11/  
runtime/audio/providers/raudio/  
runtime/script/providers/lua54/  
runtime/ui/providers/raygui/
```

Do not use app paths like:

```
apps/client/rendered/raylib/  
apps/workbench/raylib/
```

Provider choices live in profiles.

8. Current Status in Plain Language

The repo is much healthier. The broad directory crisis appears to be over. The structure is not perfect, and the proof pipeline is not full green. But the project now has a credible canonical root model, structure validators, provider structure doctrine, framework boundary, and a path toward actual Workbench/client/engine/game implementation.

The correct next mindset is no longer:

We need one more giant directory refactor.

It is:

We need targeted proof hygiene and governed implementation slices.

9. Recommended Immediate Next Tasks

Depending on current live state:

If fast-strict/RepoX is blocked FAST-STRICT-EVIDENCE-MARKER-REPAIR-01

Scope:

Refresh/retire stale AuditX/identity evidence.

Fix launcher pack-verification marker debt.

Do not modify directory structure broadly.

Rerun AIDE, RepoX, fast strict, CMake, smoke.

If fast-strict is green PROJECTION-CONFORMANCE-01

Scope:

Prove CLI/text/rendered/native/headless projections over command/result/view/evidence contracts.

Maintenance after that FULL-CTEST-AUDIT-NONPATH-01

PACK-INTERNAL-LAYOUT-CANON-01

STORAGE-PACKAGE-PROVIDER-SPLIT-01

PUBLIC-HEADER-ABI-PROMOTION-01

AIDE-STATE-CLASSIFICATION-01

10. Things Future Assistants Must Not Do

Do not add framework/ as a source root.

Do not add top-level modules/, plugins/, services/, profiles/, labs/, or sdk/.

Do not reintroduce src/ or source/.

Do not treat raylib, SDL2, Lua, ImGui, raygui, rlgl, rlsw, or raudio as architecture law.

Do not put reusable modules under apps/workbench.

Do not make Workbench authority.

Do not encode provider choices into app directory names.

Do not declare PASS_WITH_WARNINGS as full green.

Do not start broad feature work while full proof and current queue say it is blocked.

11. Verification Checklist for a New Chat

Before doing any work, verify:

Current HEAD and branch.

Worktree clean/dirty state.

AIDE doctor/validate.

RepoX strict / fast strict.

Canonical structure validator.

Dependency-direction validator.

Provider structure validator.

Third-party include validator.

CMake verify/build.

Smoke CTest.

Full CTest status or last full-gate audit.

Current .aide/queue/current.toml.

Whether stale AuditX/identity evidence remains.

Whether launcher pack-verification marker debt remains.

Do not rely on old uploaded dir_tree.json files without checking report integrity.

12. Package Contents

This updated bundle includes the prior preservation package files plus this companion report and a bundle inventory with SHA-256 hashes.

29.8 Dominion Canonical Structure And Framework Accompanying Detailed Summary

Source: docs/archive/conversations/canonical_
structure_and_framework/dominium_canonical_
structure_and_framework__10_accompanying_
detailed_summary.txt

Report type: accompanying_detailed_summary

Size: 19554 bytes

29.8.1 Dominion / Domino Canonical Structure - Accompanying Detailed Human Summary

Generated: 2026-05-27 21:03:35 AEST

Scope: This conversation's accessible content, the prior generated handoff package, visible user status reports, and uploaded/mounted summaries.

Purpose: A human-readable companion report that explains what happened in the conversation, what was

decided, what was done, what was deferred, and what the next continuation chat should preserve.

1. Executive Status

The conversation started as a directory-structure emergency and ended as a broader architecture-and-proof program for Dominium and the reusable Domino substrate.

The main outcome is:

Domino Framework = contracts + public headers + service/provider law + conformance tests.

Dominium = the game/product family built on Domino.

Workbench = production/editing/validation/evidence environment.

AIDE = repo/control-plane harness.

The structure is no longer primarily a root-chaos problem. The canonical top-level root model has been established and repeatedly defended:

```
apps/  
engine/  
game/  
runtime/  
contracts/  
content/  
docs/  
tests/  
tools/  
scripts/  
cmake/
```

external/
release/
archive/

Allowed project/tooling roots:

.aide/
.aide.local.example/
.github/
.vscode/

Generated/local roots must not become active tracked
source:

.aide.local/
.dominium.local/
build/
out/
dist/
artifacts/
reports/
tmp/
__pycache__/

The most important current status from the
conversation:

Canonical structure: credible, PASS_WITH_WARNINGS.

Major old active path debt: mostly cleared.

Fast strict / CMake / smoke: reported passing in latest
maintenance lanes.

Full CTest / T4 release gate: still not green.

Broad feature work: still blocked.

Narrow governed work: allowed only under the proof/queue model.

The single most important rule to preserve:

Path is not identity.
Implementation is not contract.
UI is not authority.
Generated output is not source truth.

2. Conversation Narrative

2.1 Initial problem: repo structure blocked real work The user began from the position that months of planning and refactoring had not produced a clean enough repo to continue development. The user was especially frustrated that prior attempts kept adding new root directories and reintroducing `src/` and `source/` folders, which had already been identified as a core failure mode.

The early goal was to determine the best future-proof directory layout for Dominion. The first answers proposed broad structures, routing maps, and distribution/install/media layouts. The user rejected proposals that reintroduced root sprawl, long ambiguous paths, source wrappers, vague folders, or status words in active paths.

That rejection hardened the core root doctrine:

No root `src/`.
No root `source/`.
No nested first-party `src/`.
No generic `common/shared/misc/lib/core/data` junk drawers.
No new top-level roots without contract approval.

2.2 Source layout versus distribution/install

layout The conversation clarified that source repo layout is separate from:

- release dist output
- .dmpkg files
- compressed archives
- portable installs
- installed desktop/server layouts
- read-only media layouts
- cache/staging/rollback layouts
- save/instance/replay/diagnostic bundles
- symbols/provenance layouts

The source repo should not contain install/runtime roots such as:

- store/
- instances/
- saves/
- exports/
- cache/
- ops/
- media/

Those are projections of virtual roots, not source ownership roots.

2.3 Canonical root model

The conversation converged on the durable root model:

- apps thin product shells
- engine deterministic substrate
- game Dominion game meaning/law/domain

runtime services, providers, projections, platform/render/
storage/input/audio/package infrastructure
contracts machine-readable law
content authored payloads: packs, profiles, domains, themes,
assets, templates
docs explanation, not law
tests proof and fixtures
tools validators, codegen, migration, release, audit
scripts thin automation wrappers
cmake build-system support
external third-party source/provenance
release release definitions, profiles, packaging, signing,
updates
archive retained historical/generated/superseded material

This root model became the basis for all later
architecture.

2.4 Massive cleanup prompts and status reports

Many Codex/AIDE prompts were generated to fix actual
directories, not only document the ideal state. The
prompts included cleanup of runtime names, game rules,
engine include boundaries, schema taxonomy, content
packs, Workbench naming, AIDE scan boundaries, tools
taxonomy, docs taxonomy, RepoX/TestX stale paths,
full remediation, and final canonical structure passes.

The user reported many task results and commits. Some
important reported commits/statuses include:

```
6e0dd93f263815667135bbf94b445c44cff6f733  
refactor(repo): finalize canonical structure  
52ac5707e
```

```
repo:  enforce service-first provider structure
1406490bba4a8db617911f54cc85a8e939d29baa
refactor(repo):  finish canonical structure cleanup
3243fab7d7e4b9c32dc296a2583d4c5fa5ad8301
tests:  route full-gate legacy path expectations
ce9ca005f5922fd562f7f4c1aeb3240edd2d4823
repo:  define Domino framework boundary
```

The reported outcomes moved the repo from obvious active root chaos to PASS_WITH_WARNINGS structure credibility.

2.5 Shift from folders to public-surface governance

The user asked how to make code portable, modular, extensible, and reusable across not just Dominion but other games and engine projects. The conversation moved from folders to public surface governance:

```
public surface registry
C-compatible ABI rules
dependency-direction validation
compatibility corpus
replacement protocol
capability/refusal law
provider model
schema/protocol evolution law
assurance profiles
fast strict and full release gates
```

The key doctrine became:

Freeze public contracts.

Replace private implementations.

Preserve artifacts.

Verify behavior.

2.6 Domino Framework versus Dominion Game

The conversation then clarified that Domino is the reusable substrate/framework and Dominion is one game/product family. The user considered a new framework/ root, but the final decision rejected it.

Accepted doctrine:

Domino Framework is not a top-level framework/ source root.

Domino Framework = contracts + public headers + service/provider law + conformance tests.

The public API homes are:

engine/include/domino/ deterministic substrate public headers
runtime/include/domino/ runtime/service/provider public headers
game/include/dominium/ game/product-specific public headers where needed

A future SDK can be generated from registered public surfaces, likely under `release/sdk/` or equivalent, but not as a new active root unless explicitly contracted.

2.7 Language baseline

The conversation moved from old C89/C++98 ideas to:

Mainline language baseline: C17 + C++17

Public ABI boundary: C-compatible, POD-only, versioned, no C++ ABI leakage

Architecture: 64-bit first, little-endian first, x86_64 + arm64 first

C17 should own boring stable law:

- ABI-facing structs
- packets
- save/replay records
- fixed-point math
- stable IDs/handles
- renderer command IR
- low-level facades

C++17 should own machinery:

- game orchestration
- runtime services
- providers
- renderer/platform backends
- apps
- Workbench
- tools
- resource ownership
- composition/profile resolution

The important carry-forward is not to expose C++ classes, STL containers, exceptions, templates, or compiler object layout across stable ABI/data boundaries.

2.8 Workbench, modules, packs, and composition

The user brought in Workbench brainstorming around Project Editor, Interface Studio, Module/Pack Foundry, App Composer, Release Forge, and related modules. The conversation clarified the vocabulary:

component = source/build ownership unit

service = callable runtime capability
provider = replaceable implementation
pack = distributable authored payload
module = declared functional extension unit
workspace = large Workbench composition
app = shipped product composition
artifact = persisted/versioned object

The key rule:

A folder can contain a component implementation.
A manifest defines a module.
A pack can provide modules.
An app composes modules, providers, services, and packs.
A workspace presents workflows over modules and services.

Reusable modules do not live under apps/workbench.
Workbench modules are Workbench UI/presentation modules only. General module law lives under contracts/module/, and module payloads may be delivered through content/packs/.../modules/.

2.9 Presentation and projection model The chat refined the UI architecture so CLI, text/TUI, rendered GUI, native GUI, headless reports, Workbench, CI, server/admin, and AIDE project the same semantic truth.

Final spine:

intent
-> command
-> capability/refusal check

- > service
- > result | document | snapshot
- > diagnostics/evidence
- > view
- > action model
- > projection
- > shell

Projection modes belong under runtime implementation, not semantic contract roots:

- runtime/projection/cli/
- runtime/projection/text/
- runtime/projection/rendered/
- runtime/projection/native/
- runtime/projection/headless/

Contracts should define semantic surfaces:

- contracts/command/
- contracts/action/
- contracts/result/
- contracts/refusal/
- contracts/diagnostic/
- contracts/document/
- contracts/patch/
- contracts/view/
- contracts/presentation/
- contracts/projection/

Do not create separate contract worlds for contracts/tui, contracts/rendered, or contracts/native.

2.10 Raylib, SDL2, Lua, ImGui and provider architecture

The user asked how raylib, SDL2, Lua, and other libraries should fit. The final doctrine:

Dominium owns service contracts.
Providers satisfy service contracts.
Profiles select providers.
Third-party libraries are fenced.
Apps stay generic.

Raylib, rlgl, rlsw, raygui, raudio, SDL2, Lua, and ImGui are providers, not architecture. Correct structure:

```
external/upstream/<third_party>/  
runtime/<service>/providers/<provider>/  
contracts/provider/  
contracts/capability/  
contracts/schema/runtime/<service>/  
release/profiles/<profile>.toml  
content/profiles/<runtime_profile>.toml  
apps/<product>/
```

Use names like:

```
sdl2 now, sdl as family  
raylib for broad provider  
rlgl for low-level raylib GL abstraction  
rlsw for raylib software renderer provider  
win32 for Windows API provider, windows as OS family  
direct3d11 for renderer provider, direct3d as family  
opengl33 for direct OpenGL 3.3 provider, opengl as family  
lua54 or lua55 for concrete script provider, lua as family
```

Third-party headers and types are forbidden in:

```
engine/
```

```
game/  
contracts/  
content/  
saves/replays/packs/public ABI/stable data law
```

2.11 Framework boundary The user then considered whether the framework approach implied `framework/`, `engine/domino_reference/`, and `game/dominium/` roots. The final answer was no.

Corrected model:

```
Domino Framework = public-surface package generated from  
contracts and public headers.  
Domino Reference Implementation = current engine/ + runtime/.  
Dominator Game = current game/ + content/ + apps/.  
Providers = runtime/<service>/providers/<provider>.  
Profiles = release/content profile descriptors.
```

The user later reported a commit defining the Domino framework boundary and adding a validator that guards against top-level `framework/` and `sdk/`.

2.12 Current proof posture The latest visible statuses indicate:

```
Canonical structure: PASS_WITH_WARNINGS  
Provider structure: PASS_WITH_WARNINGS  
Domino framework boundary: PASS  
Full-gate legacy path subset: PASS  
Fast strict / AIDE / CMake / smoke: mostly PASS by report  
RepoX/fast-strict stale evidence/launcher marker debt: still  
blocking in a later status
```

Full CTest/T4: still not green

Feature readiness: LIMITED

Broad feature/release readiness: NO

A future assistant must verify latest live repo state before acting.

3. What Was Done

The conversation produced both reasoning and executable task prompts. The user reported actual maintenance runs and commits. The main completed categories were:

1. **Canonical structure cleanup** - major old active paths routed away; top-level roots normalized.
2. **Runtime/game/engine include cleanup** - old runtime names, game/rules, and engine include leakage were reportedly removed in active paths.
3. **Schema residual routing** - 130+ residual schema paths reportedly moved into canonical categories in one major cleanup pass.
4. **Test taxonomy and full-gate retired-root routing** - stale tests expecting retired roots were updated, and a `check_full_gate_legacy_paths.py` validator was added.

5. **Provider structure** - service-first provider structure, release provider profiles, provider policy/docs, and validators were added.
6. **Domino framework boundary** - framework defined as contracts/public headers/provider-service law, not a root; guards added against framework/ and sdk/.
7. **Structure report integrity** - validators and reports were discussed/added to prevent mixed stale tree exports.
8. **Handoff package** - prior preservation files were generated, and this companion report now extends them.

4. What Was Put Off for Later

The following were not finished or intentionally deferred:

Full CTest/T4 release proof.

Stale AuditX/identity evidence refresh.

Launcher pack-verification marker debt.

Storage/package provider split.

Actual raylib/SDL2/Lua provider implementations.

Provider conformance suites beyond skeleton/planned warnings.

Projection conformance.

Presentation contract completion if not already done.

Public header ABI promotion warnings.

Pack internal layout content/ vs data/ final law.

AIDE state-like directory classification.

Runtime/engine residual session/serialization/foundation taxonomy.

Broad Workbench UI.

Runtime module loader.

Provider runtime implementation beyond planned structure.

Package runtime.

Gameplay expansion.

Renderer implementation beyond provider planning.

Native GUI.

Release publication.

The deliberate policy was to stop broad structure churn once the active tree became credible and then move to targeted maintenance and narrow governed product slices.

5. Final Canonical Root Doctrine

Active roots:

`.aide/`

`.aide.local.example/`

`.github/`

`.vscode/`

`apps/`

`engine/`

`game/`

`runtime/`

`contracts/`

`content/`

`docs/`

- tests/
- tools/
- scripts/
- cmake/
- external/
- release/
- archive/

Forbidden active roots:

- src/
- source/
- sources/
- framework/
- sdk/
- modules/
- plugins/
- services/
- profiles/
- labs/
- core/
- control/
- data/
- lib/
- libs/
- packs/
- profiles/
- bundles/
- meta/
- validation/
- governance/
- performance/
- models/

templates/

net/

ide/

Allowed exceptions:

archive/legacy/

archive/generated/

archive/historical/

archive/quarantine/

docs/compatibility/

contracts/compatibility/

external/upstream or external/vendor third-party source
preserving upstream layout

tests/fixtures intentionally modeling old layouts

finite documented compatibility shims

6. Final Architectural Laws

Path is not identity.

Implementation is not contract.

UI is not authority.

Generated output is not source truth.

Module identity is not path.

Pack identity is not path.

Provider identity is not implementation directory.

App identity is not executable filename.

Workspace identity is not layout file.

Commands are behavioral boundaries.

Documents/snapshots/results/patches are data boundaries.

Diagnostics/evidence are audit boundaries.

Contracts and tests decide stability.

7. Final Provider Directory Doctrine

Best pattern:

```
external/upstream/<third_party>/  
runtime/<service>/providers/<provider>/  
contracts/provider/  
contracts/capability/  
contracts/schema/runtime/<service>/  
release/profiles/<profile>.toml  
content/profiles/<runtime_profile>.toml  
apps/<product>/
```

Example runtime layout:

```
runtime/platform/providers/sdl2/  
runtime/platform/providers/raylib/  
runtime/input/providers/sdl2/  
runtime/input/providers/raylib/  
runtime/render/providers/null/  
runtime/render/providers/software/  
runtime/render/providers/rlsw/  
runtime/render/providers/raylib/  
runtime/render/providers/rlgl/  
runtime/render/providers/opengl33/  
runtime/render/providers/direct3d11/  
runtime/audio/providers/raudio/  
runtime/script/providers/lua54/  
runtime/ui/providers/raygui/
```

Do not use app paths like:

```
apps/client/rendered/raylib/  
apps/workbench/raylib/
```

Provider choices live in profiles.

8. Current Status in Plain Language

The repo is much healthier. The broad directory crisis appears to be over. The structure is not perfect, and the proof pipeline is not full green. But the project now has a credible canonical root model, structure validators, provider structure doctrine, framework boundary, and a path toward actual Workbench/client/engine/game implementation.

The correct next mindset is no longer:

We need one more giant directory refactor.

It is:

We need targeted proof hygiene and governed implementation slices.

9. Recommended Immediate Next Tasks

Depending on current live state:

If fast-strict/RepoX is blocked FAST-STRICT-EVIDENCE-MARKER-REPAIR-01

Scope:

Refresh/retire stale AuditX/identity evidence.

Fix launcher pack-verification marker debt.

Do not modify directory structure broadly.

Rerun AIDE, RepoX, fast strict, CMake, smoke.

If fast-strict is green PROJECTION-CONFORMANCE-01

Scope:

Prove CLI/text/rendered/native/headless projections over command/
result/view/evidence contracts.

Maintenance after that FULL-CTEST-AUDIT-NONPATH-01

PACK-INTERNAL-LAYOUT-CANON-01

STORAGE-PACKAGE-PROVIDER-SPLIT-01

PUBLIC-HEADER-ABI-PROMOTION-01

AIDE-STATE-CLASSIFICATION-01

10. Things Future Assistants Must Not Do

Do not add framework/ as a source root.

Do not add top-level modules/, plugins/, services/, profiles/,
labs/, or sdk/.

Do not reintroduce src/ or source/.

Do not treat raylib, SDL2, Lua, ImGui, raygui, rlgl, rlsw, or
raudio as architecture law.

Do not put reusable modules under apps/workbench.

Do not make Workbench authority.

Do not encode provider choices into app directory names.

Do not declare PASS_WITH_WARNINGS as full green.

Do not start broad feature work while full proof and current
queue say it is blocked.

11. Verification Checklist for a New Chat

Before doing any work, verify:

Current HEAD and branch.

Worktree clean/dirty state.
AIDE doctor/validate.
RepoX strict / fast strict.
Canonical structure validator.
Dependency-direction validator.
Provider structure validator.
Third-party include validator.
CMake verify/build.
Smoke CTest.
Full CTest status or last full-gate audit.
Current .aide/queue/current.toml.
Whether stale AuditX/identity evidence remains.
Whether launcher pack-verification marker debt remains.

Do not rely on old uploaded dir_tree.json files
without checking report integrity.

12. Package Contents

This updated bundle includes the prior preservation package files plus this companion report and a bundle inventory with SHA-256 hashes.

29.9 Dominium Canonical Structure And Framework Accompanying Detailed Summary Report

Source: docs/archive/conversations/canonical_
structure_and_framework/dominium_canonical_

structure_and_framework__10_accompanying_
detailed_summary_report.md

Report type: detailed_summary_report

Size: 25112 bytes

29.9.1 Accompanying Detailed Human-Readable Summary Report

Project: Dominion / Domino architecture, repository structure, framework boundary, provider model, Workbench, and proof gates

Generated: 2026-05-27 21:02 AEST

Purpose: This report accompanies the earlier preservation package. It gives a readable end-to-end summary of the entire visible conversation: what was discussed, what was decided, what was rejected, what was executed, what remains unresolved, and what a future chat should do next.

0. Scope, Method, and Reliability

This report summarizes the accessible conversation and the mounted artifacts available in `/mnt/data`. It incorporates the prior handoff/preservation files, user-reported commit statuses, repeated directory-structure reviews, and the architecture decisions made in the chat.

This is **not** a verbatim transcript. It is a synthesized, human-readable report intended for project continuity.

Reliability notes:

- The visible chat contained many user-provided summaries of Codex/AIDE runs. Those are treated as user-reported project state, not independently verified source control facts.
- Several uploaded files expired during the conversation. This report preserves the accessible substance, not every raw upload.
- Multiple structure bundles were found to be mixed or stale across commits. The conversation repeatedly identified report-bundle integrity as a required future proof check.
- Live repo state may have changed after the final status report. Verify the current branch, commit, validators, and CTest state before acting.

Recommended usage:

1. Read this report for human orientation.
2. Read `dominium_canonical_structure_and_framework__02_context_transfer_packet.md` before continuing in another chat.

3. Read `dominium_canonical_structure_and_framework__04_registers.md` for task/decision registers.
 4. Verify the live repo before executing any task.
-

1. Executive Summary

This conversation started as an urgent attempt to fix Dominium's directory structure after months of stalled development. It evolved into a complete architectural doctrine for turning Dominium from a messy game repository into a deterministic, contract-governed, provider-backed, pack-composed simulation platform.

The major outcome is that the project's core structure is now understood as:

`Domino = reusable deterministic/runtime substrate and framework contract surface`

`Dominium = one game/product family built on Domino`

`Workbench = production, validation, editing, inspection, packaging, and evidence environment`

`AIDE = repo/control-plane harness`

`Contracts = machine-readable law`

`Runtime = services, projections, providers, and host/platform/render/package machinery`

`Apps = thin product shells`

`Content = authored payloads: packs, profiles, assets, domain data, themes, templates`

`Tests/replay/evidence = proof system`

The central doctrine is:

Freeze public contracts.
Replace private implementations.
Preserve artifacts.
Verify behavior.

The directory-structure doctrine is:

Paths encode ownership, not identity.
Stable identities live in contracts, manifests, registries, IDs,
public headers, and artifacts.
Private implementations live in folders and files.
Generated output is not source truth.
UI is not authority.

The repo's top-level root model is settled and should not
be redesigned casually:

apps/
engine/
game/
runtime/
contracts/
content/
docs/
tests/
tools/
scripts/
cmake/
external/
release/
archive/

Allowed project/tooling roots:

.aide/
.aide.local.example/

.github/
.vscode/

Generated/local roots must not become active tracked source:

.aide.local/
.dominium.local/
build/
out/
dist/
artifacts/
reports/
tmp/
__pycache__/

The conversation's final posture is: broad structure cleanup is largely resolved, but the project is not full-release-ready. Fast strict, canonical structure, CMake verify/build, smoke CTest, and targeted subsets were repeatedly reported passing or passing with warnings; full CTest/T4 remains debt; broad feature work remains blocked; narrow governed product-spine work can continue after proof hygiene is maintained.

2. The Problem We Were Solving

The user's core complaint was that Dominium had spent months in refactor/planning cycles without enabling real feature work. The repository had repeatedly accumulated new roots and redundant folder shapes.

The user identified one recurring failure mode: assistants and tooling kept adding new root directories and `src//source/` wrappers, directly violating the project's own doctrine.

The specific problems included:

- Too many top-level roots.
- Product roots, runtime roots, domain roots, schemas, generated artifacts, packs, profiles, tools, and docs mixed at the same hierarchy level.
- Reintroduction of `src/`, `source/`, `core`, `lib`, `common`, `shared`, `misc`, `data`, and similar junk drawers.
- Duplicate or split concepts such as `appcore`, `appshell`, `shell`, `runtime/app`, `runtime/shell`, `tools/appshell`, and `libs/appcore`.
- Old schema buckets mirroring retired roots.
- Pack payloads mixed with package law.
- Workbench modules and general modules confused.
- Third-party libraries threatening to shape architecture.
- Generated reports and archived snapshots confusing active-source scans.

- Full-gate tests still expecting retired paths.

The goal became not merely to make folders tidy, but to create a system in which future development would not re-create the same structural drift.

3. Canonical Root Structure and Source Layout

The conversation converged on a small, closed top-level source-root model.

3.1 Final active roots apps/ product shells and product-specific composition

engine/ deterministic Domino substrate implementation

runtime/ reusable host/service/projection/provider machinery

game/ Dominion law, rule, world, and domain meaning

contracts/ machine-readable law, public surfaces, ABI/service/provider/package/schema law

content/ authored payloads: packs, profiles, assets, themes, templates, domain data

docs/ human explanation, not primary law

tests/ proof system

tools/ validators, codegen, migration, repo/audit/release tooling

scripts/ thin command wrappers

cmake/ build-system support

external/ third-party source/provenance

release/ release definitions, profiles, packaging, provenance

archive/ historical/generated/quarantine/superseded retained material

3.2 Forbidden active roots The conversation repeatedly rejected active roots such as:

```
src/  
source/  
sources/  
core/  
control/  
data/  
lib/  
libs/  
packs/  
profiles/  
bundles/  
compat/  
locks/  
repo/  
safety/  
security/  
specs/  
updates/  
meta/  
governance/  
performance/  
validation/  
modding/  
models/  
templates/  
net/  
ide/  
modules/  
plugins/
```

```
services/  
workspaces/  
sdk/  
framework/  
labs/
```

Some words may still appear inside valid contexts, such as `archive/legacy`, `contracts/compatibility`, `docs/compatibility`, or third-party upstream source layouts.

3.3 No `src/` or `source/` The user explicitly rejected `src/` and `source/` as a root or repeated module wrapper. The final rule is:

```
No root src/.  
No root source/.  
No nested first-party module src/.  
No source/ wrappers.
```

Implementation files live directly under ownership modules.

Correct:

```
engine/time/  
runtime/render/providers/software/  
game/domain/geology/  
tools/validators/repo/
```

Wrong:

```
engine/src/time/  
runtime/src/render/software/  
game/src/domain/geology/  
tools/source/validators/repo/
```

4. Source Layout versus Runtime/Install/Distribution Layout

A major conclusion was that source layout must not be confused with runtime/install/distribution layout.

The project needs many physical layouts:

- source repository layout;
- CI/build output layout;
- .dompkg internal logical export layout;
- downloadable compressed archive layout;
- portable uncompressed install layout;
- installed desktop/server layout;
- read-only media layout;
- save/instance/bundle/diagnostic/replay layouts;
- cache/staging/update/rollback layouts;
- symbol/provenance/release metadata layouts.

The source repo should not contain active install/runtime roots such as:

store/


```
instances/  
saves/  
exports/  
cache/  
ops/  
media/
```

These belong to generated runtime/install projections or release artifacts.

The clean model is:

```
logical roots -> physical projection -> package export map ->  
install/store/runtime binding -> deterministic verification
```

5. Naming Doctrine

The conversation spent substantial time refining naming rules.

5.1 Four-level naming model

Paths encode ownership.

Public symbols encode namespace.

Stable IDs encode semantic identity.

Private names encode local role.

Examples:

Path: runtime/render/providers/direct3d11/

Public symbol: domino_render_backend_create()

Stable ID: domino.provider.render.direct3d11.v1

Private file: backend.c

Private static function: parse_header()

5.2 When to use domino / dominium Use `domino_` and `dominium_` for public/global C symbols and stable IDs:

```
domino_package_manifest_parse()
domino_replay_log_append()
dominium_world_tick()
dominium_law_validate_action()
```

Do not put `domino` or `dominium` in every directory and file.

Good paths:

```
runtime/render/providers/raylib/
game/domain/economy/
contracts/schema/package/
```

Bad paths:

```
runtime/domino_runtime_render/domino_raylib_renderer/
game/dominium_game_domain/dominium_economy/
```

5.3 Singular versus plural Use singular for authority categories and subsystem planes; use plural for collections of peer artifacts.

Examples:

```
contracts/schema/ category root
contracts/registry/ category root
contracts/package/ package law
contracts/profile/ profile law
game/domain/ implementation plane
runtime/render/ subsystem
```

```
content/packs/ many packs
content/domains/ many domain payloads
docs/domains/ many domain docs
tests/fixtures/ many fixtures
```

Some current repo choices may use plural for historical reasons; future assistants should follow the live contract if one is already enforced.

5.4 Provider naming Use generic service roots and exact provider implementation folders:

```
runtime/render/providers/direct3d11/
runtime/platform/providers/sdl2/
runtime/script/providers/lua54/
```

Use family metadata in manifests:

```
family = "direct3d"
version = "11"

family = "sdl"
major = 2
```

Provider directory choices established:

```
sdl2 now, sdl as family
raylib for broad high-level provider
rlgl for lower-level raylib GL abstraction provider
rlsw for raylib software-render provider
win32 for native Windows API provider, windows as OS family
direct3d11 for Direct3D 11 provider, direct3d as family
opengl33 for first OpenGL 3.3 implementation, opengl as family
lua54 or lua55 for concrete Lua provider, lua as family
raygui for UI provider
```

raudio for audio provider when used directly

6. Domino Framework, Dominium Game, Workbench, and AIDE

6.1 Domino Framework Domino Framework is not a top-level framework/ root.

It is:

contracts + public headers + public surface registry + ABI law + service/provider law + conformance tests

The public API homes are:

engine/include/domino/ deterministic engine public surfaces

runtime/include/domino/ runtime/service/provider public surfaces

game/include/dominium/ game/product-specific public surfaces, if needed

6.2 Domino reference implementation The current engine/ and runtime/ are the reference implementation of the Domino substrate and services. Do not rename to engine/domino_reference/ unless there is a real second engine implementation.

6.3 Dominium Game game/, content/, and product apps are Dominium's game/product implementation.

6.4 Workbench Workbench is the production/ editing/validation/evidence environment over the same

contracts, commands, services, providers, packs, modules, and artifacts. Workbench is not authority.

6.5 AIDE AIDE is the repo/control-plane harness. It should be governed by `.aide/` policy/control-plane data, `.aide.local/` mutable local state, and `archive/generated/aide/` retained generated evidence. State-like AIDE dirs require classification.

7. API, ABI, and Language Baseline

The chat settled on:

Mainline language baseline: C17 + C++17

Mainline architecture: 64-bit, little-endian, x86_64 + arm64
first

Boundary rule: C-compatible ABI, POD-only, versioned, no C++ ABI leakage

C17 owns:

ABI-facing structs
packets
save/replay records
deterministic math
stable handles
renderer command IR
small low-level facades

C++17 owns:

runtime services
providers
game orchestration

Workbench

tools

resource ownership

composition/profile resolution

Public ABI rule:

opaque handles

versioned structs

struct_size fields

explicit ownership/lifetime

explicit allocator/free path

result/refusal codes

no exceptions across boundary

no STL containers

no C++ classes

no native object layout serialization

no third-party types

The chat rejected pure C99, pure C17, pure C++11, pure C++17 at stable boundaries, and C++20/23/26 as mainline for now.

8. Public Surface Governance

The project needs machine-readable public surface law.

Surface kinds discussed:

c_header

abi_table

schema

registry

protocol

command
view
event
refusal
diagnostic_code
package_format
profile_format
save_format
replay_format
provider_abi
capability_id
release_artifact

Stability classes:

frozen_abi
stable_api
stable_data_contract
stable_command_contract
stable_protocol
provisional
internal
experimental
generated
fixture
historical
retired

Rule:

Most things start internal or provisional.

Only proven surfaces become stable.

Every stable surface should have:

owner
stable ID
path
version
stability class
compatibility policy
replacement policy
proof
deprecation policy

9. Dependency Direction and Boundary Law

Canonical dependency direction:

apps -> runtime, game, contracts
game -> engine, contracts
runtime -> engine, contracts
engine -> contracts only where necessary
tools -> may inspect all, but runtime must not depend on tools
content -> no code dependency
contracts -> no implementation dependency
archive -> no active dependency

Forbidden:

engine -> runtime
engine -> game
engine -> apps
runtime -> apps
runtime owns game truth
contracts import implementation
content imports code
active source imports archive

Third-party libraries are only allowed in provider/proof/tooling boundaries, not stable law or deterministic simulation.

10. Service-First Provider Architecture

The final provider doctrine:

Service identity is first-party.

Provider implementation is replaceable.

Profiles select providers.

Third-party libraries are fenced.

Apps stay generic.

Contracts define the law.

Service roots should be generic:

platform

input

render

audio

asset

script

ui

storage

diagnostics

command

view

projection

Provider implementations should live under:

runtime/<service>/providers/<provider>/

Example:

```
runtime/render/providers/raylib/  
runtime/render/providers/rlgl/  
runtime/render/providers/rlsw/  
runtime/platform/providers/sdl2/  
runtime/script/providers/lua54/
```

Third-party source belongs under:

```
external/upstream/<third_party>/
```

or an explicitly chosen equivalent such as `external/vendor/`.

Provider choices belong under profiles:

```
release/profiles/dev/client.raylib.toml  
release/profiles/dev/client.sdl2_opengl33.toml  
release/profiles/dev/workbench.raylib.toml  
release/profiles/dev/server.null.toml
```

Not under app paths.

11. Raylib, SDL2, Lua, ImGui, and Third-Party Libraries

The chat accepted using raylib aggressively, but only as a provider suite.

Raylib ecosystem roles:

```
raylib = broad seed provider suite  
rlgl = raylib low-level OpenGL abstraction provider  
rlsw = raylib software renderer provider  
raygui = early Workbench/debug UI provider  
raudio = audio provider if used directly
```

raymath = provider-local render/editor math helper
rttextures/rmodels = import/preview helpers

SDL2 role:

platform/input/audio provider family

Lua role:

pinned script provider behind dominium.script.v1 or domino.
script.v1

not raw mod ABI

Dear ImGui role:

optional Workbench/debug tooling UI provider

not UI law

Forbidden leakage:

raylib.h
rlgl.h
raymath.h
SDL.h
lua.h
imgui.h

should not appear in:

engine/
game/
contracts/
content/
saves/
replays/
public SDK headers

Allowed only in provider/proof/experiment boundaries.

12. Workbench, Modules, Packs, Apps, and Composition

Vocabulary:

Component = source/build ownership unit

Service = callable runtime capability

Provider = replaceable implementation

Pack = distributable authored payload

Module = declared functional extension unit

Workspace = large user-facing Workbench composition

App = shipped product composition

Artifact = persisted versioned thing

Rules:

A folder can contain a component.

A manifest defines a module.

A pack can provide modules.

An app composes modules, providers, services, and packs.

A workspace presents workflows.

Workbench structure:

apps/workbench/shell/

apps/workbench/module/

apps/workbench/workspace/

Reusable modules should not live under `apps/workbench/module/`; Workbench modules are UI modules only.

General module law:

contracts/module/

Pack-delivered module payload:

13. Presentation / Projection Model

The final semantic spine:

```
intent
-> command
-> capability/refusal check
-> service
-> result | document | snapshot
-> diagnostics/evidence
-> view
-> action model
-> projection
-> shell
```

Definitions:

```
command = behavior request
result = command output
refusal = typed failure
document = persistent/editable logical object
snapshot = read-only state projection
patch = lawful proposed mutation
view = semantic presentation model
action = operation available from a view
projection = CLI/text/rendered/native/headless realization
shell = product host
```

Do not create separate semantic contract worlds for CLI/TUI/rendered/native. These are projections:

```
runtime/projection/cli/
```

runtime/projection/text/
runtime/projection/rendered/
runtime/projection/native/
runtime/projection/headless/

Contracts should define semantic surfaces:

contracts/command/
contracts/action/
contracts/result/
contracts/refusal/
contracts/diagnostic/
contracts/document/
contracts/patch/
contracts/view/
contracts/presentation/
contracts/projection/

14. Content, Packs, Saves, Replays, and Artifacts

Preferred pack layout:

content/packs/<category>/<pack_id>/
pack.manifest.json
data/
assets/
docs/
ui/
scenarios/
locks/ # only if authored, not generated

Pack IDs do not depend on path.

Pack composition should be deterministic:

base pack
-> official extension pack
-> mod pack
-> theme pack
-> user profile pack
-> workspace override pack

Rules:

no silent overwrite
ordered overlays
declared conflicts
schema validation
hash/provenance tracking
capability negotiation
lockfile output

Save/replay law:

stable IDs
schema IDs
explicit little-endian encoding
fixed-width values
canonical serialization
migration/refusal behavior
no native layout serialization

15. Proof Gates and Current Project State

The conversation distinguished between normal development gates and full release gates.

Normal fast strict gate:

AIDE doctor/validate

RepoX strict
root/layout/no-src/forbidden-name validators
canonical structure validator
public surface / ABI / dependency direction checks
CMake verify/build
smoke CTest
focused capability/provider/package/schema checks

Full/T4 gate:

full CTest
compatibility corpus
public header consumer tests
replay determinism tests
save migration tests
pack/mod trust tests
provider conformance tests
release promotion checks

As of the latest user-reported state:

canonical structure: credible / pass with warnings
fast strict: not fully clean if stale evidence/marker debt persists
smoke CTest: reported passing in several runs
full CTest: not green
feature readiness: limited
broad feature work: blocked

16. Major Reported Commits and Statuses

The user reported several task results. Important ones include:

- `refactor(repo):` `finish canonical structure cleanup` - actual structure cleanup, schema routing, CompatX cleanup, DomUI routing, test taxonomy moves, provider/profile guardrails, Workbench/projection boundaries, validators, fresh structure export. Status: `PASS_WITH_WARNINGS`.
- `tests:` `route full-gate legacy path expectations` - routed full-gate tests away from retired paths such as `game/rules`, `contracts/schemas`, `data/profiles`, `libs/appcore`, old docs roots, old tools roots. Status: `PASS_WITH_WARNINGS`; targeted subset 9/9 passed.
- `repo:` `define Domino framework boundary` - defined Domino Framework as `contracts + public headers + service/provider law`, not `top-level framework/`. Added validator and doc. Status: `PASS_WITH_WARNINGS`.
- `repo:` `enforce service-first provider structure` - moved `null/software render providers` under `runtime/render/providers`, added provider structure law, release profiles, external roots, validators. Status: `PASS_WITH_WARNINGS`.

These statuses are user-reported and should be verified against the live repository if used operationally.

17. What We Put Off for Later

Deferred or still pending:

FULL-GATE-GENERATED-EVIDENCE-REFRESH-01
FAST-STRICT-EVIDENCE-MARKER-REPAIR-01
PROJECTION-CONFORMANCE-01
PRESENTATION-CONTRACT-01
PACK-INTERNAL-LAYOUT-CANON-01
RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01
AIDE-STATE-CLASSIFICATION-01
PUBLIC-HEADER-ABI-PROMOTION-01
STORAGE-PACKAGE-PROVIDER-SPLIT-01
FULL-CTEST-AUDIT-NONPATH-01
PROVIDER-WEDGE-01 / RAYLIB-SEED-PROVIDER-01
SDL2-PROVIDER-01
LUA-PROVIDER-PIN-01

Broad feature work remains blocked until proof gates improve:

large Workbench UI
runtime module loader
provider runtime
package runtime
gameplay expansion
renderer implementation
native GUI
release publication

Narrow governed slices are acceptable if gates are green.

18. Things Future Assistants Must Not Do

Do not:

```
add top-level framework/  
add top-level modules/  
add top-level plugins/  
add top-level services/  
add top-level profiles/  
add top-level labs/  
add top-level sdk/  
reintroduce src/ or source/  
make raylib/SDL/Lua architecture law  
put provider choices in app paths  
treat Workbench as authority  
move authored pack payloads under contracts/package  
let tools become a second source tree  
treat generated reports as source truth  
overclaim PASS_WITH_WARNINGS as full green  
restart broad structure cleanup unless a hard validator fails
```

19. What a New Chat Should Do First

A new chat should ask for the latest current repo status or inspect the live repository. Then:

1. Verify current branch/commit.
2. Verify AIDE doctor/validate.
3. Verify fast strict / RepoX strict.

4. Verify canonical structure validators.
5. Verify current queue/current.toml.
6. Do not run broad restructure by default.
7. Pick the next task based on actual gate state.

Likely next tasks:

If fast-strict/RepoX blocked:

FAST-STRICT-EVIDENCE-MARKER-REPAIR-01

If proof gates are clean:

PROJECTION-CONFORMANCE-01

If provider implementation is now allowed:

PROVIDER-WEDGE-01 / RAYLIB-SEED-PROVIDER-01

20. Final Synthesis

The ultimate design is:

Dominium is a deterministic, contract-governed, provider-backed, pack-composed simulation platform built on reusable Domino surfaces.

The key architectural formula is:

contracts define semantic surfaces

runtime implements services and providers

engine preserves deterministic substrate

game defines Dominium meaning

content supplies authored payloads

apps compose products

Workbench operates the system

tools validate/generate/migrate

tests and evidence prove behavior

The most important preserved decision is:

Domino Framework is not a top-level framework/ root.

It is public surfaces, contracts, ABI law, service/provider law, public headers, conformance tests, and profiles.

The most important operational status is:

Structure is credible.

Full proof is not complete.

Feature readiness is limited.

Continue with targeted proof and governed slices, not broad refactors.

Chapter 30

Development Routes

30.1 Dominium Development Routes Reader Brief

Source: docs/archive/conversations/development_routes/dominium_development_routes__05_reader_brief.md

Report type: reader_brief

Size: 4099 bytes

30.1.1 Reader Brief - Dominium Development Routes

What This Chat Was About

This chat primarily concerned early technical planning for Dominium. The user asked what development routes are usual for games like Dominium and what technical/feature components should be worked on in which order. The prior assistant answered with a route framework and recommended a kernel-first deterministic simulation path. It proposed that Dominium should start from mathematical/time/RNG/serialization primitives, then

build a deterministic simulation kernel, world model, core systems, persistence/replay/sync, tooling, presentation, content, and finally modding/multiplayer/distribution.

The crucial caveat is that the user did not visibly accept the proposed route. Treat it as a strong assistant proposal, not as a confirmed project decision. The chat then shifted into continuity preservation: the user requested a maximum-fidelity Context Transfer Packet, and later requested this final downloadable report package. This package is chat-specific and intended for later aggregation with reports from other old chats into a full Project Spec Book and Master Living State File.

Most Important Things to Know

- Route C - Kernel-First Deterministic Simulation was proposed, not confirmed.
- Exact principle to preserve: "Simulation precedes representation. State precedes UI. Determinism precedes performance. Data precedes content."
- Proposed stack: primitives -> deterministic kernel -> world model -> systems -> persistence/replay/sync -> tooling -> presentation -> content -> distribution/modding/multiplayer.
- Proposed feature completion rule: deterministic,

serializable, reproducible, inspectable, moddable.

- No engine, language, codebase, files, genre, team, or timeline was established.
- Dominion's actual core loop is unknown.
- Modding and multiplayer were assistant-proposed considerations, not user-confirmed requirements.
- Rejected options are tentative assistant warnings, not final user decisions.
- This package is for this chat only.
- External technical claims should be verified before being used as evidence.

Active Plans or Workstreams

- WORKSTREAM-01: Dominion technical development route and component ordering.
- WORKSTREAM-02: Chat-specific context transfer and reusable handoff packaging.
- WORKSTREAM-03: Future aggregation into Project Spec Book and Master Living State File.

Decisions Already Made

- Use chat label: Dominion Development Routes.

- Use date anchor: 2026-05-27 Australia/Melbourne.
- Limit package scope to this chat only.
- Create the requested Markdown/YAML files and ZIP archive.
- Preserve Route C as proposal-level, not confirmed.

Pending Tasks

- Confirm whether Route C is accepted.
- Define Dominion genre and core player loop.
- Define minimum deterministic simulation slice.
- Define time, numeric, RNG, entity, mutation, and serialization primitives.
- Confirm engine/language/platform constraints.
- Confirm whether modding and multiplayer are real goals.
- Verify whether external files or repositories exist.

Open Questions

- Is Dominion actually simulation-heavy?
- What is the core player loop?
- What are target platforms?

- What tech stack will be used?
- What level of determinism is required?
- Is cross-platform replay needed?
- Are mods and multiplayer in scope?
- What is prototype v0.1 success?

Files / Artifacts / Prompts to Preserve

- Initial Dominion development-route prompt.
- Prior assistant Route A/B/C roadmap answer.
- Maximum-fidelity Context Transfer Packet.
- Current final packaging prompt.
- Later aggregator prompt.
- Generated report files and ZIP archive.

What to Verify Before Acting

- Route C acceptance.
- Genre/core loop.
- Simulation scale.
- Determinism/replay target.
- Tech stack and platforms.

- Modding/multiplayer requirements.
- External files or repositories.
- Whether any later chats superseded this proposal.

Best Next Step

Produce **Dominium Technical Roadmap v0.1** using this report as input, explicitly marking every assumption and open question.

Chapter 31

Documentation Standards Readme

31.1 Documentation Standards Readme Handoff Reader Brief

Source: docs/archive/conversations/
documentation_standards_readme/documentation_
standards_readme_handoff__05_reader_brief.md

Report type: reader_brief

Size: 4743 bytes

31.1.1 Reader Brief - Documentation Standards and README Handoff

What This Chat Was About

This chat designed a documentation strategy and automation plan for a long-lived C/C++ systems codebase in the Dominion Game project context. It did not scan the repository or modify files. Its outputs were standards, examples, Codex prompts, and a context-transfer packet. The key theme was how to make

documentation support maintainability, extensibility, determinism, explicit invariants, and mechanically enforced discipline.

The main documentation model is layered: public API contracts belong in headers; implementation rationale belongs in source files; architecture, dependencies, build topology, style, contracts, and subsystem specs belong in docs/. README.md should serve as a public landing page and technical navigation hub until a website exists.

The chat also produced a user-specified documentation ratio quality gate plan: Python 3 script, CMake integration, local warnings, CI failures, exact min/max thresholds, and no compiler/clang/AST dependency.

Later, the user said the project had been heavily refactored in setup, launcher, and game design chats, and asked for a prompt to scan the repo, update every docs/ file, and overhaul README. That latest prompt is the highest-priority repo-facing artifact.

Most Important Things to Know

- No repo was scanned in this chat.
- No files were modified before this package.
- Actual repo contents must be treated as authoritative once inspected.
- The wider project context is Dominion Game, but

exact repo name remains unverified.

- Headers are intended to be canonical homes for public API contracts.
- Source files should document rationale, invariants, hazards, and non-obvious implementation details only.
- docs/ should hold architecture and policy.
- README should be a layperson-readable and engineer-useful landing page.
- Documentation quality should be mechanically enforced if that workstream is implemented.
- The quality gate thresholds are exact user-provided requirements.
- Local quality-gate violations should warn; CI violations should fail.
- Generated/vendor/build/external code should be excluded from ratios.
- The latest post-refactor docs reconciliation prompt is the most relevant next-action prompt.
- Prompt A.4 has a known typo that must be corrected before reuse.
- Other setup/launcher/game chats are not visible here.

- Future aggregation must preserve FACT / INFERENCE / UNCERTAIN / PROJECT-CONTEXT labels.

Active Plans or Workstreams

- WORKSTREAM-01: Per-file documentation standard.
- WORKSTREAM-02: docs/ technical documentation structure.
- WORKSTREAM-03: README landing page.
- WORKSTREAM-04: Documentation ratio quality gate.
- WORKSTREAM-05: Standards-aligned Codex prompt optimization.
- WORKSTREAM-06: Post-refactor documentation reconciliation.
- WORKSTREAM-07: Reusable report package.

Decisions Already Made

- README should be public landing page and docs hub.
- Documentation should not be speculative.

- Documentation ratio quality should be mechanically enforceable.
- Python 3 and CMake are the chosen quality-gate implementation route.
- Local builds warn; CI fails.
- No compiler-specific diagnostics, clang-tools, AST parsing, or external linters for the ratio gate.
- Generated/third-party/build code is excluded from ratio checks.

Pending Tasks

- Inspect the repo.
- Inventory components and CMake targets.
- Reconcile docs/.
- Overhaul README.
- Implement doc ratio checker if chosen.
- Correct Prompt A.4 typo.
- Apply source/header documentation pass later if desired.

Open Questions

- Exact project name, license, supported platforms, and maturity.
- Actual top-level directories and refactored component structure.
- Actual docs/ inventory.
- Actual CMake target graph.
- Whether source/header docs should be applied before or after quality gate.
- Whether Codex should commit changes or produce patches.

Files / Artifacts / Prompts to Preserve

- ARTIFACT-08: full-file documentation Codex prompt.
- ARTIFACT-07: README landing page prompt.
- ARTIFACT-09: documentation ratio quality gate prompt.
- ARTIFACT-11: upgraded standards prompt with known typo.
- ARTIFACT-12: latest post-refactor docs reconciliation prompt.

- ARTIFACT-17: prior Context Transfer Packet.
- This final package.

What to Verify Before Acting

- Repository tree.
- CMake targets.
- Existing docs/.
- README.md.
- LICENSE.
- Platform support.
- CI config.
- Source roots.
- Generated/vendor/build exclusions.
- Existing documentation style.

Best Next Step

Inspect the current repository, then use the post-refactor docs reconciliation prompt to update docs/ and README.md without changing runtime or build behavior.

Chapter 32

Dominium Domino Codex Planning

32.1 Dominium Domino Codex Planning Reader Brief

Source: docs/archive/conversations/dominium_domino_codex_planning/dominium_domino_codex_planning__05_reader_brief.md

Report type: reader_brief

Size: 9132 bytes

32.1.1 Reader Brief - Dominium Domino Codex Planning

What This Chat Was About

This chat was about preserving and extending planning state for the Dominium + Domino project. The user first supplied a detailed master architecture prompt:

Domino is a deterministic ISO C89 fixed-point engine; Dominium is a portable C++98 product suite; platform access goes through dsys; rendering goes through dgfx;

GUI/TUI/CLI must share logic; old saves/mods/packs/tools must not silently break. The assistant generated a forward plan and then multiple Codex prompts to implement the architecture in stages.

The first major output was a six-prompt Milestone-0 sequence: specs, repo/CMake skeleton, deterministic numeric core, DOMINIUM_HOME manifest/launcher enumeration, minimal deterministic sim/headless game/TLV save, and launcher-to-game process spawning. The second major output was three prompts for universal CLI, TUI, and GUI layers across launcher, game, setup, and tools. The third output branch generated prompts for shared input/state/IME and multi-window GUI/dgfx pipelines/GPU backend detection. A requested third prompt for pack-system integration was not generated and remains a carry-forward task.

The chat also developed a startup policy: product executables should use CLI when run from a terminal, and when double-clicked/no terminal should try GUI, then TUI, then CLI. Explicit --mode flags should override auto mode; game server/headless flags should force headless. A Codex prompt was generated for this, but it needs design repair before implementation because a shared C startup module directly referencing product-specific C++ wrappers may create link failures.

The user later asked for platform/render/API questionnaire filling based on dominium.zip. The answer

given in the prior chat must be treated as unverified until the repo is inspected in a new chat.

Most Important Things to Know

- Domino must remain ISO C89.
- Dominion product/UI code must remain portable C++98.
- dsys is the only platform abstraction boundary.
- dgfx is the only rendering boundary.
- Simulation must be deterministic and fixed-point.
- Old saves/mods/packs/tools must not silently break.
- Vector-only UI fallback must always work.
- Six Milestone-0 Codex prompts were generated.
- Universal CLI/TUI/GUI prompts were generated.
- Shared input/state/IME and multi-window/dgfx/backend prompts were generated.
- Pack-system integration prompt was requested but not generated.
- Startup policy was designed and prompted.
- Startup prompt has a linker-risk caveat.

- Repo-specific questionnaire facts require verification.

Active Plans or Workstreams

- WORKSTREAM-01 - Dominion + Domino Core Architecture Continuity
- WORKSTREAM-02 - Milestone-0 Vertical Slice via Six Codex Prompts
- WORKSTREAM-03 - Shared CLI, TUI, and GUI Layers Across Products
- WORKSTREAM-04 - Shared Input System, State Machines, and Cross-Platform IME
- WORKSTREAM-05 - Multi-Window GUI, dgfx Pipelines, and GPU Backend Detection
- WORKSTREAM-06 - Pack System Integration Prompt
- WORKSTREAM-07 - Unified Startup and Mode Dispatcher
- WORKSTREAM-08 - Platform, Renderer, API Questionnaire and Repo-State Answer
- WORKSTREAM-09 - Chat Handoff and Report Package

Decisions Already Made

- DECISION-01 - Domino engine is ISO C89. (FACT)
- DECISION-02 - Dominion product/UI code may use portable C++98 subset where needed. (FACT)
- DECISION-03 - All platform interaction must go through dsys. (FACT)
- DECISION-04 - All rendering must go through dgfx IR/backends. (FACT)
- DECISION-05 - Software renderer is mandatory baseline. (FACT)
- DECISION-06 - Vector-only fallback UI/pack path must always function. (FACT)
- DECISION-07 - DOMINIUM_HOME repository model is canonical for products, mods, packs, and instances. (FACT)
- DECISION-08 - Backward/forward compatibility is mandatory for saves, packs, mods, tools where possible. (FACT)
- DECISION-10 - Codex prompts should be self-contained implementation packages. (FACT)

- DECISION-11 - CLI/TUI/GUI should be common/shared across launcher, game, setup, and tools. (FACT)
- DECISION-12 - TUI and GUI must use dsys/dgfx abstractions, not direct platform drawing. (FACT)
- DECISION-13 - Input/IME/state machine systems should be shared and deterministic. (FACT)
- DECISION-14 - GPU backend detection must always include software fallback. (FACT)
- DECISION-15 - Terminal invocation with no explicit mode defaults to CLI. (FACT)
- DECISION-16 - Double-click/no-terminal launch defaults to GUI, then TUI, then CLI fallback. (FACT)
- DECISION-21 - This old chat should be exported as a downloadable multi-file report package. (FACT)

Pending Tasks

- TASK-01 - Verify actual repo state from dominium.zip or current checkout.
- TASK-02 - Determine which generated Codex prompts were executed.

- TASK-03 - Build and run golden target set.
- TASK-04 - Audit generated numeric core for strict C89 and deterministic overflow behavior.
- TASK-05 - Define endian-stable TLV IO helpers if not already present.
- TASK-06 - Verify launcher->game process spawn path and manifest path handling.
- TASK-07 - Keep docs/specs aligned with generated code.
- TASK-08 - Implement or refine common CLI framework.
- TASK-09 - Implement or refine common TUI framework.
- TASK-10 - Implement or refine common GUI framework.
- TASK-11 - Implement shared input/state/IME after inspecting existing modules.
- TASK-12 - Implement or rewrite multi-window/dgfx pipeline/backend detection prompt against actual renderer API.

Open Questions

- QUESTION-01 - Which generated Codex prompts have actually been applied to the repo?
- QUESTION-02 - What is the actual current repo structure and target list in dominium.zip/current checkout?
- QUESTION-03 - Does the repo already contain files from prompts 1-6?
- QUESTION-04 - How should strict C89 support exact 64-bit types and q48_16 across retro compilers?
- QUESTION-05 - Should TLV save format be native-endian temporarily or endian-stable immediately?
- QUESTION-06 - Where should startup mode dispatch live to avoid C/C++ linker issues?
- QUESTION-07 - Should Windows products be console-subsystem, windows-subsystem, or dual wrapper binaries?
- QUESTION-08 - Should future startup mode overrides via env/config be supported?
- QUESTION-09 - What are final Tier 1/2/3 platform priorities?

- QUESTION-10 - Should common CLI/TUI/GUI prompts be applied as broad prompts or split into smaller implementation prompts?

Files / Artifacts / Prompts to Preserve

- ARTIFACT-01 - Extended Master Starter Prompt
- Dominium + Domino
- ARTIFACT-03 - Forward plan phases 0-12
- ARTIFACT-04 - Codex Prompt 1 - SPEC__
IDENTITY + SPEC__PRODUCTS
- ARTIFACT-05 - Codex Prompt 2 - Repo layout,
CMake skeleton, core/product targets
- ARTIFACT-06 - Codex Prompt 3 - Deterministic
numeric core + test harness
- ARTIFACT-07 - Codex Prompt 4 - DOMINIUM__
HOME repo model + launcher enumeration
- ARTIFACT-08 - Codex Prompt 5 - Deterministic
sim + headless game + TLV save
- ARTIFACT-09 - Codex Prompt 6 - Launcher to
game process spawn via dsys
- ARTIFACT-10 - Prompt A - Universal CLI Layer
- ARTIFACT-11 - Prompt B - Universal TUI Layer

- ARTIFACT-12 - Prompt C - Universal GUI Layer
- ARTIFACT-13 - Prompt - Shared Input System + State Machines + Cross-Platform IME
- ARTIFACT-14 - Prompt - Multi-window GUI + dgfx Pipelines + GPU Backend Detection
- ARTIFACT-15 - Missing Prompt - Pack system integration
- ARTIFACT-16 - Prompt - Unified Startup / Mode Dispatcher
- ARTIFACT-17 - User questionnaire: Platforms, Renderers, APIs, Completion, Repo/build context
- ARTIFACT-18 - User-pasted Codex-in-repo questionnaire answer
- ARTIFACT-19 - Assistant refined questionnaire answer
- ARTIFACT-20 - User-pasted root CMakeLists.txt
- ARTIFACT-21 - User-pasted source/CMakeLists.txt

What to Verify Before Acting

- VERIFY-01 - Open/extract dominium.zip or current checkout.

- VERIFY-02 - Inspect root CMakeLists.txt, source CMake files, and CMakePresets.json.
- VERIFY-03 - Search for files specified by prompts 1-6.
- VERIFY-04 - Build golden target set and run numeric/headless tests.
- VERIFY-05 - Inspect numeric typedef/fixed-point implementation for C89 compliance.
- VERIFY-06 - Inspect existing CLI/TUI/GUI/ input/state modules before applying common-framework prompts.
- VERIFY-07 - Inspect dgfx/render headers and backend implementation names.
- VERIFY-08 - Inspect content/pack/data format docs before generating pack-system prompt.
- VERIFY-09 - Inspect CMake target graph before implementing startup dispatcher.
- VERIFY-10 - Verify platform tier/render backend claims in docs/source.

Best Next Step

Verify the current repo from `dominium.zip` or the active checkout. Then determine which generated Codex

prompts were actually applied. After that, either generate the missing pack-system integration prompt or repair the startup dispatcher implementation design, depending on the user's next instruction.

Chapter 33

Dominium Full Conversation

33.1 Dominium Full Conversation Companion Report Manifest

Source: docs/archive/conversations/dominium_full_conversation/companion_report/dominium_full_conversation_companion_report__00_manifest.md

Report type: companion_report

Size: 2867 bytes

33.1.1 Manifest - Dominium Workbench, AIDE, Presentation Architecture, Provider Strategy, Product-Spine Planning, and Preservation Companion

Date anchor: 2026-05-27 Australia/Melbourne

This package is an accompanying human-readable companion report for the long Dominium/Domino/Workbench/AIDE planning conversation. It supplements the earlier preservation package already created in this

chat and bundles the earlier reports, screenshots/mockups, uploaded preservation instruction, and this new companion report into one archive.

Files created in this companion package

Row 1 File: dominium_full_conversation_companion_report__00_manifest.md; Purpose: Manifest for the companion package.

Row 2 File: dominium_full_conversation_companion_report__01_complete_human_report.md; Purpose: Main human-readable detailed report of the whole conversation.

Row 3 File: dominium_full_conversation_companion_report__02_decision_task_risk_registers.md; Purpose: Condensed structured registers for decisions, workstreams, tasks, risks, and verification.

Row 4 File: dominium_full_conversation_companion_report__03_current_plan_and_prompt_queue.md; Purpose: Current practical execution plan and prompt queue as of the end of the visible conversation.

Row 5 File: dominium_full_conversation_companion_report__04_specbook_bridge.md;

Purpose: How this chat should feed into a future master Project Spec Book.

Row 6 File: `dominium_full_conversation_companion_report__05_verification_checklist.md`; Purpose: Verification queue and stale fact checklist.

Row 7 File: `dominium_full_conversation_companion_report__06_artifact_index.md`; Purpose: Index of previous generated files and visual/source artifacts bundled in the ZIP.

Row 8 File: `dominium_full_conversation_companion_report__07_reader_brief.md`; Purpose: Shorter reader brief for quick review.

Row 9 File: `dominium_full_conversation_companion_report__FULL_COMBINED_REPORT.md`; Purpose: All companion report files combined into one Markdown file.

Row 10 File: `dominium_full_conversation_companion_report__handoff_package.zip`; Purpose: ZIP containing this companion package, previous preservation package files, screenshots/mockups, and uploaded instruction text.

Included supporting material in the ZIP

- Previous preservation files named `dominium_workbench_aide_architecture_preservation_*.*`
- Earlier package files named `Dominium_Workbench_AIDE_Presentation_Provider_Strategy_*.*` if present.
- Uploaded screenshot/mockup images from May 18, 2026, if present in `/mnt/data`.
- The uploaded `Pasted text.txt` preservation instruction.

Caveats

- This is a reconstruction from visible conversation context plus generated files available in the sandbox. It is not a forensic raw transcript export.
- Live repository status must still be verified before executing the next Codex prompt.
- External software facts about raylib, SDL2, Lua, CMake, language baselines, and OS support should be verified before becoming formal requirements.

33.2 Dominion Full Conversation Companion Report Complete Human Report

Source: docs/archive/conversations/dominium_full_conversation/companion_report/dominium_full_conversation_companion_report__01_complete_human_report.md

Report type: companion_report

Size: 32212 bytes

33.2.1 Complete Human-Readable Companion Report - Dominion Workbench, AIDE, Presentation Architecture, Provider Strategy, Product-Spine Planning, and Preservation Companion

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Dominion Workbench, AIDE, Presentation Architecture, Provider Strategy, Product-Spine Planning, and Preservation Companion

Row 2 Field: Date anchor; Assessment: 2026-05-27 Australia/Melbourne

Row 3 Field: Source scope; Assessment: This chat only
1661

unless explicitly labelled PROJECT-CONTEXT.

Row 4 Field: Apparent access; Assessment: Partial-to-broad visible context. The current chat includes a very large visible transcript, user-provided handoffs, generated prompts, uploaded screenshots, uploaded preservation instructions, and prior generated report files. It is not guaranteed to include every skipped early turn verbatim.

Row 5 Field: Previously generated files available?; Assessment: Yes. Earlier preservation packages and report files are present in `/mnt/data`.

Row 6 Field: Uploaded files/artifacts present?; Assessment: Yes. The uploaded preservation instruction, multiple screenshots/mockups, and prior generated packages are visible in the sandbox.

Row 7 Field: Contains future plans?; Assessment: Yes. The chat contains extensive near-term, medium-term, and long-term roadmaps.

Row 8 Field: Contains decisions?; Assessment: Yes. Several major architectural decisions were accepted or revised.

Row 9 Field: Contains pending tasks?; Assessment: Yes. The latest user-selected maintenance tasks and future queue are preserved.

- Row 10** Field: Contains unresolved questions?; Assessment: Yes. Live repo status, execution state of prompts, full-gate debt, and sequencing decisions require verification.
- Row 11** Field: Staleness risk; Assessment: High for live repo/current software status; medium for roadmap sequencing; low for settled chat doctrine.
- Row 12** Field: Extraction confidence; Assessment: 4/5 for visible chat substance; 3/5 for live repo status.
- Row 13** Field: Safe for later aggregation?; Assessment: Yes, with caveats.
- Row 14** Field: Main limitations; Assessment: No raw full transcript export, some early turns were summarized/skipped, external repo/software facts may be stale, and screenshots were treated as design references rather than formal specs.

Plain-language limitations This companion report preserves the meaning, decisions, and task plan of the conversation as visible in this chat. It is not a line-by-line transcript. Where the chat discussed the live `julesc013/dominium` repo, those statements should be treated as **UNCERTAIN / UNVERIFIED** until checked against the repository. Where the user pasted status reports from Codex or another chat, those are

preserved as **FACT stated in chat**, not independently verified facts unless separately checked.

1. One-Page Orientation

This conversation records a major architectural transition in the Dominion project. It began with an immediate practical concern: the existing Windows launcher UI looked broken, flickered, and needed a better way to design UIs. The initial direction was to create a Windows-first UI Editor and eventually a Tool Editor that could visually design pixel-perfect UIs and generate TLV-based layouts/code. That early plan was explored in depth: TLV schemas, UI IR, deterministic JSON mirrors, layout engines, Win32/DGFX/null backends, action code generation, import/export, CLI scripting, native widget previews, and launcher/setup redesign tests were all discussed and turned into prompt plans.

Over the course of the chat, the user corrected the direction. The old UI Editor/Tool Editor path was judged useful as an idea but wrong as a final architecture. OS-native GUI design can rely on first-party tools such as Visual Studio and Xcode where appropriate. Dominion's real internal tooling should instead prove and reuse the same runtime, command, rendering, UI, package, and diagnostic systems that the client and other products need. This led to the replacement concept: **Dominium Workbench**

Platform.

Workbench is the central product concept of this chat. It is not a monolithic IDE and not an authority over the project. It is a cross-platform rendered, modular, command-driven, agent-aware production environment. It should eventually let the user and agents build the entire project: code scaffolds, packs, modules, UI/HUD documents, themes, renderer tests, release artifacts, diagnostics, evidence packets, AIDE/Codex work units, and product compositions. Its design doctrine is: Workbench designs artifacts; runtime executes artifacts; contracts constrain artifacts; content packages artifacts; apps consume artifacts; tests prove artifacts; AIDE governs changes; Codex patches code.

The second major outcome was an AIDE development model. The chat established that **development should be non-blocking, promotion should be evidence-blocked**. This means work can accumulate on task branches and dev with classified partials or warnings, but main should only receive checkpointed, evidence-backed promotion. Codex is a bounded patch executor; AIDE is a scheduler, context compiler, evidence ledger, blocker classifier, repair/resume generator, checkpoint coordinator, and promotion gatekeeper. A series of AIDE prompts were generated or planned: workflow law, WorkUnit schema, dev/main policy, checkpoint loop, and capability reality ledger.

The third major outcome was a unified presentation doctrine. CLI, TUI/text, rendered GUI, OS-native GUI, and headless reports must not become separate product implementations. They are projections over the same semantic spine: command/result/refusal/document/view/action/diagnostic/evidence. The TUI is a serious first-class projection. The rendered GUI is a backend-agnostic system of layouts, controls, widgets, style tokens, themes, view documents, draw lists, and provider-backed renderers. OS-native widgets remain optional projections, not core semantics.

The fourth major outcome was a service-first provider strategy. The chat concluded that raylib, rlgl, rlsw, raygui, raudio, SDL2, Lua, and related libraries can be used aggressively, but only as replaceable providers behind Dominion service contracts. Dominion owns contracts, packets, saves, replays, packs, UI documents, asset identity, provider law, and simulation law. Third-party types must not leak into engine, game, contracts, content, pack schemas, save/replay formats, command/result schemas, or public SDK surfaces.

By the end, the project direction had moved from broad structure cleanup to targeted maintenance and product-spine development. The user reported that PRESENTATION-CONTRACT-01 had completed with PASS_WITH_WARNINGS, then selected six targeted maintenance prompts to generate before replanning: FULL-GATE-

LEGACY-TEST-ROUTE-01, PACK-INTERNAL-LAYOUT-CANON-01, RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01, PUBLIC-HEADER-ABI-PROMOTION-01, STORAGE-PACKAGE-PROVIDER-SPLIT-01, and POINTER-WIDTH-SERIALIZATION-AUDIT-01. The last generated prompt in the visible chat was FULL-GATE-LEGACY-TEST-ROUTE-01.

A future assistant must understand that this chat is not simply about UI. It records the architecture for how Dominion should build itself: through contracts, commands, services, providers, documents, patches, modules, packs, apps, workspaces, diagnostics, evidence, AIDE, Codex, and eventually Workbench.

2. The Story of the Conversation

2.1 The initial problem: launcher UI and the old UI Editor concept The conversation began with the user wanting a product designer/software architect/prompt engineer to help design an internal graphical UI creation tool. The immediate pain point was the Dominion Windows launcher UI: it looked mangled and flickered. The user wanted a tool that could visually design pixel-perfect UIs and broad-strokes functionality for setup, launcher, game, and tools.

Early discussion clarified target operating systems, the current DUI/TLV schema/state system, and the existing

backends: Win32 common controls, DGFX, and null. The user wanted true native controls where possible, custom rendering only where necessary, and eventually universal UI definitions across products. We discussed DPI scaling, canonical TLV, deterministic JSON mirrors, codegen, event binding, flicker causes, and bootstrap phases.

A large prompt plan followed: repo scaffolding, canonical UI IR, TLV I/O, capability system, layout engine, splitter/tabs/scroll widgets, action codegen, Phase A UI Editor, Tool Editor bootstrap, Win32 batching/flicker fixes, tests, docs, and capability tests through launcher/setup redesigns.

2.2 UI Editor gains CLI/import/export and launcher/setup tests The user then asked for the UI Editor to import existing tools and modify their UIs, expose a CLI so Codex could automate designs, and use the UI Editor to create launcher/setup designs similar to Minecraft's logical layout. The key correction was that "Minecraft-style" meant logical layout, not graphical skin: native Win32 widgets only, structured around header/sidebar/tabs/footer for launcher and wizard-style pages for setup. We generated prompts for UI discovery/import/export, CLI modes, `ops.json` scripts, launcher generation, setup generation, and hardening.

2.3 The pivot: old editors are not the final

product The user then redirected the entire plan. They argued that native OS widget GUI tools already exist through Visual Studio, Xcode, etc. The project needed a cross-platform rendered tool environment that uses the same CLI, TUI, and rendered GUI systems as the client. The old UI Editor and Tool Editor were good exploratory ideas but bad final products. This produced the Dominion Workbench concept.

2.4 Workbench becomes the production

environment Workbench was defined as a rendered, modular, command-driven userland tools environment over the same spine as the client. It should not be one giant editor. It should be one shell with modules and workspaces. Workspaces include Project Graph Explorer, Interface Studio, Module/Pack Foundry, App Composer, Release Forge, Agent Control Workspace, Renderer/Theme Laboratory, and Replay/Trace Workspace. Smaller reusable modules include validation runner, pack browser, command browser, document inspector, patch/diff viewer, evidence viewer, renderer test panel, theme previewer, and asset browser.

The user stressed that as the sole developer, they want to use Workbench and tool modules to actually build the entire Domino/Dominium project. That expanded Workbench from a command dashboard into a visual/

agentic production environment: a place to create code scaffolds, data, packs, UI, graphics, sounds, tests, release artifacts, and AIDE/Codex work units. The governing loop became: intent -> plan -> generate/edit -> preview/simulate -> validate/test -> patch/commit/package -> evidence -> next action.

2.5 AIDE and parallel development The discussion then moved into AIDE. The user wanted to automate as much as possible through Codex and AIDE. We developed a task operating model: AIDE creates WorkUnits, tracks attempts, blockers, evidence, repairs, resumes, checkpoints, and promotion decisions. Codex executes bounded tasks. Development can continue with classified partials and repairable blockers; promotion to main requires evidence.

This led to generated prompts: STATUS-RECONCILE-02, AIDE-WORKFLOW-LAW-01, AIDE-WORKUNIT-SCHEMA-01, AIDE-DEV-MAIN-POLICY-01, AIDE-CHECKPOINT-LOOP-01, and AIDE-CAPABILITY-REALITY-LEDGER-01. The user wanted to run AIDE workflow, WorkUnit schema, and dev/main policy concurrently on the same machine via separate Codex tasks.

2.6 Presentation architecture, TUI, rendered GUI, themes, and Workbench authoring The conversation then formalized presentation. CLI, TUI,

rendered GUI, native GUI, headless reports, Workbench panels, AIDE, CI, launcher, setup, client, and server should all project the same command/result/refusal/document/view/action/evidence model. The TUI should be a serious keyboard-first terminal projection, not an ASCII toy. Rendered GUI should be a data-described UI system built from layout engines, semantic controls, widget compositions, style tokens, theme profiles, draw-list renderers, and projection conformance tests.

The user shared images as references for Workbench themes and modules. These were interpreted as visual/mood and layout examples: GNOME-like command studio, macOS-like release forge, Win11 UI/HUD sandbox, pack browser, agent work board, XP/Win7-style boards, worldgen lab, TUI studio, and primitive-only themes. The conclusion was that one cross-platform Workbench app can mix and match module + workspace layout + theme + renderer + platform capability.

2.7 Provider strategy: raylib, SDL2, Lua Later, the chat incorporated raylib, rlsw, rlgl, raygui, raudio, SDL2, and Lua. The core doctrine became: use these aggressively, but only as replaceable providers.

Dominium owns service contracts. Providers satisfy those contracts. Profiles select providers. Third-party types are fenced. Apps stay generic. Workbench edits Dominium documents, not raylib objects.

This prompted a provider structure: `runtime/`
`<service>/providers/<provider>`, `contracts/`
`provider`, `contracts/capability`, `contracts/schema/`
`runtime/<service>`, `release/profiles`, `content/`
`profiles`, and `external/upstream`. The plan explicitly
rejected `runtime/raylib/render`, `apps/client/`
`rendered/raylib`, `contracts/raylib`, top-level
`profiles`, and top-level `labs`.

2.8 Product spine, structure cleanup, and current maintenance path The final part of the conversation dealt with repo status and task queue. The user pasted status reports indicating that `PRESENTATION-CONTRACT-01` completed with warnings, and then chose to generate six maintenance prompts before replanning. `FULL-GATE-LEGACY-TEST-ROUTE-01` was generated. This preservation task followed.

3. Main Topics Discussed

Topic 1 - UI Editor / Tool Editor bootstrap plan

The early UI Editor plan was a structured attempt to fix the launcher UI and build a tool for UI authoring. It covered `DUI/TLV`, native widgets, Win32 preview, deterministic `TLV/JSON`, layout, validation, codegen, `ops.json`, import/export, and launcher/setup generation. The conclusion was that these are useful components but not the final product architecture. They

should become Workbench modules and services.

Topic 2 - Dominion Workbench Platform

Workbench became the central product concept. It is a cross-platform rendered production environment, not a monolithic editor. It hosts workspaces and modules over contracts, commands, documents, patches, services, providers, packs, artifacts, evidence, and tests. It eventually self-hosts by editing its own safe artifacts first, then product UIs, packs, modules, apps, and AIDE work units.

Topic 3 - AIDE/Codex workflow and parallel development AIDE should govern work; Codex should execute bounded patches. The chat designed branch roles, lifecycle states, WorkUnit schemas, blocker taxonomy, dirty worktree policy, dev/main promotion, checkpoint loops, and capability reality ledger. The key doctrine is non-blocking dev and evidence-blocked promotion.

Topic 4 - Unified presentation/projection system CLI, TUI, rendered GUI, OS-native GUI, and headless output should all project the same command/result/document/view/action model. This enables reuse across client, server, launcher, setup, Workbench, tests, CI, and AIDE.

Topic 5 - TUI as first-class projection The TUI should be a deterministic, keyboard-first, low-dependency projection of the operating environment. It should use panels, tables, trees, forms, logs, status bars, progress meters, and command input. It should support server/admin, setup recovery, CI, SSH, accessibility, and agent workflows.

Topic 6 - Rendered GUI, themes, and Workbench UX The rendered GUI must be backend-agnostic. Layouts, controls, widgets, themes, styles, views, and workspaces should be modular and extensible. Themes include first-party primitive-only profiles and OEM+ mimic profiles. The GUI should be task-first, command-backed, evidence-visible, and projection-neutral.

Topic 7 - Provider strategy Use raylib/SDL2/Lua as seed providers. Keep service identity first-party and provider implementation replaceable. Fence third-party types. Add provider manifests, service contracts, forbidden include validators, and conformance tests before implementation wedges.

Topic 8 - Repo structure and maintenance The chat concluded not to restart broad structure cleanup. Active structure is good enough; remaining issues should be targeted maintenance. The immediate generated

maintenance prompt was FULL-GATE-LEGACY-TEST-ROUTE-01.

Topic 9 - Universe Explorer Universe Explorer became a future north-star: a read-only product/Workbench/Client slice for seamless inspection of universe-scale data, reference frames, materialization, streaming, fidelity degradation, and provenance. It should start contract-first and headless before visual implementation or embodiment.

4. What We Were Trying to Achieve

4.1 Explicit Goals

- Fix the launcher UI problem by creating better UI tooling.
- Build tools that can help create setup, launcher, game, and future tools.
- Support cross-platform CLI, TUI, rendered GUI, and optional native GUI.
- Make code reusable for other Domino-based games and different engine projects.
- Preserve all decisions, tasks, prompts, constraints, files, and artifacts in a downloadable package.

- Automate as much as possible with Codex/AIDE without losing governance.
- Develop visually with Workbench as soon as the foundation allows.

4.2 Inferred Goals INFERENCE: The user wants to stop losing context across long chats and future assistants. The user also wants a professional-grade architecture more like a serious engine/platform/OS project than a one-off indie game. The user wants a single system where tools are also tests of the engine/client architecture.

4.3 Goals That Changed Over Time The first goal was a UI Editor. That changed to a Workbench Platform. The first idea of native widget UI tooling changed to rendered cross-platform UI tooling. Broad repo structure cleanup changed to targeted maintenance. Raylib changed from possible engine dependency to seed provider suite.

4.4 Goals Still Unresolved The live repo state must be verified before executing future prompts. Full-gate debt remains. Workbench shell is not implemented. Provider runtime, package runtime, module loader, renderer, native GUI, gameplay, and Universe Explorer remain future work.

5. Decisions Made and Why

- Row 1** ID: DECISION-01; Decision: Supersede UI Editor/Tool Editor as final products.; Status: Accepted; Why it mattered: Avoids one-off tooling architecture.; Confidence: 5; Label: FACT
- Row 2** ID: DECISION-02; Decision: Make Workbench the production environment.; Status: Accepted; Why it mattered: Unifies tool, UI, agent, pack, module, app, release workflows.; Confidence: 5; Label: FACT
- Row 3** ID: DECISION-03; Decision: Workbench is not authority.; Status: Accepted; Why it mattered: Prevents GUI/tool drift from contracts/services.; Confidence: 5; Label: FACT
- Row 4** ID: DECISION-04; Decision: AIDE governs, Codex patches.; Status: Accepted; Why it mattered: Enables safe automation.; Confidence: 5; Label: FACT
- Row 5** ID: DECISION-05; Decision: Development non-blocking, promotion evidence-blocked.; Status: Accepted; Why it mattered: Supports parallel dev without corrupting main.; Confidence: 5; Label: FACT
- Row 6** ID: DECISION-06; Decision: CLI/TUI/ rendered/native/headless are projections.; Status:

Accepted; Why it mattered: Enables reuse across apps.; Confidence: 5; Label: FACT

Row 7 ID: DECISION-07; Decision: Use raylib/SDL/ Lua as providers, not architecture.; Status: Accepted conceptually; Why it mattered: Gives speed without lock-in.; Confidence: 4; Label: FACT/INFERENCE

Row 8 ID: DECISION-08; Decision: Stop broad structure cleanup.; Status: Accepted; Why it mattered: Avoids churn after structure is good enough.; Confidence: 4; Label: FACT

Row 9 ID: DECISION-09; Decision: Run targeted maintenance before replanning.; Status: Accepted; Why it mattered: User explicitly selected six maintenance prompts.; Confidence: 5; Label: FACT

Row 10 ID: DECISION-10; Decision: Build Workbench read-only first.; Status: Accepted conceptually; Why it mattered: Safer progressive self-hosting.; Confidence: 4; Label: FACT/INFERENCE

Explanation of major decisions The Workbench decision was the largest architectural correction. The old UI Editor plan could have solved the immediate launcher problem, but it would not have proven the client/

runtime stack. Workbench makes the tools use the same systems they help produce.

The AIDE decision was driven by the scale of the project. Once many Codex tasks run in parallel, stale queues, dirty worktrees, partial branches, and unclear blockers become first-order risks. AIDE gives those states names and evidence requirements.

The projection decision was driven by portability and reuse. If CLI, TUI, rendered GUI, and native GUI are implemented separately, they will drift. If they are projections of the same command/result/view system, all products share semantics.

The provider decision was driven by speed and long-term independence. Raylib/SDL/Lua can accelerate development, but Dominium must own the contracts.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

ID	Option	Status	Reason	Reconsider conditions
REJECTED-01	Windows-only UI Editor as final tool.	Superseded	Too narrow and not client/runtime proof.	As a temporary bootstrap or Workbench module only.
REJECTED-02	Monolithic Tool Editor.	Superseded	Would become separate architecture.	Never as final architecture.
REJECTED-03	Workbench as central authority.	Rejected	Contracts/services/proof must remain authority.	Not expected.
REJECTED-04	Broad root cleanup forever.	Deprioritized	Structure good enough; targeted maintenance now.	If validators block narrow slices.
REJECTED-05	Raylib-shaped app/runtime directories.	Rejected	Vendor-shaped architecture creates lock-in.	Only as temporary proof folders.
REJECTED-06	Pure C99 / pure C++11	Rejected conceptually	C17+C++17 with C ABI	Verify if live repo requires

7. Important Reasoning, Rationale, and Tradeoffs

The main tradeoff was speed versus structural integrity. The user wants visible progress, but not at the cost of another rewrite spiral. The final doctrine uses providers and profiles to get speed while preserving replaceability.

A second tradeoff was governance versus momentum. A project with this many moving parts can stall under too much governance, but without enough, it risks false claims and hidden coupling. The compromise is fast strict and targeted validators on dev, and stronger checkpoint/full-gate proof for main.

A third tradeoff was UI richness versus semantic safety. Workbench should eventually be visual and powerful, but it must first be read-only and command-backed. Visual editing comes only after document/patch/presentation/projection law.

A fourth tradeoff was native GUI versus rendered portability. Native GUIs can exist as optional projections and can use Visual Studio/Xcode tooling, but the core cross-platform Workbench must be rendered through Dominium's own document/view/control/theme/render system.

8. Plans, Future Work, and Next Steps

Immediate user-selected maintenance path

1. `FULL-GATE-LEGACY-TEST-ROUTE-01` - generated immediately before this preservation request. It routes/retired full-gate tests that still expect old roots.
2. `PACK-INTERNAL-LAYOUT-CANON-01` - decide/canonicalize pack-internal layout.
3. `RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01` - classify residual runtime/engine taxonomy.
4. `PUBLIC-HEADER-ABI-PROMOTION-01` - reduce ABI/public header promotion warning debt.
5. `STORAGE-PACKAGE-PROVIDER-SPLIT-01` - clarify storage/package provider boundaries.
6. `POINTER-WIDTH-SERIALIZATION-AUDIT-01` - audit `size_t`, pointers, pointer-width-dependent serialization risks.

Mainline after maintenance/replanning

1. `PROJECTION-CONFORMANCE-01`.
2. `ACCESSIBILITY-CONTRACT-01`.
3. `TEXT-LOCALIZATION-CONTRACT-01`.
4. `WORKBENCH-SHELL-READONLY-01`.
5. `UNIVERSE-EXPLORER-CONTRACT-01`.

6. UNIVERSE-EXPLORER-HEADLESS-01.
7. PROVIDER-MANIFEST-WEDGE-01.
8. SERVICE-CONTRACT-WEDGE-01.
9. THIRD-PARTY-FENCE-01.
10. RAYLIB-PROVIDER-WEDGE-01.
11. UNIVERSE-EXPLORER-VISUAL-01.

Recommended next action Verify live repo state, then review/run FULL-GATE-LEGACY-TEST-ROUTE-01, then generate PACK-INTERNAL-LAYOUT-CANON-01 if the queue/status supports the maintenance lane.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit constraints and preferences

- Use source-grounded, audit-ready answers.
- Preserve uncertainty labels.
- Do not silently infer.
- Do not turn brainstorming into decisions.
- Do not treat assistant suggestions as accepted decisions unless the user accepted them.

- Create actual downloadable files if tools are available.
- Include a human-readable report in-chat even if files are generated.
- Do not restart settled doctrine unnecessarily.
- Use exact, bounded Codex prompts.

9.2 Inferred constraints and preferences

- The user prefers professional-grade modular architecture over quick hacks.
- The user values long-term reuse for future games and engines.
- The user wants a system that can support human and agentic development together.

9.3 Uncertain preferences

- Exact balance between maintenance and product-spine work after the six maintenance tasks.
- Exact timing of provider implementation relative to Universe Explorer.
- Exact format of future master Project Spec Book.

10. Files, Artifacts, Outputs, and Prompts

Artifacts include uploaded screenshots/mockups, preservation instruction text, earlier preservation packages, generated prompt plans, generated Codex prompts, and this companion report package. The most important generated prompt immediately before preservation was `FULL-GATE-LEGACY-TEST-ROUTE-01`.

The screenshots should be preserved as visual references, not treated as formal specs. The prior preservation package should be preserved because it contains sections and structured registers already generated. This companion package bundles those files and adds a more direct accompanying report.

11. Open Questions and Unresolved Issues

ID	Question	Why it matters	Resolution path
QUESTION-01	What is the live repo state after this chat?	Determines next prompt.	Inspect repo, <code>.aide/queue/current.toml</code> , audits.
QUESTION-02	Has FULL-GATE-LEGACY-TEST-ROUTE-01 been executed?	Determines next maintenance prompt.	Check <code>commits/audits</code> .
QUESTION-03	Is C17+C++17 fully implemented in build files?	Affects portability and provider plan.	Verify CMake/toolchain docs.
QUESTION-04	When should raylib provider work start?	Affects speed vs governance.	After provider manifest/service/fence tasks.
QUESTION-05	How soon should Universe Explorer enter implementation?	Affects product identity.	After projection/workbench read-only and headless contract proof.

12. Risks and Failure Modes

Future chats could easily misunderstand this conversation by treating Workbench as a GUI framework, treating raylib as the engine, resuming broad structure cleanup, skipping projection conformance, or assuming live repo status from stale pasted reports. The mitigation is to verify repo state first, preserve the contract/service/projection/provider distinctions, and follow the task queue.

13. Contribution to Future Spec Book

This chat should feed the following spec book chapters:

- Workbench Platform.
- AIDE/Codex development process.
- Presentation/projection architecture.
- Provider strategy and third-party fencing.
- TUI/rendered GUI/theme design.
- Progressive self-hosting.
- Product-spine task sequencing.
- Universe Explorer north-star.
- Maintenance/full-gate debt strategy.

Formal requirements candidates include: Workbench is not authority; third-party providers are fenced; CLI/TUI/rendered/native/headless are projections; development non-blocking/promotion evidence-blocked; every visual edit becomes a document patch; provider capabilities require evidence.

14. What I Should Remember

- The old UI Editor/Tool Editor plan is superseded.
- Workbench is a production environment over contracts/services/proof, not authority.
- AIDE governs work; Codex executes bounded patches.
- CLI, TUI, rendered GUI, OS-native GUI, and headless are projections of one semantic spine.
- Raylib/SDL/Lua are providers, not architecture.
- Broad structure cleanup should stop; targeted maintenance remains.
- The next immediate maintenance prompt after `FULL-GATE-LEGACY-TEST-ROUTE-01` is likely `PACK-INTERNAL-LAYOUT-CANON-01`, pending live repo verification.

15. Best Follow-Up Questions

- "Generate PACK-INTERNAL-LAYOUT-CANON-01."
- "What did we decide about Workbench self-hosting?"
- "List all pending generated prompts and their order."
- "What must be verified in the live repo before continuing?"
- "Turn the provider/raylib doctrine into formal requirements."
- "Create a Project Spec Book chapter outline from this conversation."
- "Which maintenance tasks can run in parallel?"
- "What should the first read-only Workbench shell actually show?"
- "What is the difference between provider profiles and content profiles?"
- "What are the top risks if we skip projection conformance?"

16. Compact Human Summary

This conversation records a major architectural evolution for Dominion. It began with a concrete launcher UI problem: the current Windows launcher looked bad and flickered. The first proposed solution was a Windows-first UI Editor followed by a Tool Editor, using the existing DUI/TLV system, Win32 previews, layout engines, codegen, CLI scripting, import/export, and launcher/setup layout generation. That path was explored deeply and converted into many prompts, but it was later judged wrong as a final product.

The user reframed the goal. Instead of building a one-off editor, Dominion needs a cross-platform rendered Workbench that uses the same systems as the client, server, launcher, setup, CLI, TUI, and headless tools. Workbench should not be authority; it should be the rich production environment over contracts, commands, services, documents, patches, providers, packs, modules, diagnostics, evidence, and tests. It should eventually help the user and agents build the whole project: code scaffolds, packs, modules, themes, UI/HUDs, tests, releases, AIDE/Codex work units, and future Domino-based games.

AIDE became the governance system. Codex is a bounded patch executor. AIDE is the scheduler, context compiler, blocker classifier, evidence ledger, repair/

resume generator, checkpoint coordinator, and promotion gatekeeper. The central doctrine is: development is non-blocking, promotion is evidence-blocked. Dev and task branches can hold classified partials and warnings; main requires evidence-backed checkpoints.

The UI architecture became projection-based. CLI, TUI/text, rendered GUI, OS-native GUI, and headless outputs should all present the same command/result/refusal/document/view/action/diagnostic/evidence model. The TUI is first-class. Rendered GUI is a modular system of layouts, semantic controls, widgets, themes, style tokens, draw lists, and renderer providers. OS-native GUI is optional projection, not semantic authority.

The provider strategy became service-first. Raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua can be used aggressively, but only as replaceable providers. Dominium owns service contracts, provider law, commands, saves, replays, packs, UI documents, and asset identity. Third-party types must not leak into stable law or public SDK surfaces.

The current end state is targeted maintenance plus product-spine continuation. The user reported PRESENTATION-CONTRACT-01 complete with warnings, then selected six maintenance tasks before replanning: full-gate legacy test routing, pack internal layout canon,

runtime/engine residual taxonomy, public header ABI promotion, storage/package provider split, and pointer-width serialization audit. `FULL-GATE-LEGACY-TEST-ROUTE-01` was generated immediately before this preservation task.

The best next action is to verify live repo state, then review or run `FULL-GATE-LEGACY-TEST-ROUTE-01`, then generate `PACK-INTERNAL-LAYOUT-CANON-01` if status supports it.

33.3 Dominion Full Conversation Companion Report Decision Task Risk Registers

Source: `docs/archive/conversations/dominium_full_conversation/companion_report/dominium_full_conversation_companion_report__02_decision_task_risk_registers.md`

Report type: `companion_report`

Size: 7172 bytes

33.3.1 Structured Registers - Dominion Full Conversation Companion

Workstream Register

- Row 1** ID: WORKSTREAM-01; Name: Workbench Platform; Objective: Build a modular production environment over Dominion contracts/services.; Current state: Planned; validation/static/projection slices discussed.; Desired end state: Read-only then authoring Workbench that builds artifacts and products.; Status: Active future; Priority: P0; Confidence: 4; Label: FACT/INFERENCE
- Row 2** ID: WORKSTREAM-02; Name: AIDE/Codex Process; Objective: Safely automate development.; Current state: Workflow prompts generated; some reported done.; Desired end state: Task branches, dev integration, checkpoint main promotion.; Status: Active; Priority: P0; Confidence: 4; Label: FACT
- Row 3** ID: WORKSTREAM-03; Name: Presentation Architecture; Objective: Unify CLI/TUI/rendered/native/headless.; Current state: PRESENTATION-CONTRACT-01 reported complete.; Desired end state: Projection conformance and read-only shell.; Status: Active; Priority: P0; Confidence: 4; Label: FACT stated in chat

- Row 4** ID: WORKSTREAM-04; Name: Provider Strategy; Objective: Use third-party libraries as replaceable providers.; Current state: Doctrine established.; Desired end state: Provider manifests/fences/conformance and raylib wedge.; Status: Planned; Priority: P1; Confidence: 4; Label: FACT/INFERENCE
- Row 5** ID: WORKSTREAM-05; Name: Maintenance/ Full-Gate Debt; Objective: Resolve targeted structural/test debt.; Current state: Six maintenance tasks selected; first prompt generated. ; Desired end state: Full-gate debt reduced without broad cleanup.; Status: Active; Priority: P0; Confidence: 4; Label: FACT
- Row 6** ID: WORKSTREAM-06; Name: Universe Explorer; Objective: Prove read-only seamless universe inspection.; Current state: Conceptual north-star.; Desired end state: Headless proof then visual explorer.; Status: Future; Priority: P1; Confidence: 3; Label: INFERENCE
- Row 7** ID: WORKSTREAM-07; Name: UI/TUI/ Theme System; Objective: Modular layouts, controls, themes, TUI/rendered profiles.; Current state: Conceptual architecture settled.; Desired end state: Theme Lab, Interface Studio, Renderer Sandbox.; Status: Future; Priority: P1;

Confidence: 4; Label: FACT/INFERENCE

Row 8 ID: WORKSTREAM-08; Name: Product Apps;
Objective: Reuse same modules/packs/UI across
client/server/launcher/setup.; Current state:
Barebones/product-spine discussion.; Desired end
state: Apps as thin compositions over shared
services.; Status: Active/future; Priority: P1;
Confidence: 4; Label: FACT/INFERENCE

Decision Register

Row 1 ID: DECISION-01; Decision: Supersede old UI
Editor/Tool Editor as final products.; Status:
Accepted; Evidence / basis: User explicitly said
abandon/recycle them.; Rationale: Avoid one-off
Windows-first editor architecture.; Implications:
Recycle ideas into Workbench modules.; Related
workstream: WORKSTREAM-01; Confidence: 5;
Label: FACT

Row 2 ID: DECISION-02; Decision: Workbench is not
authority.; Status: Accepted; Evidence / basis:
Repeated in chat.; Rationale: Prevent GUI/tools
from bypassing contracts.; Implications:
Workbench uses commands/services/documents/
patches.; Related workstream: WORKSTREAM-
01; Confidence: 5; Label: FACT

Row 3 ID: DECISION-03; Decision: AIDE governs,

Codex patches.; Status: Accepted; Evidence / basis: Repeated in plans.; Rationale: Enables parallel automation safely.; Implications: Create WorkUnits, checkpoints, evidence.; Related workstream: WORKSTREAM-02; Confidence: 5; Label: FACT

Row 4 ID: DECISION-04; Decision: Development non-blocking, promotion evidence-blocked.; Status: Accepted; Evidence / basis: User handoff/plan.; Rationale: Speeds dev while protecting main.; Implications: dev/task branches vs main/checkpoints.; Related workstream: WORKSTREAM-02; Confidence: 5; Label: FACT

Row 5 ID: DECISION-05; Decision: Presentation modes are projections.; Status: Accepted; Evidence / basis: Repeated doctrine.; Rationale: Prevent separate CLI/TUI/GUI implementations.; Implications: Projection contract/conformance required.; Related workstream: WORKSTREAM-03; Confidence: 5; Label: FACT

Row 6 ID: DECISION-06; Decision: Use raylib/SDL/Lua as providers.; Status: Accepted conceptually; Evidence / basis: User asked and accepted doctrine.; Rationale: Fast progress without lock-in.; Implications: Provider manifests/fencing needed.; Related workstream: WORKSTREAM-04;

Confidence: 4; Label: FACT/INFERENCE

Row 7 ID: DECISION-07; Decision: Stop broad structure cleanup.; Status: Accepted; Evidence / basis: User/current plans.; Rationale: Avoid churn after structure is clean enough.; Implications: Run targeted maintenance tasks.; Related workstream: WORKSTREAM-05; Confidence: 4; Label: FACT

Row 8 ID: DECISION-08; Decision: Workbench starts read-only.; Status: Accepted conceptually; Evidence / basis: Assistant recommendation accepted in later plan.; Rationale: Avoid unsafe self-hosting.; Implications: Workbench shell read-only before authoring.; Related workstream: WORKSTREAM-01; Confidence: 4; Label: INFERENCE

Task Register

Row 1 ID: TASK-01; Task: Verify live repo state.; Priority: P0; Urgency: U0; Owner: Human/Codex; Dependencies: None; Inputs needed: repo checkout; Expected output: current queue/audits/status; Next step: inspect before next task; Related workstream: all; Label: UNCERTAIN

Row 2 ID: TASK-02; Task: Run/review FULL-GATE-LEGACY-TEST-ROUTE-01.; Priority: P0; Urgency: U0; Owner: Codex; Dependencies:

Presentation complete; Inputs needed: generated prompt; Expected output: stale full-gate paths routed; Next step: generate next maintenance prompt; Related workstream: WORKSTREAM-05; Label: FACT

Row 3 ID: TASK-03; Task: Generate PACK-INTERNAL-LAYOUT-CANON-01.; Priority: P0; Urgency: U1; Owner: Assistant/Codex; Dependencies: TASK-02 or live approval; Inputs needed: status; Expected output: prompt/report; Next step: execute if selected; Related workstream: WORKSTREAM-05; Label: FACT

Row 4 ID: TASK-04; Task: PROJECTION-CONFORMANCE-01.; Priority: P0; Urgency: U1; Owner: Codex; Dependencies: presentation contract; Inputs needed: schemas/fixtures; Expected output: proof of projection parity; Next step: continue UI path; Related workstream: WORKSTREAM-03; Label: FACT

Row 5 ID: TASK-05; Task: WORKBENCH-SHELL-READONLY-01.; Priority: P1; Urgency: U2; Owner: Codex; Dependencies: projection conformance; Inputs needed: runtime/app context; Expected output: read-only Workbench shell; Next step: validate usefulness; Related workstream: WORKSTREAM-01; Label: INFERENCE

- Row 6** ID: TASK-06; Task: PROVIDER-MANIFEST-WEDGE-01.; Priority: P1; Urgency: U2; Owner: Codex; Dependencies: AIDE/presentation path; Inputs needed: provider doctrine; Expected output: provider schemas/manifests; Next step: service/fence wedge; Related workstream: WORKSTREAM-04; Label: INFERENCE
- Row 7** ID: TASK-07; Task: UNIVERSE-EXPLORER-CONTRACT-01.; Priority: P1; Urgency: U2; Owner: Codex; Dependencies: Workbench/read-only/projection; Inputs needed: explorer doctrine; Expected output: contract-only explorer spec; Next step: headless proof; Related workstream: WORKSTREAM-06; Label: INFERENCE

Risk Register

- Row 1** ID: RISK-01; Risk / failure mode: Treat stale repo status as current.; Consequence: Wrong next prompt.; Likelihood: Medium; Severity: High; Mitigation: Verify live repo.; Related workstream: all; Label: FACT/INFERENCE
- Row 2** ID: RISK-02; Risk / failure mode: Build Workbench UI too early.; Consequence: Parallel UI system.; Likelihood: Medium; Severity: High; Mitigation: Complete projection conformance first. ; Related workstream: WORKSTREAM-01; Label:

INFERENCE

Row 3 ID: RISK-03; Risk / failure mode: Raylib types leak into contracts/engine/game.; Consequence: Provider lock-in.; Likelihood: Medium; Severity: High; Mitigation: Forbidden include/type validators.; Related workstream: WORKSTREAM-04; Label: INFERENCE

Row 4 ID: RISK-04; Risk / failure mode: Full-gate debt hidden as pass.; Consequence: False readiness.; Likelihood: Medium; Severity: High; Mitigation: Keep warning/T4 status explicit.; Related workstream: WORKSTREAM-05; Label: FACT/INFERENCE

Row 5 ID: RISK-05; Risk / failure mode: Assistant suggestions treated as decisions.; Consequence: Spec corruption.; Likelihood: Medium; Severity: Medium; Mitigation: Use labels and decision evidence.; Related workstream: all; Label: FACT

Row 6 ID: RISK-06; Risk / failure mode: Screenshots treated as specs.; Consequence: Overfitted UI.; Likelihood: Medium; Severity: Medium; Mitigation: Treat as references/moodboards only.; Related workstream: WORKSTREAM-07; Label: INFERENCE

Verification Queue

- Row 1** ID: VERIFY-01; Item requiring verification:
Live `.aide/queue/current.toml`.; Why
verification is needed: Determines next prompt.;
Suggested source/type: repo file; Priority: P0;
Related workstream: all; Label: UNCERTAIN
- Row 2** ID: VERIFY-02; Item requiring verification:
Execution status of maintenance prompts.; Why
verification is needed: Avoid duplicate work.;
Suggested source/type: git log/audits; Priority:
P0; Related workstream: WORKSTREAM-05;
Label: UNCERTAIN
- Row 3** ID: VERIFY-03; Item requiring verification:
C17/C++17 baseline.; Why verification is needed:
Affects provider/build policy.; Suggested source/
type: CMake/docs; Priority: P1; Related
workstream: WORKSTREAM-04; Label:
UNCERTAIN
- Row 4** ID: VERIFY-04; Item requiring verification:
Full CTest debt status.; Why verification is needed:
Affects release readiness.; Suggested source/type:
test reports; Priority: P1; Related workstream:
WORKSTREAM-05; Label: UNCERTAIN
- Row 5** ID: VERIFY-05; Item requiring verification:
Existing prior preservation files completeness.;

Why verification is needed: Affects aggregation.;
Suggested source/type: file package; Priority: P2;
Related workstream: archival; Label:
UNCERTAIN

33.4 Dominium Full Conversation Companion Report Current Plan And Prompt Queue

Source: docs/archive/conversations/dominium_full_conversation/companion_report/dominium_full_conversation_companion_report__03_current_plan_and_prompt_queue.md

Report type: companion_report

Size: 1597 bytes

33.4.1 Current Plan and Prompt Queue - Dominium Companion Report

Current immediate state from this chat

The user reported PRESENTATION-CONTRACT-01 as PASS_WITH_WARNINGS and then selected six maintenance tasks to generate before replanning. The assistant generated FULL-GATE-LEGACY-TEST-ROUTE-01 immediately before the preservation request.

Immediate maintenance sequence

1. FULL-GATE-LEGACY-TEST-ROUTE-01 - generated.
Route/retire full-gate tests still expecting retired roots.
2. PACK-INTERNAL-LAYOUT-CANON-01 - decide and enforce pack-internal layout law.
3. RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01 - classify residual runtime/engine taxonomy warnings.
4. PUBLIC-HEADER-ABI-PROMOTION-01 - burn down public header ABI promotion debt.
5. STORAGE-PACKAGE-PROVIDER-SPLIT-01 - clarify storage/package/provider boundaries.
6. POINTER-WIDTH-SERIALIZATION-AUDIT-01 - audit pointer-width-dependent persistence/serialization risks.

Mainline after maintenance/replanning

1. PROJECTION-CONFORMANCE-01
2. ACCESSIBILITY-CONTRACT-01
3. TEXT-LOCALIZATION-CONTRACT-01
4. WORKBENCH-SHELL-READONLY-01
5. UNIVERSE-EXPLORER-CONTRACT-01

6. UNIVERSE-EXPLORER-HEADLESS-01
7. PROVIDER-MANIFEST-WEDGE-01
8. SERVICE-CONTRACT-WEDGE-01
9. THIRD-PARTY-FENCE-01
10. RAYLIB-PROVIDER-WEDGE-01
11. UNIVERSE-EXPLORER-VISUAL-01

Execution rule

Verify live repo state before each prompt. Do not assume the pasted status remains current. Do not run broad Workbench, renderer, gameplay, package runtime, provider runtime, or module loader work until gates authorize it.

33.5 Dominium Full Conversation Companion Report Specbook Bridge

Source: docs/archive/conversations/dominium_full_conversation/companion_report/dominium_full_conversation_companion_report__04_specbook_bridge.md

Report type: companion_report

Size: 1615 bytes

33.5.1 Spec Book Bridge - Dominion Companion Report

Chapters this chat should inform

1. Domino/Dominium architecture overview.
2. Workbench Platform.
3. AIDE/Codex development process.
4. Presentation/projection architecture.
5. TUI and rendered GUI doctrine.
6. Theme/style/control/widget system.
7. Provider strategy and third-party fencing.
8. Product-spine task sequence.
9. Targeted maintenance/full-gate debt.
10. Universe Explorer north-star.
11. Progressive self-hosting.

Formal requirements candidates

- Workbench is not authority.
- CLI/TUI/rendered/native/headless are projections of the same semantic spine.

- Every user-visible action maps to a command or documented view/action binding.
- All edits go through document/patch/transaction/evidence flow.
- Third-party providers must not leak stable types into contracts/engine/game/content/save/replay/public SDK.
- Provider profiles select providers; apps remain generic.
- Development is non-blocking; promotion is evidence-blocked.
- Capability status must distinguish planned, specified, fixture-only, stubbed, implemented, tested, exposed, release-supported.

Background context candidates

- The original UI Editor/Tool Editor exploration.
- Minecraft-style launcher/setup logical layout test.
- Screenshot/mockup themes and UI inspiration.
- Early open-world/civilization/domain ambitions.

Items needing verification before spec merge

- Live repo task queue and audits.
- C17/C++17 build baseline.
- Current full-gate/CTest status.
- Actual provider/contract directories in repo.
- Whether each generated prompt has been executed.

33.6 Dominion Full Conversation Companion Report Verification Checklist

Source: docs/archive/conversations/dominium_full_conversation/companion_report/dominium_full_conversation_companion_report__05_verification_checklist.md

Report type: companion_report

Size: 1661 bytes

33.6.1 Verification Checklist and Audit - Dominion Companion Report

Verification queue

1. Check `.aide/queue/current.toml` before generating or executing the next prompt.

2. Check whether `FULL-GATE-LEGACY-TEST-ROUTE-01` has already been run.
3. Check whether the current repo still reports `PRESENTATION-CONTRACT-01` as latest completed mainline task.
4. Check whether the six maintenance tasks remain the selected next sequence.
5. Verify `C17/C++17` in live build files.
6. Verify current full `CTest/T4` debt status.
7. Verify whether provider `manifest/service/fence` tasks already exist in repo.
8. Verify whether the previous preservation package is complete and not superseded.

Self-audit

Issue	Severity	Correction needed	Correction applied?	Residual risk
No raw complete transcript export	High	Mark coverage as partial-to-broad	Yes	Some early details may be missing
Live repo facts may be stale	High	Add verification queue	Yes	Must verify before action
Assistant suggestions could be mistaken for decisions	Medium	Label decisions and inferences	Yes	Some acceptance inferred from user agreement
Screenshots could be overinterpreted	Medium	Treat as visual references	Yes	Future visual spec should review them
Prior generated files may be superseded	Medium	Bundle but mark as supporting material	Yes	Future package may be needed

Reliability

- Completeness: 4/5
- 1709

- Reliability: 4/5 for chat doctrine, 3/5 for live repo state
- Human-readability: 4/5
- Aggregation-readiness: 4/5

Manual review is recommended before merging into a master Project Spec Book.

33.7 Dominium Full Conversation Companion Report Artifact Index

Source: docs/archive/conversations/dominium_full_conversation/companion_report/dominium_full_conversation_companion_report__06_artifact_index.md

Report type: companion_report

Size: 919 bytes

33.7.1 Artifact Index - Dominium Companion Report

Included report files

This companion package includes new report files plus previously generated preservation files.

Included prior packages

The ZIP includes, if present:

- `dominium_workbench_aide_architecture_preservation__handoff_package.zip`
- `Dominium_Workbench_AIDE_Presentation_Provider_Strategy__handoff_package.zip`
- individual prior preservation `.md` and `.yaml` files

Included uploaded/reference artifacts

The ZIP includes, if present:

- `Pasted text.txt` - uploaded preservation instructions.
- Screenshot/mockup images from May 18, 2026.
- Additional uploaded image-like files named `file_000...` where present.

Important generated prompt artifact

The most recent major prompt generated in the chat was `FULL-GATE-LEGACY-TEST-ROUTE-01`, a targeted maintenance task for routing stale full-gate tests that still expect retired roots.

33.8 Dominium Full Conversation Companion Report Reader Brief

Source: docs/archive/conversations/dominium_full_conversation/companion_report/dominium_full_conversation_companion_report__07_reader_brief.md

Report type: companion_report

Size: 1571 bytes

33.8.1 Reader Brief - Dominium Companion Report

Top things to know

1. The old UI Editor/Tool Editor plan was superseded.
2. Dominium Workbench is now the central production-environment concept.
3. Workbench is not authority; it operates over contracts, commands, services, documents, patches, providers, packs, modules, diagnostics, evidence, and tests.
4. AIDE governs development; Codex executes bounded patches.

5. Development is non-blocking; promotion is evidence-blocked.
6. CLI/TUI/rendered/native/headless are projections of one semantic spine.
7. TUI is first-class.
8. Rendered GUI must be modular and backend-agnostic.
9. OS-native GUI is optional projection, not core semantic law.
10. Raylib/SDL/Lua are seed providers, not architecture.
11. Apps stay generic; profiles select providers.
12. Broad structure cleanup should stop.
13. Targeted maintenance remains.
14. PRESENTATION-CONTRACT-01 was reported complete with warnings.
15. FULL-GATE-LEGACY-TEST-ROUTE-01 was generated next.
16. The next likely prompt is PACK-INTERNAL-LAYOUT-CANON-01, pending live status.
17. Universe Explorer is a future read-only/headless north-star before visual/gameplay work.

18. Workbench should start read-only, then progressively self-host.
19. Live repo facts must be verified before execution.
20. This package is a preservation and aggregation aid, not a live repo authority.

Best next action

Verify current repo state and whether FULL-GATE-LEGACY-TEST-ROUTE-01 has run. If it has not, run/review it. If it has, generate PACK-INTERNAL-LAYOUT-CANON-01.

33.9 Dominion Full Conversation Companion Report Future Chat Bootstrap Prompt

Source: docs/archive/conversations/dominium_full_conversation/companion_report/dominium_full_conversation_companion_report__08_future_chat_bootstrap_prompt.md

Report type: companion_report

Size: 1439 bytes

33.9.1 Future Chat Bootstrap Prompt - Dominium Companion Report

I am continuing from a long Dominium/Domino/
Workbench/AIDE planning chat. Treat the following as
starting context, not live repo truth.

Preserve FACT / INFERENCE / UNCERTAIN /
PROJECT-CONTEXT labels. Do not assume access to
the old chat. Do not re-ask answered questions. Verify
stale facts before relying on them. Flag contradictions.
Use the task register and open questions to determine
next actions.

Core carry-forward:

- Domino = reusable deterministic substrate.
- Dominium = game/product family.
- Workbench = production, validation, editing,
inspection, packaging, and evidence environment.
- AIDE = repo/control-plane harness.
- Codex = bounded patch executor.
- Contracts = law.
- Tests/replay/evidence = proof.
- Workbench is not authority.

- CLI/TUI/rendered/native/headless are projections of one command/result/refusal/document/view/action spine.
- Raylib/SDL/Lua are providers, not architecture.
- Broad structure cleanup should stop; targeted maintenance remains.
- The latest visible user-selected next sequence starts with FULL-GATE-LEGACY-TEST-ROUTE-01, then PACK-INTERNAL-LAYOUT-CANON-01, then other maintenance tasks, before returning to projection/Workbench/Universe Explorer tasks.

When you respond, state:

- context loaded;
- active priorities;
- key constraints;
- open questions;
- contradictions or uncertainties;
- recommended next action.

33.10 Dominion Full Conversation Companion Report Full Combined Report

Source: docs/archive/conversations/dominium_
full_conversation/companion_report/dominium_
full_conversation_companion_report__FULL_
COMBINED_REPORT.md

Report type: companion_report

Size: 51850 bytes

33.10.1 FILE:
dominiumfullconversationcompanionreport

**33.10.2 Manifest - Dominion Workbench,
AIDE, Presentation Architecture,
Provider Strategy, Product-Spine
Planning, and Preservation
Companion**

Date anchor: 2026-05-27 Australia/Melbourne

This package is an accompanying human-readable companion report for the long Dominion/Domino/Workbench/AIDE planning conversation. It supplements the earlier preservation package already created in this chat and bundles the earlier reports, screenshots/mockups, uploaded preservation instruction, and this new companion report into one archive.

Files created in this companion package

Row 1 File: dominium_full_conversation_companion_report__00_manifest.md; Purpose: Manifest for the companion package.

Row 2 File: dominium_full_conversation_companion_report__01_complete_human_report.md; Purpose: Main human-readable detailed report of the whole conversation.

Row 3 File: dominium_full_conversation_companion_report__02_decision_task_risk_registers.md; Purpose: Condensed structured registers for decisions, workstreams, tasks, risks, and verification.

Row 4 File: dominium_full_conversation_companion_report__03_current_plan_and_prompt_queue.md; Purpose: Current practical execution plan and prompt queue as of the end of the visible conversation.

Row 5 File: dominium_full_conversation_companion_report__04_specbook_bridge.md; Purpose: How this chat should feed into a future master Project Spec Book.

Row 6 File: dominium_full_conversation_companion_report__05_verification_

checklist.md; Purpose: Verification queue and stale fact checklist.

Row 7 File: dominium_full_conversation_companion_report__06_artifact_index.md;
Purpose: Index of previous generated files and visual/source artifacts bundled in the ZIP.

Row 8 File: dominium_full_conversation_companion_report__07_reader_brief.md;
Purpose: Shorter reader brief for quick review.

Row 9 File: dominium_full_conversation_companion_report__FULL_COMBINED_REPORT.md;
Purpose: All companion report files combined into one Markdown file.

Row 10 File: dominium_full_conversation_companion_report__handoff_package.zip;
Purpose: ZIP containing this companion package, previous preservation package files, screenshots/mockups, and uploaded instruction text.

Included supporting material in the ZIP

- Previous preservation files named dominium_workbench_aide_architecture_preservation__*.
- Earlier package files named Dominium_Workbench_

`AIDE_Presentation_Provider_Strategy_*` if present.

- Uploaded screenshot/mockup images from May 18, 2026, if present in `/mnt/data`.
- The uploaded `Pasted text.txt` preservation instruction.

Caveats

- This is a reconstruction from visible conversation context plus generated files available in the sandbox. It is not a forensic raw transcript export.
- Live repository status must still be verified before executing the next Codex prompt.
- External software facts about raylib, SDL2, Lua, CMake, language baselines, and OS support should be verified before becoming formal requirements.

- 33.10.3 FILE:
dominiumfullconversationcompanionreport
- 33.10.4 Complete Human-Readable Companion Report - Dominion Workbench, AIDE, Presentation Architecture, Provider Strategy, Product-Spine Planning, and Preservation Companion

0. Coverage and Reliability Assessment

- Row 1 Field: Chat label; Assessment: Dominion Workbench, AIDE, Presentation Architecture, Provider Strategy, Product-Spine Planning, and Preservation Companion
- Row 2 Field: Date anchor; Assessment: 2026-05-27 Australia/Melbourne
- Row 3 Field: Source scope; Assessment: This chat only unless explicitly labelled PROJECT-CONTEXT.
- Row 4 Field: Apparent access; Assessment: Partial-to-broad visible context. The current chat includes a very large visible transcript, user-provided handoffs, generated prompts, uploaded screenshots, uploaded preservation instructions, and prior generated report files. It is not guaranteed to include every skipped early turn verbatim.

- Row 5 Field: Previously generated files available?;

Assessment: Yes. Earlier preservation packages and report files are present in `/mnt/data`.

Row 6 Field: Uploaded files/artifacts present?; Assessment: Yes. The uploaded preservation instruction, multiple screenshots/mockups, and prior generated packages are visible in the sandbox.

Row 7 Field: Contains future plans?; Assessment: Yes. The chat contains extensive near-term, medium-term, and long-term roadmaps.

Row 8 Field: Contains decisions?; Assessment: Yes. Several major architectural decisions were accepted or revised.

Row 9 Field: Contains pending tasks?; Assessment: Yes. The latest user-selected maintenance tasks and future queue are preserved.

Row 10 Field: Contains unresolved questions?; Assessment: Yes. Live repo status, execution state of prompts, full-gate debt, and sequencing decisions require verification.

Row 11 Field: Staleness risk; Assessment: High for live repo/current software status; medium for roadmap sequencing; low for settled chat doctrine.

Row 12 Field: Extraction confidence; Assessment: 4/5 for visible chat substance; 3/5 for live repo status.

Row 13 Field: Safe for later aggregation?; Assessment: Yes, with caveats.

Row 14 Field: Main limitations; Assessment: No raw full transcript export, some early turns were summarized/skipped, external repo/software facts may be stale, and screenshots were treated as design references rather than formal specs.

Plain-language limitations This companion report preserves the meaning, decisions, and task plan of the conversation as visible in this chat. It is not a line-by-line transcript. Where the chat discussed the live `julesc013/dominium` repo, those statements should be treated as **UNCERTAIN / UNVERIFIED** until checked against the repository. Where the user pasted status reports from Codex or another chat, those are preserved as **FACT stated in chat**, not independently verified facts unless separately checked.

1. One-Page Orientation

This conversation records a major architectural transition in the Dominium project. It began with an immediate practical concern: the existing Windows launcher UI looked broken, flickered, and needed a better way to design UIs. The initial direction was to create a Windows-first UI Editor and eventually a Tool Editor

that could visually design pixel-perfect UIs and generate TLV-based layouts/code. That early plan was explored in depth: TLV schemas, UI IR, deterministic JSON mirrors, layout engines, Win32/DGFX/null backends, action code generation, import/export, CLI scripting, native widget previews, and launcher/setup redesign tests were all discussed and turned into prompt plans.

Over the course of the chat, the user corrected the direction. The old UI Editor/Tool Editor path was judged useful as an idea but wrong as a final architecture. OS-native GUI design can rely on first-party tools such as Visual Studio and Xcode where appropriate. Dominium's real internal tooling should instead prove and reuse the same runtime, command, rendering, UI, package, and diagnostic systems that the client and other products need. This led to the replacement concept: **Dominium Workbench Platform**.

Workbench is the central product concept of this chat. It is not a monolithic IDE and not an authority over the project. It is a cross-platform rendered, modular, command-driven, agent-aware production environment. It should eventually let the user and agents build the entire project: code scaffolds, packs, modules, UI/HUD documents, themes, renderer tests, release artifacts, diagnostics, evidence packets, AIDE/Codex work units, and product compositions. Its design doctrine is:

Workbench designs artifacts; runtime executes artifacts; contracts constrain artifacts; content packages artifacts; apps consume artifacts; tests prove artifacts; AIDE governs changes; Codex patches code.

The second major outcome was an AIDE development model. The chat established that **development should be non-blocking, promotion should be evidence-blocked**. This means work can accumulate on task branches and dev with classified partials or warnings, but main should only receive checkpointed, evidence-backed promotion. Codex is a bounded patch executor; AIDE is a scheduler, context compiler, evidence ledger, blocker classifier, repair/resume generator, checkpoint coordinator, and promotion gatekeeper. A series of AIDE prompts were generated or planned: workflow law, WorkUnit schema, dev/main policy, checkpoint loop, and capability reality ledger.

The third major outcome was a unified presentation doctrine. CLI, TUI/text, rendered GUI, OS-native GUI, and headless reports must not become separate product implementations. They are projections over the same semantic spine: command/result/refusal/document/view/action/diagnostic/evidence. The TUI is a serious first-class projection. The rendered GUI is a backend-agnostic system of layouts, controls, widgets, style tokens, themes, view documents, draw lists, and provider-backed renderers. OS-native widgets remain

optional projections, not core semantics.

The fourth major outcome was a service-first provider strategy. The chat concluded that raylib, rlgl, rlsw, raygui, raudio, SDL2, Lua, and related libraries can be used aggressively, but only as replaceable providers behind Dominion service contracts. Dominion owns contracts, packets, saves, replays, packs, UI documents, asset identity, provider law, and simulation law. Third-party types must not leak into engine, game, contracts, content, pack schemas, save/replay formats, command/result schemas, or public SDK surfaces.

By the end, the project direction had moved from broad structure cleanup to targeted maintenance and product-spine development. The user reported that PRESENTATION-CONTRACT-01 had completed with PASS_WITH_WARNINGS, then selected six targeted maintenance prompts to generate before replanning: FULL-GATE-LEGACY-TEST-ROUTE-01, PACK-INTERNAL-LAYOUT-CANON-01, RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01, PUBLIC-HEADER-ABI-PROMOTION-01, STORAGE-PACKAGE-PROVIDER-SPLIT-01, and POINTER-WIDTH-SERIALIZATION-AUDIT-01. The last generated prompt in the visible chat was FULL-GATE-LEGACY-TEST-ROUTE-01.

A future assistant must understand that this chat is not simply about UI. It records the architecture for how Dominion should build itself: through contracts,

commands, services, providers, documents, patches, modules, packs, apps, workspaces, diagnostics, evidence, AIDE, Codex, and eventually Workbench.

2. The Story of the Conversation

2.1 The initial problem: launcher UI and the old UI Editor concept The conversation began with the user wanting a product designer/software architect/prompt engineer to help design an internal graphical UI creation tool. The immediate pain point was the Dominion Windows launcher UI: it looked mangled and flickered. The user wanted a tool that could visually design pixel-perfect UIs and broad-strokes functionality for setup, launcher, game, and tools.

Early discussion clarified target operating systems, the current DUI/TLV schema/state system, and the existing backends: Win32 common controls, DGFX, and null. The user wanted true native controls where possible, custom rendering only where necessary, and eventually universal UI definitions across products. We discussed DPI scaling, canonical TLV, deterministic JSON mirrors, codegen, event binding, flicker causes, and bootstrap phases.

A large prompt plan followed: repo scaffolding, canonical UI IR, TLV I/O, capability system, layout engine, splitter/tabs/scroll widgets, action codegen, Phase A UI

Editor, Tool Editor bootstrap, Win32 batching/flicker fixes, tests, docs, and capability tests through launcher/setup redesigns.

2.2 UI Editor gains CLI/import/export and launcher/setup tests The user then asked for the UI Editor to import existing tools and modify their UIs, expose a CLI so Codex could automate designs, and use the UI Editor to create launcher/setup designs similar to Minecraft's logical layout. The key correction was that "Minecraft-style" meant logical layout, not graphical skin: native Win32 widgets only, structured around header/sidebar/tabs/footer for launcher and wizard-style pages for setup. We generated prompts for UI discovery/import/export, CLI modes, `ops.json` scripts, launcher generation, setup generation, and hardening.

2.3 The pivot: old editors are not the final product The user then redirected the entire plan. They argued that native OS widget GUI tools already exist through Visual Studio, Xcode, etc. The project needed a cross-platform rendered tool environment that uses the same CLI, TUI, and rendered GUI systems as the client. The old UI Editor and Tool Editor were good exploratory ideas but bad final products. This produced the Dominion Workbench concept.

2.4 Workbench becomes the production

environment Workbench was defined as a rendered, modular, command-driven userland tools environment over the same spine as the client. It should not be one giant editor. It should be one shell with modules and workspaces. Workspaces include Project Graph Explorer, Interface Studio, Module/Pack Foundry, App Composer, Release Forge, Agent Control Workspace, Renderer/Theme Laboratory, and Replay/Trace Workspace. Smaller reusable modules include validation runner, pack browser, command browser, document inspector, patch/diff viewer, evidence viewer, renderer test panel, theme previewer, and asset browser.

The user stressed that as the sole developer, they want to use Workbench and tool modules to actually build the entire Domino/Dominium project. That expanded Workbench from a command dashboard into a visual/ agentic production environment: a place to create code scaffolds, data, packs, UI, graphics, sounds, tests, release artifacts, and AIDE/Codex work units. The governing loop became: intent -> plan -> generate/edit -> preview/simulate -> validate/test -> patch/commit/ package -> evidence -> next action.

2.5 AIDE and parallel development The discussion then moved into AIDE. The user wanted to automate as much as possible through Codex and AIDE. We

developed a task operating model: AIDE creates WorkUnits, tracks attempts, blockers, evidence, repairs, resumes, checkpoints, and promotion decisions. Codex executes bounded tasks. Development can continue with classified partials and repairable blockers; promotion to main requires evidence.

This led to generated prompts: STATUS-RECONCILE-02, AIDE-WORKFLOW-LAW-01, AIDE-WORKUNIT-SCHEMA-01, AIDE-DEV-MAIN-POLICY-01, AIDE-CHECKPOINT-LOOP-01, and AIDE-CAPABILITY-REALITY-LEDGER-01. The user wanted to run AIDE workflow, WorkUnit schema, and dev/main policy concurrently on the same machine via separate Codex tasks.

2.6 Presentation architecture, TUI, rendered GUI, themes, and Workbench authoring The conversation then formalized presentation. CLI, TUI, rendered GUI, native GUI, headless reports, Workbench panels, AIDE, CI, launcher, setup, client, and server should all project the same command/result/refusal/document/view/action/evidence model. The TUI should be a serious keyboard-first terminal projection, not an ASCII toy. Rendered GUI should be a data-described UI system built from layout engines, semantic controls, widget compositions, style tokens, theme profiles, draw-list renderers, and projection conformance tests.

The user shared images as references for Workbench

themes and modules. These were interpreted as visual/mood and layout examples: GNOME-like command studio, macOS-like release forge, Win11 UI/HUD sandbox, pack browser, agent work board, XP/Win7-style boards, worldgen lab, TUI studio, and primitive-only themes. The conclusion was that one cross-platform Workbench app can mix and match module + workspace layout + theme + renderer + platform capability.

2.7 Provider strategy: raylib, SDL2, Lua Later, the chat incorporated raylib, rlsw, rlgl, raygui, raudio, SDL2, and Lua. The core doctrine became: use these aggressively, but only as replaceable providers.

Dominium owns service contracts. Providers satisfy those contracts. Profiles select providers. Third-party types are fenced. Apps stay generic. Workbench edits Dominium documents, not raylib objects.

This prompted a provider structure: runtime/<service>/providers/<provider>, contracts/provider, contracts/capability, contracts/schema/runtime/<service>, release/profiles, content/profiles, and external/upstream. The plan explicitly rejected runtime/raylib/render, apps/client/rendered/raylib, contracts/raylib, top-level profiles, and top-level labs.

2.8 Product spine, structure cleanup, and current maintenance path The final part of the conversation dealt with repo status and task queue. The user pasted status reports indicating that PRESENTATION-CONTRACT-01 completed with warnings, and then chose to generate six maintenance prompts before replanning. FULL-GATE-LEGACY-TEST-ROUTE-01 was generated. This preservation task followed.

3. Main Topics Discussed

Topic 1 - UI Editor / Tool Editor bootstrap plan

The early UI Editor plan was a structured attempt to fix the launcher UI and build a tool for UI authoring. It covered DUI/TLV, native widgets, Win32 preview, deterministic TLV/JSON, layout, validation, codegen, `ops.json`, import/export, and launcher/setup generation. The conclusion was that these are useful components but not the final product architecture. They should become Workbench modules and services.

Topic 2 - Dominion Workbench Platform

Workbench became the central product concept. It is a cross-platform rendered production environment, not a monolithic editor. It hosts workspaces and modules over contracts, commands, documents, patches, services, providers, packs, artifacts, evidence, and tests. It eventually self-hosts by editing its own safe artifacts

first, then product UIs, packs, modules, apps, and AIDE work units.

Topic 3 - AIDE/Codex workflow and parallel development AIDE should govern work; Codex should execute bounded patches. The chat designed branch roles, lifecycle states, WorkUnit schemas, blocker taxonomy, dirty worktree policy, dev/main promotion, checkpoint loops, and capability reality ledger. The key doctrine is non-blocking dev and evidence-blocked promotion.

Topic 4 - Unified presentation/projection system CLI, TUI, rendered GUI, OS-native GUI, and headless output should all project the same command/result/document/view/action model. This enables reuse across client, server, launcher, setup, Workbench, tests, CI, and AIDE.

Topic 5 - TUI as first-class projection The TUI should be a deterministic, keyboard-first, low-dependency projection of the operating environment. It should use panels, tables, trees, forms, logs, status bars, progress meters, and command input. It should support server/admin, setup recovery, CI, SSH, accessibility, and agent workflows.

Topic 6 - Rendered GUI, themes, and Workbench

UX The rendered GUI must be backend-agnostic.

Layouts, controls, widgets, themes, styles, views, and workspaces should be modular and extensible. Themes include first-party primitive-only profiles and OEM+ mimic profiles. The GUI should be task-first, command-backed, evidence-visible, and projection-neutral.

Topic 7 - Provider strategy

Use raylib/SDL2/Lua as seed providers. Keep service identity first-party and provider implementation replaceable. Fence third-party types. Add provider manifests, service contracts, forbidden include validators, and conformance tests before implementation wedges.

Topic 8 - Repo structure and maintenance

The chat concluded not to restart broad structure cleanup. Active structure is good enough; remaining issues should be targeted maintenance. The immediate generated maintenance prompt was `FULL-GATE-LEGACY-TEST-ROUTE-01`.

Topic 9 - Universe Explorer

Universe Explorer became a future north-star: a read-only product/Workbench/Client slice for seamless inspection of universe-scale data, reference frames, materialization, streaming, fidelity degradation, and provenance. It

should start contract-first and headless before visual implementation or embodiment.

4. What We Were Trying to Achieve

4.1 Explicit Goals

- Fix the launcher UI problem by creating better UI tooling.
- Build tools that can help create setup, launcher, game, and future tools.
- Support cross-platform CLI, TUI, rendered GUI, and optional native GUI.
- Make code reusable for other Domino-based games and different engine projects.
- Preserve all decisions, tasks, prompts, constraints, files, and artifacts in a downloadable package.
- Automate as much as possible with Codex/AIDE without losing governance.
- Develop visually with Workbench as soon as the foundation allows.

4.2 Inferred Goals INFERENCE: The user wants to stop losing context across long chats and future assistants. The user also wants a professional-grade

architecture more like a serious engine/platform/OS project than a one-off indie game. The user wants a single system where tools are also tests of the engine/client architecture.

4.3 Goals That Changed Over Time The first goal was a UI Editor. That changed to a Workbench Platform. The first idea of native widget UI tooling changed to rendered cross-platform UI tooling. Broad repo structure cleanup changed to targeted maintenance. Raylib changed from possible engine dependency to seed provider suite.

4.4 Goals Still Unresolved The live repo state must be verified before executing future prompts. Full-gate debt remains. Workbench shell is not implemented. Provider runtime, package runtime, module loader, renderer, native GUI, gameplay, and Universe Explorer remain future work.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Supersede UI Editor/Tool Editor as final products.; Status: Accepted; Why it mattered: Avoids one-off tooling architecture.; Confidence: 5; Label: FACT

Row 2 ID: DECISION-02; Decision: Make Workbench

the production environment.; Status: Accepted;
Why it mattered: Unifies tool, UI, agent, pack,
module, app, release workflows.; Confidence: 5;
Label: FACT

Row 3 ID: DECISION-03; Decision: Workbench is not
authority.; Status: Accepted; Why it mattered:
Prevents GUI/tool drift from contracts/services.;
Confidence: 5; Label: FACT

Row 4 ID: DECISION-04; Decision: AIDE governs,
Codex patches.; Status: Accepted; Why it
mattered: Enables safe automation.; Confidence: 5;
Label: FACT

Row 5 ID: DECISION-05; Decision: Development non-
blocking, promotion evidence-blocked.; Status:
Accepted; Why it mattered: Supports parallel dev
without corrupting main.; Confidence: 5; Label:
FACT

Row 6 ID: DECISION-06; Decision: CLI/TUI/
rendered/native/headless are projections.; Status:
Accepted; Why it mattered: Enables reuse across
apps.; Confidence: 5; Label: FACT

Row 7 ID: DECISION-07; Decision: Use raylib/SDL/
Lua as providers, not architecture.; Status:
Accepted conceptually; Why it mattered: Gives
speed without lock-in.; Confidence: 4; Label:
FACT/INFERENCE

- Row 8** ID: DECISION-08; Decision: Stop broad structure cleanup.; Status: Accepted; Why it mattered: Avoids churn after structure is good enough.; Confidence: 4; Label: FACT
- Row 9** ID: DECISION-09; Decision: Run targeted maintenance before replanning.; Status: Accepted; Why it mattered: User explicitly selected six maintenance prompts.; Confidence: 5; Label: FACT
- Row 10** ID: DECISION-10; Decision: Build Workbench read-only first.; Status: Accepted conceptually; Why it mattered: Safer progressive self-hosting.; Confidence: 4; Label: FACT/INFERENCE

Explanation of major decisions The Workbench decision was the largest architectural correction. The old UI Editor plan could have solved the immediate launcher problem, but it would not have proven the client/runtime stack. Workbench makes the tools use the same systems they help produce.

The AIDE decision was driven by the scale of the project. Once many Codex tasks run in parallel, stale queues, dirty worktrees, partial branches, and unclear blockers become first-order risks. AIDE gives those states names and evidence requirements.

The projection decision was driven by portability and

reuse. If CLI, TUI, rendered GUI, and native GUI are implemented separately, they will drift. If they are projections of the same command/result/view system, all products share semantics.

The provider decision was driven by speed and long-term independence. Raylib/SDL/Lua can accelerate development, but Dominium must own the contracts.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

ID	Option	Status	Reason	Reconsider conditions
REJECTED-01	Windows-only UI Editor as final tool.	Superseded	Too narrow and not client/runtime proof.	As a temporary bootstrap or Workbench module only.
REJECTED-02	Monolithic Tool Editor.	Superseded	Would become separate architecture.	Never as final architecture.
REJECTED-03	Workbench as central authority.	Rejected	Contracts/services/proof must remain authority.	Not expected.
REJECTED-04	Broad root cleanup forever.	Deprioritized	Structure good enough; targeted maintenance now.	If validators block narrow slices.
REJECTED-05	Raylib-shaped app/runtime directories.	Rejected	Vendor-shaped architecture creates lock-in.	Only as temporary proof folders.
REJECTED-06	Pure C99 / pure C++11	Rejected conceptually	C17+C++17 with C ABI	Verify if live repo requires

7. Important Reasoning, Rationale, and Tradeoffs

The main tradeoff was speed versus structural integrity. The user wants visible progress, but not at the cost of another rewrite spiral. The final doctrine uses providers and profiles to get speed while preserving replaceability.

A second tradeoff was governance versus momentum. A project with this many moving parts can stall under too much governance, but without enough, it risks false claims and hidden coupling. The compromise is fast strict and targeted validators on dev, and stronger checkpoint/full-gate proof for main.

A third tradeoff was UI richness versus semantic safety. Workbench should eventually be visual and powerful, but it must first be read-only and command-backed. Visual editing comes only after document/patch/presentation/projection law.

A fourth tradeoff was native GUI versus rendered portability. Native GUIs can exist as optional projections and can use Visual Studio/Xcode tooling, but the core cross-platform Workbench must be rendered through Dominium's own document/view/control/theme/render system.

8. Plans, Future Work, and Next Steps

Immediate user-selected maintenance path

1. `FULL-GATE-LEGACY-TEST-ROUTE-01` - generated immediately before this preservation request. It routes/retired full-gate tests that still expect old roots.
2. `PACK-INTERNAL-LAYOUT-CANON-01` - decide/canonicalize pack-internal layout.
3. `RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01` - classify residual runtime/engine taxonomy.
4. `PUBLIC-HEADER-ABI-PROMOTION-01` - reduce ABI/public header promotion warning debt.
5. `STORAGE-PACKAGE-PROVIDER-SPLIT-01` - clarify storage/package provider boundaries.
6. `POINTER-WIDTH-SERIALIZATION-AUDIT-01` - audit `size_t`, pointers, pointer-width-dependent serialization risks.

Mainline after maintenance/replanning

1. `PROJECTION-CONFORMANCE-01`.
2. `ACCESSIBILITY-CONTRACT-01`.
3. `TEXT-LOCALIZATION-CONTRACT-01`.
4. `WORKBENCH-SHELL-READONLY-01`.
5. `UNIVERSE-EXPLORER-CONTRACT-01`.

6. UNIVERSE-EXPLORER-HEADLESS-01.
7. PROVIDER-MANIFEST-WEDGE-01.
8. SERVICE-CONTRACT-WEDGE-01.
9. THIRD-PARTY-FENCE-01.
10. RAYLIB-PROVIDER-WEDGE-01.
11. UNIVERSE-EXPLORER-VISUAL-01.

Recommended next action Verify live repo state, then review/run FULL-GATE-LEGACY-TEST-ROUTE-01, then generate PACK-INTERNAL-LAYOUT-CANON-01 if the queue/status supports the maintenance lane.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit constraints and preferences

- Use source-grounded, audit-ready answers.
- Preserve uncertainty labels.
- Do not silently infer.
- Do not turn brainstorming into decisions.
- Do not treat assistant suggestions as accepted decisions unless the user accepted them.

- Create actual downloadable files if tools are available.
- Include a human-readable report in-chat even if files are generated.
- Do not restart settled doctrine unnecessarily.
- Use exact, bounded Codex prompts.

9.2 Inferred constraints and preferences

- The user prefers professional-grade modular architecture over quick hacks.
- The user values long-term reuse for future games and engines.
- The user wants a system that can support human and agentic development together.

9.3 Uncertain preferences

- Exact balance between maintenance and product-spine work after the six maintenance tasks.
- Exact timing of provider implementation relative to Universe Explorer.
- Exact format of future master Project Spec Book.

10. Files, Artifacts, Outputs, and Prompts

Artifacts include uploaded screenshots/mockups, preservation instruction text, earlier preservation packages, generated prompt plans, generated Codex prompts, and this companion report package. The most important generated prompt immediately before preservation was **FULL-GATE-LEGACY-TEST-ROUTE-01**.

The screenshots should be preserved as visual references, not treated as formal specs. The prior preservation package should be preserved because it contains sections and structured registers already generated. This companion package bundles those files and adds a more direct accompanying report.

11. Open Questions and Unresolved Issues

ID	Question	Why it matters	Resolution path
QUESTION-01	What is the live repo state after this chat?	Determines next prompt.	Inspect repo, <code>.aide/queue/current.toml</code> , audits.
QUESTION-02	Has FULL-GATE-LEGACY-TEST-ROUTE-01 been executed?	Determines next maintenance prompt.	Check commits/audits.
QUESTION-03	Is C17+C++17 fully implemented in build files?	Affects portability and provider plan.	Verify CMake/toolchain docs.
QUESTION-04	When should raylib provider work start?	Affects speed vs governance.	After provider manifest/service/fence tasks.
QUESTION-05	How soon should Universe Explorer enter implementation?	Affects product identity.	After projection/workbench read-only and headless contract proof.

12. Risks and Failure Modes

Future chats could easily misunderstand this conversation by treating Workbench as a GUI framework, treating raylib as the engine, resuming broad structure cleanup, skipping projection conformance, or assuming live repo status from stale pasted reports. The mitigation is to verify repo state first, preserve the contract/service/projection/provider distinctions, and follow the task queue.

13. Contribution to Future Spec Book

This chat should feed the following spec book chapters:

- Workbench Platform.
- AIDE/Codex development process.
- Presentation/projection architecture.
- Provider strategy and third-party fencing.
- TUI/rendered GUI/theme design.
- Progressive self-hosting.
- Product-spine task sequencing.
- Universe Explorer north-star.
- Maintenance/full-gate debt strategy.

Formal requirements candidates include: Workbench is not authority; third-party providers are fenced; CLI/TUI/rendered/native/headless are projections; development non-blocking/promotion evidence-blocked; every visual edit becomes a document patch; provider capabilities require evidence.

14. What I Should Remember

- The old UI Editor/Tool Editor plan is superseded.
- Workbench is a production environment over contracts/services/proof, not authority.
- AIDE governs work; Codex executes bounded patches.
- CLI, TUI, rendered GUI, OS-native GUI, and headless are projections of one semantic spine.
- Raylib/SDL/Lua are providers, not architecture.
- Broad structure cleanup should stop; targeted maintenance remains.
- The next immediate maintenance prompt after FULL-GATE-LEGACY-TEST-ROUTE-01 is likely PACK-INTERNAL-LAYOUT-CANON-01, pending live repo verification.

15. Best Follow-Up Questions

- "Generate PACK-INTERNAL-LAYOUT-CANON-01."
- "What did we decide about Workbench self-hosting?"
- "List all pending generated prompts and their order."
- "What must be verified in the live repo before continuing?"
- "Turn the provider/raylib doctrine into formal requirements."
- "Create a Project Spec Book chapter outline from this conversation."
- "Which maintenance tasks can run in parallel?"
- "What should the first read-only Workbench shell actually show?"
- "What is the difference between provider profiles and content profiles?"
- "What are the top risks if we skip projection conformance?"

16. Compact Human Summary

This conversation records a major architectural evolution for Dominion. It began with a concrete launcher UI problem: the current Windows launcher looked bad and flickered. The first proposed solution was a Windows-first UI Editor followed by a Tool Editor, using the existing DUI/TLV system, Win32 previews, layout engines, codegen, CLI scripting, import/export, and launcher/setup layout generation. That path was explored deeply and converted into many prompts, but it was later judged wrong as a final product.

The user reframed the goal. Instead of building a one-off editor, Dominion needs a cross-platform rendered Workbench that uses the same systems as the client, server, launcher, setup, CLI, TUI, and headless tools. Workbench should not be authority; it should be the rich production environment over contracts, commands, services, documents, patches, providers, packs, modules, diagnostics, evidence, and tests. It should eventually help the user and agents build the whole project: code scaffolds, packs, modules, themes, UI/HUDs, tests, releases, AIDE/Codex work units, and future Domino-based games.

AIDE became the governance system. Codex is a bounded patch executor. AIDE is the scheduler, context compiler, blocker classifier, evidence ledger, repair/

resume generator, checkpoint coordinator, and promotion gatekeeper. The central doctrine is: development is non-blocking, promotion is evidence-blocked. Dev and task branches can hold classified partials and warnings; main requires evidence-backed checkpoints.

The UI architecture became projection-based. CLI, TUI/text, rendered GUI, OS-native GUI, and headless outputs should all present the same command/result/refusal/document/view/action/diagnostic/evidence model. The TUI is first-class. Rendered GUI is a modular system of layouts, semantic controls, widgets, themes, style tokens, draw lists, and renderer providers. OS-native GUI is optional projection, not semantic authority.

The provider strategy became service-first. Raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua can be used aggressively, but only as replaceable providers. Dominium owns service contracts, provider law, commands, saves, replays, packs, UI documents, and asset identity. Third-party types must not leak into stable law or public SDK surfaces.

The current end state is targeted maintenance plus product-spine continuation. The user reported PRESENTATION-CONTRACT-01 complete with warnings, then selected six maintenance tasks before replanning: full-gate legacy test routing, pack internal layout canon,

runtime/engine residual taxonomy, public header ABI promotion, storage/package provider split, and pointer-width serialization audit. FULL-GATE-LEGACY-TEST-ROUTE-01 was generated immediately before this preservation task.

The best next action is to verify live repo state, then review or run FULL-GATE-LEGACY-TEST-ROUTE-01, then generate PACK-INTERNAL-LAYOUT-CANON-01 if status supports it.

33.10.5 FILE:
dominiumfullconversationcompanionrepo

33.10.6 Structured Registers - Dominion Full Conversation Companion

Workstream Register

Row 1 ID: WORKSTREAM-01; Name: Workbench Platform; Objective: Build a modular production environment over Dominion contracts/services.; Current state: Planned; validation/static/projection slices discussed.; Desired end state: Read-only then authoring Workbench that builds artifacts and products.; Status: Active future; Priority: P0; Confidence: 4; Label: FACT/INFERENCE

Row 2 ID: WORKSTREAM-02; Name: AIDE/Codex

Process; Objective: Safely automate development.;
Current state: Workflow prompts generated; some
reported done.; Desired end state: Task branches,
dev integration, checkpoint main promotion.;
Status: Active; Priority: P0; Confidence: 4; Label:
FACT

Row 3 ID: WORKSTREAM-03; Name: Presentation
Architecture; Objective: Unify CLI/TUI/rendered/
native/headless.; Current state: PRESENTATION-
CONTRACT-01 reported complete.; Desired end
state: Projection conformance and read-only shell.;
Status: Active; Priority: P0; Confidence: 4; Label:
FACT stated in chat

Row 4 ID: WORKSTREAM-04; Name: Provider
Strategy; Objective: Use third-party libraries as
replaceable providers.; Current state: Doctrine
established.; Desired end state: Provider
manifests/fences/conformance and raylib wedge.;
Status: Planned; Priority: P1; Confidence: 4;
Label: FACT/INFERENCE

Row 5 ID: WORKSTREAM-05; Name: Maintenance/
Full-Gate Debt; Objective: Resolve targeted
structural/test debt.; Current state: Six
maintenance tasks selected; first prompt generated.
; Desired end state: Full-gate debt reduced without
broad cleanup.; Status: Active; Priority: P0;

Confidence: 4; Label: FACT

Row 6 ID: WORKSTREAM-06; Name: Universe Explorer; Objective: Prove read-only seamless universe inspection.; Current state: Conceptual north-star.; Desired end state: Headless proof then visual explorer.; Status: Future; Priority: P1; Confidence: 3; Label: INFERENCE

Row 7 ID: WORKSTREAM-07; Name: UI/TUI/Theme System; Objective: Modular layouts, controls, themes, TUI/rendered profiles.; Current state: Conceptual architecture settled.; Desired end state: Theme Lab, Interface Studio, Renderer Sandbox.; Status: Future; Priority: P1; Confidence: 4; Label: FACT/INFERENCE

Row 8 ID: WORKSTREAM-08; Name: Product Apps; Objective: Reuse same modules/packs/UI across client/server/launcher/setup.; Current state: Barebones/product-spine discussion.; Desired end state: Apps as thin compositions over shared services.; Status: Active/future; Priority: P1; Confidence: 4; Label: FACT/INFERENCE

Decision Register

Row 1 ID: DECISION-01; Decision: Supersede old UI Editor/Tool Editor as final products.; Status: Accepted; Evidence / basis: User explicitly said

abandon/recycle them.; Rationale: Avoid one-off Windows-first editor architecture.; Implications: Recycle ideas into Workbench modules.; Related workstream: WORKSTREAM-01; Confidence: 5; Label: FACT

Row 2 ID: DECISION-02; Decision: Workbench is not authority.; Status: Accepted; Evidence / basis: Repeated in chat.; Rationale: Prevent GUI/tools from bypassing contracts.; Implications: Workbench uses commands/services/documents/patches.; Related workstream: WORKSTREAM-01; Confidence: 5; Label: FACT

Row 3 ID: DECISION-03; Decision: AIDE governs, Codex patches.; Status: Accepted; Evidence / basis: Repeated in plans.; Rationale: Enables parallel automation safely.; Implications: Create WorkUnits, checkpoints, evidence.; Related workstream: WORKSTREAM-02; Confidence: 5; Label: FACT

Row 4 ID: DECISION-04; Decision: Development non-blocking, promotion evidence-blocked.; Status: Accepted; Evidence / basis: User handoff/plan.; Rationale: Speeds dev while protecting main.; Implications: dev/task branches vs main/checkpoints.; Related workstream: WORKSTREAM-02; Confidence: 5; Label: FACT

Row 5 ID: DECISION-05; Decision: Presentation modes are projections.; Status: Accepted; Evidence / basis: Repeated doctrine.; Rationale: Prevent separate CLI/TUI/GUI implementations.; Implications: Projection contract/conformance required.; Related workstream: WORKSTREAM-03; Confidence: 5; Label: FACT

Row 6 ID: DECISION-06; Decision: Use raylib/SDL/Lua as providers.; Status: Accepted conceptually; Evidence / basis: User asked and accepted doctrine.; Rationale: Fast progress without lock-in.; Implications: Provider manifests/fencing needed.; Related workstream: WORKSTREAM-04; Confidence: 4; Label: FACT/INFERENCE

Row 7 ID: DECISION-07; Decision: Stop broad structure cleanup.; Status: Accepted; Evidence / basis: User/current plans.; Rationale: Avoid churn after structure is clean enough.; Implications: Run targeted maintenance tasks.; Related workstream: WORKSTREAM-05; Confidence: 4; Label: FACT

Row 8 ID: DECISION-08; Decision: Workbench starts read-only.; Status: Accepted conceptually; Evidence / basis: Assistant recommendation accepted in later plan.; Rationale: Avoid unsafe self-hosting.; Implications: Workbench shell read-only before authoring.; Related workstream:

WORKSTREAM-01; Confidence: 4; Label:
INFERENCE

Task Register

Row 1 ID: TASK-01; Task: Verify live repo state.;
Priority: P0; Urgency: U0; Owner: Human/Codex;
Dependencies: None; Inputs needed: repo
checkout; Expected output: current queue/audits/
status; Next step: inspect before next task; Related
workstream: all; Label: UNCERTAIN

Row 2 ID: TASK-02; Task: Run/review FULL-GATE-
LEGACY-TEST-ROUTE-01.; Priority: P0;
Urgency: U0; Owner: Codex; Dependencies:
Presentation complete; Inputs needed: generated
prompt; Expected output: stale full-gate paths
routed; Next step: generate next maintenance
prompt; Related workstream: WORKSTREAM-05;
Label: FACT

Row 3 ID: TASK-03; Task: Generate PACK-
INTERNAL-LAYOUT-CANON-01.; Priority: P0;
Urgency: U1; Owner: Assistant/Codex;
Dependencies: TASK-02 or live approval; Inputs
needed: status; Expected output: prompt/report;
Next step: execute if selected; Related workstream:
WORKSTREAM-05; Label: FACT

Row 4 ID: TASK-04; Task: PROJECTION-
1757

CONFORMANCE-01.; Priority: P0; Urgency: U1;
Owner: Codex; Dependencies: presentation
contract; Inputs needed: schemas/fixtures;
Expected output: proof of projection parity; Next
step: continue UI path; Related workstream:
WORKSTREAM-03; Label: FACT

Row 5 ID: TASK-05; Task: WORKBENCH-SHELL-
READONLY-01.; Priority: P1; Urgency: U2;
Owner: Codex; Dependencies: projection
conformance; Inputs needed: runtime/app context;
Expected output: read-only Workbench shell; Next
step: validate usefulness; Related workstream:
WORKSTREAM-01; Label: INFERENCE

Row 6 ID: TASK-06; Task: PROVIDER-MANIFEST-
WEDGE-01.; Priority: P1; Urgency: U2; Owner:
Codex; Dependencies: AIDE/presentation path;
Inputs needed: provider doctrine; Expected
output: provider schemas/manifests; Next step:
service/fence wedge; Related workstream:
WORKSTREAM-04; Label: INFERENCE

Row 7 ID: TASK-07; Task: UNIVERSE-EXPLORER-
CONTRACT-01.; Priority: P1; Urgency: U2;
Owner: Codex; Dependencies: Workbench/read-
only/projection; Inputs needed: explorer doctrine;
Expected output: contract-only explorer spec;
Next step: headless proof; Related workstream:

WORKSTREAM-06; Label: INFERENCE

Risk Register

Row 1 ID: RISK-01; Risk / failure mode: Treat stale repo status as current.; Consequence: Wrong next prompt.; Likelihood: Medium; Severity: High; Mitigation: Verify live repo.; Related workstream: all; Label: FACT/INFERENCE

Row 2 ID: RISK-02; Risk / failure mode: Build Workbench UI too early.; Consequence: Parallel UI system.; Likelihood: Medium; Severity: High; Mitigation: Complete projection conformance first. ; Related workstream: WORKSTREAM-01; Label: INFERENCE

Row 3 ID: RISK-03; Risk / failure mode: Raylib types leak into contracts/engine/game.; Consequence: Provider lock-in.; Likelihood: Medium; Severity: High; Mitigation: Forbidden include/type validators.; Related workstream: WORKSTREAM-04; Label: INFERENCE

Row 4 ID: RISK-04; Risk / failure mode: Full-gate debt hidden as pass.; Consequence: False readiness. ; Likelihood: Medium; Severity: High; Mitigation: Keep warning/T4 status explicit.; Related workstream: WORKSTREAM-05; Label: FACT/INFERENCE

Row 5 ID: RISK-05; Risk / failure mode: Assistant suggestions treated as decisions.; Consequence: Spec corruption.; Likelihood: Medium; Severity: Medium; Mitigation: Use labels and decision evidence.; Related workstream: all; Label: FACT

Row 6 ID: RISK-06; Risk / failure mode: Screenshots treated as specs.; Consequence: Overfitted UI.; Likelihood: Medium; Severity: Medium; Mitigation: Treat as references/moodboards only.; Related workstream: WORKSTREAM-07; Label: INFERENCE

Verification Queue

Row 1 ID: VERIFY-01; Item requiring verification: Live `.aide/queue/current.toml`.; Why verification is needed: Determines next prompt.; Suggested source/type: repo file; Priority: P0; Related workstream: all; Label: UNCERTAIN

Row 2 ID: VERIFY-02; Item requiring verification: Execution status of maintenance prompts.; Why verification is needed: Avoid duplicate work.; Suggested source/type: git log/audits; Priority: P0; Related workstream: WORKSTREAM-05; Label: UNCERTAIN

Row 3 ID: VERIFY-03; Item requiring verification: C17/C++17 baseline.; Why verification is needed:

Affects provider/build policy.; Suggested source/
type: CMake/docs; Priority: P1; Related
workstream: WORKSTREAM-04; Label:
UNCERTAIN

Row 4 ID: VERIFY-04; Item requiring verification:
Full CTest debt status.; Why verification is needed:
Affects release readiness.; Suggested source/type:
test reports; Priority: P1; Related workstream:
WORKSTREAM-05; Label: UNCERTAIN

Row 5 ID: VERIFY-05; Item requiring verification:
Existing prior preservation files completeness.;
Why verification is needed: Affects aggregation.;
Suggested source/type: file package; Priority: P2;
Related workstream: archival; Label:
UNCERTAIN

33.10.7 FILE:
 dominiumfullconversationcompanionreport

33.10.8 Current Plan and Prompt Queue -
 Dominium Companion Report

Current immediate state from this chat

The user reported PRESENTATION-CONTRACT-01 as PASS_ WITH_WARNINGS and then selected six maintenance tasks to generate before replanning. The assistant generated

FULL-GATE-LEGACY-TEST-ROUTE-01 immediately before the preservation request.

Immediate maintenance sequence

1. FULL-GATE-LEGACY-TEST-ROUTE-01 - generated. Route/retire full-gate tests still expecting retired roots.
2. PACK-INTERNAL-LAYOUT-CANON-01 - decide and enforce pack-internal layout law.
3. RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01 - classify residual runtime/engine taxonomy warnings.
4. PUBLIC-HEADER-ABI-PROMOTION-01 - burn down public header ABI promotion debt.
5. STORAGE-PACKAGE-PROVIDER-SPLIT-01 - clarify storage/package/provider boundaries.
6. POINTER-WIDTH-SERIALIZATION-AUDIT-01 - audit pointer-width-dependent persistence/serialization risks.

Mainline after maintenance/replanning

1. PROJECTION-CONFORMANCE-01
2. ACCESSIBILITY-CONTRACT-01
3. TEXT-LOCALIZATION-CONTRACT-01

4. WORKBENCH-SHELL-READONLY-01
5. UNIVERSE-EXPLORER-CONTRACT-01
6. UNIVERSE-EXPLORER-HEADLESS-01
7. PROVIDER-MANIFEST-WEDGE-01
8. SERVICE-CONTRACT-WEDGE-01
9. THIRD-PARTY-FENCE-01
10. RAYLIB-PROVIDER-WEDGE-01
11. UNIVERSE-EXPLORER-VISUAL-01

Execution rule

Verify live repo state before each prompt. Do not assume the pasted status remains current. Do not run broad Workbench, renderer, gameplay, package runtime, provider runtime, or module loader work until gates authorize it.

33.10.9 FILE:
dominiumfullconversationcompanionrepo

**33.10.10 Spec Book Bridge - Dominion
Companion Report**

Chapters this chat should inform

1. Domino/Dominium architecture overview.

2. Workbench Platform.
3. AIDE/Codex development process.
4. Presentation/projection architecture.
5. TUI and rendered GUI doctrine.
6. Theme/style/control/widget system.
7. Provider strategy and third-party fencing.
8. Product-spine task sequence.
9. Targeted maintenance/full-gate debt.
10. Universe Explorer north-star.
11. Progressive self-hosting.

Formal requirements candidates

- Workbench is not authority.
- CLI/TUI/rendered/native/headless are projections of the same semantic spine.
- Every user-visible action maps to a command or documented view/action binding.
- All edits go through document/patch/transaction/evidence flow.

- Third-party providers must not leak stable types into contracts/engine/game/content/save/replay/public SDK.
- Provider profiles select providers; apps remain generic.
- Development is non-blocking; promotion is evidence-blocked.
- Capability status must distinguish planned, specified, fixture-only, stubbed, implemented, tested, exposed, release-supported.

Background context candidates

- The original UI Editor/Tool Editor exploration.
- Minecraft-style launcher/setup logical layout test.
- Screenshot/mockup themes and UI inspiration.
- Early open-world/civilization/domain ambitions.

Items needing verification before spec merge

- Live repo task queue and audits.
- C17/C++17 build baseline.
- Current full-gate/CTest status.
- Actual provider/contract directories in repo.

- Whether each generated prompt has been executed.
-

33.10.11 FILE:
dominiumfullconversationcompanionrep

33.10.12 Verification Checklist and Audit - Dominium Companion Report

Verification queue

1. Check `.aide/queue/current.toml` before generating or executing the next prompt.
2. Check whether `FULL-GATE-LEGACY-TEST-ROUTE-01` has already been run.
3. Check whether the current repo still reports `PRESENTATION-CONTRACT-01` as latest completed mainline task.
4. Check whether the six maintenance tasks remain the selected next sequence.
5. Verify `C17/C++17` in live build files.
6. Verify current full `CTest/T4` debt status.
7. Verify whether provider manifest/service/fence tasks already exist in repo.
8. Verify whether the previous preservation package is complete and not superseded.

Self-audit

Issue	Severity	Correction needed	Correction applied?	Residual risk
No raw complete transcript export	High	Mark coverage as partial-to-broad	Yes	Some early details may be missing
Live repo facts may be stale	High	Add verification queue	Yes	Must verify before action
Assistant suggestions could be mistaken for decisions	Medium	Label decisions and inferences	Yes	Some acceptance inferred from user agreement
Screenshots could be overinterpreted	Medium	Treat as visual references	Yes	Future visual spec should review them
Prior generated files may be superseded	Medium	Bundle but mark as supporting material	Yes	Future package may be needed

Reliability

- Completeness: 4/5

- Reliability: 4/5 for chat doctrine, 3/5 for live repo state
- Human-readability: 4/5
- Aggregation-readiness: 4/5

Manual review is recommended before merging into a master Project Spec Book.

33.10.13 FILE: **dominiumfullconversationcompanionrep**

33.10.14 Artifact Index - Dominium Companion Report

Included report files

This companion package includes new report files plus previously generated preservation files.

Included prior packages

The ZIP includes, if present:

- dominium_workbench_aide_architecture_preservation__handoff_package.zip
- Dominium_Workbench_AIDE_Presentation_Provider_Strategy__handoff_package.zip
- individual prior preservation .md and .yaml files

Included uploaded/reference artifacts

The ZIP includes, if present:

- Pasted `text.txt` - uploaded preservation instructions.
- Screenshot/mockup images from May 18, 2026.
- Additional uploaded image-like files named `file_000...` where present.

Important generated prompt artifact

The most recent major prompt generated in the chat was `FULL-GATE-LEGACY-TEST-ROUTE-01`, a targeted maintenance task for routing stale full-gate tests that still expect retired roots.

33.10.15 FILE:
`dominiumfullconversationcompanionrep`

**33.10.16 Reader Brief - Dominion
 Companion Report**

Top things to know

1. The old UI Editor/Tool Editor plan was superseded.

2. Dominion Workbench is now the central production-environment concept.
3. Workbench is not authority; it operates over contracts, commands, services, documents, patches, providers, packs, modules, diagnostics, evidence, and tests.
4. AIDE governs development; Codex executes bounded patches.
5. Development is non-blocking; promotion is evidence-blocked.
6. CLI/TUI/rendered/native/headless are projections of one semantic spine.
7. TUI is first-class.
8. Rendered GUI must be modular and backend-agnostic.
9. OS-native GUI is optional projection, not core semantic law.
10. Raylib/SDL/Lua are seed providers, not architecture.
11. Apps stay generic; profiles select providers.
12. Broad structure cleanup should stop.
13. Targeted maintenance remains.

14. PRESENTATION-CONTRACT-01 was reported complete with warnings.
15. FULL-GATE-LEGACY-TEST-ROUTE-01 was generated next.
16. The next likely prompt is PACK-INTERNAL-LAYOUT-CANON-01, pending live status.
17. Universe Explorer is a future read-only/headless north-star before visual/gameplay work.
18. Workbench should start read-only, then progressively self-host.
19. Live repo facts must be verified before execution.
20. This package is a preservation and aggregation aid, not a live repo authority.

Best next action

Verify current repo state and whether FULL-GATE-LEGACY-TEST-ROUTE-01 has run. If it has not, run/review it. If it has, generate PACK-INTERNAL-LAYOUT-CANON-01.

33.10.17 FILE:
dominiumfullconversationcompanionrep

33.10.18 Future Chat Bootstrap Prompt - Dominium Companion Report

I am continuing from a long Dominium/Domino/
Workbench/AIDE planning chat. Treat the following as
starting context, not live repo truth.

Preserve FACT / INFERENCE / UNCERTAIN /
PROJECT-CONTEXT labels. Do not assume access to
the old chat. Do not re-ask answered questions. Verify
stale facts before relying on them. Flag contradictions.
Use the task register and open questions to determine
next actions.

Core carry-forward:

- Domino = reusable deterministic substrate.
- Dominium = game/product family.
- Workbench = production, validation, editing,
inspection, packaging, and evidence environment.
- AIDE = repo/control-plane harness.
- Codex = bounded patch executor.
- Contracts = law.
- Tests/replay/evidence = proof.

- Workbench is not authority.
- CLI/TUI/rendered/native/headless are projections of one command/result/refusal/document/view/action spine.
- Raylib/SDL/Lua are providers, not architecture.
- Broad structure cleanup should stop; targeted maintenance remains.
- The latest visible user-selected next sequence starts with FULL-GATE-LEGACY-TEST-ROUTE-01, then PACK-INTERNAL-LAYOUT-CANON-01, then other maintenance tasks, before returning to projection/Workbench/Universe Explorer tasks.

When you respond, state:

- context loaded;
- active priorities;
- key constraints;
- open questions;
- contradictions or uncertainties;
- recommended next action.

**33.11 Dominion Workbench Aide
 Architecture Preservation
 Human Readable Report**

Source: docs/archive/conversations/dominium_
full_conversation/previous_preservation_files/
dominium_workbench_aide_architecture_
preservation__01_human_readable_report.md

Report type: human_readable_report

Size: 51033 bytes

**33.11.1 COMPLETE CHAT
 PRESERVATION REPORT -
 Dominion Workbench, AIDE,
 Presentation Architecture, Provider
 Strategy, and Product-Spine Planning**

0. Coverage and Reliability Assessment

- Row 1** Field: Chat label; Assessment: Dominion
Workbench, AIDE, Presentation Architecture,
Provider Strategy, and Product-Spine Planning
- Row 2** Field: Date anchor; Assessment: 2026-05-27
Australia/Melbourne
- Row 3** Field: Source scope; Assessment: This chat only
unless labelled PROJECT-CONTEXT

Row 4 Field: Apparent access; Assessment: Partial-to-
1774

broad visible context: I can see the current conversation state, user-provided handoffs, uploaded preservation instructions, and much of the transcript, but not a guaranteed raw complete line-by-line export of every prior turn.

Row 5 Field: Previously generated files available?; Assessment: No previously generated downloadable package is visible in `/mnt/data`; many prompts and handoff texts were generated in-chat.

Row 6 Field: Uploaded files or artifacts present?; Assessment: Yes: screenshots/mockups and `Pasted text.txt` preservation instruction are present; older screenshots are referenced in chat.

Row 7 Field: Contains future plans?; Assessment: Yes

Row 8 Field: Contains decisions?; Assessment: Yes

Row 9 Field: Contains pending tasks?; Assessment: Yes

Row 10 Field: Contains unresolved questions?; Assessment: Yes

Row 11 Field: Staleness risk; Assessment: High for live repo and external software status; medium for project doctrine; low for in-chat decision history.

Row 12 Field: Extraction confidence; Assessment: 4/5 for visible chat themes and decisions; 3/5 for exact

current repo state; 2/5 for any omitted/skipped early messages.

Row 13 Field: Safe for later aggregation?; Assessment: Yes, with caveats

Row 14 Field: Main limitations; Assessment: No raw full transcript export; live repo facts are user-reported and may be stale; some earlier turns were summarized/skipped; uploaded images are preserved as references but not exhaustively analyzed here.

Plain-language limitations: this preservation package is based on the visible current chat context and user-provided handoff material. It is not a fresh clone of the repo and not a forensic transcript with line numbers. Where the chat discussed live repo state, I preserve what was said and mark verification items. Where assistant recommendations evolved into accepted user plans, I treat them as accepted only when the user clearly endorsed or operationalized them. External facts about raylib, SDL, Lua, GitHub, and the live repo should be verified before implementation.

1. One-Page Orientation

This chat was a long, architecture-heavy planning conversation for Dominium, Domino, AIDE, and the

proposed Workbench. It began around a concrete tooling problem: the Windows launcher UI looked mangled and flickered, and the user wanted a visual internal "Tool Editor" or "UI Editor" that could design pixel-perfect UIs for setup, launcher, game, and tools. Over time, that concrete UI tooling goal was deliberately replaced by a much broader and more durable architecture: a cross-platform, rendered, modular Dominion Workbench that would act as a production environment for the entire project rather than a one-off editor.

The user's deeper aim was not just to make a GUI editor. They wanted the codebase to be portable, modular, extensible, replaceable, and professional-grade: closer to an operating environment or serious game engine platform than an indie one-off. The chat therefore expanded into a general doctrine for how Dominion should be structured: Domino as reusable deterministic substrate, Dominion as a product/game family on that substrate, Workbench as a production, validation, editing, packaging, inspection, evidence, and agent-control environment, and AIDE as the repo/task/checkpoint control plane. The most repeated idea was that the center of the system must not be a folder, an app, a renderer, a UI, a vendor library, or a tool. The center must be contracts, commands, services, documents, patches, providers, modules, packs, artifacts, diagnostics, evidence, tests, and replay/proof.

The discussion repeatedly corrected itself. Early plans had Windows-only UI Editor phases, TLV generation, Minecraft-style launcher/setup layout tests, and native Win32 previews. Those ideas were later judged useful as inspiration but wrong as final products. The user explicitly said the old UI Editor and Tool Editor should be abandoned and recycled. The replacement became "Dominator Workbench Platform": one coherent cross-platform rendered app with many modules, workspaces, themes, provider profiles, and projections, all sharing the same command/result/refusal/document spine as the client, launcher, setup, server, CLI, TUI, and headless modes.

A second major line of discussion was repo governance and development process. The project's queue evolved through Foundation Lock, product-spine slices, package mount, replay proof, barebones client shell, product-spine review, presentation contracts, and targeted cleanup. The final process doctrine became: development is non-blocking, promotion is evidence-blocked. AIDE should manage task branches, WorkUnits, blockers, repair tasks, checkpoint branches, warning dispositions, capability reality status, and main promotion; Codex should execute bounded patch tasks. Full CTest should not be the everyday prompt gate, but should remain release/full-gate debt.

A third line of discussion covered reusable presentation

architecture: CLI, TUI/text, rendered GUI, OS-native GUI, and headless reports should be projections of the same semantic view/action model. Workbench should eventually author views, layouts, widgets, TUI panels, rendered UI documents, themes, workspaces, HUDs, setup/launcher screens, and evidence/test artifacts by editing structured documents through patches, not by hardcoding one-off GUI code.

A fourth line covered provider strategy. The user asked whether to use raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua. The conclusion was yes, aggressively, but only as fenced, replaceable providers behind first-party Dominion service contracts. Dominion should own commands, saves, replays, packs, UI documents, provider law, asset identity, and simulation law; raylib/SDL/Lua should provide early window/input/render/audio/asset/script/UI implementations.

The chat's future relevance is high. It records not only current task order but the rationale for avoiding architecture drift: no GUI-only truth, no vendor-shaped architecture, no Workbench as authority, no broad structure cleanup after the structure became clean enough, no gameplay before product/projection foundations, and no false claims about fixture-only or stubbed capabilities. A future assistant must treat this as a planning and continuity package, verify the live repo before acting, and continue with narrow, evidence-

backed prompts rather than restarting doctrine.

2. The Story of the Conversation

2.1 From mangled launcher UI to a proposed UI Editor The conversation began with a practical problem: the current Windows launcher UI was mangled and flickered. The user wanted a graphical internal UI creation tool capable of visually designing pixel-perfect UIs and broad-strokes functionality for setup, launcher, game, and other tools. The early framing assumed a Windows-first UI Editor that could generate TLV directly and eventually feed a fuller Tool Editor. The user answered batches of questions about target systems, language constraints, current DUI/TLV backends, native widgets, DPI, and launcher flicker sources.

The early plan produced a sequence of Codex prompts: repo scan, canonical UI IR, TLV I/O, capability system, layout engine, splitter/tabs/scroll widgets, action codegen, Phase A UI Editor, Tool Editor bootstrap, flicker fixes, tests/docs, CLI import/export/ops scripting, and Minecraft-style launcher/setup logical layout tests. Screenshots were uploaded and used to define "Minecraft-style" as logical layout rather than graphical skin. These early outputs are preserved as artifacts, but they were later superseded.

2.2 Replacing UI Editor/Tool Editor with

Workbench The user then corrected the direction: OS-native SDK GUI design can be handled by first-party tools like Visual Studio and Xcode. What Dominion really needs is a cross-platform rendered tools environment using the same CLI, TUI, rendered GUI, renderer, command, pack, and runtime services as the client. The old UI Editor/Tool Editor became examples of the wrong final product: too Windows-first, too editor-specific, and too disconnected from the runtime. Their ideas should be recycled as Workbench modules, not preserved as the architecture.

This became the Dominion Workbench Platform: an integrated but modular production environment. Its purpose is not just to inspect Dominion but to build it-code, contracts, data, packs, UI/HUDs, themes, tests, artifacts, releases, and AIDE/Codex work units. Workbench must not own semantics. It must operate over commands, documents, patches, validation, diagnostics, evidence, and providers.

2.3 Generalizing UI across CLI, TUI, rendered GUI, native GUI, and headless

A large part of the chat established that all product surfaces should share one semantic spine. CLI, TUI/text, rendered GUI, OS-native GUI, and headless reports are projections of the same commands, typed results, diagnostics, refusals,

evidence, documents, views, and actions. This was applied to client, launcher, setup, server, Workbench, AIDE, and future modules. The TUI was explicitly framed as a first-class deterministic, keyboard-first projection, not a toy ASCII fallback.

The rendered GUI system was framed as a data-described, themeable, renderer-agnostic system. Layouts, controls, themes, widgets, workspaces, HUDs, and views should be documents. Themes should be token-driven and packable. The user explored OEM+ mimic themes for Windows, Linux, and macOS styles, and primitive-only themes that require no external assets.

2.4 Repo governance, AIDE, queues, and product

spine The conversation then became strongly repo-governance oriented. It established a sequence around Foundation Lock, product-spine slices, AIDE workflow law, WorkUnit schemas, dev/main/checkpoint policy, checkpoint loop, capability reality ledger, presentation contracts, projection conformance, Workbench read-only shell, and universe explorer contracts. The user repeatedly pasted current status summaries and requested next prompts. The assistant generated numerous Codex prompts, including STATUS-RECONCILE-02, AIDE-WORKFLOW-LAW-01, AIDE-WORKUNIT-SCHEMA-01, AIDE-DEV-MAIN-POLICY-01, AIDE-CHECKPOINT-LOOP-01, AIDE-

CAPABILITY-REALITY-LEDGER-01, and FULL-GATE-LEGACY-TEST-ROUTE-01.

A key process doctrine emerged: development is non-blocking, promotion is evidence-blocked. AIDE should manage parallel task branches, dirty worktree classification, blockers, repair tasks, checkpoints, and main promotion. Full CTest remains full-gate debt, not everyday prompt validation.

2.5 Provider strategy and raylib/SDL/Lua Later, the user introduced a transcript about raylib/SDL/Lua. The result was a provider doctrine: use raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua aggressively, but only behind Dominion-owned service contracts. Directory structure should be service-first-runtime/render/providers/raylib, runtime/input/providers/sdl2, etc.-not vendor-first. Apps must remain generic, with profiles selecting providers. Third-party types must be fenced out of engine, game, contracts, content, saves, replays, packs, command/result schemas, and public SDK. Raylib is a seed provider suite, not the architecture.

2.6 Final state before preservation At the point of this preservation request, the latest user-provided state said PRESENTATION-CONTRACT-01 had completed with warnings, and the user wanted to generate maintenance prompts first: FULL-GATE-LEGACY-

TEST-ROUTE-01, PACK-INTERNAL-LAYOUT-CANON-01, RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01, PUBLIC-HEADER-ABI-PROMOTION-01, STORAGE-PACKAGE-PROVIDER-SPLIT-01, and POINTER-WIDTH-SERIALIZATION-AUDIT-01, before replanning the next mainline sequence. The assistant generated FULL-GATE-LEGACY-TEST-ROUTE-01 immediately before the user requested this preservation package.

3. Main Topics Discussed

Topic 1 - Launcher UI, DUI/TLV, and the old UI Editor plan FACT: The chat began with a plan to solve a mangled/flickering launcher UI and build an internal UI authoring tool. The user described an existing Dominion UI schema/state system using TLV and backends such as Win32 common controls, DGFX, and null. Early answers explored native widgets, custom rendering, DPI, layout, action codegen, and flicker mitigation. The conclusion changed: the specific UI Editor/Tool Editor product was superseded, but its concepts-visual layout, property inspector, canonical documents, CLI scripting, codegen, validation, preview-were preserved as future Workbench modules.

Topic 2 - Dominion Workbench as the replacement product FACT: The user accepted that

the old UI Editor/Tool Editor were good general ideas but bad final products. The new direction became a cross-platform rendered Workbench that uses the same runtime, renderer, UI, command, pack, and diagnostics systems as the client. It should be one coherent shell with many modules and workspaces, not a monolithic editor. It should support human graphical work, CLI/TUI workflows, Codex/AIDE agent work, validation, preview, testing, release, pack/mod creation, and eventual self-hosting.

Topic 3 - Unified

command/result/view/projection architecture

FACT: A central conclusion was that CLI, TUI/text, rendered GUI, OS-native GUI, and headless reports should not be separate semantic implementations. They should be projections of the same commands, typed results, documents, views, diagnostics, refusals, evidence, and actions. This doctrine underpins presentation contracts, projection conformance, Workbench, launcher/setup/client/server reuse, and future OS-native UI.

Topic 4 - AIDE as control plane and Codex as

patch executor **FACT:** AIDE was defined as a repo/control-plane harness that manages task state, queue state, evidence, blockers, repair tasks, branches,

checkpoints, and promotion. Codex is a bounded patch executor. Development can be non-blocking on dev, but main promotion must be evidence-blocked. This led to a series of AIDE prompts: workflow law, WorkUnit schema, dev/main policy, checkpoint loop, and capability reality ledger.

Topic 5 - Provider/service architecture and third-party acceleration FACT: The user explored raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua. The conclusion was to use them aggressively as seed providers while keeping Dominion service contracts first-party. Raylib can provide early visible rendering, Workbench UI, assets, and audio, but cannot define simulation, replay, saves, commands, UI law, packs, or public SDK. Provider choices belong in profiles, not app directory names.

Topic 6 - Themes, TUI, rendered GUI, and OEM+ mimic styles FACT: The chat covered modular themes, style tokens, control skins, layout metrics, TUI themes, primitive-only rendered themes, and OEM+ OS-style mimic profiles for Windows, Linux, and macOS. The goal is not legal/pixel-perfect copying of proprietary assets but a rendered, modular, packable system that can mimic layout/geometry/density/control language and support modder expectations.

Topic 7 - Repo structure, full-gate cleanup, and targeted maintenance FACT: The chat repeatedly emphasized that top-level structure was now clean enough and broad structure cleanup should stop. Remaining work should be targeted: full-gate legacy test routing, pack-internal layout canon, runtime/engine residual taxonomy, public header ABI promotion, storage/package provider split, and pointer-width serialization audit. FULL-GATE-LEGACY-TEST-ROUTE-01 was generated as the next maintenance prompt.

Topic 8 - Universe Explorer as future product north-star INFERENCE: The user and assistant converged on a possible first meaningful product identity: a read-only Universe Explorer proving 1:1-scale inspection, no-modal-loading, reference frames, streaming, sparse materialization, fidelity degradation, provenance, and renderer purity. It should begin headlessly and read-only, then become visual, and only later transition toward embodiment/gameplay.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted to preserve and understand this entire chat; build a durable future-proof architecture; make Workbench into a production environment; reuse all code/data/modules/

packs across client, server, setup, launcher, Workbench, and future Domino games; use Codex/AIDE efficiently; support CLI/TUI/rendered/native projections; and avoid one-off indie-project architecture.

4.2 Inferred Goals INFERENCE: The user wants to avoid losing months of architectural reasoning, prevent future assistants from restarting old debates, and convert the chat into material for a master Project Spec Book. The user also wants to accelerate development with parallel Codex tasks without sacrificing repo integrity.

4.3 Goals That Changed Over Time The original goal was a Windows UI Editor to fix the launcher. That changed into a modular cross-platform Workbench. The goal of native widgets changed into optional OS-native projections, while rendered GUI became a first-party portable path. The goal of folder cleanup changed into targeted maintenance once structure became clean enough. The goal of raylib usage changed from possible dependency to seed provider suite behind service contracts.

4.4 Goals Still Unresolved Unresolved goals include confirming live repo state, executing pending maintenance and projection prompts, building read-only Workbench, implementing provider manifests/fences,

building raylib provider wedge, proving Universe Explorer headlessly/visually, and eventually building authoring/self-hosting modules.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Domino is the reusable deterministic substrate; Dominium is one product/game family on it.; Status: Accepted/active; Why it mattered: Allows reuse for other games and prevents Dominium-specific systems from contaminating the engine substrate.; Confidence: 4; Label: FACT

Row 2 ID: DECISION-02; Decision: Workbench is not authority; it is a production environment over contracts, commands, services, documents, patches, evidence, modules, packs, and providers.; Status: Accepted/active; Why it mattered: Prevents a second GUI framework and keeps CLI/TUI/GUI/headless parity.; Confidence: 4; Label: FACT

Row 3 ID: DECISION-03; Decision: Abandon old UI Editor / Tool Editor as final products; recycle their ideas into Workbench modules.; Status: Accepted/active; Why it mattered: Avoids Windows-first one-off tooling and moves to cross-platform rendered tools.; Confidence: 5; Label: FACT

Row 4 ID: DECISION-04; Decision: CLI, TUI/text,

rendered GUI, OS-native GUI, and headless are projections of the same semantic command/result/view system.; Status: Accepted/active; Why it mattered: Prevents separate implementations and enables reuse across client, server, setup, launcher, and Workbench.; Confidence: 4; Label: FACT

Row 5 ID: DECISION-05; Decision: Use progressive self-hosting: seed substrate first, then Workbench edits safe artifacts, then its own presentation, then products.; Status: Accepted/active; Why it mattered: Avoids circular dependency and unsafe self-modifying Workbench.; Confidence: 4; Label: FACT

Row 6 ID: DECISION-06; Decision: Stop broad structure cleanup once canonical active structure is clean enough; use targeted maintenance lanes.; Status: Accepted/latest plan; Why it mattered: Avoids cleanup spiral.; Confidence: 4; Label: FACT

Row 7 ID: DECISION-07; Decision: Use AIDE as governor/ledger/checkpoint/repair system and Codex as bounded patch executor.; Status: Accepted/active; Why it mattered: Supports parallel development without corrupting main.; Confidence: 4; Label: FACT

Row 8 ID: DECISION-08; Decision: Development is non-blocking; promotion is evidence-blocked.;

Status: Accepted/active; Why it mattered: Allows progress on dev while protecting main.;
Confidence: 5; Label: FACT

Row 9 ID: DECISION-09; Decision: Use task branches/worktrees; do not let multiple agents mutate shared dev directly.; Status: Accepted/active; Why it mattered: Prevents conflicts and stale AIDE state.; Confidence: 4; Label: FACT

Row 10 ID: DECISION-10; Decision: Current mainline language doctrine moved from C89/C++98 ideas to C17 + C++17 with C-compatible ABI boundaries.; Status: Accepted but verify live repo; Why it mattered: Improves maintainability while preserving ABI portability.; Confidence: 3; Label: UNCERTAIN / UNVERIFIED

Row 11 ID: DECISION-11; Decision: Use raylib/rigl/rslw/raygui/raudio/SDL2/Lua aggressively only as replaceable providers behind Dominion service contracts.; Status: Accepted/conceptual; Why it mattered: Gives fast visible progress without architectural lock-in.; Confidence: 4; Label: FACT/INFERENCE

Row 12 ID: DECISION-12; Decision: Do not make raylib, SDL2, Lua, or app variants define the architecture.; Status: Accepted/conceptual; Why it mattered: Prevents vendor-shaped directory and

API lock-in.; Confidence: 4; Label: FACT/
INFERENCE

Row 13 ID: DECISION-13; Decision: Raylib rlsw is a raylib software-provider, not the canonical Dominion software renderer.; Status: Accepted/conceptual; Why it mattered: Preserves first-party reference renderer slot.; Confidence: 4; Label: FACT/INFERENCE

Row 14 ID: DECISION-14; Decision: Universe Explorer is a north-star read-only inspection product, not gameplay.; Status: Tentative/accepted as plan; Why it mattered: Proves no-modal-loading, reference frames, streaming, materialization, projection before embodiment.; Confidence: 3; Label: INFERENCE

Row 15 ID: DECISION-15; Decision: Workbench should begin read-only before editing artifacts.; Status: Accepted/active; Why it mattered: Reduces risk and proves projection/evidence first.; Confidence: 4; Label: FACT/INFERENCE

Row 16 ID: DECISION-16; Decision: Full CTest remains full-gate/T4 debt, not every-prompt gate.; Status: Accepted/latest plan; Why it mattered: Supports parallel productivity.; Confidence: 4; Label: FACT/UNVERIFIED

DECISION-01 - Domino is the reusable deterministic substrate; Dominium is one product/game family on it. Status: Accepted/active. Basis: Repeated user acceptance and later handoffs.. Rationale: Allows reuse for other games and prevents Dominium-specific systems from contaminating the engine substrate. Implications: Affects roots, APIs, provider boundaries, Workbench, packs, and future game projects. Caveat/label: FACT.

DECISION-02 - Workbench is not authority; it is a production environment over contracts, commands, services, documents, patches, evidence, modules, packs, and providers. Status: Accepted/active. Basis: User accepted replacement of old UI/Tool Editor with Workbench platform.. Rationale: Prevents a second GUI framework and keeps CLI/TUI/GUI/headless parity. Implications: Workbench must call command/service/document/patch flows, not mutate truth privately. Caveat/label: FACT.

DECISION-03 - Abandon old UI Editor / Tool Editor as final products; recycle their ideas into Workbench modules. Status: Accepted/active. Basis: User explicitly said old UI editor/tool editor were good general ideas but bad final products, to abandon and recycle them.. Rationale: Avoids Windows-first one-

off tooling and moves to cross-platform rendered tools. Implications: Old concepts become Interface Studio, UI/ HUD Sandbox, Theme Lab, Module/Pack Foundry, App Composer. Caveat/label: FACT.

DECISION-04 - CLI, TUI/text, rendered GUI, OS-native GUI, and headless are projections of the same semantic command/result/view system.

Status: Accepted/active. Basis: Repeatedly discussed and accepted.. Rationale: Prevents separate implementations and enables reuse across client, server, setup, launcher, and Workbench. Implications: Requires presentation contracts and projection conformance before rich UI. Caveat/label: FACT.

DECISION-05 - Use progressive self-hosting: seed substrate first, then Workbench edits safe artifacts, then its own presentation, then products.

Status: Accepted/active. Basis: User accepted progressive self-hosting model.. Rationale: Avoids circular dependency and unsafe self-modifying Workbench. Implications: Defines Workbench maturity ladder S0-S6. Caveat/label: FACT.

DECISION-06 - Stop broad structure cleanup once canonical active structure is clean enough; use targeted maintenance lanes. Status: Accepted/

latest plan. Basis: User shifted to full-gate legacy test routing and targeted cleanup.. Rationale: Avoids cleanup spiral. Implications: Maintenance tasks include full-gate legacy test route, pack layout canon, residual taxonomy, public ABI promotion. Caveat/label: FACT.

DECISION-07 - Use AIDE as governor/ledger/checkpoint/repair system and Codex as bounded patch executor. Status: Accepted/active. Basis: Explicit in multiple handoff plans.. Rationale: Supports parallel development without corrupting main. Implications: Requires workflow law, WorkUnit schema, dev/main policy, checkpoint loop, capability reality ledger. Caveat/label: FACT.

DECISION-08 - Development is non-blocking; promotion is evidence-blocked. Status: Accepted/active. Basis: Repeated as doctrine in user and assistant handoffs.. Rationale: Allows progress on dev while protecting main. Implications: Shapes branch model and testing policy. Caveat/label: FACT.

DECISION-09 - Use task branches/worktrees; do not let multiple agents mutate shared dev directly. Status: Accepted/active. Basis: Discussed as required before larger parallelism.. Rationale: Prevents conflicts

and stale AIDE state. Implications: Requires branch
role policy and checkpoint loop. Caveat/label: FACT.

**DECISION-10 - Current mainline language
doctrine moved from C89/C++98 ideas to C17 +
C++17 with C-compatible ABI boundaries.**

Status: Accepted but verify live repo. Basis: User pasted
synthesis stating this; earlier chat had C89/C++98
assumptions.. Rationale: Improves maintainability while
preserving ABI portability. Implications: Needs live
verification in CMake/build policy before final spec
merge. Caveat/label: UNCERTAIN / UNVERIFIED.

**DECISION-11 - Use
raylib/rlgl/rlsw/raygui/raudio/SDL2/Lua
aggressively only as replaceable providers behind
Dominium service contracts.** Status: Accepted/
conceptual. Basis: User asked what I thought; assistant
agreed and refined.. Rationale: Gives fast visible
progress without architectural lock-in. Implications:
Provider work should follow AIDE/presentation and
manifest/fence/service contract wedges. Caveat/label:
FACT/INFERENCE.

**DECISION-12 - Do not make raylib, SDL2, Lua,
or app variants define the architecture.** Status:
Accepted/conceptual. Basis: Explicitly discussed as

service-first/provider-backed doctrine.. Rationale: Prevents vendor-shaped directory and API lock-in. Implications: Requires forbidden include validator and provider manifests. Caveat/label: FACT/INFERENCE.

DECISION-13 - Raylib rlsw is a raylib software-provider, not the canonical Dominium software renderer. Status: Accepted/conceptual. Basis: Discussed and accepted in provider doctrine.. Rationale: Preserves first-party reference renderer slot. Implications: Directory split: runtime/render/providers/software vs runtime/render/providers/rlsw. Caveat/label: FACT/INFERENCE.

DECISION-14 - Universe Explorer is a north-star read-only inspection product, not gameplay. Status: Tentative/accepted as plan. Basis: Discussed after structure cleanup as first meaningful product identity.. Rationale: Proves no-modal-loading, reference frames, streaming, materialization, projection before embodiment. Implications: Implementation should start with contracts/headless proof, not renderer/gameplay. Caveat/label: INFERENCE.

DECISION-15 - Workbench should begin read-only before editing artifacts. Status: Accepted/active. Basis: Repeated in self-hosting safety

stages.. Rationale: Reduces risk and proves projection/ evidence first. Implications: First Workbench shell should inspect commands, diagnostics, artifacts, graphs, packs, status. Caveat/label: FACT/INFERENCE.

DECISION-16 - Full CTest remains full-gate/T4 debt, not every-prompt gate. Status: Accepted/ latest plan. Basis: Repeated; user wants speed on dev with full suite at main/checkpoints.. Rationale: Supports parallel productivity. Implications: Requires targeted validators, fast strict, and checkpoint policy. Caveat/label: FACT/UNVERIFIED.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

REJECTED-01 - Old Windows-only UI Editor as final product Status: Superseded. It was considered because it addressed a real need, but it was rejected or superseded because too narrow, native-widget-first, not cross-platform/reusable; replaced by workbench modules. Reconsideration: Temporary for bootstrap only if ever needed.. Related workstream: WORKSTREAM-05. Label: FACT.

REJECTED-02 - Tool Editor as monolithic editor Status: Superseded. It was considered because it addressed a real need, but it was rejected or superseded

because would become separate architecture; replaced by modular workbench platform. Reconsideration: Could reappear as workspace/module concept only.. Related workstream: WORKSTREAM-05. Label: FACT.

REJECTED-03 - Workbench as authority/center of architecture Status: Rejected. It was considered because it addressed a real need, but it was rejected or superseded because contracts/commands/services/providers/packs/artifacts/proof are center; workbench is surface. Reconsideration: Reconsider only as UX language, not architecture.. Related workstream: WORKSTREAM-05. Label: FACT.

REJECTED-04 - Broad root/directory rewrites after structure became clean enough Status: Deprioritized. It was considered because it addressed a real need, but it was rejected or superseded because risk of cleanup spiral; remaining work targeted. Reconsideration: If validators show blockers.. Related workstream: WORKSTREAM-07. Label: FACT.

REJECTED-05 - Raylib-shaped architecture or apps/client/rendered/raylib variants Status: Rejected. It was considered because it addressed a real need, but it was rejected or superseded because vendor/provider must not define architecture. Reconsideration:

Temporary proof folders allowed with retirement notes..
Related workstream: WORKSTREAM-06. Label:
FACT/INFERENCE.

REJECTED-06 - Pure C99 or pure C++11 as mainline baseline Status: Rejected/superseded. It was considered because it addressed a real need, but it was rejected or superseded because c17+c++17 with c-compatible abi deemed better fit. Reconsideration: If platform/toolchain evidence contradicts.. Related workstream: WORKSTREAM-01. Label: UNCERTAIN / UNVERIFIED.

7. Important Reasoning, Rationale, and Tradeoffs

The dominant tradeoff was speed versus architectural contamination. The user wanted fast progress, visual tools, and agentic development, but not at the cost of making temporary tools permanent architecture. The recurring solution was to split identity from implementation: commands, contracts, documents, packs, modules, services, and provider manifests are stable; individual folders, renderers, libraries, UI skins, and Workbench panels are replaceable.

A second tradeoff was governance versus motion. Too little governance had already produced cleanup debt; too much governance risked stalling the project. The

compromise became a queue model: Foundation and product-spine contracts first, then narrow slices, then limited parallel dev on task branches, then checkpointed promotion. Full CTest remains a release/full-gate concern rather than a prompt-by-prompt requirement.

A third tradeoff was native GUI versus rendered portability. The user recognized that Visual Studio/Xcode/AppKit/Win32 tooling already exists for OS SDK GUI work. The unique Dominium value is a rendered, cross-platform, modular presentation system shared by the client, Workbench, launcher, setup, server, CLI/TUI/headless, and future mods.

A fourth tradeoff was third-party acceleration versus engine independence. Raylib, SDL2, and Lua can speed up the first visible product slices; they become safe only if fenced behind Dominium service contracts and conformance tests. This preserves replaceability while allowing practical progress.

8. Plans, Future Work, and Next Steps

The immediate future work from the latest state in this chat is a maintenance set followed by mainline projection and Workbench planning. The user explicitly said to generate first: FULL-GATE-LEGACY-TEST-ROUTE-01, PACK-INTERNAL-LAYOUT-CANON-01, RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01,

PUBLIC-HEADER-ABI-PROMOTION-01, STORAGE-PACKAGE-PROVIDER-SPLIT-01, and POINTER-WIDTH-SERIALIZATION-AUDIT-01. FULL-GATE-LEGACY-TEST-ROUTE-01 was generated immediately before this preservation task.

After maintenance and replan, the user listed the sequence: PROJECTION-CONFORMANCE-01, ACCESSIBILITY-CONTRACT-01, TEXT-LOCALIZATION-CONTRACT-01, WORKBENCH-SHELL-READONLY-01, UNIVERSE-EXPLORER-CONTRACT-01, UNIVERSE-EXPLORER-HEADLESS-01, PROVIDER-MANIFEST-WEDGE-01, SERVICE-CONTRACT-WEDGE-01, THIRD-PARTY-FENCE-01, RAYLIB-PROVIDER-WEDGE-01, and UNIVERSE-EXPLORER-VISUAL-01. PRESENTATION-CONTRACT-01 was already reported complete before that list.

Recommended next-action sequence in this chat:

1. Execute or review FULL-GATE-LEGACY-TEST-ROUTE-01.
2. Generate PACK-INTERNAL-LAYOUT-CANON-01.
3. Generate RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01.
4. Generate PUBLIC-HEADER-ABI-PROMOTION-

01.

5. Generate STORAGE-PACKAGE-PROVIDER-SPLIT-01.
6. Generate POINTER-WIDTH-SERIALIZATION-AUDIT-01.
7. Replan based on live repo state.
8. Continue to PROJECTION-CONFORMANCE-01.

Major dependencies: live repo state must be verified; broad feature blockers must remain; full CTest debt should be tracked, not hidden; presentation contracts should precede Workbench and renderer implementation; provider fences should precede raylib implementation.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences

- CONSTRAINT-01: Do not turn Workbench into authority. Practical implication: Workbench must use commands/services/documents/patches/evidence.
- CONSTRAINT-02: Broad Workbench UI, provider runtime, package runtime, module loader, renderer/native GUI, gameplay, release publication remain blocked until gates authorize. Practical

implication: Future prompts must preserve non-goals.

- CONSTRAINT-03: No broad structure cleanup unless validator/audit blocks next slice. Practical implication: Use targeted maintenance lanes.
- CONSTRAINT-04: CLI/TUI/rendered/native/headless must be projections of same command/result/view spine. Practical implication: Presentation work must avoid separate semantics per UI mode.
- CONSTRAINT-05: Third-party types must not leak into engine/game/contracts/content/saves/replays/public SDK. Practical implication: Provider code must translate to Dominion types.
- CONSTRAINT-06: Full CTest is full-gate/T4 debt, not ordinary prompt gate. Practical implication: Dev tasks run targeted validators/fast strict; main/checkpoint more stringent.
- CONSTRAINT-07: Stable identity is contractual, not path-based. Practical implication: Use IDs/manifests/registries, not folder names, as identity.
- CONSTRAINT-08: Apps compose; runtime implements; contracts define law; content supplies authored payloads; tools validate/generate/

migrate. Practical implication: Prevents apps/tools becoming runtime or truth owners.

- CONSTRAINT-09: Development non-blocking; promotion evidence-blocked. Practical implication: Dev may accept classified warnings; main requires checkpoint/evidence.
- CONSTRAINT-10: The preservation report itself is for THIS CHAT ONLY unless labelled PROJECT-CONTEXT. Practical implication: Do not merge other chat facts unlabelled.

9.2 Inferred Constraints and Preferences

- The user wants maximal continuity and minimal repeated re-explanation. Future assistants should treat this preservation package and the generated handoffs as continuity artifacts.
- The user values engineering discipline over short-term demos, but still wants speed through parallel Codex/AIDE work.

9.3 Uncertain or Unestablished Preferences

- Exact final degree of raylib/SDL/Lua usage is conceptual until provider tasks are run.
- Exact live repo state after this preservation task is unverified.

10. Files, Artifacts, Outputs, and Prompts

Row 1 ID: ARTIFACT-01; Artifact / file / prompt / output: Prompt UU1-UU6 plan for UI Editor import/CLI/ops/Minecraft launcher/setup tests; Type: Prompt plan; Purpose: Earlier plan for old UI Editor toolchain.; Status: Superseded/recycled; Origin: This chat; Carry forward?: Yes, as rejected/superseded context; Notes: Ideas became Workbench modules and CLI/presentation concepts.; Label: FACT

Row 2 ID: ARTIFACT-02; Artifact / file / prompt / output: Final UI Editor implementation prompt with import/CLI/Minecraft tests; Type: Codex prompt; Purpose: Implementation prompt for old Phase A UI Editor.; Status: Superseded; Origin: This chat; Carry forward?: Yes, as historical artifact; Notes: Shows why old approach was abandoned.; Label: FACT

Row 3 ID: ARTIFACT-03; Artifact / file / prompt / output: Screenshots/mockups of Workbench themes/modules; Type: Images; Purpose: Visual references for OEM+ themes, modules, TUI, Workbench surfaces.; Status: Reference only; Origin: Uploaded in chat; Carry forward?: Yes; Notes: Used to derive modular theme/workspace concepts, not exact design requirements.; Label:

FACT

Row 4 ID: ARTIFACT-04; Artifact / file / prompt / output: AIDE/Workbench/product-spine handoff prompt; Type: Handoff text; Purpose: Long carry-forward prompt summarizing repo state and next tasks.; Status: Important; Origin: This chat; Carry forward?: Yes; Notes: Should inform future chat bootstrap; live repo must still be verified.; Label: FACT

Row 5 ID: ARTIFACT-05; Artifact / file / prompt / output: Codex prompts: STATUS-RECONCILE-02, AIDE-WORKFLOW-LAW-01, AIDE-WORKUNIT-SCHEMA-01, AIDE-DEV-MAIN-POLICY-01, AIDE-CHECKPOINT-LOOP-01, AIDE-CAPABILITY-REALITY-LEDGER-01; Type: Codex prompts; Purpose: Detailed prompts for AIDE workflow layer.; Status: Generated; Origin: This chat; Carry forward?: Yes; Notes: Some reportedly executed later; verify live repo.; Label: FACT

Row 6 ID: ARTIFACT-06; Artifact / file / prompt / output: Codex prompt: FULL-GATE-LEGACY-TEST-ROUTE-01; Type: Codex prompt; Purpose: Targeted full-gate stale-root cleanup prompt.; Status: Generated immediately before preservation request; Origin: This chat; Carry forward?: Yes;

Notes: Potential next execution depending on user plan.; Label: FACT

Row 7 ID: ARTIFACT-07; Artifact / file / prompt / output: User-uploaded Pasted text.txt preservation instruction; Type: Uploaded file; Purpose: Maximum-fidelity preservation package instructions.; Status: Active input; Origin: Uploaded file; Carry forward?: Yes; Notes: Primary instruction for this response.; Label: FACT

Row 8 ID: ARTIFACT-08; Artifact / file / prompt / output: This preservation package files and zip; Type: Generated export; Purpose: Downloadable report/register/spec/handoff files created by this response.; Status: Created now; Origin: This response; Carry forward?: Yes; Notes: Use for aggregation and future chat transfer.; Label: FACT

11. Open Questions and Unresolved Issues

Row 1 ID: QUESTION-01; Question / issue: What is the live repo status at the time of continuation?; Why it matters: The chat includes changing queue and commit state.; Known information: User reported latest tasks and statuses.; Unknown information: Whether subsequent Codex runs changed queue/audits after this preservation.; Resolution path: Verify `.aide/queue/current.`

toml, latest commits, relevant audits.; Priority: P0; Related workstream: WORKSTREAM-03; Label: UNCERTAIN / UNVERIFIED

Row 2 ID: QUESTION-02; Question / issue: Should the next executable task be FULL-GATE-LEGACY-TEST-ROUTE-01 or PROJECTION-CONFORMANCE-01?; Why it matters: User said first generate maintenance prompts, then replan.; Known information: FULL-GATE prompt was generated immediately before preservation request.; Unknown information: Whether user will run maintenance lane first or mainline projection.; Resolution path: Ask user or check queue after files.; Priority: P0; Related workstream: WORKSTREAM-07; Label: FACT/UNCERTAIN

Row 3 ID: QUESTION-03; Question / issue: Is C17/C++17 fully implemented in CMake or partially doctrinal?; Why it matters: Language baseline changed during chat.; Known information: Later handoffs state C17/C++17 baseline.; Unknown information: Live CMake/toolchain details.; Resolution path: Verify repo files before spec lock.; Priority: P1; Related workstream: WORKSTREAM-01; Label: UNCERTAIN / UNVERIFIED

Row 4 ID: QUESTION-04; Question / issue: How

aggressively should provider/raylib work start relative to presentation/projection and Workbench read-only shell?; Why it matters: Provider strategy was accepted conceptually but timing deferred.; Known information: Recommended after AIDE/presentation/fence/manifest tasks.; Unknown information: Exact queue timing after maintenance tasks.; Resolution path: Replan after projection conformance and maintenance status.; Priority: P1; Related workstream: WORKSTREAM-06; Label: INFERENCE

Row 5 ID: QUESTION-05; Question / issue: What is the final target for Universe Explorer as first product identity?; Why it matters: It is a north-star but not implemented.; Known information: Headless/read-only first, visual later, embodiment after.; Unknown information: How soon to schedule after read-only Workbench/presentation.; Resolution path: Formalize UNIVERSE-EXPLORER-CONTRACT-01 when gates allow.; Priority: P2; Related workstream: WORKSTREAM-08; Label: INFERENCE

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Row 1 ID: RISK-01; Risk / failure mode: Future assistant treats stale repo status as current.;

Consequence: Wrong prompts, duplicate tasks, stale queues.; Likelihood: Medium; Severity: High; Mitigation: Always verify live repo before current-status decisions.; Related workstream: WORKSTREAM-03; Label: UNCERTAIN / UNVERIFIED

Row 2 ID: RISK-02; Risk / failure mode: Workbench implementation starts before projection conformance.; Consequence: Separate UI semantics, later rewrite.; Likelihood: Medium; Severity: High; Mitigation: Require presentation/projection contracts first.; Related workstream: WORKSTREAM-04; Label: FACT/INFERENCE

Row 3 ID: RISK-03; Risk / failure mode: Raylib/SDL/ Lua provider types leak into stable contracts or game state.; Consequence: Vendor lock-in, replay/save instability.; Likelihood: Medium; Severity: High; Mitigation: Forbidden include/type validator and provider fences.; Related workstream: WORKSTREAM-06; Label: FACT/INFERENCE

Row 4 ID: RISK-04; Risk / failure mode: Full CTest debt remains ignored too long.; Consequence: Release gate remains blocked.; Likelihood: Medium; Severity: Medium; Mitigation: Targeted full-gate legacy test routing and debt ledger.; Related workstream: WORKSTREAM-07; Label:

FACT/UNVERIFIED

Row 5 ID: RISK-05; Risk / failure mode: Capability status overclaimed.; Consequence: Docs/Workbench say implemented when only fixture/stub exists.; Likelihood: High; Severity: High; Mitigation: Capability Reality Ledger.; Related workstream: WORKSTREAM-02; Label: FACT

Row 6 ID: RISK-06; Risk / failure mode: Parallel Codex tasks conflict on coordinator files.; Consequence: Queue/status corruption.; Likelihood: Medium; Severity: High; Mitigation: Coordinator ownership rules and checkpoint loop.; Related workstream: WORKSTREAM-02; Label: FACT

Row 7 ID: RISK-07; Risk / failure mode: Universe Explorer free-camera becomes implicit gameplay movement.; Consequence: Wrong gameplay model and authority leakage.; Likelihood: Low-Medium; Severity: High; Mitigation: Separate observer inspection authority from embodied actor authority.; Related workstream: WORKSTREAM-08; Label: INFERENCE

Row 8 ID: RISK-08; Risk / failure mode: Preservation report overstates decisions from assistant suggestions.; Consequence: Bad spec aggregation.; Likelihood: Medium; Severity: High; Mitigation:

Label decisions accepted by user vs assistant suggestions.; Related workstream: WORKSTREAM-01; Label: FACT

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat should feed several chapters of a future master Project Spec Book. It contributes the clearest articulation of the Workbench replacement plan; the AIDE/Codex development process; unified presentation architecture; provider/service/fence strategy; modular theme/UI/TUI/rendered GUI doctrine; and the transition from broad cleanup to targeted maintenance plus product-spine slices.

Formal requirement candidates include: Workbench must not be authority; CLI/TUI/rendered/native/headless must project the same command/result/view model; third-party libraries must be providers behind service contracts; capability reality status must distinguish planned/specified/fixture/stubbed/implemented/tested/exposed/release_supported; and promotion to main must be evidence-blocked.

Background context includes the abandoned UI Editor/Tool Editor plan, Minecraft-style launcher/setup logical layouts, OEM+ theme aspirations, TUI design mockups, and long-range world simulation doctrine. These should

not all become immediate requirements.

Verification before merge: live repo status, C17/C++17 baseline, current queue, and whether latest generated prompts have run.

14. What I Should Remember

- The chat's center of gravity moved from a Windows UI Editor to a cross-platform Dominion Workbench Platform.
- Workbench is an interface over contracts, commands, documents, patches, services, providers, packs, modules, artifacts, diagnostics, and evidence. It is not authority.
- AIDE is the coordination system; Codex is the patch executor. Development should be non-blocking; promotion should be evidence-blocked.
- The old UI Editor/Tool Editor plan is superseded but not wasted; it becomes Interface Studio, UI/ HUD Sandbox, Theme Lab, Module/Pack Foundry, and App Composer modules.
- The immediate current path in this chat is maintenance and projection: full-gate legacy test routing, pack layout canon, runtime/engine residual taxonomy, public ABI promotion, storage/

package provider split, pointer-width audit, then projection conformance.

- Raylib/SDL/Lua are accelerators, not architecture. They belong behind Dominium-owned service contracts and provider manifests.
- Universe Explorer is a future read-only inspection north-star, not gameplay and not free-camera movement authority.
- The biggest live risk is stale repo status; verify before generating current-state prompts.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- "Explain why the old UI Editor plan was abandoned and how its ideas survive in Workbench."
- "Summarize the difference between Workbench as surface and contracts/services as system center."

15.2 Decisions

- "Which decisions in this chat are final, and which are only tentative?"
- "What decision would most likely need revisiting if the live repo contradicts our assumed queue state?"

15.3 Tasks and Next Actions

- "Generate PACK-INTERNAL-LAYOUT-CANON-01 next."
- "Make a dependency map showing which maintenance prompts can run in parallel."

15.4 Artifacts and Files

- "List all Codex prompts generated in this chat and their intended execution order."
- "Which uploaded images influenced the Workbench/theme direction?"

15.5 Risks and Verification

- "What should be verified in the repo before executing the next prompt?"
- "What are the highest-risk false implementation claims to avoid?"

15.6 Future Spec Book / Aggregation

- "Which parts of this report should become formal requirements in the Project Spec Book?"
- "What overlaps should I expect with other old-chat reports?"

15.7 Deep-Dive Questions Specific to This Chat

- "Design the first read-only Workbench shell slice using the presentation contract."
- "Write the provider manifest doctrine as formal requirements."
- "Explain the Universe Explorer headless proof as a task sequence."

16. Compact Human Summary

This chat began as a practical tool/UI problem and became one of the central architecture-planning conversations for Dominion. The original issue was that the Windows launcher UI looked mangled and flickered, and the user wanted a graphical Tool Editor/UI Editor to design pixel-perfect UIs and broad functionality for launcher, setup, game, and tools. Early discussion produced a detailed UI Editor plan: Windows-first editor, canonical UI IR, TLV/JSON output, layout engine, capability system, code generation, CLI scripting, import/export, and launcher/setup layout tests. That plan was useful but later superseded.

The major turning point was the user's recognition that the old UI Editor and Tool Editor were good ideas but wrong final products. The user wanted a cross-platform rendered tools environment using the same systems as

the client, not a Windows-native one-off editor. This became the Dominion Workbench Platform: one coherent shell with many modules and workspaces, built on the same command, document, patch, view, projection, renderer, pack, diagnostics, and evidence spine as the rest of Dominion.

The core architectural doctrine became: Domino is the reusable deterministic substrate; Dominion is a game/product family built on it; Workbench is the production environment; AIDE is the repo/control-plane harness; Codex is a bounded patch executor; contracts are law; tests/replay/evidence are proof. Workbench must not be authority. It should design artifacts, while runtime executes artifacts, contracts constrain them, content packages them, apps consume them, tests prove them, AIDE governs changes, and Codex patches code.

The chat repeatedly unified CLI, TUI/text, rendered GUI, native GUI, and headless operation. These are not separate products or separate behavioral systems. They are projections of the same command/result/refusal/document/view/action model. This is essential for client, server, launcher, setup, Workbench, AIDE, and future modding. The TUI was framed as a serious low-dependency deterministic interface, not a toy fallback. The rendered GUI was framed as a cross-platform data-described UI system with layouts, controls, themes, widget compositions, workspaces, HUDs, and theme

packs.

Another major theme was development governance. The chat moved through many repo-state plans: Foundation Lock, package mount, replay proof, barebones client shell, product spine review, AIDE workflow law, WorkUnit schema, dev/main policy, checkpoint loop, and capability reality ledger. The doctrine became: development is non-blocking; promotion is evidence-blocked. Dev can accumulate classified work and warnings, but main should receive only checkpointed, evidence-backed states. Full CTest remains release/full-gate debt; fast strict and targeted validators are normal task gates.

The provider strategy was also clarified. Raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua should be used aggressively to accelerate visible progress. However, they must be replaceable providers behind Dominion service contracts. Dominion owns commands, views, assets, saves, replays, packs, modules, provider law, and simulation law. Third-party types must not leak into engine, game, contracts, content, save/replay formats, pack schemas, public SDK, or deterministic domain law. Apps must remain generic; profiles select providers.

By the final state of the chat, PRESENTATION-CONTRACT-01 had reportedly completed with warnings, and the user wanted a targeted maintenance sequence before continuing: FULL-GATE-LEGACY-

TEST-ROUTE-01, PACK-INTERNAL-LAYOUT-CANON-01, RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01, PUBLIC-HEADER-ABI-PROMOTION-01, STORAGE-PACKAGE-PROVIDER-SPLIT-01, and POINTER-WIDTH-SERIALIZATION-AUDIT-01. FULL-GATE-LEGACY-TEST-ROUTE-01 was generated immediately before this preservation task. After those, the plan returns to PROJECTION-CONFORMANCE-01, ACCESSIBILITY-CONTRACT-01, TEXT-LOCALIZATION-CONTRACT-01, WORKBENCH-SHELL-READONLY-01, Universe Explorer contracts, provider manifest/service/fence/raylib wedges, and later visual Explorer.

The most important preservation point is that this chat records a mature architecture doctrine and transition plan. It should not be reduced to a list of prompts. Future assistants must understand why the plan changed: from one-off editor to reusable Workbench, from UI widgets to semantic projections, from folder structure to contracts/proof, from direct Codex execution to AIDE-governed task branches, and from vendor libraries to fenced providers. The best next action is to continue the targeted maintenance sequence, starting with or reviewing FULL-GATE-LEGACY-TEST-ROUTE-01, then generate the remaining maintenance prompts, verify the live repo, and proceed to projection conformance only when current status

supports it.

33.12 Dominion Workbench Aide Architecture Preservation Reader Brief

Source: docs/archive/conversations/dominium_
full_conversation/previous_preservation_files/
dominium_workbench_aide_architecture_
preservation__06_reader_brief.md

Report type: reader_brief

Size: 1661 bytes

33.12.1 Reader Brief - Dominion Workbench, AIDE, Presentation Architecture, Provider Strategy, and Product-Spine Planning

What this chat was about

This chat documents the evolution from a concrete UI Editor idea into a broad Dominion Workbench/AIDE architecture. It covers Workbench, AIDE, command/result/view/projection architecture, rendered/TUI/native UI, theme systems, provider strategy with raylib/SDL/Lua, targeted repo maintenance, product-spine planning, and future Universe Explorer.

Top 20 things to know

1. Old UI Editor/Tool Editor is superseded.
2. Workbench is the new production environment.
3. Workbench is not authority.
4. Domino is reusable substrate; Dominion is product family.
5. AIDE governs tasks/evidence; Codex executes patches.
6. CLI/TUI/rendered/native/headless are projections of one semantic spine.
7. Presentation contracts precede rich Workbench UI.
8. TUI is first-class.
9. Rendered GUI is portable and data-described.
10. OS-native GUI is optional projection.
11. Development non-blocking, promotion evidence-blocked.
12. Full CTest remains release/full-gate debt.
13. Capability Reality Ledger prevents overclaims.
14. Raylib/SDL/Lua are providers, not architecture.
15. Third-party types must be fenced.

16. Apps remain generic; profiles select providers.
17. Structure cleanup should now be targeted, not broad.
18. Universe Explorer is future read-only product north-star.
19. Workbench self-hosting must be progressive.
20. Verify live repo before executing next tasks.

Best next step

Review/run FULL-GATE-LEGACY-TEST-ROUTE-01, then generate PACK-INTERNAL-LAYOUT-CANON-01 unless live repo state has changed.

33.13 Dominion Workbench Aide Architecture Preservation In Chat Reader

Source: docs/archive/conversations/dominium_full_conversation/previous_preservation_files/dominium_workbench_aide_architecture_preservation__09_in_chat_reader.md

Report type: in_chat_reader

Size: 1057 bytes

33.13.1 In-Chat Reader - Dominion Workbench, AIDE, Presentation Architecture, Provider Strategy, and Product-Spine Planning

This package preserves the chat as a human-readable report plus structured registers, context transfer, spec sheet, aggregator packet, and verification/audit materials.

Read first:

1. File 06 Reader Brief for a quick overview.
2. File 01 Human-Readable Report for the full story.
3. File 04 Registers for actionable IDs.
4. File 02 Context Transfer Packet when starting a new chat.
5. File 03 Spec Sheet and File 05 Aggregator Packet for merging with other reports.

Safest next actions:

- Verify live repo state.
- Confirm whether FULL-GATE-LEGACY-TEST-ROUTE-01 has been run.
- Generate PACK-INTERNAL-LAYOUT-CANON-01 if not.

- Preserve Workbench/AIDE/provider/presentation doctrines.

Question menu:

- What did we decide about Workbench?
- What replaced the old UI Editor plan?
- Which prompts were generated?
- Which tasks are next?
- What needs verification before implementation?
- Which parts should go into the master spec book?

33.14 Dominion Workbench Aide Presentation Provider Strategy Human Readable Report

Source: docs/archive/conversations/dominium_
full_conversation/previous_preservation_files/
Dominium_Workbench_AIDE_Presentation_Provider_
Strategy__01_human_readable_report.md

Report type: human_readable_report

Size: 42369 bytes

**33.14.1 COMPLETE CHAT
PRESERVATION REPORT -
Dominium Workbench, AIDE,
Presentation Spine, and Provider
Strategy**

0. Coverage and Reliability Assessment

- Row 1** Field: Chat label; Assessment: Dominium Workbench, AIDE, Presentation Spine, and Provider Strategy
- Row 2** Field: Date anchor; Assessment: 2026-05-27 Australia/Melbourne
- Row 3** Field: Source scope; Assessment: This chat only unless labelled PROJECT-CONTEXT
- Row 4** Field: Apparent access; Assessment: Partial-to-high: visible transcript context in this conversation plus the uploaded preservation instructions; no guarantee of access to every hidden/prior artifact outside this visible chat.
- Row 5** Field: Previously generated files available?; Assessment: No previous downloadable package from this chat was available in the sandbox; this response creates a new package.
- Row 6** Field: Uploaded files or artifacts present?; Assessment: Yes: the user uploaded a text file

containing the preservation-task instructions. It is cited in the final response as requested.

Row 7 Field: Contains future plans?; Assessment: Yes

Row 8 Field: Contains decisions?; Assessment: Yes

Row 9 Field: Contains pending tasks?; Assessment: Yes

Row 10 Field: Contains unresolved questions?;
Assessment: Yes

Row 11 Field: Staleness risk; Assessment: High for live repository status and current external dependencies; medium for internal doctrine; low for decisions explicitly made in this chat.

Row 12 Field: Extraction confidence; Assessment: 4/5 for visible chat substance; 3/5 for exact live repo state because this package does not re-query the repository.

Row 13 Field: Safe for later aggregation?; Assessment: Yes, with caveats: verify current repo state and distinguish FACT from PROJECT-CONTEXT and UNVERIFIED repo-status claims.

Row 14 Field: Main limitations; Assessment: The report is reconstructed from the visible chat transcript and pasted status reports. Some repo facts were pasted from other chats and should be verified before operational use.

This preservation package is based on the visible conversation plus the uploaded preservation instructions file. I can produce files in this environment, and I created a downloadable report set and ZIP. The uploaded file is the explicit preservation request and output specification for this task, not an independent evidence source about Dominion architecture. ?filecite?turn0file0?

Important limitation: many repository-state facts in this chat came from pasted status blocks or previous assistant web checks. In this preservation report, those are retained because they were part of the chat, but they are marked as FACT only when explicitly stated in the chat and marked UNVERIFIED when they would require a fresh repository check. Future operational work must verify live repository files before acting.

1. One-Page Orientation

This chat was a long-running architecture and execution-planning conversation for the Dominion / Domino project. It began around a practical UI problem: the existing Windows launcher UI looked mangled and flickered, and the user wanted a graphical internal editor that could create pixel-perfect UIs for setup, launcher, game, and other tools. That early goal became much larger. Over time, the chat moved from a Windows-only "UI Editor" plan, to a broader "Tool Editor," and finally to a more powerful and coherent concept: **Dominium**

Workbench, a cross-platform rendered, modular, command-driven, agent-aware production environment for building the whole Domino/Dominium system.

The conversation's most important shift was that Workbench stopped being "an editor" and became a userland product over the same operating-environment spine as the client, server, launcher, and setup. The user wanted all tools to reuse the actual Domino/Dominium code and data instead of becoming separate helper programs. The resulting doctrine was that CLI, TUI, rendered GUI, OS-native GUI, headless jobs, AIDE, Codex, tests, and Workbench panels should all project the same underlying command/result/refusal/document/evidence truth. UI is not authority; commands, contracts, services, providers, documents, patches, artifacts, diagnostics, and evidence are the stable center.

A large part of the chat dealt with what should be reusable and where it should live. The stable vocabulary became: **component** as source/build ownership unit; **service** as callable runtime capability; **provider** as replaceable implementation; **pack** as distributable authored payload; **module** as declared functional extension; **workspace** as user-facing Workbench composition; **app** as shipped product composition; and **artifact** as versioned persisted thing such as save, replay, pack, bundle, diagnostic, or release output. This vocabulary matters because it prevents "module" or

"tool" from becoming a junk drawer.

The chat also developed a staged execution strategy. The user and assistant converged on Foundation and AIDE gates before broad Workbench implementation. The sequence evolved as pasted repo status changed: first Foundation Lock and dependency-direction repair, then package mount, replay proof, barebones client shell, product-spine review, AIDE workflow law, WorkUnit schema, dev/main policy, checkpoint loop, capability reality ledger, presentation contract, projection conformance, and then read-only Workbench. Later, after a reported successful PRESENTATION-CONTRACT-01, the user wanted to generate targeted maintenance prompts before continuing presentation/projection work.

Another key branch of the chat involved presentation and rendering. The user wanted a single cross-platform rendered app capable of modular themes, Workbench modules, CLI/TUI/rendered/native projections, and OEM+ OS-style mimic profiles. The conclusion was that layout, controls, themes, views, and widgets should be data-described and registry-backed. Themes should alter geometry, density, palette, typography, control metrics, and procedural drawing, not product semantics. TUI should be a first-class projection, not a toy fallback. Rendered GUI should be built on a renderer-agnostic draw-list pipeline. OS-native GUI can exist as optional projection, using Visual Studio/Xcode/GTK tooling, but

it must not become canonical behavior.

Finally, the chat considered third-party acceleration. The user pasted advice about raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua. The accepted doctrine was service-first and provider-backed: use these aggressively as seed providers, but fence their types so they do not leak into engine, game, contracts, content, saves, replays, pack schemas, or public SDK. Provider choices should live in profiles; apps stay generic.

The future relevance of this chat is high. It records the architecture behind Workbench, AIDE, provider strategy, presentation contracts, self-hosting, TUI/rendered GUI unification, and near-term task order. A future assistant must understand that the project is no longer looking for a one-off UI editor. It is building a deterministic, contract-governed, modular simulation operating environment where Workbench becomes the production environment over shared law, services, providers, packs, modules, views, and evidence.

2. The Story of the Conversation

2.1 From UI Editor to Tool Editor The chat opened with a practical product-design/software-architecture task: the current Windows launcher UI was mangled and flickered. The user wanted a graphical UI creation tool to design pixel-perfect UIs and broad-

strokes functionality for setup, launcher, game, and other tools. Early questions established that the project had an in-tree DUI schema/state system using TLV, with Win32 common-control, DGMX custom-render, and null backends. The first chosen plan was a minimal Windows-only UI Editor that generated TLV directly, followed later by a more powerful Tool Editor that would edit DUI schemas/states directly and instantiate preview backends.

The assistant and user generated an 11-prompt implementation plan for this old approach, including repo scaffolding, canonical UI IR, TLV I/O, capability validation, layout engine, splitter/tabs/scroll widgets, action codegen, Phase A UI Editor, Tool Editor bootstrap, flicker fixes, and hardening. That plan then expanded into six UI Editor extension prompts covering discovery/import/export, CLI modes, ops.json scripting, Minecraft-style logical launcher/setup layouts, setup layout, and hardening. The user clarified that "Minecraft-style" meant logical layout using native Win32 controls, not graphical skinning.

2.2 Abandoning UI Editor / Tool Editor as final products The user then redirected the plan. They concluded that the old UI Editor and Tool Editor were good general ideas but bad final products. Instead, they wanted cross-platform tools using the same CLI, TUI,

and rendered GUI as the client, not OS-native SDK widgets. This led to the idea of Dominion Workbench: a rendered modular tools host and production environment over the same runtime architecture as the client.

The assistant agreed and proposed an integrated Workbench shell with focused modules rather than one monolithic editor. The user and assistant refined this into "Dominium Workbench Platform," a modular userland operating environment for inspecting, editing, validating, simulating, previewing, packaging, and proving Dominion systems. Workbench would not own semantics; it would host capability-scoped modules over shared commands, documents, packages, validation, render, diagnostics, and replay services.

2.3 Generalizing Workbench into the project production environment The user emphasized that Workbench was not merely an inspector or command dashboard. As sole developer, they wanted to use it to build the entire Domino/Dominium project: code, data, packs, UI, graphics, sounds, tests, releases, and agent work. The plan expanded to include Command OS, Creation Studio, Evidence Notebook, System Graph, Preview/Sandbox, Agent Control Board, and Release Forge. Workbench became the "visual and agentic production environment" for Domino/Dominium.

A strong rule emerged: every GUI/TUI action should

map to a command; every edit should be a patch; every operation should produce diagnostics/evidence. This enables human GUI work, CLI work, TUI work, headless work, and Codex/AIDE work to converge instead of drifting into separate systems.

2.4 Modular UI, themes, OEM+ style, TUI, and rendering The user wanted modular layouts, controls, themes, styles, views, and OEM+ native-style rendered theme profiles. The conversation concluded that UI should be separated into view model, view document, style/theme tokens, runtime UI components, and renderer backend. Layouts should be data-driven; controls should be semantic; themes should be token overlays; views should project typed documents/results; controls/widgets should have projection implementations for CLI/TUI/rendered/native.

The user supplied mockups and TUI design advice. The TUI doctrine became: terminal-native projection of the same operating environment, not a separate ASCII version. It should support command palette, module launcher, project tree, inspector, diagnostics, logs, validation reports, pack browser, setup wizard, server console, client tactical fallback, and agent work board. The TUI should be fast, exact, trustworthy, and always available.

Rendered GUI would use software renderer first,

hardware later, and could support primitive-only theme families such as Barebones, Instrument, Blueprint, Paper, Phosphor, Tactical, High Contrast, and TUI-rendered hybrid. Rich assets, exact OS mimicry, audio, fonts, textures, and advanced icons could be optional packs.

2.5 Engineering law and reusable architecture

The user then asked about portability, modularity, extensibility, replacing directories and code, and building like a proper game/OS rather than a one-off project.

The assistant proposed an engineering constitution:

Domino as reusable substrate, Dominium as product family, every boundary contract-defined, versioned, capability-checked, test-proven, and replaceable.

Important practices included stable C-compatible ABI, service interfaces, provider contracts, module/pack tiers, generated artifact policy, dependency-direction validators, schema/protocol versioning, deterministic data formats, structured refusals, and compatibility corpus.

This led to the vocabulary split: component, service, provider, pack, module, workspace, app, artifact.

Workbench modules were framed as one consumer of the general module system, not the general module system itself.

2.6 AIDE, queues, status, and prompt sequencing

A major segment focused on AIDE and Codex workflow. The project had a queue of tasks such as Foundation Lock, Package Mount Slice, Replay Proof Slice, Barebones Client Shell, Product Spine Review, AIDE Workflow Law, WorkUnit Schema, Dev/Main Policy, Checkpoint Loop, Capability Reality Ledger, Presentation Contract, Projection Conformance, and later Workbench/Explorer tasks.

The user pasted status reports indicating tasks completed and queue corrections needed. The assistant generated detailed Codex/AIDE prompts for STATUS-RECONCILE-02, AIDE-WORKFLOW-LAW-01, AIDE-WORKUNIT-SCHEMA-01, AIDE-DEV-MAIN-POLICY-01, AIDE-CHECKPOINT-LOOP-01, AIDE-CAPABILITY-REALITY-LEDGER-01, and FULL-GATE-LEGACY-TEST-ROUTE-01. These prompts emphasized allowed paths, forbidden paths, non-goals, validation, blocker handling, dirty worktree handling, and final response format.

2.7 Providers, raylib, SDL2, Lua, and future

visual progress The user pasted later advice about using raylib ecosystem libraries, SDL2, and Lua. The final doctrine was to use raylib aggressively as a seed provider suite but only behind Dominion service contracts. `r1sw` is a raylib ecosystem software provider, not the Dominion canonical software renderer. `r1gl` is a

raylib GL-abstraction provider, not the final OpenGL law. Raygui is early Workbench/debug UI provider, not UI law. SDL2 remains useful for host/input/audio. Lua can be a pinned script provider but must not define the mod ABI.

2.8 Final state of the chat At the end, the user uploaded a preservation task requesting a complete chat preservation report, registers, spec sheet, aggregator packet, self-audit, and downloadable files. This report and package are the response to that final request.

3. Main Topics Discussed

Topic 1 - Launcher UI problem and original UI Editor plan The chat began with the launcher UI looking mangled and flickering. The early solution was a Windows-only Phase A UI Editor, followed by a more ambitious Tool Editor. The UI Editor would edit existing DUI/TLV definitions, use Win32 native controls for preview, and produce canonical TLV plus generated action stubs. This mattered because launcher/setup UI was an immediate product pain and a good test of the existing DUI system.

Conclusion: the UI Editor plan was useful as a bootstrapping concept but later superseded as a final product.

Topic 2 - DUI/TLV, schema/state, and UI

runtime architecture The chat established that the existing launcher used a DUI schema/state system with TLV and backends for Win32 common controls, DGFX custom rendering, and null/headless. Early architecture options revolved around whether to extend DUI directly, introduce an authoring IR, or build a minimal direct-TLV editor. The user accepted a canonical edit model and deterministic TLV/JSON output before later abandoning the old editor product path.

Conclusion: the useful pieces remain: canonical UI documents, action bindings, validation, import/export, CLI scripting, preview, layout engine, and codegen. They become Workbench modules/services rather than a standalone UI Editor.

Topic 3 - Workbench as production environment

The conversation's largest conceptual shift was from "editor" to "Workbench." The user wanted one rendered cross-platform app using the same client/runtime code, capable of building and testing Domino/Dominium artifacts. Workbench became a modular production environment with command palette, module registry, project graph, validation, agent board, pack browser, UI/HUD sandbox, renderer sandbox, release forge, and future authoring tools.

Conclusion: Workbench is not authority; it operates the

system. It should expose commands, documents, patches, diagnostics, evidence, modules, packs, providers, and workspaces.

Topic 4 - Unified

CLI/TUI/rendered/native/headless presentation

The chat repeatedly refined a presentation doctrine: CLI, TUI, rendered GUI, native GUI, headless reports, Workbench panels, tests, and AIDE workflows should all project the same command/result/refusal/document/evidence truth. OS-native GUI may be optional, built via Visual Studio/Xcode/GTK paths, but not canonical.

Conclusion: presentation is a projection layer. This led to PRESENTATION-CONTRACT-01 and PROJECTION-CONFORMANCE-01 as essential next tasks.

Topic 5 - TUI as first-class projection The user pasted detailed TUI design advice. The TUI should be a deterministic, keyboard-first, low-dependency projection suitable for servers, setup recovery, CI, SSH, accessibility, debugging, AIDE/Codex workflows, and users who prefer terminals. It should include panels, split panes, tables, trees, command input, logs, status bars, and degraded ASCII fallback.

Conclusion: TUI is not second-class. It should share the same semantic view/action model as CLI and rendered GUI.

Topic 6 - Themes, OEM+ style, and code-only UI richness The user wanted a rendered GUI capable of mimicking Windows, macOS, and Linux styles through OEM+ theme profiles. The discussion established that many useful styles can be generated code-only through primitives: fills, borders, lines, patterns, gradients, text, procedural glyphs, and layout metrics. Richer fidelity can come later through packs/assets.

Conclusion: themes are data-packable token/control-skin profiles. They change appearance and metrics, not product semantics.

Topic 7 - Engineering law for portability, modularity, and replaceability The user asked how to build the project like a serious game/OS rather than a one-off indie project. The response developed broad coding/architecture practices: stable ABI, contracts, versioned schemas, service/provider boundaries, dependency-direction validation, modular packs/modules, patch/transaction systems, structured diagnostics/refusals, generated artifact policy, compatibility corpus, and conformance tests.

Conclusion: future-proofing depends more on contract-governed boundaries than on folders alone.

Topic 8 - Module, pack, service, provider, app vocabulary The conversation settled on strict

terminology. Component is source/build ownership; service is callable runtime capability; provider is replaceable implementation; pack is authored distributable payload; module is declared functional extension; workspace is Workbench composition; app is shipped product composition; artifact is persisted versioned output/input.

Conclusion: this vocabulary is central and should be preserved.

Topic 9 - AIDE/Codex workflow and parallel development A large part of the chat focused on AIDE as scheduler/ledger/gatekeeper and Codex as bounded patch executor. The user wanted to run parallel Codex tasks on `/dev` while keeping `/main` evidence-blocked. The assistant generated prompts for AIDE workflow law, WorkUnit schemas, dev/main policy, checkpoint loop, and capability reality ledger.

Conclusion: development is non-blocking; promotion is evidence-blocked. Large parallelism requires AIDE workflow controls.

Topic 10 - Product spine, replay proof, barebones client, and Universe Explorer As task state evolved, the plan moved from product-spine slices toward presentation/projection and eventually a read-only Workbench/Client Universe Explorer. The explorer is a

north-star product slice: a lawful read-only universe inspection/navigation surface proving reference frames, no modal loading, streaming/degradation, sparse materialization, and provenance before gameplay or embodiment.

Conclusion: Universe Explorer is valid as a future product-spine direction, but must enter through contracts, headless proof, and read-only projection, not broad renderer/gameplay work.

Topic 11 - Raylib/SDL2/Lua provider strategy

Later discussion refined use of raylib, rlsw, rlgl, raygui, raudio, SDL2, and Lua. The final doctrine is service-first/provider-backed/profile-selected/third-party-fenced/evidence-tested. Raylib can accelerate visible progress but must not define architecture.

Conclusion: third-party libraries are seed providers, not law.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted to solve the damaged launcher UI, build UI/tools capable of visually designing interfaces and functionality, support Windows/Linux/macOS cross-platform tools, reuse client/runtime code in tools, create modular and extensible rendered GUI/TUI/CLI surfaces, and use

Workbench to build the entire Domino/Dominium project. Later explicit goals included preserving the entire chat, producing a handoff package, and generating bounded Codex prompts in correct sequence.

4.2 Inferred Goals INFERENCE: The user is trying to prevent architecture drift and avoid a future in which the project becomes a collection of disconnected tools, UIs, and one-off scripts. The repeated emphasis on portability, modularity, replaceability, and reuse suggests a desire for a professional, long-lived platform architecture where every subsystem can be tested, replaced, and reused for other games or engine projects.

4.3 Goals That Changed Over Time The goal changed from a Windows UI Editor to a full Workbench platform. The early goal was immediate UI repair and graphical editing. The later goal became a cross-platform rendered production environment, agent-control surface, module/pack/app composer, and self-hosting authoring tool. Another shift was from OS-native widgets to rendered, provider-backed UI, with native SDK GUIs optional projections.

4.4 Goals Still Unresolved The Workbench is not implemented. Presentation/projection conformance is only beginning. Provider runtime, package runtime,

module loader, renderer implementation, native GUI, Universe Explorer, gameplay, and self-hosting authoring remain future work. Repo live state must always be verified before assuming a queued task is done.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Abandon old UI Editor/Tool Editor as final products; recycle ideas into Workbench modules.; Status: Accepted; Why it mattered: Avoid one-off Windows/native editor architecture.; Confidence: High; Label: FACT

Row 2 ID: DECISION-02; Decision: Dominion Workbench is a modular production environment, not authority.; Status: Accepted; Why it mattered: Prevents GUI/tool drift; keeps contracts/commands/services central.; Confidence: High; Label: FACT

Row 3 ID: DECISION-03; Decision: CLI/TUI/rendered/native/headless are projections of one semantic spine.; Status: Accepted; Why it mattered: Enables reuse across apps, tests, Workbench, AIDE, and products.; Confidence: High; Label: FACT

Row 4 ID: DECISION-04; Decision: Use progressive self-hosting.; Status: Accepted; Why it mattered:

Avoids circular bootstrap.; Confidence: High;
Label: FACT

Row 5 ID: DECISION-05; Decision: Use service/
provider/profile vocabulary and strict module/
pack/app terms.; Status: Accepted; Why it
mattered: Prevents taxonomy collapse.;
Confidence: High; Label: FACT

Row 6 ID: DECISION-06; Decision: Use AIDE as
governor/ledger/scheduler and Codex as bounded
patch executor.; Status: Accepted; Why it
mattered: Enables parallel dev without corrupting
main.; Confidence: High; Label: FACT

Row 7 ID: DECISION-07; Decision: Use raylib/SDL2/
Lua as providers, not architecture.; Status:
Accepted conceptually; Why it mattered:
Accelerates visible progress without lock-in.;
Confidence: Medium-High; Label: FACT/
UNVERIFIED for implementation

Row 8 ID: DECISION-08; Decision: Stop broad
structure cleanup once canonical structure passes;
use targeted maintenance lanes.; Status: Accepted
conceptually; Why it mattered: Avoids churn after
structure became credible.; Confidence: Medium-
High; Label: FACT/PROJECT-CONTEXT

Row 9 ID: DECISION-09; Decision: Universe Explorer
is a read-only/headless proof before visual/

gameplay.; Status: Accepted conceptually; Why it mattered: Proves scale/refinement/reference frames without broad gameplay.; Confidence: Medium; Label: FACT

Row 10 ID: DECISION-10; Decision: Full CTest remains full-gate/T4 debt, not normal prompt gate.; Status: Accepted from pasted status; Why it mattered: Keeps dev loop practical.; Confidence: Medium; Label: PROJECT-CONTEXT/
UNVERIFIED

DECISION-01 - Abandon old UI Editor / Tool Editor as final products The user explicitly said the old UI Editor and Tool Editor were good general ideas but bad actual final products, and should be abandoned and recycled. The retained pieces are UI documents, CLI scripting, import/export, validation, preview, codegen, layout, theme, and property editing. These become Workbench modules and services. Alternative considered: continue with a Windows-only UI Editor followed by full Tool Editor. Revisit only if a tiny one-off tool is needed as a temporary bridge.

DECISION-02 - Workbench is production environment, not authority Workbench should operate the system, not define truth. This decision protects the project from GUI-only behavior and allows

CLI/TUI/headless/AIDE to use the same commands and results. Workbench edits artifacts through documents/patches/transactions.

DECISION-03 - Unified projection model CLI, TUI, rendered GUI, native GUI, headless reports, Workbench panels, tests, and AIDE workflows should all present typed results/documents/refusals/evidence from one command spine. This supports reuse and prevents duplicated product logic.

DECISION-04 - Progressive self-hosting Workbench should not build itself from nothing. It starts from hand-coded seed substrate, then edits safe artifacts, then its own themes/layouts, then modules/apps/packs, and later orchestrates AIDE/Codex code work.

DECISION-05 - Strict vocabulary split The terms component/service/provider/pack/module/workspace/app/artifact were adopted to avoid future junk drawers. This affects directory structure, manifests, contracts, and Workbench UX.

DECISION-06 - AIDE governs, Codex executes AIDE should manage task packets, context, policies, evidence, blockers, branch roles, dev/main promotion, and repair/resume work. Codex executes bounded patch

tasks. This enables parallel development while keeping main evidence-blocked.

DECISION-07 - Third-party libraries as providers Raylib, SDL2, Lua, and related libraries are accelerators. The project should use them aggressively but fence their types behind Dominion service contracts and provider manifests.

DECISION-08 - Stop broad structure cleanup after structure is credible Once the active structure passes hard gates, broad cleanup should stop. Remaining work should be targeted: full-gate legacy test routing, pack layout canon, residual taxonomy, ABI promotion, storage/package/provider split.

DECISION-09 - Universe Explorer as read-only proof path Universe Explorer is a future north-star product slice but should begin as read-only/headless inspection. It must not become gameplay or renderer authority.

DECISION-10 - Full CTest as full-gate debt The chat repeatedly treated fast strict and targeted validators as normal task gates, while full CTest remains release/full-gate debt unless explicitly run and passed.

This makes Codex throughput practical while preserving release rigor.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

The Windows-only UI Editor as final product was superseded. It is still useful as a historical bridge idea, but final architecture should be Workbench modules over shared services. A native-widget-first Tool Editor was also superseded because the user wants rendered cross-platform tools using client/runtime code. OS-native GUI design is not abandoned; it becomes optional projection using Visual Studio/Xcode/GTK-style tooling.

A monolithic everything editor was rejected. The accepted form is one integrated Workbench shell built from small reusable modules, services, commands, documents, views, and workspaces.

A raylib-shaped architecture was rejected. Raylib is useful, but source structure must be service-first: `runtime/<service>/providers/<provider>`, not `runtime/raylib/<service>`. App variants like `apps/client/rendered/raylib` were rejected as long-term architecture.

Broad structure cleanup was deprioritised after the active structure reportedly passed hard gate checks. Remaining cleanup should be targeted maintenance.

Pure C89/C++98 was superseded by a C17/C++17 baseline in pasted repo status. This must be verified in live repo before operational use.

Broad gameplay, renderer implementation, native GUI, module loader, provider runtime, release publication, and full Workbench UI were repeatedly deferred until appropriate gates.

7. Important Reasoning, Rationale, and Tradeoffs

The dominant tradeoff was speed versus architectural durability. The user wants rapid progress using Codex/AIDE and third-party libraries, but also wants the project to be reusable, replaceable, deterministic, and professional. The compromise is narrow vertical slices, provider-backed acceleration, and evidence-gated promotion.

Another tradeoff was GUI richness versus semantic safety. Workbench should eventually be visual and powerful, but if it directly mutates files or calls private tools, it will become a second authority. The adopted solution is document/patch/transaction flow.

The UI discussion balanced OS-native familiarity with cross-platform rendered control. Native SDK GUIs remain useful, but the core Workbench/client UI should be rendered and service-backed to support themes, TUI parity, and modding.

Parallel development was treated as desirable but dangerous. The visible rationale was that `/dev` can accumulate classified partial progress, but `/main` must remain evidence-backed. This leads to task branches, repair branches, checkpoint branches, and promotion decisions.

The third-party provider discussion balanced visible progress against lock-in. Raylib/SDL/Lua are valuable because they provide quick windows, rendering, input, audio, assets, scripting, and UI; but their types must not enter stable contracts or deterministic truth.

8. Plans, Future Work, and Next Steps

The latest immediate sequence in this chat ended with a generated `FULL-GATE-LEGACY-TEST-ROUTE-01` prompt. The user had just reported `PRESENTATION-CONTRACT-01` complete and wanted to generate a maintenance batch before replanning.

Recommended short-term sequence from the chat:

1. Generate and run maintenance prompts:

- `FULL-GATE-LEGACY-TEST-ROUTE-01`
- `PACK-INTERNAL-LAYOUT-CANON-01`
- `RUNTIME-ENGINE-RESIDUAL-TAXONOMY-01`

- PUBLIC-HEADER-ABI-PROMOTION-01
- STORAGE-PACKAGE-PROVIDER-SPLIT-01
- POINTER-WIDTH-SERIALIZATION-AUDIT-01

1. Replan after the maintenance batch.
2. Continue mainline product/presentation sequence:

- PROJECTION-CONFORMANCE-01
- ACCESSIBILITY-CONTRACT-01
- TEXT-LOCALIZATION-CONTRACT-01
- WORKBENCH-SHELL-READONLY-01
- UNIVERSE-EXPLORER-CONTRACT-01
- UNIVERSE-EXPLORER-HEADLESS-01
- PROVIDER-MANIFEST-WEDGE-01
- SERVICE-CONTRACT-WEDGE-01
- THIRD-PARTY-FENCE-01
- RAYLIB-PROVIDER-WEDGE-01
- UNIVERSE-EXPLORER-VISUAL-01

Each task should be narrow, with allowed/forbidden paths, non-goals, validation, blocker handling, and final

response format. Broad Workbench UI, renderer/gameplay/native GUI, provider runtime, package runtime, module loader, and release publication remain blocked until gates authorize them.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences The user prefers direct, source-grounded, audit-ready answers with explicit uncertainty. They want decisions preserved, rejected ideas recorded, and tentative items not promoted to final decisions. They repeatedly asked for copy-paste-ready prompts in single code blocks. They want the project to be portable, modular, extensible, deterministic, replaceable, and reusable for other games or engines.

9.2 Inferred Constraints and Preferences

INFERENCE: The user values architectural continuity and dislikes repeating settled debates. They prefer long, complete handoffs over terse summaries. They want future chats to continue from current queue status and verify live repo state before acting.

9.3 Uncertain or Unestablished Preferences

UNCERTAIN: Exact preferred path names may change as the repo evolves. Live repository queue state may

have advanced after this preservation package. The actual CMake language baseline should be verified.

10. Files, Artifacts, Outputs, and Prompts

Important artifacts from this chat include the generated prompt series for the old UI Editor plan, the UU1-UU6 UI Editor extension prompts, the final UI Editor prompt with import/CLI/Minecraft tests, many Workbench/AIDE/product-spine prompt blocks, and the final generated FULL-GATE-LEGACY-TEST-ROUTE-01 prompt.

The user uploaded a text file containing the preservation instruction set for this task. This response creates a new preservation package with Markdown, YAML, and ZIP outputs.

No previous downloadable package from this chat was available in the sandbox at the time of this response. The present response creates:

- manifest
- human-readable report
- context transfer packet
- spec sheet
- structured registers
- aggregator packet

- reader brief
- verification/audit file
- future chat bootstrap prompt
- in-chat reader
- ZIP bundle

11. Open Questions and Unresolved Issues

The biggest unresolved issue is live repo state: future work must verify whether the generated prompts have been run and whether the queue has advanced. Another unresolved issue is the exact implementation order after the maintenance batch. The user wants to run maintenance prompts first, then replan.

Technical open questions include how soon provider manifests should be implemented relative to projection conformance, how to split raylib provider wedge into manifest/service/fence/implementation tasks, which Lua version to pin, whether `external/upstream` or `external/vendor` matches repo convention, and how full-gate tests should be routed without weakening active structure gates.

Workbench remains unresolved at the implementation level: read-only first is agreed, but exact UI shell design, renderer path, modules, and self-hosting milestones remain future tasks.

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

A future assistant might restart the UI Editor/Tool Editor plan as if it were still current. It might treat Workbench as authority or a monolithic editor. It might claim raylib/SDL/Lua implementation support when only provider doctrine exists. It might assume pasted repo status is current without checking live files. It might run broad structure cleanup after the structure is already clean enough. It might treat full CTest debt as normal prompt blocker or, conversely, claim full CTest is green without evidence.

The main mitigation is to verify live repo state, preserve labels, and keep using bounded prompts with allowed paths, forbidden paths, non-goals, validation, blocker handling, and explicit final response format.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat should feed several chapters of a future Project Spec Book: Workbench Platform, AIDE/Codex Workflow, Presentation Architecture, UI/TUI/Rendered GUI Systems, Provider Architecture, Engineering Constitution, Module/Pack/App Vocabulary, Progressive Self-Hosting, TUI Doctrine, Theme/OEM+ Style System, and Product-Spine Execution Plan.

It contributes especially valuable context about why the project moved away from a one-off UI Editor and toward a unified production environment. It also provides concrete prompt templates and task sequencing that can be reused.

Before merging into a master spec, all repo-status claims should be verified. The doctrine is reusable; the live task status is perishable.

14. What I Should Remember

- The old UI Editor / Tool Editor plan is superseded as final product, but its ideas become Workbench modules/services.
- Workbench is the production environment over the system, not authority.
- CLI, TUI, rendered GUI, OS-native GUI, headless, Workbench, AIDE, and tests should all project the same command/result/refusal/document/evidence truth.
- Development is non-blocking; promotion is evidence-blocked.
- AIDE governs queues, blockers, work units, evidence, checkpoints, and dev/main policy; Codex executes bounded patches.

- Themes, layouts, controls, widgets, and views should be modular and data-described.
- TUI is a first-class projection.
- Raylib/SDL2/Lua are providers, not architecture.
- Full CTest remains full-gate debt unless verified otherwise.
- Broad Workbench UI, renderer, native GUI, gameplay, provider runtime, package runtime, module loader, and release publication remain blocked until gates authorize them.
- The last generated prompt in this chat was FULL-GATE-LEGACY-TEST-ROUTE-01.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- "Explain again why we abandoned the old UI Editor / Tool Editor path."
- "What is the difference between Workbench, AIDE, and Codex?"

15.2 Decisions

- "Which decisions from this chat are final versus tentative?"

- "Which decisions depend on live repo verification?"

15.3 Tasks and Next Actions

- "Generate PACK-INTERNAL-LAYOUT-CANON-01."
- "Generate the next maintenance prompt after FULL-GATE-LEGACY-TEST-ROUTE-01."
- "Replan after the maintenance batch."

15.4 Artifacts and Files

- "List every Codex prompt generated in this chat."
- "Which artifacts from this package should feed a master Project Spec Book?"

15.5 Risks and Verification

- "What repo-state claims from this chat need fresh verification?"
- "What could a future assistant misunderstand most badly?"

15.6 Future Spec Book / Aggregation

- "Convert this preservation report into formal spec-book chapters."

- "Compare this chat's doctrine against another old chat report."

15.7 Deep-Dive Questions Specific to This Chat

- "Deep-dive the presentation/projection architecture."
- "Deep-dive the provider strategy for raylib/SDL2/Lua."
- "Deep-dive the progressive self-hosting ladder."

16. Compact Human Summary

This chat began with a practical UI problem: the Dominion launcher UI was broken-looking and flickery. The original plan was to build a Windows-only UI Editor that could visually design DUI/TLV UIs, then later a full Tool Editor. That early plan became a detailed prompt sequence, including schema/IR, TLV I/O, layout, action codegen, import/export, CLI scripting, and launcher/setup redesign tests. But the user later changed direction decisively: the old UI Editor and Tool Editor were not the right final products.

The better plan became Dominion Workbench: a cross-platform rendered, modular, command-driven, agent-aware production environment that uses the same runtime, renderer, UI, command, package, diagnostics,

evidence, and provider systems as the client. Workbench is not a separate GUI framework and not authority. It is the richest human/agent interface over the same contracts and services used by CLI, TUI, headless, native GUI, setup, launcher, server, client, and tests.

A central doctrine emerged: everything meaningful should go through commands, typed results, refusals, diagnostics, documents, patches, services, providers, packs, modules, artifacts, views, and projections. GUI actions should map to commands. Edits should become document patches. Results should produce evidence. AIDE governs task state and proof; Codex executes bounded patches. Development can move quickly on dev/task branches, but promotion to main is evidence-blocked.

The chat also developed a strong UI architecture. CLI, TUI, rendered GUI, OS-native GUI, and headless output are projections of the same semantic view model. TUI is first-class, not a toy fallback. Rendered UI should be data-described through layouts, controls, themes, style tokens, widgets, and workspace documents. Themes can be code-only at first, using rectangles, lines, text, hatching, gradients, and procedural glyphs. Rich assets, textures, fonts, sounds, and exact OS-mimic profiles can come later through packs.

The conversation established a reusable vocabulary: component, service, provider, pack, module, workspace,

app, artifact. This distinction prevents architectural junk drawers. Workbench modules are not the general module system; they are one consumer of it.

The repo/task planning thread evolved through many reported states. Foundation Lock passed with warnings; package mount, replay proof, barebones client, product-spine review, AIDE workflow law, WorkUnit schema, dev/main policy, checkpoint loop, capability reality ledger, and presentation contract were discussed as completed or queued at different points. Because repo status changes rapidly, future assistants must verify live files before assuming current status. The last user-provided status said **PRESENTATION-CONTRACT-01** completed, and the user wanted to generate maintenance prompts first: full-gate legacy test routing, pack internal layout canon, runtime/engine residual taxonomy, public header ABI promotion, storage/package provider split, and pointer-width serialization audit. The assistant generated **FULL-GATE-LEGACY-TEST-ROUTE-01**.

A later strategic thread established how to use third-party libraries. Raylib, rlgl, rlsw, raygui, raudio, SDL2, and Lua should be used aggressively as seed providers, but not allowed to define architecture. Dominion owns service contracts, data formats, commands, views, saves, replays, packs, modules, and asset identity. Providers implement replaceable backends. Profiles select providers. Apps remain generic.

The most important next action inside this old chat is to continue generating the requested maintenance prompts after **FULL-GATE-LEGACY-TEST-ROUTE-01**, or to replan after that maintenance batch. The most important preservation requirement is to carry forward the doctrine: Workbench designs artifacts; runtime executes artifacts; contracts constrain artifacts; content packages artifacts; apps consume artifacts; tests prove artifacts; AIDE governs changes; Codex patches code.

33.15 Dominion Workbench Aide Presentation Provider Strategy Reader Brief

Source: docs/archive/conversations/dominium_full_conversation/previous_preservation_files/Dominium_Workbench_AIDE_Presentation_Provider_Strategy__06_reader_brief.md

Report type: reader_brief

Size: 1585 bytes

33.15.1 Reader Brief - Dominion Workbench, AIDE, Presentation Spine, and Provider Strategy

This chat records the transition from an old UI Editor/
Tool Editor plan to the Dominion Workbench/AIDE/

presentation/provider architecture.

Top 20 things to know

1. Old UI Editor / Tool Editor are superseded as final products.
2. Their useful pieces become Workbench modules and services.
3. Workbench is a production environment over the system, not authority.
4. CLI, TUI, rendered GUI, native GUI, headless, tests, AIDE, and Workbench should share command/result/refusal/document/evidence truth.
5. TUI is first-class.
6. Rendered GUI is cross-platform and data-described.
7. OS-native GUI is optional projection.
8. Themes are token/control-skin profiles, not semantics.
9. AIDE governs; Codex patches.
10. Development can be non-blocking; promotion is evidence-blocked.
11. Workbench self-hosting is progressive.

12. Modules are declared functional extensions, not source folders.
13. Packs are distributable authored payloads.
14. Providers implement services and are replaceable.
15. Raylib/SDL2/Lua are seed providers, not architecture.
16. Full CTest remains full-gate debt unless verified.
17. Broad Workbench/renderer/gameplay remains blocked until gates authorize.
18. Universe Explorer is a future read-only/headless proof before gameplay.
19. The last generated prompt was FULL-GATE-LEGACY-TEST-ROUTE-01.
20. Future work must verify live repo queue before acting.

Best next step: verify live repo state, then continue the maintenance prompt sequence or replan if repo advanced.

33.16 Dominion Workbench Aide Presentation Provider Strategy In Chat Reader

Source: docs/archive/conversations/dominium_full_conversation/previous_preservation_files/Dominium_Workbench_AIDE_Presentation_Provider_Strategy__09_in_chat_reader.md

Report type: in_chat_reader

Size: 1119 bytes

33.16.1 In-Chat Reader - Dominion Workbench, AIDE, Presentation Spine, and Provider Strategy

Package overview

This package preserves the chat's substance, decisions, tasks, constraints, artifacts, risks, open questions, and future handoff material. The main report is human-readable. Registers and YAML are for aggregation.

File index

- 00_manifest.md: package contents and caveats.
- 01_human_readable_report.md: sections 0-16 narrative report.

- `02_context_transfer_packet.md`: section 29 handoff.
- `03_spec_sheet.yaml`: structured YAML-style spec sheet.
- `04_registers.md`: sections 17-28 registers.
- `05_aggregator_packet.md`: compact merge packet.
- `06_reader_brief.md`: short human guide.
- `07_verification_and_audit.md`: self-audit and verification queue.
- `08_future_chat_bootstrap_prompt.md`: standalone future-chat prompt.
- `09_in_chat_reader.md`: this guide.

Question menu

Ask: "What did we decide about Workbench?", "What was superseded?", "What prompt should I generate next?", "What repo facts need verification?", or "Turn this into a spec-book chapter."

Chapter 34

Dominium Setup

34.1 Dominium Setup Handoff Reader Brief

Source: docs/archive/conversations/dominium_setup/dominium_setup_handoff__05_reader_brief.md

Report type: reader_brief

Size: 4737 bytes

34.1.1 Reader Brief - Dominium Setup Handoff

What This Chat Was About

This chat was primarily about the Dominium / Domino Setup system: a deterministic, auditable, reversible installer/updater/verifier/rollback authority. It began with cross-platform setup architecture and directory planning, then shifted to canonical repository alignment, build enforcement, engine purity, schema handoff, application-layer canon, and finally context transfer/report packaging.

The most important outcome is that setup is the only component allowed to install files, modify installation/system state, verify/repair installations, perform rollback, and enforce package layout. Launcher may invoke setup and read its outputs, but may not reimplement setup logic. Engine must not know setup exists.

Earlier proposed layouts using `setup/adapters`, `adapter-local` packaging, and `core/source` were superseded. The canonical setup layout is `setup/core/{fetch,verify,install,rollback}`, `setup/include/{dsk,dsu}`, `setup/packages`, `setup/platform`, `setup/tests`, and `setup/ui`.

The user stated that two large Codex prompts had already been applied: one for VS2026/CMake/build repair and one for engine purity/contract ownership. A further prompt was generated in this chat to finish setup code/docs/spec alignment. Actual repo state after those prompts was not verified in this chat.

Most Important Things to Know

- Setup is sole installation mutation authority.
- Launcher invokes setup; launcher does not repair/install directly.
- Engine must remain pure and unaware of setup.

- Setup depends only on `libs/` and `schema/`.
- Public setup APIs live in `setup/include/dsk` and `setup/include/dsu`.
- Packaging belongs under `setup/packages/`.
- Cross-product contracts belong in `libs/contracts/include/dom_contracts`.
- CMake is authoritative; IDE projects are generated, not source of truth.
- Use C/C++ for setup execution, not .NET.
- CLI is canonical; GUI/TUI are views.
- UI is declarative data, not business logic.
- RepoX/TestX and BUILD-ID-0 govern release metadata and validation.
- Existing implementation is authoritative unless it violates locked rules.
- Do not delete code; quarantine under `legacy/`.
- Actual repo state must be verified before acting.

Active Plans or Workstreams

- WORKSTREAM-01: Setup / Installer System

- WORKSTREAM-02: Canonical Repository and Build Enforcement
- WORKSTREAM-03: Engine Purity and Contract Ownership
- WORKSTREAM-04: Setup Schemas and Launcher Handoff
- WORKSTREAM-05: Application Layer Canon
- WORKSTREAM-06: Packaging/Reproducibility/Offline-Online Install
- WORKSTREAM-07: UI Frontend and Command Graph
- WORKSTREAM-08: RepoX/TestX/BUILD-ID Governance
- WORKSTREAM-09: Legacy Platform Support
- WORKSTREAM-10: Context Transfer/Report Packaging

Decisions Already Made

- Setup sole authority.
- Canonical setup directory layout.
- Setup depends only on libs/schema.

- Launcher reads setup outputs and invokes setup only.
- Engine purity and neutral contracts are mandatory.
- CMake is authoritative.
- CLI canonical, UI as data.
- Repair/update/downgrade are explicit, not hidden.
- Rollback is first-class and journaled.
- Existing implementation must be audited, not redesigned.

Pending Tasks

- Verify actual repo state.
- Run CMake configure/build/test.
- Run setup and launcher CLI smoke tests.
- Execute or audit the generated setup hardening prompt.
- Verify schema/setup files.
- Verify libs/contracts.
- Verify engine purity enforcement scripts.
- Audit setup platform/UI/packages/tests/docs.

Open Questions

- Did the applied Codex prompts fully succeed?
- Has the setup hardening prompt been executed?
- What is the current setup/ tree?
- What setup schemas exist?
- Does libs/contracts compile?
- Are setup/launcher CLIs runnable?
- Is app command spine implemented?
- Where exactly does UI IR live?

Files / Artifacts / Prompts to Preserve

- ARTIFACT-11 canonical repo/setup prompt.
- ARTIFACT-14 applied VS2026/CMake refactor prompt.
- ARTIFACT-15 applied engine purity/contract prompt.
- ARTIFACT-17 generated setup hardening prompt.
- ARTIFACT-19/20 application/project canon prompts.
- ARTIFACT-22 previous Context Transfer Packet.
- This report package.

What to Verify Before Acting

- `tree /f setup`
- `tree /f schema`
- `tree /f libs\contracts`
- `root CMakeLists.txt`
- `setup/CMakeLists.txt`
- `CMakePresets.json`
- `cmake --preset vs2026-x64-debug`
- `cmake --build --preset vs2026-x64-debug`
- `ctest --preset vs2026-x64-debug`
- `setup --help`
- `launcher --help`

Best Next Step

Verify actual repository state, then execute or audit the generated "Dominium Setup: Final Spec Alignment, Gap Closure, and Hardening Pass" prompt.

Chapter 35

Domino Engine Refactor Prompts

35.1 Dominium Domino Engine Refactor Prompts Reader Brief

Source: docs/archive/conversations/domino_engine_refactor_prompts/dominium_domino_engine_refactor_prompts__05_reader_brief.md

Report type: reader_brief

Size: 5362 bytes

35.1.1 Reader Brief - Dominium/Domino Engine Refactor Prompts

What This Chat Was About

This chat designed and organized a major deterministic-engine refactor for Dominium/Domino. The user established that the Domino engine must remain ISO C89, deterministic, fixed-point where determinism matters, free of OS/UI/rendering/platform APIs, and free of hardcoded gameplay semantics. Dominium

remains portable C++98 UI/tools only. The chat started with advanced simulation systems, moved into agents/life/ecology, generalized performance and extensibility methods across the whole engine, added heavy infrastructure/building optimization, generalized Interstellar/Wargames DLC needs into core primitives, integrated a sister-chat prompt requiring arbitrary 3D placement and TRANS/STRUCT/DECOR systems, and finally produced a 14-prompt Codex implementation roadmap plus a documentation validation prompt.

The most important architecture pattern is: generic engine primitives, not game semantics. Systems communicate through typed packets, fields, events, messages, observations, intents, actions, deltas, ports, probes, and canonical graphs. Mutation happens through sorted delta commit. Heavy work is budgeted and deferred deterministically. Large-scale simulation uses R0/R1/R2/R3 representations with interest volumes and accumulators. Placement uses fixed-point pose plus parametric anchors; grids are UI-only; baked geometry is never authoritative. TRANS uses corridors with cross-section slots for co-location. STRUCT uses parametric authoring and compiled occupancy/enclosure/surface/support/carrier artifacts. DECOR is host-agnostic rulepacks plus manual overrides. Space/war features are generalized as domains, frame graph, propagators, observer contexts, beliefs, visibility, and

comms.

No repository was inspected and no code was implemented in this chat. The prompts are planning artifacts until executed or adapted against the actual repo.

Most Important Things to Know

- Domino Engine = C89, deterministic, fixed-point, no platform/UI/rendering APIs.
- Dominion = portable C++98 UI/tools/visualization only.
- No hardcoded gameplay semantics.
- All data/commands are TLV-versioned.
- Mutation through sorted delta commit.
- Bounded work with deterministic carryover queues.
- R0/R1/R2/R3 representation ladder is engine-wide.
- Agents are composition, not inheritance.
- Behavior pipeline: sensors -> observations -> minds/controllers -> intents -> actions -> deltas.
- Plants/wildlife can be fields/populations and promoted when relevant.

- No global grid dependency in engine logic.
- All placement uses pose + parametric anchor.
- No baked geometry as source of truth.
- TRANS co-location uses cross-section slots.
- STRUCT/DECOR are authoring-plus-derived-cache systems.
- Space/war needs are generalized into domains/knowledge/comms primitives.
- A 14-prompt Codex implementation roadmap exists.
- VM-only vs native plugin policy is unresolved.
- Exact Q-format table is unresolved.
- Repository structure/build system must be verified.

Active Plans or Workstreams

- WORKSTREAM-01 - Dominion/Domino deterministic engine architecture
- WORKSTREAM-02 - Advanced physics and simulation stack
- WORKSTREAM-03 - Agents, actors, life, ecology, and behavior

- WORKSTREAM-04 - Engine-wide extensibility, modularity, and performance methodology
- WORKSTREAM-05 - Custom buildings and infrastructure at scale
- WORKSTREAM-06 - Generalized Interstellar/Wargames core primitives
- WORKSTREAM-07 - Arbitrary placement and TRANS/STRUCT/DECOR spatial primitives
- WORKSTREAM-08 - Master Codex prompt implementation roadmap
- WORKSTREAM-09 - Documentation validation and normalization

Decisions Already Made

- Hard engine/layer constraints are non-negotiable.
- Use typed IO, delta commit, fields/events/messages, budgets, LOD, graphs, and replay hashes.
- Use pose+anchors and no baked geometry for placement.
- Use TRANS slots for corridor co-location.
- Use STRUCT/DECOR authoring vs compiled artifacts.

- Generalize DLC requirements into core primitives.

Pending Tasks

- Inspect repository structure and build system.
- Run/adapt Codex Prompts 1-14 in sequence.
- Run the documentation validation prompt.
- Decide VM-only versus native plugins.
- Define exact Q-format table.
- Generate detailed physics subsystem prompts later.

Open Questions

- VM-only vs native plugins?
- Exact Q formats?
- Actual repository/build layout?
- Existing TLV/scheduler/graph/BUILD/TRANS/STRUCT/DECOR state?
- Rotation representation for pose/frame transforms?
- How derived caches should be hashed?

Files / Artifacts / Prompts to Preserve

- The 14 Codex prompts.
- Documentation validation prompt.
- Sister GPT-5.1 requirements prompt.
- Maximum-fidelity context transfer packet.
- This report package.

What to Verify Before Acting

- Repository paths and build system.
- Existing TLV/res layer.
- Existing deterministic scheduler, if any.
- Existing BUILD/TRANS/STRUCT/DECOR assumptions.
- Whether any engine code uses floats, unordered containers, grid-locked logic, raw memory hashing, or platform APIs.

Best Next Step

Inspect the repository, then run or adapt CODEX PROMPT 1. Do not execute later prompts until Prompt 1 establishes specs, invariants, and scaffolding or until repository reality is mapped.

Chapter 36

Engine Baseline Architecture

36.1 Domino Dominion Engine
Baseline Architecture Human
Readable Report

Source: docs/archive/conversations/engine_
baseline_architecture/domino_dominium_engine_
baseline_architecture__01_human_readable_
report.md

Report type: human_readable_report

Size: 48379 bytes

36.1.1 COMPLETE CHAT PRESERVATION
REPORT - Domino/Dominium Engine
Baseline, Architecture, and Feasibility

0. Coverage and Reliability Assessment

Row 1 Field: Chat label; Assessment: Domino/
Dominium Engine Baseline, Architecture, and
Feasibility

Row 2 Field: Date anchor; Assessment: 2026-05-27
1882

Australia/Melbourne

Row 3 Field: Source scope; Assessment: This chat only unless labelled PROJECT-CONTEXT

Row 4 Field: Apparent access; Assessment: Partial but broad

Row 5 Field: Previously generated files available?; Assessment: No previously generated preservation files were visible before this task

Row 6 Field: Uploaded files or artifacts present?; Assessment: Yes - **Pasted text.txt**, containing the preservation/export instructions

Row 7 Field: Contains future plans?; Assessment: Yes

Row 8 Field: Contains decisions?; Assessment: Yes, mostly tentative/strategic decisions and sequencing decisions

Row 9 Field: Contains pending tasks?; Assessment: Yes

Row 10 Field: Contains unresolved questions?; Assessment: Yes

Row 11 Field: Staleness risk; Assessment: Medium

Row 12 Field: Extraction confidence; Assessment: 4/5

Row 13 Field: Safe for later aggregation?; Assessment: Yes, with caveats

Row 14 Field: Main limitations; Assessment: I can access the visible chat context and files fetched during the chat, but I cannot independently verify the user's local `C:/Inbox/...` paths, local build outputs, CTest results, or generated evidence files unless those are uploaded. Some repo facts may have changed after they were fetched.

The uploaded preservation prompt requires a maximum-fidelity human-readable handoff, structured registers, spec sheet, aggregator packet, self-audit, and downloadable file package for this chat. That prompt itself is an artifact of this chat and should be preserved.
Source: ?filecite?turn44file0?

Limitations in plain language: this report reconstructs the conversation from the visible transcript and from repository files that were fetched during the chat. It does not prove that the local commands the user reported were actually run, because those local logs were pasted as claims rather than uploaded as verifiable build artifacts. It also does not claim that the full GitHub repository was exhaustively audited. The report is therefore suitable as a high-fidelity chat handoff, not as a final technical audit of the current repo.

1. One-Page Orientation

This chat was about the architecture, feasibility, state, and future direction of the Domino engine and Dominion game. The user wanted to understand what the project is, whether the engine exists in a meaningful sense, whether it is ready to support a game, how it can become portable and modular enough to be reused for multiple games or even CAD-like tools, and how to sequence the work without being overwhelmed by the enormous scope.

The conversation began with a request to read the project docs and GitHub repository and describe the total project, especially the engine and game. The answer framed Domino as the deterministic simulation substrate, Dominion as the rule/game layer, and XStack/governance as the proof/audit discipline. Repository CMake files were fetched showing real `domino_engine` and `dominium_game` targets, with substantial engine and game source composition. Later, the user pushed beyond description into strategy: how to make the engine better, how to make it do more for less, how to support many hardware/software targets, and how to build something durable for decades.

A major theme emerged: Domino should not try to become a conventional monolithic game engine. It should become a deterministic, contract-driven,

capability-negotiated simulation kernel. Dominium should be one product/game built on top, not the reason the engine exists. The engine should define mechanisms such as Work IR, Access IR, scheduling, replay, domains, fields, capabilities, registries, and law gates. The game should define meaning: construction, life, agents, economy, governance, survival, war, CAD-like design, terraforming, and so on. Client/rendering should be projection and intent only. Server/domain authority should own authoritative multiplayer truth.

The user repeatedly emphasized portability, modularity, extensibility, future-proofing, and reuse. The recommendations converged on stable public contracts with replaceable implementations: semantic contracts version meaning; schemas version shape; capabilities define power; lockfiles define exact composition; proof/replay validates actual behavior. Directory layout was discussed, but the conclusion was that folder names alone do not create modularity. Real modularity comes from public/private boundaries, stable APIs, provider interfaces, explicit ownership, deterministic artifact formats, and compatibility tests.

Another major branch concerned building and destruction. The user asked whether the game would need to give players a full CAD suite. The answer proposed a layered authoring model: ordinary players use templates, snapping, assisted intent, or accessible UI;

advanced users and developers can use CAD-grade design tools. The canonical artifact should be a design/blueprint/assembly/process object, independent of the UI surface used to create it. When the user asked about voxels versus B-Rep/CSG/NURBS, the conclusion was a hybrid: CSG/feature graphs for authoring, B-Rep for canonical solids, NURBS for advanced curved faces, meshes as derived render/collision proxies, and voxels/SDF/fields only for terrain, mining, fluids, debris, and sparse damage approximation.

The feasibility discussion became the most important practical correction. Initially the project was described as having a working deterministic foundation but not a finished game engine. The user then pasted a more precise local assessment: the repo builds real engine/game/client/server targets and some targeted tests pass, but the client advertises `support_claim_playable=false`, the playtest path is blocked, server Python startup has circular import issues, baseline bundle/session paths are not hardened, and a time-anchor policy registry is missing or invalid. The final corrected sequencing was: do not start with destruction, CAD, economy, agents, or broad gameplay. First complete **Milestone 0: Make the baseline path honest** - one canonical repo-local playable baseline command must pass strict validation. Only then should a tiny builder/destruction lab begin.

The final design vision explored a deterministic full-scale solar-system/civilization game. The answer rejected "full fidelity everywhere" as impossible. The viable design is a multi-resolution universe: rail-orbit solar systems, hierarchical planet domains, sparse terrain overlays, collapsed civilization capsules, fog-of-war epistemic filtering, sparse construction/destruction, domain-sharded multiplayer authority, and proof-carrying client compute only where safe. Unreal was judged useful for rendering/editor-like presentation but not sufficient as the authoritative deterministic substrate for this project.

Future relevance: this chat should feed several chapters of a master project spec book: engine/game boundary, deterministic execution, portability and modularity doctrine, baseline milestone sequencing, CAD/build/destruction model, sparse full-scale universe architecture, MMO/sharding constraints, and project feasibility discipline. The most important thing to preserve is the corrected sequencing: **baseline first, vertical slice second, grand systems later.**

2. The Story of the Conversation

2.1 Project description and first architecture

framing The chat began with the user asking for a total description of the project by reading the docs and GitHub repository. Repository files were inspected, especially `engine/CMakeLists.txt` and `game/`

`CMakeLists.txt`. This established that the repo contains real build targets rather than only planning notes: `domino_engine` is a static engine library and `dominium_game` is a static game library linking against it. The early report described Dominium as a deterministic, law-governed, multi-scale reality simulation game and product stack. The distinction between Domino and Dominium became foundational: Domino is the deterministic substrate; Dominium is the game/rules layer.

The report also emphasized that the project is not merely "Minecraft but real life." It is more accurately a simulation OS or deterministic world runtime, with engine, game, client, server, launcher, setup, tools, and governance layers.

2.2 Learning from historical engines and making Domino better The user then pasted a large analysis of revolutionary game engines - SCUMM, Doom, Quake, Unreal, RenderWare, Source, Havok, CryEngine, RAGE/Euphoria, Frostbite, Unity, id Tech 5, UE4, UE5, Decima - and asked what else Domino could do to be better, more efficient, portable, powerful, and durable. The response identified the deeper lessons: visibility/world representation, data-driven content, physics as gameplay, large-world streaming, material realism, and toolchains as the real engine.

The conversation then connected those lessons to Domino. The key recommendation was to virtualize everything expensive: geometry, textures, terrain, agents, economy, history, AI, physics, networking, and tools. Domino should maintain active sets rather than full world state in memory; it should use event queues instead of global scans; and it should use sufficient statistics and capsules for collapsed domains. This aligned with the repo's documented deterministic execution model and invariants.

2.3 Live repo correction: more work exists than previously discussed The user then clarified that the live repo had much more implemented work than the earlier description captured. New docs and code were fetched: Status Now, MVP Scope Lock, Extension Map, Repo Review 3, Frozen Invariants, AppShell Constitution, Capability Negotiation Constitution, Pack Verification Pipeline, execution IR source files, registry compile docs, determinism envelope report, canon conformance report, and more.

The conclusion changed: the project already had more infrastructure than the generic future plan assumed. The recommendation shifted from "invent the architecture" to "converge the architecture." The answer proposed that Dominion should become a **contract-compiled simulation platform**: contracts -> registries ->

compiled locks -> capabilities -> Work IR ->
deterministic runtime -> proof/replay -> product shells.

The main practical recommendations were validation unification, extension classification, provider descriptors, registry bundle grouping, deterministic scheduler/backend optimization, generated docs, platform matrix, pack doctor/mod templates, and AuditX triage.

2.4 CAD, building, and player freedom The user then raised the problem of construction: if players can build all items, machines, and structures, does the game need a CAD suite? The answer proposed layered authoring. Ordinary players should not be forced to use CAD. Instead, the same canonical design artifact can be produced from different surfaces: simple templates, assisted snapping, parametric objects, constraint editing, full CAD, or generative/programmatic design.

The answer also rejected recipes/tech trees as the default gameplay model. Instead, it proposed affordances and constraints: materials, tools, machines, capabilities, processes, authority, environment, time, and energy determine what can be made. Recipes, tech trees, levels, or certifications can exist as optional mod/game-mode layers via packs, law, profiles, and capabilities.

2.5 Portability, modularity, and future-proofing

The user then emphasized that all code should be

portable, modular, extensible, and reusable for different games or entirely different engine projects. The response gave a broad engineering doctrine: stable outside, flexible inside. Stable public APIs, semantic contracts, schema versioning, capability negotiation, provider interfaces, opaque handles, canonical serialization, golden replay corpora, and strict public/private boundaries matter more than directory aesthetics.

A long-term directory model was suggested, but with the caveat that the current repo's v0.0.0 state should not be disrupted by broad restructuring. The repo should be organized by authority and ownership: contracts define what may exist; engine implements reusable mechanisms; runtime adapts host/platform/product shells; game defines Dominion meaning; content defines packs/profiles; products expose binaries; tools inspect/compile/validate/migrate; tests prove contracts; docs explain contracts.

2.6 B-Rep / CSG / NURBS versus voxels The user then focused on construction/destruction representation and argued that Dual Universe got it wrong by going voxel-based. The answer agreed with the direction but recommended a hybrid. Voxel-first is useful for terrain, mining, craters, debris, and volumetric fields, but poor for precise machines, CAD semantics, thin structures, interfaces, mass properties, and network

replication. The preferred model was CSG/feature graphs for editable authoring, B-Rep for canonical validated solids, NURBS/B-splines for advanced curved surfaces, meshes only as derived proxies, and sparse voxels/SDFs for terrain/damage/fluid/debris approximations. Destruction should be tiered and sparse rather than global geometry recomputation.

2.7 Engine readiness and feasibility anxiety The user expressed worry that the project might be impossible and that they did not know how to write engines. The initial answer said the project has a working deterministic substrate but not a finished game engine. Then the user pasted a more precise local assessment: targeted build and CTest slices had passed, but the client still reports `support_claim_playable=false`; the local playtest path is blocked; Python server entrypoint has circular import issues; session creation defaults to a lab bundle; the MVP bundle path refuses; and the baseline universe verifier fails because the time-anchor policy registry is missing or invalid.

This corrected the plan. The final sequencing became: first **Milestone 0: Make the baseline path honest**. That means fixing server/runtime circular import, CLI forwarding, `session_create -> session_boot`, missing time-anchor policy registry, and making the strict local

playtest validator pass. Only after that should the builder/destruction lab begin.

2.8 Full-scale universe architecture and Unreal comparison The user then described the full dream: deterministic procedural/authored hybrid solar systems, fully recreated/customizable planets, civilizations, terraforming, terrain cut/fill, megaprojects, designed machines/factories, fog of war, collapsed simulation for unobserved areas, MMO persistent universe, client-shared compute, sparse construction/destruction, and low latency/high FPS at scale. The answer said full scale is possible only if full fidelity everywhere is abandoned. The architecture should use domains, collapsed capsules, sparse overlays, planet tiles, event-driven processes, fog as epistemic filtering, domain sharding, and proof-carrying client delegation.

Unreal was judged incapable of the full authoritative simulation model out-of-the-box. It can help as a rendering/editor/client shell but not as the deterministic substrate for this architecture.

2.9 Preservation request The final user action uploaded `Pasted text.txt`, a detailed instruction prompt requiring a full preservation report, structured registers, spec sheet, aggregator packet, self-audit, and downloadable file package for this chat. That request is

the current task.

3. Main Topics Discussed

Topic 1 - Domino engine versus Dominion game

The central architecture distinction was that Domino should be a reusable deterministic simulation substrate, while Dominion should be one game/product layer built on it. This came up immediately when the user asked for a total description of the project. It mattered because the user wants the code to be reusable for future games and possibly other engine projects.

The conclusion was that engine and game responsibilities must stay separated. Engine owns deterministic mechanisms, execution, storage, domains, fields, replay, capabilities, and law gates. Game owns meanings and rules such as construction, life, agents, economy, governance, survival, and war. Client/render is projection and intent only; server owns multiplayer authority. This aligns with the canonical constitution fetched later in the chat. [?filecite?turn40file0?L3-L3?](#)

Uncertainty: the repo has many modules and some historical/legacy surfaces, so exact implementation maturity varies by subsystem. The distinction should remain a formal requirement in a master spec.

Topic 2 - Deterministic execution and proof The conversation repeatedly returned to determinism. The project's value depends on identical inputs producing identical outputs, replay equivalence, stable commit ordering, named RNG streams, explicit Work IR/Access IR, and no hidden global scans. The assistant inspected execution model docs and source files such as task graph, access set, scheduler, and work queue.

The visible conclusion was that the engine has real deterministic substrate pieces, but these must be hardened through small playable slices and proof/replay tests. Determinism is not only a mathematical preference; it enables multiplayer authority, replay, audit, mod compatibility, and long-term artifact preservation.

Uncertainty: the full determinism envelope across all current modules was not independently re-run in this chat. User-supplied local validation results should be preserved but verified before being used as audit proof.

Topic 3 - Engine improvement strategy When the user asked how to make the engine better and more efficient, the answer was "virtualize everything expensive. " Domino should do more for less by activating only what matters, using event queues, collapsed capsules, sufficient statistics, budgeted fidelity, and provider backends. Large worlds should not be simulated by ticking every entity. Instead, the engine should preserve truth through

sparse representations and deterministic expansion.

The conclusion was that future work should build active-set runtime, capsule/refinement contracts, compatibility edition matrix, golden replay corpus, and toolchain-first authoring. Later, after reading live repo docs, this plan was tightened into "contract-compiled simulation platform."

Topic 4 - Live repo maturity and convergence

The user corrected that the live repo had more work than the early answer acknowledged. After reading more docs and code, the plan shifted from speculative architecture to convergence. The repo already had MVP scope locks, AppShell, capability negotiation, pack verification, semantic contracts, frozen invariants, registry compile, determinism envelope reporting, and canon conformance reports. The answer recommended fewer stronger contracts, validation unification, provider descriptors, registry bundles, generated docs, and a pack doctor.

This topic matters because it prevents wasted work. The project should not add broad new systems before stabilizing existing baseline paths.

Topic 5 - CAD, construction, and player freedom

The user wanted players to build anything but also enjoy the game without skill. The solution was layered authoring. Ordinary players should use simple

templates, assisted snapping, intent-based building, or accessibility-first controls. Advanced players and developers can use a full CAD suite. All authoring surfaces compile into a canonical design artifact.

The conclusion was that the engine should support CAD-like design without requiring CAD-like gameplay. Recipes and tech trees should not be default mechanics; they should be optional pack/law/profile/capability layers for modders and game modes.

Topic 6 - Geometry representation for building/destruction The user suggested B-Rep, CSG, NURBS, or hybrid rather than voxels. The recommendation was hybrid: CSG/feature graphs for authoring, B-Rep for canonical solids, NURBS for advanced curved faces, meshes as derived render/collision proxies, voxels/SDFs for terrain/mining/damage/fluid/debris. Destruction should be tiered and sparse.

This connects to future work because a construction/destruction game cannot rely on one universal representation. Different domains need different data structures.

Topic 7 - Portability, modularity, and future-proofing The user wanted code reusable across games and engine projects. The answer emphasized stable public contracts, opaque handles, C ABI-like

boundaries, public/private header discipline, provider interfaces, schema/semantic-contract separation, canonical serialization, artifact versioning, and golden compatibility corpora. Directory structure matters, but enforcement matters more.

The conclusion was "stable at the boundary, ruthless inside." Internal implementation can be rewritten if public contracts, tests, formats, and migration/refusal behavior hold.

Topic 8 - Baseline readiness and Milestone 0 This became the practical pivot. The user pasted evidence that the repo has compiled targets and some passing tests but lacks a finished playable path. The final decision was that before game features, the team must create one canonical repo-local playable baseline command that passes strict validation.

This matters because it turns overwhelming ambition into concrete blocker-fixing: circular imports, CLI forwarding, session materialization, bundle mismatch, missing time-anchor registry, and strict playtest validator.

Topic 9 - Full-scale solar-system/civilization/MMO architecture The user described the full ambition. The answer stated that full-scale is only possible with collapsed multi-resolution

simulation. Domains, fields, sparse overlays, planet tiles, collapsed civilization capsules, fog-of-war epistemics, sharding, and proof-carrying delegated compute are required. Unreal can help with rendering/editor surfaces but not with this deterministic authoritative simulation model.

Uncertainty: this remains strategic architecture, not an implemented decision.

4. What We Were Trying to Achieve

4.1 Explicit Goals The user explicitly wanted to understand and preserve:

- what the project is;
- whether Domino has a working engine;
- whether Dominium can start game work;
- how to make the engine portable, modular, reusable, and future-proof;
- how to support full-scale solar systems, planets, construction, destruction, and civilizations;
- how to avoid requiring recipes/tech trees in normal gameplay;
- how to support CAD-level freedom without overwhelming players;

- how to sequence the project so it is not impossible;
- how to preserve this chat for future aggregation.

These goals mattered because the project scope is extremely large and the user was concerned about feasibility and implementation ability.

4.2 Inferred Goals INFERENCE: the user wants a development doctrine that makes the project less emotionally and technically overwhelming. The repeated questions about impossibility, not knowing how to write engines, and needing future-proof structure indicate a need for practical sequencing as much as architecture.

INFERENCE: the user wants the engine to outlive Dominion and serve as a platform for multiple future projects.

4.3 Goals That Changed Over Time The conversation moved from broad architecture description to implementation readiness. Early answers focused on what the project could be and how to architect it. Later, the user's pasted local assessment forced a stricter current-state view: the immediate goal is not more game design; it is making the baseline path honest.

The goal also shifted from "support everything" to "support everything through sparse, collapsed, capability-gated, provider-based architecture."

4.4 Goals Still Unresolved Unresolved goals include:

- exact Milestone 0 implementation plan at code level;
- verification of local build/test claims;
- exact canonical playtest command name and behavior;
- exact design artifact schema for CAD/building;
- exact geometry kernel approach and library/provider strategy;
- exact sharding and client-compute proof model;
- whether Unreal will be used at all as a client/editor shell.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Domino is the reusable deterministic substrate; Dominium is one game layer; Status: Accepted direction; Why it mattered: Prevents Dominium-specific coupling in engine; Confidence: High; Label: FACT / accepted framing

Row 2 ID: DECISION-02; Decision: Do not treat the engine as a finished playable game engine; Status: Accepted after correction; Why it mattered: Avoids

premature feature work; Confidence: High; Label: FACT / user-corrected

Row 3 ID: DECISION-03; Decision: Milestone 0 must precede builder/destruction lab; Status: Accepted by user framing; Why it mattered: Fixes baseline blockers first; Confidence: High; Label: FACT / accepted sequencing

Row 4 ID: DECISION-04; Decision: Use contract/capability/registry/proof architecture for extensibility; Status: Strong recommended direction, broadly accepted; Why it mattered: Enables modularity, modding, compatibility; Confidence: Medium-high; Label: INFERENCE

Row 5 ID: DECISION-05; Decision: CAD should be supported as an advanced/tooling surface, not required normal gameplay; Status: Strong recommended direction; Why it mattered: Balances freedom and accessibility; Confidence: Medium-high; Label: INFERENCE

Row 6 ID: DECISION-06; Decision: Avoid recipes/tech trees as default gameplay; support them via packs/law/capabilities; Status: User-stated preference + assistant architecture; Why it mattered: Preserves freedom while enabling modded modes; Confidence: High; Label: FACT

Row 7 ID: DECISION-07; Decision: Use hybrid CSG/B-Rep/NURBS/mesh/SDF approach, not voxel-first; Status: Assistant recommendation, not explicitly rejected; Why it mattered: Supports precise building/destruction; Confidence: Medium; Label: INFERENCE

Row 8 ID: DECISION-08; Decision: Unreal is not sufficient as authoritative simulation substrate; Status: Assistant conclusion; Why it mattered: Clarifies engine strategy; Confidence: Medium; Label: INFERENCE / external facts stale risk

Row 9 ID: DECISION-09; Decision: Full-scale universe must be multi-resolution and collapsed, not full-fidelity everywhere; Status: Strong strategic conclusion; Why it mattered: Makes project feasible; Confidence: High; Label: INFERENCE

Row 10 ID: DECISION-10; Decision: Avoid broad restructuring before v0.0.0/baseline hardening; Status: Based on repo docs and assistant recommendation; Why it mattered: Prevents breaking fragile paths; Confidence: High; Label: FACT / recommendation

DECISION-01 - Domino as substrate, Dominium as game The conversation consistently preserved this boundary. The engine should define deterministic

1904

mechanisms; the game defines rules and meaning. This affects directory layout, APIs, naming, dependencies, modding, and future reuse. It would be revisited only if the project intentionally stopped trying to make Domino reusable, which contradicts the user's stated goal.

DECISION-02 - Engine exists, but playable game engine does not This was corrected over time. Earlier descriptions could have sounded too optimistic. The user's pasted local assessment tightened it: compiled substrate exists, targeted tests may pass, but the client itself advertises no playable support. This affects all next steps.

DECISION-03 - Milestone 0 first The user explicitly accepted the correction that before a builder/destruction lab, the first deliverable should be one canonical repo-local playable baseline command passing strict validation. This is the most important sequencing decision in the chat.

DECISION-04 - Contract-compiled simulation platform This was a recommended architectural conclusion after reading live repo docs. It is not a single user-typed acceptance sentence, but the user continued building on it. It should be treated as a strong strategic direction, not a final immutable spec.

DECISION-05 - CAD-capable, not CAD-required

The user wanted full freedom and low-skill enjoyment. The answer resolved this with layered authoring. This should become a future spec requirement: every advanced design surface should compile to a canonical artifact, but ordinary users should have assisted modes.

DECISION-06 - No default recipes/tech

trees/levels The user explicitly stated they do not want usual gameplay to rely on technology trees, recipes, or experience levels, while wanting the engine to support those for mods/game modes. This is a strong product preference.

DECISION-07 - Geometry hybrid The assistant recommended a hybrid representation. The user did not explicitly confirm after that response in the visible transcript. Treat as a strong recommendation to preserve, not a final decision.

DECISION-08 - Unreal as possible shell, not authoritative core The answer said Unreal can help with rendering/editor/client presentation but cannot solve the deterministic full universe architecture out-of-the-box. This is a strategic conclusion and should be verified if the project later seriously considers Unreal integration.

DECISION-09 - Full-scale means collapsed multi-resolution This became the core feasibility doctrine. It should become a formal architecture principle: full scale yes, full fidelity everywhere no.

DECISION-10 - No broad restructure before baseline The assistant recommended this based on repo docs. It should remain a strong constraint unless the repo state changes.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

REJECTED-01 - Build the full engine before game work Rejected because it creates an infinite engine project. The better plan is narrow vertical slices that harden the engine.

REJECTED-02 - Start with CAD/destruction before baseline Superseded by Milestone 0. CAD/destruction remains important but should not precede playable baseline hardening.

REJECTED-03 - Voxel-first universal building engine Rejected/deprioritized because voxels are poor for precise machines, CAD semantics, interfaces, thin surfaces, and network-efficient authoritative objects.

Voxels/SDF remain useful for terrain, mining, damage approximation, fluids, and debris.

REJECTED-04 - Full fidelity universe everywhere

Rejected as impossible. The viable plan is full-scale but collapsed multi-resolution fidelity.

REJECTED-05 - Unreal as authoritative

simulation substrate Rejected as a default plan.

Unreal may still be useful as a client/editor/rendering shell.

REJECTED-06 - Recipes/tech trees/levels as

default gameplay Rejected by user preference. Still supported as optional mod/game-mode layers.

REJECTED-07 - Broad repo restructure before

baseline Deprioritized because current baseline paths are fragile and v0.0.0/current-reality docs prioritize preserving them.

7. Important Reasoning, Rationale, and Tradeoffs

The main tradeoff was ambition versus feasibility. The user wants a universe-scale game with CAD-like construction, sparse destruction, civilization building, fog of war, and MMO persistence. The answer

repeatedly reframed this as possible only through representations that do less work: collapsed capsules, event-driven processes, sparse overlays, active sets, domain sharding, and derived proxies.

Another key tradeoff was player freedom versus accessibility. A full CAD suite gives freedom but alienates low-skill players and increases complexity. Templates and simple parts are accessible but can feel restrictive. The proposed compromise is layered authoring over a single canonical design artifact.

A third tradeoff was portability versus feature richness. Full modern rendering, CAD, and massive multiplayer cannot be required for all hardware. The answer proposed capability profiles and explicit degradation/refusal. Every target gets the same truth contract, not the same fidelity.

A fourth tradeoff was modularity versus restructuring. The user wanted replaceable directories and reusable code. The answer argued that contracts, APIs, tests, and provider interfaces matter more than directory layout. Broad restructure before baseline would risk breaking fragile run paths.

The final tradeoff was engine-first versus game-first. The conclusion was neither: build minimal vertical slices that are simultaneously game work and engine proof.

8. Plans, Future Work, and Next Steps

8.1 Milestone 0 - Make the baseline path honest

This is the immediate next work. It matters because without a reliable local playable path, later feature work will be built on unstable foundations.

Tasks:

1. Fix Python server/runtime circular import and entrypoint CLI forwarding.
2. Make `session_create` -> `session_boot` pass for intended MVP/baseline bundle.
3. Fix missing/invalid time-anchor policy registry.
4. Add one canonical repo-local playtest command.
5. Make `check_local_playtest_path.py --strict` pass.
6. Preserve user-reported validation evidence or rerun it as audit-grade logs.

Expected output: one command that creates/materializes session, boots it, starts local loopback authority, runs deterministic client/server smoke path, emits proof/replay/hash/compat artifacts, and exits with deterministic pass/fail.

8.2 Milestone 1 - One deterministic game action

After Milestone 0, implement one accepted and one refused game action, likely a simple build/place-part action. It should pass through authority/law, Work IR/process, commit, replay/hash, and client projection.

8.3 Milestone 2 - One deterministic

removal/damage action Add simple part removal or damage that affects an assembly/support graph. Do not start with full CAD/CSG destruction.

8.4 Milestone 3 - Minimal projection/rendering

Show the object/action through null/software/TUI/simple rendered projection. Rendering must not mutate truth.

8.5 Milestone 4 - Pack-driven part/action

Move the part/action into pack/registry/lockfile flow so modding and data-driven content are proven early.

8.6 Long-term work Long-term work includes CAD/design substrate, geometry kernel providers, sparse terrain overlays, planet/domain hierarchy, collapsed civilization capsules, sharding, proof-carrying client compute, generated docs, platform matrix, validation unification, and modder toolchain.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences

- User wants direct, rigorous, source-grounded answers.
- User wants uncertainty labelled and framing corrected when evidence disagrees.
- User wants Domino code portable, modular, extensible, and reusable for other games/engines.
- User wants not to rely on recipes, tech trees, or levels in usual gameplay.
- User wants those systems supported for mods/game modes through capabilities/law/packs.
- User wants players limited by imagination but also able to play without special skills.
- User wants low-performance, disability controls, minimal GUI, and accessibility support.
- User wants full preservation of this chat for later aggregation.

9.2 Inferred Constraints and Preferences

- INFERENCE: prefer engineering plans that reduce overwhelm and produce concrete next actions.

- INFERENCE: prefer reusable contracts over ad hoc feature code.
- INFERENCE: prefer maximal freedom only if bounded by deterministic validation, budgets, and law/capability systems.
- INFERENCE: prefer not to depend on Unreal or another engine as authoritative core.

9.3 Uncertain or Unestablished Preferences

- UNCERTAIN: whether the user wants Unreal integration at all as a client/editor.
- UNCERTAIN: exact tolerance for using external geometry libraries.
- UNCERTAIN: whether broad repo restructuring will be desired after Milestone 0.
- UNCERTAIN: exact visual style, UI stack, or programming language evolution.

10. Files, Artifacts, Outputs, and Prompts

ARTIFACT - Uploaded preservation prompt

Pasted text.txt contains the current preservation/export instructions. It is the source for this report's requested structure. Preserve it. ?filecite?turn44file0?

ARTIFACT - Repository/docs/code references
fetched during chat Important fetched files included:

- `engine/CMakeLists.txt` - evidence for `domino_engine` and engine module composition.
- `game/CMakeLists.txt` - evidence for `dominium_game` and game module composition.
- `docs/canon/constitution_v1.md` - authoritative architecture/execution contract.
- `docs/repo/REPO_NON_NEGOTIABLES_AND_CURRENT_REALITY.md` - current baseline-first repo reality packet.
- `apps/client/main_client.c` - evidence that client advertises unavailable gameplay/world/package/provider/module runtime and `support_claim_playable=false`.
- `tools/xstack/registry_compile/constants.py` - evidence of `DEFAULT_BUNDLE_ID = "bundle.base.lab"`.
- `game/rules/physical/parts_and_assemblies.cpp`, `construction_processes.cpp`, `fab_interpreters.cpp` - evidence for early construction/fabrication substrates.
- Various docs: AppShell, capability negotiation, pack verification, semantic contracts, frozen

invariants, MVP scope lock, registry compile, determinism envelope, canon conformance.

ARTIFACT - User pasted local validation assessment The user pasted a detailed local assessment listing passing build/test slices and blockers. Treat as user-supplied evidence unless independently verified. Preserve it because it corrected the plan.

ARTIFACT - This preservation package Created by this task: Markdown/YAML files and ZIP handoff package. These should feed future aggregation and spec book creation.

11. Open Questions and Unresolved Issues

1. Does the current live repo still match the fetched GitHub state?
2. Are the user-reported build/CTest results reproducible now?
3. What is the exact canonical playtest command name and interface?
4. Should `session_create` default stay `bundle.base.lab`, or should the MVP wrapper explicitly select `profile.bundle.mvp_default`?

5. What file/module refactor fixes the circular import without violating boundaries?
6. What registry artifact is missing or invalid for time-anchor policy?
7. What exact design artifact schema should support CAD/building later?
8. Which geometry provider/library strategy should be used for B-Rep/CSG/NURBS?
9. Will Unreal be used as a client/editor/render shell, or avoided entirely?
10. What is the minimum acceptable first playable slice?

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

Future chats might overstate engine readiness by saying "the engine works" without noting `support_claim_playable=false` and blocked baseline paths. They might also forget Milestone 0 and jump into CAD, destruction, multiplayer, or universe-scale simulation prematurely.

Another risk is treating assistant recommendations as user decisions. For example, the CSG/B-Rep/NURBS hybrid is a strong recommendation but not yet a user-

finalized decision. Unreal-as-shell is also a recommendation, not a committed plan.

A future assistant might merge this chat into a master spec as if every ambitious idea is a requirement for v0.0.0. That would be wrong. The chat clearly moved toward baseline-first sequencing and deferred major systems.

A future assistant might also miss the distinction between full scale and full fidelity. The project can aim for full-scale solar systems and planets only by using collapsed, sparse, hierarchical representations.

Finally, there is a risk of relying on stale GitHub/doc facts. Any implementation-sensitive claim should be verified against the current repo.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat contributes:

- a high-level project identity: Domino substrate, Dominion game, XStack/governance;
- a modularity/future-proofing doctrine;
- a current readiness correction and baseline-first roadmap;
- a CAD/building authoring model;
- a geometry representation strategy;

- a full-scale universe architecture strategy;
- an MMO/client-compute feasibility frame;
- a list of immediate blockers and tasks.

It should feed spec book chapters on:

- architecture principles;
- engine/game/product boundaries;
- deterministic execution and proof;
- content/mod/pack system;
- construction/design/CAD;
- geometry/destruction;
- universe/planet/domain simulation;
- multiplayer/sharding/client compute;
- project roadmap and milestones;
- risks and non-goals.

Do not merge the full universe/MMO/CAD material as near-term implementation requirements. They belong as long-term architecture context unless later accepted as staged requirements.

14. What I Should Remember

- The engine substrate is real, but the playable game engine is not finished.
- The client explicitly reports `support_claim_playable=false`.
- The most important next milestone is not building/destruction; it is making one canonical repo-local playable baseline command pass strict validation.
- Domino should be reusable and deterministic; Dominium should not contaminate the engine substrate with game-specific meaning.
- The project is only feasible if it uses collapsed multi-resolution simulation, sparse overlays, event-driven scheduling, and proof/replay.
- Normal gameplay should avoid default recipes/tech trees/levels; those remain optional pack/law/profile features.
- CAD should be supported as an advanced/tooling surface, not required for casual play.
- Geometry should likely be hybrid: CSG/feature authoring, B-Rep canonical solids, NURBS optional faces, mesh derived, voxel/SDF for terrain/damage fields.

- Unreal cannot simply solve the authoritative simulation problem, though it might be useful for rendering/editor surfaces.
- Preserve the user's pasted local validation correction because it changed the plan.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

- "Explain the difference between Domino substrate and Dominion game layer again, using examples from the repo."
- "What changed after I pasted the local engine readiness assessment?"

15.2 Decisions

- "Which recommendations in this chat are actually accepted decisions, and which are just candidate architecture?"
- "What exactly did we decide about Milestone 0?"

15.3 Tasks and Next Actions

- "Turn Milestone 0 into a detailed implementation prompt for Codex/AIDE."

- "Write the task card for fixing the server circular import."
- "Write the task card for `session_create` -> `session_boot` baseline validation."

15.4 Artifacts and Files

- "List every repo file cited in this chat and what it proved."
- "Make a minimal evidence bundle checklist for the local validation results I pasted."

15.5 Risks and Verification

- "What claims in this chat need to be re-verified against the live repo?"
- "What would make the project impossible again if we ignore it?"

15.6 Future Spec Book / Aggregation

- "Extract only the formal requirements from this chat."
- "Extract only the future-spec context, excluding near-term tasks."

15.7 Deep-Dive Questions Specific to This Chat

- "Design the exact first builder/destruction lab after Milestone 0."
- "Define the CAD DesignArtifact schema from this chat's recommendations."
- "Define the sparse planet/terrain overlay model as a formal contract."
- "Compare using Unreal as a renderer shell versus not using it at all."

16. Compact Human Summary

This chat was a deep architecture and feasibility discussion about the Domino engine and Dominion game. It began with a request to read the project docs and GitHub repo and describe the whole project, especially the engine and game. The answer established the core distinction: Domino is the deterministic simulation substrate and Dominion is a game/rules/product layer built on top. The repository contains real CMake targets for `domino_engine` and `dominium_game`, so the project is not just a set of notes, but the engine is not yet a finished playable game engine.

The user then asked how to make the engine better, more efficient, portable, extensible, and durable. The

answer compared lessons from major historical engines and concluded that Domino should not be a monolithic engine. It should be a deterministic, contract-driven, capability-negotiated simulation kernel. It should activate only what matters, use event-driven work rather than global scans, collapse inactive regions into capsules and sufficient statistics, and treat rendering as projection rather than truth.

The user later clarified that the live repo had more implemented work than discussed. After reading more docs and code, the plan shifted from broad future architecture to convergence. The project already has many serious systems: AppShell, capability negotiation, pack verification, semantic contracts, frozen invariants, registry compilation, determinism envelope reports, and execution source files. The recommendation became to reduce the architecture into fewer stronger extension contracts, improve validation unification, generate docs from registries, group registry bundles, add provider descriptors, create platform matrices, and harden the existing baseline instead of expanding.

A major topic was building and destruction. The user wanted players to build machines, structures, items, and megaprojects with maximum freedom, but not require skill. The answer proposed a CAD-capable but not CAD-required design substrate. Ordinary players should use templates, snapping, assisted intent, and

accessibility-friendly controls. Advanced players and developers should be able to use CAD-like tools. All surfaces should compile into canonical design artifacts. The user also stated they do not want ordinary gameplay to rely on recipes, tech trees, or experience levels. The answer preserved that as a strong product preference: those systems may exist for mods or game modes via packs, law, capabilities, and profiles, but not as default core gameplay.

The geometry discussion concluded that a voxel-first approach like Dual Universe is not ideal. The recommended model is hybrid: CSG or feature graphs for authoring, B-Rep for canonical solids, NURBS/B-splines for advanced curved faces, meshes as derived render/collision proxies, and sparse voxels/SDF/fields for terrain, mining, damage approximation, fluids, debris, and volumetric phenomena. Destruction should be tiered and sparse, often represented as damage events and structural graph updates rather than constant full geometry recomputation.

The most important correction came when the user expressed worry that the project might be impossible and pasted a precise local readiness assessment. The corrected view is: the project has a working deterministic substrate, but not a finished playable engine. Some targeted tests and MP0 smoke paths reportedly pass, but the client source advertises

unavailable gameplay/world/package/provider/module runtime and `support_claim_playable=false`; the playtest path is blocked; server Python entrypoint has circular import issues; session bundle paths are not hardened; and the time-anchor policy registry is missing or invalid. Therefore, before any CAD, destruction, economy, agents, or broad gameplay, the project needs **Milestone 0: Make the baseline path honest.**

Milestone 0 means fixing server/runtime circular import and CLI forwarding, making `session_create -> session_boot` pass for the intended MVP/baseline bundle, fixing the time-anchor policy registry, adding one canonical repo-local playtest command, and making the strict local playtest validator pass. Only after that should the first builder/destruction lab begin: one deterministic accepted/refused build action, one deterministic damage/removal action, minimal projection/rendering, and then pack-driven content.

The final architectural discussion concerned full-scale solar systems, planets, civilizations, fog of war, client-shared compute, and MMO persistence. The answer made the feasibility rule explicit: full scale is possible only if full fidelity everywhere is rejected. Domino should use hierarchical domains, rail-orbit providers, planet tiles, sparse terrain overlays, collapsed civilization capsules, fog-of-war epistemic filtering, domain-sharded authority, and proof-carrying client delegation. Unreal

can help with rendering or editor tooling, but it cannot provide the full deterministic authoritative simulation model out-of-the-box.

The most important thing to carry forward is the sequencing: **baseline first, tiny vertical slice second, grand systems later.**

36.2 Domino Dominion Engine Baseline Architecture Reader Brief

Source: docs/archive/conversations/engine_
baseline_architecture/domino_dominium_engine_
baseline_architecture__06_reader_brief.md

Report type: reader_brief

Size: 2525 bytes

36.2.1 Reader Brief - Domino/Dominium Engine Baseline, Architecture, and Feasibility

What this chat was about

This chat examined the Domino engine and Dominion game at three levels: what the project is, what the current engine can actually do, and how to make the long-term vision feasible. It moved from broad

architecture into hard current-state sequencing.

Top 20 things to know

1. Domino should be the reusable deterministic substrate.
2. Dominion should be one game/rules/product layer on top.
3. The engine substrate is real, but not a finished playable engine.
4. The client advertises `support_claim_playable=false`.
5. The current playtest path is a target recipe, not one finished command.
6. Milestone 0 must come before builder/destruction work.
7. Milestone 0 means fixing local playtest blockers and strict validation.
8. Broad gameplay expansion should wait.
9. CAD should be supported but not required for normal play.
10. Ordinary players need templates/assisted authoring/accessibility modes.

11. Advanced users can have CAD-grade tooling.
12. Default gameplay should not rely on recipes, tech trees, or XP.
13. Those systems can exist in mods/game modes.
14. Building/destruction should likely use hybrid geometry, not voxel-first.
15. Full-scale universe simulation requires collapse/sparse fidelity.
16. Full fidelity everywhere is impossible.
17. Unreal may help as renderer/editor shell but not authoritative simulation core.
18. Clients can help compute only with proof/verification and no hidden-truth leaks.
19. Stable contracts matter more than directory aesthetics.
20. Preserve uncertainty: not every assistant recommendation is a user decision.

Decisions

The key accepted decision is Milestone 0 first. The key architectural direction is contract-driven deterministic substrate with game/product layers on top.

Pending tasks

Fix server import/CLI path, fix session create/boot, resolve bundle seam, fix time-anchor registry, add canonical playtest command, pass strict local playtest validator.

Open questions

Exact playtest command, bundle default strategy, missing time-anchor registry, geometry provider choice, DesignArtifact schema, Unreal shell decision.

Artifacts

The package includes full human report, registers, context transfer packet, spec sheet, aggregator packet, verification/audit file, future chat prompt, and in-chat reader.

Best next step

Ask: "Write the Milestone 0 implementation prompt/task card for fixing the local playable baseline."

36.3 Domino Dominion Engine Baseline Architecture In Chat Reader

Source: docs/archive/conversations/engine_
baseline_architecture/domino_dominium_engine_
baseline_architecture__09_in_chat_reader.md

Report type: in_chat_reader

Size: 1691 bytes

36.3.1 In-Chat Reader - Domino/Dominium Engine Baseline, Architecture, and Feasibility

Package overview

This package preserves a chat about the Domino engine and Dominion game: project identity, engine state, feasibility, modularity, CAD/building, geometry/destruction, full-scale universe architecture, Unreal comparison, and baseline-first roadmap.

File index

Read 01_human_readable_report.md first for context. Use 04_registers.md for structured tasks and decisions. Use 05_aggregator_packet.md for merging with other old-chat reports. Use 03_spec_sheet.yaml

for machine-assisted aggregation.

Plain-English explanation

The chat concluded that the project has a real deterministic substrate but not a finished playable game engine. The most important next action is Milestone 0: make one canonical repo-local playable baseline command pass strict validation. Only after that should the project begin a tiny builder/destruction lab.

Question menu

- What exactly is Milestone 0?
- Which blockers are P0?
- Which ideas are final decisions and which are recommendations?
- What should go into the master spec book?
- What needs verification?
- How should the first build/destroy slice be scoped?

Top things to preserve

- Baseline-first sequencing.
- Domino/Dominium boundary.

- No default recipes/tech trees/levels.
- CAD-capable but not CAD-required authoring.
- Hybrid geometry recommendation.
- Full scale, not full fidelity.
- User-pasted readiness correction.

Safest next actions

1. Reproduce current baseline blockers.
2. Fix circular import.
3. Fix session create/boot.
4. Fix time-anchor registry.
5. Implement one canonical strict playtest command.

36.4 Domino Dominion Engine Baseline Architecture Companion Detailed Summary Report

Source: docs/archive/conversations/engine_
baseline_architecture/domino_dominium_engine_
baseline_architecture__10_companion_detailed_
summary_report.md

Report type: detailed_summary_report

Size: 21377 bytes

36.4.1 Companion Human-Readable Detailed Summary and Report

Chat label: Domino/Dominium Engine Baseline, Architecture, and Feasibility

Date anchor: 2026-05-27 Australia/Melbourne

Source scope: This chat only, plus files generated or uploaded inside this chat.

Purpose: This companion report sits beside the full handoff package. It is written as a readable "what happened and what matters" account rather than a register dump.

1. Executive Summary

This conversation was a long architecture, feasibility, implementation-readiness, and preservation discussion for the **Domino engine** and **Dominium game** project. The central question was how to build a deterministic, portable, modular, reusable simulation engine that can support a deeply ambitious game: full-scale solar systems and planets, sparse construction and destruction, civilization-building, terraforming, machine and

megaproject design, modding, multiplayer persistence, fog of war, and eventually CAD-like authoring.

The most important outcome was a correction of project sequencing. The project has a **real deterministic substrate**: there are engine and game build targets, execution/scheduler pieces, pack/registry/capability infrastructure, AppShell/product surfaces, early construction/fabrication modules, and strong architecture/governance doctrine. However, it does **not** yet have a finished playable game engine. The client still advertises that gameplay/world/package/provider/module runtime surfaces are unavailable and that `support_claim_playable=false`. The local playtest path is a target recipe rather than one fully hardened command. Therefore the immediate project priority is not CAD, destruction, economy, agents, MMO sharding, or broad gameplay. The immediate priority is **Milestone 0: make the baseline path honest**.

Milestone 0 means one canonical repo-local playtest command should create/materialize a session, boot it, start local loopback authority, run deterministic client/server smoke or baseline behavior, emit proof/replay/hash/compat artifacts, and pass strict validation. The known blockers discussed were: Python server/runtime circular import, server CLI forwarding/import path problems, `session_create -> session_boot` not passing for the intended baseline bundle, the

implementation-vs-guidance seam around `bundle.base.lab` versus the MVP baseline bundle, missing/invalid time-anchor policy registry, and strict local playtest validation reporting blocked proof status.

After Milestone 0, the first true game slice should be tiny: one deterministic accepted/refused build action, one deterministic removal/damage action, one replay/hash proof, one minimal projection/rendering path, and one pack-driven object/action. This sequence was chosen because trying to "finish the engine" first would become an infinite engine project, while trying to build the full game before baseline hardening would build on unstable ground.

The conversation also produced several long-term architecture principles. Domino should be the reusable deterministic simulation substrate; Dominion should be one game/product layer built on it. The engine should define mechanisms, not Dominion-specific meanings. Game content and rules should be data/pack/registry/capability driven where possible. Client and renderer should project state and issue intents, not mutate truth. Server/domain authority should own authoritative multiplayer truth. Silent fallback is forbidden; explicit degradation and refusal are required.

For building and destroying, the conversation concluded that the game should be **CAD-capable but not CAD-required**. Ordinary players should be able to

build through templates, snapping, intent-based actions, accessible controls, or low-performance interfaces.

Advanced players and developers can use a full CAD-like suite. All authoring surfaces should compile into the same canonical design artifact. The user explicitly does not want normal gameplay based on recipes, tech trees, or experience levels, though those systems should remain available for modders and alternate game modes through packs, law, profiles, and capabilities.

For geometry, the recommended direction was a hybrid rather than voxel-first: use CSG/feature graphs for editing and procedural authoring, B-Rep for canonical validated solids, NURBS/B-spline faces for advanced curved surfaces, meshes for derived rendering/collision, and sparse voxels/SDFs/fields only where they are the right tool, such as terrain, mining, fluids, debris, craters, and damage approximation.

For the full-scale universe dream, the key feasibility doctrine is: **full scale, yes; full fidelity everywhere, no**. Domino should use hierarchical domains, orbits on rails, planet tiles, sparse terrain/terraforming overlays, collapsed civilization capsules, fog-of-war epistemics, active-fidelity budgets, domain-sharded multiplayer authority, and proof-carrying client compute. Unreal can help with rendering/editor/client presentation if desired, but it should not be treated as the authoritative deterministic simulation substrate.

2. What We Actually Discussed

2.1 Project identity: Domino, Dominion, and XStack

The conversation began with the user asking for a total project description by reading the docs and the GitHub repo. The project was framed as a stack:

- **Domino engine:** deterministic mechanisms, execution, storage, domains, replay, capabilities, law gates, pack/runtime primitives, rendering/platform abstraction.
- **Dominium game:** rules and meanings built on top of the engine: construction, physical systems, life, agents, economy, governance, survival, war, knowledge, scale, and eventually CAD-like design and full-scale universe gameplay.
- **Client/server/product shells:** presentation, intent projection, multiplayer authority, launcher/setup/tools, AppShell surfaces.
- **XStack/governance:** RepoX/TestX/AuditX/CompatX-like validation, proof, and drift control.

This distinction became one of the most important pieces of the chat. The engine should be reusable for other games or even other engine projects. That means Dominion-specific concepts should not leak into the Domino substrate.

2.2 Historical engine lessons The user supplied a detailed comparison of revolutionary engines and technologies: SCUMM, Wolfenstein/Doom/Quake/id Tech, Build Engine, Unreal, RenderWare, Source, Havok, CryEngine, RAGE/Euphoria, Frostbite, Unity, id Tech 5, UE4, UE5, Decima, and related lineages.

The resulting lesson was not "copy Unreal" or "copy Doom." It was that historically successful engines made hard ideas shippable, toolable, reusable, and production-friendly. The enduring lessons were:

- represent the world so the engine can cheaply decide what matters;
- make game logic and content data-driven;
- use physics and systems only where they are controllable and meaningful;
- stream worlds and assets rather than loading everything;
- treat the toolchain as part of the engine;
- build workflows that designers, modders, and developers can actually use.

The answer mapped those lessons onto Domino: virtualize expensive systems, use active sets, event queues, sufficient statistics, and collapsed capsules; make

tooling and validation first-class; and do not build a monolithic renderer-first game engine.

2.3 Live repo reality and convergence The user then corrected that the live repo had much more work than previously discussed. More docs and source were read or referenced during the chat, including MVP scope lock, frozen invariants, AppShell, capability negotiation, semantic contracts, pack verification, registry compile, determinism envelope, and construction/fabrication source files.

This changed the answer. The project was no longer treated as a blank-slate architecture. It was treated as a real but unfinished system that needs convergence. The recommended direction became:

Turn Dominion from many good systems
into one coherent modular machine: contracts
-> registries -> compiled locks -> capabilities
-> Work IR -> deterministic runtime ->
proof/replay -> AppShell/product surfaces.

The advice shifted toward validation unification, provider descriptors, registry bundle grouping, generated docs, a platform matrix, pack/mod tooling, and AuditX/RepoX triage.

2.4 CAD, construction, and player freedom The user asked whether the game would need to provide a full CAD suite so players can build all items, machines, and structures. The answer separated **canonical design substrate** from **authoring surface**.

The recommended model is layered:

1. Low-skill players use templates, snapping, part placement, assisted actions, and text/intent commands.
2. Intermediate players modify parametric templates.
3. Advanced players edit constraints and assemblies.
4. Developers and expert players use CAD-grade design tools.
5. Generative assistants or scripts can propose designs, but those designs must still compile and validate into canonical artifacts.

This allows freedom without making CAD knowledge mandatory.

2.5 No default recipes, tech trees, or levels The user explicitly stated that usual gameplay should not depend on technology trees, recipes, or experience levels. The answer preserved this preference. The alternative

design is an affordance-and-constraint system: what a player can build depends on available materials, tools, processes, authority, law, environment, time, energy, and knowledge, not an arbitrary recipe unlock.

However, the engine should still support recipes, tech trees, levels, certifications, and similar systems for mods and alternate modes through packs, law profiles, capabilities, and policies.

2.6 Geometry: voxels versus CSG/B-Rep/NURBS

The user argued that Dual Universe went wrong by using a voxel-based engine and suggested B-Rep, CSG, NURBS, or a hybrid. The answer agreed that a pure voxel-first foundation would be a poor fit for precise machines, CAD semantics, interfaces, mass properties, and network-efficient authoritative objects.

The recommended model:

- **CSG / feature graph:** editable authoring and procedural history.
- **B-Rep:** canonical validated solid representation.
- **NURBS / B-splines:** optional advanced curved faces inside B-Rep.
- **Mesher:** derived rendering and collision proxies only.

- **Sparse voxels/SDF/fields:** terrain, mining, craters, fluids, fire, debris, damage approximation, volumetric phenomena.

Destruction should be event-driven and tiered: cosmetic damage, component damage, interface breakage, local cut/puncture, fracture/separation, and salvage/wreck states. The answer explicitly warned against doing full B-Rep booleans for every impact in real time.

2.7 Portability, modularity, and future-proofing

The user emphasized that all code should be portable, modular, extensible, and reusable for other games and engine projects, with directories and files replaceable if refactored. The answer said future-proofing does not come mainly from folders. It comes from contracts.

Key guidance:

- stable public APIs, flexible internals;
- public/private include boundaries;
- opaque handles and C ABI-like surfaces where needed;
- semantic contracts for behavior meaning;
- schemas for structure;
- capabilities for declared power;

- lockfiles for exact composition;
- proof/replay artifacts for actual execution;
- provider interfaces for replaceable backends;
- canonical serialization for persistent artifacts;
- golden replay/save/pack compatibility corpora.

A long-term directory shape was discussed, but the immediate advice was not to restructure broadly before baseline hardening.

2.8 Engine readiness and feasibility anxiety The user then expressed worry that the project might be impossible and that they did not know how to write engines. The first answer framed the repo as a working deterministic foundation but not a full playable engine. The user then pasted a much more precise local assessment showing a stronger but still blocked current reality.

The corrected conclusion:

- Yes, there is a working compiled deterministic substrate.
- No, there is not yet a finished playable game engine.

- Yes, it is ready for baseline hardening and narrow vertical slices.
- No, it is not ready for broad game feature expansion.

The biggest correction was inserting **Milestone 0** before any builder/destruction lab.

2.9 Full-scale universe, planets, civilizations, MMO, and Unreal The user described the full dream: full-scale solar systems, recreated and customizable planets, procedural/data-authored universe, civilization building, terraforming, cut/fill, megaprojects, machines, factories, fog of war, collapsed simulation for unsensed areas, client-shared compute, MMO persistent universe, single-player multiverse, and sparse construction/destruction.

The answer stated clearly that no conventional engine, including Unreal, can straightforwardly do the full version without major custom architecture. The only viable model is a deterministic multi-resolution universe:

- solar systems using orbits on rails initially;
- hierarchical planet domains and tiles;
- base procedural/data terrain plus sparse overlays;
- collapsed civilization capsules;

- fog of war as epistemic filtering, not just rendering;
- active simulation driven by sensing, interaction, due events, hazards, law obligations, and consistency;
- domain-sharded authority for MMO scale;
- clients allowed to contribute only derived/speculative/proof-carrying compute;
- proof/replay/hash validation for authoritative outcomes.

Unreal may be useful as a rendering/editor shell, but not as the authoritative deterministic simulation kernel.

3. What Was Decided

3.1 Strong decisions or accepted conclusions

- 1. Domino should remain the reusable deterministic engine substrate.**

Dominium should be one game/product built on top, not the reason the engine exists.

- 1. The current repo has a real deterministic substrate but not a finished playable game engine.**

This was sharpened after the user pasted the local readiness assessment.

1. **Milestone 0 comes before builder/destruction work.**

The first deliverable is one canonical repo-local playable baseline command passing strict validation.

1. **Do not start with full CAD, destruction, agents, economy, MMO, or broad renderer work.**

Those are downstream of baseline hardening.

1. **Normal gameplay should not rely on recipes, tech trees, or experience levels.**

Those can exist through packs/law/capabilities for mods and alternate modes.

1. **Full scale does not mean full fidelity everywhere.**

Full-scale solar systems and planets require collapsed sparse multi-resolution simulation.

3.2 Strong recommendations, not final decisions

1. Use a hybrid CSG/B-Rep/NURBS/mesh/SDF representation model for construction/destruction.
2. Treat CAD as an advanced tool surface, not required casual gameplay UI.
3. Use Unreal only as a possible client/editor/rendering shell, not the authoritative core.
4. Avoid broad repo restructuring until baseline proof exists.
5. Use a contract-compiled simulation platform model as the architecture north star.

These should be preserved as recommendations unless the user later explicitly confirms them as decisions.

4. What Was Put Off for Later

The chat explicitly or implicitly deferred:

- full CAD/B-Rep/CSG/NURBS implementation;
- arbitrary destruction;
- real chemistry/material simulation;
- economy/governance/war/agent loops;

- full vehicle/transport simulation;
- MMO-scale sharding and external multiplayer;
- full planet/civilization simulation;
- Unreal integration decision;
- broad directory restructure;
- full mod ecosystem;
- production renderer;
- client-shared authoritative compute;
- geometry provider selection;
- master Project Spec Book aggregation.

The reason these were deferred is not that they are unimportant. They were deferred because the local playable baseline path is not yet reliable. Feature work before baseline proof would increase uncertainty and project risk.

5. What We Actually Did in the Chat

The chat did not modify the repo. It did perform conceptual analysis and generated preservation artifacts.

Concrete things done:

- read and discussed repo docs and source files;
- identified the real project split between engine/game/product/governance;
- compared Domino against historical engine lessons;
- produced a modularity/future-proofing doctrine;
- proposed a CAD-capable layered authoring model;
- proposed a hybrid geometry/destruction strategy;
- corrected engine readiness after the user pasted local evidence;
- defined Milestone 0 as the immediate next milestone;
- described sparse full-scale universe/MMO feasibility architecture;
- generated a full preservation handoff package with reports, registers, spec sheet, aggregator packet, context transfer prompt, and ZIP;
- now generated this additional companion report and complete bundle.

6. What the Project Should Do Next

Immediate next sequence

1. **Re-verify current local state.**

Confirm the exact build/test status and capture logs.

1. **Fix the server/runtime circular import.**

Entry points should import runtime; runtime should not import entry points.

1. **Fix server CLI forwarding and public entrypoint behavior.**

The server help/smoke command should work cleanly.

1. **Make @@V10@@ pass for the intended baseline bundle.**

Resolve the `bundle.base.lab` versus MVP baseline seam explicitly.

1. **Fix the missing/invalid time-anchor policy registry.**

The baseline universe verifier should pass or refuse for an explicitly different reason.

1. **Create one canonical repo-local baseline playtest command.**

It should be an executable command, not a prose recipe.

1. **Make strict local playtest validation pass.**

`proof_status` should no longer be blocked.

1. **Only then build the tiny builder/destruction lab.**

Start with one accepted/refused build action, then one simple removal/damage action.

What the first gameplay slice should look like A realistic first gameplay slice should not be a full game. It should be:

- one small domain/world;
- one player/actor/authority context;
- one buildable part;
- one valid build intent;
- one invalid/refused build intent;
- one simple damage/remove action;

- one deterministic replay/hash result;
- one minimal client projection;
- one pack-driven object/action.

This is enough to prove the engine and game can actually cooperate.

7. Things Future Assistants Must Not Get Wrong

1. Do not say the playable game engine is finished.
2. Do not skip Milestone 0.
3. Do not treat CAD/destruction/MMO/universe architecture as near-term implementation scope.
4. Do not treat assistant recommendations as final user decisions.
5. Do not assume local pasted test results are audit-grade unless logs are uploaded or rerun.
6. Do not collapse "full scale" into "full fidelity everywhere."
7. Do not make Unreal the authoritative simulation core by default.
8. Do not reintroduce recipes/tech trees/levels as normal gameplay without explicit user approval.

9. Do not recommend broad directory restructuring before baseline proof.
 10. Do not forget that accessibility and low-performance support are core user concerns, not polish.
-

8. Key Open Questions

1. What exact code path causes the server/runtime circular import?
2. What command should become the canonical repo-local baseline playtest command?
3. Should the generic session default stay `bundle.base.lab`, while the playtest wrapper explicitly selects the MVP bundle?
4. What exact time-anchor policy registry artifact is missing or invalid?
5. What local validation logs should be preserved for audit-grade evidence?
6. What is the exact first build action after Milestone 0?
7. What is the exact DesignArtifact schema?

8. Which geometry provider/library approach should be used later?
 9. Should Unreal be used as a presentation shell at all?
 10. How should this chat's output be merged with other old-chat reports into the master Project Spec Book?
-

9. Summary of Package Contents

This complete bundle includes:

- the original full handoff package files generated earlier;
- the uploaded preservation prompt for provenance;
- the previous original ZIP package;
- this new companion human-readable report;
- a new combined package manifest with hashes and file status.

The earlier files remain the formal preservation package. This new companion report is intended to be easier to read and more narrative, while the earlier registers/spec sheets remain better for machine-assisted aggregation and planning.

10. Final Takeaway

The project is not impossible if it is treated as a sequence of small deterministic proofs. It is impossible if treated as a single giant engine/game to build all at once.

The correct next step is not to design the whole universe again. It is to make the baseline path real:

**one repo-local command, one session,
one loopback authority path, one
deterministic proof, one strict pass.**

After that, build one tiny piece of gameplay. Then another. Then another.

That is how this engine becomes real.

Chapter 37

Gui Binary Content

37.1 Dominium Gui Binary Content Handoff Reader Brief

Source: docs/archive/conversations/gui_binary_content/dominium_gui_binary_content_handoff__05_reader_brief.md

Report type: reader_brief

Size: 5212 bytes

37.1.1 Reader Brief - Dominium GUI/Binary + CONTENT0 Handoff

What This Chat Was About

This chat preserved two connected Dominium / Domino planning threads. The first was CONTENT0, a data/schema/docs-only canonical content prompt for universe, Milky Way, Sol, Earth, populations, civilizations, economies, technology, history, and start scenarios. The binding content principle is that nothing appears without causal provenance: entities must be created, populations born, cities built, resources extracted or

produced, and civilizations historically emerged. An assistant produced a cleaned CONTENT0 prompt, but the user then corrected the process: discussion must happen before planning or generating prompts. Treat that cleaned prompt as draft only.

The second and current main thread was GUI and binary planning. The user decided to redo GUIs from scratch. Every product should always work through CLI, is expected to work through TUI, and may have multiple modular GUIs attached to product backends to compile native applications. Products include setup, launcher, client, server, tools, and related products. Windows frontends proposed by the user: Win32 ANSI x86 for setup/client; Win32 Unicode x64/ARM64 for setup/client; WinForms .NET Framework family for launcher/server; WinUI 3 x64/ARM64 for launcher/server. macOS frontends proposed: AppKit for setup/launcher/client/server and SwiftUI for future launcher/server. Linux and Android are in scope but undecided.

The next chat should not start coding. It should first define GUI/binary doctrine, the shared backend/UI contract, product matrix, platform matrices, compatibility lanes, visual profiles, packaging strategy, and build/toolchain verification plan.

Most Important Things to Know

- GUI rebuild from scratch is an explicit user decision.
- CLI is mandatory for every product.
- TUI is expected for every product, but parity level is unresolved.
- Modular GUIs attach to shared product backends.
- Backend/UI contract is the first unresolved architecture issue.
- Windows frontend families are user-specified but need build-lane normalization.
- WinForms old .NET support needs real target/build lanes; verify before use.
- macOS AppKit/SwiftUI lanes are user-specified but macOS 10.9 needs verification/toolchain decision.
- Linux stack is not chosen.
- Android role/stack is not chosen.
- CONTENT0 is content-layer-only and provenance-driven.
- Assistant-generated CONTENT0 prompt is not final.

- Default datapack should include Milky Way and high-detail Sol.
- External platform/toolchain facts must be reverified.
- Do not treat assistant suggestions as user decisions.

Active Plans or Workstreams

- WORKSTREAM-06 - GUI and binary rebuild.
- WORKSTREAM-07 - Shared backend/UI contract.
- WORKSTREAM-08 - Windows frontend family matrix.
- WORKSTREAM-09 - macOS frontend family matrix.
- WORKSTREAM-10 - Linux frontend strategy.
- WORKSTREAM-11 - Android frontend strategy.
- WORKSTREAM-15 - TUI doctrine.
- WORKSTREAM-01 through WORKSTREAM-05 - paused content/universe/datapack work.

Decisions Already Made

- DECISION-01 - CONTENT0 is data/schema/docs-only.

- DECISION-02 - Content requires causal provenance.
- DECISION-03 - Discuss before generating further content prompts.
- DECISION-06 - Redo GUIs from scratch.
- DECISION-07 - CLI always works.
- DECISION-08 - TUI expected.
- DECISION-09 - Modular GUIs attach to backends.
- DECISION-10 - Windows frontend family list.
- DECISION-12 - macOS AppKit/SwiftUI family list.

Pending Tasks

- TASK-01 - Define GUI/binary doctrine.
- TASK-02 - Define backend/UI contract.
- TASK-03 - Build product matrix.
- TASK-04/TASK-05 - Build Windows/macOS matrices.
- TASK-06/TASK-07 - Decide Linux/Android strategies.
- TASK-08 - Define TUI parity.

- TASK-10/TASK-11 - Define packaging and build/toolchain matrix.
- TASK-12 - Inspect repo/files before implementation.

Open Questions

- What is the backend/UI contract shape?
- Should stable C ABI be used?
- Is the client host-native, renderer-driven, or hybrid?
- What TUI parity level is required?
- What exact Windows and .NET floors are required?
- Is macOS 10.9 a hard requirement?
- What Linux GUI stack should be used?
- What is Android's role?

Files / Artifacts / Prompts to Preserve

- ARTIFACT-01 - Original CONTENT0 prompt.
- ARTIFACT-02 - Assistant cleaned CONTENT0 draft, draft only.

- ARTIFACT-03 - Assistant CONTENT0 critique.
- ARTIFACT-07 - User GUI rebuild decision and frontend list.
- ARTIFACT-10 - Previous Context Transfer Packet.
- ARTIFACT-19 - This final report package.

What to Verify Before Acting

- Actual repository/files.
- Windows App SDK / WinUI 3 support floor.
- .NET Framework targeting and Visual Studio support.
- macOS/Xcode deployment floors and old SDK availability.
- Linux toolkit/package strategy.
- Android product role and API floor.
- TUI parity expectations.
- Backend contract feasibility.

Best Next Step

Define the GUI/binary architecture doctrine and backend/UI contract first, then map products and

platforms into matrices. Do not generate code or final prompts until those decisions are explicit.

Chapter 38

Launcher App Layer

38.1 Dominium Launcher App Layer Handoff Reader Brief

Source: docs/archive/conversations/launcher_app_layer/dominium_launcher_app_layer_handoff__05_reader_brief.md

Report type: reader_brief

Size: 4743 bytes

38.1.1 Reader Brief - Dominium Launcher Application Layer Handoff

What This Chat Was About

This chat focused on the Dominium / Domino application layer, especially the Dominium Launcher. It began with the user asking whether the post-Plan 8 system could support a fast, native-feeling, cross-platform launcher with one common codebase and minimal/no renderer, inspired by pre-2016 Minecraft launcher examples. The discussion evolved into a deeper launcher architecture and repository-boundary

conversation.

The major outcome is that older standalone launcher layouts and IDE-project advice were superseded by a canonical product-first monorepo structure. The launcher is now a first-class product under `launcher/`, not an engine subcomponent. Engine purity is locked: engine must not include or export `launcher/setup/tools/contracts`. Cross-product contracts belong in `libs/contracts/include/dom_contracts`, and canonical schemas live in `schema/`. Setup is the only install mutation authority; launcher consumes setup output through `schemas/manifests/contracts` only.

The user later pasted application-layer and overall project canon from another chat. That canon says architecture is closed, applications are orchestration shells, CLI is canonical, GUI/TUI are views over the same command graph, UI is data via UI IR, accessibility/localisation are mandatory, tools are read-only by default, and RepoX/TestX/BUILD-ID-0 enforcement must not be bypassed.

Two Codex prompts were said to have been applied: one for CMake/VS2026/build repair and one for engine purity/contract ownership. Their results must still be verified against the repo. The likely next action is to run verification gates, then execute or audit the generated "Launcher Spec Completion, Gap Closure, and Documentation Hardening" prompt.

Most Important Things to Know

- Architecture is closed; do not redesign.
- Launcher is first-class under `launcher/`.
- Canonical launcher structure: `core/discover`, `core/profile`, `core/invoke`, `ui`, `platform`, `tests`.
- Engine purity is non-negotiable.
- Engine must not include `dom_contracts`.
- Cross-product contracts live in `libs/contracts/include/dom_contracts`.
- Setup is the only install mutation authority.
- Launcher/setup communicate only via schemas, manifests, and contracts.
- CMake is authoritative for VS2026; manual IDE-project advice is superseded.
- CLI is canonical.
- GUI/TUI are views over the same command graph.
- UI is data via UI IR, never business logic.
- Accessibility and localisation are mandatory.
- Apps must run with zero content packs installed.

- RepoX generates changelogs and compatibility/build metadata.
- BUILD-ID-0 versioning and mismatch refusal are locked.
- Actual repo state remains unverified in this package.

Active Plans or Workstreams

- Launcher product hardening.
- Engine purity verification.
- Setup/launcher boundary enforcement.
- Neutral contracts and schema usage.
- UI command graph/UI IR validation.
- CMake/VS2026/CI gate verification.
- RepoX/TestX/BUILD-ID-0 integration.

Decisions Already Made

- Launcher is not in engine.
- CMake is the authoritative build graph.
- Setup alone mutates installs.
- CLI is canonical.

- UI IR is required.
- Tools are read-only by default.
- Manual changelogs are rejected.

Pending Tasks

- Run sanity scripts and build/test gates.
- Determine whether launcher hardening prompt was applied.
- Verify CANON_INDEX and docs status headers.
- Verify command graph and UI IR implementation.
- Verify UI binding validator and zero-pack tests.
- Verify RepoX/build identity launcher behavior.

Open Questions

- Did applied Codex prompts fully succeed?
- Has launcher hardening already run?
- Do command graph/UI IR/binding validator exist?
- Does launcher run with zero packs and missing locale?
- Does launcher refuse BUILD-ID mismatches?
- Are any shared headers ambiguously owned?

Files / Artifacts / Prompts to Preserve

- The two applied Codex prompts.
- The launcher hardening prompt.
- Canonical repo update prompt.
- Application-layer canon prompt.
- Latest overall canon prompt.
- Previous Context Transfer Packet.
- Uploaded Plan 8 file/repo archive if available.
- Minecraft launcher reference images as UX background only.

What to Verify Before Acting

- `scripts/verify_tree_sanity.bat`
- `python scripts/verify_includes_sanity.py`
- `cmake --preset vs2026-x64-debug`
- `cmake --build --preset vs2026-x64-debug`
- `ctest --preset vs2026-x64-debug`
- `launcher --help`
- `setup --help`

Best Next Step

Run the verification gates, then execute or audit the launcher spec completion/hardening prompt. Do not generate new feature plans until launcher code/docs are confirmed up to spec.

Chapter 39

Omega Xi Pi Architecture Future

39.1 Dominion Omega Xi Pi Architecture Future Proofing Planning Human Readable Report

Source: docs/archive/conversations/omega_xi_pi_
architecture_future/dominium_omega_xi_pi_
architecture_future_proofing_planning__01_
human_readable_report.md

Report type: human_readable_report

Size: 36314 bytes

39.1.1 COMPLETE CHAT PRESERVATION REPORT - Dominion Omega/Xi/Pi Architecture & Future-Proofing Planning

0. Coverage and Reliability Assessment

- Row 1** Field: Chat label; Assessment: Dominion Omega/Xi/Pi Architecture & Future-Proofing Planning
- Row 2** Field: Date anchor; Assessment: 2026-05-27 Australia/Melbourne
- Row 3** Field: Source scope; Assessment: This chat only unless labelled PROJECT-CONTEXT
- Row 4** Field: Apparent access; Assessment: Partial-to-broad. I can see a large portion of the conversation, the active in-memory context, and uploaded snapshot/prompt files, but I cannot guarantee literal access to every original token in the full old transcript.
- Row 5** Field: Previously generated files available?; Assessment: Yes, some are present in /mnt/data during this response: `NEW.txt`, `dir (1).txt`, and `Pasted text.txt`. Many repo files referenced in the chat live in the user's local/public GitHub repo, not in this runtime.

- Row 6** Field: Uploaded files or artifacts present?; Assessment: Yes. A repo directory snapshot (`NEW.txt / dir (1).txt`) and a pasted preservation instruction (`Pasted text.txt`) are present.
- Row 7** Field: Contains future plans?; Assessment: Yes, extensive: Omega, Xi, Pi, ?, Sigma, Phi, Upsilon, Zeta, plus release/distribution/runtime/component/agent-governance planning.
- Row 8** Field: Contains decisions?; Assessment: Yes, many strategic decisions and tentative/final planning decisions.
- Row 9** Field: Contains pending tasks?; Assessment: Yes, chiefly ?-series next, then Sigma/Phi/Upsilon/Z after snapshot reconciliation.
- Row 10** Field: Contains unresolved questions?; Assessment: Yes: current repo truth, status of `src`, exact readiness of future prompts, current vendor-agent docs, exact final mapping from blueprint to live repo.
- Row 11** Field: Staleness risk; Assessment: Medium to High. Repo state, GitHub state, and vendor AI tooling docs may change.
- Row 12** Field: Extraction confidence; Assessment: 4/5 for major themes and plans; 3/5 for exact current repo state without live verification.

Row 13 Field: Safe for later aggregation?; Assessment: Yes, with caveats: mark repo-specific claims as user-reported or snapshot-dependent unless verified.

Row 14 Field: Main limitations; Assessment: Some earlier uploaded files expired at times; the visible conversation is very large and not guaranteed token-complete; many current repo facts were user-reported, not independently verified in this preservation response.

Plain-language limitations: this preservation reconstructs the substance of this chat from the visible conversation, the assistant's model context, user-reported repo status, and the currently uploaded files. It should not be treated as a verified audit of the public GitHub repo as of 2026-05-27 unless a future assistant browses/inspects that repo. The uploaded directory snapshot is valuable evidence of repo structure and drift concerns, but parts of it may be stale relative to the user's latest pushed `main`.

1. One-Page Orientation

This chat was a long-running architectural planning and continuity-preservation conversation for the user's *Dominium* project. *Dominium* began in the conversation as a game/engine project, but the user and assistant

progressively reframed it as something larger: a deterministic simulation platform, reusable engine, runtime/service host, package ecosystem, product family, agent/human development environment, and release/archive/control plane. The engine layer was repeatedly referred to as Domino; Dominion is the game/product layer built on top of it.

The core user concern throughout the later parts of the chat was future-proofing. The user wanted to know how to make the code portable, modular, extensible, reusable for different games on the same engine, reusable for different engine projects, able to survive rewrites/refactors of entire directories, and engineered like a proper game or operating system rather than a one-off indie project. This led to a doctrine that became the spine of the chat: stable contracts, replaceable implementations, deterministic behavior, manifest-based identity, tool-agnostic development, XStack-enforced architecture, and human-readable plus machine-readable everything.

A huge amount of planning was produced. Earlier material covered MVP scope, Earth/Sol/Galaxy simulation stubs, CLI/TUI/GUI/AppShell concerns, capability negotiation, pack compatibility, library/install/save management, diagnostics and repro bundles, release/distribution, meta-stability, time anchors, architecture audits, universal identity, migration

lifecycle, numeric discipline, concurrency, observability, store GC, governance, performance envelopes, archive policy, and final distribution. Later, those became grouped into Omega-series for ultimate MVP/runtime/distribution freezing, Xi-series for repository convergence and drift immunity, and Pi-series for future-series meta-blueprint planning.

The user later reported that substantial parts of these plans had already been executed in the public GitHub repo `julesc013/dominium/main`. In particular, the user reported Xi-6 (architecture graph freeze), Xi-7 (XStack CI guardrails), Xi-8 (repository structure freeze), and Pi-0/Pi-1/Pi-2 (meta-blueprint / execution strategy / final prompt inventory) as complete, with specific commit hashes and fingerprints. These are important as user-reported state, but a future assistant should verify current GitHub state before making implementation-specific claims.

A major thread was concern about AI/human drift: Codex or other agents may have implemented temporary code under `src/` to make things compile, duplicating older code or bypassing intended modules. This produced the Xi-series: architecture graph generation, duplicate implementation detection, scoring, convergence planning, mechanical refactor, `src` removal, architecture graph freeze, CI guardrails, and repository freeze. The uploaded repo snapshot shows why this matters: it

contains both strong domain roots such as `engine`, `game`, `client`, `server`, `setup`, `launcher`, `packs`, `schema`, `tools`, and `docs`, and older/generic pockets such as `src`, `app/src`, `libs/*/src`, `legacy/source`, and `tools/ui_shared/src`.

The latest strategic direction is not to implement the next advanced features directly. Instead, the next step is the `?`-series: snapshot-driven final planning. This will consume current repo reality and the completed Pi blueprint to decide what already exists, what should be kept, extended, merged, replaced, or quarantined, and only then produce final repo-specific plans for Sigma (agent/human governance), Phi (runtime component/service/kernel architecture), Upsilon (build/release/control plane), and Zeta (future live operations such as hot-swappable renderers, live schema migration, distributed shard relocation, live mod activation, and deterministic canary cutovers).

A future assistant must understand that this chat is not mainly a feature list; it is a systems architecture and project-governance blueprint. The user wants ambitious long-term capabilities, but only if they are built through contracts, validators, proof gates, stable identities, explicit migrations, and XStack enforcement. The right next action is to continue with `?`-series planning or verify the live repo before generating final Sigma/Phi/Upsilon/Z execution prompts.

2. The Story of the Conversation

2.1 From MVP feature planning to deterministic platform thinking The conversation began around MVP readiness: whether the engine, game, client, server, setup, launcher, CLI/TUI/GUI, and distribution paths were ready. The user wanted every product to run standalone and portably, with GUI where available but CLI/TUI as reliable baselines. At first the discussion sounded like game/engine distribution planning, but it quickly expanded into a much broader platform problem: every executable should work on its own, every artifact should be portable, and every user-facing mode should degrade cleanly.

The user then questioned whether Earth/Sol/Galaxy systems were realistic enough: sky, night/day cycles, tides, lakes, rivers, hydrology, mountains, geology, sounds, climate, collisions, pathfinding, epistemics, freecam, godmode, and so on. The assistant helped separate MVP scope from future simulation scope. The key decision was not to implement full fluids, chemistry, or geology in v0.0.0, but to add minimal proxy/stub layers such as material proxy, surface flags, albedo proxy, unified illumination geometry, orbit visualization, galaxy metadata proxies, and compact object stubs. These were chosen because they future-proof interfaces without exploding scope.

2.2 Compatibility and ecosystem discipline The user repeatedly pushed on version compatibility: arbitrary versions of engines, games, clients, servers, launchers, setup tools, packs, saves, profiles, schemas, and protocols should be mix-and-matchable when possible, and should degrade/refuse deterministically when not. This led to a dense set of ecosystem mechanisms: semantic contracts, capability negotiation, pack compatibility manifests, content-addressed stores, install/instance/save separation, universal identity blocks, migration lifecycle policies, release indices, component graphs, install profiles, update models, trust models, target matrices, archive policy, yanking, rollback, and latest-compatible resolution.

The important conceptual shift was that the suite version should remain as a curated tested snapshot, but it should not determine compatibility. Products, protocols, schemas, formats, packs, and releases all need independent versions and identities. Compatibility should come from contract ranges, capability negotiation, protocol ranges, migration policies, and trust rules.

2.3 Distribution and release professionalisation

The release/distribution planning began as packaging and archiving but became much more complete. DIST was reframed as the final polish and operational integrity layer that proves a real product ecosystem. It should

validate portable installs, installed mode, standalone products, mixed-version negotiation, export/import, offline verification, clean-room runs, cross-platform packaging matrices, UX polish, interop tests, deterministic archives, release indices, archive records, mirror policies, old-version preservation, and setup/update behavior.

The user then asked how to handle individual product versions, suite versions, downloadable components, commercialisation, paid features, closed-source engine/game with open-source tools, forks, and handoff/public-domain transitions. The resulting doctrine: keep suite and individual versions; make Setup a package-manager/orchestrator; use component graphs and release indices; support exact-suite and latest-compatible policies; use signed capability artifacts for paid features; keep protocols/schemas open if desired; avoid reliance on GitHub/toolchains; and make artifacts content-addressed and archived.

2.4 Repository drift crisis and Xi-series A central late-chat problem was the user's worry that Codex may have reimplemented systems from scratch under `src/` just to make the repo compile, instead of extending or replacing existing code. The user did not know which implementations were best, whether docs were correct, or whether unique approaches needed to be merged. The

assistant argued that the solution was not guessing, but repository archaeology and convergence: build an architecture graph, index symbols, detect duplicates, score implementations, produce a convergence plan, execute it safely, remove `src`, enforce architecture graph v1, add CI guardrails, and freeze repository structure. This became the Xi-series.

The user later reported that Xi phases had been implemented: residual `src` blockers resolved or classified, architecture graph v1 frozen, RepoX/AuditX rules added, ControlX planning helper added, XStack CI profiles integrated, and repository structure frozen. Specific commits and hashes were provided. These reports became authoritative user statements but should still be verified against current GitHub if relied upon.

2.5 Agent governance and vendor-neutral human/agent interface The user asked whether to take advantage of `AGENTS.md` and other agent-specific features. The assistant recommended not building a vendor-specific AI framework, but instead using a canonical, tool-agnostic repo contract plus generated mirrors for Codex, Claude, Copilot, and future systems. `AGENTS.md` should be a short normative entry point, but not the source of truth; it should point to architecture graph, module boundaries, repo structure lock, contracts, schemas, registries, and XStack gates. The future Sigma-

series would create this governance layer, mirrors, task catalogs, MCP wrappers, and safety policies.

2.6 OS-like runtime architecture and future live operations The user brought in advice from another GPT conversation about runtime components, render devices, framegraph, shader IR, platform services, plugin modules, capability tiers, and module manifests. The assistant agreed directionally and reframed Dominion as similar to an OS kernel architecture: engine kernel, runtime services, modules/drivers, applications/shells, content packages, tools, and release/control plane. This led to Phi-series planning for runtime componentization and a large Zeta-series of future live operations.

The user listed an extremely ambitious Z capability list: hot-swappable renderers, live module reload, restartless core replacement, fully live save migration, distributed shard relocation, live trust-root rotation, canary releases, deterministic operator playbooks, multi-render-backend mirrored execution, live pack mount/unmount, mod quarantine, stateful service mirroring, checkpoint/event-tail synchronization, deterministic distributed simulation, cluster failover, and more. The assistant grouped these into families and argued that the real innovation is deterministic live operations: live changes with explicit plans, capability/trust/compatibility checks, proof anchors, rollback, and replayable evidence.

2.7 Pi-series and snapshot-driven next step After Xi, the assistant planned Pi-series as a meta-blueprint: architecture diagrams, dependency maps, capability ladders, readiness matrices, prompt inventories, and snapshot mapping templates. The user reported Pi-0/Pi-1/Pi-2 were completed and restarted from Xi-8 ground truth, with fingerprints and prompt counts.

The final strategic direction before this preservation request was to run a ?-series: snapshot intake, reality extraction, blueprint reconciliation, foundation readiness, and final prompt synthesis. This is needed because plans must now be mapped onto current repo reality rather than executed abstractly. The next chat should pick up there.

3. Main Topics Discussed

Topic 1 - Dominion as deterministic simulation platform This topic is the conceptual core of the chat. The user repeatedly asked how to make the project modular, portable, extensible, reusable, and future-proof. The assistant framed Dominion not just as a game but as a deterministic simulation platform with OS-like properties. This matters because it determines every later decision: stable boundaries, replaceable implementations, runtime service architecture, package graphs, versioned schemas, proof anchors, and governance tooling.

The conclusion was that Dominion's long-term value lies in the contracts and ecosystem, not in any single current file tree or implementation. The user should remember that this framing is an architectural discipline: keep simulation truth deterministic, use interfaces and manifests, and make everything replaceable behind explicit contracts.

Topic 2 - MVP scope and avoiding feature

explosion The user asked if the MVP needed full materials, fluids, molecules, dirt, concrete, wood, chemistry, hydrology, geology, sky, weather, tides, and more. The assistant repeatedly distinguished between structural stubs and full simulation features. The decision was to avoid implementing full fluids, chemistry, economy, vehicles, crafting, ecology, and other domains before v0.0.0. Instead, minimal future-proofing stubs were proposed: `material_proxy`, `surface_flags`, `albedo_proxy`, unified illumination geometry, orbit proxies, galaxy metadata proxies, and compact object stubs.

This remains important because the user has a tendency to expand scope ambitiously. The project should keep adding foundations before feature-heavy domains.

Topic 3 - Product standalone behavior and

AppShell The user wanted each product to work standalone: engine, game, client, server, setup, launcher,

tools. The assistant proposed deterministic UI mode selection and AppShell bootstrap: every product should support CLI, TUI, rendered GUI or OS-native GUI where appropriate, with fallback chains based on invocation context and capabilities.

The conclusion was that all products should boot through AppShell, use shared command registries, expose descriptors, validate packs/contracts before session start, and never have ad hoc boot paths. This matters for portability, setup/launcher integration, and future distribution.

Topic 4 - Compatibility, versioning, and release ecosystem The user wanted arbitrary versions of products, protocols, packs, saves, profiles, and forks to work together when possible. The assistant developed a multi-axis versioning model: suite releases as curated snapshots; products with independent SemVer/build IDs; protocols, schemas, formats, semantic contracts, packs, and component graphs versioned independently; compatibility via ranges and negotiation, not equality.

The user later asked whether suite version should be removed. The answer was no: keep suite version as a human-facing tested baseline, but do not use it as runtime compatibility authority.

Topic 5 - Distribution, update, trust, archive, commercialisation Release and distribution planning expanded into a package ecosystem. Setup should be a small bootstrap/package manager/orchestrator. The release index should list components across platforms and versions. Component graphs and install profiles drive exact-suite, latest-compatible, and lab installs. Trust model supports signed artifacts, unsigned default/mock warnings, strict refusal, and future paid capability artifacts. Archive policy preserves old artifacts and release index history.

Commercialisation was discussed: paid features should not be hidden by obscurity but controlled by signed license/capability artifacts and trust/capability policies. Open/closed splits are possible if protocols/schemas/ABI boundaries are clean.

Topic 6 - Repository convergence and drift prevention This topic arose from the user's concern about `src/` and duplicated Codex work. The assistant recommended a repository archaeology pipeline and later the Xi-series. The key insight was to use XStack as the architecture immune system: RepoX structural invariants, AuditX drift scans, ControlX convergence planning, TestX deterministic verification.

The user later reported Xi completion. The enduring lesson is that prompt instructions are not enough;

architecture must be machine-readable and enforced.

Topic 7 - Agent governance across

GPT/Claude/Copilot/Codex The user wanted agent support to work across vendors. The answer was to define a canonical vendor-neutral layer and generate vendor-specific mirrors. Canonical files include `AGENTS.md`, `.agentignore`, `data/agents/agent_context.json`, and `docs/agents/AGENT_TASKS.md`; mirrors may include Copilot instructions, Claude agent files, Codex skills/MCP wrappers. XStack, not agent prose, enforces truth.

Topic 8 - Runtime microkernel/component/service model

The assistant accepted earlier advice about runtime components, render devices, framegraph, shader IR, asset pipelines, platform service abstraction, and module manifests. The long-term Phi-series was defined to formalize runtime/kernel doctrine, components, module loader, runtime services, state externalization, lifecycle manager, framegraph, render device, asset pipeline, sandboxing, multi-version coexistence, event log, snapshot service, and distributed authority.

Topic 9 - Zeta live operations and extreme future capabilities

The user listed a very large set of live operations. The assistant grouped them into

replaceability, live state movement, rollout/control, distributed simulation, content/mod live operations, and resilience/validation. The core principle: implement foundations first, especially lifecycle manager, state externalization, event log, snapshot service, capability negotiation, trust policy, and operator transaction log.

Topic 10 - Final handoff/preservation The user asked for a comprehensive handoff and later an even more complete preservation package. This response is itself the preservation package, intended to let a new chat continue without re-explaining everything.

4. What We Were Trying to Achieve

4.1 Explicit Goals

1. Make Dominion/DOMINO portable, modular, extensible, and reusable for other games and engine projects.
2. Allow entire code/data/files/directories to be replaced or rewritten while preserving stable contracts and behavior.
3. Develop the project like a proper game/OS/platform, not a one-off indie project.
4. Preserve all plans, tasks, constraints, risks, decisions, artifacts, and future directions across

chats.

5. Prevent AI/human drift.
6. Support standalone products, portable and installed distributions, and long-term archives.
7. Support future live operations such as hot-swappable renderers and live mod activation, but only after foundations.
8. Support GPT/Claude/Copilot/Codex and humans through tool-agnostic governance.

4.2 Inferred Goals

1. Avoid losing value from earlier Codex work while still correcting structural drift.
2. Avoid massive rewrites unless evidence proves them necessary.
3. Make the repo robust to future tool/vendor changes.
4. Preserve ambition without sacrificing determinism or maintainability.
5. Eventually build a Workbench/AIDE environment for human and agent production workflows.

4.3 Goals That Changed Over Time The conversation shifted from MVP feature readiness to ecosystem and repo architecture. It started with questions about CLI/TUI/GUI and world realism, then moved into compatibility, distribution, repo convergence, agent governance, runtime componentization, and live operations. The goal changed from "what features should MVP have?" to "how do we build a platform that can evolve for decades?"

4.4 Goals Still Unresolved

1. Current repo reality must be reconciled with the blueprint.
2. Final Sigma/Phi/Upsilon/Z execution plans must be generated from current repo reality.
3. Exact runtime component boundaries must be mapped to existing code.
4. Agent governance and vendor mirrors must be implemented.
5. Build/release preset drift must be audited and consolidated.
6. Long-term live operations remain future work.

5. Decisions Made and Why

Row 1 ID: DECISION-01; Decision: Treat Dominium as deterministic simulation platform / OS-like runtime, not merely game engine; Status: Final direction; Why it mattered: Guides all architecture; Confidence: High; Label: FACT

Row 2 ID: DECISION-02; Decision: Preserve suite version as curated snapshot, but decouple from compatibility; Status: Final; Why it mattered: Avoid brittle version equality; Confidence: High; Label: FACT

Row 3 ID: DECISION-03; Decision: Version products/protocols/schemas/formats independently; Status: Final; Why it mattered: Enables mix-and-match; Confidence: High; Label: FACT

Row 4 ID: DECISION-04; Decision: Use CAP-NEG/contracts/migration/trust for compatibility; Status: Final; Why it mattered: Compatibility must be explicit; Confidence: High; Label: FACT

Row 5 ID: DECISION-05; Decision: Avoid generic source-code `src` roots; Status: Final direction; Why it mattered: Prevents drift and shadow modules; Confidence: Medium-High; Label: FACT with repo-state caveat

- Row 6** ID: DECISION-06; Decision: Use XStack existing components instead of adding ArchX; Status: Final; Why it mattered: Avoids subsystem sprawl; Confidence: High; Label: FACT
- Row 7** ID: DECISION-07; Decision: Use AGENTS.md as guidance, not source of truth; Status: Final; Why it mattered: Vendor-neutral and safer; Confidence: High; Label: FACT
- Row 8** ID: DECISION-08; Decision: Run ?-series before Sigma/Phi/Upsilon/Z implementation; Status: Final recommended next step; Why it mattered: Grounds plans in live repo; Confidence: High; Label: FACT
- Row 9** ID: DECISION-09; Decision: Do not implement Z directly before foundations; Status: Final; Why it mattered: Avoids hacks; Confidence: High; Label: FACT
- Row 10** ID: DECISION-10; Decision: Packs should be data/contracts, not silent executable authority; Status: Final direction; Why it mattered: Security and determinism; Confidence: High; Label: FACT
- Row 11** ID: DECISION-11; Decision: Use C-compatible ABI at frozen boundaries; Status: Strong recommendation; Why it mattered: Portability/open-closed split; Confidence: Medium; Label: INFERENCE/accepted direction

Row 12 ID: DECISION-12; Decision: Workbench should start with command/validation spine, not full GUI; Status: Recommended direction; Why it mattered: Avoids GUI-first trap; Confidence: Medium; Label: INFERENCE

Detailed rationale is covered throughout sections 2-8.

6. Ideas Considered but Rejected, Superseded, or Deprioritised

1. **Remove suite version entirely.** Rejected. Suite version remains useful as a curated snapshot, but not a compatibility authority.
2. **Per-product long-lived branches.** Rejected. Recommended single trunk/main with short-lived feature branches and independent product versioning.
3. **Create a new ArchX subsystem.** Rejected. Use existing RepoX/AuditX/ControlX/TestX with architecture graph artifact.
4. **Implement full materials/fluids/chemistry before MVP.** Rejected/deprioritised. Use proxy stubs for future-proofing.
5. **Implement Z hot-swap/live operations immediately.** Rejected/deprioritised. Requires Phi/Upsilon foundations.

6. **Build Workbench GUI first.** Rejected/deprioritised. Command/package/presentation floors first.
7. **Use @@V10@@/@@V11@@ as active code root.** Rejected for active code. Content provenance **source** pockets remain valid if policy-classified.
8. **Optimize for one AI vendor's instruction format.** Rejected. Use canonical governance + generated mirrors.
9. **Treat documentation as unquestioned authority.** Rejected. Docs can drift; code/artifacts/XStack must reconcile reality.
10. **Make every file stable.** Rejected. Stable boundaries, replaceable files.

7. Important Reasoning, Rationale, and Tradeoffs

The central tradeoff was between ambition and discipline. The user wanted extreme future capabilities: restartless updates, distributed shards, live mods, live trust-root rotation, hot-swappable renderers. The assistant repeatedly argued these should not be implemented as direct features. Instead, they should emerge from deep primitives: lifecycle manager, state externalization, event log, snapshot service, capability

negotiation, trust/migration policy, and operator transaction logs.

Another major tradeoff was repo elegance versus actual code reuse. The assistant repeatedly warned against a new grand rewrite. The right process is evidence-based: inspect the live repo, keep existing good subsystems, extend them, merge duplicates, replace only when necessary, and quarantine ambiguity.

The `src` issue illustrates the reasoning. A generic `src` root is not bad in all projects, but in *Dominium* it hides ownership and allowed Codex/humans to create shadow implementations. Therefore, the policy became ownership-rooted layout rather than language-oriented layout. However, the discussion also preserved nuance: `packs/source` can be valid because it means raw content provenance, not active code.

The agent-governance reasoning followed the same pattern. `AGENTS.md` is useful, but prompts and agent docs are not authoritative. They must point to machine-readable artifacts and executable XStack gates. This keeps the repo usable by GPT, Claude, Copilot, Codex, future agents, and humans without locking to any vendor.

8. Plans, Future Work, and Next Steps

Immediate recommended sequence

1. **?-0 Snapshot Intake**
2. **?-1 Reality Extraction**
3. **?-2 Blueprint Reconciliation**
4. **?-3 Foundation Readiness**
5. **?-4 Final Prompt Synthesis**
6. Then final Sigma/Phi/Upsilon/Z planning/
execution in repo-specific order.

Why ? first Because current repo reality may differ from both stale snapshots and the blueprint. The project must avoid executing theoretical plans against wrong assumptions.

Expected ? outputs

- Normalised snapshot/repo intake.
- Module/product/build/doc/tool/preset inventory.
- Keep/extend/merge/replace/quarantine decisions.
- Prompt readiness classification.
- Final repo-specific prompt plans.

Main blockers

- Current GitHub state must be verified.
- Stale uploaded snapshots may mislead.
- Some future prompts may already be partially implemented or obsolete.

9. Constraints, Preferences, and Non-Negotiables

9.1 Explicit Constraints and Preferences

- Preserve FACT / INFERENCE / UNCERTAIN labels.
- Do not invent or flatten uncertainty.
- Do not treat brainstorming as decisions.
- Preserve rejected/superseded options.
- Provide exhaustive structured output.
- Support future aggregation/spec-book generation.
- Code must be portable/modular/extensible/reusable.
- Develop like proper OS/proper game platform.

9.2 Inferred Constraints and Preferences

- User values ambitious but gated planning.
- User wants Codex-ready prompts.
- User wants no repeated re-explanation.
- User prefers architecture diagrams and registries over loose prose.

9.3 Uncertain or Unestablished Preferences

- Exact preferred physical directory layout after latest repo changes should be verified.
- Exact priority among Sigma/Phi/Upsilon after ? may need confirmation if multiple are ready.

10. Files, Artifacts, Outputs, and Prompts

Uploaded files currently present

1. `NEW.txt / dir (1).txt`: repo directory snapshot. It shows domain roots and also generic `src/legacy/` source pockets. It is useful as drift evidence but may be stale.
2. `Pasted text.txt`: the user's preservation-package instruction. It is the immediate task specification for this response.

Important repo artifacts reported by user See section 9 of the handoff above for full list. These include Xi/Pi outputs such as `architecture_graph.v1.json`, `module_boundary_rules.v1.json`, `repository_structure_lock.json`, `FINAL_PROMPT_INVENTORY.md`, and `final_prompt_inventory.json`.

Prompts/series artifacts generated in this chat

The chat generated detailed prompts for many series. Key future prompt sets: Omega, Xi, Pi, ?, Sigma, Phi, Upsilon, Zeta. These should be preserved in any master spec book.

11. Open Questions and Unresolved Issues

1. Is current GitHub `main` exactly as reported by user?
2. Does current repo have any active `src` roots?
3. Which Sigma/Phi/Upsilon prompts are already partially implemented?
4. Which runtime/service/component concepts map to existing code?
5. Which build/release scripts/presets are canonical?
6. How should stable ABI prefixes/names be finalised?
7. What should be the first second-game proof target?

8. How much vendor-agent integration already exists?
9. Which Z capabilities are truly foundation-ready after Phi/Upsilon?

12. Risks, Failure Modes, and What Future Chats Might Get Wrong

1. Treating stale snapshots as current.
2. Restarting Omega/Xi/Pi instead of continuing.
3. Assuming user-reported commits are verified.
4. Treating assistant suggestions as final decisions.
5. Over-implementing Z early.
6. Creating another grand directory redesign.
7. Ignoring XStack.
8. Making AGENTS.md authority instead of guidance.
9. Forgetting suite/product/protocol/schema version separation.
10. Freezing files instead of contracts.

13. What This Chat Contributes to the Larger Project or Future Spec Book

This chat should feed into chapters on:
2000

- Project doctrine and architecture philosophy.
- MVP/runtime/distribution freeze plan.
- Repository convergence and drift prevention.
- Agent/human governance.
- Runtime component/kernel model.
- Build/release/control-plane evolution.
- Live operations roadmap.
- Coding/naming/API/schema practices.
- Future-proofing and portability.

It should not be merged as final implementation fact without current repo verification.

14. What I Should Remember

- The next phase is ?-series.
- Stable contracts, replaceable implementations.
- XStack is the immune system.
- `src` drift was a central concern.
- Use current repo reality, not stale snapshots.
- Do not implement Z before foundations.

- Agent governance must be vendor-neutral.
- Workbench should start with command/validation spine.
- The user wants OS-grade architecture, not indie one-off architecture.

15. Best Questions I Can Ask Next in This Chat

15.1 Understanding the Chat

1. "Explain the difference between Omega, Xi, Pi, ?, Sigma, Phi, Upsilon, and Zeta again."
2. "What is the single highest-level doctrine from this chat?"

15.2 Decisions

1. "Which decisions were final and which were tentative?"
2. "Why did we reject per-product long-lived branches?"

15.3 Tasks and Next Actions

1. "Generate ?-0 now."
2. "What exact inputs should I provide for ?-1?"

15.4 Artifacts and Files

1. "List all artifacts I should verify in GitHub main."
2. "Which uploaded snapshot details should be treated as stale?"

15.5 Risks and Verification

1. "What should I verify before starting Sigma?"
2. "What are the highest-risk assumptions in this handoff?"

15.6 Future Spec Book / Aggregation

1. "Which sections of a Project Spec Book should this chat contribute to?"
2. "Create a spec-book outline from this chat."

15.7 Deep-Dive Questions Specific to This Chat

1. "Design the stable ABI boundary table."
2. "Design the declarative cutover DSL."
3. "Plan the second-game proof for Domino reuse."
4. "Compare the stale snapshot to the current GitHub repo."

16. Compact Human Summary

This chat was about preserving and extending the architecture of Dominium, a deterministic simulation/game platform built on the reusable Domino engine. Over the conversation, Dominium was repeatedly reframed from a normal game project into a systems platform: deterministic engine kernel, runtime service host, content/package ecosystem, product family, XStack-governed repository, and eventual live-operations environment.

The user's central concern was future-proofing. They want all code and data to be portable, modular, extensible, reusable for other games and engine projects, and replaceable even at the directory/file level. This led to the doctrine that the system should not try to make every file stable; it should make every boundary explicit, versioned, testable, and replaceable.

Earlier parts of the chat generated many plans for MVP foundations: Earth/Sol/Galaxy stubs, AppShell, capability negotiation, pack compatibility, library/install/save management, diagnostics, MVP gates, release/distribution, meta-stability, time anchors, architecture audits, universal identity, migration lifecycle, numeric/concurrency/observability/store policies, governance, performance, archive, and more.

Later, the user became concerned that AI agents may

have created duplicate or temporary implementations under generic **src** directories. The solution became the Xi-series: repo archaeology, architecture graph, duplicate scanning, implementation scoring, convergence planning, source removal, architecture graph freeze, XStack CI guardrails, and repository structure freeze. The user later reported that Xi-6, Xi-7, and Xi-8 were complete in the public repo.

After Xi, the Pi-series was planned and reportedly executed to create meta-blueprints, execution strategy, and final prompt inventory for future series. Those future series are Sigma for agent/human governance, Phi for runtime component/service/kernel architecture, Upsilon for build/release/distribution/control plane, and Zeta for future live operations such as hot-swappable renderers, live save migration, live protocol upgrades, mod hot activation, and deterministic distributed simulation.

The next correct step is not to start Sigma/Phi/Upsilon/Z directly. It is ?-series: snapshot-driven final planning. ? should extract current repo reality, compare it to the blueprint, classify each subsystem as keep/extend/merge/replace/quarantine, assess foundation readiness, and synthesize final repo-specific execution plans.

The most important thing to preserve is the doctrine: stable contracts, replaceable implementations, deterministic behavior, manifest-based identity, tool-

agnostic development, XStack-enforced architecture, and human-readable plus machine-readable everything.

39.2 Dominion Omega Xi Pi Architecture Future Proofing Planning Reader Brief

Source: docs/archive/conversations/omega_xi_pi_architecture_future/dominium_omega_xi_pi_architecture_future_proofing_planning__06_reader_brief.md

Report type: reader_brief

Size: 1676 bytes

39.2.1 Reader Brief - Dominion Omega/Xi/Pi Architecture & Future-Proofing Planning

What this chat was about

This chat transformed a large game/engine planning effort into a coherent architecture doctrine for Dominion/Domino as a deterministic simulation platform. It covered MVP foundations, release/distribution, repo convergence, agent governance, runtime component architecture, and future live operations.

Top 20 things to know

1. Dominium is being treated as a platform, not just a game.
2. Domino is the reusable deterministic engine/kernel layer.
3. Dominium is the game/product layer.
4. The core doctrine is stable contracts and replaceable implementations.
5. Compatibility is not based on suite version equality.
6. Products/protocols/schemas/formats/packs have independent versions.
7. XStack is the architecture immune system.
8. Repo drift and `src` shadow implementations were a central concern.
9. Xi-series was designed to fix repo convergence and drift.
10. User reports Xi-8 is complete and repo structure frozen.
11. Pi-series was designed to plan future Sigma/Phi/Upsilon/Z.
12. User reports Pi-2 is complete with 110 future prompts.

13. The next phase is ? snapshot-driven planning.
14. Sigma handles agent/human governance.
15. Phi handles runtime component/service/kernel architecture.
16. Upsilon handles build/release/control plane.
17. Zeta handles live operations but only later.
18. Workbench should begin with command/validation spine, not full GUI.
19. Uploaded snapshots may be stale.
20. Verify current GitHub main before repo-specific implementation.

Best next step

Generate ?-0 - SNAPSHOT-INTAKE-0 or verify current GitHub repo state first.

39.3 Dominion Omega Xi Pi Architecture Future Proofing Planning In Chat Reader

Source: docs/archive/conversations/omega_xi_pi_
architecture_future/dominium_omega_xi_pi_
2008

architecture_future_proofing_planning__09_in_
chat_reader.md

Report type: in_chat_reader

Size: 1140 bytes

39.3.1 In-Chat Reader - Dominium Omega/Xi/Pi Architecture & Future-Proofing Planning

Package overview

This preservation package captures the substance of the chat: what was discussed, why it mattered, what was decided, what remains pending, what is uncertain, and how a future chat should continue.

File index

- 00_manifest: package index.
- 01_human_readable_report: full human-readable report.
- 02_context_transfer_packet: future-chat handoff.
- 03_spec_sheet: machine-friendly YAML spec.
- 04_registers: workstream/decision/task/etc. registers.

- 05_aggregator_packet: compact cross-chat merge packet.
- 06_reader_brief: short readable brief.
- 07_verification_and_audit: self-audit and verification queue.
- 08_future_chat_bootstrap_prompt: prompt for a new chat.
- 09_in_chat_reader: this guide.

Best reading order

1. Reader brief.
2. Human-readable report sections 0-16.
3. Context transfer packet.
4. Registers.
5. Spec sheet and aggregator packet if building a master spec book.

Best next actions

- Ask for ?-0.
- Ask to verify current GitHub main.
- Ask for a repo-specific ?-series plan.

39.4 Dominion Omega Xi Pi Architecture Future Proofing Planning Accompanying Human Readable Detailed Summary And Report

Source: docs/archive/conversations/omega_xi_pi_architecture_future/dominium_omega_xi_pi_architecture_future_proofing_planning__10_accompanying_human_readable_detailed_summary_and_report.md

Report type: human_readable_detailed_summary

Size: 16387 bytes

39.4.1 Accompanying Human-Readable Detailed Summary and Report

Chat label: Dominion Omega/Xi/Pi Architecture & Future-Proofing Planning

Date anchor: 2026-05-27 Australia/Melbourne

Scope: This chat and the artifacts generated from this chat. Repo-state claims remain subject to verification against the live repository.

1. What this conversation was really about

This conversation was not merely about producing a summary. It was the culmination of a long architectural planning arc for **Dominium**, a project that increasingly moved from "game plus engine" into the territory of a **deterministic simulation platform**. The reusable engine substrate was repeatedly referred to as **Domino**, while **Dominium** is the game/product family that sits on top of it.

The user's central concern was that the project must be built to last. The desired system is not a throwaway indie codebase. It should be portable, modular, extensible, reusable for different games on the same Domino engine, reusable for other engine projects, and resilient to full rewrites or refactors of entire files and directories. The user repeatedly asked how to make the system future-proof, backward-compatible, forward-compatible, and safe against both human drift and AI-agent drift.

The eventual doctrine became:

Stable contracts.

Replaceable implementations.

Deterministic behavior.

Manifest-based identity.

Tool-agnostic development.

XStack-enforced architecture.

Human-readable and machine-readable everything.

This report should be read as the human explanation accompanying the more formal handoff, registers, spec sheet, and aggregator packet. It exists so that a future reader can understand not just what was planned, but why the plans exist and how the different series relate.

2. The major arc of the chat

2.1 MVP readiness and product surface The early planning revolved around whether the engine, game, client, server, setup, launcher, tools, and app shell were ready for an MVP. The user wanted each product to be usable standalone, not only through a launcher or monolithic suite. The desired behavior was:

- each executable can be launched directly;
- each product can run in CLI and TUI;
- the client is special because it can have a rendered GUI independent of OS-native widgets;
- setup, launcher, server, and tools should eventually support OS-native GUIs where available;
- every product must still fall back to TUI/CLI;
- direct console access and IPC should remain available.

This produced the **AppShell** direction: all products should boot through a unified app shell, select UI mode deterministically, expose commands consistently, and never bypass validation, compatibility checks, or logging.

2.2 Earth/Sol/Galaxy and MVP simulation scope

The user then scrutinized the world simulation: sky, night sky, day/night cycle, tides, rivers, mountains, hydrology, weather, sounds, climate, smell/gas/temperature, collisions, pathfinding, memory, epistemics, freecam, godmode, and more. The key conclusion was that the MVP should not attempt full realism in every domain. Instead, it should add **minimal extensible proxies** where they prevent future refactors:

- EARTH-10: material proxy, surface flags, albedo proxy;
- SOL-1: unified illumination geometry so Moon phase is derived from Sun/Moon/viewer geometry rather than stored phase variables;
- SOL-2: orbit visualisation and ephemeris proxy;
- GAL-0: galaxy metadata proxies;
- GAL-1: compact object and hazard stubs.

The principle was that visible phenomena should be derived from underlying fields, geometry, emitters,

receivers, and processes rather than stored as presentation shortcuts.

2.3 Compatibility and ecosystem architecture A large portion of the conversation focused on versions and compatibility. The user wanted arbitrary versions of engines, games, clients, servers, launchers, setup tools, tools, packs, protocols, schemas, saves, profiles, blueprints, and resources to be mix-and-matchable when safe. That led to a layered compatibility model:

- semantic contracts;
- CAP-NEG capability negotiation;
- PACK-COMPAT manifests;
- LIB install/instance/save separation;
- component graphs;
- install profiles;
- release index;
- update model;
- trust model;
- migration lifecycle;
- universal identity blocks;

- exact suite versus latest-compatible install policies;
- yanked builds and rollback.

A key decision was that the **suite version remains**, but it is only a curated tested snapshot. It does not define compatibility. Compatibility is determined by contract ranges, protocol ranges, schema/format migration rules, pack compatibility, trust policy, and capability negotiation.

2.4 Distribution, archive, and commercialisation planning

The distribution plan evolved from simple packaging into a complete ecosystem. Setup is best understood as a package manager and orchestrator. It can install itself, manage portable/per-user/all-users installs, repair, rollback, update, export, and verify. Bundles and packs can be distributed separately. A release index and archive policy preserve old versions and allow old releases to remain downloadable.

Commercialisation was considered but not implemented. The preferred model is capability-based and trust-based: paid features can be delivered via signed capability artifacts, premium packs, or trust-gated capabilities. The assistant warned not to rely on obscurity of source code. Commercial controls should be handled through signed artifacts, policy, server enforcement, and trust roots.

2.5 Repository drift and the Xi-series A major pivot occurred when the user worried that Codex may have reimplemented systems from scratch under `src/` just to make the repo compile. The user did not know which implementations were best, whether docs were wrong, or whether multiple unique approaches should be merged.

The solution was not to guess. It was to make the repository self-inspecting:

- generate architecture graph;
- index symbols;
- detect duplicate implementations;
- score implementations;
- generate convergence plans;
- execute convergence safely;
- eliminate generic `src` directories;
- freeze architecture graph v1;
- enforce boundaries with RepoX/AuditX/TestX;
- freeze repository structure.

This became the **Xi-series**. The user later reported that Xi phases were completed, including Xi-6 architecture

graph freeze, Xi-7 XStack CI guardrails, and Xi-8 repository freeze. Some uploaded snapshots still showed old `src` pockets, so current repo truth must still be verified before implementation-specific planning.

2.6 Agent governance and tool-agnostic AI integration The user asked whether to use `AGENTS.md` and agent-specific features. The answer was yes, but with a correction: do not optimise for one vendor. The durable design is:

- canonical human/agent governance source;
- machine-readable context;
- task catalog;
- XStack command surface;
- generated mirrors for Codex, Claude, Copilot, and future agents.

`AGENTS.md` should be treated as an instruction layer, not the source of truth. The source of truth is architecture graph, contracts, schemas, registries, and XStack gates. Prompts are untrusted; XStack is authoritative.

2.7 Runtime as a microkernel/service host The conversation then absorbed external advice that the repo

2018

should move toward a componentized runtime, render devices, framegraph, module manifests, shader IR, asset pipeline, thin platform services, and module discovery. The assistant agreed directionally and reframed Dominion as a deterministic simulation kernel plus runtime services and modules.

This produced the **Phi-series** plan:

- runtime kernel model;
- component model;
- module loader;
- runtime services;
- state externalization;
- lifecycle manager;
- framegraph;
- render device abstraction;
- hotswap boundaries;
- asset pipeline;
- sandboxing;
- multi-version coexistence;
- event log;

- snapshot service;
- distributed authority.

2.8 Extreme Z-series live operations The user listed an enormous set of future capabilities: hot-swappable renderers, in-place runtime service restarts, partial live module reload, restartless core engine replacement, fully live save migration, distributed shard relocation, mod install without interruption, deterministic distributed simulation, live protocol/schema evolution, renderer virtualization, canaries, rollback, trust-root rotation, untrusted mod isolation, proof-anchor monitors, mirrored render execution, world streaming, and more.

The assistant grouped these into capability families:

- replaceability;
- live state migration;
- rollout/cutover operations;
- trust/security live ops;
- distributed simulation;
- live content/mod operations;
- multi-path validation and resilience.

The key innovation angle became **deterministic live operations**: live changes with explicit cutover plans, compatibility checks, trust enforcement, proof anchors, rollback, and replayable evidence.

2.9 Pi-series and next ?-series After Xi, the conversation planned a Pi-series meta-blueprint to organise all future work. The user later reported Pi-0, Pi-1, and Pi-2 were completed in the live repo. Pi-2 reportedly produced 110 future prompts, 40 critical path prompts, 9 parallelizable prompts, 86 manual-review prompts, 15 pre-snapshot-safe prompts, and 95 post-snapshot-required prompts.

The next correct step is **?-series**, a snapshot-driven final planning pass. It should consume current repo reality and the completed Pi blueprints to decide which future prompts are already solved, partially solved, blocked, obsolete, dangerous, or ready.

3. Decisions and conclusions that matter most

3.1 Stable contracts, replaceable implementations

The most important conclusion is that Dominion should not try to make every file stable. Files and directories can move, and implementations can be rewritten. What must remain stable or explicitly migrated are identities, contracts, schemas, protocols, save/replay formats, pack

IDs, command IDs, refusal IDs, proof expectations, and public boundaries.

3.2 Use suite version as a curated snapshot, not compatibility authority The suite version should stay because it gives humans a coherent release label. But product interoperability should not depend on exact suite equality. Setup and runtime must consult release indices, component graphs, contract ranges, protocol ranges, pack compatibility, and migration policies.

3.3 Repo state must be evidence-driven Because AI and human developers can drift, the repo itself must be inspected and scored. Architecture graph, duplicate scan, implementation scoring, and ControlX convergence plans are preferred over guessing or rewriting.

3.4 XStack is the immune system XStack should enforce repository structure, architecture boundaries, determinism, duplicate semantic engines, trust, compatibility, and packaging invariants. Agent instructions and human docs are helpful, but XStack is what prevents bad prompts and casual edits from damaging the system.

3.5 Do not implement Z directly The future live-ops ambitions are plausible only if foundations exist first.

Hot-swappable renderers require render service, framegraph, render device abstraction, lifecycle manager, state externalization, cutover plans, and proof hooks. Live save migration requires snapshot service, event logs, migration lifecycle, state integrity proofs, and rollback. Distributed shard relocation requires event-tail synchronization, authority handoff, deterministic replication, and proof-anchor quorum.

3.6 Finish ? before Sigma/Phi/Upsilon/Z Even though Sigma/Phi/Upsilon/Z are planned, they should not be executed from theory. The current repo must be mapped to the blueprint first.

4. What was put off for later

Several things were explicitly deferred or treated as future layers:

- full fluids, chemistry, molecular materials, full erosion, tectonics, ecology, economy, vehicles, combat, and advanced domain simulation;
- OS-native GUI completion for all platforms;
- full Workbench GUI;
- native plugin systems and untrusted executable mods;

- restartless core engine replacement;
- fully live protocol/schema evolution;
- deterministic distributed cluster-of-clusters;
- seamless shard relocation and player transfer;
- live renderer hot-swap and renderer virtualization;
- full commercial licensing/paywall systems;
- marketplace or decentralised pack ecosystems;
- full public stable release.

The reason was not lack of interest. The reason was sequencing. Each requires foundation layers and proof gates.

5. Current unresolved issues

The most important unresolved issue is repo reality. The user reported that Xi and Pi are complete in public GitHub `main`, but some uploaded snapshots in the chat still showed old `src` pockets. Therefore, a future assistant should not assume the snapshot is current. It should verify current GitHub or current local snapshot before making implementation-specific claims.

Other unresolved issues:

- exact current architecture graph and repository lock should be verified;
- exact current XStack CI state should be verified;
- current vendor docs for AGENTS.md / Copilot / Claude / Codex should be rechecked before implementing mirrors;
- exact mapping from existing `engine/modules`, `runtime`, `tools/xstack`, `client`, `server`, `setup`, and `launcher` into Phi-series should be performed by ?;
- final ABI/API naming/prefix policy remains to be formalised;
- second-game proof for Domino reuse remains to be designed;
- Workbench first slice remains to be planned after command/package/presentation floors are clear.

6. Recommended next actions

The best next step is:

?-0 - SNAPSHOT-INTAKE-0

Then:

?-1 - REALITY-EXTRACTION-0

?-2 - BLUEPRINT-RECONCILIATION-0

?-3 - FOUNDATION-READINESS-0

?-4 - FINAL-PROMPT-SYNTHESIS-0

Only after that should the final Sigma/Phi/Upsilon/Z plans be executed.

7. Practical doctrine for future assistants

A future assistant should work like this:

1. Preserve all uncertainty labels.
2. Do not restart Omega/Xi/Pi unless the user explicitly asks.
3. Treat user-reported repo status as user-reported fact, not independently verified truth.
4. Verify live repo reality before exact implementation plans.
5. Do not treat uploaded historical snapshots as current unless the user says they are current.
6. Use XStack as authority.
7. Generate Codex-ready prompts with phases, deliverables, enforcement, TestX, final reports, commit messages, and stop conditions.

8. Keep ambition high but sequence foundations before extreme features.
 9. Challenge shortcuts that would damage determinism, modularity, or compatibility.
-

8. Why this chat matters

This chat matters because it captured the moment where Dominium's planning matured from feature accumulation into platform architecture. It connected many previous plans into one doctrine:

- deterministic engine kernel;
- replaceable runtime services;
- manifest-based identity;
- package/content ecosystem;
- XStack-enforced repo governance;
- agent/human development interface;
- release/archive/control plane;
- future live operations with deterministic proofs.

If merged into a future master spec book, this chat should inform chapters on architecture doctrine,

repository governance, XStack, versioning and release, agent governance, runtime componentization, live operations roadmap, and long-term future-proofing.

9. Key facts to preserve

- Chat label: **Dominium Omega/Xi/Pi Architecture & Future-Proofing Planning.**
- Next phase: **?-series**, not Sigma/Phi/Upsilon/Z directly.
- Doctrine: stable contracts, replaceable implementations, deterministic behavior, manifest-based identity, tool-agnostic development, XStack-enforced architecture, human/machine-readable everything.
- User reports Xi-6, Xi-7, Xi-8, Pi-0, Pi-1, Pi-2 complete in public GitHub main.
- Do not assume current repo truth from old snapshots.
- Do not implement Z before foundations.
- Workbench should grow from command/service/validation spine, not start as a monolithic GUI.
- Code must be reusable for other games and engine projects.

- Paths are not identity; manifests/contracts/IDs are identity.
- XStack is the long-term architecture immune system.

Chapter 40

Platform Renderer Api Plan

40.1 Dominium Codex Platform Renderer Api Plan Reader Brief

Source: docs/archive/conversations/platform_renderer_api_plan/dominium_codex_platform_renderer_api_plan__05_reader_brief.md

Report type: reader_brief

Size: 3982 bytes

40.1.1 Reader Brief - Dominium Codex Platform Renderer API Plan

What This Chat Was About

This chat built a continuity-safe implementation plan for the Dominium/Domino engine. The user wanted ready-to-run prompts for Codex CLI / VS Code Codex running GPT-5.2 on Windows at c:\Inbox\Git Repos\dominium, with no interactive approvals and no dependency downloads unless approved. The work evolved from a broad platforms/renderers/APIs prompt request into a final nine-prompt post-master

implementation pack. The pack assumes a previous master engine-core prompt plan (prompts 1-14) exists and is implemented for planning purposes, but this is unverified.

The active nine-prompt pack focuses on C89/C++98 baseline enforcement, stable ABI/vtable facades, central capability registry, determinism grades, launcher profiles, CMake presets, null backends, Win32/win32-headless Tier-1 platform completion, software/DX9 Tier-1 renderer completion, Tier-2 gating, TLV-as-ABI, net handshakes, launcher/game GUI smoke tests, end-to-end verification, and changelog tooling. Each prompt requires Codex to make detailed git commits suitable for later changelog generation.

The chat also captured a future AAA roadmap: advanced graphics, audio, input, AI, navigation, terrain, vehicles, economy, combat, weather, networking, setup/download, tools, modding, and scripting. These are future architectural slots, not current implementation scope.

Most Important Things to Know

- Final active plan is the 9-prompt pack, not earlier superseded plans.
- Domino core must remain ISO C89.
- Dominium layer must remain C++98.

- C99/C11/C++11+ are optional acceleration only.
- No runtime language-standard switching.
- ABI boundaries use C ABI, POD structs, function tables.
- Every interface starts with `u32 abi_version` and `u32 struct_size`.
- Systems use facade + backend architecture.
- Central capability registry chooses backends deterministically.
- Determinism grades are D0/D1/D2.
- Lockstep strict requires D0.
- Serialization is ABI: one chunked/TLV format, skip-unknown, migrations.
- Net handshake includes build ID, schema ID/version, determinism profile, content hashes.
- Codex must commit throughout with detailed messages.
- Repo archive was uploaded but not inspected in this chat.
- Prompt plans are not evidence of implemented code.

Active Plans or Workstreams

- WORKSTREAM-01: Master deterministic engine-core prompts 1-14.
- WORKSTREAM-02: Active post-master platform/render/API/product completion pack.
- WORKSTREAM-03: Stable API architecture model.
- WORKSTREAM-04: Future AAA roadmap.
- WORKSTREAM-05: This chat report package.

Decisions Already Made

- Use final 9-prompt pack as active plan.
- Use fewer larger prompts.
- Enforce C89/C++98 baseline.
- Use C ABI/POD/vtable interfaces.
- Use central caps registry and D0/D1/D2 determinism.
- Add launcher/game GUI smoke tests.
- Add changelog-ready commit discipline.

Pending Tasks

- Verify repo/archive contents.
- Run Prompt 1, then prompts 2-9 in order.
- Resolve DSYS vs domino_sys_* if conflict appears.
- Confirm target names.
- Confirm UI text strategy if needed.
- Confirm legacy format migration scope.

Open Questions

- Were master prompts 1-14 actually implemented?
- Which system API is authoritative?
- What are exact launcher/game targets?
- Is full Tier-2 runtime completion required later?
- What old save/replay/pack formats must be migrated?

Files / Artifacts / Prompts to Preserve

- /mnt/data/dominium.zip
- MASTER prompt plan 1-14
- Final prompts 1-9

- Previous Context Transfer Packet
- This report package and ZIP

What to Verify Before Acting

- Repo build state.
- CMake target list.
- DSYS/DGFX headers and callsites.
- Public header baseline violations.
- Existing serialization formats.
- Existing UI/text rendering support.

Best Next Step

Download this package, then if continuing implementation, inspect the repo/archive and run Final Prompt 1 in Codex.

Chapter 41

Platform Support

41.1 Dominium Platform Support Planning Reader Brief

Source: docs/archive/conversations/platform_support/dominium_platform_support_planning__05_reader_brief.md

Report type: reader_brief

Size: 8870 bytes

41.1.1 Reader Brief - Dominium Platform Support Planning

What This Chat Was About

This chat was about platform support planning for Dominium. It started with broad questions about what systems and operating systems Dominium could support, then expanded into a large device-family inventory covering PlayStation, Xbox, Nintendo, PC handhelds, Android, Apple/iOS, Web, AR, VR, and cross-cutting graphics/runtime technologies. The most important user decision came near the end: Android, iOS, and Web will

be primary top-tier support after PC.

The current working model is therefore: PC first; Android, iOS, and Web also Tier-0. PC implies Windows, macOS, and Linux planning, though exact minimums are not known. Android, iOS, and Web must not be treated as late ports. Consoles remain desired but should be treated as gated secondary targets unless elevated. Legacy consoles, old handhelds, retro OSES, and bare-metal/firmware targets should be treated as constrained/research/engine-only unless the user later makes them a priority.

The best continuation is to create a formal Platform Support Matrix and platform contracts. Do not rely on prior assistant claims or old citation IDs for current platform facts. Verify SDKs, stores, browser features, console rules, hardware status, and legal constraints from current official sources before implementation planning.

Most Important Things to Know

- PC is the first primary platform family.
- Android is primary top-tier support after PC.
- iOS is primary top-tier support after PC.
- Web is primary top-tier support after PC.
- Mobile and Web should not be treated as late ports.

- Use a formal platform support matrix/platform contracts.
- Consoles are desired but gated/secondary unless elevated.
- Legacy consoles/retro OSes are constrained/research by default.
- "Domino" engine/core name is unconfirmed.
- All external platform facts require verification before future use.
- Do not bundle proprietary BIOS/firmware for legacy/emulated support.
- Closed console support must use official access for commercial/native support.
- PC handhelds map primarily to PC OS targets plus handheld QA/UX profiles.
- XR priority is unresolved; runtime-first OpenXR/WebXR was only an assistant proposal.
- Engine/toolchain choice is not established.
- Support must be defined by scope: full product, reduced build, engine-only, emulator-hosted, remote-play, unsupported.
- This package is chat-specific and not a whole-project summary.

Active Plans or Workstreams

- WORKSTREAM-01: Overall Dominion platform support strategy and tier model - highest priority.
- WORKSTREAM-02: Tier-0 PC desktop operating system support - highest priority.
- WORKSTREAM-03: Tier-0 Android ecosystem support - highest priority.
- WORKSTREAM-04: Tier-0 iOS/iPadOS and Apple-family classification - highest priority.
- WORKSTREAM-05: Tier-0 Web/WASM/browser/PWA support - highest priority.
- WORKSTREAM-06: Closed console support strategy: PlayStation, Xbox, Nintendo - medium-high priority.
- WORKSTREAM-07: PC handheld support - high priority.
- WORKSTREAM-10: Cross-cutting architecture, backend, runtime, and compatibility strategy - highest priority.
- WORKSTREAM-11: Chat-specific handoff/report package - highest for current task priority.

Decisions Already Made

- DECISION-01: PC is the first primary platform family for Dominion. (FACT; decided by explicit user wording).
- DECISION-02: Android is primary top-tier support after PC. (FACT; decided by explicit user statement).
- DECISION-03: iOS is primary top-tier support after PC. (FACT; decided by explicit user statement).
- DECISION-04: Web is primary top-tier support after PC. (FACT; decided by explicit user statement).
- DECISION-05: Mobile and Web should be treated as co-equal roots with PC, not late ports. (INFERENCE; inferred working decision based on explicit user priority and prior assistant framing).
- DECISION-06: Closed commercial consoles are secondary/gated support targets, not current Tier-0. (INFERENCE; assistant-proposed working classification; not explicitly accepted or rejected).
- DECISION-07: Legacy consoles and retro systems are constrained/research/engine-only targets by default, not full Dominion parity targets.

(INFERENCE; assistant-proposed working classification).

- DECISION-08: PC handhelds should map primarily to PC OS targets rather than become separate base platform families. (INFERENCE; assistant-proposed working classification).
- DECISION-09: Web/WASM is both a universal fallback and a Tier-0 support target. (FACT / INFERENCE; decided/inferred from latest user priority).
- DECISION-10: Use platform contracts/support matrix rather than an undifferentiated "support everything" claim. (INFERENCE; assistant-proposed; strongly implied by scope).
- DECISION-11: Closed console support must use official platform-holder paths for commercial/native support. (UNCERTAIN / UNVERIFIED; assistant-proposed constraint requiring external verification).
- DECISION-12: Do not bundle proprietary BIOS/firmware or rely on firmware redistribution for legacy support. (INFERENCE; assistant-proposed legal/safety constraint).

Pending Tasks

- TASK-01: Create a formal Platform Support Matrix for Dominium.
- TASK-02: Define Tier-0 platform contracts.
- TASK-03: Define PC OS baselines.
- TASK-04: Define Android minimum API, ABI, graphics, and subdevice scope.
- TASK-05: Define iOS/iPadOS minimum OS/device baseline and Apple-family classification.
- TASK-06: Define Web/WASM/browser/PWA baseline.
- TASK-07: Create PC handheld profile.
- TASK-08: Define console Tier-1 strategy.
- TASK-09: Define legacy/retro target policy.
- TASK-10: Decide XR priority and runtime strategy.
- TASK-11: Choose or confirm engine/toolchain strategy.
- TASK-12: Confirm or retire the "Domino" engine/core name.

Open Questions

- QUESTION-01: What exact target families are Tier-0 besides PC, Android, iOS, and Web?
- QUESTION-02: What minimum Android API level and ABI set will be supported?
- QUESTION-03: What minimum iOS/iPadOS version and device generations will be supported?
- QUESTION-04: What is the Web/browser baseline?
- QUESTION-05: Is WebGPU required, preferred, or optional, and is WebGL fallback mandatory?
- QUESTION-06: Which Windows versions and CPU architectures are supported?
- QUESTION-07: Which macOS versions and Mac architectures are supported?
- QUESTION-08: What Linux baseline is required?
- QUESTION-09: Are Android TV / Google TV first-class Tier-0 targets or secondary Android subtargets?
- QUESTION-10: Are tvOS, watchOS, and visionOS in scope, and at what tier?

- QUESTION-11: What engine/toolchain will Dominion use?
- QUESTION-12: Is "Domino" the real name for the engine/core layer?
- QUESTION-13: Is Steam Deck a named Tier-0 subtarget?
- QUESTION-14: Which current console generations are commercial targets?

Files / Artifacts / Prompts to Preserve

- ARTIFACT-07: User comprehensive hardware/software target inventory.
- ARTIFACT-09: User's decisive Android/iOS/Web top-tier statement.
- ARTIFACT-11: Previous Context Transfer Packet.
- ARTIFACT-13: This final generated report package.
- ARTIFACT-14: Prior citation IDs are non-portable and should not be relied on.

What to Verify Before Acting

- VERIFY-01: Current official commercial console developer access requirements for PlayStation,

Xbox, and Nintendo.

- VERIFY-02: Current PlayStation hardware/platform support status, including PS4/PS5/PS5 variants and PlayStation Portal role.
- VERIFY-03: Current Xbox One/Series development and compatibility model.
- VERIFY-04: Current Nintendo Switch/successor developer support and platform facts.
- VERIFY-05: Current Android SDK/NDK, Google Play target API, ABI, Android TV/Go/Automotive requirements.
- VERIFY-06: Current Apple SDK, App Store, iOS/iPadOS/macOS/tvOS/watchOS/visionOS submission and support requirements.
- VERIFY-07: Current WebGPU browser support and limitations.
- VERIFY-08: Current WebAssembly features needed for Dominium, including SIMD, threads, memory limits, filesystem/storage patterns.
- VERIFY-09: Steam Deck/SteamOS current developer guidance, Steam Deck Verified requirements, and Linux runtime assumptions.

- VERIFY-10: Feasibility of listed Apple devices: iPod Touch, Apple TV, Apple Watch, Vision Pro, Intel Mac, Apple Silicon Mac.
- VERIFY-11: Android TV, Google TV, Android Automotive, Chromebook Android compatibility, and vendor ROM constraints.
- VERIFY-12: Current OpenXR, WebXR, ARKit, ARCore, SteamVR, Meta, PlayStation VR, Windows Mixed Reality, and Pico runtime status.

Best Next Step

Create the formal Platform Support Matrix. Start with Tier-0 PC, Android, iOS/iPadOS, and Web; then classify PC handhelds, consoles, XR, and legacy systems below that hierarchy unless the user explicitly changes priority.

Chapter 42

Readme Ports Determinism

42.1 Dominium Readme Ports Determinism Reader Brief

Source: docs/archive/conversations/readme_ports_determinism/dominium_readme_ports_determinism__05_reader_brief.md

Report type: reader_brief

Size: 4665 bytes

42.1.1 Reader Brief - Dominium README Ports Determinism

What This Chat Was About

This chat refined the Dominium / Domino root README and produced a maximum-fidelity handoff for future continuation. Dominium / Domino is described as a deterministic, integer-math, multi-platform simulation game and engine. Domino is the deterministic engine core; Dominium is the official game/runtime/tooling layer. The user pasted README versions, requested critique, asked for Codex prompts, pasted Codex

outputs, and refined the result.

The most important change came from the user's direct requirement that ports should not be stored in a separate directory or system, should work within the same structure, and should degrade gracefully without flowing upstream. The final README now says all platforms build from one unified source hierarchy, platform-specific behavior is expressed through thin shims, compile-time flags, and capability tables, and `/ports` contains only optional metadata/build configurations/capability descriptors with no code or behavior. This metadata-only `/ports` interpretation should be verified before formalizing directory specs.

The chat also clarified determinism rules: no floats in `/engine` or engine-controlled formats; non-authoritative tools/renderers may use floats only if they never feed back; RNG streams advance only during deterministic tick phases; tick phases are immutable; parallel commits must be deterministic; lockstep is canonical networking; build numbers/timestamps are diagnostic only; content-lock mismatches are fatal; and disk versions are immutable contracts.

Most Important Things to Know

- User explicitly rejected separate port directories/systems.

- All platforms must share one source hierarchy.
- Reduced functionality must degrade locally through capability descriptors and not flow upstream.
- Final README keeps `/ports` only as metadata/build config/capability descriptors, but this may need confirmation.
- No engine/runtime source code or behavior may live in `/ports` if retained.
- Section 9 is normative for platform/render targets.
- Intro mentions OS/2 strata, but Section 9 omits OS/2; this is an unresolved contradiction.
- Full Domino engine targets 286-class-and-newer systems.
- CP/M-80/86 is tooling/limited frontends only, not complete world simulation.
- Floats are banned in authoritative engine/format paths, not in all tools/renderers.
- Lockstep is canonical networking; rollback/prediction must converge to lockstep.
- README is descriptive; `/docs/spec` files are normative.

- Codex previously introduced duplicate README content, so prompts must be tightly scoped.

Active Plans or Workstreams

- WORKSTREAM-01: README architecture.
- WORKSTREAM-02: unified ports and capability-based degradation.
- WORKSTREAM-03: determinism contract.
- WORKSTREAM-04: data formats and save/content-lock contract.
- WORKSTREAM-05: future normative specs.
- WORKSTREAM-06: Codex prompt workflow.

Decisions Already Made

- Domino = engine core; Dominion = game/runtime/tooling layer.
- Specs are normative; README is descriptive.
- 286+ full engine target; CP/M limited only.
- Unified source hierarchy for all platforms.
- No divergent port implementations.
- Capability-based graceful degradation.

- Lockstep is canonical.
- Disk versions immutable.
- `content.lock` mismatch fatal until reconciled.

Pending Tasks

- Verify actual repo README against final pasted README.
- Resolve OS/2 matrix inconsistency.
- Confirm metadata-only `/ports` vs no `/ports`.
- Update `DIRECTORY_CONTRACT.md`.
- Create capability-system spec.
- Create determinism and data-format specs.

Open Questions

- Should OS/2 be added to Section 9 or removed/qualified in intro?
- Does the user accept metadata-only `/ports`?
- Where exactly do platform shims live?
- What is the capability descriptor schema?
- What are exact RNG/hash/TLV algorithms?

Files / Artifacts / Prompts to Preserve

- Final README pasted by user.
- User port requirement quote.
- First Codex prompt for README fixes.
- Second Codex prompt for unified ports.
- Cleanup Codex prompt.
- Previous Context Transfer Packet.
- References to README.md, /docs/spec/DIRECTORY_CONTRACT.md, /docs/spec/MILESTONES.md, content.lock.

What to Verify Before Acting

- Actual repository state.
- /ports interpretation.
- OS/2 target status.
- Existence/content of referenced spec files.
- That no source code lives under /ports if retained.

Best Next Step

Verify the actual repository README, then patch the OS/2 inconsistency. If moving into specs, confirm /

ports semantics before editing DIRECTORY_CONTRACT.
md.

Chapter 43

Testx Repox Governance

43.1 Dominion Testx Repox Governance Handoff Reader Brief

Source: docs/archive/conversations/testx_repoX_governance/dominium_testx_repoX_governance_handoff__05_reader_brief.md

Report type: reader_brief

Size: 4262 bytes

43.1.1 Reader Brief - Dominion TestX RepoX Governance Handoff

What This Chat Was About

This chat was about preserving and formalising the governance layer around Dominion / Domino: TestX, RepoX, build identity, application/UI rules, IDE/toolchain projections, authority-based demo/free distribution, control-layer containment, language/ABI policy, old OS support, and eventual cross-chat handoff.

It was not primarily about adding gameplay content. The latest upstream canon pasted by the user is now controlling: architecture is closed, canon is locked, and future chats should implement, audit, optimise, and maintain rather than redesign.

The biggest change to preserve is versioning: earlier global-build-number ideas were superseded by BUILD-ID-0. Product SemVer remains manual. Release-grade GBN is centrally allocated only for beta/rc/release/hotfix and shared by all products in a release. BII is always present for dev/ci. No distributed artifact may lack GBN. Protocol/schema/API/ABI versions are orthogonal and mismatches refuse loudly.

The second major contribution is RepoX: IDEs are projections, not source of truth. CMake + TestX/RepoX own the build graph. IDE files should be generated/disposable under /ide except documented packaging inputs. The user supplied a strict legacy/modern toolchain matrix and later installed Windows 10 + VS2022 for MVP testing.

The third major contribution is authority-based distribution: demo/free/tourist/full/service experiences should use the same binaries/content and differ by authority/entitlement. Piracy is contained by denying durable value extraction, not by corrupting the core with DRM.

Most Important Things to Know

- Latest upstream canon supersedes earlier assistant drafts.
- Architecture is closed; do not redesign.
- Core ontology: Assemblies, Fields, Processes, Agents, Law.
- Engine C89; game C++98; public headers parseable forever.
- BUILD-ID-0 replaces earlier build-number ideas.
- CLI is canonical; GUI/TUI bind to the same command graph.
- UI is data, never logic.
- Setup is only install mutation authority.
- Tools are read-only by default.
- IDEs are projections; CMake/TestX/RepoX own graph.
- Demo/free/full/tourist are authority profiles, not code forks.
- Control systems are optional external layers and non-interfering.

- Old OS support can be dropped by stopping new binaries, not by breaking old compatible binaries.
- VS2022/v141_xp is an MVP path, not archival replacement for VC7.1/VC6.

Active Plans or Workstreams

WORKSTREAM-01 Dominion / Domino core engine and game, WORKSTREAM-02 Content system, packs, and zero-asset boot, WORKSTREAM-03 Application layer canon, WORKSTREAM-04 Renderers, platforms, and headless operation, WORKSTREAM-05 TESTX / VALIDATE verification system, WORKSTREAM-06 TESTX2 control layers and non-interference, WORKSTREAM-07 TESTX3 authority, demo, tourist, services, and piracy containment, WORKSTREAM-08 REPOX repository ownership and IDE projections, WORKSTREAM-09 Toolchain and legacy OS matrix, WORKSTREAM-10 TESTX4 language profiles and ABI discipline, WORKSTREAM-11 Policy-as-data and rules-to-checks automation, WORKSTREAM-12 Windows MVP build/test plan with VS2022

Decisions Already Made

See DECISION-01 through DECISION-27 in the registers. Highest priority: DECISION-01, DECISION-06, DECISION-10, DECISION-21, DECISION-23.

Pending Tasks

Start with TASK-01 through TASK-06: repo audit, canon index/status, build/version reconciliation, and IDE inventory.

Open Questions

Highest priority: QUESTION-01 through QUESTION-08.

Files / Artifacts / Prompts to Preserve

Preserve the Context Transfer Packet, latest upstream SYSTEM ROLE block, TESTX original prompt, TESTX2/TESTX3/REPOX prompts, TESTX4 proposal, Windows VS2022 MVP recommendation, repo tree snapshot, and this report package.

What to Verify Before Acting

Verify CANON_INDEX, BUILD-ID-0 files/scripts, docs status headers, CMakePresets, CI, IDE artifacts, public header parseability, language compliance, zero-asset boot tests, and VS2022 toolsets/SDKs.

Best Next Step

Run a Phase 0 audit of the actual repo before changing code: canon docs, build/version files, CMake/CI, IDE artifacts, APP/UI binding, and language/profile compliance.

Chapter 44

Ui Editor Tool Editor Planning

44.1 Dominion Ui Editor Tool Editor Planning Reader Brief

Source: docs/archive/conversations/ui_editor_tool_editor_planning/dominium_ui_editor_tool_editor_planning__05_reader_brief.md

Report type: reader_brief

Size: 4230 bytes

44.1.1 Reader Brief - Dominion UI Editor + Tool Editor Planning

What This Chat Was About

This chat planned a Dominion/Domino UI authoring ecosystem. The immediate problem was a Windows launcher UI that looks mangled and flickers. The broader goal is a deterministic, native-widget UI authoring pipeline for launcher, setup, game, and other Dominion tools. The chat established that the repo already has an in-tree DUI system using TLV schema/state and backends including win32/comctl32, dgfx, and 2060

null. The selected strategy is Phase A UI Editor first, then Phase B Tool Editor. Phase A is Windows NT/latest / Windows 10+ only, system DPI, 100% design scale, and native Win32 preview. Phase B Tool Editor is the long-term cross-platform/self-hosting editor.

The chat produced a large set of Codex prompts, not verified implementation. These include an 11-prompt foundational plan, a final UI Editor implementation prompt, a six-prompt UU plan for import/CLI/ops/launcher/setup/hardening, and an IDE live-editing prompt. The user later clarified that Minecraft-style launcher/setup means logical layout only, not graphical design, and native Win32 widgets only.

Most Important Things to Know

- UI Editor first, Tool Editor second.
- Existing DUI/TLV system is the foundation.
- TLV is canonical; JSON mirror is read-only/default if small.
- Do not assume any prompt was implemented.
- Native widgets are a hard requirement.
- Minecraft-style means structure only, not skinning.
- UI Editor must import existing tool UIs.

- UI Editor must expose CLI automation for Codex.
- ops.json is the deterministic edit DSL.
- Launcher/setup canonical docs are the capability tests.
- Tool Editor eventually self-hosts and edits itself/ other tools.
- Uploaded files exist but were not inspected.
- IDE live editing prompt uses preview hosts; user acceptance of that approach is unverified.

Active Plans or Workstreams

- Existing DUI system extension.
- Phase A UI Editor.
- Phase B Tool Editor.
- Canonical UI IR/TLV/JSON toolchain.
- Backend capability model.
- Deterministic layout engine.
- Splitter/Tabs/Scroll widgets.
- Action/event codegen.
- UI discovery/import/export.

- CLI and ops.json automation.
- Launcher/setup native logical layout tests.
- CI/determinism hardening.
- IDE live editing workflows.

Decisions Already Made

- Phase A UI Editor precedes Tool Editor.
- Phase A is modern Windows-only, system DPI, 100% design scale.
- Tool Editor eventually targets all supported compiled systems.
- Use native OS controls.
- Use existing DUI; do not replace it with Qt/WPF/wx.
- TLV canonical with optional deterministic JSON mirror.
- Generic backend capability system.
- Splitter/Tabs/Scroll first-class in Phase A.
- Minecraft-style launcher/setup is logical layout only, native Win32 widgets only.

Pending Tasks

- Verify whether any Codex prompts were executed.
- Inspect repo/source bundle.
- Implement or verify foundational prompts.
- Implement or verify UI Editor, CLI, ops.json engine.
- Generate launcher/setup canonical UI docs from scripts.
- Run hardening/determinism tests.
- Clarify IDE live-editing expectations.

Open Questions

- Actual repo state and file paths.
- Actual TLV wire format.
- Setup tool location and UI integration.
- Generated code commit policy.
- Exact dependency license policy.
- Whether preview-host IDE strategy satisfies the user.
- Whether AppKit/GTK backends exist.

Files / Artifacts / Prompts to Preserve

- `/mnt/data/source.zip`
- `/mnt/data/SetupC.zip`
- `/mnt/data/LauncherC.zip`
- `launcher_ui_v1.tlv`
- `dui_win32.c`, `dui_dgfx.c`, `dui_null.c`
- Prompts 1-11
- Final UI Editor prompt
- UU1-UU6 prompts
- IDE live editing prompt

What to Verify Before Acting

- Whether prompts were implemented.
- Contents of uploaded zip files.
- Actual repo paths/targets.
- Existing TLV parser/writer.
- Setup tool existence.
- User acceptance of IDE preview-host approach.

Best Next Step

If continuing implementation, first inspect the repo/source bundle or ask the user whether Codex executed any prompts. If continuing planning only, refine the IDE live-editing prompt or begin Tool Editor prompts after confirming UI Editor capability-test status.

Chapter 45

Xstack Lab Galaxy

45.1 Dominium Xstack Lab Galaxy Handoff Reader Brief

Source: docs/archive/conversations/xstack_lab_galaxy/dominium_xstack_lab_galaxy_handoff__05_reader_brief.md

Report type: reader_brief

Size: 3916 bytes

45.1.1 Reader Brief - Dominium XStack Lab Galaxy Handoff

What This Chat Was About

This chat was about preserving and continuing work on Dominium / Domino, a deterministic universe simulation/game project, and XStack, a portable agentic development/governance suite. The chat began from a long history of planning and frustration with agentic development failures, especially gates that stopped work instead of fixing mechanical problems. It produced a robust doctrine: no hardcoded modes, process-only

mutation, deterministic replay, TruthModel -> PerceivedModel -> RenderModel separation, pack-driven integration, and XStack as removable development tooling rather than runtime dependency.

A later uploaded transcript records a 13-prompt run in a new chat that reportedly implemented a Lab Galaxy deterministic substrate: canonical docs/schemas, pack system, registry compile/lockfile/cache, modular tools/xstack runner, session boot, observation kernel, lab camera/time processes, astronomy/Earth packs, descriptor-driven tool UI, interest regions, SRZ scheduling, deterministic packaging, setup/launcher, and 10 thematic commits. The user explicitly said they are not sure what was actually done and need completion/accuracy/consistency verification.

The future assistant should treat this report as continuity context, not as verified repo truth.

Most Important Things to Know

- Verify the repository before trusting the transcript.
- XStack must be portable and removable.
- Runtime must not depend on tools/xstack.
- No mode flags: use LawProfile / ExperienceProfile / ParameterBundle.
- All mutation must happen through Processes.

- Renderer/UI must not access TruthModel.
- Packs must be data-only.
- Registry compile and lockfile enforce deterministic integration.
- Local verification should be fast, cached, sharded.
- Run-meta should not dirty tracked files except explicit snapshots.
- Lab Galaxy milestone is reported complete but must be audited.
- Survival is planned but not implemented as the main 13-prompt result.
- Survival default should be diegetic-only: no non-diegetic HUD/freecam/console.
- Future realism should be packs/solvers/budgets, not global micro-sim.

Active Plans or Workstreams

- Verify 13-prompt Lab Galaxy implementation.
- Reconcile XStack entrypoints and artifact policy.
- Preserve runtime/XStack boundary.
- Possibly proceed to Survival Vertical Slice after audit.

- Preserve future domain/realism extensibility.

Decisions Already Made

- Profiles, not modes.
- Process-only mutation.
- Truth/Perceived/Render separation.
- XStack removable.
- Mechanical failures remediated.
- Packs data-only and compiled into registries.
- Deterministic packaging and lockfile enforcement.

Pending Tasks

- Run git status/log.
- Run tools/xstack/run fast.
- Audit 13 prompt deliverables.
- Verify XStack removability.
- Inspect .gitignore and docs/audit tracking.
- Decide push/rebase status.
- Decide next feature focus.

Open Questions

- Is tools/xstack/run now canonical over gate.py?
- Are docs/audit artifacts tracked intentionally?
- Are all 13 prompts truly complete?
- Is Lab Galaxy interactive or mainly headless?
- Is survival next?

Files / Artifacts / Prompts to Preserve

- transcript.txt
- AGENTS.md
- docs/canon/constitution_v1.md
- docs/canon/glossary_v1.md
- docs/roadmap/milestone_lab_galaxy.md
- tools/xstack/run
- tools/setup/build
- tools/launcher/launch
- packs/
- bundles/bundle.base.lab
- schemas/

- tools/xstack/sessionx
- docs/architecture/
- 10 reported commits

What to Verify Before Acting

- Actual git state.
- Commit list and push status.
- XStack command path.
- Runtime no-XStack imports.
- Run-meta tracked files.
- Fast/strict validation.
- Prompt-by-prompt file existence.

Best Next Step

Run a repository verification and completion audit against the 13-prompt transcript before writing any new feature prompt.